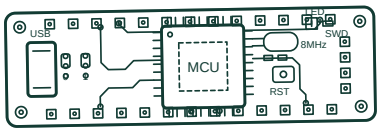
通往单片机之路

把一颗芯片讲透：以 STM32F407 为主线，对照 GD32 与 ESP32-S3，从寄存器一路走进双 RTOS。



v1.0.0 · 2026-10-23
zhuguang-ZFG 与贡献者
正文与图片 Apache-2.0 · 代码 Apache-2.0

版权与版本

书名	通往单片机之路
版本	v1.0.0 · 2026-10-23
作者	zhuguang-ZFG 与贡献者
许可	正文与图片 Apache-2.0 · 代码 Apache-2.0（仓库 LICENSE）
来源	https://github.com/zhuguang-ZFG/STM32/ · 在线阅读 https://zhuguang-zfg.github.io/STM32/mcu/
证据冻结点	75 项来源注册（附录 E）；被引用的上游代码冻结到 tag/commit；未上板实测的现象一律标「待上板实测」。
构建	由 `mcu-book/build_html.py` + `build_print.py` 从仓库 88 章源文档一键构建；同一输入两次构建的页数与开本一致（构建信息含时间戳除外）。
书号	自制版本页，未申请 ISBN/CIP；发行渠道另行决定。

这不是文章合集，是一本书

全书 94 章沿同一条主线推进：先把 **C 与构建**的地基打牢，再把 **STM32F407 寄存器**逐个拆开，随后上 **双 RTOS**、跨到 **ESP32-S3**、对照 **GD32 双系**，最后在 **实物实验与双板项目**里把知识钉在板子上，收官教你怎么把 **大模型变成结对工程师**。每章开头的「本章罗盘」约定这一章回答什么问题、交付什么、边界在哪；章末自测的答案与逐项解析集中在附录 C。

零基础第一遍

C0 → B0 → S0 → S2/S3 → 实验 E01；编译出第一个 .bin 时，你已经摸到真机器码。

只想先点灯

S0 → S3 → E01；半小时内让霸天虎的红灯按你写的寄存器位闪烁。

面试/校招深挖

C 篇全读 → B2/B3/B4 → S4 → S16 HardFault 取证 → F2 上下文切换。

AIoT 方向

S 篇 → P 篇（重点 P8 Wi-Fi、P12 音频）→ J2 S3 记录器。

RTOS 进阶

F 篇源码级 → R 篇 → 对照篇选型 → E04 优先级反转复现。

换国产芯

G 篇 GD32F4 逐字段对照 → V 篇 RISC-V/CLIC → 拿旧工程逐项迁移。

想用 AI 加速

A0 定位 → A2 手册对账 → A1 提示词模式；模型写假设，你对账拿终审权。

证据口径：手册参数写文档号与修订点（附录 E 来源注册表）；被逐行引用的代码冻结到 tag/commit（FreeRTOS V11.1.0、ESP-IDF v5.5.2 等）；凡是没接仪器量过的现象，正文与实验页一律标「待上板实测」，不冒充已验证数据。

关于插图：书中 100 余张机制动画以内嵌 SVG 收录——HTML 版打开即播（SMIL 动画随翻页运行）；PDF 版为静态首帧。交互式计算器与在线视频在离线书中替换为提示卡片并给出在线版入口。

目录与分部目标

第一部分 C 语言精髓：嵌入式是另一种 C

在碰任何外设之前，把内存模型、指针、volatile、ABI、栈帧与未定义行为的地基打进肌肉记忆，每一课都能用宿主机上的反汇编与 probe 脚本复现。

1. 01 . C0 为什么嵌入式 C 是另一种 C
2. 02 . C1 内存模型：一个全局变量的三段旅程
3. 03 . C2 指针：点灯公式的全部秘密
4. 04 . C3 volatile：跟编译器约法三章
5. 05 . C4 结构体与 ABI：一个结构体罩住整组寄存器
6. 06 . C5 函数指针、状态机与环形缓冲
7. 07 . C6 调用约定与栈帧
8. 08 . C7 未定义行为与 MISRA-C 精要
9. 09 . C8 串口协议：从字节流到可恢复的消息

第二部分 构建与运行全过程

从源码到机器码再到板上运行：四步构建、ELF 解剖、链接脚本、启动过程、map 审计、烧录调试与构建系统，每道工序的产物都亲手拆开对账。

1. 10 . B0 工具链全景：四个软件一台戏
2. 11 . B1 四步构建：从 main.c 到机器码
3. 12 . B2 ELF 解剖：把固件放上解剖台
4. 13 . B3 链接脚本：固件的建筑图纸
5. 14 . B4 启动过程：上电到 main 的每一条指令
6. 15 . B5 map 与体积：谁吃了我的 Flash
7. 16 . B6 烧录与调试：SWD 与断点的真相
8. 17 . B7 构建系统：从手写 Makefile 到 CMake 与 scons

第三部分 STM32F407 寄存器主线

不碰 HAL、不点 CubeMX：从 RCC 时钟树到看门狗，18 站走完「手册 → 寄存器 → 库源码 → 实物」的完整闭环，对芯片的信任来自手册而不是库。

1. 18 . S0 环境搭建：裸机工具链
2. 19 . S1 F407 架构总览：地址背后的故事
3. 20 . S2 RCC 时钟树：168MHz 是怎么算出来的
4. 21 . S3 GPIO：七个寄存器位级图解
5. 22 . S4 NVIC 与 EXTI：打断的艺术
6. 23 . S5 SysTick：内核的心跳
7. 24 . S6 定时器 TIM：一器多用的瑞士军刀
8. 25 . S7 USART：让芯片开口说话
9. 26 . S8 DMA：让数据自己搬家
10. 27 . S9 ADC：把模拟世界变成数字
11. 28 . S10 DAC：芯片也能输出模拟量
12. 29 . S11 I2C：两根线的社交礼仪
13. 30 . S12 SPI：全双工的移位艺术
14. 31 . S13 内部 Flash 与 IAP：固件给自己动手术
15. 32 . S14 低功耗：让电池活过一年
16. 33 . S15 SPL 标准库解剖：库是怎么封装寄存器的
17. 34 . S16 HardFault 与排错：崩溃现场的法医技术
18. 35 . S17 看门狗：任务失去进展之后

第四部分 FreeRTOS 精讲（内核源码级）

对着 FreeRTOS V11.1.0 源码逐行走读：任务与 TCB、上下文切换、调度器、队列、信号量与互斥量、内存管理，直到亲手把它移植上霸天虎。

1. 36 . F0 为什么需要 RTOS：先失败一次
2. 37 . F1 任务与 TCB：xTaskCreate 逐行走查
3. 38 . F2 上下文切换：PendSV 换魂术
4. 39 . F3 调度器：一条 CLZ 选出下一个任务
5. 40 . F4 队列：传送带的源码解剖
6. 41 . F5 信号量与互斥量：优先级反转事件
7. 42 . F6 任务通知、事件组与软件定时器
8. 43 . F7 内存管理：heap_1 到 heap_5 的人生选择
9. 44 . F8 移植 FreeRTOS 到霸天虎：三异常接管全记录

第五部分 RT-Thread 精讲

从线程调度到 IPC 全家桶、内存管理、设备框架、finsh 控制台与 env 工具链，理解国产 RTOS 的工程哲学，并完成霸天虎移植。

1. 45 . R0 RT-Thread 架构：万物皆对象
2. 46 . R1 线程与调度：256 级位图的另一种写法
3. 47 . R2 IPC 全家桶：一套范式五种武器
4. 48 . R3 RT-Thread 内存管理：确定性之美
5. 49 . R4 设备框架：驱动与应用的解耦术
6. 50 . R5 finsh 控制台：板子上的 shell
7. 51 . R6 Env 与 menuconfig：RT-Thread 的点单系统
8. 52 . R7 移植 RT-Thread 到霸天虎：两步走

第六部分 双 OS 对照与选型

同一个概念在两套内核里各长什么样：逐项对照任务、调度、IPC 与内存管理，把选型从口味之争变成按约束推导。

1. 53 . 对比 0 同概念双实现对照
2. 54 . 对比 1 选型决策树

第七部分 ESP32-S3 + ESP-IDF

换一条技术路线：GPIO 矩阵、IDF 工程解剖、双核中断、驱动框架、Wi-Fi 与 ESP-NOW、分区表/NVS/OTA、低功耗、音频链路与看门狗健康监督。

1. 55 . P0 环境搭建：ESP-IDF 与小智板的第一次对话
2. 56 . P1 S3 架构与启动：从复位到 app_main
3. 57 . P2 GPIO 与引脚矩阵：引脚自由换岗的秘密
4. 58 . P3 IDF 工程解剖：组件化构建流水线
5. 59 . P4 中断与双核：IRAM 纪律与核间分工
6. 60 . P5 UART 驱动解析：成熟驱动长什么样
7. 61 . P6 SPI/I2C 驱动框架：板载外设当教材
8. 62 . P7 定时器与 LEDC：各管一段时间
9. 63 . P8 Wi-Fi 精髓：事件循环与连接状态机
10. 64 . P9 蓝牙与 ESP-NOW：协议栈的另一种打开方式
11. 65 . P10 Flash、分区表、NVS 与 OTA
12. 66 . P11 低功耗：睡眠矩阵与 ULP
13. 67 . P12 音频链路：从 I2S 时序到"hello 语音
14. 68 . P13 看门狗与健康监督：不要喂反了

第八部分 GD32 双系对照

G 篇把 GD32F4xx 与 STM32F407 逐字段对照，V 篇用 GD32VF103 丈量 RISC-V 内核、CLIC 中断与 MTIME。迁移能力就是在对照里练出来的。

1. 69 . G0 环境与工具链：复用一套 GCC，解剖官方库 V3.3.3
2. 70 . G1 RCU 时钟树：200MHz 是怎么算出来的
3. 71 . G2 GPIO 与 AF 复用对照：七大寄存器同名不同姓
4. 72 . G3 USART 增强点：从命名到数量的差异清单
5. 73 . G4 GD32 外设差异：有没有与一不一样
6. 74 . V0 RISC-V 工具链与启动文件
7. 75 . V1 Bumblebee 内核与 CLIC 中断
8. 76 . V2 RCU 与 108MHz
9. 77 . V3 MTIME 与裸机延时
10. 78 . V4 GPIO 最小系统

第九部分 实物实验中心

逻辑分析仪、示波器、电流表下场：UART 抓帧、PWM 实测、优先级反转复现、I2C 解码、功耗账本，每个理论结论都在真实波形与读数上钉一次。

1. 79 . 实验 E01 点亮霸天虎的 RGB 红灯
2. 80 . 实验 E02 逻辑分析仪抓 UART 帧
3. 81 . 实验 E03 示波器看 PWM
4. 82 . 实验 E04 优先级反转复现
5. 83 . 实验 E05 I2C 抓包读 EEPROM
6. 84 . 实验 E06 低功耗电流实测
7. 85 . 实验 E07 S3 板载姿态传感器
8. 86 . 实验 E08 S3 音频链路放音

第十部分 双板综合项目

把全部知识组装成两台可交付的传感器记录器：采样、滤波、队列、持久化、故障注入与恢复路径，全程可观察。

1. 87 . J1 STM32F407 传感器记录器
2. 88 . J2 ESP32-S3 传感器记录器

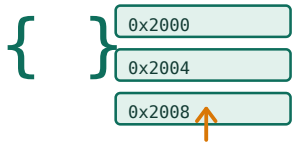
第十一部分 AI 辅助开发

把大模型变成你的结对工程师：提示词给约束、输出对手册、工具化工作流、幻觉有防御，最后用这本书自身的生产线自证。

1. 89 . A0 为什么把大模型当结对工程师
2. 90 . A1 提示词模式：三段式，把决定权从模型手里拿回来
3. 91 . A2 手册对账：模型写的寄存器代码，逐位验证给它看
4. 92 . A3 工具流：让模型读手册、跑构建、看串口
5. 93 . A4 幻觉防御：八类幻觉的类型学与验证决策树
6. 94 . A5 这本书是怎么和 AI 一起写成的：生产线的自证

附录

1. 附录 A 工程深读：从 IDLE 中断到完整帧协议的五个契约
2. 附录 B 配套工程索引（47 个可编译验证的工程）
3. 附录 C 章末自测参考答案
4. 附录 D 术语速查：一句话讲透一个名词
5. 附录 E 来源、版本与许可
6. 附录 F 常见问题 FAQ
7. 附录 G 开源项目与社区地图



PART PC

第一部分 C 语言精髓：嵌入式是另一种 C

在碰任何外设之前，把内存模型、指针、volatile、ABI、栈帧与未定义行为的地基打进肌肉记忆，每一课都能用宿主机上的反汇编与 probe 脚本复现。

> 这一部不教语法，教「同一种语言在另一种机器上的另一种脾气」。九堂课全部在宿主机上取证：probe.sh 跑出段归属，反汇编对着 -O0/-O2 讲 volatile 翻脸，AAPCS 逐寄存器对账。嵌入式 C 的地基不需要开发板，需要的是把「编译器替你做了什么」从假设变成证据。读完的标志：看到 *(volatile uint32_t*)0x40021418 = 1UL<<22 不再是咒语，你能说出地址、类型、volatile 各自管什么。

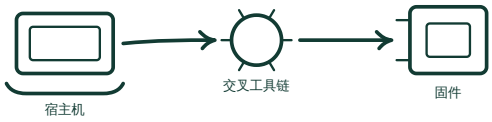
本部 9 章 · 约 390 分钟（6.5 小时）

01 . C0 为什么嵌入式 C 是另一种 C
02 . C1 内存模型：一个全局变量的三段旅程
03 . C2 指针：点灯公式的全部秘密
04 . C3 volatile：跟编译器约法三章
05 . C4 结构体与 ABI：一个结构体罩住整组寄存器
06 . C5 函数指针、状态机与环形缓冲
07 . C6 调用约定与栈帧
08 . C7 未定义行为与 MISRA-C 精要
09 . C8 串口协议：从字节流到可恢复的消息

通往单片机之路第 01 章

C0 为什么嵌入式 C 是另一种 C

难度 ★☆☆☆☆ · 约 20 分钟



本章罗盘

问题

同样一行 C，在 PC 上越界是段错误弹窗，到单片机上为什么变成隔壁外设寄存器被改写、没有弹窗只有事故？

交付

一张裸机 C 与应用 C 四维差异清单（无 OS、内存按 KB、直碰硬件、复位向量起跑），加一条用『向 0x40021418 写 32 位值』口径描述的寄存器写，并在 00-blink 三个文件里各指认一处落点。

边界

本章只立世界观：启动文件逐行走读在 B4、volatile 合同在 C3、交叉工具链配置在 B0/B1；HardFault 只交代形态，定位方法在 S16，CCM 内存不展开。

依据锚点 `a3` · `mcu-code` · `mcu-anim`

> 🎯 在电脑上，数组越界=段错误弹窗；在单片机上，数组越界=隔壁外设的寄存器被改写，电机突然转了一下——没有弹窗，只有事故。

本章精髓

1. 裸机没有"运行时爸爸"：没有 MMU、没有段错误、没有 GC，越界/野指针的后果直接落在硬件上。
2. C 在嵌入式里是"高级汇编"：每一行都能对应到确定的机器行为，这正是我们选它写驱动的原因。
3. 资源观不同：RAM 按 KB 计、Flash 按 MB 计，`malloc` 不是想用就能用。

- **能记住**：裸机 C 与应用 C 的四条差异：无 OS、内存按 KB、直接碰硬件、从复位向量起跑（即精髓三条）
- **能理解**：为什么"数组越界"在单片机上不是弹窗而是事故：没有 MMU 替你拦，写谁就是谁；为什么全局变量的初值要从 Flash 搬到 RAM
- **能用**：用"地址 + 宽度"的语言描述一次寄存器写（如"向 `0x40021418` 写 32 位值"），并在 `code/stm32/00-blink` 的三个文件里各指出一处对应

先修

- 会写基础 C（函数、指针、结构体的语法层面）。

先跑起来（10 分钟 quick win）

跳到 S0 把灯点了再回来，亲眼看到 `*(volatile uint32_t*)0x40021418 = ...` 让红灯亮灭，本章的每句话都会变得具体。

去之前先押一题：灯能亮，说明 main 函数里第一行 C 的运行前提（栈顶已装好、有初值的变量已就位）全都成立了。这些前提是谁、在哪一步做好的？答案不在你写的那份源码里。带着它去看灯，回来对第一节。

小节结构

小节	内容	四件套
没有 OS 的世界	复位向量→main 的启动事实；崩溃时没有蓝屏只有 HardFault	代码分析
内存按 KB 计	F407 的 SRAM 怎么分（栈/堆/全局）；为什么嵌入式慎用 malloc	配置
C 与硬件的直线	寄存器就是"有名字的地址"；volatile 预告（C3 详讲）	代码分析
工具链即世界观	交叉编译四件套简介，指向 B0	库解析

一、没有 OS 的世界

电脑上运行一个 C 程序，`main` 之前站着一整座运行时大厦：装载器把 ELF 映射进虚拟地址空间，动态链接器解析符号，CRT 初始化堆，最后才 `__libc_start_main → main`。程序崩溃时，内核送来段错误弹窗，进程尸体被收走，世界照旧。

单片机上没有这座大厦。上电那一刻，Cortex-M4 内核只做两件硬件规定的事（RM0090 §12.2，具体值以你手上芯片的参考手册为准）：

1. 从 Flash 起始的**向量表第 0 项**取出初始栈顶地址，装进 MSP；
2. 从**第 1 项**取出复位处理函数 `Reset_Handler` 的入口地址，跳过去。

没有装载器、没有堆初始化、没有任何人替你准备 C 语言的运行前提。`Reset_Handler` 必须亲手做完三件事，`main` 里的第一行 C 才合法（本工程 `startup_stm32f407xx.s:35-80`，逐行走读在 B4 启动过程）：

- 使能 FPU（复位后 CP10/CP11 默认关闭，浮点指令一碰就 UsageFault）；
- 把 `.data` 段（有初值的全局变量）从 Flash 搬到 RAM；
- 把 `.bss` 段（无初值的全局变量）清零。

崩溃的形态也完全不同。越界写在 PC 上：内核通过 MMU 拦截，进程死给你看。越界写在裸机上：没有 MMU，写谁就是谁：写到隔壁外设的寄存器，电机可能真的转一下；写到向量表映射区，下一次中断就跳进荒野。内核唯一的兜底是 HardFault：一张没有弹窗、没有日志、只有调试器才看得到的"死亡通知"。

取证（xPack arm-none-eabi-gcc 15.2.1，`-mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard`，完整命令见工程 Makefile）：

```
$ mingw32-make            # 在 code/stm32/00-blink 下，零警告
   text    data     bss     dec     hex filename
    660       0    1536    2196     894 build/blink.elf
```

660 字节代码 + 1536 字节 `.bss`（其中 1024 字节是链接脚本预留的最小栈、512 字节最小堆），这就是"点灯"在最裸的世界里的全部开销。没有 OS 的世界，从 2196 字节开始。

二、内存按 KB 计

PC 上"内存不够"是个遥远概念；F407ZGT6 上它是链接脚本里两行硬数字（`stm32f407xx.ld:22-29`）：

区域	起始	容量	住什么
FLASH	`0x08000000`	1024 KB	`.text` 代码、`.rodata` 常量、`.data` 的初值仓库
RAM	`0x20000000`	128 KB	`.data`、`.bss`、栈（从尾部向下长）、堆

（F407 另有 64KB CCM @ `0x10000000`，CPU 直连、DMA 够不着，本章不展开。）

128 KB 是什么概念：一张未压缩的 320×240 灰度图就要 76.8 KB。于是三条应用层的"常识"全部作废：

- **`malloc` 不是想用就能用**。堆没有操作系统替你扩，碎片化了没有交换分区兜底；本工程给堆的"最小预算"只有 512 字节（`_Min_Heap_Size = 0x200`），链接时 RAM 摆不下就直接报错，把"运行时踩内存"提前成"链接期拒签"。
- **栈是奢侈品**。默认 1 KB 起步，递归深一点、局部数组大一点就撞进 `.bss`，而且撞得无声无息。
- **全局变量的初值要付两份钱**。`int x = 42;` 运行时在 RAM，但 `42` 这个初值还占 4 字节 Flash，启动时由 `Reset_Handler` 搬过去（C1 逐字节追踪这三段旅程）。

上面的 size 输出就是证据：`data 0` 表示本工程没有带初值的全局变量，`bss 1536` 几乎全是链接脚本强制预留的栈/堆，点灯程序本身一个全局变量都没用。

三、C 与硬件的直线

裸机编程的全部秘密就一句：**寄存器就是有名字的地址**。GPIOF 的 BSRR 寄存器住在 `0x40021418`（本工程 `main.c:29,34` 用 `GPIOF_BASE + 0x18` 拼出来，地址经 ST 官方 CMSIS 头文件核对），对它写 32 位值，红灯亮灭：

```
#define GPIOF_BSRR  (*(volatile uint32_t *)(GPIOF_BASE + 0x18UL))
/* ... */
GPIOF_BSRR = (1UL << (LED_R_PIN + 16));  /* BR6：PF6 拉低 → 红灯亮 */
```

这行赋值编译出来就是一条 `str` 指令——高级语言到硬件之间**没有中间商**。这正是嵌入式选 C 的原因：每一行都能对应到确定的机器行为，性能与副作用都可预测。

注意那个 `volatile`，它不是装饰：硬件寄存器会被"程序之外的力量"改变（状态位被外设置位、输入数据寄存器随引脚变化），编译器的常规假设"没人动的内存不会变"在这里不成立。`volatile` 是告诉编译器"每次都真去访存，不许缓存、不许合并、不许删"的合同条款。为什么是三条、边界在哪，C3 用两套工具链的反汇编逐条取证。本章先立下这条：**凡指向硬件寄存器的指针，必须戴 volatile**。

四、工具链即世界观

你在 PC 上敲 `gcc hello.c`，编译器默认目标是"本机"。嵌入式里这句默认不再成立：你在 x86_64 的电脑上，为一颗 Cortex-M4 的芯片产代码，这叫**交叉编译**，工具链也从此变成四件套（以本工程 Makefile 为实例）：

角色	本工程里的身份	干什么
编译器	`arm-none-eabi-gcc` 15.2.1	C→汇编→目标文件，`-mcpu=cortex-m4` 指明目标内核
链接器	同上驱动 `-T stm32f407xx.ld`	按链接脚本把 `.o` 拼成落在 `0x08000000` 的固件
二进制工具	`arm-none-eabi-objcopy / objdump / size`	ELF↔bin/hex 转换、反汇编取证、体量统计
烧录/调试	OpenOCD + ST-Link + `arm-none-eabi-gdb`	把 ELF 写进 Flash，从复位向量单步

"即世界观"的意思是：工具链的每一项配置都在替你回答一个世界观问题：目标是谁（`-mcpu`）、代码放哪（链接脚本）、初值谁来搬（启动文件）、错了怎么看（GDB）。把这些串起来的完整走读在 B0 交叉工具链 与 B1 四步构建，本章只需建立直觉：**同一份 C 源码，换一组 `-mcpu` 和链接脚本，就是另一颗芯片的世界**。

附录：工程完整源码

本章引用的"最小裸机程序"全文如下（仓库 `code/stm32/00-blink/`，`mingw32-make` 一键构建）：

main.c：开时钟、配模式、控数据，三步点灯。

<<< ../../code/stm32/00-blink/main.c

startup_stm32f407xx.s：复位向量起跑前的全部准备（FPU、`.data` 搬运、`.bss` 清零、向量表）。

<<< ../../code/stm32/00-blink/startup_stm32f407xx.s

stm32f407xx.ld：链接脚本管 FLASH/RAM 两块地的划分与 `.data` / `.bss` /栈堆的安置。

<<< ../../code/stm32/00-blink/stm32f407xx.ld

附属入口：去仓库看 00-blink 工程（star / 提 issue）。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **应用 C 出错弹窗，裸机 C 出错冒烟。** 写每一行前问一句：这行代码落在硬件的哪个地址上？

延伸：嵌入式 C 与应用 C 的四维差异动画见 C0 动画；内存布局与段分配在 C1 展开；volatile 的总线纪律在 C3 详解。

实物实验

- 用 E01 的 GDB 环节：故意把 BSRR 地址写错一位（`0x40021419`），烧录观察：灯不亮，芯片也不报错。体会"硬件不弹窗"。

常见坑

- **把 PC 习惯带过来**：`printf` 调试、无限内存假设、阻塞死等，裸机里每个都要重新挣得。
- **以为全局变量"天生在 RAM"**：它的初值其实存在 Flash，启动代码搬过来的（C1 的料）。
- **忽略初始化顺序**：main 之前还有 Reset_Handler 的一整套准备（B4）。

短自测

1. 上电后、进入 `main` 之前，`Reset_Handler` 必须完成哪三件事，缺了哪一件会让哪类 C 代码出错？

▼ **答案**
① 使能 FPU：缺了它，任何浮点指令触发 UsageFault；② 把 `.data` 初值从 Flash 搬到 RAM：缺了它，有初值的全局变量读到的是 RAM 里的随机值；③ 清零 `.bss`：缺了它，无初值全局变量不为 0，违反 C 标准对静态存储期的承诺。（对应 `startup_stm32f407xx.s:35-72`。）

2. 用"地址+宽度"的语言描述这条语句的硬件行为：`GPIOF_BSRR = (1UL << (6 + 16));`

▼ **答案**
向地址 `0x40021418`（GPIOF 的 BSRR 寄存器）做一次 32 位写，写入值 `0x00400000`：bit22（BR6）为 1，表示把 PF6 输出拉低，共阳接法的红灯点亮；写 0 的位无影响，所以不需要读-改-写。

3. 本工程链接结果 `text 660 / data 0 / bss 1536`。为什么 `data` 是 0？`bss` 里那 1536 字节主要是谁？

▼ **答案**
`data = 0` 因为工程没有"带初值的全局变量"（所有常量要么是宏、要么是局部量）。`bss 1536` 几乎全是链接脚本强制预留的最小堆（`_Min_Heap_Size = 0x200`，512 字节）+ 最小栈（`_Min_Stack_Size = 0x400`，1024 字节），这是"链接期拒签"机制占的位子，不是真正的全局变量。

4. 同事说："寄存器指针不加 `volatile` 也能编译过，先不管。" 用本章与 C3 的取证逻辑反驳他，一句就够。

▼ **答案**
"能编译过"与"能对"是两回事：不加 `volatile`，编译器有权把多次访存合并/缓存/删除：`-O2` 下轮询状态位的循环会被优化成只读一次甚至整个消失（C3 的反汇编实测），程序在真硬件上死等，而源码看起来毫无问题。

对照表：本章概念 → 仓库落点

本章概念	仓库落点
向量表 = Flash 最前面的地址条	`code/stm32/00-blink/startup_stm32f407xx.s:96-113`（`g_pfnVectors` 开头两项：栈顶、`Reset_Handler`）
Reset_Handler 三件套（FPU/搬 `.data`/清 `.bss`）	`code/stm32/00-blink/startup_stm32f407xx.s:35-72`，走读 B4 第三节
寄存器 = 有名字的地址（`0x40021418`）	`code/stm32/00-blink/main.c:29,34,60-62`
内存按 KB 计（FLASH 1MB / RAM 128KB / 栈 1KB / 堆 512B）	`code/stm32/00-blink/stm32f407xx.ld:19-29,116-125`
`size` 取证：text 660 / bss 1536	`code/stm32/00-blink/Makefile` 构建输出（gcc 15.2.1，零警告）
volatile 的合同	C3 volatile，六变体反汇编对照
初值从 Flash 到 RAM 的三段旅程	C1 内存模型
交叉编译四件套	B0 工具链、B1 四步构建

你做到了

- 建立了"每行 C 都落在硬件上"的世界观；
- 拿到 C 篇的地图：C1 内存 → C2 指针 → C3 volatile → C6 栈帧，逐章点亮。

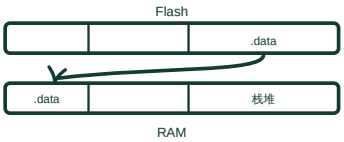
回到开场那一下电机的抖动：裸机不给弹窗，不是因为脾气好，而是 C 和硬件之间根本没人值守。你现在知道上电后内核只做两件事、main 之前是谁在铺路，下一次再遇到「没有事故现场的事故」，就知道该去向量表和启动文件里找证据，而不是在业务代码里乱翻。

✅ 下一站：C1 内存模型：一个全局变量从源码到上电，三段旅程逐字节跟上。

通往单片机之路第 02 章

C1 内存模型：一个全局变量的三段旅程

难度 ★★☆☆☆ · 约 65 分钟



本章罗盘

问题

int g_init = 5 上电瞬间 RAM 里为什么不是 5？初值存在哪、谁搬来，.bss 凭什么一个字节 Flash 都不占？

交付

一张 11 个对象的段归属实测表（nm 地址/字母/占哪种介质）加一笔 const 经济学手算：256 字节表 .data 256→0、.text 148→404，sh probe.sh 一条命令复现。

边界

结论以本工程链接脚本为准（.rodata 并入 .text，nm 字母会骗人，判断介质只看地址前缀）；_Min_Stack 只是链接期下限，运行期栈溢出它不管；CCM 与 DMA 的坑在 S8。

依据锚点 `c3 · c7 · mcu-code · mcu-anim`

> 🎯 `int led = 5;` 在你的固件里其实**没有**这个 5。RAM 里那位 `led` 上电瞬间还是一堆随机值，5 住在别处，靠人搬过来。谁搬的、从哪搬、搬几个字节？本章把这三问一次答完。

本章精髓

1. **为什么段要分裂**：Flash 掉电不丢但写不了，RAM 能写但断电即失。"有初值的全局变量"必须同时住两边：RAM 里住本体、Flash 里住初值，这份"双重户籍"就叫 `.data`；没有有意义初值的只登记不给货，就叫 `.bss`。段不是语法分类，是两种介质的分工。
2. **为什么 C 的承诺需要别人兑现**：语言保证 `int g_init = 5;` 一执行就能读到 5，硬件可不保证。兑现承诺的是启动文件里那个不到 40 字节的拷贝循环，一出链接器与启动代码合演的戏：链接脚本把"行李"和"人"分成两个地址（VMA/LMA），`Reset_Handler` 照着这两个地址把行李搬进来。
3. **为什么 `const` 在嵌入式里值钱**：PC 上 `const` 是写给同事看的，MCU 上是写给链接脚本看的，它决定一块表是占 Flash 还是占 Flash 又占 RAM。本章用 256 字节的表实测这个词的价钱。

- **能记住**：五个住处（`.text` / `.rodata` / `.data` / `.bss` /栈堆）各自放什么；`_sidata/_sdata/_edata/_sbss/_ebss` 五个符号各是什么
- **能理解**：为什么 ELF 里 `.data` 的运行地址是 RAM、文件偏移却在 Flash；`nm` 的字母为什么会"骗人"（`T` 也可以是数据）
- **能用**：`size` / `nm` / `readelf` / `objdump` 四条命令查自己工程里任何变量的住址与花费；改一行 `const` 前后能算出差额

先修

- C0 嵌入式 C 是另一种 C；B1 四步构建 可并行读。段是编译与链接两步的产物，链接脚本逐行讲解在 B3。

先跑起来（10 分钟 quick win，不用开发板）

```
cd code/c/01-memory-model
sh probe.sh mem          # 只跑段归属这一组
```

看到 `----- readelf -l -----` 那一行了吗？

```
LOAD  0x002000  0x20000000  0x080002a0  0x0000c  0x0002c RW
      ↑文件偏移  ↑运行地址     ↑存储地址     ↑文件里   ↑RAM 里
```

同一份东西，**运行地址在 RAM、存储地址在 Flash**，且文件里只有 12 字节、RAM 里要 44 字节。那 12 与 44 的差、以及"0x20000000 怎么会有个 Flash 的孪生地址"，就是本章要讲完的全部内容。跑不出来也别慌，下面每一步都带命令，照着敲即可。

跑之前先押两问：一个带初值的全局变量，此刻躺在固件的哪个介质里？上电之后它又住在哪？两问若答成同一个地方，第一段输出会当场纠偏。

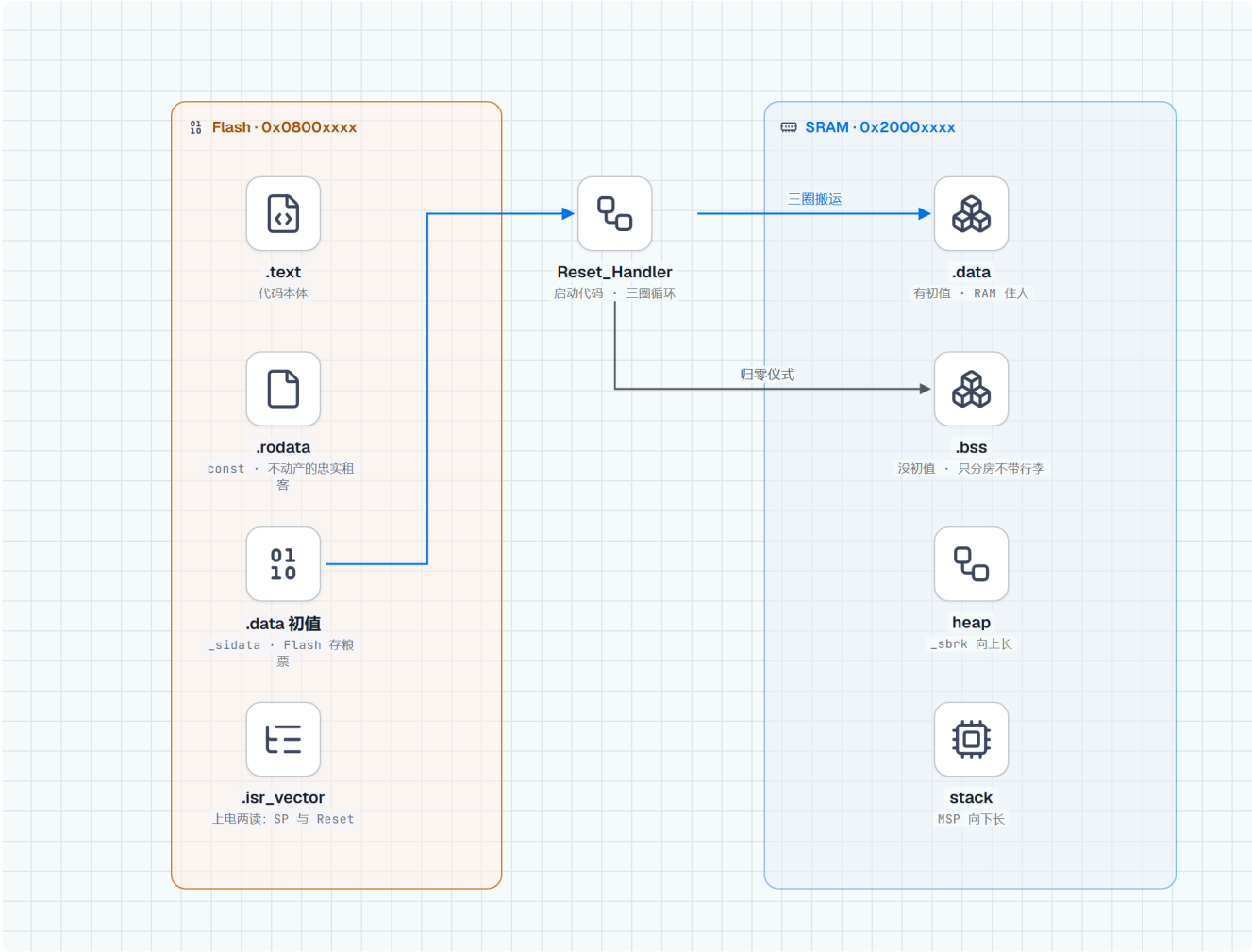
小节结构

小节	内容	四件套
一、五段论	`.text` / `.rodata` / `.data` / `.bss` /栈堆逐段讲解 + F407 地址实拍	配置
二、三段旅程	以 `int g_init = 5` 追踪：源码 → ELF 的 `.data` +LMA → 启动拷贝 → RAM（含两地生活全景动画）	代码分析
三、const 经济学	`const uint8_t lut[]` 为什么该留 Flash；256 字节对照实验	代码分析
四、nm/map 实操	亲手查符号地址；与链接脚本对照；字母与地址谁可信	代码分析

> 本章不涉及引脚设置，"库解析"在第四节以链接脚本为对象：**内存布局不是编译器单独决定的，链接脚本有一半话语权。**

一、五段论：先看地图，再认门牌

一张地图先收下：Flash 存粮，SRAM 住人，搬运工是 Reset_Handler。



F407 内存地图：Flash 段落与 SRAM 段落各住着谁，Reset_Handler 负责搬运与归零

`code/stm32/00-blink` 那份链接脚本把 F407 的地址空间切成三块（`stm32f407xx.ld:24,28` 与文件头注释，数值以本工程脚本为准，芯片侧口径见 ST datasheet）：

区域	起始	本工程给的容量	谁能直接访问
FLASH	`0x08000000`	1024K	CPU；掉电不丢
RAM	`0x20000000`	128K	CPU + DMA
CCM	`0x10000000`	64K（**本脚本未使用**）	只有 CPU——DMA 够不着（见 S8 DMA）

再看编译产物里真实的段表（`sh probe.sh mem`，`readelf -S` 摘五行）：

```
  [ 1] .isr_vector       PROGBITS   08000000 001000 000188 00   A
  [ 2] .text             PROGBITS   08000188 001188 000118 00  AX
  [ 3] .data             PROGBITS   20000000 002000 00000c 00  WA
  [ 4] .bss              NOBITS     2000000c 00200c 000020 00  WA
  [ 5] ._user_heap_stack NOBITS     2000002c 00202c 000604 00  WA
```

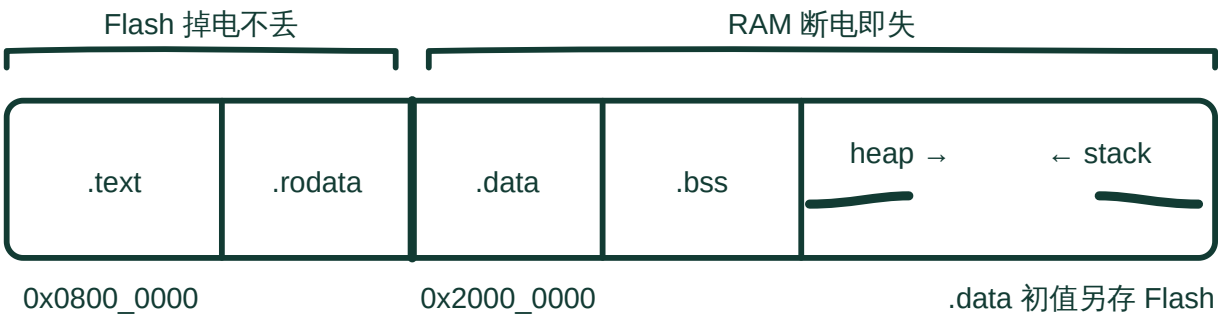
每一行都值得停一下：

- **`.isr_vector` 在 `0x08000000`，392 字节**：向量表。Cortex-M 复位后**先**从这里第 0 个字取栈顶、第 1 个字取复位入口。这时候 C 运行时还不存在，`.data` 里一个变量都没就位。
- **`.data` 的地址是 `0x20000000`（RAM），类型却是 `PROGBITS`（文件里有货）**：一个住在 RAM 的段，凭什么在二进制文件里占 12 个字节？因为那 12 字节是它的**初值备份**。第二节的三段旅程就从这条看起来矛盾的行列开始。
- **`.bss` 是 `NOBITS`**：文件里一个字节都不给。它只是链接器给启动代码留的一张便条："这段 RAM 请你清零"。`.bss` 的大小（32 字节）**不占 Flash，只占 RAM**。
- **`._user_heap_stack` 也是 `NOBITS`，`0x604` 字节**：链接脚本按 `_Min_Heap_Size = 0x200` + `_Min_Stack_Size = 0x400` 预留的下限（`stm32f407xx.ld:19-20,117-125`），末尾那个 `+4` 是 8 字节对齐的填充（`0x2000002c` 起、`ALIGN(8)` → `0x20000030`）。栈从 `_estack = 0x20020000`（RAM 顶，`ld:16`）向下长，堆从 `0x20000030` 向上长，中间这一百多 KB 是它们还没碰面的空地。
- **`.text` 里混着 `.rodata`**：上面五行里**没有 `.rodata`**。因为本工程链接脚本把 `*(.rodata)`、`*(.rodata*)` 收进了 `.text` 输出段（`stm32f407xx.ld:48-49`）。这是本站要提醒的一处工程取舍：只读数据放在可执行段里，功能完全正常，但"代码段只读、数据段另立"的这条边界就没了。**想验证代价很小**：`sh probe.sh rodata` 会从原脚本现场生成一份把 `.rodata` 单列的变体（原件不动），同一批符号的地址一格没动、字母却变了：

`原脚本（并进 .text）：` `08000298 T g_ro 0800029c T k_tab` `变体脚本（单列）：` `08000298 R g_ro 0800029c R k_tab` `[ 2] .text PROGBITS 08000188 001188 000108 00 AX` `[ 3] .rodata PROGBITS 08000290 001290 000010 00 A`

这就是"字母会骗人"的实锤：**T 与 R 说的是"在哪个输出段、什么属性"，不是"是函数还是数据"。**

把五段钉牢：**能执行的进 `.text`，只读数据进 `.rodata`（本工程并进 `.text`），有初值的活数据进 `.data`（双户籍），零初值的进 `.bss`（单户籍），函数里的自动变量既不在 ELF 也不在段表，它在栈上。**



五段内存地图：代码与只读数据住 Flash，已初始化数据、零初始化数据、堆与栈住 RAM，堆向上长、栈向下长；.data 的初值另存一份在 Flash

全景动画：一个变量的两地生活

下面这张图把本章要讲的五个瞬间压进一轮循环：上电时 RAM 一个字节都不可信 → `Reset_Handler` 从 `0x080002a0` 逐字搬 12 字节进 `0x20000000` → 顺手把 `0x2000000c..0x2c` 这 32 字节扫成 0 → 立起栈顶 `_estack` 让堆与栈相向生长 → 进 `main` 后所有写只落 RAM、永不回写 Flash。**图里每个数字都是 `mem_probe.elf` 的实测值**，`sh probe.sh mem` 一条命令复现。

全局变量的两地生活动画

二、三段旅程：`int g_init = 5;` 的一生

取证工程 `mem_probe.c` 里一次摆了 11 个对象。本节只跟 `g_init` 走完全程，末尾回头清点其余的。

第一程：源码里，它只是一行声明

```
int g_init = 5;              /* 文件作用域，有初值 */
```

此刻它有三个信息：类型（`int`，4 字节）、初值（5）、**存储期**（全程序）。第三个信息决定了它后面所有的遭遇：存储期是"全局"的变量，编译器必须为它**预先定址**，不能像自动变量那样运行时算。

第二程：ELF 里，它有了"户口"和"仓库"两个地址

编译链接后先查门牌：

```
$ arm-none-eabi-nm -n --print-size build/mem_probe.elf | grep g_init
20000000 00000004 D g_init
```

地址 `0x20000000`，RAM 开头第一个字节。字母 `D`：住在数据段、全局可见。**这是它出生后的住址，不是它出生前的所在地。**

出生前它在哪？看程序头：

```
$ arm-none-eabi-readelf -W -l build/mem_probe.elf | sed -n '/Program Header/,/Section to Segment/p'
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  LOAD           0x001000 0x08000000 0x08000000 0x002a0 0x002a0 R E 0x1000
  LOAD           0x002000 0x20000000 0x080002a0 0x0000c 0x0002c RW  0x1000
  LOAD           0x00002c 0x2000002c 0x080002ac 0x00000 0x00604 RW  0x1000
```

第二行是本章的心脏：

字段	值	含义
VirtAddr	`0x20000000`	**VMA**：运行时这段要在哪（RAM）
PhysAddr	`0x080002a0`	**LMA**：烧录时这段实际压在哪（Flash 尾部）
FileSiz	`0x0000c`	文件里真有 12 字节
MemSiz	`0x0002c`	RAM 里要 44 字节（`.data` 12 + `.bss` 32）

烧进 Flash 的固件里，`0x080002a0` 处躺着 12 个字节，它们是 `.data` 段的"初值备份"。把这份备份单独抠出来看（`objcopy -j .data -O binary`，或直接 `objdump -s -j .data`）：

```
Contents of section .data:
 20000000 05000000 01000000 68690000           ........hi..
```

四个字节四个字节地读：`0500 0000` = `g_init` 的 5；`0100 0000` = `g_flag` 的 1；`6869 0000` = `g_msg` 的 `"hi"`（`68 69 00`）加一个对齐填充字节。正好 12 字节。**`.bss` 的对象一个都不在这里**：它们的初值全是 0，存进 Flash 是纯浪费空间，所以只在符号表里留个名字。

为什么 `VirtAddr` 和 `PhysAddr` 能不一样？链接脚本里两个词（`stm32f407xx.ld:91-100`）：

```
  _sidata = LOADADDR(.data);
  .data : {
    . = ALIGN(4);  _sdata = .;  *(.data)  *(.data*)  . = ALIGN(4);  _edata = .;
  } >RAM AT> FLASH
```

`>RAM` 说"运行时在 RAM"（定 VMA），`AT> FLASH` 说"存储时在 Flash"（定 LMA），`_sidata = LOADADDR(.data)` 把这个 LMA 变成一个符号交给启动代码。三个符号对上实测：`_sidata = 0x080002a0`（Flash）、`_sdata = 0x20000000`、`_edata = 0x2000000c`。

第三程：上电瞬间，搬运工逐字拷进 RAM

`Reset_Handler` 的反汇编里，那几个 `.word` 就是链接脚本发下来的地址条（`startup_stm32f407xx.s:35-80`，逐行走读在 B4 启动过程 第三节）：

```
 8000270:  20020000        @ _estack  → 装栈顶
 8000274:  e000ed88        @ SCB->CPACR → 使能 FPU
 8000278:  20000000        @ _sdata   → 拷贝目标起点
 800027c:  2000000c        @ _edata   → 拷贝目标终点
 8000280:  080002a0        @ _sidata  → 拷贝源起点（Flash！）
 8000284:  2000000c        @ _sbss    → 清零起点
 8000288:  2000002c        @ _ebss    → 清零终点
```

于是循环本体 `ldr r4, [r2, r3] / str r4, [r0, r3] / adds r3, #4` 从 `0x080002a0` 读、往 `0x20000000` 写，跑 `(0x2000000c - 0x20000000) / 4 = 3` 圈，正好 12 字节。接着 `str.w r3, [r2], #4` 把 `0x2000000c..0x2000002c` 这 32 字节扫成 0。然后 `bl main`。

`main` 里第一次读 `g_init` 拿到 5，靠的全是这 3 圈 + 8 圈。 这就是"三段旅程"：源码声明 → 链接器分两个地址 → 启动代码搬运。

> 💡 **提示 · 顺带看懂 Flash 里的字节**
>
> `0500 0000` 而不是 `0000 0005` ——Cortex-M 是小端：低位字节放低地址。`objdump -s` 的右边 ASCII 栏里那个 `hi` 是 `"hi"` 的两个字符，剩下的点表示不可打印。**看二进制文件不神秘，只是没人告诉你列宽。**

回头清点：另外八个对象的归宿

`nm -n --print-size` 的实测输出（另有 91 行同名 IRQ 别名指向 `0800028c`，此处省略）：

```
08000298 00000004 T g_ro          # const int      → Flash
0800029c 00000004 T k_tab         # const uint8_t[4] → Flash
080002a0 A _sidata
20000000 00000004 D g_init        # int =5         → .data
20000004 00000004 D g_flag        # volatile int =1 → .data
20000008 00000003 D g_msg         # char[] = "hi"  → .data（3 字节 + 对齐）
2000000c 00000004 B g_zero        # int =0         → .bss  ★
20000010 00000004 B g_uninit      # int            → .bss
20000014 00000010 b s_buf         # static uint8_t[16] → .bss
20000024 00000008 b s_local.0     # 函数内 static[8] → .bss
20020000 R _estack
```

三个反直觉的答案都在这张表里：

1. **`int g_zero = 0;` 竟在 `.bss`**。它明明写了初值。但 GCC 一看初值是 0，"清零"这件事 `.bss` 顺便就办了，何必占 Flash 四个字节再拷一遍？于是**"写没写 `=` "不重要，"初值是不是全零"才重要**。后果：`g_zero` 的值由启动代码现场清零产生，`g_init` 的值由 Flash 里的备份搬运产生，两条路来源不同，`main` 里看起来一样自然。
2. **`volatile int g_flag = 1;` 照样在 `.data`**。`volatile` 不改住址，只改访问方式：它管的是"每次都要真访存"（C3），跟"住在哪"是两个正交的问题。**volatile 是访问权限，段是存储地点。**
3. **函数里的 `static uint8_t s_local[8]` 在 `.bss`，而同一个函数里的 `int auto_var = 3;` 连符号都没有**。`nm build/mem_probe.elf | grep auto_var` 查无此人：自动存储期的变量活在栈上，链接期根本不存在。你只能从 `probe` 的反汇编里看到它（`-O0` 实测）：

`asm 80001a2: movs r3, #3 80001a4: str r3, [r7, #0] @ 栈帧里的一个槽，函数返回即蒸发`

写在函数里的 `static` 是"寿命全程序、可见性本文件"，所以它有固定地址（`b` = 局部符号）；`auto_var` 只有"这次调用的栈帧"这个临时地址。**`static` 与 `auto` 的差别是住址的差别，不是位置的差别。**

还有第四点最容易看错：`g_ro` 的地址是 `0x08000298`（Flash 里！），`nm` 的字母却是 `T`（"代码段符号"）。**因为本工程的链接脚本把 `.rodata` 并进了 `.text` 输出段**（`ld:48-49`），链接器眼里它就在代码段里。紧跟着的一行反证（`sh probe.sh mem` 的"字面量住哪"那一节）：

```
8000288 2c000020 fee70000 666c6173 68000000   ,.. ....flash...
```

`0x08000290` 起是那串 `"flash"` 字面量（源码里 `const char *lit = "flash";`），**没有符号名，但确实在 Flash**。而 `lit` 这个指针本身是自动变量，活在栈上。**同一行代码里，字符串和指针分住两种介质**，这是本章最省话的一张图。

三、const 经济学：一个词值 256 字节 RAM

一张 256 字节的查找表，两种写法，各编译一次（`sh probe.sh lut`，实测值）：

写法	`.text`	`.data`	`lut` 的 `nm` 行	第二个 LOAD 段
`uint8_t lut[256] = {1,2,3};`	148	**256**	`20000000 00000100 D lut`	`VirtAddr 0x20000000 PhysAddr 0x0800021c FileSiz 0x100 MemSiz 0x700`
`const uint8_t lut[256] = {1,2,3};`	**404**	**0**	`0800021c 00000100 T lut`	`VirtAddr 0x20000000 PhysAddr 0x0800031c FileSiz 0x00000 MemSiz 0x600`

三个数对着读：

- **RAM 从 256 字节变成 0 字节**：`.data` 那一列从 256 塌到 0，第二个 LOAD 段的 `FileSiz` 从 `0x100` 变成 `0x00000`，Flash 里不再需要为它存备份，启动代码也少拷 64 个字。**一个 `const` 换 256 字节 RAM**，在 128K 的 F407 上不算什么，在 RAM 只有几十 KB 的小型号上就是两位数的家底。同一笔交易的金额由你手上的 RAM 总量决定，不由 `const` 决定。
- **Flash 从 148 涨到 404**：`.text` 多出 256 字节（404 − 148 = 256），表还在，只是**只住 Flash 了**。所以"加 const 省一半"的说法要看口径：省的是 RAM 那一半，Flash 那一半本来就要花（初值也得存）。
- **`MemSiz 0x700 → 0x600`**：可写版的 `MemSiz` = 256(`.data`) + 0x600(堆栈预留) = 0x700；只读版只剩堆栈预留。链接器的 RAM 记账直接少了一笔。

为什么 const 数组**该**留 Flash：查表是只读行为，从 Flash 读和从 RAM 读，Cortex-M 都支持直接寻址（XIP），结果一模一样；差别只在访问速度（Flash 有等待周期，RAM 没有；量化数字要按你的 `FLASH_ACR` 配置与主频实测，这里不给结论）。**用一点访问延迟，换整块 RAM 归还**，这是嵌入式里最划算的一笔常规交易。

反过来，什么时候 `const` 不该乱加？表要被运行时改写（比如自动校准出来的补偿表）就不能 `const`，只能 `__attribute__((section(".data")))` 或者干脆普通全局；`const` 数组想"存初值 + 运行时改"就得自己写"从 Flash 模板拷一份到 RAM 工作区"的两段式，这正是 P10 Flash/NVS/OTA 里出厂参数的标准做法。

四、nm/map 实操：把符号表变成审计工具

前面三节的所有结论都来自四条命令。本节把它们排成一张可背的检查单（`sh probe.sh` 会连着跑完）：

想知道	命令	实测里看哪一行
每段多大、住在哪	`size -A build/mem_probe.elf`	`.data 12 / .bss 32 / ._user_heap_stack 1540`
某个变量住哪个段、几个字节	`nm -n --print-size build/mem_probe.elf`	`20000008 00000003 D g_msg`（3 字节，不是 4）
文件里到底有几字节货	`readelf -W -l build/mem_probe.elf`	第二 LOAD 行 `FileSiz 0x0000c / MemSiz 0x0002c`
初值备份长什么样	`objdump -s -j .data build/mem_probe.elf`	`05000000 01000000 68690000`
谁在拷它	`objdump -d build/mem_probe.elf | sed -n '/<Reset_Handler>:/,/^$/p'`	字面量池里 `080002a0`
全工程最大对象是谁	`less build/mem_probe.map`（`-Wl,-Map=...` 产出）	`mem_probe.map` 里 `.bss` 段最大项是 `s_buf` 16 字节

字母表背这张（大写=全局可见，小写=局部/`static`）：

字母	含义	本章实例
`T` / `t`	代码段（含本工程并入的 `.rodata`）	`T probe`、**`T g_ro`（数据！）**
`D` / `d`	`.data`（有非零初值）	`D g_init`（大写=全局可见；同一声明加 `static` 就变小写 `d`，与下面的 `b` / `t` 同一套规则）
`B` / `b`	`.bss`（零初值）	`B g_zero`、`b s_buf`
`R` / `r`	只读段	`R _estack`、`r g_pfnVectors`
`A`	绝对值符号（不是地址，是常量）	`A _sidata`、`A _Min_Heap_Size`
`W` / `w`	弱符号	`W Reset_Handler`、`W GPIOE_IRQHandler`

两条读表纪律：

1. **字母会骗人，地址不会。** 判断"占 Flash 还是占 RAM"唯一可靠的依据是地址前缀：`0x0800xxxx` 是 Flash，`0x2000xxxx` 是 RAM。`g_ro` 顶着 `T` 却住在 Flash，就是 `ld:48-49` 那两行的直接后果；`sh probe.sh rodata` 生成的变体脚本（把 `.rodata` 单列）里，同一个符号字母变成 `R`、地址一格不动。另一个方向的验证也很快：把 `g_init` 加上 `static` 重新链接，同一地址 `0x20000000`，字母从 `D` 变 `d`，**变的只是可见性，住址没变。先查脚本，再下结论。**
2. **`nm` 的总和不等于固件大小。** `readelf -S` 里那些 `.debug_*`（本工程 `size -A` 的 Total 是 26775，而真正进 Flash+RAM 的只有 `392+280+12+32+1540 = 2256`）是调试信息，`objcopy -O binary` 之后全部蒸发。想知道烧进去多少字节，用 `size -A` 挑那五行加总，或者看 `.bin` 的文件大小。别拿 Total 吓人（也**别拿它做站内统计**，站内数字一律走 `npm run docs:gen`）。

`_Min_Stack_Size` 那 0x400（1KB）是**下限检查**，不是上限：链接期只要 RAM 剩余装得下 `._user_heap_stack` 就放行，运行期栈真溢出它不管。本工程实测高水位：`_ebss` 之后预留段跑到 `0x20000630`，离 `_estack = 0x20020000` 还有很大空地——空地多大？`(0x20020000 - 0x20000630) / 1024 ≈ 126.5 KB`。想把这个数变成硬约束，得自己在链接脚本加 `. = ASSERT(...)`，见 B5 map 与体积。

附录：工程完整源码

十一个对象的段归属现场（正文所有地址、大小、字母都出自它）：

<<< ../../code/c/01-memory-model/mem_probe.c

const 对照实验：

<<< ../../code/c/01-memory-model/lut_probe.c

一条命令跑完全部取证（正文每条命令的包装在这里）：

<<< ../../code/c/01-memory-model/probe.sh

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **有初值住 `.data`（Flash 存粮票，RAM 住人），没初值住 `.bss`（只分房不带行李），const 留 Flash（不动产的忠实租客）。** 粮票由 `_sidata` 发、`_sdata.._edata` 收货，搬运工是 `Reset_Handler` 那三圈循环。 **判断占哪种介质，看地址前缀，不看 `nm` 字母。**

延伸：内存模型与链接脚本（B3）直接对应；`.data` 搬运由启动代码（B4）完成；指针操作这些地址（C2）是寄存器访问的基础。

实物实验

- **本机已全部实测（不需要板子）**：`cd code/c/01-memory-model && sh probe.sh`，把 `nm` 里 `g_zero` 那行、`readelf -l` 第二行的 `FileSiz/MemSiz`、`.data` 那 12 个字节抄进实测笔记，注明工具链版本（本机 xPack `arm-none-eabi-gcc 15.2.1`，2026-10-06）。
- **板上（三步，编译期已核实、肉眼现象待回填）**：1. GDB 里 `p &g_init` 与 `p _sidata`，一个 `0x2000xxxx`、一个 `0x0800xxxx`，"人在 RAM、粮票在 Flash"落在两条命令上；2. `set g_init = 99`、`p g_init` 确认改了，`monitor reset` + `load` 之后再 `p g_init`，**又变回 5**，因为 Flash 里的备份没动；3. 把 `Reset_Handler` 的拷贝循环上下断点（`b *0x0800024c` 那种形式，地址按你的构建查 `nm`），单步看 `p g_init` 从垃圾值变成 5 的那一刻。
- 观测点：GDB/`p`，不需要示波器。
- 预期现象：第 2 步复位后回到 5，这一条把"段"从抽象名词变成能按出来的事实。

> ⚠️ **注意 · 本轮取证状态**
>
> 上述第 1–3 步需要接 F407 与 ST-Link，本轮写这一版时手边未接入，**板上现象待回填**；正文里所有地址、字节数、段大小均已在 `xPack arm-none-eabi-gcc 15.2.1`（`-mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard -O0 -g`，链接 `code/stm32/00-blink/stm32f407xx.ld`）下实测，命令与输出记录在 `code/c/01-memory-model/README.md`。

常见坑

1. **`int x = 0;` 你以为在 `.data`，其实在 `.bss`**：后果不是 bug，而是你对 RAM 的账算错：加 `= 0` 不会让它在 Flash 里多占四字节，改初值也不会改变镜像大小。要看真相，`nm` 一个字母就够了（出现频率第一，因为几乎人人踩过）。
2. **以为 `.bss` 不占空间**：它不占 Flash 但**照占 RAM**。`uint8_t buf[100*1024];` 一句就爆 128K 的 RAM，而且链接脚本没写 ASSERT 时它可能真的链接通过，上电踩内存。自查：`size -A` 里 `.bss` 那一行。
3. **大初始化数组放错地方**：`uint8_t lut[1024] = {...}` 既吃 Flash（备份）又吃 RAM（本体）；加 `const` 立省 1024 字节 RAM。第三节实测：256 字节的表，`.data` 从 256 变 0。
4. **中断开启太早，`.data` 还没搬完**：如果你在 `Reset_Handler` 的拷贝循环之前就让能了某个中断（或把初始化搬进了启动文件的早期），中断里读全局变量读到的是 Flash 备份还没到位的垃圾值。现象是"偶发、换个编译器版本就变"。纪律：**外设初始化必须在 `main` 之后，别塞进启动文件的搬运前面**（见 B4）。
5. **把 `nm` 的字母当段名读**：`T g_ro` 不是"函数"，是"在 `.text` 输出段里"，本工程 `.rodata` 并进了 `.text`（`ld:48-49`）。看到字母与直觉不符，先去读链接脚本，别先改代码。
6. **拿 `volatile` 当段归属开关**：`volatile int x = 1;` 照样在 `.data`。想强制某个对象去特定区域用的是 `__attribute__((section(".xxx")))` / `__attribute__((aligned(n)))`，跟 `volatile` 没关系。
7. **忘了 `--gc-sections` 会删掉"没人用的变量"**：`-fdata-sections` + `-Wl,--gc-sections` 下，没有引用者的全局变量直接不进固件，你会以为"我明明定义了，`nm` 里怎么找不到"。本工程的对策是写一个真的引用者 `sink()`（`mem_probe.c:15`，每个变量取一次地址），需要保留但不想引用时用 `__attribute__((used))`。
8. **拿调试信息当固件体积**：`size` 的 Total 26775 里绝大部分是 `.debug_*`。汇报体积要么报 `.bin` 大小，要么报 `size -A` 里那五行加总，**并且别让这些数字进站内正文**，站点统计一律由 `npm run docs:gen` 生成。

短自测

1. `static int counter;` 和 `static int counter = 0;` 有区别吗？`nm` 里分别是什么字母？

▼ 答案
在这份 GCC 下**没有区别**：两者都进 `.bss`，字母都是 `b`（`static` → 局部符号）。`int counter = 5;` 才进 `.data`（字母 `d`）。判据是"初值是否全零"，不是"有没有写等号"。实测对照就在本章第二节：`B g_zero` 与 `D g_init`。

2. 把 `stm32f407xx.ld` 里 `.data` 的 `AT> FLASH` 删掉（只留 `>RAM`），程序还能跑吗？

▼ 答案
跑不了（或者说：初值全错）。删掉 `AT>` 后 LMA 与 VMA 相同，链接器认为这段"本来就住在 RAM"，于是 `.data` 的初值备份不再被安排进 Flash 镜像；`Reset_Handler` 仍然去 `0x2000xxxx` 拷 `0x2000xxxx`（甚至 `_sidata` 会变成 RAM 地址），搬回来的全是没定义的内容。现象是"全局变量的初值全都是随机值"。**`.data` 的双重户籍靠 `AT>` 建立**，逐行推导见 [B3](../build/03-linker-script.md)。

3. `size -A` 显示 `.data 12` / `.bss 32`，为什么 `.data` 里 `g_msg` 只有 3 字节却写成 4 的对齐？

▼ 答案
`ld:94,98` 两个 `. = ALIGN(4);` 把段的头尾对齐到 4 字节；实测 `objdump -s -j .data` 的 `68690000` 里最后那个 `00` 是填充。收益：`.data` 里的每个 4 字节对象都落在 4 的边界上，启动拷贝循环才敢固定 `adds r3, r3, #4` 一个字一个字搬：**搬运循环的简单，是链接脚本的对齐换来的**。

4. 你想让一张 4KB 的波形表既不占 RAM、又要能被 DMA 读走。放到 CCM（`0x10000000`）行不行？

▼ 答案
不行。CCM 只有 CPU 直连，DMA 够不着，硬塞进去的后果是"CPU 看着数据对，DMA 搬出来全是 0/垃圾"（[S8](../stm32/08-dma.md) 有专门一小节讲这个坑）。正确做法：留在 Flash（`const`）+ DMA 从 Flash 读，或放主 SRAM `0x20000000`。只有"纯 CPU 访问、追求零等待"的大缓冲才值得搬去 CCM。

对照表：本章概念 → 仓库落点

概念	仓库落点
11 个对象的段归属现场	`code/c/01-memory-model/mem_probe.c`（附录全文内嵌）
const 对照实验	`code/c/01-memory-model/lut_probe.c`（用 `-DLUT_CONST` 切）
全部取证命令	`code/c/01-memory-model/probe.sh`（`sh probe.sh [mem|lut|boot]`）
本机实测数值与记录	`code/c/01-memory-model/README.md`
`>RAM AT> FLASH` / `_sidata`	`code/stm32/00-blink/stm32f407xx.ld:91-100`，逐行讲解 B3
`.rodata` 并进 `.text`（`T g_ro` 的成因）	同上 `:48-49`
`_Min_Heap_Size`/`_Min_Stack_Size`/`_estack`	同上 `:16,19-20,117-125`
拷贝与清零循环	`code/stm32/00-blink/startup_stm32f407xx.s:48-72`，走读 B4 第三节
`volatile` 与段归属正交	C3 volatile
DMA 够不着 CCM	S8 DMA
map 文件与体积审计	B5 map 与体积

你做到了

- 任何一条变量声明，能当场说出"住哪个段、占哪种介质、几个字节"，并且**能用 `nm` / `readelf` 自证**；
- 能解释 `_sidata` / `_sdata` / `_edata` / `_sbss` / `_ebss` 五个符号：谁发的（链接脚本）、谁消费的（`Reset_Handler`）、怎么对上实测地址；
- 会用 `const` 做预算：一个词省 256 字节 RAM，也能说清这笔交易在什么时候不划算。

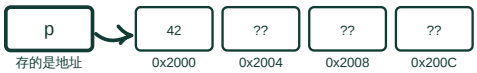
回到开场那个「其实没有的 5」：它出厂就躺在 Flash 里，上电后由启动文件里的拷贝循环搬进 RAM。谁搬的、从哪搬、搬几个字节，这三问你现在都有凭证可查，工具就是本章第四节教你的那几件。

✅ 下一站：C2 指针：`*(volatile uint32_t *)0x40021418` 凭什么能点灯？地址、类型、解引用各管一件事。

通往单片机之路**第 03 章**

C2 指针：点灯公式的全部秘密

难度 ★★★☆☆ · 约 40 分钟



本章罗盘

问题

点灯咒语里地址、类型、volatile 各管什么？为什么把 uint32_t* 换成 uint16_t*，灯就『亮得下不来』？

交付

一份 read/write 六函数的宽度对照汇编（ldrb/ldrh/ldr、strb/strh/str）加一张 const 三组合『锁谁/放行谁』表，索引与宏两种写法编出逐字节相同机器码的对照记录。

边界

『合法的 C 指针值』不等于『可访问的地址』，最终裁判是存储映射；volatile 的完整合同在 C3、野指针 HardFault 定位在 S16、结构体封装寄存器在 C4，均不在本章展开。

依据锚点 `c1 · a3 · mcu-code · mcu-anim`

🎯 先破一桩点灯怪案：地址没写错，数值也没写错，只把指针类型换窄，灯就可能“亮得下不来”。编译器没有报错，芯片也没有闹脾气，真正变掉的是哪一条机器指令？沿着 `*(volatile uint32_t *)0x40021418 = 1UL << 22;` 往下查，你会发现指针从来不是玄学，它只管三件事：**地址 + 宽度 + 纪律**。

本章精髓

1. 指针的值是地址，指针的类型决定**每次读写的字节宽度**（uint8/16/32 对应 LDRB/LDRH/LDR）：寄存器必须 32 位访问，类型错了硬件不理你。
2. 指针运算的步长由类型定：`p+1` 对 uint32_t* 是 +4 字节，这正是"寄存器数组"能用结构体/数组索引访问的原因（C4 展开）。
3. 数组名与指针的"纠缠"在嵌入式里有一个实用出口：`((uint32_t*)0x40021400)[6]` 就是 BSRR。
4. `const` 在 `*` 左边锁目标、右边锁指针，全部在**编译期**拦截，不要一分钱运行时开销，驱动里满街都是 `volatile uint32_t * const`。

▼ 🌐 **English Abstract**

Pointer = address + width + discipline. The pointer value is the address; the pointer type determines **bytes per access** (uint8/16/32 → LDRB/LDRH/LDR). Hardware registers require 32-bit access—wrong type, hardware ignores you. **Pointer arithmetic**: `p+1` steps by the pointed-to type's size (uint32_t* → +4 bytes). **Array-pointer equivalence**: `((uint32_t*)0x40021400)[6]` is BSRR. **const placement**: left of `*` locks the target, right locks the pointer—all compile-time checks, zero runtime cost.

- **能记住**：地址定位置、类型定宽度、volatile 定纪律（三板斧）

先修

- C1 内存模型。

先跑起来（10 分钟 quick win）

本章全部事实由一个取证工程跑出（宿主 gcc 16.1.0 + xPack arm-none-eabi-gcc 15.2.1，均本机实测）：

先别执行。拿张纸押三个答案：三种读取会不会生成同一条指令？三种指针的 `p+1` 会不会都只加一个字节？索引写法和偏移写法会不会编成不同机器码？押完再开庭，避免看过答案后误以为自己早就知道。

```
cd code/c/02-pointer
sh probe.sh        # 交叉汇编 + 宿主真跑 + const 反例，一次跑完
```

你将看到：`ldrb/ldrh/ldr` 三条指令并排、`p+1` 的 +1/+2/+4 字节增量、`[6]` 与 `+0x18` 编出**逐字节相同**的机器码、三条 const 越界"如期报错"。`probe.sh` 不在 PATH 的 xPack 会自动垫上，ARM 工具链位置不同就用 `ARM_PREFIX=... sh probe.sh`。

现在给刚才的下注判分：读取指令不同，步长不同，最后两种写法的机器码反而相同。三个结果看似互相打架，其实都由同一条规则解释：类型决定一次访问多宽，也决定一次“前进一步”跨多远；只要最终地址与访问宽度相同，写法的外形就不重要。

有板子的再加一道：对 `code/stm32/00-blink` 构建产物跑 `arm-none-eabi-objdump -d`，找到 main 里写 BSRR 的那条 `str`，对照 `code/stm32/00-blink/main.c:60`，亲眼看 C 咒语变成机器指令。

动画：指针三件事

取地址是抄门牌、指针是存门牌的格子、解引用是顺箭头登门。`p+1` 为什么跳 4 格、野指针登门为什么有三种下场，动画逐阶段演一遍。

指针三件事：取地址 · 存门牌 · 解引用登门

小节结构

小节	内容	四件套
一、咒语逐词翻译	地址字面量→指针类型→解引用；LDR/STR 宽度对照	代码分析
二、步长的真相	`p+1` 的字节数实验；寄存器块的"索引访问"写法	代码分析
三、野指针三宗罪	HardFault 定位（联动 S16）；静默改写案例	代码分析
四、const 与指针三组合	`const int*` / `int* const` / `const int* const` 在驱动里的真实用途	配置

一、咒语逐词翻译

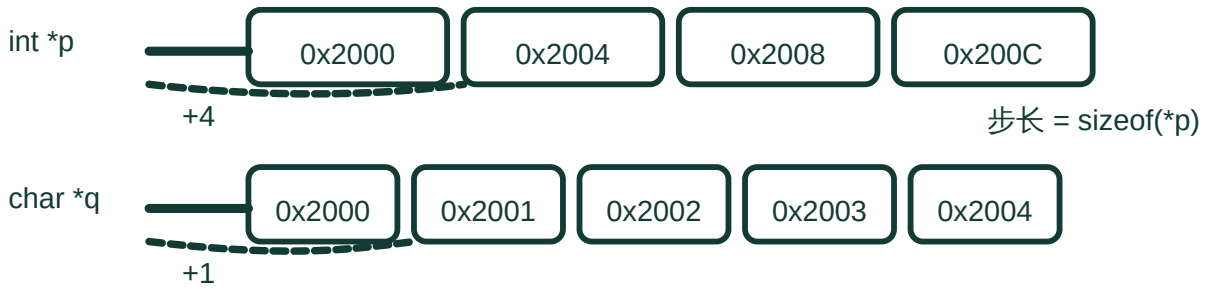
把 `*(volatile uint32_t *)0x40021418 = 1UL << 22;` 从右往左拆：

token	人话
`0x40021418`	门牌号：GPIOF 基址 `0x40021400` + BSRR 偏移 `0x18`（出处 `code/stm32/00-blink/main.c:29,34`，STM32F407）
`(volatile uint32_t *)`	给裸地址装上类型：**32 位宽**的钥匙 + volatile 的叮嘱（C3 展开）
`*`	解引用：这一次真的要登门访存
`= 1UL << 22`	一次 32 位写：BSRR 高 16 位的 BR6，PF6 拉低，红灯亮（共阳，见 00-blink 头注释）

"类型定宽度"不是修辞，是指令层面的分叉。取证（`probe.c` 的 read/write 六函数，`arm-none-eabi-gcc -std=c11 -Wall -Wextra -O2 -mcpu=cortex-m4 -mthumb -S`，版本 15.2.1）：

```
read8:                          read16:                         read32:
        ldrb    r0, [r0]                ldrh    r0, [r0]                ldr     r0, [r0]
        bx      lr                      bx      lr                      bx      lr
write8:                         write16:                        write32:
        strb    r1, [r0]                strh    r1, [r0]                str     r1, [r0]
        bx      lr                      bx      lr                      bx      lr
```

后缀即宽度：`B` =8 位字节、`H` =16 位半字、无后缀=32 位字。**同一个 `*p`，类型换了，生成的指令就换。** F407 的 GPIO 寄存器全是 32 位，所以 00-blink 的宏一律 `uint32_t`；你若用 `uint16_t*` 去写 BSRR，编出来是 `strh`：只写低 16 位，高 16 位（复位区）根本够不着（指令宽度编译期已核实，板上现象待回填，见实物实验）。



指针步长：int *p 加一前进 4 字节，char *q 加一前进 1 字节

二、步长的真相

C 标准把 `p+1` 定义为"前进一个**指向对象**"，不是前进一个字节。编译器干脆把步长折进寻址偏移，一条指令都不多花（同一取证汇编）：

```
next8:                          next16:                         next32:
        ldrb    r0, [r0, #1]            ldrh    r0, [r0, #2]            ldr     r0, [r0, #4]
        bx      lr                      bx      lr                      bx      lr
```

`p[1]` 的地址增量是 `#1/#2/#4`，就是 `sizeof` 三兄弟。宿主侧真跑（`gcc -std=c11 -Wall -Wextra -O2`，MinGW gcc 16.1.0，断言全过）：

```
sizeof: uint8_t=1 uint16_t=2 uint32_t=4  指针自身=8 字节
p+1 字节增量: uint8_t* +1  uint16_t* +2  uint32_t* +4
```

（指针自身 8 字节是 x86-64 宿主的口径，与"指向宽度"无关，别把这两件事搅在一起。）

这一步直接通往寄存器块的索引写法。 既然 `p[i]` 恒等于 `*(p+i)`，而 `i` 的单位是类型宽度，那么：

```
((volatile uint32_t *)0x40021400)[6]   /* 地址 = 0x40021400 + 6×4 = 0x40021418 = BSRR */
```

`6 × sizeof(uint32_t) == 0x18` 这条等式用 `_Static_assert` 锁死在 `probe.c` 里（随每次编译校验，编过即证）。宿主真跑同样确认（只取地址、不解引用）：

```
寄存器索引: &((uint32_t*)0x40021400)[6] = 0x40021418 (= 基址 + 6*4 = +0x18)
```

更狠的证据在汇编：索引写法 `led_on_indexed` 与 00-blink 宏写法 `led_on_macro` 编出**逐字节相同**的机器码：

```
led_on_indexed:                 @ led_on_macro 与此完全相同
        ldr     r3, .L12
        mov     r2, #4194304            @ 1<<22 = BR6
        str     r2, [r3, #1048]
        bx      lr
.L12:
        .word   1073876992              @ 0x40021000；+1048(0x418) = 0x40021418
```

编译器把最终地址 `0x40021418` 拆成"字面值 `0x40021000` + 12 位偏移 `1048`"，殊途同归。**两种 C 写法，零差别**：选哪个是风格问题，不是正确性问题。这也是 C4 用结构体罩住整组寄存器的地基：结构体字段偏移与数组下标，走的是同一套"下标 × 宽度"算术。

三、野指针三宗罪

裸机没有 MMU 页保护，指错了没人立刻拦你。宿主上野指针通常吃到 segfault，那是操作系统的馈赠；板子上只有三种下场：

1. **HardFault**：访问了不存在的地址（未使能时钟的外设区、保留区、越出存储映射），总线错误升级成 HardFault。这是最好的一种：当场死给你看，定位方法见 S16。
2. **静默改写**：地址合法但指错了对象，写穿到隔壁变量、别的外设寄存器。没有任何异常，系统在几天后、另一个看似无关的功能里爆雷。排查成本以天计。
3. **碰巧能用**：最可怕的一种。野到的内存当前恰好无害，功能"正常"，换优化档、换芯片、加一行代码才复发，与 C3 里"`-O0` 一切正常"同源：**运气不是设计**。

注意 `probe.c` 的一个细节：宿主运行只敢对 `0x40021400` **取地址**做算术，从不敢解引用：在 Windows 宿主上那个地址摸不得。这本身就是活教材："合法的 C 指针值"与"可访问的地址"是两回事，指针合法性的最终裁判是**存储映射**，不是编译器。

防御纪律就三条：指针必先初始化再使用；地址常量集中在一处定义（像 00-blink 那样 `GPIOF_BASE + 偏移`，别满篇魔法数）；需要按地址打印/存储时用 `uintptr_t`，需要表偏移差用 `size_t` / `ptrdiff_t`。

四、const 与指针三组合

`const` 的位置决定锁谁，口诀"从右往左念，const 锁左边最近的那个"：

写法	锁住的	放行的
`const int *p`（同 `int const *p`）	`*p = x` 透过指针改目标	`p = &b` 换指向
`int * const p`	`p = &b` 换指向	`*p = x` 改目标
`const int * const p`	全锁	只能读

三条越界全部**编译期**拦截（`probe.sh const` 段，宿主 gcc 16.1.0 实测，如期报错）：

```
probe.c:50:52: error: assignment of read-only location '*p'        @ const int *p : *p = 1
probe.c:52:58: error: assignment of read-only variable 'p'         @ int * const p : p = &b
probe.c:54:59: error: assignment of read-only location '*(const int *)p'  @ 全锁
```

合法半边真跑通过：`*pc=2 *cp=42 *cpc=2`，断言全过。const 不要钱——跑都不用跑，错在编译期就死了。

为什么驱动里满街是 `volatile uint32_t * const`：指针本身锁死（这个句柄永远指向 GPIOF，不许被改去指别人），目标保持 volatile（硬件会绕过 CPU 改它，C3 的合同照旧）。两个限定词各管一件事，叠在一起恰好是"寄存器句柄"的完整语义。CMSIS 的 `GPIO_TypeDef` 结构体指针用法（C4 展开）就是这一组合的批量化。

附录：工程完整源码

取证工程（宿主 gcc 与 Cortex-M4 交叉各一套，一分钟可复现）：

<<< ../../code/c/02-pointer/probe.c

构建与取证命令（`sh probe.sh` 从工程目录直接跑）：

<<< ../../code/c/02-pointer/probe.sh

板级对照工程（点灯咒语的出处）：

<<< ../../code/stm32/00-blink/main.c

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **地址定位置，类型定宽度，volatile 定纪律**（width/where/don't-optimize）。指针三板斧，念念有回响。

延伸：指针与内存布局的关系在 C1 铺垫；volatile 防止编译器优化指针读取在 C3 展开；指针访问外设寄存器是 S3 GPIO 的核心操作。

实物实验

- **无板上（本机已全部实测通过）**：`cd code/c/02-pointer && sh probe.sh`，把 `read8/16/32` 三条指令、`p+1` 的 +1/+2/+4 输出、两条 `led_on_*` 的相同汇编抄进实测记录，注明工具链版本与完整命令行（本机：arm-none-eabi-gcc 15.2.1 `-mcpu=cortex-m4 -mthumb -O2 -S`；宿主 gcc 16.1.0 `-O2`）。
- **板上（编译期结论已核实，肉眼现象待回填）**：在 `code/stm32/00-blink/main.c:34` 把 `GPIOF_BSRR` 的类型从 `uint32_t*` 改成 `uint16_t*`，构建烧录：`str` 变 `strh`，只写低 16 位：置位（BSx 低 16 位）仍有效，复位（BRx 高 16 位）够不着，灯将"亮得下不来"（预期现象，待回填）。亲手证明"类型=访存宽度"。

> ⚠️ **注意 · 本轮取证状态**
>
> 本章汇编与宿主运行输出均为本机实测（版本与命令行如上）；**板上肉眼现象本轮未跑**。谁先烧录验证，欢迎把现象回填到这一节。

常见坑

- *用 int 访问寄存器*：多数平台 int=32 位碰巧对，但风格即隐患；寄存器访问一律定宽类型 `uint32_t`。
- **忘记括号**：`*GPIOF_BSRR = x` 与宏定义展开优先级。所以我们的宏写成 `(*(volatile uint32_t *)(...))`，括号一个不能少。
- **把指针当整数打印**：嵌入式 printf 打印指针用 `%p` 且强转 `(void*)`，否则行为未定义。
- **以为 `const` 有运行时开销**：三组合全部编译期拦截，生成的机器码与不加 const 逐字节相同——不加白不加。
- **`p+1` 当字节加**：`uint32_t *p = (uint32_t *)0x40021400; p + 6` 是 `0x40021418`，`(char *)p + 6` 才是 `0x40021406`，单位是指向对象，不是字节（第二节汇编里 `#1/#2/#4` 的铁证）。

短自测

1. 把 OO-blink 的 BSRR 宏改成 `*(volatile uint16_t *)0x40021418 = 1U << 22;`，会发生什么？为什么？

> **▼ 答案**
> 编译产出 `strh`（16 位写），且 `1U << 22` 截成 16 位后是 0，这行实际往 BSRR 低 16 位写 0，"写 0 无影响"，什么都不发生。就算改成 `uint16_t*` 写 `1U << 6`，也只能碰置位区，复位区（高 16 位）永远够不着：灯"亮得下不来"。指令宽度由指针类型决定（第一节 `strb/strh/str` 对照），寄存器必须按手册宽度 32 位访问。（指令侧已核实，板上现象待回填。）

2. `uint32_t *p = (uint32_t *)0x40021400;` 时，`p + 6` 与 `(char *)p + 6` 各等于多少？

> **▼ 答案**
> `p + 6` = `0x40021400 + 6×4` = `0x40021418`（正是 BSRR）；`(char *)p + 6` = `0x40021406`。指针加减的单位是**指向对象的大小**，不是字节，`probe.sh run` 的实测输出 +1/+2/+4 即证据。

3. `const int *p` 与 `int * const p` 各拦什么？驱动句柄为什么常写成 `volatile uint32_t * const`？

> **▼ 答案**
> `const int *p` 拦"透过 p 改目标"（`error: assignment of read-only location`），放行换指向；`int * const p` 拦"换指向"（`error: assignment of read-only variable`），放行改目标。`volatile uint32_t * const` 是两者合体的驱动语义：指针锁死在这个外设地址上不许改指（const 在 `*` 右），目标可被硬件异步修改、每次访问都必须真访存（volatile 管目标）。全在编译期生效，零运行时开销。

4. 凭什么说 `((uint32_t *)0x40021400)[6]` 与 `*(uint32_t *)(0x40021400 + 0x18)` 等价？证据链有几环？

> **▼ 答案**
> 三环，全部可复现：① 语义环：`p[i]` 恒等于 `*(p+i)`，`i` 以 `sizeof(*p)` 为单位，`6×4=24=0x18`；② 编译期环：`_Static_assert(6U*sizeof(uint32_t)==0x18)` 随每次编译校验；③ 指令环：`-O2` 下 `led_on_indexed` 与 `led_on_macro` 编出逐字节相同的汇编（`str r2, [r3, #1048]`，字面池 `0x40021000`，落点 `0x40021418`）。

对照表：本章概念 → 仓库落点

概念	仓库落点
类型定访存宽度（ldrb/ldrh/ldr、strb/strh/str）	`code/c/02-pointer/probe.c` read/write 六函数，正文第一节汇编引证
`p+1` 步长 = 类型宽度	同上 next8/16/32 汇编（`#1/#2/#4`）+ `sh probe.sh run` 输出
BSRR = 基址 `[6]`（索引=偏移）	同上 `_Static_assert` + `led_on_indexed` / `led_on_macro` 同码对照
点灯咒语 `0x40021418` / `0x40021400` / `0x18`	`code/stm32/00-blink/main.c:29,34,60,62`（全部地址的唯一出处）
const 三组合编译期拦截	`sh probe.sh const` 三条如期报错
`volatile uint32_t * const` 驱动惯用法	`probe.c` main() 运行输出；volatile 合同见 C3
野指针 HardFault 定位	S16
取证脚本入口（host-probe）	`code/c/02-pointer/probe.sh`

你做到了

- 点灯咒语再无秘密，还能随手写出同类；
- 会按"地址+宽度+纪律"三问审查任何寄存器访问代码；
- 会用 `_Static_assert` + 同码对照证明"两种写法等价"，而不是靠嘴。

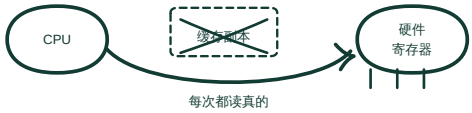
回到开场那盏“亮得下不来”的灯：地址与数值都可以正确，类型仍能让访问宽度悄悄换掉。以后再遇到寄存器写入怪案，先查生成的是哪种 load/store，再查门牌号。

✅ 下一站：C3 volatile：编译器会把你的寄存器读写"优化"成什么样，volatile 又能管到哪一截。

通往单片机之路第 04 章

C3 volatile：跟编译器约法三章

难度 ★★★☆☆ · 约 70 分钟



本章罗盘

问题

volatile 到底向编译器承诺了什么，为什么加了 volatile 的共享标志仍然可能出错？

交付

一份 -O0/-O2 反汇编对照（volatile 有无各一版）和一张『volatile 保证/不保证』清单。

边界

volatile 不提供原子性与顺序性；并发正确性要靠本章末尾的访问函数/关中断方案，而非更多 volatile。

依据锚点 `c1 · d7 · mcu-code · mcu-anim`

> 🎯 同一段轮询，`-O0` 跑得好好的，`-O2` 一开就死等。编译器没坏，它只是不知道那个地址后面藏着一台会自己变状态的机器。

本章精髓

1. **为什么优化会"吃掉"访存**：编译器按 as-if 规则工作，只要可观察行为不变，缓存、删除、合并你的读写都合法。它眼里的 `status` 是一块内存；而 `USART1->SR` 后面是硬件，你不读它也在变。`volatile` 就是收回这项授权的合同条款。
2. **为什么 volatile 只解决一半问题**：它保证"每次使用都真实访存"，不保证"这一串访存之间没人插队"。把 `volatile int flag` 当锁用，是把类型限定符当成了同步原语。
3. **为什么值得单独一章**：本站所有"库源码逐行解析"读的都是带 volatile 的封装（CMSIS 的 `__IO`、SPL 的 `__IO` 局部变量、IDF 的寄存器访问）。懂它的合同与边界，才知道那些封装在替谁兜底。

▼ 🌐 **English Abstract**

Why volatile matters: The compiler's as-if rule allows it to cache, delete, or merge memory accesses as long as observable behavior stays the same. But hardware registers change state independently—`volatile` revokes the compiler's authorization to optimize those accesses. **What volatile guarantees**: Every use triggers an actual memory access. **What it doesn't guarantee**: Atomicity or ordering—`volatile int flag` is not a synchronization primitive. **Three must-have scenarios**: (1) Hardware registers (MMIO), (2) Interrupt-shared variables, (3) Signal handlers. **Two anti-patterns**: (1) Using volatile as a lock, (2) Adding volatile to "fix" race conditions (use atomics/critical sections instead).

- **能记住**：volatile 的"管三件事 / 不管两件事"清单（本章记忆锚点）
- **能理解**：为什么 `-O2` 把轮询变成原地空转、把延时循环删成一条返回——四段本机取证汇编，x86-64 与 Cortex-M4 各一份
- **能用**：拿到驱动代码能判断哪里的 volatile 必需、哪里的是性能税；知道中断共享变量该换什么机制

先修

- C2 指针：`(volatile uint32_t *)0x40021418` 里限定符该贴谁；
- B1 四步构建：`-O` 在编译层起作用，本章的翻车全发生在编译这一步；
- S3 GPIO 的 ODR 读-改-写隐患：同一个问题在外设侧的另一种表现。

先跑起来（10 分钟 quick win，不用开发板）

```
cd code/c/03-volatile
gcc -std=c11 -Wall -Wextra -O2 -S -o opt_probe.O2.s opt_probe.c
arm-none-eabi-gcc -std=c11 -Wall -Wextra -mcpu=cortex-m4 -mthumb -O2 -S -o opt_probe.m4.O2.s opt_probe.c
```

跑之前先押：那个空转一千圈的延时函数，在优化后的产物里会剩几条指令？总该留下点什么吧。押个数字再搜，命中率通常低得让人印象深刻。

没有 ARM 工具链只跑第一条也够。两条命令产物里搜 `delay_plain`：x86 那份整个函数只剩一条 `ret`，Cortex-M4 那份只剩一条 `bx lr`。你写的 `while (n--) {}` 一千圈延时，在优化器眼里根本没存在过。这就是本章的全部主题，后面四十分钟是解释它为什么合法、以及怎么避免。

动画：同一段轮询的两条命运

volatile 的访存合同：读一次 vs 每次读

视频讲解

本章有配套视频（约 12 分钟），用 4 个犯罪现场、6 个变体实验把 volatile 讲透。视频将在 B 站发布后在此更新链接。

视频的分幕、旁白与动画绑定见 `video/episodes/c3-volatile/episode.json`（成片的唯一事实源，渲染脚本只吃这一个文件），写作阶段的分镜见 `storyboard.md`。两个都在仓库里，未发布到站点。

四阶段：优化器的世界观 → 不加 volatile 的下场 → 加 volatile 的下场 → 边界在哪里。

小节结构

小节	内容	四件套
一、as-if：优化器的世界观	C 标准给编译器的授权书；volatile 收回哪三项授权	配置（语义规则）
二、四个现场取证	轮询 / 延时 / 写合并 / 共享标志，`-O0` 与 `-O2` 汇编逐条对账（x86-64 与 Cortex-M4 各一份）	代码分析
三、管什么与不管什么	"每次必登门"三条保证 vs 原子性与顺序性两个空白	配置 + 代码分析
四、库源码解析	CMSIS `GPIO_TypeDef` 的 `__IO`、SPL 的 `__IO` 局部变量、IDF 的对应做法	库解析
五、代码分析：`00-blink`	逐行看三处 volatile：寄存器宏、`delay` 参数、`__asm__ volatile ("nop")`	代码分析
六、补齐边界	临界区、外设侧原子寄存器、屏障与 cache 各自站哪个岗	库解析

> 本章不涉及引脚设置：`volatile` 是语言层的合同，作用域覆盖**所有** MMIO 访问，与具体引脚无关。

一、as-if：优化器的世界观

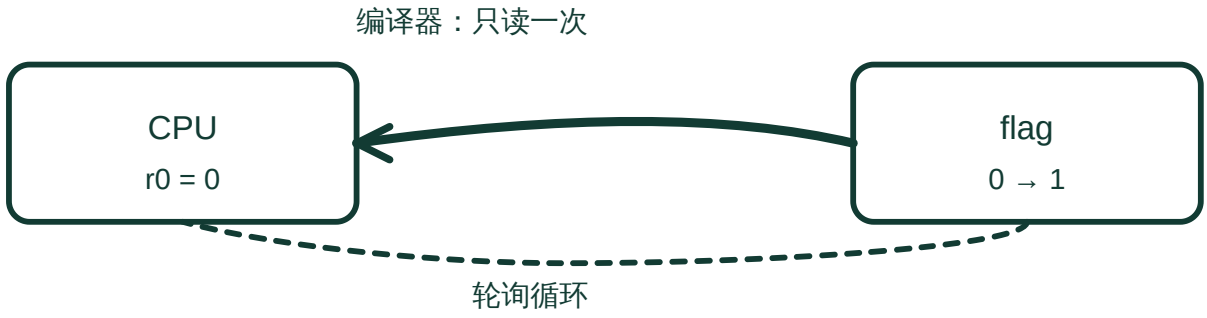
C 标准（ISO/IEC 9899:2018，类型限定符在 §6.7.3、程序执行语义在 §5.1.2.3，具体措辞以你手上的标准版本为准）给编译器的授权是：**只要可观察行为不变，中间的机器指令随你怎么排**。

于是优化器做三件天经地义的事：

优化动作	对普通内存	对硬件寄存器
缓存：多次读合成一次，值留在寄存器	完全正确	**错**——这几次读之间硬件自己变了
删除：写了没人再读的赋值（死 store）	可以删	**错**——那次写本身就是目的
合并：连续两次 `|=` 合成一次	等价	**错**——硬件要两个动作，你只发了一个

`volatile` 的语义就是把这三项授权收回：**每次使用都必须真实访存，不许缓存、不许删、不许凭空造**。注意它只管"访存"这一件事，不改值的含义，它不是 `atomic`，也不是锁。

另有一条常被忽略：**没有副作用的空循环，编译器可以假定它不会无限跑**（C11 起此类循环按未定义处理）。这解释了下一节的 `delay_plain`，它连"缓存"都没用上，直接整个消失。



编译器把 flag 缓存进寄存器，ISR 在内存里写 1；轮询循环因此看不到更新

二、四个现场取证

取证环境（两套工具链，同日实测，均 `-std=c11 -Wall -Wextra` 零告警）：宿主 MinGW-Builds **gcc 16.1.0**（x86_64-posix-seh）与 **xPack arm-none-eabi-gcc 15.2.1**（`-mcpu=cortex-m4 -mthumb`，就是你板子上那颗 M4 的目标后端）。下面每个现场都摆两份：**x86 那份方便你本机 30 秒复现，Cortex-M4 那份才是你板子上真正会跑出来的东西**。两边的优化决策一模一样：同一个优化器、同一条 as-if 规则，只是指令形式不同。

现场 1：轮询状态位——读一次 vs 每次读

源码只有一处差别：`unsigned int` 与 `volatile unsigned int`。

`-O0` 下的 `poll_plain`，每圈都真访存（这才是"-O0 能跑"的真相）：

```
poll_plain:
.L2:
    movl    g_status(%rip), %eax     ; 每圈真的去内存读
    andl    $1, %eax
    testl   %eax, %eax
    je      .L2                      ; 没置位就再来一圈
    movl    $42, %eax
```

同一函数在 `-O2` 下（x86-64）：

```
poll_plain:
    testb   $1, g_status(%rip)       ; 整段代码只读这一次
    je      .L3
    movl    $42, %eax
    ret
.L3:
    jmp     .L3                      ; 原地死转，再也不看那块内存
```

判断依据被缓存了：bit0 后来被硬件置成 1，代码不会知道。

换到 Cortex-M4（`-O2`），同一件事，指令形式不同：

```
poll_plain:
    ldr     r3, .L5
    ldr     r3, [r3]                ; 整个函数只读这一次
    lsls    r3, r3, #31             ; 把 bit0 顶到符号位
    bpl     .L3                     ; 没置位 → 跳 .L3
    movs    r0, #42
    bx      lr
.L3:
    b       .L3                     ; 死转：连那块内存都不再碰
```

注意 M4 这份的残酷之处：`.L3: b .L3` 是**一条指令的紧死循环**，没有访存、没有中断以外的出口。板子看起来"活着"（SWD 还能连），但这段代码永远不会往下走。这就是"轮询少了 volatile"在真实芯片上的长相。

换成 `poll_volatile`，`-O2` 下循环回到每圈访存：

```
poll_volatile:                      ; x86-64
.L6:
    testb   $1, v_status(%rip)      ; 每圈一次真实读
    je      .L6
    movl    $42, %eax
    ret
```

```
poll_volatile:                      ; Cortex-M4
.L8:
    ldr     r3, [r2, #4]            ; 每圈一次真实 ldr
    lsls    r3, r3, #31
    bpl     .L8
    movs    r0, #42
    bx      lr
```

现场 2：软件延时——整个循环被删

`delay_plain(unsigned n){ while (n--) {} }` 在 `-O2` 下的全部产物：

```
delay_plain:
    ret                             ; x86-64
```

```
delay_plain:
    bx      lr                      ; Cortex-M4：整个函数一条返回
```

一条指令，你的"500ms"变成 0ms。参数改成 volatile（`delay_volatile`）后循环保住了：

```
delay_volatile:
    movl    %ecx, 8(%rsp)           ; x86-64：计数落在内存（栈槽），不放寄存器
.L11:
    movl    8(%rsp), %eax
    leal    -1(%rax), %edx
    testl   %eax, %eax
    movl    %edx, 8(%rsp)           ; 每圈写回
    jne     .L11
    ret
```

```
delay_volatile:
    sub     sp, sp, #8
    str     r0, [sp, #4]            ; Cortex-M4：计数落栈槽
.L15:
    ldr     r3, [sp, #4]
    subs    r2, r3, #1
    str     r2, [sp, #4]            ; 每圈写回
    cmp     r3, #0
    bne     .L15
    add     sp, sp, #8
    bx      lr
```

延时的另一条出路是**给它真实副作用**：`00-blink` 用的是 `__asm__ volatile ("nop")`，见第五节。

现场 3：配置序列被合并成一次写

三个连续 `|=` 想表达"依次置位三个使能位"。普通结构体（等于寄存器访问漏了 volatile）：

```
cfg_plain:
    orl     $7, (%rcx)              ; x86-64：三次读-改-写 → 一次，写进去 7
    ret
```

```
cfg_plain:
    ldr     r3, [r0]                ; Cortex-M4：三次并成一次
    orr     r3, r3, #7              ;   M4 没有"内存读-改-写"单指令，
    str     r3, [r0]                ;   所以整段只访存两次
    bx      lr
```

字段带 `volatile`（CMSIS `__IO` 的等价写法）：

```
cfg_volatile:
    orl     $1, (%rcx)              ; x86-64：三条独立读-改-写
    orl     $2, (%rcx)
    orl     $4, (%rcx)
    ret
```

```
cfg_volatile:
    ldr     r3, [r0]                ; Cortex-M4：三组 ldr/orr/str
    orr     r3, r3, #1
    str     r3, [r0]
    ldr     r3, [r0]
    orr     r3, r3, #2
    str     r3, [r0]
    ldr     r3, [r0]
    orr     r3, r3, #4
    str     r3, [r0]
    bx      lr
```

对普通内存两者等价；对外设**不等价**：有的寄存器每写一次就触发一次动作（压 FIFO、启动一次转换、打一个脉冲）。折叠写入 = 少发了两个动作。这也是"顺序"问题的另一种形态：优化器没重排，它直接**并了**。

现场 4：volatile 保证访存，不保证原子

```
volatile unsigned int shared_flag;
void isr_toggle(void) { shared_flag ^= 1u; }
```

`-O2` 下：

```
isr_toggle:
    xorl    $1, shared_flag(%rip)    ; x86-64：一条指令，仍是"读-改-写"三步
    ret
```

```
isr_toggle:
    ldr     r3, [r2, #8]            ; Cortex-M4：连"一条指令"的假象都没有
    eor     r3, r3, #1              ;   读、改、写摊成三条
    str     r3, [r2, #8]
    bx      lr
```

`volatile` 让这次访问真的发生了，**但不阻止中断在这三步中间插进来**：主循环读到旧值后被抢占、中断改了值、主循环回到"写"这步把带旧值的成果写回去。中断的动作就地蒸发。`volatile` 一个字都没帮你。M4 这份反汇编值得多看一眼：三条指令之间有两个可被打断的间隙，而 x86 那份"看起来一条指令"也绝不原子（x86 的内存读-改-写只在锁前缀下才保证原子）。这是本章最容易出事故的一点，第六节给替代方案。

三、管什么与不管什么

需求	volatile 够不够	该用什么
轮询硬件标志位直到置位	**够**	`volatile` 对象 + 循环
读一个寄存器全部位再决定	够	`volatile` 结构体字段
中断置标志、主循环读快照（32 位对齐单次访问）	够	`volatile` + 单个标志位
主循环与中断都对同一变量做读-改-写	**不够**	临界区（关中断/挂起调度），或外设侧原子入口（GPIO 的 BSRR），或位原子操作
先配 A 再配 B，顺序不能乱	不够（访存都保，但相对顺序与预取仍可能不如你预期）	内存屏障：Cortex-M 的 `DMB/DSB`，CMSIS-Core 的 `__DMB()/__DSB()/__ISB()` 封装
DMA 写的缓冲区 CPU 随后读	不够	屏障 + cache 清理/无效化（带数据 cache 的核，如 ESP32-S3；API 以 ESP-IDF 文档为准）

四、库源码解析：volatile 藏在哪一层

CMSIS 设备头把 volatile 写进结构体字段。`stm32f407xx.h:526-537`（本项目引证副本原文）：

```
typedef struct
{
  __IO uint32_t MODER;    /*!< GPIO port mode register,               Address offset: 0x00      */
  __IO uint32_t OTYPER;   /*!< GPIO port output type register,        Address offset: 0x04      */
  __IO uint32_t OSPEEDR;  /*!< GPIO port output speed register,       Address offset: 0x08      */
  __IO uint32_t PUPDR;    /*!< GPIO port pull-up/pull-down register,  Address offset: 0x0C      */
  __IO uint32_t IDR;      /*!< GPIO port input data register,         Address offset: 0x10      */
  __IO uint32_t ODR;      /*!< GPIO port output data register,        Address offset: 0x14      */
  __IO uint32_t BSRR;     /*!< GPIO port bit set/reset register,      Address offset: 0x18      */
  __IO uint32_t LCKR;     /*!< GPIO port configuration lock register, Address offset: 0x1C      */
  __IO uint32_t AFR[2];   /*!< GPIO alternate function registers,     Address offset: 0x20-0x24 */
} GPIO_TypeDef;
```

`__IO` 是 CMSIS-Core 的宏，本质是 `volatile` 的封装（同族还有只读/只写的变体；定义在 `cmsis_compiler.h` / `cmsis_gcc.h`，本项目 ref 只入了设备头，核心头以你手上的 CMSIS 版本为准）。这个设计的价值在于：**声明一次，管住整组寄存器**。应用写 `GPIOF->BSRR = ...`（`GPIOF` 即 `((GPIO_TypeDef *) GPIOF_BASE)`，同文件 :1107）根本不需要操心限定符。

SPL（StdPeriph） 则在函数内部手动加。`stm32f4xx_i2c.c:1261-1264`（引证副本原文）：

```
FlagStatus I2C_GetFlagStatus(I2C_TypeDef* I2Cx, uint32_t I2C_FLAG)
{
  FlagStatus bitstatus = RESET;
  __IO uint32_t i2creg = 0, i2cxbase = 0;
```

停下来想一层：**两个局部变量为什么要写 `__IO`？** 它们的值由本函数算出，硬件改不到；真正的保底来自 `I2C_TypeDef` 字段自带的 volatile。所以这两处是"作者求稳"的防御写法，代价是变量被钉在内存里（每次读写都访存），功能收益接近零。

这不是挑 SPL 的毛病，而是本章最该学走的读码动作：**看到一个 volatile，先问"谁会绕过编译器改这块内存"。答得出来是必需，答不出来是性能税。** 对照 S11 I2C 里 SPL 的轮询实现，你能同时看到"必需的"和"可加可不加的"两种写法。

ESP-IDF 同一路线：寄存器访问由带 volatile 的字段/宏完成，驱动层再向上封装。P6 SPI/I2C 驱动框架 里应用侧碰不到裸地址；一旦你下探到 LL 层或自己写外设访问，volatile 立刻回来。

五、代码分析：`00-blink` 里的三处 volatile

本节不重复贴板级工程，逐行看现有的 `code/stm32/00-blink/main.c`。它的 `Makefile:20` 是 `CFLAGS := $(MCU) -O0 -g3 ...`，"让第一次上板不被优化器打扰"的保险设置，也正是本章要拆掉的保险。

① 寄存器宏（`main.c:26-34`）

```
#define RCC_AHB1ENR         (*(volatile uint32_t *)(RCC_BASE + 0x30UL))
#define GPIOF_BSRR          (*(volatile uint32_t *)(GPIOF_BASE + 0x18UL))
```

`(*(volatile uint32_t *)(addr))` 拆开读：地址 → 转成"指向 volatile uint32_t 的指针" → 解引用。限定符作用在**被指向的对象**上，于是每次读写这个宏都是真访存。

这个 `volatile` 漏掉会怎样？把 `main.c:26-34` 六个宏里的 `volatile` 全部删掉（其余不动），`-O2` 交叉编译实测，`main` 变成这样（片段，Cortex-M4）：

```
main:
    ldr     r1, [pc, #104]
    ldr     r2, [pc, #108]
    ldr.w   r3, [r1, #1024]          ; 0x400 = GPIOF_MODER —— 先读了它
    ldr.w   r4, [r2, #2096]          ; 0x830 = RCC_AHB1ENR
    ldr.w   r0, [r1, #1028]          ; GPIOF_OTYPER
    orr.w   r4, r4, #32
    bic.w   r3, r3, #12288
    str.w   r4, [r2, #2096]          ; 开 GPIOF 时钟的写，排到了上面那次读之后
```

源码顺序是"先开时钟，再配引脚"，机器码顺序反过来了：**GPIOF 时钟还没使能，代码就去读 GPIOF 的寄存器**。这就是第一节表格第三行"合并/重排"的真相：优化器不是把两次写并成一次，而是把整段访问按它自己的依赖图重排（它眼里这些只是普通内存）。本工程默认 `-O0`，所以它一直是对的；这也是为什么"能跑"从来不是证据。

② 限定符贴谁：`volatile` 在指针的哪一侧（`main.c:39`）

```
static void delay(volatile uint32_t count)
```

`volatile uint32_t count`：计数本身是易变对象 → 每圈 load/store（现场 2 的保命写法）。别和 `uint32_t * volatile p`（指针自身易变）混，`volatile` 在 `*` 左边和右边说的是两件事，回 C2 指针 对照。

③ 语句级 volatile：`__asm__ volatile ("nop")`（`main.c:42`）

```
while (count--) {
    __asm__ volatile ("nop");
}
```

这里的 `volatile` 是 **asm 语句的限定符**，与对象类型限定符是两套规则：表示"这段汇编不许删、不许移动、不许合并"，同时给循环注入了副作用。所以 `00-blink` 的延时比纯 volatile 计数更稳。但周期数仍取决于主频与优化档，注释里"约 0.5s（-O0 实测校准）"就是这个意思，换时钟源要重新校准（见 S2 RCC 时钟树）。

把三处 volatile 拆开逐个实测（`arm-none-eabi-gcc 15.2.1`，`-mcpu=cortex-m4 -mthumb -O2`，`sh code/c/03-volatile/blink-variants.sh` 可全部重跑）：

变体	宏 volatile	参数 volatile	nop	-O2 下 main 的长相	灯
A（现状）	有	有	有	先写 `RCC_AHB1ENR` 开时钟，再配 MODER	闪
B	有	**无**	有	与 A 几乎一样，循环仍在（`nop / subs / bne`）	闪
C	有	**无**	**无**	延时没了：两条 `BSRR` 写（`str [r3,#0x418]`）紧挨着	不闪
D	有	有	**无**	循环保留，计数却在寄存器 r3 里（无 `ldr/str`）	闪
F	**无**	有	有	顺序倒了：读 `GPIOF_MODER` 排在开时钟那次写之前	未知
E	**无**	**无**	**无**	`main:` 只剩一条 `b.n 0` ——**所有外设写被当死 store 删光**	全黑

三点读法：

1. **B 行是对流行说法的纠正**：网上讲 volatile 几乎都说"去掉 `delay` 参数的 volatile，`-O2` 灯就不闪了"。在这份代码里不成立：`__asm__ volatile ("nop")` 已经独立挡住了删除。**要复现翻车，得连 `nop` 一起删（C 行）。**
2. **F 行比 C 行更常见也更阴**：漏的是寄存器宏里一个词，现象不是"不闪"而是"偶尔配不上/换颗芯片就不好使"：顺序倒了。
3. **D 行顺带回答了"volatile 局部变量到底在不在内存"**：`delay` 是 `static` 且被内联，计数就留在寄存器；而第二节的 `delay_volatile` 是外部函数、单独编译，参数落在 `[sp,#4]` 每圈访存。`volatile` **保的是"每次使用都可见"，不是"必须占一块内存"**；MMIO 不受这条影响：地址是硬编码的，编译器无处可藏。

六、补齐边界：volatile 不管的那两件事

- **原子性**：短到不必犹豫的做法是进临界区（根据参与者选择：任务间可用互斥，任务与 ISR 用能覆盖该 ISR 优先级的短临界区，SMP 需跨核锁或原子协议；挂起调度不会禁用中断），长一点用"只置位、只清位"的标志协议；外设侧则优先挑天生原子的入口：`BSRR` 写 1 有效、写 0 无影响，压根不需要读-改-写（S3 GPIO）。GCC 的 `__atomic_*` 内建与 C11 `<stdatomic.h>` 在多核场景（P4 中断与双核）才真正登场。
- **顺序性**：两条 volatile 访存之间插屏障，才是"配置 A 之后才能碰 B"的可靠保证。别以为"访存都保了，顺序自然对"。第五节 F 行就是反例：漏了宏上的 volatile，`RCC_AHB1ENR` 的写直接排到了 `GPIOF_MODER` 的读后面。反过来，即使 volatile 把两次访问都保成真实访存，编译器仍可能在**它们之间**重排你代码里的其他计算，而总线上的先后也不等于硬件看到的先后（带 prefetch / 写缓冲 / cache 的核尤其如此）。Cortex-M 侧是 `DMB/DSB`（CMSIS-Core 封成 `__DMB()/__DSB()/__ISB()`，定义在 CMSIS 核心头，本项目 ref 未入库，以你手上的版本为准）；带 prefetch 与数据 cache 的核还要处理"读到的不是总线上最新的那一份"：那是下一节的 cache 话题。
- **多执行流可见 ≠ 多执行流正确**：volatile 让双方都真的读写内存，剩下的正确性由上面的机制负责。这条界限画不清，就会得到"加了 volatile 还是偶发出错"的经典悬案。

附录：工程完整源码

取证工程（宿主 gcc 与 Cortex-M4 交叉各一套，30 秒可复现）：

<<< ../../code/c/03-volatile/opt_probe.c

构建与取证命令：

<<< ../../code/c/03-volatile/Makefile

六个 `00-blink` 变体的生成脚本（第五节那张表由它跑出来）：

<<< ../../code/c/03-volatile/blink-variants.sh

板级对照工程（第五节逐行过的就是它）：

<<< ../../code/stm32/00-blink/main.c

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **编译器只对你的"内存值"负责，不对你的"总线动作"负责——`volatile` 就是把总线动作写进合同的那句话。** 登门（访存）它保；排队（原子）、先后（顺序）它不管。

延伸：volatile 与寄存器操作（S3）直接相关；原子操作需中断保护（S4）或 RTOS 临界区（F5）；UB 与 MISRA 规范见 C7。

实物实验

本章没有独立实验编号，用两个现成工程做对照，结果记进你的实测笔记：

- **无板上（本机已全部实测通过）**：`cd code/c/03-volatile`，宿主侧 `gcc -std=c11 -Wall -Wextra -O2 -S -o opt_probe.O2.s opt_probe.c`，交叉侧 `make asm-arm` 或 `sh blink-variants.sh`。把 `poll_plain` / `poll_volatile` 两段与六变体表里 C、F、E 三行的 `main` 抄进实测记录，注明编译器版本与完整命令行；
- **板上（三步，编译期结论已核实，肉眼现象待回填）**：`code/stm32/00-blink` 的 `CFLAGS` 由 `-O0` 改 `-O2`：1. 什么都不删，直接烧：灯**照闪**。`-O2` 不是潘多拉魔盒，这份代码的两道防线（宏 `volatile` + `asm volatile ("nop")`）都还在；2. 把 `main.c:39` 的 `volatile` 与 `main.c:42` 的 `__asm__ volatile ("nop");` **一起**去掉再烧：红灯不再闪（延时整个消失，两条 `BSRR` 写相邻）。只去掉前者对应 B 行，灯照闪，**这是流传最广的一个错误说法，别照抄**；3. 想看到 F 行那种"更阴"的现象：把 `main.c:26-34` 六个宏的 `volatile` 去掉（`delay` 不动），灯的行为可能依旧正常（顺序倒了但硬件宽容），也可能配不出模式，**换优化档/换芯片就会变**，这正是要写进合同而不能靠运气的理由。

观测点：肉眼 + 示波器看 PF6（C 行应看到 PF6 恒定或极窄脉冲，而非 1Hz 方波）。

预期现象：差异只随"优化档 × 三处 volatile 有无"变化，其余不变。

> ⚠️ **注意 · 本轮取证状态**
>
> 上述六个变体的**编译期**结果已用 `arm-none-eabi-gcc 15.2.1`（`-mcpu=cortex-m4 -mthumb`）在本机全部实测（`code/c/03-volatile/README.md` 附命令与反汇编）；**板上肉眼现象本轮未跑**：写这一版时手边没有接上 F407 与 ST-Link。谁先跑通，欢迎把波形/现象回填到这一节。

常见坑

1. **`-O0` 一切正常，发布档翻车**：成因基本就是现场 1/2/3。纪律：**功能验证至少跑一遍与发布一致的 `-O` 档**；出问题先用同档复现，别在 `-O0` 上找 bug。（出现频率第一。）
2. **照抄"去掉 `delay` 的 volatile，`-O2` 灯就不闪"**：本站第一版也这么写，实测才发现不成立：`00-blink` 的循环体里有 `__asm__ volatile ("nop")` 独立兜底，去掉参数 `volatile` 灯照闪（第五节 B 行）。**结论要自己跑一遍才敢写**，包括本站写的结论。
3. **漏了寄存器宏的 volatile，却去查 `delay`**：宏漏词的现象是"顺序倒了 / 换档换芯片才复发"（F、E 行），比"灯不闪"隐蔽得多。自查只要一条命令，把宏展开后再数：`arm-none-eabi-gcc -mcpu=cortex-m4 -E main.c | grep -c 'volatile uint32_t'`。`00-blink` 现状是 9，只漏寄存器宏是 1，三处全漏是 0。**数的是展开结果，不是源文件行数**：宏里的 volatile 写在头文件里，源码 grep 看不全。
4. **自己封装寄存器宏漏了 volatile**：少一个词就进现场 3 与 F 行。优先用 CMSIS 的 `TypeDef` 结构体（`__IO` 一次声明全体生效），别手搓一串地址宏。
5. **拿 `volatile` 当锁**：现场 4。中断与主循环都做读-改-写时必须进临界区，或改用外设侧的原子入口：`BSRR` 写 1 有效、写 0 无影响，天生不需要读-改-写。
6. **过度 volatile 焦虑症**：大缓冲区、纯本地中间变量全标 volatile，`memcpy` 被拆成逐字节访存，性能白送。判据回到第四节那句："谁会绕过编译器改这块内存？"
7. **以为 volatile 能管 cache**：F407（Cortex-M4，无数据 cache）上 volatile + 屏障基本够用；ESP32-S3 有数据 cache，DMA 共享缓冲还要 cache 清理/无效化，光 volatile 会读到旧数据。见 P6、P10。

短自测

1. `while (!(USART1->SR & USART_SR_RXNE));` **还需要另外加** `volatile` **吗？如果** `SR` **字段本身已经带 volatile。**

> **▼ 答案**
> 不用。CMSIS 的 `USART_TypeDef` 字段自带 volatile 限定（与上面 `GPIO_TypeDef` 同一套写法），`USART1->SR` 这个读已经是真访存。再套一层只会多出不必要的访存，SPL 的 `I2C_GetFlagStatus` 里那两个 `__IO` 局部变量就是"可加而不必加"的活样本（第四节）。

2. 为什么 `delay_plain()` **会被优化成一条** `ret` **？标准哪一条给了编译器这个权利？**

> **▼ 答案**
> 两层：① 没有副作用的空循环，标准允许编译器假定它不会无限执行（C11 起按未定义处理），循环可以直接消失；② 计数变量若非 volatile，其值在循环内"没人能改"，比较结果可被缓存与折叠。加 `volatile` 后每次使用都必须真实访存，循环每圈都要读改写它，删不掉。

3. 主循环 `flag ^= 1;` **、中断里** `flag = 0;` **，** `flag` **已是** `volatile` **。安全吗？为什么？**

> **▼ 答案**
> 不安全。volatile 只保证"读一改一写"三步都真的对内存发生，不保证三步之间不被抢占：主循环读旧值后被中断打断，中断写 0，主循环回到"写"这步把基于旧值算出的结果写回去，中断的更新就丢了。要么进临界区做读-改-写，要么改成"只置位/只清位"的标志协议，要么用外设侧原子寄存器。

4. `volatile uint32_t buf[256]` **当 DMA 缓冲用，代价是什么？F407 与 ESP32-S3 的结论一样吗？**

> **▼ 答案**
> 代价：对数组的每次普通访问都被强制成真实访存，`memcpy` 之类无法做块搬运优化，性能明显下降。结论不一样：F407 的 Cortex-M4 没有数据 cache，volatile + 屏障基本够；ESP32-S3 有数据 cache，DMA 与 CPU 共享的缓冲还要 cache 清理/无效化，光 volatile 不够。

🎯 C3 volatile 自测

1. **CMSIS 的 USART_TypeDef 寄存器字段本身带 volatile。轮询 `while (!(USART1->SR & RXNE))` 还需要再套 volatile 吗？**
   - A. 需要——轮询变量一律要 volatile 双保险
   - B. 不需要——字段自带 volatile，这个读已经是真访存；再套只会多出不必要的访存
   - C. 需要——不加 volatile 编译器会把整个循环删掉
   - D. 无所谓——寄存器读不受优化影响

2. **软件延时循环 `for (i=0;i<N;i++);` 在 -O2 下被优化成一条 ret，为什么合法？**
   - A. 编译器 bug，升级版本就好
   - B. N 太小时硬件跑得太快，看起来像被优化
   - C. 没有副作用的空循环，标准允许编译器假定其终止（C11 起不终止按未定义处理）；非 volatile 的 i 又可被缓存折叠——两重权利叠加
   - D. for 循环在 C 里本来就不保证执行

3. **主循环 `flag ^= 1;`、中断里 `flag = 0;`，flag 已声明 volatile。这样安全吗？**
   - A. 安全——volatile 保证原子性
   - B. 不安全——volatile 只保证每步真访存，不保证读-改-写三步之间不被抢占
   - C. 安全——中断打断主循环是原子的
   - D. 不安全——volatile 变量根本不能在中断里访问

4. **`volatile uint32_t buf[256]` 当 DMA 缓冲区，代价和边界是什么？**
   - A. 没有代价，volatile 是免费的保险
   - B. 每次普通访问都被强制真实访存，memcpy 无法块搬运优化；且 F407（无数据 cache）基本够用，ESP32-S3（有 cache）还必须做 cache 清理/无效化
   - C. volatile 会让 DMA 传输变慢，硬件直接不工作
   - D. 只在 -O0 下有代价，-O2 会自动撤销 volatile

5. **`asm volatile("nop")` 里的 volatile 和变量上的 volatile 是一回事吗？**
   - A. 一回事，都是"别优化掉"，作用对象相同
   - B. 不是一回事：这是 inline asm 的属性，阻止的是汇编语句被删除/移动/合并，与内存可见性无关
   - C. asm 上的 volatile 已被标准废弃
   - D. 只有变量上的 volatile 有实际效果，asm 上的是装饰

先选完再翻：答案与逐项解析在附录 C 的本组；在线版可点击即时判分。

对照表：本章概念 → 仓库落点

概念	仓库落点
四个现场的完整源码	`code/c/03-volatile/opt_probe.c`（本附录全文内嵌）
轮询被缓存	同上 `poll_plain` / `poll_volatile`，正文第二节汇编引证
空循环被删	同上 `delay_plain`，对照 `code/stm32/00-blink/main.c:39-44`
写被合并	同上 `cfg_plain` / `cfg_volatile`，对照 CMSIS `stm32f407xx.h:526-537`
读-改-写非原子	同上 `isr_toggle`；外设侧原子入口见 `docs/stm32/03-gpio.md` 的 BSRR 小节
访存**顺序**被重排	`code/c/03-volatile/blink-variants.sh` 的 F 行：宏漏 volatile → 读 `GPIOF_MODER` 排到写 `RCC_AHB1ENR` 之前
外设写被当死 store 全删	同上 E 行：三处 volatile 全去 → `main` 只剩一条 `b.n 0`
`asm volatile` 是另一道独立防线	`code/stm32/00-blink/main.c:42`，对照 B 行（只去参数 volatile → 灯照闪）与 C 行（连 `nop` 一起去掉 → 延时消失）
"可加而不必加"的 volatile	`stm32f4xx_i2c.c:1261-1264`（第四节），库解剖见 `docs/stm32/15-spl-anatomy.md`
默认 `-O0` 的板级工程	`code/stm32/00-blink/Makefile:20`

延伸阅读

本章的结论有两份原文可查：

- **[D7]** Eide & Regehr 2008 — 13 款编译器对 volatile 的错编实测，以及"把 volatile 访问包进函数"的对策；本章六变体表的学术版。
- **[C1]** N1570 §5.1.2.3 — as-if 规则与"volatile 访问是可观察行为"的原文，本章结论的法律条文。

你做到了

- 会取证：能给任意驱动代码生成 `-O0/-O2` 两份汇编，指出被缓存、被删、被合并的访存；
- 会判断：拿到一个 `volatile`，能说清它保的是哪类风险，答不出来源就知道是性能税；
- 会补齐：知道 volatile 不管原子与顺序，临界区、BSRR 式原子寄存器、屏障与 cache 操作各有其岗。

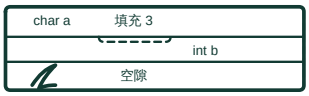
回到开场那段一开优化就死等的轮询：编译器按 as-if 规则办事，被它删掉的每一次读取，在它自己的账本上都是「结果没变」。现在你能把死等现场翻成两份汇编对齐着看，指出哪一条本该读而没读；也能在动 volatile 之前先问一句：这份合同到底要保哪类风险。

✅ 下一站：C4 结构体与 ABI：GPIO_TypeDef 凭什么用一个结构体罩住整组寄存器，字段顺序写错会撞到谁的地址。

通往单片机之路第 05 章

C4 结构体与 ABI：一个结构体罩住整组寄存器

难度 ★★☆☆☆ · 约 30 分钟



本章罗盘

问题

GPIO_TypeDef 凭什么罩住整组寄存器？成员偏移由谁担保，位域和 pack(1) 为什么是硬件寄存器的禁区？

交付

三个结构体的偏移/填充手算（Padded_t=12、Packed_t=8）经 offsetof 与 _Static_assert 验证，附对齐三规则表和位域改掩码宏的替代写法。

边界

布局规则出自 AAPCS32，位域分配顺序是实现定义、换编译器即翻车，不能外推；pack(1) 的正确用途是协议帧不是寄存器；参数怎么进函数属 C6 的调用约定，不属本章。

依据锚点 `c4` · `c1` · `f3` · `mcu-code` · `mcu-anim`

> 🎯 `GPIOF->BSRR = ...` 比 `*(volatile uint32_t*)(0x40021418)` 好看一百倍，但它凭什么是对的？答案藏在三条规则里：结构体成员按声明顺序排、地址按对齐递增、编译器不许重排。ARM 的 ABI 把这三条焊死了。

本章精髓

1. **结构体 = 编译器担保的"地址计算器"**：`GPIOF->BSRR` ≡ 基址 + 0x18，偏移由声明顺序和 ABI 对齐规则唯一确定。
2. **对齐与填充不是浪费**：成员按自身对齐要求落位（uint32_t 对 4 字节）；尾部填充保证数组 stride。你能在内存里"看见"这些洞。
3. **`__IO`（=volatile）+ `__I` / `__O` 标注的阅读法**：CMSIS 头文件的每个宏都在说话。

- **能理解**：对照 `probe.c` 的 `_Static_assert`，亲手改错一个成员顺序，看编译器如何当场报错。
- **能用**：拿到任何一颗新芯片的寄存器地图，能自己写出 TypeDef 并逐字段验证。

先修

- C2 指针、C3 volatile。

先跑起来（10 分钟 quick win）

```
cd code/c/04-struct-abi && sh probe.sh
```

先别急着跑：你能不看手册，只凭「成员按声明顺序排、宽度一致」心算出 BSRR 的偏移吗？把数字押在纸上，再跑脚本对账。

输出里找这两行：`offsetof(BSRR) = 0x18`、`sizeof(GPIO_TypeDef) = 0x28`，对上了，结构体封装当场验真。

小节结构

小节	内容	四件套
一、GPIO_TypeDef 解剖	逐成员对照 RM0090 §8.4 寄存器地图；`__IO/__I/__O` 宏展开	库解析
二、对齐三规则	成员对齐/结构体对齐/数组 stride；手算练习 ×3	配置
三、填充可视化	洞肉眼可见：`Padded_t` vs `Packed_t`	代码分析
四、位域的陷阱	实现定义布局 = 硬件寄存器禁区；用掩码宏替代	代码分析
五、端序与打包	小端确认实验；`#pragma pack` / `_Packed` 的正确用途（协议帧，不是寄存器）	配置

一、GPIO_TypeDef 解剖

先看这张图：偏移是"数"出来的，条带、填充、位域陷阱一次看全。

结构体对齐与填充动画

CMSIS 头文件把"寄存器地图"翻译成 C 结构体。以下是 `stm32f407xx.h` 第 526–537 行的官方定义（第二来源：github.com/STMicroelectronics/cmsis_device_f4）：

```
typedef struct
{
  __IO uint32_t MODER;    /*!< GPIO port mode register,               Address offset: 0x00 */
  __IO uint32_t OTYPER;   /*!< GPIO port output type register,        Address offset: 0x04 */
  __IO uint32_t OSPEEDR;  /*!< GPIO port output speed register,       Address offset: 0x08 */
  __IO uint32_t PUPDR;    /*!< GPIO port pull-up/pull-down register,  Address offset: 0x0C */
  __IO uint32_t IDR;      /*!< GPIO port input data register,         Address offset: 0x10 */
  __IO uint32_t ODR;      /*!< GPIO port output data register,        Address offset: 0x14 */
  __IO uint32_t BSRR;     /*!< GPIO port bit set/reset register,      Address offset: 0x18 */
  __IO uint32_t LCKR;     /*!< GPIO port configuration lock register, Address offset: 0x1C */
  __IO uint32_t AFR[2];   /*!< GPIO alternate function registers,     Address offset: 0x20-0x24 */
} GPIO_TypeDef;
```

三个宏展开：

- `__IO` → `volatile`（读+写都可能变）
- `__I` → `volatile const`（只读，硬件随时改）
- `__O` → `volatile`（只写，语义上强调）

`GPIOF` 的定义在头文件另一处：

```
#define GPIOF               ((GPIO_TypeDef *) GPIOF_BASE)   /* GPIOF_BASE = 0x40021400 */
```

所以 `GPIOF->BSRR = 1<<6` 的完整展开是：

```
(*(volatile uint32_t *)(0x40021400 + 0x18)) = (1 << 6);
```

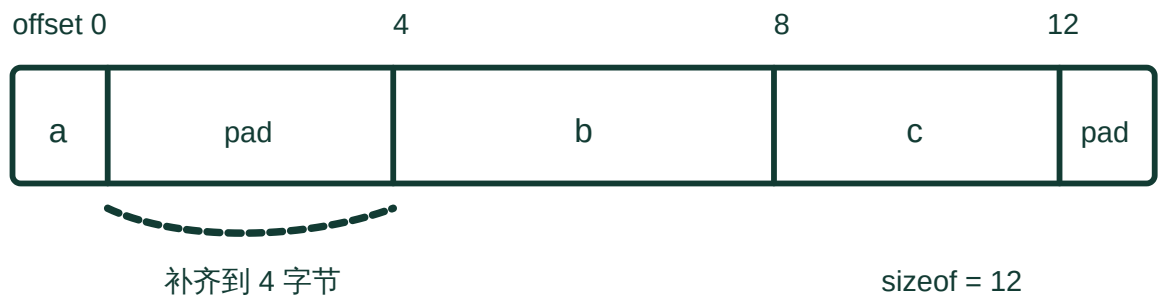
二、对齐三规则

ARM AAPCS（ARM 架构过程调用标准）把结构体布局焊死成三条：

规则	内容	结果
成员对齐	每个成员的起始偏移必须是其对齐值的倍数	`uint32_t` 永远落在 4 的倍数上
结构体对齐	结构体总大小必须是其最大成员对齐值的倍数	尾部可能补 padding
数组 stride	`sizeof(struct)` 已含尾部 padding，数组元素连续	`arr[i]` 的地址 = base + i × sizeof

手算练习（答案在附录源码的 `_Static_assert` 里）：

```
typedef struct { uint8_t a; uint32_t b; uint8_t c; } Padded_t;
/* 手算：a@0, pad 3, b@4, c@8, tail-pad 3 → sizeof=12 */

typedef struct { uint32_t b; uint8_t a; uint8_t c; } Packed_t;
/* 手算：b@0, a@4, c@5, tail-pad 2 → sizeof=8 */
```



结构体对齐：char a、int b、char c 之间插入 3 字节填充，sizeof 从 6 变 12

三、填充可视化

`probe.c` 跑出的真实输出：

```
Padded_t: sizeof=12, offsets a=0 b=4 c=8
Packed_t: sizeof=8, offsets a=4 b=0 c=5
```

`Padded_t` 的内存画像（`x` = 填充字节）：

```
偏移:  0    1 2 3    4 5 6 7    8    9 10 11
      [a] [x x x]  [b b b b]   [c]  [x x x]
```

成员重排后，`Packed_t` 省掉 4 字节。这就是"结构体成员从大到小排"省内存的力学原理。

四、位域的陷阱

```
typedef struct {
    uint32_t a : 1;
    uint32_t b : 2;
    uint32_t c : 5;
    uint32_t d : 24;
} Bitfield_1_t;
```

C 标准说：位域的分配顺序（从高到低还是从低到高）、是否跨存储单元、填充单元如何处理：**全部是实现定义的**。`probe.c` 里两种写法 `sizeof` 都是 4，但位序在不同编译器上可能完全相反。

结论：硬件寄存器映射用**掩码宏 + 移位**，不用位域：

```
#define REG_FIELD_A_MASK  (1UL << 0)
#define REG_FIELD_B_MASK  (3UL << 1)
/* 读： (reg & REG_FIELD_B_MASK) >> 1 */
/* 写： reg = (reg & ~REG_FIELD_B_MASK) | (val << 1) */
```

五、端序与打包

`probe.c` 的端序实验输出：

```
endian: 0x12345678 stored as 78 56 34 12 -> little-endian
```

最低有效字节 `0x78` 存在最低地址，小端。ARM Cortex-M4 默认小端。

`#pragma pack(1)` 或 `__attribute__((packed))` 的正确用途是**协议帧**（网络包、文件头），不是寄存器映射。给寄存器结构体加 pack(1) 会导致非对齐访问，Cortex-M4 上可能触发 `UsageFault`。

附录：工程完整源码

probe.c

<<< ../../code/c/04-struct-abi/probe.c

probe.sh

<<< ../../code/c/04-struct-abi/probe.sh

broken.c

<<< ../../code/c/04-struct-abi/broken.c

broken.sh

<<< ../../code/c/04-struct-abi/broken.sh

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **结构体罩寄存器=声明顺序定先后、ABI 定间隔、volatile 定纪律；位域别碰硬件，pack 只打包裹（协议）。**

实物实验

- **无板上（本机已实测）**：`cd code/c/04-struct-abi && sh probe.sh` 跑通，对照输出验证 `BSRR=0x18`、`sizeof=0x28`。
- **无板上（本机已实测）**：`sh broken.sh` 打印四种"写错一位"的真实布局：`AFR[2]` 写成 `AFR` 后 `sizeof=0x24`（`BSRR` 仍在 `0x18`）、寄存器地图的空洞不补保留字时 `DR` 从 `0x08` 前移到 `0x04`、`pack(1)` 下 `b` 落在偏移 `1`，并现场演示一次被 `_Static_assert` 拦下的编译。
- **板上（编译期已核实、肉眼现象待回填）**：在 STM32F407 工程里用 GDB 执行 `print &((GPIO_TypeDef*)0)->BSRR`，再读 `0x40021400+0x18` 对照：编译器算的偏移 = 手册给的偏移。

常见坑

- **成员顺序写错一位**：后面所有偏移全错，这就是"库版本和芯片不匹配"时最阴的 bug。
- **把位域当寄存器位用**：不同编译器布局不同，换工具链即翻车；正经做法是掩码 + 移位宏。
- **结构体里漏掉保留字（RESERVED）**：寄存器地图里的空洞必须用占位成员顶住，否则后续偏移整体前移。
- **给寄存器结构体加 `#pragma pack(1)`**：画蛇添足还破坏对齐访问（可能触发对齐异常）。

短自测

1. `GPIOF->BSRR = 1<<6` 展开成地址计算是什么？

▼ **答案**
`(*(volatile uint32_t *)(0x40021400 + 0x18)) = (1 << 6)`。`GPIOF_BASE = 0x40021400`，`BSRR` 偏移 `0x18`，由结构体成员顺序和 ABI 对齐规则唯一确定。

2. 为什么 `Padded_t { uint8_t a; uint32_t b; uint8_t c; }` 的 `sizeof` 是 12 而不是 6？

▼ 答案
对齐三规则：`uint32_t b` 必须落在 4 的倍数上，所以 `a` 后面插 3 字节填充；`c` 占 1 字节后，结构体总大小必须是最大成员对齐（4）的倍数，尾部再补 3 字节。`1+3+4+1+3 = 12`。

3. 为什么不用位域映射硬件寄存器？

▼ 答案
C 标准把位域布局（分配方向、跨单元策略、填充）留给编译器实现定义。同一份代码在 GCC、Clang、IAR 上可能产生不同的位序。硬件寄存器映射必须跨工具链稳定，所以用掩码宏 + 移位。

4. 小端机器上 `0x12345678` 的内存字节序是什么？

▼ 答案
`78 56 34 12`（低地址 → 高地址）。最低有效字节 `0x78` 存在最低地址。`probe.c` 已实测输出。

对照表：本章概念 → 仓库落点

概念	仓库落点
GPIO_TypeDef 成员顺序与偏移	`code/c/04-struct-abi/probe.c` 第 19–40 行
BSRR=0x18、sizeof=0x28 的编译期断言	同上 `_Static_assert`
对齐三规则手算验证	同上 `Padded_t` / `Packed_t`
端序确认（小端）	同上 `endian_check()`
位域不可移植证据	同上 `bitfield_check()`
掩码宏替代方案	同上 `REG_FIELD_*_MASK`

延伸阅读

字节账的条款出处：

- **[C4]** AAPCS32 — 基本数据类型的尺寸、对齐与位域规则的原文；本章每张字节账都能在"Data types and alignment"一节找到条款。

你做到了

- 再看到 `GPIOF->BSRR` 能心算出 `0x40021418`；
- 手握 `offsetof` / `sizeof` / `_Static_assert` 三件验证工具，任何"结构体封装"都能自证真伪；
- 知道位域为什么不能碰硬件、pack 为什么不能碰寄存器。

回到开场那句「好看一百倍，凭什么是对的」：现在你能拆开这台地址计算器亲自验账。成员按声明顺序、地址按对齐递增、编译器不许重排，三条规则都有编译期断言兜底。那行顺手好写的寄存器访问从此不是语法糖，是一笔你亲手对过账的偏移。

✅ 下一站：C5 函数指针与状态机：中断驱动代码的基本功三件套。

通往单片机之路第 06 章

C5 函数指针、状态机与环形缓冲

难度 ★★☆☆☆ · 约 30 分钟

本章罗盘

问题

中断驱动代码为什么总在排列组合『函数指针+状态机+环形缓冲』三件套？读写指针相等时，缓冲到底是空还是满？

交付

一份 560/560 断言通过的 probe 记录：fsm_feed 的 blx r3 反汇编逐行注解、环形缓冲 A/B 派容量对照（物理 8 格可用 7 vs 8）与发布/获取屏障写法。

边界

免锁证明只对单生产单消费成立：双方改同一索引就要上临界区（S7/F4 展开）；atomic_signal_fence 只挡编译器重排，总线序与多核可见性（DMB/DSB、cache）不在本章。

依据锚点 `d5` · `mcu-code` · `mcu-anim`

> 🎯 中断来了，硬件只负责跳到一个固定地址——"接下来干什么"全靠向量表里那个**函数指针**。而你要写的所有"中断驱动"代码，归根结底都是这三件套的排列组合：函数指针 + 状态机 + 环形缓冲。

本章精髓

1. 函数指针=可变的跳转目标：向量表、回调注册、命令分发表，本质都是"存起来的地址"。交叉编译后它长成一条指令：`blx r3`。
2. 状态机是中断代码的"记账本"：中断里只做记账（改状态+搬数据），不做业务，业务留给主循环。
3. 环形缓冲是生产者-消费者的最小解：中断生产、主循环消费；单生产单消费只是前提，还需索引访问原子、正确的发布/获取顺序和明确的满空规则。

- **能理解**：看 `fsm_feed` 的反汇编：`ldr.w r3, [r2, r3, lsl #2]` 查表、`blx r3` 跳过去。源码里那个 `t(f, ch)` 在机器码层面就是这么两条指令。
- **能用**：拿到"串口收到一帧命令"这类需求，直接落成"状态×事件→动作表"，不再写一坨 `if/else`。

先修

- C2 指针；S4 中断 可并行。

先跑起来（10 分钟 quick win）

```
cd code/c/05-func-pointer
sh probe.sh          # 宿主 gcc 跑 560 条断言；工具链在 PATH 时附送 Cortex-M4 反汇编
```

跑之前先押一题：同样容量的环形缓冲，「牺牲一格」与「单调索引」两种写法各能存几笔数据？数字相同，还是差一？押完再看输出第二行。

输出里找这两行：`== 断言通过 560/560 ==`、`环形缓冲容量对照：A 派（牺牲一格）可用 7 格，B 派（单调索引）可用 8 格`。

反汇编段里找 `blx`：**函数指针的调用就长这样**：

```
----- 全文件里的间接调用指令（blx = 跳到寄存器里的地址） -----
94:  9e:    4798        blx r3
133:  22:   4798        blx r3
```

小节结构

小节	内容	四件套
声明阅读法	右左法则拆解 `void (*table[8])(void)` ；typedef 的正确姿势	代码分析
向量表=指针数组	启动文件回看；weak 覆盖机制再讲一遍	库解析
查表法状态机	以一个"串口命令解析"状态机为例：状态×事件→动作表	代码分析
环形缓冲	读写指针取模回绕；满/空判定两派（牺牲一格 vs 计数）	代码分析
中断礼仪	记账不做事；重入与临界区预告（S7/F4 展开）	配置

一、声明阅读法：右左法则与 typedef

`probe.c` 里三种声明，指向的是同一个东西：

```
typedef void (*cb_t)(void);        /* cb_t = "指向 void(void) 函数的指针"这个类型 */

static cb_t key_table[8];                          /* 8 个函数指针 */
static void (*pick_raw(int id))(void);             /* 函数返回函数指针：裸写法 */
```

右左法则读 `void (*table[8])(void)` ：从名字 `table` 出发，**先右**看到 `[8]` ，说明 `table` 是数组；**再左**看到 `*` ，说明元素是指针；**再右**看到 `(void)` ，说明指针指向函数；**再左**看到 `void` ，说明函数返回 `void` 。合起来： `table` 是"8 个元素的数组，元素是返回 void 的无参函数的指针"。

`typedef` 只是给这个类型起名，不改语义。 `probe.c` 用一条断言把"两种写法等价"钉死：

```
cb_t via_typedef = on_press;
void (*via_raw)(void) = on_press;   /* 两种写法指向同一对象 */
CHECK(via_typedef == via_raw);
```

编译期还钉死了尺寸： `_Static_assert(sizeof(key_table) == 8 * sizeof(cb_t), ...)` ： `nm` 显示 `key_table` 占 0x20 = 32 字节，正好 8 × 4。

二、向量表 = Flash 里的函数指针数组

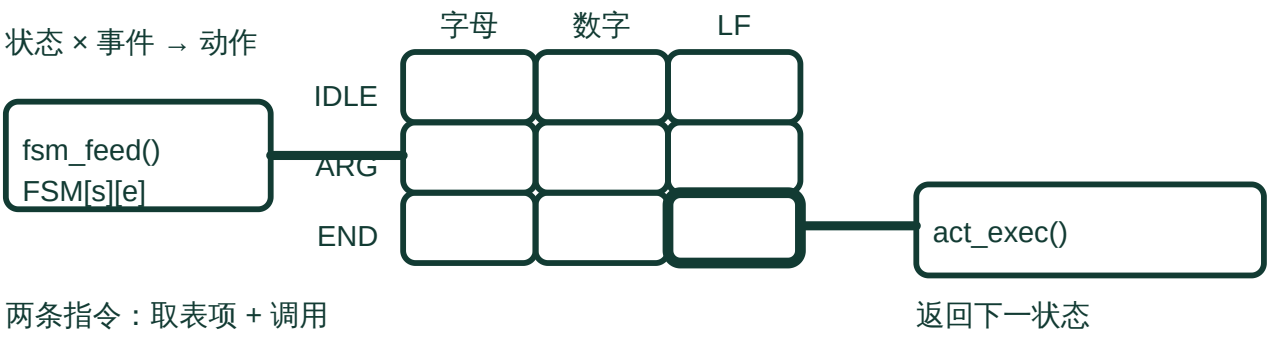
回看 B1 四步构建 里 `.isr_vector` 那 392 字节：它就是**一个 `const` 函数指针数组**，存在 Flash 起始处。启动文件里的每一行

```
.word EXTI0_IRQHandler
```

都是在表里填一个地址。 `Reset_Handler` 是 `W` （弱符号），就是"表里那个槽默认指向这里，你可以覆盖"的机制：**弱符号 = 可被覆盖的函数指针槽位**。

所以"注册中断服务函数"这件事，在 C 层面根本不需要 API：你写出同名函数，链接器就把表里那格重定位到你函数的地址。

三、查表法状态机： `fsm_feed` 的机器码只有两条指令



查表法状态机：状态×事件查表取出函数指针

`probe.c` 实现的是一个"串口命令解析"状态机：单行命令 `<字母><数字>\n` ，字母 ∈ {R,G,B}，数字 ∈ 0..9。核心是一张 **状态 × 事件 → 动作** 的 `const` 表：

```
typedef state_t (*trans_t)(fsm_t *f, char ch);   /* 动作 = 返回下一状态的函数 */

static trans_t const FSM[3][4] = {
    /*             EV_LETTER     EV_DIGIT      EV_LF       EV_OTHER  */
    /* S_IDLE */ { act_save_cmd, act_ignore,  act_ignore, act_ignore },
    /* S_ARG  */ { act_save_cmd, act_save_arg, act_error, act_error  },
    /* S_END  */ { act_error,    act_error,    act_exec,  act_error  },
};
```

整个"状态机"就是这一行：

```
__attribute__((noinline))
static void fsm_feed(fsm_t *f, char ch)
{
    trans_t t = FSM[f->state][classify(ch)];
    f->state = t(f, ch);            /* ← 查表 + 间接调用：中断里只做这点事 */
}
```

交叉编译（`-O2`）后 `fsm_feed` 的机器码，查表和跳转一目了然：

```
00000070 <fsm_feed>:
  70:   b510        push    {r4, lr}
  ...
  92:   eb03 0382   add.w   r3, r3, r2, lsl #2          @ r3 ← state*4 + ev
  96:   4a05        ldr r2, [pc, #20]   ; (ac <fsm_feed+0x3c>)  @ r2 ← FSM 基址
  98:   4620        mov r0, r4
  9a:   f852 3023   ldr.w   r3, [r2, r3, lsl #2]    @ 查表：取出函数地址
  9e:   4798        blx r3              @ 间接调用
  a0:   7020        strb    r0, [r4, #0]        @ 返回值写回 f->state
  a2:   bd10        pop {r4, pc}
```

三点值得注意：

1. **`blx r3` 就是"函数指针"在机器码层面的全部含义**：跳到寄存器里的地址。你在源码里看到的每一次"回调"，交叉编译后都是这条指令。
2. `f->state = t(f, ch)` 被翻译成 `strb r0, [r4, #0]`：**动作函数的返回值直接就是下一状态**。这个设计让整张表不需要任何"转移逻辑"代码，表驱动。
3. **`classify(ch)` 被完全内联掉了**（没有 `bl` 调用），只剩下 `cmp` / `beq` / `bls` / `ite` 一串比较。`-O2` 会把短小的分类函数摊平。

`const` 的价值在这里是硬的：`nm` 显示 `FSM` 是 `r` 类型（只读），落在 `.rodata`，**48 字节（3×4×4）一个字节都不占 RAM**。对比 `key_table`（运行时要注册，非 const）：`b` 类型，老老实实占 `.bss` 的 32 字节。**这就是 C1 讲的"const 经济学"**。

行为断言（`probe.c` 实测全绿）：

- `"R5\n"` → `n_exec==1`，`cmd=='R'`，`arg==5`，回到 `S_IDLE`；
- 逐字节喂 `'R'`、`'4'` 跨调用续命：喂完 `'4'` 时 `state==S_END`，再喂 `'\n'` 才 `n_exec` 加一：**状态活在结构体里，不在调用栈上**，这正是中断分片处理的必要条件；
- `"R\n"`（缺参数）、`"R5X\n"`（参数后混垃圾）→ `n_err` 加一、状态复位，**不崩溃不死等**。

四、环形缓冲两派：满/空判定的两条路

先看这张图：指针相等的歧义，和两条判定路线。

环形缓冲两派动画

单生产单消费环形缓冲的经典难题：**读写指针相等时，是空还是满？** 两派各有一个答案，`probe.c` 两派都实现了，用同一组测试对照。

A 派（牺牲一格）：留一个空位当哨兵：

```
if ((uint8_t)((w + 1) % RB_CAP) == q->r) return false;   /* 满 */
...
if (r == q->w) return false;                             /* 空 */
```

8 格物理容量，**只能用 7 格**（实测输出 `A 派可用 7 格`）。代价是浪费一格，好处是判定逻辑直白。

B 派（单调索引）：指针永不取模，交给 `uint8_t` 自然溢出，用"在飞元素数"判满：

```
uint8_t w = q->w;
if ((uint8_t)(w - q->r) >= RB_CAP) return false;         /* 满：无符号减法天然抗回绕 */
q->data[w % RB_CAP] = v;                                 /* 只在寻址时才取模 */
```

8 格全部可用（实测输出 `B 派可用 8 格`）。妙处在于 `(uint8_t)(w - q->r)`：即使 `w` 从 255 回绕到 0，无符号减法的模 256 语义让"在飞数量"依旧正确：**不用写任何回绕判断**。

两派共用的发布/获取顺序（这是"中断生产、主循环消费"的正确性关键）：

```
q->data[w % RB_CAP] = v;                          /* 先写数据 */
atomic_signal_fence(memory_order_release);        /* 屏障：数据先于写指针发布 */
q->w = (uint8_t)(w + 1);                          /* 再发布写指针 */
```

消费者对称：先读 `r`，再 `atomic_signal_fence(memory_order_acquire)`，最后取数据。**顺序反了就是"读到空槽"的经典 bug**（对应 C3 的屏障一节）。

长跑验证（两派各 600 步、塞取交错）：序号对 256 取模后与读出值逐一比对，`n_put == n_get`（一个不多一个不少）。B 派额外断言 `n_put > 256`：**确实跨过了 255→0 回绕**，且 `q.w == n_put mod 256`、`q.r == n_get mod 256`。A 派额外断言"满写被拒"路径真的触发过（`n_refused > 0`）。

五、中断礼仪：记账不做事

回到开篇那句"中断里只做快递分拣，不下厨房"。三件套的分工是这样的：

组件	中断里做什么	主循环做什么
函数指针（向量表/分发表）	一跳到位，不做判断	注册/替换处理函数
状态机	只改 `f->state`、搬一字节	读 `state` / `n_exec` 决定业务
环形缓冲	只 `put` 一个字节	循环 `get` 出来慢慢处理

为什么不能恋战：`fsm_feed` 的全部工作量是"一次查表 + 一次间接调用 + 一次 `strb`"：机器码 4 条有效指令。这就是中断预算的基准线：几十个时钟周期。一旦你在里面加 `printf`、`delay`、浮点运算，预算立刻爆掉（`-O0` 下 `printf` 要几百微秒，见 C3）。

重入与临界区预告：单生产单消费的成立前提是"生产者只有一个、消费者只有一个"。一旦中断和主循环都要**改同一个索引**（例如双向队列、或多优先级中断都往里塞），就必须上临界区，S7 与 F4 展开。

附录：工程完整源码

本章取证工程 `code/c/05-func-pointer/`（`sh probe.sh` 一条命令跑完，不需要开发板）：

probe.c（四个现场：回调注册表 / 查表状态机 / 环形缓冲 A 派 / B 派）：

<<< ../../code/c/05-func-pointer/probe.c

probe.sh（宿主编译运行 + 交叉反汇编取证）：

<<< ../../code/c/05-func-pointer/probe.sh

README.md（完整命令行与输出归档）：

<<< ../../code/c/05-func-pointer/README.md

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **中断三件套：指针记账、状态归档、缓冲转运；中断里只做快递分拣，不下厨房。**

实物实验

- **无板（本机已实测）**：`cd code/c/05-func-pointer && sh probe.sh`，560/560 断言通过；工具链在 `PATH` 时反汇编段应出现两条 `blx r3`。
- **无板（进阶）**：把 `probe.sh` 第 27 行的 `-O2` 换成 `-O0` 再跑，对比反汇编：`classify` 不再被内联，`fsm_feed` 变长，但 `blx r3` 依然在（**间接调用的本质不随优化等级改变**）。
- **板上（可选，肉眼现象待回填）**：在 00-blink 基础上加 EXTI 按键中断（前瞻 S4）：中断里只翻转一个 `volatile uint8_t evt=1;`，主循环检测到就换灯色，亲手体会"记账 vs 办事"的分工。

常见坑

- **函数指针类型不匹配**：签名差一个参数，调用就是 UB，编译器未必报警（老代码里 `void(*)()` 与 `void(*)(void)` 不一样）。
- **环形缓冲读写指针写反**：读指针只能消费者动、写指针只能生产者动，交叉修改即翻车。
- **状态机里做长延时**：中断/主循环统统被拖死，延时本身就该是一个状态（等超时事件）。
- **查表数组忘加 const**：白白吃掉 RAM（回 C1 的 const 经济学）。`nm` 一眼可辨：`r` = Flash，`b` / `d` = RAM。
- **分派前不判空**：`key_table[i]` 可能是 `NULL`（未注册的槽），直接调用就是跳进地址 0，`probe.c` 里 `if (key_table[i] != NULL)` 是纪律，不是装饰。这条纪律在反汇编里看得见：`main+0x20` 是一条 `cbz r3, 24 <main+0x24>`，紧接 `main+0x22` 才是 `blx r3`——空槽直接跳过。

短自测

1. 用右左法则读 `void (*table[8])(void)`。

> **▼ 答案**
> 从 `table` 出发：右看 `[8]` → 是数组；左看 `*` → 元素是指针；右看 `(void)` → 指向函数；左看 `void` → 函数返回 void。合起来：**8 个元素的数组，元素是指向"无参、返回 void 的函数"的指针**。`sizeof(table)` = 8 × 4 = 32 字节（`_Static_assert` 与 `nm` 双向验证）。

2. `blx r3` 是什么？为什么说"函数指针"在机器码层面就是它？

> **▼ 答案**
> `blx` = Branch with Link and Exchange：**跳到寄存器里的地址**，同时把返回地址放进 `lr`。函数指针变量存的就是一个地址，调用它 = 把地址装进寄存器 + `blx`。`probe.c` 交叉编译后在 `.text+0x9e`（`fsm_feed` 的查表分派）和 `.text.startup+0x22`（`main` 里内联的回调循环）各出现一条。

3. 环形缓冲 A 派和 B 派各能装多少格（物理 8 格）？为什么？

▼ 答案
A 派 7 格（牺牲一格当哨兵：`w+1 == r` 判满，所以永远留一格空）；B 派 8 格（用 `(uint8_t)(w - r) >= RB_CAP` 判满，"在飞数量"用无符号减法算，天然抗 255→0 回绕）。实测输出 `A 派可用 7 格，B 派可用 8 格`。

4. `FSM` 表和 `key_table` 分别落在哪个段？说明了什么？

▼ 答案
`FSM` 是 `const` → `nm` 类型 `r`（只读），落在 `.rodata`，链接后进 Flash，48 字节不占 RAM。`key_table` 运行时才注册（非 const）→ 类型 `b`，落在 `.bss`，占 RAM 32 字节。同一份代码里，"能不能改"直接决定它住 Flash 还是住 RAM。

5. 为什么消费者要先读指针、再屏障、最后取数据？

▼ 答案
生产者是"先写数据、再发布写指针"；消费者必须对称地"先看到写指针、再读数据"。`atomic_signal_fence(memory_order_acquire)` 阻止编译器把"取数据"提前到"读指针"之前，否则可能读到还没写进去的空槽。顺序反了就是经典的单生产单消费 bug。

对照表：本章概念 → 仓库落点

概念	仓库落点
回调注册表与右左法则	`code/c/05-func-pointer/probe.c` 34–69 行
typedef 与裸写法等价（断言）	同上 66–68 行
查表法状态机 + 动作表	同上 71–132 行（`FSM` 表在 114–119）
`fsm_feed` 的 `blx r3` 反汇编	同上，`probe.sh` 第 2 段输出
环形缓冲 A 派（牺牲一格）	同上 164–234 行
环形缓冲 B 派（单调索引）	同上 236–304 行
发布/获取屏障	同上 `atomic_signal_fence`（180/190/250/260 行）
`const` 表落 `.rodata` 的证据	`nm build/probe.m4.o`（`FSM = r`，`key_table = b`）

延伸阅读

环形缓冲"不加锁也对"不是经验，是证明过的：

- **[D5]** Lamport 1977 — 单生产者单消费者环形缓冲为什么不需要锁；本章"发布/获取顺序"那两道屏障的证明出处。
- **[D10]** Dijkstra 1965 — 互斥问题的起点，一页纸。当缓冲区不再是单生产单消费，你要走的就是这篇开出来的路。

你做到了

- 向量表在你眼里就是"Flash 里的函数指针数组"；
- 手握三件套，能搭出任何中断驱动骨架。

回到开场那个「硬件只负责跳」的瞬间：跳过去之后的世界，你已经能用三件套亲手搭出来。查表分发是一条间接跳转指令，状态记账留给主循环，环形缓冲把中断与业务接成流水线。所谓「中断驱动」，拆开看全是这三种零件。

✅ 下一站：C6 调用约定与栈帧：参数怎么进、栈帧怎么长，一次函数调用的全程直播（动画）。

通往单片机之路第 07 章

C6 调用约定与栈帧

难度 ★★★☆☆ · 约 45 分钟

本章罗盘

问题

一次函数调用，参数凭什么送得到、函数凭什么回得来？mix(int, long long) 里 r1 为什么被白白跳过？

交付

一张 AAPCS 寄存器分工表加三段汇编注解（sum6 第 5/6 参数走栈、non_leaf 的 push {r4,lr}…pop {r4,pc}、add_d 走 d0/d1），宿主一帧 64 字节、M4 同函数 24 字节的实测账。

边界

帧字节数随编译器与目标变（64 vs 24），别拿宿主数字套芯片；soft/hard ABI 目标文件不能混链只点到为止；HardFault 挖栈中 PC 在 S16、RTOS 上下文切换在 F2。

依据锚点 `c4 · a3 · mcu-code · mcu-anim`

> 🎯 参数装 r0–r3、多余的走栈、返回地址进 lr、返回值回 r0——函数调用的全部秘密就这四条。

本章精髓

1. 函数凭什么"调用完就回来"？ `bl` 把返回地址写进 lr；非叶子函数把 lr 压栈，收尾时 `pop {…, pc}` 直接弹回程序计数器。
2. 参数凭什么"送得到"？AAPCS 铁律：r0–r3 装前 4 个、第 5 个起走栈、64 位占偶寄存器对、硬浮点走 d0–d7。
3. 局部变量凭什么"用完自动回收"？栈帧随 sp 移动而生灭，分配与释放的全部成本就是改一根指针。

- **能理解**：拿 `sum6` 的 `ldr r2, [sp]`、`mix` 调用方不写的 r1、`non_leaf` 的 `push {r4, lr}` … `pop {r4, pc}` 三段真实汇编，逐条对到 ABI 规则上。
- **能用**：给你一段 Cortex-M 的 `-O2` 反汇编，能说出参数从哪进、栈帧多大、返回地址压在栈上哪个字，S16 崩溃定位要用的就是这个本事。

先修

- C4 结构体与 ABI：数据在内存里怎么排；本章讲调用发生时数据怎么进函数。
- C5 函数指针：`blx r3` 决定"跳到哪"；本章回答"跳过去之后参数怎么进、栈帧怎么长"。
- C1 内存模型：知道栈住在 SRAM 哪一端、往哪头长。
- B4 启动过程 可读可后补：知道复位后 sp 从哪来。

先跑起来（10 分钟 quick win）

```
cd code/c/06-abi-stack
sh probe.sh   # 宿主 gcc 真跑 9 条断言；arm-none-eabi-gcc 在 PATH 时附送 Cortex-M4 汇编核对
```

跑之前先押：那个非叶子函数只开了一个局部数组，它的一帧会吃掉多少字节？刚好等于数组大小，还是更多？多出来的部分又是谁？押完对第一行输出。

宿主侧（gcc 16.1.0 实测输出，每条都是真跑出来的数）：

```
一帧吃掉的栈          = 64 字节（两次 sp 之差，含返回地址+保存寄存器+buf）
buf 距 sp(non_leaf)   = 32 字节
sum6(1,2,3,4,5,6)     = 21（第 5、6 个参数 5、6 是走栈送进去的）
add64(0x100000000, 0x200000001) = 0x300000001（64 位结果走寄存器对回来）
== 断言通过 9/9 ==
```

交叉侧（本机 PATH 里的 arm-none-eabi-gcc 10.3.1，`-mcpu=cortex-m4 -mfloat-abi=hard -O2 -S`）里找这三行，AAPCS 就长这样：

```
add2:
    @ link register save eliminated.    " 叶子函数：连 lr 都不用存
    ldr r2, [sp]                        " sum6：第 5 个参数从栈上取
    pop {r4, pc}                        " non_leaf：返回地址直接弹进 PC
```

> 用哪几个寄存器接收栈上的值，是编译器的自由：10.3.1 给成 `ldr r2, [sp]`，别的版本可能是 `ldr r1, [sp]`。AAPCS 只承诺"前 4 个参数在 r0–r3，第 5 个起在栈上 `[sp]` / `[sp,#4]`"，剩下的它不管。本章引用的每一条汇编都是本机 10.3.1 实跑产物，行号对得上 `probe.sh`。

动画：栈帧的一生

四个阶段循环播放：① 参数装车（r0–r3）② `push {r4, lr}` 保护现场 ③ 局部变量占坑（`sub sp`）④ `pop {r4, pc}` 满血返回。

函数调用栈帧动画

小节结构

小节	内容	四件套
一、为什么需要"约定"	编译器不止一家，没约定就无法互相调用；寄存器分工表逐行对证据	代码分析
二、参数落位铁律	r0–r3 装前 4 个；sum6 的第 5、6 个走栈（`@ args = 8` / `strd` / `ldr [sp]`）	代码分析
三、64 位与偶奇寄存器对	add64 的 adds/adc 进位链；mix 跳过 r1 的调用方铁证	代码分析
四、硬浮点 ABI	double 走 d0/d1 不占 r0:r1；fpv4-sp 只有单精度 → `__aeabi_dadd`	代码分析
五、栈帧解剖	非叶子 `push {r4, lr}` + `sub sp` … `pop {r4, pc}`；叶子 add2 对照	代码分析
六、把 ABI 落成字节账	宿主实测一帧 64 字节、buf 距 sp 32 字节；M4 上同一函数 24 字节	代码分析

一、为什么需要"约定"

启动文件里一行 `bl main` 为什么能成？编译器是 gcc，启动文件是手写汇编，两者从没见过面，能接上头全靠 **AAPCS**（Procedure Call Standard for the ARM Architecture）：ARM 公布的调用约定，gcc / clang / IAR / Keil 全都遵守。它规定四个问题：参数放哪、返回值放哪、哪些寄存器用完要复原、栈怎么对齐。

核心寄存器分工（每一行都能在本章 probe 里找到对应证据）：

寄存器	别名	职责	调完还有效吗	本章证据
r0–r3	a1–a4	前 4 个参数 / 临时scratch；r0（64 位 r0:r1）带回返回值	被调方随便改（调用方自保）	sum6/add64/mix 全段
r4–r8、r10、r11	v1–v8	长期变量	被调方想用就得先存后恢复	non_leaf `push {r4, lr}`
r9	v6/sb	平台保留	被调方保存	——
r12	ip	链接器贴片用的 scratch	被调方随便改	——
r13	sp	栈指针，公共接口处 8 字节对齐	——	probe.c 第 113 行对齐断言
r14	lr	返回地址，`bl` 时硬件写入	每次调用都被覆盖	non_leaf 必须 push 它
r15	pc	程序计数器	——	`pop {r4, pc}` 改写它完成返回

说白了，约定 = 谁可以用、谁必须存、参数放哪、返回值放哪。约定之外没有魔法。

二、参数落位铁律：r0–r3 装前 4 个，第 5 个起走栈

`sum6` 有 6 个 int 参数（probe.c 第 40–44 行），r0–r3 只装得下 4 个。看被调方汇编（probe.sh 第 45–46 行取证输出）：

```
sum6:
    @ args = 8, pretend = 0, frame = 0
    @ frame_needed = 0, uses_anonymous_args = 0
    @ link register save eliminated.
    add r1, r1, r0
    add r2, r2, r1
    add r3, r3, r2
    ldr r2, [sp]        @ 第 5 个参数 e，从栈上取
    ldr r0, [sp, #4]    @ 第 6 个参数 f
    add r3, r3, r2
    add r0, r0, r3
    bx  lr
```

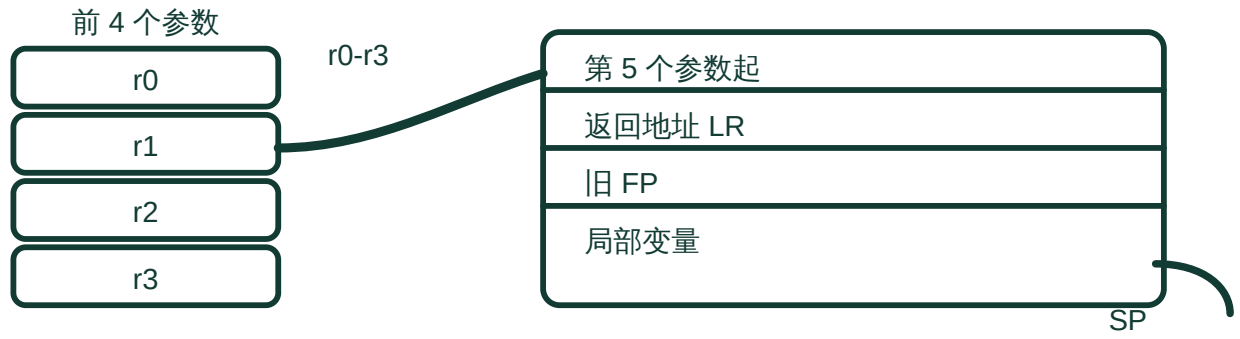
- `@ args = 8` 是编译器注释：调用方在栈上放了 **8 字节**入参（两个 int），函数自己的帧却是 0（`frame = 0`）。
- 前 4 个参数就在 r0–r3 里，三条 `add` 直接加完；第 5、6 个用 `ldr r2, [sp]` / `ldr r0, [sp, #4]` 从栈上读回来。

调用方一侧（main 内，`bl sum6` 之前，build/probe.s 第 217–224 行；probe.sh 第 48–50 行取证）：

```
    movs    r3, #5
    movs    r2, #6
    strd    r3, r2, [sp]   @ 一条指令：5 → [sp]，6 → [sp,#4]
    movs    r2, #3
    subs    r4, r0, r2     @ 断言 add2 的返回值 ==3，与传参无关
    movs    r3, #4
    movs    r0, #1
    bl  sum6
```

`strd`（store double）一次把第 5、6 参数写上栈顶，与被调方的两条 `ldr` 严丝合缝。（细心的读者会发现 r1=2 没被重写：编译器在同一编译单元内知道 `add2` 不改 r1，就沿用了，这是对函数体内情的优化，跨模块调用时 ABI 不作此承诺。）

行为断言兜底：`sum6(1,2,3,4,5,6) = 21`（宿主 9/9 之一），**语义不因传参方式改变**，变的只是成本：栈传参多出内存写+内存读，寄存器传参零额外访存。性能敏感的接口，参数控制在 4 个以内。



AAPCS 参数落位：前四个参数进 r0-r3，第五个起压栈；栈帧里还有 LR、旧 FP 与局部变量

三、64 位参数：必须落在偶奇寄存器对

`add64`（probe.c 第 47–51 行）两个 `long long`：

```
add64:
    @ args = 0
    adds    r0, r0, r2    @ 低 32 位相加，s 后缀更新进位标志
    adc r1, r1, r3      @ 高 32 位 + 进位
    bx  lr
```

x 在 **r0:r1**、y 在 **r2:r3**，返回值也在 r0:r1。`adds` / `adc` 进位链就是"两个 32 位寄存器扮一个 64 位数"的铁证。宿主实测：`add64(0x100000000, 0x200000001) = 0x300000001`。

真正的反直觉点在 `mix(int tag, long long v)`（probe.c 第 53–57 行）：tag 占了 r0，v **不能**放 r1:r2，AAPCS 要求 64 位参数对齐到**偶数号**寄存器，于是 v 落 r2:r3，**r1 被跳过**。

- 被调方从没读过入参 r1：`mov r1, r0`（r1 只当返回值高半的暂存）、`adds r0, r0, r2`、`adc r1, r3, r1, asr #31`：读的入参只有 r0、r2、r3。
- 调用方铁证（`bl mix` 之前，probe.sh 第 55–60 行取证段）：

```
    movs    r2, #0     @ v 低 32 位
    movs    r3, #1     @ v 高 32 位 → v = 0x1_0000_0000
    movs    r0, #7     @ tag
    bl  mix        @ r1 压根没被赋值——这就是"跳过"
```

此刻 r1 里躺的是上一个函数的返回高半（垃圾值），而 mix 正确地对它视而不见。宿主断言：`mix(7, 0x100000000) = 0x100000007`。推论：给函数签名排序时把 64 位参数放前面、int 填后面，能少留寄存器空洞——ABI 不会替你重排。

四、硬浮点 ABI：double 走 d0/d1，不占 r0:r1

编译旗 `-mfpu=fpv4-sp-d16 -mfloat-abi=hard`（probe.sh 第 37 行）。hard 表示**浮点参数走 VFP 寄存器**：单精度 s0–s15、双精度 d0–d7。`add_d(double, double)`（probe.c 第 60–64 行）的汇编（build/probe.s 第 90–99 行）：

```
add_d:
    @ args = 0            @ 栈上参数 0 字节
    push    {r3, lr}
    vmov    r0, r1, d0    @ 把 d0 搬进内核寄存器对……
    vmov    r2, r3, d1    @ ……只是因为 fpv4-sp 只有单精度硬件，
    bl  __aeabi_dadd    @   double 加法得调软件库（它按 r0:r1/r2:r3 收参）
```

两个 double 是从 **d0**、**d1** 进来的，`vmov r0, r1, d0` 恰好证明了这一点；占 r0:r1 是被调的软件库 `__aeabi_dadd` 的内部约定，不是本函数的传参方式。对照：`-mfloat-abi=soft` 时 double 按 64 位整数规则直接走 r0:r1。两种 ABI 编出的目标文件**不能混链**（链接器会报 VFP 参数 mismatch）：这就是"ABI"三个字的分量。宿主断言：`add_d(0.5, 0.25) == 0.75`（9/9 之一）。

五、栈帧解剖：push {r4, lr} … pop {r4, pc}

`non_leaf`（probe.c 第 78–88 行）内部还要 `bl sink`、`bl add2`，每次 `bl` 都会覆盖 lr，所以 prologue 必须先把 lr 存起来。为什么还带着 r4？因为 `sink` 标了 `__attribute__((noipa))`（probe.c 第 70–76 行注释）：禁用一切过程间分析，调用方只能按 ABI 做最坏假定：r0–r3、r12 全被改写。想活过这次调用的值，只能放进被调方保存寄存器，r4 由此登场。

完整帧（probe.sh 第 62–63 行取证输出）：

```
non_leaf:
    push    {r4, lr}        @ 8 字节：保存 r4 与返回地址
    sub sp, sp, #16     @ 16 字节：buf[16] 的家
    @ …
    bl  sink
    bl  add2
    pop {r4, pc}        @ 恢复 r4，返回地址直接弹进 PC——连 bx lr 都省了
```

返回地址弹进 PC 是 Thumb-2 的经典收尾：弹栈与跳转合一。口诀里"lr 记着回家道"的下半场：回家不一定要经过 lr，可以直接进门。这个 Cortex-M4 帧 = push 8 + sub 16 = **24 字节**。

对照组 `add2`（probe.c 第 33–37 行）：叶子函数不调用任何人，编译器直接注释 `@ link register save eliminated.`，全身就 `add r0, r0, r1` + `bx lr` 两条指令。计数核对（probe.sh 第 66–70 行）：全文件 `push {r4, lr}` 恰出现 1 次、`pop {r4, pc}` 恰出现 1 次，正是 non_leaf 这一对；`strd , [sp]` 也恰 1 次，就是 sum6 那次的栈传参。

六、把抽象 ABI 落成字节账（宿主实测）

宿主 gcc 16.1.0（x86-64）真跑 non_leaf，探针打印（probe.c 第 115–121 行）：

量	实测值
sp(main, 调用前)	`0x7d73dff980`
sp(non_leaf 体内)	`0x7d73dff940`
一帧吃掉的栈	**64 字节**（含返回地址 + 保存寄存器 + buf + 对齐填充）
buf 地址	`0x7d73dff960`
buf 距 sp(non_leaf)	**32 字节**

布局断言同步通过（probe.c 第 110–113 行）：sp 向下走（`sp_callee < sp_main`）、buf 落在本帧之内、帧差是 8 的倍数：AAPCS 要求公共接口处 sp **8 字节对齐**，主流桌面 ABI 同样有对齐条款。

注意口径：64 字节是宿主编译器的帧；**同一个 non_leaf 编到 Cortex-M4 是 24 字节**（第五节的 8+16）。字节账随目标与编译器而变，不变的是机制：**sp 一根指针的移动 = 分配与释放的全部成本**。

与前后章的呼应：

- C5 函数指针：函数指针在机器码层面是 `blx r3`，跳到寄存器里的地址；本章补的是跳过去之后"参数怎么进、帧怎么长"。传参规则与直接调用完全一致，函数指针才能与直调无缝互换。
- S16 HardFault 与排错：HardFault 时硬件把 r0–r3、r12、lr、pc、xPSR 自动压进当前栈：反查崩溃行号，就是按本章的栈帧知识把那个 pc 从栈里挖出来。
- F2 上下文切换：切任务 = 把一整套寄存器现场换栈保存/恢复，机制与本章 prologue/epilogue 同源。

附录：工程完整源码

本章取证工程 `code/c/06-abi-stack/`（`sh probe.sh` 一条命令跑完，不需要开发板）：

probe.c（六个实验函数：add2 / sum6 / add64 / mix / add_d / non_leaf）：

<<< ../../code/c/06-abi-stack/probe.c

probe.sh（宿主编译运行 + 交叉汇编取证）：

<<< ../../code/c/06-abi-stack/probe.sh

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **参数 r0 到 r3，多余的上栈找；64 位占偶奇对，r1 可以被跳掉；返回值 r0 里抱；r4–r11 谁用谁保管；lr 记着回家道，pop 进 pc 一把销。**

实物实验

- **装备**：霸天虎板 + ST-Link + 第 0 章工程 + OpenOCD/GDB。
- **做法**：断点打在任一非叶子函数入口，`si` 单步过 prologue；每步 `info registers sp lr pc`，再 `x/4xw $sp` 看栈顶。
- **预期现象**（数字来自本章 Cortex-M4 汇编）：`push {r4, lr}` 后 sp 减 8，栈顶第二个字就是返回地址（lr 的拷贝）；`sub sp, #16` 后再减 16，整帧 24 字节；`pop {r4, pc}` 后 sp 复原，pc 精确落回 `bl` 的下一条指令。

- **进阶**：在 HardFault_Handler 里读硬件压栈帧的第 7 个字（偏移 24，即 stacked pc），对照 map 文件找崩溃行，完整流程见 S16。
- 交叉引用：实验中心 E02 待建。

常见坑

- **误以为 `lr` 不用保存**：函数里再调函数（非叶子），第一次 `bl` 就把 lr 覆盖了，所以 non_leaf 的 prologue 一定有 `push {…, lr}`，而叶子 add2 才配拥有 `@ link register save eliminated.`。手写汇编忘了 push lr，返回时程序直接乱飞。
- **64 位参数落位想当然**：`mix(int, long long)` 的 long long 在 r2:r3 而不在 r1:r2，调试时按 r1 找值，找到的是上一个函数留下的垃圾（本章调用方汇编里 r1 残留 add64 的返回高半）。
- **中断里也在用同一个栈**：ISR 的硬件压栈发生在被中断者的栈上，栈预算要算上最坏中断嵌套，F2 上下文切换会回头算这笔账。
- **递归无界 = 栈溢出踩堆/全局区**：裸机没有 MMU 兜底，症状是"变量莫名其妙变了"，HardFault 都不一定报。
- **`main` 里 `return` 不会"回到命令行"**：裸机没有命令行可回；B4 启动过程里 main 返回后的去向是启动代码的死循环。

短自测

1. `sum6` 的第 5、6 个参数怎么进函数？给出调用方与被调方双向证据。

> **▼ 看答案**
> 调用方：main 在 `bl sum6` 前用一条 `strd r3, r2, [sp]` 把 5 → `[sp]`、6 → `[sp,#4]` 写上栈（build/probe.s 第 224 行）。被调方：`sum6` 内 `ldr r2, [sp]` / `ldr r0, [sp, #4]` 取回，且编译器注释 `@ args = 8` 标明栈上入参 8 字节（自己的 `frame = 0`）。宿主断言 `sum6(1..6)=21` 通过。

2. `mix(int tag, long long v)` 里 v 为什么在 r2:r3 而不是 r1:r2？此刻 r1 里是什么？

> **▼ 看答案**
> AAPCS 要求 64 位参数落在**偶奇寄存器对**。tag 占了 r0 后，下一个可用的偶奇对是 r2:r3，r1 被跳过。调用方汇编（`movs r2, #0` / `movs r3, #1` / `movs r0, #7` / `bl mix`）从头到尾没写 r1，r1 里残留的是 add64 返回值的高半，mix 对它视而不见。

3. 为什么 non_leaf 必须 `push lr`，而 add2 不用？各引一条证据。

> **▼ 看答案**
> non_leaf 内部 `bl sink`、`bl add2`，任何一次 `bl` 都会覆盖 lr，不保存就回不了家，所以 prologue 有 `push {r4, lr}`。add2 是叶子函数（不调用任何人），lr 不会被覆盖，编译器直接注释 `@ link register save eliminated.`，全身只有 `add r0, r0, r1` + `bx lr`。

4. `pop {r4, pc}` 一条指令完成了哪两件事？为什么能省掉 `bx lr`？

> **▼ 看答案**
> ① 从栈上恢复 r4（被调方保存寄存器复原）；② 把 prologue 存下的返回地址直接弹进 pc，弹栈与跳转合一，程序计数器被改写的瞬间就完成了返回，自然不需要再走 lr + `bx lr` 这条路。这是 Thumb-2 的经典收尾。

5. 硬浮点 ABI 下 `add_d` 的两个 double 从哪进来？汇编里哪条指令能证明？

> **▼ 看答案**
> 从 d0、d1 进来（`-mfloat-abi=hard` 时浮点参数走 VFP 寄存器，不占 r0:r1，所以 `@ args = 0`）。证据是 `vmov r0, r1, d0` / `vmov r2, r3, d1`：编译器把 d0/d1 搬进内核寄存器，只是因为 fpv4-sp 没有双精度硬件，要调软件库 `__aeabi_dadd`。

对照表：本章概念 → 仓库落点

概念	仓库落点
叶子函数不保存 lr	`code/c/06-abi-stack/probe.c` 33-37 行；汇编取证 probe.sh 42-43 行
第 5、6 参数走栈（sum6）	probe.c 40-44 行；`@ args = 8` 与 `ldr [sp]` 见 probe.sh 45-46 行输出
调用方 strd 写栈传参	probe.sh 第 48-50 行取证；build/probe.s 第 224 行（`strd r3, r2, [sp]`，main 内）
64 位寄存器对 / adds+adc	probe.c 47-51 行；probe.sh 52-53 行输出
r1 跳过（mix）	probe.c 53-57 行；调用方取证 probe.sh 55-60 行
硬浮点 d0/d1（add_d）	probe.c 60-64 行；编译旗见 probe.sh 37 行；汇编 build/probe.s 89-95 行
noipa 迫使 r4 登场	probe.c 70-76 行（sink 的 `__attribute__((noipa))` 注释）
非叶子栈帧三件套	probe.c 78-88 行；`push/sub … pop` 取证 probe.sh 62-63 行
帧字节账与对齐断言	probe.c 94、110-121 行（一帧 64 字节、buf 距 sp 32 字节）
9/9 行为断言	probe.c 103-109、127 行
push/pop 计数核对	probe.sh 61-64 行（各恰 1 次）

延伸阅读

9 条断言逐条对应的条款：

- **[C4]** AAPCS32 — r0–r3 传参、r4–r11 被调用者保存、64 位参数落偶奇寄存器对、公共接口处栈 8 字节对齐，全在"Procedure Call Standard"一节。

你做到了

- 看反汇编不再发怵：prologue/epilogue 一眼认出，`@ args = 8` 这种注释也会读了。
- 明白局部变量"自动回收"其实是 sp 一根指针的移动，一帧 24～64 字节的账自己会算。
- 能预言参数落位（包括 64 位跳过 r1 这种反直觉情况），并能拿汇编验证自己的预言。
- 给 S16 的崩溃现场分析备好了全部读帧本领。

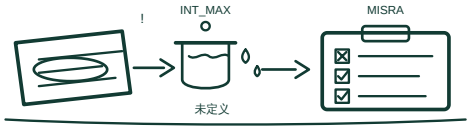
回到开场那四条口诀：参数装 r0–r3、多余的走栈、返回地址进 lr、返回值回 r0，现在每一条都有你亲手跑出来的汇编行作证。一次函数调用不再是黑盒，是一笔算得出的栈账。

✅ 下一站：C7 未定义行为与 MISRA-C 精要：看看约定管不到的"未定义行为"能野到什么程度；也可以拐去 S16 HardFault 与排错，把栈帧里的 PC 挖出来反查崩溃行号。

通往单片机之路**第 08 章**

C7 未定义行为与 MISRA-C 精要

难度 ★★★☆☆ · 约 35 分钟



本章罗盘

问题

昨天还能跑的代码，换个优化等级就变脸，编译器凭什么把 x+1>x 折叠成恒真？哪些写法是裸机上的定时炸弹？

交付

一份三档实测记录：is_bigger_after_inc(INT_MAX) 在 -O0/-O2/-fwrapv 下读数变化、1u<<33 两档对照，配『嵌入式 MISRA 十条』与警告/UBSan/静态分析三道防线的编译旗清单。

边界

新工具链连 -O0 都做折叠，『-O0 能跑=代码正确』作废，且读数随工具链版本变、引用汇编须注明版本；-fwrapv 只救有符号加减乘，移位三条红线一条不救。

依据锚点 `c1` · `c2` · `d8` · `mcu-code` · `mcu-anim`

> 🎯 UB（未定义行为）最阴险的地方：代码**看起来对、编译通过、今天能跑**——直到换编译器版本、换优化等级、或者客户演示那天，它才炸给你看。嵌入式没有操作系统替你挡枪，UB 就是定时炸弹。

本章精髓

1. UB ≠ 报错：标准甩手不管，编译器就可以**以"你没有 UB"为前提做优化**：`x+1 > x`（有符号溢出）能被优化成恒真。
2. 高危三兄弟：有符号溢出、移位越界（`1<<33`）、严格别名冲突（用不兼容类型的指针访问同一对象）。
3. MISRA-C 不是繁文缛节：它把"炸弹区"画成红线，汽车/医疗固件靠它活命；我们取其中性价比最高的 10 条自用。

- **能理解**：拿 probe 工程的**真跑输出**逐条对账：`is_bigger_after_inc(INT_MAX)` 在 -O0/-O2/-fwrapv 三档下的读数、`1u<<33` 在两档下的变脸、ARM 汇编里 `movs r0, #1` 那两行铁证，以及宿主 x86 上 `-O0` 函数体就已经是 `movl $1, %eax`、`-O2` 连 `call` 都不剩的那两段。
- **能用**：给一段新代码，先扫三兄弟；给一次"昨天还好好的"灵异故障，第一反应查优化等级差异，而不是怀疑硬件。

先修

- C2 指针、C3 volatile、C4 结构体与 ABI。

先跑起来（10 分钟 quick win）

```
cd code/c/07-ub-misra
sh probe.sh   # 宿主 gcc 真跑三档对照 + 编译器防线现场 + ARM 交叉汇编铁证
```

跑之前先押：第一个对照实验里的边界比较，在两个优化档位下各返回几？十年前的老教程有标准答案，它今天还作数吗？两注都下完，再看读数。

先看两个读数（gcc 16.1.0 实测）：

- `is_bigger_after_inc(INT_MAX) = 1`：**两个优化档位都返回 1**。十年前的老教程说"-O0 返回 0、-O2 返回 1"，在 gcc 16 上前半句已经翻案：折叠器在 -O0 也工作。亲眼看到优化器"利用"UB 之后，本章后面每句话你都会读得很认真。
- `shift_oob(1u, 33u)`：**-O0 是 2，-O2 是 0**：真正当场变脸的是移位。同一行代码，两档答案，哪个都不算"错"，因为 UB 没有标准答案。

小节结构

小节	内容	四件套
UB 的优化逻辑	编译器为什么"有权使坏"；INT_MAX 案例全程复盘	代码分析
移位雷区	`1<<33`、负数移位、移位超过位宽；寄存器掩码的安全写法	代码分析
严格别名	float 与 uint32 互看的正确姿势（memcpy/union 之争）	代码分析
MISRA 十条	精选取舍：类型转换、运算符优先级括号化、禁递归等	配置
检查工具链	-Wall -Wextra -Wconversion -fstack-usage；静态分析简介	库解析

动画：编译器如何利用 UB 做优化

源代码 → 编译器推理（"你保证没有 UB"）→ 汇编折叠为常量 → 运行时行为与数学期望偏离。盯住"编译器假设"那一步，它是整个优化链的起点。

UB 编译器优化陷阱

一、UB 的优化逻辑：编译器凭什么"使坏"

C 标准把程序行为分成三类：**定义良好的**、**实现定义的**（编译器得选一个并写进文档）、**未指定的**（随便哪个都行但不许炸）。UB 是第四类：**标准直接撒手**：行为可以是任何东西，包括"看起来正常工作"。这不是标准的疏忽，是刻意的交易：把某些情形划成"不许发生"，编译器才能拿着这条禁令当优化燃料。

`is_bigger_after_inc`（probe.c 第 26 行）就是教科书案例：

```
int is_bigger_after_inc(int x) { return x + 1 > x; }   // 有符号溢出：x=INT_MAX 时 UB
```

数学上 `x+1 > x` 只有在 `x = INT_MAX` 时为假，而那一刻 `x+1` 已经溢出，是 UB。编译器的推理链很短：**你保证不喂我 INT_MAX，那这个比较永远为真，函数可以整段折叠成 `return 1`**。ARM 交叉汇编（arm-none-eabi-gcc 15.2.1，`-mcpu=cortex-m4 -O2`，probe.sh 第 5 步取证）：

```
is_bigger_after_inc:
    movs    r0, #1        @ 整个函数只剩"装载常量 1"
    bx  lr
```

两行。没有加法、没有比较，它们被"你不会溢出"这条假设吃掉了。对照 `-fwrapv -O2`：这个编译旗把有符号溢出**重新定义**为按补码回绕，编译器失去"假设权"，只能真算：

```
is_bigger_after_inc:
    mvn r3, #-2147483648   @ r3 = ~0x80000000 = 0x7FFFFFFF（INT_MAX）
    subs    r0, r0, r3          @ x - INT_MAX：x==INT_MAX 时 Z 置位
    it  ne
    movne   r0, #1              @ x ≠ INT_MAX → 1；x == INT_MAX → 0
    bx  lr
```

gcc 的推理：既然回绕定义下 `x+1 > x` 等价于"x 不等于 INT_MAX"，那就用一次减法+条件置位实现。

> 换一把编译器，这段的形状会变、结论不变。本机另一把 arm-none-eabi-gcc 10.3.1（PATH 里默认那把）给的是 `adds r3, r0, #1` / `cmp r3, r0` / `ite le` / `movle r0, #0` / `movgt r0, #1`，老老实实加一次、比一次。**两把编译器只是"真算"的方式不同，都不肯把函数折成常量**，这才是 `-fwrapv` 要证明的事。这也是本章反复强调的一条纪律：引用汇编必须注明工具链版本，`probe.sh` 第 5 步之所以先把版本号打出来。

宿主实测三档读数（gcc 16.1.0）：

编译档	is_bigger_after_inc(INT_MAX)	解释
`-O0`	**1**	老梗"-O0 返回 0"已在 gcc 16 翻案：折叠在 -O0 也发生
`-O2`	**1**	恒真折叠，两行汇编
`-O2 -fwrapv`	**0**	溢出被定义为回绕 → INT_MAX+1 = INT_MIN，比较为假

这张表比任何说教都狠：**同一个二进制符号，三种编译配置，两种答案**。"昨天还好好的"这种灵异故事，第一嫌疑人从来不是硬件，是优化等级。

"-O0 也折叠"不是嘴上说说，把宿主汇编拉出来（probe.sh 第 6 步，gcc 16.1.0 x86-64）：

```
is_bigger_after_inc:              @ -O0
    pushq   %rbp
    movq    %rsp, %rbp
    movl    %ecx, 16(%rbp)
    movl    $1, %eax               ← 加法、比较都不在，只剩装载常量
    popq    %rbp
    ret
```

`-O0` 的 `main` 仍然规规矩矩 `call is_bigger_after_inc`，可函数体里已经没人算账了。`-O2` 更狠：`main` 里 `is_bigger_after_inc` / `shift_oob` / 两种位读法**四个 call 全消失**，喂给 printf 的直接是立即数：

```
    movl    $1, %edx               @ is_bigger_after_inc(INT_MAX) → 恒真折叠
    xorl    %edx, %edx             @ shift_oob(1u, 33u) → 0（UB 之后编译器爱给什么给什么）
    movl    $1065353216, %edx      @ bits_union(1.0f)   → 0x3F800000
    movl    $1065353216, %edx      @ bits_memcpy(1.0f)  → 同一个立即数
```

而 `-O0` 的 `shift_oob` 函数体里，躺着一条真真切切的 `sall %cl, %edx`，计数 33 被移位器截成 1，所以读数是 2。**结论：`-O0` 不等于"逐行照实执行"，它只是不在调用点做折叠、不在函数间做传播；同一个函数体内的模式匹配，`gcc 16` 在 `-O0` 照做。** 这就是"我 -O0 跑过了"永远不能当正确性证据的原因。

一个容易搞反的推论：UB 最可怕的形态不是崩溃，是**悄悄改变逻辑**。`movs r0, #1` 让程序"正常跑着"，只是从此这个函数对 INT_MAX 撒谎，而你可能在它上面盖了三层缓存逻辑。回到 C3 volatile 的主题：编译器不是"老实人"，它是"规则怪兽"：规则内它可以把你的代码改得亲妈都不认识，规则外（UB）它连改都不用跟你商量。

UB 的另一面是 HardFault：数组越界、空指针解引用、未对齐访问，这些 UB 在 ARM 上会直接触发 S16 HardFault。用 `-fsanitize=undefined`（主机侧）或 `-fstack-protector`（嵌入式侧）提前抓人，比等 HardFault 再查效率高十倍。

二、移位雷区：最便宜也最频繁的一刀

移位是嵌入式使用频率最高的运算（每一个寄存器掩码都是它），也是 UB 密度最高的运算。C 标准划了三条红线：

1. **移位计数 ≥ 操作数位宽**：UB，与操作数有无符号**无关**：`1u << 33` 同样炸；
2. **移位计数为负**：UB；
3. **左移把 1 移进/移出有符号数的符号位**：UB（如 `(int)1 << 31`）。

probe.c 第 29 行把计数走运行期变量（编译期拦不住），实测 `1u << 33`：

```
----- -O0 -----          shift_oob(1u, 33u) = 2
----- -O2 -----          shift_oob(1u, 33u) = 0
```

-O0 为什么是 2？因为真机指令就在那儿跑：x86 的 `shl` 与 Cortex-M 的 `lsl` 都把计数截到操作数位宽的低 5 位（32 位操作数 → 计数 & 31），`33 & 31 = 1`，于是 `1u << 1 = 2`。-O2 为什么是 0？编译器认出 UB 后，结果可以"任意"，它选了个最省指令的答案。**两个答案都不是"错"，因为 UB 题目本身没有标准答案。**

这就是 C6 栈帧实验里 BSRR 掩码写成 `1UL << (LED_R_PIN+16)` 的原因，三条纪律层层设防：

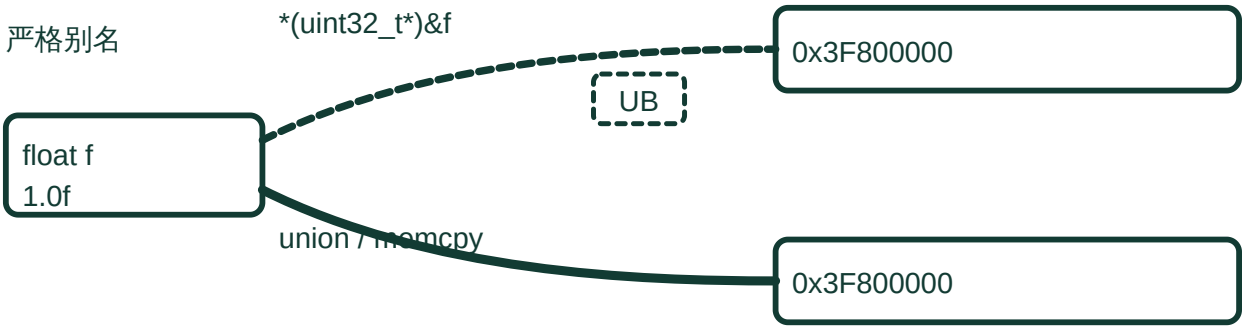
- **字面量类型先行**：`1` 是 int（32 位有符号），`1 << 31` 踩红线 3，`1 << 33` 踩红线 1；写 `1UL`（64 位宿主上仍是 32 位无符号字面量，但无符号左移不存在符号位问题）只防住红线 3，**防不住红线 1**。
- **计数必须可证 < 位宽**：`LED_R_PIN` 是枚举常量（编译期可见），`+16` 后是 22/23/24 < 32，两条红线都躲开。若计数来自运行期数据（协议字节、寄存器值），先检查再移：

```
if (n < 32u) {
    mask = 1u << n;          /* 计数已证 < 位宽：安全 */
} else {
    return false;            /* 拒绝，别让硬件替你决定 */
}
```

- **负数禁止入场**：`int8_t` 的计数先转 `unsigned` 再检查范围，有符号计数在"检查前"就已经踩了红线 2。

顺带一句 `-fwrapv` 的边界（实测）：它只救有符号加/减/乘的溢出，**移位三条红线一条都不救**（probe 第 2 步：`-fwrapv -O2` 下 `1u<<33` 依旧是 0）。别指望一个编译旗包打天下。

三、严格别名：float 与 uint32 互看的正确姿势



严格别名：指针强转是 UB，union/memcpy 才是合法写法

C 规定：访问对象必须用**兼容类型**的左值表达式，少数例外（`char*` 谁都能指、加过限定符的同类型、union 自己的成员）。拿 `uint32_t*` 去解引用一个 float 对象，就是"严格别名违规"，UB 三兄弟里最隐蔽的一个，因为**它十年都不一定发作**。

编译器为什么在意：别名分析是优化的地基。如果编译器敢假设"这个 uint32_t 写入不会改到那个 float"，它就能把两次 float 读合并成一次、把 store 重排到 load 前面。你的违规代码就是在这条假设上打洞。抓捕现场（probe.sh 第 3 步，alias-violate.c 第 13 行）：

```
warning: dereferencing type-punned pointer will break strict-aliasing rules [-Wstrict-aliasing]
   13 |     return *(uint32_t *)&f;
```

两种**合法**写法（probe.c 第 32–44 行，实测输出）：

```
/* 读法一：union 双关——C11 起脚注给了正式说法（写一个成员、读另一个是"指定位模式重解释"） */
static uint32_t bits_union(float f)
{
    union { float f; uint32_t u; } pun;
    pun.u = 0u;
    pun.f = f;
    return pun.u;
}

/* 读法二：memcpy——标准层面永远合法，编译器会把它优化成一条 load */
static uint32_t bits_memcpy(float f)
{
    uint32_t u;
    memcpy(&u, &f, sizeof u);
    return u;
}
```

```
bits_union(1.0f)   = 0x3F800000
bits_memcpy(1.0f)  = 0x3F800000
```

两种读法位级一致（断言 2/2 之一）。选型建议很朴素：

- **跨类型取位**（float→位串、CRC 算法吃结构体字节）：**memcpy**：零心智负担，"看似函数调用"会被优化器消掉，生成的就是一条访存指令；
- **既取位又想省一次拷贝**：union 双关：嵌入式老代码的传统写法，C11 已转正，但记得 C4 的提醒：它依赖"同存储、同对齐"这一层实现约定，跨平台代码仍以 memcpy 为锚；
- **字节序敏感**（网络协议、EEPROM 布局）：走 `unsigned char*` 逐字节拼（alias-violate.c 第 18 行的对比组），char 是别名规则的天生例外，且逐字节代码读得出字节序。

什么情况**不需要**双关：调 S9 ADC 时把半字数据当 int 看，那不叫别名违规，叫"本来就该用正确的类型读正确的地址"。别名规则管的是**同一块存储的两种身份**，不是"我 cast 了一个外设寄存器指针"。外设寄存器访问走 volatile 指针（回 C3），那是另一个学科。

四、嵌入式 MISRA 十条：自选的"红线地图"

MISRA-C:2012 全本 300+ 条，汽车/医疗行业全量合规。取信条的思路是：把历史上**真实炸过**的 UB 高发区画成红线，剩下的按团队成本自选。我们的自选十条（编号给到 MISRA-C:2012 中有明确对应、且值得背号的）：

1. **声明即初始化**（Rule 8.9 的精神 + -Wuninitialized 抓现行）：`int x;` 后面读 = UB；写代码时变量定义挪到能初始化的位置。
2. **隐式转换显式化**（Rule 10.x 类型模型）：`uint16_t` 与 `int` 混算、有符号无符号比较，一律显式 cast 或统一类型，C4 的整型提升坑在这条线上。
3. **移位计数先验证**：上节的三条红线；掩码宏的参数必须"编译期可证 < 位宽"或运行期先查。
4. **位运算与比较不裸混**：`flags & MASK == MASK` 解析成 `flags & (MASK==MASK)`，`==` 优先级高于 `&`，加括号不丢人（Rule 12.x 的精神）。
5. **禁递归**（Rule 17.2）：裸机栈按 C6 算过账：每层一帧，没有哨兵，爆栈不报错。树形逻辑用显式栈迭代。
6. **指针运算只在数组圈内**（Rule 18.1 的精神）：指向"尾后一"可以、解引用"尾后一"是 UB；跨对象做指针差是 UB。
7. **禁 goto 直穿初始化**：向上 goto 跳过变量初始化 = 读未初始化 = UB；错误处理出口统一（单出口风格），跳转只向下。
8. **switch 必有 default**（Rule 16.x 的精神）：枚举 switch 加齐 default 并 abort/log，新增枚举值时固件"炸得响"比"悄悄漏"好。
9. **volatile 纪律**：外设寄存器、ISR 与主循环共享的变量一律 volatile（C3 全章）；但 volatile 不解决原子性，那是 S4 NVIC 的活。
10. **无副作用依赖**：`&&` / `||` 右侧、`if` 条件、`sizeof` 里的表达式，其副作用不许是程序依赖的一部分：表达式既要求值又要求副作用，重排/短路就会改变行为。

十条的共同句式：**把"编译器可以任意解释"的地带，用写法约束收窄成"只有一种读法"**。这正是 MISRA 的哲学：语言的自由度是给写编译器的人的，不是给写固件的人的。

五、检查工具链：三道防线自动化

第一道防线是**编译警告**，零成本、可自动执行。抓捕现场（probe.sh 第 3 步真实输出）：

```
oob-static.c:19:14: warning: 'x' is used uninitialized [-Wuninitialized]
oob-static.c:12:10: warning: array subscript 7 is above array bounds of 'int[4]' [-Warray-bounds=]
alias-violate.c:13:13: warning: dereferencing type-punned pointer will break strict-aliasing rules [-Wstrict-aliasing]
```

三个 UB 三兄弟的"编内成员"，三个警告各就各位。军备建议按档加：

- **起步价**：`-Wall -Wextra`，配 `-Werror` 让 CI 拦人（本站所有工程的标准旗）；
- **加一档**：`-Wconversion -Wsign-conversion`：隐式收窄/符号转换全部点名，前三章所有坑这条旗都能提前吹哨；
- **专项**：`-Wstrict-aliasing`（别名）、`-Warray-bounds`（越界）、`-Wuninitialized`（未初始化）、`-fstack-usage`（每函数栈账单：C6 的栈预算从猜测变核对，喂给 B5 map 与体积 的体积审计）。

第二道防线是**运行期仪表**：`-fsanitize=undefined`（UBSan）在真机上给 UB 点名。诚实记录：本站宿主工具链（MinGW-Builds gcc 16.1.0）没有带 UBSan 运行库，probe.sh 第 4 步自动跳过（"编译期防线不受影响"）；有 Linux 环境时同一段 ub.c 会打出 `runtime error: signed integer overflow`。交叉链上 UBSan 支持有限，所以嵌入式的主战场是第一、三道防线。

第三道防线是**静态分析**：cppcheck（免费、快、对 MISRA 子集有专用检查）、clang-tidy、gcc 自带 `-fanalyzer`。它们与 -Wall 的分工：-Wall 看"这一行"，静态分析看"这条数据流"：未初始化的传播链、空指针的所有分支路径。

三道防线的成本-收益排序很清晰：警告 0 成本、CI 0 成本、静态分析一杯咖啡、UBSan 一台 Linux。**先让前两道变成默认**，你的 UB 故事会少 90%。

附录：工程完整源码

本章取证工程 `code/c/07-ub-misra/`（`sh probe.sh` 一条命令跑完，不需要开发板）：

probe.c（运行期取证：溢出/移位/两种合法位读法）：

<<< ../../code/c/07-ub-misra/probe.c

alias-violate.c / oob-static.c（专供编译器警告抓人的违规现场）：

<<< ../../code/c/07-ub-misra/alias-violate.c

<<< ../../code/c/07-ub-misra/oob-static.c

ub.c（UBSan 点名用例，工具链支持时启用）：

<<< ../../code/c/07-ub-misra/ub.c

probe.sh（三档对照 + 防线现场 + ARM 交叉汇编取证）：

<<< ../../code/c/07-ub-misra/probe.sh

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **UB 三兄弟：溢出、移位、别名，今天跑通不等于明天安全；加括号不丢人，显式转换不寒碜。**

实物实验

- 把 00-blink 的 `1UL << (LED_R_PIN+16)` 故意写成 `1 << (LED_R_PIN+16)` 再改成 `1 << 33` 对比反汇编，体会"1 的类型和移位数"为什么是掩码代码的生命线（我们的写法用 `1UL`）。
- 进阶：把 `-O0` 与 `-O2` 两版固件都烧进板子，看移位越界版本在两种固件里**行为不一致**，板子会诚实地告诉你 `-fstack-usage` 之外，优化等级也是一种"硬件配置"。

常见坑

- **以为警告=错误**：`-Wall` 只是起步价；嵌入式建议 `-Wall -Wextra -Wconversion`，警告清零。
- **`char` 当数值用忘定符号**：`char` 默认有无符号是实现定义，寄存器/协议字节一律 `uint8_t`。
- **联合体类型双关**：C11 允许 union 读别的成员但有尾洞/表示风险；跨类型 reinterpret 首选 `memcpy`（编译器会优化成一条 load）。
- **递归**：MISRA 禁递归不是教条，裸机栈就那么点，爆栈无哨兵（回 C6）。
- **把"-O0 能跑"当"代码正确"**：gcc 16 连 -O0 都开始折叠 `x+1>x`；证明正确性的从来不是"跑起来过"，是"没有 UB + 警告清零"。

短自测

1. `is_bigger_after_inc(INT_MAX)` 在本站宿主 gcc 16.1.0 的 -O0、-O2、`-O2 -fwrapv` 三档下读数各是多少？为什么前两档相同？

> **▼ 看答案**
> 1、1、0（probe.sh 第 1、2 步实测）。前两档相同是因为折叠 `x+1>x → 1` 的模式匹配在 -O0 也执行（gcc 16 行为，老教程"-O0 返回 0"已翻案；probe.sh 第 6 步直接把 `-O0` 的函数体拉出来，里面只剩一句 `movl $1, %eax`）；`-fwrapv` 把有符号溢出定义为回绕，编译器失去"假定不溢出"的权利，只能真算：INT_MAX+1 回绕成 INT_MIN，比较为假，读数 0。

2. `1u << 33` 为什么在 -O0 下等于 2？-O2 下等于 0 说明什么？

> **▼ 看答案**
> -O0 走真机指令：x86 `shl`/Cortex-M `lsl` 都把移位计数截到低 5 位，`33 & 31 = 1`，`1u << 1 = 2`（probe.sh 第 6 步：`-O0` 的 `shift_oob` 函数体里确实躺着一条 `sall %cl, %edx`）。-O2 等于 0 说明编译器认出了 UB（计数 ≥ 位宽，无符号也一样）并选择了"任意结果"里最省指令的那个：第 6 步里 `-O2` 的 `main` 连 `call shift_oob` 都没有了，直接 `xorl %edx, %edx` 把 0 塞给 printf。两个答案都不算错，UB 题目没有标准答案，这正是要避开它的原因。

3. 想看 float 的位模式，写出两种合法写法并说出它们的法理依据；`*(uint32_t*)&f` 错在哪？

> **▼ 看答案**
> 合法一：union 双关（C11 脚注对"写一成员读另一成员"给出位模式重解释的正式说法）；合法二：`memcpy` 到 uint32_t（对象表示复制，永远合法，优化后就是一条 load）。`*(uint32_t*)&f` 用不兼容类型的左值访问 float 对象，违反严格别名规则，编译器可基于"这类访问不存在"的假设做 load 合并与 store 重排，行为无保证。

4. `-fwrapv` 能救 `1u << 33` 吗？依据是什么？

> **▼ 看答案**
> 不能。`-fwrapv` 只把**有符号加/减/乘**的溢出定义为按补码回绕；移位的三条红线（计数 ≥ 位宽、计数为负、有符号左移穿符号位）一条都不在其保护范围（probe 实测：`-fwrapv -O2` 下 `1u<<33` 读数仍为 0）。移位安全的唯一途径是计数先验证。

5. 背出"嵌入式 MISRA 十条"中与你最近一次 bug 最相关的一条，并说出它的 UB/缺陷机理。

> **▼ 看答案**
> （开放题，示例）第 4 条"位运算与比较不裸混"：`flags & MASK == MASK` 因 `==` 优先级更高，实际解析为 `flags & (MASK == MASK)`，即 `flags & 1`，逻辑悄悄变成"最低位为 1"，编译器不警告（表达式合法），测试可能恰好全过。机理是 C 运算符优先级表把 `&` 排在 `==` 之后，与人类阅读直觉相反；红线写法就是给位运算加括号。

对照表：本章概念 → 仓库落点

概念	仓库落点
溢出折叠 `movs r0,#1`	`code/c/07-ub-misra/probe.sh` 第 5 步（build/probe-arm.s）
-fwrapv 真算汇编（x≠INT_MAX 变换）	probe.sh 第 5 步对照段（build/probe-arm-wrapv.s）
换一把编译器的另一形状（ `adds` / `cmp` / `ite` ）	`XPACK=/none sh probe.sh` （退回 PATH 默认那把 10.3.1）
-O0 宿主函数体已折叠成 `movl $1, %eax`	probe.sh 第 6 步（build/host-o0.s）
-O2 `main` 四个 call 全消失、printf 吃立即数	probe.sh 第 6 步（build/host-o2.s）
三档读数表（1/1/0）	probe.sh 第 1、2 步实测输出
移位越界两档变脸（2/0）	probe.sh 第 1 步； `shift_oob` 见 probe.c 29 行
合法位读法 + 位级断言	probe.c 32–44、59–66 行（bits_union/bits_memcpy）
别名违规抓捕现场	alias-violate.c 13 行；probe.sh 第 3 步
越界/未初始化警告现场	oob-static.c 12、19 行；probe.sh 第 3 步
UBSan 用例与跳过逻辑	ub.c 全文；probe.sh 第 4 步
掩码纪律 `1UL << (LED_R_PIN+16)`	code/stm32/00-blink 工程与 S3

延伸阅读

本章只讲了嵌入式最常撞的几条；完整清单与"UB 真会咬人"的证据在这里：

- **[C1]** N1570 附录 J.2 — C11 全部未定义行为的官方清单，两百余条。
- **[D8]** Wang et al. 2013 — 优化器利用 UB 删掉空指针检查、溢出检查的真实案例集（Linux、PostgreSQL……）。
- **[C2]** MISRA C:2012 / C:2023 — 本章规则取舍的原文。

你做到了

- 拿到一张"UB 雷区地图"和一套编译器防线；
- 看老代码时眼里多了三兄弟的探测器。

回到开场那颗「客户演示那天才炸」的定时炸弹：你现在知道引信在哪了——不是硬件不稳，是编译器拿「你保证没有 UB」当优化燃料。三兄弟过一遍、防线拉起来，下次「昨天还好好的」再出灵异故障，第一嫌疑人换成优化等级，而不是示波器。

✅ C 篇收官。下一站：B 篇 构建与运行：看你写的 C 如何一步步变成固件，全程开盒。

通往单片机之路第 09 章

C8 串口协议：从字节流到可恢复的消息

难度 ★★★☆☆ · 约 55 分钟

本章罗盘

问题

UART 只保证字节按序到，丢一个字节之后，解析器凭什么还认得出下一帧、不把两段残帧拼成一段？

交付

一笔 seq=7 INFO 请求的全手算：6 字节 raw→CRC 0xB0CC→COBS 编码 10 字节线上帧，与 Python 独立实现逐字节核对；一张三态八计数的状态转移表。

边界

CRC 检得出随机损坏、防不了篡改（要带密钥的 MAC）；序号只用于配对响应，不是安全凭据；COBS 开销恒 1 字节的前提是帧 ≤254 字节，74 字节上限不能套到更大帧。

依据锚点 d13 · mcu-code · mcu-anim

> 🎯 UART 只保证"字节按顺序到"，从不告诉你一条命令在哪结束。丢一个字节之后，下一帧还认得出来吗？

本章精髓

1. **帧边界要协议自己画**：UART_DATA 事件、IDLE 中断、一次 `read()` 拿到多少字节，都不是一条命令的边界。COBS 把数据里的 0 全部换掉，让 0x00 只当帧尾，丢字节后等到下一个 0 就能重新对齐。
2. **每道检查只管一件事**：CRC 管随机损坏，管不了篡改；序号只用来配对响应，不是安全凭据；所有检查都通过之前，不往调用方的输出里写一个字节。
3. **失败要有出口和计数**：超时、超长、已知丢字节一律进 DISCARD，八种结局各自计数；发送队列满了丢新帧并计数，确认帧发不出去就不执行命令。"出错后仍知道发生了什么"靠的就是这些计数。

- **能理解**：EMPTY 态收到 0 为什么什么都不做；transport loss 之后为什么第一帧可能被牺牲；为什么"确认帧入队失败"等于"命令不执行"。
- **能用**：主机上用分块、粘包、错误流跑生产 codec；用 `device-console.py` 在两块板上查 INFO/STATUS。

先修

- 必需：C5 函数指针与状态机（状态机、环形缓冲）、S7 USART（波特率、IDLE+DMA）。
- 建议：S8 DMA、P5 UART 驱动解析。

先跑起来（10 分钟 quick win）

在仓库根执行：

```
sh code/common/reliability/probe.sh
```

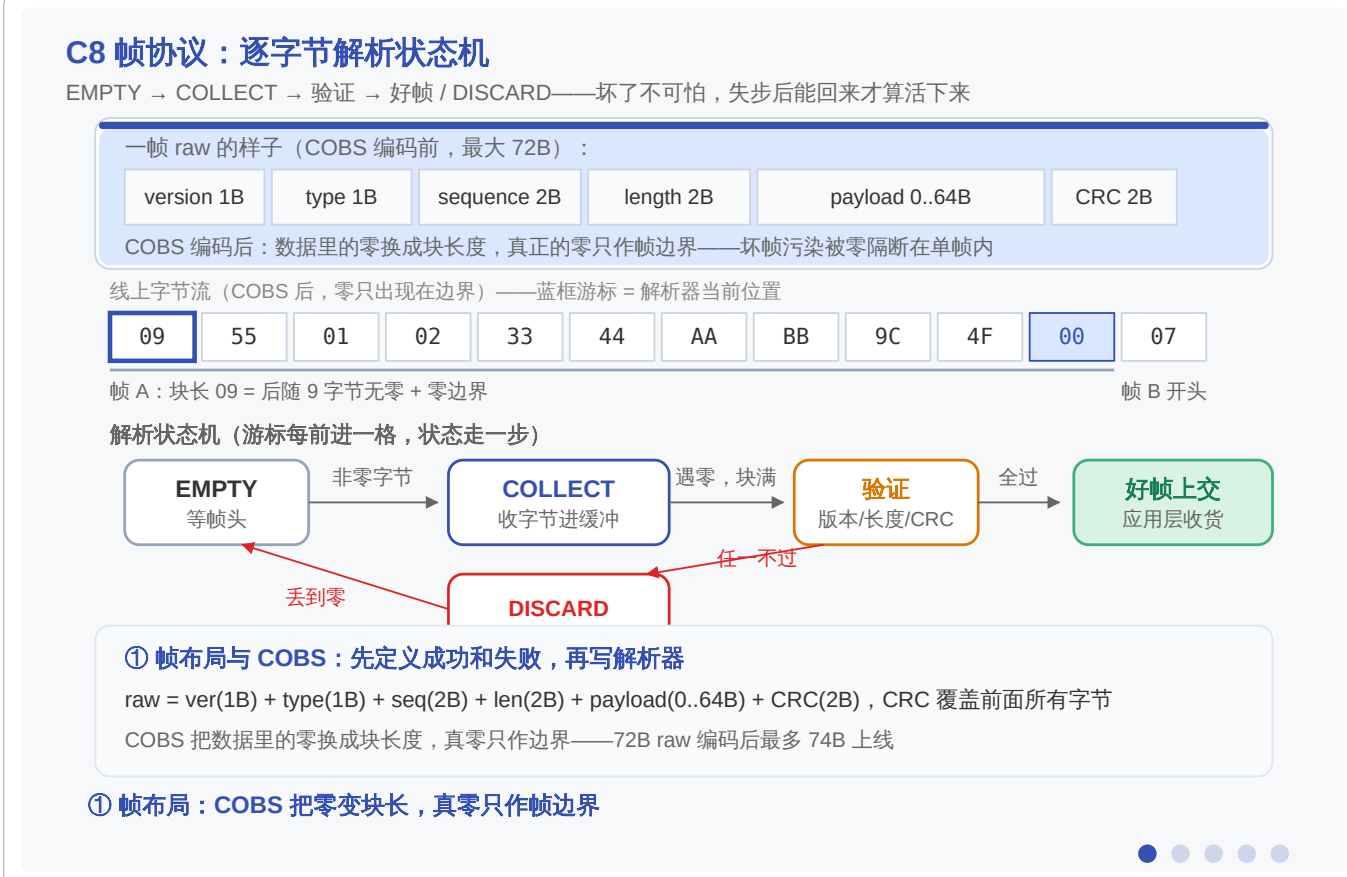
跑之前先押开场那一问：往字节流中间丢一个字节，接收端是从此再也对不上，还是能重新找回边界？若能找回，第一帧是完好无损，还是先被牺牲？答案就藏在下面的断言里。

脚本用 `-std=c11 -Wall -Wextra -Werror` 编译生产用的 codec/health/transport/service，跑 `tests.c` 的边界断言；再编一个 `codec-tool`，交给 `scripts/device-console.py self-test` 和独立的 Python 实现互相编解码。在 Linux 上还会加跑一遍 ASan/UBSan。看到下面两行就算通过，不需要串口库和开发板：

```
protocol, health, TX/RX backpressure, service and reset-record tests passed
self-test passed: codec vectors + session matching + sync/retry + C cross-validation
```

动画：逐字节解析状态机

字节流如何变成可恢复的消息：EMPTY → COLLECT → 验证 → DISCARD 的每一步，游标沿字节流连续前进、状态随之切换，坏帧路径与重新同步一图看完。



C8 帧协议：逐字节解析状态机与重新同步

小节结构

小节	内容	四件套
一、为什么要定界	长度前缀 / 转义 / COBS 三种定界法的失步代价	配置
二、帧布局与容量	6+N+2 字节逐字段、74 字节上限从哪来	配置
三、CRC-16/CCITT-FALSE	四个参数、逐位实现、独立实现交叉验证	库解析
四、COBS 手算一帧	seq=7 的 INFO 请求逐字节编码，解码的四个拒绝条件	代码分析
五、接收状态机	三态、八计数、超时回绕、丢失后的第一帧	代码分析
六、两板的字节来源	F407 DMA 环形 + 字节队列；S3 驱动环形 + 周期取数	引脚
七、发送与背压	4 槽 TX、部分写、100ms 放弃与补同步零	代码分析
八、服务层与主机	响应配对、INFO/STATUS 布局、确认先于执行	库解析

一、为什么要定界：一次 read 不是一条命令

UART 硬件只管一个字节：起始位、8 个数据位、停止位。它不知道哪几个字节组成一条命令。驱动层给的 UART_DATA 事件、IDLE 中断、`read()` 返回的长度，都只说明"此刻手里有这些字节"。主机一次写出的命令可能分三次到，三条命令也可能一次全到。S7 的波形和 P5 的驱动源码都演示过这一点。

所以边界必须由协议写进字节流本身。常见的三种做法：

定界法	做法	丢一个字节之后
长度前缀	帧头写"后面还有 N 字节"	长度字段本身错位，之后每一帧都按错的长度切，一路错下去；得再加魔数和超时才能找回同步
转义（SLIP/HDLC）	保留一个帧标志字节，数据里出现它就转义成两字节	等到下一个帧标志就重新对齐；但最坏情况数据长度翻倍
COBS	把数据里的所有 0 换掉，0x00 只当帧尾	等到下一个 0 就重新对齐；开销固定，每 254 字节最多多 1 字节

本协议的 raw 最长 72 字节，小于 254，COBS 开销恒为 1 字节，缓冲区大小可以在编译期写死并用 `_Static_assert` 锁住。选 COBS 的理由就是这两条：能重新同步，最坏开销可以提前算进预算。

二、帧布局与容量

raw 帧共 8+N 字节，多字节字段都是小端：

偏移	字段	长度	含义
0	version	1	固定为 1；解码见到别的值就报 VERSION，给将来改格式留余地
1	type	1	最高位 0=请求、1=响应（请求类型 | 0x80）；设备收到响应型帧直接忽略，两边不会互相回声
2	sequence	2	主机递增，设备原样带回，主机靠它配对响应
4	length	2	负载长度 0~64
6	payload	N	负载
6+N	crc	2	CRC-16/CCITT-FALSE，覆盖偏移 0 到 5+N 的全部字节

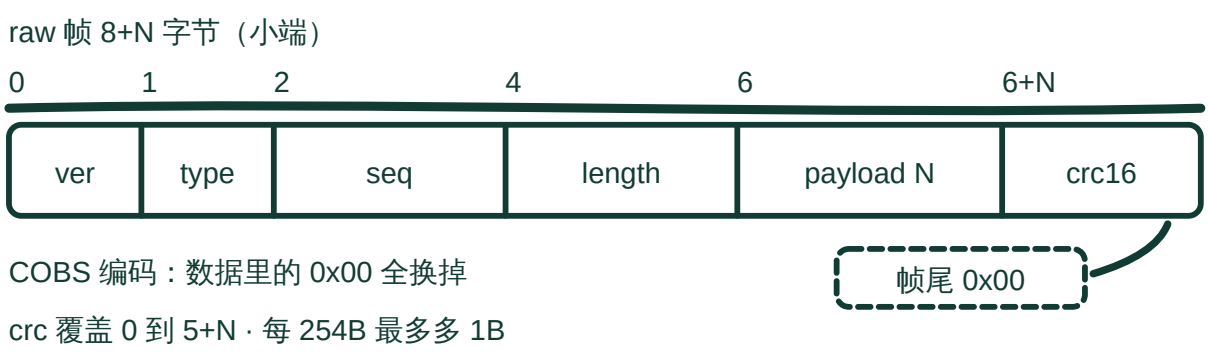
解码时要求 COBS 还原出的字节数**恰好等于** length+8，多一个少一个都报 LENGTH。容量一路推下来：raw 最大 6+64+2=72，COBS 后 73，加帧尾 0x00 共 74。三个数在 `protocol.h` 里是 `PROTOCOL_RAW_MAX / ENCODED_MAX / WIRE_MAX`，`protocol.c` 开头的两条 `_Static_assert` 把它们和布局绑在一起。

响应的状态字节也占负载容量。INFO、STATUS 的 payload[0] 就是状态码，不能把它当成 64 字节之外的"额外一字节"。

多字节字段一律显式移位读写，不把 `frame_t` 直接 `memcpy` 上线：

```
uint16_t read_le16(const uint8_t *p) { return (uint16_t)((uint16_t)p[0] | (uint16_t)p[1]<<8); }
void write_le16(uint8_t *p, uint16_t v) { p[0]=(uint8_t)v; p[1]=(uint8_t)(v>>8); }
```

结构体的填充和端序由编译器和 ABI 决定（C4）。直接发结构体内存，等于让编译器替你定了一份没写在任何地方的协议。



raw 帧布局：version、type、sequence、length、payload、CRC-16 共 8+N 字节，COBS 编码后以 0x00 收尾；每 254 字节最多多 1 字节开销

三、CRC-16/CCITT-FALSE：先把名字叫全

"CRC-16"不是一个算法，而是一族算法。要确定是哪一个，得给全多项式、初值、是否反转输入输出、结果是否再异或这几个参数。本协议用的是：

参数	值
多项式	0x1021
初值	0xFFFF
输入/输出反转	都不反转
结果异或	0x0000
检查值（ASCII `123456789`）	**0x29B1**

检查值就是验收标准：哪个实现对 `123456789` 算出 0x29B1，才和本协议一致。`protocol.c` 用逐位实现，没有查表，72 字节以内一帧只需要几百次循环：

```
uint16_t protocol_crc(const uint8_t *data, size_t n)
{
    uint16_t crc=0xffffU;
    for (size_t i=0;i<n;i++) {
        crc ^= (uint16_t)((uint16_t)data[i]<<8);          /* 新字节对齐到高 8 位 */
        for (unsigned bit=0;bit<8;bit++)
            crc=(uint16_t)((crc&0x8000U) ? ((uint32_t)crc<<1)^0x1021U : (uint32_t)crc<<1);
    }
    return crc;
}
```

主机侧的 `device-console.py` 没有照抄这段 C，而是用 Python 标准库的 `binascii.crc_hqx(data, 0xFFFF)`。这是一份参数相同、但独立写成的实现，两边互相校验，才能排除"同一个 bug 写了两遍"。`tests.c` 和 `self-test` 都先断言检查值 0x29B1。

CRC 的边界要说清：

- 它能检出随机损坏（单比特错、突发错），**防不住有意篡改**：改了数据的人顺手重算一遍 CRC 就行。要防篡改得用带密钥的 MAC。
- 它覆盖的是 COBS **之前**的 raw 字节，所以接收端的顺序是先 COBS 解码、再验 CRC。

四、COBS 手算一帧

COBS 的规则：把数据按 0 切成若干段，每段前面放一个"段长+1"的码字，原来的 0 不再出现在输出里。

以主机发出的第一条请求为例：INFO（type=0x01），seq=7，无负载。

第 1 步，拼 raw。 `01 01 07 00 00 00`，依次是 version、type、seq 低字节、seq 高字节、length 低字节、length 高字节。

第 2 步，算 CRC。 对这 6 字节算出 0xB0CC，小端追加，raw 变成 8 字节：

```
01 01 07 00 00 00 CC B0
```

第 3 步，按 0 切段，加码字。

段	内容	码字 = 段长+1
1	`01 01 07`	04
2	（空，两个 0 之间）	01
3	（空）	01
4	`CC B0`（帧末）	03

第 4 步，拼起来，加帧尾。

```
04 01 01 07 01 01 03 CC B0 00
```

10 字节 = 8 raw + 1 COBS 开销 + 1 帧尾，和第二节的公式一致。可以在仓库根用 Python 复核：

```
python3 -c "import importlib.util as u;s=u.spec_from_file_location('d','scripts/device-console.py');d=u.module_from_spec(
# 04 01 01 07 01 01 03 cc b0 00
```

解码倒过来走：读码字 c，原样拷贝后面 c-1 字节；c 不等于 255 且后面还有数据时，补一个 0。`protocol_cobs_decode` 有四个拒绝条件，任何一个成立就返回 0：

```
if (!code || (size_t)(code-1)>n-in) return 0;   /* 码字为 0，或声称的段长超过剩余字节 */
...
if (!src[in] || out==capacity) return 0;        /* 段内出现 0，或输出已满 */
```

段内出现 0 说明这一段被截断或拼接过，长度对不上说明有字节丢了。两种情况都不能"尽量解出一点"。

`protocol_decode` 在 COBS 之后依次检查：字节数 ≥8 → version==1 → length≤64 且字节数恰为 length+8 → CRC。全部通过才写 `frame_t`：

```
/* Publish only after every check. A rejected frame leaves the caller's output untouched. */
f->type=raw[1]; f->sequence=read_le16(raw+2); f->length=length;
```

`tests.c` 专门验证了这一点：先放一个 `type=99` 的哨兵帧，喂进 VERSION/LENGTH/CRC 三种坏帧，断言哨兵原封不动。

五、接收状态机：三个状态，八个计数

`protocol_rx_feed` 每次吃一个字节。状态转移如下：

当前状态	收到	动作	计数
EMPTY	0x00	什么都不做	—
EMPTY / COLLECT	非零，缓冲未满	存入缓冲，记下 `last_ms`，进 COLLECT	—
COLLECT	非零，缓冲已满 73 字节	清空，进 DISCARD	oversize
COLLECT	0x00	调 `protocol_decode`，回 EMPTY	ok / cobs / version / length / crc 之一
COLLECT	距 `last_ms` ≥100ms 后的任意字节	先超时：清空，进 DISCARD	timeout
DISCARD	非零	吞掉	—
DISCARD	0x00	回 EMPTY	—
任意	传输层报丢失	清空，进 DISCARD	transport_loss

几处值得逐行看的细节：

- **EMPTY 收 0 不计数是有意的。** 主机重试前会多发一个 0x00 做同步。如果它落在 EMPTY 态，只是个空操作，不会污染错误统计。
- **超时用无符号减法判断**：`(uint32_t)(now-rx->last_ms)>=rx->timeout_ms`。毫秒计数回绕时差值仍然正确。`tests.c` 从 `UINT32_MAX-50` 起跑，分别验证间隔 99、100、101ms：99 能收到完整帧，100 和 101 计一次 timeout；紧接着的下一帧仍能收到，因为超时帧自己的帧尾 0 把 DISCARD 结束了。
- **超时按适配层看到字节的时间算。** F407 的一次 DMA 中断可能一下子交出几十个字节，它们共用同一个 `now`。100ms 量的是"解析器多久没收到新字节"，不是线上每个字节的物理到达时间。
- **计数器是饱和加法** `sat_add`：加到 `UINT32_MAX` 就停住，不会回绕成 0，把"错过很多次"伪装成"从没错过"。

丢失之后，第一帧为什么可能被牺牲

传输层确认丢了字节时，会调 `protocol_rx_loss`，解析器进 DISCARD，等下一个 0。如果丢掉的恰好是上一帧的帧尾 0，那么解析器遇到的"下一个 0"其实是**下一帧**的帧尾，下一帧会被整个当成残段吞掉。

所以主机侧的规矩是：info/status 有限重试，每次重试前先单独发一个 0x00。那个 0 落在 DISCARD 态，就提前结束了丢弃；落在 EMPTY 态，就是空操作。

为什么丢失必须先于新字节交付

F407 的字节队列溢出时，不是丢掉一个字节，而是把整段不确定的数据清空，并置一个 `loss` 标志。`byte_queue_pop` 先把这个标志交出去，再交新字节：

```
bool byte_queue_pop(byte_queue_t *q, uint8_t *byte, bool *loss)
{
    *loss=q->loss; q->loss=false;
    if (!q->count) return false;
    ...
}
```

通信任务看到 `loss` 就先调 `protocol_rx_loss`，再喂这个字节。顺序反了，解析器会把溢出前的半截帧和溢出后的半截帧拼成一段。拼出来的东西 CRC 大概率不对，但"大概率"不是保证。

六、两板的字节来源

同一个解析器，两块板的字节从不同的路上来。

F407：USART1 + DMA2 Stream5 环形 + 字节队列（`code/stm32/platform/uart.c`）

- PA9 TX / PA10 RX，AF7。`BRR=139`，按复位后的 HSI16 计算，16MHz/139≈115108 波特，误差约 0.08%。`f407_uart_init` 先检查 RCC_CFGR 里时钟源和三个预分频都还是复位值，不是就返回失败，不让一个错的 BRR 静默跑起来。
- DMA 把字节搬进 256 字节环形区。HT、TC、TE 中断和 USART 的 IDLE 中断都调 `receive()`，把新到的一段推进 512 字节的 `byte_queue_t`。Stream5 的标志在 HISR 的第 10/11/9 位（HT/TC/TE），不是常被误抄的 28/29/27 位；HISR 只读，清标志写 HIFCR。
- `dma_consume` 有三种判定丢失的情况：TE；HT 和 TC 同时挂着（说明服务晚了至少半圈，数据可能已被覆盖）；位置没动却来了 TC（正好被套了一整圈）。USART 的 PE/FE/NF/ORE 也走同一条丢失路径。

S3：IDF UART 驱动环形 + 周期取数（`code/esp32/06-framed-protocol`）

- UART1，GPIO10 TX / GPIO11 RX，驱动的 RX 环形区 1024 字节。
- 驱动的事件队列满了只写日志，没有公开的精确计数，所以任务不能只等 UART_DATA。它每轮最多等 5ms 事件，然后不管有没有事件，都用 `uart_get_buffered_data_len` + `uart_read_bytes` 取一次数兜底；每轮最多处理 512 字节，防止 RX 洪水把 TX 饿死。
- UART_FIFO_OVF、UART_BUFFER_FULL、帧错、校验错、BREAK 都按传输丢失处理：`uart_flush_input` 清掉驱动缓冲，`xQueueReset` 清掉积压的事件，再调 `protocol_rx_loss`。

接线：3.3V USB-TTL，TX 接板子 RX、RX 接板子 TX，GND 共地。协议口上不混入任何 printf：F407 的调试走 SWD，S3 的文本日志走默认控制台。

七、发送与背压：加大数组解决不了问题

`protocol_tx_t` 只有 4 个槽，每槽 74 字节，入队时就编码好：

- **队列满了丢新帧**，`dropped` 加一，已经排着的帧不受影响。编码失败也算丢。
- `protocol_tx_step` 每轮只推进底层 `write` **实际接受**的字节数。F407 的 `f407_uart_write` 只在 TXE=1 时写 DR，写不进就返回已写的数量；S3 用非阻塞的 `uart_tx_chars`。
- **一帧发了 100ms 还没发完，就放弃剩下的部分**，计一次丢，并置 `sync`：下一帧之前先单独发一个 0x00，把对端可能残留的半帧结束掉。`sync` 初值就是 true，所以开机后的第一件事是发一个同步 0。
- `write` 返回的数量超过请求值，说明底层违反了接口约定，同样按丢失处理，不让 `offset` 越界。

```
if ((uint32_t)(now-q->start_ms)>=100) {
    q->dropped=sat_add(q->dropped,1); tx_pop(q); q->sync=true; return;
}
```

为什么不把队列加大？加多大都会有满的时候，真正要设计的是"满了怎么办"。这里的答案是丢新帧、计数，主机靠超时重试兜底。STATUS 里的 `tx dropped` 在涨，主机就知道是设备发不出去，而不是没收到。

八、服务层与主机：确认先于执行

`device_service_feed` 每收到一个合法帧：

1. type 最高位是 1（响应帧）→ 忽略。
2. type 1/2（INFO/STATUS）→ 交给 `device_service_query` 组装响应；请求带了负载就回 BAD_PAYLOAD(2)。
3. type 0x7F（TEST_FAULT）→ 普通固件没有 fault 回调，回 UNSUPPORTED(1)；测试固件校验参数，不合法回 2。
4. 其他类型 → UNSUPPORTED(1)。

最关键的是最后一行：

```
/* No accepted command side effect if its acknowledgement cannot be queued. */
if (protocol_tx_enqueue(&s->tx,&response) && invoke)
    s->fault(s->fault_user,request.payload[0],request.payload[1]);
```

反过来写会怎样？命令先执行了，确认帧因为 TX 队列满被丢掉。主机看到的是超时，以为没执行，于是重试，同一条不可重试的命令就执行了两次。主机侧也守着这条线：info/status 最多重试两次，fault 只发一次，绝不自动重试。

两种查询响应的布局（payload[0] 都是状态码，0=OK）：

响应	长度	字段
INFO	5	[1] schema=1 · [2] 平台（1=F407，2=S3）· [3..4] 能力位：bit0 协议、bit1 健康监督、bit2 允许故障注入
STATUS	25	[1] schema · [2..5] 运行毫秒 · [6..9] 复位原因原值 · [10] 必需任务 mask · [11] 过期任务 mask · [12] 标志（bit0 宽限期、bit1 已锁存重启）· [13..16] 合法帧数 · [17..20] 接收错误总数 · [21..24] TX 丢帧数

主机脚本 `device-console.py` 一次只挂一个请求：生成 seq，在**绝对期限**内等 type|0x80 且 seq 相同的帧；收到无关帧只记日志，不延长期限。超时就先发 0x00，再重发。所有收发都写进 CSV 日志，事后可以复盘。

两板命令实操

```
# F407（ST-Link 烧录 build 产物）
cd code/stm32/05-framed-protocol && make

# S3（IDF 5.5.2 环境）
cd code/esp32/06-framed-protocol
idf.py set-target esp32s3 && idf.py build flash

# 主机
pip install -r scripts/device-console-requirements.txt
python3 scripts/device-console.py info   --port COMx
python3 scripts/device-console.py status --port COMx
```

`info` 打印 schema、平台号和能力位；`status` 先问 INFO 拿平台号，再打印运行状态、复位原因和三个计数。

附录：工程完整源码

<<< ../../code/common/reliability/protocol.c

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **0 只当帧尾，CRC 只管损坏，序号只管配对；失步就等下一个 0，丢了就计数，确认发不出去就不执行。**

实物实验

- **装备**：3.3V USB-TTL、杜邦线；逻辑分析仪选配。
- **观测点 1**：`info` 返回的平台号，F407 是 1，S3 是 2；连续执行两次 `status`，合法帧数应该增加。
- **观测点 2**：用逻辑分析仪抓 `info` 请求，线上应该恰好是第四节手算的 10 个字节 `04 01 01 07 … 00`（seq 以主机实际发出的为准）。
- **观测点 3（翻车实验）**：用串口助手往协议口发一行文本（不带 0x00），再执行 `status`。预期：文本被当成半帧，主机第一次请求可能超时，重试前补的 0 让解析器恢复，之后 STATUS 里的接收错误数加一。

实验记录

模型/编译验证不等于上板实测。当前待上板；请记录日期、板版本、固件版本、接线、输入、原始输出与结论。

日期	板卡/版本	故障输入	原始结果	结论
待实测				

常见坑

- **协议口里混了 printf**：文本没有 0x00 结尾，会和下一帧拼在一起。表现为偶发 CRC 错，越调试越多。
- **TX/RX 没交叉或没共地**：完全没有响应。先查线、端口和波特率，再怀疑代码。
- **直接发结构体内存**：填充和端序变成隐藏协议，换个编译选项就不兼容（第二节）。
- **只测"一次完整到达"**：真实链路会分块、粘包、断在中间。`tests.c` 对每个负载长度、每个切分点都测了分两次到达。
- **出错时也写了输出**：调用方可能把半截数据当结果用。本实现所有检查通过之前不写 `frame_t`（第四节）。

短自测

1. 一帧分 10 次到达，结果会和一次到达不同吗？

> **▼ 参考答案**
> 只要相邻两次之间不超过 100ms 就不会。COLLECT 态逐字节累积，遇到帧尾 0 才解码，结果和一次到达完全相同。tests.c 对每个切分点都验证了这一点。

2. 坏 CRC 的帧后面紧跟一个合法帧，能收到合法帧吗？

▼ 参考答案
能。坏帧有自己的帧尾 0，解码失败只计一次 crc 并回到 EMPTY，下一段从头收集。只有已知传输丢失（可能丢的正是帧尾）时，才可能牺牲紧接着的一帧，所以主机重试前要补一个 0。

3. 为什么确认帧入队失败时不能执行故障命令？

▼ 参考答案
否则命令已经生效，确认却没发出去。主机看到超时会重试，同一条不可重试的命令就执行了两次。对这类操作，"没执行但主机知道失败"比"执行了但主机不知道"安全。

4. raw 长 200 字节时 COBS 开销是多少？本协议为什么可以把 74 写成常量？

▼ 参考答案
200 小于 254，开销仍是 1 字节；超过 254 字节，每 254 字节至少多 1 字节。本协议的 raw 最长 72 字节，所以开销恒为 1，加帧尾后线上最长 74 字节，可以在编译期确定并用 _Static_assert 锁住。

对照表：本章概念 → 仓库落点

概念	落点
容量常量与布局断言	`code/common/reliability/protocol.h` 的 enum；`protocol.c` 开头两条 `_Static_assert`
CRC 与检查值	`protocol.c` `protocol_crc()`；`tests.c` 断言 0x29B1；Python 侧 `binascii.crc_hqx`
COBS 编解码	`protocol.c` `protocol_cobs_encode/decode()`
三态八计数	`protocol.c` `protocol_rx_feed/poll/loss()`
丢失先于新字节	`transport.c` `byte_queue_pop()`；`code/stm32/platform/uart.c` `receive()`
4 槽 TX 与补同步 0	`transport.c` `protocol_tx_step()`
确认先于执行	`service.c` `device_service_feed()` 最后一行
主机配对与重试	`scripts/device-console.py` `Transport.request()`

延伸阅读

定界与校验都有一手出处：

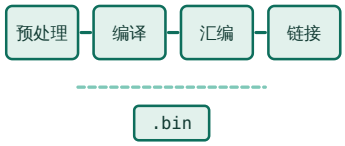
- **[D13]** Cheshire & Baker 1999：COBS 的原始论文。本章"开销固定、可以提前算进预算"这条选型理由，出处就是它。
- **[D9]** Koopman & Chakravarty 2004：按报文长度给出各多项式的汉明距离表。CRC-16/CCITT 对本章这种短帧能检出几位错，是查表查出来的，不用凭经验猜。

你做到了

- 字节流 → 帧 → 命令，每一层都有明确的失败出口和对应计数；
- 能手算一帧的 74 字节上限从哪来，也能在线上逐字节核对；
- S17 看门狗、P13 健康监督 和两个记录器项目用的都是这一份 codec，STATUS 是它们共同的观察窗口。

回到开场那一问：丢一个字节之后，下一帧还认得出来吗？现在你不仅能答「认得」，还能说清代价——等下一个帧尾重新对齐、残帧可能被牺牲、每一笔失败都有计数可查。字节流从此不是「希望能对上」，是一条逐字节可审计的流水线。

✅ 下一站：S17 看门狗：协议负责说清"出了什么错"，看门狗负责"卡住就重来"。



PART PB

第二部分 构建与运行全过程

从源码到机器码再到板上运行：四步构建、ELF 解剖、链接脚本、启动过程、map 审计、烧录调试与构建系统，每道工序的产物都亲手拆开对账。

> 源码到灯亮之间隔着四道工序和一串契约，多数教程一句「make 就行了」带过，这一部把每道工序的产物拆开对账：预处理的上千行展开、.o 里恒为 0 的地址栏、链接脚本删一个词初值就变随机数、map 文件里谁吃了 Flash。取证全部可在宿主机复跑。读完的标志：固件出错时，你的排查顺序从「重烧一遍」变成「先看死在哪一环」。

本部 8 章 · 约 280 分钟（4.7 小时）

10 . B0 工具链全景：四个软件一台戏
11 . B1 四步构建：从 main.c 到机器码
12 . B2 ELF 解剖：把固件放上解剖台
13 . B3 链接脚本：固件的建筑图纸
14 . B4 启动过程：上电到 main 的每一条指令
15 . B5 map 与体积：谁吃了我的 Flash
16 . B6 烧录与调试：SWD 与断点的真相
17 . B7 构建系统：从手写 Makefile 到 CMake 与 scons

通往单片机之路第 10 章

B0 工具链全景：四个软件一台戏

难度 ★★☆☆☆ · 约 25 分钟

本章罗盘

问题

在 x86 的电脑上给 ARM 芯片产代码，到底是谁在干活？裸机没有 glibc，printf 的输出凭什么还有出路？

交付

一张四件套分工表（gcc/binutils/newlib/gdb）加 newlib 钩子表（_write/_sbrk 干什么），附 --version、-print-search-dirs、where 三条工具链环境自查输出。

边界

本章是全景：ELF 解剖在 B2、链接脚本在 B3、GDB 十命令实战在 B6、printf 重定向实操在 S7，钩子只点名不实现；工具链行为随版本（本站 xPack 15.2.1）变。

依据锚点 `mcu-code` · `mcu-anim`

> 🎯 电脑是 x86 的，芯片是 ARM 的。在"鸡"上孵"鸭"蛋：编译器在电脑运行，生成的程序交给芯片运行。这就是交叉编译。

本章精髓

1. "交叉"在哪：编译器本身跑在 x86-64（build），产出的机器码属于 ARM Cortex-M4（target）。先分清谁在运行、谁是产物，再看 `arm-none-eabi-` 前缀。
2. 工具链（toolchain）是配合使用的一组工具。四件套各有分工：gcc 调度编译和链接，binutils 提供 as/ld/objcopy/objdump/nm/readelf 等工具，gdb 负责调试，newlib 提供给裸机用的精简 C 库。
3. 为什么裸机没有 glibc 可用：没有 Linux 系统调用，printf 就没有现成的输出去处。newlib 留出底层接口，你实现几个 `_write` 钩子，就能把 printf 接到串口（S7 实操）。

- **能理解**：`arm-none-eabi-` 三段命名各管什么；为什么裸机不能用 glibc。
- **能用**：`-print-search-dirs` / `-dumpspecs` 自查工具链环境，PATH 冲突能自救。

先修

- 无（建议与 S0 的安装步骤对照）。

先跑起来（10 分钟 quick win）

先查两个问题：这套工具链到哪里找文件？gcc 又怎样调用其他工具？

跑之前先猜：ARM 编译器会使用自己安装目录里的库，还是宿主机本机编译器的库？运行后对照目录清单，看它实际指向哪里。

```
arm-none-eabi-gcc -print-search-dirs     # 它去哪儿找库和头文件
arm-none-eabi-gcc -dumpspecs | more      # gcc 内部的"调度剧本"
```

`-print-search-dirs` 列出 libraries/startup files/include 的搜索路径。先看目录属于哪套工具链，再找 `crt0.o`、`libc.a` 的来源。`-dumpspecs` 则给出 gcc 的内部规则：哪个阶段调用哪个 binutils 工具、按什么顺序调用。原来那份"调度剧本"，说的就是这些规则。

本机工具链事实

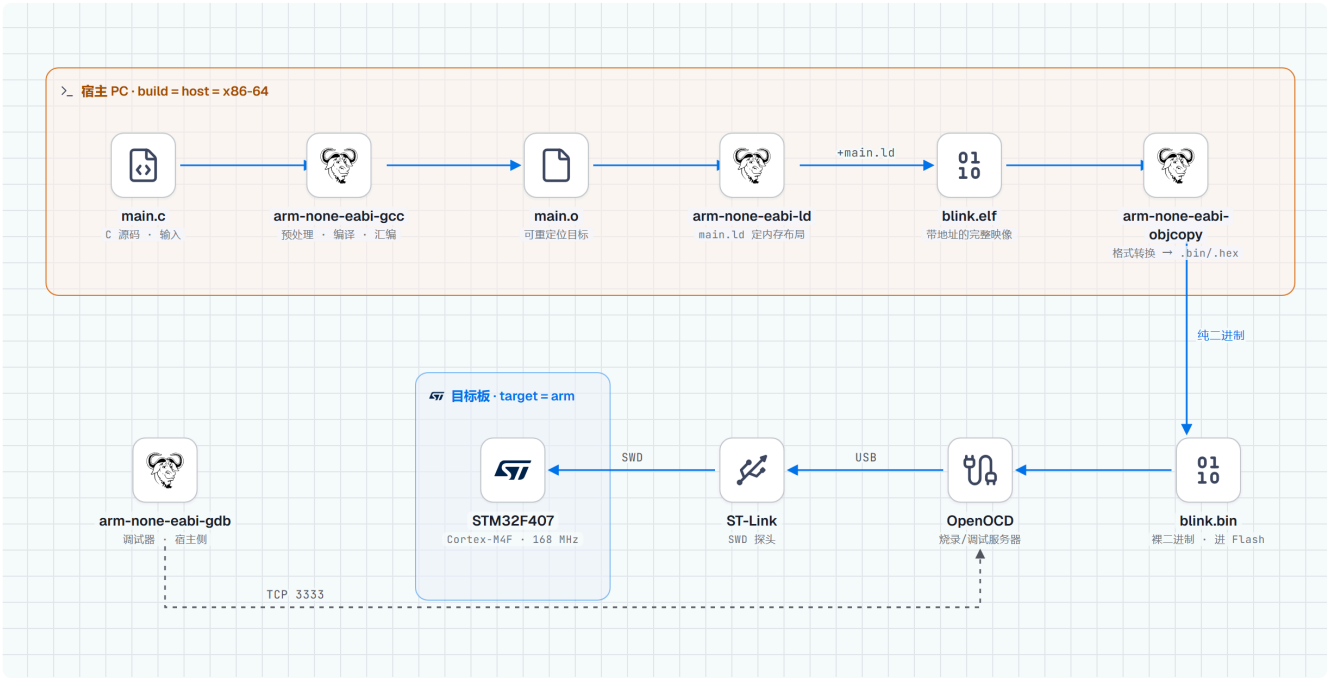
- 本站取证用 **xPack GNU Arm Embedded GCC 15.2.1**（`D:/zhugu-home/tools/armgcc/xpack-arm-none-eabi-gcc-15.2.1-1.1/bin`），与 CI 的 arm-host job 同源；宿主侧 **MinGW-Builds gcc 16.1.0**（x86-64）。
- 交叉编译旗：`-mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard`（F407 的 Cortex-M4F；C6 栈帧已用这套旗取证 AAPCS）。
- 前缀 `arm-none-eabi-`：`arm`=arch（ARM 体系）、`none`=vendor（无特定厂商）、`eabi`=ABI（嵌入式应用二进制接口，裸机无 OS），区别于 `arm-linux-gnueabihf-`（跑 Linux 的）。

小节结构

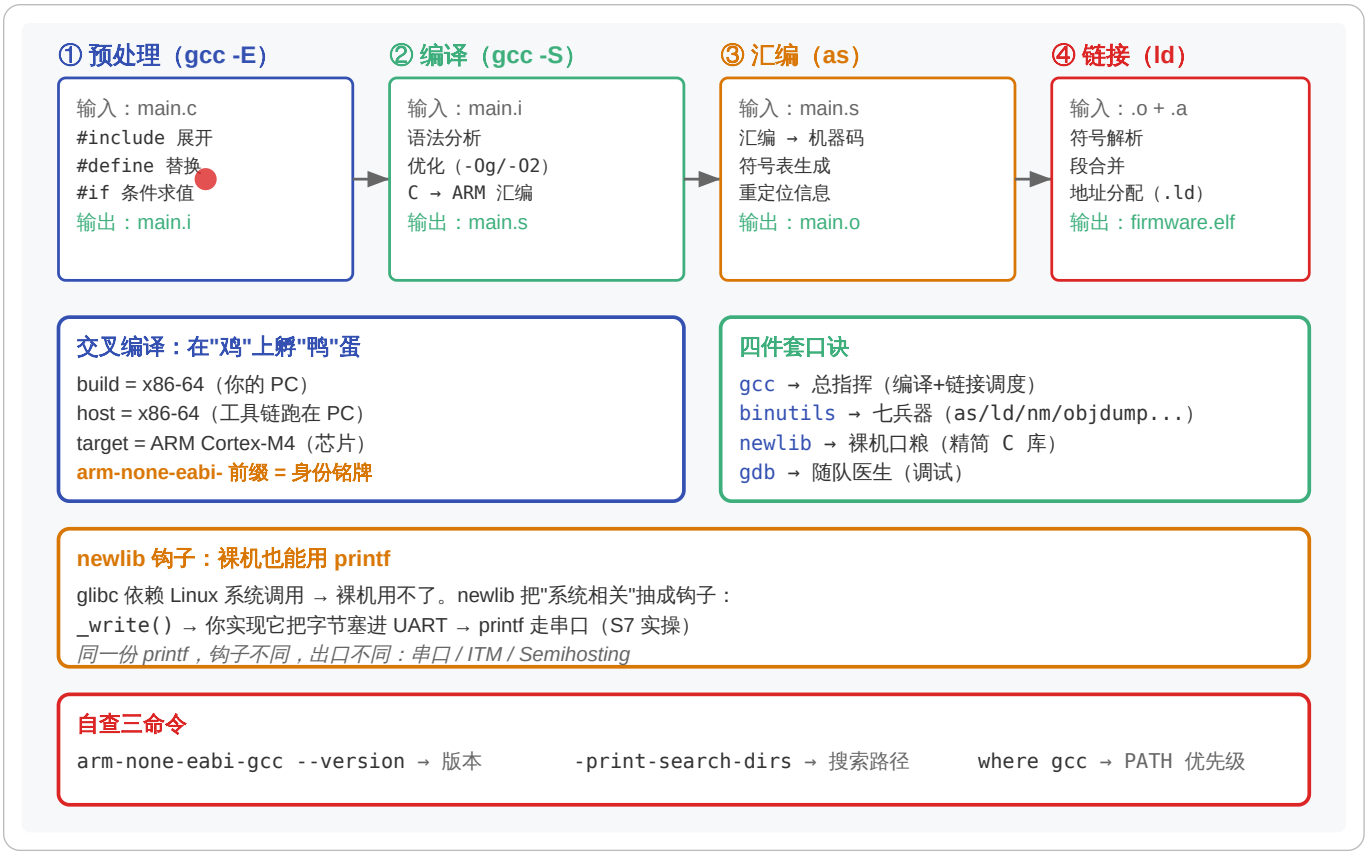
小节	内容	四件套
交叉编译是什么	build/host/target 三胞胎；为什么不能直接用系统 gcc	配置
binutils 兵器谱	as/ld/ar/nm/objcopy/objdump/readelf/strings 各一招鲜	库解析
newlib 与钩子	`_write/_sbrk` 钩子机制预览（S7 printf 重定向的伏笔）	库解析
版本与来源	Arm GNU Toolchain 官方发行；为什么别用"打包版杂牌军"	配置

一、交叉编译是什么：build/host/target 三胞胎

先把全程收进一张图：main.c 在 PC 上被四件套加工成 blink.bin，再经 OpenOCD 与 ST-Link 落进 F407 的 Flash。gdb 那条虚线回头再讲，先记住实线这条主路。



交叉编译四件套全景：从 main.c 到 F407 的完整旅程



先分清两个对象：编译器本身也是一个程序；它处理源码后，还会生成另一个程序。读下面的三个"角色"时，别把编译器与它的产物混在一起：

- **build**：编译器自己跑在什么机器上（你的 x86-64 PC）；
- **host**：产出的**编译器**跑在什么机器上（一般也是 x86-64 PC）；
- **target**：产出的**机器码**属于什么体系（ARM Cortex-M4）。

普通 PC 开发 build=host=target=x86，叫**本机编译**。嵌入式 build=host=x86、target=ARM，叫**交叉编译**：运行编译器的机器，与运行编译产物的芯片，使用不同体系。`arm-none-eabi-gcc` 就是 target=arm 的交叉 gcc。它的 build/host 都是 x86，也就是在 PC 上运行；生成的却是交给 ARM 芯片执行的机器码。

为什么不能直接用系统 gcc 编 F407：系统 gcc 的 target 是 x86-64，吐 x86 机器码，F407 的 ARM 内核根本不认识。必须装一套 target=arm 的交叉工具链。前缀 `arm-none-eabi-` 是区分本机版与交叉版的标准做法：`gcc` 编 x86，`arm-none-eabi-gcc` 编 ARM。

二、binutils 兵器谱：七件兵器各一招

gcc 负责调度，binutils 里的工具各做一项具体工作：汇编、链接、转换格式或查看固件。查本站 B2/B5 的编译产物时，会反复用到下表里的工具：

工具	一招鲜	本站用到处
`as`	汇编器：把 `.s` 汇编成 `.o`	B1 四步构建第二步
`ld`	链接器：把 `.o` + 库合成 `.elf`	B1 第四步、B3 链接脚本
`ar`	归档：把多个 `.o` 打成 `.a` 静态库	库的打包形式
`nm`	符号表：列出 `.elf` 里所有符号+地址	B5 审计、C6 栈帧
`objcopy`	格式转换：`.elf` → `.bin` / `.hex`	B1 烧录产物
`objdump`	反汇编+段信息：`.elf` → 汇编	C3/C6 取证、B2
`readelf`	ELF 头/段/程序头：比 objdump 更专	B2 ELF 解剖、C1
`size`	三列体积：text/data/bss	B5 体积审计
`strings`	抽字符串：固件里的字面量	调试辅助

记住一条：**读固件 80% 靠 binutils**。gcc 只负责"写出来"，binutils 负责"看清楚"。`objdump -d` 反汇编、`nm -n` 按地址排符号、`readelf -S` 看段、`size` 看体积，这四把刀是嵌入式工程师的日常。

三、newlib 与钩子：裸机的精简 C 库

C 标准库（printf/malloc/string.h）在 PC 上由 glibc 提供，但 glibc 依赖 Linux 系统调用，裸机没有 OS，没有 `write()` 系统调用，printf 的"输出"不知往哪去。

newlib 也是 C 库，但面向裸机/嵌入式。它把依赖具体系统的工作留给一组**钩子函数**：库调用这些函数，底层实现由你补上。这样，上层仍然使用 printf/malloc，底层才处理串口输出、堆空间等具体问题：

钩子	干什么	本站实现处
`_write(int fd, char *buf, int len)`	写一字节流到"文件描述符"	S7 USART 把 printf 接到串口
`_sbrk(ptrdiff_t incr)`	给 malloc 增长堆	链接脚本留堆空间（B3）
`_close/_lseek/_read/_isatty`	文件操作占位	裸机一般空实现
`_exit/_kill/_getpid`	进程占位	裸机一般空实现

以 printf 为例，**newlib 不替你决定"输出到哪"**。你实现 `_write`，把收到的字节写进 UART 寄存器，printf 就走串口。也可以把输出接到 SWO，经 ITM 输出；或接到 Semihosting（B6），经调试器送回 PC。

上层仍是同一份 printf 代码，改变的是底层钩子的实现。这就是 S7 的 printf 重定向。再对照 C7 里的"UB 最怕库内部优化"：库内的行为会影响上层代码；这里 newlib 则把系统相关的工作明确交给钩子，让裸机也能使用标准 C 库。

四、版本与来源：为什么别用"打包版杂牌军"

ARM 官方维护 **Arm GNU Toolchain**（gcc/binutils/newlib/gdb 一体打包），每个版本经过 ARM 验证、与 CMSIS 头文件对齐。本站用 xPack 发行版（基于官方源码、跨平台、版本明确）。

为什么别用某 IDE 捆绑的"杂牌军"：

- **版本不明**：捆绑版常改了 specs 却不写清楚，`-mfloat-abi=hard` 行为与官方版有微妙差异；
- **PATH 冲突**：装了官方版又装捆绑版，`where arm-none-eabi-gcc` 谁前用谁，编译行为飘忽；
- **newlib 补丁滞后**：`_sbrk` 的默认实现、`printf` 的浮点支持（`%f`）在老版 newlib 上可能缺，官方版定期更新。

自查纪律：`arm-none-eabi-gcc --version` 看版本号、`-print-search-dirs` 看搜索路径、`where arm-none-eabi-gcc`（Windows）/ `which`（Linux）看 PATH 优先级，三条命令把"我到底在用哪套工具链"查清楚。本站 CI 的 arm-host job 与本地用同一套 xPack 15.2.1，保证本地能编过 CI 就能编过。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **gcc 是总指挥，binutils 是七件兵器，newlib 是裸机口粮，gdb 是随队医生**，前缀 `arm-none-eabi-` 就是他们的工牌。

延伸：工具链产出的 ELF 结构在 B2 ELF 双重视角 展开；GDB 调试链路在 B6 SWD 调试 完整走一遍；RISC-V 工具链对照见 V0。

实物实验

- 对 00-blink 的 `build/blink.elf` 连发四枪：`nm`（看符号）、`objdump -h`（看段）、`readelf -l`（看加载段）、`objcopy -O binary`（出 bin），每枪都在 B2/B5 展开。

常见坑

- **混用多个工具链**：系统里装了 Arm 官方版又装了某 IDE 捆绑版，PATH 谁前用谁。`where arm-none-eabi-gcc` 查一下。
- **工具链与目标不匹配**：拿 `arm-linux-gnueabihf-gcc`（跑 Linux 的）编裸机，启动文件/库全不对。
- **把 binutils 当 gcc 附属品**：实际上读固件 80% 靠 binutils，B2/B5 两章全是它们的戏份。
- **newlib 浮点 printf 不工作**：老版 newlib 默认不带 `%f`，要加 `-u _printf_float` 链接浮点版 printf。

短自测

1. `arm-none-eabi-gcc` 的三段前缀各代表什么？为什么不能用系统 `gcc` 编 F407？

> ▼ **参考答案**
> `arm`=arch（ARM 体系）、`none`=vendor（无特定厂商）、`eabi`=ABI（嵌入式应用二进制接口，裸机无 OS）。系统 gcc 的 target 是 x86-64，吐 x86 机器码，F407 的 ARM Cortex-M4 内核不认识。必须用 target=arm 的交叉 gcc，前缀区分本机版与交叉版。

2. 四件套里谁负责把 `.elf` 变成 `.bin`？谁负责反汇编？

▼ **参考答案**
`objcopy -O binary` 把 `.elf` 转成 `.bin`（剥掉 ELF 头、按地址铺平）；`objdump -d` 反汇编（把机器码翻译回汇编）。两者都属于 binutils，不是 gcc 的功能：gcc 只负责编译，binutils 负责格式转换与查看。

3. 裸机为什么不能用 glibc？newlib 用什么机制让 printf 工作？

▼ **参考答案**
glibc 依赖 Linux 系统调用（write/read/sbrk 等），裸机没有 OS 没有系统调用，glibc 链接不上也跑不了。newlib 把"系统相关"部分抽成钩子函数（`_write`/`_sbrk` 等），你实现 `_write` 把字节塞进 UART 寄存器，printf 上层不变就走串口。同一份 printf，钩子不同出口不同。

4. PATH 里有两套 arm-none-eabi-gcc，怎么查"我到底在用哪套"？

▼ **参考答案**
`where arm-none-eabi-gcc`（Windows）/ `which`（Linux）看 PATH 优先级排第一个的是哪套；`arm-none-eabi-gcc --version` 看版本号；`-print-search-dirs` 看它的库/头文件搜索路径，三条命令把版本、路径、来源查清楚，确认是不是你想要的那套。

对照表：本章概念 → 仓库与上游落点

概念	落点
交叉编译旗（-mcpu/mfpu/mfloat-abi）	C6 栈帧 probe.sh 37 行；本站 CI arm-host job
binutils 兵器谱	B2 ELF 解剖 / B5 体积审计 全章实证
newlib `_write` 钩子	S7 USART printf 重定向
工具链版本与来源	xPack GNU Arm Embedded GCC 15.2.1；`-print-search-dirs` 自查

你做到了

- 工具链从"一坨 exe"变成职责分明的四件套；
- 会查 gcc 的搜索路径和调度剧本，环境问题能自救。

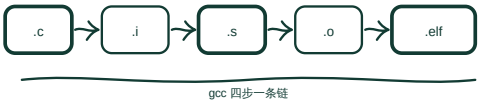
开场那句「在鸡上孵鸭蛋」，现在对应的是明确的分工：工具在电脑上运行，产物交给芯片执行。下次环境报错说找不到某个文件，先别急着重装。先查工具链的搜索路径，确认它找的是哪套文件。

✅ 下一站：B1 四步构建：把 main.c 一步步变成机器码，中间产物全留下。

通往单片机之路第 11 章

B1 四步构建：从 main.c 到机器码

难度 ★★☆☆☆ · 约 30 分钟



本章罗盘

问题

make 一秒跑完的构建拆开是哪四道工序？『头文件找不到』和『undefined reference』为什么死在不同阶段？

交付

一套手动四步构建的全套中间产物：main.i、main.s、main.o 地址栏全 0、blink.elf 段表，加三类报错原文与阶段的对照分诊表。

边界

标本是两文件裸机工程：库的反复扫描、multilib 目录选择与链接脚本语法在 B0/B3；.o 的地址栏恒为 0，最终地址只能查 .elf，不能拿 .o 证据推断烧录地址。

依据锚点 e3 · mcu-code · mcu-anim

> 🎯 `make` 一秒跑完的事，拆开是四道工序：预处理展开宏、编译生成汇编、汇编产出机器码、链接拼成固件。会拆就会查——90% 的"莫名编译错误"都能定位到具体某一环。

本章精髓

1. gcc 是"调度员"不是"全能选手"：它按后缀名和参数把活分包给 cc1/as/collect2（ld），`-###` 参数能把它的心里话全打印出来。
2. 每一步都产出一个可检查的中间文件：`.i`（纯 C）、`.s`（汇编）、`.o`（可重定位目标）、`.elf`（可执行）。
3. "头文件找不到"是预处理的事，"undefined reference"是链接的事：**报错阶段=嫌疑人范围**。

- **能理解**：亲手把 `main.c`（65 行）喂给 `-E`，看它膨胀成 1575 行，并在里面找到被摊平的 `RCC_BASE` 宏。
- **能用**：拿到任何一条编译报错，先看关键字（`fatal error: ... No such file` / `error: expected ...` / `undefined reference`）就能报出阶段。

先修

- B0 工具链：`arm-none-eabi-gcc` 必须在 `PATH` 里（本机实测版本 15.2.1 / xPack）。

先跑起来（10 分钟 quick win）

先猜猜预处理后的源文件和汇编文件各有多少行，把两个数写下来。跑完再和实际输出比较：同一份源码，经过不同工序，文件内容和行数会怎样变化？

```
cd code/toolchain/01-four-steps
sh probe.sh steps      # 只走四步，build/ 里留下 .i/.s/.o/.elf 全套证据
```

这里用的是 `code/stm32/00-blink` 的真实工程（`main.c` + 启动文件 + 链接脚本），不需要开发板。跑完先对照三处输出：`main.i` 1575 行、`main.s` 3757 行，以及 `.text` 从地址 0 变成 `0x08000188`。前两项看文件内容的变化，后一项看链接前后的位置变化。

接着运行下面的命令，观察三个阶段各自的报错文字：

```
sh probe.sh errors     # 预处理/编译/链接 各制造一个错误，原文对照
```

小节结构

小节	内容	四件套
调度员 gcc	`-###` 打印内部分工；`-save-temps` 一键留证据	库解析
预处理	宏展开/#include 嵌套/条件编译；`#define` 与 const 之争	代码分析
编译与优化	cc1 干的事；`-O0/-O2` 汇编对照入口（C3 联动）	代码分析
汇编与目标文件	.o 的"半成品"状态：符号还没地址（重定位预告）	代码分析
链接	符号决议 + 重定位 + 按脚本排座（B3 深讲）	库解析

一、调度员 gcc：`-###` 看它把活分给谁

先把整条流水线画出来：一份 65 行的 `main.c` 是怎样在四道工序里变成能烧进 Flash 的 `.elf`。

B1 四步构建流水线动画

`arm-none-eabi-gcc` 本身不编译代码，而是读参数、查文件后缀，再 fork 出实际工作的工具。加上 `-###` 后，它**只打印将要调用的工具，不执行编译**。先看每行开头的程序路径，再看后面的输入、输出文件，就能对上各自的分工（本机实测，路径已截断）：

```
$ arm-none-eabi-gcc -### -E -mcpu=cortex-m4 -mthumb main.c -o main.i
 ".../libexec/gcc/arm-none-eabi/15.2.1/cc1.exe" ... -E ... -o main.i      # 预处理也是 cc1 干的

$ arm-none-eabi-gcc -### -c -mcpu=cortex-m4 -mthumb main.c -o main.o
 ".../libexec/gcc/arm-none-eabi/15.2.1/cc1.exe" ... -o /tmp/ccXXXX.s     # C → 汇编
 ".../arm-none-eabi/bin/as.exe" ... -o main.o                            # 汇编 → 目标文件

$ arm-none-eabi-gcc main.o startup.o -T stm32f407xx.ld -o blink.elf
 ".../libexec/gcc/arm-none-eabi/15.2.1/collect2.exe" ...                 # 由它再去调 ld
```

四条事实：

1. **预处理不是独立程序**。老教程说的 `cpp` 只是个壳，现代 GCC 里预处理和编译都是 `cc1`，靠 `-E` / `-S` 开关切换输出。

2. `-c` 这条命令实际调用了**两个**工具（cc1 + as）：先编译，再汇编。命令可以写在一起，但两道工序各有产物，不能把它们当成同一步。
3. 链接由 `collect2` 代理：它负责收拾构造函数表（`__libc_init_array` 之类）再调 `ld`。所以链接报错里署名 `ld.exe`。
4. `-###` 还会打印 `COMPILER_PATH` / `LIBRARY_PATH`。找不到 `-lm`，或怀疑混用了不同版本的库时，先检查这两行指向的路径。

不想逐条敲命令，也可以用 `-save-temps` 保留中间产物：它会把 `.i/.s/.o` 自动留在当前目录。在 `make` 的构建目录里查看这些文件，就能回头检查每道工序。

二、第 1 步：预处理 `-E` ——宏在纸上摊平

`-E` 只做三件事：展开 `#include`、替换宏、按条件编译裁剪。输出仍是 C 源码（带 `# 行号 "文件"` 标记，给调试器认行用的）。

```
arm-none-eabi-gcc -E main.c -o main.i
```

实测：**65 行 → 1575 行**。多出来的 1510 行全是 `#include <stdint.h>` 等头文件的内容。打开 `main.i` 找 `main`，签名没变，但宏全没了：

```
/* main.i 第 1556 行 */
int main(void)
```

更有意思的是源码里的 `RCC_BASE`，它在 `main.i` 第 1533 行还留了个 `#define`，但**用到它的地方已经摊成了纯算术式**：

```
/* main.i 第 1559 行：源码里那行 RCC->AHB1ENR |= RCC_AHB1ENR_GPIOFEN; */
(*(volatile uint32_t *)(((0x40000000UL + 0x00020000UL) + 0x3800UL) + 0x30UL)) |= (1UL << 5);
```

这就是"宏只是文本替换"的实物：`0x40000000 + 0x20000 + 0x3800 + 0x30` 加法是留给编译器的活（常量折叠），`1UL << 5` 同理。**指针、volatile、位运算的语义从这一行开始就已经确定**。后面两步只是在"翻译"它。

三、第 2 步：编译 `-S` ——C 变成汇编

`-S` 让 cc1 停在汇编层。它会做语法检查、类型检查、优化，然后吐出目标架构的汇编：

```
arm-none-eabi-gcc -S main.i -o main.s      # 实测 3757 行
```

`main` 的开场（本机实测节选）：

```
main:
    push    {r7, lr}          @ 函数序言：压栈保存返回地址
    add r7, sp, #0
    ldr r3, .L6
    ldr r3, [r3]           @ 读 RCC->AHB1ENR（volatile：必须真读）
    ldr r2, .L6
    orr r3, r3, #32        @ 32 = 1<<5 = GPIOFEN
    str r3, [r2]           @ 写回 RCC->AHB1ENR
```

三点可验证的观察：

- `orr r3, r3, #32`：源码的 `|= (1UL << 5)` 被常量折叠成 `#32`。**读-改-写**在汇编里看得一清二楚：先 `ldr` 读、再 `orr` 改、最后 `str` 写。
- 为什么是"读-改-写"而不是一条原子指令？因为 C 里写的就是 `|=`。想避免（S3 讲过的翻车），要在**外设侧**换原子入口，这是后话。
- 地址 `0x40023830` 没出现在指令里，而是被塞进"字面池"（`.L6`），用 `ldr r3, .L6` 取。ARM 的立即数字段装不下 32 位地址，只能这么干。

`-O0` 下函数序言固定存在（`push {r7, lr}` + `add r7, sp, #0`，为了调试时栈回溯方便）；换 `-O2` 这两条会消失，这就是 C3 里"优化等级改变汇编"的入口。

四、第 3 步：汇编 `-c` ——半成品 `.o`

```
arm-none-eabi-gcc -c main.s -o main.o
```

`.o` 叫**可重定位目标文件**。汇编已经生成机器码，但这些代码最终放在哪个地址，还要等链接时决定。下面分别查符号和段的位置：

```
$ arm-none-eabi-nm main.o
（一个 U 都没有——外设访问是硬编码地址常量，不需要符号决议）

$ arm-none-eabi-nm startup_stm32f407xx.o | grep ' U '
    U _ebss
    U _edata
    U _estack
    U _sbss
    U _sdata
    U _sidata
    U main

$ arm-none-eabi-objdump -h main.o
  6 .text         000000a8  00000000  00000000  0000007c  2**2
```

- 先看 `nm` 输出：`U` 表示"未定义符号"，也就是本文件用到了这个名字，定义却在别处，需要链接器来补地址。`main.o` 里**一个 `U` 都没有**，因为这里访问外设用的是硬编码地址常量，不需要查找符号定义。再看 `startup_stm32f407xx.o`，其中的 `U` 就是在列出它还缺哪些定义：`_sidata` / `_sdata` / `_edata` / `_sbss` / `_ebss` / `_estack` 是链接脚本定义的段边界符号，`main` 则由应用定义。
- 再看 `objdump -h`：地址栏里的 `00000000` 不是最终运行位置。`.text` 此时是个 0xa8 字节的块，链接时才决定把它放到哪里，这就是这里的"可重定位"。

查 `nm` 时，先认这三个符号类型：`T/t` 表示代码段（大写全局、小写局部），`U` 表示还要去别处找定义，`W` 表示弱符号、可以被覆盖。

五、第 4 步：链接——决议符号 + 按脚本排座

```
arm-none-eabi-gcc main.o startup_stm32f407xx.o \
    -T stm32f407xx.ld -nostartfiles -o blink.elf
```

链接器（ld）接着解决两个问题。先做**符号决议**：给每个 `U` 找到对应的定义。再做**重定位**：按链接脚本安排各个段的位置，并填入代码里需要的地址。

实测结果：

```
$ arm-none-eabi-objdump -h blink.elf
Idx Name          Size      VMA       LMA
  0 .isr_vector   00000188  08000000  08000000    ← 向量表钉在 Flash 起始
  1 .text         0000010c  08000188  08000188    ← 代码紧随其后
  2 .data         00000000  20000000  20000000    ← 变量在 RAM（0x2000_0000）
  3 .bss          00000000  20000000  08000294

$ arm-none-eabi-size blink.elf
   text    data     bss     dec     hex
    660       0    1536    2196     894

$ arm-none-eabi-nm -n blink.elf | grep -E ' (Reset_Handler|main)$'
080001aa T main
08000230 W Reset_Handler
```

读法：

- **VMA（Virtual Memory Address）** = 运行时的地址。`.isr_vector` 和 `.text` 在 `0x0800_0000` 段（Flash），`.data` / `.bss` 在 `0x2000_0000` 段（SRAM）。这就是链接脚本把"哪块东西放哪块存储"焊死的效果，B3 整章讲这个。
- `text 660` = 392（向量表）+ 268（代码）。`bss 1536` 是 `._user_heap_stack` 预留的栈/堆空间，不占 Flash。
- `Reset_Handler` 是 **W（弱符号）**：启动文件里给了默认实现，用户可以覆盖。芯片复位后 CPU 从 `0x0800_0000` 取向量表第 2 个字跳到这里。
- `main` 的地址是 `080001aa`：**最低位是 1**。这不是错位：Cortex-M 只跑 Thumb 指令，函数符号地址的最低位 1 是"这是 Thumb 代码"的标记位。

到这里，四道工序走完：一份 65 行的 C 源码变成了一个能烧进 Flash 的 `.elf`。

报错分诊：三个现场实测

`sh probe.sh errors` 会故意制造三个错误，报错原文（本机实测）如下，**只看关键字就能定位阶段**：

阶段	关键字	实测报错原文
预处理	`fatal error: ... No such file or directory`	`bad_pp.c:1:10: fatal error: no_such_header_xyz.h: No such file or directory`
编译	`error: expected ...`	`bad_cc.c:1:22: error: expected expression before ';' token`
链接	`undefined reference to ...`	`ld.exe: bad_ld.o: in function 'main': bad_ld.c:2:(.text+0x4): undefined reference to 'SystemInit'`

三点：

1. **`fatal error` 只在预处理阶段出现**，预处理一旦失败就直接停，连语法分析都到不了。
2. **`error:` 是编译期语法/类型检查**，行号精确到列（`1:22`）。
3. **`undefined reference` 一定是链接期**，此时**编译已经全部通过**。修法是"补定义/补 `.o` /补库"，不是去改语法。上面这条报错还附带了 `warning: cannot find entry symbol Reset_Handler`，因为 `-nostartfiles` 没给启动文件，链接器找不到入口，这正说明"链接阶段在管符号和入口"。

附录：工程完整源码

本章取证工程 `code/toolchain/01-four-steps/`（`sh probe.sh` 直接跑通，需工具链在 `PATH`）：

probe.sh（四步 + 三错误现场，全自动取证）：

<<< ../../code/toolchain/01-four-steps/probe.sh

被构建的标本 `code/stm32/00-blink/main.c`（B1 全篇的输出都由它而来）：

<<< ../../code/stm32/00-blink/main.c

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **E、S、c、T 四步走：-E 出 .i，-S 出 .s，-c 出 .o，-T 定格局（链接脚本）。** 报错先问：死在哪一步？

延伸：四步构建的工具链全景见 B0；链接脚本详解见 B3；ELF 产物解剖见 B2。

实物实验

- **无板（本机已实测）**：`cd code/toolchain/01-four-steps && sh probe.sh`，四步输出与本文数字逐条对上（`main.i` 1575 行、`.text` 起始 `0x08000188`、`size` text=660）。
- **无板（进阶）**：把 `CFLAGS` 里的 `-O0` 改成 `-O2` 再跑 `sh probe.sh steps`，对比 `main.s`：函数序言 `push {r7, lr}` 消失，开时钟那三行 `ldr/orr/str` 可能被合并，亲手看优化等级如何改写汇编。
- **板上（可选，肉眼现象待回填）**：`cd code/stm32/00-blink && make flash`，PF6 红灯亮，那盏灯就是本章四步流水线的最终产物。

常见坑

- **把链接错当编译错**：`undefined reference to 'SystemInit'` 是链接阶段缺符号，不是语法问题，补定义或补 .o。
- **多文件忘了都传给链接器**：只链接 main.o，启动文件的符号全缺；Makefile 的 OBJS 列表就是干这个的。
- **汇编器参数漏 `-mcpu/-mthumb`**：.s 文件也走 gcc 调度，MCU 参数少了照样报错。
- **只 `-E` 就以为看到了真相**：`main.i` 里 `#define` 行还在（第 1533 行），但使用处已摊平，要看展开结果，得找**使用处**，不是找 `#define`。
- **以为 `.o` 里能看到最终地址**：`.o` 的地址栏永远是 0。想核对地址，只能看 `.elf`（`objdump -h` / `nm`）。

短自测

1. `-E` 之后 `main.i` 从 65 行涨到 1575 行，多出来的是什么？

> **▼ 答案**
> `#include ` 等头文件被整份贴进来的内容，加上宏展开后变长的行。预处理**只做文本层面的三件事**：展开 include、替换宏、按条件编译裁剪，不检查语法，更不生成任何机器码。

2. `nm main.o` 一个 `U` 符号都没有，为什么？

▼ 答案
因为 `main.c` 里的外设访问全是**硬编码地址常量**（`0x40023830` 等），不需要符号决议。汇编里那些地址走"字面池"（`ldr r3, .L6`）取常量，不是 `U` 引用。对比 `startup_stm32f407xx.o`：那里有 `U _sidata`、`U _sdata`、`U _edata`、`U _sbss`、`U _ebss`、`U _estack`（链接脚本负责定义）和 `U main`，才需要链接器去别处找定义。

3. `undefined reference to 'SystemInit'` 死在哪个阶段？怎么修？

▼ 答案
链接阶段。编译已经全部通过，问题是"某个符号被引用了却没人定义"。修法三选一：补上实现、把定义它的 `.o` 加进链接命令、或链接对应的库（`.a`）。改语法是南辕北辙。

4. `-c` 之后 `objdump -h main.o` 里 `.text` 的地址栏是 0，为什么？

▼ 答案
`.o` 是**可重定位**目标文件：代码块已经生成，但"放哪个地址"还没决定。地址由链接阶段按链接脚本分配（本例 `.text` 最终落在 `0x08000188`）。这也是为什么只有 `.elf` 才能查最终地址。

对照表：本章概念 → 仓库落点

概念	仓库落点
四步流水线的自动化脚本	`code/toolchain/01-four-steps/probe.sh`（32–72 行 `steps`）
预处理展开对照（65 → 1575 行）	同上，第 1 步（35–43 行）
`main` 的汇编序言与开时钟的读-改-写（`ldr` / `ldr` / `orr` / `str`）	同上，第 2 步（45–50 行）
`.o` 重定位状态（地址栏 = 0）与 `U` 符号对照	同上，第 3 步（52–61 行）
段 VMA 与 `size` 汇总	同上，第 4 步（64–71 行）
三个报错现场原文	同上，`errors`（74–109 行）
被构建的标本源码	`code/stm32/00-blink/main.c`

你做到了

- 亲手走完四道工序，每步留证；
- 编译报错能按阶段定位嫌疑人。

开场说 make 一秒跑完四道工序，这一秒你现在拆开看过了：宏展开后的上千行、还没住址的目标文件、最后才拿到门牌的固件。下次报错，先问它发生在哪一环，嫌疑人名单立刻从四个缩到一个。

✅ 下一站：B2 ELF 解剖：readelf/objdump 实拆一个固件，段与节不再傻傻分不清。

通往单片机之路第 12 章

B2 ELF 解剖：把固件放上解剖台

难度 ★★☆☆☆ · 约 65 分钟

本章罗盘

问题

34KB 的 blink.elf 烧进芯片只有 660 字节，差额是什么？节表和程序头两套目录各自在回答谁的问题？

交付

一份五刀解剖记录：ELF 头五字段、A 标记筛出 660 字节、.data 行 VirtAddr/PhysAddr/FileSiz/MemSiz 四数读法、nm 地址流水账、tag 加减 const 的 8 字节 RAM 对照实验。

边界

实测数字出自 xPack 15.2.1 -g3，换版本 .debug_* 个数会变、地址与结构关系不变；readelf 与 nm 差 1 是 Thumb 位口径，只属 Cortex-M；反汇编里的『乱码』是数据被读成指令，不是编译器坏了。

依据锚点 `c3` · `e2` · `e3` · `mcu-code` · `mcu-anim`

> 🎯 烧进芯片的只有 **660 字节**，可你的 `blink.elf` 有 **34,604 字节**。那 33,944 字节的差额不是 waste，是一整份写给活人看的说明书。本章动五刀，把它读完，每一刀都配本机实测输出。

本章所有数字来自同一套工具链：xPack GNU Arm Embedded GCC **15.2.1 (20251203)**，`-mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard -O0 -g3`。版本不同，`.debug_*` 的个数和符号数量会变，**地址与结构关系不会变**。

本章精髓

1. **为什么一个文件要装两套目录**：链接器与装载器要查的信息不同，所以 ELF 同时提供两套目录。链接器按**节（section）**分抽屉：`.isr_vector` / `.text` / `.rodata` / `.data` / `.bss` 各归各；装载器按**段（segment，即程序头里的 `PT_LOAD`）**装车，把若干节打包成一批，一次性摆到指定地址。查体积时，先分清是在数文件里的内容，还是芯片要用的内容：blink.elf 的 20 个节里只有 5 个带 `A`（alloc）标记，真正在文件里占字节的只有 2 个。**"34 KB 的固件"和"660 字节的固件"说的是同一件事。**
2. **为什么 VMA 与 LMA 必须分家**：`.data` 在 RAM 中运行，初值存放在 Flash。两个地址由链接脚本规定：`stm32f407xx.ld:91,100` 里的 `_sidata = LOADADDR(.data)` 与 `} >RAM AT> FLASH`，对应到 `elf_probe.elf` 的程序头，就是 `VirtAddr=0x20000000 / PhysAddr=0x08000244`。第三刀会对照两份标本：`blink.elf` 的 `.data` 是空的，同一个位置算出的是 `0x08000294`，不要把两份输出混用。C1 解释初值由谁搬、怎样搬；本章则练习**从文件里读出这对地址**。
3. **为什么所有地址都可能差 1**：`readelf -h` 说入口是 `0x8000189`，`nm` 说 `Reset_Handler` 在 `0x08000188`。谁错了？都没错。Cortex-M 只跑 Thumb 态，**最低位是状态位不是地址位**。不懂这条约定，你会在 GDB 里对着一句 "Breakpoint at 0x8000189" 找不到函数。

- **能记住**：ELF 头五个字段各说什么；节表与程序头分别回答谁的问题；`FileSiz` 与 `MemSiz` 的差从哪来
- **能理解**：为什么 `Addr` 与 `Off` 差 `0x1000`；为什么 `nm` 与 `readelf -s` 给出的函数地址差 1；为什么反汇编里会冒出 `MCU.....` 这种"乱码"
- **能用**：拿到任何一颗 MCU 的 `.elf`，五条命令内说清"入口在哪、什么占 Flash、什么占 RAM、每个符号住在第几字节"

先修

- B1 四步构建：本章拆的是**链接**那一步的产物；C1 内存模型：段的双重户籍已在该章建立直觉。
- 想知道"这些地址是谁规定的"→ B3 链接脚本；想知道"谁照着这两个地址搬运行李"→ B4 启动文件。

先跑起来（10 分钟 quick win，不用开发板）

先猜：ELF 里记录的入口地址，和符号表里的复位例程地址，是否相同？再猜它们若不相同，会差多少。运行后找到这两项输出，直接比较。

```
cd code/toolchain/02-elf
sh probe.sh head
```

```
Entry point address:               0x8000189
08000000 r g_pfnVectors
08000188 W Reset_Handler
```

`0x8000189` 与 `08000188` 差的那个 **1**，本章第三节会亲手从向量表里再挖出来一次。CPU 复位后读的第一口数据里，写的就是 `0x08000189`。跑不出来别慌，下面五刀每刀都给命令。

动画：ELF 的双重视角

看动画时先问两件事：链接器怎样查各个节，装载器怎样查程序头？它们看的都是同一份固件，用的却是两套目录。

ELF 节表与程序头双重视角动画

解剖台上的两具标本

标本	怎么来的	为什么请它上桌
`build/blink.elf`	`../../stm32/00-blink` 的 `main.c` + `startup_stm32f407xx.s` + `stm32f407xx.ld`，按它 Makefile 里那三条命令原样构建（不依赖 make）	你已经在 B1 见过的真固件；缺点是**太干净**：`.data` 大小 0，VMA/LMA 分家看不全
`build/elf_probe.elf`	本目录 `elf_probe.c`（完整源码见本章末「附录：工程完整源码」），刻意留了一个有初值的全局、一个字符数组、一个 const 表、一个零初值全局、一个未初始化数组	五个对象刚好占满 ELF 的四种归宿，`FileSiz ≠ MemSiz` 那一行必须靠它才出现

小节结构

小节	内容	四件套
第一刀 ELF 头	魔数/类别/机器/入口/程序头计数；Thumb 位初现	代码分析
第二刀 节表	`Addr` / `Off` / `Size` / `Flg` 四列怎么读； `PROGBITS` 与 `NOBITS` 的分界	配置
第三刀 程序头	`PT_LOAD` 与 section→segment 映射；VMA/LMA、 `FileSiz` / `MemSiz` 实读	配置
全景动画 两副目录	文件字节带居中，节表在上、程序头在下，四阶段连线的； `objcopy` 那一刀切掉了什么	代码分析
第四刀 符号表	`nm -n` 地址流水账；91 个弱别名； `readelf -s` 与 `nm` 的 ±1	代码分析
第五刀 反汇编	`main` 的 prologue、字面量池、 `Reset_Handler` 的搬运循环、数据混进代码区	代码分析

第一刀：ELF 头——五个字段就够

```
sh probe.sh head
```

```
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF32
  Type:                              EXEC (Executable file)
  Machine:                           ARM
  Entry point address:               0x8000189
  Number of program headers:         2
  Number of section headers:         20
```

五件事一次说完：

- **`7f 45 4c 46`** = `0x7F` + `'E' 'L' 'F'` 。前四字节就是签名， `file blink.elf` 认的就是它。第五字节 `01` = ELF32（Cortex-M 是 32 位，**不会是 ELF64**），第六字节 `01` = 小端。
- **`Type: EXEC`** ：可执行文件，不是共享对象、不是 PIE。裸机没有动态链接器，也没有 ASLR，地址从链接那一刻就焊死了。这正是你能在链接脚本里写死 `0x08000000` 的前提（B3）。
- **`Machine: ARM`** + 头里的 `Flags: 0x5000400, Version5 EABI, hard-float ABI` ：这一行是"我用的是硬浮点 ABI"的**唯一官方声明**。编译选项和库的 ABI 对不上时，症状是链接通过、跑起来浮点全错，第一个该查的就是这里。
- **`Entry point address: 0x8000189`** ：CPU 复位后**不**看这个字段（它看向量表，见第五刀），但 GDB 的 `load` + `break *0x8000189` 看它。
- **`Number of program headers: 2 / section headers: 20`** ：**两套目录各有几项**。这 2 与 20 不相等这件事，就是本章第二、三刀的全部内容。

顺手把"差 1"钉死：

```
08000000 r g_pfnVectors
08000188 W Reset_Handler
```

`0x08000188 | 1 = 0x08000189` 。**最低位是 Thumb 状态位**。ARMv7-M 架构里 Thumb 位必须为 1，否则硬件直接 fault，所以你写中断向量、写函数指针表时那个 `|1` 是必需的，不是洁癖。

第二刀：节表——链接器的抽屉

```
sh probe.sh sect
```

先看干净的 blink：

```
  [ 1] .isr_vector  PROGBITS  08000000 001000 000188 00   A
  [ 2] .text        PROGBITS  08000188 001188 00010c 00  AX
  [ 3] .data        PROGBITS  20000000 001294 000000 00  WA
  [ 4] .bss         NOBITS    20000000 000000 000000 00  WA
  [ 5] ._user_heap_stack NOBITS 20000000 002000 000600 00   WA
```

再看标本 elf_probe（ `.text` 短一点、 `.data` / `.bss` 有货）：

```
  [ 1] .isr_vector  PROGBITS  08000000 001000 000188 00   A
  [ 2] .text        PROGBITS  08000188 001188 0000bc 00  AX
  [ 3] .data        PROGBITS  20000000 002000 00000c 00  WA
  [ 4] .bss         NOBITS    2000000c 00200c 000014 00  WA
  [ 5] ._user_heap_stack NOBITS 20000020 002020 000600 00   WA
```

读表时分别问：运行时放哪、在文件里放哪、占多少字节，以及是什么类型、带什么标志。逐项对照下面的列名：

- **`Addr`（VMA，运行地址）**：先用这一列判断这个节运行时位于哪块存储器。`0x08000000` 属于 Flash，`0x20000000` 属于 RAM。blink 的 `.data` / `.bss` / `._user_heap_stack` 都写着 `0x20000000`，并不是三块内容挤在一起：前两个大小为 0，所以堆栈段直接从 RAM 起始位置开始。
- **`Off`（文件偏移，货架号）**：这个节在 `.elf` 文件里的第几字节。`.text` 的 `Addr=0x08000188` 而 `Off=0x001188`，**差 `0x1000`**。别把两者当一个：换算关系是"减掉 Flash 基址、加上文件里那段表头垫片"。第一个节要到文件第 `0x1000`（4096）字节才开始，前面 4096 字节里只装了 52 字节的 ELF 头和 64 字节的程序头，剩下全是对齐垫子。
- **`Size`**：这一列以字节为单位记录大小。`0x188` = 392，也就是 **98 个 4 字节表项**。可以到 `startup_stm32f407xx.s` 中对照 `g_pfnVectors`：共 98 条 `.word`，分别存放栈顶、复位入口，以及其余异常与中断入口。
- **`Type` + `Flg`**：**`PROGBITS` = 文件里有货，`NOBITS` = 只登记不给货**。`.bss` 是 `NOBITS`：它的 `Off` 甚至是 `000000`，文件里根本没有它的位置，只有一张"请清零"的便条（C1 的第一课）。`Flg` 里的 `A`（alloc）是本章最重要的一列：**没有 A 的节，烧录时与你无关**。

blink.elf 有 20 个节，带 `A` 的只有这 5 个。剩下 14 个是给人看的说明书：`.symtab`、`.strtab`、`.shstrtab`、`.comment`、`.ARM.attributes`，以及 **9 个 `.debug_*`**。而 5 个 alloc 节里，`.data` 大小 0、`.bss` 与 `._user_heap_stack` 是 `NOBITS`，**真正在文件里占字节的只有 `.isr_vector` 与 `.text` 两个**。

所以：392 + 268 = **660 字节**，正好是 `size` 的 `text` 列，也正好是 bin 的体积：

```
   text    data     bss     dec     hex filename
    660       0    1536    2196     894 build/blink.elf
```

```
sh probe.sh sect | tail -3     # 顺手看 bin
# bin 只装 ALLOC 段里有货的部分：blink.bin 660 字节
```

34,604 − 660 = 33,944 字节全是调试信息与符号表。`objcopy -O binary` 一刀把它们全扔掉，这也是为什么"烧 bin 才谈第几字节"（见常见坑）。

一个易踩的小坑：`size -A` 的地址列是**十进制**。你会看到 `134218120` 这种数，它就是 `0x08000188`；`536870912` 是 `0x20000000`。别以为是编译出问题了。

第三刀：程序头——装载器的装车清单

```
sh probe.sh seg
```

blink.elf：

```
  Type   Offset    VirtAddr    PhysAddr    FileSiz  MemSiz  Flg Align
  LOAD   0x001000  0x08000000  0x08000000  0x00294  0x00294 R E 0x1000
  LOAD   0x001000  0x20000000  0x08000294  0x00000  0x00600 RW  0x1000

  Segment Sections...
   00     .isr_vector .text
   01     .bss ._user_heap_stack
```

elf_probe.elf（重头戏在第二行）：

```
  LOAD   0x001000  0x08000000  0x08000000  0x00244  0x00244 R E 0x1000
  LOAD   0x002000  0x20000000  0x08000244  0x0000c  0x00020 RW  0x1000
  LOAD   0x000020  0x20000020  0x08000250  0x00000  0x00600 RW  0x1000

  Segment Sections...
   00     .isr_vector .text
   01     .data .bss
   02     ._user_heap_stack
```

这张表才是"装车清单"：

- **一个 `LOAD` = 一批要一起安排到某个地址的东西**。第一个 LOAD 把 `.isr_vector` + `.text` 装进 `0x08000000`，`FileSiz == MemSiz == 0x244`：Flash 里的货有多少，内存里就摆多少，一比一。
- **第二个 LOAD 是全章最精彩的一行**：`VirtAddr=0x20000000`（住址在 RAM），`PhysAddr=0x08000244`（行李在 Flash 的 `.text` 尾巴后面）。**`FileSiz=0x0c` 而 `MemSiz=0x20`**：文件里给 12 字节，RAM 里要 32 字节，差的 20 字节是 `.bss`（`0x14`），文件里不给货，装载者（这里就是启动代码）负责清零。**"有初值才占 Flash，为 0 只占 RAM"这条规则，在 ELF 里就是这两个数的差。**
- **第三个 LOAD** `FileSiz=0`：堆栈预留区，纯粹"分房不带行李"。它的 `Offset=0x000020` 是个提示值，别当真：`FileSiz` 为 0 时文件偏移没有意义。
- **`Flg`：`R E` = 只读可执行（代码），`RW` = 可写（数据）**。想验证有没有把常量错放进可写区，看这里最快。

- **section→segment 映射**：`.data` 与 `.bss` 被并进**同一个** LOAD，因为它们共享一次搬运（搬完 `.data` 顺手清 `.bss`），这正好解释了 `Reset_Handler` 里为什么是两个紧挨着的循环（第五刀见）。

对回源码，链接脚本就那几行（`stm32f407xx.ld`）：

```
 91:  _sidata = LOADADDR(.data);
 95:  _sdata  = .;
 99:  _edata  = .;
100:  } >RAM AT> FLASH
```

`>RAM` 给 VMA，`AT> FLASH` 给 LMA。`_sidata` 于是等于 `0x08000244`：**程序头里的 `PhysAddr` 和符号 `_sidata` 是同一个数的两种写法**，本章实测两者一致。

顺手把 `objdump -s` 与 `readelf -x` 也叫出来，看看 Flash 里那 12 字节的行李长什么样：

```
Contents of section .data:
 20000000 c5ab3412 68656c6c 6f000000           ..4.hello...
```

`c5ab3412` 是 `0x1234ABC5`（`int g_init`，小端倒着写），`68656c6c 6f00` 是 `"hello\0"`，末尾两个 `00` 是补齐对齐。**左边这列地址是 VMA（`0x20000000`），不是它在文件里的位置**。想找那份镜像，得回程序头读 `PhysAddr`。工具默认给你住址，这是新手最容易上当的一处。

全景动画：同一份文件，两副目录

第二刀与第三刀读的是同一个 `elf_probe.elf`：中间那条带子是文件里真实的字节顺序，上面挂节表、下面挂程序头。动画四个阶段依次演示：**两套目录各自连到文件的哪一块、节表怎么读**（`Addr` 与 `Off` 差 `0x1000`、`NOBITS` 只登记不给货）、**程序头怎么读**（第二条 LOAD 的 `FileSiz=0x0c` 与 `MemSiz=0x20` 之差正是 `.bss`）、**`objcopy` 那一刀切掉了什么**（换 `blink.elf` 看：20 个节只有 5 个带 `A`，进芯片的只有 660 字节）。

ELF 的两副目录动画

第四刀：符号表——nm 一排序就是地图

```
sh probe.sh sym
```

`nm -n --print-size build/elf_probe.elf`（省掉 91 行中断别名）：

```
08000000 00000188 r g_pfnVectors
08000188 00000012 t sink
0800019c 0000003c T main
080001d8 00000042 W Reset_Handler
08000238 00000002 t Default_Handler
0800023c 00000008 T tag
08000244 00000000 T _etext
08000244 00000000 A _sidata
20000000 00000004 D _sdata
20000000 00000004 D g_init
20000004 00000006 D g_msg
2000000c 00000004 B _sbss
2000000c 00000004 B g_zero
20000010 00000010 B buf
20000020 00000004 B _ebss
20020000     00000000 R _estack
```

按地址一排序，固件立刻变成地图：

- **`0x08000000 → 0x08000244` 是 Flash**：向量表 392 字节、`sink` / `main` / `Reset_Handler` 依次排开、`tag`（`const` 表）被链接脚本并进了 `.text`，最后 `_etext` / `_sidata` 落在同一个地址 `0x08000244`：**代码的终点就是数据镜像的起点**，一个字节都没浪费。
- **`0x20000000 → 0x20000020` 是 RAM 的已命名部分**：`_sdata` / `g_init` / `g_msg`，紧接着 `_sbss` / `g_zero` / `buf` / `_ebss`。`_edata` 与 `_sbss` 相等（C1 里那对"接力棒"符号），搬运与清零的边界就交接在这里。
- **`0x20020000` 是 `_estack`**：RAM 顶，栈从这里往下长。它与 `_ebss` 之间那一大片空地，就是还没人碰过的自由栈/堆空间。
- **字母的含义比你想的少**：`T/t` 是 text 段（大写全局、小写局部），`D/d` 是 `.data`，`B/b` 是 `.bss`，`R` 是只读，`A` 是绝对值符号（链接脚本里 `=` 出来的那些）。但 `tag` 明明只有读的分量却标成 `T`，因为本工程的 `.rodata` 并进了 `.text`。**字母说的是"在哪个输出段、什么属性"，不是"是代码还是数据"。要看真相，看地址。**

再看弱符号：blink.elf 里有 **91 个** 符号指向同一个地址：

```
sh probe.sh sym | grep -A1 弱
# 指向 0x080001e8 的 W 别名：91 个
```

```
080001e8 W ADC_IRQHandler
080001e8 W BusFault_Handler
...
```

它们全是 `.globl xxx_IRQHandler; .set xxx_IRQHandler, Default_Handler` 造出来的**弱别名**，真正住在 `0x080001e8` 的是那条 2 字节的死循环 `Default_Handler`（`b.n .`，就地打转等调试器）。**这就是"你没写 `USART1_IRQHandler`，中断来了程序不会跑飞、只会卡在这里"的机理**（弱符号与覆盖规则回到 C5 函数指针与 B4 启动文件）。

最后把那个 ±1 钉死，两个工具同问一个人：

```
   19: 080001e9     2 FUNC  LOCAL  DEFAULT  2 Default_Handler
   78: 08000189    66 FUNC  WEAK   DEFAULT  2 Reset_Handler
  106: 0800020d   136 FUNC  GLOBAL DEFAULT  2 main
```

`nm` 说 `Reset_Handler` 在 `08000188`、`main` 在 `0800020c`；`readelf -s` 说 `08000189`、`0800020d`。**`readelf` 保留 Thumb 位、`nm` 抹掉它**。所以：往向量表里填函数地址时那位 `|1` 必须有；而在链接脚本/map 文件里看到的地址通常不带。差 1 不是 bug，是两位工具的不同口径。

第五刀：反汇编——把固件慢放

```
sh probe.sh dis
```

先看 `main` 的三条开头（`blink.elf`，`-O0`）：

```
0800020c <main>:
 800020c:  b580        push {r7, lr}
 800020e:  af00        add  r7, sp, #0
 8000210:  4b19        ldr  r3, [pc, #100]  @ (8000278 <main+0x6c>)
 8000212:  681b        ldr  r3, [r3, #0]
 8000216:  f043 0320   orr.w r3, r3, #32
```

- `push {r7, lr}` + `add r7, sp, #0` 就是 C6 栈帧讲的 prologue，**源码里看不到，机器码里跑不掉**。
- `orr.w r3, r3, #32` 里的 `#32` 就是源码里的 `1UL << 5`（`RCC_AHB1ENR_GPIOFEN`，bit5）。**十六进制/十进制的转换是编译器干的活**，这也是你日后在 GDB 里认指令的老本行。
- 注意 `ldr r3, [pc, #100]`：**指令里没有 `0x40023830` 这个外设地址**。它躺在 `main` 尾巴后面：

```
 8000278:  40023830   .word 0x40023830
 800027c:  40021400   .word 0x40021400
 ...
 800028c:  40021418   .word 0x40021418
 8000290:  001e8480   .word 0x001e8480
```

这叫**字面量池（literal pool）**：Thumb 的 16 位指令塞不下 32 位常量，编译器就把常量就近放在代码段里，用 `pc` 相对寻址取。`0x40021400/404/408/40c/418` 正是 `GPIOF` 的 `MODER/OTYPER/OSPEEDR/PUPDR/BSRR` 五个寄存器，`0x001e8480` 是 `delay(2000000)` 的那个 2000000（`0x1E8480`）。**它们都在 `.text` 里，占的是 Flash。** 想知道你的常量到底花在哪，`objdump -d` 比 `size` 更诚实。

再看 `Reset_Handler`（`elf_probe.elf`，B4 的主角；本章只点出它和第三节的关系）：

```
080001d8 <Reset_Handler>:
 80001d8:  f8df d040   ldr.w sp, [pc, #64]   @ 800021c   → 0x20020000
 80001dc:  4810        ldr  r0, [pc, #64]    @ 8000220   → 0xe000ed88
 80001e0:  f441 0170   orr.w r1, r1, #15728640  @ 0xf00000
 80001f8:  58d4        ldr  r4, [r2, r3]
 80001fa:  50c4        str  r4, [r0, r3]
 80001fc:  3304        adds r3, #4
 8000214:  f7ff ffc2   bl   800019c <main>
 ...
 800021c:  20020000    .word 0x20020000   @ _estack
 8000220:  e000ed88    .word 0xe000ed88   @ SCB->CPACR
 8000224:  20000000    .word 0x20000000   @ _sdata
 8000228:  2000000c    .word 0x2000000c   @ _edata
 800022c:  08000244    .word 0x08000244   @ _sidata
 8000230:  2000000c    .word 0x2000000c   @ _sbss
 8000234:  20000020    .word 0x20000020   @ _ebss
```

第三节那两个看起来神秘的地址，最后就是在这里被读进寄存器的：`0x20000000`（VMA，目的地）、`0x08000244`（LMA，出发地）、`0x2000000c`（`_edata`，终点），`ldr r4,[r2,r3] / str r4,[r0,r3] / adds r3,#4` 就是一个字一个字往 RAM 里搬。搬完再照着 `_sbss → _ebss` 清一遍，然后 `bl main`。顺带看见 `0xe000ed88 = SCB->CPACR`、`#0xf00000` = CP10/CP11 全权限（`startup_stm32f407xx.s:42`），**硬浮点在 `main` 的第一行代码之前就已经开好了**，这正是 ELF 头里 `hard-float ABI` 敢写死的原因。

最后是本章最"惊悚"的一幕，`objdump -d` 会顺手把数据当代码反汇编：

```
08000238 <ADC_IRQHandler>:
 8000238:  e7fe   b.n  8000238 <ADC_IRQHandler>
0800023c <tag>:
 800023c:  434d 0055 0000 0000    MCU.....
```

`tag` 是 `const uint8_t tag[8] = {'M','C','U',0,...}`，**它就住在 `.text` 里**。反汇编器照着 Thumb 的两字节规则切成 `434d 0055`，你看到的是"MCU....."，这不是乱码，是**你把数据读成了指令**。以后在 `objdump -d` 里看见成片的 `.short 0x0000` / "undefined instruction"，先怀疑那是字面量池或常量表，而不是编译器坏了（回 C1 的 `.rodata` 并入 `.text` 那一节）。

实物实验：一个 8 字节数组，加不加 const

```
cd code/toolchain/02-elf
sh probe.sh tag
```

同一份 `elf_probe.c`，只把 `tag` 前面的 `const` 拿掉（`-DTAG_RAM`）：

实测项	`const uint8_t tag[8]`	`uint8_t tag[8]`
`nm` 里的 `tag`	`0800023c 00000008 T tag`	`2000000c 00000008 D tag`
`.text`	188	**180**（少 8）
`.data`	12	**20**（多 8）
`.bss`	20	20
第二个 LOAD	`VirtAddr=0x20000000 PhysAddr=0x08000244 FileSiz=0x0000c MemSiz=0x00020`	`VirtAddr=0x20000000 PhysAddr=0x0800023c FileSiz=0x00014 MemSiz=0x00028`

读这张表：加 `const` 之后，`tag` 从 `D` 变 `T`、住址从 RAM 的 `0x2000000c` 搬到 Flash 的 `0x0800023c`；`.data` 从 20 缩回 12、`.text` 从 180 涨到 188。程序头第二行的 `FileSiz` 也从 `0x14`(20) 回到 `0x0c`(12)、`MemSiz` 从 `0x28`(40) 回到 `0x20`(32)。

净效果：省掉 8 字节 RAM（那 8 字节的本体不再需要在 RAM 里存在），Flash 总量一格没变。 一个 `const` 字，买的是"别再往 RAM 搬一遍"，这正是 C1 用 256 字节表算过的那笔账，在 ELF 结构层面的样子。

附录：工程完整源码

五刀切的那具标本本体（正文所有地址、大小、字母都出自它和 `../../stm32/00-blink`）：

<<< ../../code/toolchain/02-elf/elf_probe.c

一条命令跑完五刀（正文每条命令的包装、`-DTAG_RAM` 对照也在这里）：

<<< ../../code/toolchain/02-elf/probe.sh

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **头说我是谁，节说我有几抽屉，程序头说怎么装车；`A` 标记之外全是说明书，`FileSiz` 与 `MemSiz` 的差就是启动代码的工作量。**

> ⚠️ **注意 · 那个 1**
>
> `readelf` 给你 `0x8000189`，`nm` 给你 `0x08000188`。**差 1 的是 Thumb 位**：往向量表和函数指针里填地址时必须带 1，工具之间对照时必须先抹平。

常见坑

- **把文件偏移当内存地址**：`.text` 的 `Addr=0x08000188`、`Off=0x001188`。谈"烧到第几字节"要用 bin 的偏移（`.text` 在 bin 里就是第 392 字节起），谈"程序跳到哪"要用 VMA。两套口径混用，就会把断点打在数据上。
- **拿 `.elf` 的体积估固件体积**：`blink.elf` 34,604 字节 vs 烧进去 660 字节。**看 `size` 的 `text`+`data` 估 Flash、`data`+`bss` 估 RAM**（blink：Flash 660，RAM 0+1536）；`dec` 是两者相加，别当成 Flash 占用。B5 会把这笔账算到 map 文件级别（B5 体积从 map 倒推）。
- **只盯 `.text` 就下结论**：`.rodata` 在本工程被并进 `.text`（`stm32f407xx.ld:48-49`），常量、字符串、跳表都藏在里面。`objdump -d` 看到 `.word` 成串才是真相（C1 的 `.rodata` 单列实验会告诉你字母为什么会变）。
- **"反汇编出现 undefined instruction"**：多半是数据被当代码（本章的 `MCU.....` 就是现场）。要读原始字节，用 `objdump -s -j .text` 或 `readelf -x`，别用 `-d`。
- **符号找不到就先怀疑编译档**：函数里的 `int auto_var` 在符号表里查无此人，它活在栈上，链接期根本不存在（C1 已实测）。同理，`static` 局部变量有符号、`auto` 局部变量没有。

短自测

1. `blink.elf` 有 34,604 字节，烧进芯片只有 660 字节。差额是什么？为什么 `size` 命令的 `text`+`data` 才是 Flash 占用？

> ▼ **参考答案**
> 差额是调试信息（`.debug_*` 段）和符号表，它们写给活人看，不进芯片。`size` 的 `text` 含 `.text`+`.rodata`（代码与常量），`data` 含 `.data`（有初值全局变量），两者之和才是真正占 Flash 的字节；`bss` 只占 RAM 不占 Flash。

2. `.data` 的 `VirtAddr=0x20000000`、`PhysAddr=0x08000244`。启动代码在进 `main` 之前必须做什么？搬多少字节？

> ▼ **参考答案**
> 启动代码必须把 `.data` 的初值从 Flash（LMA=0x08000244）拷贝到 RAM（VMA=0x20000000），搬运字节数 = `FileSiz`（不是 `MemSiz`：`MemSiz` − `FileSiz` 的部分是 `.bss`，用 memset 清零而非拷贝）。这段逻辑在启动文件的 `LoopCopyDataInitDone` 标签处。

3. `readelf -h` 显示入口 `0x8000189`，`nm` 显示 `Reset_Handler` 在 `0x08000188`。差 1 是谁错了？在 GDB 里设断点该用哪个地址？

> ▼ **参考答案**
> 都没错。Cortex-M 只跑 Thumb 态，函数地址的最低位（bit 0）是 Thumb 状态标志位，不是地址位。ELF 入口地址保留了这一位（`0x...9` = `0x...8 | 1`），告诉 CPU "跳过去之后用 Thumb 指令集"。`nm` 显示的是纯地址 `0x...8`。在 GDB 里设断点用 `break *0x08000188`（纯地址），但往向量表或函数指针里填值时必须带 1。

延伸阅读

两副目录的原始定义与两本教科书：

- **[C3]** TIS ELF 1.2 — 节头表/程序头表的原始规范，二十来页讲完。
- **[E2]** Levine《Linkers and Loaders》第 3 章 · **[E3]**《程序员的自我修养》第 3 章 — ELF 结构的教科书式讲法，中英各一本。

你做到了

- 五刀拆完一具真固件：ELF 头 → 节表 → 程序头 → 符号表 → 反汇编；
- 能读出 `FileSiz` / `MemSiz` 之差背后的启动工作量，并认出那对 VMA/LMA 是从链接脚本哪一行来的；
- 见过 Thumb 位、字面量池、91 个弱别名，以及"数据被反汇编成指令"的现场。

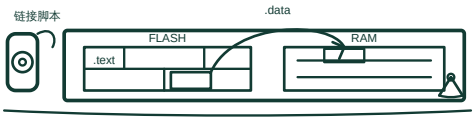
开场那笔「烧进去的只有几百字节、文件却有几十 KB」的差额账，你现在能逐项报出明细：哪些是写给链接器的目录，哪些是给装载器的，哪些根本不进芯片。那份「写给活人看的说明书」从此是你能查的档案，不是玄学。

✅ 下一站：B3 链接脚本。本章读了 `_sidata = LOADADDR(.data)` 与 `AT> FLASH` 兑现出来的地址；下一站逐行讲图纸是怎么规定这些地址的，包括那个把 `.rodata` 并进 `.text` 的决定。

通往单片机之路第 13 章

B3 链接脚本：固件的建筑图纸

难度 ★★★☆☆ · 约 45 分钟



本章罗盘

问题

只删掉 AT> 一个词，链接烧录全通过、初值却成了随机数，链接脚本到底跟谁签了哪几份契约？

交付

六刀可复跑取证：删 KEEP 后 bin 少 392 字节、删 AT> 后 _sidata==_sdata、CCM 段三态 bin 胀到 134MB 再收回 576 字节，附 RAM 手算与 overflowed 验算。

边界

图纸与启动文件是一套榫卯：链接脚本单方面开 .ccmram 房间、启动文件没有第二条拷贝循环，初值照旧是垃圾，『图纸对』推不出『运行对』；板上现象未实测，结论停在链接期与文件期。

依据锚点 `c7` · `e2` · `e3` · `mcu-code` · `mcu-anim`

> 🎯 删掉一个词（`AT> FLASH` 里的 `AT>`），固件照样链接成功，烧进去照样跑，只是每个初值都成了随机数。图纸的力量不在语法，在契约。

本章全部数字出自同一套工具链的实测：xPack GNU Arm Embedded GCC **15.2.1 (20251203)**，binutils **2.45.1**。图纸是 `code/stm32/00-blink/stm32f407xx.ld`（136 行），六刀由 `code/toolchain/03-linker/probe.sh` 现场切，每刀只用 `sed` 改一行，diff 打在输出里，原文不动，你在自己机器上复跑得到同一串字节。

本章精髓

1. **图纸产出的不是代码，是三份地址契约**：`MEMORY` 圈地，规定哪块地能住人；`SECTIONS` 分房，规定谁住哪间、谁住头排；符号门牌（`_estack` / `_sidata` / `_sdata` ……）则留给启动代码兑现。契约写错，链接器多半不报错，它只按你说的做，做得很彻底。
2. **一个段有两个住址**：VMA（运行时在哪）与 LMA（初值存在哪）。`>RAM AT> FLASH` 这一句在 ELF 里兑换成第二条 LOAD；`objcopy -O binary` 只认 LMA，所以少写一个词，bin 能从 660 字节变成 **134,217,744 字节**。
3. **图纸能在链接期就拦下运行期事故**：`._user_heap_stack` 那一间一个真实字节都不占，它只是向 RAM 借的额度。`region 'RAM' overflowed by 4616 bytes` 出现在构建窗口里，好过出现在客户手里。

- **能理解**：六刀各改一个词，输出为什么长成那样，每段下面都贴了实测文本，可以逐字核。
- **能用**：给自己的变量加一间自定义段（本章用 F407 的 64KB CCM），并知道加完还必须补哪一段启动代码，否则初值到不了。

先修

- B2 ELF 解剖（节表 / 程序头 / VMA 与 LMA 的分家，本章负责讲清"是谁规定它们的"）
- C1 内存模型（一个全局变量的三段旅程）
- B4 启动过程（本章的门牌由谁来读。顺序反了也不影响，两边互相引用）

先跑起来（10 分钟 quick win，不用开发板）

先别急着跑：这一刀要删掉链接脚本里的 KEEP。押两问：删完链接器会不会报错？固件变大还是变小？押完再看两份尺寸。

```
sh code/toolchain/03-linker/probe.sh keep
```

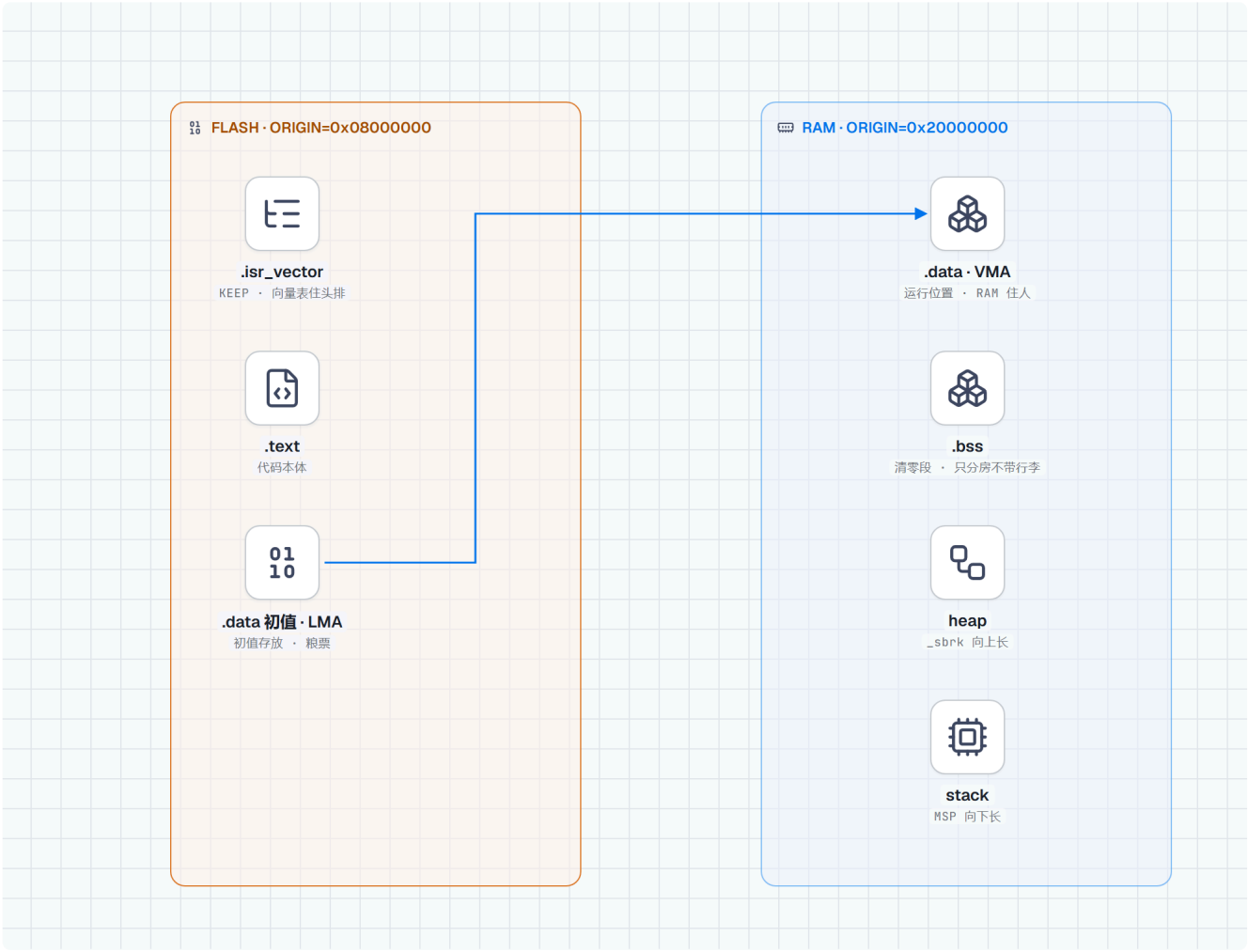
盯住输出里这两行：

```
blink.bin  = 660 字节
nokeep.bin = 268 字节
差 = 392 字节 = 0x188
```

只把 `KEEP(` 这个字母序列删掉，向量表那 392 个字节就从固件里蒸发了，而且 `readelf -S` 还给你留一个尺寸 0 的段名当遗照。这一刀是本章最短的一条命令、最长的一节课。

图纸全貌：136 行，三段结构

三份契约先看图：MEMORY 圈地、SECTIONS 分房、符号留门牌——.data 的双住址是全图的主角。



链接脚本三契约：FLASH 与 RAM 两块圈地，.data 的双住址连线

```
stm32f407xx.ld
├── 契约头        :13-20   ENTRY / _estack / 两份最小预算
├── MEMORY 圈地 :22-29   FLASH 1MB @0x0800_0000、RAM 128KB @0x2000_0000
└── SECTIONS 分房 :31-136  .isr_vector → .text → .data → .bss → ._user_heap_stack → 收尾
```

小节结构

小节	内容	四件套
MEMORY 地块	F407 的三块地（FLASH/RAM/CCM）；属性字母只有 `r w x a i l`；map 的 `Memory Configuration` 回显	配置
向量表落位	头排 + `KEEP` 的分工：一个管地址，一个管生死	配置
.data 的双地址	`>RAM AT> FLASH` 逐词讲；map 里 `load address` 那一行；程序头三条 LOAD	配置
符号界碑	`_estack/_sidata/_sdata/_edata/_sbss/_ebss` 六个门牌在反汇编字面量池里的真身	代码分析
自定义段实战	`.ccmram` 三态：orphan 撒谎 / 128MB 空洞 / `AT>` 收场 / 启动文件不搬	代码分析
栈堆预算	`._user_heap_stack` 检查段与 `overflowed by N bytes`	配置
取证六刀	每刀的 `sed` diff + 原文报错，全部 `probe.sh` 可复跑	代码分析

逐行读：契约头与 MEMORY

```
ENTRY(Reset_Handler)                        /* :13 */
_estack = ORIGIN(RAM) + LENGTH(RAM);        /* :16 */
_Min_Heap_Size  = 0x200;                    /* :19 */
_Min_Stack_Size = 0x400;                    /* :20 */
```

- `ENTRY(Reset_Handler)` 只对**链接器**说话：它决定 ELF 头里的 `Entry point address`，也决定 `-u` / `--gc-sections` 的根集合之一。而**芯片不看这一行**：上电后硬件从 `0x08000000` 连读两个字：第一个当初始 SP，第二个当复位 PC（B2 第五刀 拆过那条向量表，B4 讲它的来源）。你把 bin 烧进芯片，ELF 头连字节都不剩。所以"入口"有两套口径：`readelf -h` 的那个是给装载器看的，向量表第 2 项才是给 silicon 的。
- `_estack = ORIGIN(RAM) + LENGTH(RAM)` 算出来是 `0x20000000 + 0x20000 = 0x20020000`，实测 `nm` 给 `20020000 R _estack`，`blink.bin` 的头一个字正是它（第 1 刀会看到它被顶掉）。注意这是**栈顶**，栈向下生长，从 RAM 最后一字节的上一格开始。
- 两份 `_Min_*` 先记的是**额度**，并没有分配空间。稍后 `._user_heap_stack` 用 `. = . + _Min_Stack_Size;` 把这笔预算记在 RAM 的账上（`:122-123`）。

```
MEMORY
{
  FLASH (rxa)  : ORIGIN = 0x08000000, LENGTH = 1024K   /* :24 */
  RAM   (xrw)  : ORIGIN = 0x20000000, LENGTH = 128K    /* :28 */
}
```

- `ORIGIN` / `LENGTH` 两个数错一个，链接出来的就是"盖在别人家的房子"。F407ZGT6 的事实：Flash 1MB 从 `0x0800_0000`（RM0090 存储器映射表，S1 已核过）、SRAM 128KB 从 `0x2000_0000`。
- 括号里是**区域属性**。合法字母只有 `r w x a i l` 六个。老教程（包括 ST 早年的模板）常写 `(xrwah)`，多一个 `h`。旧 binutils 睁只眼闭只眼，2.45.1 直接拒绝：

```
invalid character %c (104) in flags
error: ld returned 1 exit status
```

`104` 就是字母 `h` 的 ASCII 码。那句 `%c` 不是我们抄漏了字，是 binutils 自己的格式串没带参数。**报错原文照抄才有价值**，遇到它别去找"字符 104 是什么"，直接把 `h` 删掉。 - 图纸写完，链接器会在 map 文件里回显它理解的地块：

```
Memory Configuration

Name             Origin             Length             Attributes
FLASH            0x08000000         0x00100000         axr
RAM              0x20000000         0x00020000         xrw
*default*        0x00000000         0xffffffff
```

属性被按固定顺序重排成 `axr`（你写的是 `rxa`），这是**图纸生效的第一处证据**。`*default*` 那行是"没进任何地块的东西"的去向，第 5 刀会用到它。

SECTIONS 第一间：向量表必须住头排

```
  .isr_vector :
  {
    . = ALIGN(4);
    KEEP(*(.isr_vector))     /* :38 */
    . = ALIGN(4);
  } >FLASH
```

- `.` **是"当前地址"，不是"当前偏移"**：在没有 `AT>` 的输出段里，`.` 就是 VMA。`. = ALIGN(4)` 把它抬到 4 字节边界，Cortex-M 的向量表项必须是字对齐的 32 位地址。
- `*` **的两层含义**：`*(.isr_vector)` 读作"所有输入文件里名叫 `.isr_vector` 的段"。输入段名来自启动文件的汇编（`startup_stm32f407xx.s` 里 `.section .isr_vector,"a",%progbits`），图纸与启动文件是**一套榫卯**：一个声明名字，一个认领名字。
- `KEEP()` **管生死**：本工程链接带 `-Wl,--gc-sections`（B2 见过它砍掉 33,944 字节里的哪几刀），GC 的判据是"有没有人引用"。向量表没有被任何代码引用，是**硬件**读它，而硬件不出现在符号表里。`KEEP()` 就是给链接器塞的小抄：这个段不许回收。

第 1 刀：把 KEEP 去掉

```
38c38
<     KEEP(*(.isr_vector))
---
>     *(.isr_vector)
```

后果按三层看：

```
blink.bin  = 660 字节
nokeep.bin = 268 字节
差 = 392 字节 = 0x188
```

```
  [ 1] .isr_vector  PROGBITS  08000000 001000 000188 00  A  0  0  1     ← 正常
  [ 1] .isr_vector  PROGBITS  08000000 00110c 000000 00  W  0  0  1     ← 空壳
```

段名、段号都还在，**尺寸变成 0**，`readelf -S` 不会告诉你出事了。第三层最要命：硬件上电从 `0x08000000` 读的那两个字。

```
blink.bin  头两个字:  20020000 08000189
nokeep.bin 头两个字:  d040f8df 68014810
```

正常版本第一个字是 `_estack`、第二个是 `Reset_Handler | 1`（Thumb 位，B2 那个"差 1"的坑）。被砍掉向量表之后，`.text` 顶到 `0x08000000`，于是芯片把**两条指令的机器码**当成 SP 和 PC 用：`SP = 0xd040f8df`，第一条压栈指令就打死机。`nm` 里 `08000000 r g_pfnVectors` 这个符号也一并消失，**它是你排查"板子毫无反应"时最该先看的一行**。

`.data` 的双地址：一句 `>RAM AT> FLASH`

先把"人在 RAM、货在 Flash"画出来：两个住址、一个搬运工、一个删词即翻车的反例。

.data 的 VMA 与 LMA 动画

```
_sidata = LOADADDR(.data);              /* :91 — 门牌在段之前就发 */
.data :
{
  . = ALIGN(4);
  _sdata = .;                           /* :95 */
  *(.data)
  *(.data*)
  . = ALIGN(4);
  _edata = .;                           /* :99 */
} >RAM AT> FLASH                        /* :100 */
```

逐词读：`>RAM` 说**运行时住在 RAM**（VMA），`AT> FLASH` 说**初值行李寄存在 Flash**（LMA）。这一句把两个住址绑起来。`LOADADDR(.data)` 取 LMA，所以 `_sidata` 指明"去 Flash 哪里取货"。

实测两副样子。带 `AT> FLASH` 的基线，程序头三条：

```
  LOAD  0x001000  0x08000000  0x08000000  0x00218  0x00218 R E
  LOAD  0x002000  0x20000000  0x08000218  0x00004  0x01008 RW
  LOAD  0x000008  0x20001008  0x0800021c  0x00000  0x00600 RW
```

第二行 `VirtAddr=0x20000000 / PhysAddr=0x08000218 / FileSiz=4 / MemSiz=0x1008`：`FileSiz` 只有 4 字节（标本里 `g_live` 一个 int），`MemSiz` 是 4104：这就是 B2 说的"差值是启动代码的工作量"，本页补上"差额从哪来"。`nm` 给 `08000218 A _sidata`。map 文件里那一行更是直说：

```
.data   0x20000000   0x4  load address 0x08000218
```

第 2 刀：删掉 `AT> FLASH`

```
100c100
<   } >RAM AT> FLASH
---
>   } >RAM
```

```
  LOAD  0x001000  0x08000000  0x08000000  0x00218  0x00218 R E
  LOAD  0x002000  0x20000000  0x20000000  0x00004  0x01608 RW
```

三条 LOAD 塌成两条，`.data` 的 `PhysAddr` 变成 `0x20000000`，和 `VirtAddr` 一模一样。符号表随之变成：

```
20000000 D _sdata
20000004 D _edata
20000000 A _sidata        ← 本来是 0x08000218
```

启动文件那个循环（`ldr r0,=_sdata / ldr r1,=_edata / ldr r2,=_sidata`，B4 逐条讲过）于是"把 `_sdata` 拷到 `_sdata`"，一个字节也没搬。而 Flash 里那 4 个初值字节仍在，**它们只是没人来取的快递**。上电瞬间 RAM 是随机值，`g_live` 读到多少全看运气和温度。

这个 bug 的恶心之处：链接不报错、烧录不报错、`readelf -h` 漂亮、多数时候程序"看起来在跑"（RAM 恰好残留了上次的值）。**判据只有一条：`nm` 里 `_sidata != _sdata`。**

符号界碑：六个门牌在机器码里的真身

`_estack/_sidata/_sdata/_edata/_sbss/_ebss` 不是给 C 用的，是给启动文件用的。链接完成后它们会被"物化"进 `Reset_Handler` 的字面量池。实测（`objdump -d build/base.elf`，标本为 `link_probe.c`）：

```
 80001cc:  .word 0x20020000    ← _estack（第一条 ldr.w sp,[pc,#64] 直接写进 SP）
 80001d0:  .word 0xe000ed88    ← SCB->CPACR，使能 FPU（B4 的第 0 步）
 80001d4:  .word 0x20000000    ← _sdata
 80001d8:  .word 0x20000004    ← _edata
 80001dc:  .word 0x08000218    ← _sidata
 80001e0:  .word 0x20000004    ← _sbss
 80001e4:  .word 0x20001008    ← _ebss
```

把上面字面量池里的值和 `nm` 逐项对照，一个不差。接着看这三个细节：

- `_sbss == _edata == 0x20000004`：`.bss` 紧贴在 `.data` 后面，所以清零循环的起点是"搬运循环的终点"，两个循环共用一个指针推进方式（`str.w r3,[r2],#4` 那条自增存）。
- `_ebss = 0x20001008 = _sbss + 0x1004`，`0x1004 = 4100` 就是 `g_dark + g_big[4096]`。
- 这些是**链接期算出来的常量**，不是运行期查询。改图纸、加数组后，池子里的字面量会跟着变。启动文件使用这些值，不需要先"知道"你有几个全局变量。

第 3 刀：图纸在链接期拦住你

先量一次账（`link_probe.c` 的标本，原图纸）：

```
Memory region         Used Size  Region Size  %age Used
           FLASH:         540 B         1 MB      0.05%
             RAM:        5640 B       128 KB      4.30%
```

5640 是能手算的：`.data` 4 + `.bss` (4 + 4096) + `._user_heap_stack` (0x200 + 0x400 = 1536) = 5640。把这一格记住，下面两个报错就能自己验算。

3a：`sed 's/LENGTH = 128K/LENGTH = 2K/'`，只改地块大小，C 代码一个字没动：

```
build/tinyram.elf section `.bss' will not fit in region `RAM'
region `RAM' overflowed by 3592 bytes
error: ld returned 1 exit status
```

`5640 − 2048 = 3592`，与 ld 报的数一字不差。ld 先点名段（`.bss`），再说总量：**先看点名，就知道该减谁的量**。

3b：把数据全删干净，只把栈预算从 `0x400` 撑到 `0x20000`：

```
build/bigstack.elf section `._user_heap_stack' will not fit in region `RAM'
region `RAM' overflowed by 4616 bytes
             RAM:      135688 B       128 KB    103.52%
```

点名的 `._user_heap_stack` 里**一个真实字节都没有**，纯粹是 `. = . + _Min_Stack_Size;`（`:122-123`）立起来的界碑。它换来的是：栈不够用这件事，在构建窗口里以 `103.52%` 的形式失败，而不是在客户手上以踩内存的形式失败。`135688 − 131072 = 4616`，同样能手算。

第 5 刀：CCM 的三态（本章最值钱的一刀）

F407 有第三块地：64KB CCM RAM @ `0x1000_0000`（S1 的总线矩阵动画里那个"DMA 够不着、只有 CPU 能到"的房间）。标本 `link_probe.c` 加 `-DWITH_CCM` 会多一个成员：

```
__attribute__((section(".ccmram"), used)) int g_ccm[4] = { 1, 2, 3, 4 };
```

态一：只改 C，图纸没留房间

用原图纸链接，`objdump -h` 的第三行：

```
  3 .ccmram  00000010  20000004  08000230  00002004  2**2
```

VMA 是 `0x2000_0004`：**它就在普通 SRAM 的第 5 个字节**，跟 CCM 毫无关系。ld 一声不响（orphan 段被按 `*default*` 的规矩随手安置了）。你在 C 里写 `g_ccm`，编译器满意，链接器满意，只有你对"它在 CCM"这件事是误解。这一态的杀伤力在于：**测不出来**，它明明能用，只是不在你以为的地方（于是 DMA 能不能访问、速度多少，全都与预期不符）。

态二：图纸加了房间，但只写 `>CCM`

```
  CCM   (rw)   : ORIGIN = 0x10000000, LENGTH = 64K
  ...
  .ccmram : { _siccm = .; _sccm = .; KEEP(*(.ccmram)) _eccm = .; } >CCM
```

程序头多出第四条：

```
  LOAD  0x003000  0x10000000  0x10000000  0x00010  0x00010 RW
```

`VirtAddr = PhysAddr = 0x10000000`：没有 `AT>`，LMA 就跟着 VMA 走。现在 `objcopy -O binary` 面对的是"要把货物放在 0x1000_0000，而镜像从 0x0800_0000 开始"，它老实照办、一声不响地补零：

```
objcopy  exit=0
bin 尺寸 = 134217744 字节
```

`0x10000000 − 0x08000000 = 134,217,728`，再加 16 字节初值 = **134,217,744**。660 字节的固件写"占用了 Flash 1 MB 的 0.05%"，改一个词的图纸让 bin 膨胀到 128MB，而烧录器会忠实地把这 128MB 往 Flash 里灌。

态三：补上 `AT> FLASH`

同一个位置，`>CCM AT> FLASH`：

```
  LOAD  0x002000  0x10000000  0x0800022c  0x00010  0x00010 RW
bin 尺寸 = 576 字节
```

LMA 回到 Flash（`0x0800022c`，紧贴在 `.text` 之后），bin 从 134,217,744 收回 **576 字节**：`0x0800023c + 4 = 0x240 = 576`，一个字节都不多。**`AT>` 就是把"人在 RAM"和"货在 Flash"这两件事分开写的那半句**，CCM 这种"离群的地块"最能看出它的价值。

态四：可是初值到得了 CCM 吗？

```
startup 里引用过这些符号的次数：_sidata=3 / _siccm=0
```

`Reset_Handler` 的反汇编里只有一个搬运循环和一段清零循环：

```
 80001a8:  ldr  r4, [r2, r3]     ← 从 _sidata 取
 80001aa:  str  r4, [r0, r3]     ← 写进 _sdata
 80001ac:  adds r3, #4
 80001b0:  cmp  r4, r1           ← 到 _edata 为止
 80001b2:  bcc.n 80001a8
```

`_siccm/_sccm/_eccm` 三块门牌立在图纸上，**启动文件里没有一条指令走过去**。即使 LMA 正确、初值躺在 Flash 里等取，也没人来取；上电后 CCM 里那 16 个字节仍是随机值。想让带初值的变量真住进 CCM，得自己在启动文件补第二段拷贝（照抄 `.data` 那四条指令，把三个符号换成 `_siccm/_sccm/_eccm`）。

这就是"图纸与启动文件是一套榫卯"的反面教材：图纸单方面开了房间，施工队不知道。

第 6 刀：让链接器自己交代它干了什么

三条开关，出问题的第一步都该按它们来：

- **`-Wl,--print-memory-usage`**：每块地的账（上面那两个表就是它打的）。RAM 从 `4.30%` 变成 `103.52%` 这类跳跃，一眼就知道是栈预算还是大数组。
- **`-Wl,--trace`**：谁被链进来了。实测前几行：

```
build/startup.o
build/main.o
  …thumb/v7e-m+fp/hard\libgcc.a
  …thumb/v7e-m+fp/hard\libc.a
  …thumb/v7e-m+fp/hard\libm.a
```

`thumb/v7e-m+fp/hard` 这一串目录名就是 **多架构（multilib）选择**：由 `-mcpu=cortex-m4 -mfpu=fpv4-sp-d16 -mfloat-abi=hard` 三个开关共同决定。改成 soft-float 会去另一个目录找库，找不到就报 `cannot find -lgcc`，这明明是装了库的机器上最容易看不懂的一类错，答案在 `--trace` 里一行就能看到。另外 `libc.a` 出现两次不是重复链接：archive 会被反复扫描，直到不再有新的目标文件被拉出来。 - **`ld --verbose`**：不打 `-T` 时它默认用哪张图纸。实测开头：

```
Supported emulations:
 armelf
using internal linker script:
OUTPUT_FORMAT("elf32-littlearm", "elf32-bigarm", "elf32-littlearm")
OUTPUT_ARCH(arm)
ENTRY(_start)
```

内置图纸的入口是 `_start`、且完全没有 `MEMORY` 块，所以裸用 `ld` 不写 `-T`，程序会被塞进默认的 `TEXT / DATA` 布局，跟 0x08000000 没有半点关系。**`-T` 不是可选的美化，是芯片能启动的前提。**

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **链接脚本不产代码，产三份地址契约：`MEMORY` 圈地、`SECTIONS` 分房、符号留门牌；一个段有两个住址：VMA 管运行、LMA 管初值，少写一个 `AT>`，bin 从 660 字节胀成 134MB。**

延伸：.data 的 VMA/LMA 双住址在 B2 ELF 段视图 有动画演示；栈堆的 `_Min_*` 预算段与启动文件配合见 B4 启动文件；Flash 调试时断点与地址关系见 B6。

实物实验

> ⚠️ **注意 · 这一节还没上板**
>
> 下面三条需要接 F407 + ST-Link，本章只做到"链接期与文件期已核实"。**板上现象一律标待回填**，不许拿 `objdump` 的结论冒充眼睛看到的结论。

实验	观测点	预期（工具链侧已核实）	板上
第 1 刀的固件烧进去	串口有无输出、SWD 能否连上	`SP=0xd040f8df`，第一条压栈即 HardFault/锁死	待回填
第 2 刀烧进去	`g_live` 打印值	每次上电不同（RAM 残留）	待回填
第 5 刀态二	烧录器进度条	灌 128MB，耗时以十秒计，且地址越界报错	待回填

安全做法：**用第 6 刀的三条开关在构建期判死**，不要靠"烧进去看看"。链接脚本的错，绝大多数在链接期就该被拦下。

常见坑

按新手实际撞上的频率排：

1. **忘 `KEEP()`，向量表被 gc**：`--gc-sections` 只认引用关系，不认硬件。指纹是 `readelf -S` 里 `.isr_vector` 尺寸 0（段名还在！）、`nm` 查不到 `g_pfnVectors`、bin 突然小 392 字节。
2. **自定义段没在图纸登记就开用**：orphan 段被静默安置到 `0x20000004`，零警告。判据：`objdump -h` 看那个段的 VMA 是不是你预期的地块地址。
3. **`AT>` 漏写**：`.data` 的 LMA 塌进 RAM，`_sidata == _sdata`，初值永远搬不到位。判据一条：`nm | grep _sidata`，它必须是个 `0x0800xxxx`。
4. **区域属性照抄老教程**：`(xrwah) → invalid character %c (104) in flags`。合法字母只有 `r w x a i l`。
5. **改了 `ORIGIN` 忘了改烧录地址**：图纸把代码放 `0x08004000`（bootloader 场景常见），烧录器还在往 `0x08000000` 写，芯片从老地方读栈顶，读到你上一版固件。两侧地址必须一起改。
6. **把 CCM 当普通 RAM**：S8 讲过 DMA 够不着 CCM（总线矩阵只把 CCM 接到 CPU）；本章补上另一半：**图纸没登记 + 启动文件不搬**，所以带初值的 CCM 变量上电是垃圾。DMA 缓冲区放 CCM 是无声失败，初值放 CCM 也是无声失败，两种失败还不一样。
7. **把 `._user_heap_stack` 的报错当成"代码太大"**：它点名的是检查段，问题往往是 `_Min_Stack_Size` 或某个 `.bss` 大数组。先跑 `--print-memory-usage` 看占比。

短自测

1. `.data` 的 VMA 和 LMA 分别是什么？谁读 VMA、谁读 LMA？
2. 删掉 `KEEP(*(.isr_vector))` 之后，为什么 `readelf -S` 还能看到 `.isr_vector` 这个段？
3. 你一个数组都没加，链接却报 `region 'RAM' overflowed by 4616 bytes`，最可能改了图纸哪一行？
4. 带初值的全局变量放进 `.ccmram`，图纸三行都写对了（`MEMORY` 加地块、`SECTIONS` 加房间、`AT> FLASH` 也不漏），上电读到的为什么还是随机值？

▼ **参考答案（先自己想完再展开）**
1. 实测 `VMA=0x20000000`（程序运行时的住址，CPU 的 `ldr` 按它寻址）、`LMA=0x08000218`（初值在 Flash 的寄存地址）。CPU 读 VMA，装载器与 `objcopy -O binary` 读 LMA；`_sidata = LOADADDR(.data)` 取的是 LMA。 2. 因为段名是**输入段被登记过的痕迹**，`--gc-sections` 回收的是段里的**内容**，不是输出段目录里那一行。尺寸 0 的 `.isr_vector` 是一具空壳：`PROGBITS` 还在、`Size` 从 `000188` 变 `000000`。 3. `_Min_Stack_Size`（或 `_Min_Heap_Size`）。检查段只记账不占货，实测：`0x400 → 0x20000` 之后点名 `._user_heap_stack`，`RAM 135688 B = 103.52%`，`135688 − 131072 = 4616`。 4. 图纸单方面开了房间，**启动文件里没有对应的第二条拷贝循环**。实测 `grep -c` 数启动文件符号引用：`_sidata` 出现 3 次、`_siccm` 出现 0 次；`Reset_Handler` 反汇编只有一对 `ldr/str`。初值安安静静待在 Flash，没人来取。

对照表：本章概念 → 仓库落点

本章说的	仓库里哪一行
136 行图纸本体	stm32f407xx.ld（关键行：`:16` `_estack`、`:24/28` MEMORY、`:38` KEEP、`:91` `_sidata`、`:100` `AT> FLASH`、`:117-123` 检查段）
六个门牌被谁读走	`startup_stm32f407xx.s:23-27`（`.global`）、`:48-51`（搬运）、`:64-65`（清零）
六刀与实测输出	`code/toolchain/03-linker/probe.sh`（`keep/at/overflow/flags/ccm/trace`）
标本的三个变量	`code/toolchain/03-linker/link_probe.c`（4 + 4100 + 1536 = 5640 的出处）
向量表第 0/1 项的含义	B2 第五刀、B4 第一节
CCM 只接 CPU	S1 架构总览 的总线矩阵动画、S8 DMA 的翻车实验

你做到了

- 一张 136 行的图纸从天书变成能逐行批改的东西：圈地、分房、留门牌，三件事各管一段；
- 见过三个"链接器不报错但固件是错的"现场：空壳 `.isr_vector`、`_sidata == _sdata`、orphan 到 `0x20000004` 的 `.ccmram`；
- 会用手算复核工具链：`4 + 4100 + 1536 = 5640 → overflowed by 3592`、`0x10000000 − 0x08000000 + 16 = 134,217,744`；
- 知道"图纸与启动文件是一套榫卯"：单方面加房间，机器会安静地给你一个错答案。

开场那句「图纸的力量不在语法，在契约」现在有了三个现场作证：不报错的空壳向量表、不报错的错位门牌、不报错的孤儿段。链接器沉默的地方，正是图纸替你扛住的地方；它扛不住时，就安静地给你一个错答案。

✅ 下一站：B4 启动过程。本章发的六块门牌，看施工队怎么一条条读回去：上电到 `main` 之间的每一条指令。

附录：工程完整源码

标本本体（`.data` / `.bss` / `.ccmram` 三类住户，正文所有字节数出自它和 `../../stm32/00-blink`）：

<<< ../../code/toolchain/03-linker/link_probe.c

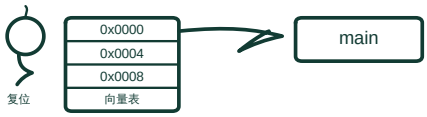
一条命令跑完六刀（每刀的 `sed` diff、报错原文、换算式都在这里）：

<<< ../../code/toolchain/03-linker/probe.sh

通往单片机之路第 14 章

B4 启动过程：上电到 main 的每一条指令

难度 ★★★☆☆ · 约 30 分钟



本章罗盘

问题

从复位向量到 main 之间的每一步，谁配栈、谁搬 .data、谁清 .bss，各是哪段代码在干活？

交付

一张从 0x08000000 取栈指针、取复位向量到进入 main 的时序清单，和一份自写启动文件逐行注解。

边界

本章讲裸机启动；RTOS 启动（调度器先于 main 的场景）在 F 篇另讲，不在本章外推。

依据锚点 a3 · c7 · mcu-code · mcu-anim

> 🎯 按一下复位，到你的 `main()` 跑起来，中间其实只有**十几条指令**。就是它们，完成了栈的准备、FPU 的开门、全局变量的搬家、空房间的清零。这十几条指令，是 C 世界的"接生婆"，值得一条条过目。

本章精髓

1. 硬件只做两件事：从 0 号地址读初始 MSP、从 4 号地址读 PC，剩下的全是软件的事（这就是"启动文件"存在的理由）。
2. Reset_Handler 的四道工序：**使能 FPU**（CPACR，硬浮点编译的生死开关）→ 搬 .data（Flash→RAM）→ 清 .bss → 跳 main：C1 的"三段旅程"在这里闭环。
3. Boot 引脚决定"0 号地址是谁"：用户 Flash / 系统 Bootloader / SRAM 三种映射的区别（霸天虎正常就是从用户 Flash 启）。

- **能理解**：指出"全局变量天生有初值"这句话背后，初值存在哪、由谁搬到变量的位置（第三节的搬家现场）。
- **能用**：打开 startup_stm32f407xx.s 逐行讲出每条指令在干什么，并在 GDB 里从复位单步验证。

先修

- B3 链接脚本（五个符号）、C6 调用约定（可读）。

先跑起来（10 分钟 quick win）

`make gdb` 后在 GDB 里：

单步之前先押个数：从复位到进入主函数，中间隔着几条指令？十几、几十、还是几百？押完一条条数过去，看实际数字离你的直觉有多远。

```
monitor reset halt    # 停在 Reset_Handler 第一条指令
x/2xw 0x08000000      # 看向量表头两项：栈顶值 + Reset_Handler 地址
si                    # 一条条单步，info registers sp pc 跟着看
```

动画：复位之后一毫秒

七站流水线循环播放：硬件两读 → 开 FPU → 搬 .data → 清 .bss → bl main。盯紧那三个橙色小方块，它们就是全局变量的初值在"搬家"。

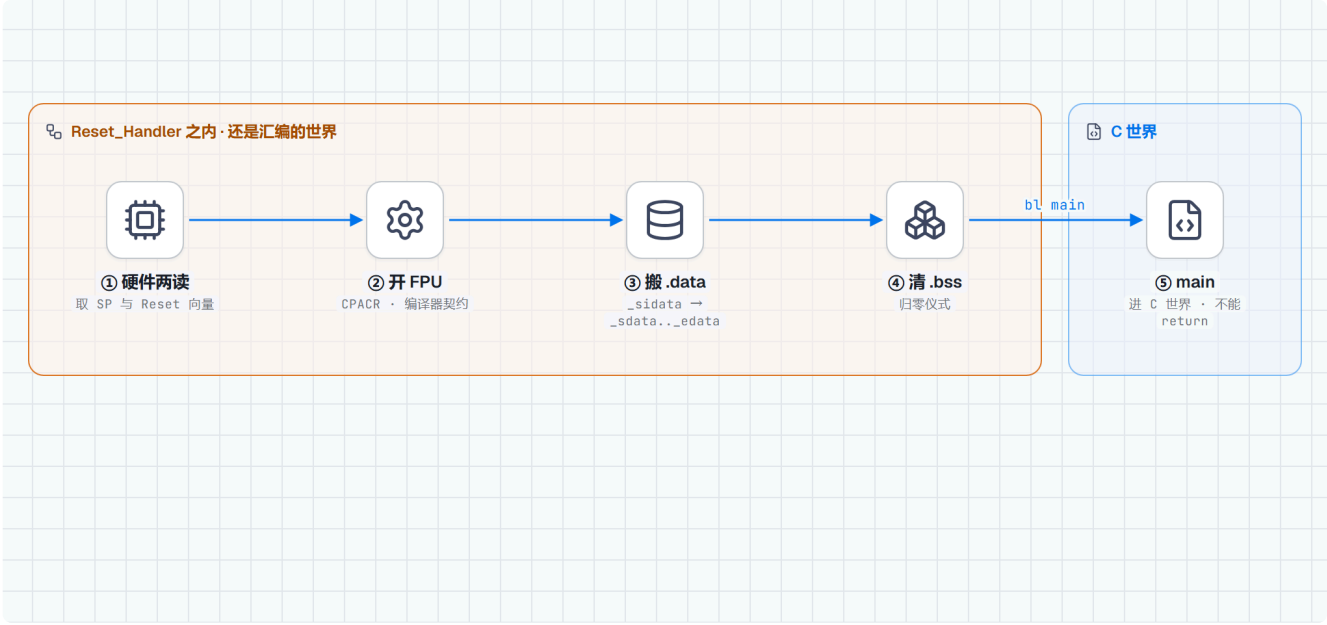
启动时序动画

小节结构

小节	内容	四件套
一、硬件的两读	0 号地址/4 号地址；MSP 与 PC 的分工	配置
二、使能 FPU	CPACR 的 CP10/CP11 置位；不做它，第一条 FPU 指令就 UsageFault(NOCP)	配置
三、搬家 .data	搬运循环五条指令逐条讲	代码分析
四、清零 .bss	C 标准的"零初始化"承诺谁来兑现	代码分析
五、跳 main 与返回	bl main 之后 main return 去哪	代码分析
六、Boot 引脚	BOOT0/BOOT1 与三种地址映射	配置
七、weak 向量表	默认死循环 + 同名覆盖机制	库解析

一、硬件的两读：上电第一纳秒

五站流水线先收进一张图：前四站挤在 Reset_Handler 里，第五站一出去就是 C 世界。



启动五站流水线：硬件两读、开 FPU、搬 .data、清 .bss、bl main

复位松手那一刻，CPU 醒来了：RAM 里是上电垃圾，没有操作系统，没有 C 世界，连栈都没有。硬件（注意，是**硅片里的硬逻辑**，不是任何程序）只会做两件事：

1. 从 **0 号地址**读 4 字节，塞进 **MSP**（主栈指针）；
2. 从 **4 号地址**读 4 字节，塞进 **PC**（程序计数器），开始取指。

"0 号地址是谁"由 BOOT 引脚的映射决定（第六节细讲）；正常情况它就是用户 Flash 0x08000000 的**别名**。所以 Flash 最前面必须躺一张向量表，头两项长这样（摘自我们的启动文件）：

```
g_pfnVectors:
  .word _estack        /* 0x00 初始 MSP */
  .word Reset_Handler  /* 0x04 复位入口 */
  .word NMI_Handler
  .word HardFault_Handler
  /* ...共 16 项内核 + 82 项外设中断（RM0090 §12.2，IRQn 0..81） */
```

> 【注】**为什么第 0 项不是指令而是栈顶值？** 因为 C 函数调用要用栈保存现场，而栈指针得先有着落：硬件先把"拐杖"递给你，才让你走路。顺序不能反。
>
> 【注】**复位后 CPU 用 MSP 跑。** 等 FreeRTOS 上场，任务会改用 PSP，MSP 留给内核与中断，F2 上下文切换会接着讲这条线。

二、第 0 步：使能 FPU——编译器与硬件的契约

Makefile 里写着 `-mfloat-abi=hard -mfpu=fpv4-sp-d16`：这是**编译器的信念**："FPU 随时可用，浮点运算直接生成 FPU 指令"。但复位后硬件的默认状态是：**CP10/CP11（FPU 协处理器）关闭**。编译器的信念和硬件的现实之间，差一道开门手续（PM0214 §4.6.6）：

```
  ldr   r0, =0xE000ED88        /* SCB->CPACR */
  ldr   r1, [r0]
  orr   r1, r1, #(0xF << 20)   /* CP10/CP11 全权限（bit20~23） */
  str   r1, [r0]
  dsb                          /* 数据同步屏障：写必须生效 */
  isb                          /* 指令同步屏障：后续取指重新来 */
```

漏了这一步会怎样？第一条 FPU 指令触发 **UsageFault(NOCP)**——灯不闪、串口无声、没有任何"报错弹窗"，程序悄悄死在 fault handler 里。这是硬浮点工程最阴的坑，所以它是启动的**第 0 步**，排在搬家清零之前（与 ST 官方 SystemInit 同款）。

三、搬家：.data 的 Flash→RAM 之旅

"全局变量天生就有初值"，这是 C 给你的承诺，但硬件不认账。先看两处位置：初值**存**在 Flash（占空间、断电不丢），变量**用**在 RAM（可读写）。把初值从前一处搬到后一处，是启动代码的工作；下面就是负责搬运的循环（启动文件原文）：

```
  ldr   r0, =_sdata        /* RAM 目标起点 */
  ldr   r1, =_edata        /* RAM 目标终点 */
  ldr   r2, =_sidata       /* Flash 源起点 */
  movs  r3, #0
  b     .L_check_copy
.L_copy_loop:
  ldr   r4, [r2, r3]       /* 从 Flash 读一个字 */
  str   r4, [r0, r3]       /* 写到 RAM     */
  adds  r3, r3, #4
.L_check_copy:
  adds  r4, r0, r3
  cmp   r4, r1
  bcc   .L_copy_loop
```

指令	在干什么	为什么
`ldr r0, =_sdata` 等三条	把链接脚本给的三个边界装进寄存器	搬家先得知道：从哪、到哪、搬多长
`movs r3, #0`	r3 = 偏移量，从 0 开始	用"基址+偏移"寻址，一份代码通吃任意长度
`ldr r4, [r2, r3]` / `str r4, [r0, r3]`	读 Flash 一字 → 写 RAM 一字	搬运本体，每次 4 字节
`adds r3, r3, #4`	偏移前进一个字	
`cmp r4, r1` / `bcc`	目标地址到 `_edata` 了吗？没到继续	先判断后干活（`b .L_check_copy` 开局直跳），**长度为零的 .data 也安全**

三个符号都是链接脚本给启动文件用的地址：`_sidata = LOADADDR(.data)` 指向 Flash 中的初值起点，`_sdata` / `_edata` 给出 .data 段在 RAM 的头尾。把这两处位置放在一起看，`>RAM AT> FLASH` 表达的就是"运行在 RAM，初值存 Flash"。

四、清零：.bss 的归零仪式

没写初值的全局变量（`static int cnt;`），C11 标准（§6.7.9）承诺它们是 0。这批变量住 .bss 段，**不占 Flash 一个字节**（初值全是 0，仓库不存货），启动时现场清零：

```
  ldr   r2, =_sbss
  ldr   r4, =_ebss
  movs  r3, #0
  b     .L_check_zero
.L_zero_loop:
  str   r3, [r2], #4       /* 写 0，r2 自动 +4 */
.L_check_zero:
  cmp   r2, r4
  bcc   .L_zero_loop
```

比搬家循环还瘦：源操作数都不需要，一路 `str r3` 扫过去。删掉这段"优化"会怎样？全局变量全是上电垃圾值，bug 玄学到怀疑人生。而且这种 bug **换块板子、换个温度就变样**。

五、进 C 世界与"main 不能 return"

```
  bl    main
.L_hang:
  b     .L_hang
```

`bl main` ——C 世界开门营业。注意第二行：裸机的 `main` **没有命令行可以退回**。它真敢 return，就掉进 `.L_hang` 原地转圈，等看门狗或调试器收尸。所以裸机 main 的标配是 `while(1)`，return 是事故不是设计。

六、Boot 引脚：0 号地址是谁

第一节说"0 号地址由 BOOT 引脚决定"，F4 这里三选一：

BOOT0	BOOT1(PB2)	0 号地址映射到	用途
0	×	用户 Flash（0x08000000）	**日常**：跑你的固件
1	0	系统存储器（0x1FFF0000）	ST 出厂 ISP bootloader：串口/USB DFU 烧录，**救砖专用**
1	1	内部 SRAM（0x20000000）	调试花活：固件下 RAM 里跑，不烧 Flash

霸天虎板子上 BOOT0 跳帽默认接 GND（下拉），所以一上电就跑你的固件。哪天"烧录把芯片锁死了"，把 BOOT0 接高从 ISP 启动，就是留的后门。

七、weak 向量表：同名覆盖的温柔机关

向量表 98 项，绝大多数中断你这辈子都用不到，它们的默认出口全是 `Default_Handler: b .`（原地死循环，调试时一眼能抓到"谁闯进来了"）。机关在于每个默认处理函数都是 **weak 符号**（启动文件尾部 `IRQ_DEFAULT` 宏批量定义）：

你在任何 C 文件里写个同名函数，链接器就悄悄"顶掉"默认实现，不用改启动文件一个字：

```
void SysTick_Handler(void) {  /* 你写的强符号，覆盖 weak 默认 */
    tick++;
}
```

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **上电两读定生死，开门搬家再清零，bl 一声进 main；向量表头不是代码是栈顶，第二条才是入口。**

延伸：启动过程与 IAP 跳转（S13）同构；`.data` 搬运与内存模型（C1）直接对应；调用约定（C6）决定 main 入口的栈帧布局。

实物实验

- 在 main 里定义 `volatile int g = 0x12345678;`，GDB 从复位单步：搬运循环结束后 `x/wx &g` 应为 `0x12345678`。那一刻，就是"搬家完成"的现场。
- 进阶：把 CPACR 那五行注释掉重烧，main 里加一行 `volatile float f = 1.5f * 2.0f;`，观察程序卡死，再用 GDB 读 SCB->CFSR 的 NOCP 位，亲手抓到"门没开"的现场。

常见坑

- **换芯片忘换启动文件**：F407 有 98 项向量（16 内核 + 82 外设），F103 按密度只有 59~76 项（C8 中密度 59，ZE 高密度 76）。用错启动文件，中断号对不上，直接 HardFault 或行为诡异。
- **在 Reset_Handler 之前下断点**：那是硬件行为，断不住。调试从 `monitor reset halt` 开始才对。
- **把 .data 搬运删掉"优化"**：全局变量初值全无，bug 隐蔽到怀疑人生。
- **BOOT0 悬空/接错**：从系统存储器启动了，你的固件"没被运行"（霸天虎 BOOT0 默认下拉，自己飞线时注意）。
- **硬浮点工程忘开 CPACR**：第一条 FPU 指令 UsageFault(NOCP)，静默卡死（本章第二节）。

短自测

1. 向量表第 0 项是什么？为什么不能是指令？
2. 硬浮点工程忘了开 CPACR，运行时会发生什么？
3. .data 的初值存在哪、运行在哪、谁负责搬？
4. BOOT0=1、BOOT1=0 时 CPU 从哪启动？这个模式拿来干嘛？

▼ 参考答案（先自己想完再展开）
1. 初始 MSP 值。没栈就没法保存现场、没法函数调用：硬件先把拐杖递上，才让你走路。 2. 第一条 FPU 指令触发 UsageFault(NOCP)，程序静默卡死，没有任何直观报错。 3. 初值存 Flash（`_sidata`），变量运行在 RAM（`_sdata`..`_edata`），Reset_Handler 的搬运循环负责搬。 4. 系统存储器 0x1FFF0000 的 ST 出厂 ISP bootloader：串口/USB DFU 烧录，救砖后门。

对照表：本章概念 → 仓库落点

本章说的	仓库里哪一行
向量表头两项	startup_stm32f407xx.s `g_pfnVectors` 开头
使能 FPU 五条 + dsb/isb	同文件 Reset_Handler 第 0 步
搬家循环五条指令	同文件 `.L_copy_loop`
`_estack` / `_sidata` / `_sdata` / `_edata`	stm32f407xx.ld 第 16 / 88-97 行
weak 同名覆盖	启动文件尾部 `IRQ_DEFAULT` 宏
Boot 三映射	hardware.md 板卡资料

你做到了

- 上电到 main 的每一纳秒都讲得出来；
- 启动文件从此不是"官方给的神秘文件"，是你可以逐行批改的自己人；
- 再听到"全局变量天生有初值"，你会心一笑：那是十几条指令搬出来的。

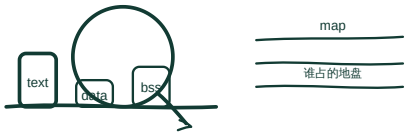
开场那位「接生婆」，你现在能一条条报名字了：硬件的两读、给浮点单元开的门、搬家的循环、清零的循环，最后才轮到你的代码出场。所谓「全局变量天生有初值」，其实是这十几条指令在人没有到场时就把家搬好了。

✅ 下一站：B5 map 与体积：固件 200 字节还是 200KB，这笔账该谁认？

通往单片机之路第 15 章

B5 map 与体积：谁吃了我的 Flash

难度 ★★☆☆☆ · 约 25 分钟



本章罗盘

问题

加了一个库固件从 20KB 涨到 200KB，谁吃的？Flash=text+data、RAM=data+bss 这两笔账凭什么成立？

交付

一次『揪大尾巴』审计记录：size 三列心算 Flash/RAM、nm --size-sort 大户排名、map 三板斧（Archive/Memory Map/Discarded）定位，加 4KB 数组三连改（bss/data/const）的 size 对比。

边界

_Min_Stack 只是链接期下限，运行期栈溢出不报错、其检测不在本章；bin 比 text+data 大是段间空隙按地址铺平所致，不是账错；先砍大项再谈 -Os，方向错了优化级别救不了。

依据锚点 `c7` · `mcu-code` · `mcu-anim`

> 🎯 加了一个库，固件从 20KB 涨到 200KB——谁干的？map 文件就是"固件户口本"：每个符号多大、住哪、谁引进的，一查便知。嵌入式工程师的基本功：**先审账，再优化**。

本章精髓

1. `size` 的三列是三种资源：text（Flash：代码+只读）、data（Flash 存初值+RAM 住人）、bss（纯 RAM）：Flash 占用=text+data，RAM 占用=data+bss。
2. map 文件的三板斧各查一件事：Archive member 表查谁被从库里捞出来，Memory Map 查每个符号的落位， discarded 段查谁被 gc-sections 辞退。
3. 优化的顺序：先看大项（库/表/缓冲区），再谈 -O 级别；方向错了 -O2 也救不了你。

- **能理解**：解释为什么带初值的数组让 Flash 和 RAM 双涨，加 const 后 data 回落、text 涨；再解释为什么 bin 比 text+data 大。
- **能用**：`size` / `nm --size-sort` / `map` 三把刀完成一次"揪出大尾巴"审计。

先修

- B2 ELF、B3 链接脚本。

先跑起来（10 分钟 quick win）

跑之前先押：这个只点灯的小固件，Flash 里体积最大的单个符号会是谁？主函数、启动代码、还是库里的东西？押一个名字，再看按大小排队的队尾。

```
arm-none-eabi-size build/blink.elf
arm-none-eabi-nm -S --size-sort build/blink.elf    # 符号按大小排，倒数五个就是大户
grep -A3 "Discarded input" build/blink.map | more  # 谁被辞退了
```

`size` 的三列用来算 Flash/RAM 占用；要找体积最大的几个符号，就看 `nm --size-sort`；要查 gc-sections 辞退了谁，就看 `map` 的 Discarded 段。

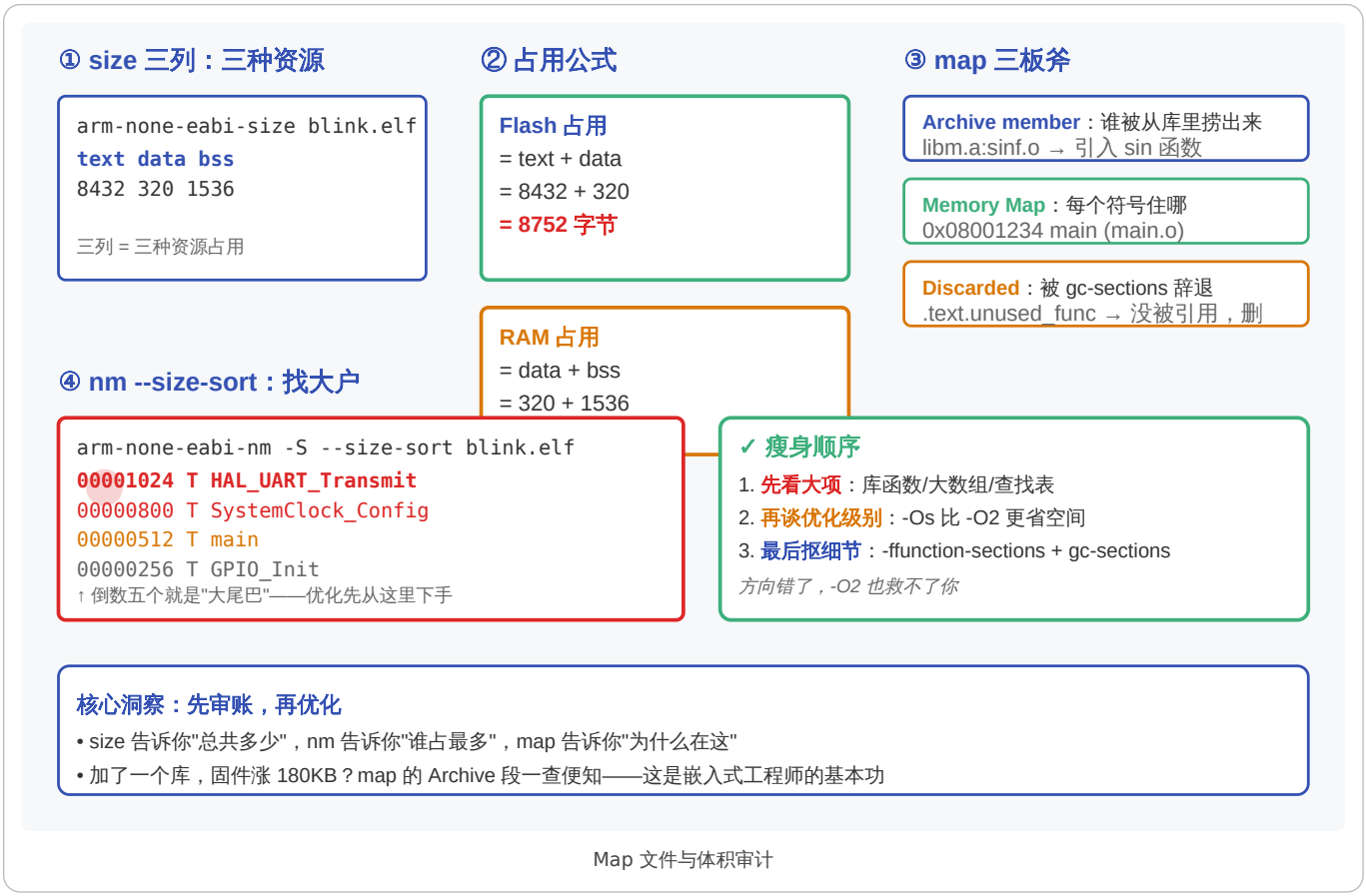
小节结构

小节	内容	四件套
size 三列	Flash=text+data、RAM=data+bss 的推导与验证	配置
map 三板斧	Archive 表/Memory Map/Discarded 逐段实读	库解析
nm 审计	--size-sort 找大户；A/B/C 实验各一次	代码分析
瘦身工具箱	-ffunction-sections+--gc-sections、-Os、去未用库、const 归位	配置
CCM 腾挪	大缓冲搬进 CCM 的链接脚本配合（B3 联动）	配置

map 分析的下游应用：审计完体积，遇到"Flash 不够装"或"要做 IAP 双区"，就回看 S13 Flash 与 IAP 的分区布局，以及 B3 链接脚本 的段排布。这里分清两件事：map 文件告诉你"现在住哪"，链接脚本决定"将来搬哪"。

动画：size → map → nm 三板斧

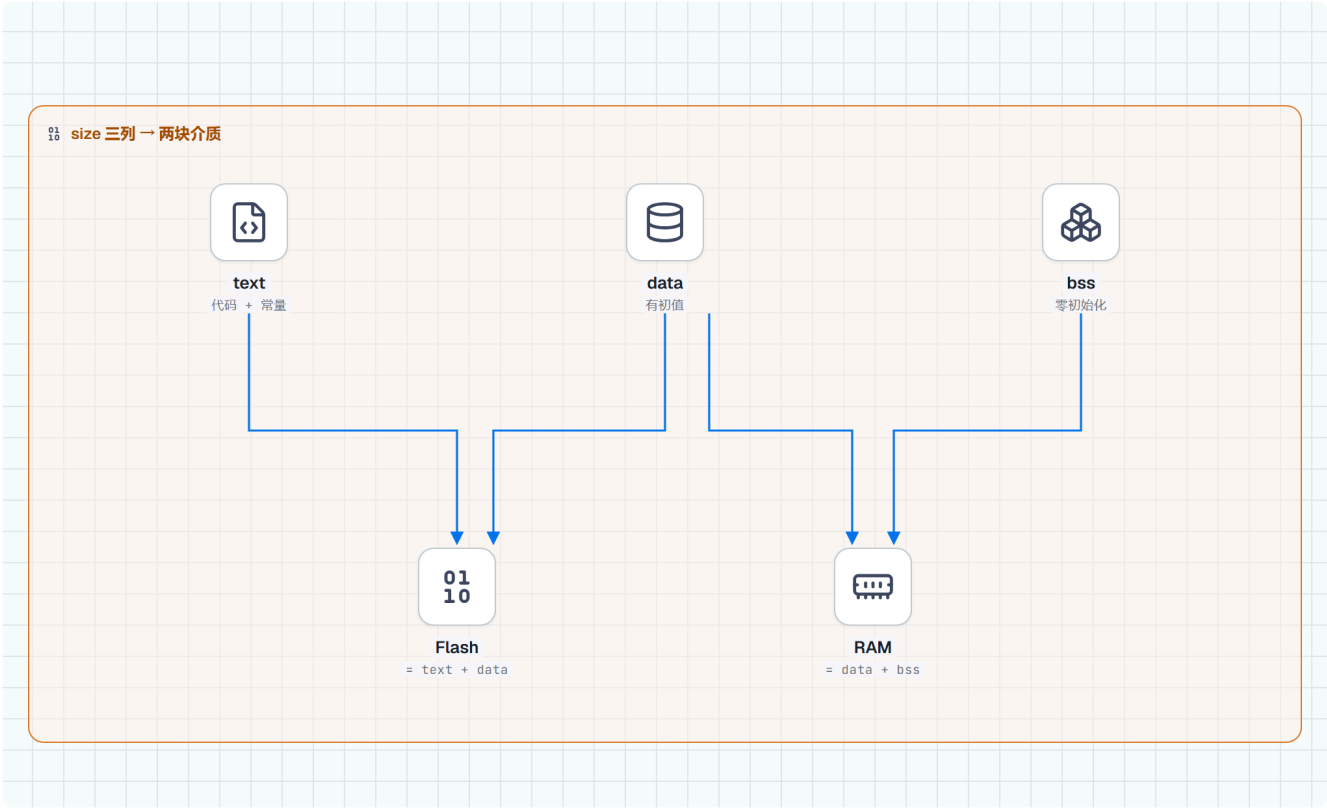
size 三列算总账，map 文件查明细，nm --size-sort 找大户。盯住"先审账再优化"五个字，这是体积审计的核心纪律。



Map 文件与体积审计

一、size 三列：Flash 与 RAM 各吃哪几份

账目公式先看图：三列输入两条公式，Flash 与 RAM 各吃哪几份一目了然。



size 三列的账：text/data/bss 汇入 Flash=text+data 与 RAM=data+bss

`arm-none-eabi-size blink.elf` 输出三列（本站 00-blink 实测，具体数以你本地 build 为准）：

```
   text    data     bss     dec     hex filename
    280      12      32     324     144 blink.elf
```

三列对应三种资源，推导靠 B3 链接脚本 的段属性：

列	装的是什么	住哪	算进谁的占用
text	代码（.text）+ 只读数据（.rodata）+ 向量表	Flash	Flash
data	有初值的全局变量（.data）	运行在 RAM，**初值存 Flash**	**Flash + RAM 各一份**
bss	零初始化的全局变量（.bss）	纯 RAM（启动时清零，不占 Flash）	RAM

两条占用公式立刻出来：

$$\text{Flash 占用} = \text{text} + \text{data}$$ $$\text{RAM 占用} = \text{data} + \text{bss}$$

为什么 data 算两份？因为 .data 有初值，初值必须存在断电不丢的 Flash 里（B3 的 `AT> FLASH`），运行时启动代码把它搬到 RAM（B4 启动 的 Reset_Handler 搬运循环）。所以 data 在 Flash 里存初值、在 RAM 里住运行值，两份都吃。bss 没初值，启动时清零即可，只吃 RAM。

> 【注】 这是 C1 内存模型 "const 经济学"的账本版：const 数组进 .rodata（算 text，只吃 Flash），可写数组进 .data（算 data，Flash+RAM 双吃），未初始化数组进 .bss（只吃 RAM）。B5 审计就是把 C1 的段归属用 size 数字验证一遍。

二、map 三板斧：Archive 表、Memory Map、Discarded

map 文件是链接器 `ld` 加 `-Map=` 旗生成的"链接户口本"（00-blink 的 Makefile 里 `... -Map=build/blink.map`）。三块内容各有用途：

第一板斧：Archive member 表：列出从哪些 `.a` 静态库里捞了哪些 `.o`。如果链接了 `libc.a`，这里能看到"为了 printf 捞了 `vfprintf.o` 、`__assert.o` ..."，库被引用的粒度一目了然，是 gc-sections 是否生效的证据。

第二板斧：Memory Map：每个 `.o` 的每个段落进哪个地址、每个符号多大住哪。找 `main.o` 的记录就能看到 main 函数多大、它引用了谁。这是"揪大户"的主要战场：按段汇总的数字告诉你 `.text` 里哪块最大。

第三板斧：Discarded input：被 `--gc-sections` 辞退的段。链接器发现某段没被任何入口（entry/中断向量）引用，就把它丢弃，不进最终 `.elf`。这里列出"谁被辞了"。如果某本该保留的段出现在这里，说明链接脚本没把它标 `KEEP()`（B3 的 KEEP 用法）。

三、nm 审计：按大小排队找大户

`nm -S --size-sort` 把所有符号按体积从大到小排，倒数五个就是固件里最占地方的：

```
arm-none-eabi-nm -S --size-sort build/blink.elf | tail -10
```

典型大户（00-blink 量级很小，加库后才看得出）：

大户类型	典型来源	瘦身手段
`vfprintf` 一族	printf 带浮点	不用 `%f` 就能关浮点版（`-u _printf_float` 反向）
大 `const` 表	正弦表/字库/查找表	必要的 Flash 开销，挪不走的
大缓冲区	`uint8_t buf[4096]`	搬进 CCM（第五节）或改环形
启动代码	`crt0.o` / `startup_*.s`	固定开销，几十百来字节

审计纪律：**先看大项再谈优化级别**。一个 4KB 的未用数组比 `-O2` 到 `-Os` 省的那几百字节大一个数量级。方向错了 -O2 也救不了你。

四、瘦身工具箱：四把刀按顺序用

1. **`-ffunction-sections -fdata-sections` + `--gc-sections`**：把每个函数/数据各放一个段，链接时丢弃没引用的。这是"没用到的库函数不进固件"的前提：不加 `-ffunction-sections`，整个 `.o` 要么全进要么全不进，gc-sections 无从下手。00-blink 的 Makefile 已默认开这套旗。
2. **`-Os`**：优化体积（比 -O2 更省 Flash，代价是速度略慢）。Flash 敏感场景首选；速度敏感的热路径单独标 `-O2` / `-O3`。
3. **去未用库**：链接了 `libc` 但没用 `malloc`，gc-sections 会把 `malloc.o` 辞退（前提是 `-ffunction-sections`）。整库链接不可怕，gc-sections 救场。
4. **const 归位**：把有初值的"只读"数组标 `const`，从 .data（Flash+RAM 双吃）挪进 .rodata（只吃 Flash）。C1 的"const 经济学"在 B5 的账本上直接体现：加一个 `const`，data 降、text 涨、RAM 净省。

五、CCM 腾挪：大缓冲搬进核心耦合内存

F407 除了 128KB SRAM，还有 **64KB CCM RAM**（Core Coupled Memory，0x10000000）——CPU 直连、零等待、但 **DMA 访问不到**。适合放 CPU 专用的大缓冲（DMA 缓冲不能放这，S8 的坑）。

链接脚本加一段（B3 的 MEMORY 范式）：

```
MEMORY { RAM (xrw) : ORIGIN=0x20000000, LENGTH=128K
         CCM (xrw) : ORIGIN=0x10000000, LENGTH=64K }
/* 大缓冲指定进 CCM */
.bss.dma_buf (NOLOAD) : { *(.dma_buf) } > RAM      /* DMA 缓冲留 RAM */
.bss.cpu_buf  (NOLOAD) : { *(.cpu_buf)  } > CCM    /* CPU 专用缓冲进 CCM */
```

代码侧给缓冲加 section 属性：`__attribute__((section(".cpu_buf")))`。CCM 腾挪的收益：128KB 主 SRAM 留给 DMA/栈/堆，64KB CCM 专门放大缓冲，两个内存域分工，不互相挤。代价：CCM 的数据 DMA 搬不动，配错了 DMA 静默失败（S8）。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **Flash=text+data，RAM=data+bss；优化先审账，nm 按大小排队，map 查户口本。**

延伸：size 审计的 .data/.bss 来源在 B3 链接脚本 讲透；CCM 腾挪与 DMA 的冲突在 S8 DMA 展开；构建系统自动生成 map 的机制见 B7。

实物实验

- 实验三连：① 加一个 4KB 未初始化数组→看 bss 涨、Flash 不涨；② 改成带 `={...}` 初始化→data 涨、Flash/RAM 双涨；③ 加 `const`→data 回落、text 涨。三次 `size` 截图对比，规律亲手验证。

常见坑

- **bin 比 text+data 大**：bin 里段间空隙也占字节（按地址铺开）；hex 才是真"按需记录"。
- **Debug 信息背锅**：-g3 让 elf 巨大，但调试信息不进 Flash，烧录大小以 map/size 为准。
- **栈堆不算进 bss**：它们靠 `_Min_*` 预算段占位+运行时生长（B3）；爆 RAM 的另一种死法。
- **优化级别迷信**：-Os 通常赢 -O2（Flash 敏感场景）；但先砍大项再谈级别。

短自测

1. 为什么 .data 同时算进 Flash 和 RAM 占用，.bss 只算 RAM？

> **▼ 参考答案**
> .data 有初值，初值必须存在断电不丢的 Flash（链接脚本 AT> FLASH），运行时启动代码搬到 RAM，所以 Flash 存初值一份、RAM 住运行值一份，两份都吃。.bss 零初始化，启动时清零即可，初值不需要存 Flash，只占 RAM。

2. `nm --size-sort` 倒数五个符号里，最大的一般是哪类？瘦身优先级怎么排？

> **▼ 参考答案**
> 典型大户：vfprintf 一族（printf 带浮点）、大 const 表、大缓冲区、启动代码。瘦身优先级：先砍大项（未用数组、未用库函数靠 gc-sections）→ 再调优化级别（-Os）→ 最后抠 const 归位。一个 4KB 未用数组比 -O2 到 -Os 省的几百字节大一个数量级，方向错了 -O2 救不了。

3. gc-sections 为什么必须配 `-ffunction-sections` 才有效？

> **▼ 参考答案**
> 不加 `-ffunction-sections`，一个 .o 里的所有函数被打包进同一个 .text 段，链接器只能整段进或整段丢：只要有一个函数被引用，整段都保留，没引用的函数也跟着进固件。加了 `-ffunction-sections`，每个函数各占一个 .text.funcname 段，链接器能精确丢弃没被引用的单个函数。gc-sections 的"按段丢弃"粒度依赖 -ffunction-sections 把段切细。

4. CCM RAM 适合放什么、不适合放什么？为什么？

> **▼ 参考答案**
> 适合放 CPU 专用的大缓冲（零等待、不与主 SRAM 抢空间）；不适合放 DMA 缓冲：CCM 直连 CPU 不挂总线矩阵，DMA 访问不到，配错了 DMA 静默失败。判断标准：这块数据 DMA 要不要碰？要碰就留主 SRAM，不碰就腾进 CCM。

对照表：本章概念 → 仓库与上游落点

概念	落点
size 三列公式	本节第一节；C1 内存模型 段归属
map 三板斧	00-blink 的 `build/blink.map`（`make` 生成）
nm --size-sort 审计	`arm-none-eabi-nm`；C6 栈帧 也用 nm
gc-sections + -ffunction-sections	00-blink Makefile 默认旗
CCM 链接脚本	B3 链接脚本 MEMORY 范式 + 本节第五节
const 经济学验证	C1 第二节 + 本节实验三连

你做到了

- 固件体积在你眼里是一笔能逐条对账的账；
- 手握 size/nm/map 三把审计刀，优化不再靠猜。

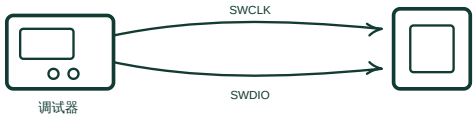
开场那本「固件户口本」你已经会翻了：每个符号多大、住在哪个段、由谁引进，三把刀各查一类。下次体积暴涨，第一反应不再是删代码，而是先审账，找出真正的大户再动手。

✅ 下一站：B6 烧录与调试：SWD 两根线，怎么把固件送进 Flash，又凭什么让 CPU 说停就停。

通往单片机之路第 16 章

B6 烧录与调试：SWD 与断点的真相

难度 ★★☆☆☆ · 约 30 分钟



本章罗盘

问题

烧录和调试为什么是同一件事，SWD 两根线凭什么在 CPU 停住时也能读写全片地址空间？

交付

一条 GDB 实战记录（reset halt/load/x/set/si/break/hbreak 各一次）加 OpenOCD program 五步日志逐行注解：halt→探测→擦写算法灌 RAM→verify→reset。

边界

FPB 只有 6 个比较器（Cortex-M0/M0+ 只有 2 个），断点预算随芯片变不能外推；Semihosting 在不接调试器时 BKPT 即 HardFault，量产必须关；SWD 报文格式与 JTAG 不在本章。

依据锚点 `a6` · `mcu-code` · `mcu-anim`

> 🎯 烧录和调试看起来像两个功能，其实是同一件事：通过 SWD 两根线，直接读写芯片的内存、寄存器，甚至让它停下来。OpenOCD 和 GDB 只是这对"超能力"的两个客户端。

本章精髓

1. SWD 只有三根线（SWDIO/SWCLK/GND）：一套请求-响应协议，读写一个叫 DP（调试端口）的家伙，再由 AP（访问端口）摸到整个地址空间，**CPU 跑不跑都能摸**。
2. 烧录=主机帮你执行擦写：OpenOCD 把一小段"擦写算法"灌进 RAM，让它替自己干活（所以 program 命令要选对 target 配置）。
3. 断点有两副面孔：硬件断点（FPB 比较器，数量有限，Flash 里也能下）vs 软件断点（改写成 BKPT 指令，RAM 才行，数量不限）；GDB 的 `hbreak` 与 `break` 由此分家。

- **能理解**：为什么烧录要把"擦写算法"灌进 RAM 代劳；为什么 Flash 里的代码用 `break` 有时失败而 `hbreak` 总能成。
- **能用**：先让 OpenOCD 常驻，再用 GDB 连上；随后完成 halt/load/读写内存/单步/断点这一整套调试流。

先修

- B4 启动过程；S0（环境已就绪）。

先跑起来（10 分钟 quick win）

连上之前先押一题：调试器往 RAM 里写一个新值，芯片必须先停下来配合吗？还是边跑边改？押一个答案，连上后当场试。

```
make debug &        # OpenOCD 常驻
make gdb            # 连上即：reset halt → load → break main → continue
```

GDB 里再玩三招：`x/4xw 0x20000000`（读 RAM）、`set {int}0x20000000 = 0x1234`、`hbreak main`。

小节结构

小节	内容	四件套
链路全图	PC/OpenOCD/ST-Link/SWD/DP/AP 逐环讲	配置
program 拆解	OpenOCD 日志逐行：halt→擦→写→verify→reset	库解析
GDB 十命令	target remote/load/monitor/x/set/si/break/hbreak/info reg/continue	代码分析
断点的硬件	FPB 比较器 vs BKPT 改写；各 6 个的限额从哪来	库解析
Semihosting 一瞥	让 printf 走调试通道回 PC——原理与代价（S7 的替代方案）	库解析

一、链路全图：从 GDB 命令到芯片内存

以 `x/4xw 0x20000000` 读 RAM 为例：从左边的 GDB 开始，沿箭头看到最右边的内存地址：

```
GDB ──(TCP 3333)──> OpenOCD ──(USB)──> ST-Link ──(SWD)──> 芯片 DP ──> AP ──> 总线矩阵 ──> 0x20000000
                    调试服务器        调试探针        调试端口    访问端口    AHB
```

每一环的职责：

- **GDB**：人机界面+命令调度，把 `x` 命令翻成 JTAG/SWD 包发给 OpenOCD；
- **OpenOCD**：调试服务器，把 GDB 协议翻成 SWD 请求，常驻在 PC 上监听 3333 端口；
- **ST-Link**：USB 转 SWD 的硬件探针（板载调试器），把 USB 包翻成 SWDIO/SWCLK 电平；
- **SWD**：两根线（SWDIO 双向数据、SWDCK 时钟，加 GND 共三根）的请求-响应协议；
- **DP（Debug Port）**：芯片上的调试端口，SWD 请求先进它，它再通过 AP 访问内部总线；
- **AP（Access Port）**：访问端口，把 DP 的请求转成 AHB 总线读写，**直通整个 4GB 地址空间**。

关键性质：**AP 访问总线不经过 CPU**。CPU 在跑代码也好、停在断点也好、甚至睡死，AP 都能读写任意地址。这就是"烧录和调试是同一件事"的物理基础：读写 Flash 寄存器就是烧录，读写 RAM/寄存器就是调试，读写 PC/MSP 就是控制 CPU，全走同一条 SWD 链路。

二、program 拆解：烧录的本质是请 RAM 里的算法代劳

`make flash` 或 GDB 的 `load` 触发 OpenOCD 的 `program` 命令，日志逐行（以 00-blink 烧录为例）：

```
> reset halt                          # ① 复位并停住 CPU（不然它在乱跑，写 Flash 会冲突）
> flash probe 0                       # ② 探测 Flash：读 ID、容量、扇区布局
> flash write_image erase build/blink.bin 0x08000000   # ③ 擦+写
> verify_image build/blink.bin 0x08000000              # ④ 校验
> reset run                           # ⑤ 复位运行（PC 指向复位向量）
```

第 ③ 步"擦+写"最绕：OpenOCD 不能直接从 USB 往 Flash 寄存器写：Flash 编程有严格时序（解锁→擦→写→等 BSY，S13）。OpenOCD 的办法是**把一段"擦写算法"（几十行 ARM 机器码）灌进芯片 RAM**，让 CPU 自己跑这段算法去操作 Flash 控制器；CPU 跑算法比 OpenOCD 从 USB 逐个寄存器写，更快也更可靠。OpenOCD 只负责传数据、发"开始执行算法"的命令。这就是 program 必须选对 target 配置的原因：不同芯片的擦写算法不同，配错了算法跑错寄存器，Flash 写不进去，甚至写坏。

> 【注】 这也解释了 S13 的"擦写关键路径放 RAM 执行"为什么是工业标准：OpenOCD 的烧录本身就是这么干的。你的 IAP bootloader 跳转函数（关中断→MSP→VTOR→跳）做的事和 OpenOCD 的"灌算法进 RAM 再让它跑"同构。

三、GDB 十命令：调试的日常

连上 OpenOCD 后（`target remote :3333`），GDB 十命令覆盖 90% 调试场景：

命令	干什么	本站用到处
`target remote :3333`	连 OpenOCD	每次 `make gdb`
`load`	烧 elf 进 Flash	改完代码重新烧
`monitor reset halt`	复位并停 CPU	烧前/调试前必做
`x/4xw 0x20000000`	读 4 个字（RAM/寄存器）	C6 看栈帧、S16 挖 PC
`set {int}0x20000000 = 0x1234`	写内存（改寄存器/变量）	不重新编译快速试值
`si` / `ni`	单步指令/指令步过	C6 走 prologue
`break main`	软件断点	RAM 代码或 Flash 代码（占 FPB）
`hbreak main`	硬件断点	Flash 代码（独占 FPB 比较器）
`info registers`	看所有寄存器	崩溃现场、调用约定核对
`continue`	继续跑	单步完放它走

带 `monitor` 前缀时，命令发给 OpenOCD（reset/flash/sleep）；不带 monitor 时，由 GDB 自己处理（break/x/continue）。对照这两个例子：`monitor reset halt` 是 OpenOCD 让芯片复位停住，`break main` 是 GDB 设断点。

四、断点的硬件：FPB 比较器 vs BKPT 改写

断点两副面孔，硬件实现完全不同：

- **硬件断点（`hbreak`）**：Cortex-M4 内置 **FPB（Flash Patch and Breakpoint）单元**，6 个比较器。CPU 取指时 FPB 比对地址，命中就把 PC 停住，**不改 Flash 内容**，所以 Flash 代码能下。代价是只有 6 个（Cortex-M4 标配），超了报错。
- **软件断点（`break`）**：GDB 把目标地址的指令**改写成 `BKPT #imm`**（0xBE00 系列），CPU 执行到 BKPT 触发调试异常停下。**要能改写指令**：RAM 代码随便改，Flash 代码改不动（Flash 写要先擦整扇区），所以 `break` 在 Flash 代码上经常悄悄退回硬件断点或失败。

实战纪律：**Flash 里的代码用 `hbreak`，RAM 里的代码用 `break`**。00-blink 这种全 Flash 代码，6 个硬件断点够用；当代码大到断点多于 6 个，要么清旧断点，要么把要调试的函数搬进 RAM（`__attribute__((section(".ramfunc")))`，S13）用软件断点。

> 【注】6 这个数字来自 Cortex-M4 的 FPB 设计（RM0090 调试章节）：FPB 有 6 个比较器，可配代码断点或数据字面量补丁。Cortex-M0/M0+ 只有 2 个，调试大固件时更紧张。选芯片时，这是个常被忽略的调试资源约束。

五、Semihosting 一瞥：printf 走调试通道

S7 USART 把 printf 接到串口靠 newlib 的 `_write` 钩子。Semihosting 是另一条路：让 `_write` 触发一条 `BKPT` 指令，OpenOCD 捕获后把字节通过 USB 送回 PC 的 GDB 控制台。**不用接串口，调试器一条线搞定输出**。

代价：CPU 执行 BKPT 会停住等 OpenOCD 处理，每次 printf 都要付出几十 μs 的调试通道往返，比串口慢一个数量级。适合调试期临时输出，不适合量产（量产要把 Semihosting 关掉，否则没接调试器时 BKPT 直接 HardFault）。

OpenOCD 开 Semihosting：`monitor arm semihosting enable`；newlib 链接加 `--specs=rdimon.specs`。两条配齐，printf 就走调试通道回 PC。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **SWD 两线通天，CPU 停不停都能读写；烧录是请 RAM 里的算法代劳，断点分软硬——Flash 用硬，RAM 随意。**

延伸：SWD 链路完整解剖（DP/AP/内存访问）在本章正文展开；FreeRTOS 任务级 GDB 调试见 F2；硬件断点 FPB 与中断调试配合见 S4 NVIC。

实物实验

- `monitor flash erase_sector 0 0 0` 擦掉首扇区再复位：板子不闪了（固件没了）；重新 `make flash` 救回，亲手体验"烧录的本质是改内存"。

常见坑

- **GDB 连不上 3333**：OpenOCD 没常驻/防火墙拦/上一个 GDB 没退（`target remote` 是 TCP 连接）。
- **load 之后跑飞**：忘了 `monitor reset halt` 就 continue，芯片可能已在乱跑。
- **硬件断点用完**：`break` 在 Flash 区域悄悄占用 FPB 槽位（通常 6 个），超了报错，得清旧断点或换软件断点思路。
- **ST-Link 固件过旧**：OpenOCD 报 SWD/JTAG 错误，用 ST 官方工具升级固件。
- **Semihosting 没关量产**：没接调试器时 BKPT 触发 HardFault，量产构建必须关 `--specs=rdimon.specs`。

短自测

1. SWD 只有几根线？AP 访问总线为什么不需要 CPU 配合？

▼ 参考答案
三根：SWDIO（双向数据）、SWDCK（时钟）、GND。AP（访问端口）直挂 AHB 总线矩阵，读写请求由 AP 发起、走总线到目标地址，不经 CPU。CPU 在跑代码、停断点、睡觉都不影响 AP 读写。这就是"烧录和调试是同一件事"的物理基础：读写 Flash 寄存器=烧录，读写 RAM/寄存器=调试，全走同一条 AP 路径。

2. `program` 命令为什么要把"擦写算法"灌进 RAM？直接从 USB 写 Flash 寄存器不行吗？

▼ 参考答案
Flash 编程有严格时序（解锁→擦→写→等 BSY），不是简单的"写寄存器就生效"。OpenOCD 把一段几十行 ARM 机器码的擦写算法灌进芯片 RAM，让 CPU 自己跑这段算法去操作 Flash 控制器：CPU 跑算法比 OpenOCD 从 USB 逐个寄存器写，更快也更可靠。不同芯片擦写算法不同，所以 target 配置必须选对。

3. Flash 里的代码为什么推荐 `hbreak` 而非 `break`？两者硬件实现有何不同？

▼ 参考答案
软件断点 `break` 要把目标指令改写成 BKPT：RAM 代码能改，Flash 代码改不动（写 Flash 要先擦整扇区），所以 break 在 Flash 上常失败或退回硬件断点。硬件断点 `hbreak` 用 FPB 比较器比对取指地址，命中就停，不改 Flash 内容，Flash 代码能下。代价是 FPB 只有 6 个（Cortex-M4 标配），超了报错。RAM 代码用 break（不限量），Flash 代码用 hbreak（6 个限额）。

4. Semihosting 的 printf 比串口 printf 慢一个数量级，原因是什么？量产为什么要关？

▼ 参考答案
Semihosting 的 printf 触发 BKPT 指令，CPU 停住等 OpenOCD 捕获、通过 USB 把字节送回 PC，每次输出都有几十 μs 的调试通道往返，比串口直接写 UART 寄存器慢一个数量级。量产要关是因为：没接调试器时 BKPT 指令没人捕获，直接触发 HardFault 系统挂掉。量产构建去掉 `--specs=rdimon.specs`，printf 走真实串口或彻底去掉。

对照表：本章概念 → 仓库与上游落点

概念	落点
SWD/DP/AP 链路	RM0090 调试章节；OpenOCD target 配置
擦写算法灌 RAM	OpenOCD `program` 实现；S13 IAP 跳转函数同构
GDB 十命令	00-blink 的 `make debug` / `make gdb`（Makefile target）
FPB 6 个比较器	Cortex-M4 TRM / RM0090 调试章节
Semihosting	OpenOCD `monitor arm semihosting enable`；对比 S7 串口 printf

你做到了

- 烧录/调试从魔法变成一条讲得通的链路；
- GDB 十命令上手，软硬断点知道何时切换。

开场那对「超能力」现在拆开了：烧录是往 Flash 写，断点是往指令上改，窥视是读寄存器与内存，走的都是同一条两根线的链路。所谓统治一颗芯片，凭的就是这份直通内存的读写权。

✅ 下一站：B7 构建系统：手写 Makefile 已经上手，接着看 CMake/Ninja 和 ESP-IDF、RT-Thread(scons) 怎么把它规模化。

通往单片机之路第 17 章

B7 构建系统：从手写 Makefile 到 CMake 与 scons

难度 ★★☆☆☆ · 约 30 分钟

本章罗盘

问题

两个文件用 Makefile 绰绰有余，两百个文件二十个组件怎么办？依赖图和 Kconfig 各自解决哪一维问题？

交付

一张手画并用 make -n 干跑验证的 00-blink 依赖图，一个 toolchain.cmake 在 project() 之前的两目录 CMake 裸机骨架，加 IDF（CMake）与 RT-Thread（scons+Kconfig）的机制对照表。

边界

构建系统管『怎么编』，Kconfig 管『编什么』，两维正交、不可互相外推；IDF 组件机制在 P3、RT-Thread 的 env/menuconfig 全流程在 R6；CMake 全语法与 IDE 生成器不在本章。

依据锚点 `mcu-code` · `mcu-anim`

> 🎯 一张最气人的构建工单：你明明改了头文件，编译器却说没有东西需要重建；固件顺利烧进去，行为还是旧的。谁在撒谎？编译器没撒谎，Make 也没撒谎，是依赖图根本不知道那次修改。沿着这张工单追下去，Makefile、CMake、IDF 与 scons 都会显出同一副骨架：**依赖图 + 配置层**。

本章精髓

1. Make 的本质是一张依赖图：目标←依赖←规则，mtime 比新旧决定要不要重来；`make -n` 干跑一遍，图就摆在眼前。
2. CMake 不构建，它**生成**构建文件（Ninja/Makefile）：多一层抽象换来跨平台+组件化；ESP-IDF 的 `idf_component_register` 就是在这层抽象上圈地。
3. 配置是独立一维：Kconfig 把"编不编这个组件/开不开这个功能"变成菜单（IDF 的 sdkconfig、RT-Thread 的 menuconfig 同源）：**构建系统管怎么编，Kconfig 管编什么**。

- **能理解**：为什么头文件改了 Makefile 不重建是"没写依赖"；为什么 CMake 工具链文件必须在 `project()` 之前设。
- **能用**：`make -n` 干跑看依赖图；读懂 00-blink Makefile 的每一行在画什么图。

先修

- B1 四步构建；用过 P0 的 idf.py 更佳。

先跑起来（10 分钟 quick win）

先预测再执行：第一次干跑会列出哪些命令？只碰一下源文件后，会全部重建，还是只重建受它影响的目标？如果你的预测是“构建系统会自己理解文件内容”，这十分钟会把这个直觉推翻。

```
make clean && make -n        # 干跑：把将执行的命令全列出，不真做
touch main.c && make -n      # 只动了 main.c，看依赖图如何"局部重建"
```

`make -n` 是构建系统的"X 光"：它把将执行的命令全列出来但不真做，依赖图的形状一目了然。`touch` 改一个文件的 mtime，再 `make -n` 看只有哪些目标会被重建，这就是依赖图"局部重建"的实证。

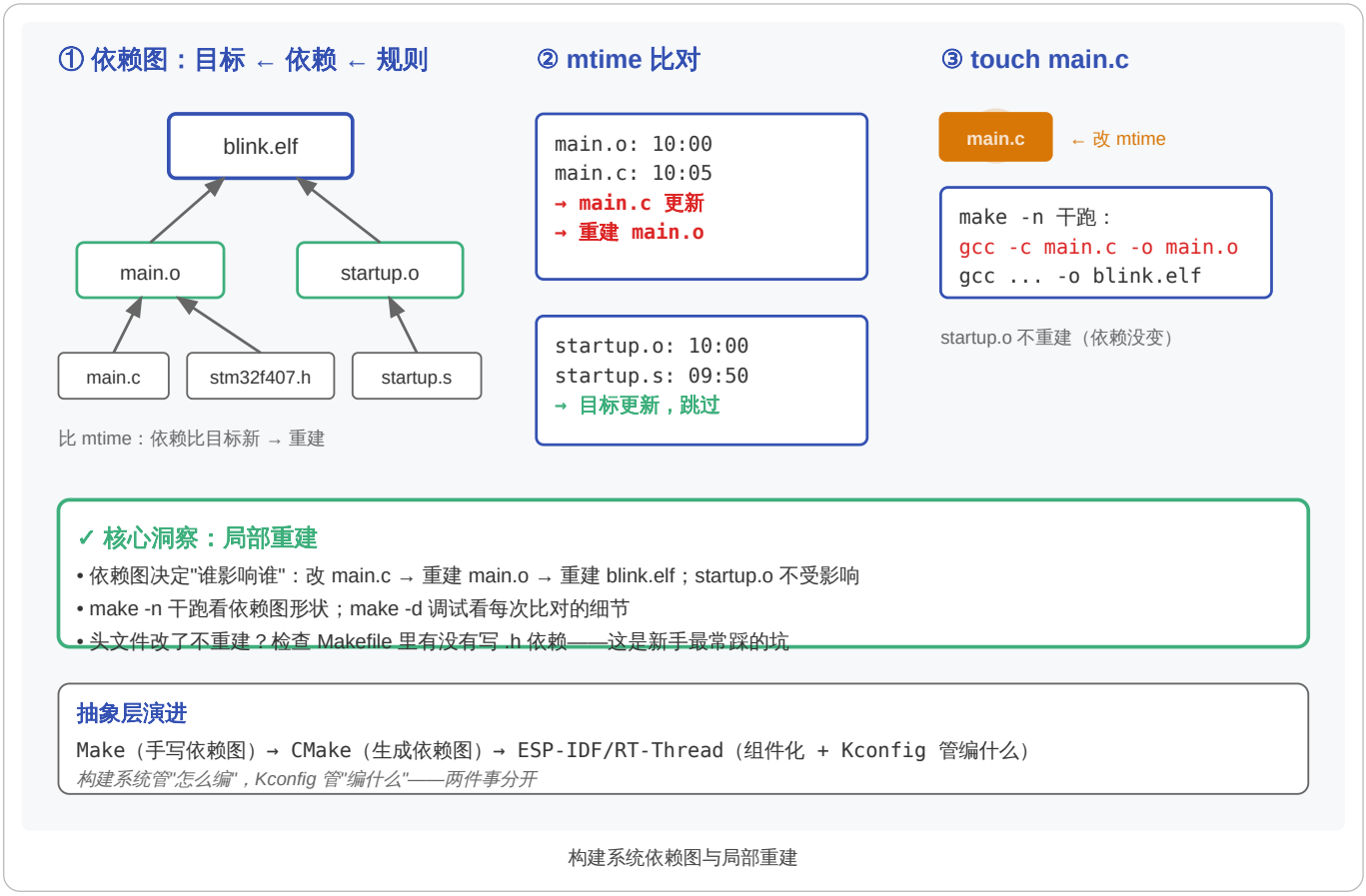
立即核对：Make 没读懂你的源代码，它只比较图上的节点和时间。图上没有的关系，在它眼里就不存在。开场工单里“头文件改了，目标没动”的怪现象，到这里已经抓到第一名嫌疑人：缺失的依赖边。

小节结构

小节	内容	四件套
Make 依赖图	目标/依赖/规则三要素；模式规则 %.o:%.c；自动变量 $@$<	库解析
Makefile 解剖	00-blink 的 Makefile 逐行回看：我们其实写了张什么图	代码分析
CMake 一层抽象	add_executable/target_compile_options；工具链文件 toolchain.cmake	库解析
IDF 组件机制	idf_component_register 做了什么；REQUIRES/PRIV_REQUIRES 依赖声明	库解析
Kconfig 一维	menuconfig→sdkconfig→宏：配置如何流进代码（联动 P3）	配置
scons 对照	RT-Thread 的 SConscript 思路（联动 R6）	库解析

动画：依赖图与局部重建

依赖图画出"谁影响谁"，mtime 比对决定"要不要重建"。touch 改一个文件后，沿图看看哪些目标会受影响。读这张图时，盯住"局部重建"三个字。



构建系统依赖图与局部重建

一、Make 依赖图：目标/依赖/规则三要素

Make 的世界只有三样东西：**目标**（要产出的文件，如 `blink.elf`）、**依赖**（产出它需要的文件，如 `main.o startup.o`）、**规则**（怎么从依赖造目标，如 `$(CC) ... -o $@ $^`）。三者用 `:` 连起来就是一条 Make 规则：

```
blink.elf: main.o startup.o          # 目标: 依赖
    $(CC) $(LDFLAGS) -o $@ $^         # 规则（Tab 缩进！）
```

Make 的核心算法只有一条：**比 mtime**。目标的 mtime 比所有依赖都新 → 不重建；有依赖比目标新 → 重建。`make -n` 干跑就是把这套比对跑一遍，列出将执行的命令但不真做，依赖图的"形状"（谁依赖谁、谁该先建）一览无余。

两个自动变量背下来：`$@`=目标名、`$<`=第一个依赖、`$^`=所有依赖。模式规则 `%.o: %.c` 让 Make 自动把每个 `.c` 编成 `.o`，不用为每个文件写一条规则，00-blink 的 Makefile 就靠这条管所有源文件。

二、Makefile 解剖：00-blink 写了张什么图

回看 00-blink 的 Makefile（`code/stm32/00-blink/Makefile`），它画的依赖图：

```
blink.elf ─┬── main.o ─── main.c + stm32f407.h
           ├── startup_stm32f407xx.o ─── startup_stm32f407xx.s
           └── linker.ld（链接脚本，不编译，直接喂 ld）
blink.bin ─── blink.elf（objcopy 转格式）
flash ─── blink.bin（调 OpenOCD 烧录，伪目标）
debug ─── 无依赖（常驻 OpenOCD，伪目标）
gdb ─── 无依赖（连 OpenOCD，伪目标）
```

每个 Makefile target 对应图上一个节点。**伪目标**（`.PHONY` 声明的，如 `flash` / `debug` / `gdb`）不产出文件，只是"命令别名"：`make flash` 就是跑那行 OpenOCD 烧录命令，不比对 mtime。

骨架页坑列表第一名的"头文件改了不重建"：图里 `main.o` 的依赖只写了 `main.c`，没写 `stm32f407.h`。改了头文件，main.o 的 mtime 仍比 main.c 新（main.c 没动），Make 不重建 main.o，固件用的是旧编译的 main.o。解法：`-MMD -MP` 让 gcc 编译时自动生成 `.d` 依赖文件（把头文件依赖也写进去），Makefile `include` 这些 `.d`，头文件依赖自动维护，是进阶 Makefile 的标配。

工单到这里可以结案：不是“Make 偶尔抽风”，而是图画漏了一条边。这个结论比背一条修复命令更重要，因为以后遇到链接顺序、组件循环依赖或配置没生效，你都可以问同一句：真实关系有没有进入构建系统看得见的图？

三、CMake 一层抽象：生成构建文件

CMake 不直接构建，它**生成** Makefile 或 Ninja 构建文件，多一层抽象，换来跨平台+组件化。最小裸机工程骨架：

```
# 顶层 CMakeLists.txt
cmake_minimum_required(VERSION 3.13)
set(CMAKE_TOOLCHAIN_FILE arm-none-eabi.cmake)   # 工具链文件，必须在 project() 之前
project(blink C)
add_executable(blink main.c startup.s)
target_compile_options(blink PRIVATE -mcpu=cortex-m4 -mthumb -Os)
target_link_options(blink PRIVATE -Tlinker.ld)
add_custom_command(TARGET blink POST_BUILD
    COMMAND ${CMAKE_OBJCOPY} -O binary $<TARGET_FILE:blink> blink.bin)
```

```
# arm-none-eabi.cmake（工具链文件）
set(CMAKE_SYSTEM_NAME Generic)
set(CMAKE_SYSTEM_PROCESSOR cortex-m4)
set(CMAKE_C_COMPILER arm-none-eabi-gcc)
set(CMAKE_ASM_COMPILER arm-none-eabi-gcc)
set(CMAKE_OBJCOPY arm-none-eabi-objcopy)
set(CMAKE_TRY_COMPILE_TARGET_TYPE STATIC_LIBRARY)   # 不试链接（交叉链没有运行环境）
```

两条铁律（骨架坑列表）：① 工具链文件必须在 `project()` 之前 `set(CMAKE_TOOLCHAIN_FILE ...)`，否则 CMake 用主机 gcc 探测环境、编出 x86 的东西；② `CMAKE_TRY_COMPILE_TARGET_TYPE=STATIC_LIBRARY` 让 CMake 不试运行编译产物（交叉编译的产物在 PC 上跑不了）。

CMake 的价值在"规模"：两百个文件分在二十个目录，用 `add_subdirectory` 递归管理，每个目录各有一个 `CMakeLists.txt`，不用把所有内容挤进一个大 Makefile。代价是多一步"生成"：先用 `cmake -B build` 生成，再用 `make -C build` 构建。

四、IDF 组件机制：idf_component_register 圈地

ESP-IDF 在 CMake 之上加了"组件"抽象（P3 IDF 工程解剖 全章展开）。每个组件（component）是一个目录，含 `CMakeLists.txt` + 源码 + `Kconfig`，顶层 `idf_component_register` 声明组件名、源文件、依赖：

```
# 组件的 CMakeLists.txt
idf_component_register(
    SRCS "my_driver.c"
    INCLUDE_DIRS "include"
    REQUIRES driver          # 依赖其他组件
    PRIV_REQUIRES freertos   # 仅实现层依赖，不暴露给依赖我的组件
)
```

`REQUIRES` vs `PRIV_REQUIRES`：前者把依赖组件的头文件也暴露给"依赖我的组件"（传递），后者只给本组件实现用（不传递），这是 C/C++ 接口与实现分离在构建层的体现。骨架坑列表的"循环依赖"（A REQUIRES B、B REQUIRES A）解法：抽出公共部分进 C，A/B 都 REQUIRES C，不再互引。

IDF 的三层分工：CMake 管组件，Kconfig 管配置，idf.py 负责调度。三层叠在一起，底层还是 B1 那四步（预处理→编译→汇编→链接）。

五、Kconfig 一维：配置如何流进代码

Kconfig 把"编不编这个组件/开不开这个功能"变成菜单（`menuconfig` 命令打开），选择结果存进 `sdkconfig`（键值对），构建时生成 `sdkconfig.h`（宏定义）供 C 代码 `#include`：

```
Kconfig（菜单定义）—menuconfig—> sdkconfig（选择结果）—生成—> sdkconfig.h（宏）
                                                                  ↓
                                                          C 代码 #include，#ifdef 走分支
```

同一个机制，IDF 叫 sdkconfig、RT-Thread 叫 .config（R6 Env 与 menuconfig 全章展开）。**Kconfig 是独立一维**，与构建系统（CMake/scons）正交。构建系统管"怎么编"（依赖图+命令），Kconfig 管"编什么"（哪些组件进、哪些宏开）。两维独立配置，不互相绑死。

骨架坑列表的"把 sdkconfig 提交进仓库"：sdkconfig 是构建产物（每个人配不同），该提交的是 `sdkconfig.defaults`（默认值的种子），新人 `idf.py build` 时由 defaults 重新生成本地 sdkconfig。

六、scons 对照：RT-Thread 的另一套衣服

RT-Thread 用 scons（Python 写的构建系统）+ Kconfig，与 IDF 的 CMake+Kconfig 对照：

维度	ESP-IDF	RT-Thread
构建系统	CMake（生成 Ninja/Makefile）	scons（Python 脚本）
配置系统	Kconfig → sdkconfig	Kconfig → .config（同源）
组件声明	`idf_component_register`	`SConscript`（Python 函数）
调度入口	`idf.py build/flash`	`scons` / `scons -j4`

两者都把 B1 的四步包成"组件化+配置化"，差别只在构建脚本语言（CMake 的 DSL vs Python）。**底层都是依赖图+配置层两件事**，会一种就能读另一种；R6 会展开 RT-Thread 的 env/menuconfig 全流程。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **Make 画图（依赖），CMake 生图（跨平台），Kconfig 管编什么，构建系统管怎么编**——IDF 用 CMake 圈组件，RTT 用 scons 走江湖，底层还是 B1 那四步。

延伸：IDF 的 CMake 组件化在 P3 IDF 解剖 展开；RT-Thread 的 scons+menuconfig 在 R6 完整走一遍；依赖图与局部重建的动画见 B1 四步构建。

实物实验

- 在 00-blink 的 Makefile 里故意删掉 `startup_stm32f407xx.o` 的依赖，touch 启动文件后 make，观察到"没重建"，亲手证明依赖图的价值，然后改回来。

常见坑

- **头文件改了不重建**：Makefile 没写头文件依赖。用 `-MMD -MP` 让 gcc 自动生成 .d 依赖文件（进阶 Makefile 标配）。
- **CMake 工具链文件顺序错**：必须在 `project()` 之前 `set(CMAKE_TOOLCHAIN_FILE ...)`，否则用的是主机 gcc。
- **IDF 组件循环依赖**：A REQUIRES B、B REQUIRES A，抽出公共组件 C 是标准解法。
- **把 sdkconfig 提交进仓库**：它是构建产物；该提交的是 `sdkconfig.defaults`。

短自测

1. Make 怎么决定一个目标要不要重建？`make -n` 干跑看的是什么？

> ▼ **参考答案**
> 比 mtime：目标比所有依赖都新就不重建，有依赖比目标新就重建。`make -n` 干跑把这套比对跑一遍，列出将执行的命令但不真做，依赖图的形状（谁依赖谁、谁该先建、谁会被重建）一目了然，是构建系统的"X 光"。

2. 改了头文件 Makefile 不重建，根因和解法各是什么？

▼ **参考答案**
根因：Makefile 里 main.o 的依赖只写了 main.c，没写 stm32f407.h。改了头文件，main.o 的 mtime 还比 main.c 新（main.c 没动），Make 不重建 main.o。解法：加 `-MMD -MP` 编译旗，gcc 编译时自动生成 .d 依赖文件（把头文件依赖也写进去），Makefile `include` 这些 .d，头文件依赖自动维护，不用手写。

3. CMake 工具链文件为什么必须在 `project()` 之前设？设晚了会怎样？

▼ **参考答案**
CMake 在 `project()` 时探测编译器、试编译、设置系统名。如果工具链文件在 project() 之后才设，CMake 已用主机 gcc 探测完环境、编出 x86 的东西，交叉工具链设了也来不及。必须在 project() 之前 set(CMAKE_TOOLCHAIN_FILE)，让 CMake 一开始就用交叉 gcc 探测。

4. Kconfig 与构建系统（CMake/scons）是什么关系？为什么说 Kconfig 是"独立一维"？

▼ **参考答案**
Kconfig 管编什么（哪些组件进、哪些宏开），构建系统管怎么编（依赖图+命令）。两者正交：同一份 Kconfig 配置可以喂给 CMake 也可以喂给 scons；同一套构建系统可以编不同 Kconfig 配置。Kconfig 的输出（sdkconfig/.config → 宏头）是构建系统的一个输入，但两者各自独立演化，不互相绑死。

对照表：本章概念 → 仓库与上游落点

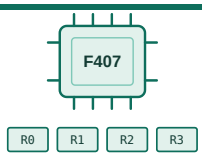
概念	落点
00-blink Makefile 依赖图	`code/stm32/00-blink/Makefile`
模式规则 %.o:%.c + 自动变量	同上 Makefile
CMake 裸机骨架	本节第三节；对比 P3 IDF 组件
idf_component_register	P3 IDF 工程解剖 全章
Kconfig → sdkconfig	P3；R6 RT-Thread 同源
scons + SConscript	R6 全章

你做到了

- 构建系统从"玄学配置"还原成"依赖图 + 配置层"两件事；
- Makefile/CMake/IDF/scons 四种形态在你眼里是同一件衣服的四个剪裁。

回看开场那张工单：命令成功不等于产物包含了你的修改。只要依赖图漏边，构建系统就会非常诚实地生成一个过期固件。你现在不仅知道怎么补边，也知道下一次该从哪里开始审讯。

✅ B 篇收官。下一站：S 篇 STM32 裸机：地基已经打完，去寄存器的世界里大干一场。



PART PS

第三部分 STM32F407 寄存器主线

不碰 HAL、不点 CubeMX：从 RCC 时钟树到看门狗，18 站走完「手册 → 寄存器 → 库源码 → 实物」的完整闭环，对芯片的信任来自手册而不是库。

> 18 站走完一条闭环：手册 → 寄存器 → 库源码 → 实物。这一部刻意不碰 HAL 与 CubeMX：工具会把会出错的决策藏起来，而学习就是把决策重新自己做一遍。每个外设都从 RM0090 的位定义出发，配上霸天虎板上可复现的工程。读完的标志：换一颗没玩过的芯片，只看手册能在两小时里把 USART 收发跑起来。

本部 18 章 · 约 645 分钟（10.8 小时）

18 . S0 环境搭建：裸机工具链
19 . S1 F407 架构总览：地址背后的故事
20 . S2 RCC 时钟树：168MHz 是怎么算出来的
21 . S3 GPIO：七个寄存器位级图解
22 . S4 NVIC 与 EXTI：打断的艺术
23 . S5 SysTick：内核的心跳
24 . S6 定时器 TIM：一器多用的瑞士军刀
25 . S7 USART：让芯片开口说话
26 . S8 DMA：让数据自己搬家
27 . S9 ADC：把模拟世界变成数字
28 . S10 DAC：芯片也能输出模拟量
29 . S11 I2C：两根线的社交礼仪
30 . S12 SPI：全双工的移位艺术
31 . S13 内部 Flash 与 IAP：固件给自己动手术
32 . S14 低功耗：让电池活过一年
33 . S15 SPL 标准库解剖：库是怎么封装寄存器的
34 . S16 HardFault 与排错：崩溃现场的法医技术
35 . S17 看门狗：任务失去进展之后

通往单片机之路第 18 章

S0 环境搭建：裸机工具链

难度 ★☆☆☆☆ · 约 25 分钟

本章罗盘

问题

不用 IDE，四个文本文件怎么变成板上闪烁的灯？每一步产物由谁生成、装错了去哪查？

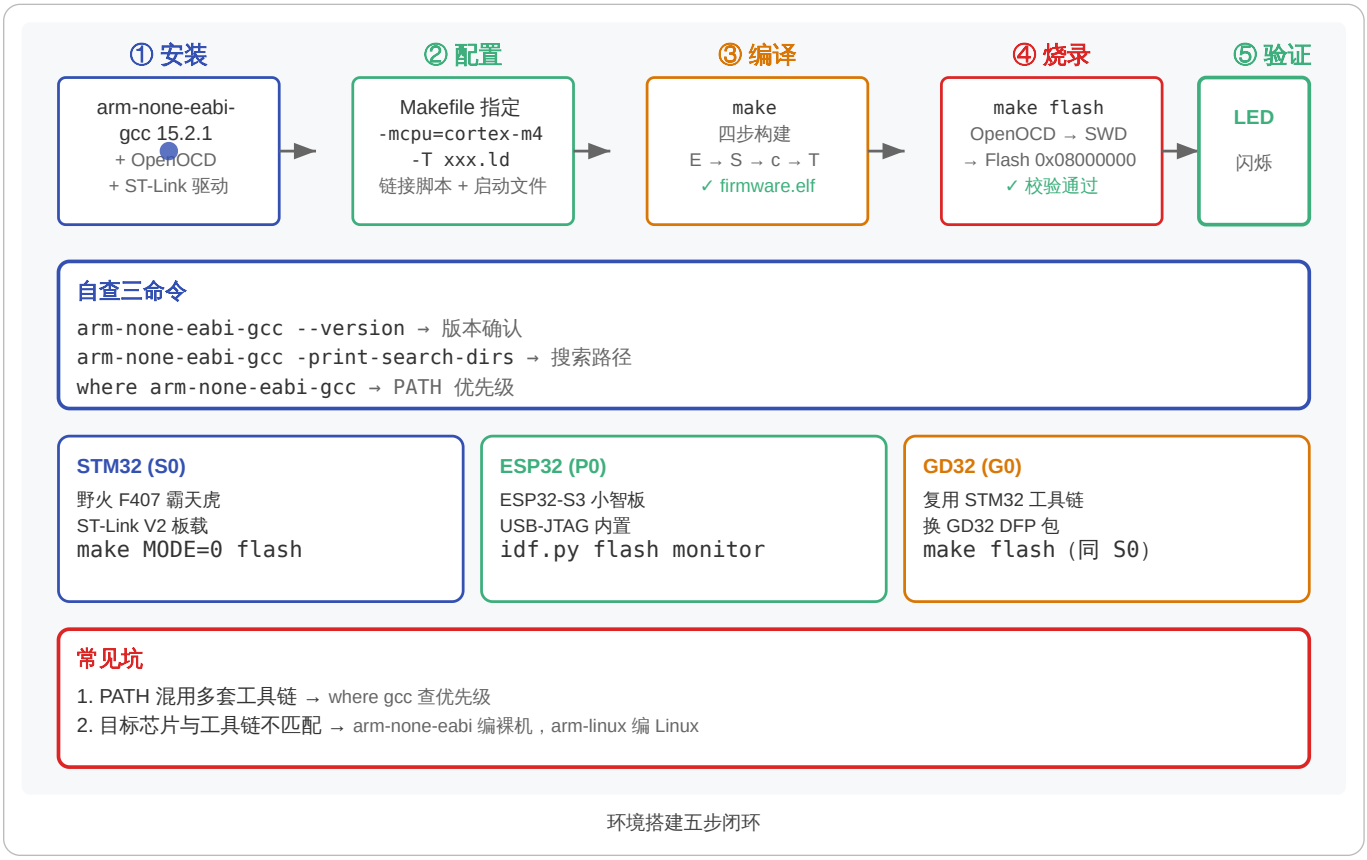
交付

一次完整的构建烧录记录：elf/bin/hex/map 四产物清单（各述用途）、make flash 的 Verified OK 日志，外加 GDB 读 RCC_AHB1ENR bit5=1 的一次会话。

边界

只管 Windows 下四件套装通与第一次点灯：编译链接四步的原理在 B 篇、SWD 协议细节在 B6，均不展开；约 1Hz 闪烁是 HSI 软件延时、频率不准属正常；代码已静态核对 RM0090，板上现象待上板实测反馈。

依据锚点 `a1` · `mcu-code` · `mcu-lab` · `mcu-anim`



环境搭建五步闭环

> 🎯 你买了一台顶配的"全自动咖啡机"（IDE），按一下就出咖啡，但豆子什么时候磨的、水温几度，你一概不知。这一章我们反着来：手动磨豆、手动控温。等你会了，再回去用咖啡机也不迟。

动手之前先押个分工：从一行 C 源码到板上灯亮，这一路要过工具链四件套的手。真正把 C 翻译成 ARM 机器码的是谁？把编译产物送进芯片的又是谁？押完再往下装，记忆锚点那节对答案。

本章精髓

1. 为什么坚持不用 CubeIDE/Keil 起步？因为 IDE 把"预处理→编译→汇编→链接→烧录"藏进一个按钮，而这条链路本身就是嵌入式的半壁江山（B 篇整个都在讲它）。

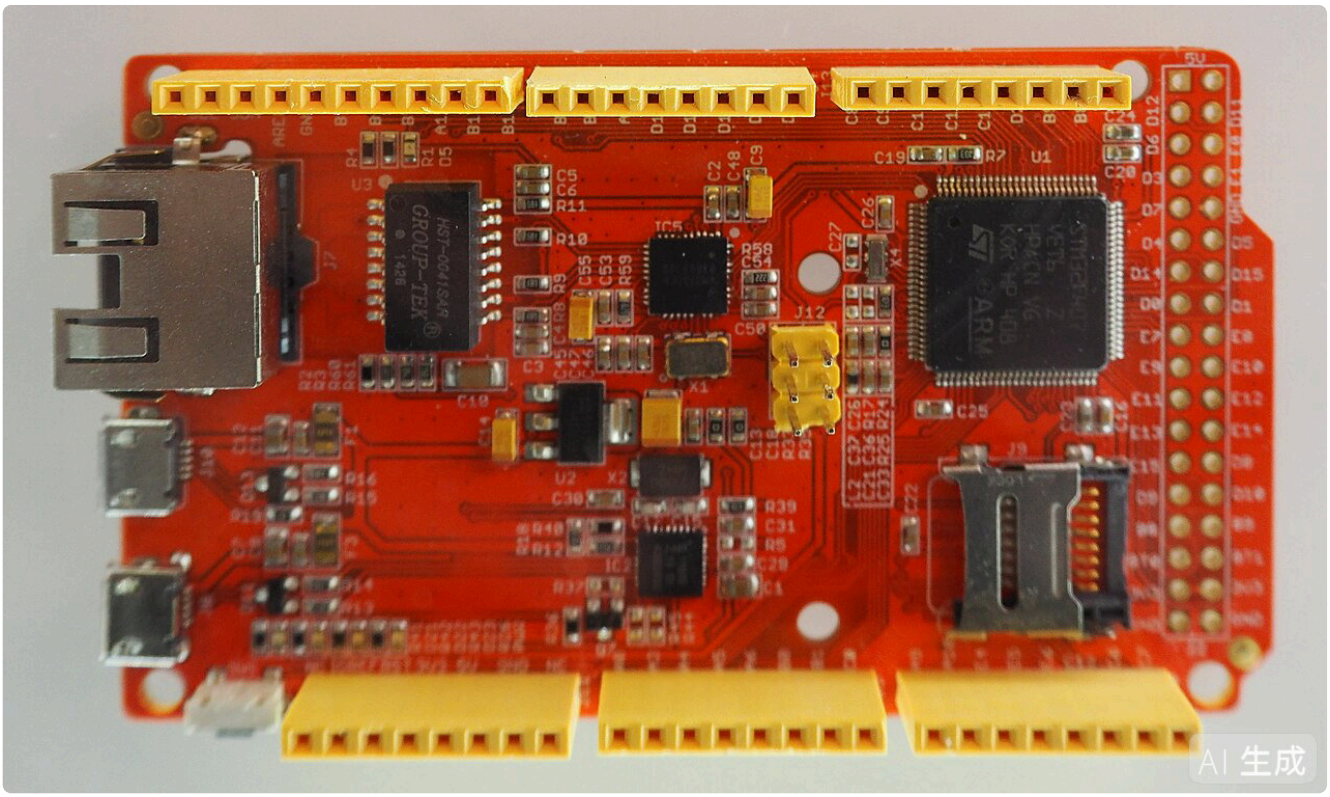
2. 交叉编译到底"交叉"在哪？在 x86 电脑上，产出 ARM Cortex-M4 的机器码。
3. 一个能跑的固件最少需要几个文件？四个：启动文件、链接脚本、main、Makefile。我们已经备好：code/stm32/00-blink。

先修

- 无。这是起点。唯一要求：Windows 10/11 + 一块霸天虎 + 一个 ST-Link（或兼容的 J-Link/DAP）。

装备清单

装备	用途	备注
野火 F407 霸天虎	目标板	板载 RGB 灯 PF6/7/8；板卡外观与原理图见官方资料页
ST-Link V2（或 J-Link/DAP）	烧录+调试	霸天虎**不板载**仿真器，必须自备
Micro-USB 线 ×2	板子供电、仿真器连接	别拿成充电专用线（没数据线芯）
杜邦线若干	接 SWD	若仿真器带排线可省



基于 STM32F407 的第三方开发板

> F407 家族板卡形态参考：图为一款第三方 F407VET6 板（LQFP100）；本站基准板霸天虎为 F407ZGT6/LQFP144，外观不同、芯片同门（来源：Wikimedia Commons，CC0）。

第一步：装 ARM 交叉编译器

1. 打开 Arm 官网开发者页，搜 **"Arm GNU Toolchain"**，下载 Windows 版 `arm-gnu-toolchain-14.x.rel*-mingw-w64-x86_64-arm-none-eabi.exe`。
2. 安装，**最后一步务必勾选 "Add path to environment variable"**。
3. 开一个**新的**终端验证（PATH 改了必须重开终端）：

```
arm-none-eabi-gcc --version
```

看到 `gcc version 14.x.x` 即成功。`arm-none-eabi` 读作"ARM、无厂商、嵌入式应用二进制接口"（专门为裸机 ARM 产代码的 GCC）。

第二步：装 make

Windows 没有自带 make。最省心的来源是 **xPack Windows Build Tools**：

1. 搜 "xPack Windows Build Tools"，下载最新 `xpack-windows-build-tools-*-win32-x64.zip`。
2. 解压到无中文无空格路径（如 `D:\tools\xpack-build-tools`）。
3. 把其 `bin` 目录加入系统 PATH，新开终端验证：

```
make --version
```

第三步：装 OpenOCD（烧录与调试的桥梁）

1. 搜 "xPack OpenOCD"，下载 Windows 版 zip 并解压（同样避开中文/空格路径）。
2. `bin` 目录加入 PATH，验证：

```
openocd --version
```

OpenOCD 的角色：一边通过 USB 跟 ST-Link 说话，一边通过 SWD 协议跟芯片说话，把我们的 `program` 命令翻译成对 Flash 的擦写。SWD 只有两根信号线（SWDIO 数据 + SWCLK 时钟），B6 章会把它讲透。

第四步：ST-Link 驱动

- **正版 ST-Link / ST-Link V2**：装 ST 官方的 **STSW-LINK009** USB 驱动（搜 ST 官网编号）。
- **DAPLink（CMSIS-DAP）**：Windows 10/11 免驱，插上即用。
- 验证：插上仿真器，设备管理器里应出现 "STM32 STLink" 或对应设备，**没有黄色感叹号**。

第五步：接线（SWD 四线）

ST-Link	霸天虎 SWD 座
SWDIO	SWDIO
SWCLK	SWCLK
GND	GND
3.3V	3.3V（仅作检测，不给板子供电；板子单独 USB 供电）

> ⚠️ **注意 · 供电纪律**
>
> 板子用自己的 USB 供电，仿真器的 3.3V 只接检测脚。两路电源对供是烧板的经典姿势。

第六步：构建第一个工程

进入仓库的工程目录，一条命令：

```
cd code/stm32/00-blink
make
```

预期输出：

```
arm-none-eabi-gcc ... -c main.c -o build/main.o
arm-none-eabi-gcc ... -o build/blink.elf
   text    data     bss     dec
   ~200       0       0    ~200   build/blink.elf
```

产物全家福：

产物	是什么
`blink.elf`	带调试信息的完整固件（GDB 用它）
`blink.bin`	纯二进制镜像（可直接按地址烧）
`blink.hex`	Intel HEX 文本格式（烧录器通用）
`blink.map`	每个符号的地址清单——B5 章教你审计它

构建失败？先去对照常见坑，90% 的问题在那里。

第七步：烧录，点灯！

```
make flash
```

OpenOCD 刷出一串日志，关键看三行：

```
** Programming Started **
** Verified OK **
** Resetting Target **
```

然后——**霸天虎上的红灯开始以约 1Hz 呼吸闪烁**。这不是 IDE 里按出来的，是你亲手从四个文本文件变出来的。

第八步（选做）：GDB 看一眼寄存器

```
make debug     # 终端 1：OpenOCD 常驻
make gdb       # 终端 2：连上后自动复位、烧录、断在 main
```

在 GDB 里：

```
continue          # 跑过 GPIO 配置
x/wx 0x40023830   # RCC_AHB1ENR：bit5 应该是 1（GPIOF 时钟开了）
```

`0x40023830` 这个地址不是魔法：RCC 基址 0x40023800 + 寄存器偏移 0x30，查 RM0090 §7.3 一字不差。C2 指针章会讲清"往地址写字"的全部原理。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **工具链四件套：gcc 翻译、make 指挥、OpenOCD 送信、ST-Link 跑腿。** 固件四文件：启动文件开门、链接脚本排座、main 干活、Makefile 记账。

延伸：环境搭建五步闭环动画见 S0 动画；工具链四件套详解在 B0；四步构建流程在 B1 展开。

实物实验

本章就是实验本身。观测点与预期：

- `make` 后 `build/` 下出现 elf/bin/hex/map 四个文件；
- `make flash` 日志出现 `Verified OK`；
- 复位后 PF6 红灯约 1Hz 闪烁（HSI 16MHz 软件延时，频率不准属正常，S5 SysTick 再较真）。

短自测

1. "交叉编译"的"交叉"具体指什么？为什么不能直接用电脑上的 gcc？

> **▼ 参考答案**
> 交叉指"编译环境"与"运行环境"的 CPU 架构不同：你在 x86-64 的 Windows 上编译，产出的是 ARM Cortex-M4 的机器码。电脑上的 gcc 默认产出 x86 指令，Cortex-M 无法执行；`arm-none-eabi-gcc` 才是 ARM 目标的后端。`none` 表示没有操作系统（裸机），`eabi` 是 ARM 嵌入式 ABI。

2. `make` 之后 `build/` 下出现 elf、bin、hex、map 四个文件。分别是什么用途？烧录用哪个？

> **▼ 参考答案**
> **elf**：完整的 ELF 固件，含代码、数据、调试信息和符号表：调试用它的符号，烧录用它的二进制内容。**bin**：纯二进制镜像，从地址 0x08000000 起的逐字节拷贝，烧录最快。**hex**：Intel HEX 格式，每行带地址和校验，可描述不连续的内存区域。**map**：链接器输出的符号地址清单，不含机器码，用于查"函数住在第几字节"。烧录用 bin 或 hex 均可（OpenOCD 两种都接受），调试必须用 elf。

3. Makefile 里命令行首为什么必须是 Tab 而不是空格？报 `missing separator` 怎么修？

> **▼ 参考答案**
> Make 的语法规则：规则体（recipe）的每一行必须以 **Tab 字符**开头，空格不被识别为命令行。这是 1970 年代 Make 的原始设计，至今未改。某些编辑器（VS Code 默认不犯，但 Notepad++ 和某些 IDE 的"智能缩进"模式会）会把 Tab 自动替换成空格。修复方法：在编辑器里关闭"用空格替换 Tab"，或直接用 `sed -i 's/^ /\t/' Makefile` 把行首四空格换成 Tab。

常见坑

- **`'arm-none-eabi-gcc' 不是内部或外部命令`**：PATH 没生效。装了新工具必须**重开终端**；还不行就是安装时没勾 Add path，手动加。
- **make 报 `missing separator`**：Makefile 的命令行首必须是 **Tab 不是空格**。某些编辑器会把 Tab 转成空格。
- **OpenOCD 报 `Error: open failed` / 找不到 ST-Link**：九成是驱动（回第四步）；一成是杜邦线虚接：SWD 对接触很敏感，重新插拔。
- **灯不亮但烧录成功**：先怀疑引脚号。霸天虎是 **PF6**；网上大量教程是其他板子的 PA8/PF9，照搬必翻车。
- **解压路径有中文/空格**：xPack 系工具对非 ASCII 路径敏感，装 `D:\tools\` 这种纯英文路径。
- **两台 USB 设备抢供电**：仿真器 3.3V 和板子 USB 供电不要同时怼，见第五步警告。

你做到了

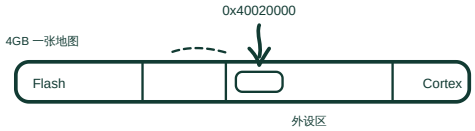
- 电脑变成了交叉编译工作站，四件工具各司其职。
- 亲手构建了固件四产物，知道每个是干什么的。
- 红灯在你自己的代码控制下闪烁，裸机世界的 Hello World 达成。

✅ 下一站：S1 F407 架构总览：0x40020000 凭什么就是 GPIOA？总线矩阵和存储器映射把门牌号拆开给你看。状态：本章代码已静态核对 RM0090，**待上板实测反馈**（欢迎 Issue 报告你的现象）。

通往单片机之路第 19 章

S1 F407 架构总览：地址背后的故事

难度 ★★☆☆☆ · 约 30 分钟



本章罗盘

问题

寄存器地址是背出来的还是推出来的，GPIOA 为什么恰好是 0x40020000，位带别名地址怎么算？

交付

一张 4GB 分区骨架图（0800/2000/4000/E000 四区口诀展开）加一笔 GPIOF->ODR bit6 的位带别名手算 0x42428298，与 probe.sh 的 _Static_assert 和 -O2 反汇编互证。

边界

位带只覆盖 SRAM/外设区前 1MB，别名公式不能外推到全区；总线仲裁的动态翻车实验（DMA 够不着 CCM）在 S8；位带断言与反汇编已在本机复现，「灯亮/不亮」的肉眼现象待上板回填。

依据锚点 `a1 · a3 · f3 · mcu-code · mcu-anim`

> 🎯 背寄存器地址是最笨的学法。GPIOA 住在 0x40020000 不是天意，是 ST 画"城市地图"时划的门牌：Cortex-M4 统一编址 4GB，外设区从 0x40000000 起，AHB1 从 0x40020000 起，GPIO 每端口隔 0x400——GPIOA 自然就是 0x40020000。

本章精髓

1. 统一编址的世界观：Flash/SRAM/外设/内核寄存器全在一张 4GB 地图上，"访问外设"就是"访问某个地址段"，这正是 C2 点灯公式合法的根源。
2. 总线矩阵决定"谁快谁慢"：内核经 I-Code/D-Code 取指取数，经 System 总线碰外设；AHB1（168MHz）/APB1（42MHz）/APB2（84MHz）三档速度，选错总线时钟源外设直接不干活。
3. 位带（bit-band）别名：把 SRAM/外设区的每一位映射成别名区的一个字：写别名地址=原子改一位，读-改-写隐患的官方解法。

- **能记住**：四句口诀：`0800` Flash、`2000` RAM、`4000` 外设、`E000` 内核；AHB1 上 GPIO、每口隔 `0x400`
- **能理解**：总线矩阵为什么能"边搬边算"；AHB/APB 分频怎么定外设上限；位带公式怎么把 1 位变成 1 个字
- **能用**：拿到任意外设名推出基址与所属总线；会用位带别名做单比特原子翻转，且知道它只覆盖前 1MB

先修

- S0 环境搭建；C2 指针。

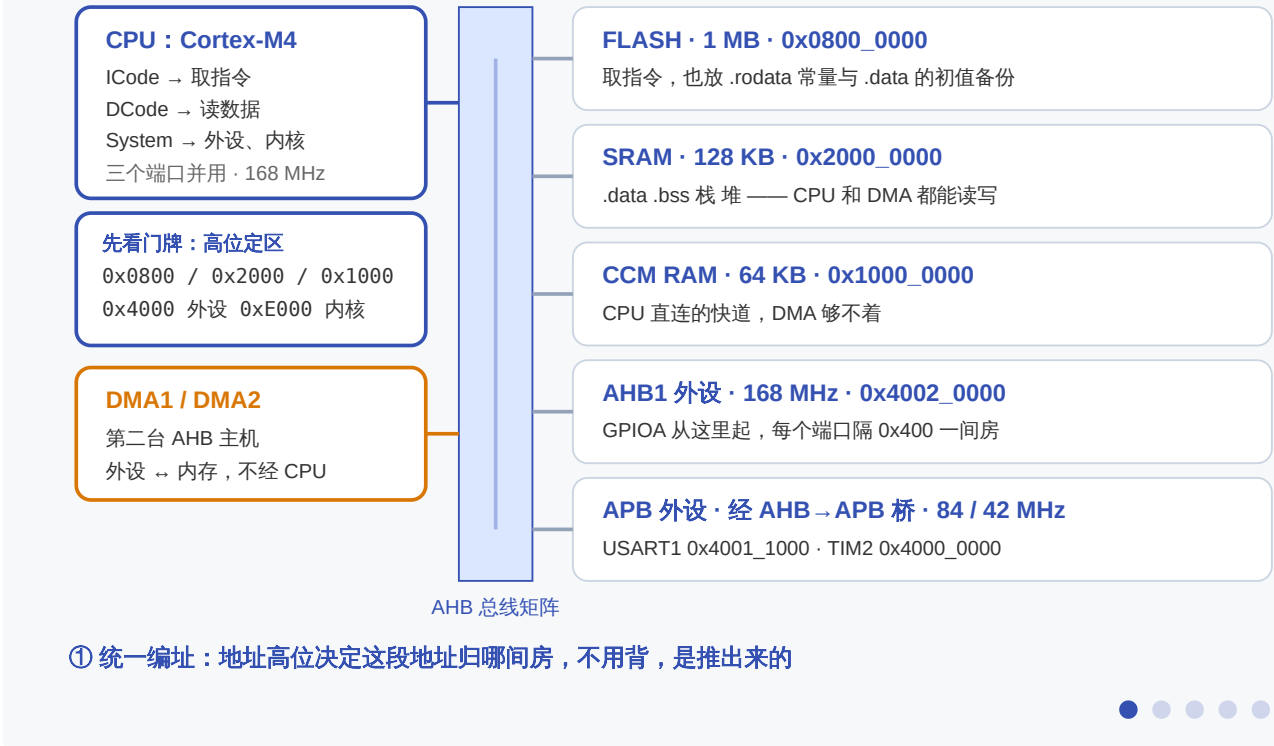
先跑起来（10 分钟 quick win）

GDB 里连读三个地址：`x/wx 0x40020000`（GPIOA_MODER）、`x/wx 0x40021400`（GPIOF_MODER）、`x/wx 0xE000ED00`（CPUID），三个街区各看一眼，地图感立刻建立。

连上之前，先押个名字：三位住户里，两位住在同一片外设街区，只有一位不算外设，是内核自家人。是 GPIOA、GPIOF，还是 CPUID？押定再读，地图感从猜的那一刻开始长。

动画：一张矩阵上的五次访问

把本章精髓第 2 条播成电影：CPU 的 ICode/DCode/System 三个端口和 DMA 一起挂在 AHB 总线矩阵上，五个阶段各走一条真实路线：**取指**每拍打到 Flash（168 MHz 下 5 个等待周期，靠预取缓冲兜住，见 S2）；**读 `USART1->SR`** 要穿 AHB→APB2 桥，两个时钟域之间还得同步；**DMA** 是矩阵上的第二台主机，搬得不占指令周期，但要跟 CPU 抢同一块 SRAM；最后一阶段故意走不通——**CCM RAM 只接到 CPU**，DMA 缓冲区放错了地方就是无声失败（S8 有翻车实验）。分区归属依据 RM0090 的存储器映射表；等待周期那条是 S2 已核过的数字，不是估的。



总线矩阵动画

动画：位带——一个位搬进一个字

精髓第 3 条的现场版。`GPIOF->ODR` 在 `0x4002_1414`，它的 8 个位在别名区各占一个字：`0x4242_8280`（bit0）到 `0x4242_829C`（bit7）。**地址是算出来的**：`0x4200_0000 + 0x21414×32 + 6×4 = 0x4242_8298`，`sh code/stm32/01-arch/probe.sh` 里三条 `_Static_assert` 替我们验算，编不过就是算错了。同一份 `-O2` 反汇编还把老问题摆了出来：`GPIOF->ODR |= 1u<<6` 编成 `ldr / orr.w / str` 三条，读与写之间有窗口；写别名字只剩 `movs #1` + `str.w [r3,#664]`，`BSRR` 同样一条 `str` 搞定。

位带别名动画

板卡与版本前提

- 板卡：野火 F407 霸天虎，F407ZGT6：168MHz、1MB Flash、192KB SRAM（SRAM1 112KB + SRAM2 16KB + CCM 64KB）；RGB 灯 PF6/7/8 共阳。
- 存储器映射与分区地址以 RM0090 §2.3（表 1）为准；位带机制以 PM0214 §2.2.5 为准。
- 寄存器基址/字段以 ST 官方 CMSIS 头 `stm32f407xx.h`（github.com/STMicroelectronics/cmsis_device_f4）为准：GPIOF= `0x40021400`、RCC= `0x40023800`、`GPIOF->ODR` 偏移 `0x14`、`BSRR` 偏移 `0x18`。
- 本章位带断言与反汇编由 `code/stm32/01-arch/probe.sh` 在本机用 `arm-none-eabi-gcc 15.2.1` 复现；**"灯亮/不亮"的肉眼现象待上板回填**，编译期结论与板上现象分开记。

小节结构

小节	内容	四件套
4GB 地图	八大分区速览；代码/SRAM/外设/内核四区重点	配置
总线矩阵	I/D/S 三总线与 DMA 的仲裁；AHB/APB 桥与分频（上面那张动画逐条演示）	配置
外设门牌推导	AHB1→GPIO、APB1→UART2-5、APB2→USART1/6 的推导练习	引脚
位带机制	别名地址公式 + ODR 单比特翻转实验（上面那张动画配 `probe.sh` 实测反汇编）	代码分析

一、4GB 地图：一块芯片怎么分房

Cortex-M4 是 32 位机，地址空间正好 4GB。ARM 架构把这 4GB 切成固定八段，谁也不能改；ST 只能在自己的段里再划分。

区段	地址范围	放什么
Code	`0x0000_0000` – `0x1FFF_FFFF`	Flash 与其别名、系统存储区
SRAM	`0x2000_0000` – `0x3FFF_FFFF`	片上 SRAM
Peripheral	`0x4000_0000` – `0x5FFF_FFFF`	全部外设寄存器
RAM	`0x6000_0000` – `0x7FFF_FFFF`	外部 RAM（经 FSMC/FMC）
Device	`0x8000_0000` – `0x9FFF_FFFF`	外部设备
保留	`0xA000_0000` – `0xDFFF_FFFF`	—
PPB	`0xE000_0000` – `0xFFFF_FFFF`	内核私有外设（NVIC/SysTick/SCB）

F407 在其中的实际落位（RM0090 §2.3 表 1）：

- `0x0800_0000`：主 Flash 1MB；同一块 Flash 在 `0x0000_0000` 还有个别名，上电从哪取指由 BOOT 引脚决定（B4 启动）。
- `0x1FFF_0000`：系统存储区，出厂固化的 bootloader 住这。
- `0x2000_0000`：SRAM1 112KB 紧接 SRAM2 16KB，共 128KB 连续。
- `0x1000_0000`：CCM RAM 64KB，只挂在 CPU 的数据总线上，**DMA 够不着**（S8 的翻车实验）。
- `0x4000_0000` 起：外设。
- `0xE000_0000` 起：PPB，`CPUID` 就在 `0xE000_ED00`。

"192KB RAM" 的账就是这么来的：128KB 通用 SRAM + 64KB CCM。

二、总线矩阵：谁快谁慢

Cortex-M4 内核单元：FPU/MPU/NVIC/DWT/ITM

内核不只有一根总线。Cortex-M4 有三个出口：**I-Code** 取指令、**D-Code** 取数据/常量（两者只覆盖 Code 区）、**System** 管其余一切（外设、SRAM、PPB）。

F407 把这些口和 DMA1、DMA2、以太网 MAC、USB HS 一起接到一张多层 AHB 总线矩阵上，再连向 Flash 接口、SRAM、AHB1/AHB2 外设、FSMC。矩阵是**并发**的：DMA 在搬 SRAM 时，CPU 能同时从 Flash 取指，这正是 S8 DMA 敢说"不占指令周期"的底气。

时钟分三档（S2 时钟树 算过）：

总线	频率	说明
AHB1/AHB2（HCLK）	168MHz	GPIO、DMA、RCC 等
APB1	42MHz	HCLK/4，硬件上限 42MHz
APB2	84MHz	HCLK/2，硬件上限 84MHz

APB 分频不为 1 时，挂在 APB 上的定时器拿到的是 `APB×2` 的时钟，S6 会用到。

Flash 取指在 168MHz 下要 5 个等待周期（RM0090 §3.3.1），靠 ART 加速器和预取缓冲把这 5 拍的坑填上。上面那张矩阵动画的"取指"阶段演示的就是这条路径。

三、外设门牌推导

外设地址不是背出来的，是推出来的。以 RM0090 存储器映射表为底：

- **AHB1** 段基址 `0x4002_0000`：GPIOA `0x4002_0000`，每端口隔 `0x400`：GPIOB `0x4002_0400`、…、GPIOF `0x4002_1400`。RCC 在 `0x4002_3800`，DMA1 `0x4002_6000`、DMA2 `0x4002_6400`。
- **APB1** 段基址 `0x4000_0000`：USART2 `0x4000_4400`、USART3 `0x4000_4800`、UART4 `0x4000_4C00`、UART5 `0x4000_5000`。
- **APB2** 段基址 `0x4001_0000`：USART1 `0x4001_1000`、USART6 `0x4001_1400`、SPI1 `0x4001_3000`。

记住归属比记住地址更重要：USART1 在 APB2（上限 84MHz）、USART2 在 APB1（上限 42MHz），波特率算错的头号原因就是把总线段记反了（S7 USART）。

四、位带：把一位搬进一个字

位带（bit-banding，PM0214 §2.2.5）给两个区各开了一片"影子地址"：

位带区	地址范围	别名区
SRAM	`0x2000_0000` – `0x200F_FFFF`	`0x2200_0000` – `0x23FF_FFFF`
外设	`0x4000_0000` – `0x400F_FFFF`	`0x4200_0000` – `0x43FF_FFFF`

换算公式：

```
别名地址 = 别名区基址 + (字节地址 − 位带区基址) × 32 + 位号 × 4
```

原位带区一个**位**，在别名区占一个**字**（32 位）；1MB 位带区于是撑出 32MB 别名区。写别名地址=硬件替你完成"读—改—写"，一条 `str` 原子改一位。

实例：`GPIOF->ODR` 的字节地址是 `0x4002_1414`，它的 bit6 别名 =

```
0x4200_0000 + (0x4002_1414 − 0x4000_0000) × 32 + 6 × 4 = 0x4242_8298
```

`probe.sh` 里三条 `_Static_assert` 把 `ODR` 地址、PF6 别名 `0x42428298`、别名区跨度都钉死，算错一个数就编不过。同一份 `-O2` 反汇编还并排摆出两种写法的差别：`GPIOF->ODR |= 1u<<6` 编成 `ldr/orr.w/str` 三条，读与写之间开着窗口；写别名只剩 `movs #1` + `str.w`，`BSRR` 同样一条 `str` 搞定。

附录：工程完整源码

本章全部取证出自下面两个文件：`bb_probe.c` 是被反复反汇编的样本，`probe.sh` 是把它编出来、算出别名地址、打印反汇编的入口。

<<< ../../code/stm32/01-arch/bb_probe.c

<<< ../../code/stm32/01-arch/probe.sh

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **0800 是 Flash，2000 是 RAM，4000 是外设，E000 是内核；AHB1 上 GPIO，每口 400 一间房。**

延伸：地址空间 layout 是理解指针（C2）和链接脚本（B3）的物理基础；总线矩阵仲裁见 S8 DMA。

实物实验

- 位带实战：用别名地址翻转 PF6 的 ODR 位（不写 BSRR），观察红灯变化，体会"原子改一位"。

常见坑

- **APB1 当 APB2 用**：USART2/3 在 APB1（42MHz 上限），波特率算错源头常在总线归属记错。
- **位带去碰多比特字段**：位带一次只能改一位，改两位字段请回 BSRR/掩码。
- **以为所有地址都能读**：未实现的地址段访问直接 HardFault，GDB 乱读也会踩雷（S16 联动）。

短自测

1. GPIOB 的基址是多少？说出你的推导。

> **▼ 答案**
> `0x4002_0400`。AHB1 段基址 `0x4002_0000`，GPIO 每端口隔 `0x400`：GPIOA `0x4002_0000` → GPIOB `0x4002_0400`（GPIOF 就是 `0x4002_1400`，本章 ODR 实验用的那个）。

2. 位带别名公式是什么？为什么 1MB 位带区要撑出 32MB 别名区？

> **▼ 答案**
> `别名 = 别名区基址 + (字节地址 − 位带区基址) × 32 + 位号 × 4`。原位带区一个**位**在别名区占一个**字**：1 位换 32 位，所以 1MB × 32 = 32MB。乘 32 的来历就是"位→字"的放大倍数。

3. CCM RAM 在哪个地址？为什么 DMA 缓冲区不能放那儿？

> **▼ 答案**
> `0x1000_0000`，64KB。它只挂在 CPU 的 D-bus 上，不在 AHB 总线矩阵上，DMA 控制器根本看不到这段地址，放进去的缓冲区会静默丢数据（[S8](08-dma.md)）。

4. USART1 和 USART2 分别在哪个总线段、时钟上限各是多少？

> **▼ 答案**
> USART1 在 APB2（`0x4001_1000`，≤84MHz）；USART2 在 APB1（`0x4000_4400`，≤42MHz）。把 USART2 当成 84MHz 算波特率，误差能大到收不到数据。

对照表：本章概念 → 仓库落点

概念	仓库落点
位带别名断言与反汇编	`code/stm32/01-arch/probe.sh`、`bb_probe.c`（本附录全文内嵌）
PF6 别名地址 `0x42428298`	同上 `bb_probe.c` 的 `_Static_assert`，与本章第四节算式互证
`|=` 读改写窗口	同上 `-O2` 反汇编 `or_bits`（`ldr/orr.w/str`）
单条 `str` 原子写	同上 `alias_set` / `bsrr_set`
GPIO 寄存器布局	`code/stm32/00-blink/main.c`，CMSIS `stm32f407xx.h` 的 `GPIO_TypeDef`
启动与 Flash 别名	B4 启动、`code/stm32/00-blink/startup_stm32f407xx.s`
总线时钟与等待周期	S2 时钟树

你做到了

- 寄存器地址从"要背的表"变成"能推的图"；
- 拿到全站反复使用的 F407 地图。

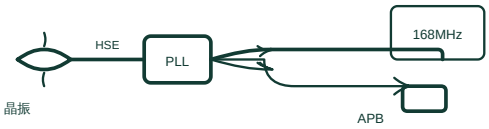
名字押对了吗？CPUID 不住外设街区，它挂在内核自家的私有地界，口诀最末那句从此有了实物。

✅ 下一站：S2 RCC 时钟树：全片外设的"心脏起搏器"归它管，168MHz 这笔账一章算清。

通往单片机之路第 20 章

S2 RCC 时钟树：168MHz 是怎么算出来的

难度 ★★★☆☆ · 约 40 分钟



本章罗盘

问题

从 8MHz 晶振到 168MHz：M/N/P/Q 怎么选才落进芯片允许的窗口？提频六步的顺序为什么不能乱？

交付

一笔任意晶振的 M/N/P/Q 手算（每项落在 RM0090 允许范围并给出寄存器字 0x07405408）、提频六步顺序表，以及 probe.sh 宿主断言全绿加 -O2 字面量池同字对账输出。

边界

HSE 晶振值未核实：正文以 8/25MHz 两例演示，套用前以板卡规格书为准；宏填错时回读自洽但引脚频率错（差 114240000Hz 的那组实账），MCO1 实测对账待上板；PLL 抖动与频谱特性不展开。

依据锚点 `a1` · `a2` · `f3` · `f4` · `mcu-code` · `mcu-anim`

> 🎯 时钟树是 STM32 的"配电网"：发电厂（HSE/HSI）→ 变电站（PLL 倍频）→ 输电线路（AHB/APB 分频）→ 每家每户的电表（外设使能位）。停电排查从电表开始，所以"先时钟、再模式、后数据"是我们的祖训。

本章精髓

1. **PLL 公式一行写尽**：`SYSCLK = HSE / M × N / P`。但每个字母都有芯片给的取值范围（RM0090 §7.2.3）：PLLM∈[2,63] 且 VCO 输入必须 1~2MHz。PLLN∈[50,432] 且 VCO 输出必须 100~432MHz。PLLQ∈[2,15] 且必须让 VCO/PLLQ **正好** 48MHz（USB OTG FS 专用）。
2. **分频有连锁反应**：AHB(/1) → APB1(/4)/APB2(/2)。挂在 APB 上的定时器还有一条隐藏规则：**APB 分频 ≠ 1 时定时器时钟 ×2**，所以 84MHz 的 APB1 上，TIM 时钟其实是 84×2=168MHz 的一半……不对，是 42×2=84MHz。这一条在 S6 会用实验实测。
3. **Flash 等得及吗**：168MHz 下 Flash 取指要 5 个等待周期，而且这张表按**供电电压分列**（RM0090 Table 10），不按电压缩放档分列。电压档（VOS）只决定 HCLK 的天花板：VOS='0' 最多 144MHz，VOS='1' 才允许 168MHz。

- **能理解**：为什么 1MHz 是 PLL 的"设计目标值"而不是外设时钟；为什么改完分频要回读 SWS 才算数；为什么把晶振值填错不会报错、只会全盘算错。
- **能用**：给任意 HSE 晶振手算一组合法的 M/N/P/Q；在 `01-rcc-clock` 上电后切到 168MHz 并用 MCO1 量到它。

先修

- S1 架构（AHB/APB 归属）；C4 结构体。

先跑起来（10 分钟 quick win）

没有板子也能把本章的账全跑完。进 `code/stm32/01-rcc-clock`：

```
sh probe.sh          # 三步：宿主跑固件自己的函数 → 错宏对照 → 交叉链 -O2 反汇编对账
```

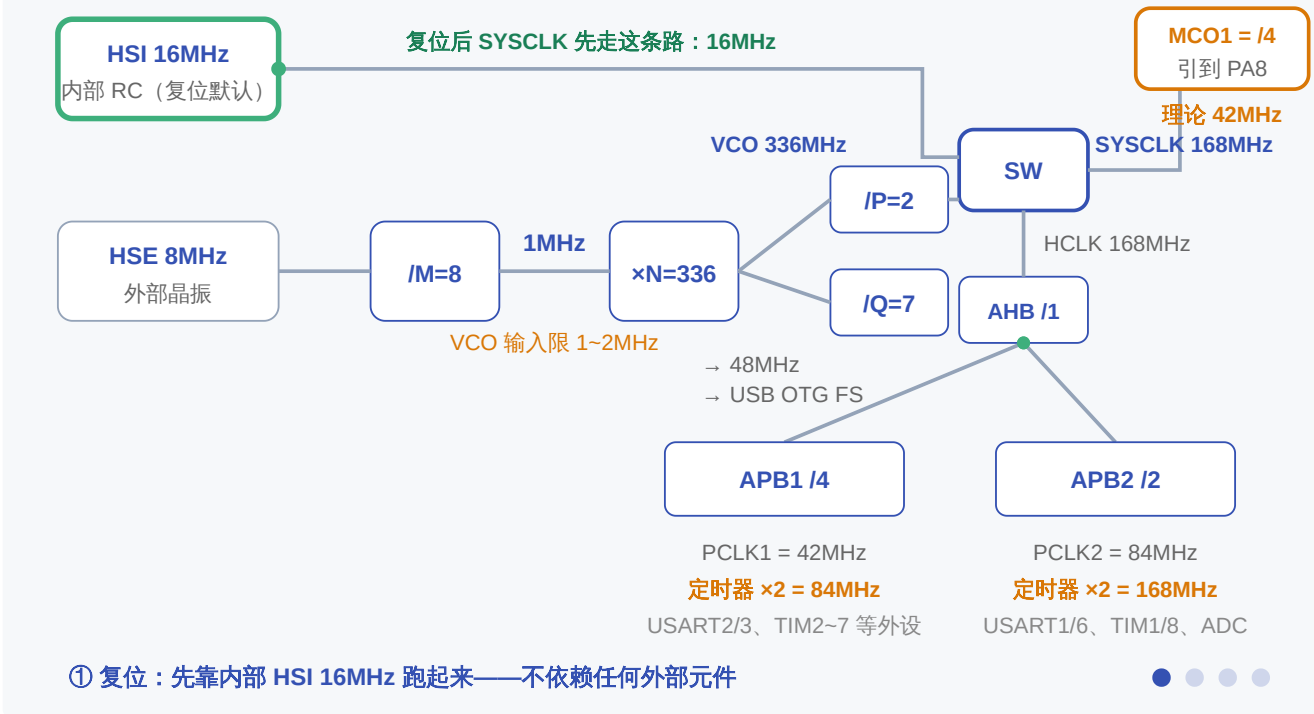
第 1 步把 `RCC_PLLCFGR` / `RCC_CFGR` 两个宏改指一对假寄存器后 `#include "main.c"`，所以断言跑的是固件本体而不是另抄一遍算术：八种晶振的 M/N/P/Q、`0x07405408` 的位段账、等待周期十个边界、时钟树回读解出的 `{168e6, 168e6, 42e6, 84e6, 84e6}`，共几十条断言全绿。第 3 步把 `main.c` 编成 Cortex-M4 的 `-O2` 目标文件，在字面量池里当场挖出同一个 `0x07405408`：**源码算出来的字 == 落到寄存器里的那个字**。

有板子再加一步：构建默认版本（留在 HSI 16MHz，不碰晶振）`make`，GDB 读 `g_clock_tree[]`，你会看到 `{16000000, 16000000, 16000000, 16000000, 16000000}`：芯片一出生就在跑 HSI 16MHz，PLL 还没上岗。这就是"先时钟"的直观证据。这句不用上板也能验：默认版编 `-O2` 后，字面量池里 `0x07405408` 和 PLLON 那个 `#16777216` **都不存在**，对 `#2052`（PLLCFGR）唯一的一次引用是一条 `ldr`：回读函数要读它，但没人写它。

先别急着跑，押个数：默认版固件一行都没碰晶振，芯片一上电跑的会是哪个钟？出厂自带的内部钟，还是外面的晶振？押完再跑，g_clock_tree 那五行数字就是裁判。

动画：时钟树的五个闸门

HSE/HSI 两个水源、PLL 这个泵站、SW 这个总闸、AHB/APB 两条输电线，每一段"通了/断了"都有对应的寄存器位。盯紧最下面的"对账"小窗：理论值先算对，上板再用仪器量回来：程序以为的时钟和引脚上量到的时钟必须一致。



RCC 时钟树动画

板卡与版本前提

- **HSE 晶振值未核实**：正文用 8MHz 与 25MHz 两例做手算演示，套用前以你手上板卡的规格书/原理图为准；工程默认 `HSE_VALUE_HZ=8000000`，这个宏改错了程序不会报错，只会全盘算错（见常见坑）。
- 寄存器位定义以 ST 官方 CMSIS 头文件 `cmsis_device_f4`（commit `9192c7b9`）为准；PLL 取值范围与 Flash 等待周期以 RM0090 Rev 18 为准。
- 本机工具链：xPack GNU Arm Embedded GCC 15.2.1 + GNU Make 4.4.1。

小节结构

小节	内容	四件套
一、四个时钟源	HSI/HSE/LSI/LSE 分工与精度对比	配置
二、PLL 手算	M/N/P/Q 逐段拆解；8MHz 与 25MHz 两例算到底	配置
三、提频六步	顺序 + 每步对应的寄存器位 + 回读确认	代码分析
四、总线分频与定时器×2	HPRE/PPRE 编码；APB 分频≠1 时定时器时钟×2	代码分析
五、Flash 与电压档	RM0090 Table 10 全表 + VOS 天花板	配置
六、MCO1 实测	PA8 复用输出 SYSCLK/4，仪器验证	引脚
七、代码分析	`01-rcc-clock/main.c` 逐段对表 + `probe.sh` 无板取证（固件本体跑断言、`-O2` 反汇编对账）	代码分析
八、上游对照	HAL `OscConfig/ClockConfig` 逐行对照（hal_rcc.c @ 1f6451c）；SPL 一手源码仍缺	库解析

一、四个时钟源：发电厂四种

源	精度	何时用
HSI（内部 RC，16MHz）	±1% 量级	复位默认、调试用、没有晶振时兜底
HSE（外部晶振，4~26MHz）	取决于晶振	需要精确波特率/定时时必选
LSI（内部低速 RC，~32kHz）	差	独立看门狗
LSE（外部 32.768kHz 晶振）	好	RTC

复位之后芯片只用 HSI 16MHz。这意味着两件事：第一，什么都不配也能跑（`00-blink` 就是这样）；第二，**只要想用更高的频率，就得亲手走一遍提频流程**。

二、PLL 手算：M/N/P/Q 一个都不能猜

公式（RM0090 Rev 18 §7.2.3）：

- `f(VCO) = f(PLL 输入) × PLLN / PLLM`
- `f(SYSCLK) = f(VCO) / PLLP`
- `f(48MHz) = f(VCO) / PLLQ`

约束（同一节的原文）：PLLM∈[2,63]，且必须让 **VCO 输入落在 1~2MHz**（原文推荐 2MHz 以限制抖动）。PLLN∈[50,432]，且 **VCO 输出必须在 100~432MHz**。PLLP 取 /2 或 /4。PLLQ∈[2,15]，且 USB OTG FS 需要正好 48MHz。

8MHz 晶振、VCO 输入 1MHz（最常见的教程写法）：

参数	值	怎么来的
M	8	8MHz / 8 = 1MHz（VCO 输入）
N	336	1MHz × 336 = 336MHz（VCO）
P	2	336 / 2 = 168MHz（SYSCLK）
Q	7	336 / 7 = 48MHz（USB）

8MHz 晶振、VCO 输入 2MHz（RM0090 推荐的低抖动写法）：M=4、N=168、P=2、Q=7。VCO 仍是 336MHz，SYSCLK 与 USB 结果不变，只是参考输入更干净。

25MHz 晶振（另一种常见板子）：M=25 → 1MHz，其余与上面相同。**M 的职责就是把晶振归一化到 1MHz（或 2MHz）**，不是"越大越好"。

> 【注】 1MHz 这个数只是设计目标，不是规范值。规范给的是 1~2MHz 的窗口；选 1 还是 2 取决于你愿不愿意为了更低的抖动牺牲一点点配置便利。这也回答了"为什么教程里永远是 M=8/N=336"：那不是唯一解，只是最常抄的解。

> 🧮 这里在在线版是一个可交互组件（`ClockCalc`，改参数即时出数）；离线书请按正文给定的参数手算，或到在线版该页把玩。

三、提频六步：顺序就是正确性

完整的提频流程，每步都有对应的寄存器位：

1. **电压档**：`PWR_CR.VOS = 1`（bit14，Scale 1，解锁 168MHz）。注意 F405/407 **没有** VOSRDY 就绪位（那是 F42x/43x 的），写入即生效，不存在可等待的硬件回执。
2. **Flash 等待周期**：`FLASH_ACR.LATENCY = 5WS`（[2:0]），并开预取 `PRFTEN`（bit8）。这一步必须在提速之前，否则 CPU 去 Flash 取指会超时跑飞。
3. **开 HSE 并等就绪**：`RCC_CR.HSEON = 1`（bit16），轮询 `HSERDY`（bit17）。晶振虚焊/负载电容不对时 HSERDY 永远不来，**必须带超时回退 HSI**，不能死等。
4. **配 PLL 并启动**：`RCC_PLLCFGR` 写 M/N/P/Q/SRC（M[5:0]、N[14:6]、P[17:16]、SRC[22]、Q[27:24]），然后 `RCC_CR.PLLON = 1`（bit24），轮询 `PLLRDY`（bit25）。
5. **切 SYSCLK**：`RCC_CFGR.SW = 10b`（[1:0]），然后**读 `SWS`（[3:2]）确认硬件真的换了**。写完 SW 不等于已切换。
6. **总线分频**：`RCC_CFGR.HPRE/PPRE1/PPRE2`。AHB=/1，APB1=/4（上限 42MHz），APB2=/2（上限 84MHz）。

> 【注】 RM0090 对 HPRE 有一条容易被忽略的 caution："分频值在写入 HPRE 后的 1~16 个 AHB 周期内才生效"。推论：**升频前先把总线分频降下来**，否则新频率会瞬间喂给一条还是老分频的总线，超过上限一拍就可能跑飞。降频则可以直接做。

提频六步：先降速，再提速

四、总线分频与定时器 ×2

AHB 上挂着 CPU、内存、DMA、GPIO；APB1/APB2 上挂着外设。所以"总线分频"不只是数字游戏，它决定每个外设能拿到多快的时钟。

还有一条不在框图里、但在寄存器说明里的规则：**当 APBx 预分频 ≠ 1 时，挂在这条 APB 上的定时器时钟 = PCLK × 2**。这条规则在 S6 会用 `02-tim-pwm` 实测出来：把 APB1 从 /1 切到 /2，PCLK1 减半，但 TIM3 的时钟纹丝不动。

五、Flash 与电压档：RM0090 Table 10

等待周期按 **VDD 电压范围** 分列（RM0090 Rev 18 Table 10）：

VDD	0WS	1WS	2WS	3WS	4WS	5WS	6WS	7WS
2.7~3.6V	≤30	≤60	≤90	≤120	≤150	≤168	—	—
2.4~2.7V	≤24	≤48	≤72	≤96	≤120	≤144	≤168	—
2.1~2.4V	≤22	≤44	≤66	≤88	≤110	≤132	≤154	≤168
1.8~2.1V（预取关）	≤20	≤40	≤60	≤80	≤100	≤120	≤140	≤160

VOS 不在这张表里，它只决定天花板：F405/407 在 VOS='0' 时 fHCLK 上限 144MHz，VOS='1' 时 168MHz。所以"能不能跑 168MHz"是两步检查：先过电压档（VOS=1），再过等待周期（3.3V 下 5WS）。

六、MCO1 实测：把内部时钟引出来看

`RCC_CFGR.MCO1PRE` ([26:24]) 选一个分频比，把 SYSCLK 送到 PA8（AF0，见 DS8626 Table 9）。/4 之后 168/4=42MHz，正好落在示波器/逻辑分析仪的舒服区间。**量到的数乘 4 就是 SYSCLK**，这是"程序以为"与"硬件实际"之间最短的对账路径。

七、代码分析：`01-rcc-clock/main.c`

逐段对表（源文件见工程目录，行号以当前提交为准）：

- `systick_init()`：SysTick 的 LOAD 用 **HCLK** 而不是 APB 时钟，这是延时函数与时钟解耦的第一步。
- `flash_latency_ws_3v3()`：只实现 2.7~3.6V 那一列（开发板常见供电），其余电压范围在注释里给全。
- `flash_config()`：先 VOS 再 LATENCY 再预取；没有"等 VOS 生效"的自旋，因为 F405/407 没有那个就绪位。

- `pll_params_from_hse()`：由晶振频率推 M/N/P/Q，超出合法范围就拒绝，而不是硬凑。
- `clock_init()`：六步顺序逐行对应第三节；HSE 起振带超时，超时回退 HSI 并把 `g_clock_status` 置 1。
- `clock_tree_readback()`：从 CFGR 反推真实时钟树，含"APB 分频≠1 时定时器 ×2"这条隐藏规则。
- `mco1_init()`：把 SYSCLK 引到 PA8；分频编码交给纯函数 `mco1_pre_for()`，好让宿主探针能直接调它。

`probe.sh`：把这一章的账全部搬到宿主机上跑

手法只有一句：`probe.c` 先把 `RCC_PLLCFGR` / `RCC_CFGR` 两个宏改指到一对 `static uint32_t` 假寄存器，再 `#include "main.c"`（顺带 `#define main firmware_main` 让位给宿主的 `main`）。于是下面这些断言跑的是**固件里那几段函数本体**，不是另抄一遍算术，这也正是"回读算出来的数"这种逻辑唯一能被离线验证的方式。

本机实跑（x86 gcc 16.1.0 + xPack 15.2.1，`sh probe.sh`）：

探针小节	实跑结论
1 · `pll_params_from_hse()`	8/25/16/12/6/4 MHz 六种晶振全部 M=晶振/1MHz、N=336、P=/2、Q=7 → SYSCLK 168000000、USB 48000000；11.0592 MHz 返回 **−2**（除不出整兆），64 MHz 返回 **−1**（M 溢出 6 位）
2 · 位段打包	`M=8 N=336 P=/2 SRC=HSE Q=7 → 0x07405408`，与 `-O2` 字面量池里 `.word 0x07405408` **同字**
3 · `flash_latency_ws_3v3()`	16→0、30→0、31→1、60→1、61→2、90→2、120→3、150→4、**151→5**、168→5 WS（每个边界各测两侧）
4 · `clock_tree_readback()`	喂 `SWS=PLL, PPRE1=0b101, PPRE2=0b100` → `{168000000, 168000000, 42000000, 84000000, 84000000}`；对照组 `PPRE1=/1` → PCLK1=TIM2CLK=168000000，×2 规则不触发
4b · 表长陷阱	旧 16 格表在 `0b100..0b111` 全读成 /1；新 8 格表读出 /2 /4 /8 /16（详见常见坑）
5 · `mco1_pre_for()`	168MHz → div=4 → `pre=0b110(6)` → PA8=42000000；按 `div=1` 写 `pre=0b011(3)` → div=1 → PA8=168000000，再按 /4 把读数乘回去得 672000000，**凭空多出 504 MHz**
6 · 错宏对照	`HSE_VALUE_HZ=25000000` 而 `BOARD_HSE_HZ=8000000`：写入 `0x07405419`（M=25）→ 程序以为 168000000、VCO 输入 320000、引脚 53760000、USB 15360000，**差 114240000 Hz，而返回值 = 0**

`-O2` 反汇编那一段（`build/main_o2.o`，text 944 字节）给出两件事。**提频顺序在指令流里单调向前推进**：`orr #16384`(VOS) → `bic #7 / orr #5 / orr #256`(LATENCY+预取) → `orr #65536`(HSEON) → `lsls #14 / bpl`(等 HSERDY) → `str [r2,#2052]`(PLLCFGR) → `orr #16777216`(PLLON) → `bic #3 / orr #2`(SW) → `and #12 / cmp #8`(读 SWS)——三处等待各自成环（等 HSERDY、等旧 PLL 停、等 SWS），但步骤本身没有一步倒回去。**常量折叠把 M/N/P/Q 压成四个立即数**：`movs r2,#8 / mov.w r3,#336 / movs r2,#2 / movs r3,#7` 直接存进 `g_pll_*`，一条除法都没有。折叠本身是好事，但宏填错时错值同样会被折叠成一个"看起来正常"的立即数，运行时再没有第二双眼睛。

八、上游对照：HAL 做同一件事

ST 官方仓库 `STMicroelectronics/stm32f4xx_hal_driver`（master @ `1f6451c`，2026-10-06 拉取）的 `HAL_RCC_OscConfig()` / `HAL_RCC_ClockConfig()` 做的是同一串动作，逐条对本工程：

本工程 `clock_init()`	HAL 对应（hal_rcc.c 行号 @ 1f6451c）	对照结论
HSEON 后轮询 HSERDY，超时回退 HSI	`HAL_RCC_OscConfig()`：轮询 `RCC_FLAG_HSERDY`，`HAL_GetTick` 超时返回 `HAL_TIMEOUT`（L231-259）	同思路：晶振可能不来，死等是 bug
先 VOS 再 LATENCY 再预取	`HAL_RCC_ClockConfig()` 只管 LATENCY——写入后**回读校验，不符直接 `HAL_ERROR`**（L613-626）；VOS 归 PWR 模块，F4 写完即生效、无就绪位可等	顺序一致；HAL 多一步回读
HPRE/PPRE 直接写目标值	切 HPRE 前**先把 PPRE1/PPRE2 打到 /16**（L635-649，注释原文：不经过非规格相位）	本工程 HPRE 恒 /1 无此窗口；做分频切换时 HAL 更保守
写完 SW 轮询 SWS 回执	同：切 SYSCLK 后轮询 `RCC_CFGR_SWS`（L681）	一致

SPL（StdPeriph）未随 ST 官方 GitHub 分发，本轮仍缺一手源码；其 `SetSysClockTo168M()` 与本节序列同构，位定义以 RM0090 为准。

附录：工程完整源码

不用跳 GitHub，整个 `main.c`、离线探针和取证脚本都在下面，滚动即达：

<<< ../../code/stm32/01-rcc-clock/main.c

<<< ../../code/stm32/01-rcc-clock/probe.c

<<< ../../code/stm32/01-rcc-clock/probe.sh

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **先电压、再等待、后切钟；先降总线、再提主频；写完 SW 要看 SWS；APB 分频不是一，定时器时钟自己乘二。**

延伸：RCC 时钟树动画见 S2 动画；Flash 等待周期与频率的关系在 B5 map 审计 有体积影响；GD32 RCU 对照见 G1。

实物实验

- MCO1 量频：`USE_HSE_PLL=1` 构建后量 PA8，应为 ≈42MHz；换回默认构建应为 ≈4MHz（16/4）。
- 对比实验：`make USE_HSE_PLL=1 HSE_VALUE_HZ=25000000`（如果你的板子是 25MHz 晶振）与 8MHz 版对照，GDB 读 `g_clock_tree[]` 应完全一致。
- 翻车实验（选做）：把 `flash_latency_ws_3v3()` 的返回值硬减 2，跑 168MHz，观察 HardFault，再用 S16 的方法读现场。
- 上板实测回填：用示波器实测霸天虎 HSE 晶振频率，回填到正文与工程宏（**待上板实测**）。

常见坑

- **MCO1PRE 编码想当然按"分频比 − 1"写**：它是 `0xx=不分频、100=/2、101=/3、110=/4、111=/5`（HAL `RCC_MCODIV_1..5` = 0/4/5/6/7）。按 `div−1` 写，/4 会落到"不分频"区，PA8 吐的是 168MHz，对账结论全部作废。
- **忘配 Flash 等待周期就切 168MHz**：取指超时，直接跑飞。顺序：先 LATENCY，再切 SYSCLK。
- **HSE 起振失败死等**：晶振虚焊/负载电容不对时 HSERDY 永远不来，代码要加超时回退 HSI。
- **改了 PLL 忘更新 SystemCoreClock**：后面的延时/波特率全错，本章工程用 `g_clock_tree[]` 回读，不依赖手工维护的全局变量。
- **HPRE 生效延迟**：写完 HPRE 立刻读分频后的时钟，读到的还是旧值；升频前先把总线分频降下来。
- **把 PLL 的 VCO 输入 1MHz 当外设时钟**：1MHz 是设计目标，不是可以喂给 USART 的时钟（见 S7 的 115200 误差算例）。
- **PPRE 查表照 HPRE 开 16 格**（本章改之前的真 bug，`probe.sh` 第 4b 节实测）：HPRE 是 4 位字段、PPRE 只有 3 位，16 格表用 3 位索引最大只到 7。`/2 /4 /8 /16` 四档全部落在第一格——**APB 分频被静默读成 /1**。症状不是崩溃而是离谱：APB1 被算成 168MHz，再套上"分频≠1 定时器 ×2"，屏幕上会出现 336MHz 这种这颗芯片根本给不出的频率。表长要按字段位宽开，`{1,1,1,1,2,4,8,16}` 八格才对。
- **`HSE_VALUE_HZ` 填错，程序一声不吭**（`probe.sh` 第 2 步实跑）：宏写 25MHz、板上焊 8MHz 时，固件写入 `PLLCFGR` 的是 `M=25 N=336` → `0x07405419`，`pll_params_from_hse()` 返回值 **0**（它只校验宏，不知道板上是什么）。`clock_tree_readback()` 也算出 168000000，而引脚上实际是 53760000 Hz，USB 那一路 15360000 Hz（要 48MHz）。回读只证明"你写的位和你的宏自洽"，不证明晶振为真。要让程序开口，得自己加一步 MCO1 实测对账。

短自测

1. 为什么切 168MHz 之前要先配 Flash 等待周期和电压档？

> **▼ 参考答案**
> Flash 取指在 3.3V 下 150~168MHz 要 5 个等待周期，不配够就取指超时、随机跑飞。电压档 VOS='0' 时 HCLK 上限是 144MHz，不配电压档连 168MHz 的资格都没有。顺序上必须先慢下来（电压档+等待周期）再提速。

2. HSE=8MHz 时给出两组合法的 M/N/P/Q，并说明哪一组更干净。

> **▼ 参考答案**
> VCO 输入 1MHz：M=8/N=336/P=2/Q=7（SYSCLK 168MHz、USB 48MHz）。VCO 输入 2MHz：M=4/N=168/P=2/Q=7，结果相同。RM0090 推荐 2MHz 输入以限制 PLL 抖动，第二组更干净。

3. HCLK=168MHz、APB1 分频 /4 时，TIM2 的时钟是多少？

> **▼ 参考答案**
> PCLK1 = 42MHz；APB 分频 ≠ 1 时定时器时钟 ×2 → TIM2 时钟 = 84MHz。只记 42MHz 会把所有定时器算慢一半。

4. 写完 `RCC_CFGR.SW` 立刻读 `SWS`，可能看到什么？该怎么办？

> **▼ 参考答案**
> 可能看到旧值，硬件切换需要时间。正确做法是轮询 SWS 直到它等于你写的 SW，并加超时；读不到回执就当切换失败处理。

5. 宏里写 `HSE_VALUE_HZ = 25000000`、板上实际焊 8MHz。编译不报错、运行也不报错，`clock_tree_readback()` 还回了个 168000000。`probe.sh` 第二步实测这一串的差值和函数返回值是多少？为什么回读救不了你？

> **▼ 参考答案**
> 实测：写入 `PLLCFGR` 的是 `M=25 N=336` → `0x07405419`，程序以为 SYSCLK = 168000000 Hz，引脚上实际 53760000 Hz，差 114240000 Hz。而 `pll_params_from_hse()` 返回值 **0**：它只校验宏本身（VCO 输入落在 1~2MHz 就放行），不知道板上焊的是什么。回读证明的只是"你写的位和你的宏自洽"，晶振真

假它管不着。要把这两个数分开，只能靠 MCO1 把内部时钟引到引脚上实测。

6. HPRE 是 4 位字段、PPRE 是 3 位字段。照 HPRE 给 PPRE 开一张 16 格的分频表，会出什么症状？

▼ 参考答案
3 位索引最大只到 7，表的后 8 格永远读不到，`/2 /4 /8 /16` 四档全部落在第一格 → APB 分频被静默读成 /1。症状不是崩溃而是离谱：APB1 被算成 168MHz，再套上"分频≠1 定时器 ×2"，屏幕上出现 336MHz 这种这颗芯片根本给不出的频率。所以表长必须按字段位宽开：`{1,1,1,1,2,4,8,16}` 八格。

🎯 S2 时钟树自测

1. **为什么切 168MHz 之前要先配 Flash 等待周期和电压档？**
   A. 降低功耗
   B. 不配够等待周期会取指超时跑飞，VOS='0' 时 HCLK 上限仅 144MHz
   C. 让 USB 先工作
   D. 防止 HSE 停振

2. **HSE=8MHz 时，哪组 M/N/P/Q 的 VCO 输入更干净（抖动更低）？**
   A. M=8, N=336, P=2, Q=7（VCO 输入 1MHz）
   B. M=4, N=168, P=2, Q=7（VCO 输入 2MHz）
   C. 两组完全相同
   D. M=2, N=84, P=2, Q=7

3. **HCLK=168MHz、APB1 分频 /4 时，TIM2 的时钟是多少？**
   A. 42MHz
   B. 84MHz
   C. 168MHz
   D. 21MHz

4. **写完 RCC_CFGR.SW 后立刻读 SWS，可能看到什么？**
   A. 一定是新值
   B. 可能是旧值——硬件切换需要时间
   C. 一定是 0
   D. SWS 只读，无法读取

5. **宏写 HSE_VALUE_HZ=25000000、板上实际焊 8MHz。probe.sh 第 2 步实测这一串的表现是什么？**
   A. 编译期就报错，压根链不出固件
   B. 一声不吭：pll_params_from_hse() 返回 0、回读算出 168000000，引脚上实际 53760000 Hz
   C. 运行时 HardFault
   D. HSE 就绪位起不来，程序卡在轮询里

6. **HPRE 是 4 位字段、PPRE 只有 3 位。照 HPRE 给 PPRE 开一张 16 格的分频表，症状是什么？**
   A. 数组越界，-Wall 直接拦下
   B. 立即数写错，APB1 被算成 21MHz
   C. /2 /4 /8 /16 四档全部落在第一格，APB 分频被静默读成 /1，屏幕上出现 336MHz
   D.「分频≠1 定时器 ×2」这条规则失效

先选完再翻：答案与逐项解析在附录 C 的本组；在线版可点击即时判分。

对照表：本章概念 → 仓库与上游落点

概念	落点
PLL 取值范围与公式	RM0090 Rev 18 §7.2.3 RCC_PLLCFGR 说明
Flash 等待周期按电压分列	RM0090 Rev 18 Table 10
VOS 天花板	同章 Note（F405/407 VOS=0 → ≤144MHz）
MCO1 = PA8 AF0	DS8626 Rev 9 Table 9
提频六步 + 回读确认	code/stm32/01-rcc-clock/main.c `clock_init()`
动画	rcc-clock-tree.svg
PLLCFGR 那个字（源码 ↔ 反汇编同字对账）	`probe.sh` 第 1 步断言 + 第 3 步 `objdump` 字面量池 `0x07405408`
提频顺序在指令流里单调	`probe.sh` 第 3 步 `-O2` 反汇编（`orr #16384 → bic #7/orr #5 → orr #65536 → str #2052 → cmp #8`）
常量折叠把 M/N/P/Q 压成立即数	同上，`movs r2,#8 / mov.w r3,#336 / movs r2,#2 / movs r3,#7`
PPRE 表长按字段位宽（本章修的 bug）	`probe.sh` 第 1 步 第 4b 节新旧表并排
错宏 `HSE_VALUE_HZ` 静默算错	`probe.sh` 第 2 步（`BOARD_HSE_HZ` 与宏不一致那一遍，差 114240000 Hz、返回值 0）
HAL 同名流程逐行对照	ST 官方仓库 stm32f4xx_hal_driver @ `1f6451c` `Src/stm32f4xx_hal_rcc.c`（见 §八）；SPL 未随官方 GitHub 分发，一手源码仍缺

你做到了

- 时钟树从"一坨框图"变成会算会配的配电网；
- MCO1 实测给了你第一个"把内部信号引出来看"的技能；
- "程序以为"与"硬件实际"之间的对账方法（回读 CFGR / 量 MCO1）从此归你。

那个数 g_clock_tree 报出来了：五行全是出厂内部钟，PLL 还没上岗。从猜一只钟到整棵树会算会配，你只花了一章。

✅ 下一站：S3 GPIO：七个寄存器逐位拆给你看，E01 那盏灯在这章彻底讲透。

通往单片机之路第 21 章

S3 GPIO：七个寄存器位级图解

难度 ★★★☆☆ · 约 30 分钟

本章罗盘

问题

配置一个引脚为什么要按序填四张表？改一个位为什么必须走 BSRR，而 ODR |= 是隐患？

交付

PF6 点亮的逐寄存器演算（MODER/OTYPER/OSPEEDR/PUPDR 每张表的位段值与先清后置写法）、一张推挽 vs 开漏对照表，加 PA9=USART1_TX 查 AF 表配 MODER+AFRH 的示范。

边界

本章讲通用机制与 F407 的寄存器版图，不背复用表，某引脚能当什么 AF 以 DS8626 Table 9 为准；OSPEEDR 与 EMI、驱动电流等电气量只给原则不给实测；I2C 开漏的总线纪律完整展开在 S11。

依据锚点 `a1` · `a2` · `f5` · `mcu-code` · `mcu-anim`

> 🎯 GPIO 是每个外设的"出发大厅"：不管多高级的功能（串口/SPI/PWM），最后都要从一个引脚进出。而这个大厅的全部管理规章，就写在七个 32 位寄存器里。本章把它们逐位拆开，以后每章的"引脚设置"环节你都能秒过。

本章精髓

1. 四个配置寄存器各管一维：**MODER**（输入/输出/复用/模拟，每脚 2 位）、**OTYPER**（推挽/开漏）、**OSPEEDR**（边沿速度）、**PUPDR**（上拉/下拉/浮空）。配置一个引脚 = 按序填这四张表。
2. 三个数据寄存器分工：**IDR** 读真实电平、**ODR** 读写输出（读-改-写有隐患）、**BSRR** 原子置位/复位：BSRR 是"改一位不扰邻居"的正解。
3. 复用（AF）是引脚的第二人生：AFRL/AFRH 每脚 4 位选 AF0~AF15：引脚还是那个引脚，数据源从 ODR 换成外设；datasheet 的复用表是唯一的真值。

- **能理解**：为什么改位要用 BSRR 而不是 ODR，第二节的翻车现场会回答你。
- **能用**：把 00-blink 的 main.c 每一行讲给同学听，讲不顺的地方回来重读。

先修

- S0 环境、S2 时钟；C4 结构体。

先跑起来（10 分钟 quick win）

打开 E01 点灯实验，把 main.c 的配置逐行注释成"这是在填哪张表"，你会发现 E01 的三步就是本章的全部。

烧录之前先押一题：板上的 RGB 灯是共阳接法，点亮红灯靠写高电平还是写低电平？押完把点灯实验烧进去，灯亮就是判分，错了就换一副理解再来。

动画：先开时钟，再盖四个章

五个阶段循环播放：① 合闸供电（RCC）②③④⑤ 四张表逐格盖章 → BSRR 一声令下，红灯亮。

GPIO 配置流程动画

板卡事实

- 霸天虎：RGB 红=PF6 / 绿=PF7 / 蓝=PF8，**共阳**接 3.3V，低电平点亮；GPIOF 时钟 = RCC_AHB1ENR bit5（出处：野火《寄存器点灯》）。
- 寄存器偏移（RM0090 §8.4）：MODER 0x00 / OTYPER 0x04 / OSPEEDR 0x08 / PUPDR 0x0C / IDR 0x10 / ODR 0x14 / BSRR 0x18 / LCKR 0x1C / AFRL 0x20 / AFRH 0x24。
- 地址已经过 ST 官方 CMSIS 头文件逐项核对：GPIOF = 0x40021400，RCC = 0x40023800，AHB1ENR 偏移 0x30。

配套视频

> 🎬 **野火 F103 入门篇（150 集，含《使用寄存器点亮 LED》《GPIO 功能框图讲解》；板子是 F103 霸道，GPIO 原理与 F407 相同，对照本章看）** @ 哔哩哔哩
> 离线书不内嵌在线视频；去 哔哩哔哩 观看 ↗

小节结构

小节	内容	四件套
一、配置四维	四寄存器位级图解 + PF6 实拍演算	配置
二、数据三剑客	IDR/ODR/BSRR；读-改-写翻车现场	代码分析
三、推挽 vs 开漏	晶体管级对比 + I2C 的必然性	引脚
四、复用机制	AFRL/AFRH 与 datasheet 复用表查法	引脚
五、SPL 对照	GPIO_Init 的形参如何落到四张表	库解析
六、输入侧	施密特触发/模拟模式/按键消抖	代码分析

一、配置四维：四张表，每脚两格

OSPEEDR 四档：越快上升沿越陡，EMI 越大

把 GPIO 外设想成一间装配车间，墙上有**四个抽屉柜**：MODER、OTYPER、OSPEEDR、PUPDR。16 个引脚在每个柜子里都占格子：除了 OTYPER 每脚 1 格，其余每脚 **2 格**。配一个引脚，就是按顺序拉开四个柜子填格子：

寄存器	每脚位宽	取值含义（以输出为例）	RM0090
MODER	2 位	00 输入 / **01 通用输出** / 10 复用 / 11 模拟	§8.4.1
OTYPER	1 位	**0 推挽** / 1 开漏	§8.4.2
OSPEEDR	2 位	**00=2MHz** / 01=25MHz / 10=50MHz / 11=100MHz	§8.4.3
PUPDR	2 位	**00 浮空** / 01 上拉 / 10 下拉	§8.4.4

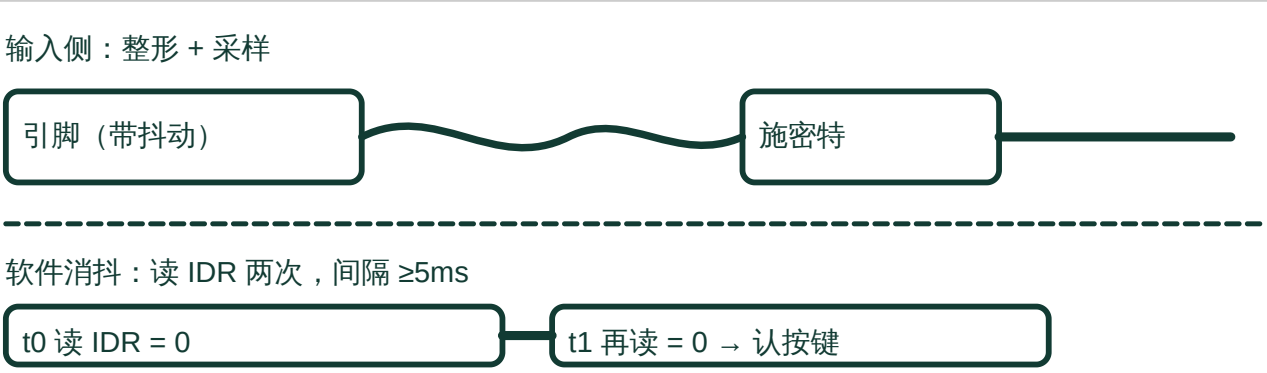
拿 PF6 实拍演算一遍。PF6 在 MODER 里占 bits[13:12]（6×2），目标是 01（输出）：

```
GPIOF_MODER   &= ~(3UL << (LED_R_PIN * 2));  /* 先清零：~0x3000，两位擦除    */
GPIOF_MODER   |=  (1UL << (LED_R_PIN * 2));  /* 再置位：0x1000，[13:12]=01  */
GPIOF_OTYPER  &= ~(1UL << LED_R_PIN);        /* 0  = 推挽                    */
GPIOF_OSPEEDR &= ~(3UL << (LED_R_PIN * 2));  /* 00 = 低速（复位默认就是）    */
GPIOF_PUPDR   &= ~(3UL << (LED_R_PIN * 2));  /* 00 = 无上下拉                */
```

> **【注】为什么坚持"先清零再置位"？** 不能直接 `|= 01`：万一格子里原来是 11（模拟），或一下变成 11。两位的格子要先擦干净再写。复位后大多数引脚是 00（输入），但 PA13/PA14 这些调试脚默认是复用。所以"先清后置"是永不翻车的写法。

还有一句顺序纪律：**时钟永远先于一切**。`RCC_AHB1ENR |= (1UL << 5);` 不合闸，外设寄存器就是没通电的橱窗：写进去不响，读回全是 0。这是新手第一大坑，没有之一。

二、数据三剑客：IDR 看、ODR 写、BSRR 办



输入侧：施密特整形 + IDR 两次采样消抖

- **IDR**（§8.4.5）：只读，引脚上此刻的真实电平。输入的耳朵。

- **ODR**（§8.4.6）：可读写，输出寄存器。看着方便，**但有暗伤**。
- **BSRR**（§8.4.7）：只写，低 16 位置位（BS）、高 16 位复位（BR），写 1 生效、写 0 无影响。

ODR 的暗伤在哪？`GPIOF_ODR |= (1UL << 6);` 这行 C，编译出来是**三条指令**：

```
ldr  r0, [r1]        ; ① 把账本读出来
orr  r0, r0, #0x40   ; ② 改自己那位
str  r0, [r1]        ; ③ 整本写回去
```

这就是"读-改-写"：三个人共改一本账。想象 ① 和 ③ 之间插进来一个中断，中断里也改了 ODR 的另一位；中断返回后，main 把 ① 时刻的旧账整本写回——**中断的改动被当场冲掉**。症状是"偶尔丢一次电平"，一周出现一回，查到你怀疑人生。

BSRR 是硬件给的解法：**一条 `str` 完成，没有读、没有改**，中断无从插足；写 0 无影响，所以连掩码都省了。00-blink 里就是这么发令的：

```
GPIOF_BSRR = (1UL << (LED_R_PIN + 16));  /* BR6 = 0x00400000：拉低 → 红灯亮 */
GPIOF_BSRR = (1UL << LED_R_PIN);         /* BS6 = 0x00000040：拉高 → 红灯灭 */
```

> 【注】**共阳反转别和自己过不去。** 霸天虎 RGB 共阳接 3.3V，引脚**低电平灯亮**。所以是"高 16 位亮、低 16 位灭"。写代码时心里默念：BR 亮、BS 灭——反人类一次，记住一辈子。
>
> 【注】**BSRR 两个冷门事实**：读它永远返回 0（只写寄存器）；对它写 0 完全不动作，这就是它能免掩码的原因。

三、推挽 vs 开漏：能推能拉，还是只会拉

	推挽（PP）	**开漏（OD）**
输出级	上 PMOS + 下 NMOS，两个开关对推	只有下 NMOS，**只会拉低**
高电平	主动推向 VDD，有力	"放手"——靠外部上拉电阻抬升
典型场景	LED、SPI、USART，绝大多数	I2C、5V 电平转换、"线与"总线

I2C 为什么**必须**开漏+上拉？这是一堂总线外交课：SDA 上挂着主机和好几个从机，**谁都有权拉低**（发 0、应答、 clock stretching）。要是用推挽，一个脚卖力推高、另一个卖力拉低，两个输出级对轰，电流直通，烧脚没商量。开漏的制度是"谁都只能拉，想高就放手"，上拉电阻把线温柔地抬回高电平。多设备共享一根线而不斗殴，这就是"线与"。

> 【注】 F4 大部分引脚标 FT（5V 容忍），开漏加上拉到 5V 就能和 5V 器件对话，但模拟功能脚（标 TTa）不行，接 5V 前先查 datasheet 引脚表。

四、复用：引脚的第二人生

引脚默认听 ODR/IDR 的（通用 IO）；写成 MODER=10 后，数据线改接到**片上外设**：USART、TIM、SPI……具体接哪个，由 AFRL/AFRH 决定：每脚 4 位，选 AF0~AF15。**哪个引脚能当什么、是 AF 几，只有 datasheet 的复用功能表说了算**（F407 在 Table 9），别背，查。

以 USART1_TX = PA9 为例，查表得 AF7，配置两行到位：

```
GPIOA->MODER  &= ~(3UL << 18);
GPIOA->MODER  |=  (2UL << 18);   /* PA9：MODER[19:18] = 10，复用模式 */
GPIOA->AFR[1] &= ~(0xFUL << 4);  /* AFRH 管 pin8~15，PA9 是第 1 个 nibble */
GPIOA->AFR[1] |=  (7UL << 4);    /* AFRH[7:4] = 0x7 = AF7 = USART1_TX */
```

> 【注】**复用要成对**：只写 AFRL/H 不改 MODER=10，等于章盖了门没开，外设根本接不进引脚。MODER=10 + AFR 选号，缺一不可。

五、SPL 对照：GPIO_Init 没有魔法

标准库（SPL）的 `GPIO_Init` 看着神秘，其实就是把上面这套流程用 for 循环走了一遍。它的形参和我们填的表一一对应：

GPIO_InitTypeDef 字段	落到哪张表
`GPIO_Mode`（IN/OUT/AF/AN）	MODER 两位
`GPIO_OType`（PP/OD）	OTYPER 一位
`GPIO_Speed`（2/25/50/100MHz）	OSPEEDR 两位
`GPIO_PuPd`（NOPULL/UP/DOWN）	PUPDR 两位
`GPIO_Pin`（位掩码）	决定改哪一格
`GPIO_PinAFConfig()`（复用时另调）	AFRL/AFRH 的 nibble

剥开 `stm32f4xx_gpio.c` 的 `GPIO_Init`，骨架就这几行（意译，保留原始逻辑）：

```
for (pinpos = 0; pinpos < 16; pinpos++) {
    if ((GPIO_InitStruct->GPIO_Pin & (1UL << pinpos)) == 0) continue;
    /* MODER：先清零 (3 << pinpos*2)，再或上 mode << pinpos*2 */
    /* 若是 OUT/AF：OTYPER / OSPEEDR / PUPDR 同样先清后或   */
    /* 若是 AF：AFR[pinpos >> 3] 的对应 nibble 写入 AF 号     */
}
```

一个字都没超出本章第一节。读完这一段，库函数在你眼里就是透明的。S15 章会把 SPL 整个掀开细讲。

六、输入侧：听外界说话

- **数字输入**：引脚经过**施密特触发器**进 IDR：滞回比较器，边沿再抖也不哆嗦。
- **模拟模式**（MODER=11）：把施密特触发器、上下拉**全部断开**，引脚直通 ADC，漏电流最小，采样最真。
- **按键消抖**两条路：硬件（RC 滤波，一分钱解决）或软件（定时采样，连续 N 次一致才算数）。实验章两条都会走。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **先时钟、再模式（M-O-S-P 四张表）、后数据；改位用 BSRR，读电平问 IDR，复用查 AF 表。共阳灯：BR 亮、BS 灭。**

延伸：GPIO 配置流程动画见 S3 动画；指针操作 GPIO 寄存器是 C2 指针 的实战应用；GD32 GPIO 对照见 G2。

实物实验

- 基础：E01 完成三色切换；
- 进阶：PF6 改输入+上拉，杜邦线对地短接，GDB 读 IDR bit6 从 1 变 0，亲手摸到"输入数据寄存器"。

常见坑

- **忘开 GPIOx 时钟**：寄存器全写不进去（读回全是 0），新手第一大坑，没有之一。
- **用 ODR 翻转单个引脚**：中断插在"读-改-写"中间就丢状态，改位请用 BSRR。
- **OSPEED 盲目拉满**：边沿越快 EMI 越猛；LED/按键用 2MHz 足够，高速信号才需要 50/100MHz。
- **复用功能开了 AF 没换 MODER**：MODER 还在"通用输出"，外设接不进引脚，复用要成对（MODER=10 + AFR 选号）。
- **对 BSRR 用读-改-写**：它读回永远是 0，`|=` 会把你的位全清掉，BSRR 只能纯赋值。

短自测

1. 不开 GPIOF 时钟就写 MODER，然后读回 MODER，看到什么？
2. 为什么 BSRR 改一位不怕被中断打断，ODR 怕？
3. I2C 的 SDA 为什么必须开漏 + 上拉？
4. PA9 用作 USART1_TX，MODER 和 AFRH 各写什么值？

▼ 参考答案（先自己想完再展开）
1. 全是 0。外设没电，写入无效，这就是"忘开时钟"坑的指纹。 2. ODR 是 ldr/orr/str 三条指令的读-改-写，中间可被插队，旧值写回冲掉别人的改动；BSRR 一条 str 原子完成，且写 0 无影响。 3. 多设备共享一根线且谁都可拉低；推挽会一推一拉对轰短路。开漏"只会拉、想高放手"，上拉电阻抬升，这就是"线与"。 4. MODER[19:18] = 10（复用模式）；AFRH[7:4] = 0x7（AF7 = USART1_TX）。

🎯 S3 GPIO 自测

1. **没开 GPIOF 时钟就往 MODER 写配置，随后读回 MODER，会看到什么？**
   A. 写入的值——寄存器照样接收，只是外设不工作
   B. 全是 0——外设没通电，写入无效、读回也无效
   C. 保持复位默认值，读回是旧配置
   D. 总线错误，触发 HardFault

2. **PF6 在 GPIOF_MODER 里占哪两位？要配成通用输出该写什么？**
   A. bits[6:5]，写 01
   B. bits[13:12]，写 10
   C. bits[13:12]，写 01
   D. bits[12:11]，写 11

3. **为什么改一个引脚电平要用 BSRR 而不是 ODR |= ？**
   A. BSRR 写入速度比 ODR 快一个数量级
   B. ODR 只能整字写入，不能按位操作
   C. ODR |= 是读-改-写三条指令，中断插在中间会把别人的改动冲掉；BSRR 一条 str 原子完成且写 0 无影响
   D. BSRR 有锁定位，能防止误写

4. **I2C 的 SDA 为什么必须开漏 + 上拉？**
   A. 开漏翻转速度更快，能跑满 400kHz
   B. 多设备共享一根线且谁都有权拉低；推挽会一推一拉对轰短路，开漏"只会拉、想高放手"实现线与
   C. 上拉电阻可以同时充当限流保护 LED
   D. 推挽模式在 I2C 频率下无法输出高电平

5. **PA9 用作 USART1_TX，除了查 datasheet 复用表得知 AF7，寄存器上还要写什么？**
   A. MODER[19:18] = 10（复用），AFRH[7:4] = 0x7
   B. MODER[19:18] = 01（输出），AFRL[7:4] = 0x7
   C. 只写 AFRH[7:4] = 0x7，MODER 保持默认
   D. MODER[19:18] = 11（模拟），AFRH[7:4] = 0x7

先选完再翻：答案与逐项解析在附录 C 的本组；在线版可点击即时判分。

对照表：本章概念 → 仓库落点

本章说的	仓库里哪一行
四张表 + BSRR 的地址	main.c:29-34 的宏定义
开时钟第一步	main.c:49 `RCC_AHB1ENR |= RCC_AHB1ENR_GPIOFEN;`
先清零再置位	main.c:52-56
BSRR 原子拉低/拉高	main.c:60-62
共阳低电平点亮	main.c:12-13 注释 + E01 接线图
IDR 进阶实验	本章"实物实验"进阶项（E01 改造）

你做到了

- GPIO 七个寄存器不再神秘，配置流程变成肌肉记忆；
- E01 的每一行代码你都能讲出"为什么"；
- 再看到 `GPIO_Init` 之类库函数，你会自动在脑子里展开成四张表。

开场那道题，答案是写低：共阳接法，写零是开灯不是关灯。方向拧对了，四张表怎么填都好说。

✅ 下一站：S4 NVIC 与 EXTI：换按键来"打断"你，中断现场有逐帧动画。

通往单片机之路第 22 章

S4 NVIC 与 EXTI：打断的艺术

难度 ★★★☆☆ · 约 40 分钟

本章罗盘

问题

门铃响到进 ISR 中间隔着哪三层配置？IRQn=37 怎么换算成 ISER 下标、位号和 IP 字节地址？

交付

一张 IRQn→向量槽/ISER 下标/位号/IP 地址换算表（52 条断言对撞过 CMSIS 头文件）加六步外部中断落地清单，每步注明漏掉后的典型死法。

边界

全部数字在 F407 上取证（4 位优先级、82 个外设中断、越界优先级在本字节内静默截断），不能外推到优先级位数不同的芯片；RTOS 下的中断管理与 BASEPRI 玩法归 F 篇；按键抢占嵌套实验是设计的实物验证项。

依据锚点 `a1` · `a3` · `f3` · `mcu-code` · `mcu-anim`

> 🎯 轮询是"每隔一会儿跑去门口看有没有快递"，中断是"快递按门铃"。门铃响了之后 CPU 做了什么：怎么去的、怎么回的、两个快递同时按铃谁先，这一章全部讲透。

本章精髓

1. 三层链路：**EXTI**（把引脚边沿变成中断请求）→ **NVIC**（按优先级仲裁、呈递给内核）→ **内核**（硬件压栈、取向量表跳转），每一层都要单独配置，漏一层铃就不响。
2. 地址全是公式：**槽号 = IRQn + 16**，**ISER 下标 = IRQn / 32**、**位号 = IRQn % 32**，**IP 字节 = 逻辑优先级 << 4**。本章每个数字都能在 CMSIS 头文件里找到行号，probe 用 52 条断言把它们全部对撞过一遍。
3. 优先级两组数：抢占优先级（能不能打断别人）与子优先级（同时挂起谁先上）：AIRCR.PRIGROUP 把 4 位优先级切成两份，写 AIRCR 必须带钥匙 0x5FA，全家要用同一把尺子。

- **能理解**：IRQn 6（EXTI0）是怎么变成向量槽 22、ISER[0] bit 6、IP 字节地址 0xE000E406 的，三条公式各推一遍。
- **能用**：照第六节清单给任意一个引脚配通外部中断，并能用 NVIC_SetPriorityGrouping 说清自己的工程用的是哪把"尺子"。

先修

- S3 GPIO；C5 函数指针（向量表就是函数指针数组）、C6 栈帧（硬件压栈的另一半）。

先跑起来（10 分钟 quick win）

```
cd code/stm32/04-nvic-exti
sh probe.sh   # 宿主 gcc 跑 52 条断言；PATH 里有交叉工具链时附送向量表/objdump 取证
```

输出里找这三行，它们是本章的骨架：

```
EXTI0_IRQn = 6    <- stm32f407xx.h:83
EXTI0          6     22 0x0058        0     6 0xE000E100 0xE000E406
== 断言通过 52/52 ==
```

读完本章再回头看这张表，每个数字你都能亲手推出来。

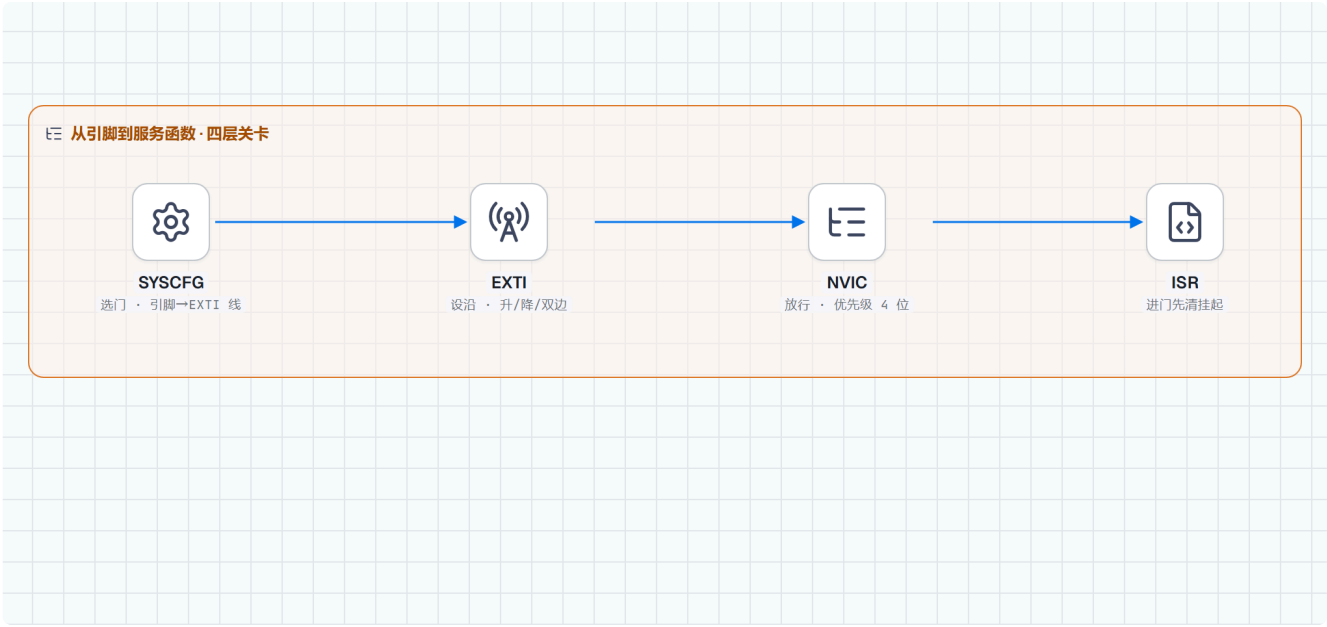
敲回车之前，先猜一把：按键走的那条中断线，搬进 NVIC 大楼后的门牌号是小于五、五到十之间、还是大于十？跑完看头文件行号那一栏，猜错了不丢分，丢了直觉才丢分。

小节结构

小节	内容	四件套
一、三层链路	EXTI/SYSCFG/NVIC/SCB 的基址地图，全部来自头文件行号	配置
二、NVIC 寄存器块	ISER/ICER/ISPR/ICPR/IABR/IPR：IRQn/32 与 IRQn%32 寻址公式	配置
三、优先级	4 位字段、<<4 编码、PRIGROUP 分组表、0x5FA 钥匙	配置
四、向量表	槽号 = IRQn+16；objdump 实证 weak 默认被强符号顶替	代码分析
五、EXTI 与 SYSCFG	六个寄存器偏移 + EXTICR 选门公式	配置
六、落地清单	六步配通一条外部中断，一步都不能少	配置
七、ISR 现场	动画逐帧：硬件压栈 8 字、取向量、EXC_RETURN 弹栈	代码分析

一、三层链路：先把地址地图画出来

四层关卡先收进一张图：选门、设沿、放行、清挂起，顺序就画在实线上。



按键到 ISR 的四层关卡：SYSCFG 选门、EXTI 设沿、NVIC 放行、ISR 清挂起

中断不是"一个开关"，而是**三个硬件模块接力**。probe.sh 第 0 段直接从 CMSIS 头文件把这条链挖了出来（每行自带行号引证）：

```
PERIPH_BASE = 0x40000000    <- stm32f407xx.h:910
APB2_OFFSET = 0x00010000    <- stm32f407xx.h:928
SYSCFG_OFFSET = 0x3800      <- stm32f407xx.h:974
EXTI_OFFSET   = 0x3C00      <- stm32f407xx.h:975
SCS_BASE      = 0xE000E000  <- core_cm4.h:1550
NVIC_OFFSET   = 0x0100      <- core_cm4.h:1556
SCB_OFFSET    = 0x0D00      <- core_cm4.h:1557
```

于是两条例链一目了然：

```
外设链：PERIPH 0x40000000 -> APB2 +0x10000 -> SYSCFG +0x3800 / EXTI +0x3C00
内核链：SCS 0xE000E000 -> SysTick +0x0010 / NVIC +0x0100 / SCB +0x0D00
```

- **EXTI 在 0x40013C00**（APB2 上），负责"检测边沿、举起请求"；
- **SYSCFG 在 0x40013800**（APB2 上），负责"EXTI 线接哪个端口"的多路选择；
- **NVIC 在 0xE000E100、SCB 在 0xE000ED00**（内核私有外设区 SCS 内），负责"仲裁、呈递、分组"。

为什么 NVIC/SCB 的地址在 `core_cm4.h` 而 EXTI/SYSCFG 在 `stm32f407xx.h`？因为前者是 **ARM 定义的 Cortex-M4 内核标配**（所有 M4 芯片都一样），后者是 **ST 加的外设**（各家不同）。这个归属差异记牢，查手册时就知道该翻 PM0214（内核）还是 RM0090（外设）。

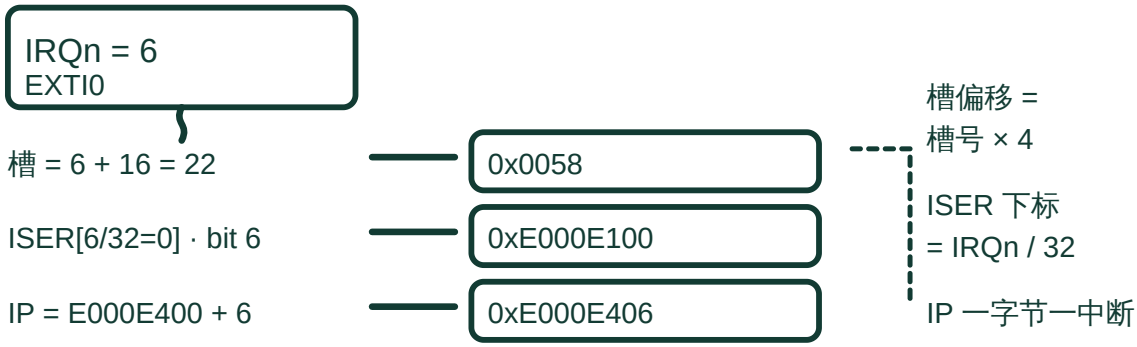
二、NVIC 寄存器块：IRQn/32 与 IRQn%32 的寻址公式

NVIC 有六组寄存器，每组都是"一位管一个中断"的位图：

寄存器	作用
ISER	置位使能（写 1 开中断）
ICER	清除使能（写 1 关中断）
ISPR / ICPR	手工挂起 / 清挂起
IABR	只读，看哪个中断正在活跃
IPR	优先级，**一个字节管一个中断**（注意：不是一位）

中断数量超过 32，所以位图是数组，寻址公式就两条：

- **数组下标 = IRQn / 32**，**位号 = IRQn % 32**（对 ISER/ICER/ISPR/ICPR/IABR）；
- **IP 字节地址 = 0xE000E400 + IRQn**（对 IPR）。



NVIC 地址公式：IRQn 6 推出向量槽 22（偏移 0x0058）、ISER 第 0 组 bit 6 与 IP 字节 0xE000E406

probe 把七个代表性中断整张表打了出来（宿主编译期 `_Static_assert` 已逐格核对，见 `probe.c:38-79`）：

```
中断       IRQn    槽 槽偏移 ISER下标   位   ISER地址 IP字节地址
SysTick      -1     15 0x003c        -     -        (SHP) 0xE000ED23
EXTI0         6     22 0x0058        0     6 0xE000E100 0xE000E406
EXTI1         7     23 0x005c        0     7 0xE000E100 0xE000E407
EXTI9_5      23     39 0x009c        0    23 0xE000E100 0xE000E417
TIM2         28     44 0x00b0        0    28 0xE000E100 0xE000E41C
USART1       37     53 0x00d4        1     5 0xE000E104 0xE000E425
EXTI15_10    40     56 0x00e0        1     8 0xE000E104 0xE000E428
```

拿 USART1 验算：IRQn = 37（`stm32f407xx.h:114`），37 / 32 = **1**、37 % 32 = **5**，所以使能位在 `ISER[1]` 的 bit 5，地址 0xE000E100 + 1×4 = **0xE000E104**；IP 字节 0xE000E400 + 37 = **0xE000E425**。两列全中。

注意第一行 SysTick：IRQn 是**负数**（-1，`stm32f407xx.h:75`），它是内核异常，优先级不在 NVIC 的 IPR 里，而在 SCB 的 SHP 寄存器（0xE000ED23）。**NVIC 的位图只服务 IRQn ≥ 0 的外设中断**。

三、优先级：4 位字段、左移 4 位、分组表与 0x5FA 钥匙

3.1 F407 只实现了高 4 位

```
NVIC_PRIO_BITS = 4    <- stm32f407xx.h:49
```

Cortex-M4 的 IP 寄存器是**每中断一个字节**（0~255），但 F407 只实现了高 4 位（`__NVIC_PRIO_BITS = 4`）。所以 `NVIC_SetPriority()` 写入 IP 字节的值 = **逻辑优先级 << 4**，probe 实测编码：

```
逻辑优先级 存入 0xE000E4xx 的字节
0          0x00
1          0x10
5          0x50
10         0xA0
15         0xF0
```

低 4 位永远是 0，读回来也永远是 0。推论：F407 上**逻辑优先级只有 0~15 这 16 档**。

那写 16 会怎样？把 `core_cm4.h:1818` 那行原话搬过来跑一遍就知道，它既不会报错、也不会进下一个中断的字节：`(priority << 4) & 0xFF` 是**在本字节里截断**，probe 实测（`probe.c` 越界编码表）：

```
逻辑优先级 存入字节 读回来当作
16         0x00     0
17         0x10     1
20         0x40     4
255        0xF0     15
256        0x00     0
```

真正危险的地方在这儿：**越界的数会被静默改写成另一个合法优先级**。你想给 16（本该最低），芯片收到的是 0（最高）；想给 17，收到的是 1。等号两边都成立，程序照跑，只有嵌套关系悄悄反了，这是真 bug 的高发区，而且它不发警报。

3.2 PRIGROUP：把 4 位切成"抢占 + 子"两份

SCB->AIRCR 的 PRIGROUP 字段（位 [10:8]，`core_cm4.h:531`）决定这 4 位里**几位管抢占、几位管子优先级**——优先级的"粗细"由这里切分。probe 打印的分组表：

```
PRIGROUP  抢占位数 子优先级位 写法（VECTKEY|组值<<8）
0         4          0          0x05FA0000
1         4          0          0x05FA0100
2         4          0          0x05FA0200
3         4          0          0x05FA0300
4         3          1          0x05FA0400
5         2          2          0x05FA0500
6         1          3          0x05FA0600
7         0          4          0x05FA0700
```

- **抢占优先级**：能不能打断正在执行的 ISR（数值越小越横）；
- **子优先级**：两个中断**同时挂起**时谁先上。注意它**没有打断能力**，同抢占级的中断永远互不嵌套。

组 0~3 在 4 位实现下效果相同（4 抢占 + 0 子），表里清清楚楚。FreeRTOS 要求全抢占位（组 4 的反面：它要 4 位全给抢占，即组 0~3 这种 4+0 切法），RT-Thread 同理；而 SPL 老代码常见组 2。**移植 RTOS 时"优先级数值没变、行为全变"的灵异事件，九成是分组尺子换了**。

把"谁能打断谁"演成五幕的，是下面这张动画：EXTI0（抢占 2）在跑 ISR 时，同抢占级的 EXTI9 只能挂起排队，子优先级没有打断能力；而抢占号更小的 EXTI1 一到，硬件立刻压栈换人，ISR1 退出后回到 ISR0 断点，ISR0 再退出才轮到 EXTI9 补位。"实物实验"那一节的按键嵌套实验，验的就是这条时间轴。

NVIC 抢占判定：谁能打断谁

3.3 写 AIRCR 必须带钥匙 0x5FA

AIRCR 是个"带锁"的寄存器：写入时高 16 位必须是 VECTKEY = **0x5FA**，否则写操作被无视。出处在 `core_cm4.h:1661` 的魔数 `0x5FAUL << SCB_AIRCR_VECTKEY_Pos`，所以上表第三列每个写法都是 `0x05FA0g00` 的形状。`NVIC_SetPriorityGrouping()` 内部就是"读 AIRCR、改 [10:8]、带钥匙写回"。

3.4 禁转辟谣：HardFault 的优先级不能设

打开 `stm32f407xx.h:68-69` 看 IRQn_Type 枚举：

```
  NonMaskableInt_IRQn         = -14,    /*!< 2 Non Maskable Interrupt   */
  MemoryManagement_IRQn       = -12,    /*!< 4 Cortex-M4 Memory Management Interrupt */
```

-13 被故意跳过了：HardFault_IRQn 根本不在枚举里。Reset、NMI、HardFault 的优先级是硬件固定的（-3/-2/-1，越来越高），不在 SHP 里、不可编程。能设优先级的是 IRQn -12（MemManage）起到 -1（SysTick）这段内核异常（走 SHP），外加全部 IRQn ≥ 0 的外设中断（走 IPR）。再看到"把 HardFault 优先级调低"的说法，直接判错。

四、向量表：槽号 = IRQn + 16，weak 被顶替的实证

C5 讲过：向量表就是 **Flash 起始处的一个 `const` 函数指针数组**。前 16 项是内核异常（含初始 MSP 与 Reset_Handler），外设中断从槽 16 开始排，所以：

- **槽号 = IRQn + 16**；**字节偏移 = 异常号 × 4**（异常号 = 槽号）。

例：EXTI0_IRQn = 6 → 槽 **22** → 偏移 22 × 4 = **0x58**，正是第二节表格里那一列。probe.sh 第 2 段直接数 `startup_stm32f407xx.s` 的 `.word` 行做了交叉核对：

```
  向量表总项数 = 98（16 内核 + 82 外设，应为 98）
  IRQn -1 -> 向量槽 15 = SysTick_Handler  ✓
  IRQn 6 -> 向量槽 22 = EXTI0_IRQHandler  ✓
  IRQn 23 -> 向量槽 39 = EXTI9_5_IRQHandler  ✓
  IRQn 40 -> 向量槽 56 = EXTI15_10_IRQHandler  ✓
```

第 3 段用交叉工具链真链了一次，objdump 给出三重实证：

```
Idx Name          Size      VMA       LMA       File off  Algn
  0 .isr_vector   00000188  08000000  08000000  00010000  2**0
08000188 g     F .text  0000001c EXTI0_IRQHandler
  槽22（EXTI0）内容 = 08000189
```

- `.isr_vector` 落在 **0x08000000**（Flash 起始），大小 0x188 = 392 字节 = 98 × 4，与项数吻合；
- 你在 C 文件里定义了 `EXTI0_IRQHandler`，它就是**强符号**，把启动文件里的 weak 默认（Default_Handler 死循环）顶替掉。函数名拼错，等于中断永远进死循环；
- 槽里装的是 `0x08000189` = 函数地址 `| 1`，**最低位是 Thumb 状态位**，向量跳转播 `bx` / `ldr pc` 时靠它保持 Thumb 模式。

五、EXTI 与 SYSCFG：选门、设沿、清挂起

5.1 EXTI 寄存器块只有六个

probe 连偏移带头文件行号全部挖出（`stm32f407xx.h:444-449`），基址 0x40013C00：

```
EXTI_IMR偏移   = 0x00    <- stm32f407xx.h:444    中断屏蔽（1=放行）
EXTI_EMR偏移   = 0x04    <- stm32f407xx.h:445    事件屏蔽
EXTI_RTSR偏移  = 0x08    <- stm32f407xx.h:446    上升沿触发使能
EXTI_FTSR偏移  = 0x0C    <- stm32f407xx.h:447    下降沿触发使能
EXTI_SWIER偏移 = 0x10    <- stm32f407xx.h:448    软件触发
EXTI_PR偏移    = 0x14    <- stm32f407xx.h:449    挂起标志，写 1 清除
```

展开成绝对地址：`IMR=0x40013C00 EMR=0x40013C04 RTSR=0x40013C08 FTSR=0x40013C0C SWIER=0x40013C10 PR=0x40013C14`。每个寄存器的 bit N 对应 EXTI 线 N（0~15 接 GPIO，16 以上接 PVD/RTC 等内部源）。

5.2 SYSCFG_EXTICR：EXTI 线接哪个端口，这里说了算

EXTI 线 0~15 是"逻辑线"，同一时刻每条线只能接**一个端口**的对应号引脚（PA6/PB6/PC6… 抢 EXTI6 这一条线）。选择开关在 SYSCFG：

- EXTICR 起始偏移 **0x08**（`stm32f407xx.h:547`），绝对地址 **0x40013808**；
- 共 EXTICR1~4 四个寄存器，**每个管 4 条线，每条线 4 bit**；
- 选 F 口写的编码是 0x5（probe 引证 `EXTICR_EXTI0_PF = 0x0005`，`stm32f407xx.h:11662`）。

probe 实测的多路选择表（选 F 口）：

```
EXTI线  寄存器 字段位    选 F 口要写的值
0        EXTICR1  [ 3: 0]   0x00000005
1        EXTICR1  [ 7: 4]   0x00000050
6        EXTICR2  [11: 8]   0x00000500
10       EXTICR3  [11: 8]   0x00000500
```

公式自己推一遍：寄存器编号 = 线号 / 4，字段起始位 = (线号 % 4) × 4。EXTI6：6/4 → EXTICR2，(6%4)×4 = 8 → [11:8]，0x5 << 8 = 0x500；EXTI10：10/4 → EXTICR3，(10%4)×4 = 8 → [11:8]。全中。

别忘了开 SYSCFG 时钟：RCC_APB2ENR 的 bit 14（probe 引证 `stm32f407xx.h:10076`）。不开时钟写 EXTICR 等于往空气里写。

5.3 PR 挂起位：写 1 清除，ISR 进门第一件事

边沿到来 → PR 置 1 → NVIC 呈递 → 进 ISR。**PR 不会自己清**，你不清它，出 ISR 的瞬间请求又成立，"无限中断、主循环停摆"就这么来的。清法是**往对应位写 1**（写 0 无效，这是 W1C 语义）。

这不是嘴上说说，probe 第 3 段的反汇编里能直接看到这条清挂起指令的绝对地址：

```
08000188 <EXTI0_IRQHandler>:
 800018c:   2101        movs    r1, #1
 800018e:   f8c3 1c14   str.w   r1, [r3, #3092] ; 0xc14
 ...
 800019c:   40013000    .word   0x40013000
```

`0x40013000 + 0xC14 = 0x40013C14`，正是 EXTI->PR 的绝对地址（5.1 的表）。ISR 把常数 1 存进去，挂起清掉，门才不堵。

六、落地清单：六步把一条外部中断配通

以 PF6 → EXTI6 为例，一步都不能少（缺哪一步，对应的症状也写上了）：

#	动作	关键寄存器	漏了会怎样
1	开 GPIOF 时钟	RCC_AHB1ENR	引脚配置写不进
2	PF6 配成输入（带上/下拉）	GPIOF MODER/PUPDR	电平悬空乱触发
3	开 SYSCFG 时钟	RCC_APB2ENR bit14（`stm32f407xx.h:10076`）	EXTICR 写了不生效
4	SYSCFG 选门：EXTI6 接 PF6	EXTICR2[11:8] = 0x5（见 5.2 表）	边沿根本到不了 EXTI
5	EXTI 设沿放行：IMR bit6=1，FTSR/RTSR bit6=1	EXTI 0x40013C00 起（见 5.1）	请求被屏蔽
6	NVIC 放行：NVIC_SetPriority(EXTI9_5_IRQn, p) + EnableIRQ	ISER[0] bit23 = 0xE000E100（第二节表）	中断进不了内核

最后写 ISR：

```
void EXTI9_5_IRQHandler(void)          /* 强符号顶替 weak 默认（第四节） */
{
    EXTI->PR = (1UL << 6);             /* 第一件事：清挂起（5.3 的反汇编实证） */
    /* ... 记账式干活：置标志、搬数据，快进快出 ... */
}
```

注意 EXTI6 没有自己的专属向量：5~9 号线**共用** EXTI9_5_IRQn（=23，槽 39）。ISR 里要按 PR 的位分辨到底是哪条线。0~4 号线才有专属向量（EXTI0_IRQn=6 → 槽 22，偏移 0x58）。

七、ISR 现场：硬件压栈 8 字与 EXC_RETURN 的归途

中断现场动画

进异常的那一刻，内核硬件自动把 **8 个字**压进当前栈：xPSR、PC（返回地址）、LR、R12、R3~R0，正好是 AAPCS 里"调用者可破坏"的那批（C6 的栈帧知识在这里闭环）。ISR 如果还要用 R4~R11，由编译器在 prologue 里补压。所以 ISR 的机器码开头常有一串 `push`，"ISR 要短"也是硬约束而非风格建议。

进去时 LR 被硬件换成魔法值 **EXC_RETURN = 0xFFFFFFFD**（表示"返回 Thread 模式、用 PSP/MSP 弹栈"的组合编码，PM0214）。ISR 末尾一条普通的 `bx lr` 就完成异常返回，probe 反汇编里 EXTI0_IRQHandler 的最后一条正是 `8000198: 4770 bx lr`。硬件认出 LR 里的魔法值，不做普通跳转，而是把之前压栈的 8 个字弹回去，主循环从断点处无缝继续。被打断的代码**毫无知觉**，这正是 `volatile` 存在的原因（C3）。

附录：工程完整源码

本章取证工程：`code/stm32/04-nvic-exti/`（`sh probe.sh` 一条命令跑完，52/52 断言通过，无需开发板）。

probe.c（_Static_assert 对撞头文件 + 四张映射表打印）：

<<< ../../code/stm32/04-nvic-exti/probe.c

probe.sh（头文件现挖常数 + 宿主编译运行 + 交叉链接 objdump 取证）：

<<< ../../code/stm32/04-nvic-exti/probe.sh

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **先 SYSCFG 选门，再 EXTI 设沿，后 NVIC 放行，ISR 进门先清挂起**。四层口诀，中断永不丢。

延伸：中断现场压栈与调用约定（C6）直接相关；RTOS 下中断管理见 F2 上下文切换 与 S16 HardFault。

实物实验

- 基础：按键中断点灯，按第六节清单配 PF6 → EXTI6，ISR 里翻转 LED（quick win 完整版）。
- 进阶（抢占眼见为实）：EXTI0 配低抢占优先级、ISR 里 delay 约 1 秒；EXTI1 配高抢占优先级、ISR 翻转另一颗灯。按住键 0 再按键 1，灯 2 **立刻**响应——高优先级打断了低优先级 ISR。把两个优先级调成同抢占级再做一遍：灯 2 要等 1 秒，子优先级没有嵌套能力（第三节）。
- 观察（可选）：逻辑分析仪夹两颗灯，对比两组的波形时序，把"抢占"两个字变成屏幕上的图。

常见坑

- **忘清 EXTI_PR 挂起位**：进 ISR 就再也出不来，主循环彻底停摆，ISR 第一行就该清（5.3，反汇编里那条 `str.w` 就是干这个的）。
- **SYSCFG 时钟没开**：RCC_APB2ENR bit14 漏置位，EXTICR 写了不生效，EXTI 死活不触发（清单第 3 步）。
- **优先级分组混用**：SPL 默认组 2、RTOS 要求 4 位全抢占，移植 RTOS 后中断优先级"数值一样行为不同"多源于此（3.2 分组表，F8/R7 联动）。
- **优先级写出 15 以上**：不报错、不拒绝写，`(prio << 4) & 0xFF` 只在**本字节里**截断：16 存成 0x00（变成最高优先级），17 存成 0x10（变成 1）。见 3.1 的实测越界表，四条断言就在 probe 里。
- **想给 HardFault 设优先级**：设不了：`stm32f407xx.h:68-69` 的枚举里 HardFault_IRQn 被故意跳过，它的优先级硬件固定（3.4）。
- **ISR 里调用不可重入函数**：printf/malloc 在中断里用，等着偶发死机（S7/F4 给出正确姿势）。
- **共用向量不分辨线号**：EXTI9_5 / EXTI15_10 是多线共用的，ISR 里不查 PR 就分不清是谁触发的（第六节）。

短自测

1. USART1_IRQn = 37，它的使能位在 ISER 哪个下标、哪一位？IP 字节地址是多少？

> ▼ **看答案**
> 下标 = 37 / 32 = **1**，位号 = 37 % 32 = **5**，使能位在 `ISER[1]` 的 bit 5，地址 0xE000E100 + 4 = **0xE000E104**；IP 字节 = 0xE000E400 + 37 = **0xE000E425**。与 probe 打印的映射表一致（该表已过 `_Static_assert` 对撞）。

2. `NVIC_SetPriority(EXTI0_IRQn, 5)` 实际写进 IP 字节的值是多少？为什么？

> ▼ **看答案**
> **0x50**。F407 的 `__NVIC_PRIO_BITS = 4`（`stm32f407xx.h:49`），IP 字节只有高 4 位有效，逻辑优先级要左移 4 位：5 << 4 = 0x50。probe 的编码表实测 5 → 0x50。

3. 往 SCB->AIRCR 写分组值 5，正确的 32 位写入值是多少？少了哪一部分会被无视？

> ▼ **看答案**
> **0x05FA0500**。高 16 位必须带钥匙 VECTKEY = 0x5FA（`core_cm4.h:1661`），组值 5 放在 PRIGROUP 位 [10:8]（`core_cm4.h:531`），即 5 << 8 = 0x500。少了钥匙，写操作被硬件无视。probe 分组表第三列可直接查到这一行。

4. EXTI10 接 PG10，该写哪个 EXTICR 寄存器的哪几位，写什么值？

▼ 看答案
寄存器 = 10 / 4 → **EXTICR2**.....不对，10 / 4 = 2（整数除法）对应 EXTICR**3**（EXTICR1 管 0~3、EXTICR2 管 4~7、EXTICR3 管 8~11）；字段 = (10 % 4) × 4 = 8 → **[11:8]**；选 G 口编码是 0x6（A=0...F=5、G=6），写入值 0x6 << 8 = **0x00000600**。probe 实测表里有同字段的 F 口版本（EXTI10 → EXTICR3[11:8] = 0x500）可对拍。

5. 为什么"HardFault 优先级调低一点方便调试"这句话是错的？

▼ 看答案
HardFault 的优先级是硬件固定值 -1（仅次于 Reset -3、NMI -2），**不可编程**：`stm32f407xx.h:68-69` 的 IRQn_Type 枚举里 NonMaskableInt=-14 之后直接跳到 MemoryManagement=-12，HardFault_IRQn（-13）被故意省略；SHP 寄存器里也没有它的字节。能调的只有 MemManage/BusFault/UsageFault/SVCall/PendSV/SysTick（走 SHP）和全部外设中断（走 IPR）。

🎯 S4 中断自测

1. **USART1_IRQn = 37，它的使能位在 NVIC 的哪里？**
   A. ISER[0] 的 bit 37
   B. ISER[1] 的 bit 5（地址 0xE000E104）
   C. ISER[37] 的 bit 0
   D. IP 字节 0xE000E104 的 bit 5

2. **NVIC_SetPriority(EXTI0_IRQn, 5) 实际写进 IP 字节的值是多少？**
   A. 0x05
   B. 0x50
   C. 0x0A
   D. 0xA0

3. **要把优先级分组切成"2 位抢占 + 2 位子"（PRIGROUP=5），往 SCB->AIRCR 写的完整 32 位值是多少？**
   A. 0x00000500
   B. 0x05FA0005
   C. 0x05FA0500
   D. 0x05FA0050

4. **EXTI10 要接 PF10，SYSCFG 的哪几位写什么值？**
   A. EXTICR2[11:8] = 0x5
   B. EXTICR3[11:8] = 0x500
   C. EXTICR3[7:4] = 0x5
   D. EXTICR2[3:0] = 0xA

5. **"把 HardFault 的优先级调低一点，方便调试嵌套"——这句话错在哪？**
   A. HardFault 优先级可以设，但只能调高不能调低
   B. HardFault 的优先级硬件固定、根本不可编程：IRQn 枚举里 -13 被故意跳过，SHP 里也没有它的字节
   C. 应该调的是 NMI 而不是 HardFault
   D. 优先级数值越小越横，调低没有意义

6. **`NVIC_SetPriority(EXTI0_IRQn, 16)` 写下去，IP 字节里实际存的是什么？**
   A. 0x100 —— 溢出到下一个中断的字节里
   B. 0x00 —— 变成最高优先级 0（在本字节里被截断）
   C. 硬件拒绝这次写入，IP 保持原值
   D. 0xF0 —— 硬件自动饱和到最低优先级

先选完再翻：答案与逐项解析在附录 C 的本组；在线版可点击即时判分。

对照表：本章概念 → 仓库落点

概念	仓库落点
基址链 PERIPH/APB2/SYSCFG/EXTI	probe.sh 第 0 段输出，引 `stm32f407xx.h:910/928/974/975`
内核链 SCS/NVIC/SCB 基址	probe.sh 第 0 段输出，引 `core_cm4.h:1550/1556/1557`
IRQn 枚举值（含 SysTick=-1）	`stm32f407xx.h:75/83/84/100/105/114/117`，对撞断言 `probe.c:53-59`
HardFault 被省略的枚举空洞	`stm32f407xx.h:68-69`
ISER/IPR 寻址公式与七中断映射表	`probe.c:97-148`（`print_irq_table`）
`__NVIC_PRIO_BITS = 4`	`stm32f407xx.h:49`，对撞断言 `probe.c:46`
逻辑优先级 << 4 编码表	`probe.c:186-213`（`print_prio_encoding`）
越界优先级实测（16→0x00、17→0x10、20→0x40、256→0x00）	同一函数下半段，四条 `CHECK` 在 `probe.c:196-212`
PRIGROUP 位 [10:8] 与分组表	`core_cm4.h:531-532`，`probe.c:157-178`（`print_prigroup_table`）
VECTKEY = 0x5FA 写入钥匙	`core_cm4.h:1661`，对撞断言 `probe.c:48`
向量槽 = IRQn+16、98 项核对	probe.sh 第 2 段输出（`probe.sh:113` 起）
weak 被强符号顶替、槽内容 |1	probe.sh 第 3 段 objdump 输出（`probe.sh:139-189`）
EXTI 六寄存器偏移	`stm32f407xx.h:444-449`，对撞断言 `probe.c:73-79`，打印 `probe.c:226-239`
EXTICR 选门公式与实测值	`stm32f407xx.h:547/11662`，`probe.c:204-224`（`print_exticr_mux`）
SYSCFG 时钟使能位	RCC_APB2ENR bit14，`stm32f407xx.h:10076`
ISR 清挂起的反汇编实证	probe.sh 第 3 段：`str.w r1, [r3, #3092]` → 0x40013C14

你做到了

- 中断从"会自动跳的函数"还原成"三层配置 + 三条寻址公式 + 两段压栈"的机制，每个数字都能在头文件里指到行号；
- 优先级与嵌套有了肌肉记忆：4 位字段、<<4 编码、分组尺子、0x5FA 钥匙，一个都绕不开，这是 RTOS 章节的地基之一；
- 手握六步清单，能配通任意引脚的外部中断，也知道每一步漏掉的典型死法。

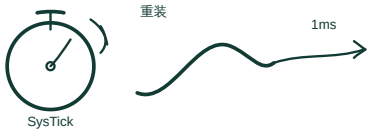
门牌是六，押在五到十之间就算中。这一章的每个数字你都不是背的，是顺着头文件行号现推的。

✅ 下一站：S5 SysTick：内核自带的"心跳发生器"，也是 RTOS tick 的真身。

通往单片机之路第 23 章

S5 SysTick：内核的心跳

难度 ★★☆☆☆ · 约 30 分钟



本章罗盘

问题

168MHz 要 1ms，LOAD 为什么是 167999？忘减一、COUNTFLAG 被偷读一次，各自的代价是什么？

交付

42/42 断言的 probe 输出加三笔账：任意频率/周期的 LOAD 手算、忘减一的 5.95ppm 日漂移 514ms 推算、回绕安全超时 (now-then)>=timeout 与朴素写法的反例对拍。

边界

只管内核自带的 24 位 SysTick：单次最长约 99.86ms，超了 SysTick_Config 拒配、硬截断则 ISR 快 737 倍；更长周期换 TIM（S6）；相位漂移是逐拍模型推算，板上长周期实测未做；RTOS tick 归 F 篇。

依据锚点 `a3 · f3 · mcu-code · mcu-anim`

> 🎯 SysTick 是内核自带的 24 位倒数秒表：归零自动重载、可申请中断——精确延时与 RTOS tick 都靠它。

本章精髓

1. 全部家当四个寄存器：**CTRL / LOAD / VAL / CALIB**，基址固定在 0xE000E010（SCS_BASE + 0x0010，`core_cm4.h:1550、1555`）：它是 ARM 内核外设，不是 ST 的，所有 Cortex-M 芯片地址一模一样，这就是 RTOS 移植层敢把它写死的原因。
2. 周期 = LOAD + 1：从 LOAD 数到 0 是 LOAD+1 个节拍，168MHz 下 1ms → `LOAD = 168000 - 1 = 167999`。忘减一不会少数次数，亏的是**相位**：每个 tick 慢 5.95ppm，日漂移 514ms（probe [3] 实测）。
3. COUNTFLAG 读过自清：「读它」这个动作本身有副作用。probe [7] 的对照：单读法 5 个周期数到 5 次，「先偷看再查」双读法数到 0 次，标志被第一次读偷走，查询式延时死等。

- **能理解**：`probe.c` 的 `sim_cycle` 模型（45-64 行）：每个周期 VAL--，**已经为 0** 的那个周期才重载并置 COUNTFLAG；「减一」和「读清」都是这条语义的必然推论，不是口诀。
- **能用**：跑通 42/42 断言；会算任意频率/周期的 LOAD；写出回绕安全的 `(now - then) >= timeout`；说清楚 `SysTick_Config` 干了哪四件事。

先修

- S2 时钟树（168MHz 从哪来）；S4 NVIC（优先级数值与嵌套规则）；C3 volatile（`g_tick` 这类共享变量）。

先跑起来（10 分钟 quick win）

```
cd code/stm32/05-systick
sh probe.sh   # 宿主 gcc 跑 42 条断言；PATH 里有交叉工具链时附送 Cortex-M4 目标文件
```

输出里找这两行：

```
[宿主] 断言通过 42/42
[6] SysTick_Config(168000)：LOAD=167999 VAL=0 CTRL=0x00000007，SHP[11]=0xF0 → 逻辑优先级 15
```

交叉编译段里再找常数：`movw r2, #16799`、`movw r1, #22783`：167999 与 24 位截断值真的进了目标文件。

出个选择题再跑：要 SysTick 每毫秒敲一次钟，LOAD 里该填原数、原数减一、还是原数加一？跑完看那行 LOAD 等于多少的输出，答案当场揭晓。

小节结构

小节	内容	四件套
寄存器版图	基址拼接 + 四寄存器偏移（`core_cm4.h:764-770`）	配置
CTRL 与 COUNTFLAG	四个位；读清语义与双读法 5:0 对照	代码分析
周期 = LOAD+1	167999 推导；忘减一的相位与日漂移	配置
24 位上限	0xFFFFFF；100ms 拒配与 737 倍速 bug	代码分析
换算与回绕	ns 整数公式；超时唯一安全写法	代码分析
SysTick_Config	三写寄存器 + 优先级最低的由来（`core_cm4.h:2022-2036`）	库解析

一、寄存器版图：四个寄存器，固定在 0xE000E010

CMSIS 原文（`core_cm4.h:764-770`）：

```
typedef struct
{
  __IOM uint32_t CTRL;   /* Offset: 0x000 (R/W) 控制与状态 */
  __IOM uint32_t LOAD;   /* Offset: 0x004 (R/W) 重载值     */
  __IOM uint32_t VAL;    /* Offset: 0x008 (R/W) 当前值     */
  __IM  uint32_t CALIB;  /* Offset: 0x00C (R/ ) 校准       */
} SysTick_Type;
```

基址是拼出来的：`SCS_BASE = 0xE000E000`（`core_cm4.h:1550`），`SysTick_BASE = SCS_BASE + 0x0010`（`:1555`），得 **0xE000E010**。probe [1] 用 `offsetof` 实测背书：

```
[1] 布局：CTRL/LOAD/VAL/CALIB 偏移 = 0x0/0x4/0x8/0xC，SysTick_BASE = 0xE000E010（内核外设，所有 Cortex-M 通用）
```

关键认识在最后半句：SCS（System Control Space）是 **ARM 内核的地盘**，不是芯片厂的外设区。STM32、GD32、nRF52 的 SysTick 全在同一个地址，RTOS 的移植层因此只需写一遍 tick 初始化。

二、CTRL 四个位：开关、中断、时钟源，和一个「读过就清」的标志

位定义出自 `core_cm4.h:773-783`，`probe.c:36-40` 同义镜像：

位	名字	作用
bit0	ENABLE	计数器使能
bit1	TICKINT	归零时是否申请 SysTick 中断
bit2	CLKSOURCE	1 = 内核时钟（168MHz），0 = 参考时钟
bit16	COUNTFLAG	归零标志——**读 CTRL 即清 0**

前三位望文生义；bit16 是本章最实用的坑。硬件语义：VAL 归零那一拍把它置 1；**软件读 CTRL 这个动作本身把它清 0**（read-to-clear，PM0214 的语义，`probe.c:71-77` 的 `sim_read_ctrl` 把它写成了代码）。probe [7] 的对照实验：

```
[7] COUNTFLAG：周期到后第一次读 CTRL=0x00010005（有标志），紧接第二次读=0x00000005（标志已被上一次读清掉）
    对照：5 个周期里单读法数到 5 次；「先偷看再查」双读法数到 0 次——标志被第一次读偷走，延时死等
```

落到代码（`probe.c:244-260`）：

```
/* 单读法：读一次，结果直接判断 —— 5 中 5 */
if (sim_read_ctrl(&b) & CTRL_COUNTFLAG) got_right++;
/* 双读法：先偷看一眼（调试打印/无意识的首次读），再查 —— 0 中 */
(void)sim_read_ctrl(&b);
if (sim_read_ctrl(&b) & CTRL_COUNTFLAG) got_wrong++;
```

实战规则：**读一次、存进变量、再判断**。调试器 watch 窗口点一下 `SysTick->CTRL` 都算「偷看」。实物上这个坑表现为延时偶发死等。

三、周期 = LOAD + 1：167999 是怎么来的，忘减一亏在哪

先看这张图：倒数、归零重载，和那个"读一次就清"的 COUNTFLAG。

SysTick 倒数与 COUNTFLAG 读清动画

硬件语义（`probe.c:45-48` 注释）：使能后每个周期 VAL--；VAL **已经为 0** 的那个周期执行重载、置 COUNTFLAG。所以从 LOAD 数到 0 再重载，一个完整周期是 LOAD+1 个时钟，「减一」不是口诀，是数出来的。

168MHz 要 1ms：$168000000 / 1000 = 168000$ 个时钟 → `LOAD = 168000 - 1 = 167999`。probe [2] 一族三例：

```
[2] 168MHz：1ms → LOAD=167999，100us → LOAD=16799，10ms → LOAD=1679999
```

模型实测（`probe.c:128-135`）：LOAD=167999 跑 1680000 周期，归零重载恰好 10 次，周期 = 168000 时钟 = 1.000000ms。

那忘减一（LOAD=168000）会怎样？反直觉的结论：**次数一样，相位不对**。probe [3]：

```
[3] 模型：LOAD=167999 跑 1680000 周期 → 归零重载 10 次（周期 = LOAD+1 = 168000 周期 = 1.000000ms）
    忘减一：同跑 1680000 周期两者都重载 10 次——差别不在次数在相位；每个 tick 慢 1/168000（5.95ppm），日漂移 514ms
    相位：第 10 次重载——正确配置在第 1512001 个时钟，忘减一在第 1512010 个，9 个间隔共欠 9 个时钟
```

逐拍对账（`probe.c:149-165`）：正确配置第 10 次重载落在第 $1 + 9 \times 168000 = 1512001$ 个时钟；忘减一落在第 $1 + 9 \times 168001 = 1512010$ 个。每个 tick 慢 1/168000 ≈ 5.95ppm，一天欠 $86400000 \times 1000 / 168000 = 514$ ms。短延时根本看不出，长期计时（数据记录器、日历时基）才暴露，这正是「忘减一」阴险的地方。

四、24 位上限：LOAD 装不下就是装不下

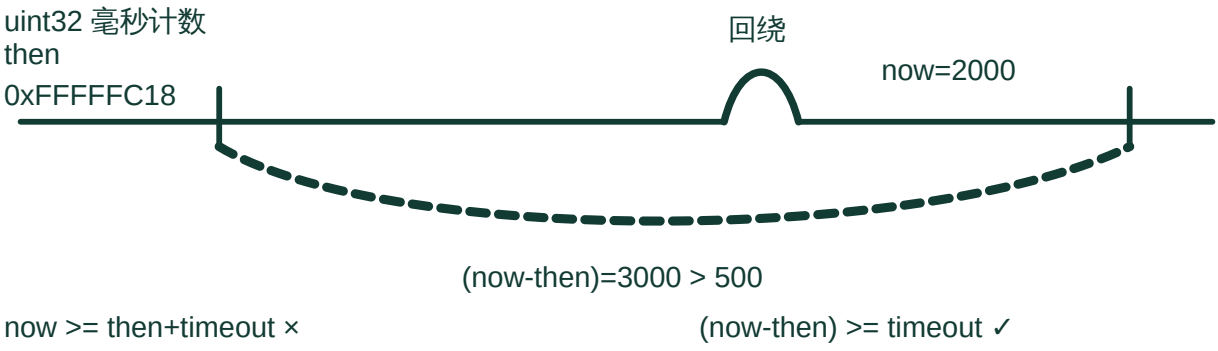
`core_cm4.h:786-791`：LOAD 与 VAL 的掩码都是 `0x00FFFFFF`，只有低 24 位，`LOAD_MAX = 16777215`。168MHz 下单次最长 $(16777215 + 1) / 168 = 99864$ µs ≈ 99.86ms：1ms tick 无压力，想单次 100ms 就不行。probe [4] 给了两级证据：

```
[4] 24 位上限：LOAD 最大 16777215（0xFFFFFF）→ 最长单次 99864us（≈99.86ms，1ms tick 无压力，100ms 单次不行）
    请求 100ms（ticks=16800000）：SysTick_Config 返回 1 拒配；若强行 &0xFFFFFF → LOAD=22783，tick 变成 135.6us，ISR 狂跳 737 倍
```

- **正规路径**：`SysTick_Config` 第一行就是 `(ticks - 1UL) > SysTick_LOAD_RELOAD_Msk` 则返回 1（`core_cm4.h:2024-2027`），即拒配，寄存器一个都没动。所以**必须查返回值**。
- **经典 bug**：不查返回值，强行 `& 0xFFFFFF` 写进去 → `LOAD = 22783`，tick 变成 135.6μs，ISR 以 737 倍速狂跳（$16800000 / 22784 = 737$），系统直接瘫在中断里（`probe.c:176-186`）。

更长的周期怎么办：要么 CLKSOURCE=0 走参考时钟（F407 上是 HCLK/8，单次上限拉到约 800ms），要么换 32 位的 TIM，那是 S6 的地盘。

五、时间换算与回绕：超时判断只有一种是安全的



无符号回绕：先减再比，跨回绕仍正确

1 个时钟 $= 1/168\text{MHz} = 125/21$ ns ≈ 5.952ns。整数公式 `ns = c*1000/168`（向零截断）：c=1 → 5ns，c=168 → 1000ns = 1μs（整 μs 无损），c=1000 → 5952ns = 5.952μs（按整数 μs 取 5μs，丢掉 0.952μs）。probe [5]：

```
[5] 换算：1 周期 = 125/21 ns ≈ 5.952ns；整数公式 ns=c*1000/168：c=1 → 5ns、c=168 → 1000ns=1us（整 us 无损）、c=1000 → 5952ns=5.
    回绕：uint32_t ms 计数 49.7 天绕一圈；then=0xFFFFFC18 now=5000 → (now-then)=6000ms（无符号减法跨回绕正确）；而 now>=then+time
```

毫秒计数用 uint32_t：$2^{32} / 86400000 \approx 49.7$ 天绕一圈。回绕不是「这辈子遇不到」，是「第 50 天前后必爆」——除非写成**无符号减法**：

```
uint32_t then = g_tick;                    /* 记录起点 */
if ((uint32_t)(g_tick - then) >= timeout)  /* 先减再比，跨回绕也正确 */
    timeout_handler();
```

反例（`probe.c:205-213`）：`then = 0xFFFFFC18`（绕前 1000ms）、`timeout = 500`，`then + timeout = 4294966796` 没回绕；而 `now = 2000` 已经回绕，朴素写法 `now >= then + timeout` 判出 0（未超时），可真实早已越过 500ms 阈值。这是**漏报**：本该立刻报超时，它却要一直等到 `now` 也爬到 4294966796 才翻转。无符号减法 `(now - then) = 3000 >= 500` 永远正确。全站的 delay、S9 的采样超时、F 篇的事件等待，统一用这个写法。

六、SysTick_Config 全貌：三写寄存器，一次设优先级

`core_cm4.h:2022-2036` 全文就六行有效代码：先查上限，然后 **写 LOAD、设优先级、写 VAL、写 CTRL**。probe [6] 实测 `SysTick_Config(168000)` 之后的现场：

```
[6] SysTick_Config(168000)：LOAD=167999 VAL=0 CTRL=0x00000007，SHP[11]=0xF0 → 逻辑优先级 15（4 位里的最低档，RTOS 就要它在最低）
```

- `CTRL = 0x00000007`：ENABLE | TICKINT | CLKSOURCE 三位全置：内核时钟、开中断、开跑。
- 优先级怎么算的：F407 的 `__NVIC_PRIO_BITS = 4`（`stm32f407xx.h:49`），`SysTick_IRQn = -1`（`:75`）；`__NVIC_SetPriority` 对负数 IRQn 走 `SCB->SHP[((IRQn)&0xF)-4]`（`core_cm4.h:1814-1824`），写入字节 $= priority \ll (8-4)$。$(1 \ll 4) - 1 = 15$，$15 \ll 4 = \text{0xF0}$ → SHP[11] = 0xF0，**4 位优先级里的最低档**。

为什么 RTOS 要它最低：tick 中断里跑的是调度器，而调度必须给一切外设中断让路：串口字节不等人的，高优先级 tick 会把 RXNE 中断顶飞（F3 展开）。

附录：工程完整源码

本章取证工程 `code/stm32/05-systick/`（`sh probe.sh` 一条命令跑完，不需要开发板）：

probe.c（七个现场：布局 / 周期与相位 / 24 位上限 / 换算回绕 / Config 全貌 / COUNTFLAG 对照）：

<<< ../../code/stm32/05-systick/probe.c

probe.sh（宿主编译运行 + 交叉编译取证）：

<<< ../../code/stm32/05-systick/probe.sh

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **LOAD 定周期、VAL 看进度、CTRL 开关机；频率除千再减一，标志一读就没，超时先减再比。**

延伸：SysTick 倒数与 COUNTFLAG 动画见 S5 动画；SysTick 作为 RTOS 时基在 F0 为什么需要 RTOS 展开；GD32 MTIME 对照见 V3。

实物实验

- 逻辑分析仪测 PF6 翻转周期：SysTick 版 `delay_ms(500)` 实测 500ms±0.1%，软件延时版误差肉眼可见。同一块板，两种「时间观」的实测对比。
- 双读法坑的实物复现：查询式延时跑到一半，用调试器 watch 窗口点一下 `SysTick->CTRL`，COUNTFLAG 被这次「偷看」清掉，延时再也出不来；再对照读一次存变量的写法，稳过。

常见坑

- **忘减一**：`LOAD = 168000` 每 tick 慢 5.95ppm、日漂移 514ms：次数对、相位错，最隐蔽的漂移源（probe [3]）。
- **COUNTFLAG 双读**：调试打印/偷看一次再判断，标志已被偷走，查询式延时死等（probe [7]：5:0 对照）。
- **超上限硬写**：LOAD 超 24 位被 `SysTick_Config` 拒配返回 1；强行截断则 tick 快 737 倍，ISR 打满 CPU（probe [4]）。
- **中断里又调查询式 delay**：ISR 里死等 COUNTFLAG，把 tick 中断自己堵死（嵌套规则回 S4）。
- **`g_tick` 忘加 volatile**：主循环永远看不到它变（C3 的经典现场）。

短自测

1. 168MHz 下要 2ms tick，LOAD 写多少？

> **▼ 看答案**
> $168000000 \times 2 / 1000 - 1 = 335999$。周期 = LOAD+1 = 336000 个时钟，整 2.000ms。与 probe [2] 同族：`reload_for_us`（`probe.c:98-102`）就是「频率换算再减一」。

2. 为什么「先读一次 CTRL 打印、再读一次判断」的延时会死等？

> **▼ 看答案**
> COUNTFLAG 是 read-to-clear：第一次读（打印那次）已把标志清掉，第二次读永远看到 0。probe [7] 对照：单读法 5 个周期数到 5 次，双读法数到 0 次（`probe.c:244-260`）。正确写法：读一次存进变量再判断。

3. `then = 0xFFFFFC18`、`timeout = 500`、`now = 2000`，超时了吗？哪种写法判得对？

> **▼ 看答案**
> 超时了：真实已过 3000ms。`(uint32_t)(now - then) = 3000 >= 500` 判对；`now >= then + timeout` 中 `then+timeout = 4294966796` 没回绕、`now = 2000` 已回绕，误判未超时（probe [5] 反例）。只有「先减再比」跨回绕永远正确。

4. `SysTick_Config(168000)` 后 CTRL 是什么值？优先级是多少？怎么算的？

> **▼ 看答案**
> `CTRL = 0x00000007`（ENABLE+TICKINT+CLKSOURCE）；`SHP[11] = 0xF0`，逻辑优先级 15，4 位优先级的最低档：$(1 \ll 4) - 1 = 15$，$15 \ll 4 = \text{0xF0}$（`stm32f407xx.h:49`、`core_cm4.h:2030`），probe [6] 实测同值。

5. 请求 100ms tick 会发生什么？

> **▼ 看答案**
> ticks = 16800000，`ticks - 1 > 0xFFFFFF`，`SysTick_Config` 返回 1 拒配、寄存器不动（`core_cm4.h:2024-2027`）。不查返回值硬写 `&0xFFFFFF` → LOAD=22783，tick 变 135.6μs、ISR 737 倍速狂跳（probe [4]）。长周期请换 TIM（S6）。

对照表：本章概念 → 仓库落点

概念	仓库落点
四寄存器布局与基址 0xE000E010	`probe.c:25-43` ； `core_cm4.h:764-770`、`1550`、`1555`
CTRL 四个位	`probe.c:36-40` ； `core_cm4.h:773-783`
COUNTFLAG 读清与 5:0 对照	`probe.c:71-77`（sim_read_ctrl）、 `232-260`
周期 = LOAD+1、相位差 9clk、日漂移 514ms	`probe.c:45-69`（模型）、 `128-165`
24 位上限与 737 倍速	`probe.c:42` 、 `167-186` ； `core_cm4.h:786-791`
ns 换算与回绕安全超时	`probe.c:188-213`
SysTick_Config 三写 + 优先级 15	`probe.c:79-96` 、 `216-230` ； `core_cm4.h:2022-2036` ； `stm32f407xx.h:49`、`75`
断言总账 42/42	`probe.c:262-263` ； `probe.sh` 第 1 段输出

你做到了

- 延时从「估」变「算」：167999 不再是魔法数，是「周期 = LOAD+1」的必然；
- 拿到全站反复使用的毫秒时基，回绕安全的超时写法入库；
- RTOS tick 前置课完成。你已经知道它为什么住在内核、为什么优先级必须最低。

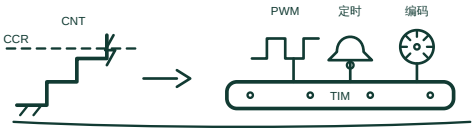
押减一的理由这章替你焊死了：差的那一拍不是误差，是计数器从零起跳的规矩。

✅ 下一站：S6 定时器 TIM：24 位不够用时，32 位 TIM 登场。从数脉冲到发 PWM，STM32 最全能的外设。

通往单片机之路第 24 章

S6 定时器 TIM：一器多用的瑞士军刀

难度 ★★★☆☆ · 约 25 分钟



本章罗盘

问题

更新频率公式里的两个 +1 从哪来？APB1=/4 时 TIM3 的时钟为什么是 84MHz 而不是 42MHz？

交付

一笔 1kHz PWM 的 PSC=83/ARR=999 手算、占空比=CCR/(ARR+1) 的呼吸灯代码，以及 APB1 从 /1 切到 /2 前后定时器时钟纹丝不动的一次对照实验记录。

边界

PWM 时基可编译可算，输入捕获的板上测频与 PA6→PA0 自测闭环仍待实测（回绕/丢边沿判定已有宿主测试覆盖）；编码器接口只有配置与状态机讲解、无配套工程；TIM1/8 的 BDTR/互补输出等高级特性不覆盖。

依据锚点 `a1` · `a2` · `f3` · `f4` · `mcu-code` · `mcu-anim`

> 🎯 TIM 是 STM32 最全能的外设：本质只是一个"会数数的计数器"，但配上预分频、自动重载、比较/捕获单元，它就能延时、发 PWM、测频率、读编码器——一招"计数"，十般变化。

本章精髓

1. **时基三件套**：PSC（预分频，先把时钟降下来）、CNT（计数器本体）、ARR（数到几归零）：更新频率 = TIMclk / (PSC+1) / (ARR+1)，两个"+1"是新手必考题。TIMclk 本身还有一条隐藏规则：**APB 分频 ≠ 1 时定时器时钟 ×2**（APB1=/4 时 TIM 是 84MHz 不是 42MHz），本章用 `02-tim-pwm` 实测出来。
2. **PWM 的本质是"比较"**：CNT 与 CCR 比较，小了输出高、大了输出低（极性可配）：占空比 = CCR/(ARR+1)，改 CCR 就改占空比，频率纹丝不动。这条"占空比和频率分家"的性质，是本章实验的核心观察点。
3. **输入捕获是"拍照"**：引脚边沿一到，硬件把当时的 CNT 拍进 CCR：两次拍照的差值×计数周期 = 脉宽/频率，全程不用 CPU 掐表。回绕必须满足计数位宽和无丢边沿前提。

- **能理解**：为什么改 CCR 不改频率；为什么 APB 分频 ≠ 1 时定时器时钟 ×2；无符号模差在哪些前提下才正确。
- **能用**：手算任意目标的 PSC/ARR；把 PWM 输出和捕获接成自测闭环，用硬件验证硬件。

先修

- S2 时钟（APB 倍频）、S3 GPIO（复用 AF）、S4 中断。

先跑起来（10 分钟 quick win）

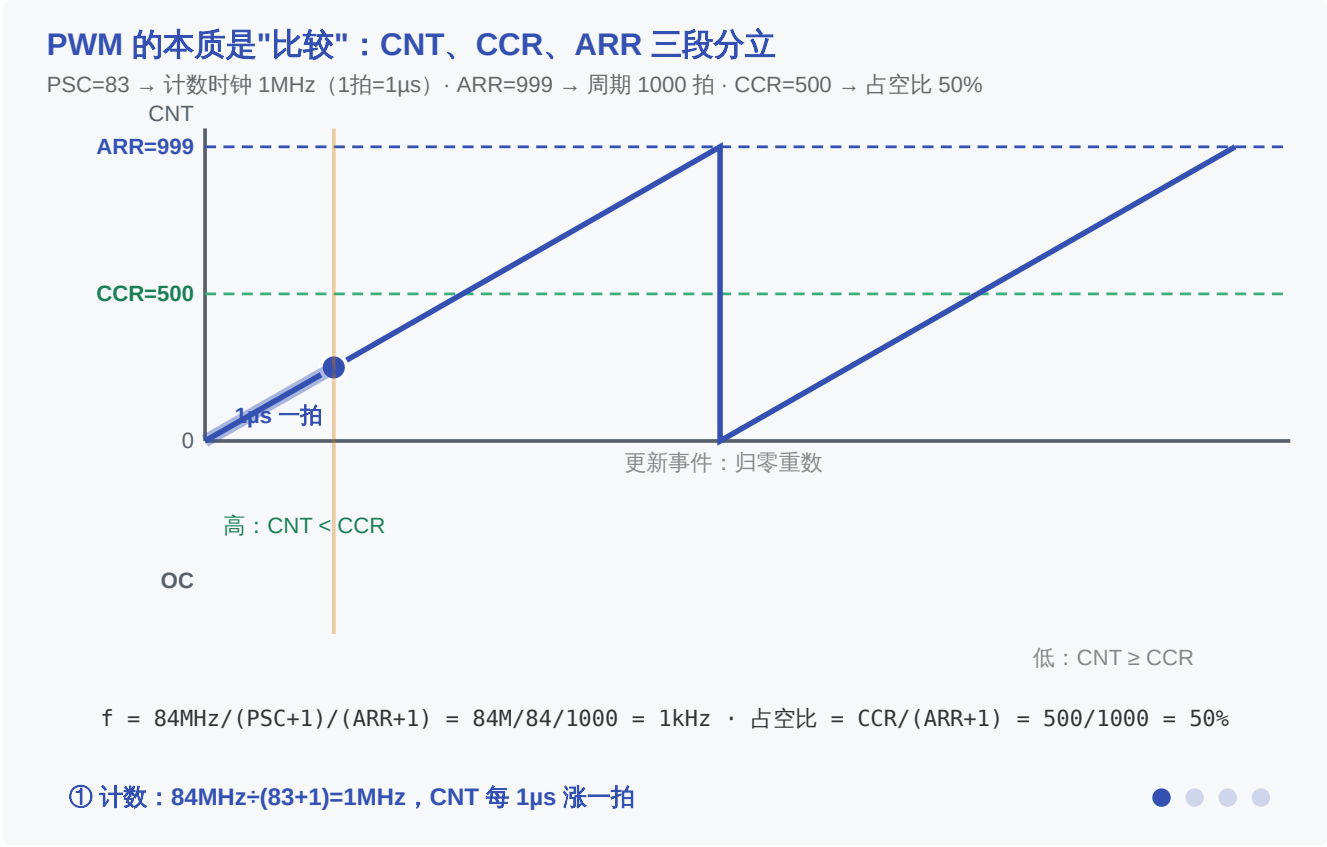
TIM3 CH1 输出 1kHz/50% PWM 到 **PA6**（AF2，见 DS8626 Table 9），红灯立刻"半亮"，这就是 PWM 调光的直观体感。

> 原文档曾写"TIM3 CH1 输出到 PF6，若复用表无 TIM 通道则换脚"。这不对：F407 的 **Port F 上根本没有 TIM3 通道**（PF6/PF7/PF8 在 AF3 上分别是 TIM10_CH1 / TIM11_CH1 / TIM13_CH1，见 DS8626 Table 9 的 Port F 段）。正确的做法不是"换个脚试试"，而是回查复用表：TIM3_CH1 在 PA6、PB4、PC6 上都有（AF2）。本工程选 PA6。

先别急着烧，三个答案里押一个：想让 PWM 变亮变暗，该动的是 PSC、ARR、还是 CCR？动错一个，频率跟着跑；动对一个，节拍纹丝不动只有亮度变。

动画：计数器怎么变成 PWM

CNT 往上涨、ARR 给它设了终点、CCR 在中间设了一个"开关点"，三段分立，改 CCR 只动开关点。看下面这张图时盯住三件事：CNT 的斜率（频率由 PSC/ARR 定）、OC 输出的高低翻转点（由 CCR 定）、以及"改 CCR 后频率纹丝不动"。



TIM PWM 计数器动画

板卡与版本前提

- 定时器归属：TIM2–5/12–14 在 APB1，TIM1/8–11 在 APB2（RM0090 §2/§7）。APB1=/4 时 PCLK1=42MHz，TIM 时钟 ×2 = 84MHz。
- 引脚-通道映射以 **F407ZGT6 datasheet 复用表**为准。本工程固定用 PA6（TIM3_CH1，AF2）与 PA7（TIM3_CH2，AF2）。
- 板载 RGB 红灯在 PF6（共阳、低电平点亮），与 TIM3 不冲突，这正是我们不用 PF6 的原因。

小节结构

小节	内容	四件套
一、时基三件套	PSC/CNT/ARR 与更新事件；频率公式推导	配置
二、定时中断	1s 定时翻转 LED；与 SysTick 分工对比	代码分析
三、PWM 输出	OC 模式/极性/CCR；ARR 与分辨率的取舍	配置
四、呼吸灯	渐变改 CCR；伽马校正的直觉版	引脚
五、输入捕获	边沿拍照测频率；独立 32 位时基与失效检测	代码分析
六、编码器接口	TI1/TI2 正交解码的硬件实现（概念+场景）	配置
七、代码分析	`02-tim-pwm/main.c` 逐段对表	代码分析
八、库解析对照	HAL 三函数逐行对照（hal_tim.c @ 1f6451c）；SPL 一手源码仍缺	库解析

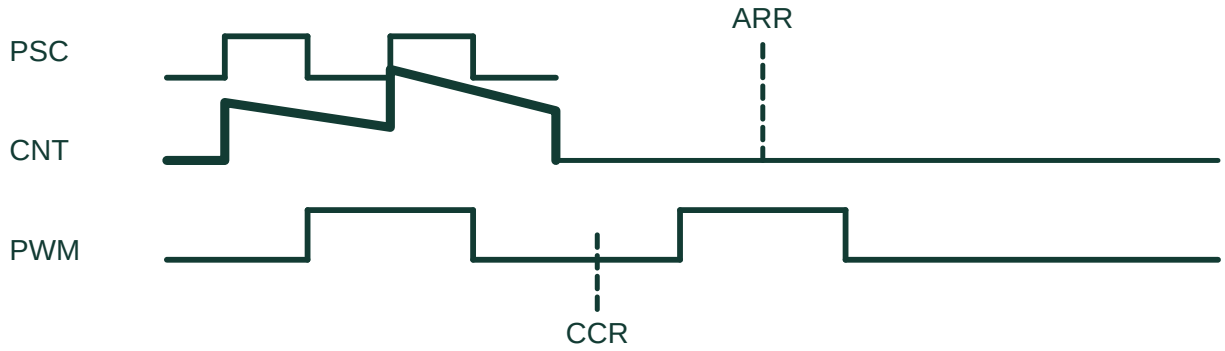
一、时基三件套：PSC、CNT、ARR

更新事件频率的公式：**f = TIMclk / (PSC+1) / (ARR+1)**。两个"+1"来自硬件的分频语义：PSC 是"每多少个时钟打一拍"，ARR 是"数到几归零"，两个都从 0 开始数。

例子：TIM3 时钟 84MHz，要 1kHz 更新：

- 选 PSC = 83 → 计数时钟 84MHz/84 = 1MHz；
- 选 ARR = 999 → 1000 拍一个周期 → 1kHz。

为什么是 83 和 999 而不是 84 和 1000？因为硬件数的是"0 到 PSC"和"0 到 ARR"，写进去的数比你想的少 1。**漏掉这个"+1"，1kHz 会配成 999Hz 或 1001Hz**，示波器上才发现。



定时器时基：PSC 分频后的计数时钟推动 CNT 从 0 数到 ARR，CNT 小于 CCR 时 PWM 为高

二、定时中断：1 秒翻灯，和 SysTick 分工

定时中断是最简单的用法：配置 PSC/ARR，开 `DIER.UIE`，在 `TIMx_IRQHandler` 里清 `SR.UIF` 再翻灯。

和 SysTick 的分工：**SysTick 归内核**，管"毫秒级心跳"（延时、RTOS 节拍）；**TIM 归外设**，管"这件事要精确到多少个时钟"。SysTick 不要用来做 PWM，TIM 不要用来做系统延时。两个都能用，但放错位置会很别扭。

三、PWM 输出：OC 模式、极性、CCR

PWM 模式 1（`CCMR1.OC1M = 110`）的规则：CNT < CCR 时输出有效电平，否则无效电平。配上极性位（`CCER.CCxP`）就得到"高电平有效"或"低电平有效"两种 PWM。

三个关键位，一个都不能少：

位	干什么
`CCMR1.OC1M = 110`	选 PWM 模式 1
`CCMR1.OC1PE = 1`	CCR 预装载：改 CCR 在**下一个周期整点**生效，不会在半中间突然变
`CCER.CCxE = 1`	输出使能：不开这个，引脚永远没有波形——新手第一大坑

占空比 = CCR/(ARR+1)。ARR=999 时 CCR=500 是 50%，CCR=250 是 25%。**分辨率由 ARR 决定**：ARR 越大，占空比可调的最小步进越小，代价是频率上限变低（f = 计数时钟/(ARR+1)）。

四、呼吸灯：只改 CCR 的渐变

呼吸灯 = 周期性把 CCR 从 0 扫到 ARR 再扫回来。本工程用三角形扫描（每 20ms 一步，约 4 秒一个来回）。**关键观察：扫 CCR 的过程中频率始终是 1kHz**，变的只是占空比。这就是"占空比和频率分家"的直观演示。

想更像人眼感受的"呼吸"，把线性扫描换成对数/伽马曲线：数字上非线性，视觉上才线性。

五、输入捕获：独立时基与回绕前提

PWM 的 TIM3 每 1000 tick 重载，不能拿它测量自己的完整周期：每次同相位边沿都可能得到同一个 CNT，差值为零。现在 PA6 输出，跳线接 **PA0（TIM2_CH1 AF1）**，TIM2 以 1MHz 自由计数、ARR=0xffffffff，捕获 ISR 及时读取时间戳。

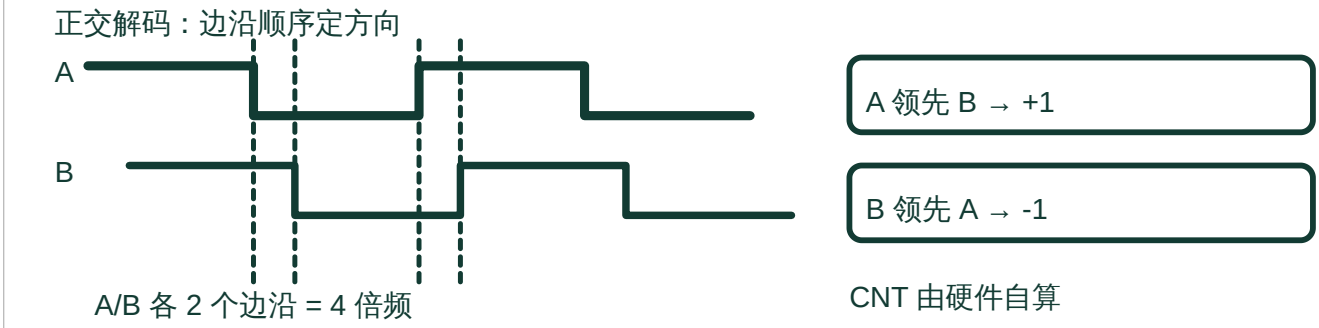
只有计数模数与运算位宽一致、间隔小于一整圈且没有丢边沿时，uint32_t 无符号相减才正确处理回绕。ARR=999 时 last=990、now=10，真实差为 20；uint32_t 相减却得 4294966316。即使改为模 1000，也不能区分经过了零圈还是一整圈。

本例接受 100us–500ms 的边沿间隔；首次捕获建立基准，过捕获后重新建立基准，无信号超过 1 秒清除频率。TIM2 的约 4294 秒硬件回绕不是本例的测频承诺。主循环只读取受短临界区保护的快照，不每 20ms 才消费 1kHz 捕获。

共享判定函数在 code/common/behavior/capture.h，宿主测试覆盖跨 0xffffffff 回绕、首拍、丢边沿、超量程与超时。板上测频仍待实测。

输入捕获：边沿拍照 → CNT 差值 → 脉宽/频率

六、编码器接口：正交解码的硬件状态机



正交编码器：A/B 边沿顺序定方向，4 倍频

TIM 的编码器接口模式（`SMCR.SMS = 0b001/010/011`）让 CNT 直接跟着正交编码器走：TI1 和 TI2 两路信号的**边沿顺序**决定加减，硬件自己算方向，CPU 只要定期读 CNT。这是"硬件做状态机"的极致例子。

正交信号：两路方波，90° 相差

旋转编码器输出两路方波 A/B，相位差 90°。正转时 A 领先 B，反转时 B 领先 A：

```
正转 (CW):  A ┐  ┌  ┐  ┌
              └  ┘  └  ┘
            ┌  ┐  ┌  ┐
B:          ┘  └  ┘  └    (A 领先 B 90°)

反转 (CCW): B 领先 A 90°，方向相反
```

SMCR.SMS：三档编码模式

SMS 值	模式	计数的边沿	分辨率
`001`	编码器模式 1	只计 TI1 边沿	1x
`010`	编码器模式 2	只计 TI2 边沿	1x
`011`	编码器模式 3	TI1 + TI2 **双沿**	**4x**

模式 3 最常用：两路信号各有上升/下降两个边沿，每个边沿都计数，分辨率 = 编码器线数 × 4。

硬件状态机：4 态决定加减

TIM 内部有一个 2-bit 状态机，跟踪 A/B 的当前电平组合：

当前状态	下一状态	CNT 动作
A=0, B=0	A=1, B=0	+1（正转）
A=1, B=0	A=1, B=1	+1
A=1, B=1	A=0, B=1	+1
A=0, B=1	A=0, B=0	+1
反向序列	—	-1

硬件在每个边沿自动判断"这是正转的第几步还是反转的第几步"，CNT 跟着加减。CPU 完全不需要参与方向判断。

配置步骤

```
/* TIM2 编码器模式 3：TI1 + TI2 双沿计数 */
RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;

/* PA0=TI1, PA1=TI2 → AF1 (TIM2) */
GPIOA->MODER  |= (2 << 0) | (2 << 2);   // 复用模式
GPIOA->AFR[0] |= (1 << 0) | (1 << 4);   // AF1

TIM2->SMCR  &= ~TIM_SMCR_SMS;
TIM2->SMCR  |= 0b011;                    // SMS=011: 编码器模式 3
TIM2->CCMR1  = 0;                        // CC1S=01 (TI1), CC2S=01 (TI2)
TIM2->CCMR1 |= (1 << 0) | (1 << 8);     // IC1F=0000 (无滤波)
TIM2->CCER   = 0;                        // 不反相
TIM2->ARR    = 0xFFFFFFFF;               // 32 位满量程
TIM2->CR1   |= TIM_CR1_CEN;

/* 读位置：直接读 CNT */
int32_t position = (int32_t)TIM2->CNT;   // 有符号，正反转自动区分
```

分辨率计算

编码器标称 500 线/圈（每圈 500 个 A 脉冲），用模式 3 双沿计数：

```
实际分辨率 = 500 × 4 = 2000 计数/圈
角度精度   = 360° / 2000 = 0.18°
```

如果用 16-bit 定时器（ARR=65535），最大计数 65535，超过需要软件溢出计数。TIM2 是 32-bit，直接 `CNT` 就是有符号位置。

滤波与噪声

实际编码器信号有抖动。`CCMR1.IC1F` 设置输入滤波：

IC1F	滤波	效果
`0000`	无	最快，噪声敏感
`0010`	f_CLK/4 × N=6	推荐起点
`1000`	f_DTS/8 × N=6	强滤波，延迟增大

滤波本质是"连续 N 次采样一致才认可边沿"，和消抖开关的原理一样。

七、代码分析：`02-tim-pwm/main.c`

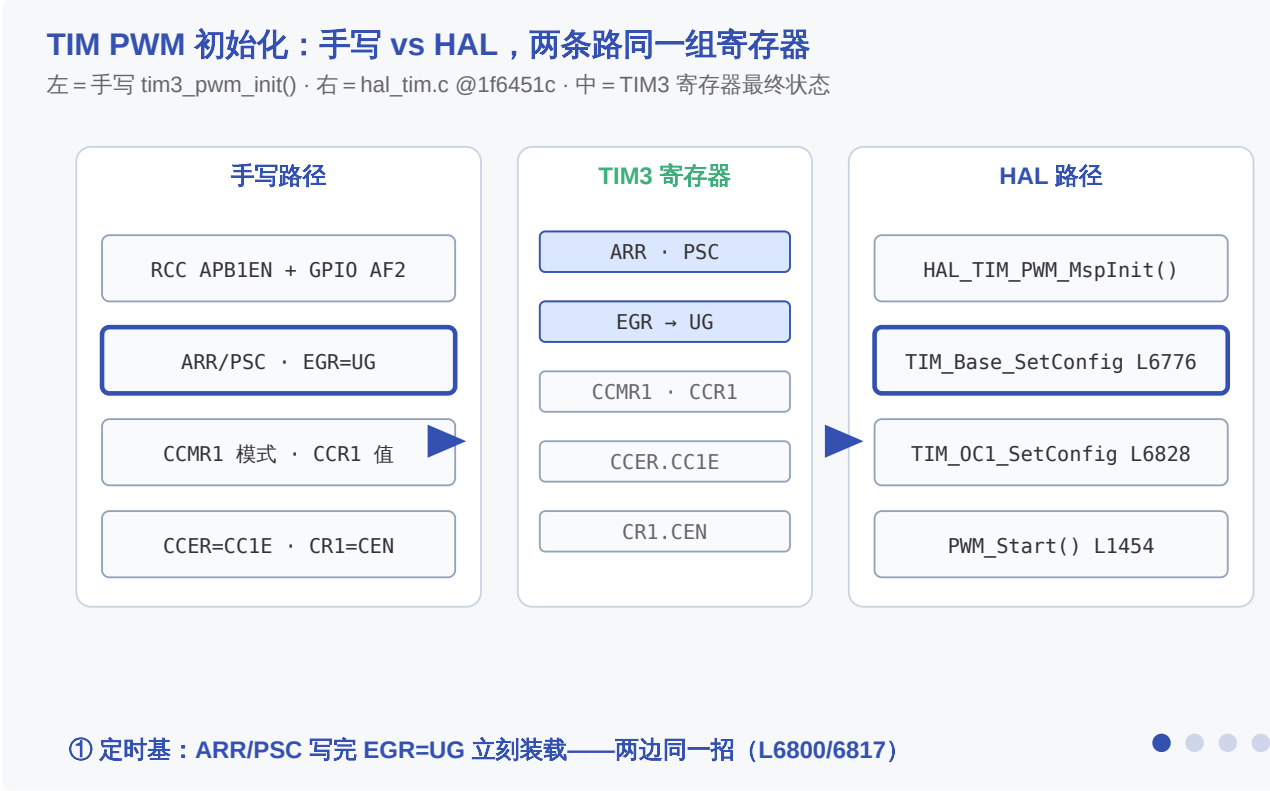
- `clocks_read()`：从 CFGR 反推 HCLK/PCLK1，HPRE/PPRE 的编码表直接按 RM0090 写（注意 AHB 没有 /32 这一档）。
- `timer_clock()`：APB 分频≠1 时 ×2，一行代码。
- `tim3_pwm_init()`：PSC/ARR/CCMR1/CCER 按上面第三、五节逐位落位；`EGR.UG` 让 PSC/ARR 立刻生效而不是等第一个更新事件。
- 捕获 ISR：读取 TIM2 时间戳；主循环只读一致快照，过捕获/超时不输出可信频率。
- 两个对照实验（改 APB1 分频前后读定时器时钟）实测 ×2 规则。

八、库解析对照：SPL/HAL 同名初始化

ST 官方仓库 `STMicroelectronics/stm32f4xx_hal_driver`（master @ `1f6451c`）把本章寄存器动作拆成三个函数：`HAL_TIM_PWM_Init()` 定时基、`HAL_TIM_PWM_ConfigChannel()` 输出通道、`HAL_TIM_PWM_Start()` 放行。逐条对标本工程 `tim3_pwm_init()`：

本工程 `tim3_pwm_init()`	HAL 对应（hal_tim.c 行号 @ 1f6451c）	对照结论
PSC/ARR 直接写、`EGR.UG` 立刻生效	`TIM_Base_SetConfig()`（L6776）：先 ARR（L6800）后 PSC（L6803），收尾 `EGR = UG`（L6817）	同一招：UG 把影子值立刻装进来，不等第一个更新事件
CCMR1 放 OC 模式、使能预装载	`HAL_TIM_PWM_ConfigChannel()`（L4219）→ `TIM_OC1_SetConfig()`（L6828）：**先清 CCER.CC1E** 再改模式，OC1PE 单独置位（L4245），CCR1 初值写入（L6893）	HAL 改参数前先关通道——防止中途输出毛刺波形
最后 CCER 使能 + CR1.CEN	`HAL_TIM_PWM_Start()`（L1454）：`TIM_CCxChannelCmd()` 移位置 CCxE（L7626）；带 BKIN 的定时器（TIM1/8）才额外开 BDTR.MOE，从机触发模式不抢 CEN	你的两步 = HAL 一步；TIM1/8 还多一个主输出开关
自己开 APB 时钟、配 GPIO	放进 `HAL_TIM_PWM_MspInit()` 回调，仅句柄状态 RESET 时被调	HAL 把"芯片初始化"与"板级初始化"分成两层

SPL 的 `TIM_TimeBaseInit()` / `TIM_OC1Init()` 做的是同一组落位，但 StdPeriph 未随 ST 官方 GitHub 分发，一手源码仍缺；位定义以 RM0090 为准。



HAL 三段初始化与手写落位的对账图

附录：工程完整源码

<<< ../../code/stm32/02-tim-pwm/main.c

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **PSC 除一除，ARR 数一数，频率=(时钟÷俩加一)；PWM 靠 CCR 比一比，捕获靠边沿拍一拍；回绕先看模数与丢边沿。**

延伸：TIM TRGO 可触发 ADC 采样（S9）；硬件 TIM 与 RTOS 软件定时器的分工见 F6。

实物实验

- E03 示波器看 PWM：改 CCR 看占空比、改 ARR 看频率、逻辑分析仪读数与手算对表；
- 呼吸灯：CCR 三角渐变，肉眼验证"数字调光"；
- 自测闭环：PA6 接 PA0，捕获实测频率应与设定一致；
- 对照实验：把 APB1 从 /1 切到 /2，读 `g_tim_clk_before/after`：定时器时钟纹丝不动。

常见坑

- **两个"+1"漏算**：1kHz 配成 999Hz 的隐形误差；
- **忘开 GPIO 复用 + CC 通道使能**：CCxE 位不开，引脚永远没波形（输出使能是独立一步）；
- **APB 倍频规则记错**：APB1 分频≠1 时 TIM 时钟×2，TIM2 按 42MHz 算全错，它是 84MHz；
- **ARR=65535 溢出盲区**：测低频时 CNT 溢出忘计，结果差一整圈。必须使用溢出计数或独立宽计数器，不能只靠 uint32_t 强转；
- **改 CCR 没开预装载**：占空比在周期中间突然变，波形上出现一个"半边高半边低"的怪周期。

短自测

1. 为什么改 CCR 不会改 PWM 的频率？

▼ **参考答案**
频率由 PSC 和 ARR 决定（计数周期）；CCR 只决定"在一个周期里的哪一拍翻转电平"。改 CCR 只是移动开关点，不改周期的总长度。

2. TIM3 挂在 APB1 上，APB1 分频 /4，PCLK1=42MHz，TIM3 的时钟是多少？

▼ 参考答案
84MHz。规则：APB 分频 ≠ 1 时定时器时钟 = PCLK × 2。只记 42MHz 会把所有定时器算慢一半。

3. 用 PSC=83、ARR=999 配 1kHz，为什么写成 PSC=84、ARR=1000 是错的？

▼ 参考答案
硬件数的是"0 到 PSC"和"0 到 ARR"：84 对应分频 85，1000 对应周期 1001。更新频率 = 84MHz/85/1001 ≈ 987Hz，差一个"%"级。

4. 输入捕获里"相邻两个上升沿的 CCR 差值"在 CNT 回绕时怎么算才对？

▼ 参考答案
先确认计数模数等于 2^32、间隔小于整圈且未丢边沿，本例 TIM2 才可用 uint32_t 模差；ARR=999 的 TIM3 不能套用。

🎯 S6 定时器自测

1. **为什么改 CCR 不会改 PWM 的频率？**
   A. CCR 被硬件锁定了
   B. 频率由 PSC 和 ARR 决定，CCR 只决定周期内哪一拍翻转电平
   C. CCR 只能增大不能减小
   D. CCR 影响的是占空比，不是频率

2. **TIM3 挂在 APB1 上，APB1 分频 /4，PCLK1=42MHz，TIM3 的时钟是多少？**
   A. 42MHz
   B. 84MHz
   C. 168MHz
   D. 21MHz

3. **用 PSC=83、ARR=999 配 1kHz，为什么写成 PSC=84、ARR=1000 是错的？**
   A. 数值太大超出寄存器范围
   B. 硬件数的是 0 到 PSC 和 0 到 ARR，84 对应分频 85、1000 对应周期 1001，结果 ≈987Hz
   C. PSC 和 ARR 不能同时为偶数
   D. 没有区别，两种写法等价

4. **输入捕获里'相邻两个上升沿的 CCR 差值'在 CNT 回绕时怎么算才对？**
   A. 直接相减，负数就取绝对值
   B. 先确认计数模数，用无符号差值自动处理回绕
   C. 回绕时无法测量
   D. 必须用软件计数器

先选完再翻：答案与逐项解析在附录 C 的本组；在线版可点击即时判分。

对照表：本章概念 → 仓库与上游落点

概念	落点
PSC/ARR/CCR/CCMR1/CCER 位定义	ST CMSIS `stm32f407xx.h`（TIM_* 段）
APB ×2 规则实测	code/stm32/02-tim-pwm/main.c `timer_clock()` 与对照实验
无符号差值处理回绕	同上捕获循环
引脚复用	DS8626 Rev 9 Table 9（PA6/PA7 AF2）
动画	tim-pwm-counter.svg、tim-input-capture.svg、tim-hal-init-path.svg（§八对账）
HAL 同名初始化逐行对照	ST 官方仓库 stm32f4xx_hal_driver @ `1f6451c` `Src/stm32f4xx_hal_tim.c`（见 §八）；SPL 未随官方 GitHub 分发，一手源码仍缺

你做到了

- 时基/PWM/捕获三大件各自跑通，且有一个自测闭环可以验证；
- "计数器+比较器"的组合思维建立，后面 DAC 触发、编码器都是它的变体；
- 亲手实测了"APB 分频≠1 时定时器×2"这条不在框图里的规则。

押 CCR 的那把，中的其实是整章地基：开关点挪了，斜率没动，呼吸灯怎么喘都不跑调。

✅ 下一站：S7 USART：让芯片开口说话，波特率怎么算、printf 怎么接、数据怎么一位位飞出去。

通往单片机之路第 25 章

S7 USART：让芯片开口说话

难度 ★★★☆☆ · 约 30 分钟

本章罗盘

问题

波特率误差、TXE/TC 标志与 IDLE+DMA 环形接收怎样连成一条不丢字节的串口链路？

交付

一笔 168MHz 下 USARTDIV 手算、一张 TXE/TC/IDLE 事件表，以及 IDLE+DMA 环形缓冲的读写指针不变量。

边界

IDLE 只表示线路空闲，不等于业务协议的一帧；本章不覆盖 RS485 半双工与流控。

依据锚点 `a1` · `a2` · `f3` · `mcu-code` · `mcu-anim`

> 🎯 串口是单片机的"嘴巴"和"耳朵"。它不讲道理，只认节拍：双方约定好波特率，然后一位一位地数。你发 0x41，对方收到 0x41，前提是两边数拍子的速度不能差太多——差 2% 还能凑合，差 3.549%（1MHz 喂 115200 正是这个数）就开始丢帧，差 10% 就只剩乱码。

本章精髓

1. **一帧的结构是固定的**：空闲高 → 起始位（拉低）→ 8 个数据位（LSB 先发）→ 停止位（拉高）。在逻辑分析仪上认出一帧，就是认出这个节奏。
2. **波特率是算出来的，不是填出来的**：默认 16 倍过采样（OVER8=0）时 `baud = f_PCLK / BRR`。时钟一变，波特率跟着变，这就是"改了时钟树、串口就乱码"的原因。
3. **TXE ≠ TC**：TXE 只说"数据寄存器空了，可以塞下一个"；TC 才说"停止位已经发出去"。在 RS-485 这类需要切方向的场合，漏掉 TC 会把最后一位切掉。

- **能理解**：为什么 1MHz 不能当 115200 的时钟；为什么 IDLE 标志必须"读 SR 再读 DR"才清；为什么 DMA 不理解"帧"。
- **能用**：寄存器级把 USART1 跑起来回显；用逻辑分析仪在 PA9 上认出 0x41 的每一位。

先修

- S2 时钟（PCLK2 怎么来）、S3 GPIO（复用 AF7）、S4 中断。

先跑起来（10 分钟 quick win）

`code/stm32/03-uart-dma`：烧录后 USB-TTL 接 PA9/PA10，115200 8N1，电脑上发任意字符串，板子原样回显，这就是最小可跑通的串口。

连串口之前，先押个结局：如果波特率算错了一点点，回显会变成原样一句、一堆乱码、还是彻底安静？押完把 BRR 故意拨错一格，眼睛当场收账。

动画：一帧的解剖

0x41 是怎么一位一位飞出去的：起始位把线拉低、然后 LSB 先发、八位走完、停止位拉高。对照 0x55（0101 0101）看会更清楚：同一波特率下，0x41 和 0x55 的波形完全不同，但帧结构一模一样。

USART 帧动画

板卡与版本前提

- USART1 在 APB2 上，PA9=TX、PA10=RX（AF7，DS8626 Table 9）。8 位字长、1 停止位、无校验、无流控是默认也最常用的配置。
- 波特率误差的可接受范围不是"经验上 ±2%"这么含糊：8N1 一帧最危险的采样点是最后一个数据位的中点（8.5 位时刻），倒推总容差 ≈5.882%，再由收发两端对半分成每端 2.941%（推导见第二节）。这也是为什么不能用 1MHz 喂 115200（实测误差 -3.549%，见下一节那张表）。

小节结构

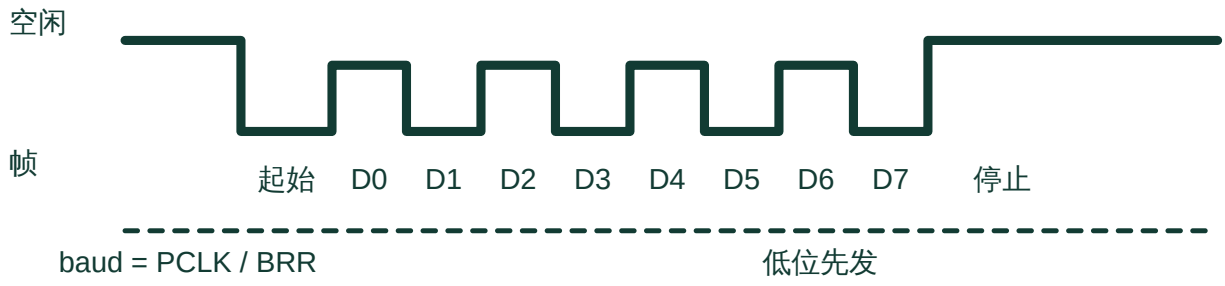
小节	内容	四件套
一、一帧的结构	起始/数据/停止/LSB；0x41 与 0x55 对照	引脚
二、波特率	`baud = PCLK/BRR` ；误差表；1MHz 反例	配置
三、发送	TXE/TC 两级；"可写"不等于"发完"	代码分析
四、接收	RXNE、IDLE、错误位；DMA 环形 + IDLE 帧边界	代码分析
五、printf 与日志	retarget 与代价	库解析
六、代码分析	`03-uart-dma/main.c` 逐段对表	代码分析
七、库解析对照	HAL Init/Transmit 逐行对照（hal_uart.c @ 1f6451c）；SPL 一手源码仍缺	库解析

一、一帧的结构：起始、数据、停止

空闲时线是**高电平**。一帧从这里开始：

1. **起始位**：拉低一位的时间，意思是"注意，数据来了"。
2. **数据位**：8 位（可配 7 或 9），**LSB 先发**，0x41 = `0100 0001`，上线顺序是 `1,0,0,0,0,0,1,0`。
3. **停止位**：拉高一位（可配 0.5/1/1.5/2），意思是"我说完了"。

用逻辑分析仪看 0x41 和 0x55（`0101 0101`）：数据位波形完全不同，但起始/停止的节奏一模一样。**帧结构是协议的骨架，数据只是填进去的肉。**



UART 一帧：空闲高、起始位拉低、8 个数据位低位在前、停止位拉高；波特率由 PCLK/BRR 决定

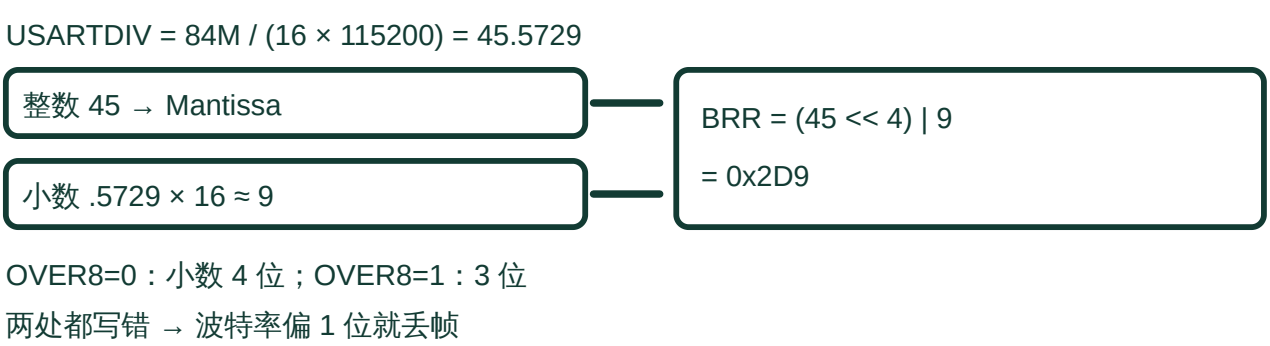
二、波特率：`baud = PCLK / BRR`

默认 **16 倍过采样**（OVER8=0）时：

```
BRR = f_PCLK / baud
baud = f_PCLK / BRR
```

> 【注】 OVER8=1 时是 8 倍过采样，公式变成 `baud = 2 × f_PCLK / BRR`：同样的 BRR，波特率翻倍。高速率场合用得上，代价是每个位只采 8 次，抗噪变差。本工程保持默认 16 倍。

所以 BRR 必须由**实际的** PCLK 算出来，不能写死。PCLK 变了，BRR 就得跟着变，改了时钟树、串口就乱码，根子在这里。



BRR 的来历：PCLK 与 baud 相除后拆成整数 Mantissa 与小数 Fraction 两个字段

> 【注 2】 上面那两条公式看起来把 BRR 当纯整数分母，而 RM0090 把它拆成 `DIV_Mantissa[15:4]` + `DIV_Fraction[3:0]`，写法是 `USARTDIV = Mantissa + Fraction/16`、`baud = f_PCLK/(16 × USARTDIV)`。把 16 乘进去：`16 × USARTDIV = 16 × Mantissa + Fraction`，右边正好就是寄存器里那 16 位，**整个寄存器的值就是分母**。所以固件直接写一个整数（139 = 0x8B → Mantissa 8、Fraction 11），不需要先算小数再拼字段；寄存器的 1 位粒度已经是硬件能给的最细分频，不存在"小数位还没用上"。

误差表（`code/stm32/03-uart-dma/probe.sh` 第 1 步本机实跑打印的那八行，逐行由 `BAUD_FROM_PCLK` / `PCLK_OF_BRR` 这两个固件宏算出；第 2 步再拿这张表与正文逐行 diff，改一个数就红）：

PCLK2	目标波特率	BRR	实际波特率	误差
1 MHz	115200	9	111111	-3.549%
8 MHz	115200	69	115942	+0.644%
16 MHz	9600	1667	9598	-0.020%
16 MHz	115200	139	115108	-0.080%
16 MHz	460800	35	457143	-0.794%
16 MHz	921600	17	941176	+2.124%
42 MHz	115200	365	115068	-0.114%
84 MHz	115200	729	115226	+0.023%

第三列不是查表来的，是 `BRR = round(PCLK2 / baud)` 那一次四舍五入的结果；第四列再拿这个整数除回去。所以**误差的上界就是半个量化格**：16 MHz / 115200 那一格宽 822.2 baud（0.714%），半格 0.357%。探针在 1..65535 里穷举过最优 BRR，这八档的最优解与固件的四舍五入**完全一致**，这个误差不是实现瑕疵，是整数 BRR 的物理下限。

量化误差本身往往很小，但**误差是沿整帧累积的**：接收端在每个位的中点重采样，最危险的时刻是最后一个数据位的中点，8N1 帧里就是 8.5 位时刻。从这里倒推预算：`|ε| × 8.5 < 0.5` → 总容差 ≈ **5.882%**，还要由收发两端的时钟**对半分**，这才是"±2% 经验值"的完整来历。帧越长（8E2 有 12 位，预算收到 4.762%）、时钟源越差（HSI ±1%），预算越紧。

> 🧮 这里在在线版是一个可交互组件（ `BaudCalc` ，改参数即时出数）；离线书请按正文给定的参数手算，或到在线版该页把玩。

反例：**1MHz 的时钟下配 115200**，BRR = 1e6/115200 ≈ 8.68 → 取 9 → 实际 111111.1，误差 **-3.549%**，单这一项就吃掉 5.882% 预算的六成，剩下每端只有 1.166%，刚够 HSI 自己的 ±1%，一点余量都不剩；两端再各带一点时钟源容差，通信必然不可靠。这就是原文档里"115200×16=1843200，1MHz 不满足该文同句给定条件"这句话的根源：**1MHz 是 PLL 的 VCO 参考输入，不是能直接喂 USART 的外设时钟**（回 S2）。

同理，16 MHz 下想要 **921600**：OVER8=0 只能取 BRR=17，实际 941176，量化误差 **+2.124%** 单项就超过每端 2.941% 的一半额度：要么按注里说的改 OVER8=1（等价于把时钟当 32 MHz 算，BRR=35，误差回到 -0.794%），要么把 PCLK2 提上去。这一格是"高速率不是想要就要"的硬边界。

三、发送：TXE 与 TC 是两级

- `USART_SR.TXE`（bit7）：**数据寄存器 DR 空了**，可以塞下一个字节。此时数据可能还在移位寄存器里往外蹦。
- `USART_SR.TC`（bit6）：**整个字节（含停止位）已经发完**。只有 TC 置位，才能安全地切 RS-485 方向、断电、进低功耗。

发送函数的正确姿势：

```
/* 塞数据只看 TXE，全部塞完再等 TC */
while ((USART1_SR & USART_SR_TXE) == 0) { }
USART1_DR = byte;
/* ... 循环 ... */
while ((USART1_SR & USART_SR_TC) == 0) { }   /* 最后必须等这个 */
```

漏掉最后的 TC 等待，在 RS-485 上会切方向太早，把最后一位切掉一半，对方收到的是"半个字节"，帧校验直接挂。

TXE vs TC：数据寄存器空 ≠ 移位寄存器发完

四、接收：RXNE、IDLE 与 DMA 环形缓冲

接收有两个问题：**字节级**和**帧级**。

- **字节级**：`USART_SR.RXNE`（bit5）置位 = 收到一个字节，读 `DR` 取走。
- **帧级**：数据是一串一串来的，怎么知道"这一串到哪儿结束"？答案是 **IDLE**（bit4）：总线空闲一个帧时间，IDLE 就置位。

IDLE 标志有个坑：**它不是靠写 0 清的，必须"先读 SR、再读 DR"这个序列才清**。少一步，IDLE 一直挂着，中断风暴。（旧答案："IDLE 标志必须先读 SR 再读 DR 才清，否则中断风暴"）

配 DMA 之后的经典结构：

```
串口移位寄存器 → (RXNE 触发 DMA) → 环形数组（我们的内存）
                  ↑ IDLE 置位时，DMA 指针位置就是帧边界
```

DMA 不理解"帧"，它只理解"搬够 N 个字节"；帧边界由 IDLE 标志告诉你。`03-uart-dma` 用 DMA2 Stream5 环形接收，每收到一帧就 IDLE 中断一次、把这一帧回显。

接收错误：`ORE`（溢出：CPU/DMA 没来得及取，新字节盖掉旧字节）、`FE`（帧错误：没等到停止位）、`NE`（噪声）。这三个位也必须"读 SR 再读 DR"清。

五、printf 与日志：retarget 的代价

`printf` 好用，但它是 newlib 的，默认往 stdout 写。嵌入式上没有 stdout，需要 retarget：实现 `_write()` 把字节送进 USART。代价不小：newlib 的 printf 会把几百字节到几 KB 的代码拉进来，还会用到堆。

本仓库的工程**不用 printf**，用一个手写的 `uart_write_blocking()`，几十字节，行为完全可控。等你真的需要格式化输出时，再决定要不要为 printf 付这个代价。

六、代码分析：`03-uart-dma/main.c`

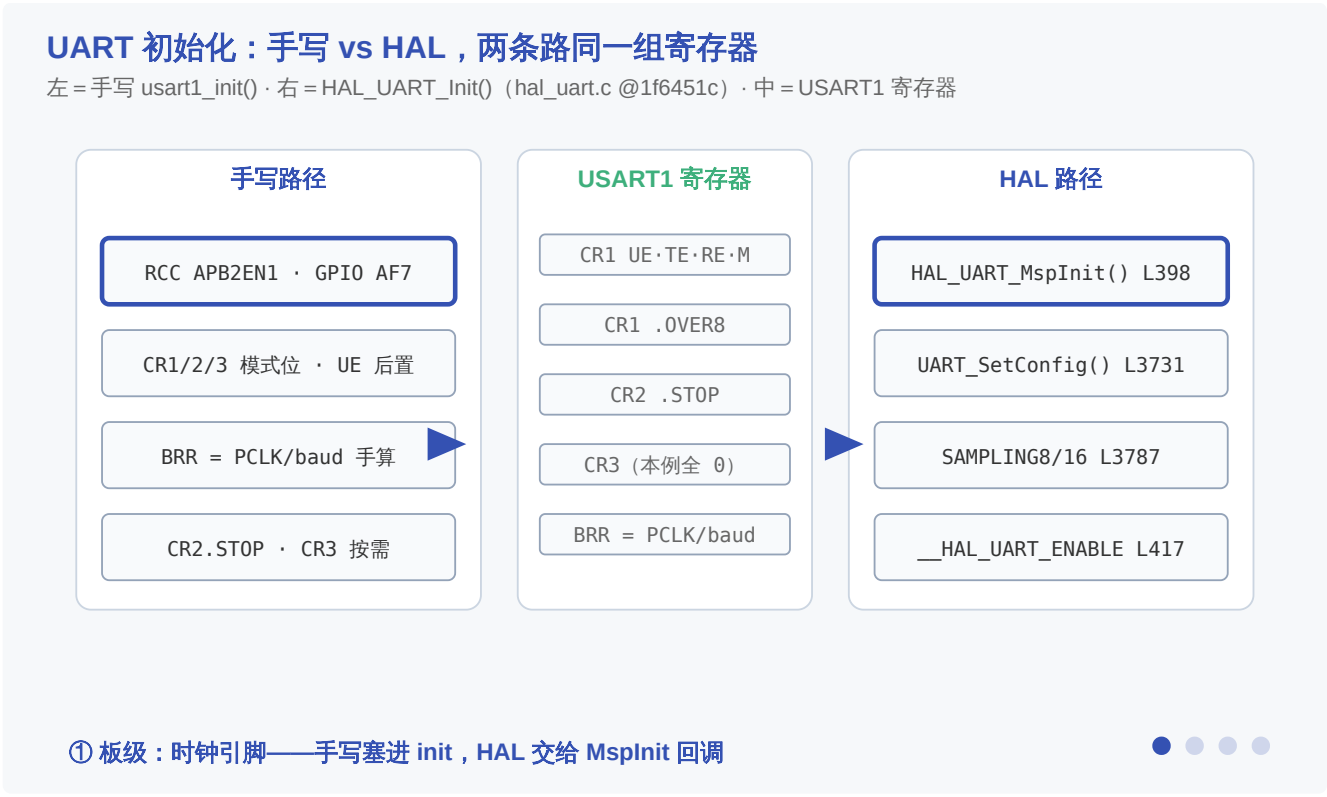
- `clocks_read()`：从 CFGR 反推 HCLK/PCLK2，BRR 由真实 PCLK2 算出来；
- `usart1_init()`：UE/TE/RE/IDLEIE 四个位，BRR = PCLK2/115200；
- `uart_write_blocking()`：塞数据看 TXE，收尾等 TC；
- `dma2_stream5_rx_init()`：环形、8 位、内存递增、DMA2 Stream5 Channel4（取值依据见工程 README 的说明，为什么不是 DMA1 见 S8 的总线分工）；
- `USART1_IRQHandler()`：错误位计数、IDLE 判帧边界、回显。

七、库解析对照：SPL/HAL 同名初始化

ST 官方仓库 `STMicroelectronics/stm32f4xx_hal_driver`（master @ `1f6451c`）的 `HAL_UART_Init()` / `HAL_UART_Transmit()` 做的是同一组寄存器落位，逐条对标本工程：

本工程	HAL 对应（hal_uart.c 行号 @ 1f6451c）	对照结论
`usart1_init()`：开时钟、配引脚、写 CR1/BRR	`HAL_UART_Init()`（L357）先调 `HAL_UART_MspInit()`（时钟与引脚归板级回调），关掉外设再 `UART_SetConfig()`（L3731）一次落 CR1/CR2/CR3——**OVER8 位同批写入**（L3752-3756）	你的初始化顺序 = MspInit + SetConfig 两段，HAL 只是把它们拆进两层回调
`baud = PCLK/BRR` 手算	`UART_SetConfig()` 尾部按过采样选 `UART_BRR_SAMPLING8/16` 宏算 BRR（L3787/L3791），最后 `__HAL_UART_ENABLE()`（L417）	同一个公式：OVER8 只是换分频宏，HAL 不多写一个寄存器
`uart_write_blocking()`：TXE 塞字节、收尾等 TC	`HAL_UART_Transmit()`：每字节前 `UART_WaitOnFlagUntilTimeout(TXE)`（L1172），结束再等一次 TC（L1191），全程 `HAL_GetTick` 计时超时	你总结的"TXE 只管塞、TC 才管完"就是 HAL 的实现；死等被超时兜底
`USART1_IRQHandler()`：IDLE 判帧、错误位计数	`HAL_UART_IRQHandler()`（L2355）按"**SR 标志 & 对应 CR 使能位**"的组合分发处理（如 L2599 TXE 要 TXEIE 开着才走发送分支）	结构同构：先取标志与使能的交集再分支，不是见标志就处理

SPL 的 `USART_Init()` 计算 BRR 用 `DIV_Mantissa/DIV_Fraction` 一对字段，公式同构；但 StdPeriph 未随 ST 官方 GitHub 分发，一手源码仍缺，位定义以 RM0090/CMSIS 为准。



HAL UART 初始化与手写落位的对账图

附录：工程完整源码

<<< ../../code/stm32/03-uart-dma/main.c

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **起始拉低、数据八位、低位先发、停止拉高；波特率跟着 PCLK 走；TXE 只管塞、TC 才管完；IDLE 读 SR 再读 DR。**

延伸：UART+DMA 环形缓冲是经典生产-消费模式，与 FreeRTOS 队列（F4）同构；DMA 原理见 S8。

实物实验

- E02 逻辑分析仪看串口：抓 0x41 与 0x55 各一帧，逐位对表；
- 对比实验：改 BRR 偏 5%，看什么时候开始丢帧，亲手感受误差的边界；
- DMA 环形缓冲：连发一长串（超过 64 字节），看 `g_half_events` / `g_full_events` 增长，理解"半满/全满中断"的节奏。

常见坑

- **TXE/TC 不分**：RS-485 切方向太早，最后一位被切掉；
- **IDLE 不清**：少了"读 SR 再读 DR"，中断风暴；
- **BRR 写死**：时钟树一改，串口乱码；
- **用 1MHz 配 115200**：BRR 只能取 9，实际 111111，误差 -3.549%，单这一项就吃掉 5.882% 总预算的六成；
- **ORE 溢出**：中断/DMA 处理不及时，新字节盖掉旧字节，数据悄悄丢。

短自测

1. 0x41 在串口线上的位序是什么样的？为什么 LSB 先发？

> **▼ 参考答案**
> 空闲高 → 起始位低 → 1,0,0,0,0,0,1,0（LSB 先发）→ 停止位高。LSB 先发是 UART 的协议规定，双方约定俗成；逻辑分析仪解码时按这个顺序读就对了。

2. PCLK2 = 16MHz，要 115200，BRR 是多少？误差多少？

> **▼ 参考答案**
> BRR = 16000000/115200 ≈ 138.9 → 取 139。实际波特率 = 16000000/139 ≈ 115108，误差 -0.08%，完全可用。

3. TXE 和 TC 的区别是什么？什么时候必须等 TC？

> **▼ 参考答案**
> TXE = 数据寄存器空了可以塞下一个；TC = 移位寄存器也空了、停止位已经发完。RS-485 切方向、断电、进低功耗之前必须等 TC，否则最后一位被切掉。

4. IDLE 标志怎么清？为什么？

> **▼ 参考答案**
> 读 SR 再读 DR。硬件规定了这个清除序列：光写 SR 没用，不把 DR 里的字节取走，IDLE 就一直挂着，中断会一直触发。

🎯 S7 串口自测

1. **0x41 在串口线上的位序是什么样的？为什么是这个顺序？**
   A. 空闲高 → 起始位低 → 高位先发的 8 位数据 → 停止位高
   B. 空闲高 → 起始位低 → 1,0,0,0,0,0,1,0（LSB 先发）→ 停止位高，UART 协议规定低位先发
   C. 起始位高 → 数据 8 位 → 停止位低，顺序由波特率决定
   D. 空闲低 → 起始位高 → LSB 先发 → 停止位低

2. **PCLK2=16MHz，要配 115200 波特率，BRR 应取多少？**
   A. 115
   B. 139
   C. 592
   D. 10417

3. **RS-485 半双工发完要切方向，应该等哪个标志？**
   A. TXE=1 就切——数据寄存器空了说明发完了
   B. RXNE=1 说明对方已收到，可以切回
   C. TC=1 才切——停止位跑完、移位寄存器也空了
   D. 等一个字节时间后直接切，不需要读标志

4. **IDLE 标志怎么清除？**
   A. 向 SR 寄存器对应位写 1
   B. 读 SR 之后再读 DR
   C. 写 CR1 的 IDLEIE 位关中断
   D. 硬件自动清，软件不用管

5. **为什么不能拿 1MHz 时钟的总线波特率发生器配 115200？**
   A. BRR 寄存器只有 12 位，装不下
   B. 分频后误差约 -3.549%，单项就吃掉 8N1 帧总预算的六成
   C. 1MHz 下 USART 硬件不支持 115200
   D. 误差可以靠过采样 8 位完全消除，可以放心用

先选完再翻：答案与逐项解析在附录 C 的本组；在线版可点击即时判分。

对照表：本章概念 → 仓库与上游落点

概念	落点
TXE/TC/RXNE/IDLE/ORE 位定义	ST CMSIS `stm32f407xx.h`（USART_SR 段）
`baud = PCLK/BRR` 实算	code/stm32/03-uart-dma/main.c `usart1_init()`
IDLE 判帧 + DMA 环形	同上 `USART1_IRQHandler()` 与 `dma2_stream5_rx_init()`
PA9/PA10 AF7	DS8626 Rev 9 Table 9
第二节那张误差表的八行	`BAUD_FROM_PCLK` / `PCLK_OF_BRR`（`main.c`）→ `probe.c` §1/§6 → `probe.sh` 第 2c 步与正文逐行 diff，改一个数就红
33 条位号、25 个寄存器地址	CMSIS `stm32f407xx.h` + `core_cm4.h`（本站 `.trellis/ref/cmsis/`）；`probe.c` §2/§11 现挖，`probe.sh` 第 2a/2b 步逐条 diff
机身里的 magic 乘法、`/1000`、BRR 落寄存器、DMA 那六个字、`-O0` 与 `-O2` 的形状差	`probe.sh` 第 3 步（`arm-none-eabi-gcc -O2/-O0 -c main.c` + `objdump -d`）
动画	uart-frame.svg、usart-txe-tc.svg、uart-hal-init-path.svg（§七对账）
HAL 同名初始化逐行对照	ST 官方仓库 stm32f4xx_hal_driver @ `1f6451c` `Src/stm32f4xx_hal_uart.c`（见 §七）；SPL 未随官方 GitHub 分发，一手源码仍缺

你做到了

- 串口从"黑盒外设"变成"节拍+寄存器"的组合；
- 亲手算过一次波特率，知道误差怎么来、多少算多；
- IDLE + DMA 环形的组合拳，是后面所有"不定长接收"的模板。

乱码的账现在能算了：波特率差上百分之三点几，采样点滑出最后一位的中点，一帧错一位满屏皆花。

✅ 下一站：S8 DMA：数据自己搬家，CPU 只等敲门声。

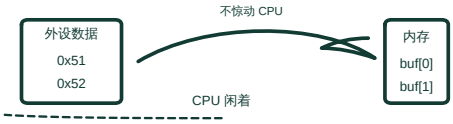
接收边界与验证更新

IDLE 是服务通知，不是应用帧边界。消费者从上次已消费位置读到新的 DMA 写位置；HT/TC 也推动消费。两种边界标志同时积压时丢弃不可信片段并计数，不能从相同位置猜测漏了几圈。ISR 只做有界搬运，主循环非阻塞回显。64 字节环形缓冲在 115200 8N1 下半缓冲约 2.78ms，服务期限必须留余量。宿主行为测试覆盖首次单字节、连续短块、跨圈与积压；这不是无限吞吐或上板实测证明。

通往单片机之路第 26 章

S8 DMA：让数据自己搬家

难度 ★★★☆☆ · 约 30 分钟



本章罗盘

问题

时钟引脚全配对但数据不来：流和通道怎么选？环形与双缓冲为什么保证不停机、却不保证不丢数据？

交付

USART1_RX 的 DMA2 逐位落位（PAR/M0AR/NDTR/CIRC，清标志写 HIFCR 不碰 HISR）加一张环形/正常/双缓冲覆盖风险对照表，配 GDB 读 NDTR 从 64 倒数回跳的观察记录。

边界

DMA 请求映射表（RM0090 Table 42/43）本轮未取得表正文：工程取值按常用映射给出、显式标注待上板实测确认；FIFO/突发模式不展开（只走直接模式）；CCM 够不着 DMA 是 F407 布局事实，不外推。

依据锚点 `a1` · `f3` · `f4` · `mcu-code` · `mcu-anim`

> 🎯 串口收 1KB 数据，中断方式要打断 CPU 一千次；DMA 方式：CPU 说一句"把串口数据搬到这个数组，搬完叫我"，然后该干嘛干嘛——搬完一次中断。这就是 DMA 存在的全部理由。

本章精髓

1. **DMA 是"第二总线主人"**：它和 CPU 共享总线矩阵（回 S1），谁用谁让路，所以 DMA 传输时 CPU 变慢不是错觉，是仲裁。
2. **三个地址一个长度**：外设地址（固定，PAR）、内存地址（自增，M0AR）、剩余个数（NDTR 倒数）：外设↔内存、内存↔内存三种组合，方向寄存器 `DMA_SxCR.DIR` **占两位**（bit7:6），不是一位。
3. **环形/双缓冲不自动防丢**：环形模式 NDTR 数到 0 自动回装、接着写开头。**CPU 处理得比数据来得慢，旧数据就会被新数据盖上**。双缓冲（乒乓）同理：它只保证"DMA 写和 CPU 读不在同一块"，不保证"CPU 一定来得及"。处理期限是硬约束，不是风格建议。

- **能理解**：为什么方向位是两位；为什么 NDTR 是"个数"不是"字节数"；为什么环形缓冲会静默覆盖。
- **能用**：给 USART1 配上 DMA2 环形接收，用 IDLE 判帧边界，亲手看到 `NDTR` 在倒数。

先修

- S7 USART、S1 总线、S4 中断。

先跑起来（10 分钟 quick win）

`code/stm32/03-uart-dma`：烧录后串口助手连发一长串，GDB 里读 `g_rx_bytes` 和 `g_frames` 在动，数据真的在自己搬家。再加一步：主循环打印 `DMA2_S5_NDTR` 的倒数，看它从 64 一路减到 0 又跳回 64。

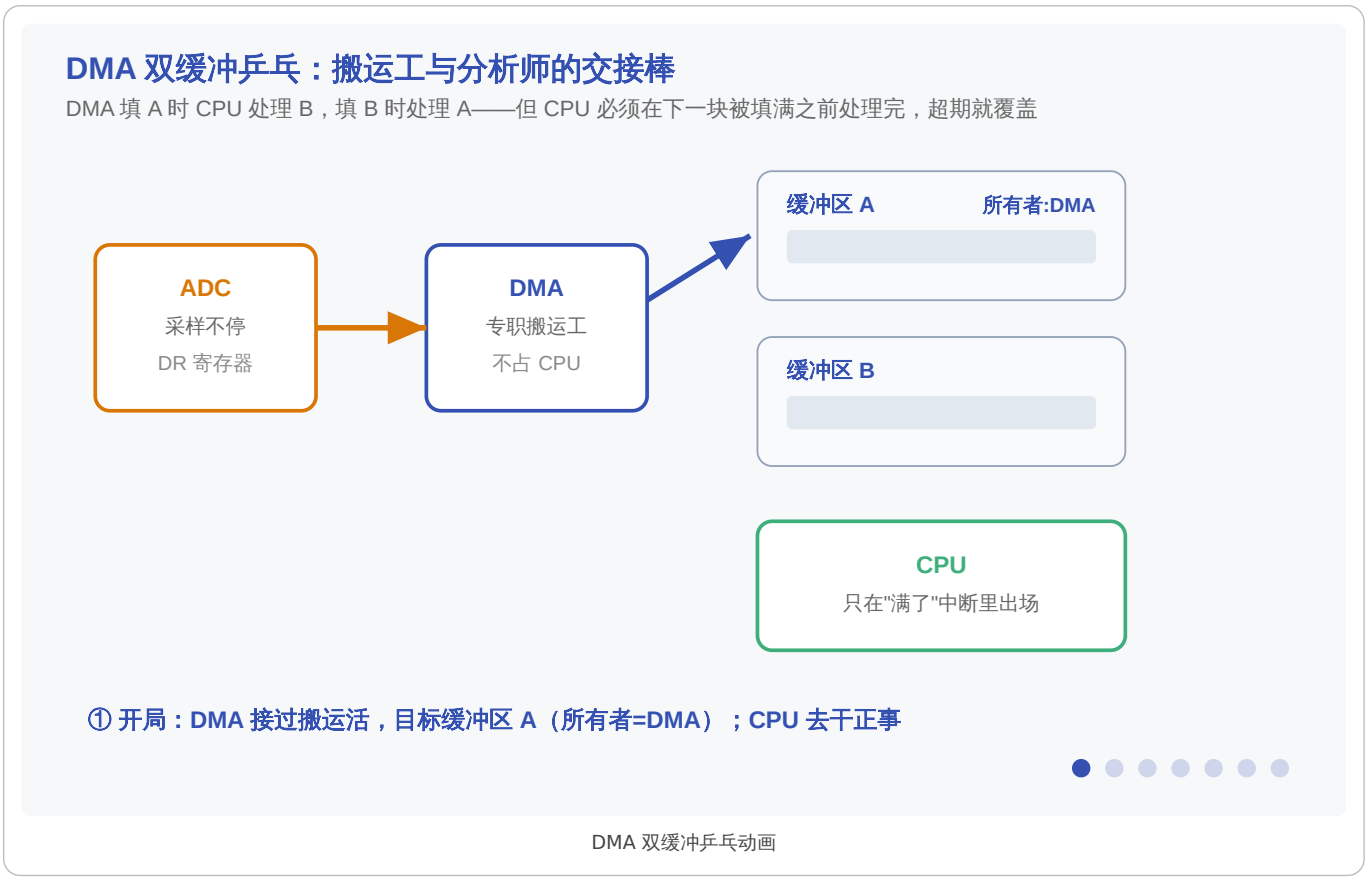
连上调试器之前，先押个现场：DMA 自己搬家的证据，是那个剩余长度计数器一动不动、减到零就停、还是减到零自己跳回去接着来？盯住它几秒，答案比讲解直观。

动画：环形缓冲的水位

环形缓冲像一圈跑道：DMA 是生产者（往跑道里放数据），CPU 是消费者（从跑道里取数据）。NDTR 是"还剩多少空位"。盯住两件事：生产者追上消费者会发生什么（覆盖），以及 NDTR 数到 0 时的"回装"动作。

DMA 环形缓冲动画

再对照老的乒乓图（已修订）：双缓冲把"写"和"读"分到两块地上，**但 CPU 必须在 DMA 把下一块也填满之前处理完**，超期就覆盖，这不是理论风险，是真实坑。



DMA 双缓冲乒乓动画

板卡与版本前提

- DMA 有两条控制器：**DMA1 管 APB1 上的外设请求，DMA2 管 APB2 上的外设请求**。USART1 在 APB2 上，所以它的 DMA 请求归 DMA2（这是常见选错点）。

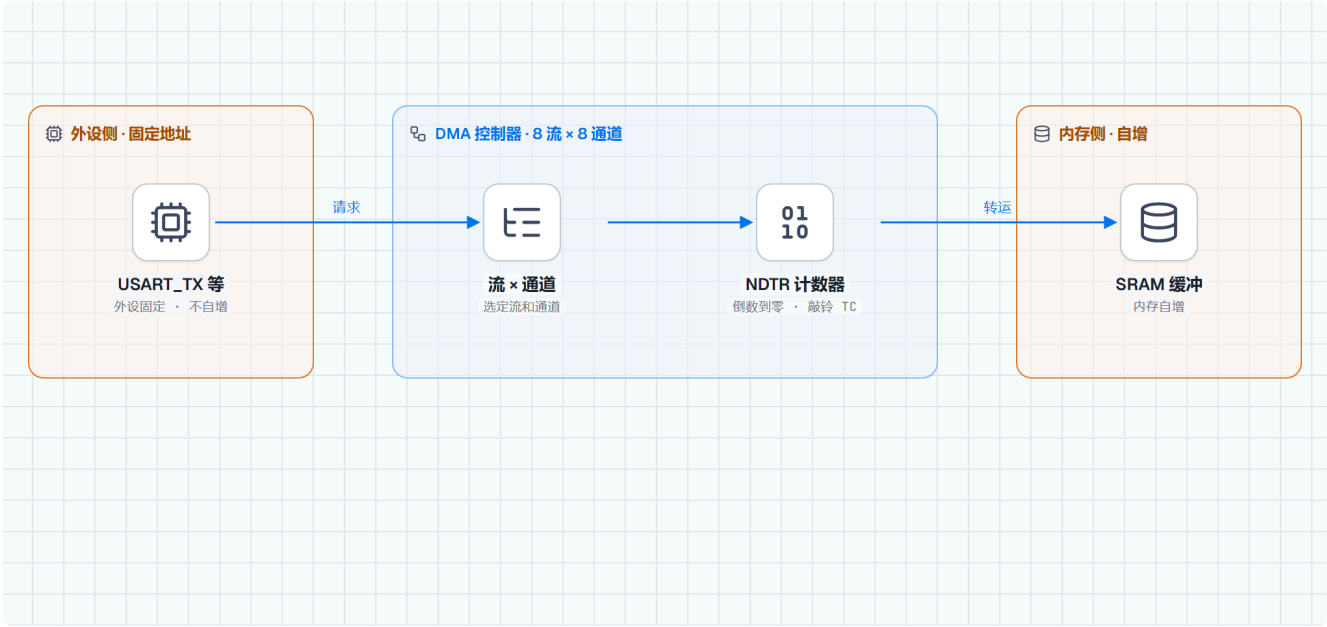
- 请求映射表：RM0090 §10.3.3，Table 42（DMA1）/ Table 43（DMA2）。本轮未取得表正文（见研究记录），工程里的取值以常用映射给出并显式标注"待上板实测确认"。
- F4 的 DMA 有 FIFO（`FCR`），本工程走直接模式（FIFO 关），先讲清主链路再谈 FIFO。

小节结构

小节	内容	四件套
一、流与通道	二级选择的含义；DMA1/DMA2 分工；映射表去哪查	配置
二、三地址一长度	PAR/M0AR/NDTR；方向位是两位；宽度与对齐	配置
三、环形 vs 正常	CIRC 位；NDTR 自动回装；环形不防覆盖	代码分析
四、半满/全满中断	HTIF/TCIF 的节奏；与 IDLE 判帧配合	代码分析
五、双缓冲（乒乓）	DBM/CT 位；处理期限是硬约束	代码分析
六、总线仲裁	DMA 与 CPU 抢总线；为什么"变慢"是正常	库解析
七、代码分析	`03-uart-dma/main.c` 的 `dma2_stream5_rx_init()` 逐位落位	代码分析
八、库解析对照	HAL Init/Start 逐行对照（hal_dma.c @ 1f6451c）；SPL 一手源码仍缺	库解析

一、流与通道：二级选择是什么

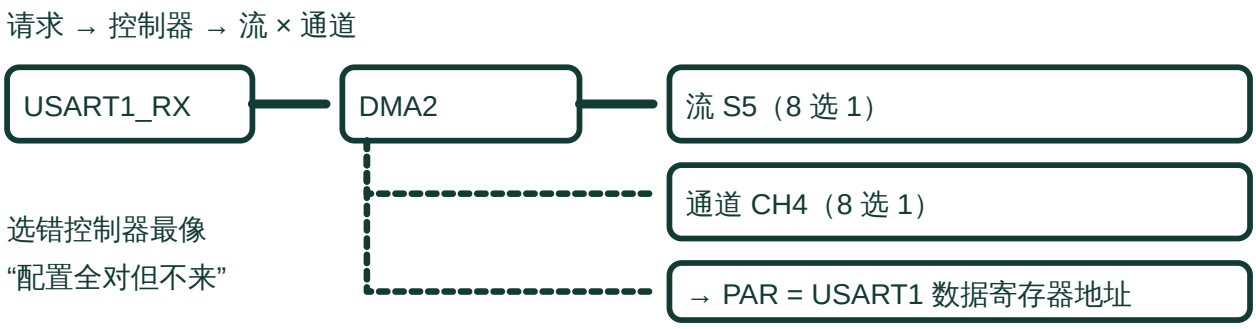
转运全程先收进一张图：请求从外设出发，流×通道选定路径，NDTR 倒数完敲铃收工。



DMA 转运通路：外设固定、流通道选择、NDTR 计数、内存自增

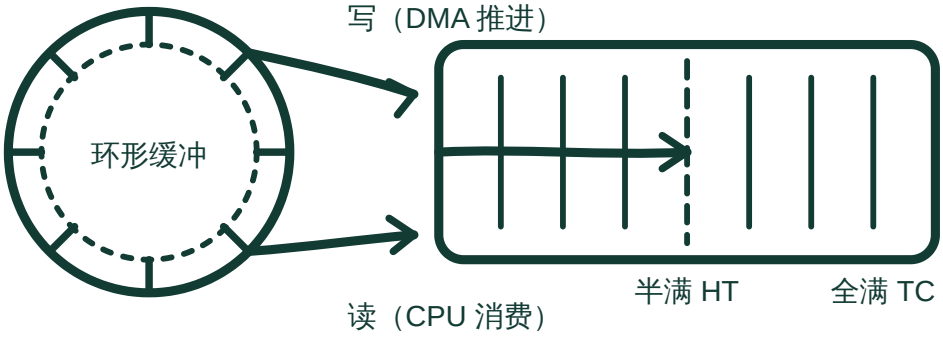
F4 的每个 DMA 控制器有 8 条**流（Stream）**，每条流可以接 8 个**通道（Channel）**里的一个。所以一个 DMA 请求要选对两样东西：流 × 通道。

- **DMA1**：APB1 上的外设（USART2/3、I2C、SPI2/3、TIM2~7 等）。



- **DMA2**：APB2 上的外设（USART1/6、SPI1、ADC3、TIM1/8 等）以及 SDIO、DCMI。

选错控制器是最常见的"配置都对但数据不来"：时钟开了、引脚对了、流也配了，就是控制器选成了 DMA1。USART1_RX 的常用取值是 **DMA2 Stream5 Channel4**（也可走 Stream2 Channel4）。以 RM0090 Table 43 为准，别背。



DMA 环形缓冲：写指针由 DMA 推进，读指针由 CPU 消费；半满 HT 与全满 TC 各触发一次中断

二、三地址一长度：PAR、M0AR、NDTR

一次 DMA 传输要告诉硬件三件事：

寄存器	干什么
`PAR`	外设地址（如 USART1 的 DR），通常**固定**
`M0AR`	内存地址，通常**自增**（`MINC=1`）
`NDTR`	还要搬几个"数据项"

方向寄存器 `DMA_SxCR.DIR` 占两位（bit7:6），三种组合：00=外设到内存、01=内存到外设、10=内存到内存。这是个"看起来是一位其实是两位"的典型陷阱（原文档写"DIR 是一位"就是错的，CMSIS 头文件里 `DMA_SxCR_DIR_Pos=6`、掩码 `0x3<<6`）。

NDTR 是"个数"不是"字节数"：数据宽度 16 位时，NDTR=64 表示 64 个半字 = 128 字节。宽度和长度对不上，差两倍的经典错。

三、环形 vs 正常：CIRC 位与自动回装

- **正常模式**：NDTR 数到 0，传输结束，要再启动得重新配置。
- **环形模式**（`CIRC=1`）：NDTR 数到 0 自动回装成初始值，接着写开头，**永远不停**。

环形模式是流式数据的灵魂（串口接收、ADC 连续采样）。但它有个必须直面的性质：**CPU 处理得比数据来得慢，旧数据就会被新数据盖上**。环形缓冲不会等你，它只会绕圈。

所以"消费速度 ≥ 生产速度"不是风格建议，是硬约束。判断方法很简单：跟踪"已读位置"和"DMA 写位置"的差，如果差值在涨，你就在欠债。

四、半满/全满中断：HTIF 与 TCIF 的节奏

环形模式下，DMA 会在两个固定点敲铃：

- **HTIF**（半传输）：搬到一半时置位，意思是"前半圈满了，可以开始读了"。
- **TCIF**（全传输）：搬完一圈时置位，意思是"后半圈也满了，再不读就回绕了"。

这两个中断把"处理期限"变成了具体数字：从 HTIF 到 TCIF 之间的时间，就是你处理前半圈的全部预算。处理不完，TCIF 之后数据就被覆盖了。

清标志要写 `HIFCR` / `LIFCR` 不是 `HISR` / `LISR`：状态寄存器是只读的，往它写 1 不会清任何东西，这是和中断标志打交道时最容易栽的坑（EXTI 的清 pending 是写 1，DMA 是写另一个寄存器，别混）。

五、双缓冲（乒乓）：DBM 与 CT，以及"处理期限"

双缓冲模式（`DBM=1`）给 DMA 两块内存（M0AR/M1AR）：DMA 写 A 时 CPU 读 B，A 满了自动切 B（`CT` 位告诉你当前在写哪块），同时敲一次 TCIF。

它解决的问题是"写和读不打架"，**没解决的问题是"CPU 来不及"**：CPU 必须在 DMA 把下一块也填满之前处理完当前这块。超期了？新数据盖上旧数据，而且是整块整块地丢，比环形模式丢得还整齐。

所以乒乓图里必须画出"CPU 的完成期限"那条线。本章的修订版动画（`dma-pingpong.svg`）补上了这个此前被遗漏的元素。

六、总线仲裁：DMA 和 CPU 抢总线

DMA 和 CPU 共享 AHB 总线矩阵（回 S1）。传输密集时 CPU 取指/访存会等 DMA，这不是 bug，是设计如此。理解这一点的实际意义：性能敏感代码不要把大数组放在 DMA 正在写的内存旁边，也别在 DMA 高负载时测代码执行时间然后以为代码变慢了。

另一个边界：**DMA 够不着 CCM**（0x10000000 的 64KB 是 CPU 直连）。把接收数组放 CCM，DMA 写不进去，数据静悄悄丢失。链接脚本默认不放，但如果你手动加了 `__attribute__((section(".ccmram")))`，就要自己扛这个后果。

七、代码分析：`03-uart-dma/main.c`

`dma2_stream5_rx_init()` 逐位落位：

- 先开 DMA2 时钟（`RCC_AHB1ENR.DMA2EN`），不开时钟，流配置写了也白写；
- 先关流再配（`EN=0` 时才能改 CHSEL/DIR/CIRC）；
- `PAR = USART1 DR` 、`M0AR = 我们的环形数组` 、`NDTR = 64`；
- `CIRC=1` 、`MINC=1` 、`PS=MSIZE=字节` 、`CHSEL=4`；
- 最后 `USART1_CR3.DMAR=1` 告诉 USART：RXNE 由 DMA 来取。

`USART1_IRQHandler()` 用 IDLE 判帧边界；`DMA2_Stream5_IRQHandler()` 用 HTIF/TCIF 跟踪水位。两者配合，正好把"帧边界"和"水位"两个问题分开处理。

八、库解析对照：SPL/HAL 同名初始化

ST 官方仓库 `STMicroelectronics/stm32f4xx_hal_driver`（master @ `1f6451c`）的 `HAL_DMA_Init()` 把流属性一次落位、`HAL_DMA_Start()` 把本次传输参数落位。逐条对标本工程：

本工程 `dma2_stream5_rx_init()`	HAL 对应（hal_dma.c 行号 @ 1f6451c）	对照结论
配置前先 `CR=0`、等 EN 落下	`HAL_DMA_Init()`（L170）先 `__HAL_DMA_DISABLE()` 再轮询 `CR.EN`，等不到就按 `HAL_GetTick` 超时返回 `HAL_TIMEOUT`（L208-227）	同一硬约束：**EN=1 时改 CHSEL/CIRC 无效**；HAL 多给超时兜底，不会死锁在 while
CHSEL/CIRC/MINC/PSIZE 逐位拼 CR	同函数：把 CHSEL 到 DBM 一整段位一次清掉（L231-237），`Init` 各字段或成一笔写回 CR（L250）；FIFO 的 DMDIS/FTH 另写 FCR（L285）	你手工拼的那串掩码 = HAL 的 tmp 读-改-写；直接模式就是 FCR 不置 DMDIS
自己写 NDTR/PAR/M0AR	留到 `HAL_DMA_Start()` 里的 `DMA_SetConfig()`：NDTR（L1157）、PAR/M0AR 按 DIR 定向（L1163-1175）	HAL 把"流属性"与"本次传输参数"分两步；环形重满时 NDTR 由硬件自动重装
`DMA2_Stream5_IRQHandler()` 管 HT/TC、写 IFCR 清标志	`HAL_DMA_IRQHandler()` 按 HISR 调半传输/完成/错误回调，清标志同样回写 IFCR（如 L708 `IFCR = (HTIF|TCIF) << StreamIndex`）	同构：清标志写 HIFCR/LIFCR、绝不碰只读的 HISR/LISR——这条你已经在坑里验证过

SPL 的 `DMA_Init()` 做的是同一组落位，但 StdPeriph 未随 ST 官方 GitHub 分发，一手源码仍缺；位定义以 RM0090 为准。

EN 可改窗口与 HAL 两段式初始化

附录：工程完整源码

<<< ../../code/stm32/03-uart-dma/main.c

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **选定流和通道，外设固定内存自增，NDTR 倒数到零敲个铃；循环搬流水，双缓冲不停机，但都不替你还债；清标志写 HIFCR/LIFCR，别碰 HISR/LISR。**

延伸：DMA 环形缓冲与 FreeRTOS 队列（F4）是同构的生产-消费模式；DMA 总线仲裁与地址空间布局（S1）相关。

实物实验

- UART DMA 环形接收 + GDB 观察 `g_rx_bytes` / `g_frames` / `g_half_events` / `g_full_events` 的增长节奏；
- 对比实验：同一 1KB 数据，中断版 vs DMA 版的 CPU 占用（用 S5 时基粗测）；
- 翻车实验（选做）：把 `g_rx_buf` 挪到 CCM（链接脚本加 `.ccmram` 段或 `__attribute__`），看 DMA 静默丢数据；
- 翻车实验（选做）：把 IDLE 的处理从"读 SR 再读 DR"改成"只读 SR"，看中断风暴。

常见坑

- **控制器选错**：USART1 的 DMA 请求在 DMA2，不在 DMA1，配置都对但数据不来，先查这个；
- **忘开 DMA 时钟**：`RCC_AHB1ENR.DMA2EN` 没开，流配置写了也白写；
- **清标志写错寄存器**：HISR/LISR 是只读状态，清标志要写 HIFCR/LIFCR；
- **流/通道选错**：映射表查错一格，数据永远不来，以 RM0090 Table 42/43 为准，别背；
- **缓冲区放 CCM**：DMA 写不进 CCM，数据静悄悄丢失；
- **NDTR 与数据宽度不匹配**：宽度 16 位时 NDTR 是"个数"不是"字节数"；
- **以为环形/双缓冲不会丢数据**：CPU 处理慢了就覆盖，处理期限是硬约束。

短自测

1. `DMA_SxCR.DIR` 占几位？为什么"外设到内存"写成 0 时不要以为它只是一位？

▼ **参考答案**
占两位（bit7:6），CMSIS 头文件 `DMA_SxCR_DIR_Pos=6`、掩码 `0x3<<6`。三种组合 00/01/10。把它当一位来理解，遇到"内存到外设"（01）时就会以为自己看错了寄存器。

2. USART1_RX 的 DMA 请求在哪个控制器上？为什么容易选错？

▼ **参考答案**
DMA2（Table 43），Stream5 Channel4（或 Stream2 Channel4）。容易选错是因为直觉上"数字小的外设配数字小的 DMA"，但 F4 的分工是按 APB 总线划的：APB1 外设→DMA1，APB2 外设→DMA2，USART1 在 APB2 上。

3. 环形模式下 NDTR 数到 0 会发生什么？为什么这不等于"数据不会丢"？

▼ **参考答案**
NDTR 自动回装成初始值，DMA 接着从缓冲区开头写，传输永不停。但如果 CPU 还没来得及读走旧数据，新数据就会把它盖上。环形模式保证"不停"，不保证"不丢"。

4. 清 DMA 中断标志要写哪个寄存器？写 HISR 会发生什么？

▼ **参考答案**
写 HIFCR（或 LIFCR，取决于流在 0~3 还是 4~7）。HISR/LISR 是只读状态寄存器，往它写 1 什么都不会清，中断会无限进。

🎯 S8 DMA 自测

1. **USART1 的 RX 数据来了半天不来，配置看起来全对——最先该查什么？**
   - A. 波特率误差
   - B. DMA 控制器选错：USART1 在 APB2，请求归 DMA2，不在 DMA1
   - C. NDTR 初值填成了字节数
   - D. 中断优先级不够高

2. **环形模式下 NDTR 数到 0 会发生什么？**
   - A. DMA 停止，等 CPU 重新启动
   - B. 触发传输完成中断后自动退出循环模式
   - C. NDTR 自动回装成初始值，从头继续写——保证'不停'，不保证'不丢'
   - D. 数据溢出到缓冲区旁边的内存

3. **清 DMA 中断标志应该写哪个寄存器？**
   - A. HISR 或 LISR 对应位写 1
   - B. HIFCR 或 LIFCR 对应位写 1（按流所在半区选）
   - C. CR 的 EN 位先关再开
   - D. 读一次 NDTR 硬件自动清

4. **在 EN=1（传输进行中）时修改 CR 的 CHSEL/CIRC 位，结果是什么？**
   - A. 立即生效，DMA 无缝切换
   - B. 硬件自动先停 EN 再改
   - C. 改不进去——这些位只有 EN=0 时可写，必须先关流并等 EN 落 0
   - D. 触发一次传输错误中断后忽略

5. **把 DMA 接收缓冲区放到 CCM（核心耦合内存）里会发生什么？**
   - A. 速度更快，官方推荐
   - B. 编译报错，链接器拦住
   - C. DMA 写不进 CCM，数据静悄悄丢失
   - D. 自动回退到 SRAM，无副作用

先选完再翻：答案与逐项解析在附录 C 的本组；在线版可点击即时判分。

对照表：本章概念 → 仓库与上游落点

概念	落点
DIR/CHSEL/CIRC/MINC 位定义	ST CMSIS `stm32f407xx.h`（DMA_SxCR 段）
USART1_RX → DMA2 Stream5 Ch4	code/stm32/03-uart-dma/main.c（取值依据与自检步骤见工程 README）
HIFCR/LIFCR 清标志	同上 `DMA2_Stream5_IRQHandler()`
环形缓冲水位实验	同上 `g_rx_buf` + `g_half_events` / `g_full_events`
动画	dma-circular-buffer.svg、dma-pingpong.svg（已修订）、dma-hal-en-window.svg（§八对账）
HAL 同名初始化逐行对照	ST 官方仓库 stm32f4xx_hal_driver @ `1f6451c` `Src/stm32f4xx_hal_dma.c`（见 §八）；SPL 未随官方 GitHub 分发，一手源码仍缺

你做到了

- "数据自己搬家"从口号变成可调可测的机制；
- UART+DMA 组合成型，ADC 扫描（S9）、DAC 波形（S10）直接复用这套范式；
- "处理期限是硬约束"这件事，从环形缓冲和乒乓图里都亲眼看到了。

现场你看到了：减到零、翻满格、接着搬，周而复始。你盯着的每一秒，CPU 都在干别的。

✅ 下一站：S9 ADC：把旋钮/光线变成数字，采样定理眼见为实。

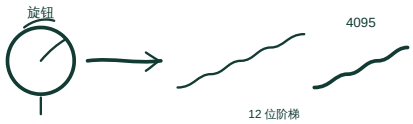
接收边界与验证更新

IDLE 是服务通知，不是应用帧边界。消费者从上次已消费位置读到新的 DMA 写位置；HT/TC 也推动消费。两种边界标志同时积压时丢弃不可信片段并计数，不能从相同位置猜测漏了几圈。ISR 只做有界搬运，主循环非阻塞回显。64 字节环形缓冲在 115200 8N1 下半缓冲约 2.78ms，服务期限必须留余量。宿主行为测试覆盖首次单字节、连续短块、跨圈与积压；这不是无限吞吐或上板实测证明。

通往单片机之路第 27 章

S9 ADC：把模拟世界变成数字

难度 ★★☆☆☆ · 约 35 分钟



本章罗盘

问题

12 次比较怎么锁定 4096 分之一？100kΩ 高阻源只给 3 拍采样，读数为什么成了虚焦照片？

交付

probe 跑出的 12 行 SAR 试探表（2.00V→0x9B2、误差 0.42LSB）加一张按源阻抗选 SMP 档的 τ=9.01·RC 换算表，含「只给 3 拍残差 2886 LSB」那一列。

边界

SAR 与采样时间是理想模型加 datasheet 典型参数（R_ADC=2kΩ/C_ADC=4pF 为假设值）的宿主推算，3 vs 480 拍的板上对比未做；VREFINT_CAL 校准地址本地无法核实、明标待回填不编造；注入组与双 ADC 交织不覆盖。

依据锚点 `a1` · `a2` · `f3` · `d6` · `mcu-code` · `mcu-anim`

> 🎯 12 位 ADC 用 12 次二分比较，把连续电压逼近成 4096 分之一——本章把每一刀都算给你看。

本章精髓

1. SAR 逐次逼近像"猜数字"：从最高位开始试，片内 DAC 生成中间值，比较器只答"大了/小了"，12 轮锁定 4096 分之一。采样保持电路先把电压"拍照"定格，转换才能从容进行。
2. 采样时间是精度的隐藏变量：保持电容经 (R_s+R_ADC) 充电，沉到半 LSB 要 t > 9.01·τ。高阻源配短采样时间，读到的是"没充满"的假值，残差可以高达上千个 LSB。
3. 规则组 + 扫描 + DMA 是采集范式：多通道按 SQR 序列自动转换、DMA 自动入数组，CPU 只拿结果。

- **能理解**：probe 打印的 12 行试探表里，每一行的 trial 码都是"保留位 | 当前试探位"，候选每轮对半砍，所以比较次数 == 位数。
- **能用**：给一个源阻抗 R_s，用 τ=(R_s+R_ADC)·C_ADC 算最少采样周期，再到 8 档 {3,15,28,56,84,112,144,480} 里选够用的那一档。

先修

- S8 DMA（扫描+DMA 范式的搬运工）、S3 GPIO 模拟模式（MODER=11）、S2 时钟树（f_ADC 来自 PCLK2 分频）。

先跑起来（10 分钟 quick win）

```
cd code/stm32/09-adc
sh probe.sh        # 宿主 gcc 跑 5 条断言：SAR×1 + 三个输入电压×3 + 采样时间×1
```

输出里找这两行：`== 断言通过 5/5 ==`、`锁定 code=2482 (0x9B2) → 1.9997 V`。不需要开发板；交叉工具链在 PATH 时还会顺手编出 Cortex-M4 目标文件。

跑探针之前，先押个精度：十二刀砍完，答案离真值能近到什么份上，整整差一格、半格以内、还是千分之一格？跑出那行锁定码值和电压，输赢立判。

动画：逐次逼近——12 刀砍到 4096 分之一

摆好一台理想 12 位 SAR：VREF=3.3 V、Vin=2.00 V。采样保持先合闸给 `C_H` 充电，把这一瞬的电压"拍照"定格；随后 SAR 寄存器从 bit11 起逐位置 1，片内 DAC 把当前码值换成电压反喂比较器，比较器一辈子只回答一个字：大了，还是小了。第一刀 2048→1.6500 V，比 Vin 小，留；第二刀 3072→2.4750 V，大了，清；第三刀 2560→2.0625 V，还是大了，清。12 轮后停在 `1001 1011 0010` = 2482 = 0x9B2，对应 1.9997 V，与真值差 0.342 mV，不到半个 LSB。候选每轮对半砍，所以位数就是"猜数字"的提问次数：12 问定 4096 档。

最后一个阶段故意把精度打下来：`C_H` 没充满就转，红色那条爬到半路就停了。此后 12 轮比较再准，逼近的也是一张虚焦的照片。**精度的天花板不在比较器，在采样窗口**，这正是本章「四、采样时间 vs 源阻抗」的物理原因。

动画里每个码值和电压都不是估的：`sh code/stm32/09-adc/probe.sh sar` 在宿主机现场跑出那 12 行试探表，换 `-DVIN=` 就能复算任意输入（1.65 V 正好落 0x800，误差 0.00 LSB）。

SAR 逐次逼近动画

小节结构

小节	内容	四件套
一、ADC 版图	三个 ADC 基址 + 寄存器偏移 + 关键位	配置
二、SAR 原理	12 轮二分表逐行拆解（probe 实测输出）	代码分析
三、换算	LSB=805.7 μV；量化误差落在 [0,1) LSB	库解析
四、采样时间	RC 沉降推导 + 4 档源阻抗实测选档表	引脚
五、工作模式	单次/连续/扫描/DMA/外部触发/EOCS	配置
六、参考电压	VREF 决定满量程；VREFINT 校准地址待回填	配置

一、ADC 版图：三个 ADC、十五个寄存器

STM32F407 有 ADC1/2/3 三个独立 ADC，挂在 APB2 上，基址一个挨一个（`stm32f407xx.h:928` 定义 `APB2PERIPH_BASE = PERIPH_BASE + 0x10000`，966-968 行给出三个基址）：

ADC	基址	出处
ADC1	0x40012000	`stm32f407xx.h:966`（APB2 + 0x2000）
ADC2	0x40012100	`stm32f407xx.h:967`
ADC3	0x40012200	`stm32f407xx.h:968`

每个 ADC 的寄存器排布由 `ADC_TypeDef` 决定（`stm32f407xx.h:180-199`），本章用到的：

寄存器	偏移	干什么
SR	0x00	状态：EOC(bit1, 1181-1183 行)、STRT(bit4, 1190-1192)、OVR(bit5, 1193-1195)
CR1	0x04	SCAN(bit8, 1215-1217)、EOCIE(bit5, 1206-1208)、RES[1:0](bit25:24, 1242-1246)
CR2	0x08	ADON(bit0, 1252)、CONT(bit1, 1255)、DMA(bit8, 1258)、DDS(bit9, 1261)、EOCS(bit10, 1264)、ALIGN(bit11, 1267)、EXTSEL[3:0](bit27:24, 1285)、EXTEN[1:0](bit29:28, 1292)、SWSTART(bit30, 1297)
SMPR1	0x0C	通道 10-18 采样时间，每通道 3 位（SMP10 在 bit2:0，1302-1307 行）
SMPR2	0x10	通道 0-9 采样时间
SQR1/SQR2/SQR3	0x2C/0x30/0x34	规则序列：总长 L[3:0] 与第 1-16 名的通道号
DR	0x4C	规则数据寄存器——一次转换的成果就躺在这

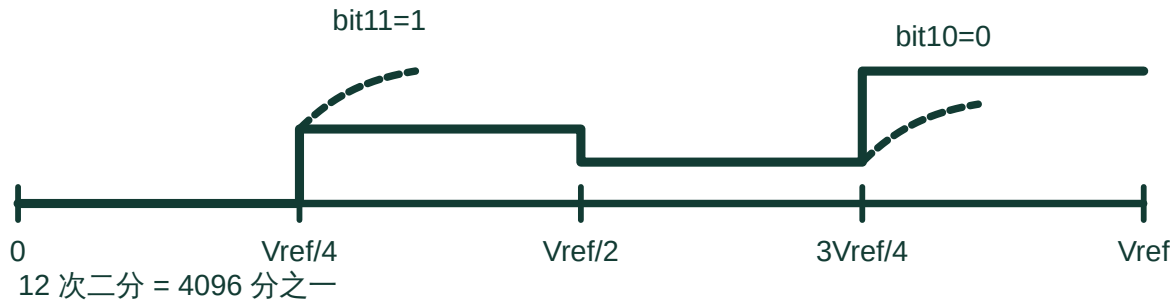
12 位是 RES=00（`st-adc.h:306`：`ADC_RESOLUTION_12B = 0`，复位默认就是 12 位）；PA0 对应 ADC1 通道 0（`ADC_CHANNEL_0 = 0`，`st-adc.h:363`）。

二、SAR 逐次逼近：12 次二分，一刀一刀看

`sar_probe.c` 在宿主机把理想 SAR 的每一刀打印出来（VREF=3.30 V、Vin=2.00 V，`sh probe.sh sar` 真实输出）：

```
轮次  试探位   trial   V_dac      比较        保留后 code
 1    bit11   2048   1.6500 V   Vin ≥ V_dac    2048
 2    bit10   3072   2.4750 V   Vin < V_dac    2048
 3    bit9    2560   2.0625 V   Vin < V_dac    2048
 4    bit8    2304   1.8562 V   Vin ≥ V_dac    2304
...
11    bit1    2482   1.9997 V   Vin ≥ V_dac    2482
12    bit0    2483   2.0005 V   Vin < V_dac    2482
```

机制就三行 C（`sar_probe.c:40-48`）：`trial = code | (1u << b)` 试探当前位，DAC 电压 `vdac = trial * lsb`，`vdac <= Vin` 就保留否则清掉。候选每轮对半砍：4096 → 2048 → … → 1，所以**比较次数恒等于位数**（probe 原话：`12 轮 = log2 4096`）。硬件上这三行对应：SAR 寄存器置位 → 片内电容 DAC 输出 → 比较器翻转（一轮一个 ADC 时钟周期量级），这就是 SAR 型 ADC"位数换时间"的底气。



SAR 逐次逼近：12 次二分把输入电压逼近到 1/4096；每一位先猜 1，再按比较结果保留或撤销

三、换算：一个 LSB 值 805.7 μV

probe 第一行就是换算：`LSB = 3.30/4096 = 0.000806 V（0.806 mV = 805.7 μV）`。12 位、3.3 V 参考下，你能分辨的最小电压差约 0.8 mV：旋钮拧出 1 mV 的变化，码值才肯动一格。

锁定结果与量化误差（probe 实测三个输入）：

Vin	锁定码	回算电压	量化误差
1.65 V	0x800 (2048)	1.6500 V	0.00 LSB（半量程正好落码心上）
2.00 V	0x9B2 (2482)	1.9997 V	0.42 LSB（0.3418 mV）
3.29 V	0xFF3 (4083)	3.2895 V	0.59 LSB（离满量程 0xFFF 差 12 码）

逐次逼近按"≤ 就保留"向下取整，量化误差落在 **[0, 1) LSB** 区间。每个二进制跑完都自检三条：终码 == floor(Vin/LSB)、误差 < 1 LSB、轮数 == 位数，任一不过 `set -e` 拦下整轮（`sar_probe.c:57-61`）。

四、采样时间 vs 源阻抗：没充满的照片是虚的

转换之前，保持电容 C_ADC 要先经 (R_s + R_ADC) 充到输入电压。一阶 RC 推导（`ts_probe.c:5-12` 注释里就是全部推导）：

- 沉降条件：e^(-t/τ) < 2^-13（12 位半 LSB）→ **t > 9.01·τ**，τ = (R_s + R_ADC)·C_ADC；
- 换算周期数：cycles > 9.01·τ·f_ADC。

`sh probe.sh ts` 实测输出（假设 R_ADC=2 kΩ、C_ADC=4 pF、f_ADC=21 MHz = PCLK2/4；R_ADC/C_ADC 以 datasheet 为准）：

源阻抗 R_s	τ	最少周期	选中档	该档残差	若只给 3 拍
1 kΩ	12 ns	2.3	3 拍	0.0277 LSB	0.0 LSB
10 kΩ	48 ns	9.1	15 拍	0.0014 LSB	208.8 LSB
50 kΩ	208 ns	39.4	56 拍	0.0111 LSB	2061.0 LSB
100 kΩ	408 ns	77.2	84 拍	0.2262 LSB	2886.0 LSB

读法：1 kΩ 低阻源 3 拍就够；**100 kΩ 高阻源若图省事一律配 3 拍，残差 2886 LSB——读数基本是垃圾**，这不是噪声，是系统性偏低。8 档可选采样时间 {3,15,28,56,84,112,144,480} 来自 ST 官方头文件（`st-adc.h:392-399`：`ADC_SAMPLETIME_3CYCLES` 到 `ADC_SAMPLETIME_480CYCLES`），每通道 3 位 SMP 字段。probe 自检"选中档 ≥ 最少周期 且残差 < 0.5 LSB"4/4 通过。

五、工作模式：单次、连续、扫描、DMA、外部触发

ADC 规则组 SQR 轮转 vs 注入组插队

五个开关在 CR1/CR2 里各就各位（行号见第一节的表）：

- **单次 vs 连续**：CONT=0 时 SWSTART 打一发转一次；CONT=1（bit1）转完立刻再来一发，`st-adc.h:834` 的 `ADC_CR2_CONTINUOUS` 宏就是往 bit1 移位。
- **扫描 SCAN**：CR1 bit8 置 1 后，规则组按 SQR3→SQR2→SQR1 的名次顺序自动轮转（rank1 通道号在 SQR3 bit4:0；序列总长 L 在 SQR1 bit23:20，`st-adc.h:787` 的 `ADC_SQR1(n)` 宏正是 `(n-1)<<20`）。`st-adc.h:848` 的 `ADC_CR1_SCANCONV` 往 bit8 写。
- **DMA**：CR2 bit8 开 DMA 请求、bit9 DDS 让请求在扫描序列结束后继续发。扫描+DMA+循环缓冲是多通道采集的终点形态，NDTR 必须与通道数对齐（S8 的 NDTR 语义在这里直接用）。
- **外部触发**：EXTSEL[3:0] 选源（TIM1/TIM2/TIM3/TIM4/TIM5/TIM8 的 CC/TRGO 事件，`st-adc.h:331-346` 列了全部候选），EXTEN[1:0] 选上升/下降沿（`st-adc.h:318-320`）；EXTEN=00 即软件触发，`ADC_SOFTWARE_START` 在 `st-adc.h:346`。
- **EOC 语义**：CR2 bit10 EOCS 决定 SR.EOC 是"每个通道转完置位"还是"整条序列转完才置位"（`st-adc.h:855` 的 `ADC_CR2_EOCSelection` 宏）。轮询多通道时这个位决定你会不会串数。

六、参考电压与 VREFINT

满量程 4095 对应 VREF+（通常接 VDDA=3.3 V）：**参考电压晃，读数跟着晃**，这就是"VDDA 没接好读数乱跳"的原理。通道 17 是内部基准 VREFINT（`st-adc.h:383`：`ADC_CHANNEL_VREFINT = ADC_CHANNEL_17`），用它反推 VDDA 是电池供电设备的常规操作。

⚠️ **待回填**：VREFINT 出厂校准值（VREFINT_CAL）的存储地址**无法在本地核实**：`.trellis/ref/` 的头文件里没有对应定义（全文检索无结果）。本章不编造地址；用到时请查 STM32F407 datasheet 的"内部电压基准"一节回填。

附录：工程完整源码

本章取证工程 `code/stm32/09-adc/`（`sh probe.sh` 一条命令跑完，不需要开发板）：

sar_probe.c（12 轮二分表 + 量化误差自检）：

<<< ../../code/stm32/09-adc/sar_probe.c

ts_probe.c（RC 沉降推导 + 采样时间选档）：

<<< ../../code/stm32/09-adc/ts_probe.c

probe.sh（宿主编译运行 + 交叉编译取证）：

<<< ../../code/stm32/09-adc/probe.sh

README.md（完整命令行与输出归档）：

<<< ../../code/stm32/09-adc/README.md

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **先拍照再逼近，十二轮定乾坤；高阻慢充多给采样时间，多路扫描交给 DMA。**

延伸：ADC 外部触发常由 TIM TRGO 提供（S6）；传感器就绪通知可用任务通知（F6）替代队列，更轻量。

实物实验

- **电位器/光敏电阻分压读取**（quick win 上板版）：PA0（ADC1_IN0，以 datasheet 复用表为准）接电位器中间脚，两端接 3.3V/GND，串口打印原始值：旋一下，数字跟着动。
- **采样时间对比**：同一电压下 3 cycles vs 480 cycles，读数稳定性对比，串口实时打印方差；把本章 ts 表里的"若只给 3 拍"一列在板上复现一遍。
- **VREFINT 反推 VDDA**：读通道 17，用待回填的校准地址算出当前 VDDA（这比万用表多一个"程序自知"的维度）。

常见坑

- **引脚忘配模拟模式**：MODER=11（模拟），数字上下拉全关。配成输入浮空，读数是玄学。
- **VDDA 没接/虚接**：读数满量程乱跳；查原理图 VDDA 与 VREF+ 供电。
- **EOC 等待位置错**：规则组 EOC 每通道都置位（EOCS 的默认语义）。多通道要用扫描+DMA 或注入组，轮询会串数。
- **采样时间一律最短**：高阻信号读数系统性偏低：ts 表里 100 kΩ 只给 3 拍残差 2886 LSB，按源阻抗算 τ 再选档。动画第 ⑤ 阶段那条半路停下的红曲线就是它的样子。

短自测

1. 12 位 ADC 为什么只要 12 次比较？

> ▼ **看答案**
> 每轮比较把候选区间对半砍：4096 → 2048 → … → 1，log2(4096) = 12。probe 原话：`比较次数=12，候选从 4096 档收到 1 档`。位数就是"猜数字"的提问次数。

2. Vin=2.00 V、VREF=3.30 V，理想 12 位码值是多少？量化误差多少？

> ▼ **看答案**
> LSB = 3.30/4096 = 805.7 μV；floor(2.00/LSB) = 2482 = 0x9B2，回算 1.9997 V，差 0.3418 mV = 0.42 LSB，落在向下取整式 ADC 的 [0, 1) LSB 误差区间。

3. 100 kΩ 源阻抗、f_ADC=21 MHz，最少要多少采样周期？只给 3 拍会怎样？

> ▼ **看答案**
> τ = (100k+2k)·4pF = 408 ns，最少 9.01·τ·f_ADC ≈ 77.2 周期，选 84 拍档（残差 0.2262 LSB < 0.5）。只给 3 拍残差 2886 LSB，读数系统性偏低，基本是垃圾。

4. 扫描+DMA 三通道采集，SQR1 的 L 字段该写几？NDTR 呢？

▼ **看答案**
L = 通道数−1 = 2（`ADC_SQR1(3)` = `(3-1)<<20`，`st-adc.h:787`）；通道号按名次填 SQR3 的前三个 5 位字段。DMA 的 NDTR 必须与通道数对齐（3 或 3 的整数倍，循环模式），否则数据错位。

5. VREFINT_CAL 的地址本章给了吗？为什么？

▼ **看答案**
没有：`.trellis/ref/` 头文件全文检索无对应定义，无法在本地核实，本章明标「待回填」而不编造。用内部基准反推 VDDA 前先查 datasheet 回填。

🎯 S9 ADC 自测

1. **12 位 SAR ADC 为什么只需要 12 次比较？**
   A. 每个时钟周期并行比较所有位
   B. 每轮比较把候选区间对半砍：4096 → 2048 → … → 1，log2(4096) = 12，比较次数恒等于位数
   C. 前 8 位查表，只逐位逼近低 4 位
   D. 比较器精度是 12 位，一次锁定

2. **VREF=3.30 V、Vin=2.00 V，理想 12 位 ADC 锁定的码值是多少？**
   A. 2482（0x9B2）
   B. 2483（0x9B3）
   C. 2048（0x800）
   D. 2480（0x9B0）

3. **100 kΩ 源阻抗、f_ADC=21 MHz，采样时间图省事一律配 3 拍，后果是什么？**
   A. 只是噪声大一点，均值后能救回来
   B. 残差约 2886 LSB 的系统性偏低——保持电容没充满就转，读数基本是垃圾
   C. ADC 自动延长采样窗口，无影响
   D. OVR 溢出，转换停止

4. **三通道规则扫描 + DMA 采集，SQR1 的 L 字段和 DMA 的 NDTR 各写什么？**
   A. L = 3，NDTR = 3
   B. L = 2（通道数−1），NDTR 与通道数对齐（3 或 3 的整数倍）
   C. L = 2，NDTR = 1（DR 只有一个）
   D. L = 3，NDTR 随意

5. **ADC 输入引脚为什么必须配成模拟模式（MODER=11）而不是输入浮空？**
   A. 模拟模式采样速度更快
   B. 输入浮空会烧坏引脚
   C. 模拟模式把施密特触发器和上下拉全部断开，引脚直通 ADC——漏电流最小、采样最真；配成数字输入读数是玄学
   D. 两种配法等效，只是习惯

先选完再翻：答案与逐项解析在附录 C 的本组；在线版可点击即时判分。

对照表：本章概念 → 仓库落点

概念	仓库落点
12 轮二分试探表（2.00 V 实测输出）	`code/stm32/09-adc/sar_probe.c` 40-48 行
LSB=805.7 μV / 量化误差自检	同上 34-61 行
RC 沉降 9.01·τ 推导与选档表	`code/stm32/09-adc/ts_probe.c` 36-64 行
8 档采样时间	`.trellis/ref/st-adc.h` 392-399 行
ADC1/2/3 基址 0x40012000/2100/2200	`.trellis/ref/cmsis/stm32f407xx.h` 966-968 行
SR/CR1/CR2/SMPR/SQR/DR 偏移	同上 180-199 行（ADC_TypeDef）
ADON/CONT/DMA/EOCS/SWSTART 位号	同上 1252-1298 行
SCAN 位与 SQR 序列宏	同上 1215-1217 行；`.trellis/ref/st-adc.h` 787、811、848 行
外部触发源与沿	`.trellis/ref/st-adc.h` 318-346 行
VREFINT 通道 17	`.trellis/ref/st-adc.h` 383 行
断言 5/5 汇总	`code/stm32/09-adc/probe.sh` 59-60 行

延伸阅读

"12 刀砍到 4096 分之一"的电路从哪来：

- **[D6]** McCreary & Gray 1975 — 片上 SAR ADC 的电容阵列电荷再分配法就是这篇定下的；也解释了采样电容为什么要先充够电（采样时间的物理来源）。

你做到了

本章的真实定时采样实现已接入 J1 F407 记录器：ADC1 由 TIM2 TRGO 触发，DMA2 Stream0 传输，采样任务接收通知。工程位于 `code/stm32/07-sensor-logger`，复用实采驱动而不再复制一份同样的外设代码；原 `09-adc/probe.sh` 仍只承担 SAR/采样时间的宿主模型取证。

- 模拟量进入你的程序，传感器时代开启；
- "采样时间/源阻抗/参考电压"三兄弟成为调参直觉；
- 12 轮二分不再是黑话，你能把每一刀的 trial 码和 DAC 电压背出来。

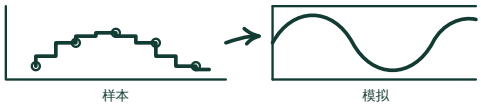
半格以内，这就是十二刀的极限：真正的天花板不在这十二刀里，在采样窗口那瞬间的照片清不清楚。

✅ 下一站：S10 DAC：反过来玩，让芯片自己输出真正的模拟波形。

通往单片机之路第 28 章

S10 DAC：芯片也能输出模拟量

难度 ★★☆☆☆ · 约 30 分钟



本章罗盘

问题

TIM 打拍、DMA 递谱、DAC 开嗓，84MHz 为什么除不出精确 1kHz/64 点？右对齐写错会怎样？

交付

一笔样本率/表长/波形频率手算（含 1312.5 不整除的两条取舍与表长 84 的精确解）加 TIM6→DAC→DMA 的寄存器级启动序列（DAC 侧先就位再使能 DMA 流）。

边界

本章代码为内联示例、无独立可编译工程；示波器看 1kHz 与砍表长看退化的板上实验未记录；输出缓冲阻抗给量级、具体值以 datasheet 为准；DAC 的 WAVE 噪声/三角生成与双通道同步不覆盖。

依据锚点 `a1 · f3 · f5 · mcu-anim`

> 🎯 示波器上只有一条直线，程序却没有报错：DAC 里有数，TIM 也在跑，DMA 甚至显示已使能。到底是谁没听见谁？让芯片唱出 1kHz 正弦不是一次写寄存器，而是一场三重奏：**DAC** 把数字变电平，**TIM** 打拍子，**DMA** 递乐谱。少接一根暗线，三个外设都“正常”，整首曲子仍然沉默。

本章精髓

1. DAC 本质是"电阻梯形网络"：12 位数字→4096 级电压（0~VREF）；建立时间决定它"唱多快不跑调"（与 PWM+滤波冒充 DAC 的方案对比：真 DAC 无纹波、响应快）。
2. 触发+DMA 是自动播放的关键：TIM 每次更新事件触发 DAC 取下一个样本，DMA 从正弦表自动递数：更新率 × 表长 = 波形频率（奈奎斯特定律在此现身：每周期至少几十个点才像正弦）。
3. 对齐方式是个坑：12 位数据有左/右对齐两种写法，写错对齐=幅度/波形全乱。

- **能理解**：为什么波形频率 = 样本率 ÷ 表长；为什么"84MHz 出 64kHz 样本率"在 F407 上不能整除（以及怎么取舍）。
- **能用**：把 DAC1 接示波器，跑出一条 1kHz 正弦；把表长从 64 砍到 8，亲眼看"正弦退化成阶梯三角"。

先修

- S6 TIM、S8 DMA、S9 ADC（对照）。

先跑起来（10 分钟 quick win）

PA4（DAC_OUT1，以 datasheet 为准）输出：先写一个固定值 2048，万用表量到约 VREF/2，芯片第一次"唱出"一个稳定的音高。

测量前先下注：如果万用表读数正确，能不能直接证明正弦播放链也正确？答案是不能。这个 quick win 只给 DAC 本体做单人试音；TIM 的拍子与 DMA 的递谱还没上场。先把“能出一个电平”和“能连续出波形”分开，后面的排错就不会三件事一起猜。

动画：三重奏的协奏

四阶段循环：① **TIM6 打拍**：计数器从 0 数到 ARR，更新事件（TRGO）标志亮起；② **DMA 递谱**：正弦表的索引前进一格，下一个样本被搬进 DAC 的 DHR12R1；③ **DAC 开嗓**：DHR 在触发沿搬到 DOR，电压跳到新台阶；④ **波形出来**：一串台阶连成的近似正弦随时间向左滚动。底部的样本指针在 64 点表里循环扫过，正是"DMA 替你递乐谱"的全部含义。

DAC 三重奏动画

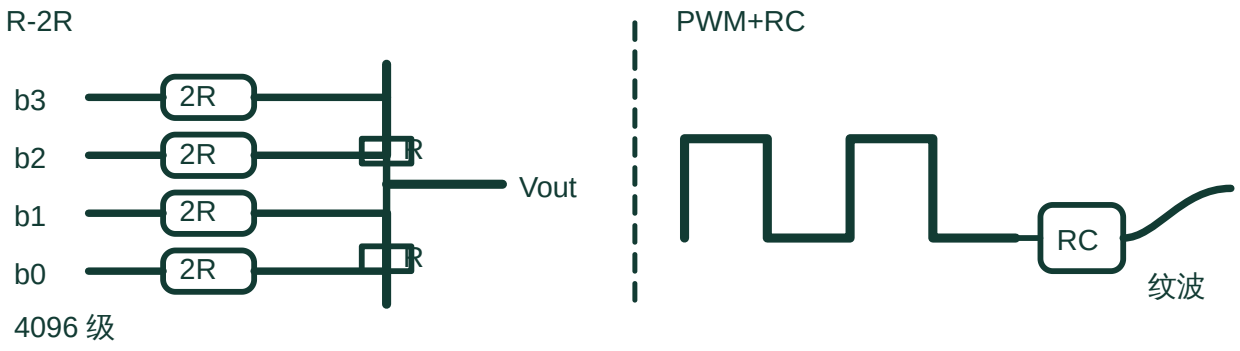
板卡事实

- DAC1 输出在 **PA4**（DAC_OUT1），DAC2 在 **PA5**（DAC_OUT2）；引脚模式 **模拟**（MODER=0b11，S3），**不需要 AF 编号**：DAC 是模拟外设，不走复用矩阵。
- DAC 控制器基址 `DAC_BASE = APB1 + 0x7400 = 0x40007400`（CMSIS 已核对）；时钟在 RCC_APB1ENR bit29 = `RCC_APB1ENR_DACEN_Msk`（0x20000000，CMSIS 已核对）。
- DAC1 的 DMA 请求：**DMA1 Stream5 Channel7**（RM0090 Table 43，与 S8 同源映射表；DAC 在 APB1 → DMA1）。
- 12 位数据有三种写寄存器：`DHR12R1`（右对齐 12 位）、`DHR12L1`（左对齐）、`DHR8R1`（8 位），本章用右对齐。
- 电压参考 VREF 由板卡供电决定（多数板 VREF = VDD = 3.3V，以你的板卡规格书为准）。

小节结构

小节	内容	四件套
DAC 原理	电阻网络 vs PWM+RC 滤波对比实测	库解析
静态输出	右对齐写 DHR12R1；万用表验证 VREF 关系	配置
TIM 触发	TRGO 选择+DAC 触发使能的链路	配置
DMA 递谱	正弦表生成（const 数组，回 C1 经济学）+循环模式	代码分析
波形数学	样本率/表长/频率关系；点太少变三角波演示	代码分析
输出缓冲	带载能力与建立时间的取舍	配置
SPL 对照	DAC_Init/DAC_DMACmd 落位	库解析

一、DAC 原理：电阻网络 vs PWM+RC 滤波



R-2R 逐位加权 vs PWM+RC 平均值近似

DAC 把数字变成电压靠的是 **R-2R 电阻梯形网络**：12 位数字每一位接一个电阻，按权重加出模拟电压。结果是一条**真正连续**的电压（0~VREF 分 4096 级），不是占空比方波。

与"PWM + RC 滤波冒充 DAC"对照：

维度	真 DAC	PWM+RC
输出	4096 级直流	平均值近似，纹波随截止频率变
响应	一个建立时间（μs 级）出值	RC 充放电，时间常数大才平滑
高频	能跟到几十 kHz	滤波器把高频也一起滤了
引脚	专用 DAC 引脚	任何定时器通道引脚
成本	只有带 DAC 的型号	所有 STM32 都行

结论：要稳、要快、要带宽，用真 DAC；只是要个慢变量（背光亮度），PWM+RC 够了。F407 有两个 DAC 通道，本章用 DAC1。

二、静态输出：先写一个数，量一个电压

最小配置：不开触发、不开 DMA，写一个数读一个电压：

```
#include "stm32f407.h"   /* CMSIS：DAC_TypeDef + 各 _Msk */

static void dac1_init_static(void)
{
    RCC->APB1ENR |= RCC_APB1ENR_DACEN_Msk;     /* 开 DAC 时钟 */
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN_Msk;   /* 开 GPIOA 时钟（[S3] 纪律） */
    /* PA4 模拟模式：MODER=0b11，无上下拉，无 AF */
    GPIOA->MODER |= (0x3u << (4u * 2u));
    GPIOA->PUPDR &= ~(0x3u << (4u * 2u));

    /* DAC_CR：使能通道 1、开启输出缓冲（BOFF1=0） */
    DAC->CR |= DAC_CR_EN1_Msk;                 /* EN1 = bit0（CMSIS 已核对） */
    /* BOFF1 = bit1：0 = 缓冲开（默认带载能力强）；1 = 缓冲关（高阻，rail-to-rail） */
}

static void dac1_set(uint16_t code)   /* code: 0~4095（右对齐 12 位） */
{
    DAC->DHR12R1 = code & 0x0FFFu;             /* 写 DHR12R1：右对齐，高 20 位忽略 */
    /* 写完 DHR 后，无触发模式下约 3 个 APB1 时钟搬到 DOR（输出寄存器） */
}
```

写 `2048`：DOR 输出 ≈ VREF × 2048/4096 = VREF/2。VREF=3.3V → 1.65V；万用表量到的就是这个数。**12 位的一半写法是 2048（0x800），不是 4096：右对齐时高 4 位是 0，写 0x800 是对的，写 0x8000 就是左对齐寄存器的事。** 这是骨架坑列表里"对齐写错=幅度只剩 1/16"的来源：把右对齐的 2048（0x800）写进左对齐寄存器 DHR12L1（数据在 bit4~15），实际输出只有 2048>>4 = 128 级 → 幅度 1/16。

三、TIM 触发：让 DAC 跟着拍子取数

自动播放的第一根线：TIM6 每次数到 ARR 产生更新事件，经 TRGO 通知 DAC"该取下一个样本了"。配置链路（TIM6 在 APB1，`RCC_APB1ENR_TIM6EN_Msk` = 0x10，CMSIS 已核对）：

```
static void tim6_dac_trigger_init(uint16_t psc, uint16_t arr)
{
    RCC->APB1ENR |= RCC_APB1ENR_TIM6EN_Msk;
    TIM6->PSC = psc;                            /* 预分频（[S6] 公式 f=TIMclk/(PSC+1)/(ARR+1)） */
    TIM6->ARR = arr;
    TIM6->CNT = 0;
    /* CR2.MMS = 010：更新事件作 TRGO 输出（CMSIS：MMS 在 bits4-6，010 = TIM_CR2_MMS_1 = 0x20） */
    TIM6->CR2 = (TIM6->CR2 & ~TIM_CR2_MMS_Msk) | TIM_CR2_MMS_1_Msk;   /* update → TRGO */
    TIM6->CR1 |= TIM_CR1_ARPE_Msk | TIM_CR1_CEN_Msk;   /* ARR 预装载 + 启动 */
}

static void dac1_trigger_enable(void)
{
    /* DAC_CR：TEN1=bit2 使能触发、TSEL1=bits3-5 选 000 = TIM6 TRGO（CMSIS：0x38 掩码） */
    DAC->CR |= DAC_CR_TEN1_Msk;                  /* 触发使能；TSEL1 复位值就是 000 = TIM6，不用改 */
}
```

链路闭合：TIM6 数满 → TRGO → DAC 在触发沿把 DHR12R1 搬到 DOR → 电压跳变。但 CPU 还得在每次 TRGO 后手动写下一个 DHR12R1，这活交给 DMA，CPU 才真的"不用当指挥"。

在脑中按一次暂停：如果 DHR 里的数已经变化，DOR 却不动，先查触发链；如果 DOR 会动但只重复少量台阶，再查 DMA 的地址、宽度与循环模式。把症状分流后，开场那条直线就不再是“DAC 坏了”，而是一张能沿信号链逐站追查的工单。

四、DMA 递谱：正弦表 + 循环模式

正弦表是 const 数组，由编译器在 Flash 里生成（C1 内存模型 的"经济学"：const 数据住 Flash，不占 SRAM）。64 点正弦，幅度归一到 0~4095：

```
#include <math.h>
#define SINE_N  64
static const uint16_t sine_table[SINE_N] = {
    /* 2048 + round(2047 * sin(2π i / 64))，i = 0..63；预生成，运行期直接用 */
    /* 生成脚本见本节末尾；占 128 字节 Flash */
};

static void dma1_stream5_sine_init(const uint16_t *table, uint16_t n)
{
    /* DAC1 → DMA1 Stream5 Channel7（RM0090 Table 43） */
    RCC->AHB1ENR |= RCC_AHB1ENR_DMA1EN_Msk;     /* 开 DMA1 时钟 */
    DMA1_Stream5->CR  = 0;                       /* 先禁用流再配（[S8] 纪律） */
    DMA1_Stream5->PAR = (uint32_t)&DAC->DHR12R1; /* 外设地址：DAC 的数据保持寄存器 */
    DMA1_Stream5->M0AR = (uint32_t)table;        /* 内存地址：正弦表 */
    DMA1_Stream5->NDTR = n;                      /* 传输个数 = 表长 */
    /* 配置：Ch7、内存递增、外设不递增、16→16 位、循环模式、高优先级 */
    DMA1_Stream5->CR = (0x7u << DMA_SxCR_CHSEL_Pos)   /* Channel 7 */
                     | DMA_SxCR_MINC_Msk              /* 内存递增：读完一个样本指针前进 */
                     | DMA_SxCR_PSIZE_16BITS_Msk      /* 外设 16 位 */
                     | DMA_SxCR_MSIZE_16BITS_Msk      /* 内存 16 位 */
                     | DMA_SxCR_CIRC_Msk              /* 循环模式：传完自动回卷到表头 */
                     | DMA_SxCR_DIR_Msk ? 0 : 0       /* 内存→外设（DIR=0） */
                     | DMA_SxCR_PL_Msk;               /* 高优先级 */
    /* DAC 侧使能 DMA 请求 */
    DAC->CR |= DAC_CR_DMAEN1_Msk;                /* DMAEN1 = bit12（CMSIS 已核对） */
    DMA1_Stream5->CR |= DMA_SxCR_EN_Msk;         /* 最后使能流 */
}
```

正弦表生成（编译期可由脚本预生成填进 const 数组，免得 main 里跑 `sinf`，无 FPU 的型号连 math.h 都不一定方便）：

```
/* 一次性生成器（仅用于离线生成 sine_table 的内容，不进固件） */
#include <math.h>
#include <stdio.h>
int main(void) {
    for (int i = 0; i < 64; i++) {
        printf("%d, ", (int)(2048 + round(2047 * sin(2*M_PI*i/64))));
        if (i % 8 == 7) printf("\n");
    }
}
```

启动顺序很关键：**先配 DAC（TEN1 + DMAEN1）再使能 DMA 流**：DMA 使能前 DAC 已经准备好接 TRGO；反过来 DMA 早早开始往 DHR 写，DAC 还没开触发，第一个样本会被下一个覆盖。这是 S8 DMA "外设侧先就位"纪律的 DAC 版。

五、波形数学：样本率、表长、频率，以及"算不整"

三者的关系只有一条：

$$f_{\text{波形}} = \frac{f_{\text{样本}}}{N} = \frac{f_{\text{TIM6 更新}}}{N} = \frac{f_{\text{TIM6 时钟}}}{(PSC+1)(ARR+1) \cdot N}$$

霸天虎标准时钟树下 TIM6 时钟 = APB1 ×2 = 84MHz（S2：APB1=42MHz，定时器时钟加倍）。要 1kHz、64 点：

$$f_{\text{样本}} = 1000 \times 64 = 64\text{kHz} \quad\Rightarrow\quad (PSC+1)(ARR+1) = \frac{84\text{MHz}}{64\text{kHz}} = 1312.5$$

1312.5 不是整数：这是 F407 上一个真实的尴尬，84MHz 除不出 64kHz。两种取舍（实测可证）：

取法	PSC	ARR	实际样本率	实际波形频率	误差
舍入 ARR=1311	0	1311	84e6/1312 = 64.024 kHz	**1000.38 Hz**	+0.038%
舍入 ARR=1312	0	1312	84e6/1313 = 63.988 kHz	**998.94 Hz**	−0.11%

两条都够准（音频/演示场景听不出），但**没有一条是精确 1kHz**。要精确 1kHz 的办法是改表长：`N × 1000 = 84e6/k`，找能整除的 k。`84e6/64000` 不整，但 `84e6/10500 = 8000` 整 → 样本率 8kHz、表长 8 → 1kHz 精确，但 8 点太疏（见下）；`84e6/84000=1000` → 样本率 84kHz，表长 84 → 1kHz 精确且 84 点够密。**选表长不光看频率，还要看 84MHz 能不能整除**，这是骨架"波形数学"小节最值得讲的一刀。

点太少会发生什么：奈奎斯特定律要求每周期至少 2 点（理论下限），实战要几十点才像正弦。把 N 从 64 砍到 8：8 个台阶连成的"正弦"更像阶梯三角：DAC 输出本身就是阶梯（零阶保持），表长越短阶梯越粗。骨架 quick win 后的"砍表长看退化"实验就是把这条数学眼见为实。

六、输出缓冲：带载能力 vs 建立时间

DAC 输出后有一个**输出缓冲放大器**（buffer），由 `DAC_CR.BOFF1`（bit1，CMSIS 已核对）控制：

- **缓冲开（BOFF1=0，默认）**：输出阻抗低（约 15kΩ 量级，手册值以 datasheet 为准），能直接驱动 ADC 输入或高阻负载；代价是输出范围不到 rail-to-rail（典型 0.2V~VDD−0.2V）；
- **缓冲关（BOFF1=1）**：输出直接是电阻网络，rail-to-rail 全幅；但输出阻抗高（MΩ 量级），带任何负载都会被拉低，必须外接高阻运放跟随。

选型很朴素：**接 ADC/高阻测量** → 缓冲开；**接外部运放做有源滤波** → 缓冲关 + 运放跟随。骨架坑里"输出接大负载掉电压"就是缓冲关了还接低阻负载，DAC 输出被分压，波形幅度凭空减半。

七、SPL 对照：`DAC_Init` / `DAC_DMACmd` 落位

SPL 的 DAC API 把 CR 的位组合打包成结构体字段（stm32f4xx_dac.c V1.8.0）：

SPL API / 字段	寄存器落位
`DAC_Init(DAC_Channel_1, &init)`	`init.DAC_Trigger` → TEN1 + TSEL1；`init.DAC_WaveGeneration` → WAVE1；`init.DAC_LFSRUnmask_*` → MAMP1
`init.DAC_OutputBuffer`	`DAC_OutputBuffer_Enable` → BOFF1=0；`_Disable` → BOFF1=1
`DAC_Cmd(DAC_Channel_1, ENABLE)`	CR `|= EN1`
`DAC_DMACmd(DAC_Channel_1, ENABLE)`	CR `|= DMAEN1`
`DAC_SetChannel1Data(Align, data)`	写 `DHR12R1`（右对齐）/ `DHR12L1`（左对齐）/ `DHR8R1`
`DAC_SoftwareTrigger(DAC_Channel_1)`	CR `|= SWTRIG1`（软件触发，不靠 TIM）

读库姿势照旧（S15）：拿到 `DAC_Init` 先问"我的 trigger/buffer 选择被翻成了 CR 的哪几位"，逐位与本节 `dac1_trigger_enable` 对答案。**SPL 的 `DAC_Init` 不替你算 TIM6 的 PSC/ARR**：样本率与表长是数学关系，得自己按第五节算完填进 TIM6。库负责"配置 DAC"，数学负责"配出对的频率"，分工清晰。

附录：本章代码落点

本章 TIM6/DAC1/DMA1 链路配置以内联代码呈现（无独立工程），可直接进任何项目主组件。DAC 与 DMA1 Stream5 的请求映射依据 RM0090 Table 43（与 S8 同源）；正弦表由离线生成器预填进 const 数组（见第四节末尾）。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **TIM 打拍、DMA 递谱、DAC 开嗓；波形频率=更新率÷点数，对齐写错全盘皆输。**

延伸：DAC 三重奏动画见 S10 动画；DMA 搬运机制在 S8 DMA 详解；TIM 触发 DAC 的配合在 S6 TIM 展开。

实物实验

- 万用表验证静态电压（2048≈VREF/2）；
- 示波器看 1kHz 正弦；把表长从 64 点砍到 8 点，亲眼看"正弦退化成阶梯三角"。

常见坑

- **对齐方式错**：右对齐的值写进左对齐寄存器，波形幅度只有 1/16。
- **DMA 数据宽度与 DAC 寄存器不匹配**：半字/字选错，波形全是毛刺。
- **忘开 DAC 时钟在 APB1**：DAC 在 APB1，RCC 位别找错（RM0090 §7）。
- **输出接大负载掉电压**：缓冲没开/负载太重，带载前先算输出阻抗。
- **DMA 与 DAC 启动顺序反了**：DMA 先使能会盖掉第一样本，DAC 侧先就位。

短自测

1. 为什么 PA4 配 DAC 不需要 AF 编号，而 SPI1 的 PA5 需要 AF5？

> ▼ **参考答案**
> DAC 是模拟外设，输出的是连续电压，不走数字复用矩阵，引脚只要设成"模拟模式"（MODER=0b11）就直连 DAC 内部。SPI 的 SCK/MOSI 是数字信号，必须通过 AF 矩阵把定时器的输出连到具体引脚，所以有 AF5 这种编号。模拟外设（DAC/ADC）和数字外设（SPI/USART/TIM）的引脚路由是两套机制。

2. 84MHz 的 TIM6 为什么出不来精确的 1kHz/64 点正弦？怎么取舍？

> ▼ **参考答案**
> 样本率要 64kHz，84MHz/64kHz = 1312.5 不是整数，PSC/ARR 都是整数乘积，凑不出 1312.5。取舍：舍入到 1312 或 1313，误差约 ±0.1%（听不出）；或改表长让 84MHz/目标样本率能整除（如表长 84、样本率 84kHz → 84MHz/84kHz=1000 整，精确 1kHz）。后者用更多点换精确，前者用小误差换简单。

3. DAC 输出缓冲开与关各适合什么负载？为什么"缓冲关 + 大负载"波形会减半？

> ▼ **参考答案**
> 缓冲开：低输出阻抗、带载能力强，但输出范围不到 rail-to-rail，适合直驱 ADC 输入或高阻测量。缓冲关：rail-to-rail 全幅，但输出阻抗 MΩ 量级，必须接高阻运放跟随。"缓冲关 + 大负载"时，DAC 高内阻与负载分压，输出电压被拉到内阻与负载的分压比，波形幅度按比例减半甚至更多。

4. DMA 为什么要循环模式？传完一次停了会发生什么？

> ▼ **参考答案**
> 正弦是周期信号，要无限循环播放。循环模式（CIRC）让 DMA 传完表长后自动把内存指针回卷到表头、NDTR 重载，无需 CPU 重新启动，这是"CPU 不用当指挥"的关键。若不用循环，DMA 传完 64 个样本就停，DAC 停在最后一个样本的电压，波形变成一条直流线。

5. 启动顺序为什么是"先 DAC 后 DMA"？

> ▼ **参考答案**
> DAC 侧（TEN1 + DMAEN1）先就位，意味着触发已使能、DMA 请求通道已开。这时再使能 DMA 流，第一个 TRGO 到来时 DAC 已准备好接样本、DMA 已准备好递样本。反过来若 DMA 先使能，它在 DAC 触发未开时就把样本往 DHR 写，等 DAC 触发一开，第一个样本已被后续样本覆盖，波形开头几个点错位。

对照表：本章概念 → 仓库与上游落点

概念	落点
DAC_CR 位（EN1/BOFF1/TEN1/TSEL1/DMAEN1）	CMSIS `stm32f407xx.h`（本站 `.trellis/ref/cmsis/`，逐位核对）
DHR12R1/DOR1 偏移	同上（DAC_TypeDef）
DAC1 → DMA1 Stream5 Ch7	RM0090 Table 43（与 S8 同源）
TIM6 CR2.MMS=010（update TRGO）	CMSIS `TIM_CR2_MMS_1_Msk`；S6 TIM 时基公式
样本率/表长/频率公式	本章第五节；C1 const 经济学
三重奏协奏动画	dac-trio.svg（四阶段）

你做到了

- 芯片从"只认数字"变成"能输出连续电压"；
- TIM+DMA+外设的协奏范式再下一城——这是 RTOS 之前"硬件自动化"的巅峰。

回到开场的沉默三重奏：三个外设各自使能，不代表链路已经闭合。以后看到“寄存器都有值、引脚却没波形”，按 TIM 触发、DMA 搬运、DAC 输出的顺序逐站验票，比从头重配一遍更快。

✅ 下一站：S11 I2C：两根线上开茶话会，START 到 STOP 每一拍都有事件位等你签收。

通往单片机之路第 29 章

S11 I2C：两根线的社交礼仪

难度 ★★★☆☆ · 约 35 分钟

本章罗盘

问题

I2C 难在哪一拍？清 ADDR 为什么必须先读 SR1 再读 SR2，读 EEPROM 收尾为什么先 NACK 再 STOP？

交付

42MHz 下 CCR=210、TRISE=43 的 100kHz 手算，一张写事务事件链表（START→SB→ADDR→TXE→BTF 每拍的等待与清除动作），加 AT24C02 页 8 字节/tWR 5ms/ACK polling 三连坑对策。

边界

E05 寄存器级工程编译通过，但板上波形与 PulseView 解码待实测；多主仲裁只讲线与原理，SMBus/10 位地址不覆盖；总线恢复 9 脉冲与 DMA 模式给的是代码框架，未逐拍验证。

依据锚点 `b1 · a1 · f5 · mcu-code · mcu-anim`

> 🎯 I2C 像一场有规矩的茶话会：两根线上挂着一群设备，谁说话先报名字（地址），说完等对方点头（ACK）。本章把这套礼仪逐拍翻译成 SR1/SR2 的状态流转：I2C 难，难在**每一拍都有一个必须等、必须按顺序清的事件位**。把事件链走通一遍，I2C 从"最容易卡死的协议"变成"最有规律的社交"。

本章精髓

1. **时序三要素**：START（SCL 高时 SDA 落）/STOP（SCL 高时 SDA 升）/ACK（第 9 拍拉低）：所有 I2C 波形都是这三块积木；数据跳变只允许发生在 SCL 低区。
2. **开漏+上拉是"线与"**：任何人都能拉低、没人能主动推高，这是 ACK、多主仲裁的物理基础（回 S3 开漏节）。
3. **寄存器级收发 = 事件驱动状态机**：写 CR1.START→等 SB→写 DR 地址→等 ADDR→读 SR1+SR2 清 ADDR→写数据→等 TXE/BTF，顺序错一拍就卡死，这也是 SPL `I2C_CheckEvent` 存在的全部理由。

- **能理解**：为什么数据只能在 SCL 低区变；为什么清 ADDR 必须"先 SR1 后 SR2"；为什么读完最后一个字节要先 NACK 再 STOP。
- **能用**：寄存器级向 AT24C02 写一页再读回；用逻辑分析仪抓包并对照动画逐拍对表。

先修

- S3 开漏、S7 状态机思维。

先跑起来（10 分钟 quick win）

接一块 AT24C02（7 位地址 0x50）到 I2C1（PB6/PB7，AF4，以 datasheet 与板卡规格书为准），写入 0x5A 再读回，串口打印，两线社交一次达成。

写入之前先押个归宿：往同一页多塞一个字节，越过了这一页的边界，它是接着排进后面的页、原地丢掉，还是绕回这一页的开头，把最先写下的那个盖掉？三个去处押一个，再往下读，页边界的三连坑会当庭宣判。

动画：一帧的旅程

START/地址/ACK/数据/STOP 六站逐拍扫过。盯紧 SDA 的每次跳变：**全部躲在 SCL 低区**完成，高电平期间动 = 控制信号。地址 0x50、数据 0x69，和 quick win 的 AT24C02 完全同帧。

I2C 时序动画

板卡事实

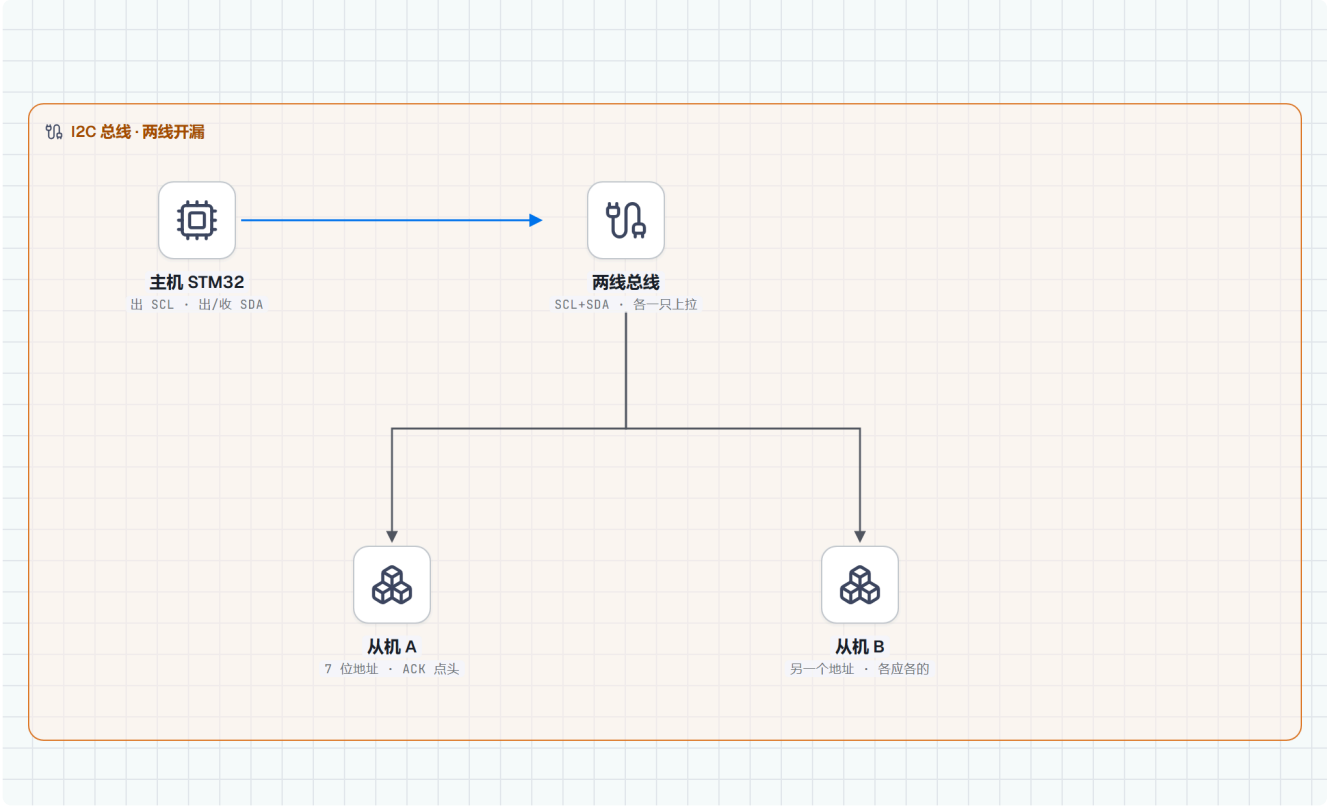
- AT24C02：2Kbit = 256 字节，**页大小 8 字节**，写周期 tWR 典型 5ms，这三个数决定第五、六节的全部坑。
- 7 位地址 0x50 = `1010 000`：高 4 位 `1010` 是 EEPROM 的"姓氏"（出厂焊死），低 3 位 A2/A1/A0 由引脚电平决定，全接地 = 0x50。
- 上拉电阻：模块一般自带 10k；速度要上去就换小（1k~4.7k），代价是功耗和下降沿负担，实物实验里亲手看差别。

小节结构

小节	内容	四件套
一、礼仪三件套	START/STOP/ACK 波形与"线与"电气基础	引脚
二、地址与方向	7 位地址+R/W 位如何拼成一个字节（SPL 原文）	配置
三、事件链	SB/ADDR/TXE/BTF 状态机 + `I2C_CheckEvent` 拆解	代码分析
四、SPL 对照	`I2C_Init` 六字段落位 CR2/CCR/TRISE，42MHz 实算	库解析
五、写 EEPROM	页边界、tWR、ACK polling 三连坑	代码分析
六、读回与重复 START	重复 START 的意义；NACK+STOP 收尾礼仪	代码分析

一、礼仪三件套：START、STOP、ACK

总线拓扑先看图：两线开漏、上拉必备、各挂各的地址。



I2C 总线拓扑：主机、两线总线（各一只上拉）、从机按地址挂载

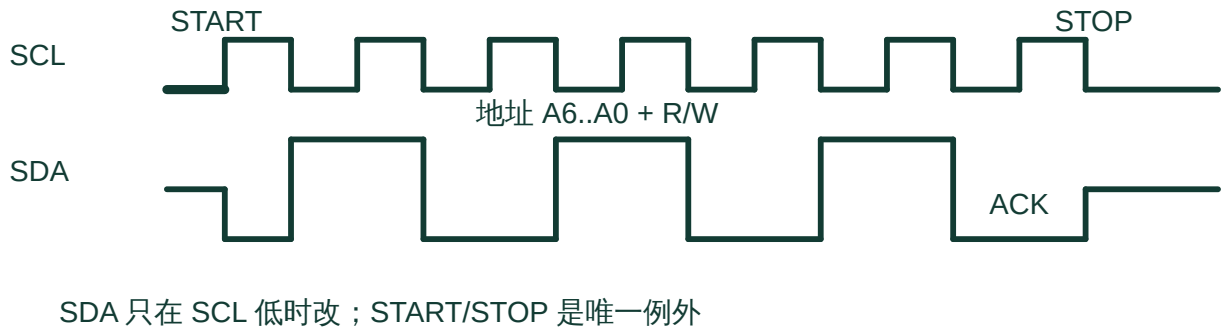
START：SCL 保持高，SDA 从高落低（"大家注意，我要说话了"）。**STOP**：SCL 保持高，SDA 从低升高（"我说完了"）。**ACK**：每个字节第 9 拍，接收方把 SDA 拉低（"听到了"）。

推论立刻出来：**数据位只能在 SCL 低电平期间变化**。因为 SCL 高期间的 SDA 跳变被硬件定义为控制信号（START/STOP），你数据线这时候动一下，全场都以为你喊了"注意"或"散会"。动画里 18 拍波形把这条铁律画得很清楚。

电气上这一切成立，靠的是回 S3 学过的开漏+上拉：SDA/SCL 都是"线与"，任何一个设备都能把线拉低，但谁也无法主动推高（高电平靠上拉电阻"放手"获得）。所以：

- 从机想 ACK，就在第 9 拍把 SDA 拽低，主机读到低电平=点头；
- 主机想 NACK（最后一口不吃了），第 9 拍放手不管，线自己弹高；
- 多主同时讲话时，谁先推高谁"输"（回读发现线被别人拽着），仲裁白送。

> 【注】 上拉电阻是把双刃剑：越大越省电但上升沿越肉（RC 充电），小到 1k 时沿陡了、速度能冲 400kHz，代价是拉低时电流大。E05 实验让你亲眼对比 10k vs 1k 的上升沿。



I2C 起止条件与地址字节：SCL 高时 SDA 下降为 START，上升为 STOP；地址后跟 R/W 与 ACK

二、地址与方向：一个字节里藏着 8 个 bit 的规矩

7 位地址出厂时只占字节高 7 位，**最低位是 R/W 方向位**：0=主机要写，1=主机要读。AT24C02 地址 0x50，上线时是 `0x50<<1 | 0 = 0xA0`（写）或 `0x50<<1 | 1 = 0xA1`（读），分析仪上你看到的是 A0/A1，不是 50。

SPL 的 `I2C_Send7bitAddress` 就干这件事，逐字原文（stm32f4xx_i2c.c V1.8.0）：

```
void I2C_Send7bitAddress(I2C_TypeDef* I2Cx, uint8_t Address, uint8_t I2C_Direction)
{
  /* Test on the direction to set/reset the read/write bit */
  if (I2C_Direction != I2C_Direction_Transmitter)
  {
    Address |= I2C_OAR1_ADD0;                /* 读:bit0 置 1 */
  }
  else
  {
    Address &= (uint8_t)~((uint8_t)I2C_OAR1_ADD0); /* 写:bit0 清 0 */
  }
  I2Cx->DR = Address;                         /* 写 DR = 送上总线 */
}
```

注意调用方传的 `Address` 必须是**已经左移过一位**的地址（0xA0 那种），这个函数只管方向位。而 `I2Cx->DR = Address;` 这一行就是全部"发送"：写 DR 寄存器，硬件自己完成移位输出。**I2C 的发送就是写 DR，剩下的全是等事件。**

三、事件链：I2C 的难点不在时序，在"等"

寄存器级一次写事务的法定流程：

1. `CR1 |= START` → 等 **SB**（SR1 bit0：START 已发出）
2. `DR = 地址|W` → 等 **ADDR**（SR1 bit1：从机认了地址）
3. **清 ADDR：先读 SR1，再读 SR2**（硬件规定的顺序，错一步 ADDR 永远挂着，后面全卡）
4. `DR = 数据` → 等 **TXE**（SR1 bit7：DR 空了，可以塞下一个）
5. 最后一个字节发完等 **BTF**（SR1 bit2：移位寄存器也空了，字节真正上了总线）
6. `CR1 |= STOP`

SPL 把"等事件"封装成 `I2C_CheckEvent`，原文只有十行核心（stm32f4xx_i2c.c）：

```
flag1 = I2Cx->SR1;
flag2 = I2Cx->SR2;
flag2 = flag2 << 16;
lastevent = (flag1 | flag2) & FLAG_MASK;        /* SR1 低16位 | SR2 高16位 */
if ((lastevent & I2C_EVENT) == I2C_EVENT)
  status = SUCCESS;
```

而"事件"就是状态位的组合宏（stm32f4xx_i2c.h）：

I2C 事件链：六步各等一组状态位，事件号是 SR1 与 SR2 的位组合

```
#define  I2C_EVENT_MASTER_MODE_SELECT  ((uint32_t)0x00030001)  /* BUSY, MSL and SB flag */
#define  I2C_EVENT_MASTER_TRANSMITTER_MODE_SELECTED ((uint32_t)0x00070082) /* BUSY, MSL, ADDR, TXE and TRA */
```

看懂这两个数，你就看懂了 SPL 的事件模型：0x00030001 = SR2 的 BUSY|MSL（0x0003 在高 16 位）+ SR1 的 SB（0x0001）："我成了主机且 START 已发"。**等事件 = 等一串状态位同时成立**，比单等一个标志位严谨，也更不容易被"幽灵置位"骗过。

> **【注】** TXE 和 BTF 是新手翻车第一名：TXE 只说"DR 空了"，字节还在移位寄存器里往外蹦；BTF 才说"整个字节发完了"。发完最后一字节只看 TXE 就发 STOP，会把最后一拍拦腰砍断，从机收到半个字节，NACK 伺候。

四、SPL 对照：`I2C_Init` 六字段没有魔法

初始化结构体（stm32f4xx_i2c.h）：`I2C_ClockSpeed / I2C_Mode / I2C_DutyCycle / I2C_OwnAddress1 / I2C_Ack / I2C_AcknowledgedAddress` 六个字段。`I2C_Init` 把它们落进三个寄存器，原文关键三段：

```
freqrange = (uint16_t)(pclk1 / 1000000);   /* CR2.FREQ = PCLK1 的 MHz 数 */
tmpreg |= freqrange;
I2Cx->CR2 = tmpreg;
...
if (I2C_InitStruct->I2C_ClockSpeed <= 100000)            /* 标准模式 */
{
  result = (uint16_t)(pclk1 / (I2C_InitStruct->I2C_ClockSpeed << 1)); /* CCR = PCLK1/(2×目标速率) */
  if (result < 0x04) { result = 0x04; }                  /* 硬件下限 */
  tmpreg |= result;
  I2Cx->TRISE = freqrange + 1;                           /* 标准模式最大上升时间 */
}
else                                                      /* 快速模式 */
{
  result = (uint16_t)(pclk1 / (I2C_InitStruct->I2C_ClockSpeed * 3));   /* Duty 2:1 */
  /* 或 I2C_DutyCycle_16_9 时 ×25 并置 DUTY 位 */
  I2Cx->TRISE = (uint16_t)(((freqrange * (uint16_t)300) / (uint16_t)1000) + (uint16_t)1);
}
```

拿霸天虎标准时钟树实算（PCLK1 = 42MHz，见 S2 RCC）：

目标	公式	结果
CR2.FREQ	42MHz ÷ 1MHz	**42**（告诉外设输入时钟多少 MHz）
CCR @100kHz	42e6 ÷ (100e3 × 2)	**210**（SCL 高低各计 210 个 PCLK1）
TRISE @100kHz	42 + 1	**43**（1μs 内必须升完，42+1 拍）

所以"100kHz"不是填进去的，是**分频算出来的**：I2C 外设拿 42MHz，按 CCR=210 数拍子，数出 100kHz 的 SCL。改时钟树不改 CCR？SCL 频率跟着飘，时钟配置和波特率从来是一根绳上的。

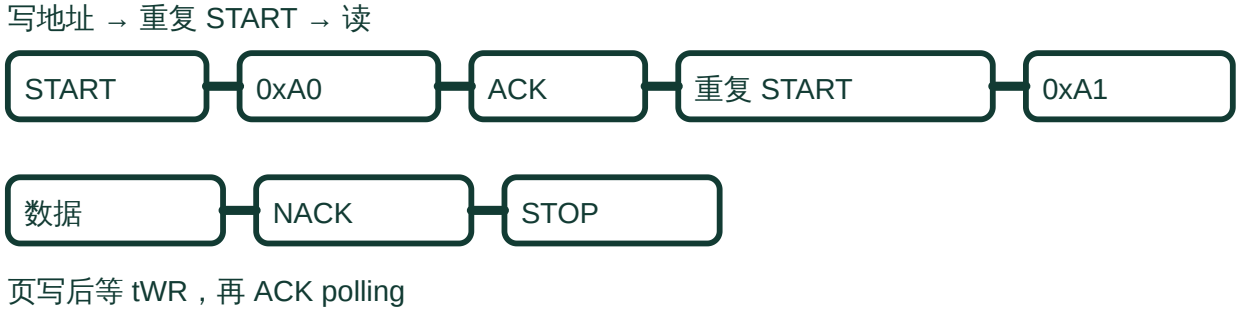
五、写 EEPROM：三个数引发的连环坑

AT24C02 写一字节的完整事务：`START → 0xA0 → 字地址 → 数据 → STOP`。看起来简单，但板子上躺着三个数：**页 8 字节、tWR 5ms、ACK polling**。

1. **页写回绕**：连写超过 8 字节，超出部分**卷回页首覆盖**：连写 20 字节到地址 0，实际第 9~20 字节盖掉了第 1~12 字节的位置附近（严格按"当前页内回绕"）。对策：按 8 字节对齐分批。
2. **写周期 tWR**：STOP 之后 EEPROM 进入内部擦写（典型 5ms），这期间它**不应答**：你立刻发起下一笔，地址阶段就被 NACK。
3. **ACK polling（应答轮询）**：利用"不应答"特性做等待：循环发 START+写地址，收到 NACK 就再来，直到某次收到 ACK = 擦写完成。比死等 5ms 优雅，也是工业代码的标准写法。

> 【注】 很多"I2C 偶发丢数据"的玄学 bug，根源就是页边界和 tWR 没伺候好。协议没错，是没按器件的脾气来。

六、读回与重复 START：不松手的艺术



EEPROM 读回：重复 START + NACK/STOP

读 EEPROM 要先"写"字地址再读数据，但中间**不能发 STOP 松手**：松手期间别的主机（如果有）可能插进来，而且某些器件会把内部地址指针复位。解法是**重复 START**（Repeated START）：

`START → 0xA0 → 字地址 → **重复 START** → 0xA1 → 读数据 → NACK → STOP`

收尾礼仪也有讲究：读最后一个字节前，先 `I2C_AcknowledgeConfig(DISABLE)`（下一拍回 NACK），读完立即 STOP——告诉从机"最后一口，不吃了"。若该 NACK 时回了 ACK，从机会继续往线上怼数据，总线就乱了。

整条链路和第三节的事件链一一对应：每个"→"背后都是一次 `I2C_CheckEvent` 的等待。**时序图是给人看的，事件链是给代码走的，两者逐拍同构**：这就是本章标题的含义。

附录：工程完整源码

E05 实验 现有独立的寄存器级 EEPROM 工程，包含有界等待、ACK polling 和单字节重复 START 读取；编译通过，板上波形待实测。

<<< ../../code/stm32/04-i2c-eeprom/main.c

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **先 START 后地址，ACK 点头再继续；数据只在 SCL 低区变，事件位按顺序等；清 ADDR 先 SR1 后 SR2，收尾 NACK 再 STOP。**

延伸：多任务共享 I2C 总线需互斥锁保护（F5）；I2C DMA 模式见 S8。

实物实验

- E05：抓一次完整"写 0x5A 读回"事务，导出 PulseView 解码，与动画逐拍对表；
- 对比实验：上拉电阻 10k vs 1k，示波器看上升沿变陡，速度 vs 功耗眼见为实；
- 翻车实验（选做）：故意不清 ADDR（删掉读 SR2 那行），看程序卡死在哪一步，再用调试器读 SR1/SR2 抓现场。

常见坑

- **ADDR 不会清**：先读 SR1 再读 SR2，顺序法定——新手十有八九栽这里；
- **等 TXE 不等 BTF**：STOP 发早，最后一字节没上总线（第三节注）；
- **上拉电阻省掉**：开漏没上拉=永远没有高电平，波形全是 0；
- **页写跨边界**：回绕覆盖页首（第五节）；
- **时钟树改了 CCR 没改**：SCL 频率跟着 PCLK1 飘（第四节实算）。

总线卡死恢复：SDA 被从机拉低的急救

I2C 最经典的翻车：主机复位或通信中途崩溃，从机正在发数据（SDA=0），主机重启后 SCL 开始翻转但 SDA 仍被从机锁低，总线"死了"，所有后续事务全部 NACK。

放进茶话会里看：主持人中场晕倒，爬起来重新宣布开场；刚才说到一半的那位客人还占着话头，半个字悬在嘴边，专等下一拍 SCL 把它送出去。

根因：从机是纯硬件状态机，不知道主机复位了。它还在等 SCL 时钟把最后一位移完，SDA 保持输出低电平（ACK 或数据 0）。主机 I2C 外设重启后发 START，从机认为"当前事务还没结束"，继续拉低 SDA。

恢复方案：GPIO 位带 9 个 SCL 脉冲

```
void i2c_bus_recovery(void)
{
    /* 1. 关闭 I2C 外设 */
    I2C1->CR1 &= ~I2C_CR1_PE;

    /* 2. 把 SCL/SDA 切到 GPIO 开漏输出 */
    GPIOB->MODER &= ~(0xF << (8 * 2));   // PB6=SCL, PB7=SDA
    GPIOB->MODER |=  (0x5 << (8 * 2));   // 01 = 输出模式
    GPIOB->OTYPER |= (0x3 << (6));       // 开漏
    GPIOB->PUPDR  |= (0x1 << (6 * 2));   // 上拉（辅助）

    /* 3. 手动发 9 个 SCL 脉冲 */
    for (int i = 0; i < 9; i++) {
        GPIOB->BSRR = (1 << (6 + 16));   // SCL 拉低
        delay_us(5);
        GPIOB->BSRR = (1 << 6);          // SCL 释放（上拉拉高）
        delay_us(5);
    }

    /* 4. 检查 SDA 是否释放 */
    if (GPIOB->IDR & (1 << 7)) {
        /* SDA 高了 → 从机已释放总线 */
        GPIOB->BSRR = (1 << (7 + 16));   // 主机发 STOP：SDA 高→低期间 SCL 高
        delay_us(5);
        GPIOB->BSRR = (1 << (6 + 16));   // SCL 拉低
        delay_us(5);
        GPIOB->BSRR = (1 << 7);          // SDA 释放（上拉高）→ STOP
        delay_us(5);
    }

    /* 5. 切回 AF 模式，重新初始化 I2C */
    GPIOB->MODER &= ~(0xF << (8 * 2));
    GPIOB->MODER |=  (0xA << (8 * 2));   // 10 = 复用
    GPIOB->AFR[0] &= ~(0xFF << (6 * 4));
    GPIOB->AFR[0] |=  (0x44 << (6 * 4)); // AF4 = I2C1

    I2C1->CR1 |= I2C_CR1_SWRST;          // 软件复位
    I2C1->CR1 &= ~I2C_CR1_SWRST;
    /* ... 重新 I2C_Init() ... */
}
```

为什么是 9 个脉冲：I2C 一字节 8 bit + 1 ACK = 9 个 SCL 周期。从机最多在等 9 个时钟完成当前字节，9 个脉冲保证它走完状态机释放 SDA。这九拍就是主持人补敲的九下杯子：每敲一下，它送出一位；数到九，含在嘴里的必然说完，总线交还。

DMA + I2C：非阻塞传输

轮询模式等 TXE/RXNE 会阻塞 CPU：主人一个字节一个字节亲自递话，整场就耗在递话上。DMA 让数据搬运脱离 CPU，请个帮手端着整盘字节，按节拍往 DR 里放：

```
/* DMA 发送：DMA1_Stream6 → I2C1_TX */
DMA1_Stream6->PAR = (uint32_t)&I2C1->DR;
DMA1_Stream6->M0AR = (uint32_t)tx_buf;
DMA1_Stream6->NDTR = len;
DMA1_Stream6->CR = DMA_SxCR_CHSEL_1       // Channel 1
                 | DMA_SxCR_DIR_0          // Memory → Peripheral
                 | DMA_SxCR_MINC           // 内存地址自增
                 | DMA_SxCR_TCIE           // 传输完成中断
                 | DMA_SxCR_EN;            // 使能

I2C1->CR2 |= I2C_CR2_DMAEN;              // I2C DMA 使能
/* DMA 自动把 tx_buf 逐字节写入 DR，I2C 自动发 */
```

DMA 模式的关键： - **DMA 传输完成 ≠ I2C 传输完成**：DMA TC 中断触发时，最后一字节还在 I2C 移位寄存器里（帮手放空了盘子，不等于最后一个字已经出口）。必须等 BTF=1（移位寄存器也空了）再发 STOP。 - **DMA 接收要用 LAST 位**：`I2C1->CR2 |= I2C_CR2_LAST`，让 DMA 收到最后一字节时自动发 NACK（而不是 ACK），从机知道该停了。

短自测

1. 为什么 SDA 的数据跳变必须落在 SCL 低区？

▼ **参考答案**
SCL 高电平期间的 SDA 跳变被定义为控制信号：下落=START、上升=STOP。数据若在高区变，会被全场误读为"开始/结束"，帧当场报废。所以协议规定数据只在 SCL 低区换脸，高区只负责被采样。

2. 清 ADDR 标志的法定动作是什么？写反会怎样？

▼ **参考答案**
先读 SR1，再读 SR2。这是硬件约定的清除序列；不读或读反，ADDR 一直挂着，后续事件判断全卡死，I2C 寄存器版最经典的死循环现场。

3. TXE 和 BTF 有何区别？为什么最后一字节必须等 BTF？

▼ **参考答案**
TXE=数据寄存器 DR 空（可以写下一字节）；BTF=移位寄存器也空了（当前字节完整上了总线）。最后一字节只看 TXE 就 STOP，会把正在移出的字节拦腰砍断，从机收到半帧回 NACK。

4. 向 AT24C02 从地址 0 连写 20 字节会发生什么？正确做法？

▼ **参考答案**
页大小 8 字节，超出部分页内回绕覆盖页首附近数据；且 STOP 后有约 5ms 擦写期。正确做法：按 8 字节分页写，每页之间用 ACK polling 等擦写完成。

🎯 S11 I2C 自测

1. **为什么 SDA 的数据跳变必须落在 SCL 低电平区？**
   A. 低电平区电流更大，信号更稳定
   B. SCL 高电平期间的 SDA 跳变被定义为控制信号（START/STOP），数据跳变会被误读
   C. 这是 I2C 协议的随机规定
   D. 低电平区功耗更低

2. **清 ADDR 标志的法定动作是什么？**
   A. 写 0 到 ADDR 位
   B. 先读 SR1，再读 SR2
   C. 写 1 到 ADDR 位
   D. 复位 I2C 外设

3. **TXE 和 BTF 有何区别？为什么最后一字节必须等 BTF？**
   A. TXE 和 BTF 完全相同
   B. TXE=数据寄存器空；BTF=移位寄存器也空。最后一字节只看 TXE 就 STOP 会截断正在移出的字节
   C. BTF 比 TXE 更早置位
   D. 只有接收模式才需要等 BTF

4. **向 AT24C02 从地址 0 连写 20 字节会发生什么？**
   A. 正常写入 20 字节
   B. 页大小 8 字节，超出部分页内回绕覆盖页首数据
   C. I2C 总线报错
   D. 从机自动分页

先选完再翻：答案与逐项解析在附录 C 的本组；在线版可点击即时判分。

对照表：本章概念 → 仓库与上游落点

概念	落点
事件位组合判定	SPL V1.8.0 `stm32f4xx_i2c.c` `I2C_CheckEvent()`（SR1|SR2<<16 比对）
地址+方向位拼字节	同上 `I2C_Send7bitAddress()`（DR 写入前处理 bit0）
100kHz 分频实算	同上 `I2C_Init()`：CCR=PCLK1/(2×速率)、TRISE=MHz+1
页写/ACK polling 实验	E05 配套固件
波形逐拍对表	动画 i2c-timing.svg（0x50/0x69 同帧）

你做到了

- I2C 从"最容易卡死的协议"变成"最有规律的社交"；
- SPL 的 I2C 事件模型被拆成 SR1/SR2 的位运算，以后看 HAL/LL 也只是换皮；
- 分析仪抓包对表成为肌肉记忆，S12 SPI 直接复用这套方法。

✅ 下一站：S12 SPI：四根线全双工移位，快是快了，时钟极性和相位的坑也跟着来了。

通往单片机之路第 30 章

S12 SPI：全双工的移位艺术

难度 ★★★☆☆ · 约 45 分钟

本章罗盘

问题

只想读一个字节为什么也必须往 DR 假写 0xFF？CPOL/CPHA 错配为何不报错、症状是什么？

交付

一段四步 spi_xfer（等 TXE→按字节写 DR→等 RXNE→读走，BSY 清零后才放片选）加 W25Q 读 JEDEC ID/页写/忙等的命令序列，附模式 0 与 3 的时序四宫格。

边界

W25Q 命令集以你手上型号的 Winbond 手册为准，本章给框架不背型号表；代码为内联示例，读 ID 与页写读回的板上实测未记录；42MHz 档只给布线约束，信号完整性设计不展开。

依据锚点 `a1 · a2 · b2 · f3 · f5 · mcu-anim`

> 🎯 SPI 是协议里的"直性子"：没有地址、没有 ACK、没有仲裁——片选一拉低，两个移位寄存器头尾相接成环，时钟每跳一下，两边各换一位。发和收是**同一个动作**的两面，这就是全双工的本质。

本章精髓

1. 移位环的真相：写 DR 发一个字节的同时必然收一个字节，所以"只想读"也要"假写"（发 0xFF/0x00），"只想写"也要"读走"收到的字节（否则溢出）。
2. CPOL/CPHA 四模式是"时钟相位"的排列组合：空闲电平（CPOL）× 第几沿采样（CPHA）：主从必须同款，模式错了数据错位还不报错（逻辑分析仪一眼识破）。
3. NSS 的管理哲学：硬件 NSS（SSM/SSOE）vs 软件 GPIO 拉片选：多从机/长帧场景软件片选更可控，大多数实战选软件。

- **能理解**：为什么"只想读一个字节"也必须写 DR；为什么 OVR（溢出）是"只写不读"的必然结局而不是偶发故障。
- **能用**：寄存器级把 W25Q 的 JEDEC ID 读出来、页写 256 字节再读回比对；分析仪上指认模式 0/3 的差异。

先修

- S11 I2C（抓包方法论）、S3 复用。

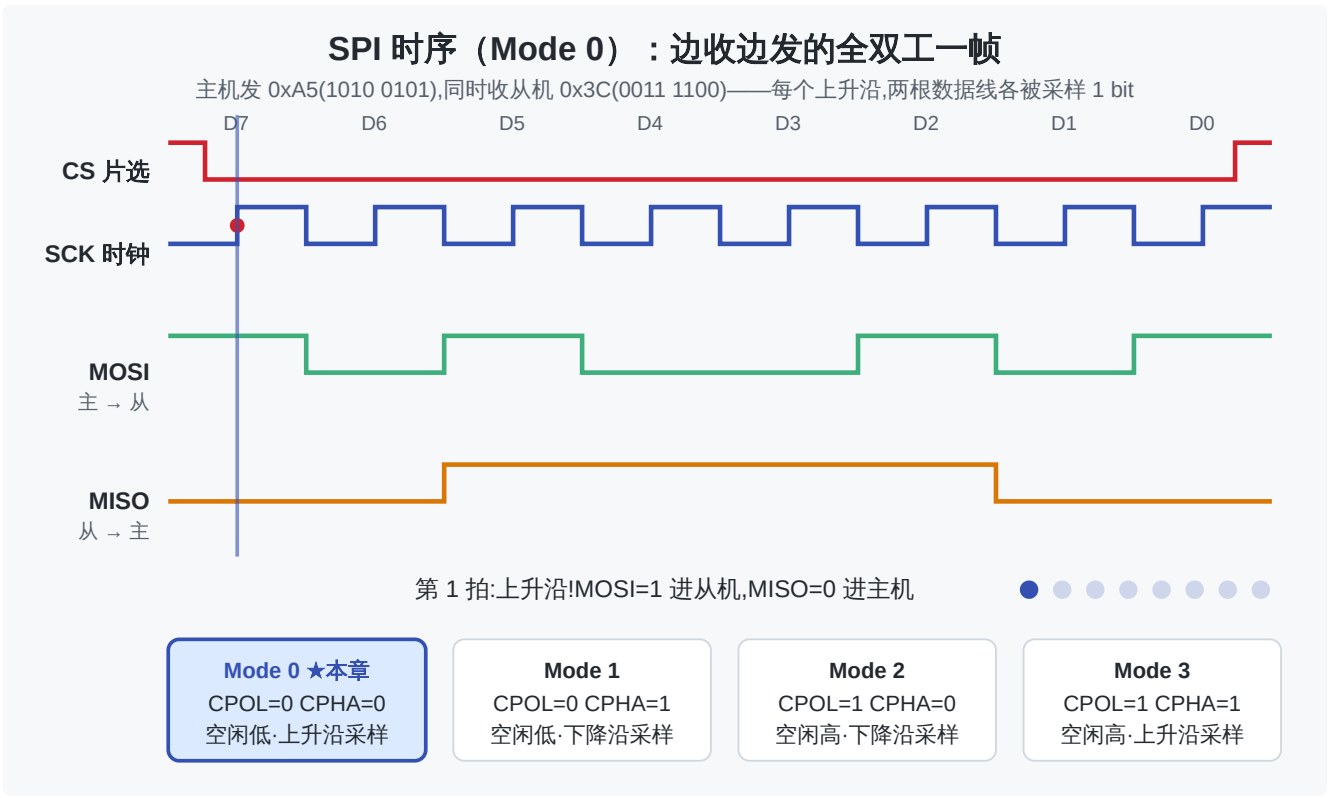
先跑起来（10 分钟 quick win）

SPI1（PA5/6/7，AF5，以 datasheet 为准）接 W25Q，发 `0x9F` 读三字节，串口打印出厂商/容量 ID，链路即通。

读回之前先押个交换：主机每移出去一位，手里是不是同时就收进一位？还是得等一个字节全部发完，才开始收自己的？押完往下读，答案就藏在移位环里：收发同拍到底是巧合还是必然。

动画：Mode 0 逐拍采样

一帧 8 拍：CS 拉低后，SCK 每个**上升沿**两根数据线各被采走 1 bit：主机发 0xA5 的同时收回 0x3C，收发同拍是移位环的必然，不是巧合。底部四宫格对照四种 CPOL/CPHA 模式，W25Q 系列 FLASH 支持 Mode 0/3。



SPI 时序动画

板卡事实

- SPI1 引脚：SCK=PA5、MISO=PA6、MOSI=PA7，复用功能 **AF5**；片选没有专用引脚的硬性要求，本章用软件 GPIO（PA4，以 datasheet 与板卡规格书为准）。
- SPI1 挂 **APB2**（RCC_APB2ENR bit12 = `RCC_APB2ENR_SPI1EN_Msk` 0x1000，CMSIS 已核对）；霸天虎标准时钟树下 APB2 = 84MHz（S2）→ 最高分频档 SCK = 42MHz。
- W25Q 系列（以你手上型号的 Winbond 手册为准）：JEDEC ID 命令 0x9F 回三字节（厂商 0xEF=Winbond、类型/容量两个字节）；状态寄存器命令 0x05，**BUSY = bit0**；页写 0x02（页 256 字节）、快读 0x03、写使能 0x06、扇区擦 0x20。
- 逻辑分析仪采样率 ≥ 4×SCK（抓 1MHz 的包至少 4MS/s；E02 的方法论直接复用）。

小节结构

小节	内容	四件套
移位环	主从移位寄存器成环；收发同拍的必然	库解析
四模式	CPOL/CPHA 波形四宫格；器件手册怎么标	配置
最小收发	写 DR→等 TXE/RXNE→读 DR；BSY 收尾	代码分析
W25Q 实战	JEDEC ID/页写/读状态寄存器忙等	代码分析
片选策略	软 NSS vs 硬 NSS；多从机星型接线	引脚
提速与 DMA	分频表、长帧 DMA（S8 复用）	配置
SPL 对照	SPI_Init 八字段到 CR1/CR2 的落位	库解析

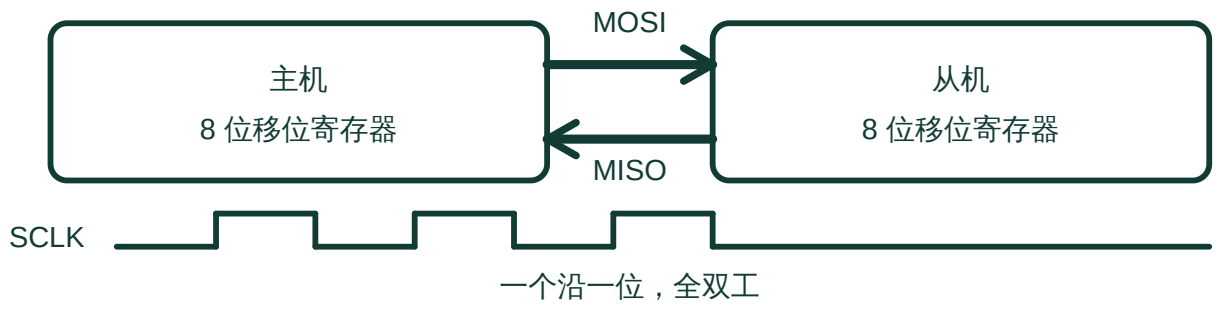
一、移位环：发即是收，收即是发

SPI 的硬件模型一张图说完：主机和从机各有一个 8 位移位寄存器，MOSI 把主机的"出"接到从机的"进"，MISO 把从机的"出"接回主机的"进"，**头尾相接，闭合成环**。时钟每来一拍，两边寄存器同步挪一位：主机移出去的是它 DR 里的新字节，移进来的是从机 DR 里的内容；8 拍之后，两个字节**完成对调**。

这个模型立刻推出三条在 I2C 里不成立的规矩：

1. **没有"只发"或"只收"**：8 拍一跑，两边都换了一个字节。想读 Flash 的状态寄存器？你也得**发**点什么（惯例 0xFF 或 0x00）去"顶"出从机的数据。
2. **发完必须收**：哪怕你根本不要从机回话，DR 里那个交换回来的字节也得读走；不读，下一个字节交换完成时 **OVR（overrun，SR bit6）** 置位，后续接收悄悄断线。

3. **没有地址阶段、没有 ACK**：选谁靠片选，收没收对靠应用层校验。SPI 的"简单"是把协议的活转嫁给了"片选+全双工"这两件事。



SPI 全双工移位环：主机和从机各一个 8 位移位寄存器，MOSI 与 MISO 同时移出一位、移入一位

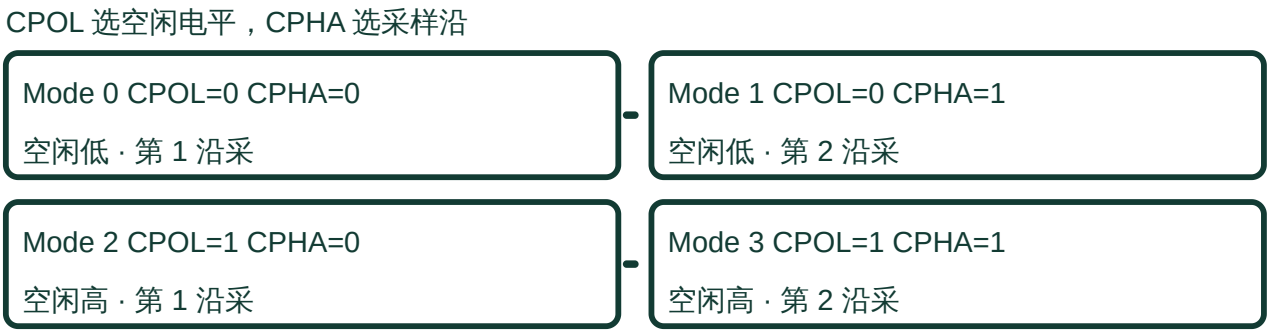
二、四模式：CPOL/CPHA 的排列组合

SPI 帧只有两个自由度，都写在 CR1 的最低两位（CMSIS 已核对：CPHA=bit0、CPOL=bit1）：

- **CPOL**：空闲时 SCK 停在什么电平（0=低、1=高）；
- **CPHA**：在第几个时钟沿采样（0=第一个沿、1=第二个沿）。

模式	CPOL	CPHA	空闲 SCK	采样沿	数据何时变
0	0	0	低	第 1 个沿（上升）	每个下降沿
1	0	1	低	第 2 个沿（下降）	每个上升沿
2	1	0	高	第 1 个沿（下降）	每个上升沿
3	1	1	高	第 2 个沿（上升）	每个下降沿

模式 0 与模式 3 是实战双主流：**采样沿同为"数据稳定后被抓"的安排**，区别只在空闲电平。W25Q 手册标"支持 Mode 0 与 Mode 3"，很多器件两档都能跑。但**主从必须同款**。模式错配不会报错，症状是：分析仪上 MOSI 波形完全正常，从机读到的却全错位一位。破案只能靠把 SCK 空闲电平和采样沿对表。



SPI 四模式：CPOL 定空闲电平、CPHA 定第几个沿采样，四格对照

器件手册的读法：一般写"CPOL=0/CPHA=0"，或者直接画时序图。图里可能标 CPOL/CPHA，也可能用 TI 的"CKP/CKE"。

TI 记法与 Motorola 记法之间还要换算，所以**认波形，别认缩写**。

三、最小收发：写 DR、等两个标志、一个 BSY 收尾

寄存器级一次字节交换，无项目依赖，可直接进任何工程：

```
#include "stm32f407.h"   /* CMSIS 设备头：SPI_TypeDef 与 SPI_SR_TXE/RXNE/BSY */

static uint8_t spi_xfer(SPI_TypeDef *spi, uint8_t b)
{
    while (!(spi->SR & SPI_SR_TXE)) { }        /* ① 等 TXE=1（bit1）：TX 缓冲空 */
    *((__IO uint8_t *)&spi->DR) = b;           /* ② 写 DR——8 位帧必须按字节访问（见下注） */
    while (!(spi->SR & SPI_SR_RXNE)) { }       /* ③ 等 RXNE=1（bit0）：交换回来的字节就位 */
    return *((__IO uint8_t *)&spi->DR);         /* ④ 读走它：RXNE 自动清，OVR 无机可乘 */
}
```

四步一步都不能省：

- ① **TXE**（SR bit1，CMSIS 已核对）：写 DR 前必须等它。上一字节还没搬进移位寄存器就写，等于把它覆盖掉；
- ② **按字节访问 DR**：F4 的 SPI 在 DFF=0（8 位帧）时规定 DR 按**单字节**访问（RM0090 data packing：按半字访问会一次塞进两个字节）；CMSIS 里 DR 是 32 位成员，所以要 `(__IO uint8_t*)&spi->DR` 这刀 cast；
- ③ **RXNE**（SR bit0）：交换完成的证据：**等它就是等"对方的 8 拍全部走完"**；
- ④ **读走**：就算你不需要返回值也要读（丢弃返回值），这是"只写场景"必须履行的另一半合同。

收尾第三件事：等 BSY=0 再放片选。 一帧最后一个字节交换完，TXE 早就置位了，但移位寄存器可能还在蹦最后几拍，此刻拉高 CS，从机收到的就是被掐断的半个字节。法定顺序：

```
static void spi_cs_high(void) { GPIOA->BSRR = (1u << 4) << 16; }   /* PA4 置高（BSRR 复位半区） */

/* 帧收尾 */
while (!(SPI1->SR & SPI_SR_TXE)) { }
while (SPI1->SR & SPI_SR_BSY) { }   /* BSY（bit7）=0：移位寄存器真正空闲 */
(void)*((__IO uint8_t *)&SPI1->DR);
spi_cs_high();
```

（BSRR 复位半区的 `1UL << (pin+16)` 掩码纪律在 C7 刚复习过。）

四、W25Q 实战：读 ID、页写、忙等

三个动作把 SPI 的典型工作流全覆盖。命令时序以 Winbond 手册为准，代码框架如下：

```
/* 读 JEDEC ID：0x9F → 连收三字节。每收一字节都要"假写"顶出来 */
static void w25q_read_jedec(uint8_t id[3])
{
    spi_cs_low();
    (void)spi_xfer(SPI1, 0x9F);     /* 发命令，同时收回一个无所谓字节（读走） */
    id[0] = spi_xfer(SPI1, 0xFF);   /* 想读：发哑数据 0xFF 顶出对方字节 */
    id[1] = spi_xfer(SPI1, 0xFF);
    id[2] = spi_xfer(SPI1, 0xFF);
    spi_cs_high();                   /* 单字节帧，BSY 会在 spi_xfer 的 RXNE 等待里自然走完 */
}

/* 读状态寄存器：0x05 → 1 字节，BUSY=bit0 */
static uint8_t w25q_read_status(void)
{
    spi_cs_low();
    (void)spi_xfer(SPI1, 0x05);
    uint8_t s = spi_xfer(SPI1, 0xFF);
    spi_cs_high();
    return s;
}
```

读 JEDEC ID 是**链路自检**的标准第一步。`0xEF 40 xx` 三字节一出来，说明物理层、模式、片选、时序全部 OK。烧到板子后，第一条要跑的就是它。**页写**多三样规矩：

```
static void w25q_wait_idle(void)
{
    while (w25q_read_status() & 0x01) { }   /* BUSY 位轮询：写/擦期间必须等 */
}

static void w25q_page_program(uint32_t addr, const uint8_t *src, uint32_t len)
{
    /* 预防性断言：页 256 字节 + 页内不回绕（W25Q 页编程越界不会帮你换页） */
    if (len == 0 || len > 256 || (addr & 0xFF) + len > 256) { return; }

    spi_cs_low();
    (void)spi_xfer(SPI1, 0x06);        /* 写使能：每次编程/擦除前都要重发 */
    spi_cs_high();                      /* WE 是"一次性令牌"，靠 CS 边沿生效 */

    spi_cs_low();
    (void)spi_xfer(SPI1, 0x02);        /* Page Program */
    (void)spi_xfer(SPI1, (uint8_t)(addr >> 16));   /* 24 位地址，高字节先行 */
    (void)spi_xfer(SPI1, (uint8_t)(addr >> 8));
    (void)spi_xfer(SPI1, (uint8_t)addr);
    for (uint32_t i = 0; i < len; i++) {
        (void)spi_xfer(SPI1, src[i]);
    }
    spi_cs_high();
    w25q_wait_idle();                   /* tPP 期间 BUSY=1，轮询等它落地 */
}
```

三条 W25Q 的脾气（全部有对照物）：**写使能是一次性的**（CS 拉高即失效，下次操作前重发），对应 I2C 里 AT24C02 的页写回绕；**页内不回绕**（越界部分被丢弃/拒绝，按你的手册为准），对应 S11 的页对齐纪律；**编程/擦除期间 BUSY=1**（只能轮询状态寄存器，器件不 NACK 也不报错），对应 S11 的 tWR+ACK polling。**每个协议都要伺候"器件的擦写周期"，只是 SPI 里没有 ACK 帮你踩刹车，全靠应用层主动查。**

五、片选策略：软 NSS vs 硬 NSS

NSS 的两种管法：

- **软件片选**（本章默认）：NSS 相关位全不使能，CS 就是普通 GPIO：拉低开帧、BSY 清零后拉高收帧。优点有三条。其一，**片选时机 100% 由代码掌控**，能精确覆盖"命令+数据"整段。其二，多从机随便加，每个从机一根 CS。其三，可以控制"一个逻辑事务跨多帧"，比如 JEDEC 三字节必须落在同一次 CS 低电平里。
- **硬件 NSS**：当 CR1 的 SSM=bit9（CMSIS 已核对）=0 时，NSS 引脚接管片选。主模式下再配 CR2 的 SSOE=bit2 输出片选，每**一帧**自动起落。坑在于：SSOE 的片选是"字节级"的，多字节命令会被逐字节切断；而主模式忘配 SSOE、NSS 又被外部拉低，触发 **MODF**（模式错误，SR bit5）：SPE 被硬件清掉、MSTR 被降级成从机，一个静默的"我咋不发了"。
- **单主机的"防误伤"写法**：SSM=1 且 SSI=bit8 置 1：NSS 信号内部强制为高，外部引脚完全脱钩，MODF 从根上不可能。这是"软件片选+硬件免疫"的组合拳，主模式裸跑（不用 SSOE、CS 走 GPIO）时的标准姿势。

多从机接线：**星型**：SCK/MOSI/MISO 三线共享，每从机独享一根 CS；MISO 是"谁被选中谁开口"的总线，没人选中时其输出脚应呈高阻（模块一般用输出使能自动处理，自己拼电路时要看器件的 MISO 三态条件）。

六、提速与 DMA：分频表与"长帧交给 DMA"

波特率没有"位数"可填，只有**分频档**：CR1 的 BR=bits3-5（CMSIS：0x38），编码 000~111 对应 fpclk/2 ~/256。SPI1 挂 APB2=84MHz：

BR 编码	分频	SCK（SPI1@84MHz）
000	/2	**42 MHz**（极限档，等长短线+回流地）
001	/4	21 MHz（短杜邦线可用）
010	/8	10.5 MHz
011	/16	5.25 MHz
…	…	…
111	/256	328 kHz（调通起步档）

调通纪律：**先 /256 或 /16 把链路读对，再逐档提速**：分析仪采样率跟不上时，先修仪器再修速度；42MHz 档上杜邦线就是天线（骨架坑列表里那句"30cm 杜邦线还想 42MHz"的现实版）。

长帧（整页 256 字节、屏刷新缓冲）逐字节 `while` 等标志是浪费 CPU：移位环是全场最不用盯的员工，时钟一响必然换位，递料的活交给 DMA 就行。开 CR2 的 TXDMAEN=bit0 / RXDMAEN=bit1（CMSIS 已核对），收发两路 DMA 流按 S8 的选型表配置，CPU 只管铺数据、配计数、等传输完成标志。注意 S8 的旧坑直接适用：**SPI 的 DMA 请求不在**

DMA1，F407 的 SPI1 在 DMA2（S8 章内表的既有结论）；且 DMA 收发要**成对配置**：这就是第一节"发即是收"的 DMA 版：只发不配收流，OVR 照样等你。

七、SPL 对照：`SPI_Init` 八字段落位

SPL 的 `SPI_InitTypeDef` 八字段到寄存器的落位（对照 stm32f4xx_spi.c V1.8.0 的翻译逻辑，位号已经 CMSIS 核对）：

结构体字段	取值 → 寄存器落位
`SPI_Direction`	全双工不动位；`Direction_1Line_Tx/Rx` → BIDIMODE/BIDIOE；`Direction_2Lines_RxOnly` → RXONLY（CR1 bit15/bit14/bit10）
`SPI_Mode`	`Mode_Master` → **MSTR**（CR1 bit2）；从机则清
`SPI_DataSize`	`DataSize_8b/16b` → **DFF**（CR1 bit11）
`SPI_CPOL`	→ **CPOL**（CR1 bit1）
`SPI_CPHA`	→ **CPHA**（CR1 bit0）
`SPI_NSS`	`NSS_Soft` → **SSM**（CR1 bit9）=1；`NSS_Hard` →SSM=0+CR2 **SSOE**（bit2）看主从
`SPI_BaudRatePrescaler`	`_2.._256` → **BR**（CR1 bits3-5），编码即 0x00/0x08/0x10/.../0x38
`SPI_FirstBit`	`FirstBit_MSB` 不动位；`_LSB` → **LSBFIRST**（CR1 bit7）

`SPI_Cmd(SPIx, ENABLE)` 就是 SPE（CR1 bit6）置 1，**先 Init 后 Cmd**，SPE 置 1 前的配置都算"备机"，置 1 那一刻外设才咬合齿轮。读库姿势照旧（S15 的方法论）：拿到任何 SPL 工程先问"SPI_Init 把我的参数翻成了 CR1 的哪几位"，拿本节的表对答案，RM0090 永远是终审。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **片选拉低成环，发即是收收即是发；CPOL 定闲电平，CPHA 定哪沿采样；读 ID 先行，忙等看 BUSY。**

延伸：多任务共享 SPI 总线需互斥锁（F5）；长帧传输用 DMA（S8）释放 CPU。

实物实验

- 读 W25Q JEDEC ID + 页写 256 字节再读回比对；
- 分析仪对比模式 0/3 波形差异，导出双图存档。

常见坑

- **只写不读**：RXNE 溢出标志置位，通信悄悄断线，收发必须成对处理。
- **模式与器件不符**：W25Q 支持模式 0/3，换错模式读出的 ID 全错。
- **片选提前拉高**：BSY 还在忙就释放 NSS，最后字节报废，收尾等 BSY=0。
- **长走线跑高速**：杜邦线 30cm 还想 42MHz，先降到 1MHz 调通再提速。
- **8 位帧按 32 位写 DR**：违反 data packing：一次塞两个字节，波形"莫名"多一拍（第三节注）。
- **SPI 的 DMA 去找 DMA1**：F407 的 SPI1 请求在 DMA2（S8 结论），配错流连"不工作"都算幸运。

短自测

1. 为什么"只读一个字节"也必须往 DR 写东西？发 0xFF 那一步在硬件上干了什么？

> **▼ 参考答案**
> 移位环两侧每拍同步交换：不写 DR，主机移位寄存器里就没有"料"推过去顶出从机的数据。发 0xFF（哑数据）就是用主机的移位输出把从机字节一拍一拍"顶"回 MISO，8 拍之后 DR 里躺着要读的字节。只想读 = 用假写驱动时钟，这是移位环的物理必然。

2. OVR 什么时候置位？为什么说它是"设计错误"而不是"偶发故障"？

> **▼ 参考答案**
> 上一个交换完成的字节没被读走、新字节又收满时，SR 的 OVR（bit6）置位。它必然发生在"只写不读"的循环里，每写 DR 一次，硬件就同步收进一个字节，不读走就攒不下第二个。清 OVR 要读 SR 再读 DR（顺序见 RM0090），但根治是像 spi_xfer 那样收发成对。

3. 模式 0 和模式 3 在分析仪上怎么一眼区分？它们为什么经常可以互换？

▼ **参考答案**
看 CS 拉低后、第一个数据变化前的 SCK 电平：低=模式 0，高=模式 3。两档的采样沿安排，都能让数据在稳定期被采样——都是"一沿变、一沿采"的互补节奏。所以支持"第一沿采样"的器件常常两档都兼容。 但主从必须同款。错配的症状是数据整体错位一位，且不报任何错。

4. 为什么单主机的寄存器版代码常配 SSM=1、SSI=1，哪怕片选用的是 GPIO？

▼ **参考答案**
SSM=1 时 NSS 信号由 SSI 位内部提供（SSI=1 = 一直"高"=没人在选我当从机），外部 NSS 引脚彻底脱钩。这样即使 NSS 脚被外部干扰拉低、或复用成别的功能，MODF（模式错误）也无从触发，SPE 不会被硬件偷偷清掉。这是"软件片选+硬件免疫"的组合拳。

5. 42MHz 需要什么条件？"先 /256 再提速"省的是什么？

▼ **参考答案**
等长或极短的走线、完整回流地、尽量少的过孔与转接（必要时上端接阻抗匹配）；杜邦线原型上请回到 5~10MHz 以内。先低分频调通省的是"分不清是协议错还是信号完整性错"的排查成本：把变量一个一个引入，是 E02 就立下的调试纪律。

🎯 S12 SPI 自测

1. **读 W25Q 的 JEDEC ID 时，为什么每收一个字节都要先往 DR 写 0xFF？**
   A. 0xFF 是 W25Q 规定的"空操作"命令字
   B. 移位环两侧每拍同步交换——不写 DR，主机移位寄存器就没有"料"推过去顶出从机的数据；发哑数据就是花钱雇时钟
   C. 写 0xFF 是为了把 MISO 预充电到高电平
   D. 不写 DR 的话 SCK 不会输出时钟

2. **循环里只写 DR 从不读回，攒到第二个字节交换完成时会发生什么？**
   A. 新字节覆盖旧字节，无任何标志
   B. SR 的 OVR（bit6）置位，后续接收悄悄断线——这是"只写不读"设计错误的必然结局
   C. TXE 永远不再置位，写 DR 被硬件拒绝
   D. MODF 触发，主机被降级成从机

3. **逻辑分析仪上怎么一眼区分模式 0 和模式 3？**
   A. 看 CS 拉低后、第一个数据变化前 SCK 停在哪：低=模式 0，高=模式 3
   B. 看 MOSI 第一位是 0 还是 1
   C. 模式 3 没有空闲电平
   D. 数每字节的时钟沿数：模式 0 是 8 沿，模式 3 是 16 沿

4. **单主机、片选走普通 GPIO 的工程，为什么还常配 SSM=1 + SSI=1？**
   A. 让硬件自动管理片选时序
   B. NSS 信号内部强制为高、外部引脚彻底脱钩——即使 NSS 脚被干扰拉低也不会触发 MODF，SPE 不会被偷偷清掉
   C. 提高 SPI 的最高时钟频率
   D. 关闭 MISO 方向以节省一个 GPIO

5. **给 SPI1 配长帧 DMA 传输，流该选在哪一侧？**
   A. DMA1——SPI 挂在 APB1 上
   B. DMA2——F407 的 SPI1 请求在 DMA2，且收发两路要成对配置，只发不配收流 OVR 照样等你
   C. DMA1 或 DMA2 都行，SPI 请求两边都映射
   D. 用 DMA2 但只需配发送流，接收靠轮询 RXNE

先选完再翻：答案与逐项解析在附录 C 的本组；在线版可点击即时判分。

对照表：本章概念 → 仓库与上游落点

概念	落点
CR1/SR 位号（CPHA/CPOL/MSTR/BR/SPE/SSM/SSI/TXE/RXNE/OVR/BSY）	CMSIS `stm32f407xx.h`（本站 `.trellis/ref/cmsis/`，逐位核对）
8 位帧按字节访问 DR	RM0090 SPI 章 data packing 说明；本章 `spi_xfer` 第 ② 步
`SPI_Init` 八字段翻译	SPL V1.8.0 `stm32f4xx_spi.c`（第七节落位表）
W25Q 命令集（0x9F/0x05/0x02/0x06/0x20）	Winbond W25Q 系列手册（以你的型号为准）
波形逐拍对表	动画 spi-timing.svg + E02 抓包方法论
分频表与 84MHz 实算	本站 S2 RCC 时钟树 + 第六节表
DMA 选型（SPI1→DMA2）	S8 DMA 请求映射表

你做到了

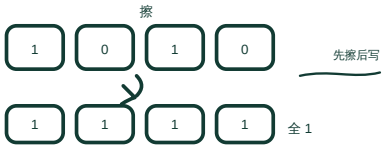
- 全双工移位从抽象变直觉；
- 拿下第一块 SPI 器件，抓包对表技能二次实战。

✅ 下一站：S13 内部 Flash 与 IAP：让固件学会给自己动手术。

通往单片机之路第 31 章

S13 内部 Flash 与 IAP：固件给自己动手术

难度 ★★★☆☆ · 约 40 分钟



本章罗盘

问题

程序能改写自己住的 Flash 吗？IAP 跳转为什么必须先关中断再设 VTOR 和 MSP，这个顺序能换吗？

交付

一张 11 扇区布局图（参数区选址依据）加 flash_unlock/erase/write 的逐位寄存器序列，和 jump_to_app 五步跳转函数，每步附「为什么是这个顺序」的注解。

边界

例程为内联代码、无独立工程：断电半擦回退、RDP 三级都未经板载验证（Level 2 是一行代码毁片的单向门，量产前别碰）；外部 SPI Flash 备份只链接到 S12；擦写关键路径搬 RAM 执行只给做法不展开。

依据锚点 `a1 · a3 · f3 · f5 · mcu-anim`

> 🎯 程序住在 Flash 里，能不能让程序**自己改写** Flash？能——存参数、做 IAP（应用内编程/远程升级）都靠它。但 Flash 是个怪房东：写只能 1→0，想 0→1 必须整页（扇区）推倒重建。

本章精髓

1. 擦写铁律：按"位"只能写 0，按"扇区"才能擦回全 1，所以存参数要先擦后写，且擦除单位是扇区（F407 扇区大小 16K~128K 不等，布局见 RM0090 §3）。
2. 擦写要"解锁+等空闲+设并行位数"：KEYR 序列解锁 → BSY 空闲 → PSIZE（按供电电压选 x8/x16/x32/x64）→ 擦/写 → 上锁，漏一步要么不动要么 HardFault。
3. IAP 的双固件结构：Bootloader 住扇区 0，App 住后面；跳转前三件事：关中断→设 MSP（App 栈顶）→跳 App 复位入口；向量表偏移（VTOR）别忘改。

- **能理解**：为什么"擦在运行的代码扇区"会让 PC 瞬间跑飞；为什么 VTOR 必须在跳转前改、不能让 App 的 reset handler 自己改。
- **能用**：写一个"参数不随断电丢失"的小框架；画出 IAP 双固件内存布局并实现跳转函数。

先修

- B3 链接脚本、B4 启动（向量表）、S2 时钟。

先跑起来（10 分钟 quick win）

在最后一个扇区写入 `0xA5A5`，断电重启后读回，数据还在，你就拥有了"不丢的记忆"。

写入之前先押个顺序：Flash 这块地方，是想写就写，还是得先走完什么手续才轮得到你写？再顺手押一个：扇区里原有的旧数据，不做任何处理直接盖上去，行吗？押完往下读，两把钥匙和擦的规矩会轮流登场。

动画：IAP 跳转接力

四阶段循环：① 双固件布局（Bootloader 住扇区 0、App 住后面，一条蓝线连接两块）；② 关中断 + 复位 SysTick（PRIMASK 置 1、SysTick 的红框熄灭）；③ 设 VTOR + MSP（向量表指针从扇区 0 切到 App 起始、MSP 从 App 第一字载入）；④ 跳 App 复位入口（PC 接力棒从 Bootloader 末尾滑到 App 的 reset handler）。每一步的"此刻向量表指向谁、此刻栈顶是谁"在画面上随阶段切换：这是 IAP 区别于"普通函数调用"的全部秘密。

IAP 跳转接力动画

板卡事实

- Flash 控制器基址 `FLASH_R_BASE = AHB1 + 0x3C00 = 0x40023C00`（CMSIS 已核对）；寄存器序：ACR(0x00)、KEYR(0x04)、OPTKEYR(0x08)、SR(0x0C)、CR(0x10)、OPTCR(0x14)、OPTCR1(0x18)。
- 解锁键两把：`KEY1 = 0x45670123`、`KEY2 = 0xCDEF89AB`（RM0090 FLASH 解锁键，写 KEYR 序列即解锁；这两把不是 CMSIS 宏，是手册常数）。
- F407 擦除并行度 PSIZE：x8/x16/x32 三档（2.7–3.6V 选 **x32**）；F427/F439 才有 x64，**F407 不要碰 x64**，以 RM0090"电压-并行度"对照表为准。

小节结构

小节	内容	四件套
扇区地图	F407 扇区布局与"参数区选哪扇"的考量	配置
解锁序列	KEYR 两把钥匙；PSIZE 与电压的关系	配置
擦写例程	扇区擦除+字写入逐行；BSY 等待纪律	代码分析
参数框架	结构体+魔数+校验和的防呆设计	代码分析
IAP 原理	双固件布局图；跳转函数逐行（MSP/VTOR/关中断）	代码分析
选项字节	读保护 RDP 三级与"锁死救砖"常识	配置
SPL 对照	FLASH_Unlock/FLASH_EraseSector 落位	库解析

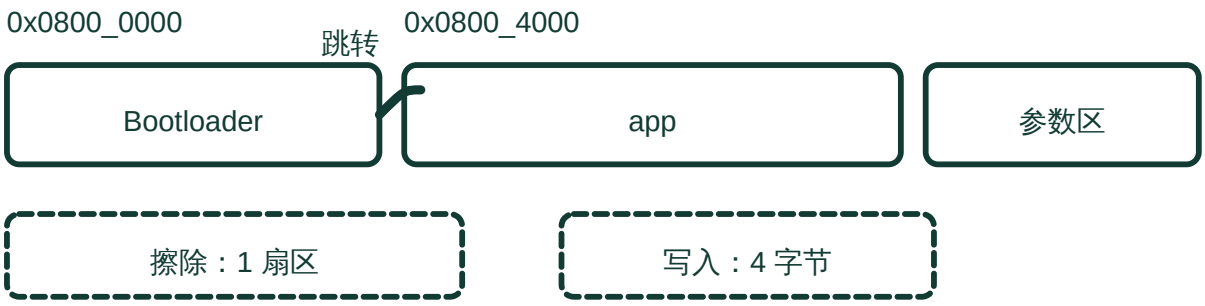
一、扇区地图：参数区选哪一扇

F407 是 1MB Flash，11 个扇区大小不等（RM0090 §3，CMSIS F4 头文件此芯片一档）：

扇区	起始地址	大小	用途建议
0	`0x08000000`	16 KB	Bootloader（IAP）
1	`0x08004000`	16 KB	App 起点 / 或 Bootloader 续
2	`0x08008000`	16 KB	App
3	`0x0800C000`	16 KB	App
4	`0x08010000`	64 KB	App
5	`0x08020000`	128 KB	App
6	`0x08040000`	128 KB	App
7	`0x08060000`	128 KB	App
8	`0x08080000`	128 KB	App
9	`0x080A0000`	128 KB	App
10	`0x080C0000`	128 KB	App
11	`0x080E0000`	128 KB	**参数区**（最后扇区，与代码隔离）

两条选址铁律：

1. **参数区与代码区物理隔离**：别把参数塞进代码扇区的空隙，擦参数等于擦代码，PC 瞬间跑飞。F407 最后一个 128KB 扇区是首选（如果 App 用不满前面 10 个扇区）。
2. **IAP 双固件布局**：Bootloader 住扇区 0（16KB 足够放一个最小 bootloader），App 从扇区 1 起；App 的起始地址 = `0x08004000`，这正是第五节跳转函数的 `app_addr`。Bootloader 的链接脚本 `ORIGIN` 落 `0x08000000`、App 的 `ORIGIN` 落 `0x08004000`，这就是 B3 链接脚本里那条 MEMORY 规则的实战场景。



Flash 扇区与 IAP：Bootloader 跳转到 app，参数区单独占一扇；擦除按扇区，写入按字

二、解锁序列：两把钥匙与并行度

Flash 上锁后，CR 的 PG/SER/MER 全部被硬件忽略（防误写）。解锁是把 KEY1、KEY2 顺序写进 KEYR：

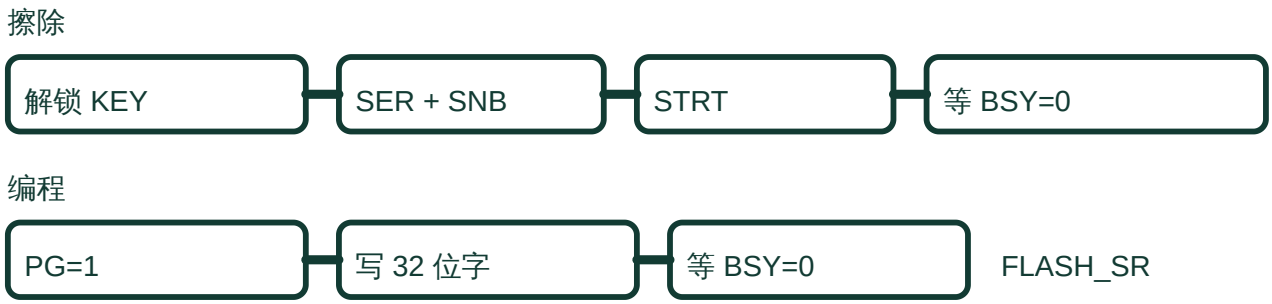
```
#include "stm32f407.h"   /* CMSIS：FLASH_TypeDef + 各 _Msk */

static void flash_unlock(void)
{
    /* 已上锁才需要解（重复写 KEY1/KEY2 在已解锁态是错的，会触发误写保护） */
    if (FLASH->CR & FLASH_CR_LOCK_Msk) {     /* LOCK = CR bit31（CMSIS 已核对） */
        FLASH->KEYR = 0x45670123u;            /* KEY1 */
        FLASH->KEYR = 0xCDEF89ABu;            /* KEY2 */
    }
}

static void flash_lock(void)
{
    FLASH->CR |= FLASH_CR_LOCK_Msk;           /* 一位置 1 即重新上锁 */
}
```

擦/写前还有一步"设并行度"：CR 的 PSIZE（bits8-9，CMSIS：0x300）。F407 在 2.7–3.6V 供电下选 `0b10 = x32`（一次写 32 位）：

```
#define FLASH_PSIZE_X32   (0x2u << FLASH_CR_PSIZE_Pos)   /* 0x200 */
```

PSIZE 必须在 STRT 之前设好，硬件按 PSIZE 决定每次 DRAM-flash 的字节数；选错档（低压跑 x32）会写坏数据不报错。

三、擦写例程：扇区擦除、字写入、BSY 等待



Flash 擦除/编程序列：解锁→SER/SNB→STRT→等 BSY

扇区擦除完整序列（以擦参数扇区 11 为例）：

```
static void flash_erase_sector(uint8_t snb)
{
    flash_unlock();
    FLASH->CR &= ~FLASH_CR_PSIZE_Msk;
    FLASH->CR |= FLASH_PSIZE_X32;                 /* 设并行度 */
    FLASH->CR &= ~FLASH_CR_SNB_Msk;                /* 清旧扇区号（SNB = CR bits3-6） */
    FLASH->CR |= (snb << FLASH_CR_SNB_Pos);        /* 写新扇区号 */
    FLASH->CR |= FLASH_CR_SER_Msk;                /* 选扇区擦除（SER = bit1） */
    FLASH->CR |= FLASH_CR_STRT_Msk;                /* 触发擦除（STRT = bit16）——硬件置后自清 */
    while (FLASH->SR & FLASH_SR_BSY_Msk) { }      /* BSY = SR bit16：擦完才清 */
    /* 收尾：清可能的 EOP、上锁 */
    if (FLASH->SR & FLASH_SR_EOP_Msk) { FLASH->SR = FLASH_SR_EOP_Msk; }  /* EOP = SR bit0，写 1 清 */
    FLASH->CR &= ~FLASH_CR_SER_Msk;
    flash_lock();
}
```

字写入（每次 32 位）：

```
static void flash_write_word(uint32_t addr, uint32_t data)
{
    flash_unlock();
    FLASH->CR &= ~FLASH_CR_PSIZE_Msk;
    FLASH->CR |= FLASH_PSIZE_X32 | FLASH_CR_PG_Msk;  /* PG = CR bit0：编程使能 */
    *(volatile uint32_t *)addr = data;               /* 直接写 Flash 地址——硬件自己编程 */
    while (FLASH->SR & FLASH_SR_BSY_Msk) { }
    if (FLASH->SR & FLASH_SR_EOP_Msk) { FLASH->SR = FLASH_SR_EOP_Msk; }
    FLASH->CR &= ~FLASH_CR_PG_Msk;
    flash_lock();
}
```

三条等待纪律，缺一就翻车：

1. **解锁与擦/写之间不能有别的 Flash 访问**（含向量取指之外的中断处理函数若在 Flash 里运行），把这段代码搬进 RAM 执行是严谨做法（见骨架坑列表）；
2. **BSY 必须等到底**（擦一个 128KB 扇区最坏几百毫秒，`while` 轮询要给 WDT 喂狗）；
3. **EOP 写 1 清零**：SR 大多标志是"写 1 清"，按 RM0090 §3 顺序处理。

> **【注】** 擦写期间任何中断如果处理函数也在 Flash 里运行，CPU 取指会和 Flash 编程抢总线，某些 F4 版本上会卡死。擦写关键路径放 SRAM 执行（`__attribute__((section(".ramfunc")))` + 链接脚本，见 B3）是工业代码的标准姿势。

四、参数框架：魔数 + 校验和的防呆设计

存进 Flash 的不是裸数据，是一份**自描述的快照**：

```
#include <stdint.h>
#include <string.h>

#define PARAM_MAGIC   0x50415241u   /* 'PARA' */
#define PARAM_ADDR    0x080E0000u   /* 扇区 11 起始 */

typedef struct {
    uint32_t magic;
    uint32_t version;
    /* 业务字段 */
    uint32_t baud;
    uint32_t sample_rate;
    /* 校验 */
    uint32_t crc;
} params_t;

static uint32_t crc32_simple(const uint8_t *p, uint32_t n)   /* 简易 CRC32，足以防单字节错 */
{
    uint32_t c = 0xFFFFFFFFu;
    for (uint32_t i = 0; i < n; i++) { c ^= p[i]; for (int b = 0; b < 8; b++) c = (c >> 1) ^ (0xEDB88320u & -(c & 1u)); }
    return ~c;
}

/* 复位时调：找不到合法快照 → 用默认值回填 */
params_t params_load_or_default(const params_t *defaults)
{
    const params_t *stored = (const params_t *)PARAM_ADDR;   /* Flash 直接可读 */
    if (stored->magic == PARAM_MAGIC) {
        params_t probe;
        memcpy(&probe, stored, sizeof probe);
        if (crc32_simple((const uint8_t *)&probe, offsetof(params_t, crc)) == probe.crc) {
            return probe;      /* 校验通过：用快照 */
        }
    }
    return *defaults;          /* 魔数错或校验败：回退默认，防"半擦"数据被当合法 */
}
```

三道防呆：**魔数**（`'PARA'` 当签名，擦空时读到 0xFFFFFFFF 不等于合法）+ **校验和**（防单字节损坏）+ **默认回退**（校验败不报错，回退，这是"半擦"状态下的安全选择）。骨架坑列表里的"擦除中断电进入半擦状态"靠这个三层防御兜底：即便某次擦了一半断电，下次复位读到的是错魔数或坏 CRC，照样回退默认值。

> 【注】 升级型产品用**双备份**：A/B 两个快照交替写、各自带版本号：写 A 成功才切到 A、失败停在 B。这样擦 A 中途断电，B 仍是上次有效值。本节框架是单备份基础版，双备份是把"半擦"风险降到零的工业级升级。

五、IAP 原理：双固件布局与跳转函数

Bootloader 与 App 各有独立的向量表与复位入口。Bootloader 跑完升级逻辑后，把控制权交给 App，这不是"函数调用"（App 有自己的栈、自己的中断表、自己的复位序列），是**硬件级接力**。跳转函数（CMSIS-Core 内建 intrinsic 全部就位）：

```
#include "stm32f407.h"   /* SCB、SysTick、__set_MSP、__disable_irq */

typedef void (*reset_handler_t)(void);

#define APP_ADDR  0x08004000u   /* App 起始 = 扇区 1 */

static void jump_to_app(void)
{
    /* ① 关掉所有可屏蔽中断：PRIMASK 置 1，防 NVIC 里挂着的请求在 VTOR 切换瞬间触发 */
    __disable_irq();

    /* ② 复位内核 SysTick：清 CTRL、清挂起位——App 启动时它必须是干净的 */
    SysTick->CTRL = 0;
    SCB->ICSR = SCB_ICSR_PENDSTCLR_Msk;

    /* ③ 重设向量表偏移：VTOR 指向 App 起始处的向量表 */
    SCB->VTOR = APP_ADDR;

    /* ④ 设主栈指针：App 向量表第一字就是它的初始栈顶 */
    __set_MSP(*(uint32_t *)APP_ADDR);

    /* ⑤ 取 App 复位入口（向量表第二字）并跳——Thumb 地址位 0 已由链接器置好 */
    reset_handler_t handler = (reset_handler_t)(*(uint32_t *)(APP_ADDR + 4u));
    handler();

    /* 跳过去就回不来了；理论兜底死循环 */
    while (1) { }
}
```

五步为什么是这个顺序（每一步都能在动画里对上阶段）：

- **关中断优先**：VTOR 切换是个"窗口期"，期间任何中断都会按**旧向量表**取入口，而旧表马上要被覆盖，取到垃圾地址就 HardFault。关 PRIMASK 把窗口期焊死；
- **VTOR 在 MSP 之前**：先让向量表归位，再从新表读栈顶；顺序反了，MSP 是从旧表读的；
- **跳复位入口而非 `main`**：App 的复位入口才是 App 启动代码的真正起点（B4 启动 里 .data/.bss 初始化在那之前），直接跳 main 会跳过这些初始化；
- **不复位外设**：IAP 的 Bootloader 已经配好的外设（如 UART）由 App 接管决定复位与否，这是设计选择，不是 bug。需要干净环境时由 Bootloader 自己复位（RCC 一刀切）。

App 一侧记得两件事：链接脚本 `ORIGIN = 0x08004000`、`SCB->VTOR` 在 reset handler 早期就指向自己（或保留 Bootloader 设的 VTOR，因为已经指向 App）；通常 App 的 startup 代码自己再写一次 VTOR 兜底。

六、选项字节：读保护 RDP 与"锁死救砖"

Flash 顶上还有一层"选项字节"（OPTCR/OPTCR1，CMSIS 已核对偏移 0x14/0x18），存的是芯片级配置：读保护（RDP）、写保护（WRP）、看门狗硬件使能、复位阈值等。对开发者影响最大的是 **RDP 三级**：

- **Level 0（默认）**：OPTCR 的 RDP 字段写 `0xAA`，无读保护，调试器可读可擦；
- **Level 1**：RDP 写任意非 `0xAA`、非 `0xCC` 的值，读保护启用，调试器能擦但不能读 Flash 内容；从 RAM 启动也被禁；
- **Level 2**：RDP 写 `0xCC`：**永久**读保护，Flash 永远不能再被调试器访问，**也不能降回 Level 0/1**（这一档是单向门，写错了芯片基本作废，只能当 OTP 用）。

救砖常识：Level 2 是"锁死"，没有"解锁"，只能擦整片回 Level 0 但 Flash 内容全失（这已经是好消息）；写 Level 2 之前先用 `0xAA`/Level 1 跑产品验证。选项字节解锁有自己的两把钥匙（OPTKEYR，`0x08192A3B` / `0x4C4D4E4F`，与主 Flash 解锁键不同），写法和主解锁同构。**写 Level 2 是一行代码就毁片的操作，没有板子量产前别碰。**

七、SPL 对照：`FLASH_Unlock` / `FLASH_EraseSector` 落位

SPL 的 Flash 操作 API 几乎是寄存器序列的逐字直译（stm32f4xx_flash.c V1.8.0）：

SPL API	寄存器序列（对照本节实现）
`FLASH_Unlock()`	写 KEYR = KEY1、KEY2（解锁；本节 `flash_unlock` 同）
`FLASH_Lock()`	CR `|= LOCK`（本节 `flash_lock` 同）
`FLASH_EraseSector(sn)`	设 PSIZE → 清/写 SNB → SER → STRT → 等 BSY → 清 SER
`FLASH_ProgramWord(addr, data)`	设 PSIZE → PG → 写地址 → 等 BSY → 清 PG
`FLASH_GetStatus()`	读 SR 的 BSY/PGSERR/WRPERR
`FLASH_WaitForLastOperation()`	`while (FLASH->SR & BSY)` + 清 EOP + 返回错误码

看一眼就明白：SPL 没有替你做任何"魔法判断"，它把"等 BSY、清 EOP、报 WRPERR"这些**纪律性步骤**包成函数。库的价值在此：寄存器序列太长，手写容易漏一步；SPL 把漏步做成"漏 API 调用"，编译期就看得见。**读库姿势**照旧：拿到 `FLASH_EraseSector` 先问"它动了 CR 的哪几位"，逐位与本节的 `flash_erase_sector` 对答案（S15 的方法论在 Flash 上特别好用，因为这里的"翻译"几乎是 1:1）。

附录：本章代码落点

本章擦写例程、参数框架、跳转函数全部以内联代码呈现（无独立工程），可直接进任何项目主组件。Flash 控制器基址与位定义对照见 `docs/.vitepress` 工具链对 CMSIS 头文件的核对记录（`.trellis/ref/cmsis/stm32f407xx.h`）；W25Q 外部 Flash 的 SPI 读写（参数备份到外部 Flash 时复用）见 S12。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **写 1 变 0 随意，0 回 1 整扇区；解锁等空再动手，IAP 跳转三件宝：关断、换栈、改向量。**

延伸：IAP 跳转与启动过程（B4）同构；Flash 擦写算法灌 RAM 的做法与 OpenOCD 烧录（B6）原理一致。

实物实验

- 参数保存：改参数→断电→重上电，串口打印参数仍在；
- 故意写坏校验和，看固件回退默认值，防呆机制亲自验证。

常见坑

- **擦正在运行的扇区**：代码自己擦掉自己，瞬间跑飞，参数区别放代码扇区。
- **擦除中断电**：扇区进入"半擦"状态；参数框架必须有双备份或掉电检测。
- **PSIZE 与电压不配**：低压跑 x32 并行写，写坏数据不报错；F407 上别碰 x64。
- **IAP 跳完中断向量没偏移**：App 的中断全跳到 Bootloader 的表，VTOR 是 IAP 的第一大坑。
- **解锁重复写 KEY1/KEY2**：已解锁态再写钥匙触发误写保护；先查 LOCK 位。

短自测

1. 为什么存参数到 Flash 必须"先擦后写"？

> **▼ 参考答案**
> Flash 按位只能 1→0 写，0→1 必须整扇区擦除。如果 Flash 上已有 0xA5A5、想改写成 0x3C3C（位级上需要把某些 0 翻回 1），不擦就只能 1→0，写不出目标位模式。先擦把整个扇区复位为全 1（全 0xFF），再写就只有 1→0 一种方向，目标位模式自然落成。

2. IAP 跳转函数为什么先关中断、再设 VTOR，而不是反过来？

> **▼ 参考答案**
> VTOR 切换是个"窗口期"：期间任何可屏蔽中断都会按 VTOR 当时指向的向量表取入口。若先改 VTOR 再关中断，窗口期内挂着的请求会按**新表**取入口，而新表（App 的）此刻 MSP 还是 Bootloader 的、栈布局未就绪，极易取到未对齐地址 HardFault。先关 PRIMASK 焊死窗口期，再改 VTOR，中断进不来，安全切换。

3. 跳转函数为什么不跳 App 的 `main`，而跳它的"复位入口"（向量表第二字）？

> **▼ 参考答案**
> App 的复位入口是它启动代码的真正起点：[B4 启动](../build/04-startup.md)里 .data 拷贝、.bss 清零、SystemClock 配置都在 main 之前发生。直接跳 main 会跳过这些初始化，App 运行在"半初始化"状态：全局变量全是 0 或未定义、系统时钟没配好。跳复位入口等于让 App 重新走一次完整的上电启动序列，正是 IAP 的本意。

4. RDP 的三个级别里，哪一级是"单向门"？为什么量产前别用它做"防抄板调试"？

> **▼ 参考答案**
> Level 2（写 0xCC）是单向门：一旦写入，Flash 永远不能再被调试器访问，也永远降不回 Level 0/1（只能整片擦除回 Level 0，但内容全失）。它适合真正量产定型后做终极防抄板，而不是开发期用来"挡调试"，开发期写错 Level 2 就意味着这块芯片基本作废，只能当 OTP 用。量产前用 Level 1（可降回 Level 0）就够防调试器读，且能擦回来。

5. 给一段"擦写关键路径放 SRAM 执行"的最小做法（提示：链接脚本 + section 属性）。

> **▼ 参考答案**
> 链接脚本加一段 `.ramfunc : { *(.ramfunc) } >RAM AT>FLASH`（VMA 在 RAM、LMA 在 Flash，[B3 链接脚本](../build/03-linker-script.md)的 VMA/LMA 范式）；启动代码把这段从 Flash 拷到 RAM；擦写函数加 `__attribute__((section(".ramfunc")))`。这样擦写期间 CPU 从 RAM 取指，不与 Flash 编程抢总线，规避某些 F4 版本上的

卡死风险。

对照表：本章概念 → 仓库与上游落点

概念	落点
FLASH_TypeDef 偏移（KEYR/SR/CR/OPTCR）	CMSIS `stm32f407xx.h`（本站 `.trellis/ref/cmsis/`，逐位核对）
CR 位（PG/SER/MER/SNB/PSIZE/STRT/LOCK）	同上，本节第二、三节
SR 位（BSY/EOP/WRPERR）	同上；WRPERR = bit4
解锁键 KEY1/KEY2	RM0090 FLASH 解锁键（手册常数，非 CMSIS 宏）
扇区布局表	RM0090 §3（F407 1MB / 11 扇区）
跳转用 CMSIS-Core 内建	`__disable_irq`、`__set_MSP`、`SCB->VTOR`、`SysTick->CTRL`
IAP 跳转动画	iap-handoff.svg（四阶段接力）
SPI 擦写外部 Flash	S12 SPI W25Q 实战

你做到了

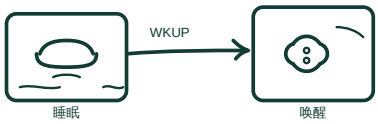
- 固件拥有了"长期记忆"；
- IAP 的原理图刻进脑子，OTA（P10）与 bootloader 设计不再神秘。

✅ 下一站：S14 低功耗：电池供电的产品想活过一年，先学会睡觉。

通往单片机之路第 32 章

S14 低功耗：让电池活过一年

难度 ★★☆☆☆ · 约 50 分钟



本章罗盘

问题

Stop 醒来一切照跑但数全错：慢 10.5 倍怎么算出来的？Standby 为什么醒来即复位、遗言放哪？

交付

三档睡眠四列对照表（停什么/留什么/谁唤醒/醒来到哪）加三笔实账：SysTick 1ms 变 10.5ms、BRR=729 变 10973 baud、IWDG 八码七分频超时表；位号与地址已同 CMSIS 逐条对账。

边界

位号、地址、指令流均经宿主对账，但三档电流只给量级，具体数值以 datasheet Power consumption 节为准，实测在 E06 且未回填；RTC 日历只给链路框架，细节以 RM0090 RTC 章为准；IWDG 深入玩法在 S17。

依据锚点 `a1` · `a2` · `f3` · `f5` · `mcu-code` · `mcu-anim`

> 🎯 电池产品的灵魂拷问：99% 的时间你的设备在睡觉。睡眠有多深、醒得多快、醒来还记得什么，决定了电池活一个月还是一年。

本章精髓

1. 三档睡眠的取舍：**Sleep**（只停 CPU，外设全醒，μs 级唤醒）→ **Stop**（停 1.2V 域时钟，SRAM/寄存器保留，RTC/EXTI 唤醒，μA~几十μA）→ **Standby**（几乎全断电，只留备份域，nA~μA，但 SRAM 全丢、复位式唤醒）。
2. 唤醒源是设计出来的：EXTI/RTC 闹钟/WKUP 引脚/IWDG。"谁能叫醒我"必须在入睡前配好，漏配=睡死。
3. 省电不止睡：降频、关外设时钟、GPIO 模拟态、关调试接口（DBGMCU 在 Stop 下也吃电）。系统级抠电。

- **能理解**：为什么 Stop 醒来后系统跑在 HSI 16MHz（PLL 停了得重配）；为什么 Standby 唤醒是"复位"而不是"继续"。
- **能用**：进 Stop + RTC 闹钟周期唤醒，万用表量到电流从 mA 掉到 μA 的跳水。

先修

- S2 时钟、S4 EXTI、S5 SysTick。

先跑起来（10 分钟 quick win）

没有板子也能把本章的账全跑完。进 `code/stm32/14-pwr`：

```
sh probe.sh          # 三步：宿主跑固件自己的函数 → 位号/地址与 CMSIS 逐条对账 → 交叉链 -O2 反汇编三档睡眠
```

第 1 步把 `PWR_CR` / `PWR_CSR` / `RCC_*` / `SCB_SCR` / `DBGMCU_CR` / `SysTick_LOAD` / `BKPSRAM_BASE` 全部改指到一组假寄存器再 `#include "main.c"`，所以断言跑的是固件本体：三档各自写出的那两个位（`Sleep CR=0x4200 SCR=0x10`、`Stop CR=0x4201 SCR=0x14`、`Standby CR=0x4202 SCR=0x14`）、醒来那笔时钟账（1 ms 变 10.5 ms、115200 变 **10973**）、看门狗八个分频码、备份域四笔解锁、唤醒源矩阵、复位原因解码。第 2 步把本章列出的**每一条位号**和 **10 个寄存器地址**拿去和 ST 官方 CMSIS 头文件逐条 diff（地址那一遍是现场从头文件的 `base + 结构体成员 Offset` 注释拼出来的，脚本里一个地址都没写死）。第 3 步把 `main.c` 编成 Cortex-M4 的 `-O2` 目标文件，三档睡眠在指令流里就是「改 CR + 改 SCR + `dsb sy` + `wfi`」，而 `wfi` 之后剩多少条指令，恰好是三档的本质差别（实测 1 / 6 / 7 条，见第八节）。

有板子再加一步：主循环跑 5 秒→进 Stop→RTC 2 秒后唤醒→串口打印"我醒了"循环，万用表串在供电上，看电流从 mA 掉到 μA 的跳水瞬间。

睡前先押个失忆：Standby 睡上一整晚再醒来，主 SRAM 和备份 SRAM，谁还记得睡前的事？两个都忘、都记得，还是只留一个在守夜？押完往下读，看备份域凭什么能撑过长夜。

动画：低功耗三档对比

Sleep 只停 CPU、Stop 停 1.2V 域时钟保留 SRAM、Standby 几乎全断电。三档"停什么/留什么/谁唤醒/醒来到哪"逐档对照，深睡与快醒的取舍一目了然。

低功耗三档对比：Sleep/Stop/Standby

板卡事实

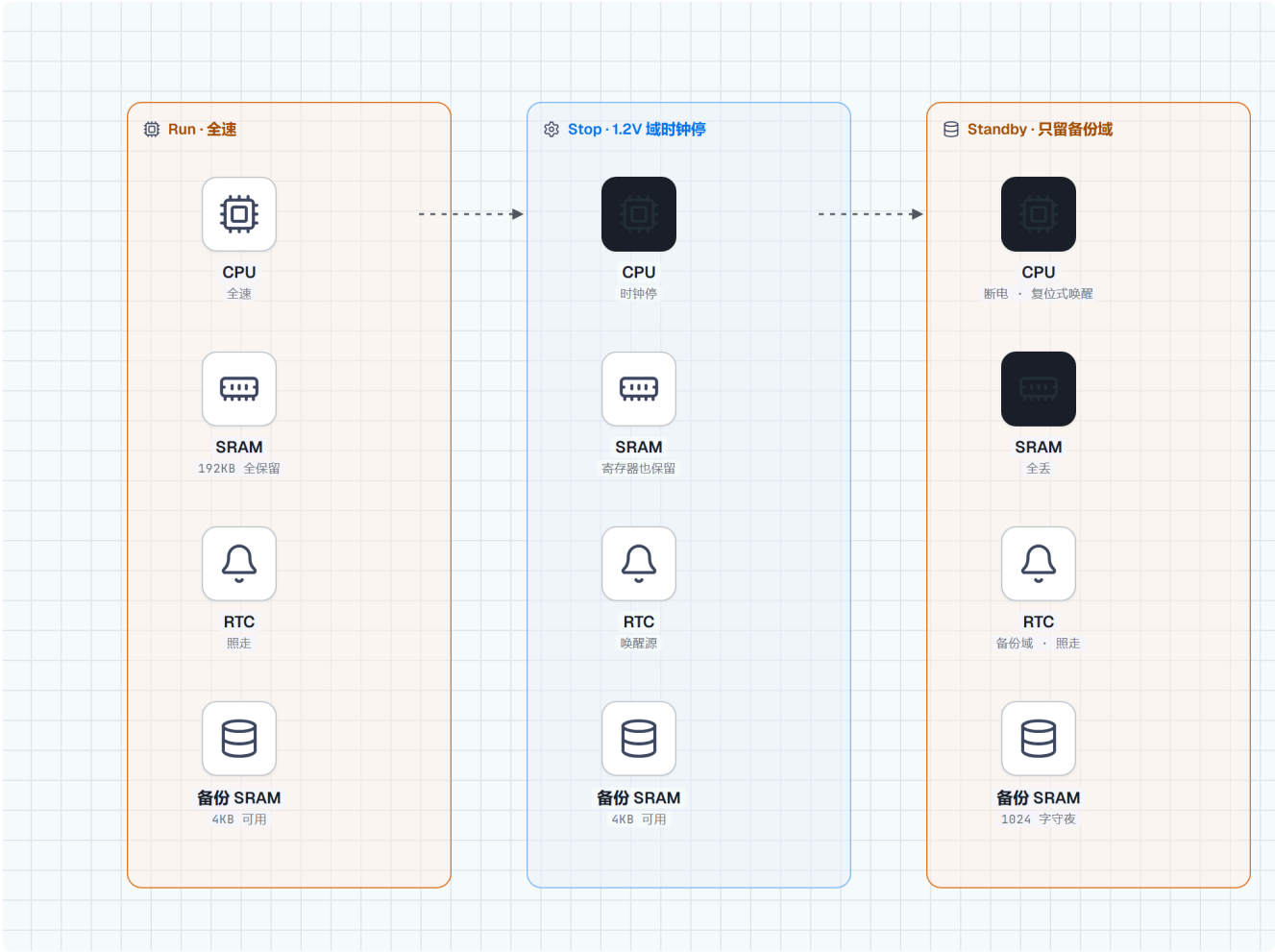
- PWR 控制器基址 `PWR_BASE = APB1 + 0x7000 = 0x40007000`（CMSIS 已核对）；时钟在 RCC_APB1ENR bit28 = `RCC_APB1ENR_PWREN_Msk`。
- PWR_CR 关键位：`LPDS`（bit0，Stop 下调压器低功耗）、`PDDS`（bit1，=1 进 Standby）、`CWUF`（bit2，清唤醒标志）、`CSBF`（bit3，清 Standby 标志）、`DBP`（bit8，解除备份域写保护）、`FPDS`（bit9，Stop 下 Flash 掉电），CMSIS 逐位核对。
- PWR_CSR 关键位：`WUF`（bit0，唤醒标志）、`SBF`（bit1，Standby 标志）、`BRR`（bit3，备份调压器**就绪**）、`EWUP`（bit8，使能 WKUP 引脚 PA0）、`BRE`（bit9，备份调压器**使能**）。`BRR` / `BRE` 这两位本章第五节要用，探针实测它们各自管什么。
- 内核那一侧只有一个位定档：`SCB->SCR`（`0xE000ED10`）的 `SLEEPDEEP`（bit2）；`SLEEPONEXIT`（bit1）与 `SEVONPEND`（bit4）是两个会被睡眠顺带动到的邻居位，实测它们不被定档逻辑碰。
- `RCC_CSR`（`0x40023874`）高位的七个复位标志：`RMVF`（bit24，写 1 清全部）、`BORRSTF` 25、`PINRSTF` 26、`PORRSTF` 27、`SFTRSTF` 28、`IWDGRSTF` 29、`WWDGRSTF` 30、`LPWRRSTF` 31。
- `DBGMCU_CR`（`0xE0042004`）低三位 `DBG_SLEEP` / `DBG_STOP` / `DBG_STANDBY`（bit0/1/2），睡前一次 `bic #7` 清掉。
- 备份 SRAM **4 KB** 在 `BKPSRAM_BASE = 0x40024000`（CMSIS 已核对），时钟在 RCC_AHB1ENR bit18 = `RCC_AHB1ENR_BKPSRAMEN_Msk`（0x40000）；写之前必须 `PWR_CR.DBP=1` 解除备份域写保护。4096 字节 = **1024 个 32 位字**，这是"睡前遗言"的硬上限。
- Stop 唤醒源在线路层的落点：`WKUP` = EXTI 线 **0**（PA0，但由 `PWR_CSR.EWUP` 单独管，不在 EXTI_IMR 里）、`PVD` = 线 16、`RTC 闹钟` = 线 **17**、`RTC 唤醒` = 线 18。
- RTC 基址 `RTC_BASE = APB1 + 0x2800 = 0x40002800`；时钟使能在 RCC_BDCR.RTCEN（bit15），时钟源选 LSE/LSI（RCC_BDCR.RTCSEL）。
- IWDG 预分频寄存器 `PR` 是 **3 位（8 个码）但只有 7 档分频**：`/4 /8 /16 /32 /64 /128 /256 /256`。`PR=6` 与 `PR=7` 是同一个 256，不存在 /512。LSI 按 32 kHz 算，`RLR=4095` 时超时从 511 ms（PR=0）到 32760 ms（PR=6/7）。
- 电流数值以 F407 datasheet §"Power consumption"为准，本章给量级（mA → μA → μA/nA）而非具体数，实测见 E06。

小节结构

小节	内容	四件套
三档对比	四列对照表 + 唤醒延迟实测数据	配置
Sleep 与 WFI/WFE	`__WFI()` 一条指令的事；唤醒后时钟恢复检查	代码分析
Stop 全流程	入睡前配置清单；唤醒后重配 PLL（HSI 默认接管）	代码分析
RTC 闹钟	RTC 初始化与闹钟中断；LSI/LSE 精度对续航的影响	配置
Standby 与备份域	备份寄存器传"睡前遗言"；WKUP 引脚	代码分析
抠电清单	GPIO/调试接口/外设时钟/调压器档位逐项抠	配置
SPL 对照	PWR_EnterSTOPMode 的参数含义	库解析
无板取证	`code/stm32/14-pwr/probe.sh`：固件本体跑断言 + CMSIS 逐位逐址对账 + `-O2` 反汇编三档睡眠	代码分析

一、三档对比：停什么、留什么、谁唤醒、醒来到哪

供电域矩阵先看图，深底是断电的域：Run 全活，Stop 停 1.2V 域时钟但 SRAM/RTC 还在，Standby 只剩备份域守夜。



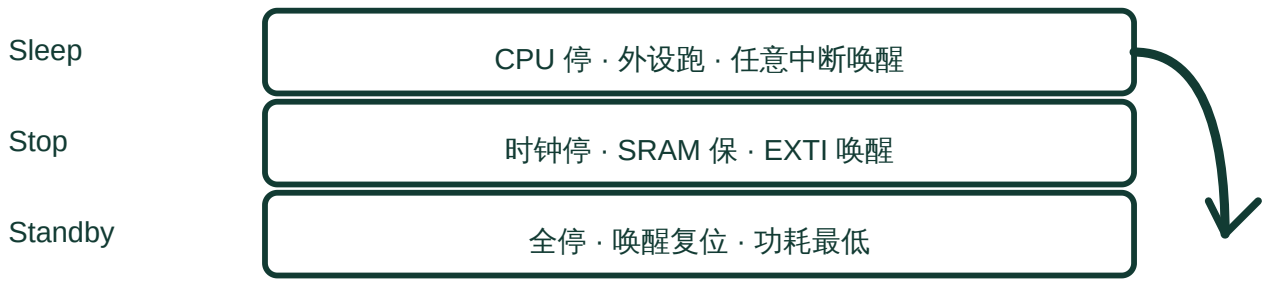
三档睡眠供电域矩阵：深底为断电域，Standby 只留 RTC 与备份 SRAM

低功耗不是"省电模式"一个词，是一张取舍表。F407 提供三档，越深越省电，但醒来代价越大：

维度	Sleep	Stop	Standby
停什么	CPU 时钟	1.2V 域时钟（CPU+大部分外设）	调压器（1.2V 域断电）
留什么	SRAM、寄存器、所有外设时钟	SRAM、寄存器、RTC、备份域	**只有备份域**（备份寄存器+备份 SRAM）
谁唤醒	任何使能的中断	EXTI（上升/下降沿）、RTC 闹钟、部分外设	WKUP 引脚（PA0）、RTC 闹钟、IWDG 复位、NRST
醒来到哪	WFI 的下一条指令	WFI 的下一条指令（但 HSI 接管时钟）	**复位入口**（像重新上电）
唤醒延迟	μs 级	μs 级（时钟要重启）	ms 级（调压器重启+复位）
电流量级	接近运行（mA）	μA ~ 几十 μA	μA ~ nA
核心寄存器	SCB->SCR.SLEEPDEEP=0	SCR.SLEEPDEEP=1 + PWR_CR.PDDS=0	SCR.SLEEPDEEP=1 + PWR_CR.PDDS=1

读这张表的方法是"看代价"：Sleep 最浅（什么都没忘，但也没省多少）；Standby 最深（SRAM 全丢，醒来像重生，但电流最低）。**选哪档取决于"醒来还需要记得什么"**：要无缝继续 → Stop；可以接受重置 → Standby。

对照阅读：ESP32-S3 的低功耗策略在 P11，架构不同但取舍相似：STM32 靠硬件模式切换，ESP32 靠 light/deep sleep + ULP 协处理器守夜。两条线对照看，低功耗设计的"停什么/留什么/谁唤醒"三问是通用的。



低功耗三档：Sleep 只停 CPU，Stop 停时钟保 SRAM，Standby 全停且唤醒相当于复位

二、Sleep 与 WFI/WFE：一条指令的事

Sleep 是最轻的，只停 CPU 时钟，外设全活：

```
#include "stm32f407.h"   /* CMSIS：SCB + __WFI/__WFE 内建 */

static void enter_sleep(void)
{
    SCB->SCR &= ~SCB_SCR_SLEEPDEEP_Msk;     /* SLEEPDEEP=0 → Sleep（不是 Stop） */
    __WFI();                                /* Wait For Interrupt：CPU 停在这里 */
    /* 任何使能的中断触发后，CPU 在这里醒来，执行 ISR 后回到 WFI 下一条 */
}
```

`__WFI()` 是一条 ARM 指令（CMSIS-Core 内建），CPU 停在这里直到中断到来。Sleep 的用途是"等事件"，主循环没事干时进 Sleep，比 `while(1){}` 死循环省 CPU 那一份动态功耗。唤醒后**时钟没变**（PLL 还在跑），不用重配。

WFE（Wait For Event）是变体：靠事件而非中断唤醒，可以用 `SEV` 指令跨核发事件（双核场景）。单核裸机绝大多数用 WFI。

三、Stop 全流程：入睡前清单 + 醒来重配 PLL

Stop 比 Sleep 深一档：1.2V 域时钟全停，但 SRAM 与寄存器保留。代价是醒来后**HSI 16MHz 默认接管**（PLL 在 Stop 期间也停了），系统时钟从 168MHz 掉到 16MHz。倍率是 `168/16 = 10.5`，不是"十倍左右"，`probe.sh` 第 1 步实测：1 ms 的装载值运行档是 `167999`、醒来档是 `15999`；`167999` 不改就带着它回 16MHz，真实周期变成 **10500 μs**，也就是"以为 1 ms，量出来 10.5 ms"。

```
static void enter_stop(void)
{
    /* 入睡前清单 */
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN_Msk;   /* 确保 GPIO 时钟开着才能改模式 */
    /* 1. 抠电：未用 GPIO 设模拟输入（见第六节清单） */
    /* 2. 关掉不需要的外设时钟（ADC、SPI 等若不用） */
    /* 3. 可选：Flash 在 Stop 下掉电（FPDS=1），更省一点 */
    PWR->CR |= PWR_CR_FPDS_Msk;

    /* 4. 选 Stop 模式：SLEEPDEEP=1 + PDDS=0 + 调压器低功耗 */
    SCB->SCR |= SCB_SCR_SLEEPDEEP_Msk;        /* SLEEPDEEP=1 → 进深睡 */
    PWR->CR &= ~PWR_CR_PDDS_Msk;              /* PDDS=0 → Stop（不是 Standby） */
    PWR->CR |= PWR_CR_LPDS_Msk;               /* LPDS=1：调压器进低功耗档 */

    __WFI();                                  /* 进 Stop——电流跳水到 μA */

    /* 5. 醒来：HSI 16MHz 接管，必须立刻重配 PLL */
    SystemClock_Config();                      /* 恢复 PLL 168MHz（[S2](02-rcc-clock.md) 的配置函数） */
    /* 醒来后第一件事检查 PWR_CSR.WUF（bit0）：是哪个唤醒源叫醒的 */
    if (PWR->CSR & PWR_CSR_WUF_Msk) {
        PWR->CR |= PWR_CR_CWUF_Msk;           /* 清 WUF：写 CWUF（bit2）清 */
    }
}
```

唤醒源必须在**入睡前**配好：EXTI 配上升沿（S4）、RTC 闹钟（第四节）。Stop 唤醒是"继续"。WFI 下一条指令，但若忘了重配 PLL，程序会以为还在 168MHz，所有定时器慢 10.5 倍，串口更直接： `BRR=729` 是按 `PCLK2=84MHz` 算出来的 115226.0 baud（误差 +0.023%），醒来 `PCLK2` 掉到 8MHz 后那个 729 变成**实际打出去 10973 baud**。对面那台终端一个字节都收不到。这两笔账都不是崩溃，是"看起来在跑、数全错"，典型的"醒来变痴呆"。

四、RTC 闹钟：周期唤醒的精准节拍

RTC 是 Stop/Standby 下唯一还在走的外设（靠 LSE/LSI 独立时钟），所以是周期唤醒的首选。配置链路（RTC 在备份域，写前要 DBP）：

```
static void rtc_alarm_init_for_wakeup(uint32_t seconds)
{
    RCC->APB1ENR |= RCC_APB1ENR_PWREN_Msk;
    PWR->CR |= PWR_CR_DBP_Msk;                /* 解除备份域写保护（动 RTC 必备） */
    /* 时钟源选 LSE（外部 32.768kHz 晶振，精度高）或 LSI（内部 RC，省一个晶振但精度差） */
    RCC->BDCR |= RCC_BDCR_RTCEN_Msk;          /* 使能 RTC（RCC_BDCR bit15） */
    /* 选时钟源、配置 RTC 预分频与闹钟寄存器——细节以 RM0090 RTC 章为准 */
    /* 闹钟中断经 RTC_Alarm_IRQn → EXTI Line 17 唤醒（[S4](04-nvic-exti.md) 的 EXTI 路由） */
    /* 设闹钟 = 当前时间 + seconds，周期唤醒由"闹钟中断里重设下一次"实现 */
}
```

LSI vs LSE 的取舍直接关系续航：**LSE**（外部 32.768kHz）精度高（±20ppm）、功耗极低，是 RTC 标配；**LSI**（内部 RC，约 32kHz）省一个外部晶振但精度差（±5%）、随温度漂，长期定时（闹钟日历）必须 LSE，短时唤醒（"每 2 秒醒一次"）LSI 够用。RTC 闹钟唤醒经 EXTI Line 17（S4），所以"RTC 唤醒 Stop"在硬件层面就是"EXTI 17 上升沿"。

五、Standby 与备份域：只留遗言的深度睡眠

Standby 是最深的，调压器关掉，1.2V 域断电，SRAM 全丢。醒来**不是继续**，是**复位**（PC 从 0 起跑，像重新上电）。唯一活下来的是备份域：备份寄存器（20 个 32 位字，RTC 寄存器区）+ 备份 SRAM（4KB）。

```
static void enter_standby_with_message(const uint32_t *msg, uint32_t n)
{
    /* 1. 把"睡前遗言"写进备份 SRAM（Standby 后还在）——四笔解锁，一笔都不能少 */
    RCC->APB1ENR |= RCC_APB1ENR_PWREN_Msk;          /* bit28：不开 PWR 时钟，后面全白写 */
    PWR->CR  |= PWR_CR_DBP_Msk;                     /* bit8 ：解除备份域写保护          */
    PWR->CSR |= PWR_CSR_BRE_Msk;                    /* bit9 ：开备份调压器（Standby 下给备份域供电）*/
    RCC->AHB1ENR |= RCC_AHB1ENR_BKPSRAMEN_Msk;      /* bit18：开备份 SRAM 时钟          */
    uint32_t *bkp = (uint32_t *)BKPSRAM_BASE;       /* 0x40024000，4KB = 1024 个字 */
    for (uint32_t i = 0; i < n && i < 1024; i++) bkp[i] = msg[i];

    /* 2. 清旧标志、配唤醒源 */
    PWR->CR |= PWR_CR_CWUF_Msk | PWR_CR_CSBF_Msk;  /* 清 WUF/SBF */
    PWR->CSR |= PWR_CSR_EWUP_Msk;                  /* 使能 WKUP 引脚（PA0） */

    /* 3. 进 Standby：SLEEPDEEP=1 + PDDS=1 */
    SCB->SCR |= SCB_SCR_SLEEPDEEP_Msk;
    PWR->CR |= PWR_CR_PDDS_Msk;                    /* PDDS=1 → Standby */
    __WFI();                                       /* 进 Standby——不会返回 */

    /* 醒来 = 复位：从这里开始重来，靠 SBF 判断"我是不是刚从 Standby 醒来" */
}
```

这四位里最容易被漏的是 `BRE`（bit9，备份调压器使能）：`DBP` 管的是"写保护"，`BRE` 管的是"给备份域供电"，两件事。Standby 期间主调压器关掉，备份域靠这个独立的备份调压器活着。没开它，写进 `BKPSRAM` 的东西醒来一定不在。`probe.sh` 的写法是把这三关做成一个函数，逐关返回码可查（实测：`DBP=0` 就写 → **−1**，`DBP` 有但 `BRE` 没开 → **−2**，`n > 1024` → **−3**，三关全过 → **0**，且前两关失败时 `f_bkpsram[0]` 断言仍为 0——**写进去的确实是空气**）。对应的就绪位是 `PWR_CSR.BRR`（bit3）：`BRE` 是使能，`BRR` 是回执，探针里两遍调用实测出"第一遍回执 1（没就绪）、置上 `BRR` 之后第二遍回执 0（就绪）"。**动备份域之前等的是 BRR，不是 BRE**。

顺带一提，`bkp_store()` 那种"越界就整笔拒掉（−3）"而不是"写到 1024 个字为止"的取舍：遗言写一半比写不下更糟，因为醒来那份状态看起来是完整的。

复位后怎么知道"是上电"还是"刚从 Standby 醒来"？查 PWR_CSR.SBF（bit1）：

```
/* 在 reset handler 或 main 早期 */
if (PWR->CSR & PWR_CSR_SBF_Msk) {
    /* 是从 Standby 醒来，备份 SRAM 里的遗言还在 */
    PWR->CR |= PWR_CR_CSBF_Msk;     /* 清 SBF，下次上电就不会误判 */
    /* 读 BKPSRAM 取遗言 */
} else {
    /* 真上电，备份 SRAM 是 0 或上次值——按上电处理 */
}
```

"醒来是复位"是 Standby 与 Stop 的本质区别：Stop 醒来继续跑（只需重配时钟），Standby 醒来重生（要从备份域捡回状态）。设计取舍很直接：要无缝继续 → Stop；可以接受重置且要极致省电 → Standby。

但 `SBF` 只回答"是不是从 Standby 醒的"，不回答"这一笔是谁给的复位"。真上电、看门狗咬、软复位、欠压，在代码看来都是"我从 0 起跑"，分辨它们靠 `RCC_CSR` 高位那七个标志。`probe.sh` 把解码函数喂了五组实测标志位（第 7 节，断言全绿）：

现场	RCC_CSR	PWR_CSR	解码出的位图	读法
刚上电	`0x0C000000`	`0x00000000`	`0x0C`	POR + PIN（`NRST` 也被拉过）
Standby 醒来	`0x08000000`	`0x00000002`	`0x09`	`SBF` + `PORRSTF` ——Standby 的"醒"走的是上电复位那条序列，两个标志一起置起（口径以 RM0090 复位章为准）
看门狗咬的	`0x20000000`	`0x00000000`	`0x20`	`IWDGRSTF`，没有 SBF
软复位	`0x10000000`	`0x00000000`	`0x10`	`SFTRSTF`（NVIC 系统复位请求）
欠压 + BOR	`0x82000000`	`0x00000000`	`0x82`	`BORRSTF` + `LPWRRSTF`

读完必须写 `RCC_CSR.RMVF`（bit24）清标志：否则下一次开机照抄这一次的结论，把"上次是狗咬的"当成"这次也是"。探针里的动作断言只断言"固件发出了那个写"（假寄存器不会自己变，硬说硬件清了就是自欺）。

六、抠电清单：睡前的系统级准备

光进低功耗模式不够，"漏电"的元凶常常是没伺候好的外设。入睡前逐项检查：

1. **未用 GPIO 设模拟输入**：悬空的数字输入脚在漏电流上"喝电"，配成模拟输入（MODER=0b11）或确定上下拉，把漏电流通道关掉；

2. **关掉不需要的外设时钟**：ADC、SPI、USART 若不用，RCC 位清掉，外设静态功耗归零；
3. **关调试接口**：DBGMCU->CR 的 `DBG_SLEEP` / `DBG_STOP` / `DBG_STANDBY`（bit0/1/2）若置着，Stop/Standby 下调试器还吃电，实测功耗时**必须拔掉仿真器**并清这三位，否则电流虚高一个数量级。固件里就一句 `DBGMCU->CR &= ~(7U << 0)`，`-O2` 编出来是一条 `bic.w r3, r3, #7`（`probe.sh` 第 3 步实测，见第八节）；
4. **调压器档位**：Stop 下用 LPDS=1（低功耗档）；F4 的 VOS（电压调节输出）在运行时也能降档（PWR_CR.VOS，bit14），降一档省一档电，代价是最高频率受限（S2 里 VOS='0' 时 fHCLK 上限只有 144MHz）；
5. **Flash 掉电**：FPDS=1 让 Flash 在 Stop 下断电，省一点，代价是唤醒后第一次取指稍慢；
6. **IWDG 还在跑**：S5 提过 IWDG 靠 LSI 独立跑，Stop 期间也计数，若 Stop 时间超过 IWDG 超时，醒来就不是"唤醒"而是 IWDG 复位。要么睡前喂狗+算好超时，要么进 Stop 前关 IWDG（但关了就没了看门狗保护）。

第 6 项那笔账必须算，因为 `PR` 那个字段有个坑：**3 位宽 = 8 个码，但只有 7 档分频**。`probe.sh` 第 4 节把八个码全跑了一遍（LSI 按 32 kHz、`RLR=4095` 上限，本机实跑）：

PR	0	1	2	3	4	5	6	7
分频	/4	/8	/16	/32	/64	/128	/256	**/256**
超时 ms	511	1023	2047	4095	8190	16380	32760	**32760**
LSI ±5%	485~536	971~1074	1944~2149	3890~4299	7780~8599	15561~17199	31122~34398	同左

按"8 个码就是 $/4\times2^7=/512$"去想，会把最长超时算成 **65520 ms**（探针把这一版也当场算了一遍贴在旁边），正好是实际值的两倍，而 LSI 本身 ±5% 的漂移盖不住这个错误。写代码时 `div = 4 << pr` 是错的，要么查表、要么 `pr >= 6 ? 256 : (4 << pr)`（本工程用查表，`probe.sh` 断言 `PR=6` 与 `PR=7` 同一个分频）。

这张清单的全部项都能在 E06 实测里逐一开/关对比，每关一项量一次电流，量化每一项的省电贡献。

七、SPL 对照：`PWR_EnterSTOPMode` 的参数

SPL 把"进 Stop"包成一个函数（stm32f4xx_pwr.c V1.8.0）：

```
void PWR_EnterSTOPMode(uint32_t PWR_Regulator, uint8_t PWR_STOPEntry)
{
    /* PWR_Regulator: PWR_Regulator_LowPower → LPDS=1；PWR_Regulator_ON → LPDS=0 */
    /* PWR_STOPEntry: PWR_STOPEntry_WFI / PWR_STOPEntry_WFE */
    tmpreg = PWR->CR;
    if (PWR_Regulator == PWR_Regulator_LowPower) tmpreg |= PWR_CR_LPDS;
    else tmpreg &= ~PWR_CR_LPDS;
    PWR->CR = tmpreg;
    SCB->SCR |= SCB_SCR_SLEEPDEEP_Msk;       /* 第三节逐行版 */
    if (PWR_STOPEntry == PWR_STOPEntry_WFI) __WFI();
    else __WFE();
    /* 醒来后 SPL 不替你重配 PLL——这是用户的责任（坑列表第一名） */
}
```

看一眼就明白：SPL 的 `PWR_EnterSTOPMode` 就是第三节的几行寄存器序列，参数 `PWR_Regulator` 选调压器档位、`PWR_STOPEntry` 选 WFI 还是 WFE。**SPL 不替你重配 PLL**：醒来后时钟恢复仍是用户责任（这是为什么"Stop 醒来变痴呆"是 SPL 用户和寄存器用户共同的坑）。`PWR_EnterSTANDBYMode` 同理：设 PDDS + SLEEPDEEP + WFI，不返回。

读库姿势照旧（S15）：拿到 `PWR_EnterSTOPMode` 先问"它动了 CR 与 SCR 的哪几位"，逐位与第三节对答案，低功耗的"魔法"全在 PDDS/LPDS/SLEEPDEEP 这三个位上，库没多做什么。

八、`probe.sh`：把这一章的账全部搬到宿主机上跑

手法与 S2 那一章相同：`probe.c` 先把 `PWR_CR` / `PWR_CSR` / `RCC_APB1ENR` / `RCC_AHB1ENR` / `RCC_BDCR` / `RCC_CSR` / `SCB_SCR` / `DBGMCU_CR` / `SysTick_LOAD` 九个宏改指到一组 `static volatile uint32_t` 假寄存器（`BKPSRAM_BASE` 改指一个 1024 字的数组），再 `#define MCU_PROBE_NO_MAIN` + `#include "main.c"`。于是每条断言跑的都是**固件本体**：`pwr_mode_words` / `enter_sleep` / `enter_stop` / `enter_standby` / `backup_domain_unlock` / `bkp_store` / `wake_source_armed` / `iwdg_timeout_ms` / `usart_baud_from_brr` / `reset_cause_decode` / `firmware_main`。

有一条纪律值得单独说：假寄存器**不会自己变**。所以凡是"硬件会清掉标志 / 硬件会就绪"的地方，探针只断言"固件发出了那个动作"（写 `CWUF`、写 `RMVF`），绝不冒充硬件的回执。

本机实跑（x86 gcc 16.1.0 + xPack ARM 15.2.1，`sh probe.sh`，三步全绿退出 0）。九个探针小节，结论都已落在正文对应那节，这里只留一张"哪一节被哪一遍跑过"的对照，不重复议价：

探针小节	结论在哪	探针小节	结论在哪
§1 三档定档	第三节	§6 唤醒源矩阵	第四节
§2 位号表（29 条）	本节第 2 步	§7 复位原因解码	第五节
§3 醒来时钟账	第三节	§8 固件本体	本节下段
§4 IWDG 八码	第六节	§9 地址表（10 个）	本节第 2 步
§5 备份域四笔	第五节	三档机身指令	本节第 3 步

只有 §8 那几个数是正文别处没出现的：把 `firmware_main()` 整跑一遍，得到 `CR=0x4205`、`SCR=0x4`、`DBGMCU_CR=0x30`（`0x37 & ~0x07`，其余位保留）、`SysTick_LOAD=0x3E7F`；这一遍发了 `CWUF`、没发 `CSBF`。把现场换成 `SBF=1` 再跑：发了 `CSBF`、没发 `CWUF`。单独调三个 `enter_*`：`SLEEPDEEP` 依次 0/1/1、`PDDS` 依次 0/0/1，并且 **Standby 那条路径不碰 SysTick 装载值**，醒来是复位，那个变量早就不存在了。

第 2 步：位号和地址都不是我说的，是头文件说的

`probe.sh` 的第 2 步先从探针输出里挖出那两块表，再回 CMSIS 挖对照组：位号一侧 `grep` 出全部 `#define XXX_Pos`；地址一侧顺着 `PERIPH_BASE → AHB1PERIPH_BASE → RCC_BASE`、`SCS_BASE → SCB_BASE / SysTick_BASE` 把这些宏里的 `+ 0x...UL` **一段段现挖**再相加，成员偏移则从结构体定义的 `Address offset:` 注释里取。`APB1PERIPH_BASE` 那条脚本会先断言它确实等于 `PERIPH_BASE`（没有隐藏偏移），否则 `PWR_BASE` 那条链就白算。两侧 `diff` 必须为空：

```
位号对账：29 条逐位一致（diff 为空）
地址对账：10 个全中（diff 为空）
  PWR_CR 0x40007000        ← 左：CMSIS 拼的    右：固件那一遍打印的
  SCB_SCR 0xE000ED10       ← 内核外设，不在 STM32 地址空间里
  SysTick_LOAD 0xE000E014
  DBGMCU_CR 0xE0042004
  BKPSRAM 0x40024000
```

第 3 步：三档睡眠在指令流里长什么样

`main.c` 用板上那套参数编一遍（`-mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard -std=c11 -Wall -Wextra -Werror -O2 -c`，只编不链），`size` 回 **text 796 / data 4 / bss 4**。三档的机身分别是（完整机身见 `probe.sh` 第 3 步输出）：

```
enter_sleep :  ldr r0,[PWR_CR] → str r0,[PWR_CR]      ← 读出来原样写回去，中间一条修改都没有
               ldr r3,[SCB+0xd10] → bic r3,#4 → str   ← 只清 SLEEPDEEP(bit2)
               dsb sy → wfi → bx lr                   ← wfi 之后：1 条指令 / 2 字节

enter_stop  :  bic r3,#2 → orr r3,#1                  ← PDDS=0、LPDS=1
               orr r1,#4 ([r2,#3344] = SCB_SCR)       ← SLEEPDEEP=1
               dsb sy → wfi
               ... str 16MHz 的装载值 → bx lr          ← wfi 之后：6 条指令 / 14 字节，会执行

enter_standby:  bic r3,#1 → orr r3,#2                   ← LPDS=0、PDDS=1
                orr r1,#4                               ← SLEEPDEEP=1
                dsb sy → wfi
                ... bl reset_cause_decode → pop         ← wfi 之后：7 条指令 / 18 字节，永不执行
```

`wfi` 之后剩多少条指令，就是三档的本质：**Sleep 直接返回；Stop 的那一段会执行（所以重配时钟的代码写在这里才有地方放）；Standby 的那一段永远不会执行**：编译器照样生成了它，18 个字节静静躺在 Flash 里。这一条是"醒来是复位而不是继续"最硬的机器层证据，也是本章为什么反复强调"状态要放备份域"：`enter_standby()` 里 `wfi` 之后那半截代码，包括它准备读复位标志的那条 `ldr`，一句都不会跑。

另外三处对账值得看一眼：

- `dsb sy`（`f3bf 8f4f`）与 `wfi`（`bf30`）在三档里都是**相邻两条**，中间没插任何指令，脚本用"上一条正则接下一条正则"断言相邻，不是简单两个 `grep`；
- `SCB->SCR` 在机身里是 `[r2, #3344]`，`r2 = 0xE000E000`，`3344 = 0xD10`，objdump 自己把偏移还原出来了。内核外设和芯片外设不是一张地址图，S2 里已经遇见过一次（`0xE000E000` 那个 SysTick）；同理 `enter_standby` 里读复位标志编成 `[r3, #2164]` 而 `r3 = 0x40023000`：源码写的 `RCC_BASE(0x40023800) + 0x74` 被编译器合成成一个偏移，`0x40023000 + 0x874 = 0x40023874`，**base 是源码里的口径，机身里只有 base+偏移那一个数**，所以对地址要像第 2 步那样从两头拼，不能只信一边；
- `backup_domain_unlock()` 编出来是五个 `orr` 加一条 `ubfx`：`#268435456`(PWREN bit28) / `#256`(DBP bit8) / `#512`(BRE bit9) / `#262144`(BKPSRAMEN bit18) / `#32768`(RTCEN bit15)，最后 `ubfx r0, r0, #3, #1` 取的就是 `BRR` 那一位；`firmware_main` 里查 `WUF` / `SBF` 编成 `lsls r1, r3, #31` 与 `lsls r2, r2, #30` + `bpl`，**把标志位搬到符号位再决定跳不跳**，和 S2 等 `HSERDY` 那一段用的是同一招。

附录：工程完整源码

不用跳 GitHub，整个 `main.c`、离线探针和取证脚本都在下面，滚动即达：

<<< ../../code/stm32/14-pwr/main.c

<<< ../../code/stm32/14-pwr/probe.c

<<< ../../code/stm32/14-pwr/probe.sh

PWR/BKPSRAM/RTC 的位定义与地址一律对照 CMSIS（本站 `.trellis/ref/cmsis/`），第 2 步就是那场对账；电流实测见 E06。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **Sleep 停脑，Stop 停钟（记忆在），Standby 断电（只留遗言）；唤醒源睡前配，醒来先查钟。**

延伸：低功耗三档对比动画见 S14 动画；Stop 模式唤醒后时钟恢复在 S2 RCC 有完整流程；ESP32 睡眠矩阵对照见 P11。

实物实验

- E06 功耗实测：三种模式电流逐一记录，算出"1 节 CR2032 能活多久"的估算表。

常见坑

- **Stop 唤醒后忘重配时钟**：系统跑在 HSI 16MHz，定时全慢 10 倍，唤醒后第一件事恢复 PLL。慢多少不是"大概"：`systick_period_us10` 实测，带着 168MHz 的装载值 `167999` 回 16MHz，周期从 1000 µs 变成 **10500 µs**。
- **只重配 SysTick，不重配 USART**：第三节那笔账，`BRR=729` 是按 PCLK2=84MHz 算的，PCLK2 掉回 8MHz 后同一个 729 打出 **10973 baud**（差 5%），波特率寄存器也是"随时钟走"的账，醒来一起重算（应得 `BRR=69`）。
- **开备份域只想到 DBP，漏了 BRE**：DBP（`PWR_CR` bit8）是写保护开关，备份 SRAM 的**供电**开关是 `PWR_CSR.BRE`（bit9），两个东西、两个寄存器。探针实测：有 DBP、无 BRE 时 `bkp_store()` 返回 **−2**，写进去的内容数组里仍是 0，看着写成功，其实是空气。
- **等备份域就绪却等错标志**：`backup_domain_unlock()` 的回执是 `BRR`（bit3），不是 `BRE`。第一遍读到 1（还没就绪）、置上 `BRR` 后第二遍才 0，睡前不等这一位，遗言就在断电前丢了。
- **把 `EWUP` 当"RTC 唤醒开关"**：`EWUP` 只管 PA0/EXTI **线 0**。RTC 闹钟是 EXTI 线 **17**、RTC 唤醒是 **18**，得走 `EXTI_IMR`。探针实测：只配 `EWUP` 再去查 17 号线 → **−1**，照样睡死。
- **IWDG 分频按"2 的幂续上去"想当然**：`PR` 是 8 个码但只有 7 档分频，`PR=6` 与 `PR=7` **同为 /256**。按 /512 算，`RLR=4095` 的最长超时从 **32760 ms** 被高估成 65520 ms——你以为狗还能撑一分多钟，它半分钟就咬。
- **调试器插着测电流**：SWD 连接让 Stop 电流虚高一个数量级，测功耗时拔掉仿真器，并把 `DBGMCU_CR` 的低三位清掉（固件里那一笔编出来是 `bic.w r3, r3, #7`）。
- **GPIO 悬空耗电**：悬空输入在漏电流上"喝电"，睡前全配模拟输入或确定上下拉。
- **Standby 后找变量**：SRAM 已清空，"睡前变量"必须用备份寄存器/备份 SRAM。`enter_standby()` 里 `wfi` 之后那 18 个字节，编译器生成了、Flash 里躺着、一句不会跑。
- **IWDG 在 Stop 里还在数**：超时就是 IWDG 复位，不是"醒来"，睡前算好超时或关 IWDG。
- **读完复位原因不清 `RMVF`**：`RCC_CSR` 那七个标志不清，下一次开机照抄这一次的结论（第五节表）。

短自测

1. Stop 和 Standby 醒来的"到哪"有什么本质区别？为什么 Standby 后变量全丢？

> **▼ 参考答案**
> Stop 醒来回到 WFI 的下一条指令（继续跑），SRAM 与寄存器都保留，只是时钟停了得重配。Standby 醒来回到**复位入口**（像重新上电），因为调压器关了、1.2V 域断电、SRAM 内容全失。Standby 后变量全丢正是因为 SRAM 没有供电保留，只有备份域（备份寄存器+备份 SRAM）还活着，要留状态就得睡前写进备份域。

2. 为什么 Stop 唤醒后系统时钟是 HSI 16MHz 而不是 168MHz？醒来后第一件事该做什么？

> **▼ 参考答案**
> Stop 期间 1.2V 域时钟全停，包括 PLL（PLL 靠 HSE/HSI 倍频，停了就不出 168MHz）。醒来硬件自动启 HSI 作为系统时钟（保证最低能跑），但 PLL 没自动重启。第一件事是调 SystemClock_Config 重新配 PLL 切回 168MHz，否则所有定时器、UART 波特率都按 16MHz 算，全错 10 倍。

3. 怎么在复位后区分"真上电"和"刚从 Standby 醒来"？

> **▼ 参考答案**
> 查 PWR_CSR.SBF（bit1，Standby Flag）。硬件在 Standby 唤醒时置 SBF，上电时 SBF=0。读 SBF=1 就是"刚从 Standby 醒来"，可以从备份 SRAM 捡回遗言；读 SBF=0 就是真上电。处理后写 PWR_CR.CSBF 清标志，避免下次上电误判。

4. LSI 和 LSE 给 RTC 用，精度与续航怎么取舍？

> **▼ 参考答案**
> LSE（外部 32.768kHz 晶振）精度高（±20ppm）、功耗极低，是 RTC 标配，长期定时（日历、闹钟）必须用它。LSI（内部 RC，约 32kHz）省一个外部晶振、BOM 成本

低，但精度差（±5%）且随温度漂，短时周期唤醒（"每 2 秒醒一次"）够用，长期累计会偏。续航上两者都很省（μA 级），主要差别是精度而非功耗。

5. 测 Stop 电流时为什么要拔掉 ST-Link？

▼ 参考答案
ST-Link 通过 SWD 连着芯片，DBGMCU->CR 的 DBG_STOP 位若置着，Stop 下调试器还维持着调试接口的供电与时钟，电流虚高一个数量级。即便清了 DBG_STOP，SWD 物理连接本身的漏电也会引入误差。测真实 Stop 电流必须拔掉仿真器、清 DBG_STOP/DBG_STANDBY，让芯片真正"独自睡眠"。

下面这三轮直接用本机 `probe.sh` 的实跑数提问：三档定出的寄存器字、醒来漏算的那笔账、看门狗最长能撑多久。

🎯 S14 低功耗自测

1. **Stop 与 Standby 在寄存器层的差别，说到底动了哪两位？**
   A. PWR_CR.LPDS 与 PWR_CR.FPDS
   B. SCB->SCR.SLEEPDEEP 与 PWR_CR.PDDS
   C. PWR_CSR.SBF 与 PWR_CSR.WUF
   D. RCC_CSR.DEEPSLEEPRESETF 与 PWR_CR.VOS

2. **Stop 醒来把 SysTick 恢复成 168MHz 的装载值，但 USART 的 BRR 留着没重算（PCLK2 此刻还是 8MHz）。实际打出去的波特率是多少？**
   A. 仍是 115200，BRR 与时钟无关
   B. 115226 baud，误差千分之二，能通
   C. 10973 baud，差 5%，对端收到的是帧错
   D. 0 baud，外设直接停摆

3. **BKPSRAM 写入前只开了 DBP、没开 PWR_CSR.BRE。probe.sh 实测 bkp_store() 的返回值和数组内容是？**
   A. 返回 0，数组写进去了，但断电会丢
   B. 返回 −2，数组仍是 0——看着写成功，其实是空气
   C. 返回 −1，说明 DBP 也没生效
   D. 硬卡死在写总线上

4. **想让 RTC 闹钟（EXTI 线 17）唤醒 Stop，只把 PWR_CSR.EWUP 置 1 够不够？**
   A. 够，EWUP 就是深度睡眠唤醒总开关
   B. 不够，还要额外置 RCC_BDCR.RTCEN 才算唤醒使能
   C. 不够，EWUP 只管 PA0/EXTI 线 0；线 17 得写 EXTI_IMR
   D. 够，但必须先关掉 SysTick 才能用 RTC 唤醒

5. **IWDG 的 PR 有 8 个编码值，RLR=4095、LSI=32kHz 时最长超时是多少？按『每个码再翻一倍』推到 /512 会算成多少？**
   A. 32760 ms；想当然会算成 65520 ms——高估整整两倍
   B. 65520 ms；PR=7 就是 /512
   C. 16380 ms；PR=7 与 PR=5 同档
   D. 8190 ms；第八个码保留不可用

6. **反汇编 enter_standby()，wfi 之后还有 7 条指令 / 18 个字节。这说明了什么？**
   A. 编译器没优化好，应该删掉
   B. 唤醒后依次执行这 7 条，所以重配时钟的代码可以写在这里
   C. 这段代码永远不执行：Standby 的醒法是复位，18 字节静静躺在 Flash 里
   D. 这 7 条由硬件在唤醒前代为执行

先选完再翻：答案与逐项解析在附录 C 的本组；在线版可点击即时判分。

对照表：本章概念 → 仓库与上游落点

概念	落点
PWR_CR/CSR 全部位号（LPDS/PDDS/CWUF/CSBF/DBP/FPDS/VOS/WUF/SBF/BRE/BRR/EWUP）	CMSIS `stm32f407xx.h`（本站 `.trellis/ref/cmsis/`）；与 `_Pos` 的逐条对账 = `probe.sh` 第 2 步
`SCB->SCR.SLEEPDEEP`	CMSIS-Core `core_cm4.h`（bit2 @ `0xE000ED10`）；地址同表对账
三档定档写出的那两个位	`probe.c` §1 → 第三节
醒来时钟账（SysTick 周期、USART `BRR`）	`systick_period_us10` / `usart_baud_from_brr`，`probe.c` §3 → 第三节
IWDG 八码七分频与超时区间	`iwdg_pr_div` / `iwdg_timeout_ms`，`probe.c` §4 → 第六节
备份域四种回执与 `BRR` 两遍解锁	`bkp_store` / `backup_domain_unlock`，`probe.c` §5 → 第五节
唤醒源矩阵（`EWUP` / `EXTI_IMR`）	`wake_source_armed`，`probe.c` §6 → 第四节
复位原因七标志解码	`reset_cause_decode`，`probe.c` §7 → 第五节
三档机身与 `wfi` 之后的指令数	`probe.sh` 第 3 步（`arm-none-eabi-gcc -O2 -c` + `objdump -d`）→ 第八节
备份 SRAM 4KB @ `0x40024000`、RTC 基址、`DBGMCU_CR`	`probe.sh` 第 2 步地址表（10 个地址现场重拼）
RTC 闹钟/唤醒的 EXTI 线号路由	S4 EXTI
三档电流实测	E06 低功耗电流实测
PLL 重配函数	S2 RCC SystemClock_Config

你做到了

- 低功耗从"省电模式"三个字变成可设计、可测量、可核算的工程；
- 电池产品的续航估算有了第一性原理。

✅ 下一站：S15 SPL 库解剖：拆开标准库，看它怎样把我们手写过的每一行包进函数。

通往单片机之路第 33 章

S15 SPL 标准库解剖：库是怎么封装寄存器的

难度 ★★☆☆☆ · 约 35 分钟

本章罗盘

问题

GPIO_Init 拆开还是不是我们手写的那几行？SPL 枚举值为什么直接就是寄存器位段的值？

交付

一份 SPL 三板斧拆解（结构体打包/位操作翻译/断言看门）加 GPIO_Init 逐行与手写版的配对表，和一张「何时手写何时用库」的判断框架（初始化用库、热路径手写）。

边界

解剖对象是 SPL V1.8.0，HAL 只有哲学对比、逐行对照已分散在各章的库解析小节；体积/开销的具体数字指向 B5 的 map 审计方法，本章不实测；库行号以你手头的分发为准。

依据锚点 `f5 · f3 · a1 · mcu-code · mcu-anim`

> 🎯 前面 14 章我们手写寄存器；这一章把 SPL（标准外设库）放上解剖台：`GPIO_Init` 一个函数，里面就是我们手写的"先时钟、再模式、后数据"。读懂库的封装术，你就拥有了"用库而不被库绑死"的自由。

本章精髓

1. 库 = 结构体 + 查表 + 断言：`GPIO_InitTypeDef` 把四张寄存器表打包成四个字段；`GPIO_Init` 逐字段翻译成位操作；`assert_param` 在 Debug 版替你查参数。
2. 封装的三层价值：防错（参数检查/位序查表）、可读（`GPIO_Mode_OUT` vs `0x01`）、可移植（同 API 跨 F1/F4），代价是代码体积与运行开销（B5 审计它）。
3. 读库的正确姿势：**从 API 反推寄存器**，遇到任何库函数，先问"它动了哪几个寄存器的哪几位"，用 RM0090 对答案。

- **能理解**：为什么 `GPIO_Init` 要逐 pin 循环（一次配多个引脚）、为什么用"先清后置"而不是直接赋值（保留别的引脚的配置）。
- **能用**：拿到任何 SPL 函数（GPIO_Init/USART_Init/I2C_Init），用本章的"API→寄存器反推法"拆开它。

先修

- S3 GPIO、S2 RCC；C4 结构体（库的根基）。

先跑起来（10 分钟 quick win）

打开任意 SPL 工程的 `stm32f4xx_gpio.c`，找到 `GPIO_Init`，对照 S3 你的手写版，逐段配对（MODER 段→你的 MODER 行），10 分钟完成"相认"。

相认之前先押个体积：同一段功能，SPL 版和你手写版做出来，谁占的地方更大？差不多、差几倍，还是库反而更省？押完往下读，库与手写的三轴实测会给出量过的答案。

动画：GPIO_Init 解剖

结构体打包 → 逐 pin 循环 → 位偏移计算 → 四段寄存器写入，库的封装术全在这四步里。

GPIO_Init 解剖动画

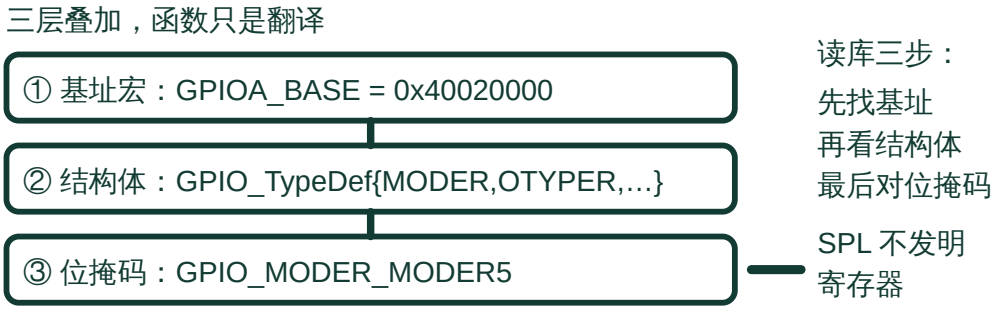
本节对照源

本章 SPL 引用以 **stm32f4xx_gpio.c / stm32f4xx_rcc.c V1.8.0** 为准（行号以你手头的库分发核对）；寄存器结构与位掩码以本站 CMSIS 设备头 `stm32f407.h` 为底（`.trellis/ref/cmsis/`，逐位核对）。

小节结构

小节	内容	四件套
头文件三板斧	stm32f4xx.h 的基址宏/结构体/位掩码体系	库解析
GPIO_Init 逐行	四段寄存器写法的位操作翻译	库解析
时钟使能的读回	RCC_AHB1PeriphClockCmd 的 `__DSB()` /读回细节	库解析
assert_param	断言宏的开关与调试价值	代码分析
库 vs 手写	体积/开销/可读性三轴对比（用 B5 方法实测）	代码分析
从库反推手册	方法演练：给 USART_Init，反推 CR1/CR2/BRR 全落位	库解析

一、头文件三板斧：基址宏、结构体、位掩码



SPL 三层基础设施：基址宏给名字、结构体收字段、位掩码给位号；函数只是把三层翻译成寄存器写

SPL 读懂任何函数之前，先读懂 `stm32f4xx.h` 的三层基础设施，它们在 CMSIS 设备头里逐字可见（本站 `.trellis/ref/cmsis/stm32f407.h`）：

第一板斧：基址宏：把"地址"变成"名字"：

```
#define AHB1PERIPH_BASE  (PERIPH_BASE + 0x00020000UL)   /* = 0x40020000 */
#define GPIOA_BASE       (AHB1PERIPH_BASE + 0x0000UL)  /* = 0x40020000 */
#define GPIOA            ((GPIO_TypeDef *) GPIOA_BASE)
```

`GPIOA` 就是一个指向 0x40020000 的结构体指针，`GPIOA->MODER` 在编译期就是 `*(uint32_t*)0x40020000`。**所有"寄存器访问"在 C 层面都是结构体成员解引用**，没有魔法。

第二板斧：结构体：把一串地址掰成命名字段（CMSIS 已核对偏移）：

```
typedef struct {
  __IO uint32_t MODER;    /* 0x00：模式（2 位/引脚 × 16 引脚） */
  __IO uint32_t OTYPER;   /* 0x04：输出类型（1 位/引脚） */
  __IO uint32_t OSPEEDR;  /* 0x08：输出速度（2 位/引脚） */
  __IO uint32_t PUPDR;    /* 0x0C：上下拉（2 位/引脚） */
  __IO uint32_t IDR;      /* 0x10：输入数据（只读） */
  __IO uint32_t ODR;      /* 0x14：输出数据 */
  __IO uint32_t BSRR;     /* 0x18：位设置/复位（写 1 触发） */
  __IO uint32_t LCKR;     /* 0x1C：配置锁 */
  __IO uint32_t AFR[2];   /* 0x20-0x24：复用功能（8 位/4 引脚 × 2 组） */
} GPIO_TypeDef;
```

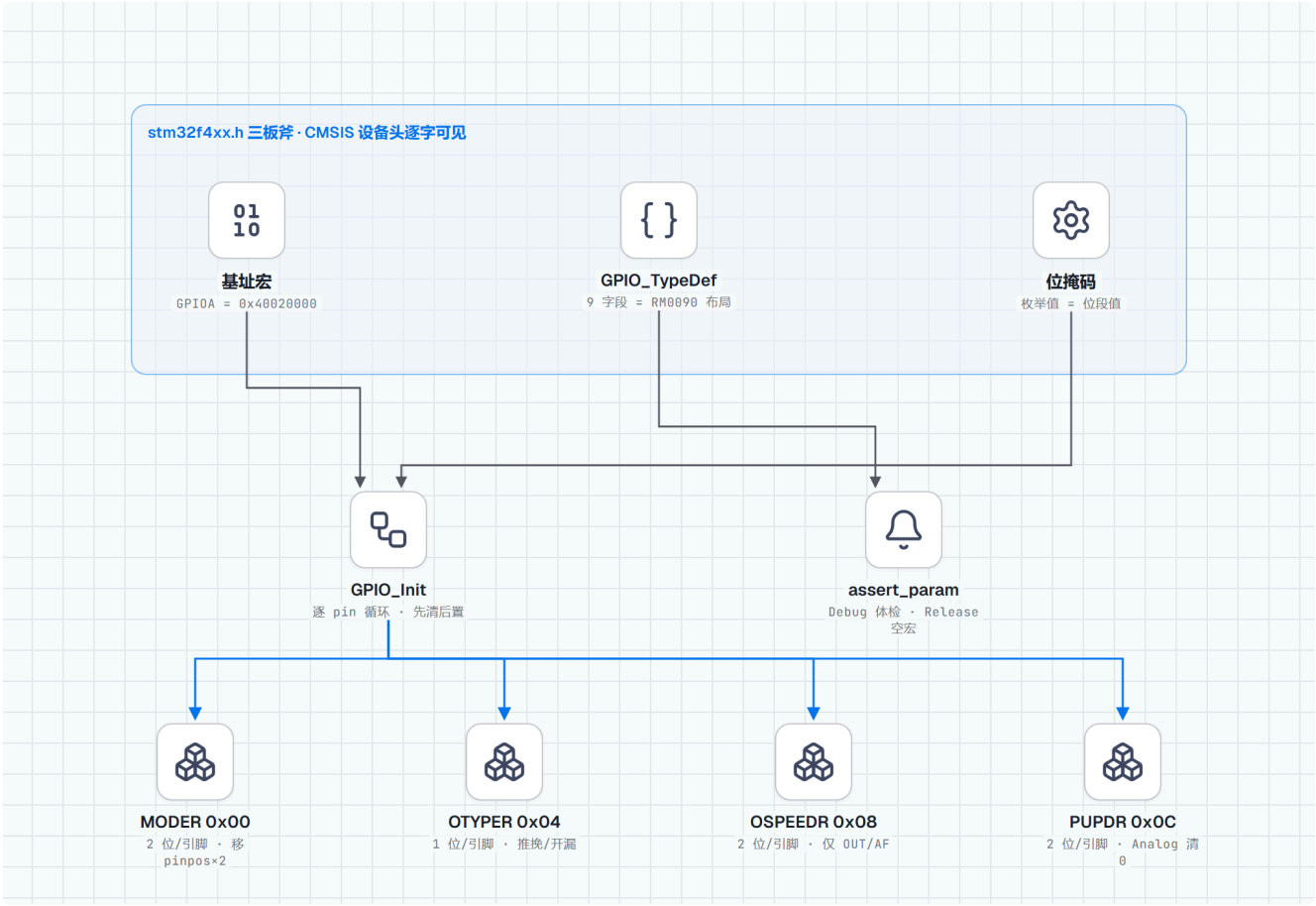
`__IO` 是 `volatile`，回 C3：寄存器访问必须 volatile，否则编译器把连续两次写合并。**结构体的字段顺序与偏移就是 RM0090 的寄存器布局**，所以"读 CMSIS 结构体 = 读 RM0090 寄存器表"。

第三板斧：位掩码：把"第 N 位的值"变成可读宏（CMSIS 已核对 `GPIO_MODER_MODER0_Msk = 0x3`）：

```
#define GPIO_MODER_MODER0_Msk   (0x3UL)              /* 2 位 */
#define GPIO_MODER_MODER0_Pos   (0U)
/* 模式枚举（值即寄存器位段值）：00=IN, 01=OUT, 10=AF, 11=Analog */
```

关键设计：**SPL 的枚举值 = 寄存器位段的实际值**：`GPIO_Mode_IN=0x00`、`GPIO_Mode_OUT=0x01`、`GPIO_Mode_AF=0x02`、`GPIO_Mode_AN=0x03`，直接就是 MODER 那两位的值。所以"`GPIO_Init` 翻译"本质是"把枚举值左移到对应引脚的位段"，没有查表，只有移位。

三板斧怎么喂进 `GPIO_Init`，又落到哪四段寄存器，一张图收干净：



SPL 解剖：基址宏、GPIO_TypeDef、位掩码三板斧供料给 GPIO_Init，落到 MODER/OTYPER/OSPEEDR/PUPDR 四段寄存器

二、GPIO_Init 逐行：四段寄存器的位操作翻译

`GPIO_Init` 的核心逻辑（V1.8.0 结构还原，行号以库源码为准）：逐 pin 循环，每个被选中的引脚走"清旧值 → 置新值"两步：

```
void GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef *init)
{
  uint32_t pinpos, pos, currentpin;
  /* —— assert_param 四连：参数体检（见第四节）—— */
  assert_param(IS_GPIO_ALL_PERIPH(GPIOx));
  assert_param(IS_GPIO_PIN(init->GPIO_Pin));
  assert_param(IS_GPIO_MODE(init->GPIO_Mode));
  /* ... PuPd / OType / Speed 同样断言 ... */

  for (pinpos = 0; pinpos < 16; pinpos++) {
    pos = ((uint32_t)0x01) << pinpos;             /* 1 << pinpos：当前引脚的位掩码 */
    currentpin = init->GPIO_Pin & pos;
    if (currentpin == pos) {                       /* 这个引脚在 GPIO_Pin 掩码里被选中 */
      /* ① MODER：2 位/引脚，先清后置 */
      GPIOx->MODER  &= ~(GPIO_MODER_MODER0_Msk << (pinpos * 2));
      GPIOx->MODER  |= ((uint32_t)init->GPIO_Mode << (pinpos * 2));

      if (init->GPIO_Mode == GPIO_Mode_OUT || init->GPIO_Mode == GPIO_Mode_AF) {
        /* ② OTYPER：1 位/引脚（推挽/开漏） */
        GPIOx->OTYPER &= ~((uint16_t)0x01 << pinpos);
        GPIOx->OTYPER |= (uint16_t)((uint16_t)init->GPIO_OType << pinpos);
        /* ③ OSPEEDR：2 位/引脚 */
        GPIOx->OSPEEDR &= ~(GPIO_OSPEEDR_OSPEEDR0_Msk << (pinpos * 2));
        GPIOx->OSPEEDR |= ((uint32_t)init->GPIO_Speed << (pinpos * 2));
      }

      /* ④ PUPDR：2 位/引脚（输入/输出都要配上下拉；Analog 清成 0） */
      GPIOx->PUPDR &= ~(GPIO_PUPDR_PUPDR0_Msk << (pinpos * 2));
      if (init->GPIO_Mode != GPIO_Mode_AN) {
        GPIOx->PUPDR |= ((uint32_t)init->GPIO_PuPd << (pinpos * 2));
      }
    }
  }
}
```

四个翻译要点，每个都能对照 S3 你手写的版本：

1. **为什么逐 pin 循环**：`GPIO_Pin` 是 16 位掩码（如 `GPIO_Pin_5 | GPIO_Pin_6`），一次调用可能配多个引脚。循环逐位检测"这个引脚在不在掩码里"，在就走翻译，这是"一次 API 配多引脚"的实现。
2. **为什么"先清后置"而不是直接赋值**：`MODER &= ~(mask << shift)` 先把这一位段清成 0，`|=` 再置新值。直接 `=` 会覆盖别的引脚的配置，这是"按位段操作"的基本功，C7 的位运算纪律在库里的体现。
3. **移位量 `pinpos * 2`**：MODER/OSPEEDR/PUPDR 是 2 位/引脚，所以移 `pinpos*2`；OTYPER 是 1 位/引脚，移 `pinpos`。**移位量由寄存器布局决定**，不是库作者拍脑袋。
4. **枚举值即位段值**：`init->GPIO_Mode` 直接左移到 MODER 位段，没有查表，因为 SPL 在头文件里就把枚举定义成了与硬件编码相同的值（`GPIO_Mode_OUT=0x01` 就是 MODER 的 01）。这是 SPL 设计的精髓：**让 API 值与硬件值零翻译**。

对照 S3 你手写的 `GPIOA->MODER |= (0x01 << (5*2))`，`GPIO_Init` 干的是同一件事，只是包了"循环多引脚 + 清旧值 + 断言"三层。**库没有魔法，只有更周全的边界处理。**

三、时钟使能的读回：一句话置位背后的延迟

`RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA, ENABLE)` 看起来就是 `RCC->AHB1ENR |= bit`，但实际更绕一圈（stm32f4xx_rcc.c）：

```
void RCC_AHB1PeriphClockCmd(uint32_t RCC_AHB1Periph, FunctionalState NewState)
{
  /* assert_param 略 */
  if (NewState != DISABLE) {
    RCC->AHB1ENR |= RCC_AHB1Periph;
  } else {
    RCC->AHB1ENR &= ~RCC_AHB1Periph;
  }
  /* 关键：写后补一次"读回"——防时钟生效延迟 */
  (void)RCC->AHB1ENR;   /* 或某些版本用 __DSB() */
}
```

为什么要读回：RM0090 §7 有一条"外设时钟使能后，访问外设寄存器前需等几个时钟周期生效"。如果刚 `|=` 完立刻访问外设，可能在时钟还没真正喂到外设的那几个周期里读/写落空，现象是"配置了但没生效"。**读回 `RCC->AHB1ENR` 强制一次总线往返**，等流水线把写落地再继续——这是 SPL 替你踩的"时钟生效延迟"坑。

手写寄存器版本（S3 及之前所有章节）的对应做法：开时钟后插一句 `(void)RCC->AHB1ENR;` 或 `__DSB();`。**这是库的"隐形价值"之一：把硬件手册里的小字条款编进函数体**，手写时容易漏。

四、assert_param：Debug 版的参数体检

SPL 每个函数开头一串 `assert_param(IS_GPIO_PIN(init->GPIO_Pin))`。`assert_param` 在 Release 版是空宏（编译期消失），Debug 版是"失败进死循环 + 打印位置"，和标准 `assert` 一个套路，但跨编译器、不依赖 libc。

```
#ifdef USE_FULL_ASSERT
  #define assert_param(expr) ((expr) ? (void)0 : assert_failed((uint8_t *)__FILE__, __LINE__))
#else
  #define assert_param(expr) ((void)0)   /* Release 版：零开销 */
#endif
```

`IS_GPIO_PIN(x)` 是一组参数检查宏（stm32f4xx_gpio.h）：

```
#define IS_GPIO_PIN(PIN) (((PIN) & GPIO_Pin_All) != (uint32_t)0x00)   /* 至少选了一个引脚 */
#define IS_GPIO_MODE(MODE) (((MODE) == GPIO_Mode_IN)  || ((MODE) == GPIO_Mode_OUT) || \
                            ((MODE) == GPIO_Mode_AF)  || ((MODE) == GPIO_Mode_AN))
```

价值：Debug 版开 `USE_FULL_ASSERT`，参数错（如 `GPIO_Pin = 0`、`GPIO_Mode = 0x05`）当场进 `assert_failed` 死循环，看调用栈就知道哪个调用方传错了。**Release 版编译期消失**，零运行开销，这是 SPL 把"调试价值"与"运行成本"分离的标准做法。

骨架坑列表里"assert 卡住当死机"就是没认出 `assert_failed` 死循环——它在喊"参数错了"，看调用栈别慌，不是硬件挂了。

五、库 vs 手写：体积、开销、可读性三轴

同一功能（配 PA5 为推挽输出）两版对比：

维度	手写寄存器	SPL 版
代码	`GPIOA->MODER = (GPIOA->MODER & ~(3<<10)) | (1<<10);` 等 4 行	`GPIO_InitTypeDef init = {GPIO_Pin_5, GPIO_Mode_OUT, GPIO_OType_PP, GPIO_Speed_2MHz, GPIO_PuPd_NOPULL}; GPIO_Init(GPIOA, &init);`
可读性	位操作密集，需对 RM0090	结构体字段自描述，新人 30 秒看懂
体积	4 条访存指令	函数调用 + 循环 + 断言，几十条指令（B5 实测）
运行开销	固定几周期	循环 16 次 + 分支，~百周期
防错	错位移/错值不报错	assert_param 拦参数错
可移植	F1/F4 寄存器布局不同，重写	同 API 跨 F1/F4（库适配）

判断框架很朴素：

- **初始化代码**（启动时跑一次，体积/开销不重要）→ **用库**：可读性 + 防错价值远超几周期开销；
- **中断处理/性能热路径**（每次中断都跑、纳秒级预算）→ **手写**：库的循环+分支开销吃不消，直接位操作；
- **学习阶段**（理解机制）→ **手写**：亲手配过一遍寄存器，再用库才知道库在替你做什么（这是 S3 到本章的路径）；
- **量产 → 混合**：初始化用库（可维护），热路径手写（性能），用 `-gc-sections`（B5）把没用到的库函数链接时剔除。

六、从库反推手册：USART_Init 方法演练

把"GPIO_Init 反推法"用在 `USART_Init` 上，拿到这个函数，先问"它动了哪几个寄存器的哪几位"，答案落在 CR1/CR2/CR3/BRR：

结构体字段	寄存器落位	值怎么来
`USART_BaudRate`	**BRR**	`BRR = PCLK / BAUD`（整数分频，S7 实算过）
`USART_WordLength`	**CR1.M**（bit12）	8 位 M=0、9 位 M=1
`USART_StopBits`	**CR2.STOP**（bits12-13）	1/0.5/2/1.5 编码
`USART_Parity`	**CR1.PCE**（bit10）+ **PS**（bit9）	使能 + 奇偶选择
`USART_Mode`	**CR1.TE**（bit3）+ **RE**（bit2）	TX 使能 / RX 使能
`USART_HardwareFlowControl`	**CR3.CTSE/RTSE**	CTS/RTS 流控
`USART_CLKInit` 的 CPOL/CPHA	**CR2.CPOL/CPHA**	同步时钟模式

`USART_Init` 的翻译逻辑（与 `GPIO_Init` 同构）：读 CR1/CR2/CR3，逐字段"先清后置"，最后写 BRR。**断言保证 BAUD 不超范围、WordLength 合法**，和 GPIO 的 assert_param 一个套路。

这就是"API → 寄存器反推法"的全部：**拿到任何库函数，列出它的结构体字段 → 查 RM0090 每个字段对应哪个寄存器位 → 对照库源码核对翻译逻辑**。HAL/LL/RT-Thread 驱动都只是换皮，结构体字段名变了、断言宏变了，但"结构体打包 + 位操作翻译 + 断言"三板斧不变。**学会这一刀，任何新库都能解剖。**

附录：本章对照源

- SPL 引用：stm32f4xx_gpio.c / stm32f4xx_rcc.c V1.8.0（行号以库分发为准）。
- 寄存器结构与位掩码：CMSIS `stm32f407.h`（本站 `.trellis/ref/cmsis/`，逐位核对）。
- 手写对照版：S3 GPIO 的寄存器级配置；E01 工程 `code/stm32/00-blink/main.c`。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **库=结构体打包+位操作翻译+断言看门；读库先问动了谁，API 反推寄存器，RM 永远是法官。**

延伸：GPIO_Init 解剖动画见 S15 动画；SPL 与 HAL 的设计哲学对比在 S1 架构 展开；GD32 固件库对照见 G0。

实物实验

- 把 E01 改成 SPL 版（GPIO_Init 三行），`size` 对比手写版体积差异；GDB 单步进 `GPIO_Init` 内部，看你熟悉的位操作逐条执行。

常见坑

- **库与芯片型号错配**：F10x 库编 F407 工程，头文件结构体全错位。
- **忘定义 USE_STDPERIPH_DRIVER / 器件宏**：编译出来的配置表是空的。
- **assert 卡住当死机**：Debug 断言失败进死循环，那是库在喊"参数错了"，看调用栈别慌。
- **无脑全库链接**：只用一个 GPIO 却把整个库编进去（B5：gc-sections 救场的前提是分节编译）。

短自测

1. 为什么 `GPIO_Init` 用"先清后置"而不是直接 `MODER = 新值` ？

> **▼ 参考答案**
> MODER 是 32 位、16 个引脚各占 2 位。直接 `=` 会覆盖其他 15 个引脚的配置，而一次 `GPIO_Init` 通常只配一个或几个引脚。"先清后置"（`&= ~(mask<<shift)` 再 `|= 新值<<shift`）只动目标引脚的位段，保留其他引脚的配置。这是按位段操作的基本功，[C7](../c/07-ub-misra.md) 的位运算纪律在库里的体现。

2. 为什么 SPL 的 `GPIO_Mode_OUT` 枚举值是 0x01 而不是随便一个数？

> **▼ 参考答案**
> SPL 把枚举值定义成与 MODER 寄存器位段值**相同**的编码（00=IN, 01=OUT, 10=AF, 11=AN）。这样 `GPIO_Init` 翻译时直接 `枚举值 << (pinpos*2)` 落到位段，没有查表，零翻译成本。这是 SPL 设计的精髓：让 API 值与硬件值同构，减少"翻译"环节出错的可能。

3. `RCC_AHB1PeriphClockCmd` 写完 `AHB1ENR |= bit` 后为什么还要读回一次？

▼ 参考答案
RM0090 §7 规定外设时钟使能后，访问外设寄存器前需等几个时钟周期生效。若刚写完立刻访问外设，可能在时钟还没喂到外设的那几个周期里读/写落空。读回 `RCC->AHB1ENR` 强制一次总线往返，等流水线把写落地再继续，这是 SPL 替你踩的"时钟生效延迟"坑。手写版对应做法是开时钟后插 `(void)RCC->AHB1ENR;` 或 `__DSB();`。

4. assert_param 在 Release 和 Debug 版分别是什么？为什么不会拖慢量产固件？

▼ 参考答案
Release 版（未定义 USE_FULL_ASSERT）：`assert_param(expr)` 展开为 `((void)0)`，编译期完全消失，零运行开销。Debug 版（定义 USE_FULL_ASSERT）：展开为 `expr ? (void)0 : assert_failed(__FILE__, __LINE__)`，参数错时进 `assert_failed` 死循环并打印位置。量产固件用 Release 配置编译，断言消失，不拖慢运行；开发期用 Debug 配置，参数错当场抓人。这是 SPL 把"调试价值"与"运行成本"分离的标准做法。

5. 给"何时手写、何时用库"一个判断框架。

▼ 参考答案
初始化代码（启动跑一次，开销不重要）→ 用库，可读性+防错价值远超几周期；中断处理/性能热路径（每次中断跑、纳秒预算）→ 手写，库的循环+分支开销吃不消；学习阶段（理解机制）→ 手写，亲手配过才知道库在替你做什么；量产 → 混合，初始化用库、热路径手写，配 -gc-sections 链接时剔除没用到的库函数。

对照表：本章概念 → 仓库与上游落点

概念	落点
GPIO_TypeDef 结构与位掩码	CMSIS `stm32f407.h`（本站 `.trellis/ref/cmsis/`，逐位核对）
GPIO_Init 翻译逻辑	SPL V1.8.0 `stm32f4xx_gpio.c`（行号以库源码为准）
时钟使能读回	SPL `stm32f4xx_rcc.c` `RCC_AHB1PeriphClockCmd`
assert_param 宏	SPL `stm32f4xx_conf.h` + 各外设头 `IS_*` 宏
手写对照版	S3 GPIO + `code/stm32/00-blink/main.c`
体积实测方法	B5 map 与体积

你做到了

- 库在你眼里是"透明的封装"，不再是必须迷信的黑盒；
- 掌握"API→寄存器"的反推法，任何新库（HAL/LL/RT-Thread 驱动）都能用同一把刀解剖。

✅ 下一站：S16 HardFault 与排错：崩溃之后别猜，去现场把死因验出来。

通往单片机之路第 34 章

S16 HardFault 与排错：崩溃现场的法医技术

难度 ★★★☆☆ · 约 50 分钟

本章罗盘

问题

程序跳进 HardFault 后，怎样从压栈的寄存器现场反查出是哪条指令、哪个非法访问引发的？

交付

一份 CFSR/HFSR/BFAR 解读表和一次从 fault 地址回到 C 源码行的完整取证记录。

边界

本章解决『事后取证』；时序类 bug（偶发、与优化相关）还需 DWT/逻辑分析仪，不在本章范围。

依据锚点 `a3 · a6 · mcu-code`

> 🎯 程序跑着跑着不动了，灯不闪了，printf 沉默了——恭喜你遇到 HardFault。新手靠猜，法医靠证据：故障寄存器里写着"死因"，栈里躺着"案发现场"。

本章精髓

1. 故障是"异常升级"：MemManage / BusFault / UsageFault 没使能或处理不当就升级成 HardFault，HFSR 的 `FORCED` 位会告诉你：这是"升级案"，真凶要去 CFSR 里找。
2. 三份验尸报告：**CFSR**（可配置故障状态：19 个位逐位点名死因）、**BFAR**（总线故障的具体地址）、**MMFAR**（存储管理故障地址）。地址一指，凶手现形，但只有对应的 `BFARVALID` / `MMARVALID` 为 1 时才可信。
3. 栈帧即现场：异常进入时硬件压的 8 个字里有**案发时的 PC**。从栈里挖出 PC，用 `addr2line` 反查源码行：这是"死机"和"知道死在哪一行"的区别。

- **能理解**：看 naked `HardFault_Handler` 的反汇编：`tst.w lr, #4` / `ite eq` / `mrseq r0, MSP` / `mrsne r0, PSP`。短短四条指令就解决了"案发时用的是哪个栈"。
- **能用**：板子一死，先读 CFSR，再读栈帧 PC，`addr2line` 定位到行。所有裸机疑难，都按同一套流程破案。

先修

- S4 中断与 NVIC：异常号、向量表、优先级，本章大量复用。
- C6 调用约定与栈帧：`push {r4, lr}` 与"返回地址弹进 PC"是读懂本章汇编的前提。
- B2 ELF 与 objdump：`nm` / `objdump` / `addr2line` 的用法。

先跑起来（10 分钟 quick win）

```
cd code/stm32/16-debug-hardfault
sh probe.sh          # 宿主 274 条断言；工具链在 PATH 时附送 Cortex-M4 反汇编与 addr2line 全链路
```

输出里找这两行：

```
== 断言通过 274/274 ==
  ----- 对账结果：逐位一致（39 条全中） -----
```

"逐位一致"指的是：本章代码里的 39 条故障寄存器位号（CFSR 19 位 + SHCSR 14 位 + HFSR 3 位，另有 3 条子寄存器基名），与 ST 官方 CMSIS 头文件 `core_cm4.h` **逐条比对、零差异**。后面正文里每一个位号都不是背出来的，是当场从官方头文件挖出来对过的。

在板上写一行 `*(volatile uint32_t*)0xFFFFFFFF = 0;` 就能触发 HardFault：第一次"按地址抓凶手"。

结案之前先押个真凶：把栈帧里抓到的 PC 交给 addr2line，报出来的那一行，一定是真凶本人吗？还是可能只是被牵连的邻居？押完往下读，看真凶指令和函数入口之间隔着什么。

小节结构

小节	内容	四件套
故障家族	HardFault / MemManage / BusFault / UsageFault 的升级关系	配置
验尸三寄存器	SCB 版图；CFSR / HFSR / MMFAR / BFAR 的偏移与分工	配置
CFSR 19 位	MMFSR 6 + BFSR 7 + UFSR 6 逐位点名；两个"有效性位"	库解析
栈帧取证	MSP / PSP 判定（EXC_RETURN bit2）；8 个字里找 PC	代码分析
Handler 实现	naked 汇编取栈指针 + C 侧抄报告的最小实现；四条指令的字节学与那个"工具链不拦"的位	代码分析
addr2line 反查	PC → 源码行；函数入口 vs 真凶指令的区别	代码分析
高频死因	空指针 / 未对齐 / 除零 / 爆栈 / 优先级五大命案	库解析

一、故障家族：四个异常，一个终点

Cortex-M4 有四个"故障"类异常。它们的异常号与 `IRQn` 枚举值（`stm32f407xx.h:67-75`）：

异常	异常号	IRQn 枚举	向量表内偏移
HardFault	3	**无**（见下）	0x0C
MemManage	4	`MemoryManagement_IRQn = -12`（69 行）	0x10
BusFault	5	`BusFault_IRQn = -11`（70 行）	0x14
UsageFault	6	`UsageFault_IRQn = -10`（71 行）	0x18

第一处反直觉：HardFault 的异常号是 3、排在 MemManage 前面，但 `IRQn_Type` 枚举里**故意没有它**：`stm32f407xx.h` 第 68 行是 `NonMaskableInt_IRQn = -14`，第 69 行直接跳到 `MemoryManagement_IRQn = -12`，中间的 `-13`（= 异常号 3）被整个省略了。原因是 CMSIS 的 `NVIC_SetPriority()` 只接受 `IRQn_Type`，省略它等于从类型层面禁止你给 HardFault 设优先级。HardFault 的优先级是**固定**的最高档：它不可屏蔽、不可设。

第二处：另外三个故障是"可配置"的。它们各自在 SHCSR 里有一个使能位，位号实测（`core_cm4.h:545-580`）：

```
SHCSR:MEMFAULTENA 16
SHCSR:BUSFAULTENA 17
SHCSR:USGFAULTENA 18
```

关上时，对应的故障**不单独上报**，而是直接升级成 HardFault，这时 HFSR 的 `FORCED` 位会亮起来，告诉你"这是一起升级案，真凶在 CFSR 里"。所以看到 HardFault 不要先去翻 HardFault，先看 `FORCED`。

二、验尸三寄存器：SCB 版图

所有故障寄存器都挂在 `SCB`（System Control Block）下，基址 `0xE000ED00`。各寄存器偏移出自 `core_cm4.h:449-459`，下面是 probe 打印并已比对过的绝对地址：

```
SCB 基址            = 0xE000ED00
SHCSR（使能开关）    = 0xE000ED24
CFSR （可配置故障）  = 0xE000ED28
HFSR （硬故障状态）  = 0xE000ED2C
MMFAR（存储故障地址）= 0xE000ED34
BFAR （总线故障地址）= 0xE000ED38
```

分工很清楚：

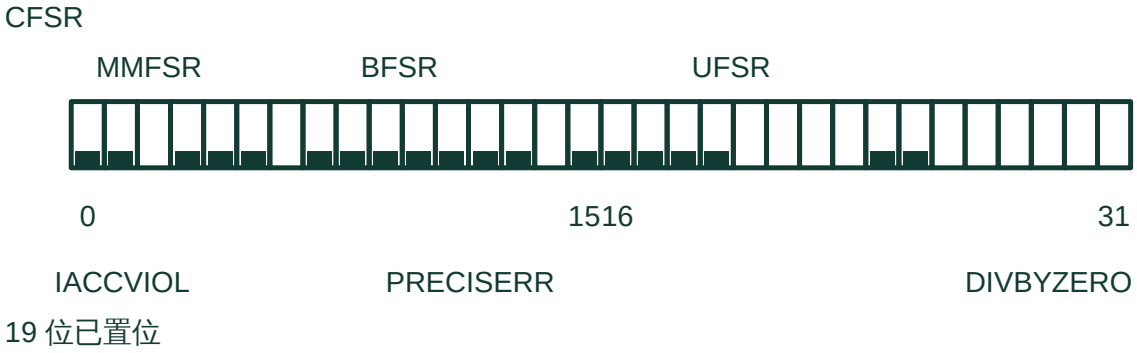
- **CFSR**：`configurable fault status`，19 个位逐个点名死因。它管的是"可配置故障"，但 HardFault 升级上来时它**同样有内容**：真凶往往就在这。
- **HFSR**：`hard fault status`，只有 3 个位，定性用。
- **MMFAR / BFAR**：两个"现场地址"。BFAR 存总线故障的访问地址，MMFAR 存存储管理故障的地址。

最关键的坑：MMFAR / BFAR 里的地址**不是永远可信**。CFSR 里有两个专门的"有效性位"：`MMARVALID`(7) 和 `BFARVALID`(15)。只有它们为 1 时，对应的地址寄存器才保真。probe 里的解码示例：

```
CFSR = 0x00008200
  命中 2 位：PRECISERR + BFARVALID
  BFAR = 0x1FFFFFFF  （BFARVALID=1，这个地址可信）
HFSR = 0x40000000 → FORCED —— 被升级：真凶在 CFSR 里，去查它
```

`0x00008200` 拆开看就是 bit9 + bit15：bit9 = `PRECISERR`（精确总线错误），bit15 = `BFARVALID`，两相印证，所以 `BFAR = 0x1FFFFFFF` 这个"访问了不存在的地址"结论站得住。

三、CFSR 19 位逐位点名



CFSR 19 位点图：MMFSR/BFSR/UFSR 三段

CFSR 是三个子寄存器拼成的（`core_cm4.h:616-684`）：低 8 位 MMFSR、次 8 位 BFSR、高 16 位 UFSR。probe 的表实测是 **MMFSR 6 位 + BFSR 7 位 + UFSR 6 位 = 19 位**（子寄存器的基名 `MEMFAULTSR` / `BUSFAULTSR` / `USGFAULTSR` 只占 3 条记录，不参与点名的位数）。

子寄存器	位	名字	白话
MMFSR	0	`IACCVIOL`	取指越权
MMFSR	1	`DACCVIOL`	访存越权
MMFSR	3	`MUNSTKERR`	出栈时存储管理错
MMFSR	4	`MSTKERR`	入栈时存储管理错
MMFSR	5	`MLSPERR`	懒浮点保存时出错
MMFSR	7	`MMARVALID`	**MMFAR 可信**
BFSR	8	`IBUSERR`	取指总线错
BFSR	9	`PRECISERR`	精确总线错（地址就是 BFAR）
BFSR	10	`IMPRECISERR`	不精确总线错（地址不可信！）
BFSR	11	`UNSTKERR`	出栈时总线错
BFSR	12	`STKERR`	入栈时总线错
BFSR	13	`LSPERR`	懒浮点保存时总线错
BFSR	15	`BFARVALID`	**BFAR 可信**
UFSR	16	`UNDEFINSTR`	未定义指令
UFSR	17	`INVSTATE`	非法状态（Thumb 位丢了）
UFSR	18	`INVPC`	非法 PC
UFSR	19	`NOCP`	用了没开的协处理器
UFSR	24	`UNALIGNED`	未对齐访问
UFSR	25	`DIVBYZERO`	除以零

全 1 解码（probe 实测，19 个名字全中）：

```
  IACCVIOL + DACCVIOL + MUNSTKERR + MSTKERR + MLSPERR + MMARVALID
+ IBUSERR + PRECISERR + IMPRECISERR + UNSTKERR + STKERR + LSPERR + BFARVALID
+ UNDEFINSTR + INVSTATE + INVPC + NOCP + UNALIGNED + DIVBYZERO
```

为什么这张表要一位一位对：故障解码是枯燥的体力活，但位号记错一位，你整晚的推断方向就全歪了。probe 的 `--dump-positions` 把表导出成机器可读格式，`probe.sh` 用 `awk` 从官方头文件现挖同样的 39 条位号，两边 `diff`，**零差异**才算过。这就是"不许凭印象背位号"的工程做法。

四、栈帧取证：案发时用的是哪个栈

先看"崩溃破案"的整条流水线：案发、升级、取证、PC 对行号。

HardFault 故障取证全流程动画

异常进入时，硬件**自动**把 8 个寄存器压进当前栈（ARMv7-M 异常机制），probe 实测的版图（`fault_ctx.c:22-24` 的 `hw_frame_t`）：

```
偏移 0/4/8/12 = r0-r3，16 = r12，20 = lr，24 = pc，28 = xpsr（共 32 字节）
```

三个位置最关键：

- **偏移 20 = LR**：这是"从哪儿调用来的"，还原调用链靠它。
- **偏移 24 = PC**：**案发时正在执行的指令地址**，法医的主证据。
- **偏移 28 = xPSR**：其中的 IPSR 字段告诉你当时在哪个异常里。

问题来了：栈指针有两个（MSP 主栈、PSP 进程栈），硬件压的是**哪一个**？答案藏在异常进入时硬件写进 LR 的那个特殊值 `EXC_RETURN`（`core_cm4.h:1636-1641`）里：

```
0xFFFFFFF1 处理模式/MSP/无FPU | 0xFFFFFFF9 线程/MSP/无FPU
0xFFFFFFFD 线程/PSP/无FPU     | 0xFFFFFFE1/E9/ED 同上但带 FPU 上下文
```

判据是 **bit2**：

- `bit2 = 0`（如 `0xFFFFFFF1`、`0xFFFFFFF9`）→ 案发时用 **MSP**（裸机主循环、中断嵌套）。
- `bit2 = 1`（如 `0xFFFFFFFD`）→ 案发时用 **PSP**（RTOS 任务）。

probe 实测样例：`EXC_RETURN = 0xFFFFFFFD → 案发时用 PSP（RTOS 任务栈），不带 FPU 上下文`。**拿错栈指针，读出来的 8 个字全是垃圾，后面所有推断都是假的**。这是 HardFault Handler 最容易写错的一处。

另外两个位也顺手用上：bit3（`0x8`）=0 表示案发时处于 handler 模式；bit4（`0x10`）=0 表示带 FPU 上下文（浮点寄存器也被压栈了，栈帧更长）。

五、Handler 实现：naked 汇编 + C 报告

写一个正确的 Handler，核心就是上面那一条判据。用 `naked` 属性让编译器别插桩，自己写那五条指令（`fault_ctx.c:50-61`）：

```
__attribute__((naked))
void HardFault_Handler(void)
{
    __asm volatile(
        "tst   lr, #4            \n"  /* bit2 决定用哪个栈 */
        "ite   eq                \n"
        "mrseq r0, msp           \n"  /* eq：MSP */
        "mrsne r0, psp           \n"  /* ne：PSP */
        "b     hardfault_report  \n"  /* 交给 C 侧抄报告 */
    );
}
```

交叉编译后的真实反汇编（`probe.sh` 第 3 段输出）：

```
08000070 <HardFault_Handler>:
 8000070:   f01e 0f04   tst.w   lr, #4
 8000074:   bf0c        ite eq
 8000076:   f3ef 8008   mrseq   r0, MSP
 800007a:   f3ef 8009   mrsne   r0, PSP
 800007e:   f7ff bfbf   b.w 8000000 <hardfault_report>
 8000082:   bf00        nop
```

值得逐条读一遍：

- `tst.w lr, #4` 测 bit2，结果进标志位。
- `ite eq` 是 Thumb-2 的 `if-then-else` 块声明：它告诉后续两条指令"第一条按 eq 执行、第二条按 ne 执行"。**没有它，条件指令就是非法的**。
- 两条 `mrs` 分别读 MSP / PSP，结果都放进 `r0`，因为 `r0` 是 AAPCS 的第一个参数（见 C6）。
- `b.w hardfault_report` 直接尾跳到 C 函数，`r0` 就是栈帧指针。

C 侧只做一件事：抄报告（`fault_ctx.c:27-48`），读四个 SCB 寄存器、从栈帧里取 LR/PC/xPSR，然后**死循环不动**：

```
g_cfsr       = *(volatile uint32_t *)0xE000ED28UL;   /* CFSR  */
g_hfsr       = *(volatile uint32_t *)0xE000ED2CUL;   /* HFSR  */
g_mmfar      = *(volatile uint32_t *)0xE000ED34UL;   /* MMFAR */
g_bfar       = *(volatile uint32_t *)0xE000ED38UL;   /* BFAR  */
g_frame_lr   = frame[5];
g_frame_pc   = frame[6];                             /* ← 案发时的 PC */
g_frame_xpsr = frame[7];
```

现场全部落到 `volatile` 全局变量里，然后死等 GDB 来读，**故意不打印**。这是刻意的设计：故障上下文里跑 `printf` 这种复杂库函数，很可能触发二次故障，把唯一的现场也毁掉。

那四条指令的字节学：条件到底存在哪个词里

`probe.sh` 第 4 段把 Handler 单独编成裸二进制，逐字节拆开看。先看正确版和"把 ne 槽也写成 msp"的笔误版：

```
00000000 <h_psp>:                    00000000 <h_msp>:
   0:   f01e 0f04   tst.w   lr, #4          0: f01e 0f04   tst.w   lr, #4
   4:   bf0c        ite eq                  4:   bf0c        ite eq
   6:   f3ef 8008   mrseq   r0, MSP            6:   f3ef 8008   mrseq   r0, MSP
   a:   f3ef 8009   mrsne   r0, PSP            a:   f3ef 8008   mrsne   r0, MSP
   e:   4770        bx  lr                  e: 4770        bx  lr
```

两份机身各 16 字节，`cmp -l` 只报出一行（它给的是 octal）：

```
  正确版 hex: 1ef0040f0cbfeff30880eff309807047
  笔误版 hex: 1ef0040f0cbfeff30880eff308807047
  差异字节数 = 1，明细：13  11  10
```

第 13 个字节从 octal `11`（0x09）变成 `10`（0x08），**整个语义差别就是一个位**。因为 MRS 的编码是 `f3ef 8<Rd><imm5>`，`imm5` 里 MSP= `0x08`、PSP= `0x09`，两者只差 bit0。

更值得记的一件事：**`mrseq` 和 `mrsne` 的字节完全相同**。`probe.sh` 编了一张编码矩阵，每一对 eq/ne 槽机身一字不差：

```
    6: f3ef 8008   mrseq   r0, MSP        a:   f3ef 8008   mrsne   r0, MSP
   16: f3ef 8108   mrseq   r1, MSP       1a:   f3ef 8108   mrsne   r1, MSP
   26: f3ef 8c08   mrseq   ip, MSP       2a:   f3ef 8c08   mrsne   ip, MSP
   36: f3ef 8009   mrseq   r0, PSP       3a:   f3ef 8009   mrsne   r0, PSP
   46: f3ef 8000   mrseq   r0, CPSR      4a:   f3ef 8000   mrsne   r0, CPSR
```

条件根本不在 `mrs` 里，它只存在于 `ite` 那一个词 `bf0c` 里（前一条 `tst` 决定了它_eq 还是_ne，见第四节 bit2）。顺带把 `Rd` 的位置也钉死了：`r0` / `r1` / `ip` 分别给出 `8008` / `8108` / `8c08`，`Rd` 就在 bits[11:8]。

于是"写错 Handler"有三种姿势，工具链只拦得住其中两种（`probe.sh` 把 GAS 的报错原文整段抄了下来）：

```
[bad_cond 拒绝] Error: incorrect condition in IT block -- `mrseq r0,psp'
[bad_noite 拒绝] Error: thumb conditional instruction should be in IT block -- `mrsne r0,psp'
```

第三种它照收不误：把 eq 槽写成 `psp`、ne 槽写成 `msp`：

```
00000000 <bad_slot>:
   0:   f01e 0f04   tst.w   lr, #4
   4:   bf0c        ite eq
   6:   f3ef 8009   mrseq   r0, PSP
   a:   f3ef 8008   mrsne   r0, MSP
   e:   4770        bx  lr
```

条件后缀跟 IT 块匹配，语法完全合法，GAS 一声不吭——但**语义整个反了**：主循环崩的时候读 PSP，任务崩的时候读 MSP，两种情况都拿到错的栈帧。真正咬人的从来不是工具链拦得住的那两种。

六、addr2line 反查：从栈里的 PC 到源码行

拿到 `g_frame_pc` 之后，剩下的问题就是"这个地址对应哪一行"。`fault_ctx.c` 里预备了三具"尸体"（只编译、不运行），probe 用它们把整条链路走通：

```
080000c4 T crash_divzero
08000084 T crash_null
080000a4 T crash_unaligned
08000070 T HardFault_Handler
08000000 T hardfault_report
```

这里有一个必须讲清的细节：`nm` 给的 `crash_null = 0x08000084` 是**函数入口**，而栈里躺的是**出故障那一条指令**的地址。两者指的往往不是同一行：

```
crash_null 入口    = 0x08000084（nm 给的，指向函数第一行）
crash_null
D:\Users\mcu\code\stm32\16-debug-hardfault/fault_ctx.c:69
crash_null 的 str  = 0x800008e（反汇编找的，HardFault 时栈帧里躺的就是它）
crash_null
D:\Users\mcu\code\stm32\16-debug-hardfault/fault_ctx.c:70
```

同一次崩溃，两个地址差 10 个字节，行号差 1 行：`:69` 是函数体开头（`{`），`:70` 才是那行真正的空指针写。**要拿到真凶行，就得从反汇编里找出那条出错的指令地址**，而不是拿 `nm` 的入口地址去凑。

反汇编里那条 `str` 就是凶手。`probe.sh` 第 3 段现在把 `crash_null` / `crash_unaligned` / `crash_divzero` / `fault_demo_entry` 四具机身整段打出来，本机实跑的第一具：

```
08000084 <crash_null>:
 8000084:   b480        push    {r7}
 8000086:   af00        add r7, sp, #0
 8000088:   4b04        ldr r3, [pc, #16]   ; (800009c <crash_null+0x18>)
 800008a:   681b        ldr r3, [r3, #0]
 800008c:   4a04        ldr r2, [pc, #16]   ; (80000a0 <crash_null+0x1c>)
 800008e:   601a        str r2, [r3, #0]
 8000090:   bf00        nop
 8000092:   46bd        mov sp, r7
 8000094:   f85d 7b04   ldr.w   r7, [sp], #4
 8000098:   4770        bx  lr
 800009a:   bf00        nop
 800009c:   0801012c    .word   0x0801012c
 80000a0:   deadbeef    .word   0xdeadbeef
```

`r3` 从 `g_null_ptr` 载入（值为 0），`str r2, [r3, #0]` 就是往地址 0 写，**空指针写的机器码长相**。它的地址 `0x800008e` 送进 `addr2line` 得到 `fault_ctx.c:70`，与源码里那行 `*g_null_ptr = 0xDEADBEEF;` 严丝合缝。

最后用链接器 map 文件交叉验证同一个地址（三处独立来源互相印证）：

```
                0x08000084                crash_null
```

`.text` 段落在 Flash 上（`objdump -h`）：`0 .text 0000010c 08000000 08000000`，VMA 就是 `0x08000000`，与 B1 链接四步 讲的 Flash 起始地址一致。所以 PC 落在 `0x0800xxxx` 时，你在看的是 Flash 里的代码地址，直接喂给 `addr2line` 就对。

> `addr2line` 能查到行号的前提是编译时带了 `-g3`。probe 里交叉编译用的是 `-O0 -g3`，**优化会重排指令，`-O2` 下行号可能对不上**。调试版本务必开 `-g3` 且优先 `-O0`。

七、高频死因五虎

前文是"怎么破案"，这里给"常见案由"速查表。

1. 空指针：`PRECISERR` + `BFARVALID`，BFAR 里是那个非法地址（接近 0）。本章的 `crash_null` 就是这桩命案。

2. 未对齐访问：`UNALIGNED` 位。**但这桩案子平时不报案**：Cortex-M4 对单字 `LDR` / `STR` 的未对齐访问默认是硬件兜住的，只有把 `CCR.UNALIGN_TRP`（`core_cm4.h:563`，bit3）打开，它才升格成故障。多字访问（`LDM` / `STM`、`LDRD` / `STRD`）则**永远**不许未对齐。所以这类 bug 的可怕之处不是崩溃，而是**不崩**：硬件帮你偷偷做完了，你以为没事。

3. 除零：`DIVBYZERO` 位。同样有开关：`CCR.DIV_0_TRP`（`core_cm4.h:560`，bit4）。开关不开时 `SDIV` 只是**返回 0**，一声不吭地给你个错答案（probe 的反汇编里那条 `sdiv r3, r2, r3` 就是这个场景）。

> 所以本章的 `crash_unaligned` / `crash_divzero` 两具"尸体"，光把代码放那儿是死不了的。`fault_demo_entry` 第一句就是开这两个开关（`*(volatile uint32_t *)0xE000ED14UL |= (1<<3)|(1<<4);`，即 `SCB->CCR`，地址见 `core_cm4.h:452` 的偏移 `0x014`），反汇编里对应 `orr.w r3, r3, #24`（24 = 0x18 = bit3|bit4 的常量折叠）。你自己上板复现时，也别忘了先开。
>
> 两个 CCR 开关的复位默认值以 ARM PM0214 为准；本地未存该文档 PDF，**待回填核对**。

上面这几句不是转述二手结论。`probe.sh` 第 3 段现在把四具机身整段打出来，并对机身本身下断言，以下全是本机实跑输出：

```
  [中] fault_demo_entry：orr[^[:space:]]*[[:space:]]+r3,[[:space:]]+r3,[[:space:]]+#24$
  [中] fault_demo_entry：0xe000ed14
  [中] crash_null：str[[:space:]]
  [中] crash_divzero：sdiv
  [中] crash_divzero：[[:space:]](bl|b\.[a-z]+|udf|bkpt|svc)[[:space:]]    （期望命中=0）
  [中] crash_unaligned：literal pool 里存在奇地址 0x08010131
```

拆开看这三具尸体的死法凭什么成立：

```
080000a4 <crash_unaligned>:            080000c4 <crash_divzero>:
 80000ac:   607b        str r3, [r7, #4]     80000d0:   68fb        ldr r3, [r7, #12]
 80000ae:   687b        ldr r3, [r7, #4]     80000d2:   687a        ldr r2, [r7, #4]
 80000b0:   2201        movs    r2, #1         80000d4: fb92 f3f3   sdiv    r3, r2, r3
 80000b2:   601a        str r2, [r3, #0]
 ...                                    （机身再无 bl / b.xx / udf / bkpt / svc）
 80000c0:   08010131    .word   0x08010131
```

- **开关确实被拧开了**：入口第一条写操作就是 `orr.w r3, r3, #24`，而 `r3` 的来源是字面量池里那个 `0xe000ed14`，`SCB->CCR` 无误。
- **未对齐那具是真的未对齐**：`str` 的目标地址来自 literal pool 里的 `0x08010131`，**奇地址**，而指令本身只是最普通的 `601a str r2, [r3, #0]`。所以它死不死完全取决于那个 trap 位，不取决于代码长相。
- **除零那具没有任何陷阱**：`sdiv` 前后没有 `bl`、没有分支、没有 `udf` / `bkpt` / `svc`（这条是反向断言：命中数必须为 0）。硬件不拦它，它就老老实实返回 0。

这两条合起来说明一件事：**这两类 bug 在默认配置下根本不表现为崩溃**，它们只是悄悄给你一个错答案。想复现，先开开关；想防住，别指望它会自己炸。

4. 爆栈：栈溢出到非法区域时，往往在"入栈/出栈"阶段就出错，看 `MSTKERR` / `MUNSTKERR`（存储管理）或 `STKERR` / `UNSTKERR`（总线）。**这是最有诊断价值的一类**：报错位直接告诉你"是在压栈还是弹栈时死的"，一眼就能想到栈深问题。

5. 优先级配置错误：`PendSV` / `SysTick` 用了 S4 讲的优先级规则，配错时表现为"中断里又触发中断"或"中断永不返回"。这类通常不产生 CFSR 位，而是**卡死在某个 handler 里**，得靠栈帧的 LR/PC 还原调用链。

附录：工程完整源码

probe.c（411 行：SCB 版图 / CFSR·SHCSR·HFSR 三张位表 / 解码器 / EXC_RETURN 判栈 / 向量槽 / 栈帧版图 / 全链路演示）：

<<< ../../code/stm32/16-debug-hardfault/probe.c

fault_ctx.c（103 行：naked `HardFault_Handler` + C 侧报告 + 三具只编译不运行的"尸体"，外加 `fault_demo_entry` 里开 CCR 两个 trap 位）：

<<< ../../code/stm32/16-debug-hardfault/fault_ctx.c

probe.sh（宿主断言 + 位号对账 + 反汇编与 addr2line 全链路 + 机身断言 + 条件指令字节学）：

<<< ../../code/stm32/16-debug-hardfault/probe.sh

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **先看 CFSR 定罪名，再查 BFAR 指现场，栈里挖 PC 对行号；死因五虎：空针、错位、除零、爆栈、优先级。**

延伸：栈帧挖 PC 依赖调用约定（C6）；HardFault 与 RTOS 下看门狗超时（S17、P13）常伴生。

实物实验

- 三大命案各复现一次：空指针写（`*(volatile uint32_t*)0 = 0;`）、未对齐访问（奇地址强转 `uint32_t*`，记得先开 `CCR.UNALIGN_TRP`）、`x/0`。每次记录 CFSR 值与 `addr2line` 出来的行号，贴进实验记录。
- 故意把 `HardFault_Handler` 里的 `mrsne r0, psp` 改成 `mrsne r0, msp`，**注意别写成 `mrseq r0, psp`，那样根本编不过**（GAS 直接 `Error: incorrect condition in IT block`，见第五节）。改完之后机身只变一个字节（`0x09` → `0x08`），烧进板子，在 RTOS 任务里触发一次故障，感受"拿错栈指针"之后 `g_frame_pc` 读出来的垃圾数据长什么样。
- 再来一次更狠的：把 eq/ne 两槽的 `msp` / `psp` 对调。这个工具链不报错，但语义整个反了：裸机崩溃读 PSP、任务崩溃读 MSP。
- 在有 RTOS 的工程里（F 篇）触发一次任务内的故障，确认 `EXC_RETURN` 的 bit2 确实是 1。

常见坑

- **Handler 里用 `printf` 本身再崩**：故障里跑复杂库函数可能二次故障，Handler 保持极简（直接写寄存器串口或半主机），或者像本章一样只存全局变量等 GDB。
- **MSP / PSP 拿错**：RTOS 下任务用 PSP，拿 MSP 读栈全是错的（判据是 `EXC_RETURN` 的 bit2，见第四节）。
- **没开 `-g3`**：`addr2line` 查不到行号，只会回 `??:0`，调试版本务必 `-g3`，并优先 `-O0`。
- **拿 `nm` 的函数入口地址当案发 PC**：入口地址指向函数第一行，真凶是反汇编里那条出错指令的地址，两者常常差好几天行（本章实测差 1 行、10 字节）。
- **只看 PC 不看 LR**：LR（压栈值）告诉你"从谁调用来"，调用链还原靠它，配合 map 文件追 caller。
- **以为工具链会拦住 Handler 笔误**：GAS 只校验条件后缀跟 IT 块匹不匹配，**不校验语义**。`mrseq r0, psp`（后缀错位）它拒；`mrsne r0, msp`（少一个位，RTOS 下永远读 MSP）它照收。这种地方只能靠反汇编自己核。

短自测

1. 为什么 `IRQn_Type` 里没有 `HardFault_IRQn`？

> **▼ 看答案**
> 因为 HardFault 的优先级是固定的、不可设。CMSIS 的 `NVIC_SetPriority()` 只接受 `IRQn_Type`，省略该枚举等于从类型层面禁止你设优先级。`stm32f407xx.h` 第 68 行 `NonMaskableInt_IRQn = -14` 直接跳到第 69 行 `MemoryManagement_IRQn = -12`，中间的 `-13`（异常号 3）被整个略过。

2. CFSR 显示 `PRECISERR`，但 `BFARVALID` 是 0，BFAR 里的地址能信吗？

> **▼ 看答案**
> 不能。`BFARVALID`(bit15) 是 BFAR 的"有效性位"，它为 0 时 BFAR 里是**上次残留的旧值**。同理 `MMARVALID`(bit7) 管 MMFAR。另外 `IMPRECISERR`（不精确总线错）情况下地址本身就不可靠，总线写是缓冲的，出错时 PC 可能已经跑远。

3. 硬件压栈的 8 个字里，案发 PC 在哪个偏移？

> **▼ 看答案**
> 偏移 **24**。完整版图：0/4/8/12 = r0-r3，16 = r12，20 = LR，24 = PC，28 = xPSR，共 32 字节。写 Handler 时用 `frame[6]` 取 PC、`frame[5]` 取 LR。

4. `crash_null` 的 `nm` 入口是 `0x08000084`，为什么 `addr2line` 要喂 `0x800008e`？

> **▼ 看答案**
> `nm` 给的是函数入口，指向函数第一行（`:69` 的 `{`）；栈帧里的 PC 是**实际出错的那条指令**（`:70` 的空指针写）。`0x800008e` 是从反汇编里找出来的那条 `str r2, [r3, #0]` 的地址。差 10 个字节就是差一行，定位到正确行号才对得上源码。

5. 除零为什么常常"不崩"？

> **▼ 看答案**
> 因为 `CFSR.DIVBYZERO` 的触发要先打开 `CCR.DIV_0_TRP`（`core_cm4.h:560`，bit4）。开关关着时 `SDIV` 只返回 0，不产生故障，程序继续跑，只是结果悄悄错

了。未对齐访问同理（开关是 `CCR.UNALIGN_TRP`，bit3）。**这两类"不崩的 bug"比崩溃难查得多。**

6. `mrseq r0, MSP` 和 `mrsne r0, MSP` 的机器码有区别吗？条件存在哪里？

▼ 看答案
没有区别，两条都是 `f3ef 8008`（本机编码矩阵实测，eq 槽与 ne 槽机身一字不差）。MRS 的编码是 `f3ef 8`，`imm5` 只选源寄存器：MSP=`0x08`、PSP=`0x09`、APSR/CPSR=`0x00`；`Rd` 在 bits[11:8]。**条件只存在于前一条 `ite` 的那个词 `bf0c` 里**，不带后缀的 `mrs` 本身压根没有条件域。这也解释了为什么把 `psp` 写成 `msp` 只改一个位（`imm5` 的 bit0），GAS 还不报错：它看的是后缀与 IT 块匹不匹配，那一位它不管。

对照表：本章概念 → 仓库落点

概念	仓库落点
SCB 版图与六个故障寄存器地址	`code/stm32/16-debug-hardfault/probe.c` 41–68 行
CFSR 19 位表（MMFSR 6 + BFSR 7 + UFSR 6）	同上 71–103 行
SHCSR 14 位表（三个 ENA 位在 16/17/18）	同上 104–120 行
HFSR 3 位表（`FORCED` = 30）	同上 121–126 行
位号与 CMSIS 头文件逐条对账（39 条）	`probe.sh` 第 2 段（`--dump-positions` + awk diff）
解码器：位 → 死因名字	`probe.c` 172–227 行
`EXC_RETURN` 判栈（bit2）与 FPU（bit4）	同上 248–280 行（`exc_return_uses_psp` / `has_fpu`）
向量槽 = `IRQn + 16`	同上 281–310 行
栈帧版图（PC 在偏移 24）	同上 311–335 行；`fault_ctx.c` 22–24 行
naked `HardFault_Handler` 汇编	`fault_ctx.c` 50–61 行
C 侧报告（读 SCB + 抄栈帧）	同上 27–48 行
三具"尸体"（空指针 / 未对齐 / 除零）	同上 65–86 行
`addr2line` 全链路与 map 交叉验证	`probe.sh` 第 3 段
四具机身打印 + 机身断言（五虎里两具"死不了的尸体"）	`probe.sh` 第 3 段末（`SEC` / `chk` / 奇地址 literal pool）
条件指令的字节学：`ite` 带条件、MRS 编码矩阵、GAS 拒绝原文	`probe.sh` 第 4 段（`enc_*.s` / `mrs_matrix.s` / `bad_*.s` + `cmp -l`）

你做到了

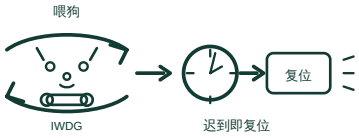
- HardFault 从"玄学死机"变成"按流程破的案"；
- 你手上有了三份验尸报告 + 一条 PC → 行号的全链路；
- S 篇收官：你已有能力独立调试任何裸机疑难。

✅ S 篇收官。下一站：RTOS 篇：从"一个超级循环"到"多个平行世界"，先看 FreeRTOS 怎么变魔术。

通往单片机之路第 35 章

S17 看门狗：任务失去进展之后

难度 ★★★☆☆ · 约 45 分钟



本章罗盘

问题

中断里准时喂狗、任务却卡死了，喂狗到底该证明什么？WWDG 为什么喂早了也复位？

交付

一张 IWDG 超时三档表（LSI 17/32/47kHz→约 1.36/2.00/3.76s）加一笔 WWDG 窗口手算（约 67.6ms 开窗、131ms 复位），和故障注入命令到 NRST 复位的时间预算线（MODE=0 约 2.7s，区间 2.06~4.46s）。

边界

时间预算与三档超时是按 RM0090/DS8626 计算：板上 NRST 脉冲时刻、复位标志组合与 LSI 实测值均待上板，实验记录表为空；health 模块平台无关、宿主可测，但三任务切分是本项目架构而非唯一解；调试冻结位只有断电重上电才清，否则复位实验无效。

依据锚点 `a1 · a2 · f3 · mcu-code · d12 · mcu-anim`

> 🎯 定时器中断里每 10ms 喂一次狗，采集任务卡死了，狗会叫吗？不会。它只证明中断还活着。

本章精髓

1. **喂狗要证明关键工作在按期完成**：每个必需任务完成一轮有界工作后上报进展，监督任务确认全部按期才喂。任何一个迟到，就锁存"该重启"，之后来的心跳也撤销不了。
2. **两只狗量两种健康**：IWDG 用独立的 LSI 时钟，只管"别太晚"；WWDG 用 PCLK1，窗口两头都是死路，喂早了也复位。超时值要按时钟的最坏情况算，不能按名义值算。
3. **复位之后要留证据**：RCC_CSR 的复位标志先读出来再清；`.noinit` 里放一条带 magic/version/CRC 的记录，只在看门狗暖复位后才解读。断电后 RAM 内容不可信，CRC 对不上就报"没有有效记录"。

- **能理解**：为什么任务期限（500ms）和硬件超时（2s）是两个参数；为什么 MODE=2 要连写两次 WWDG_CR；为什么调试器断开后还要重新上电。
- **能用**：四种 MODE 各自构建；用 `fault` 命令停掉 worker，按时间预算预测复位时刻，再用 STATUS 和复位原因核对。

先修

- 必需：S5 SysTick、F8 FreeRTOS F407 移植、C8 串口协议（数据口和 STATUS 都用它）。

先跑起来（10 分钟 quick win）

```
cd code/stm32/06-watchdog-health
make MODE=0 FAULTS=0      # IWDG，普通固件
```

用 ST-Link 烧录 `build/mode-0-faults-0` 下的产物。**烧完断开调试器，再给板子重新上电**（原因见第七节）。然后在仓库根执行：

```
python3 scripts/device-console.py status --port COMx
```

隔几秒再执行一次。运行毫秒在涨、复位原因没变、过期任务 mask 为 0，说明监督在正常喂狗，系统没有反复重启。数据口是 PA9/PA10，115200，协议和 C8 相同。

上电之前先押个喂养：喂狗是不是喂得越频繁越稳？逮着空就喂一口，行不行？押完往下看：有一种看门狗，喂得太早，下场和从来不喂一样。

动画：看门狗双雄

IWDG 倒数到 0 复位、喂狗等于重装；WWDG 只许在窗口内刷新——喂早了同样复位。两把尺子量两种健康："活着"与"节奏对"，动画把窗口的两头死路演出来。

看门狗双雄：IWDG 只管活着，WWDG 还管节奏

板卡事实

- 复位后系统时钟是 HSI16，本工程不改时钟树，所以 PCLK1 = 16MHz。WWDG 的窗口跟着 PCLK1 走，改了时钟树就得重算。
- LSI 名义 32kHz。DS8626 Rev 12 第 105 页表 35 给出的范围是 **17 / 32 / 47 kHz**（最小/典型/最大），注明是特性测试结果。手里这块板的 LSI 精确值待实测。
- IWDG 在 VDD 域，Stop 和 Standby 模式下照样计数（RM0090 Rev 22 §21.3）。用了 IWDG 再进低功耗，得把唤醒周期算进超时。

小节结构

小节	内容	四件套
一、喂狗证明什么	无条件喂狗的反例；三任务分工	代码分析
二、health 模块	required/seen/deadline 三个 mask，宽限期与锁存	库解析
三、IWDG	四步配置、表 107 实算、LSI 三档超时	配置
四、WWDG	窗口公式实算，为什么 5ms 轮询正好	配置
五、故障注入与时间预算	四种 MODE；从命令到复位的时间线	代码分析
六、复位证据	RCC_CSR 读后清；`.noinit` 记录的写入顺序	代码分析
七、调试器与看门狗	DBGMCU 冻结位只在上电复位时清零	引脚

一、喂狗到底证明了什么

最常见的写法是在定时器中断或主循环里无条件喂狗：

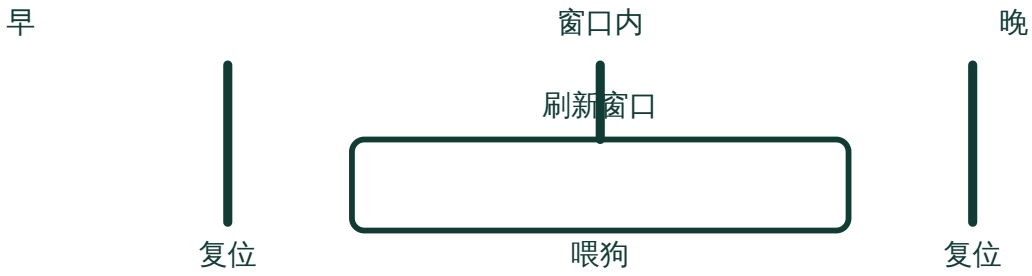
```
void TIM6_DAC_IRQHandler(void) { IWDG->KR = 0xAAAA; ... }   /* 反例 */
```

这只能证明"这个中断还在进"。采集任务死等一个永远不来的信号量、通信任务卡在一个没有超时的 while 里，中断照样准时到，狗永远不叫。看门狗该监督的是**工作有没有按期完成**，而不是**有没有代码在跑**。

本工程把职责拆成三个 FreeRTOS 任务：

任务	优先级	周期	职责
worker	2	10ms	模拟一轮采集，完成后上报进展 id=0
communication	3	1ms	最多读 128 字节喂给 C8 解析器，推进 TX，然后上报进展 id=1
supervisor	4	5ms	评估健康；**唯一**允许喂狗的地方

通信任务没有收到串口数据也会上报进展。它证明的是"这一轮有界的服务检查做完了"，不要求主机一直发数据。监督任务优先级最高，低优先级任务忙死了也挡不住它判断超时。



早也复位，晚也复位：窗口约束的是“进展发生在正确的时间段”

窗口看门狗：早喂也复位、晚喂也复位，只有落在窗口内才喂狗成功

二、health 模块：把"进展"变成可检查的 mask

`code/common/reliability/health.c` 是平台无关的纯 C，F407 和 S3 共用，主机上就能测。它不拿锁，也不碰看门狗寄存器，调用方负责串行化（F407 用 `taskENTER_CRITICAL`）。

```
typedef struct {
    uint32_t required_mask, seen_mask, deadline_ms[8], last_ms[8], start_ms, grace_ms;
    bool valid, grace_active;
} health_t;
```

- **`health_init`** 校验配置：required 不能为 0，只能用低 8 位；每个必需 id 的期限不能为 0，期限和宽限期都必须小于 0x80000000（保证无符号减法比较不会出错）。任何一项不合法就返回 false，`valid` 保持 false，之后 `health_evaluate` 永远返回 `may_feed=false`。
- **`health_progress`** 只接受 required 里的 id，记下时间并置 `seen_mask` 对应位。
- **`health_evaluate`** 先处理宽限期：上电后 1000ms 内，或者所有必需任务都至少报到一次之前，允许喂狗。宽限期一结束，任何必需 id 满足"从没报到过"或"距上次报到 ≥ 期限"，就进 `overdue_mask`：

```
if ((h->required_mask&bit) && (!(h->seen_mask&bit) ||
    (uint32_t)(now-h->last_ms[i])>=h->deadline_ms[i])) out.overdue_mask|=bit;
```

- **`health_supervise`** 第一次看到 `may_feed=false` 就锁存 `restart_latched`，并记下当时的 `first_overdue`。锁存之后永远返回 false：

```
if (!result.may_feed && !s->restart_latched) {
    s->restart_latched=true; s->first_overdue=result.overdue_mask;
}
return !s->restart_latched && result.may_feed;
```

为什么要锁存？worker 卡了 600ms 又自己恢复，偶尔来一次心跳，系统看起来"又好了"。但它已经错过了一次期限，说明有个没被理解的故障。复位决定一旦做出就不撤回，留给复位后的记录去解释。

三、IWDG：四步配置，两个时间尺度

`code/stm32/platform/watchdog.h` 的启动顺序和 HAL v1.8.5 的 `HAL_IWDG_Init` 一致：

```
REG32(IWDG,KR)=0xcccc;          /* 1. 启动；LSI 自动打开 */
REG32(IWDG,KR)=0x5555;          /* 2. 解锁 PR/RLR 写权限 */
REG32(IWDG,PR)=4;               /* 3. /64 */
REG32(IWDG,RLR)=999;
unsigned limit=1000000;while(REG32(IWDG,SR)&&--limit){}   /*    等 PVU/RVU 清零，有界 */
REG32(IWDG,KR)=0xaaaa;          /* 4. 用新值刷新 */
return limit!=0;
```

（代码里写的是基址加偏移：IWDG=0x40003000，KR/PR/RLR/SR 偏移 0/4/8/0xC。）

几个要点：

- **先启动再配置**。0xCCCC 写下去，计数器就从复位值 0xFFF、按复位分频 /4 开始倒数，约 512ms。万一后面的配置失败，狗也已经在跑，最终一定会复位，不会出现"配置失败所以没狗"的状态。
- **等 SR 必须有界**。PVU/RVU 清零要等 LSI 域同步完成。死等写成 `while(SR){}`，LSI 坏了就永远卡在这里。超时返回 false，supervisor 进 `for(;;){}`，让已经启动的狗去复位。
- **启动后不可停止**，只有复位能关掉它。

表 107 实算。 RM0090 Rev 22 第 705 页表 107 给出 32kHz 下 /64 档每个计数 2ms（RL=0 时 2ms，RL=0xFFF 时 8192ms）。RLR=999 即 1000 个计数：

LSI	一个计数	超时 = 1000 × 64 / f_LSI
47kHz（最快）	1.36ms	**约 1.36 秒**
32kHz（名义）	2.00ms	**2.00 秒**
17kHz（最慢）	3.76ms	**约 3.76 秒**

"2 秒"只是名义值。设计时要保证**最快的 LSI** 下（1.36 秒）正常喂狗也来得及，**最慢的 LSI** 下（3.76 秒）的故障检测延迟也能接受。

两个时间尺度别混。 任务期限 500ms 决定"什么时候停止喂狗"，IWDG 超时决定"停止喂狗之后多久复位"。故障到复位的延迟是两者相加，第五节会算一遍。

四、WWDG：窗口两头都是死路

`MODE=1` 换成 WWDG。它的计数器 T 是 7 位，每 `4096 × 2^WDGTB` 个 PCLK1 周期减一，从 0x40 减到 0x3F（T6 位清零）的那一刻复位。RM0090 Rev 22 第 711 页给出的公式：

t_WWDG = t_PCLK1 × 4096 × 2^WDGTB × (T[5:0] + 1)

本工程的配置（`f407_wwdg_start`）：先开 RCC_APB1ENR 的 WWDGEN（bit11）；CFR 写 `(3<<7)|0x5F`，即 WDGTB=3（/8）、窗口 W=0x5F；CR 写 0xFF，即 WDGA=1 启动、T=0x7F。

量	计算	结果
一个计数	4096 × 8 / 16MHz	**2.048ms**
刷新后到复位	(0x3F + 1) × 2.048ms	**约 131ms**
窗口何时打开	T 从 0x7F 减到 0x5E：(0x7F − 0x5E) × 2.048ms	**约 67.6ms**

刷新只能在 0x3F < T < W 时进行。早于窗口刷新（T ≥ 0x5F）会立刻复位，晚了（T 降到 0x3F）也复位。RM0090 第 710 页原话：计数器必须在"小于窗口寄存器值且大于 0x3F"时重载。

`f407_wwdg_feed` 的判断和手册条件逐字对应，`0x3F < T < 0x5F` 才写；不在窗口内就返回 false，不为了"保命"硬喂：

```
uint32_t count=REG32(WWDG,CR)&0x7f;
if(count>0x3f && count<0x5f){REG32(WWDG,CR)=0xff;return true;}
return false; /* Do not refresh early just to keep the MCU alive. */
```

supervisor 每 5ms 看一次。刷新后约 67.6ms 窗口打开，下一次轮询最多再晚 5ms，所以实际刷新间隔约 68~73ms，离 131ms 的复位还有将近 60ms 的余量。期限也得跟着收紧：WWDG 模式下两个任务的期限是 30ms，不能套用 IWDG 的 500ms，否则还没判定超时，WWDG 就已经复位了。

五、故障注入与时间预算

构建	看门狗	故障命令	预期
`MODE=0 FAULTS=0`	IWDG	拒绝（UNSUPPORTED）	正常运行
`MODE=1 FAULTS=0`	WWDG	拒绝	正常运行
`MODE=2 FAULTS=1`	WWDG	`fault --fault 2 --port COMx`	停止正常监督，连写两次 CR，**过早刷新复位**
`MODE=3 FAULTS=1`	WWDG	`fault --fault 2 --port COMx`	停止喂狗，**过晚复位**

任何 FAULTS=1 的构建都可以用 `fault --fault 1 --task 0 --port COMx` 暂停 worker 的进展上报。

MODE=2 为什么要连写两次？第一次写可能恰好落在窗口里，算一次合法刷新，T 被重置为 0x7F；紧接着的第二次一定在窗口之外，必然触发过早复位。只写一次的话，结果取决于那一刻 T 是多少，实验不可复现。

故障命令都**先回确认，200ms 之后才生效**。这和 C8 的规则一样：确认帧入队成功才安排执行，延迟 200ms 让确认帧来得及发出去。

时间预算（MODE=0，`fault 1` 暂停 worker；t0 为设备收到命令的时刻）：

时刻	事件
t0	确认帧入队，安排 200ms 后生效
t0 + 200ms	worker 停止上报，它最后一次进展大约就在这时
t0 + 700ms	supervisor 发现 worker 超过 500ms 期限：锁存重启、保存记录、停止喂狗
t0 + 2.7s	IWDG 名义超时，复位（按 LSI 范围约 2.06~4.46s）

MODE=1 下同样的故障：t0 + 230ms 判定过期，之后再过 58~131ms（取决于上次刷新在多久之前），约 t0 + 0.29~0.36s 复位。量出来的时刻落在预算区间外，就要怀疑时钟配置或调试器冻结（第七节）。

六、复位证据：先读后清，记录只信校验过的

RCC_CSR 读后清。 `f407_reset_reason` 取 RCC+0x74 的高 7 位，然后写 RMVF（bit24）清除：

位	31	30	29	28	27	26	25
标志	LPWRRSTF	WWDGRSTF	IWDGRSTF	SFTRSTF	PORRSTF	PINRSTF	BORRSTF

标志不清除会一直累积，下一次复位时就分不清这次是哪个来源。也别把 RCC+0x74 写成别的偏移（比如把 AHB2ENR 当成复位原因读）。读出来的原值通过 STATUS 的"复位原因"字段原样交给主机。

注意：看门狗复位时 **PINRSTF 通常也会置位**。内部复位源会通过 NRST 引脚输出至少 20μs 的复位脉冲（RM0090 Rev 22 §7.1，第 215 页）。所以判断"是不是看门狗复位"要看 bit29/30，不能因为看到 PINRSTF 就认定是有人按了复位键。这块板上两个标志的实际组合待上板记录。

`.noinit` 记录。 链接脚本里有一个 `.noinit (NOLOAD)` 段，启动代码不清零它。supervisor 停止喂狗前写一条 12 字节记录：

偏移	内容
0..3	magic `0x57444731`（小端，ASCII "1GDW"）
4	version = 1
5	保留，必须为 0
6..9	mask：哪几个任务过期；0x80 表示监督被停止
10..11	对前 10 字节的 CRC-16（与 C8 同一个函数）

写入顺序是"先作废、再写内容、最后写 magic"：

```
for (unsigned i=0;i<4;i++) dst->bytes[i]=0;     /* 先清 magic：写到一半复位 = 无效记录 */
for (unsigned i=4;i<12;i++) dst->bytes[i]=b[i];
for (unsigned i=0;i<4;i++) dst->bytes[i]=b[i];  /* 最后发布 */
```

启动时 `fault_record_take` 只在 IWDGRSTF 或 WWDGRSTF 置位时才解读记录，读完立刻清掉 magic，防止以后某次无关的暖复位又把旧记录读出来。上电复位后 SRAM 是随机值，magic、version、CRC 三关都过不了，就报"没有有效记录"。这条记录只用于辅助诊断暖复位，不保证掉电保存。

HardFault 里同样不能"打印一下、喂一下狗"。栈可能已经坏了，这时候该做的是停住，让狗复位（S16）。工程里的栈溢出钩子就是关中断后死循环。

七、调试器与看门狗

内核被调试器停住时，IWDG/WWDG 是继续计数还是暂停，由 DBGMCU_APB1_FZ（0xE004 2008）的 DBG_IWDG_STOP（bit12）、DBG_WWDG_STOP（bit11）决定（RM0090 Rev 22 §38.16.4）。很多 IDE 和 OpenOCD 配置会在连接时把这两位置 1，断点停住时狗不会叫，方便调试。

麻烦在于：这个寄存器**只被上电复位清零**，普通的系统复位清不掉。调试会话结束后，冻结位可能还留着。这时测"卡死后会不会复位"，得到的"没复位"不代表代码对了。

所以复位实验的固定流程是：烧录 → 断开 ST-Link → 板子断电再上电 → 再发故障命令。

附录：工程完整源码

<<< ../../code/stm32/06-watchdog-health/main.c

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **进展才喂狗，迟到就锁存；IWDG 管太晚，WWDG 还管太早；超时按最慢的 LSI 算，复位原因先读后清，冻结位只有断电才清。**

实物实验

- **装备**：F407 板、ST-Link、3.3V USB-TTL；有示波器的话加一路接 NRST。
- **实验 1（IWDG 时间预算）**：`make MODE=0 FAULTS=1`，按第七节流程上电，执行 `fault --fault 1 --task 0`。记录主机发出命令到 NRST 出现低脉冲的时间，和第五节的 2.06~4.46s 对照。复位后 `status` 的复位原因应含 bit29。
- **实验 2（WWDG 两头）**：分别用 MODE=2 和 MODE=3 执行 `fault --fault 2`。两者复位原因都应含 bit30，区别在复位时刻：MODE=2 在生效（t0+200ms）后的下一轮监督里立刻复位，MODE=3 要等到上次刷新后约 131ms。
- **实验 3（冻结位）**：不断电，直接在调试会话结束后做实验 1，看是否能复位；再断电重做一遍对比。

实验记录

模型/编译验证不等于上板实测。当前待上板；请记录日期、板版本、固件版本、接线、输入、原始输出与结论。

日期	板卡/版本	故障输入	原始结果	结论
待实测				

常见坑

- **定时中断里无条件喂狗**：只证明中断活着（第一节）。
- **调试器连着测复位**：冻结位让狗不叫，以为代码没问题。必须断电重来（第七节）。
- **按名义 LSI 算超时**：最快的 LSI 下 2 秒会变成 1.36 秒，正常喂狗的周期抖一下就误复位（第三节）。
- **IWDG 和 WWDG 用同一套期限**：WWDG 131ms 就复位，500ms 的任务期限永远来不及判定（第四节）。
- **复位标志不清**：标志累积，下一次复位分不清来源（第六节）。
- **故障处理里喂狗**：HardFault 或栈溢出钩子里"顺手喂一下"，等于把唯一能救场的复位关掉了。

短自测

1. 通信口一直没有数据，会触发看门狗吗？

> **▼ 参考答案**
> 不会。通信任务每轮做完一次有界的读取和发送推进就上报进展，"这一轮服务检查完成了"就是它的进展，不要求主机持续发数据。

2. 传感器读数报错、或者收到 STOP 命令，要不要停止喂狗？

▼ **参考答案**
不要。只要任务还能处理这个错误并按期完成下一轮，系统就是健康的，重启解决不了传感器的问题。看门狗只处理"失去进展"，不处理"业务结果不好"。

3. 用 RLR=999、/64，LSI 实际是 40kHz 时 IWDG 超时多少？

▼ **参考答案**
1000 × 64 / 40000 = 1.6 秒。所以说"2 秒"只是 32kHz 下的名义值。

4. WWDG 模式下，如果 supervisor 每 1ms 轮询一次而不是 5ms，会出什么问题吗？

▼ **参考答案**
不会更早复位，因为 f407_wwdg_feed 只在 0x3F < T < 0x5F 时才写，窗口外的轮询直接返回 false。只是 CPU 开销变大，刷新时刻更靠近窗口打开的那一刻。

对照表：本章概念 → 仓库与手册落点

概念	落点
进展、期限、锁存	`code/common/reliability/health.c`
IWDG 四步与有界等待	`code/stm32/platform/watchdog.h` `f407_iwdg_start()`；HAL v1.8.5 `HAL_IWDG_Init`
IWDG 超时表	RM0090 Rev 22 第 705 页表 107；LSI 范围见 DS8626 Rev 12 第 105 页表 35
WWDG 公式与窗口条件	RM0090 Rev 22 第 710–711 页；`f407_wwdg_feed()`
复位标志位	CMSIS `stm32f407xx.h` `RCC_CSR_*RSTF`；`f407_reset_reason()`
暖复位记录	`code/common/reliability/record.c`；链接脚本 `.noinit (NOLOAD)`
调试冻结	RM0090 Rev 22 §38.16.4 DBGMCU_APB1_FZ

延伸阅读

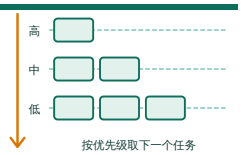
喂狗的方法论：

- **[D12]** Ganssle《Great Watchdog Timers for Embedded Systems》：为什么"主循环里定时喂狗"几乎等于没装狗，以及窗口看门狗（WWDG）的设计动机。

你做到了

- 喂狗从"定时做的一件事"变成"健康检查通过后的结论"；
- IWDG 超时能按 LSI 三档算出区间，WWDG 窗口能算到毫秒；
- 复位后能说清"是谁、因为哪个任务、在什么时候"让它复位的。

✅ 下一站：P13 健康监督：同一份 health 模块搬到 S3，看 TWDT 怎么替你管"谁该喂"。



PART PF

第四部分 FreeRTOS 精讲（内核源码级）

对着 FreeRTOS V11.1.0 源码逐行走读：任务与 TCB、上下文切换、调度器、队列、信号量与互斥量、内存管理，直到亲手把它移植上霸天虎。

> 对着 V11.1.0 的源码逐行走读：xTaskCreate 六步、PendSV 换魂术、O(1) 选优的位图与 CLZ、队列结构体的双生设计、heap_4 何时合并何时永远留洞。RTOS 最怕被讲成 API 说明书，这一部讲的是「内核到底替你做了什么、又付出了什么」。读完的标志：能向别人讲清「中断里为什么不能调用阻塞函数」，并给出源码依据。

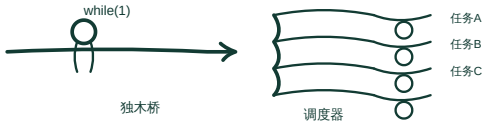
本部 9 章 · 约 385 分钟（6.4 小时）

36 . F0 为什么需要 RTOS：先失败一次
37 . F1 任务与 TCB：xTaskCreate 逐行走查
38 . F2 上下文切换：PendSV 换魂术
39 . F3 调度器：一条 CLZ 选出下一个任务
40 . F4 队列：传送带的源码解剖
41 . F5 信号量与互斥量：优先级反转事件
42 . F6 任务通知、事件组与软件定时器
43 . F7 内存管理：heap_1 到 heap_5 的人生选择
44 . F8 移植 FreeRTOS 到霸天虎：三异常接管全记录

通往单片机之路第 36 章

F0 为什么需要 RTOS：先失败一次

难度 ★☆☆☆☆ · 约 35 分钟



本章罗盘

问题

超级循环明明跑得好好的，凭什么说它注定失败，改成状态机、调换代码行序都救不回来吗？

交付

一份「任务×周期×最坏执行时间」的裸机可行性账（ΣWCET ≤ min(T)），配 probe.sh 实测记录：阻塞版该翻 12 次的灯中位只翻 5 次、15ms 短按五局全丢、2ms 的活摆第一行与最后一行分不出胜负（中位 1.49 vs 1.60 ms）。

边界

时间数字出自宿主机 mingw gcc 五局中位（Windows 随时抢走十几毫秒，单局不可引用），指令条数出自交叉链反汇编（逐位稳定）；真板复现属实物实验、RTOS 修复在 F1，本章只负责「裸机为什么死」，不负责「RTOS 怎么活」。

依据锚点 `mcu-code` · `f1` · `mcu-anim` · `mcu-lab`

> 🎯 学 RTOS 最好的方法不是先背概念，而是**亲手写一个注定失败的多任务裸机程序**：看着它怎么死，你就懂了 RTOS 为什么生。

本章精髓

1. 超级循环的原罪：所有任务共享一个时间轴，任何任务多花一毫秒，其他任务全部晚点；任务越多，"互相拖累"越无解。
2. 时间片轮询的幻觉：用标志位 + 定时器分片，看似"同时"，实则任一任务阻塞（死等延时/等 IO）就全线瘫痪。
3. RTOS 的本质承诺：**每个任务仿佛独占 CPU**：实现这个幻觉只需要三样东西：每任务一个栈、一个 TCB 记账本、一个会保存/恢复现场的切换机制（F1–F2 拆解）。

- **能理解**：为什么"把 `delay` 改成状态机"只是把问题从**等待**挪到了**优先级**：状态机不解决"紧急任务被排在小任务后面"，连调换代码行顺序都救不回来（第三节实测：一件 2 ms 的活摆第一行与摆最后一行，五局最坏迟到的中位是 1.49 ms vs 1.60 ms，两边的区间互相包住，胜负都判不出来）。
- **能用**：拿到需求先算一笔"最坏执行时间 vs 周期"的账，够用就裸机、不够才上 RTOS，不再凭感觉选型。

先修

- S5 SysTick：1ms 节拍与 `LOAD` 的算法，本章的时间账全建立在它之上。
- S7 USART：串口回显这个"会阻塞的任务"从哪来。
- C6 调用约定与栈帧：F1 讲"每任务一个栈"之前，先知道一个栈帧长什么样。

先跑起来（10 分钟 quick win）

在你的 `00-blink` 工程里，把主循环改成三个任务轮着跑：

```
for (;;) {
    led_tick();      /* 1Hz 闪烁：数到 500 翻转一次 */
    key_scan();      /* 按键：按下翻转另一颗灯 */
    uart_echo();     /* 串口：收到完整一行才返回 ← 元凶在这里 */
}
```

烧录之前先押个死活：按住键盘不停发字符，LED 还闪得下去吗？押完再按住不放，让板子宣判。

`uart_echo()` 按"**收满一行才返回**"的阻塞写法实现（死等 `RXNE`）。然后按住键盘不放——**LED 立刻卡壳**。

这不是比喻：`uart_echo` 在等你松手，`led_tick` 根本没机会被调用。用秒表或逻辑分析仪量一下"按键期间的 LED 周期抖动"，把毫秒数记进实验记录，这就是本章的论据。

手上没板子也能拿到同一笔账：本章每个数字都出自 `code/rtos/00-why-rtos/`，一条命令跑完。

```
$ cd code/rtos/00-why-rtos && sh probe.sh
```

`superloop.c` 只有一份，宿主机（mingw gcc 16.1.0）编出来跑 6 秒量时间，交叉链（arm-none-eabi-gcc 15.2.1，`-DBOARD_ARM`）编出来只出反汇编数指令。两边看到的任务代码一字不差，所以宿主机上量到的"谁等谁"结构，就是芯片上的结构。

宿主机不是真板：Windows 会随时抢走十几毫秒，单局的"最坏迟到"能差一个数量级。所以 `probe.sh` 默认**每档连跑 5 局、只报跨局中位数**（`REPS=3 sh probe.sh` 改局数，`REPS=1` 退回单局）。交叉侧的指令条数与反汇编是逐位稳定的，想复现就多跑几次；宿主机侧的时间，只有中位数值得写进结论。

真正的修复不在这一章：F1 会把"三个任务各有自己的栈"这件事造出来，让 LED 在串口忙时照闪不误。

小节结构

小节	内容	四件套
超级循环解剖	三任务版代码逐行；时间轴共享的连锁反应	代码分析
阻塞的传染性	一个 delay 拖死全系统；非阻塞改写的复杂度爆炸	代码分析
时间片改良的极限	定时器分片 + 标志位方案；为什么救不了阻塞	代码分析
RTOS 三件传家宝	栈 = 平行世界、TCB = 户口本、切换 = 换魂术	库解析
裸机够用清单	什么时候**不该**上 RTOS（资源 / 确定性 / 简单任务）	配置

一、超级循环解剖：三个人挤一条传送带

先看这张图：一根轴上的三个任务，和三条轨道上的三个世界。

裸机超级循环 vs RTOS 平行世界动画

先把"多任务"翻译成裸机的样子。三个任务，三种完全不同的时间需求：

任务	周期要求	单次最坏执行时间（实测）	晚了会怎样
`led_tick`	500ms（1Hz 闪烁 = 每 500ms 翻一次）	20 条指令（16 MHz 下 1.3 μs 量级）	灯闪得慢一点，肉眼几乎看不出
`key_scan`	每次都查（≤20ms 一次才不丢按键）	14 条指令	按键漏检
`uart_echo`	有数据就办，无硬周期	**不定**（实测最长单圈 3.299 s，5 局中位）	回显变慢、按键整条丢掉

前两个数字是 `arm-none-eabi-gcc -O2` 反汇编里数出来的行（`probe.sh` 第 3 步，逐位稳定），第三个是宿主机 6 秒一局、连跑 5 局取中位的读数（`probe.sh` 第 1 步），**同一个 `uart_echo`，指令只有 7 条，时间却是无限的**。

超级循环把它们串成一条线：

```
for (;;) {
    led_tick();
    key_scan();
    uart_echo();
}
```

关键在第三列与第一列的对比：前两个任务的单次开销是微秒级，第三个是"不定"：**只要有一个任务的最坏执行时间不可界定，整条循环的周期就不可界定**。这是超级循环的原罪，与代码写得多漂亮无关。

二、阻塞的传染性：一根轴上加不进新东西

`uart_echo` 的阻塞写法（教科书式、也最容易写出来）：

```
void uart_echo(void)
{
    char c = uart_getc();      /* 死等 RXNE */
    if (c == '\n') {
        uart_puts(line);
        line_len = 0;
    } else {
        line[line_len++] = c;
    }
}
```

`uart_getc()` 内部是 `while (!(USART2->SR & USART_SR_RXNE)) { }`。**在收到下一个字节之前，这个循环不会返回，`led_tick` 与 `key_scan` 都不会被调用。**

- 你按住键盘不放 → 不再有新字节进来 → 卡在 `uart_getc` 里 → LED 停摆。
- 就算你正常打字，两个字符之间的间隙也足以让 LED 的相位漂移。

`-O2` 之后这个死等连函数都不剩，整个 `while` 被内联进任务体尾部（`probe.sh` 第 3 步，Cortex-M4 真寄存器版）：

```
000000b4 <uart_echo>:
  b4:  mov.w  r2, #1073758208      @ 0x40004000  USART2 基址
  b8:  ldr.w  r3, [r2, #1024]      @ 0x400       SR
  bc:  lsls   r3, r3, #26          @ RXNE(bit5) 挪到符号位
  be:  bpl.n  b8 <uart_echo+0x4>   ← 字节不来，就永远在这四条里转
  c0:  ldr.w  r0, [r2, #1028]      @ 0x404       DR
  c4:  uxtb   r0, r0
  c6:  b.n    0 <consume>
```

7 条指令，一扇出口，钥匙在对方手里。这 7 条与 `led_tick` 的 20 条挂在同一根轴上，账却完全不对称。

看看代价。宿主机 gcc 16.1.0，6 秒一局、**每档连跑 5 局**，两种模式的**事件脚本一字不改**（600–875 ms 送来 `hello world!`、900 ms 一个换行；4200–5175 ms 送来 40 个 `A`、5498 ms 换行；三次按键分别是 702–717 ms 的 15 ms 短按、1000–4300 ms 的 3.3 s 长按、5499–5514 ms 的短按）。统计量取跨局中位，逐键明细取自末局，日志里 min / 中位 / max 一并给出：

观测量	阻塞版 `uart_echo`	改良版 `uart_poll`
超级循环圈数	55（五局全同）	30 682 621（中位）
单圈 µs：min / max	24 998 / **3 298 958**	0.10 / 3 472
LED 翻转次数（该翻 12 次）	中位 **5**（4–5）	中位 12（11–12）
最长的一次翻灯间隔	**3600.00 ms**	501.47 ms
15 ms 短按（落在字节流里）	**五局全丢**	迟到 0.00 ms
3.3 s 长按（这期间一个字节都没有）	迟到 **3200 ms**	迟到 0.00 ms
短按（落在回显发送里）	迟到 2.47 ms	迟到 2.47 ms
串口收 / 发字节	54 / 52	54 / 52

最后一行两栏完全相同：**阻塞版一个字节都没丢**。它没搞坏数据，它偷的是别人的时间：该翻 12 次的灯中位只翻了 5 次，15 ms 的短按五局局局凭空消失，一次长按晚了 3.2 秒才被看见。

而 3.2 秒既不是随机数、也不是宿主机噪声：五局里最长单圈全是 **3298.96 ms**（min = 中位 = max），最坏迟到钉在同一个数，丢键的局 **5/5**。这一局里长按的迟到是 **3200.00 ms**，短按那 15 ms 干脆整条没被采到。**响应上界 = 一整圈**。改良版那两栏同样自洽：最坏迟到中位 2.47 ms（min = 中位 = max），对照最长单圈中位 3.47 ms，那 3.47 ms 正是 40 个字符回显必须占用的线上时间（115200 8N1 → 86.805 µs/字符 × 40 = 3.472 ms），谁也变不走。

顺带一条诚实注脚：圈数这一栏两档的**稳定性**也完全不同：阻塞版五局都是 55 圈（被事件脚本钉死），改良版五局从 29 174 989 到 31 287 271 圈。宿主机上"一圈多快"会随负载漂，"阻塞那一圈有多长"却纹丝不动，因为它的钥匙在对方手里。

这就是"共享一根时间轴"的具体后果：任务之间不是"先后关系"，而是"连坐关系"：任何一个任务的时间不确定，所有人的时间都不确定。

三、时间片改良的极限：把大任务切碎

自然的想法是：把 `uart_echo` 改成**非阻塞状态机**，每次只处理一点点：

```
void uart_poll(void)            /* 每次进来只查一眼，立刻返回 */
{
    if (!(USART2->SR & USART_SR_RXNE)) return;
    char c = (char)(USART2->DR & 0xFF);
    ...
}
```

循环变成三个人都"很快返回"：

```
for (;;) {
    led_tick();
    key_scan();
    uart_poll();     /* 再也不阻塞 */
}
```

这确实解决了**等待**。但它没解决两件事：

1. **谁先跑**。`key_scan` 和 `led_tick` 之间没有任何优先级概念，只是按书写顺序执行。流行的说法是"你可以靠**调换代码行顺序**来调优先级，把急的活摆到第一行"。这条直觉经不起一次实测：往循环里加一件 2 ms 的活（`host_burn_us(2000)`，顶替一段跑不掉的批量采样），只把它从第一行挪到最后一行，跑同一张事件表、每档连跑 5 局：

	2 ms 的活摆第一行	**同样的活摆最后一行**
最长单圈 5 局：min / 中位 / max	5.47 / **5.49** / 8.13 ms	5.47 / **5.47** / 5.48 ms
三次按键的迟到（末局）	1.85 / 1.27 / 0.37 ms	0.25 / 1.39 / 0.17 ms
最坏迟到 5 局：min / 中位 / max	0.71 / **1.49** / 1.85 ms	0.41 / **1.60** / **5.24** ms
LED 迟到翻灯（>550 ms）	0 次（0 / 5 局）	0 次（0 / 5 局）
丢键的局	0 / 5	0 / 5

中位差 0.11 ms，而摆最后一行的五局区间（0.41–5.24 ms）把摆第一行的（0.71–1.85 ms）整个包住，统计上根本分不出胜负，摆最后一行的最大值 5.24 ms 反而是摆第一行最大值 1.85 ms 的 2.8 倍。**换行序换不来优先级，连"哪种摆法更差"都决定不了**：你只是改了自己在一圈里的站位，而上界由那一圈多长决定（≈5.5 ms），跟站位无关。需求里写的"必须 200 µs 内响应"要的是上界，行序只动得了平均值。 2. **不可预知的抖动**。每次循环的总耗时随分支变化，每个任务的实际启动时刻都在漂移。想量化"最坏情况"就得对**所有分支组合**做最坏分析：任务一多，组合爆炸。上面那 5.47 ms 的最长单圈就是实例：它不是 2 ms 加三个任务的和，而是"2 ms 撞上 3.47 ms 回显"那一个特定组合同时发生（5.47 ≈ 2.00 + 3.47，两个数各自都能对上独立测算）。

还可以再进一步：用 S5 的 SysTick 做"时间片"，给每个任务发标志位：

```
void SysTick_Handler(void) { tick++; }

for (;;) {
    if (tick % 1 == 0) led_tick();
    if (tick % 2 == 0) key_scan();
    uart_poll();
}
```

但这仍然救不了阻塞：如果某个"任务"内部又出现了一次死等，整个时间片调度一起陪葬。而且 `%` 取模、条件判断本身也在消耗时间，改良方案越叠越复杂，可预测性反而更差。

> 结论：裸机侧的所有"改良"都在**同一根时间轴上腾挪**。要真正的并行，需要让每个任务拥有**自己的执行上下文**：这就是 RTOS 要卖给你的东西。

四、RTOS 三件传家宝：一个幻觉的三个零件

"每个任务仿佛独占 CPU" 是一个**幻觉**，它由三个零件构成。这不是比喻，本仓库 `code/rtos/01-freertos-lab/` 里就能逐个指出来：

1. 私栈（平行世界的地基）：每个任务有自己独立的栈，所以一个任务"卡住"时，它的调用栈、局部变量都留在自己的地盘上，不会污染别人。内核里这个字段是 `pxStack` / `pxEndOfStack`，见 `tasks.c:358` 起 `tskTaskControlBlock` 结构体定义（该结构体注释明确写着"旧命名惯例被保留，以免破坏内核感知调试器"）。

2. TCB（户口本）：任务的身份、状态、优先级、栈指针都记在 `tskTaskControlBlock` 里。其中**第一个成员**是：

```
volatile StackType_t * pxTopOfStack;  /* THIS MUST BE THE FIRST MEMBER OF THE TCB STRUCT. */
```

`tasks.c` 里那句"必须是第一个成员"是**硬约束**：切换现场时汇编要靠固定偏移找到它（F2 展开）。同一结构体里还有 `xStateListItem`（挂在哪个状态链表：就绪 / 阻塞 / 挂起）、`uxPriority`（数字越大优先级越高）、`pxEndOfStack`。

3. 切换机制（换魂术）：`PendSV` 异常里保存当前任务的现场、恢复下一个任务的现场。现场从哪来？就是上面那个 `pxTopOfStack`。

这三件东西的成本是实打实的 RAM。看本仓库 lab 的真实配置（`code/rtos/01-freertos-lab/FreeRTOSConfig.h`）：

配置项	真值	行号	含义
`configMINIMAL_STACK_SIZE`	`128`	27	**单位是字（4 字节）**，即 512 字节——这是"最小"任务的栈
`configMAX_PRIORITIES`	`5`	23	优先级 0..4，数字越大越高
`configTICK_RATE_HZ`	`1000`	17	1ms 一个节拍
`configCPU_CLOCK_HZ`	`16000000`	16	本工程跑 HSI 16MHz，不启 PLL
`configTOTAL_HEAP_SIZE`	`16 * 1024`	36	heap_4 的池子，任务栈/TCB/队列都从这里切

"单位是字不是字节"这条在 `FreeRTOS.h:162` 的错误提示里写得很直白：`configMINIMAL_STACK_SIZE defines the size (in words) of the stack allocated to the idle task`。**单位记错一次，栈就是 4 倍或 1/4**：这是新手上 RTOS 最常见的翻车点，F7 有完整的审计方法。

五、裸机够用清单：什么时候不该上 RTOS

RTOS 不是升级，是**换一种成本结构**。它买的是"结构清晰 + 响应有保证"，花的是 RAM（每任务一个栈 + TCB）与切换开销。**小系统上 RTOS 往往更差。**

先算一笔时间账。假设最坏执行时间 `WCET` 与周期 `T` 如下，裸机可行性判据是 `Σ WCET ≤ min(T)`：

场景	任务 A	任务 B	任务 C	裸机够用？
纯点灯	`led_tick` 20 条 / 500ms	—	—	✅ 一个 `for` 循环足够
点灯 + 按键	20 条 / 500ms	`key_scan` 14 条 / 20ms	—	✅ 实测一圈 <1 µs（直方图中位数落在 `<1 µs` 桶）
点灯 + 按键 + 阻塞串口	20 条 / 500ms	14 条 / 20ms	`uart_echo` 7 条，但**不定**（实测最长单圈 3.299 s，5 局中位）	❌ 不可界定
点灯 + 按键 + 非阻塞串口 + 2 ms 采样	20 条 / 500ms	14 条 / 20ms	采样 2 ms / 一圈	⚠️ 上界 = 一圈 ≈ 5.5 ms，够用与否取决于那个 200 µs
三路高速采样 + 通信	各 200µs / 1ms	—	—	❌ 总占用接近周期，必须抢占式

不该上 RTOS 的信号：

- 任务都能在**一个固定周期**内轻松跑完，且没有"必须在 X 微秒内响应"的硬要求；
- Flash/RAM 已经吃紧（每任务栈按 512 字节起算，几个任务就是几 KB）；
- 只有"一件事"，只是这件事比较长，那需要的是**状态机**，不是日程表。

该上 RTOS 的信号：

- 有**硬实时**要求（响应时限与任务数量同时增长）；
- 任务之间**相互独立**、周期各不相同；
- 有"等待"但不想让等待拖垮别人（等信号量、等队列、等事件组）。

> 判据只有一条：**裸机管"顺序"，RTOS 管"期限"。** 需求里出现第二个"必须在……之内"，就该认真考虑 RTOS 了。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **超级循环一根轴，一人阻塞全家堵；RTOS 三件宝：私栈、户口、换魂术。**

延伸：超级循环的局限在 SysTick 节拍（S5）下暴露；RTOS 上下文切换依赖 PendSV（S4）与调用约定（C6）。

实物实验

- 复现 quick win 的失败：三任务超级循环 + `uart_echo` 阻塞写法，按住键盘，观察 LED 停摆；用逻辑分析仪记录"按键期间 LED 周期抖动"的毫秒数，写进实验记录。
- 没板子就 `cd code/rtos/00-why-rtos && sh probe.sh`，四步一条命令跑完：阻塞 vs 非阻塞（第 1 步）、行序当假优先级的对照（第 2 步）、每个任务在 Cortex-M4 上是多少条指令（第 3 步）、死等那三行在 `-O0` / `-O2` 下的两种形状（第 4 步）。本章每一个数字都出自这里；宿主机那两步默认每档跑 5 局（`REPS=1 sh probe.sh` 退回单局，`REPS=8` 更稳），日志末尾直接打 min / 中位 / max。
- 把 `uart_echo` 改成非阻塞状态机（第三节的 `uart_poll`），再测一次抖动，**会发现抖动变小但没消失**（最长单圈从 3.3 s 降到 3.47 ms，回显那一圈的线上时间仍在），这就是"状态机不解决优先级"的实测证据。
- 进阶：`cd code/rtos/01-freertos-lab && mingw32-make DEMO_SCENE=1 -B`，看 F1 用 RTOS 写同一个需求的产物（产物在 `build/scene-1/`）。注意 `FreeRTOSConfig.h:16` 写的是 **16MHz HSI**，本工程不跑 PLL，别拿 Cortex-M4 的标称主频去套。

常见坑

- **把 delay 全改成"非阻塞状态机"就以为够了**：状态机解决等待，不解决优先级，紧急任务照样排队（第三节实测：最坏迟到 = 一整圈，2 ms 的活摆第一行与摆最后一行的五局中位是 1.49 ms 与 1.60 ms，两边的区间互相包住）。
- **把"调换代码行顺序"当优先级用**：行序只改平均启动时刻，不改上界；上界由最长单圈决定，而最长单圈往往由一件你没挪位置的活决定（那 3.47 ms 的回显）。
- **以为 RTOS 会让代码变快**：它买的是"结构清晰 + 响应保证"，成本是 RAM（每任务一栈）与切换开销；`configTOTAL_HEAP_SIZE` 就是这笔账（`FreeRTOSConfig.h:36`）。
- **一上来就十来个任务**：任务越少 RTOS 越优雅；能合并的周期性小任务先用软件定时器（F6）。
- **混淆栈深单位**：`configMINIMAL_STACK_SIZE` 的单位是**字**不是字节（`FreeRTOS.h:162` 明确写了 "in words"），128 是 512 字节。
- **拿宿主机单局的数当结论**：同一份代码、同一张事件表，宿主机上一局的"最坏迟到"能比另一局大一个数量级（Windows 随时抢走十几毫秒）。本章只引用跨局中位数；反过来，交叉链的指令条数与反汇编是逐位稳定的，那种数才是结构，跑一次就够。

短自测

1. 超级循环里三任务都不阻塞，能保证响应时间吗？为什么？

> **▼ 看答案**
> 不能。不阻塞只保证"每个任务最终会被执行"，不保证**什么时候**执行。每个任务的启动时刻等于"前面所有任务本次耗时之和"，而这些耗时随分支变化，你只能保证一个上界，且任务越多这个上界越接近（甚至超过）最小周期。要**期限**保证就需要抢占式调度。

2. 为什么"把 `uart_echo` 改成状态机"不能彻底解决问题？

> **▼ 看答案**
> 状态机消除的是**等待**，不是**优先级**。改完之后每个任务都很快返回，但它们仍然按书写顺序串行执行：`key_scan` 要等 `led_tick`，`led_tick` 要等 `uart_poll`。出现"必须 200µs 内响应按键"这类硬要求时，调换代码行顺序也没用：实测把一件 2 ms 的活从第一行挪到最后一行，五局最坏迟到的中位从 1.49 ms 变成 1.60 ms（区间分别是 0.71–1.85 与 0.41–5.24，后者把前者整个包住，统计上分不出胜负），而上界始终是"一整圈"（中位 5.47 ms）。要**期限**保证就需要抢占式调度。

3. RTOS 的三件传家宝各解决什么问题？

> **▼ 看答案**
> **私栈**：让每个任务有独立的调用栈与局部变量，一个任务卡住不污染别人（内核字段 `pxStack`/`pxEndOfStack`）；**TCB**：任务的"户口本"，记录状态/优先级/栈顶指针（`tskTaskControlBlock`，第一个成员必须是 `pxTopOfStack`）；**切换机制**：`PendSV` 里保存/恢复现场，让"幻觉"得以在物理上实现。

4. `configMINIMAL_STACK_SIZE = 128` 是多少字节？

> **▼ 看答案**
> **512 字节**（128 字 × 4 字节/字）。`FreeRTOS.h:162` 的错误提示写明 "defines the size (in words)"，单位是**字**。本条来自本仓库 `code/rtos/01-freertos-lab/FreeRTOSConfig.h:27` 的真实值。

5. 什么信号说明"不该上 RTOS"？

> **▼ 看答案**
> 任务能在**一个固定周期**内轻松跑完、没有"必须在 X 微秒内响应"的硬要求、Flash/RAM 吃紧、或者"其实只有一件事只是比较长"（那需要状态机而非日程表）。判据：**裸机管顺序，RTOS 管期限。**

6. 为什么"最长单圈 3.299 s"可以直接写进结论，"最坏迟到 2 ms 还是 5 ms"却要多跑几局取中位？

> **▼ 看答案**
> 因为两者的**来源**不同。3.299 s 是阻塞那一圈的时间，长度由事件脚本决定（40 个字符在 4200–5175 ms 间陆续到达，下一个字节 5498 ms 才来），五局的 min / 中位 / max 全等于同一个数，它是**结构**。而"最坏迟到"在改良版里取决于宿主此刻被 Windows 抢走了多少时间，同一档连跑五局能从 0.41 ms 到 5.24 ms，它是**负载**，只有统计量可信。同一件事在芯片上：交叉链反汇编出的 20 / 14 / 7 条指令逐位稳定，跑一次就够；真板上的迟到要跑一批看分布。

对照表：本章概念 → 仓库落点

概念	仓库落点
超级循环三任务版（失败现场，同一份源码两边编）	`code/rtos/00-why-rtos/superloop.c` + `superloop.h`
本章所有数字怎么复核	`code/rtos/00-why-rtos/probe.sh` 四步一条命令跑完（阻塞对照 / 行序对照 / 指令条数 / `-O0` 与 `-O2` 的死等）；`REPS=n` 控制每档局数，默认 5
宿主机那层：事件脚本、直方图、翻灯与按键迟到	`code/rtos/00-why-rtos/board_host.c`、`main_host.c`（`block` / `poll` / `burn-head` / `burn-tail` 四模式）
跨局中位数怎么算出来的（min / 中位 / max 那一屏）	`board_host.c` 的 `run_stats_t` + `median_raw()` + `summary_pick()`；`main_host.c` 的 `REPS_DEFAULT`
Cortex-M4 那层：真寄存器 `static inline` 实现	`code/rtos/00-why-rtos/board_arm.h`（USART2 `0x40004400` / GPIOD `BSRR 0x40020C18`）
RTOS 版同一需求（真工程）	`code/rtos/01-freertos-lab/main.c` 场景 1（`DEMO_SCENE=1`）
任务私有栈字段（`pxStack` / `pxEndOfStack`）	`.trellis/ref/freertos-v11/kernel/tasks.c:358`（`tskTaskControlBlock`）
TCB 首成员硬约束（`pxTopOfStack` 必须第一个）	同上，结构体首字段及其 "MUST BE THE FIRST MEMBER" 注释
任务状态链表 / 优先级字段	同上，`xStateListItem` 与 `uxPriority`
栈深单位是「字」不是「字节」	`.trellis/ref/freertos-v11/kernel/include/FreeRTOS.h:162`
真实配置：栈 128 字 / 优先级 5 / 1kHz / 16MHz HSI	`code/rtos/01-freertos-lab/FreeRTOSConfig.h:16,17,23,27`
heap 池子 16KB（任务栈 + TCB 都从这里切）	同上 `:36`
SysTick 1ms 节拍的算法	S5；`core_cm4.h` 的 `SysTick_Config`

你做到了

- 亲手制造并测量了超级循环的失败；
- 能算一笔"WCET vs 周期"的账，说清裸机什么时候够用；
- 能说清 RTOS 买什么、花什么，不再为"要不要上 OS"纠结。

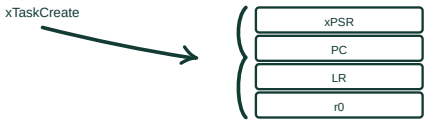
对答案：LED 死得不冤。一条阻塞的串口调用把三个任务共同的时间轴冻在原地，而 RTOS 卖给你的第一件货，就是给每个任务一条自己的时间轴。

✅ 下一站：F1 任务与 TCB：xTaskCreate 逐行走查，看"平行世界"怎么造出来。

通往单片机之路第 37 章

F1 任务与 TCB：xTaskCreate 逐行走查

难度 ★★★☆☆ · 约 30 分钟



本章罗盘

问题

从没被中断打断过的新任务，第一口栈是谁摆好的，凭什么 PendSV 一弹栈它就「开跑」了？

交付

xTaskCreate 六步走查（分栈→分 TCB→化妆→填本→挂货架→触发调度）+ GDB 实测：x/16xw 指认新任务栈里的 xPSR=0x01000000、PC、LR=prvTaskExitError、r0，再附一份 N 任务的 RAM 账单（TCB＋栈，单位是字不是字节）。

边界

行号对 FreeRTOS-Kernel V11.1.0 ARM_CM4F 端口说话（tasks.c:358 / port.c:202）；ESP-IDF 内置的 V10.5.1 SMP 修改版不同源，行号与结论不混读；栈深只给估算公式与水位实测方法，具体数值要自己上板量。

依据锚点 `f1` · `mcu-code` · `mcu-anim`

> 🎯 创建任务像"克隆一个你"去平行世界：克隆体醒来时要以为自己一直在那儿干活，所以 FreeRTOS 把它的栈**伪装成"刚被中断打断"的样子**。看懂这个伪装，就懂了任务的一半。

本章精髓

1. **TCB（tskTCB）是任务户口本**：栈顶指针 `pxTopOfStack` 永远排第一（上下文切换直接按 TCB 地址取栈）、栈底 `pxStack`、优先级、状态列表项、任务名，`tasks.c` 里逐字段看得完（V11.1.0 `tasks.c:358`）。
2. **栈初始化是一场"化妆"**：`pxPortInitialiseStack`（`portable/GCC/ARM_CM4F/port.c:202`）按异常帧格式把 xPSR/PC(=任务函数)/LR/参数 r0 摆进新栈：第一次切换时，弹栈弹出来的就是"任务开始跑"。初始 xPSR = `0x01000000`（`port.c:92` 的 `portINITIAL_XPSR`，Thumb 位）。
3. **就绪列表按优先级分数组**：`pxReadyTasksLists[prio]` 是链表数组，创建任务 = 挂进对应优先级的链表尾巴（F3 调度器的货架）。

- **能理解**：为什么 `pxTopOfStack` 必须排第一；为什么任务函数不许 return；为什么 INCLUDE_* 宏会让老工程升级后链接失败。
- **能用**：对照源码讲出 xTaskCreate 的六步；用 GDB 指认新任务栈里的 xPSR/PC/r0。

先修

- F0 为什么需要 RTOS、C6 栈帧、B4 启动。

先跑起来（10 分钟 quick win）

`code/rtos/01-freertos-lab` 场景 1（默认）：

```
make
make flash
```

串口 115200 看日志：`high` 每 100ms 一行、`low` 每 300ms 一行：**先建低优先级、再建高优先级，high 一建好就抢占**，抢占式调度直接上演。每 2 秒还会打一张 `vTaskList` 任务状态表（R/B 状态列 + 剩余栈高水位）。

烧录前先押个头一行：low 先创建、high 后创建，串口打出的第一行日志，落款是谁？押完再看 115200 的输出。

动画：新任务的第一口栈

栈顶压入的不是"参数"，是一套**完整的异常现场**：xPSR、PC（指向任务函数）、LR、r12、r3~r0（r0 是任务参数），然后 SP 退到最低。第一次 PendSV 弹栈时，弹出来的就是"任务函数开始跑"。

任务创建与栈初始化动画

版本与配置前提

- 内核：上游 **FreeRTOS-Kernel V11.1.0**，ARM_CM4F 单核端口（`portable/GCC/ARM_CM4F/port.c`）。
- 配置：本工程 `FreeRTOSConfig.h`，`configTICK_RATE_HZ=1000`、`configMAX_PRIORITIES=5`、`configMINIMAL_STACK_SIZE=128`（单位是**字**）。
- 与 ESP-IDF 内置的 V10.5.1 SMP 修改版不同源，行号与结论不混读。

小节结构

小节	内容	四件套
一、TCB 逐字段	pxTopOfStack 必须第一的汇编级原因	库解析
二、栈的化妆术	pxPortInitialiseStack 逐行；初始 xPSR 为什么是 0x01000000	库解析
三、创建六步	xTaskCreate 源码走查	库解析
四、就绪货架	链表数组结构；同级任务的轮转	配置
五、RAM 账单	栈怎么估（调用深度×帧+余量）；水位标记实测	代码分析
六、代码分析	`01-freertos-lab` 场景 1 的日志对照	代码分析

一、TCB 逐字段：户口本的第一行

`tskTaskControlBlock`（V11.1.0 `tasks.c:358`）开头几个字段：

```
typedef struct tskTaskControlBlock
{
    volatile StackType_t * pxTopOfStack;   /* 当前栈顶：切换时的存取锚点 */
    ListItem_t xStateListItem;             /* 挂在哪个状态链表上 */
    ListItem_t xEventListItem;             /* 等哪个事件（队列/信号量/…） */
    UBaseType_t uxPriority;                /* 数值越大优先级越高 */
    StackType_t * pxStack;                 /* 栈底（溢出检查用） */
    char pcTaskName[ configMAX_TASK_NAME_LEN ];
    ...
} tskTCB;
```

`pxTopOfStack` 排第一不是排版习惯，是**汇编级的硬约定**：上下文切换的 PendSV 处理器拿到 TCB 地址后，第一个字就是"当前栈顶"，不用知道结构体其余部分的布局就能完成存取。这个顺序写死了，改不得。

二、栈的化妆术：`pxPortInitialiseStack`

`port.c:202` 的 `pxPortInitialiseStack()` 把一个新栈伪装成"刚被中断打断"的现场：

```
*pxTopOfStack = portINITIAL_XPSR;        /* 0x01000000：Thumb 位，必须是 1 */
pxTopOfStack--;
*pxTopOfStack = ( StackType_t ) pxCode;  /* PC：任务函数入口 */
pxTopOfStack--;
*pxTopOfStack = ( StackType_t ) prvTaskExitError; /* LR：任务不许 return，
                                                     返回就掉这里 */
pxTopOfStack -= 5;                        /* r12, r3, r2, r1 留白 */
*pxTopOfStack = ( StackType_t ) pvParameters;     /* r0：任务参数 */
pxTopOfStack -= 8;                        /* r11~r4 留白 */
```

解读三个关键选择：

1. **xPSR = 0x01000000**：Thumb 位（bit24）。Cortex-M 只支持 Thumb 指令集，弹栈恢复时若这一位是 0 直接 UsageFault，所以初始值必须硬编码成 0x01000000。
2. **PC = 任务函数**：弹栈完成后，处理器"接着执行"的就是任务函数：任务就这样出生了。
3. **LR = `prvTaskExitError`**：任务函数的"返回地址"指向一个死循环断言。**任务不许 return**：这是这条规则的物理来源。

三、创建六步：`xTaskCreate`

`tasks.c:1718`（V11.1.0）的 `xTaskCreate` 干六件事：

1. **分栈**：`pvPortMalloc( uxStackDepth × sizeof(StackType_t) )`，注意单位是**字**（`StackType_t` = `uint32_t`，`portmacro.h:55`），传 128 就是 512 字节，不是 128 字节；
2. **分 TCB**：`pvPortMalloc( sizeof(TCB_t) )`；
3. **初始化栈**：`prvInitialiseNewTask` → `pxPortInitialiseStack`（上节那场化妆）；
4. **初始化 TCB**：名字、优先级、列表项、栈底；
5. **挂就绪表**：`prvAddTaskToReadyList` 挂进 `pxReadyTasksLists[prio]` 的尾巴；
6. **触发调度**：新任务优先级高于当前任务就 `taskYIELD`："先建低优先级、再建高优先级，high 一建好就抢占"就是这一步。

四、就绪货架：链表数组

就绪列表不是一条大链表，是**按优先级分格的链表数组** `pxReadyTasksLists[0..configMAX_PRIORITIES-1]`：

- 同优先级的任务在同一格里轮转（时间片，`configUSE_TIME_SLICING=1`）；
- 调度器从高往低扫，第一个非空格的链表头就是下一个跑的，**O(1) 找到最高优先级就绪任务**，与任务总数无关。

五、RAM 账单：栈怎么估

每个任务的 RAM 账单 = TCB（几十字节）+ 栈（`uxStackDepth` 字）。栈怎么估：

- **下限**：调用链最深处的帧深 × 每层帧大小 + 中断嵌套余量；
- **实测**：FreeRTOS 建任务时把栈填 0xA5 水位线，运行后 `uxTaskGetStackHighWaterMark()` 读"历史最低剩余"，用真实水位把栈预算从玄学变工程；
- **护栏**：`configCHECK_FOR_STACK_OVERFLOW=2` 开满，溢出会触发 `vApplicationStackOverflowHook`（本工程打印任务名后停住）。

六、代码分析：`01-freertos-lab` 场景 1

`scene_start()` 里三个任务：`low`(prio 1)、`high`(prio 2)、`rpt`(prio 1)。日志里能看到：

- `high` 的 tick 间隔稳定 100ms，`low` 稳定 300ms：优先级决定的是"谁先跑"，不是"谁跑得快"；
- `vTaskList` 表里的 `R`（运行）/ `B`（阻塞）状态列与剩余栈高水位；
- 把 `high` 的 delay 改成忙等，`low` 就再也跑不到，这是**抢占式 + 无时间片让出**的直接证据。

七、任务状态机：五个状态与两条路

任务在五个状态间流转：**就绪（Ready）→ 运行（Running）→ 阻塞（Blocked）→ 挂起（Suspended）→ 删除（Deleted）**。

```
         xTaskCreate
              ↓
[Ready] ←→ [Running] → vTaskDelay/xQueueReceive → [Blocked]
   ↑           ↓                                      ↓
   └───────────┘ ← 调度器切换                  xEventGroupSetBits/
                                                xQueueSend 到期
                                                      ↓
                                                  [Ready]
```

两条创建路径：

路径	API	栈/TCB 来源	适合
动态	`xTaskCreate`	`pvPortMalloc`（F7）	运行时决定任务数、栈深
静态	`xTaskCreateStatic`	调用者提供 `StaticTask_t` + 栈数组	安全关键、禁止动态分配

静态创建的 TCB 和栈在编译期确定位置，**不会分配失败**：医疗/汽车产品常用。代价是每个任务的 `StaticTask_t` 和栈数组要显式声明，代码量更大。

八、GDB 实战：指认新任务的第一口栈

① 连接目标

```
# 启动 OpenOCD + GDB 连接
openocd -f interface/stlink.cfg \
-f target/stm32f4x.cfg
(gdb) target remote :3333
```

✓ 连接成功

② 查看当前任务

```
# 当前任务 TCB 指针
(gdb) p pxCurrentTCB
$1 = (tskTCB*) 0x20001a80
(gdb) p pxCurrentTCB->pcTaskName
```

⚠ 正在运行 high 任务

③ 检查栈内容

```
# 栈顶指针
(gdb) p pxCurrentTCB->pxTopOfStack
$3 = (StackType_t*) 0x20001b80
(gdb) x/16xw $3
```

⚠ Thumb 位 = 1 ✓

④ 设置断点

```
# 在任务函数打断
(gdb) break high_task
Breakpoint 1 at 0x8000189
(gdb) continue
```

✓ 断点命中

FreeRTOS 调试常用命令

`p pxCurrentTCB` → 当前任务 TCB
`p pxCurrentTCB->uxPriority` → 当前优先级
`x/16xw pxCurrentTCB->pxTopOfStack` → 栈内容（指认 xPSR/PC/LR）

调试纪律

1. **断点会改变时序**：实时任务调试时断点暂停会让其他任务"饿死"
2. **看门狗会咬人**：调试暂停时 TWDT 可能超时复位——调试前禁用看门狗
3. **用 watchpoint 抓变量**：值变化时自动暂停，比断点更精准

FreeRTOS GDB 调试实战

```
arm-none-eabi-gdb build/firmware.elf
(gdb) break vTaskStartScheduler
(gdb) continue
(gdb) p pxCurrentTCB->pxTopOfStack
$1 = (StackType_t *) 0x20001a80
(gdb) x/16xw $1
  0x20001a80: 0x01000000  ← xPSR (Thumb 位)
  0x20001a84: 0x08000189  ← PC (任务入口)
  0x20001a88: 0x080002a1  ← LR (prvTaskExitError)
  0x20001a8c: 0x00000000
  0x20001a90: 0x00000000  ← r0 (任务参数)
```

对照 `port.c:202` 的 `pxPortInitialiseStack` ：**每行代码都能在栈里找到对应的字**。这就是"化妆术"的物理证据。

附录：工程完整源码

<<< ../../../code/rtos/01-freertos-lab/main.c

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **TCB 是户口本，栈顶指针写第一行；新栈化个"中断妆"，一弹栈任务就出生；栈深按字不按字节，INCLUDE 宏默认全关。**

延伸：任务栈与内存模型（C1）直接相关；TCB 结构体与 ABI 对齐（C4）影响布局；GDB 查看 TCB 实战见 B6。

实物实验

- 场景 1 日志对照： `high` / `low` 的 tick 间隔与 `vTaskList` 表；
- GDB 现场： `p pxCurrentTCB` 、 `x/16xw pxCurrentTCB->pxTopOfStack` ，指认 xPSR(0x01000000)/PC/r0；
- 翻车实验（选做）：把某个任务的栈深从 256 改到 64，看 `vApplicationStackOverflowHook` 抓到谁。

常见坑

- **栈给太小**：不报错，悄悄踩到隔壁任务/堆，症状是"莫名其妙 HardFault"（S16 取证）；
- **任务函数返回**：返回地址指向 `prvTaskExitError` 死循环断言：结尾必须 `vTaskDelete(NULL)` 或永远不返回；
- **优先级乱排**：数值越大优先级越高（与 NVIC 中断优先级"数值越小越高"相反！），两套规则混用是经典翻车；
- *INCLUDE_ 默认关**：V11 内核默认 `INCLUDE_vTaskDelay=0` 等，老工程升级后链接报 `undefined reference to vTaskDelay` ，不是代码坏了，是宏没开；
- **栈深单位是字**：传 128 是 512 字节，不是 128 字节， `configMINIMAL_STACK_SIZE` 同理。

短自测

1. 为什么 `pxTopOfStack` 必须是 TCB 的第一个字段？

> **▼ 参考答案**
> 上下文切换的 PendSV 处理器拿到 TCB 地址后，第一个字就是"当前栈顶"，不用知道结构体其余布局就能完成存取。这个顺序是汇编级的硬约定。

2. 新任务的初始栈里 xPSR 为什么是 0x01000000？

> **▼ 参考答案**
> 那是 Thumb 位（bit24）。Cortex-M 只支持 Thumb 指令集，异常返回弹栈时若这一位是 0 就 UsageFault，所以 `portINITIAL_XPSR` 硬编码为 0x01000000（port.c:92）。

3. 任务函数的"返回地址"指向哪里？为什么任务不许 return？

> **▼ 参考答案**
> 指向 `prvTaskExitError`，一个死循环断言。任务函数 return 就会掉进去；正确写法是 `vTaskDelete(NULL)` 或永远循环。

4. `xTaskCreate` 的栈深参数单位是什么？传 128 实际占多少字节？

▼ **参考答案**
单位是 `StackType_t`（字），ARM_CM4F 上是 `uint32_t`（portmacro.h:55）。传 128 = 512 字节，不是 128 字节。

对照表：本章概念 → 仓库与上游落点

概念	落点
TCB 结构体	上游 V11.1.0 `tasks.c:358`（tskTaskControlBlock）
栈初始化化妆术	上游 V11.1.0 `portable/GCC/ARM_CM4F/port.c:202`（pxPortInitialiseStack）、`port.c:92`（portINITIAL_XPSR）
创建六步	上游 V11.1.0 `tasks.c:1718`（xTaskCreate）→ `tasks.c:1793`（prvInitialiseNewTask）
就绪链表数组	上游 V11.1.0 `tasks.c`（pxReadyTasksLists）
场景实验	code/rtos/01-freertos-lab 场景 1
动画	task-create-stack.svg

你做到了

- 任务的"出生"在你眼里是透明的六步；
- 栈与 TCB 不再抽象：你会算、会看、会量；
- V11 内核的 INCLUDE_* 门控陷阱，你以后不会再踩。

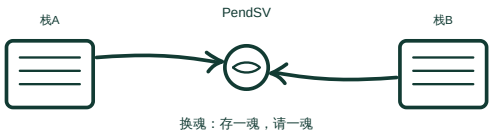
头一行判给了 high：出生顺序说了不算，优先级说了算。low 先出生，也只是在就绪链表里排了个队；读内核先看结构再排队形，比背 API 口诀可靠。

✅ 下一站：F2 上下文切换：PendSV 逐汇编指令，换魂术全公开（动画）。

通往单片机之路第 38 章

F2 上下文切换：PendSV 换魂术

难度 ★★★☆☆ · 约 40 分钟



本章罗盘

问题

PendSV 里那几十条指令凭什么能冻结一个任务的世界，哪些寄存器硬件存、哪些软件存、FPU 何时才存？

交付

一张异常入栈帧布局图和一份 xPortPendSVHandler 逐行注解，标出 r4-r11、s16-s31 各自由谁压栈。

边界

本章讲 CM4F 端口；带 MPU 的隔离切换与 RISC-V 端口不在本章，SMP 多核切换更不能从本章外推。

依据锚点 `a6 · c5 · f1 · mcu-anim`

> 🎯 两个任务"同时在跑"的幻觉，每毫秒发生一次偷梁换柱：把当前任务的魂（寄存器）抽走封进它的栈，把下一个任务的魂从它的栈里请回来。执刑官是 PendSV——全系统优先级最低的中断。本章把这段编出来只有 **27 条机器指令、104 字节**的 naked 函数逐行过堂。

动手前先押个响不响：断点压在内核源码写的 xPortPendSVHandler 上，切一次任务，断得住吗？押完进第五节的目击实验亲手试。

本章精髓

1. 为什么是 PendSV 且必须最低：切换要等所有中断服完役才进行，否则 ISR 被"切走一半"就乱套：最低优先级保证 PendSV 永远最后执行。**而且这个"最低"不用你配**：V11 的 ARMv7-M 移植层自己硬写成 255，`configKERNEL_INTERRUPT_PRIORITY` 在这个端口上是不被读取的死宏（第一节翻案）。
2. 切换就两个动作：**存**（当前任务的 r4-r11 软件补压 + 栈顶存进 TCB）与**取**（从下个任务 TCB 取栈顶 + r4-r11 弹回），硬件帧（r0-r3 等）由异常进出自动完成，软件只需管 r4-r11。
3. 选择下一个任务靠 `pxCurrentTCB` 换指针：`vTaskSwitchContext` 只是改了这个全局变量，PendSV 按新值取栈，"换魂"完成。
4. **源码名和板子上的名字不是一回事**：内核里写的是 `xPortPendSVHandler`，`FreeRTOSConfig.h:76` 一行 `#define xPortPendSVHandler PendSV_Handler` 把它改名，启动文件向量表里躺着的才是 `PendSV_Handler`，GDB 断点要断后者，断前者什么都断不到（第五节踩过）。

- **能理解**：为什么硬件管一半、软件管另一半（第二节，AAPCS 的账）。
- **能用**：在 GDB 里给 PendSV 打断点，把动画里的每一帧在真板上找到（第五节，注意断点要断改名后的符号）。

先修

- F1 任务与 TCB、S4 中断现场、C6 调用约定。

动画：换魂全程

TaskA 的魂（r4-r11）被压进它自己的栈、栈顶藏进 TCB；换指针；TaskB 的魂从它的栈里弹回，注意两个栈此消彼长，CPU 还是那个 CPU。

上下文切换动画

配套视频

> 🎬 **野火《FreeRTOS 内核实现与应用开发实战指南》配套视频（42 集，从 0 到 1 写内核）** @ 哔哩哔哩
> 离线书不内嵌在线视频；去 哔哩哔哩 观看 ↗

小节结构

小节	内容	四件套
一、PendSV 定位	为什么最低优先级；与 SysTick 的配合	配置
二、软硬分工	硬件帧 vs r4-r11：AAPCS 的账（回 C6）	代码分析
三、汇编逐行	十一条语义步骤、编出来 27 条指令，逐条过堂	库解析
四、换指针即换魂	vTaskSwitchContext 与 TCB 第一成员	库解析
五、GDB 目击	断点 + 双任务栈对照实验（含名字改名坑）	代码分析
六、第一帧是伪造的	没跑过的任务，栈帧由 pxPortInitialiseStack 手工造	库解析

一、PendSV 定位：执刑官为什么永远最后上场

SysTick 每 1ms 来一次，但它的活只是**报案**：`xTaskIncrementTick()` 发现"有更高优先级任务就绪了"，就写一笔挂起位：

```
ICSR（0xE000ED04）bit28 = PENDSVSET：把 PendSV 异常挂起
```

报案这条在本机两份源码里都能对上号，不用背：

```
/* portmacro.h:99-100 —— 寄存器地址和位号就是章首那行写的 0xE000ED04 / bit28 */
#define portNVIC_INT_CTRL_REG   ( *( ( volatile uint32_t * ) 0xe000ed04 ) )
#define portNVIC_PENDSVSET_BIT  ( 1UL << 28UL )

/* portmacro.h:91 —— portYIELD_WITHIN_API() 的全部动作，就是写这一笔 */
portNVIC_INT_CTRL_REG = portNVIC_PENDSVSET_BIT;
```

位号本身另有架构出处：CMSIS 的 `core_cm4.h:490` 写死 `SCB_ICSR_PENDSVSET_Pos = 28U`（同文件 `:487` 的 `NMIPENDSET_Pos` 才是 31U，别记混）。

而"最低优先级"也不是只写在配置文档里，是启动时真写进去的两行（`port.c:434-435`）：

```
portNVIC_SHPR3_REG |= portNVIC_PENDSV_PRI;    /* port.c:66 → portMIN_INTERRUPT_PRIORITY << 16 */
portNVIC_SHPR3_REG |= portNVIC_SYSTICK_PRI;   /* port.c:67 → portMIN_INTERRUPT_PRIORITY << 24 */
```

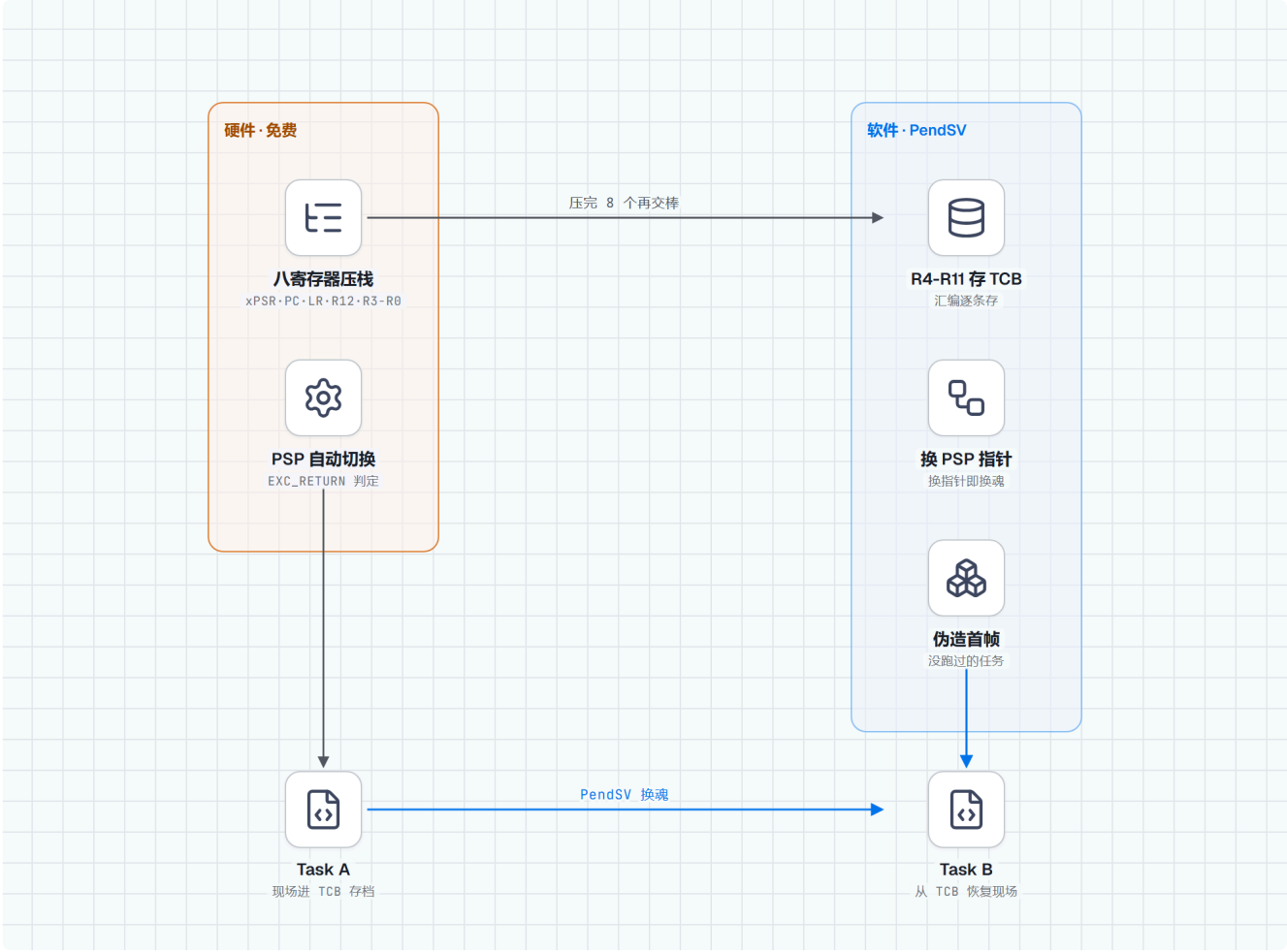
`portMIN_INTERRUPT_PRIORITY` 就是 `configKERNEL_INTERRUPT_PRIORITY`（255），Shift 16 / 24 对应 SHPR3 里 PendSV 与 SysTick 各自的 8 位槽，两个都填满 0xFF，全系统最低。

真正的执法交给 PendSV。为什么绕这一手？因为 SysTick 优先级高，**若在 ISR 堆里直接切换，等于把某个中断服务程序切走一半**：外设寄存器读到一半、标志位清到一半，回来全乱。PendSV 的优先级是 **0xFF，全系统最低**：上面 `:434` 那两行就是写这个值的现场：它永远排队到最后，等所有 ISR 收队完毕，才出来执行切换。

> **【注】"把 `configKERNEL_INTERRUPT_PRIORITY` 配错"这个坑，在 ARMv7-M 的新内核上已经不存在了。** 本仓库用的 V11.1.0 里，内核自己把两个优先级硬写成最低（`port.c:65` `portMIN_INTERRUPT_PRIORITY ( 255UL )` → `:434-435` 落到 SHPR3），**根本不读这个宏**。内核自己的变更记录把话说得很白（`History.txt:301-303`）："*Update ARMv7-M ports to ensure that kernel interrupts run at the lowest priority. configKERNEL_INTERRUPT_PRIORITY is now obsolete for ARMv7-M ports.*" 所以本站工程里 `FreeRTOSConfig.h:61` 那行 255 是一段无害的死代码，留着是为了兼容老教程的口径，删了也不会有行为变化。
>
> **真正会炸的是 `configMAX_SYSCALL_INTERRUPT_PRIORITY`**，它被 `:525` 的 `mov r0, %0` 代进汇编、又被 `vPortSetupTimerAndInterrupts` 连查三遍（`configASSERT_DEFINED == 1` 时）：
>
> | 断言 | 出处 | 配错的后果 |
> |---|---|---|
> | `configASSERT( ucMaxSysCallPriority )` | `port.c:379` | 填 0 → 优先级比所有中断都高，临界区关不掉任何东西 |
> | `( configMAX_SYSCALL_INTERRUPT_PRIORITY & ~ucMaxPriorityValue ) == 0` | `port.c:383` | 写了超出硬件优先级位宽的数 → 低位被吃掉，语义漂移 |
> | `( configMAX_SYSCALL_INTERRUPT_PRIORITY & 0x1U ) == 0` | `port.c:413` | 填奇数 → 某些芯片上会漏掉一批本该被屏蔽的中断（注释举例：设成 5 时 4 和 5 的行为反直觉） |
>
> **开机就断言炸，比"一周崩一次"好得多**。老说法里那个"偶发灵异 bug"，今天的形态是启动即死，除非你把 `configASSERT` 定义成空，那它才会退化成人格分裂式的偶发崩溃。常见坑一节还会点名。

二、软硬分工：硬件管一半，软件管另一半

先看分工图：左列是芯片免费送的，右列是 xPortPendSVHandler 一条条汇编挣来的。



PendSV 换魂软硬分工：八寄存器压栈与 PSP 切换归硬件，R4-R11 存 TCB 与换 PSP 归软件

异常入场时，硬件已自动压好 8 个字的**硬件帧**（S4 动画演示过）：xPSR、PC、LR、R12、R3~R0。这是 AAPCS 的"调用者保存"部分，硬件替你管了。

那 **r4-r11** 呢？AAPCS 说它们是"被调者保存"：函数要用就自己存。硬件不认识这个约定，所以 PendSV 里**软件手工补压 r4-r11**。软硬两半合起来，才是一个任务完整的"魂"。

FPU 再加一层：若任务用过浮点（EXC_RETURN **bit4 = 0** 为证），硬件帧会扩展带上 s0-s15+FPSCR，软件还要加存 **s16-s31**。

> 【注】**懒堆叠（lazy stacking）**：M4F 的 FPCCR（LSPEN 默认开）让硬件"先占座、后写真"：入场时先给 FPU 帧**预留空间**，真用到 FPU 才写数，大多数不用浮点的中断省下这笔开销。所以 EXC_RETURN bit4 是"这趟有没有 FPU 行李"的牌子。

三、汇编逐行：xPortPendSVHandler 过堂

以下摘自本机 `.trellis/ref/freertos-v11/kernel` 的 FreeRTOS-Kernel **V11.1.0** `portable/GCC/ARM_CM4F/port.c:509-556`（M4F + GCC 移植层的 naked 内联汇编，注释为本章所加）。两处转写与本仓库真实源码不同，注意分辨：② 号源码写 `ldr r3, pxCurrentTCBConst`（:514），这里写成教学常见的 `=pxCurrentTCB`；⑥ 号源码写 `mov r0, %0`（:525），操作数在 :557 由 `"i" ( configMAX_SYSCALL_INTERRUPT_PRIORITY )` 代入，所以编出来是一个硬立即数，下面实测段会看到它落地成 `#80`：

```
mrs   r0, psp                ; ① 取出当前任务的栈顶（PSP）
isb
ldr   r3, =pxCurrentTCB      ; ② r3 = &pxCurrentTCB
ldr   r2, [r3]               ;    r2 = 当前任务的 TCB
tst   r14, #0x10             ; ③ 这趟有 FPU 行李吗（EXC_RETURN bit4=0）？
it    eq
vstmdbeq r0!, {s16-s31}      ;    有：先把 FPU 高 16 寄存器压栈
stmdb r0!, {r4-r11, r14}     ; ④ 软件帧：r4-r11 + EXC_RETURN 压进任务栈
str   r0, [r2]               ; ⑤ 新栈顶存进 TCB 第一个成员——封棺！
stmdb sp!, {r0, r3}          ;    （保护现场，进临界区）
mov   r0, #configMAX_SYSCALL_INTERRUPT_PRIORITY
msr   basepri, r0            ; ⑥ 临界区：屏蔽低优先级中断
dsb
isb
bl    vTaskSwitchContext     ; ⑦ 换指针：pxCurrentTCB = 下一个任务
mov   r0, #0
msr   basepri, r0            ;    出临界区
ldmia sp!, {r0, r3}
ldr   r1, [r3]               ; ⑧ r1 = 新 TCB（pxCurrentTCB 已换人）
ldr   r0, [r1]               ;    r0 = 新任务的栈顶
ldmia r0!, {r4-r11, r14}     ; ⑨ 还魂：软件帧从新任务栈弹回
tst   r14, #0x10
it    eq
vldmiaeq r0!, {s16-s31}      ;    FPU 行李也领回
msr   psp, r0                ; ⑩ PSP 指向新任务的栈顶
isb
bx    r14                    ; ⑪ 异常返回：硬件弹出硬件帧，新任务复活
```

步骤	指令	一句话
①	`mrs r0, psp`	当前任务的栈顶在哪？（线程用的是 PSP，不是 MSP）
②	`ldr r3/ldr r2`	拿到"当前任务档案"TCB
③	`tst + it + vstmdb`	有 FPU 行李就先托运 s16-s31
④	`stmdb {r4-r11, r14}`	软件帧入栈——魂魄抽离
⑤	`str r0, [r2]`	**栈顶存进 TCB 第一个成员**——为什么敢写偏移 0？第四节揭晓
⑥	`basepri`	换指针时要安静：临界区
⑦	`bl vTaskSwitchContext`	全剧就这一句：换个全局指针
⑧⑨	`ldr/ldmia`	新 TCB → 新栈顶 → 魂魄归还
⑪	`bx r14`	异常返回，硬件弹硬件帧——**新任务从它上次停下的那条指令接着跑**

这十一条编出来是多少条？本机实测

源码是十一个语义步骤，编译器会把 `ldr r3, =pxCurrentTCB` 拆成真指令、给 naked 函数补对齐。`code/rtos/01-freertos-lab/probe.sh` 第 2 步直接数 `bx lr` 之前的指令行：

```
正文指令条数 = 27 条（数到 bx lr 为止，后面的 nop 填充与字面量池不算）
段大小 = 104 字节（含对齐填充和 pxCurrentTCBConst 字面量）
```

27 条按十一段落地的分布（`probe.sh` 第 2 步的完整反汇编就在这段前面，可逐条对）：

段	指令	条数	编译器多做了什么
①	`mrs PSP` + `isb`	2	—
②	`ldr r3,[pc,#84]` + `ldr r2,[r3,#0]`	2	源码 `port.c:514` 写的是 `ldr r3, pxCurrentTCBConst`（字面量池符号，`:556` 那行 `.word pxCurrentTCB` 就是它读的地址）；本章上面转写成教学常见的 `ldr r3, =pxCurrentTCB`
③	`tst.w lr,#16` + `it eq` + `vstmdbeq`	3	`#0x10` 打印成十进制 16
④⑤	`stmdb` 9 字 + `str [r2,#0]`	2	—
—	`stmdb sp!, {r0, r3}`	1	进临界区前先保 r0/r3，源码有、上表没单列
⑥	`mov #80` + `msr BASEPRI` + `dsb` + `isb`	4	—
⑦	`bl vTaskSwitchContext`	1	—
—	`mov #0` + `msr BASEPRI` + `pop {r0, r3}`	3	出临界区，同上
⑧	`ldr r1,[r3,#0]` + `ldr r0,[r1,#0]`	2	—
⑨	`ldmia` 9 字 + `tst` + `it` + `vldmia`	4	—
⑩⑪	`msr PSP` + `isb` + `bx lr`	3	`bx r14` 打印成 `bx lr`（r14 就是 lr）

合计 2+2+3+2+1+4+1+3+2+4+3 = **27**。字节账也对得上：27 条里有 9 条是 16 位 Thumb 编码（`4b15` / `681a` / `bf08` / `6010` / `bc09` / `6819` / `6808` / `bf08` / `4770`），所以正文只占 9×2 + 18×4 = **90 字节**（`0x00` – `0x5a`）；`objdump -s` 看尾部：`0x5a` / `0x5c` 是 `port.c:555` 那句 `.align 4` 补的两条 nop（6 字节），`0x60` 是 4 字节的 `pxCurrentTCBConst` 字面量，`0x64` 再两条 nop 凑到 8 字节对齐：**90 + 6 + 4 + 4 = 104**，正是 `size -A` 报的段大小。

数字会随工具链浮动，**账不在条数而在"一次切换只走这一段直线代码，没有循环、没有函数内的分支跳转"**（`tst/it` 那两条是条件执行，不改变控制流），所以第四节才敢说百周期量级。

三行最该亲眼看到的指令（`probe.sh` 第 3 步）：

```
  16:  e920 4ff0   stmdb  r0!, {r4, r5, r6, r7, r8, r9, sl, fp, lr}   ; ④ 一条指令 9 个字 = 36 字节
  1a:  6010        str    r0, [r2, #0]                                ; ⑤ 偏移 0——凭据见第四节
  20:  f04f 0050   mov.w  r0, #80    ; 0x50                          ; ⑥ = 5 << (8 - configPRIO_BITS)
```

`stmdb` 那一行是整章最贵的一条：一条指令写 9 个字。⑥ 的立即数在源码里写作宏，编出来是硬碰硬的 `#80`：`configPRIO_BITS` 是 4（F407 有 4 个优先级位），所以 `5 << 4 = 80`。`probe.sh` 第 4 步把这条账的两个来源并排打出：`FreeRTOSConfig.h:61` 的 `configKERNEL_INTERRUPT_PRIORITY 255` 和 `:62` 的 `(5 << (8 - configPRIO_BITS))`。

四、换指针即换魂

`vTaskSwitchContext`（tasks.c）的核心动作只有一步：`pxCurrentTCB = 最高优先级就绪任务的 TCB`（怎么选出最高的？位图 + CLZ 一条指令，F3 调度器细讲）。

⑤ 号指令 `str r0, [r2]` 为什么敢往偏移 0 写？因为 FreeRTOS 立下铁律：**TCB 结构体的第一个成员永远是 `pxTopOfStack`**。这条铁律不是口口相传，内核源码里就写在成员自己的注释上（`tasks.c:360`，本机 `.trellis/ref/freertos-v11/kernel` 实测行号）：

```
volatile StackType_t * pxTopOfStack; /**< Points to the location of the last item placed on the tasks stack.
                                          THIS MUST BE THE FIRST MEMBER OF THE TCB STRUCT. */
```

偏移 0 就是栈顶的家——封棺与起棺都从这进门。反汇编里三处 `#0` 全都指向同一个约定：`ldr r2, [r3, #0]`（取当前 TCB）、`str r0, [r2, #0]`（封）、`ldr r1, [r3, #0]` + `ldr r0, [r1, #0]`（起）。

顺带一个容易踩的空格：`include/task.h:171` 也有个同名成员 `pxTopOfStack`，但它属于 `TaskStatus_t`，那是 `uxTaskGetSystemInfo()` 用来给外部看的信息结构，**不是** TCB。移植层要写的偏移 0，永远指 `tasks.c` 里的那个。

整个切换，C 部分只换了一个指针；真正的体力活全在 PendSV 那十一条汇编里。**换指针是决策，PendSV 是执行**：决策与执行分离，这就是 FreeRTOS 切换设计的全部智慧。

> **【注】切换有多快？** M4 @168MHz 下一次完整切换是**百周期量级**，微秒不到，"RTOS 开销大"的直觉多为谣言，F7 会算这笔账。

五、GDB 目击实验

双任务各闪一灯（E 系列实验工程），把动画的每一帧在真板上抓到。

先解决名字：断点必须断在向量表里那个名字上，不是内核源码里那个。本工程三层对应关系（`probe.sh` 第 1 步一次打全，含 `nm` 的实际符号）：

```
port.c:505        void xPortPendSVHandler( void )        ← 内核源码写的名字
FreeRTOSConfig.h:76  #define xPortPendSVHandler PendSV_Handler  ← 一行改名
startup_stm32f407xx.s:111  .word PendSV_Handler          ← 向量表躺的是这个
nm port.o         00000000 T PendSV_Handler              ← 编出来只剩这个
```

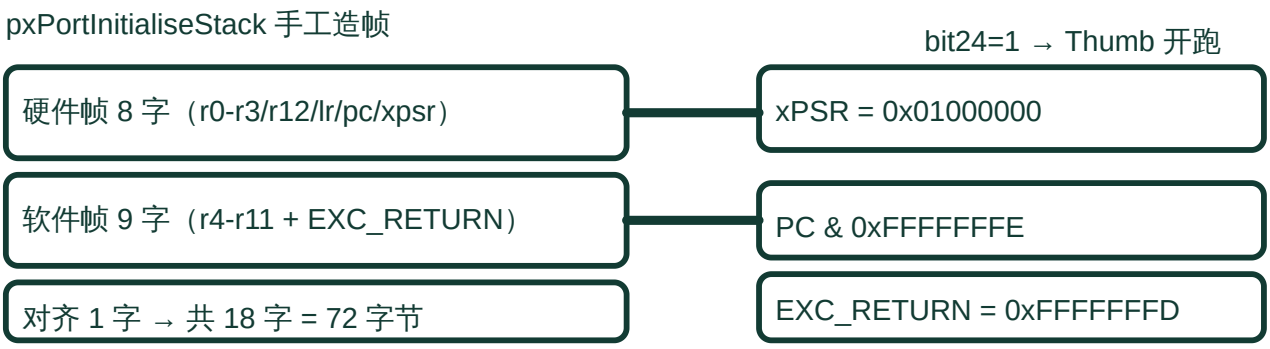
改名之后 `xPortPendSVHandler` 在目标文件里根本不存在，`b xPortPendSVHandler` 会直接报 "Function not defined"（或者更坑：某些前端默默接受然后永不命中）。

```
b PendSV_Handler          # 断在换魂入口——用向量表的名字，不是源码的名字
p pxCurrentTCB            # 旧任务 TCB 地址
p/x $psp                  # 此刻 PSP 指硬件帧最低字（R0）——软件帧还没压
x/8xw $psp                # 硬件帧 8 字：R0 R1 R2 R3 R12 LR PC xPSR（回 S4）
stepi 8                   # 走到第 9 条 str r0, [r2, #0]（断下时它还没执行）
p/x $r0                   # ④ 压完之后的任务栈顶。注意：PSP 到 ⑩ 才改，这里只能看 r0
x/9xw $r0                 # 软件帧 9 字：r4 r5 r6 r7 r8 r9 sl fp lr(EXC_RETURN)
x/8xw $r0+36              # 紧接其上就是硬件帧，$r0+36 应当等于 $psp
finish                    # 跑完整个 handler
p pxCurrentTCB            # 已换人！对比两次地址
```

三个容易读错的点：

- **别在入口读软件帧**。`stmdb r0!` 改的是 r0，不是 PSP，PSP 直到 ⑩ 那条 `msr PSP, r0` 才被更新。所以想看 ④ 的成果，必须 `stepi` 过 ④ 再读 `$r0`，读 `$psp` 只会读到旧值。
- **`x/9xw` 不是 `x/8xw`**。软件帧数下来 9 个字：r4~r11 八个 + EXC_RETURN 一个。第 3 步的寄存器列表 `{r4, r5, r6, r7, r8, r9, sl, fp, lr}` 正好 9 项，`sl` 是 r9、`fp` 是 r11 的 ABI 别名（C6 见过）。
- **`$r0+36 == $psp` 这条等式只在没有 FPU 行李时成立**。若 ③ 走了 `eq` 分支（任务跑过浮点），`vstmdbeq` 先把 s16-s31 压了 16 个字 = 64 字节，`$r0` 会再低 64。这正是 `tst lr, #16` 存在的理由，也是"同一个任务两次切换栈顶差得不一样"的合法解释。

六、没跑过的任务，第一帧是伪造的



伪造首帧：pxPortInitialiseStack 手工造 18 字栈帧

前面所有动作都假设"这个任务曾经被切走"。可 `xTaskCreate` 刚建好的任务从没被中断过，**没有硬件会替它压那一帧**。所以第一帧是手工造的：`port.c:202` 的 `pxPortInitialiseStack()` 直接往任务栈里写出一份和"被切走时"形状完全一致的栈帧。

形状必须严格一致，否则 ⑧⑨⑩⑪ 那四条"起棺"指令会读错位置。`probe.sh` 第 5 步把源码里的回退量当场加了一遍：

```
指针回退 17 个字，加上最后停住的那个落点共 18 个字 = 72 字节
对得上账：硬件帧 8 + 软件帧 9 + 对齐 1 = 18
```

三个立即数撑起这 18 个字里被真正写入的 5 个（xPSR / PC / LR / R0 / EXC_RETURN）；其余 13 个槽位（1 个对齐空位 + R12/R3/R2/R1 + R4-R11）源码明说 *Save code space by skipping register initialisation*，故意留脏：

写进帧的值	宏（port.c 行号）	为什么是它
`0x01000000` → xPSR	`portINITIAL_XPSR`（:92）	bit24 = 1。ARMv7-M 规定异常返回时这个保留位必须为 1，它同时就是 THUMB 状态位——新任务从此以 Thumb 模式开跑
`pxCode & 0xfffffffe` → PC	`portSTART_ADDRESS_MASK`（:100）	清掉函数指针低位的 Thumb bit。Thumb 状态由 xPSR bit24 说，不许 PC 的 bit0 越权——所以 `bic r2, r3, #1`
`0xfffffffd` → EXC_RETURN	`portINITIAL_EXC_RETURN`（:93）	bit2=1 出栈用 PSP、bit3=1 回 Thread 模式、**bit4=1 表示这趟没有 FPU 行李**（所以初始帧是 18 个字而不是 26 个）
`pvParameters` → R0	—	新任务第一次拿到的实参，AAPCS 说整型参数走 r0（C6）
`prvTaskExitError` → LR	`portTASK_RETURN_ADDRESS`（:125 → :234）	任务函数不许 `return`。真返回了就落进这个"触发 configASSERT + 关中断 + 死循环"的陷阱

编译产物里能直接看到前两条（`probe.sh` 第 5 步末尾的 objdump）：

```
  14:  f04f 7280   mov.w  r2, #16777216   ; 0x1000000  ← portINITIAL_XPSR
  22:  f023 0201   bic.w  r2, r3, #1                   ← & portSTART_ADDRESS_MASK
```

这一节的实用价值：任务函数跑完没死、程序"莫名卡住"、`configASSERT` 在没人预料的地方炸：先怀疑有人 `return` 了，断点打在 `prvTaskExitError` 上。

顺手分清两类符号（`nm build/scene-1/port.o` 实测）：

```
00000000 T PendSV_Handler          ← 大写 T：全局，被宏改过名，源码名已不存在
00000000 T SVC_Handler             ← 同上
00000000 T pxPortInitialiseStack   ← 大写 T：全局，没被改名，源码名 == 符号名
00000000 t prvTaskExitError        ← 小写 t：static 局部符号，没被改名
```

改名只发生在 `FreeRTOSConfig.h:75-77` 那三行宏覆盖的三个 handler 上；`pxPortInitialiseStack`、`prvTaskExitError` 都按源码名直接断即可。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **PendSV 最后上场，r4-r11 手工进栈，栈顶藏进 TCB；换个指针换个魂，异常返回新人生。**

实物实验

- **不上板也能全过一遍**：`cd code/rtos/01-freertos-lab && sh probe.sh`，五步一条命令跑完（符号改名链、27 条反汇编、三行铁账、优先级账、伪造帧的长度账）。本章每一个数字都出自这里。
- 见第五节 GDB 目击清单；装备：霸天虎板 + FreeRTOS 双任务工程 + ST-Link。
- 进阶：在任务里加一行浮点运算，再观察 `tst r14, #0x10` 走 `eq` 分支，亲眼确认 FPU 行李托运。

常见坑

- **中断里调 API 忘 FromISR 版**：普通版 API 在 ISR 里可能直接触发切换，现场混乱，ISR 一律 `xQueueSendFromISR` 家族（F4 队列）。
- **以为还要自己把 PendSV/SysTick 设最低**：V11.1.0 的 ARMv7-M 移植层自己硬写成最低（`port.c:65/434-435`），`configKERNEL_INTERRUPT_PRIORITY` 在这个端口上是**不被读取的死宏**（`History.txt:301-303` 明说 obsolete）。真正要盯的是 `configMAX_SYSCALL_INTERRUPT_PRIORITY`：内核连查三遍（`port.c:379/383/413`），填 0、超出优先级位宽、填奇数都会开机即断言。
- **把 `configASSERT` 定义成空**：上面三道检查全部消失，配置错误就从"启动即死"退化成"一周崩一次的灵异现场"。调试期一定留着。
- **handler 没接到向量表**：`port.c:343-344` 会查 `SVC` / `PendSV` 两个槽是不是指向内核的 handler，这正是 `FreeRTOSConfig.h:75-77` 那三行宏改名的用途（第五节）。直接把 `xPortPendSVHandler` 写进启动文件反而链接不上。
- **栈对齐不是 8 字节**：xPSR 异常入场要求 8 对齐，栈起点错对齐直接 UsageFault（S16 取证）。
- **GDB 里 `b xPortPendSVHandler` 断不上**：这个符号在目标文件里不存在（`FreeRTOSConfig.h:76` 已经把它宏改名为 `PendSV_Handler`），向量表认的是后者（第五节有三层对应表）。同理 `SVC_Handler`、`SysTick_Handler`。
- **任务函数写了 `return`**：没地方可回（源码原话 *there is nothing to return to*），落进 `prvTaskExitError` 关中断死循环（第六节）。要退出就 `vTaskDelete( NULL )`。
- **以为切换很慢**：M4 上一次切换是百周期量级，"RTOS 开销大"的直觉多为谣言。

短自测

1. 切换为什么在 PendSV 里做，而不是在 SysTick 中断里直接做？

2. 硬件帧和软件帧各包含哪些寄存器？（FPU 情况各加什么？）
3. `str r0, [r2]` 把栈顶存到了 TCB 的哪里？凭什么敢这么写？
4. `bx r14` 之后硬件做了什么？新任务从哪条指令开始跑？
5. 内核源码里函数明明叫 `xPortPendSVHandler`，为什么 `b xPortPendSVHandler` 断不上？
6. 从没跑过的任务，它的第一帧是谁压的？这帧多大、哪几个字被真正写过？
7. `portINITIAL_EXC_RETURN` 是 `0xffffffd`，bit4 = 1 说明什么？

▼ 参考答案（先自己想完再展开）
1. SysTick 优先级高，ISR 堆里直接切换会把某个中断服务切走一半；PendSV 优先级最低（0xFF），等所有 ISR 收队才执法。 2. 硬件帧：R0-R3、R12、LR、PC、xPSR（FPU 加 s0-s15+FPSCR）；软件帧：R4-R11 和 EXC_RETURN（FPU 加 s16-s31）。 3. TCB 第一个成员 `pxTopOfStack`，偏移 0 是 FreeRTOS 的铁律，原文写在 `tasks.c:360` 的成员注释里（*THIS MUST BE THE FIRST MEMBER OF THE TCB STRUCT*）。注意 `task.h:171` 那个同名字段属于 `TaskStatus_t`，不是 TCB。 4. 硬件按 EXC_RETURN 选用 PSP，从任务栈弹出硬件帧；新任务从它上次被切走时的 PC（就在硬件帧里）继续执行。 5. `FreeRTOSConfig.h:76` 有一行 `#define xPortPendSVHandler PendSV_Handler`，预处理阶段就把源码里的名字整体换掉了；向量表（`startup_stm32f407xx.s:111 .word PendSV_Handler`）和 `nm` 里只剩 `PendSV_Handler`。断点要断后者。`SVC_Handler`、`SysTick_Handler` 同理（:75、:77）。 6. `port.c:202` 的 `pxPortInitialiseStack()` 手工造的，形状和"被切走时"完全一致：18 个字 = 72 字节 = 硬件帧 8 + 软件帧 9 + 对齐 1。真正写入的只有 5 个字（xPSR、PC、LR、R0、EXC_RETURN），剩下 13 个槽位故意留脏：省代码空间，反正这些寄存器接下来会被任务自己覆盖。 7. bit4 = 1 表示这趟异常没有 FPU 行李：硬件按 8 字的核心帧出栈，③ 号 `tst r14, #0x10` 也不等零，`vstmdbeq/vldmiaeq` 双双跳过。任务真跑过浮点后被切走，软件侧还要多压 s16-s31 共 16 个字（64 字节）。

对照表：本章概念 → 源码落点

本章说的	源码里哪里
换魂十一条语义步骤（编出来 27 条指令 / 正文 90 字节）	FreeRTOS-Kernel V11.1.0 `portable/GCC/ARM_CM4F/port.c:505-559` 的 `xPortPendSVHandler`（naked）
`mov r0, %0` 为什么编出来是 `#80`	同文件 `:525` 的 GCC 扩展汇编操作数 + `:557` 的 `::"i" ( configMAX_SYSCALL_INTERRUPT_PRIORITY )`
换指针	同内核 `tasks.c` 的 `vTaskSwitchContext`
TCB 首成员铁律（`str r0,[r2,#0]` 的凭据）	`tasks.c:360`（不是 `task.h:171`，那个属于 `TaskStatus_t`）
源码名 → 符号名的改名	本工程 `FreeRTOSConfig.h:75-77` + `startup_stm32f407xx.s:111`
新任务第一帧的伪造	`port.c:202-231` `pxPortInitialiseStack`；三个立即数 `port.c:92/93/100`
任务 `return` 掉进哪	`port.c:125` `portTASK_RETURN_ADDRESS` → `port.c:234` `prvTaskExitError`
PendSV 挂起位	ICSR = 0xE000ED04，bit28（PENDSVSET）——本机出处：移植层 `portmacro.h:99-100`（`0xe000ed04` / `1UL << 28UL`）与 `:91` 的 `portYIELD_WITHIN_API`；架构侧 `core_cm4.h:490` `SCB_ICSR_PENDSVSET_Pos = 28U`
内核把两个 handler 硬设最低	`port.c:65` `portMIN_INTERRUPT_PRIORITY ( 255UL )` → `:66-67` → `:434-435` 写 SHPR3
`configKERNEL_INTERRUPT_PRIORITY` 作废	内核 `History.txt:301-303`；本机实测该宏在 `ARM_CM4F/port.c` 与 `portmacro.h` 中出现 **0 次**
优先级配置错误的三道断言	`port.c:379`（非 0）、`:383`（不超出实现的优先级位宽）、`:413`（必须为偶数）
handler 有没有真接进向量表	`port.c:343-344` 查 `SVC` / `PendSV` 两个槽（受 `configCHECK_HANDLER_INSTALLATION` 控制）
栈帧约定	C6 调用约定 + S4 中断动画
启动时 MSP 的来历	B4 启动过程 第一节
选最高优先级任务	F3 调度器（位图 + CLZ）
本章所有数字怎么复核	`code/rtos/01-freertos-lab/probe.sh` 五步一条命令跑完（符号名 / 27 条反汇编 / 三行铁账 / 优先级账 / 伪造帧）

延伸阅读

懒堆叠与异常帧的权威出处：

- **[C5]** AN298 — 懒堆叠（lazy stacking）的官方应用笔记：FPCCR.LSPEN 怎么让硬件"先占座、后写真"、EXC_RETURN bit4 的含义，本章【注】的全文版。
- **[A6]** ARMv7-M 架构参考手册 — 异常入栈帧与 EXC_RETURN 的规范定义。

你做到了

- "换魂术"全程目击并能逐行讲解；
- 上下文切换从此没有黑盒，这是理解一切 RTOS 行为的总钥匙；
- 三个数张口就来：软件帧 9 个字、正文 27 条指令、新任务第一帧 18 个字 72 字节；
- 拿到任何一份 RTOS 移植层，先看它的 `FreeRTOSConfig.h` 改了哪些 handler 名，这是 GDB 能不能断上的分水岭；
- 再看到 `taskYIELD()` 或 `portYIELD_FROM_ISR`，你知道它们在跟 PendSV 打招呼。

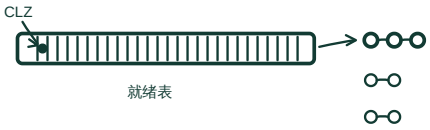
断点没响，不是 GDB 坏了：源码里的名字被一行宏换掉了，向量表里躺着的才是真身。往后拿到任何移植层，先翻改名表再下断点，这一炮才响得起来。

✅ 下一站：F3 调度器：位图就绪表 32 选 1 只要一条 CLZ，tick 里到底发生了什么。

通往单片机之路第 39 章

F3 调度器：一条 CLZ 选出下一个任务

难度 ★★★☆☆ · 约 50 分钟



本章罗盘

问题

每毫秒都要选人，凭什么 O(1)，两条选优路径谁在生效，tick 为什么只瞄一眼延时队首？

交付

一张双路径实测表：三份 tasks.o 逐字节对照（不写与写 1 完全相同、.text 均 140 字节，写 0 为 160），两条路 58 vs 70 条指令，宿主机模拟器判空对照，外加 (660−100)÷(33−5)=20 字节/级的 List_t 推算。

边界

位图＋CLZ 只在 configUSE_PORT_OPTIMISED_TASK_SELECTION=1 且优先级 ≤32 时成立（=33 会被 portmacro.h 的 #error 拦下）；优先级该怎么分（速率单调、利用率上界）不在本章，那是 D1 论文的领地；全部数字出自 V11.1.0 三份编译产物实测。

依据锚点 `f1 · d1 · mcu-code · mcu-anim`

> 🎯 现在冻结一个 tick：当前任务刚调用 `vTaskDelay`，另一个同优先级任务在就绪表里，更高优先级任务还没到唤醒时刻。下一位是谁？别凭“优先级高就先跑”的口号猜，真正答案藏在三样东西里：就绪速查器、两条链表和下一次唤醒时间。调度器不是黑箱，它是一场可以逐帧判案的搬家游戏。

本章精髓

1. **就绪表 = 链表数组 + 一个"最高非空"速查器**：`pxReadyTasksLists[configMAX_PRIORITIES]` 是货架（F1），`uxTopReadyPriority` 是速查器：通用路径里是标量（递减扫描），M4F 移植优化路径里是 **32 位位图**（CLZ 一把出最高位）。
2. **tick 中断干三件事**：节拍++、查延时列表队首谁该醒了、（必要时）请求切换，`xTaskIncrementTick` 就读这三段。
3. **`vTaskDelay` 不是"等"，是"搬"**：把自己挂进按唤醒时间排序的延时链表，然后让出 CPU：任务真的"不在"了，这是 RTOS 延时与裸机死等的本质区别。

▼ 🌐 **English Abstract**

Ready list = array of linked lists + a "highest non-empty" lookup. `pxReadyTasksLists[configMAX_PRIORITIES]` is the shelf; `uxTopReadyPriority` is the lookup—scalar in generic path (linear scan), **32-bit bitmap** in M4F optimized path (CLZ instruction finds highest set bit in O(1)). **Tick interrupt does three things**: increment tick count, check delay list head for tasks to wake, request context switch if needed. **`vTaskDelay` is not "wait", it's "move"**: task moves to delay list (sorted by wake time), then yields CPU—task is truly "gone", the fundamental difference between RTOS delay and bare-metal busy-wait.

- **能理解**：为什么 tick 只查延时表**队首**（O(1) 的秘密）；为什么位图路径要求优先级 ≤32；为什么 `vTaskDelay(1)` 不能当精确 1ms 用。
- **能用**：RunTimeStats 实测任务占比验证调度行为；按场景选 vTaskDelay / vTaskDelayUntil。

先修

- F1 任务与 TCB、F2 上下文切换、S5 SysTick。

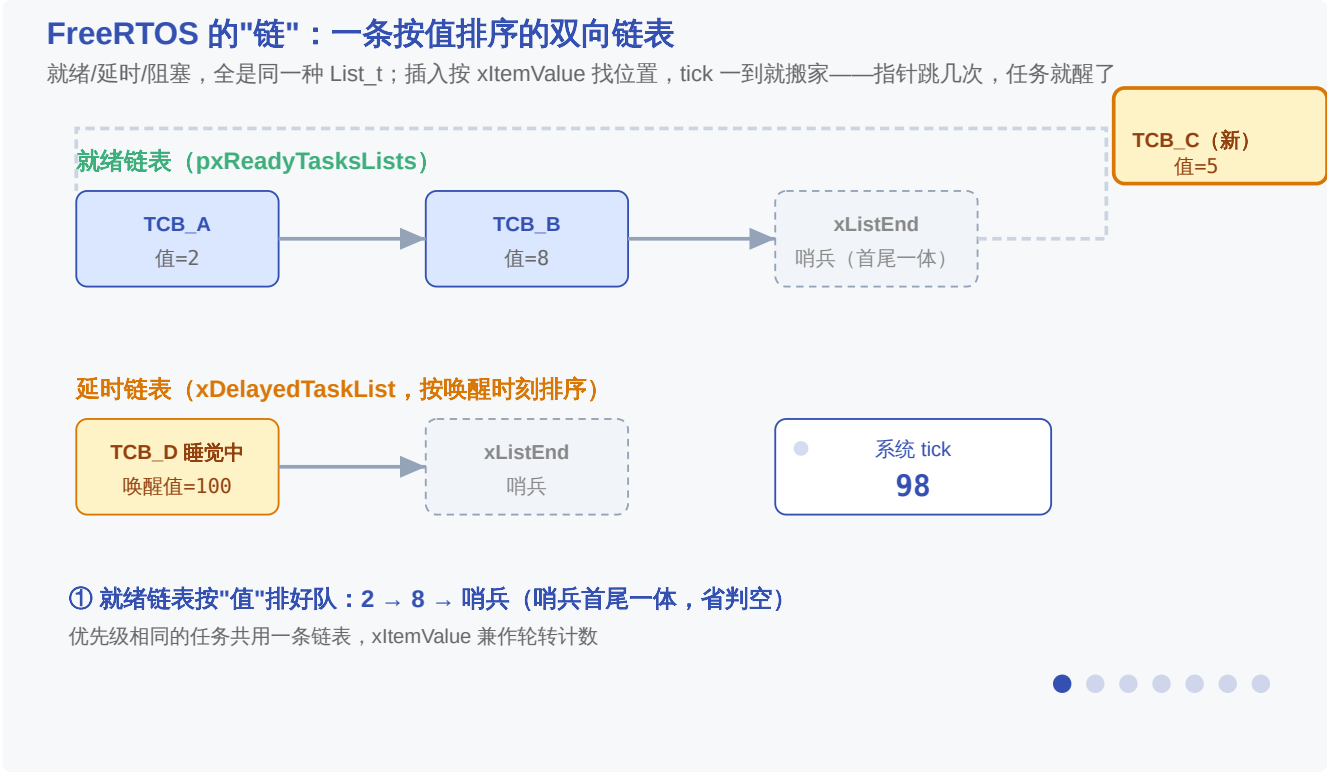
先跑起来（10 分钟 quick win）

两任务同优先级各 `vTaskDelay(1)`：逻辑分析仪测两个 LED 翻转时刻，1ms 交替出现，时间片轮转眼见为实。

接线前先画一条只有“运行、延时、就绪”的时间线，押下一次亮灯属于谁，再看逻辑分析仪。若实测与你的图不一致，不要先怪时间片：先问当前任务被搬进了哪条链表、延时队首是否到点、同优先级就绪表的索引停在哪里。每一次翻转都能在数据结构里找到座位。

动画：一条链表撑起整个调度

新任务按值找位插入（只改 4 根指针）；tick 到点，延时链表队首搬家到就绪链表：**vTaskDelay 没有魔法，就是挂进延时链表睡 N 拍**。



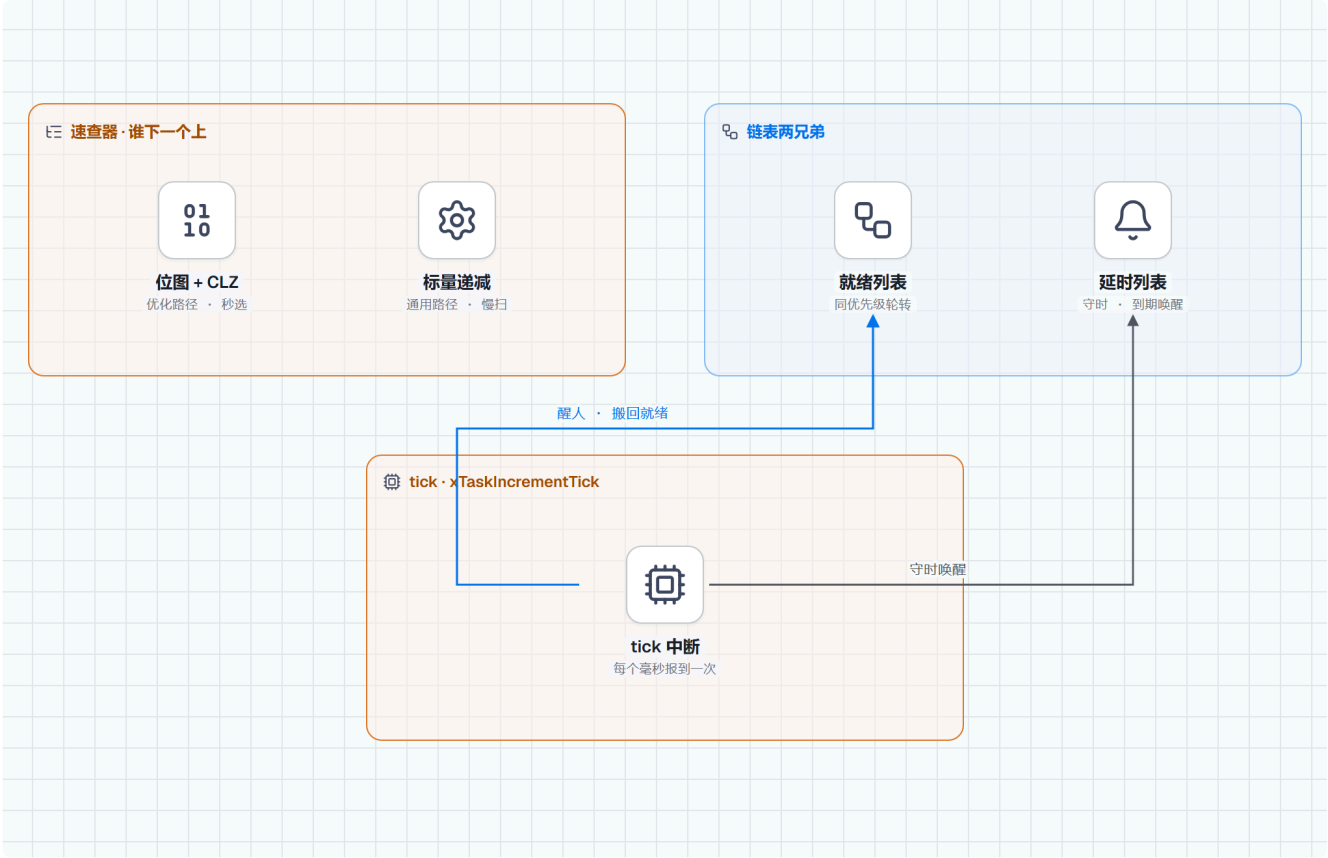
就绪链表插入动画

小节结构

小节	内容	四件套
一、速查器双实现	通用扫描 vs 位图 CLZ（两版宏原文对照）	库解析
二、入库动作	`prvAddTaskToReadyList` 两步走	库解析
三、链表两兄弟	`vListInsertEnd` 轮转 vs `vListInsert` 排序	代码分析
四、tick 三段	`xTaskIncrementTick` 走查 + 队首剪枝	库解析
五、vTaskDelay 真相	挂延时表原文；抢占点全集	代码分析
六、CPU 统计	RunTimeStats 配置与解读	配置

一、速查器双实现：通用扫描 vs 一条 CLZ

调度器的家当先看图：速查器两副面孔、链表两兄弟、tick 居中调度。



调度器结构：速查器双实现、就绪与延时链表、tick 到期唤醒

选下一个任务 = 找"最高的非空就绪链表"。tasks.c（V11.1.0）里这条宏有**两副面孔**，由 `configUSE_PORT_OPTIMISED_TASK_SELECTION` 切换。

通用路径（=0）：标量 + 递减扫描。 `uxTopReadyPriority` 只是个数：

```
#define taskRECORD_READY_PRIORITY( uxPriority ) \
do {                                            \
    if( ( uxPriority ) > uxTopReadyPriority )   \
    {                                           \
        uxTopReadyPriority = ( uxPriority );    \
    }                                           \
} while( 0 )
```

选择时从它开始往下找非空链表：

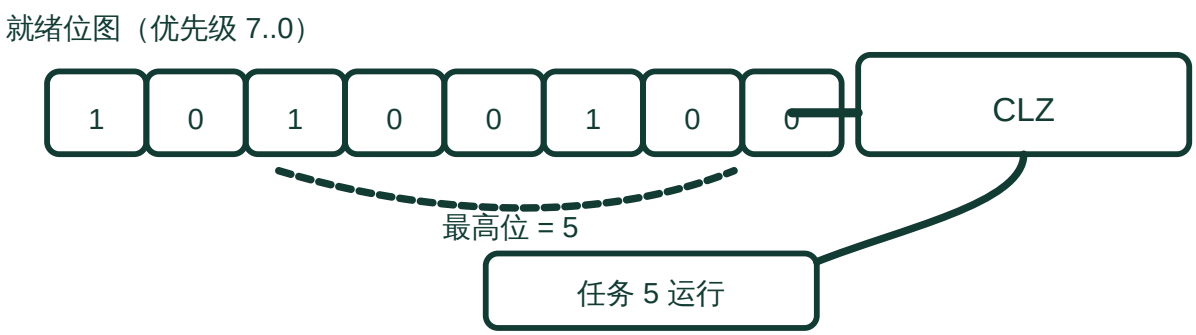
```
while( listLIST_IS_EMPTY( &( pxReadyTasksLists[ uxTopPriority ] ) ) != pdFALSE )
{
    configASSERT( uxTopPriority );
    --uxTopPriority;                        /* 一层层往下摸 */
}
listGET_OWNER_OF_NEXT_ENTRY( pxCurrentTCB, &( pxReadyTasksLists[ uxTopPriority ] ) );
```

移植优化路径（=1，M4F 默认）：位图 + CLZ。 同一个变量摇身变成位图。portmacro.h（ARM_CM4F）原文：

```
#define portRECORD_READY_PRIORITY( uxPriority, uxReadyPriorities )  ( uxReadyPriorities ) |= ( 1UL << ( uxPriority ) )
#define portRESET_READY_PRIORITY( uxPriority, uxReadyPriorities )   ( uxReadyPriorities ) &= ~( 1UL << ( uxPriority ) )
#define portGET_HIGHEST_PRIORITY( uxTopPriority, uxReadyPriorities ) uxTopPriority = ( 31UL - ( uint32_t ) ucPortCountLea
```

第 N 位=1 ⟺ 优先级 N 有就绪任务。`ucPortCountLeadingZeros` 在 M4F 上就是一条 `CLZ` 汇编：**32 位里找最高置位，一条指令，O(1)，与任务数无关**。代价：位图 32 位，所以此路径要求 `configMAX_PRIORITIES ≤ 32`。

> 【注】 网上很多文章说"`uxTopReadyPriority` 是位图"，只对了一半：位图是**移植优化路径**的形态；通用路径里它是标量。读源码先看 `configUSE_PORT_OPTIMISED_TASK_SELECTION` 是哪条路，再谈语义。这也是读 FreeRTOS 的通用方法论：**一切结论先问配置宏**。



调度器就绪位图：uxTopReadyPriority 标出哪些优先级有任务，CLZ 找最高位，再到对应就绪链表取任务

二、入库动作：两步走

任务就绪时（新建/解除阻塞/延时到期），走同一个宏（tasks.c）：

```
#define prvAddTaskToReadyList( pxTCB )                                                  \
do {                                                                                    \
    taskRECORD_READY_PRIORITY( ( pxTCB )->uxPriority );             /* ① 速查器记账 */             \
    listINSERT_END( &( pxReadyTasksLists[ ( pxTCB )->uxPriority ] ), &( ( pxTCB )->xStateListItem ) ); /* ② 挂到货架队尾 */
} while( 0 )
```

①记账（标量取大 / 位图置位）②挂队尾。为什么队尾不是队首？因为 `listGET_OWNER_OF_NEXT_ENTRY` 从"上次的下一个"继续取：**队尾插入 + 循环取头 = 同优先级天然轮转**，时间片不需要额外数据结构。

三、链表两兄弟：一个管轮转，一个管守时

list.c 全文件只有两个插入函数，分工明确（V11.1.0 原文节选）：

`vListInsertEnd`：不排序，插到"索引位置之前"（即下一个被取走的队尾）：

```
pxNewListItem->pxNext = pxIndex;
pxNewListItem->pxPrevious = pxIndex->pxPrevious;
pxIndex->pxPrevious->pxNext = pxNewListItem;
pxIndex->pxPrevious = pxNewListItem;
```

`vListInsert` 则**按 xItemValue 升序**找位插入：

```
for( pxIterator = ( ListItem_t * ) &( pxList->xListEnd );
     pxIterator->pxNext->xItemValue <= xValueOfInsertion;
     pxIterator = pxIterator->pxNext )
{
    /* 空循环体:纯粹迭代到插入点 */
}
```

延时列表用后者，链表项的值 = **唤醒时刻**，于是队首永远是"最早该醒的"。这就是下一节 O(1) 剪枝的地基。四根指针改完收工，插入动画里看得清清楚楚。

四、tick 三段：醒人、换片、（必要时）换片场

SysTick 每拍调用一次 `xTaskIncrementTick`（tasks.c 节选）：

```
const TickType_t xConstTickCount = xTickCount + ( TickType_t ) 1;
xTickCount = xConstTickCount;                /* ① 节拍++ */

if( xConstTickCount >= xNextTaskUnblockTime ) /* ② 剪枝:队首都没到点就整个跳过 */
{
    for( ; ; )
    {
        if( listLIST_IS_EMPTY( pxDelayedTaskList ) != pdFALSE )
        {
            xNextTaskUnblockTime = portMAX_DELAY;
            break;
        }
        else
        {
            /* 只看队首:它的 xItemValue 就是最早唤醒时刻 */
            ...
        }
    }
}
```

注意那行剪枝：`xNextTaskUnblockTime` 记着"下一个该醒的时刻"，tick 值没追上它，整个延时检查**一行比较就跳过**：1kHz 下每拍 1 次比较，这就是 FreeRTOS 敢让 tick 跑高频率的底气。追上了才进循环：队首到点→摘出→`prvAddTaskToReadyList`→看下一个队首，直到队首未到点。

现在重判开场题：更高优先级任务“存在”不等于它“就绪”。只要唤醒时刻没到，它仍在延时表里，速查器根本看不见它；调度器会从当前可见的最高非空就绪链表选人。优先级决定就绪任务之间谁先跑，链表状态决定谁有资格参选。

第三件事在函数尾部（configUSE_PREEMPTION + configUSE_TIME_SLICING 时）：若有同优先级就绪任务等着轮转，返回 `pdTRUE`，port 层据此**挂起 PendSV**。至于 PendSV 怎么换魂，F2 的十一条汇编已经讲透，调度器只管"发信号"，搬寄存器是 PendSV 的活。

五、vTaskDelay 真相：挂表、让出、不等任何人

```
void vTaskDelay( const TickType_t xTicksToDelay )
{
    if( xTicksToDelay > ( TickType_t ) 0U )
    {
        vTaskSuspendAll();                                      /* 调度器暂停,搬家期间不许捣乱 */
        prvAddCurrentTaskToDelayedList( xTicksToDelay, pdFALSE ); /* 挂进延时链表,值=唤醒时刻 */
        xAlreadyYielded = xTaskResumeAll();
    }
    if( xAlreadyYielded == pdFALSE )
    {
        taskYIELD_WITHIN_API();                                 /* 主动让出 → PendSV */
    }
}
```

三行核心：**挂表 → 恢复调度 → 让出 CPU**。没有任何"等"：此刻起这个任务在就绪表里查无此人，直到 tick 把它搬回来。裸机的 `delay_ms()` 是原地空转烧 CPU，`vTaskDelay` 是"挂牌睡觉"，CPU 转头去伺候别人。

由此推出**抢占点全集**：任务只在三处可能被换下：① tick 中断（醒人/轮转触发 PendSV）② 任务自己调 API（delay/队列/信号量内部 YIELD）③ 中断退出（FromISR 系列置了 xHigherPriorityTaskWoken）。

> 【注】 `vTaskDelay(1)` 的延时区间是 (0, 1] 拍：你在拍中任何时刻调用，都睡到**下一拍**边界。想要严格周期（如 10ms 整的采样环），用 `vTaskDelayUntil`（它以上次唤醒点为锚，消除抖动累积）。

六、CPU 统计：用数据证明调度

`configGENERATE_RUN_TIME_STATS=1` + 一个高精度定时器（如 TIM2 跑 10kHz+），`vTaskGetRunTimeStats` 打印各任务绝对/相对占用。实验要求：两同优先级任务 ≈50/50；其中一个 +1 优先级 → 100/0。**调度行为是可测量的事实，不是信仰**，这也是全站"眼见为实"的落脚点。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **位图置位 CLZ 秒选（优化路径），标量递减慢扫（通用路径）；tick 三事：醒人、轮转、换片场；vTaskDelay 是挂表睡觉，不是原地死等。**

延伸：SysTick 与硬件 TIM 分工，SysTick 管调度节拍，TIM 管外设时基（S6）；软件定时器依赖 tick，见 F6。

实物实验

- **不用板子也能把本章每一个数字打出来**：`cd code/rtos/03-scheduler && sh probe.sh`。七步一条命令跑完：把 `configUSE_PORT_OPTIMISED_TASK_SELECTION` 掰成不写 / 1 / 0 编三份 `tasks.o` 逐字节对照（实测：不写与写 1 的两份**逐字节相同**，`.text.vTaskSwitchContext` 都是 140 字节，写 0 那份 160 字节）；数两条路的指令（58 条无后向跳转 vs 70 条含一个循环）；跑一遍把两条规则抄成的宿主机模拟器（同一张 10 次选择的事件表：通用路累计判空 21 次、平均 2.10、最坏 5；位图路 10 次、平均 1.00、最坏 1）；用 `configMAX_PRIORITIES=5` 与 `=33` 两个测量反推 `sizeof(List_t) = (660−100)÷(33−5) = 20` 字节；再把 `NPRIO=33` 交给位图路径，看 `portmacro.h:162` 那条 `#error` 原文。需要交叉工具链与 V11.1.0 内核（CI 的 arm-host 作业两样都装）。
- quick win 轮转证据 + RunTimeStats 占比实验（本章目标三）；
- 进阶：逻辑分析仪夹 LED，把 `configTICK_RATE_HZ` 从 1000 改 100 再测，看交替周期跟着变，tick 频率物理化。

常见坑

- **高优先级任务不延时**：永不放弃 CPU，低优先级全体饿死，`vTaskDelay` 是基本教养；
- **`vTaskDelay(1)` 当精确 1ms**：实际 (0,1] 拍区间，周期任务请用 `vTaskDelayUntil`；
- **`configTICK_RATE_HZ` 盲目拉 10k**：每拍都有醒人检查开销，1kHz 是通用甜点；
- **优先级开几十上百**：每个优先级一条链表，RAM 按条收租；位图路径还硬性 ≤32；
- **中断里调非 FromISR API**：list.c 注释里的崩溃清单第二名，`configMAX_SYSCALL_INTERRUPT_PRIORITY` 不是摆设。

短自测

1. 位图路径下 `uxTopReadyPriority` 第 12 位为 1 表示什么？选最高优先级要走几条指令？

▼ 参考答案
优先级 12 至少有一个就绪任务。选优 = 一次 CLZ 加一次减法（31-CLZ），M4F 上两条指令，O(1)。

2. tick 中断为什么敢只查延时链表队首？

▼ 参考答案
延时链表用 vListInsert 按唤醒时刻升序排列，队首即最早该醒者；队首未到点则全员未到点，直接 break。配合 xNextTaskUnblockTime 剪枝，常态每拍只需一次比较。

3. `vTaskDelay(10)` 在 tick=1kHz 时实际睡多久？为什么不是精确 10ms？

▼ 参考答案
(9, 10] ms。调用时刻落在拍内任意位置，唤醒对齐到第 10 个拍边界，不足一拍的部分被"抹零"。要精确周期用 vTaskDelayUntil（以唤醒点为锚）。

4. 同优先级轮转不需要额外数据结构，靠哪两个配合实现？

▼ 参考答案
prvAddTaskToReadyList 把任务插到就绪链表队尾（listINSERT_END），taskSELECT_HIGHEST_PRIORITY_TASK 用 listGET_OWNER_OF_NEXT_ENTRY 循环取"下一个"：队尾进、循环出，天然 Round-Robin。

对照表：本章概念 → 源码落点

概念	落点（**FreeRTOS-Kernel V11.1.0**）
速查器双实现	`tasks.c` taskRECORD/SELECT_HIGHEST_PRIORITY_TASK 两组宏（configUSE_PORT_OPTIMISED_TASK_SELECTION 分路）
位图+CLZ	`portable/GCC/ARM_CM4F/portmacro.h` portRECORD/GET_HIGHEST_PRIORITY
入库两步	`tasks.c` prvAddTaskToReadyList
排序/队尾插入	`list.c` vListInsert / vListInsertEnd
tick 醒人剪枝	`tasks.c` xTaskIncrementTick（xNextTaskUnblockTime 比较+队首循环）
延时挂表	`tasks.c` vTaskDelay → prvAddCurrentTaskToDelayedList
本章所有数字怎么复核	`code/rtos/03-scheduler/probe.sh` 七步一条命令跑完（三份 `tasks.o` 逐字节对照 / 两条路指令数 / 模拟器判空次数 / 20 字节每级 / 33 级被 `#error` 拦下 / tick 三段 / 链表两兄弟）

延伸阅读

本章讲"高优先级先跑"这台机器怎么造；优先级该怎么分才能保证期限，是另一篇文章：

- **[D1]** Liu & Layland 1973 — 速率单调（周期越短优先级越高）与 69.3% 利用率上界都出自这里；读完你会知道 `configMAX_PRIORITIES` 里的数字不该拍脑袋。

你做到了

- 调度器从"魔法"变成"速查器+链表+三条抢占路径"；
- 网上"位图"说法的适用边界被你亲手钉死（配置宏分路）；
- 调度行为可测量，RunTimeStats 是你的测谎仪。

回到那帧被冻结的 tick：你已经不需要背“调度器会选最高优先级”这句半真话。先看任务在哪张表里，再看速查器指向哪里，最后看这一拍是否请求切换，下一位运行者就能被你逐步推出来。

✅ 下一站：F4 队列：任务间的"传送带"，环形存储+两个阻塞列表的源码级解剖。

通往单片机之路**第 40 章**

F4 队列：传送带的源码解剖

难度 ★★★☆☆ · 约 45 分钟

本章罗盘

问题

队列满了发送方睡在哪、中断里为什么不能调普通版，一个结构体怎么同时当队列和信号量？

交付

一张 Queue_t 五要素结构图（环形三指针＋两张按优先级排的候客名单＋ union 双生）＋「按键 EXTI → xQueueSendFromISR → 任务 portMAX_DELAY 收走翻灯」的可跑管道与串口日志。

边界

只解剖 V11.1.0 queue.c 的单核路径；队列集（Queue Set）仅点名、选型细节留 F6；大对象传指针只给所有权纪律不给方案；SMP 行为不外推，中断优先级撞 configMAX_SYSCALL 的崩溃机制在 F2/F5 展开。

依据锚点 `f1` · `mcu-code` · `mcu-anim`

> 🎯 队列是 RTOS 里最忙碌的传送带：数据从这头进那头出，满了送件人排队，空了取件人排队。FreeRTOS 用一个结构体装下全部：一段环形存储 + 两张"排队名单"。本章把 `xQueueGenericSend` 拆开看，你会发现**信号量、互斥锁、队列全是这一个结构体的不同用法**。

本章精髓

1. **Queue_t 五要素**：`pcHead/pcWriteTo/pcReadFrom` 环形存储 + `xTasksWaitingToSend/xTasksWaitingToReceive` 两张阻塞名单，一切队列行为都是这五样的组合。
2. **发送是"拷贝"不是"传指针"**：`memcpy` 按值进环形区，简单安全，代价是大数据拷贝慢（大对象传指针，这是设计决策不是偷懒）。
3. **阻塞有公平也有超时**：等待者按**优先级**挂名单，唤醒时高优先级先拿；超时到点自动复活带错误码，永不死等（除非你坚持 `portMAX_DELAY`）。

- **能理解**：为什么写满阻塞和读空阻塞是**两张**名单而不是一张；为什么 `xQueueOverwrite` 适合"最新值"场景；为什么 FromISR 版本不能阻塞。
- **能用**：串口中断→队列→任务的标准管道；会算队列 RAM 账（深度×项大小）。

先修

- F3 调度、S7 USART。

先跑起来（10 分钟 quick win）

按键中断里 `xQueueSendFromISR` 发按键值，任务里 `xQueueReceive(..., portMAX_DELAY)` 收到就翻灯，第一个"中断→任务"标准管道，睡觉不耗 CPU。

按键不来的时候，收件任务在干嘛：原地空转数羊，还是干脆睡死让出 CPU？先押个睡法，押完看它睡在队列的哪张名单上。

动画：传值不传址的传送带

生产者拷贝进 tail、消费者从 head 取走，head/tail 指针此消彼长；取空就睡、放满也睡——**睡觉不耗 CPU** 是 RTOS 通信和 while 轮询的本质区别。

队列传送动画

小节结构

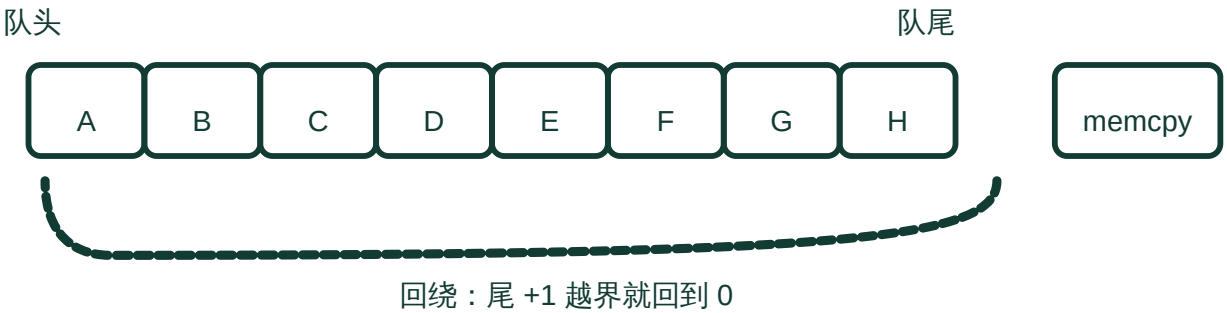
小节	内容	四件套
一、结构体五要素	QueueDefinition 逐字段（含 cRxLock/cTxLock 锁计数）	库解析
二、拷贝入队	`prvCopyDataToQueue`：memcpy+指针回绕，互斥锁彩蛋	库解析
三、发送三分支	空位/覆盖/阻塞——`xQueueGenericSend` 走查	库解析
四、唤醒与公平	`xTaskRemoveFromEventList`：按优先级叫号	代码分析
五、FromISR 家族	为什么不能阻塞；`pxHigherPriorityTaskWoken` 延迟让出	代码分析
六、RAM 账与设计	深度×项大小；水位查询；队列集预告	配置

一、结构体五要素：一个结构体三种人生

queue.c（V11.1.0）的 `QueueDefinition`，关键字段原文：

```
typedef struct QueueDefinition
{
    int8_t * pcHead;     /* 存储区起点 */
    int8_t * pcWriteTo;  /* 下一个可写位置 */

    union
    {
        QueuePointers_t xQueue;     /* 当队列用时:pcReadFrom/pcTail */
        SemaphoreData_t xSemaphore; /* 当信号量/互斥锁用时:持有者等 */
    } u;

    List_t xTasksWaitingToSend;     /* 写满阻塞的送件人名单(按优先级排) */
    List_t xTasksWaitingToReceive;  /* 读空阻塞的取件人名单(按优先级排) */

    volatile UBaseType_t uxMessagesWaiting; /* 当前存货数 */
    UBaseType_t uxLength;                   /* 容量:能装几项(不是几字节) */
    UBaseType_t uxItemSize;                 /* 每项几字节 */

    volatile int8_t cRxLock;  /* 队列锁定期间被取走的项数 */
    volatile int8_t cTxLock;  /* 队列锁定期间被塞进的项数 */
} xQUEUE;
```

读三遍那个 `union`：**队列和信号量共享同一个结构体**，信号量就是 `uxItemSize=0` 的队列（没有数据只有计数），互斥锁再多带一个持有者指针。所以 F5 的信号量 API 全是 `xQueueGenericSend/Receive` 的马甲。`uxItemSize=0` 这个分支马上在下一节现身。



FreeRTOS 队列环形缓冲：入队 memcpy 到队尾、出队 memcpy 到局部变量；指针回绕，传值不传址

二、拷贝入队：memcpy + 指针回绕

真正搬数据的 `prvCopyDataToQueue`（节选）：

```
if( pxQueue->uxItemSize == ( UBaseType_t ) 0 )
{
    /* 信号量/互斥锁:没有数据可拷 */
    if( pxQueue->uxQueueType == queueQUEUE_IS_MUTEX )
    {
        xReturn = xTaskPriorityDisinherit( pxQueue->u.xSemaphore.xMutexHolder ); /* 还锁→去继承 */
        pxQueue->u.xSemaphore.xMutexHolder = NULL;
    }
}
else if( xPosition == queueSEND_TO_BACK )
{
    ( void ) memcpy( ( void * ) pxQueue->pcWriteTo, pvItemToQueue, ( size_t ) pxQueue->uxItemSize );
    pxQueue->pcWriteTo += pxQueue->uxItemSize;
    if( pxQueue->pcWriteTo >= pxQueue->u.xQueue.pcTail )
    {
        pxQueue->pcWriteTo = pxQueue->pcHead;   /* 到尾回绕:环形 */
    }
}
else  /* queueSEND_TO_FRONT */
{
    ( void ) memcpy( ( void * ) pxQueue->u.xQueue.pcReadFrom, pvItemToQueue, ... );
    pxQueue->u.xQueue.pcReadFrom -= pxQueue->uxItemSize;
    ...
}
```

三行看清全部本质：**memcpy 按值拷贝、写指针前进、到尾回绕**。`uxItemSize==0` 分支就是互斥锁"还钥匙"的现场，还锁时顺手做**优先级去继承**（F5 的重头戏，先记住这行 `xTaskPriorityDisinherit`）。

> **【注】** "按值拷贝"是 FreeRTOS 的标志性取舍：小数据（结构体、传感器读数）直接拷，安全无脑；大对象（帧缓冲）改为**传指针**：队列里装的是地址，所有权归你自己管。拷指针时记得指向的生命周期必须长过消费时刻，栈变量地址是头号翻车素材。

三、发送三分支：`xQueueGenericSend` 走查

主循环的判别式（节选）：

```
taskENTER_CRITICAL();
{
    if( ( pxQueue->uxMessagesWaiting < pxQueue->uxLength ) || ( xCopyPosition == queueOVERWRITE ) )
    {
        xYieldRequired = prvCopyDataToQueue( pxQueue, pvItemToQueue, xCopyPosition );

        /* 有取件人在等? 立刻叫醒 */
        if( listLIST_IS_EMPTY( &( pxQueue->xTasksWaitingToReceive ) ) == pdFALSE )
        {
            if( xTaskRemoveFromEventList( &( pxQueue->xTasksWaitingToReceive ) ) != pdFALSE )
            {
                queueYIELD_IF_USING_PREEMPTION();  /* 叫醒的比我优先级高→当场让出 */
            }
        }
        ...
        taskEXIT_CRITICAL();
        return pdPASS;
    }
    else  /* 满了且不是覆盖写 */
    {
        vTaskPlaceOnEventList( &( pxQueue->xTasksWaitingToSend ), xTicksToWait ); /* 送件人排队 */
        ...
    }
}
```

三分支一目了然：**有空位→拷贝+叫醒取件人；覆盖写→照样拷（`queueOVERWRITE` 直接放行，长度 1 的队列专用，"只要最新值"场景如心跳/状态）；满了→自己挂上 `xTasksWaitingToSend` 名单睡觉**。注意让出发生在**叫醒的取件人比自己优先级高**时，公平就藏在这行 `xTaskRemoveFromEventList` 的返回值里。

四、唤醒与公平：按优先级叫号

`xTaskRemoveFromEventList` 从名单上摘下的是**优先级最高**的等待者（名单按优先级排序插入，F3 的 `vListInsert` 老熟人），而不是来得最早的。设计哲学：**RTOS 的公平是"重要的先来"，不是"先来的先来"**。被叫醒的任务从事件名单搬家到就绪表，如果它优先级压过当前任务，立刻 YIELD，数据还没到手的任务就已经被调度去取数据了，这就是"叫醒"与"交接"一气呵成的流水线。

接收侧 `xQueueGenericReceive` 是它的镜像：有货→拷出+叫醒送件人；没货→挂 `xTasksWaitingToReceive`。对称结构，读源码时对照看事半功倍。

五、FromISR 家族：中断里的单行桥

中断里为什么不能调普通版？两个死穴：**不能阻塞**（中断没有"任务"可挂）、**不能当场切上下文**（中断里 YIELD 时机非法）。FromISR 版的做法：

1. 永远不等：满了直接返回 `errQUEUE_FULL`，丢不丢由你决定；
2. 让出延迟：把"要不要换任务"写进 `pxHigherPriorityTaskWoken` 输出参数，由你在中断末尾调 `portYIELD_FROM_ISR(xHigherPriorityTaskWoken)` 统一结算。

标准姿势：

```
void EXTI0_IRQHandler( void )
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    uint8_t key = 0x01;
    xQueueSendFromISR( xKeyQueue, &key, &xHigherPriorityTaskWoken );
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );  /* 需要才切,一次结清 */
}
```

> 【注】 中断优先级必须 ≥ `configMAX_SYSCALL_INTERRUPT_PRIORITY`（数值上 ≥，Cortex-M 数值越大优先级越低，F3 常见坑清单的崩溃第二名），否则 FromISR 在临界区里"裸奔"，list.c 注释里那份崩溃清单就是给你准备的。

六、RAM 账与设计：传送带要多长

- **RAM = uxLength × uxItemSize + 结构体本身**，深度 10、项 16 字节的队列 ≈ 160B+结构体，开队列前先算账；
- **深度设计**：按"生产-消费速度差×最长延迟"估，再用水位 API `uxQueueSpacesAvailable` / `uxQueueMessagesWaiting` 实测修正：**先估后测**，别拍脑袋；
- **队列集**（Queue Set）：一个任务同时等多路来源时用 `xQueueSelectFromSet`，F6 事件组章会对比选型。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **队列=环形仓库+两张候客名单；发送按值 memcpy，满则睡空则睡；ISR 走专用通道延迟让出；信号量就是 uxItemSize=0 的队列。**

延伸：队列的生产-消费模式与 DMA 环形缓冲（S8）同构；1:1 通知场景可用任务通知（F6）替代队列，更轻量。

实物实验

- quick win 按键管道 + 串口打印队列水位：连按 20 次看 `uxQueueMessagesWaiting` 涨到满；
- 翻车实验（选做）：中断里故意调普通版 `xQueueSend`，观察断言/HardFault，再读 list.c 崩溃清单对号。

常见坑

- **大结构体直接拷**：队列深度 5 × 结构体 1KB = 5KB RAM 蒸发，传指针；
- **传了栈变量地址**：函数返回后指针成野指针，所有权纪律；
- **FromISR 忘了结算让出**：高优先级任务明明醒了却跑到下一拍才执行，延迟一拍在电机控制里就是事故；
- **`portMAX_DELAY` 当默认值用**：队列一满/一空任务永久失踪，调试先给有限超时，上线再放宽；
- **中断优先级撞 configMAX_SYSCALL**：见第五节注。

短自测

1. 为什么说"信号量就是队列"？源码依据是什么？

> **▼ 参考答案**
> QueueDefinition 里用 union 复用存储：uxItemSize=0 时 prvCopyDataToQueue 不做 memcpy 只维护计数/持有者：信号量=无数据的队列，互斥锁再带持有者指针和继承逻辑。F5 会看到 xSemaphoreGive 直接展开成 xQueueGenericSend。

2. `xQueueOverwrite` 和普通发送在"队列满"时行为有何不同？各自适合什么场景？

▼ 参考答案
普通发送：满了→调用者阻塞挂 xTasksWaitingToSend;Overwrite:直接覆盖队首旧值永远成功。长度 1+Overwrite 适合"只要最新值"(心跳/状态广播),普通阻塞适合"一个都不能丢"(命令/数据流)。

3. FromISR 版本为什么不能阻塞？`pxHigherPriorityTaskWoken` 解决了什么？

▼ 参考答案
中断上下文没有"任务"可挂起，也不能在中断中途切换上下文。pxHigherPriorityTaskWoken 把"是否需要切换"作为输出参数带回，由代码在中断末尾调 portYIELD_FROM_ISR 统一结算，让出时机合法且只切一次。

4. 发送方拷贝完数据后，为什么可能立刻 YIELD？依据什么判断？

▼ 参考答案
若 xTasksWaitingToReceive 非空，xTaskRemoveFromEventList 叫醒等待者；该函数在被叫醒者优先级高于当前任务时返回 pdTRUE,于是 queueYIELD_IF_USING_PREEMPTION 当场让出，数据一到，最高优先级的消费者立刻接手。

🎯 F4 队列自测

1. **为什么说"信号量就是队列"？源码依据是什么？**
   A. API 名字相近，只是叫法习惯
   B. QueueDefinition 用 union 复用存储：uxItemSize=0 时发送只维护计数、不 memcpy——信号量=无数据的队列
   C. 信号量内部另有一套独立结构，只是接口长得像
   D. 互斥锁才是队列，普通信号量不是

2. **队列已满，xQueueGenericSend 与 xQueueOverwrite 的行为差异是？**
   A. 两者都返回失败，只是错误码不同
   B. 普通发送阻塞在 xTasksWaitingToSend；Overwrite 直接覆盖队首旧值、永远成功
   C. Overwrite 会先清空整个队列再写入
   D. 普通发送丢新数据返回成功；Overwrite 丢旧数据返回失败

3. **中断里调队列 API 为什么必须用 FromISR 版本？**
   A. 中断里堆栈太小，普通版本参数放不下
   B. FromISR 版本更快，纯性能考虑
   C. 中断上下文没有"任务"可挂起、不能中途切换上下文；是否需要让出用 pxHigherPriorityTaskWoken 带回，在中断末尾统一结算
   D. 普通版本会破坏队列数据，FromISR 版本不会

4. **发送方拷贝完数据后可能当场 YIELD，判断依据是什么？**
   A. 队列剩余空间为零
   B. 被叫醒的等待者优先级高于当前任务——xTaskRemoveFromEventList 返回 pdTRUE
   C. 系统节拍中断恰好到来
   D. 发送超时时间到了

5. **prvCopyDataToQueue 写数据时队列缓冲区已到尾端，怎么处理？**
   A. 拒绝本次写入返回失败
   B. memmove 整个缓冲区腾出空间
   C. 分两段拷贝：先写到物理尾端，再从头写剩余字节（环形回绕）
   D. 自动扩容队列存储区

先选完再翻：答案与逐项解析在附录 C 的本组；在线版可点击即时判分。

对照表：本章概念 → 源码落点

概念	落点（**FreeRTOS-Kernel V11.1.0**）
五要素结构体	`queue.c` QueueDefinition（union 队列/信号量双生）
按值拷贝+回绕	`queue.c` prvCopyDataToQueue（memcpy/pcWriteTo 回绕/互斥锁去继承彩蛋）
发送三分支	`queue.c` xQueueGenericSend（空位/覆盖/阻塞+唤醒取件人）
优先级叫号	`tasks.c` xTaskRemoveFromEventList（名单按优先级排序）
ISR 通道	`queue.c` xQueueSendFromISR（不阻塞+pxHigherPriorityTaskWoken）

你做到了

- 队列从黑盒 API 变成"环形存储+两张名单"的可推理机器；
- 信号量/互斥锁的血缘被提前揭穿，F5 只剩继承逻辑可讲；
- ISR→任务管道成为你的通信肌肉记忆。

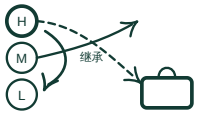
它真睡了，睡在名单上：按键不来，收件任务挂进队列的阻塞名单，CPU 转身去喂别的任务。轮询是站着等，阻塞是躺下睡，这一觉就是 RTOS 通信和裸机轮询的分界线。

✅ 下一站：F5 信号量与互斥锁：计数牌与钥匙，以及优先级继承怎么把 H 从死亡螺旋里捞出来。

通往单片机之路第 41 章

F5 信号量与互斥量：优先级反转事件

难度 ★★☆☆☆ · 约 40 分钟



本章罗盘

问题

H 等 L 手里的锁、M 在一旁狂跑，高优先级为什么反而垫底，继承在源码哪几行把 L 抬起来？

交付

E04 三任务剧本双版本执行序判读（二值信号量 waited≈6000ms vs 互斥量≈2500ms，并说清两个数的理论/模型值身份）+ xTaskPriorityInherit「判→改→搬」三步逐行注解与四种锁选型表。

边界

≈6000/≈2500 是按剧本参数推出的理论值，sim_invert.c 复现也只是模型值，E04 板上实测表至今空白，不能当串口日志引用；继承不防死锁（D2 的结论），多锁只给顺序纪律；行号与 ISR 取锁无持有者的结论对 V11.1.0 说话。

依据锚点 `d2` · `d11` · `f1` · `mcu-lab` · `mcu-code` · `mcu-anim`

> 🎯 1997 年火星探路者在火星上神秘重启——元凶是地球上的经典 bug：**优先级反转**。高优先级任务等资源、资源在低优先级手里、中优先级任务疯狂插队。本章在你的霸天虎上复现它，再用互斥量的"优先级继承"破解它——源码级别，逐行看清。

本章精髓

1. **二值信号量管"同步"，计数信号量管"几个"**：车位牌（一手交一手接）与车位池（N 个资源），它们**不保护数据**，只是发令枪。
2. **互斥量是"带继承的钥匙"**：低优先级持锁被高优先级等时，**临时继承高优先级**（`xTaskPriorityInherit`），中优先级插不进来；还锁时 `xTaskPriorityDisinherit` 回落。
3. **血缘关系**：信号量/互斥锁全是 F4 那个队列结构体的马甲：`xSemaphoreGive` 展开就是 `xQueueGenericSend`，互斥锁 = 长度 1、项大小 0 的队列 + 持有者指针 + 继承逻辑。

- **能理解**：为什么互斥锁创建时要先"白送一次"；继承发生在哪一行代码；为什么二值信号量**不能**当互斥锁用（无继承）。
- **能用**：E04 复现反转→换互斥锁破解；多锁场景定死获取顺序。

先修

- F4 队列（信号量基于队列实现）、F3 调度。

先跑起来（10 分钟 quick win）

E04 优先级反转实验：把 `DEMO_SCENE=6` 烧上板子，串口日志就打出三任务执行序："H 等锁，M 狂跑，L 没机会还锁"的死亡螺旋。注意 E04 的"实测记录"那张表至今是空的，所以这一集里那两个数都还不来自板子（见第四节末尾的注）。

死亡螺旋先看清，解药先押下：把二值信号量换成互斥量，M 还狂跑得起来吗？L 又靠什么突然腿快了？押完往下读，破解现场见真章。

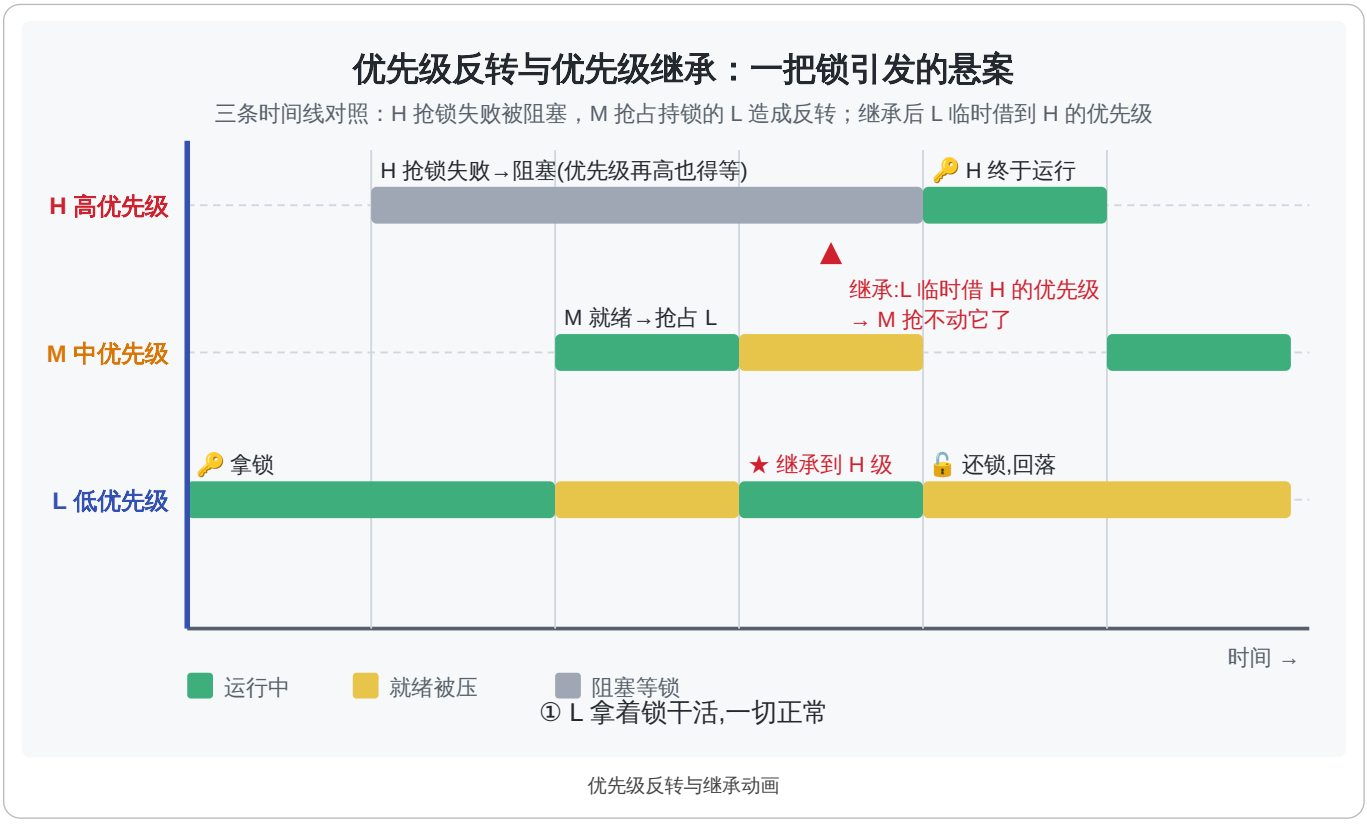
动画：计数牌与钥匙

左边停车场计数（信号量管"几个"），右边厕所钥匙（互斥锁管"谁的"）。第四帧看**优先级继承**现场：L 持锁被 H 等，L 临时升到 H 的优先级快进快出。

信号量与互斥锁动画

动画：反转现场与继承救命

三条泳道看完整悬案：L 持锁 → H 抢锁阻塞 → M 把 L 挤下台（**反转**：H 实际垫底）→ 继承把 L 抬到 H 级 → 还锁回落，H 终于开跑。游标六拍走完，你就记住了"为什么互斥锁要带继承"。



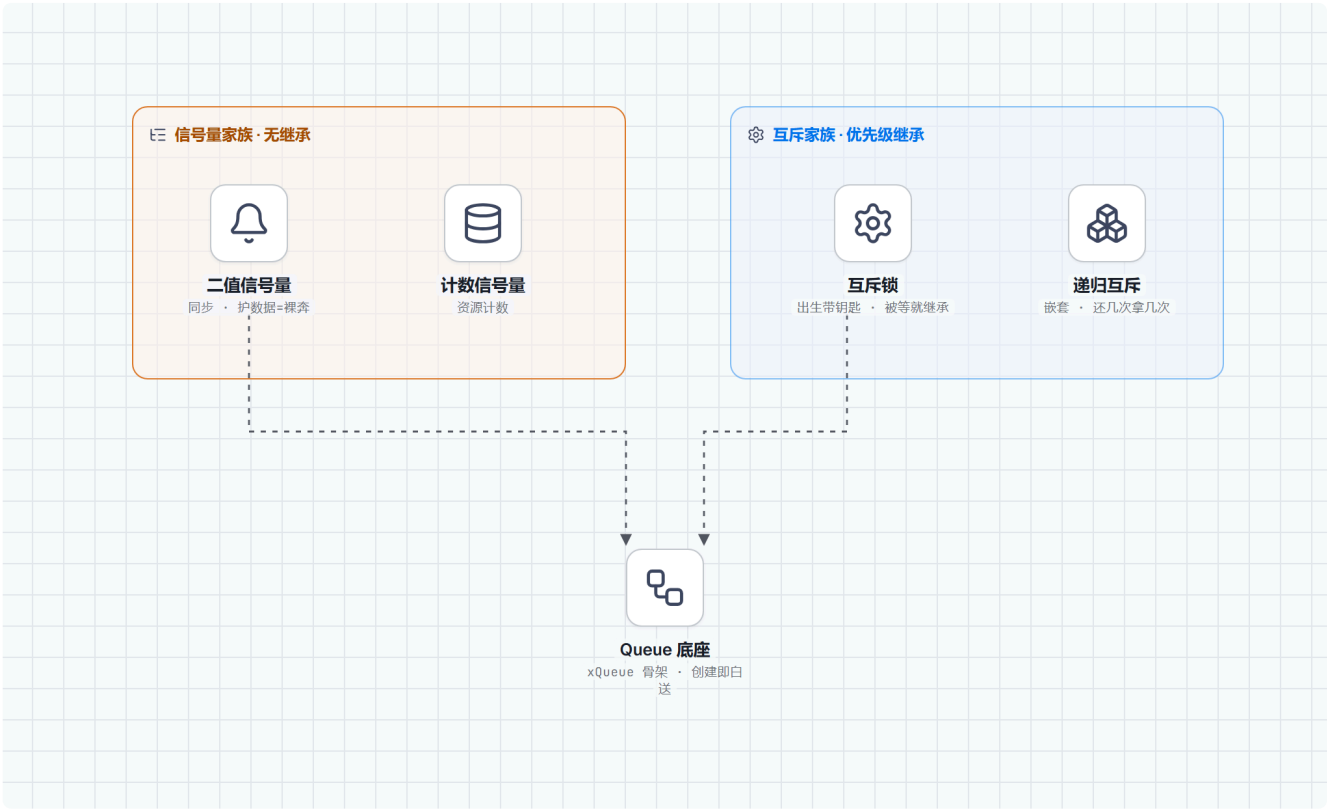
优先级反转与继承动画

小节结构

小节	内容	四件套
一、宏的血缘	semphr.h 三个宏展开：全是 queue.c 马甲	库解析
二、互斥锁出生证	`prvInitialiseMutex` ：为什么创建即"白送"	库解析
三、继承逐行	`xTaskPriorityInherit` 搬表现场	库解析
四、反转实验	E04 三任务配置；≈6000/≈2500 那两个数是什么身份	代码分析
五、选型与纪律	四种选型；递归配对；多锁顺序	代码分析

一、宏的血缘：semphr.h 没有函数，只有马甲

马甲家族先看图：两大家族四件马甲，底座是同一个 Queue。



信号量与互斥量马甲家族：信号量无继承、互斥有优先级继承，底座同为 Queue

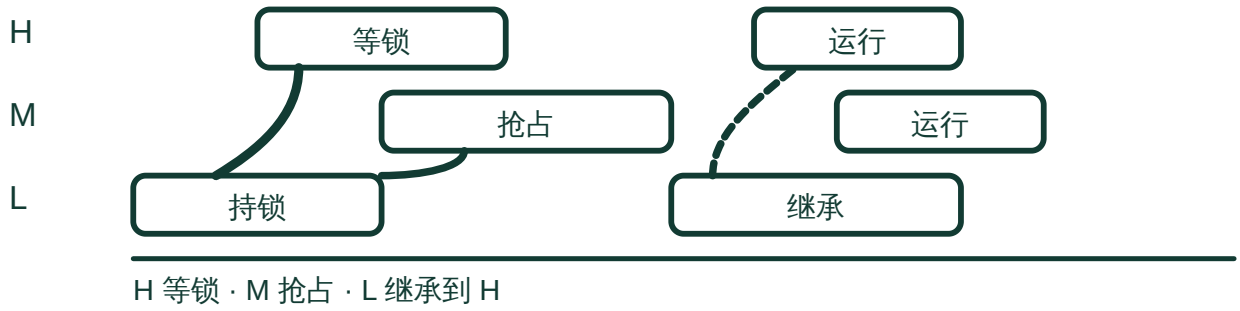
semphr.h（V11.1.0）里最常用的三个 API，原文就是三行宏：

```
#define semGIVE_BLOCK_TIME  ( ( TickType_t ) 0U )

#define xSemaphoreTake( xSemaphore, xBlockTime )  xQueueSemaphoreTake( ( xSemaphore ), ( xBlockTime ) )
#define xSemaphoreGive( xSemaphore )              xQueueGenericSend( ( QueueHandle_t ) ( xSemaphore ), NULL, semGIVE_BLOC
#define xSemaphoreCreateMutex()                   xQueueCreateMutex( queueQUEUE_TYPE_MUTEX )
```

give = 往队列发 NULL、不等、发队尾，就是 F4 的 `xQueueGenericSend` 换件外套。所以"信号量是 uxItemSize=0 的队列"不是比喻，是编译后的事实：give 时 `prvCopyDataToQueue` 走 `uxItemSize==0` 分支，不拷数据只加计数。这也解释了为什么 give 永不阻塞（ `semGIVE_BLOCK_TIME=0` ）：发令枪不需要排队。

选型由此自然分化：

类型	管什么	初始计数	带继承
二值信号量	同步（一件事发生了）	0（空）	否
计数信号量	资源池（N 个同类资源）	N（满）	否
互斥锁	共享数据排他访问	1（空着可用）	**是**
递归互斥锁	同一任务嵌套加锁	1	是



优先级反转与继承：高优先级任务等锁、中优先级任务抢占、持锁的低优先级任务临时继承高优先级

二、互斥锁出生证：创建即"白送"一次

`xQueueCreateMutex`（queue.c）先建一个**长度 1、项大小 0** 的队列，再调 `prvInitialiseMutex`：

```
static void prvInitialiseMutex( Queue_t * pxNewQueue )
{
    pxNewQueue->u.xSemaphore.xMutexHolder = NULL;        /* 尚无持有者 */
    pxNewQueue->uxQueueType = queueQUEUE_IS_MUTEX;
    pxNewQueue->u.xSemaphore.uxRecursiveCallCount = 0;   /* 递归计数归零 */

    /* Start with the semaphore in the expected state. */
    ( void ) xQueueGenericSend( pxNewQueue, NULL, ( TickType_t ) 0U, queueSEND_TO_BACK );
}
```

注意最后一行：**创建时立刻 give 一次**，互斥锁出生就"钥匙挂在门上"（计数=1，可用）。谁第一个 take 谁拿到。对照：二值信号量创建后是 0（事件发生前没牌可拿），计数信号量是 N。同一个 `xQueueGenericCreate`，初始计数不同，三种人生由此分开。

三、继承逐行：`xTaskPriorityInherit` 的搬表现场

H 任务 take 互斥锁失败、持有者 L 优先级比自己低时，`xQueueSemaphoreTake` 内部调 `xTaskPriorityInherit`（tasks.c 节选）：

```
if( pxMutexHolderTCB->uxPriority < pxCurrentTCB->uxPriority )  /* 持锁者比我低才继承 */
{
    /* 事件列表项的值改成"新优先级"的编码,保证在事件名单里按新身份排序 */
    listSET_LIST_ITEM_VALUE( &( pxMutexHolderTCB->xEventListItem ),
        ( TickType_t ) configMAX_PRIORITIES - ( TickType_t ) pxCurrentTCB->uxPriority );

    /* 若持锁者在就绪表里:先摘出来 */
    if( listIS_CONTAINED_WITHIN( &( pxReadyTasksLists[ pxMutexHolderTCB->uxPriority ] ),
                                 &( pxMutexHolderTCB->xStateListItem ) ) != pdFALSE )
    {
        if( uxListRemove( &( pxMutexHolderTCB->xStateListItem ) ) == ( UBaseType_t ) 0 )
        {
            portRESET_READY_PRIORITY( pxMutexHolderTCB->uxPriority, uxTopReadyPriority );
        }
        pxMutexHolderTCB->uxPriority = pxCurrentTCB->uxPriority;  /* ★ 继承:升级 */
        prvAddTaskToReadyList( pxMutexHolderTCB );                /* 挂进新优先级的就绪表 */
    }
    else
    {
        pxMutexHolderTCB->uxPriority = pxCurrentTCB->uxPriority;  /* 阻塞中:只改数不搬表 */
    }
}
```

流程三步：**判（持锁者更低）→ 改（优先级=等待者的）→ 搬（从旧就绪链表摘出、挂进新链表）**。`portRESET_READY_PRIORITY` + `prvAddTaskToReadyList` 正是 F3 位图/速查器那套记账，升级后 L 立刻具备和 H 同级的调度地位，**M 再也插不进来**。还锁时走镜像路径 `xTaskPriorityDisinherit`（F4 第二节那行彩蛋）：优先级回落原值，若回落后低于新就绪任务，立刻让出。

> 【注】 继承只解"中优先级插队"，不解"持锁者自己磨蹭"。临界区还是要快进快出：继承是安全带，不是油门。另外二值信号量**没有**这套逻辑：拿它保护共享数据，反转照样发生，这是选型错误最隐蔽的形态。

四、反转实验：E04 的死亡螺旋判读

三任务配置（`code/rtos/01-freertos-lab/main.c:309-358`，`DEMO_SCENE=6`，`USE_MUTEX` 编译期二选一把锁换成互斥量）：

任务	优先级	行为
L	1	t=0 拿"锁"→ 临界区 `cpu_work_ms(3000)` → 还锁 → 长眠
M	2	t=500ms 起 `busy_until(4500)`：不碰锁、不阻塞，连续占 CPU 到 t=4500ms
H	3	t=1000ms 要同一把"锁"，拿到后打印 `waited`

台词全在 `waited` 这一个数上。二值信号量没有持有者，H 在 t=1000ms 就挂住了，可 CPU 归 M 一路霸占到 4500ms，L 才轮到跑满 3000ms，按剧本参数推出的 `waited ≈ 6000ms`。换互斥量，H 挂下的同一刻 L 被抬到 3 级，M 抢不动它，L 跑完 3000ms 就还锁，`waited ≈ 2500ms`。

> 【注】 这两个数是**理论值，不是实测**。E04 文档里"实测记录"那张表到今天还是空的，谁在屏上把它们说成板子上串口打出来的毫秒，就是替 E04 撒谎。`code/rtos/05-sem-mutex/sim_invert.c` 跑出来的是同一对数，身份也只到**模型值**为止（日志判读细节见 E04）。

五、选型与纪律：剩下的都是工程规矩

- **同步选二值**：ISR give → 任务 take 是标准范式（比队列轻，无数据）；
- **资源池选计数**：如"同时只允许 3 个连接"，创建计数=3；
- **共享数据选互斥**：带上继承保险；**递归互斥**用于同一任务嵌套调用（take 几次 give 几次，`uxRecursiveCallCount` 就是出生证里那个字段）；
- **多锁定序**：全系统约定统一的加锁顺序（如永远先 A 后 B），破坏者死锁，这是防死锁的**唯一**纪律，没有银弹；
- **临界区纪律**：不调用可能阻塞的 API、不做长计算，继承救不了磨蹭。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **同步二值、资源计数、保护互斥、嵌套递归；互斥锁出生带钥匙、被等就继承、还锁即回落；二值信号量没有继承，拿它护数据等于裸奔。**

延伸：I2C/SPI 总线共享需互斥锁保护（S11、S12）；简单同步场景可用任务通知（F6）替代信号量。

实物实验

- E04 优先级反转实验：二值版死亡螺旋 + 互斥锁版起死回生，两份日志贴进实验报告；
- 进阶：把 L 的临界区从 `cpu_work_ms(3000)` 改小（如 200ms）再跑二值信号量版，反转窗口变窄但**依然存在**，理解"窗口大小≠有无"。

常见坑

- **二值信号量当互斥锁**：无继承，反转照常，选型第一坑；
- **take/give 不配对**：递归锁少 give 一次，其他任务永久等锁；
- **ISR 里 take 互斥锁**：`xSemaphoreTakeFromISR` 展开为 `xQueueReceiveFromISR`（semphr.h:675 → queue.c:2035），那条路上不写 `xMutexHolder`、不比 `uxQueueType`、也不调 `xTaskPriorityInherit`（`queue.c:2035-2132` 整个函数体里这三样各命中 0 处）：中断里就算"取到"也不会登记持有者，于是既没有继承也没有回落；所有权是任务之间才谈得成的事；
- **临界区里 vTaskDelay**：持锁睡觉，把继承机制变成全系统的瓶颈放大器；
- **多锁乱序**：A→B 与 B→A 两条路径并存，死锁迟早登门。

短自测

1. `xSemaphoreGive` 为什么不阻塞？从宏展开回答。

> **▼ 参考答案**
> 它展开为 xQueueGenericSend(队列， NULL, semGIVE_BLOCK_TIME, queueSEND_TO_BACK),而 semGIVE_BLOCK_TIME=0,等待时间为零，满了立刻返回错误，不存在挂起。

2. 互斥锁创建时为什么要调一次 xQueueGenericSend？

> **▼ 参考答案**
> 让计数初始为 1="钥匙挂在门上",第一个 take 者直接拿到。二值信号量不送(初始 0=事件未发生),计数信号量送满 N 次，初始计数区分三种选型。

3. `xTaskPriorityInherit` 的三步是什么？为什么必须"搬表"？

> **▼ 参考答案**
> 判(持锁者更低)→改(uxPriority=等待者优先级)→搬(从旧就绪链表摘除、portRESET 记账、挂进新链表)。不搬表则调度器仍在旧链表/旧位图位里找它，改了优先级也轮不到它跑，继承失效。

4. 为什么二值信号量不能替代互斥锁保护共享数据？

> **▼ 参考答案**
> 二值信号量没有持有者与继承逻辑(xTaskPriorityInherit 只在互斥锁 take 失败路径被调用)。用它护数据时，H 等、L 持、M 插队，反转完整上演，`DEMO_SCENE=6` 那出不带 `USE_MUTEX=1` 的剧本就是演这件事的。

对照表：本章概念 → 源码落点

概念	落点（**FreeRTOS-Kernel V11.1.0**）
信号量=队列马甲	`semphr.h` xSemaphoreGive/Take/CreateMutex 宏定义
互斥锁出生证	`queue.c` xQueueCreateMutex → prvInitialiseMutex（创建即 give）
继承搬表	`tasks.c` xTaskPriorityInherit（判→改→搬三步）
还锁回落	`queue.c` prvCopyDataToQueue 互斥锁分支 → xTaskPriorityDisinherit
反转实验	板子：E04（`DEMO_SCENE=6`）+动画 priority-inversion.svg；不要板子：`code/rtos/05-sem-mutex/probe.sh` 静态血缘 + `sim_invert.c` 反转模型

延伸阅读

优先级继承的理论与那次著名事故：

- **[D2]** Sha, Rajkumar, Lehoczky 1990 — 优先级继承协议的原始论文，也证明了它**不防死锁**（优先级天花板才防）；`xSemaphoreCreateMutex` 继承逻辑的理论出处。

- **[D11]** Reeves 1997 — 火星探路者优先级反转事故的 JPL 一手复盘，`configUSE_MUTEXES` 这一行的历史重量。
- **[D10]** Dijkstra 1965 — 信号量的起点。

你做到了

- RTOS 最著名的灵异事件在你板子上复现并被你亲手破解；
- "信号量 vs 互斥锁"从背题变成"看宏展开就知道"；
- 继承机制三步刻进脑子，面试白板题变成默写题。

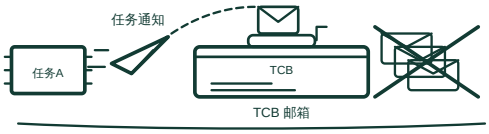
解药的名字叫继承：L 不是变强了，是临时借了 H 的优先级，M 从此插不进队，锁准时归还。优先级反转不是灵异，是调度器没被递过话；递上话，同一块板子立刻讲道理。

✅ 下一站：F6 任务通知与事件组：比队列更快的"直达电报"。

通往单片机之路第 42 章

F6 任务通知、事件组与软件定时器

难度 ★★☆☆☆ · 约 50 分钟



本章罗盘

问题

软定时器回调里为什么禁止 vTaskDelay，「共用守护任务所以零成本」这句流行说法错在哪？

交付

一笔 nm 量出的 RAM 账：configUSE_TIMERS=1 即付 1312 B（守护任务 1128＋命令队列 184）、每定时器 56 B、.bss 另有 60 B 不归 configTOTAL_HEAP_SIZE 管；加五幕模拟的迟到/补拍/欠账表（Σ(耗时÷周期) 过 1 就发散）。

边界

字节数与五幕数字出自本机 heap_4＋CM4F＋V11.1.0、配置就是 01-freertos-lab 那份（通知槽=1、队列长 8），配置一换账就换；「快 45%」是历史某版本的实测，不能当无条件结论；分派规则用行号钉在上游，内核一升级脚本变红而非悄悄给错数。

依据锚点 `f1` · `mcu-code` · `mcu-anim`

> 🎯 队列通用但有"运费"：建队列要 RAM、发送要拷贝。任务通知直接写进对方 TCB——零队列、零拷贝、一次写操作。一对一的通信场景，通知几乎总是更优解；但"更快"快多少取决于配置与场景，别背固定百分比。

本章精髓

1. **任务通知 = TCB 里的 32 位邮箱**：每个任务在 TCB 里有 `ulNotifiedValue[]`（值）＋ `ucNotifyState[]`（状态）两个字段，槽位数由 `configTASK_NOTIFICATION_ARRAY_ENTRIES` 决定（本工程=1，所以"通知容量一"）。`xTaskNotifyGive` 走 eIncrement 累加，`xTaskNotify` 可覆盖/置位，**计数语义和覆盖语义是两条不同的路**。
2. **事件组 = 24 个标志位的"公告栏"**（TickType_t 32 位，扣掉 8 个控制位）：多任务按"任意/全部"条件等待一组位。**清位不是先唤醒者独占**：`xEventGroupSetBits` 把整个等待列表走一遍、满足条件的全部唤醒，最后按 `uxBitsToClear` **汇总一次清掉**（V11.1.0 `event_groups.c:547` 起）。
3. **软件定时器跑在"守护任务"里，但不是零成本**：`configUSE_TIMERS=1` 一开就固定付掉 **1312 B**（守护任务 1128 + 命令队列 184，此时一个定时器都还没建），每建一个定时器再收 **56 B**，另有 **60 B** 落在 `.bss`、`configTOTAL_HEAP_SIZE` 调多大都管不到它（本机 `timers.o` 实测，见 §八）。回调在守护任务上下文**串行**执行，代价也不是一句"晚一拍"：Σ(回调耗时 ÷ 周期) **小于一**时，零耗时定时器的最大迟到恰好等于最大的那次回调耗时；**不小于一**时迟到跳出耗时之外发散，并连环补拍（§五 的模拟器实测）。所以回调里**禁止阻塞**，周期性小任务用它代替创建真任务，省的是那 616 B 的栈与 TCB，不是 1312 B。

- **能理解**：为什么通知比队列快（无队列对象、无拷贝）；为什么"快 45%"不能当无条件结论；为什么事件组清位是汇总清而不是独占清。
- **能用**：用通知做一对一同步；用事件组做多条件汇合；用软件定时器做周期杂务。

先修

- F1 任务与 TCB、F4 队列、F5 信号量。

先跑起来（10 分钟 quick win）

不要板子的那条先跑：本章每个字节数都来自 `code/rtos/06-timers/probe.sh`，七步一条命令跑完（只要交叉编译器和上游内核）：

```
cd code/rtos/06-timers && sh probe.sh   # 结构体尺寸、机身立即数、.bss、五幕分派模拟
```

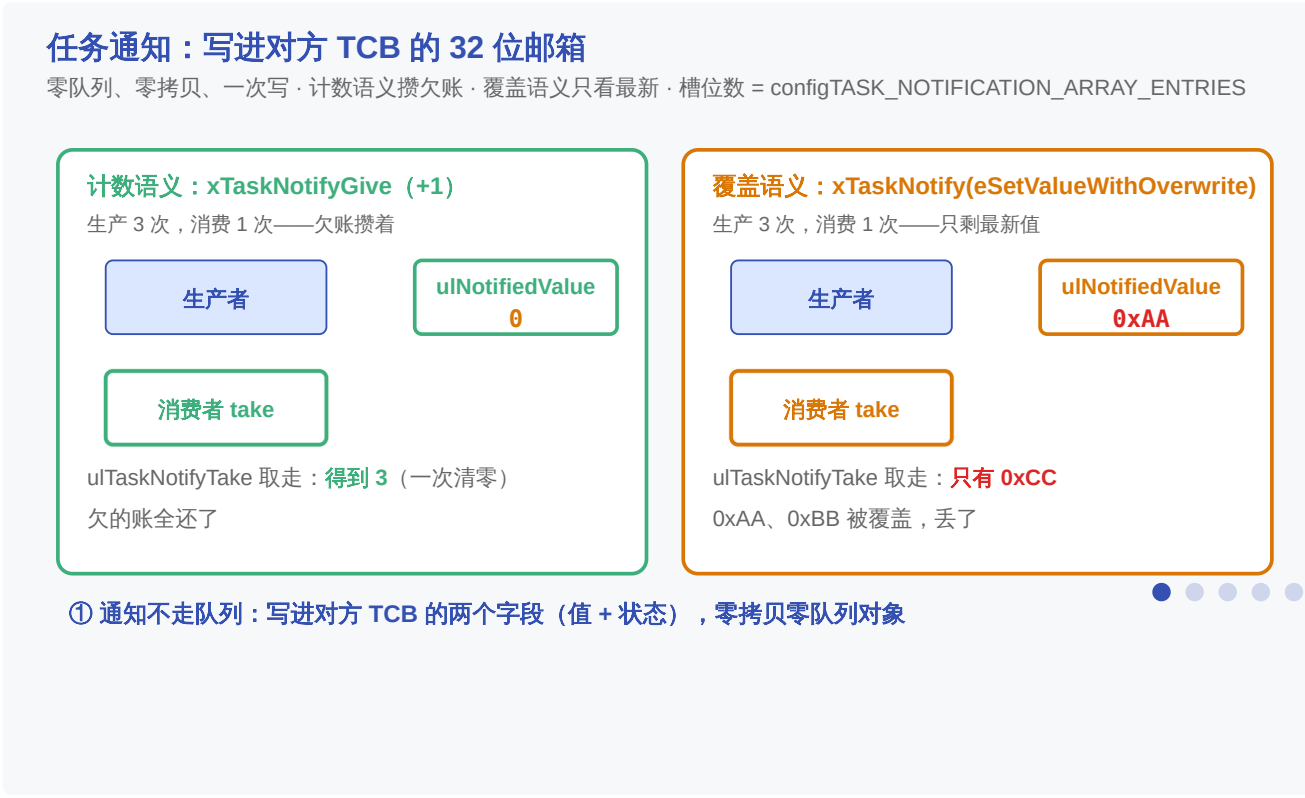
`code/rtos/01-freertos-lab` 场景 2/3/4（要板子）：

```
make DEMO_SCENE=2 flash    # 通知：took 的值就是"欠账总数"
make DEMO_SCENE=3 flash    # 事件组：ANY 先醒，ALL 等齐
make DEMO_SCENE=4 flash    # 定时器：slow cb 耗时 50ms → fast cb 最大迟到恰好 50ms
```

先押个首付：configUSE_TIMERS 这一行敲下去，一个定时器都还没建，RAM 先被划走多少？押完跑 probe.sh，尺寸表当账本。

动画：三条路各一张

通知的两种语义（计数 vs 覆盖）、事件组的 ANY/ALL 与汇总清位、软件定时器的命令队列→守护任务→回调，各自一张图：



任务通知动画

事件组等待动画

软件定时器动画

版本与配置前提

- 内核：上游 **FreeRTOS-Kernel V11.1.0**。通知/事件组/定时器的 API 名与语义以 `include/task.h`、`event_groups.c`、`timers.c` 为准。
- 本工程 `FreeRTOSConfig.h`：`configTASK_NOTIFICATION_ARRAY_ENTRIES=1`、`configUSE_TIMERS=1`、`configTIMER_TASK_PRIORITY=4`（守护任务跑最高）、`configUSE_16_BIT_TICKS=0`（事件组 24 个可用位）。

小节结构

小节	内容	四件套
一、通知原理	TCB 两字段；give/take/notify 三组 API 的语义差	库解析
二、通知 vs 队列/信号量	快在哪里、为什么；选型表（一对一/多对一/多对多）	配置
三、事件组	位语义/ANY/ALL/汇总清位	代码分析
四、软件定时器	守护任务模型；one-shot vs auto-reload	配置
五、回调纪律	不能阻塞的原因推导；回调里该干什么	代码分析
六、代码分析	场景 2/3/4 的日志对照	代码分析

一、通知原理：TCB 里的 32 位邮箱

每个任务的 TCB 里有两组字段（数组长度 = `configTASK_NOTIFICATION_ARRAY_ENTRIES`）：

- `ulNotifiedValue[]`：32 位值；
- `ucNotifyState[]`：状态（未通知/已通知待取）。

三组 API（V11.1.0 `task.h` 宏 → `tasks.c` 实现）：

API	语义	落点
`xTaskNotifyGive(x)`	通知值 +1（eIncrement），**计数语义**	`task.h:2984` → `xTaskGenericNotify`（`tasks.c:7832`）
`xTaskNotify(x, v, eSetValueWithOverwrite)`	直接覆盖，**覆盖语义**	`task.h:2611`
`ulTaskNotifyTake(pdTRUE, timeout)`	阻塞到计数非零，取走时清零	`task.h:3177` → `xTaskGenericNotifyWait`（`tasks.c:7717`）

注意函数名：是 **`ulTaskNotifyTake`**（返回计数值），不是 `xTaskNotifyTake`，后者不存在。

二、通知 vs 队列/信号量：快在哪里

通知比队列快的原因很具体：**没有队列对象、没有拷贝、没有额外的链表项**：写一次 TCB 就完事。但"快多少"取决于你的配置（队列深度、是否从中断上下文、调度器当时的状态），官方文档里出现过"约 45%"的实测数字，那是**某个版本某个配置下**的结果，不是无条件结论。

选型表：

场景	选型
一对一、容量一、只要信号	通知（give/take）
一对一、要带 32 位数据	通知（notify 覆盖语义）
多对一 / 一对多	队列
多对多、要带任意长数据	队列
要等"几个条件都齐"	事件组

三、事件组：ANY/ALL 与汇总清位

`xEventGroupWaitBits(组, 位掩码, xClearOnExit, xWaitForAllBits, timeout)`：

- `xWaitForAllBits=pdFALSE` → **ANY**：任一位满足就醒；
- `xWaitForAllBits=pdTRUE` → **ALL**：所有位都满足才醒。

清位的真相（V11.1.0 `event_groups.c` `xEventGroupSetBits`）：`SetBits` 会把整个等待列表**完整走一遍**，每个满足条件的任务都被唤醒；每个带 `xClearOnExit` 的等待者把它等的位并进 `uxBitsToClear`，**列表走完后统一清一次**（`pxEventBits->uxEventBits &= ~uxBitsToClear`）。

所以"第一个等到的人把位清掉、后来者永远等不到"是错的：**同一次 SetBits 里所有满足条件的等待者都能醒**，清位发生在所有人都看完之后。真正的坑在另一边：如果你想让"下一批人"也看到这些位，就别开 `xClearOnExit`。

四、软件定时器：守护任务模型

`xTimerCreate(名字, 周期, autoReload, id, 回调)` 创建的定时器**不是**一个独立的执行上下文。所有定时器的回调都跑在同一个**守护任务**里（`timers.c` 的 `prvTimerTask`）：

```
你的代码 xTimerStart() → 命令进 xTimerQueue → 守护任务取出执行回调
```

守护任务的优先级由 `configTIMER_TASK_PRIORITY` 决定（本工程=4，最高）。"只有一个守护任务"不是修辞，是数出来的：`timers.c` 全文创建任务的调用点 4 处（V11.1.0 的 `:260`、`:276`、`:295`、`:310`，覆盖静态/亲和性/动态几种组合），四处都在 `xTimerCreateTimerTask` 的函数体内、第一个实参全都写作 `prvTimerTask`，定时器系统不会为你多建任务。能调到用户回调的地方也只有 4 处（`:713`、`:744`、`:960`、`:1021`），分属三个 `static` 函数，而这三个函数只出现在 `prvTimerTask` 的调用树里。**所以回调串行是结构决定的，不是配置出来的。**

守护任务也不是"每拍醒一次"：它按 `xNextExpireTime - xTimeNow` 阻塞在命令队列上（`:820`），表空时下一个到期时刻是 0，于是睡到 tick 溢出；一轮处理里时钟只采样一次（`:795`），整轮判定共用同一个 `xTimeNow`。

串行磨蹭到底贵多少，§五 用模拟器把它算成了数，不是"晚一拍"这么轻。

one-shot vs auto-reload：`autoReload=pdTRUE` 周期触发，`pdFALSE` 触发一次就停。

五、回调纪律：为什么禁止阻塞

回调跑在守护任务里，守护任务是"所有定时器共用的执行线程"。你在回调里 `vTaskDelay`，守护任务阻塞，**所有**定时器停摆。所以回调里：

- **可以**：置标志、发通知、写队列（不带阻塞的版本）、改 GPIO；

- **不可以**：`vTaskDelay`、等队列满/空（带 timeout）、等信号量。

想让定时器"每 100ms 闪一下灯"：回调里只翻 GPIO，别在回调里睡。

磨蹭的代价能算：五幕实测

把守护任务的分派规则原样抄成宿主机模拟器（`code/rtos/06-timers/sim_daemon.c`，事件表写死，所以每次跑出的数一样），两个定时器 fast/slow，观测窗口 1200 拍。规则逐条钉在上游源码行号上：`probe.sh` 第 7 步用 `at_line` 校验这十一行，行号一漂这一步就失败，宁可红也不给错的数：R1 只看表头 `timers.c:859`、R2 到期比较 `:800` / `:803`、R3 睡到下一个到期 `:820`、R4 重装调用 `:735`、R5 补拍循环 `:706` / `:709` 与判据 `:904` / `:908`、R6 插入条件 `list.c:192`、R7 一轮一次采样 `:795`。

幕	fast 周期/耗时	slow 周期/耗时	Σ(耗时÷周期)	fast 最大迟到	fast 补拍	fast 连发	fast 欠账
A	100/0	120/0	0.000	0	0	1	0
B	100/0	120/50	0.416	**50**	0	1	50
E	100/0	300/200	0.666	**200**	6	3	200
C	100/0	120/200	1.666	**520**	6	6	520
D	100/0	120/250	2.083	470	5	5	**970**

一条律：**Σ(回调耗时 ÷ 周期) 小于一时，零耗时定时器的最大迟到恰好等于最大的那次回调耗时**：A 的 0、B 的 50、E 的 200 三幕全是这个形状；**一旦不小于一，迟到就跳出所有耗时之外发散**，C 幕的 520 早就盖过了那个 200，而 D 幕收工时 fast 还欠着 970 拍（观测窗口总共才 1200 拍）。注意 E 幕那个反直觉的组合：耗时 200 大于 fast 的周期 100，却小于 slow 自己的周期 300，占空比仍不满 → 迟到有界。所以关键从来不是"谁的回调长"，而是总占空比。

拍没有丢。 上面每一对（幕 × 定时器）都满足"欠账 < 周期 **或** 补拍数 > 0"：欠下的要么只是还没到，要么已经当场补开了。原因是自动重装的锚点写的是 `xExpiredTime += pxTimer->xTimerPeriodInTicks`（`:709`），不是"现在 + 周期"，周期不漂，欠的一定要还。所以正文原来那句"数 fast cb 丢了几拍"问错了问题：该数的是欠账和连发。

同一刻到期谁先跑？ 由 `list.c:192` 那个插入条件决定：`pxIterator->pxNext->xItemValue <= xValueOfInsertion`，是 `<=` 不是 `<`，等于时继续往后走，于是**先入表者先服务**。模拟里 t=600 那一拍 fast 与 slow 撞车，先动的是 slow（它在 480 那一拍就被插到 600 这个位置，比 fast 早），t=1200 再撞一次顺序一样。⚠️ 这里的"入表"是**被插进链表的那一刻**，不是 `xTimerCreate` 的先后：自动重装一跑，两者的插入时刻就交错分了，创建顺序只在第一轮管用。

六、代码分析：场景 2/3/4 日志对照

- **场景 2（通知）**：生产端每 50ms give 一次，消费端 `ulTaskNotifyTake` 取走。把生产 delay 改小，`took` 的值就会变成 >1，计数语义把"欠的账"攒着；如果改成覆盖语义，多出来的就丢了。
- **场景 3（事件组）**：只发 bit0 时只有 `ANY woke`；bit0+bit1 都置上后 `ALL woke` 才出现，而且**两个等待者同时醒**：汇总清位的直接证据。
- **场景 4（定时器）**：`slow cb` 每次磨蹭 ~50ms，你会看到紧跟其后的 `fast cb` 迟到恰好 50ms，守护任务串行执行回调的直接证据。这一条正是上面 B 幕那档（Σ=0.416，占空比不满），所以迟到有界、等于耗时本身；把 `slow cb` 的耗时改到 200ms 就等于跨进 C 幕，`fast cb` 的迟到会超过 200ms 并开始连环补拍。

七、选型决策树：通知 / 队列 / 事件组

拿到一个"任务间通信"需求，按这个顺序问：

```
1. 是"一对一"还是"多对多"？
   ├─ 一对一 → 继续问 2
   └─ 多对多 → 队列（带数据）/ 事件组（只带信号）

2. 需要传数据还是只传信号？
   ├─ 只传信号 → 继续问 3
   └─ 要传数据 → 队列（数据 > 32 位）/ 通知（数据 ≤ 32 位，覆盖语义）

3. 需要攒"欠账"还是只关心最新值？
   ├─ 攒欠账 → xTaskNotifyGive（计数语义）
   └─ 只关心最新 → xTaskNotify（覆盖语义）

4. 需要等"几个条件都齐"？
   └─ 是 → 事件组（ANY/ALL）
```

经验法则：中断 → 单任务同步，默认用通知；任务 → 任务传数据，默认用队列；多条件汇合（如"初始化完成 + 传感器就绪 + 网络连上"），用事件组。

八、软件定时器 vs 任务内 vTaskDelay

两者都能做周期任务，选型看三点：

维度	软件定时器	任务 + vTaskDelay
RAM 开销	**固定 1312 B**（一个定时器都没建就要付）+ 每个定时器 56 B	每个任务 616 B（栈 128 字 + TCB）
回调里能阻塞吗	**不能**（堵守护任务）	能（只堵自己）
时间精度	受守护任务队列负载影响	只受调度器影响
适合	周期杂务（LED 闪、看门狗喂）	需要独立逻辑、可能阻塞的任务

反模式：在软件定时器回调里做 I2C 读取，I2C 超时阻塞会拖垮所有定时器。正确做法是回调里发通知，由专门的任务做 I2C。

那句"RAM 开销：零"，两个方向都不成立

网上讲软定时器几乎都有一行"RAM 开销：零（共用守护任务）"。这台机器上量的结果（`sh probe.sh` 第 6 步，heap_4 + CM4F + V11.1.0，配置就是本工程的 `code/rtos/01-freertos-lab`）：

```
守护任务 TCB    : pvPortMalloc(88)              → 实付 96 B
守护任务栈      : pvPortMallocStack(256×4=1024) → 实付 1032 B
命令队列        : pvPortMalloc(80+8×12=176) → 实付 184 B
── 固定那笔合计：1312 B（一个定时器都没建就要付）
── 每建一个定时器：56 B ｜ 每开一个最小任务：616 B（栈 128 字）
── 固定那笔占 configTOTAL_HEAP_SIZE 的千分比：80‰，也就是 8.0%
```

三个数都是量出来的，不是抄的：`sizeof(Timer_t)=44`、`sizeof(Queue_t)=80`、`sizeof(TCB_t)=88`、`sizeof(DaemonTaskMessage_t)=12` 由 `nm --print-size` 从目标文件里读，队列那笔的长度 8、消息 12、栈深 256 字由编译器报出来；`实付` 那一步是 heap_4 每笔 malloc 收的抽头，`heap_4.c:187` 加 `xHeapStructSize`、`:198` 补对齐，`(8 + 8 - 1) & ~(8 - 1) = 8 B`。

方向一：`configUSE_TIMERS=1` 一开，1312 B 就没了，跟定时器个数无关。

方向二：还有 **60 B 落在 `.bss`**，`configTOTAL_HEAP_SIZE` 调多大都管不到它，因为它根本不在这笔账的池子里。这 60 B 是从源码数出项数再乘实测宽度预测的（两条到期表 `List_t×2` + 4 个指针 + `xLastTime`），`= 20×2 + 4×4 + 4×1 = 60 B`，跟 `nm` 打出的实际值对上了才对上。

那"共用守护任务所以便宜"这句到底对多少？拿比价基准看：**一个最小任务 616 B，等于 11 个定时器的名额**；反过来，**固定那笔 = 2.12 个最小任务的名额**（`1312 ÷ 616` 向下取整，精确值 2.129），为了能用定时器，你先付了两个任务的价钱。所以省的是每个定时器自己那 616 B 的栈与 TCB，1312 B 那笔一分不省，只用一两个定时器时最不划算。

换档配置账怎么动（同一个公式换实测输入）：

档位	固定那笔	每定时器	相对 lab
lab 默认	1312 B	56 B	+0
`configUSE_TRACE_FACILITY=0`	1296 B	48 B	−16
`INCLUDE_xTimerPendFunctionCall=1`	1344 B	56 B	+32
`configTIMER_QUEUE_LENGTH` 8→16	1408 B	56 B	+96

`+32` 恰好是"队列长度 × 消息增重"（8 × 4），`+96` 恰好是"长度增量 × 消息大小"（8 × 12），涨幅能被公式复现，说明这笔账里没有魔法数。整池 16 KiB 若只塞定时器，`(16384 − 1312) ÷ 56 = 269` 个：那是上界，不是预算，真实工程里这笔钱还要给任务和栈。

九、GDB 调试通知与事件组

通知和事件组的状态都在 TCB 里，GDB 可以直接看：

```
# 查看任务的通知值（计数语义）
(gdb) p pxCurrentTCB->ulNotifiedValue[0]
$1 = 3    # 欠了 3 次没取

# 查看通知状态
(gdb) p pxCurrentTCB->ucNotifyState[0]
$2 = 1 '\001'    # 1 = 已通知待取

# 查看事件组位
(gdb) p xEventGroup->uxEventBits
$3 = 5    # bit0 + bit2 置位

# 在通知到达时打断点
(gdb) break xTaskNotifyGive
(gdb) commands 1
> p xTaskToNotify->pcTaskName
> continue
> end
```

调试纪律：通知是"瞬时状态"，GDB 暂停时看到的值可能已经被消费。用 `watch` 命令抓变化比 `break` 更精准：

```
# 通知值变化时自动暂停
(gdb) watch pxCurrentTCB->ulNotifiedValue[0]
```

附录：工程完整源码

<<< ../../../code/rtos/01-freertos-lab/main.c

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **通知写 TCB 一步到位，事件组是位公告栏，软定时器住守护任务——回调里只办事，不等待；清位是汇总清，不是先醒者独占。**

延伸：硬件 TIM（S6）管外设时基，软件定时器管周期任务，分工不同；任务通知可替代队列用于 1:1 数据传递（F4），更轻量。

实物实验

- **不用板子也能把本章每一个数字打出来**：`cd code/rtos/06-timers && sh probe.sh`。七步一条命令跑完，用 `nm --print-size` 量出 `Timer_t 44 / ListItem_t 20 / DaemonTaskMessage_t 12 / Queue_t 80 / TCB_t 88`，再拿机身里那句 `movs r0, #44` 和静态版 `StaticTimer_t` 作为第二、三条独立路子对上同一个 44；从 `xQueueGenericCreate` 调用点读队列的（`8, 12`）（不写进脚本，写了就不叫实测）；把 heap_4 的抽头算进去，逐笔打出 96 / 1032 / 184 → 固定 **1312 B**、每定时器 **56 B**、每最小任务 **616 B**；按源码数出的项数预测 `.bss` 那 **60 B** 再跟 nm 对上；最后跑一遍把七条分派规则抄成的宿主机模拟器，打出五幕的迟到/补拍/连发/欠账（§五）。每条规则都由 `at_line` 钉住上游行号，内核一升级行号漂了这一步直接变红，而不是悄悄给一个错的数。需要交叉工具链与 V11.1.0 内核（CI 的 arm-host 作业两样都装）。
- 场景 2：把生产端 delay 从 50ms 改到 10ms，看 `took` 的值涨上去；
- 场景 3：给 ANY 等待者加 `xClearOnExit=pdFALSE`，再发一次 bit0，这次 ALL 也能等到；
- 场景 4：把 `slow cb` 的磨蹭时间从 50ms 改到 200ms，这一步正好跨过 Σ(耗时 ÷ 周期) = 1 那条线（50ms 时 0.416，200ms 时 1.666）。**别数"丢了几拍"**：模拟实测拍没丢，`owed` 那栏欠着、`catchup` 连环补拍，`fast cb` 的最大迟到从恰好 50 涨到 520。无板复现同一张表：`cd code/rtos/06-timers && sh probe.sh` 第 7 步。

常见坑

- **通知当通用队列用**：槽位数 = `configTASK_NOTIFICATION_ARRAY_ENTRIES`（本工程=1），多对多还是队列；
- **记错函数名**：是 `ulTaskNotifyTake`，不是 `xTaskNotifyTake`；
- **覆盖丢事件**：高频通知未消费就被覆盖，要攒"欠账"用 `xTaskNotifyGive`（eIncrement）；
- **事件组 xClearOnExit 误用**：想让"下一批"也看到这些位就别开自动清；
- **软定时器回调里 vTaskDelay**：守护任务一堵，所有定时器集体迟到。
- **把"晚一拍"当成结论**：它是现象，不是判据。判据是 Σ(回调耗时 ÷ 周期) 有没有过 1：没过，零耗时定时器的最大迟到恰好等于最大那次耗时（有界）；过了，迟到超出耗时本身并连环补拍（发散）。看日志数"晚了几拍"永远看不出这条线，模拟器一次就量到。
- **把定时器当零成本**：`configUSE_TIMERS=1` 一开就固定付 1312 B，另有 60 B 在 `.bss` 里不受 `configTOTAL_HEAP_SIZE` 管（§八）。只用一两个定时器时，这笔钱比开任务还贵：固定那笔 = 2.12 个最小任务。
- **"入表顺序"当成 `xTimerCreate` 的先后**：同一刻到期谁先跑，由链表插入时 `list.c:192` 那个 `<=` 决定，是"插入那一刻的先后"，不是创建顺序，也不是 ID 大小。

短自测

1. 任务通知的"计数语义"和"覆盖语义"分别对应哪个 API？

> **▼ 参考答案**
> 计数语义是 xTaskNotifyGive（eIncrement，通知值 +1）；覆盖语义是 xTaskNotify 的 eSetValueWithOverwrite（直接盖掉旧值）。欠账要攒就用前者，只关心最新值就用后者。

2. 事件组里两个任务都在等同一组位，一个等 ANY、一个等 ALL，发 SetBits 把位全置上后，谁会醒？位什么时候清？

> **▼ 参考答案**
> 两个都醒。SetBits 把整个等待列表走完、所有满足条件的都唤醒，然后按 uxBitsToClear 汇总清一次（event_groups.c:547 起）。不存在"先醒者独占"。

3. 软件定时器的回调里为什么不能 vTaskDelay？

> **▼ 参考答案**
> 回调跑在守护任务里，守护任务是所有定时器共用的执行线程。回调阻塞 = 守护任务阻塞 = 所有定时器集体停摆。回调必须快进快出。

4. 为什么"通知比队列快 45%"不能当无条件结论引用？

> **▼ 参考答案**
> 那是官方文档在某个版本、某个配置下的实测数字。通知快的根源是无队列对象、无拷贝、无额外链表项；具体快多少取决于你的队列深度、是否从中断上下文、调度器当时的状态。引用时必须带版本与配置条件。

5. `configUSE_TIMERS=1`、一个定时器都没建，堆已经被吃掉多少？另有几字节不在堆里？

> **▼ 参考答案**
> 固定 1312 B：守护任务 TCB 实付 96 + 守护任务栈（256 字 × 4）实付 1032 = 1128，命令队列 pvPortMalloc(80 + 8×12 = 176) 实付 184。另有 60 B 落在 .bss（两条到期表 List_t×2 + 4 个指针 + xLastTime），configTOTAL_HEAP_SIZE 调多大都管不到它。每建一个定时器再收 56 B，一个最小任务是 616 B。

6. 一个 100ms 的零耗时定时器，被一个耗时 200ms 的 120ms 定时器压着，它的最大迟到是 200ms 吗？

> **▼ 参考答案**
> 不是，实测 520 拍（毫秒）。Σ(耗时÷周期) = 200/120 = 1.666 ≥ 1，迟到跳出耗时本身并发散，且出现连环补拍（catchup 6 次、连发 6）。把耗时降到 50ms（Σ=0.416 < 1），最大迟到才恰好等于那次耗时 50；耗时 200ms 但周期 300ms（Σ=0.666 < 1），最大迟到又回到恰好 200，有界条件是那个和小于 1，不是单次耗时小于某个数。

对照表：本章概念 → 仓库与上游落点

概念	落点
give/take/notify API	上游 V11.1.0 `task.h:2984/3177/2611` → `tasks.c:7832/7717`
事件组汇总清位	上游 V11.1.0 `event_groups.c:547`（xEventGroupSetBits）
守护任务	上游 V11.1.0 `timers.c`（prvTimerTask）
定时器 RAM 账 + 分派规则实测	code/rtos/06-timers `sh probe.sh`（第 6 步总账、第 7 步五幕模拟，无需开发板）
场景实验	code/rtos/01-freertos-lab 场景 2/3/4
动画	task-notification.svg、event-group-wait.svg、software-timer-service.svg

你做到了

- 通信工具箱从"队列打天下"升级为按场景选型；
- 事件组的"汇总清位"不再是玄学；
- 软定时器接管周期性杂务，任务数开始下降。

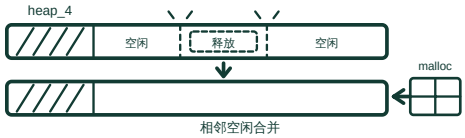
首付交了 1312 B，每建一个定时器再收 56 B：守护任务串行跑所有回调，回调小而快、禁止阻塞，是这条通道的纪律。

✅ 下一站：F7 内存管理：heap_1~heap_5 五种人生，栈水位与堆碎片实测。

通往单片机之路第 43 章

F7 内存管理：heap_1 到 heap_5 的人生选择

难度 ★★★☆☆ · 约 50 分钟



本章罗盘

问题

总空闲明明还有 4KB，为什么连 2KB 都分不出来，heap_4 什么时候合并、什么时候永远留洞？

交付

场景 5 证据链日志：boot free → after ABC → free B 后 big ok?=0（总空闲够、最大连续块不够）→ free A 相邻合并 → big ok?=1；外加 heap_1/2/4/5 指标 API 逐文件可用性表与 configTOTAL_HEAP_SIZE 预算公式（栈＋TCB＋队列，余量 30%）。

边界

指标可用性按 V11.1.0 逐文件核对（xPortGetMinimumEverFreeHeapSize 只有 heap_4/5 有），「历史最低水位」不是碎片率；heap_5 多区只讲配置与 CCM 禁 DMA 原则，硬件依据在 S8、TLSF 对照路线在 R3；动态任务栈来自堆的结论不适用于静态创建与 MSP 主栈。

依据锚点 `f1 · d4 · e5 · d3 · mcu-code · mcu-anim`

> 🎯 FreeRTOS 给了你五份"内存管理员"简历：从"只进不出的守财奴"（heap_1）到"会拼碎片的清洁工"（heap_4）再到"跨区调度的大管家"（heap_5）。选谁，取决于你的产品敢不敢承受"分配失败"与"碎片"这两件事。

本章精髓

1. **heap_1→heap_5 的能力阶梯**：只分配不释放（确定性无敌）→ 可释放不合并（碎片制造机）→ 包 libc 的 malloc（线程安全包装）→ 释放+相邻合并（碎片可控，嵌入式默认答案）→ 多区域堆（SRAM+CCM 分而治之）。
2. **碎片是时间的敌人**：长期运行产品里，"现在够用"不等于"一年后还够"。监控有两个指标要分清：**剩余总量**（`xPortGetFreeHeapSize`）和**历史最小剩余**（`xPortGetMinimumEverFreeHeapSize`），后者只在 heap_4/heap_5 上有，而且它是"水位最低点"，不是"碎片率"。
3. **动态任务的栈就是从堆里来的**：`xTaskCreate` 内部 `pvPortMalloc` 栈和 TCB（F1 创建六步的第 1、2 步）。"栈不在堆里"的说法只适用于**静态分配**的任务栈和 MSP 主栈，别把两种栈混为一谈。

- **能理解**：为什么"总空闲够"不等于"能分出大块"；为什么历史最小剩余量不是碎片率；为什么 heap_2 没有那个指标 API。
- **能用**：给"参数表/任务创建/图形缓冲"三个场景各选一版 heap；用场景 5 亲手看到碎片与合并。

先修

- F1 任务栈、C1 内存模型、B3 链接脚本。

先跑起来（10 分钟 quick win）

`code/rtos/01-freertos-lab` 场景 5：

```
make DEMO_SCENE=5 flash
```

串口日志就是证据链：`boot free` → `after ABC` → `after free B`（总空闲涨了，但 `big ok?=0`，最大连续块不够）→ `after free A`（A 与 B 相邻，heap_4 合并）→ `big ok?2=1`。

先押个接头：heap_4 凭什么知道 A、B 该不该焊成一块？释放的块按什么规矩找邻居？押完看第二节的合并算法。

动画：相邻块合并，不相邻块留洞

heap_4 的空闲链表**按地址排序**，释放时插回去：前后相邻就合并成一块，不相邻就留洞。盯住"总空闲"和"最大连续块"两个数字，它们经常不一样大。

heap_4 合并动画

版本与配置前提

- 内核：上游 **FreeRTOS-Kernel V11.1.0**，内存分配 `portable/MemMang/heap_4.c`。
- 本工程 `configTOTAL_HEAP_SIZE = 16KB`，`configSUPPORT_DYNAMIC_ALLOCATION=1`。
- 指标 API 按实际文件核对（见对照表），不凭印象书写。

小节结构

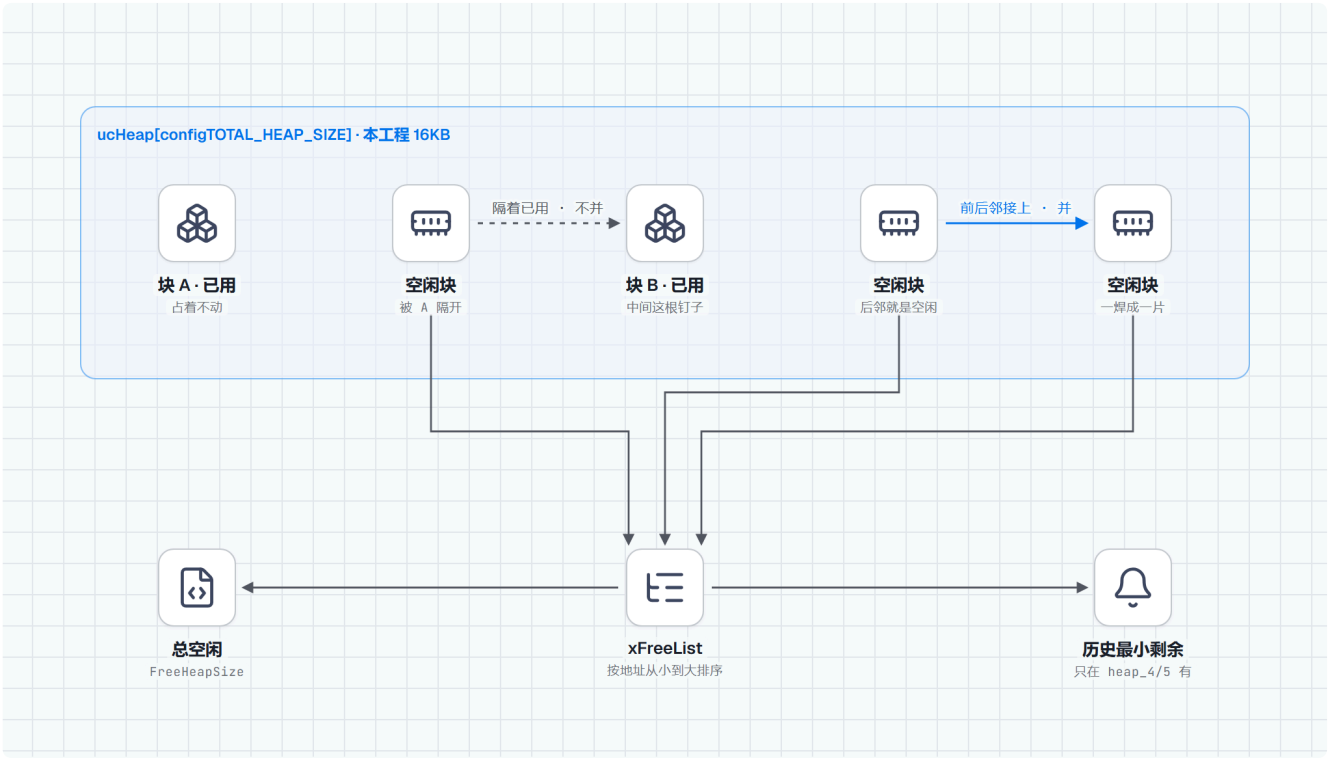
小节	内容	四件套
一、五版简历	能力/确定性/碎片三轴对照表	配置
二、heap_4 合并算法	空闲链表按地址排序 + 相邻合并源码逻辑	库解析
三、碎片实验	场景 5 的分配/释放/合并证据链	代码分析
四、栈溢出检测	canary 原理；hook 函数与 S16 联动取证	代码分析
五、heap_5 多区	SRAM+CCM 双区配置；哪类数据放哪区	配置
六、监控与兜底	两个指标的区别；分配失败时的产品级处理	代码分析

一、五版简历：能力阶梯

版本	分配	释放	合并	确定性	碎片	适合
heap_1	✅	❌	—	无敌	不产生（也没法用）	一次性分配的初始化
heap_2	✅	✅	❌	好	制造机	短期演示，别上长期产品
heap_3	包 libc malloc	✅	看 libc	看 libc	看 libc	快速原型
heap_4	✅	✅	✅ 相邻合并	好	可控	**嵌入式默认答案**
heap_5	✅	✅	✅	好	可控	多区域（SRAM+CCM）

二、heap_4 合并算法：按地址排序 + 相邻就拼

地契先看图：链表按地址排，隔着已用块的两洞拼不上，前后邻上的才焊得成一块。



heap_4 空闲链表：ucHeap 里已用块与空闲块交替，xFreeList 按地址排序，相邻空闲块合并，两个指标都从这条链表读

heap_4 的空闲链表**按地址从小到大排序**（不是按大小）。释放时 `prvInsertBlockIntoFreeList`（V11.1.0 `heap_4.c:492`）做三件事：

1. 找到插入位置（按地址）；
2. **前邻块**的尾地址若正好接上我，就并进前邻块；
3. **后邻块**的首地址若正好接上我，就把它也并进来。

推论：**只有相邻的空闲块才会合并**。交替分配释放留下的"洞"，永远不会自动拼成大块，这就是碎片的物理来源。

heap_4 源码走读：prvInsertBlockIntoFreeList

释放时的合并逻辑（`heap_4.c:492`）用三行 C 完成：

```
/* 空闲链表按地址排序。pxBlockToInsert 是刚释放的块。*/

/* 1. 找到插入位置：遍历链表，找到第一个地址比我大的块 */
pxIterator = (BlockLink_t*)&xStart;
while (pxIterator->pxNextFreeBlock < pxBlockToInsert) {
    pxIterator = pxIterator->pxNextFreeBlock;
}

/* 2. 后邻块：我后面那块如果是空闲的，且紧挨着我 → 合并 */
pxNextBlock = (uint8_t*)pxBlockToInsert + pxBlockToInsert->xBlockSize;
if ((uint8_t*)pxIterator->pxNextFreeBlock == pxNextBlock) {
    /* 把后邻块从链表摘除，大小加到我头上 */
    pxBlockToInsert->xBlockSize += pxIterator->pxNextFreeBlock->xBlockSize;
    pxIterator->pxNextFreeBlock = pxIterator->pxNextFreeBlock->pxNextFreeBlock;
}

/* 3. 前邻块：我前面那块如果是空闲的，且紧挨着我 → 合并 */
pxPrevBlock = (uint8_t*)pxIterator + pxIterator->xBlockSize;
if ((uint8_t*)pxPrevBlock == (uint8_t*)pxBlockToInsert) {
    /* 把我加到前邻块头上 */
    pxIterator->xBlockSize += pxBlockToInsert->xBlockSize;
} else {
    /* 不邻接，正常插入链表 */
    pxBlockToInsert->pxNextFreeBlock = pxIterator->pxNextFreeBlock;
    pxIterator->pxNextFreeBlock = pxBlockToInsert;
}
```

关键设计： - **边界标记法**：每个块的头部存 `xBlockSize`（含头部自身大小），通过"前块地址 + 前块大小"就能找到当前块的起始位置，不需要双向链表。 - **首次适配**：分配时从链表头开始找，第一个够大的就用。简单、快，但容易产生碎片。 - **O(n) 遍历**：释放和分配都要遍历链表。对于嵌入式的小堆（几 KB 到几十 KB），这比复杂数据结构的开销更划算。

静态分配：xTaskCreateStatic —— 零动态分配的安全答案

安全关键系统（汽车、医疗、航空）不允许运行时分配失败。FreeRTOS 提供**静态创建 API**：调用方自己提供 TCB 和栈的存储，内核零 malloc。

```
/* 静态分配：调用方提供所有存储 */
static StaticTask_t taskTCB;                    // TCB 结构体
static StackType_t taskStack[256];              // 栈数组（256 × 4 = 1024 字节）

TaskHandle_t handle = xTaskCreateStatic(
    sensor_task,        // 函数
    "sensor",           // 名字
    256,                // 栈深（字数，不是字节）
    NULL,               // 参数
    2,                  // 优先级
    taskStack,          // ← 栈数组
    &taskTCB            // ← TCB 存储
);
configASSERT(handle != NULL);                   // 永远成功，不会返回 NULL
```

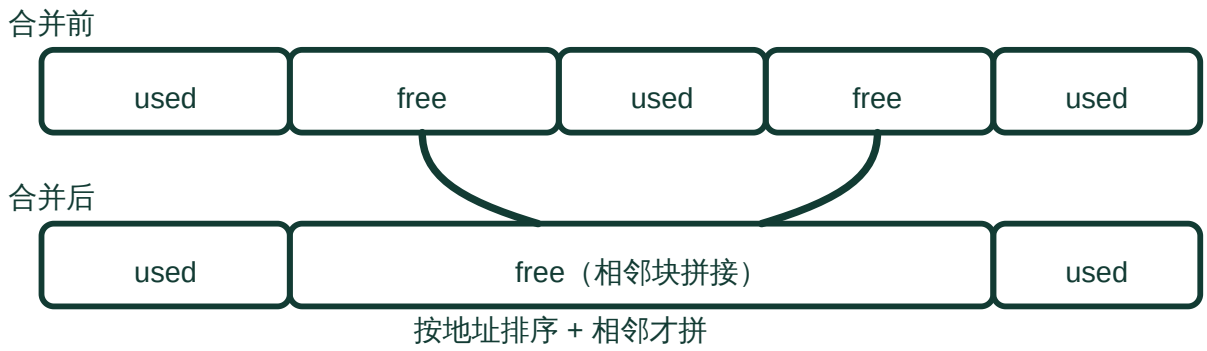
与动态创建的对比：

	xTaskCreate（动态）	**xTaskCreateStatic（静态）**
TCB 来源	`pvPortMalloc` 从堆分配	调用方提供 `StaticTask_t`
栈来源	`pvPortMalloc` 从堆分配	调用方提供 `StackType_t[]`
分配失败	可能（堆不够）	**不可能**（编译期确定）
内存释放	`vTaskDelete` 归还堆	不释放（静态生命周期）
配置宏	`configSUPPORT_DYNAMIC_ALLOCATION=1`	`configSUPPORT_STATIC_ALLOCATION=1`
适合	原型开发、非关键任务	安全关键、长期运行产品

静态分配的代价： - 每个任务的 TCB + 栈在编译期就占好 RAM，不能运行时增减任务。 - 需要实现 `vApplicationGetIdleTaskMemory` 和 `vApplicationGetTimerTaskMemory`，为内核的 Idle 和 Timer 任务提供静态存储。

```
/* 必须实现：为 Idle 任务提供静态存储 */
void vApplicationGetIdleTaskMemory(StaticTask_t **ppxIdleTaskTCB,
                                   StackType_t **ppxIdleTaskStack)
{
    static StaticTask_t idleTCB;
    static StackType_t idleStack[configMINIMAL_STACK_SIZE];
    *ppxIdleTaskTCB = &idleTCB;
    *ppxIdleTaskStack = idleStack;
}
```

选型建议：长期产品优先静态分配（零失败风险）。如果必须动态创建任务（如运行时加载插件），用 heap_4 + 严格的 `configTOTAL_HEAP_SIZE` 计算 + `xPortGetMinimumEverFreeHeapSize` 监控。



heap_4 空闲链表：按地址排序，释放时只有相邻空闲块才合并；不相邻的洞保留

三、碎片实验：总空闲 ≠ 最大连续块

场景 5 的完整证据链（串口日志）：

步骤	操作	观察
1	启动	`boot free` ≈ 16KB（减去 TCB/栈等内核开销）
2	连续分配 A/B/C 各 1KB	`after ABC` 掉了 ~3KB
3	释放 B（中间那块）	`after free B` 涨了 ~1KB，但**总空闲够 2KB，最大连续块不够**
4	要一块 2KB	`big ok?=0` —— 失败
5	释放 A（与 B 相邻）	heap_4 合并 A+B
6	再要 2KB	`big ok?2=1` —— 合并后成了

这就是"总空闲 ≠ 最大可分配块"的实验证据。原文档里"动态任务栈来自堆"与"不在堆里"自相矛盾的表述，根子也在这里：动态任务的栈确实从 heap_4 的池子里 `pvPortMalloc` 出来（F1），碎片化直接影响你能不能建新任务。

四、栈溢出检测：canary 是更早的哨兵

`configCHECK_FOR_STACK_OVERFLOW=2` 开满后，每次上下文切换时内核检查栈底附近的 0xA5 水位线是否被踩。踩了就调 `vApplicationStackOverflowHook(任务句柄, 任务名)`，本工程打印任务名后停住。

为什么需要它：**爆栈的第一现场往往不是 HardFault，而是"悄悄踩了邻居的数据"**：任务函数读到一个莫名其妙的值、通信偶尔丢包、日志里出现乱码。等 HardFault 再查，现场已经被踩得面目全非。canary 在踩的那一瞬间就抓人。

五、heap_5 多区：SRAM 和 CCM 分而治之

heap_5 允许把堆拆成多个区域（HeapRegion 表）：比如 SRAM 放常规分配，CCM（0x10000000 的 64KB，CPU 直连）放实时性敏感的数据。前提是**这些区域 DMA 够不着**（回 S8 的 CCM 禁区），所以 CCM 里绝对不能放 DMA 缓冲区。

六、监控与兜底：指标 API 不是每个版本都有

逐文件核对后的可用性（V11.1.0）：

指标	heap_1	heap_2	heap_4	heap_5
`xPortGetFreeHeapSize`	✅	✅ `:334`	✅	✅ `:435`
`xPortGetMinimumEverFreeHeapSize`	❌	❌	✅ `:413`	✅ `:441`

两个指标的区别：

- **FreeHeapSize**：现在还剩多少（瞬时值）；
- **MinimumEverFreeHeapSize**：历史最低水位（峰值压力），它回答的是"最坏的时候有多紧张"，**不是碎片率**。碎片率要自己用"最大连续块/总空闲"估。

产品级兜底：分配失败不许静默。要么进安全态，要么记日志告警，要么重启看门狗。静默往下跑，后面全是玄学。

七、堆大小怎么算：从拍脑袋到工程预算

`configTOTAL_HEAP_SIZE` 的预算公式：

```
总堆 = Σ(任务栈 × 字数 × 4) + Σ(TCB × ~400B) + Σ(队列深度 × 项大小) + 信号量/事件组 + 安全余量(30%)
```

实操步骤：

1. **列任务清单**：每个任务的栈深（用 `uxTaskGetStackHighWaterMark` 实测，不是猜）；
2. **列通信对象**：队列深度 × 项大小、信号量数量、事件组数量；
3. **加安全余量**：总需求 × 1.3，留给付给碎片和未来扩展；
4. **检查 .bss 冲突**：堆和 .bss 共享 SRAM，堆太大会挤占全局变量空间，看链接脚本的 MEMORY 定义。

反模式：`configTOTAL_HEAP_SIZE = 64KB`，理由是"反正 Flash 够大"。SRAM 只有 128KB（F407），堆占一半，.bss 和栈就没地方了。

八、长期运行产品的堆监控策略

产品上线后，堆的健康度要持续监控，不是开发时看一眼就完事：

周期性巡检（每 10 分钟或每小时）：

```
void heap_health_check(void) {
    size_t free_now = xPortGetFreeHeapSize();
    size_t free_min = xPortGetMinimumEverFreeHeapSize();

    // 趋势告警：当前值逼近历史最低
    if (free_now < free_min * 1.2) {
        LOG_WARN("heap tension: %u free, %u min-ever", free_now, free_min);
    }

    // 绝对告警：低于安全阈值
    if (free_now < 1024) {  // 1KB 安全线，按产品调整
        LOG_ERROR("heap critical: %u bytes left", free_now);
    }
}
```

碎片率估算（heap_4/heap_5 才有意义）：

```
// 分配一块测试内存，看实际能拿到多大
void* test = pvPortMalloc(4096);
if (!test) {
    LOG_WARN("fragmentation: can't alloc 4KB, but free=%u", xPortGetFreeHeapSize());
}
vPortFree(test);
```

数据记录：把 `free_now` 和 `free_min` 写进 NVS 或日志，跑一周看曲线；如果 `free_now` 持续下降不回升，说明有内存泄漏。

九、GDB 调试堆与栈溢出

堆和栈的问题在 GDB 里能直接看到证据：

```
# 查看堆的当前状态
(gdb) p xPortGetFreeHeapSize()
$1 = 12480    # 还剩 12KB

(gdb) p xPortGetMinimumEverFreeHeapSize()
$2 = 8192     # 历史最低 8KB

# 查看堆池起始地址（heap_4）
(gdb) p ucHeap
$3 = (uint8_t[16384]) @ 0x20004000

# 查看任务的栈底（溢出检查用）
(gdb) p pxCurrentTCB->pxStack
$4 = (StackType_t *) 0x20001800    # 栈底

# 看栈里有没有被踩（正常应该填 0xA5）
(gdb) x/32xb 0x20001800
0x20001800: 0xa5 0xa5 0xa5 0xa5 0xa5 0xa5 0xa5 0xa5
0x20001808: 0xa5 0xa5 0xa5 0xa5 0x48 0x65 0x6c 0x6f  ← 被踩了！
```

栈溢出取证：如果 `vApplicationStackOverflowHook` 被触发，GDB 里看调用栈：

```
(gdb) break vApplicationStackOverflowHook
(gdb) continue
# 触发后
(gdb) bt
#0  vApplicationStackOverflowHook
#1  vTaskSwitchContext
#2  PendSV_Handler    ← 上下文切换时发现的
(gdb) p pxCurrentTCB->pcTaskName
$5 = "sensor_task\000..."    ← 肇事者
```

调试纪律：栈溢出第一现场往往不是 HardFault，而是"数据莫名其妙错了"。看到灵异现象，先用 GDB 查栈底有没有被踩。

附录：工程完整源码

<<< ../../../code/rtos/01-freertos-lab/main.c

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **heap_1 只进不出，heap_4 会合并，heap_5 跨区管；碎片看曲线，爆栈靠哨兵，剩多少问 FreeHeapSize，最坏多紧张问 MinimumEver。**

延伸：堆管理与链接脚本的堆区预留（B3）相关；嵌入式慎用 malloc 的原因见 C0；内存池的确定性替代方案见 RT-Thread（R3）。

实物实验

- 场景 5 完整跑一遍，把日志抄进实验记录；
- 对比实验：把 Makefile 里 `heap_4.c` 换成 `heap_2.c` 再跑，`xPortGetMinimumEverFreeHeapSize` 直接编译不过（heap_2 没有这个 API），这就是"指标 API 要逐文件核对"的现场教学；
- 爆栈捕获：把某任务的栈深从 256 改到 64，看 OverflowHook 抓到谁。

常见坑

- **heap_2 用于长期运行产品**：碎片无解，跑着跑着分配失败，长期产品直接 heap_4；
- **configTOTAL_HEAP_SIZE 拍脑袋**：按"任务栈总和 + 队列 + 余量 30%"算，不是越大越好（挤占 .bss）；
- **把历史最小剩余当碎片率**：它是"最坏水位"，不是"碎片程度"，两者都要看；
- **栈溢出只信 HardFault**：很多时候爆栈先踩邻居数据，静默错乱，canary 是更早的哨兵；
- **中断里 pvPortMalloc**：堆分配非原子且可能耗时，ISR 禁地，预先静态分配；
- **CCM 里放 DMA 缓冲区**：DMA 够不着 CCM，数据静悄悄丢失。

短自测

1. heap_4 释放一块内存时，什么情况下会合并？

▼ **参考答案**
只有当被释放的块与空闲链表中的块**地址相邻**时才合并（前邻接尾或后邻接头）。不相邻就留洞，这就是碎片的物理来源。落点：heap_4.c 的 prvInsertBlockIntoFreeList。

2. "总空闲还有 4KB"为什么不能保证能分出 2KB？

▼ **参考答案**
空闲可能碎成多块不连续的小块。分配器要的是**最大连续块**，不是总空闲。场景 5 的第 3、4 步就是证据。

3. `xPortGetMinimumEverFreeHeapSize` 在哪些 heap 版本上有？它回答的是什么问题？

▼ **参考答案**
只有 heap_4 和 heap_5 有（V11.1.0 逐文件核对）。它回答"历史最低水位是多少"，峰值压力有多大，不是碎片率。

4. 动态创建的任务，它的栈存在哪里？静态创建的呢？

▼ **参考答案**
动态创建（xTaskCreate）的栈由内核 pvPortMalloc 从 heap_4 的池子里分；静态创建（xTaskCreateStatic）的栈由调用方提供的静态数组给。MSP 主栈（中断用）在链接脚本里，与堆无关。"栈不在堆里"只对后两种成立。

🎯 F7 内存管理自测

1. **heap_4 释放一块内存时，什么情况下会合并？**
   - A. 只要空闲链表有空位就合并
   - B. 只有当被释放的块与空闲链表中的块地址相邻时才合并
   - C. 永远不合并
   - D. 每次释放都触发全表压缩

2. **'总空闲还有 4KB'为什么不能保证能分出 2KB？**
   - A. 分配器有 50% 的保留开销
   - B. 空闲可能碎成多块不连续的小块，分配器要的是最大连续块
   - C. RTOS 内核会占用一半堆
   - D. 这个说法是错的，4KB 一定能分出 2KB

3. **`xPortGetMinimumEverFreeHeapSize` 在哪些 heap 版本上有？**
   - A. 所有五个版本都有
   - B. 只有 heap_1 和 heap_2
   - C. 只有 heap_4 和 heap_5
   - D. 只有 heap_3

4. **动态创建的任务，它的栈存在哪里？**
   - A. 由调用方提供的静态数组
   - B. 由内核 pvPortMalloc 从 heap 池子里分
   - C. 在 MSP 主栈里
   - D. 在 Flash 里

先选完再翻：答案与逐项解析在附录 C 的本组；在线版可点击即时判分。

对照表：本章概念 → 仓库与上游落点

概念	落点
heap_4 相邻合并	上游 V11.1.0 `portable/MemMang/heap_4.c:492`（prvInsertBlockIntoFreeList）
指标 API 可用性	同上逐文件核对：heap_1:149、heap_2:334、heap_4:413、heap_5:435/441
动态任务栈来自堆	上游 V11.1.0 `tasks.c:1718`（xTaskCreate → pvPortMalloc）
碎片实验	code/rtos/01-freertos-lab 场景 5
动画	heap4-coalesce.svg

延伸阅读

五份简历背后的分配器理论：

- **[D4]** Wilson et al. 1995 — 分配器综述：首次适配/最佳适配/隔离适配各自的碎片代价，heap_4 为什么选"首次适配 + 相邻合并"。
- **[E5]** Knuth TAOCP Vol.1 §2.5 — 边界标记法与伙伴系统，heap_4 合并相邻空闲块的祖师爷。
- **[D3]** Masmano et al. 2004 — TLSF：RT-Thread 走的另一条路（O(1)、碎片有界），与 heap_4 对照着看。

你做到了

- 内存从"够不够"的焦虑变成"可测可控"的工程；
- "总空闲 ≠ 最大连续块"亲手见过一次；
- FreeRTOS 内核篇收官，你已经读完一个真实 RTOS 的全部核心机制。

接头在空闲链表：释放的块按地址排进去，前后一探，门对门就焊成一整块，隔一个邻居就留洞。总空闲是账面，最大连续块才是能住人的房。

✅ 下一站：F8 移植到霸天虎：把前面所有机制在你的板上拼起来跑。

通往单片机之路第 44 章

F8 移植 FreeRTOS 到霸天虎：三异常接管全记录

难度 ★★★☆☆ · 约 45 分钟

本章罗盘

问题

裸机工程接上 FreeRTOS 到底要动几处，三个异常怎么改名接管，两份优先级宏各管谁？

交付

一份移植清单跑出可验产物：FreeRTOSConfig.h 三行 #define 映射、nm 出 SVC/PendSV/SysTick 三个 T 符号落在 port.o、size 三段 text 43156 / data 2168 / bss 19208，并算清 heap 池子与 _Min_Stack_Size 两套独立预算。

边界

所有数字是编译期/链接期实测，上板串口输出与板上现象标注「待回填」；本工程跑 HSI 16MHz 不启 PLL，拿 168MHz 套节拍计算会错 10 倍；结论限单核 ARM_CM4F 端口（-mfloat-abi=hard 与端口目录绑定），RISC-V/MPU/SMP 移植不能外推。

依据锚点 `f1 · mcu-code · mcu-anim · a3`

> 🎯 前面七章的机制，这一章全部落地：在我们的寄存器工程上，让 FreeRTOS 跑起来。移植的本质是**三个异常函数的改名接管**：SVC 启动第一任务、PendSV 切换、SysTick 报时，其余全是体力活。

本章精髓

1. 移植 = 拿对 port 层 + 改三处异常向量：`ARM_CM4F` 的 `port.c` / `portmacro.h` 拿来即用；FreeRTOS 用 `vPortSVCHandler` / `xPortPendSVHandler` / `xPortSysTickHandler` 三个名字，与我们启动文件的 SVC/PendSV/SysTick 向量对上即可（改向量表或 `#define` 映射两法）。
2. 中断优先级两组配置别踩：`configKERNEL_INTERRUPT_PRIORITY`（内核相关：PendSV/SysTick 必须最低）与 `configMAX_SYSCALL_INTERRUPT_PRIORITY`（能调 API 的中断上限），数值越大优先级越低，搞反全系统灵异。
3. heap 与栈是两套预算：heap 供任务栈/TCB/队列，与裸机 main 栈（链接脚本 `_Min_Stack_Size`）是两笔账，RTOS 化的 RAM 总账要重算。

- **能理解**：`nm` 出来的 `SVC_Handler` / `PendSV_Handler` / `SysTick_Handler` 三个符号，它们的**地址落在 `port.o` 里**：内核函数只是被 `#define` 换了名字。
- **能用**：拿到任意 Cortex-M4 工程，按"port 层 + 三个向量 + 两份优先级 + 一笔 heap"清单一遍过，`make` 出可跑镜像。

先修

- F2 上下文切换：PendSV 保存/恢复现场的机制，本章只看"怎么接上"。
- S4 中断与 NVIC：优先级分组、`AIRCR.PRIGROUP`、异常号，本章的优先级数值全靠它。
- C6 调用约定与栈帧：任务栈就是"一摞帧"，`push {r4, lr}` 的含义先熟。
- B5 map 与 size：本章的 RAM 账要用 `size` / `map` 读。

先跑起来（10 分钟 quick win）

```
cd code/rtos/01-freertos-lab
mingw32-make DEMO_SCENE=1 -B        # Linux/macOS 上用 make
```

先取内核：本工程默认从仓库的忽略缓存 `.trellis/ref/freertos-v11/kernel` 找源码（tag **V11.1.0**），获取方式见 `README.md`；也可用 `FREERTOS_KERNEL=<你的路径>` 覆盖。

实测输出（本机真跑）：

```
   text    data     bss     dec     hex filename
  43156    2168   19208   64532    fc14 build/scene-1/freertos-lab.elf
```

一条命令同时产出 `.elf` / `.bin` / `.hex`。**43KB 代码 + 2.1KB 已初始化数据 + 19KB 零初始化**：这就是一个最小 FreeRTOS 工程的入场券，第 5 节会拆开看这笔账。

make 通过先押个房东：PendSV_Handler、SysTick_Handler、SVC_Handler 三位接管者的符号落在哪个 .o？你的工程文件，还是内核的？押完用 nm 一查便知。

如果 `make` 报 `arm-none-eabi-gcc: command not found`，把工具链加进 PATH：

```
sh -c 'export PATH="/d/zhugu-home/tools/armgcc/xpack-arm-none-eabi-gcc-15.2.1-1.1/bin:$PATH"; cd code/rtos/01-freertos-la
```

小节结构

小节	内容	四件套
port 层怎么选	ARM_CM4F vs ARM_CM3、FPU 浮点寄存器的额外保存	配置
三异常接管	`vPortSVCHandler` / `xPortPendSVHandler` / `xPortSysTickHandler` 与向量表对齐的两法	代码分析
两份优先级	`configKERNEL_INTERRUPT_PRIORITY` 与 `configMAX_SYSCALL_INTERRUPT_PRIORITY` 的数值推演	配置
配置逐项	`FreeRTOSConfig.h` 真值逐条：时钟/节拍/优先级/栈/堆	配置
RAM 两套账	heap 池子 vs 裸机主栈；`size` 三段读法	代码分析
首任务跑起来	从 `make` 到上板的完整命令序列与验收点	代码分析

一、port 层怎么选：先对型号，再对 FPU

FreeRTOS 的移植代码全在 `portable/` 下按"编译器 × 架构"分目录。本工程用的是：

```
portable/GCC/ARM_CM4F/port.c
portable/GCC/ARM_CM4F/portmacro.h
```

为什么是 `ARM_CM4F` 而不是 `ARM_CM4`：最后的 `F` 表示 **FPU**（Floating Point Unit）。Cortex-M4F 带单精度硬件浮点，任务切换时必须**额外保存浮点寄存器**，所以端口代码不同。

这正好呼应 C6 硬浮点 ABI 讲的：`-mfpu=fpv4-sp-d16 -mfloat-abi=hard` 下 `double` 走 `d0/d1`。**端口选择错了（拿了不带 F 的 `ARM_CM4`），编译器照过、运行起来浮点寄存器被踩。**

本工程的 `Makefile` 里三项参数必须一致：

```
MCU      := -mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard
INC      := -I. -I$(FREERTOS_KERNEL)/include \
            -I$(FREERTOS_KERNEL)/portable/GCC/ARM_CM4F
```

也正因为 `-mfloat-abi=hard`，端口才是 CM4F 版本。**MCU 参数与端口目录是一对绑定关系**，改一个必须改另一个。

二、三异常接管：移植的全部核心

先把"三个异常改名"画出来：SVC 起死回生、PendSV 换魂、SysTick 报时。

FreeRTOS 三异常接管动画

内核的三个关键动作各挂在一个 Cortex-M 异常上：

内核函数（`port.c` 行号）	挂在哪个异常	干什么
`vPortSVCHandler`（260 行）	SVC（异常号 11）	启动第一个任务：手工恢复它的现场，从此不再返回
`xPortPendSVHandler`（505 行）	PendSV（异常号 14）	上下文切换：保存当前任务、恢复下一个任务
`xPortSysTickHandler`（562 行）	SysTick（异常号 15）	系统节拍：递增 tick、必要时挂起 PendSV 请求切换

`port.c:138-140` 处就是这三个函数的声明，其中两个带 `naked` 属性，和 S16 的 `HardFault_Handler` 同理：**现场必须手工管理，不能让编译器插桩**。

但**名字对不上**：我们的启动文件里向量表写的是 CMSIS 标准名：

```
.word SVC_Handler        ; startup_stm32f407xx.s:108
.word PendSV_Handler     ; :111
.word SysTick_Handler    ; :112
```

于是有**两种接法**：

（A）改向量表：把启动文件里的名字改成 `vPortSVCHandler` 等。侵入启动文件，将来换内核版本要再改。

（B）`#define` 映射：本工程用的就是这个，写在 `FreeRTOSConfig.h:74-77`：

```
#define vPortSVCHandler     SVC_Handler
#define xPortPendSVHandler  PendSV_Handler
#define xPortSysTickHandler SysTick_Handler
```

（B）的好处：启动文件保持标准 CMSIS 命名，内核函数在编译期被预处理改名，两边都不用为对方妥协。

实测：三个向量真的落在 port.o 里

光看 `#define` 不够，得证明**链接结果**确实如此。`nm` 一下产物：

```
08003180 T SVC_Handler
080032b0 T PendSV_Handler
08003318 T SysTick_Handler
```

三个符号都是 `T`（在 `.text` 里、已定义），它们**不是**启动文件里的 `IRQ_DEFAULT` 弱符号，而是被内核实现**顶替**了。地址间距也印证了上面的分工：`PendSV` 到 `SysTick` 相距 `0x68` 字节。

> 顺带看两个参照符号：`main` 在 `0x080010c4`，`prvIdleTask` 在 `0x08002080`（小写 `t` = 内核内部的 local 符号），`pxCurrentTCB` 在 `0x20000a7c`（`B` = `.bss`），**TCB 指针在 RAM 里**，正是 F2 讲的"换魂"要读的那个变量。

三、两份优先级：数值越大优先级越低

这是 Cortex-M 的经典迷惑点（S4 已建立概念），FreeRTOS 把它浓缩成两个宏。

FreeRTOS 使用**全部 4 位**作抢占优先级（`configPRIO_BITS = 4`）：**0 = 最高，255 = 最低**。

`FreeRTOSConfig.h:60-63` 的真值：

```
#define configKERNEL_INTERRUPT_PRIORITY         255  /* 最低（4 位域全 1） */
#define configMAX_SYSCALL_INTERRUPT_PRIORITY    (5 << (8 - configPRIO_BITS))  /* 0x50 */
#define configMAX_API_CALL_INTERRUPT_PRIORITY   configMAX_SYSCALL_INTERRUPT_PRIORITY
```

（1）`configKERNEL_INTERRUPT_PRIORITY = 255`：内核中断必须最低。

PendSV 和 SysTick 在**每个**任务切换点都要跑。如果它们优先级偏高，会打断用户的临界区；更要命的是，**低优先级的临界区保护就失效了**：FreeRTOS 进临界区靠 `BASEPRI` 屏蔽，而 `BASEPRI` 只能屏蔽"不高于某阈值"的中断。数值 255 意味着它挡不住任何用户中断，这正是设计意图：内核让路给所有应用中断。

（2）`configMAX_SYSCALL_INTERRUPT_PRIORITY = 0x50`：能调 API 的中断上限。

`5 << (8 - 4)` = `5 << 4` = `0x50` = 80。含义：**优先级数值 ≥ 80 的中断才允许调用 `...FromISR()` API**；数值 < 80（即更紧急）的中断**不许**碰任何 FreeRTOS API，因为它不受内核临界区保护。

`(8 - configPRIO_BITS)` 这个移位是把 4 位逻辑优先级"抬"到 8 位硬件寄存器的**高 4 位**：与 S4 讲过的 `IP 字节 = 逻辑优先级 << 4` 是同一件事（NVIC 寄存器只有高 4 位有效）。

搞反会怎样：把 `configMAX_SYSCALL_INTERRUPT_PRIORITY` 配成比 `configKERNEL_INTERRUPT_PRIORITY` 还"高"（数值更小），内核临界区就盖不住能调 API 的中断，队列/信号量内部链表被并发改写，症状是**随机死机**，且极难复现。这就是原文里"搞反全系统灵异"的机制。

四、配置逐项：`FreeRTOSConfig.h` 的真值

本工程配置（`code/rtos/01-freertos-lab/FreeRTOSConfig.h`）逐条读，每个数字都是正文结论的基准：

行	宏	真值	说明
16	`configCPU_CLOCK_HZ`	`16000000UL`	**本工程跑 HSI 16MHz，不启 PLL**
17	`configTICK_RATE_HZ`	`1000UL`	1ms 一个节拍（= S5 的 SysTick 1ms）
18	`configPRIO_BITS`	`4`	F407 的 NVIC 实现 4 个优先级位
21	`configUSE_PREEMPTION`	`1`	抢占式调度开
23	`configMAX_PRIORITIES`	`5`	优先级 0..4，**数字越大越高**
27	`configMINIMAL_STACK_SIZE`	`128`	**单位是字**（= 512 字节）
31	`configCHECK_FOR_STACK_OVERFLOW`	`2`	栈溢出检查开满（F7 会用）
34/35	动态/静态分配	`1` / `0`	只用动态分配
36	`configTOTAL_HEAP_SIZE`	`(16 * 1024)`	heap_4 的池子 = 16KB
41	`configTASK_NOTIFICATION_ARRAY_ENTRIES`	`1`	每任务 1 个通知槽（F6 讲"容量一"）
42	`configUSE_16_BIT_TICKS`	`0`	`TickType_t` 是 32 位
45	`configTIMER_TASK_PRIORITY`	`configMAX_PRIORITIES - 1` = `4`	定时器守护任务跑最高
53	`configUSE_TRACE_FACILITY`	`1`	`vTaskList` 等可用（场景 1 用到）

两个容易读错的点：

- `configCPU_CLOCK_HZ` 是 **16MHz**，不是 F407 的标称 168MHz：本工程故意跑内部 HSI，省掉时钟树配置的干扰。**拿 168MHz 去套这个工程的节拍计算会错 10 倍。**
- `configUSE_16_BIT_TICKS = 0` 决定了事件组**可用位数**：32 位 `TickType_t` 扣掉 8 个控制位 = **24 位**（配置第 49-50 行专门注释了这条口径）。

还有一组容易踩的宏（配置 67-72 行）：V11 内核默认把 `INCLUDE_*` 系列**全关**。从老版本升级后 `vTaskDelay` 链接不上，多半不是代码坏了，而是 `INCLUDE_vTaskDelay` 默认是 0，用哪个开哪个。

五、RAM 两套账：heap 池子 vs 裸机主栈

`size` 的实测三段（quick win 的输出改写成表）：

```
   text    data     bss     dec     hex filename
  43156    2168   19208   64532    fc14 freertos-lab.elf
```

- `text 43156`：代码 + 只读常量。FreeRTOS 内核的调度器、队列、定时器全在这里。
- `data 2168`：已初始化的可读写数据（要从 Flash 拷到 RAM）。
- `bss 19208`：零初始化数据，**16KB 的 heap 池子就躺在这儿**（`configTOTAL_HEAP_SIZE` = 16384 字节）。

关键认识：`bss` 里的 heap 池子，不等于"FreeRTOS 额外占用了 19KB"。 裸机程序也有 `.bss`（全局 buffer、栈）。真正的对照是**两套栈预算**：

	裸机工程	RTOS 工程
主栈	链接脚本里的 `_Min_Stack_Size`（启动后 SP 指向它）	只给**中断和启动阶段**用（MSP）
任务栈	不存在	每个任务从 **heap 池子**里切一块
谁在记账	链接脚本一处	`configTOTAL_HEAP_SIZE` + 每个任务的栈深

所以 RTOS 化的 RAM 账要**重算**：`heap 池子 = 所有任务栈 + 所有 TCB + 所有队列/信号量/定时器`，池子用尽时 `xTaskCreate` 会**返回失败**（`pdFAIL`），而不是像裸机那样栈溢出。

> 栈深单位陷阱再强调一次：`configMINIMAL_STACK_SIZE = 128` 是**128 字 = 512 字节**。这里填 128 字节的话，栈只有 32 字，随便一个 `printf` 就踩穿了。F7 有完整的审计方法。

六、首任务跑起来：完整命令序列

从零到可跑镜像，本工程的真实步骤：

```
# 1）取内核源码（V11.1.0），放进仓库忽略缓存
mkdir -p .trellis/ref/freertos-v11
curl -L -o .trellis/ref/freertos-v11/kernel.tar.gz \
  https://codeload.github.com/FreeRTOS/FreeRTOS-Kernel/tar.gz/refs/tags/V11.1.0
cd .trellis/ref/freertos-v11 && tar -xzf kernel.tar.gz && mv FreeRTOS-Kernel-11.1.0 kernel

# 2）构建（本工程默认从上面的路径找内核）
cd code/rtos/01-freertos-lab
mingw32-make DEMO_SCENE=1 -B      # Linux/macOS: make DEMO_SCENE=1 -B
```

验收点（每一步都有可检查的产物）：

步骤	产物	怎么验
取内核	`.trellis/ref/freertos-v11/kernel/tasks.c`	文件存在，且版本号是 V11.1.0
编译	`build/scene-1/*.o`（含 `port.o`）	十个 `.o` 齐了（见 Makefile 的 `OBJS`）
链接	`build/scene-1/freertos-lab.elf`	`nm` 出三个 `T` 的异常符号（第二节）
转换	`.bin` / `.hex`	`objcopy` 无报错
体积	`size` 三段	与本章表格对得上

上板（待回填）：`mingw32-make flash` 后，串口应能打印场景 1 的任务状态表（`vTaskList` 需要 `configUSE_TRACE_FACILITY = 1`，本工程已开）。**串口实际输出与板上现象待上板回填**：本章所有数字都是编译期/链接期实测，不含上板观测。

附录：工程完整源码

FreeRTOSConfig.h（内核行为的唯一配置源，正文第四节逐项引用）：

<<< ../../../code/rtos/01-freertos-lab/FreeRTOSConfig.h

Makefile（端口选择、内核源文件清单、五场景开关，正文第一节与第六节引用）：

<<< ../../../code/rtos/01-freertos-lab/Makefile

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **移植就是三个异常改名：SVC 起死回生、PendSV 换魂、SysTick 报时；优先级越大越低，内核永远垫底。**

延伸：三异常与 NVIC 优先级（S4）直接相关；上下文切换的汇编细节依赖调用约定（C6）；RT-Thread 移植对照见 R7。

实物实验

- 跑通第六节的完整序列，把 `size` 三段与本章表格逐项对照，数字不一致说明内核版本或场景选错了。
- `nm` 产物，找出 `SVC_Handler` / `PendSV_Handler` / `SysTick_Handler` 三个符号的地址，确认它们是 `T` 而非弱符号 `W`。
- 故意把 `FreeRTOSConfig.h:74-77` 的三行 `#define` 注释掉再编译，观察链接错误里提到的是哪三个符号（这就是"没有接管"的样子）。
- 上板：`make flash` 后抓串口日志（**待回填**）。

常见坑

- **选错 port 目录**：Cortex-M4F 用了 `ARM_CM4`（不带 F），浮点寄存器不在切换时保存，任务间浮点值互相踩。判据是 `-mfloat-abi=hard` 必须配 `ARM_CM4F`。
- **两份优先级配反**：`configMAX_SYSCALL_INTERRUPT_PRIORITY` 数值大于 `configKERNEL_INTERRUPT_PRIORITY` 才是对的（0x50 < 255）。配反 → 内核临界区盖不住能调 API 的中断 → 随机死机。
- **`configCPU_CLOCK_HZ` 填 168MHz**：本工程跑 HSI 16MHz（`FreeRTOSConfig.h:16`），填错会让节拍时长差 10 倍。
- **栈深单位当字节**：`configMINIMAL_STACK_SIZE = 128` 是 128 **字** = 512 字节。填成 128 字节 = 32 字，立刻踩穿。
- **`INCLUDE_*` 默认是 0**：V11 里这些 API 默认全关（配置 67-72 行）。`vTaskDelay` 链接不上先查这里，别怀疑代码。
- **忘了取内核源码**：Makefile 默认从 `.trellis/ref/freertos-v11/kernel` 找（该目录被 gitignore，不进仓库），新克隆的仓库必须先跑第六节第 1 步。

短自测

1. 为什么内核中断（PendSV/SysTick）优先级必须最低？

▼ 看答案
因为它们在每个任务切换点都要运行。若优先级偏高，会打断用户的临界区；更根本的是 FreeRTOS 靠 `BASEPRI` 屏蔽实现临界区，而 `BASEPRI` 只能屏蔽"不高于阈值"的中断，内核把自己的优先级设成最低（255），就等于**主动让路给所有应用中断**。本工程的取值见 `FreeRTOSConfig.h:61`。

2. `configMAX_SYSCALL_INTERRUPT_PRIORITY = (5 << (8 - 4))` 是多少？为什么这么算？

▼ 看答案
`5 << 4` = **0x50 = 80**。左移 `(8 - configPRIO_BITS)` 是把 4 位逻辑优先级抬到 8 位硬件寄存器的**高 4 位**：NVIC 的 IP 寄存器只有高 4 位有效，与 S4 讲的 `IP 字节 = 逻辑优先级 << 4` 是同一件事。含义：优先级数值 ≥ 80 的中断才允许调用 `...FromISR()` API。

3. `nm` 出来 `SVC_Handler` 是 `T` 而不是 `W`，说明了什么？

▼ 看答案
`T` = 该符号在本目标的 `.text` 段里已定义，来自 `port.o`。启动文件里原本的 `IRQ_DEFAULT SVC_Handler` 是 `W`（弱符号），链接时被 `port.c` 的强定义**顶替**了。这证明 `FreeRTOSConfig.h` 的 `#define` 映射确实生效，三个异常已经被内核接管。

4. RTOS 化之后，`configTOTAL_HEAP_SIZE` 和裸机的 `_Min_Stack_Size` 是什么关系？

▼ 看答案
没有关系，这是两套独立预算。 `_Min_Stack_Size` 是链接脚本给**主栈（MSP）**留的空间，RTOS 下只给中断和启动阶段用；`configTOTAL_HEAP_SIZE` 是 heap_4 的池子，所有**任务栈 + TCB + 队列/信号量/定时器**都从这里切。池子用尽时 `xTaskCreate` 返回失败，而不是栈溢出。

5. 为什么本工程 `configCPU_CLOCK_HZ` 是 16MHz 而不是 168MHz？

▼ 看答案
因为它跑 **HSI 16MHz、不启 PLL**（`FreeRTOSConfig.h:16` 注释写明）。这样做把"时钟树配置"这个变量从移植实验中剔除，节拍计算只依赖一个确定的常量。**拿 168MHz 去算这个工程的 tick 会差 10 倍。**

对照表：本章概念 → 仓库落点

概念	仓库落点
port 层选择与 MCU 参数绑定	`code/rtos/01-freertos-lab/Makefile`（`MCU` 与 `INC` 两行）
三个异常函数的声明（两个是 `naked`）	`.trellis/ref/freertos-v11/kernel/portable/GCC/ARM_CM4F/port.c:138-140`
`vPortSVCHandler` 实现（启动第一个任务）	同上 `:260`
`xPortPendSVHandler` 实现（上下文切换）	同上 `:505`
`xPortSysTickHandler` 实现（节拍）	同上 `:562`
向量表里的标准 CMSIS 名	`code/rtos/01-freertos-lab/startup_stm32f407xx.s:108,111,112`
`#define` 映射三行	`code/rtos/01-freertos-lab/FreeRTOSConfig.h:74-77`
两份中断优先级常量	同上 `:60-63`
时钟 / 节拍 / 优先级位 / 任务栈单位 / heap	同上 `:16,17,18,27,36`
`INCLUDE_*` 默认关（升级踩坑点）	同上 `:67-72`
`size` 三段实测（text 43156 / data 2168 / bss 19208）	`mingw32-make DEMO_SCENE=1 -B` 的输出
三个异常符号被顶替的实测	`arm-none-eabi-nm -n build/scene-1/freertos-lab.elf | grep -iE 'svc|pendsv|systick'`
五场景的另外四个	`code/rtos/01-freertos-lab/README.md`（场景 2–5 对应 F6/F7）

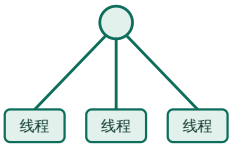
你做到了

- 把一个纯裸机工程变成了多任务工程，并知道每一处改动的**理由**而不只是步骤；

- 能对着 `nm` 和 `size` 的输出，验证"接管真的发生了"和"RAM 花在哪儿"；
- 有了这套清单，任意 Cortex-M4 工程都能按同一套路移植。

房东是内核：三位接管者全落在 port.o，你的工程只是把名字借给了向量表。霸天虎从此跑着多任务，清单在手，下一块 Cortex-M4 也照单抓药。

✅ F 篇收官。下一站：回到 RTOS 篇目录，或继续 F7 堆把 RAM 账算到底。



PART PR

第五部分 RT-Thread 精讲

从线程调度到 IPC 全家桶、内存管理、设备框架、finsh 控制台与 env 工具链，理解国产 RTOS 的工程哲学，并完成霸天虎移植。

> 同一块板子，另一套内核哲学：万物皆对象、设备框架按名取柄、finsh 控制台、Env 点单式构建。FreeRTOS 教你内核是什么，RT-Thread 教你内核之上怎么组织工程。设备框架与组件生态是另一套问题。移植走 Nano 最小集，五项验收可上板复跑。读完的标志：知道「内核 + 组件」两层各管什么，不再把一个组件的行为算到内核头上。

本部 8 章 · 约 280 分钟（4.7 小时）

45．R0 RT-Thread 架构：万物皆对象
46．R1 线程与调度：256 级位图的另一种写法
47．R2 IPC 全家桶：一套范式五种武器
48．R3 RT-Thread 内存管理：确定性之美
49．R4 设备框架：驱动与应用的解耦术
50．R5 finsh 控制台：板子上的 shell
51．R6 Env 与 menuconfig：RT-Thread 的点单系统
52．R7 移植 RT-Thread 到霸天虎：两步走

通往单片机之路第 45 章

R0 RT-Thread 架构：万物皆对象

难度 ★★☆☆☆ · 约 30 分钟

本章罗盘

问题

finsh 凭什么 list_thread/list_sem 一把梭，万物继承 rt_object 是怎么用纯 C 做出来的？

交付

一张三层架构图（内核/组件/软件包对应 src/components/packages 目录）+ rt_object→rt_thread 首成员继承的源码指认 + INIT_BOARD/DEVICE/COMPONENT/APP_EXPORT 段位触发时机表。

边界

结论以 RT-Thread 5.x master 为准，全章刻意不钉行号（版本未固定 tag、字段名会漂），落点只给到官方仓库文件路径；本章只给世界观，调度/IPC/设备机制分别在 R1/R2/R4；Nano 与标准版取舍只给规则，实际移植在 R7。

依据锚点 `f8` · `e7` · `mcu-anim`

> 🎯 如果说 FreeRTOS 是"瑞士军刀"（内核极致精简），RT-Thread 就是"工具箱"：内核之外自带设备框架、控制台、软件包生态。而理解这一切的钥匙只有一把——**万物皆对象**。

本章精髓

1. 三层架构：**内核层**（调度/IPC/内存，libcpu+BSP 之下）、**组件层**（finsh/设备框架/文件系统/网络）、**软件包**（社区三方，menuconfig 即点即用），生态是它的护城河。
2. 对象模型是统一世界观：线程/信号量/互斥量/定时器/设备全部继承自 `rt_object`（名字+类型+标志），`rt_object` 就是"基类"，容器（object container）统一管理，所以 finsh 能 `list_thread` / `list_sem` 一把梭。
3. 自动初始化有"段位"：`rt_components_board_init` → `rt_components_init` → 应用。INIT_BOARD_EXPORT/INIT_COMPONENT_EXPORT 等宏把各层初始化函数按段排好，启动时依次执行（链接脚本段机制的教科书级应用，回 B3）。

- **能理解**：为什么 finsh 能 `list_thread` / `list_sem` / `list_device` 一把梭；为什么 `INIT_*_EXPORT` 不用调用链就能按顺序跑；Nano 和标准版到底差在哪。
- **能用**：对照源码指认 `rt_object` → `rt_thread` 的继承（第一个成员就是对象头）；按移植目标选 Nano 还是标准版；看懂对象容器与 INIT 段位的配合。

先修

- F1~F3（内核概念对照锚点）；C4、B3。

先跑起来（10 分钟 quick win）

打开 RT-Thread 源码 `include/rtdef.h` 找到 `struct rt_object` 与 `struct rt_thread`，亲眼看到"线程结构的第一个成员就是对象"，继承一目了然。

翻 rtdef.h 之前先押个辈分：rt_thread 的第一个成员是谁，凭什么排在栈顶指针前面？押完往下翻两屏就见分晓。

动画：对象模型与容器

rt_object 基类、rt_thread 的首成员继承、对象容器按类型分链，橙色遍历珠沿 thread 链移动，list_thread 的"一把梭"就是它在逐节点走。

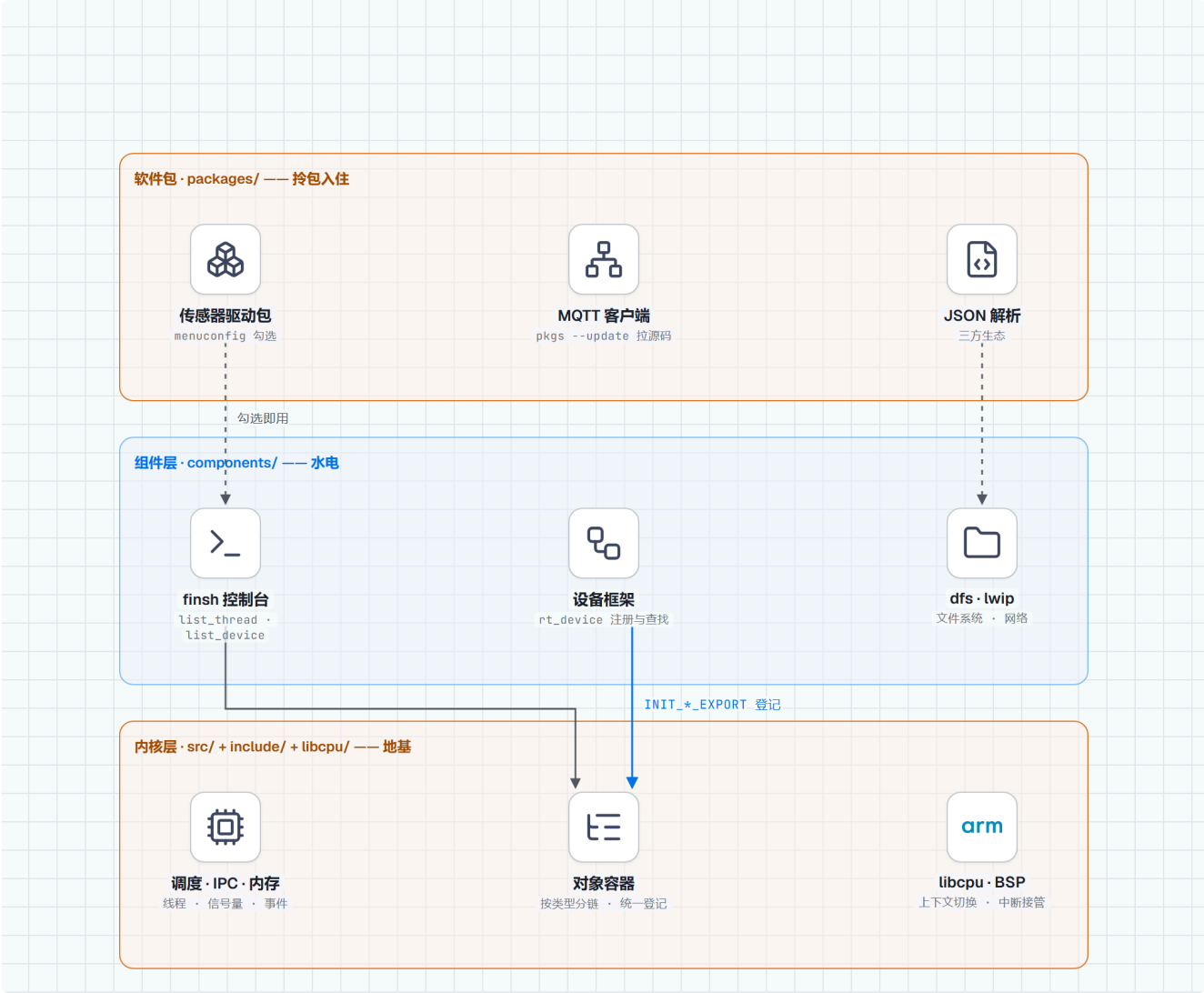
R0 对象模型：万物继承 rt_object

小节结构

小节	内容	四件套
三层架构	内核/组件/软件包目录巡礼（src/components/packages）	库解析
对象模型	rt_object 基类与容器；list API 为什么万能	库解析
自动初始化	INIT_*_EXPORT 段位表；与链接脚本的配合	库解析
Nano vs 标准版	裁剪版与完整版的定位差；R7 移植选哪条路	配置
版本与许可	5.x 主线与 Apache-2.0；商业使用无负担	配置

一、三层架构：内核/组件/软件包目录巡礼

三层楼先看图：地基管调度，组件当水电，软件包按需搬进来。



RT-Thread 三层架构：内核层打地基、组件层当水电、软件包拎包入住，组件靠 INIT_*_EXPORT 登记进内核的对象容器

RT-Thread 把整个系统分成三层，每层职责分明、目录也分得开：

- **内核层**（`src/` + `include/` + `libcpu/`）：调度器、线程、IPC（信号量/互斥量/事件/邮箱/消息队列）、定时器、内存管理、对象容器。这一层只关心"怎么把 CPU 公平地分给一堆线程"，和 FreeRTOS 的内核职责一一对应。`libcpu/` 放的是与具体 CPU 架构相关的上下文切换、中断接管（对应 F2 的 PendSV 那一层）。
- **组件层**（`components/`）：finsh 控制台、设备框架（`rt_device`）、文件系统（dfs）、网络（lwip）、shell、msh，内核之外的"水电"都住这儿。组件通过 `INIT_*_EXPORT` 自动登记自己，应用代码用统一 API 调它们，不用关心谁先初始化。
- **软件包**（`packages/`）：社区三方包，`menuconfig` 里勾选、`pkgs --update` 拉源码，编译时一并进工程。从传感器驱动到 MQTT 客户端到 JSON 解析，生态是 RT-Thread 的护城河。

对照 FreeRTOS：它的内核对应 RTT 的内核层，但 FreeRTOS 没有"组件层"这一档：设备框架/文件系统/控制台都得自己接或用第三方（ESP-IDF 那一套就相当于给 FreeRTOS 配了个"组件层"）。RTT 把这一层做进了主线。

> 口诀版区分：内核打地基（调度/IPC/内存）、组件当水电（控制台/设备/文件系统）、软件包是家具（按需拎包入住）。

二、对象模型：rt_object 基类与容器

RT-Thread 的"万物皆对象"不是说用 C++ 写了个抽象基类。它是纯 C，用**结构体嵌套**模拟继承（回 C4）。`rt_object` 是"基类"，每个具体类型（线程/信号量/定时器/设备/内存池）的结构体第一个成员都是 `rt_object`：

```
struct rt_object
{
    char       name[RT_NAME_MAX];   /* 名字 */
    rt_uint8_t type;                 /* 类型（线程/信号量/…） */
    rt_uint8_t flag;                  /* 标志（静态/动态） */
    rt_list_t  list;                  /* 挂在对象容器里 */
};

struct rt_thread
{
    struct rt_object parent;   /* 第一个成员 = 对象头，继承 */
    volatile rt_uint32_t sp;    /* 栈顶 */
    rt_uint8_t  current_priority;
    rt_uint8_t  stat;
    ...
};
```

因为 `rt_object` 排在最前，任何具体对象的指针都能"安全地"当成 `rt_object*` 用，这是 C 语言里"继承"的标准玩法。对象容器（`rt_object_container`）按类型维护链表（`rt_object_container[type]`），每创建一个对象就挂进对应链表。finsh 的 `list_thread` / `list_sem` / `list_timer` / `list_device` 能一把梭打印，本质就是遍历对应类型的容器链表，把每个对象当 `rt_object` 读名字、当具体类型读字段。这就是"万能"的来源。

> 钥匙：`rt_object` 排第一 → 任何对象都是 `rt_object` → 容器统一登记 → list API 统一遍历。

三、自动初始化：INIT_*_EXPORT 段位表

RT-Thread 不让你在 main 里一条条调初始化函数，而是用链接段（回 B3）把所有初始化函数按"段位"排好：

段位宏	触发时机	典型用途
`INIT_BOARD_EXPORT`	`rt_components_board_init()`，最早	板级 GPIO/时钟
`INIT_DEVICE_EXPORT`	`rt_components_init()`，设备段	设备驱动注册
`INIT_COMPONENT_EXPORT`	`rt_components_init()`，组件段	finsh/dfs 框架
`INIT_ENV_EXPORT`	`rt_components_init()`，环境段	环境变量/挂载
`INIT_APP_EXPORT`	`rt_components_init()`，应用段	应用入口

机制（以 RT-Thread 官方源码 `rt-thread/src/components.c` 为准，不引行号）：每个宏把函数指针放进一个特殊的链接段（如 `.rti_fn.*`），启动时 `rt_components_board_init` / `rt_components_init` 各自遍历自己负责的那一段、挨个调用。所以你只写 `INIT_DEVICE_EXPORT(my_driver_init)`，从不在 main 调它，它也会在合适时机自动跑，这就是"免调用链"。

段内顺序由链接顺序定（看链接脚本里段的排布），所以**板级依赖关系**要靠段位选择对，而不是靠"我写在前面它就先跑"。这也是常见坑的来源：设备注册依赖了某组件，但你把驱动放进了更早的段，结果"设备没注册上"。

四、Nano vs 标准版

RT-Thread 有两个裁剪档：

- **Nano**：内核 + 精简 finsh，**没有**设备框架、文件系统、网络、组件层那一套。体积极小，适合资源紧张（Flash 小于 64KB）或只需要"调度 + IPC"的场景。上手快、移植简单（R7 走 Nano 路最省事）。
- **标准版**：三层齐全，组件/软件包生态全开。要 finsh msh、要 `list_device`、要 dfs 文件系统、要 lwip 上网，就只能上标准版。

选择规则：只跑几个线程做控制，Nano 够用；要用设备框架挂传感器、要文件系统存数据、要网络通信，直接标准版。R7 移植章节会分两条路走。

> 划重点：Nano 没有设备框架/finsh 全功能，想要生态就上标准版（R6/R7）。

五、版本与许可

- **主线版本**：RT-Thread 5.x，长期演进。本书所有结论以 5.x 为准。
- **许可**：Apache-2.0，商业使用无负担，可以闭源衍生，只需保留许可声明。与 FreeRTOS 的 MIT 同属宽松档，做产品不用担心许可问题。

RT-Thread 的生态更偏国内（中文文档、国产 MCU 适配多），FreeRTOS 更国际化。两者都是"商业可用"，选哪个看生态与团队熟悉度。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **RTT 三层楼：内核打地基、组件当水电、软件包是家具；万物继承 rt_object，初始化按段自动排队。**

延伸：RTT 对象模型动画见 R0 动画；FreeRTOS 对象管理对照见 F1；双 OS 架构对比见 X0。

实物实验

- 在 R7 移植工程上执行 finsh 命令 `list_thread`、`list_device`（对象模型的"户口本"直接打印在终端上）。

常见坑

- **Nano 当标准版用**：Nano 没有设备框架/finsh，想要生态就上标准版（R6/R7）。
- **对象当线程用**：`rt_object` 是"概念基类"，直接操作它是糊涂账，走具体类型的 API。
- **自动初始化顺序误解**：段内顺序由链接顺序定，板级依赖关系错时表现为"设备没注册上"。
- **标准版工程漏了 `rt_components_init`**：段里的初始化函数不会自己跑，要在 main 流程里触发 `rt_components_init()`，漏掉就是 `list_device` 一片空、文件系统/网络没动静。
- **menuconfig 后忘了 `pkgs --update`**：Kconfig 只动了配置，软件包源码没拉下来，编译直接报找不到头文件。配置完务必同步拉包。

短自测

1. RT-Thread 的三层架构是哪三层？各自对应哪些目录？

> **▼ 看答案**
> 内核层（src/ + include/ + libcpu/）、组件层（components/）、软件包（packages/）。内核打地基、组件当水电、软件包是家具。

2. 为什么 finsh 能用 `list_thread` / `list_sem` / `list_device` 一把梭打印所有对象？

> **▼ 看答案**
> 所有对象都继承自 rt_object（结构体第一个成员就是 rt_object），对象容器按类型维护链表。list API 遍历对应类型的容器链表，把每个对象当 rt_object 读名字、当具体类型读字段，"万物皆对象"让统一遍历成为可能。

3. `INIT_*_EXPORT` 是怎么做到"免调用链"自动初始化的？

> **▼ 看答案**
> 每个宏把函数指针放进一个特殊链接段（如 .rti_fn.*），启动时 rt_components_board_init / rt_components_init 各自遍历自己负责的段、挨个调用。链接脚本（回 B3）的段机制是它的物理基础。所以你只写 INIT_DEVICE_EXPORT(my_init)，从不在 main 调它，它也会自动跑。

4. Nano 和标准版的主要差异是什么？什么场景选哪个？

> **▼ 看答案**
> Nano 只有内核 + 精简 finsh，没有设备框架/文件系统/网络/组件层；标准版三层齐全。只跑几个线程做控制选 Nano，要用设备框架/文件系统/网络选标准版。R7 移植分两条路走。

5. RT-Thread 用什么开源许可？商业使用有什么影响？

> **▼ 看答案**
> Apache-2.0。商业可用、可闭源衍生，只需保留许可声明，做产品无负担，与 FreeRTOS 的 MIT 同属宽松档。

对照表：本章概念 → 仓库与上游落点

概念	落点
三层架构目录	RT-Thread 源码 `src/` + `include/` + `libcpu/`、`components/`、`packages/`
`rt_object` 基类	以 RT-Thread 官方源码（`rt-thread/include/rtdef.h`）为准（`struct rt_object`）
`rt_thread` 继承 `rt_object`	以 RT-Thread 官方源码（`rt-thread/include/rtthread.h`）为准（`struct rt_thread`，首成员 `parent`）
对象容器	以 RT-Thread 官方源码（`rt-thread/src/object.c`）为准（`rt_object_container`）
`INIT_*_EXPORT` 段位宏	以 RT-Thread 官方源码（`rt-thread/include/rtdef.h`）为准
段位执行	以 RT-Thread 官方源码（`rt-thread/src/components.c`）为准（`rt_components_board_init` / `rt_components_init`）
版本与许可	RT-Thread 5.x 主线，Apache-2.0
对照锚点	F1 tskTCB 继承对照

你做到了

- RT-Thread 的"世界观"成型：一张架构图 + 一个对象模型；
- 拿到对照学习的第二锚点：与 F 篇逐章对表开始。

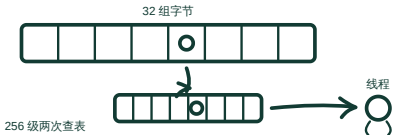
辈分第一：rt_object 领衔，万物皆对象不是口号，是结构体第一排的座位；对象挂进容器统一登记，finsh 那些一把梭的 list 命令才有根。

✅ 下一站：R1 线程与调度：rt_thread 解剖与 256 级位图，和 FreeRTOS 逐项对照。

通往单片机之路第 46 章

R1 线程与调度：256 级位图的另一种写法

难度 ★★☆☆☆ · 约 35 分钟



本章罗盘

问题

256 级优先级怎么还能 O(1) 选优，二级位图两次查表各查什么，比 FreeRTOS 多付哪笔账？

交付

一张 rt_thread × tskTCB 双列对照表（对象头、内嵌 rt_timer、优先级方向相反三处差异标出）+ 二级位图 i×8+j 选优推演 + create/init 两种创建的内存来源表与 startup 两步纪律。

边界

「数小=高」只对 RT-Thread 默认语义成立，与 FreeRTOS 反向、与 NVIC 同向，三家不可混背；源码以 master 为准不钉行号；GDB 对照双 OS 就绪表内存布局的实验要两份固件上板（R7/F8 工程）才落地，本章只给方法。

依据锚点 `f8` · `f1` · `mcu-anim`

> 🎯 FreeRTOS 用 32 位位图管 32 个优先级；RT-Thread 说 32 个不够，上 256 级——位图还是那张位图，只是升级成"二级位图"（32×8）。同样的 CLZ，加一层索引，调度依然是 O(1)。

本章精髓

1. `struct rt_thread` 与 TCB 对照：对象头（R0 的继承）+ `sp`（栈顶）+ 优先级 + 状态 + 剩余 tick + 定时器（线程自带 `rt_timer`！）。功能等价物都能找到，结构风格更"面向对象"。
2. 二级位图：`rt_thread_ready_table[32]` 每字节管 8 个优先级，`rt_thread_ready_priority_group` 管哪字节非空：两次 CLZ/查表定位最高优先级（源码 `rt_schedule` + `_get_highest_priority_thread`）。
3. 调度点的同与不同：同样有 tick（`rt_tick_increase`）、有抢占；不同的是 RT-Thread 把"线程延时"挂在**线程自带的定时器**上，定时器子系统是全家共享的基础设施。

- **能理解**：为什么二级位图能 O(1) 定位 256 级（两步查表）；为什么 `rt_thread_create` 之后还要 `rt_thread_startup`；为什么删除线程要"排队死"；优先级数值方向三家怎么背。
- **能用**：`list_thread` 看栈水位；`rt_thread_create` / `rt_thread_init` 各建一线程并说清内存来源；用 GDB 对照 RTT 与 FreeRTOS 的就绪表内存布局。

先修

- R0、F1~F3。

先跑起来（10 分钟 quick win）

finsh 里 `list_thread` 看自家线程表：名称/优先级/状态/栈用量一屏出，RT-Thread 的"可观测性"开箱即用。

list_thread 之前先押个嫁妆：rt_thread 出门自带一件东西，FreeRTOS 的延时要把任务挂进全局的阻塞列表统一管；是哪件？押完对照 rtdef.h 与 F1 的 TCB 看答案。

动画：二级位图 256 级选优

先在 32 位组选字上定位第一个非空字节，再在该字节上定位第一个非空位，两次固定步数查表，256 级优先级的 O(1) 选优全程可见。

R1 二级位图：32×8 实现 256 级优先级选优

小节结构

小节	内容	四件套
rt_thread 解剖	字段逐一对照 tskTCB	库解析
二级位图	256 级选优的两步查表；与 32 级的取舍	库解析
动态 vs 静态	rt_thread_create vs rt_thread_init 的内存来源	配置
tick 与定时器	rt_tick_increase 与线程内嵌 rt_timer	库解析
调度点	rt_schedule 的进入路径（API/中断退出/tick）	代码分析
栈与水位	list_thread 的栈用量列；与 F 篇水位线对照	代码分析

一、rt_thread 解剖：字段逐一对照 tskTCB

`struct rt_thread`（以 RT-Thread 官方源码 `rt-thread/include/rtdef.h` 为准）的核心字段，与 FreeRTOS `tskTCB`（F1）逐项对照：

rt_thread 字段	含义	tskTCB 对照	备注
`parent`（`rt_object`）	对象头：名字+类型+标志+容器链表	无直接对应	RTT 的"继承"，FreeRTOS 没有
`sp`	当前栈顶	`pxTopOfStack`	同为切换锚点
`current_priority`	当前优先级（数值越小越高）	`uxPriority`（越大越高）	**方向相反**
`stat`	状态（就绪/挂起/阻塞/...）	`eCurrentState`	
`remaining_tick`	剩余时间片/延时 tick	`xItemValue`（延时列表项值）	RTT 独立字段
`timer`（`rt_timer`）	**线程内嵌定时器**	无内嵌，延时挂阻塞列表	RTT 特色
`stack_addr` / `stack_size`	栈底地址/栈大小	`pxStack` /栈深	
`entry` / `parameter`	入口函数/参数	任务函数/参数	

最显著的两处差异：

1. **对象头 `parent`**：FreeRTOS 的 TCB 是个"光杆"结构体，RTT 的线程一上来先继承 `rt_object`，所以它有名字、有类型、自动进容器，`list_thread` 才能打印。这是 R0"万物皆对象"在 TCB 上的具体落地。
2. **内嵌 `rt_timer`**：FreeRTOS 的延时靠"阻塞列表 + 列表项的 `xItemValue`"，没有"每个任务一个定时器"的概念；RTT 直接给每个线程塞一个 `rt_timer`，线程延时就把这个定时器挂到系统定时器链上，到点了回调唤醒，**定时器子系统是全家共享的基础设施**，连线程延时都用它。

> 看字段名记风格：FreeRTOS 偏"裸数据 + 链表项"组合，RTT 偏"面向对象 + 自带定时器"。

二、二级位图：256 级选优的两步查表

RT-Thread 的就绪表是"二级位图"（以 RT-Thread 官方源码 `rt-thread/src/scheduler.c` 为准，看 `_get_highest_priority_thread` 与 `rt_schedule`）：

- `rt_thread_ready_priority_group`：一个 32 位字，bit i 表示"第 i 个字节是否有就绪线程"；
- `rt_thread_ready_table[32]`：32 字节数组，每个字节 8 位，bit j 表示"优先级 i×8+j 是否就绪"。

两层加起来 = 32×8 = **256 级优先级**。选优分两步：

1. 在 `rt_thread_ready_priority_group` 上做 CLZ（或查 `rt_lowest_bitmap` 表）找到**第一个非空字节**的索引 `i`；
2. 在 `rt_thread_ready_table[i]` 那个字节上再做一次 CLZ/查表，找到**该字节内第一个非空位** `j`；
3. 最高优先级 = `i×8 + j`。

两次查表，固定步数，与任务总数无关，**O(1)**。

对照 FreeRTOS（F3）：它用一个 32 位字，每个 bit 一个优先级，最多 32 级，一次 CLZ 完事。取舍：

	FreeRTOS	RT-Thread
最大优先级数	32（`configMAX_PRIORITIES` 上限 32）	256
查询步数	1 次 CLZ	2 次查表/CLZ
RAM 开销	1 个 32 位字	1 个 32 位字 + 32 字节
适合场景	优先级少、极致精简	优先级多、要细分

RTT 多花一丁点 RAM 和一次查表，换来 8 倍的优先级粒度，这是"位图位图，多一层索引"的代价与收益。

> 口诀：`rt_thread_ready_priority_group` 选字节，`rt_thread_ready_table[i]` 选位，两次定位 256 级。

三、动态 vs 静态：两种创建的内存来源

RT-Thread 给线程两种创建方式，内存来源不同（以 RT-Thread 官方源码 `rt-thread/src/thread.c` 为准）：

	rt_thread_create	rt_thread_init
TCB 来源	堆 `rt_malloc`	用户提供（静态）
栈来源	堆 `rt_malloc`	用户提供（静态）
返回	句柄（指针）	`rt_err_t`（错误码），句柄即传入指针
释放	`rt_thread_delete`（堆上回收）	`rt_thread_detach`（不释放内存，自管）
FreeRTOS 对照	`xTaskCreate`（动态）	`xTaskCreateStatic`（静态）

两者都**只造好线程、不挂就绪**，必须再调一次 `rt_thread_startup` 才把它放进就绪表、可能立即抢占当前线程。这是和 FreeRTOS `xTaskCreate` 最大的行为差：FreeRTOS 的 `xTaskCreate` 一步到位（造好 + 挂就绪 + 可能立即抢占），RTT 把"造"和"启动"拆成两步，少了 `startup` 线程永远不动（见常见坑）。

选哪种：

- **动态**：栈大小运行时才定、用完能 `delete` 回收，灵活，但依赖堆、有碎片风险；
- **静态**：栈和 TCB 全在编译期定好地址，不依赖堆、不怕碎片，安全核查/功能安全场景首选。

四、tick 与定时器：rt_tick_increase 与线程内嵌 rt_timer

时钟节拍由 `rt_tick_increase` 驱动（以 RT-Thread 官方源码 `rt-thread/src/clock.c` 为准）：每个 SysTick 中断调用它，做两件事：

1. 系统时钟 `rt_tick` 加 1；
2. 遍历定时器链表，把到点的定时器摘下来、执行回调。

线程延时的实现（`rt_thread_delay` / `rt_thread_mdelay` → `rt_thread_sleep`）：把当前线程从就绪表摘下，给它**自带的 `rt_timer`** 设上延时值，挂进系统定时器链，然后触发调度。到点了定时器回调把线程重新挂回就绪表，延时就这样完成。

这是 RTT 和 FreeRTOS 的关键架构差：

- **FreeRTOS**：延时靠"延时列表 + 列表项的 `xItemValue`（唤醒 tick）"，每个任务没有"自己的定时器对象"，调度器在 tick 里扫延时列表；
- **RT-Thread**：延时直接借用"线程内嵌的 `rt_timer`"，定时器子系统是共享的，线程延时、软件定时器、超时 watchdog 都走同一套机制。

> 对照着看：FreeRTOS 把延时塞进调度器的列表，RTT 把延时挂到线程肚里的定时器，基础设施复用，概念上 RTT 更"对象化"。

五、调度点：rt_schedule 的进入路径

`rt_schedule` 是 RTT 的调度入口（以 RT-Thread 官方源码 `rt-thread/src/scheduler.c` 为准），三种典型进入路径：

1. **API 主动让出**：`rt_thread_yield`（同优先级轮转）、`rt_thread_delay` / `rt_thread_mdelay`（延时阻塞自己），这些 API 内部调 `rt_schedule`；
2. **中断退出**：PendSV/上下文切换路径里，如果标记了"需要切换"，在中断退出时跑调度（libcpu 层）；
3. **tick 到点**：`rt_tick_increase` 里，如果时间片用完或延时到点，触发 `rt_schedule`。

`rt_schedule` 干的活：用二级位图找出当前最高优先级就绪线程，与当前线程比较，不一样就调 `rt_hw_context_switch` 换栈（PendSV）。这与 FreeRTOS 的"找最高就绪 → PendSV 切换"骨架一致，差别只在"怎么找最高优先级"（位图实现不同，见第二节）。

六、栈与水位：list_thread 的栈用量列

finsh 的 `list_thread` 输出列大致是：`name / pri / status / sp / stack size / stack used / max used`。其中：

- **stack size**：建线程时给的栈大小（字节，RTT 这里直接是字节不是字）；
- **max used / 栈水位**：历史最大使用深度，原理同 FreeRTOS 的 `uxTaskGetStackHighWaterMark`（F1 第五节）：建栈时把空闲区填一个魔数（RTT 默认 `'#'`），运行后扫到底部往上第一个非魔数的位置，就知道"最深用到哪"。

对照 F 篇水位线：

	FreeRTOS	RT-Thread
魔数	`0xA5`（`tskSTACK_FILL_BYTE`）	`'#'`
查询 API	`uxTaskGetStackHighWaterMark`	`list_thread` 内置列 / 扫栈
单位	字（`StackType_t`）	字节
溢出钩子	`vApplicationStackOverflowHook`	`rt_thread_stack_overflow_hook`

对照学习的硬证据：同一台板子分别刷 FreeRTOS 与 RTT 两份固件，GDB 看 `pxCurrentTCB` / `rt_current_thread` 的就绪表内存布局：单级位图（一个 32 位字）vs 二级位图（一字 + 32 字节数组），一眼看清"两种解法"。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **对象打头 sp 记账，256 级两查表；动态堆上请，静态自家造；线程肚里还揣着定时器。**

延伸：RTT 二级位图动画见 R1 动画；FreeRTOS 调度对照见 F3；任务创建与 TCB 在 F1 详解。

实物实验

- 双线程异频闪灯 + `list_thread` 观测栈水位；
- GDB 对照两 OS 的"就绪表"内存布局（同一台板子两份固件），对照学习的第一个硬证据。

常见坑

- **静态线程忘了栈对齐**：自供栈必须 8 字节对齐（RT_ALIGN），不对齐下场同 F2。
- **删除线程在自身上下文**：自杀用 `rt_thread_delete(rt_thread_self())` 并在调度中清理，理解"死亡也要排队"。
- **优先级习惯沿用 FreeRTOS 数值感**：RT-Thread 数值越小优先级越高（0 最高），与 NVIC 同向、与 FreeRTOS 反向，三家对照背熟。
- **启动线程忘 rt_thread_startup**：create 只是造好，startup 才挂就绪，少一步永远不动。
- **二级位图查表方向记反**：`rt_thread_ready_priority_group` 用 CLZ 找最低置位（=最高优先级），不是从高位往低扫，记住"数值小先跑"，与 NVIC 同向。

短自测

1. `rt_thread` 结构体的第一个成员是什么？它带来了什么能力？

> ▼ **看答案**
> 第一个成员是 rt_object（对象头）。它让线程继承"对象"，有名字、有类型、自动进对象容器，所以 list_thread 才能一把梭打印。这是 C 结构体嵌套模拟继承（回 C4）。

2. 二级位图怎么用两次查表/CLZ 定位 256 级中的最高优先级？

> ▼ **看答案**
> 第一步在 rt_thread_ready_priority_group（32 位字）上 CLZ 找第一个非空字节的索引 i；第二步在 rt_thread_ready_table[i]（那个字节）上再 CLZ 找第一个非空位 j；最高优先级 = i×8 + j。两次查表固定步数，O(1)。

3. `rt_thread_create` 和 `rt_thread_init` 的内存来源有何不同？各自对应 FreeRTOS 的哪个 API？

> ▼ **看答案**
> create 从堆 rt_malloc 分配 TCB 和栈（动态），对应 FreeRTOS 的 xTaskCreate；init 用用户自备的 TCB 和栈（静态），对应 xTaskCreateStatic。两者都还要再调 rt_thread_startup 才挂就绪，这点和 xTaskCreate 一步到位不同。

4. RT-Thread 的线程延时挂在哪个数据结构上？这和 FreeRTOS 有什么不同？

> ▼ **看答案**
> 挂在线程内嵌的 rt_timer 上，延时 = 摘下就绪 + 给自带定时器设值 + 挂系统定时器链，到点回调唤醒。FreeRTOS 没有内嵌定时器，延时靠"延时列表 + 列表项 xItemValue"。RTT 把定时器作为全家共享基础设施。

5. RT-Thread 的优先级数值方向如何？与 FreeRTOS、NVIC 各是什么关系？

> ▼ **看答案**
> RT-Thread 数值越小优先级越高（0 最高），与 NVIC 中断优先级同向，与 FreeRTOS（数值越大越高）反向。三家对照背熟：RTT/NVIC 数小高、FreeRTOS 数大高。

对照表：本章概念 → 仓库与上游落点

概念	落点
`struct rt_thread`	以 RT-Thread 官方源码（`rt-thread/include/rtdef.h`）为准（`struct rt_thread`，首成员 `parent`）
二级位图	以 RT-Thread 官方源码（`rt-thread/src/scheduler.c`）为准（`rt_thread_ready_table` / `rt_thread_ready_priority_group` / `_get_highest_priority_thread`）
`rt_thread_create` / `rt_thread_init`	以 RT-Thread 官方源码（`rt-thread/src/thread.c`）为准
`rt_tick_increase`	以 RT-Thread 官方源码（`rt-thread/src/clock.c`）为准
`rt_schedule`	以 RT-Thread 官方源码（`rt-thread/src/scheduler.c`）为准
`list_thread`	以 RT-Thread 官方源码（`rt-thread/components/finsh/`）为准
对照锚点	F1 tskTCB / F3 单级位图

你做到了

- 第二个内核的线程/调度机制打通；
- "同一问题两种解法"的对照眼练成。

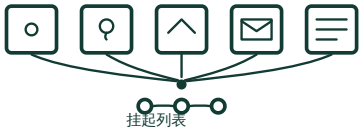
嫁妆是定时器：FreeRTOS 把延时挂进全局的阻塞列表，RT-Thread 让每个线程自带 rt_timer 出门。同一件事两种记法，你的双 OS 对照表又添一行。

✅ 下一站：R2 IPC 全家桶：信号量/互斥量/事件/邮箱/消息队列，一套范式五种武器。

通往单片机之路第 47 章

R2 IPC 全家桶：一套范式五种武器

难度 ★★☆☆☆ · 约 35 分钟



本章罗盘

问题

信号量、互斥量、事件、邮箱、消息队列要背五套 API 吗，它们共用哪副骨架，排队秩序谁说了算？

交付

一张通用 IPC 骨架图（rt_ipc_object 的 suspend_list + 五种对象的特殊字段标注）+「中断 release 信号量→线程 take、邮箱传指针」双管道代码与 list_sem/list_mailbox 在册观测。

边界

E04 的 RT-Thread 版双跑在本章是剧本与预期，板上两份日志待实测回填（E04 实测表仍空白）；源码以 master 为准不钉行号；互斥量整条路径禁入 ISR 只给纪律、中断侧源码不展开；与 FreeRTOS 的继承对照只到字段与三步动作同构，不比性能。

依据锚点 `f8 · d2 · mcu-lab · mcu-anim`

> 🎯 信号量、互斥量、事件集、邮箱、消息队列：RT-Thread 的 IPC 看起来像五件武器，拆开全是同一副骨架：**一个对象头 + 一个资源计数/存储区 + 一条按优先级排队的挂起列表**。学会一副骨架，五种武器全会。

本章精髓

1. 挂起列表（suspend list）是灵魂：`rt_ipc_list_suspend/resume` 统一处理"排队等号"：FIFO 或按优先级（PRIO）两种秩序，创建时一个标志位决定。
2. 五种 IPC 的分工：信号量=计数令牌、互斥量=带持有者与继承的令牌、事件集=32 位旗语、邮箱=定长消息（4 字节）池、消息队列=变长消息环形队。选型按"同步还是传信、消息多大"。
3. 互斥量的继承与 FreeRTOS 同构：`rt_mutex` 记录 owner 与 original priority，同样是火星 bug 的解法，读 `rt_ipc.c` 对照 F5 最有味道。

- **能理解**：为什么五种 IPC 共用一套 `rt_ipc_list_suspend/resume`；为什么互斥量的继承和 F5 同构（owner + original priority）；为什么邮箱固定 4 字节而不是变长。
- **能用**：用信号量+邮箱打通中断→线程、线程→线程双通信；E04 双 OS 双跑对比继承行为，两组日志并排核对。

先修

- R1、F4~F5。

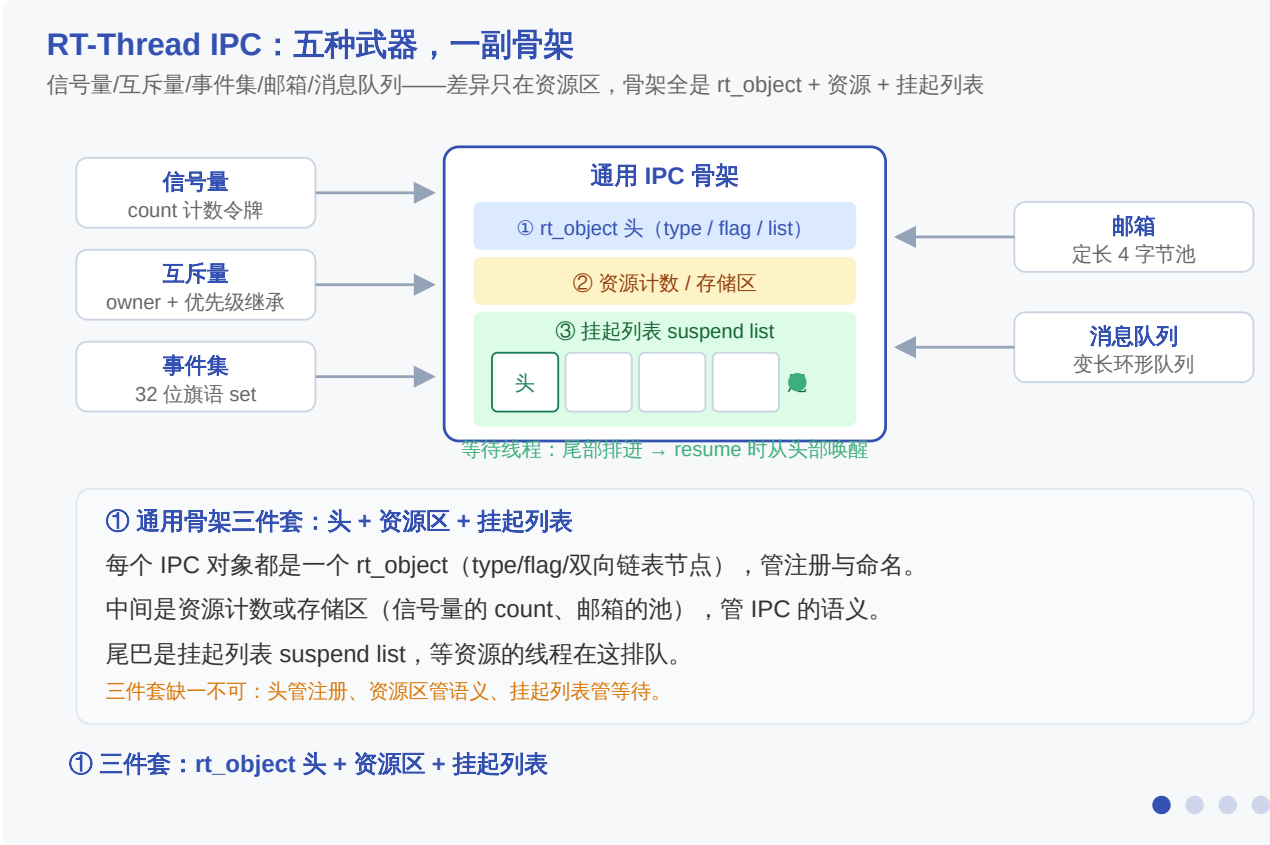
先跑起来（10 分钟 quick win）

finsh 一把梭：`list_sem`、`list_mutex`、`list_msgqueue`，刚创建的 IPC 对象全部在册，对象模型（R0）的可观测性再次立功。

先押个骨架数：信号量、互斥量、事件集、邮箱、消息队列五件兵器，拆开到底是几副骨架？押完翻 rt_ipc.c，答案会逼你把五件当一个背。

动画：五种武器一副骨架

信号量、互斥量、事件集、邮箱、消息队列轮番上阵，但每一次阻塞与唤醒都落到同一条挂起列表上。五张皮共享一副骨架，排队秩序创建时一锤定音。



R2 IPC 五种武器一副骨架

小节结构

小节	内容	四件套
通用骨架	rt_ipc_object 与挂起列表；FIFO/PRIO 两秩序	库解析
信号量/互斥量	take/release 源码路径；继承发生点对照 F5	库解析
事件集	AND/OR 等待与清零语义；与 FreeRTOS 事件组对表	配置
邮箱 vs 消息队列	定长池 vs 变长队；内存来源与拷贝成本	代码分析
反转复现 RTT 版	E04 双跑：两 OS 继承行为实测对比	代码分析
finsh 观测	list_* 系列命令的对象模型原理	库解析

一、通用骨架：rt_ipc_object 与那条挂起列表

RT-Thread 5.x 把所有 IPC 的公共部分抽到 `struct rt_ipc_object`（位于 `include/rtdef.h`，**以 RT-Thread 官方源码为准**）。它有两个灵魂字段：

- `suspend_list`：一条 `rt_list_t` 双向链表，挂着所有"在等这个 IPC"的线程。`rt_ipc_list_suspend` 把申请失败要阻塞的线程按秩序塞进这条链表；`rt_ipc_resume` 在资源到位时把"该醒的第一个"摘下来唤醒。
- `suspend_list` 的秩序二选一：`RT_IPC_FLAG_FIFO`（先到先得）或 `RT_IPC_FLAG_PRIO`（按线程优先级排序），**创建时一个标志位决定，之后不再改**。

为什么这副骨架能统一五种 IPC：信号量、互斥量、事件集、邮箱、消息队列的"等"都落到这条链表上，差别只在"等的是什么、醒了之后怎么判断能不能拿走"。读源码盯住 `rt_ipc_list_suspend` / `rt_ipc_resume` 这一对函数，五种 IPC 的阻塞/唤醒路径一通百通。

> 【对照】 FreeRTOS 的"等"挂在 `xTasksWaitingToReceive` / `xTasksWaitingToSend` 两条名单上（F4），且**默认按优先级排序**。RT-Thread 给你两种秩序的选择：PRIO 是实时系统标配，FIFO 用在"不希望高优先级任务总抢头"的场景。

二、信号量与互斥量：take/release 与继承发生点

信号量 = 计数令牌

`rt_sem` 的核心字段是 `value`（当前可用令牌数）和 `max_value`（上限）。`rt_sem_take`：`value` 大于 0 减一返回；为 0 就把自己挂进 `suspend_list` 等到点。`rt_sem_release`：`value` 加一，若有人在等就 `rt_ipc_resume` 一个。ISR 纪律同 FreeRTOS：`rt_sem_release` 可在中断里调，`rt_sem_take` 不可，ISR 没有"等"的资格。

互斥量 = 带持有者与继承的令牌

`rt_mutex` 比信号量多两个灵魂字段：`owner`（当前持有线程）和 `original_priority`（继承前的原优先级）。`rt_mutex_take` 失败时走继承路径：若 owner 的优先级比当前线程低，把 owner 临时抬到当前优先级，原值存 `original_priority`；`rt_mutex_release` 走镜像路径回落。

> 【对照 F5】 RT-Thread 的继承源码与 FreeRTOS 的 `xTaskPriorityInherit`（F5 第三节）是同一个解法的两种语言：判（持锁者更低）→ 改（升优先级）→ 唤醒路径调整。火星 bug 的解法在两个内核里长得几乎一样。读 `ipc.c`（以 RT-Thread 官方源码为准）对照 `tasks.c` 的 `xTaskPriorityInherit` / `xTaskPriorityDisinherit`，最有味道的对照学习。

三、事件集：32 位旗语 + AND/OR + CLEAR

`rt_event` 的核心是一段 32 位 `set` 字段，每位代表一个事件。`rt_event_recv` 等一组事件，支持两种模式：

- **RT_EVENT_FLAG_AND**：32 位里**所有指定的位**都置位才醒（"全部条件满足"）；
- **RT_EVENT_FLAG_OR**：**任意一位**置位即醒（"任一事件发生"）；
- **RT_EVENT_FLAG_CLEAR**：醒时顺便把命中的位清零（"消费式"）；不带 CLEAR 则位保持（"广播式"）。

> 【对照】 这套语义与 FreeRTOS 事件组（`xEventGroupWaitBits`）几乎一致：`pdTRUE` 对应 CLEAR，`pdFALSE` 对应不清零；`xAnd` 对应 AND，`xOr` 对应 OR。**API 名不同，语义同**：迁移时逐参数对照，别凭名字猜。

四、邮箱 vs 消息队列：定长池 vs 变长队

维度	邮箱 `rt_mailbox`	消息队列 `rt_messagequeue`
单条大小	**固定 4 字节**	调用方指定（变长）
拷贝成本	一次 4 字节拷贝	整条按 size 拷贝
典型用途	传指针/句柄/小型枚举值	传结构体/帧/字符串
内存来源	创建时申请 `size * 4` 字节环形池	创建时申请 `size * msg_size` 字节环形池

邮箱固定 4 字节是设计取舍，不是缺陷：它专门服务"传一个指针或一个句柄"的场景，32 位平台上指针正好 4 字节，写邮件就是把指针塞进去，对方拿到的就是指针本身（指向的对象不拷贝）。要传结构体请用消息队列（队列会按你给的 `msg_size` 整条拷贝）。

两者背后都是**环形缓冲**：send 写头、recv 读尾，满了发送方挂起、空了接收方挂起，挂起同样走 `rt_ipc_list_suspend`，再次印证"一副骨架"。

> 【对照 F4】 FreeRTOS 的 `xQueue`（F4）只有"变长队列"一种形态。RT-Thread 把"定长 4 字节"单独拎出来叫邮箱，背后是更明确的"传指针/句柄"语义。FreeRTOS 想做同样的事得自己约定 item size=4。

五、反转复现 RTT 版：E04 双跑

把 E04 的三任务剧本在 RT-Thread 工程上跑两遍：`rt_sem`（当"锁"用）版 vs `rt_mutex` 版。

预期：信号量版同样上演死亡螺旋：H 等锁、L 持锁、M 狂跑把 L 挤下台；互斥量版继承一启动，L 被抬到 H 的优先级，M 立刻让路。**两份日志并排贴**，与 FreeRTOS 版逐行对比：

- 等待时间是否一致？继承点是否在同一个相对位置？
- 不同的是 RT-Thread 的 `list_thread` 能直接打印优先级变化的现场：继承瞬间 L 的 priority 列从 1 变 3，还锁后回落 1，**优先级继承在终端上看得见**，这是 RT-Thread 可观测性的额外加分。

六、finsh 观测：list_* 系列为什么万能

`list_sem`、`list_mutex`、`list_event`、`list_mailbox`、`list_msgqueue`，一共五条 finsh 命令，一行命令列出全系统该类型的所有 IPC 对象，含名称/计数/持有者/挂起线程数。

为什么这么爽：所有 IPC 都继承自 `rt_object`（R0），对象容器按类型统一登记：`list_*` 命令就是遍历对应类型的容器链表逐条打印。这是"万物皆对象"在 IPC 篇的直接兑现：FreeRTOS 要你自己维护一份句柄表，RT-Thread 内核替你管。

> **quick win 落地**：刚 create 完一个邮箱，立刻 `list_mailbox` 看到它在册、挂起线程数=0；再 `list_thread` 看主线程阻塞在 `rt_mb_recv` 上，两屏对照，IPC 的工作状态一目了然。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **一副骨架五张皮：令牌、带主令牌、旗语、定长池、变长队；排队要么 FIFO 要么 PRIO，创建时一锤定音。**

延伸：RTT IPC 五种武器动画见 R2 动画；FreeRTOS 队列/信号量对照见 F4 和 F5；双 OS IPC 对比见 X0。

实物实验

- E04 RT-Thread 版：与 FreeRTOS 版同剧本，两组日志并排贴，继承机制的行为一致性/差异亲眼核对。

常见坑

- **事件集 flag 混用接收选项**：RT_EVENT_FLAG_AND/OR + CLEAR 组合语义，与 FreeRTOS 事件组几乎一致但 API 名不同，迁移时逐参数对照。
- **邮箱当消息队列用**：邮箱固定 4 字节（指针/句柄专用），塞结构体请用消息队列。
- **IPC 对象在中断里的权限**：ISR 只能 send/release，不能 take（同 FreeRTOS 纪律）。
- **delete 时还有线程挂着**：删除前确保无人等待，否则唤醒路径指向已释放内存（RT-Thread 会报警告/断言，别忽略）。
- **互斥量 release 也别在 ISR 调**：信号量 release 可在 ISR，但互斥量 release 涉及继承回落，ISR 里没有"持有者"概念，整条互斥量路径都是禁区，别和信号量的纪律混。

短自测

1. 五种 IPC 共用哪条数据结构？为什么这套设计能让 `rt_ipc_list_suspend/resume` 通吃？

> ▼ **参考答案**
> 共用 struct rt_ipc_object 的 suspend_list（一条 rt_list_t 双向链表）。五种 IPC 的"等"都是"把自己挂进这条链表等到点被唤醒"，差别只在"等什么、醒了怎么判断"，所以挂起/唤醒这对操作可以通用。

2. `RT_IPC_FLAG_FIFO` 与 `RT_IPC_FLAG_PRIO` 的区别是什么？什么时候选 FIFO？

> ▼ **参考答案**
> PRIO 按等待线程的优先级排序唤醒（高优先级先拿），FIFO 按进入顺序先到先得。实时系统默认 PRIO；FIFO 用在"不希望高优先级任务总抢头"或"想保证低优先级任务也公平拿到资源"的场景。一旦创建不再改。

3. 互斥量的继承与 FreeRTOS 的 `xTaskPriorityInherit` 同构，请说出对应字段与三步动作。

> ▼ **参考答案**
> RT-Thread 的 owner 对应 FreeRTOS 的 xMutexHolder；original_priority 存继承前的原优先级；继承动作都是"持锁者优先级低于等待者就升到等待者级别"，还锁时走镜像路径回落原值。落点对照：ipc.c（以 RT-Thread 官方源码为准）vs tasks.c 的 xTaskPriorityInherit/xTaskPriorityDisinherit。

4. 邮箱为什么固定 4 字节？什么场景下用它？

> ▼ **参考答案**
> 32 位平台指针正好 4 字节，邮箱专门服务"传指针/句柄/小型枚举值"，把指针塞进去，对方拿到的就是指针本身（指向对象不拷贝）。要传结构体/帧/字符串请用消息队列（按指定 msg_size 整条拷贝）。

5. RT-Thread 的事件集与 FreeRTOS 事件组语义几乎一致，迁移时容易踩什么坑？

> ▼ **参考答案**
> API 名不同：RT-Thread 用 rt_event_recv/send，FreeRTOS 用 xEventGroupWaitBits/SetBits，但参数语义对应：RT-Thread 的 RT_EVENT_FLAG_AND/OR 对应 xAnd/xOr，RT_EVENT_FLAG_CLEAR 对应 pdTRUE（消费式清零）。逐参数对照，别凭名字猜语义。

对照表：本章概念 → 源码落点

概念	落点（以 RT-Thread 5.x 官方源码为准）
rt_ipc_object 通用骨架	`include/rtdef.h`（rt_ipc_object）+ `src/ipc.c`（rt_ipc_list_suspend/resume）
FIFO/PRIO 秩序	各 IPC 创建函数的 flag 参数（rt_sem_create 等）
互斥量继承	`src/ipc.c` rt_mutex_take/release vs F5 tasks.c xTaskPriorityInherit
事件集 AND/OR+CLEAR	`src/ipc.c` rt_event_recv/send（与 FreeRTOS 事件组对表）
邮箱 4 字节定长	`src/ipc.c` rt_mb_send/recv（环形 4 字节池）
消息队列变长	`src/ipc.c` rt_mq_send/recv（按 msg_size 环形拷贝）
finsh list_* 万能	R0 对象容器遍历；list_sem/mutex/event/mailbox/msgqueue
反转实验	E04 双 OS 双跑

延伸阅读

`rt_mutex` 优先级继承的理论出处：

- **[D2]** Sha, Rajkumar, Lehoczky 1990 — 继承协议的原始论文；记住它不防死锁，天花板协议才防。

你做到了

- IPC 从"五个 API 要背"变成"一副骨架推演"；
- 双 OS 反转实验双跑——你的证据链比教科书还硬。

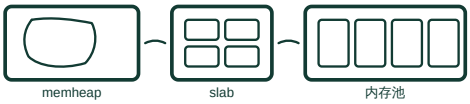
骨架只有一副：一个对象头、一份资源或存储、一条挂起列表，五件兵器是同一副骨头换皮。往后选型先问场景要什么，再问这件兵器披的是哪层皮。

✅ 下一站：R3 内存管理：memheap/slab/TLSF 三选与内存池的确定性之美。

通往单片机之路第 48 章

R3 RT-Thread 内存管理：确定性之美

难度 ★★★☆☆ · 约 35 分钟



本章罗盘

问题

实时路径上分配内存凭什么不能随手 rt_malloc，TLSF 的「近 O(1)」与内存池的 O(1) 差在哪？

交付

rt_mp_alloc 与 rt_malloc 同申请量各跑 1 万次的 rt_tick 耗时对比记录 + memheap/slab/TLSF/内存池按「规模×确定性×碎片」的分治选型表，外加池块用尽时 timeout 挂起等待的行为观察。

边界

「近 O(1)」是复杂度论证、碎片有界的证明在 D3 论文，本章不给实测数字；源码以 master 为准不钉行号；FreeRTOS 侧只对照到 heap_1~5 能力阶梯（F7），其无内置内存池的补救方案不在本章展开。

依据锚点 `f8 · d3 · d4 · mcu-anim`

> 🎯 FreeRTOS 问你"选 heap 几"，RT-Thread 问得更细：通用堆要确定性（TLSF）还是要简单（memheap）？内核小对象要不要专用快车道（slab）？实时路径上干脆别用堆，用**内存池**（分配时间是常数）。

本章精髓

1. 三套堆策略的场景分治：memheap（多地址段拼接，简单够用）、slab（内核对象的小块高速缓存，碎片免疫）、TLSF（Two-Level Segregated Fit，分配/释放都近 O(1)，大内存与实时性的双优生）。
2. 内存池（rt_mp）是实时的答案：固定块大小+空闲链表，分配=摘链表头，O(1) 且无碎片；代价是块大小一刀切（大材小用浪费，小了装不下）。
3. 静态内存潮：RT-Thread 与 FreeRTOS 一样支持"全静态创建"，安全关键场景连堆都可以不要（RT_USING_HEAP 关掉）。

- **能理解**：为什么 TLSF 近 O(1) 而不是绝对 O(1)；为什么 slab "碎片免疫"；为什么内存池没有碎片但块大小一刀切；为什么 RT_USING_HEAP 可关。
- **能用**：给"内核对象/通用大块/实时路径"三类场景各选一套；用 rt_mp_alloc vs rt_malloc 的耗时差把"确定性"量化成数字。

先修

- F7 heap（对照锚点）、C1。

先跑起来（10 分钟 quick win）

同一申请量分别用 `rt_malloc` 与 `rt_mp_alloc` 各跑 1 万次，rt_tick 差打印：内存池的确定性，数字自己会说话。

烧板之前先押个平不平：同样的申请量各跑一万次，rt_malloc 和 rt_mp_alloc 的耗时线，哪条平得像尺子？押完看 rt_tick 差，数字自己作证。

动画：内存池的确定性

块大小一刀切，空闲链表串起；alloc 摘链头、free 挂链头，没有搜索没有分裂，O(1) 从结构里长出来。右上的耗时形态对照把"恒定 vs 波动"演成两根柱子。

RT-Thread 内存池：块大小一刀切，换 O(1) 的确定性

小节结构

小节	内容	四件套
三套堆	memheap/slab/TLSF 场景分治表	库解析
TLSF 直觉	两级位图怎么做到近 O(1)（与 RTT 调度二级位图同源思想！）	库解析
内存池	固定块池的结构与 O(1) 证明；块大小设计	代码分析
静态创建	object init 系列 API；关 HEAP 的固件形态	配置
泄漏与监控	list_memheap/rt_memory_info；与 F7 监控法对照	代码分析

一、三套堆：场景分治表

堆	适用内存规模	确定性	碎片	典型用途
memheap	多地址段拼接	中（链表管理）	中	多段物理内存的统一管理，简单够用
slab	内核小对象	高（定长块）	**免疫**	内核对象（thread/timer/sem）高速缓存
TLSF	大内存	**近 O(1)**	低（按大小分级，合并友好）	通用大块分配，实时性双优

三者可同时存在，各管各的"小地盘"：memheap 管拼接的多段物理内存，slab 在其上为内核常用小对象开"快车道"，TLSF 面向应用的大块通用堆。

> 【对照 F7】 FreeRTOS 给你五份简历（heap_1~5），按"释放/合并/多区"递进（F7）；RT-Thread 给你三套正交的策略，按"用途"分治：一个走阶梯，一个走分治。

二、TLSF 直觉：两级位图怎么做到近 O(1)

TLSF = **Two-Level Segregated Fit**（两级分离适配）。第一级按 2 的幂划大块区间（如 16/32/64/128...），第二级在每个大区间内再细分子档位。

- **第一级位图**：一串 bit 标记"哪些大小档有空闲块"，查 CLZ 一次定位到最小够用的档；
- **第二级位图**：在该档内再细分几档，再查一次位图锁定具体子档；
- **取块**：摘下该子档链表头，O(1)；**释放**：检查前后邻块是否空闲，空闲就合并，合并代价也近 O(1)。

为什么是"近 O(1)"而不是绝对 O(1)：合并时查邻块状态需要 O(1)，但极端情况下位图扫描的常数较大；TLSF 的工程价值在于**最坏情况下的常数上界被压住**，对实时系统这就是"足够好的确定性"。

> 【同源】 RT-Thread 调度的二级位图（`rt_thread_ready_priority_group` + `rt_thread_ready_table[32]`，见 R1）和 TLSF 的两级位图是同一个思想的两副面孔：**位图+CLZ 把"找最值"压成常数级**。调度器用它找"最高优先级就绪线程"，TLSF 用它找"最小够用的空闲块"。同一个工程美学，读 R1 的位图再回来看 TLSF，你会感觉到"这套招式我见过"。

三、内存池：固定块池与 O(1) 证明

`rt_mp`（memory pool）是实时的答案。结构上：

- 一段连续内存被切成 N 个**等大**的块；
- 空闲块串成一条空闲链表；
- `rt_mp_alloc`：摘链表头，O(1)，**没有查表、没有合并、没有位图扫描**；
- `rt_mp_free`：放回链表头，O(1)。

碎片免疫的来源：所有块同样大，归还的块和被请求的块必然同尺寸，不存在"小请求塞不进大洞"或"大请求拼不上小洞"的问题。代价就是**块大小一刀切**：池块按"最大成员+对齐"定，宁大勿小，放不下时 alloc 直接失败或挂起等待（看 timeout 参数）。

`rt_mp_alloc` 的 timeout 参数：池块用尽时**可以挂起等待**（不像 rt_malloc 直接返回 NULL）。这把内存池变成了"资源排队"的 IPC 化形态。挂起同样走 `rt_ipc_list_suspend`，与 R2 的 IPC 骨架同源。

> 【对照 **FreeRTOS**】 FreeRTOS 没有内置内存池，你得自己用 `pvPortMalloc` 拼一个。RT-Thread 把内存池做成内核一等公民，是因为实时路径（中断下半部、DMA 缓冲）对"分配时间必须确定"的要求比"省内存"更高。

四、静态创建：object init 系列

RT-Thread 与 FreeRTOS 一样支持"全静态创建"：

- 动态版：`rt_sem_create` / `rt_thread_create` / `rt_mp_create`，内核向堆要内存；
- 静态版：`rt_sem_init` / `rt_thread_init` / `rt_mp_init`，调用方自备内存（静态数组），内核只填字段。

`RT_USING_HEAP` 这个 Kconfig 选项一关，动态创建路径直接编译不进来，固件变成"零堆"形态，安全关键场景（IEC 61508/SIL）的标配。**整个应用用静态创建 + 静态任务栈**，堆的痕迹从你的固件里消失。

> 【对照 **F7**】 FreeRTOS 的 `configSUPPORT_DYNAMIC_ALLOCATION=0` 走同一条路（F7）。两套 OS 都承认"动态分配的便利"和"静态分配的确定性"是产品形态的二选一，而不是某个永远更好。

五、泄漏与监控：list_memheap 与两个指标

finsh 提供：

- `list_memheap`：列出所有 memheap 段的总量/已用/最大连续块；
- `list_memp`（或等价命令，**以 RT-Thread 官方源码为准**）：内存池的总量/空闲块数；
- `rt_memory_info` 系列：更细的堆统计。

监控的两个指标要分清（与 F7 同名同义）：

- **当前空闲**：现在还剩多少；
- **历史最低水位**：最紧张时刻还剩多少，回答"最坏多紧张"，**不是碎片率**。

碎片率要自己估：把"最大连续块 / 总空闲"当近似指标，长期产品跑下来这个比值持续走低，说明堆在变碎，这是"产品寿命"的预警线。

> 【对照 **F7 监控法**】 `xPortGetMinimumEverFreeHeapSize`（heap_4/5）等价于 RT-Thread 的历史最低水位；`xPortGetFreeHeapSize` 等价于当前空闲。指标含义一致，API 名不同，逐项对照见下方对照表。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **通用堆选 TLSF，内核小块走 slab，实时路径上内存池；池子 O(1) 无碎片，就是块大不能挑。**

实物实验

- quick win 耗时对比 + 故意把池块用尽看 `rt_mp_alloc` 挂起等待的行为（超时参数），资源池排队的真实体感。

常见坑

- **块大小拍脑袋**：池块按"最大成员+对齐"定，宁大勿小，放不下时 alloc 直接失败。
- **内存池里放变长数据**：变长需求回 TLSF，别硬塞池子。
- **混用 alloc/free 家族**：rt_malloc 与 rt_mp_free 交叉释放=内存灾难，谁家的孩子谁领走。
- **TLSF 当万能药**：小内存（<几 KB）场景它的元数据占比过高，小堆 memheap 更划算。
- **slab 当通用堆用**：slab 是内核小对象的快车道，应用大块请走 TLSF/memheap，硬塞变长数据会让 slab 的"碎片免疫"失效。

短自测

1. 为什么说 slab 是"碎片免疫"？

> ▼ **参考答案**
> slab 为每种固定尺寸的对象开独立"快车道"：同尺寸块归还后和被请求尺寸完全匹配，不存在"小请求塞不进大洞"或"大请求拼不上小洞"。外部碎片不会产生，内部碎片被"同尺寸群体"摊薄。

2. TLSF 的"近 O(1)"为什么不是绝对 O(1)？

> ▼ **参考答案**
> 位图扫描和邻块合并都是常数级操作，但常数较大且边界情况下位图扫描的步数不是严格固定。TLSF 的工程价值是"最坏情况常数上界被压住"，对实时系统这就足够当成"确定性"用。

3. 内存池的 O(1) 是怎么做到的？代价是什么？

> ▼ **参考答案**
> alloc 摘空闲链表头、free 放回链表头，无查表无合并无位图扫描，所以 O(1)。代价是块大小一刀切：按"最大成员+对齐"定，宁大勿小，放不下 alloc 直接失败或挂起等待。变长需求回 TLSF。

4. `RT_USING_HEAP` 关掉后，哪些 API 不能再调用？固件怎么写？

> ▼ **参考答案**
> 所有 *_create 系列动态创建路径（rt_sem_create/rt_thread_create/rt_mp_create 等）编译不进来。固件要全静态：用 *_init 系列自备内存创建对象，任务栈用静态数组，零堆形态适合安全关键场景。

5. RT-Thread 的 TLSF 与调度器的二级位图有什么思想上的同源？

> ▼ **参考答案**
> 两者都用"位图+CLZ 把'找最值'压成常数级"：调度器用二级位图找最高优先级就绪线程（rt_thread_ready_priority_group + rt_thread_ready_table），TLSF 用两级位图找最小够用的空闲块。同一个工程美学，两副面孔。

对照表：本章概念 → 源码落点

概念	落点（以 RT-Thread 5.x 官方源码为准）
memheap 多段拼接	`src/memheap.c`
slab 内核小对象	`src/slab.c` （按对象类型分快车道）
TLSF 两级位图	`src/tlsf.c` （TLSF 算法实现）
内存池 rt_mp	`src/mempool.c` （rt_mp_create/init/alloc/free）
静态创建	`*_init` 系列 API（rt_sem_init/rt_thread_init/rt_mp_init）
RT_USING_HEAP 关堆	Kconfig 选项；关掉后 *_create 不可用
监控命令	finsh list_memheap/list_memp/rt_memory_info
二级位图同源	TLSF 两级位图 vs R1 调度二级位图
对照 FreeRTOS	F7 heap_1~5 vs 三套分治

延伸阅读

三套策略背后的论文：

- **[D3]** Masmano et al. 2004 — TLSF 原始论文：两级位图索引、O(1) 分配/释放、碎片有界的证明。
- **[D4]** Wilson et al. 1995 — 小内存堆、slab、memheap 三套策略背后的通用权衡。

你做到了

- 内存管理的"确定性"维度被量化；
- 两套 OS 的内存哲学在你脑中完成对照。

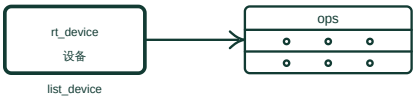
平的那条线姓 mp：固定块、空闲链表，摘头就走，一万次一个价。抖的那条不是坏，是通用堆在替灵活性付账；确定性，是要用一刀切的块大小换的。

✅ 下一站：R4 设备框架，RT-Thread 最灵魂的一章：驱动与应用如何解耦。

通往单片机之路**第 49 章**

R4 设备框架：驱动与应用的解耦术

难度 ★★☆☆☆ · 约 55 分钟



本章罗盘

问题

换一块板，应用层凭什么可以一行不动，ops 表和「按名取柄」怎么把驱动做成可插拔商品？

交付

一个能被 list_device 看见的自写设备全流程（蜂鸣器：rt_device 继承 + ops 表 + INIT_DEVICE_EXPORT 注册） + PIN 三行点灯与 S3 寄存器版的并排对照 + GET_PIN('C',12)=44 的编号推算。

边界

结构与行为以 RT-Thread 5.x components/drivers 为准、不钉行号；rx_indicate 的调用上下文因配置而异，本章按中断上下文立「只搬运不干活」的规矩；纳秒级时序（WS2812 类）应绕过框架直写寄存器，该路径实现不在本章；FreeRTOS 无框架的替代范式在 F4。

依据锚点 `f8 · e7 · mcu-anim · mcu-lab`

> 🎯 裸机写应用，`uart_send()` 里全是寄存器；换块板子，应用层推倒重来。设备框架说：应用只跟"设备句柄"说话：`rt_device_open/read/write`，底下是 USART1 还是 UART5，应用不关心。**这就是驱动与应用解耦**，也是 RT-Thread 区别于"裸内核"的核心资产。

本章精髓

1. 设备=对象+操作表：`struct rt_device` 继承对象头，挂一张 ops 表（init/open/close/read/write/control）：驱动作者实现这张表，应用作者按统一 API 调用（与 Linux 字符设备的 file_operations 一脉相承）。
2. 注册与查找两分离：驱动 `rt_device_register(dev, "uart1")` 挂进对象容器；应用 `rt_device_find("uart1")` 按名取柄：**名字是接口，实现可替换**。
3. 框架之上的"设备类型"再封装：PIN（GPIO）、UART（串口带缓冲/回调）、I2C/SPI bus，越往上层，应用越无感；finsh 的 `list_device` 一屏看全家。

- **能理解**：为什么 register 和 find 要两分离而不是"创建即用"；为什么 PIN 不直接用 `rt_device_read/write` 而单独一套 `rt_pin_*`；为什么 UART 的 `rx_indicate` 在中断上下文而不是线程。
- **能用**：PIN 三行点灯、UART 中断接收回显、把蜂鸣器封成一个能被 `list_device` 看见的 `rt_device`。

先修

- R0 对象模型、S7 USART、F4 队列。

先跑起来（10 分钟 quick win）

标准版工程上 `list_device` 找到 "pin"，三行代码点 RGB 灯：

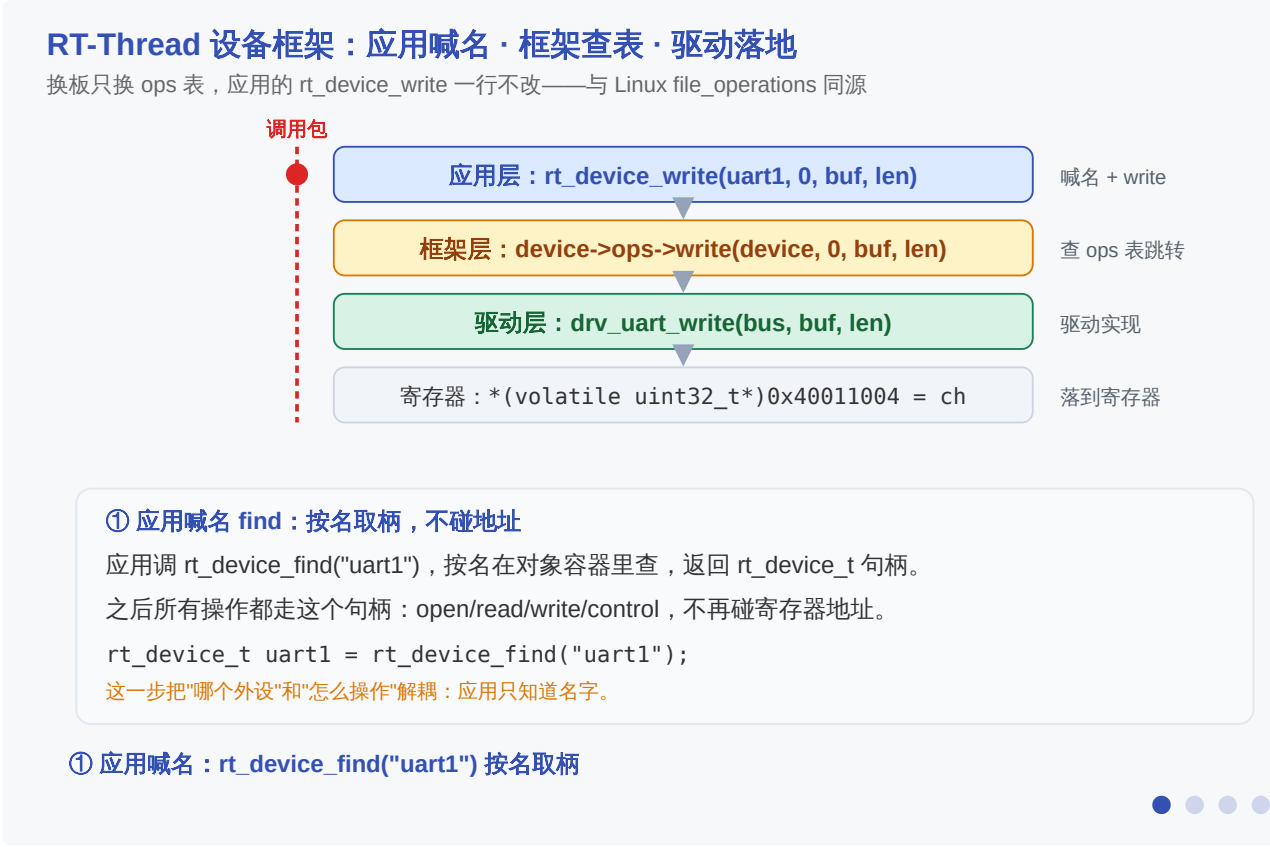
```
rt_pin_mode(LED_R, PIN_MODE_OUTPUT);
rt_pin_write(LED_R, PIN_LOW);   /* 共阳：低电平亮 */
```

不用查任何一个寄存器，体会封装的甜。

先押个翻译官：应用那一行调用落到板上，变成谁的动作？寄存器直写在哪个环节被换的手？押完看下一节的调用链动画。

动画：设备框架调用链

一次 `rt_device_write` 从应用出发，穿过框架 API、ops 表、驱动，最终落到寄存器：四层各司其职，换板只换 ops 表，应用纹丝不动。



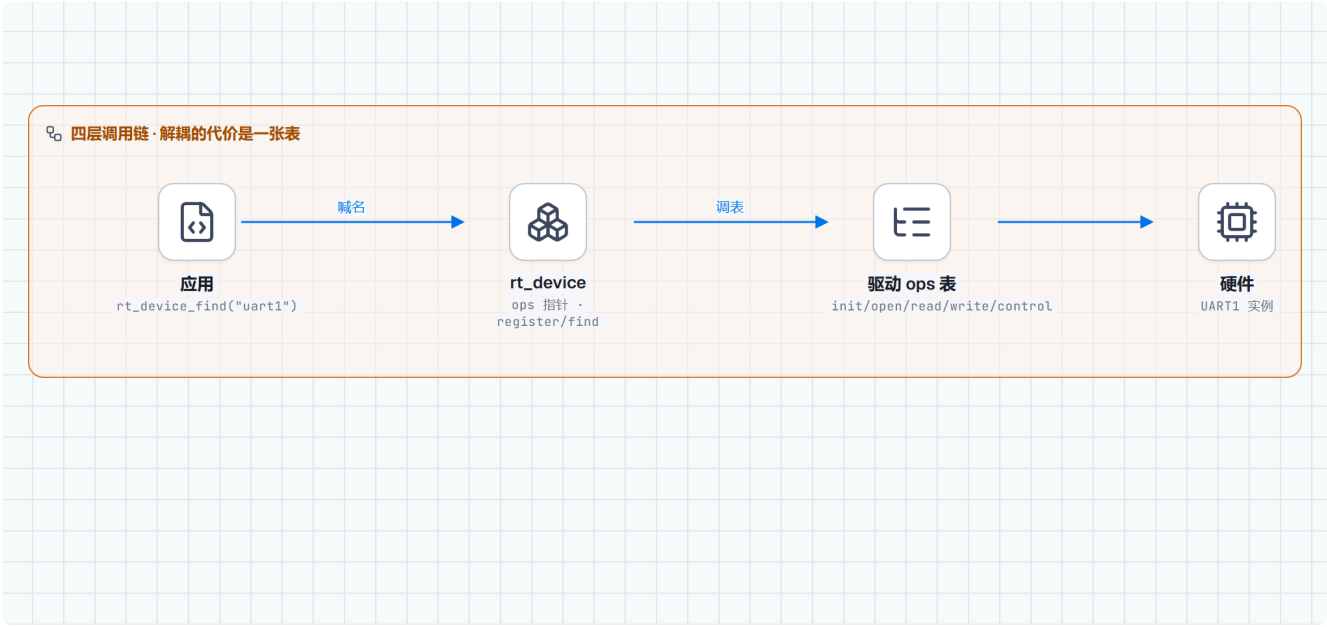
R4 设备框架调用链：应用 → 框架 → 驱动 → 寄存器

小节结构

小节	内容	四件套
调用链全图	应用/框架/驱动/寄存器四层解剖	库解析
rt_device 解剖	ops 表逐字段；register/find/open 源码路径	库解析
PIN 设备	编号体系（GET_PIN 宏）；点灯对照实验	配置
UART 设备	接收回调+缓冲模型；与手写环形缓冲对照	代码分析
I2C/SPI bus	总线设备与"挂在总线上的从设备"两级模型	配置
自己写一个设备	把蜂鸣器封装成 rt_device 的全流程	代码分析

一、调用链全图：四层各司其职

调用链先收进一张图：应用喊名、对象调表、驱动执行。



设备框架调用链：应用 rt_device_find、rt_device 调 ops 表、驱动落到硬件

一次 `rt_device_write` 从应用到寄存器，穿过四层：

```
应用层    rt_device_find("uart1") → rt_device_write(uart, 0, buf, n)
    ↓ 名字是契约
框架层    device.c：rt_device_write 查 ops->write，分发到底
    ↓ ops 表是契约
驱动层    BSP uart 驱动：ops->write 把字节塞进 USART_DR，等 TXE
    ↓ 寄存器是契约
寄存器层  USART1->DR / SR / BRR —— 硬件真相
```

每一层只认识**下一层的接口**，不关心再往下：应用不知道是 USART1 还是 UART5，框架不知道驱动写的是哪个寄存器，驱动知道寄存器但不关心应用在哪个任务里。**这就是解耦的物理实现：一张 ops 表 + 一个名字。**

把这张图和 Linux 字符设备并排看：`file->f_op` 就是 ops 表，`register_chrdev` 就是 register，`/dev/uart1` 就是 find 的名字：**RT-Thread 把 Linux 那套搬到了单片机上，只是尺度小了一档。**

二、rt_device 解剖：ops 表 + register/find

5.x 主线里 `struct rt_device` 继承 `rt_object`（RO 的对象头），核心字段：

```
struct rt_device
{
    struct rt_object parent;            /* 继承对象头：名字、类型、链表节点 */
    enum rt_device_class_type type;     /* 字符/块/网口/MTD/... */
    rt_uint16_t flag;                   /* 能力标志：可读/可写/可中断... */
    rt_uint16_t open_flag;               /* 当前打开标志 */
    rt_uint8_t  ref_count;               /* 打开计数 */
    rt_uint8_t  device_id;
    rt_err_t (*rx_indicate)(rt_device_t dev, rt_size_t size);  /* 收到数据回调 */
    rt_err_t (*tx_complete)(rt_device_t dev, void *buffer);    /* 发完回调 */
    const struct rt_device_ops *ops;    /* 操作表：解耦的载体 */
    void *user_data;
};
```

操作表（与 Linux `file_operations` 一一对应）：

```
struct rt_device_ops
{
    rt_err_t  (*init)   (rt_device_t dev);
    rt_err_t  (*open)   (rt_device_t dev, rt_uint16_t oflag);
    rt_err_t  (*close)  (rt_device_t dev);
    rt_size_t (*read)   (rt_device_t dev, rt_off_t pos, void *buffer, rt_size_t size);
    rt_size_t (*write)  (rt_device_t dev, rt_off_t pos, const void *buffer, rt_size_t size);
    rt_err_t  (*control)(rt_device_t dev, int cmd, void *args);
};
```

三个动作看清生命周期：

- **register**（`rt_device_register(dev, "uart1", flag)`）：把设备按 type 挂进对象容器（R0 的容器链表），名字写进 `parent.name`：**从此全局可见**。
- **find**（`rt_device_find("uart1")`）：在容器链表上线性比对名字，返回 `rt_device_t`：**名字是契约**，所以 BSP 与应用必须用同一套命名惯例（`uart1` / `i2c1` / `spi10`）。
- **open/read/write/control/close**：`rt_device_open` 让 `ref_count++` 并调 `ops->open`；`rt_device_read` 直接转发 `ops->read`，框架本身不搬数据，只分发。

为什么 register/find 两分离而不是"创建即用"：register 是驱动侧的事（板子启动时调一次），find 是应用侧的事（任意时刻按名取柄）。两分离意味着驱动可以晚于应用加载、应用可以不知道驱动在哪，这正是软件包"即插即用"的根基。

三、PIN 设备：编号体系与点灯对照

PIN 是设备框架之上的一层"便捷壳"：底层仍是一个 char 设备，但 `rt_pin_*` 这套 API 走的是 `struct rt_pin_ops`，比通用 `rt_device_read/write` 更贴近 GPIO 语义：

```
struct rt_pin_ops
{
    void (*pin_mode) (rt_device_t dev, rt_base_t pin, rt_uint8_t mode);
    void (*pin_write)(rt_device_t dev, rt_base_t pin, rt_uint8_t value);
    int  (*pin_read) (rt_device_t dev, rt_base_t pin);
    rt_err_t (*pin_attach_irq)(rt_device_t dev, rt_int32_t pin, rt_uint8_t mode,
                               void (*hdr)(void *), void *args);
    rt_err_t (*pin_detach_irq)(rt_device_t dev, rt_int32_t pin);
    rt_err_t (*pin_irq_enable)(rt_device_t dev, rt_base_t pin, rt_uint8_t en);
    rt_base_t (*pin_get)(const char *name);
};
```

编号体系是关键：所有引脚被压平成一个 `rt_base_t`，公式 `pin = port * 16 + pin_in_port`，由 `GET_PIN` 宏在编译期算出：

```
#define GET_PIN(PORTx, PIN)   (RT_PIN_##PORTx * 16 + RT_PIN_##PORTx##_##PIN)
/* 板级 board.h 里：RT_PIN_A=0, RT_PIN_B=1, ... */
/* GET_PIN('A', 5) = 0*16 + 5 = 5；GET_PIN('B', 0) = 1*16 + 0 = 16 */
```

`port*16` 而非 `port*32`：因为 STM32 一组最多 16 个脚（F407 是 PA0~PA15），4 bit 够存脚号，余下高位存组号：一个 32 位 int 装下全部引脚，便宜又够用。

点灯对照 S3 手写版：

```
/* S3 寄存器版：你得知道 LED 在哪个 GPIO、哪一位、怎么配时钟 */
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;
GPIOB->MODER |= GPIO_MODER_MODER5_0;
GPIOB->BSRR   = GPIO_BSRR_BR5;

/* R4 PIN 设备版：只知道一个编号 */
rt_pin_mode (GET_PIN('B', 5), PIN_MODE_OUTPUT);
rt_pin_write(GET_PIN('B', 5), PIN_LOW);
```

封装的甜是"不用查寄存器"；代价是多一次函数调用 + 一次 ops 分发，在 168MHz 的 F4 上是纳秒级，点灯完全无所谓；但在纳秒级脉冲生成（如 WS2812 时序）这种"每一拍都要抠"的场合，该走 RT 路径（`rt_pin_write` 的 fast 版或直接寄存器，常见坑第 4 条会再提）。

四、UART 设备：中断接收 + 回调 + 缓冲

UART 是设备框架的"招牌菜"：`serial.c` 把它注册成 char 设备，应用一套 `rt_device_*` 通吃。中断接收的标准管道：

```
static rt_device_t serial;
static struct rt_ringbuffer ring;        /* 环形缓冲 */
static rt_sem_t rx_sem;

/* 1. 接收回调（中断上下文！）—只搬运，不干活 */
static rt_err_t rx_ind(rt_device_t dev, rt_size_t size)
{
    rt_uint8_t tmp[32];
    while (size) {
        rt_size_t n = rt_device_read(dev, 0, tmp, size > 32 ? 32 : size);
        rt_ringbuffer_put(&ring, tmp, n);
        size -= n;
    }
    rt_sem_release(rx_sem);              /* 通知消费者：有货了 */
    return RT_EOK;
}

/* 2. 消费者任务 */
void uart_task(void *p)
{
    rt_uint8_t buf[64];
    rt_device_set_rx_indicate(serial, rx_ind);
    rt_device_open(serial, RT_DEVICE_FLAG_INT_RX);   /* 中断接收模式 */
    while (1) {
        rt_sem_take(rx_sem, RT_WAITING_FOREVER);
        while (rt_ringbuffer_data_len(&ring) > 0)
            rt_ringbuffer_get(&ring, buf, sizeof(buf));  /* 取出来处理 */
    }
}
```

为什么回调只搬运不干活：`rx_indicate` 在接收中断里被调（serial.c 里 RXNE 中断 → `dev->rx_indicate`），上下文是 ISR，在中断里跑业务逻辑等于"在中断里干活"，长任务会把中断延迟顶爆。标准做法是 ISR 搬数据 + 释放信号量，业务在任务里做。

三方对照同一件事"串口收数据回显"：

方案	ISR 做什么	通信机制	缓冲
S7 手写	DMA 搬 + IDLE 判帧	无（主循环查 DMA 指针）	DMA 环形
F4 队列	xQueueSendFromISR	队列（带阻塞）	队列本身就是
R4 UART 设备	rx_indicate 释放 sem	信号量 + 环形缓冲	rt_ringbuffer

三者本质同构：**ISR 只做有界搬运，业务在任务里**。R4 的不同是这套管道被框架标准化了：你不用每次手写中断 + 环形缓冲，驱动作者已经填好 ops 表，你只管 `set_rx_indicate` + 取数据。

五、I2C/SPI bus：总线设备 + 从设备两级

I2C/SPI 比 PIN/UART 多一层"总线"：**总线本身是一个设备，挂在总线上的从设备是另一类对象**。因为一条 I2C 总线上有多个从设备，共用同一对 SCL/SDA，必须由总线统一调度（谁占用、谁释放、谁的消息）。

两级模型：

```
应用     rt_i2c_transfer(bus, msgs, n)   /  rt_spi_transfer(spi_dev, ...)
   ↓
总线设备 rt_i2c_bus_device / rt_spi_bus
         - ops: master_xfer / acquire / release   ← 总线锁，防多任务抢总线
   ↓
从设备   rt_i2c_client / rt_spi_device
         - bus 指针 + 7-bit 地址(或 CS)           ← 数据带"我是谁"
```

应用调 `rt_i2c_transfer(bus, msg, n)`，传的是"总线 + 一组消息（带从设备地址）"，总线驱动负责起停条件、地址帧、ACK 收发：**应用不碰 SCL/SDA 寄存器**。`rt_i2c_bus_acquire/release` 是总线锁，多任务共用一条总线时由框架保证串行化。

这层抽象的价值：换一颗传感器（同总线、不同地址）只改一个从设备指针；换一块板（同芯片、不同引脚）只改总线驱动的寄存器部分，**应用层的 `rt_i2c_transfer` 一行不动**。

六、自己写一个设备：蜂鸣器封装

把一个无源蜂鸣器（PWM 驱动）封成 `rt_device` 的最小骨架，体验"驱动作者"视角：

```
struct buzzer_dev
{
    struct rt_device parent;     /* 继承设备头 */
    rt_uint32_t channel;          /* 哪路 PWM */
    rt_uint32_t freq;             /* 当前频率 */
};

static rt_err_t buz_init(rt_device_t dev)                      { /* 配 PWM 通道 */ return RT_EOK; }
static rt_err_t buz_open(rt_device_t dev, rt_uint16_t oflag)   { return RT_EOK; }
static rt_err_t buz_control(rt_device_t dev, int cmd, void *args)
{
    struct buzzer_dev *b = (struct buzzer_dev *)dev;
    switch (cmd) {
    case BUZZER_CMD_SET_FREQ:  b->freq = *(rt_uint32_t *)args; /* 改 PWM 分频 */ break;
    case BUZZER_CMD_OFF:       /* 关 PWM 输出 */ break;
    }
    return RT_EOK;
}

static const struct rt_device_ops buz_ops = {
    buz_init, buz_open, RT_NULL, RT_NULL, RT_NULL, buz_control
};

void buzzer_register(void)
{
    static struct buzzer_dev buz = { .channel = 1 };
    buz.parent.type = RT_Device_Class_Miscellaneous;
    buz.parent.ops  = &buz_ops;
    rt_device_register(&buz.parent, "buzzer", RT_DEVICE_FLAG_RDWR);
}
INIT_DEVICE_EXPORT(buzzer_register);   /* 文件作用域：自动初始化，回 [R0](00-arch.md) */
```

应用侧：

```
rt_device_t buz = rt_device_find("buzzer");
rt_device_open(buz, RT_DEVICE_FLAG_WRONLY);
rt_uint32_t f = 4000;  rt_device_control(buz, BUZZER_CMD_SET_FREQ, &f);
```

驱动写一次，全工程受益，这就是生态。**填表（ops）+ 喊名（find）+ 注册进容器**，三步闭环。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **驱动填表（ops），应用喊名（find），换板只换表，应用纹丝不动：解耦的代价是一张表，回报是整个生态。**

延伸：RTT 设备框架调用链动画见 R4 动画；ESP-IDF 驱动模型对照见 P6；FreeRTOS 无设备框架的设计差异见 F0。

实物实验

- PIN 设备点灯 vs E01 寄存器版：同一块板两种世界观，GDB 里追一次 `rt_pin_write` 到 BSRR 的完整下钻，封装在你眼前逐层剥落。

常见坑

- **忘了 register 就 find**：设备不在容器里，find 返回空，先查驱动入口函数与 INIT_DEVICE_EXPORT。
- **把设备名当字符串随便起**：命名是 BSP 与应用的契约（"uart1"/"i2c1" 惯例），乱起名上层软件包找不到。
- **UART 回调里干重活**：回调在中断/线程上下文因配置而异，查清 RX indicate 的调用上下文再写代码。
- **绕过框架直接摸寄存器**：绕过框架=破坏解耦契约，除非性能论证（RT 路径），否则别拆自己家地基。
- **open 时 oflag 与设备能力不符**：只读传感器用 `RT_DEVICE_FLAG_WRONLY` 打开，规范的 `ops->open` 应拒绝；但有些驱动不查 oflag，写到只读设备返回的是驱动作者的良心，读 spec 确认能力标志，别靠"能 open 就能写"的错觉。

短自测

1. 为什么"register 和 find 两分离"是解耦的关键？如果合并成"创建即可用"会丢失什么？

> **▼ 参考答案**
> register 是驱动侧（板级启动调一次），find 是应用侧（任意时刻按名取柄）。合并意味着应用必须在驱动创建时就在场、且要知道设备指针，驱动晚加载或软件包热插拔就废了。两分离让"名字"成为契约，驱动和应用解耦到只剩一个字符串，软件包才能即插即用。

2. GET_PIN('C', 12) 的编号是多少？为什么用 port*16+pin 而不是直接传 (port, pin) 两个参数？

> **▼ 参考答案**
> port C 是第 3 组（A=0,B=1,C=2），C12 = 2*16 + 12 = 44。压平成一个整数是因为 rt_pin_mode 等接口形参是 rt_base_t 单值：一个 32 位 int 装下全部引脚（16 组 × 16 脚 = 256，8 bit 够），调用约定简单、可作数组下标、可存表，比二元组好调度。

3. UART 的 rx_indicate 回调为什么不能直接在里面处理完所有业务？标准做法是什么？

> **▼ 参考答案**
> rx_indicate 在接收中断里被调，上下文是 ISR。在里面干长活会顶爆中断延迟、可能喂不上 watchdog、错过下一帧。标准做法：ISR 里只把字节搬进环形缓冲、释放一个信号量；业务逻辑在消费者任务里 sem_take 醒来后处理：ISR 只做有界搬运，业务在任务里。

4. rt_device_ops 和 Linux 的 file_operations 有哪些对应关系？为什么说"一脉相承"？

> **▼ 参考答案**
> 对应：init/open/read/write/close/control 一一映射 file_operations 的 init/open/read/write/release/ioctl；register 对应 register_chrdev；find 按名取柄对应 open(/dev/xxx) 拿到 file 的 f_op。一脉相承指"对象持一张 ops 表 + 按名分发"的同一套解耦哲学，RT-Thread 只是把 Linux 那套搬到单片机，尺度小一档。

5. PIN 设备为什么不用 rt_device_read/write 而单独一套 rt_pin_* API？这套 API 走的还是 rt_device 吗？

> **▼ 参考答案**
> PIN 语义是"引脚模式/电平/中断"，用 read/write 这种"流"语义不顺手，所以单独一套 pin_mode/pin_write/pin_attach_irq。但它底层仍是一个 char 设备：rt_pin_* 内部 find 出 pin 设备、调它挂的 rt_pin_ops，是设备框架之上的便捷壳，不是另起炉灶。

对照表：本章概念 → 源码落点

概念	落点（RT-Thread 5.x）
rt_device 结构 + ops 表	`include/rtdef.h`（struct rt_device / rt_device_ops）
register/find/open/read 分发	`src/device.c`（rt_device_register/find/open/read）
PIN 设备 + GET_PIN	`components/drivers/misc/pin.c` + BSP `board.h`
UART 设备 + rx_indicate	`components/drivers/serial/serial.c`
I2C/SPI 总线两级模型	`components/drivers/i2c/i2c_core.c` / `spi/spi_core.c`
list_device 命令	`components/finsh/msh_cmd.c`

你做到了

- 驱动模型从"Linux 那套很高级"变成"我也写得出的 ops 表"；
- RT-Thread 的生态逻辑打通：软件包为什么能即插即用，答案就在这层框架。

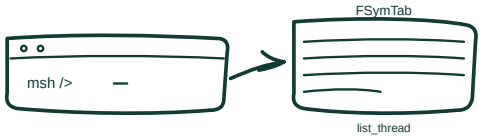
翻译官是 ops 表：应用那行调用跳进表里，落到谁家的函数就是谁的动作。换块板子，动的只是表后面那层实现，应用一行不改。

✅ 下一站：R5 finsh 控制台，在板子上跑 shell：list、ps、help 的背后。

通往单片机之路第 50 章

R5 finsh 控制台：板子上的 shell

难度 ★☆☆☆☆ · 约 25 分钟



本章罗盘

问题

敲一行 list_thread 回车就出线程表，命令是怎么被钉进固件、运行时又按名字找到的？

交付

MSH_CMD_EXPORT 宏展开等价代码（三字段结构体＋SECTION＋used）＋ map 文件里 FSymTab 段与 KEEP 界桩定位记录 ＋ 一个实测过的 led <index> <on|off> 自定义命令（含 usage 与错误返回码）。

边界

宏展开标注「字段名随版本微调」，以 RT-Thread 5.x master 为准不钉行号；RAM/Flash 是裁剪相关的量级估计（finsh 栈 2~4KB、整套 msh 约 10~20KB Flash），非实测数字；量产裁剪只给开关与攻击面提示，不给安全审计流程。

依据锚点 `f8` · `e7` · `mcu-anim`

🎯 插根串口线，板子上敲 `list_thread` 回车，线程表跃然屏上。这不是 IDE 的专利，是 finsh：RT-Thread 自带的微型 shell。它的原理就一条：**把函数名和地址编进一张符号表，运行时按名调用**。

本章精髓

1. MSH_CMD_EXPORT 的魔法：宏展开=把一个 `{名字, 描述, 函数指针}` 结构体放进专用链接段（FsymTab），链接器收齐，运行时 finsh 按名字段内查找并调用（链接段的又一神用，回 B3/R0 自动初始化）。
2. 解析器极简：按空格切词→首词查符号表→参数逐个转换（支持整型/字符串）→按原型调用，所以命令的形参列表是受约束的。
3. list 系列的实现套路：`list_thread` 就是遍历对象容器（R0）＋ 格式化打印，你会写 list，就会给自己的子系统加自检命令。

- **能理解**：为什么用链接段而不是运行时注册表；为什么命令签名必须是 `int func(int argc, char **argv)`；finsh 线程为什么需要单独预算栈而不是用主线程或空闲线程。
- **能用**：给工程加一个 `led` 命令实测；在 map 文件里定位 FSymTab 段；给自己的子系统写一个 `list_xxx` 自检命令。

先修

- R0 对象模型、S7 USART、C5 函数指针。

先跑起来（10 分钟 quick win）

串口终端连上 R7 移植工程：回车出 `msh />`，敲 `help`，全部命令清单立刻到手（自助文档系统）。

敲 help 之前先押个名册：那张命令清单存在哪、运行时又是谁按名翻它？押完去 map 文件里找 FSymTab 段，答案在链接期就排好了。

动画：finsh 命令分发四步

编译期宏钉进 FSymTab 段、链接期界桩圈地、运行期切词/查表/转换/调用：三个命令词从输入行飞向各自的步骤框，光标闪烁的终端就是板上的最小人机界面。

R5 finsh 命令分发：宏钉进段，shell 按名翻牌

小节结构

小节	内容	四件套
符号表机制	MSH_CMD_EXPORT 宏展开与 FsymTab 段	库解析
解析与调用	切词/查表/类型转换/调用四步	库解析
list 系列	对象容器遍历；给自己的子系统写 list 命令	代码分析
自定义命令	`led` 命令完整实战（参数解析+错误提示）	代码分析
裁剪与资源	finsh 的 RAM/Flash 账；生产固件关不关	配置

一、符号表机制：MSH_CMD_EXPORT 的魔法

```
/* 代表性展开（字段名随版本微调，三字段稳定） */
#define MSH_CMD_EXPORT(name, desc)                              \
    USED_CMD const struct msh_cmd_entry _cmd_##name             \
    SECTION("FSymTab") = { #name, desc, (int (*)(int, char**))name };

struct msh_cmd_entry
{
    const char *name;                       /* 名字：find 时的 key */
    const char *desc;                       /* 描述：help 时显示 */
    int (*func)(int argc, char **argv);     /* 函数指针：调用入口 */
};
```

宏做三件事：

1. 定义一个 `msh_cmd_entry` 结构体，三字段 `{名字, 描述, 函数指针}`；
2. 用 `SECTION("FSymTab")` 把它放进专用链接段；

3. `used` 防止编译器因为"没人引用"而删掉它，引用由链接段在运行时建立。

链接脚本（回 B3）划出 FSymTab 段并打起止界桩：

```
. = ALIGN(4);
__fsymtab_start__ = .;
KEEP(*(FSymTab))
__fsymtab_end__ = .;
```

`KEEP` 是关键：没有它，gc-sections 会把"没人引用"的命令全清掉（常见坑第 2 条）。运行时 finsh 在 `[__fsymtab_start__, __fsymtab_end__)` 区间遍历，按名字二分/线性查找，命中就调用 `entry->func`。**这就是 RO 自动初始化段的同一招：把"编译期已知、运行期要遍历"的东西钉进段里，链接器替你收齐。** 第三次见这个手法了（R0 INIT 段、B3 链接脚本、这里 FSymTab），你应该已经炉火纯青。

二、解析与调用：切词/查表/类型转换/调用

用户敲 `led 1 on` 回车，解析器走四步：

1. **切词**：按空格切成 `led 1 on` 三段，首词是命令名，其余是参数。
2. **查表**：拿 `"led"` 在 FSymTab 区间里按名查找（命令少时线性，多时排序后二分），命中拿到 `entry`。
3. **类型转换**：参数逐个转：字符串原样传（`"on"`），数字用 `atoi` / `strtol`（`"1"` → `1`）。所以命令形参类型受约束：finsh 只认整型和字符串，不支持浮点、结构体直接传。
4. **调用**：`entry->func(argc, argv)`：`argc=3, argv` 三段，与 C main 同型。

签名约束由此而来：**命令函数必须是 `int func(int argc, char **argv)`**（常见坑第 1 条）。原型不符（写成 `void func(void)` 或多了个参数）→ 解析器按 argc/argv 调用，栈布局对不上，一调就崩。

三、list 系列：遍历对象容器

`list_thread` 不是魔法，就是"遍历线程对象容器 + 格式化打印"。框架（R0）把所有线程挂在 `rt_object_container[RT_Object_Class_Thread]` 链表上，`list_thread` 走这张链表逐个打印：

```
int list_thread(int argc, char **argv)
{
    rt_kprintf("thread    pri  status    sp     ...\n");
    for (each tcb in thread container)
        rt_kprintf("%-8s %3d  %-8s 0x%08x ...\n",
                   tcb->name, tcb->current_priority, status_str, tcb->sp);
    return 0;
}
MSH_CMD_EXPORT(list_thread, list all threads);
```

`list_device` / `list_timer` / `list_sem` / `list_mutex` 同构，只是遍历的对象容器不同。**会写一个 list，就会给自己的子系统加自检命令**：你的驱动/中间件维护一个链表（任务、连接、缓存块），照葫芦画瓢写一个 `list_xxx`，`MSH_CMD_EXPORT` 一挂，`help` 里就有、运行时就能查，板子上的自检面板到手。

四、自定义命令：led 命令实战

把 E01 的点灯包装成 shell 命令，串口敲 `led 1 on` 就亮灯：

```
#include <rtthread.h>
#include <rtdevice.h>

static rt_base_t leds[] = { GET_PIN('B', 0), GET_PIN('B', 1), GET_PIN('B', 5) };

int led(int argc, char **argv)
{
    if (argc < 3) {
        rt_kprintf("usage: led <index 0..2> <on|off>\n");
        return -1;
    }
    int idx = atoi(argv[1]);
    if (idx < 0 || idx > 2) { rt_kprintf("index out of range\n"); return -2; }
    rt_base_t val = (strcmp(argv[2], "on") == 0) ? PIN_HIGH : PIN_LOW;
    rt_pin_write(leds[idx], val);
    return 0;
}
MSH_CMD_EXPORT(led, "led <index> <on|off> - control onboard RGB");
```

串口实测：

```
msh /> help             ← 全部命令清单
msh /> led 1 on         ← LED1 亮
msh /> led 1 off        ← LED1 灭
msh /> led 9 on         ← "index out of range"
```

注意三件事：**argc 含命令名本身**（`led` 是 argv[0]，所以 `led 1 on` 的 argc=3）；**错误路径要有提示和返回码**（用户敲错能看清）；**命令名别撞内核自带**（`help` / `list` / `ps` 等已占）。

五、裁剪与资源：RAM/Flash 账 + 生产开关

finsh 不是免费的，上板前要算账：

- **RAM**：finsh 线程（tshell）单独预算栈，`FINSH_THREAD_STACK_SIZE` 默认 2~4KB，`list` 类命令带格式化打印，栈深由 `rt_kprintf` 的格式化嵌套决定，开小了爆栈（常见坑第 3 条）。再加 RX 环形缓冲、history 缓冲。
- **Flash**：每条命令在 FSymTab 占一个 entry（名字串 + 描述串 + 函数指针 ≈ 16~24 字节），再加解析器 + list 系列代码，一整套 msh 约 10~20KB Flash（视裁剪）。

生产固件的取舍：

- **保留 msh，关 C 表达式**：`FINSH_USING_MSH_ONLY`，砍掉 VSymTab 和 C 表达式解析器，省一块 Flash，留下 `led` / `list` 这类工程命令。
- **整包关掉**：量产固件若串口对外暴露，finsh 就是攻击面，任何人接根线就能 `rt_thread_control` 改优先级、`list` 翻内存。配置 `RT_DEBUG` 或直接关 `FINSH_USING_MSH`，仅在工程版保留。

两种模式速记：

- **msh（命令行，默认）**：`MSH_CMD_EXPORT` 注册，argc/argv 调用，像 bash：轻、安全面窄。
- **finsh C 表达式（老式）**：能在 shell 里直接写 C 表达式、调任意函数（`rt_kprintf("%d", 3+4)`），靠 VSymTab 把全局变量/函数地址也编进段，重、调试强但攻击面大，量产基本不用。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **宏把函数钉进段里，shell 按名翻牌；help 即是文档，list 即是自检：finsh 是板子上的最小人机界面。**

延伸：RTT finsh 命令分发动画见 R5 动画；链接脚本段属性在 B3 详解；GDB 调试对照见 B6。

实物实验

- 自定义 `led` 命令 + 在 E04 实验里用 finsh 动态改任务优先级（`rt_thread_control` 包一个命令），现场调参不重烧。

常见坑

- **命令函数原型不符**：finsh 参数约定（int argc, char**argv）写错，调用即崩。
- **段被 gc-sections 回收**：FsymTab 段必须 KEEP，链接脚本少了它，命令"编译通过但 help 里没有"。
- **finsh 线程栈太小**：命令里调用深（如 list 带格式化打印）爆栈，finsh 栈单独预算（F7 方法）。
- **生产固件忘裁剪**：finsh 是调试利器也是攻击面，量产版 RT_DEBUG/FINSH 配置过一遍。
- **finsh 线程优先级设得比业务任务还高**：tshell 默认优先级较低是有意为之：敲命令时若它抢占业务任务，整板都在等 finsh 让出。别为了"命令响应快"把 finsh 拉到最高优先级，否则业务实时性被你亲手破坏。

短自测

1. MSH_CMD_EXPORT 的宏展开做了什么？为什么用链接段而不是运行时注册表？

> **▼ 参考答案**
> 宏定义一个 {名字， 描述， 函数指针} 结构体，用 SECTION 把它放进 FSymTab 段、用 used 防止编译器删除。用链接段而不是运行时注册表的好处：编译期就把所有命令收齐，启动即可用、无需额外 init 代码；省掉运行时注册的 RAM 开销和时序问题；与 R0 自动初始化段是同一手法。

2. 命令函数为什么必须是 int func(int argc, char **argv) 签名？签名不符会怎样？

> **▼ 参考答案**
> 解析器四步最后一步按 (int argc, char** argv) 这个原型把切好的参数压栈调用，签名是解析器和命令之间的 ABI。写成 void func(void) 或多带参数，栈布局对不上：argv 指针被当成你的"参数"、argc 落到野地址，一调就 HardFault。这是 finsh 崩溃的头号原因。

3. 链接脚本里 FSymTab 段少了 KEEP，会发生什么？

▼ **参考答案**
gc-sections 优化会以"没人引用"为由把整个 FSymTab 段删掉，因为命令函数只在段里被"按名引用"、没有直接调用点。结果：编译完全通过、烧进去 help 里一条命令都没有、敲啥都 command not found。必须 KEEP(*(FSymTab)) 把段钉死。

4. finsh 的两种模式有什么区别？为什么 msh 成为默认？

▼ **参考答案**
msh 是命令行风格（命令+参数，argc/argv 调用），轻、安全面窄；finsh C 表达式风格能在 shell 里直接写 C 表达式、调任意函数（要 VSymTab 把变量和函数都编进段），调试强但重、攻击面大。msh 成为默认是因为量产需要安全面窄和体积小，调试用得着的 C 表达式在工程版单独开。

5. finsh 线程栈为什么需要单独预算？默认大小不够时会怎样？

▼ **参考答案**
finsh 命令（尤其 list 系列）带 rt_kprintf 格式化打印，嵌套深、局部变量多，栈消耗波动大；和业务任务共用栈会互相挤爆。单独预算 FINSH_THREAD_STACK_SIZE（默认 2~4KB）才安全。不够时表现为：敲 list_thread 这种深打印就 HardFault 或栈溢出告警，且只在特定命令才暴露，调试时要把栈开大一点先验证。

对照表：本章概念 → 源码落点

概念	落点（RT-Thread 5.x）
MSH_CMD_EXPORT 宏 + msh_cmd_entry 结构	`components/finsh/msh.h`
FSymTab 段 + KEEP 界桩	链接脚本（回 B3）
切词/查表/调用解析器	`components/finsh/msh.c`（msh_exec）
list_thread / list_device	`components/finsh/msh_cmd.c`
finsh 线程 + 栈预算	`components/finsh/shell.c` + `rtconfig.h` FINSH_THREAD_STACK_SIZE

你做到了

- 板子有了"键盘鼠标"；
- 链接段技术第三次立功，你对它的运用已炉火纯青。

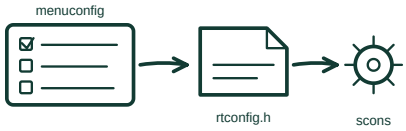
名册叫 FSymTab：编译期就把命令名和函数指针排进链接段，运行时按名翻页。以后想给自己的子系统加 list 命令，走的还是这条老路。

✅ 下一站：R6 Env 与 menuconfig，RT-Thread 的"点单系统"：scons 与 Kconfig。

通往单片机之路第 51 章

R6 Env 与 menuconfig：RT-Thread 的点单系统

难度 ★☆☆☆☆ · 约 25 分钟



本章罗盘

问题

menuconfig 勾一个 MQTT，源码凭什么自动进构建，Kconfig 到 rtconfig.h 的宏链路怎么走？

交付

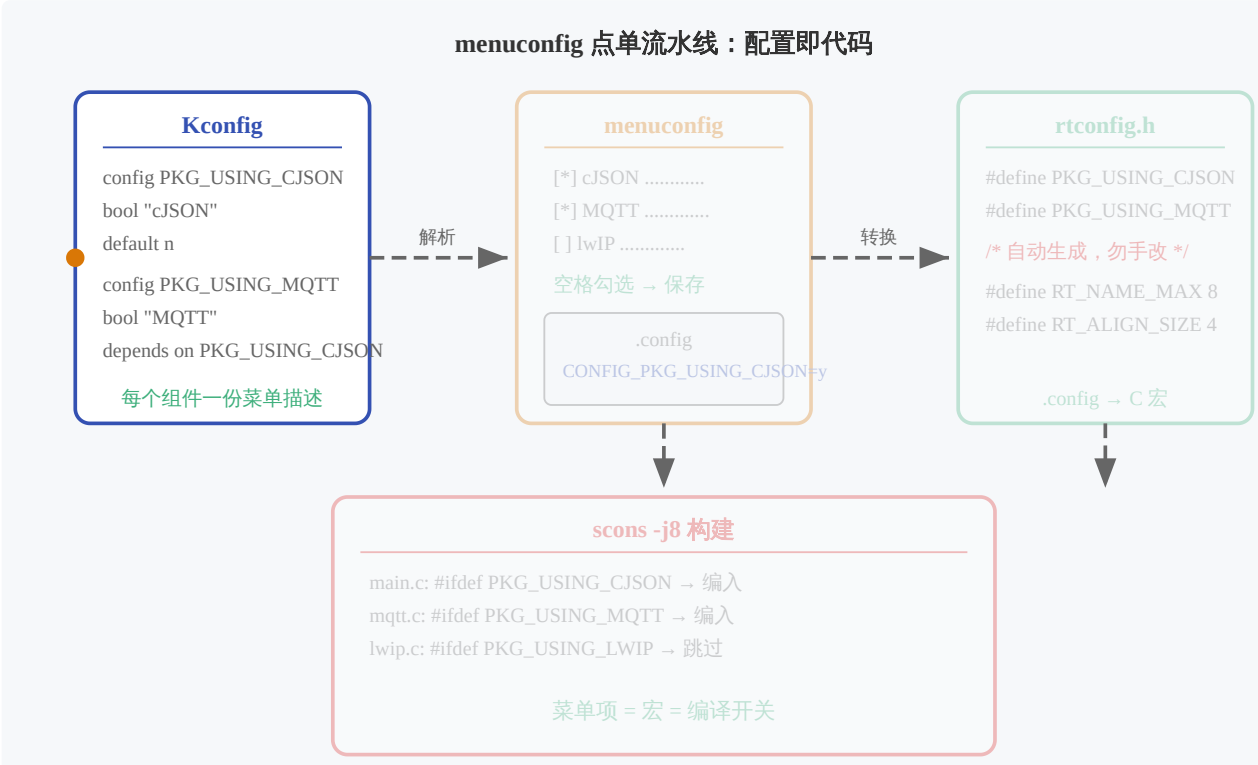
一次完整点单记录（menuconfig 勾 cJSON → pkgs --update 拉源码 → scons -j8 出固件 → 烧录解析 JSON 串口打印）+ 为自写组件补一条 Kconfig 选项并在菜单里出现，外加 BSP 三区（board/libraries/applications）职责图。

边界

本章只讲工具链路（配置与构建是正交两维），不碰内核机制；命令与菜单以当期 Env/RT-Thread master 为准，版本未固定、界面与选项会漂；scons 与 CMake 的深对照在 B7，ESP-IDF 侧只借 P3/P0 做参照不展开。

依据锚点 `f8 · e7 · mcu-anim`

🎯 RT-Thread 生态的恐怖之处：menuconfig 里勾一个"MQTT 客户端"，`pkgs --update` 一敲，源码自动下载、自动进构建——像点外卖。这背后是 Env（工具环境）+ scons（构建）+ Kconfig（配置）的三人转。



R6 menuconfig 点单流水线：Kconfig → .config → rtconfig.h → 构建

本章精髓

1. Kconfig 管"编什么"：每个组件/软件包自带 Kconfig 描述依赖与选项，`menuconfig` 生成 `.config` → `rtconfig.h`：**菜单项=宏**，代码里 `#ifdef` 应声而开（与 ESP-IDF 同源，对照 P3）。
2. scons 管"怎么编"：SConstruct/SConscript（Python）描述源码清单与依赖，`scons -j8` 出固件；`scons --target=mdk5/iar` 还能一键导出 IDE 工程：源码是单一事实源。
3. Env 是"开箱即用"的 Windows 环境：内置 Python/scons/gcc/menuconfig，装一个 Env，全站工具零配置（与 P0 的 IDF 安装器同思路）。

- **能理解**：为什么 `rtconfig.h` 是生成物不该手改；为什么 SConscript 用 Python 比纯 DSL 灵活；为什么"在 applications 里改驱动"换 BSP 会全丢。
- **能用**：menuconfig 勾选组件 → `pkgs --update` 拉源码 → `scons -j8` 出固件完整跑一遍；为一个自写组件加一条 Kconfig 选项并在菜单里出现。

先修

- B7 构建系统（CMake/scons/Kconfig 对照锚点）。

先跑起来（10 分钟 quick win）

```
cd bsp/stm32f407-zeo   # 进 BSP 目录
menuconfig             # 方向键翻菜单，空格勾选（如 cJSON），保存退出
pkgs --update          # 拉取选中软件包源码到 packages/
scons -j8              # 并行编译，出固件
```

四条命令走完：menuconfig 勾一个软件包（cJSON/MQTT/lwIP 任选），pkgs 把源码拉到 `packages/`，scons 编进固件，新组件已进镜像。配置驱动开发，第一次体感：**菜单项即宏，勾选即编译**。改了菜单一定 `scons` 重编，否则宏变了固件没变（这是后面常见坑第一条）。

打开菜单之前先押个落点：勾选那一下，最终变成哪个文件里的哪一行？编译时读的又是哪份？押完走完四条命令，打开 rtconfig.h 对答案。

小节结构

小节	内容	四件套
Env 安装与命令	env 控制台；menuconfig/pkgs/scons 三板斧	配置
Kconfig 机制	config 项→.config→rtconfig.h 的宏链路	库解析
scons 构建	SConscript 逐行：源文件分组与依赖	库解析
软件包生态	在线包索引/版本选择/离线包处理	配置
BSP 解剖	board/libraries/applications 三区职责	库解析
对比 IDF	Kconfig 同源、构建异构（CMake vs scons）的取舍	库解析

一、Env 安装与命令：三板斧

Env 是 RT-Thread 官方提供的 Windows 开箱即用环境，内置 Python/scons/gcc/menuconfig，装一个 Env，全站工具零配置（与 ESP-IDF 的 IDF 安装器同思路，回 B7）。打开 env 控制台（双击 `env.exe` 或运行 `env.bat`），进 BSP 目录，三板斧：

```
menuconfig          # 打开菜单，勾选组件/软件包
pkgs --update       # 拉取/更新选中的软件包源码
scons -j8           # 并行编译，出固件 .bin/.elf
```

三板斧的分工：menuconfig 管"编什么"（写 `.config`），pkgs 管"取什么"（下载软件包源码），scons 管"怎么编"（执行构建）。还有一个常用招，一键导出 IDE 工程：

```
scons --target=mdk5   # 导出 Keil MDK5 工程
scons --target=iar    # 导出 IAR 工程
```

源码与 SConscript 是单一事实源，IDE 工程是衍生品：改了 SConscript 重新导出，不用在 IDE 里手动维护文件清单。这也是为什么团队协作时该提交 SConscript 而不是 IDE 工程文件（`.uvprojx` / `.eww`）：前者是源头，后者是人手一份的衍生品。

二、Kconfig 机制：菜单项 = 宏

每个组件/软件包自带一份 Kconfig 文件，描述它的依赖与选项。`menuconfig` 命令收集所有 Kconfig 拼成菜单树。链路：

```
Kconfig（菜单定义）—menuconfig—> .config（选择结果）—scons 生成—> rtconfig.h（宏）
                                                                    ↓
                                                          C 代码 #include，#ifdef 走分支
```

一个 Kconfig 选项长这样：

```
config RT_USING_I2C
    bool "Enable I2C bus driver"
    select RT_USING_DEVICE
    default n
    help
      勾选后生成 #define RT_USING_I2C
```

- `bool` = 开关项（y/n）；`select` = 选中它时自动勾选依赖项（`RT_USING_DEVICE` 自动开）。
- 存盘后 `.config` 里多一行 `CONFIG_RT_USING_I2C=y`，scons 再生成 `rtconfig.h` 多一行 `#define RT_USING_I2C 1`。
- 代码里 `#ifdef RT_USING_I2C` 应声而开：**菜单项即宏，配置即代码**。

`select` 是"依赖自动满足"：勾 I2C 自动勾 DEVICE，省得忘开依赖；`depends on` 反过来，父依赖没开时子选项灰着选不上（按 `?` 看依赖表达式）。骨架坑列表第三条"Kconfig 依赖断链"就是这么来的：选项灰着选不上=父依赖没开。

与 ESP-IDF 的 sdkconfig 同源（回 B7 第五节）：同一套 Kconfig 语法，只是产物文件名不同，IDF 叫 `sdkconfig.h`，RT-Thread 叫 `rtconfig.h`。**rtconfig.h 是生成物，不该手改**，手改的下次 menuconfig 会被覆盖。要改默认值，去改 Kconfig 里的 `default`，或直接编辑 `.config`。

三、scons 构建：SConscript 逐行

scons 用 Python 写构建脚本：SConstruct 是顶层入口，SConscript 散落在各子目录描述"我这目录编什么"。一个典型 BSP 的 SConscript：

```
# bsp/stm32f407/SConscript（节选）
import rtconfig
from building import *

cwd = GetCurrentDir()
src = Glob('*.c')                    # 收集本目录所有 .c
path = [cwd]                         # 头文件搜索路径
group = DefineGroup('Drivers', src, depend=['RT_USING_DEVICE'], CPPPATH=path)
```

逐行读：- `Glob('*.c')`：把本目录所有 `.c` 收进源文件清单。- `depend=['RT_USING_DEVICE']`：只有 rtconfig.h 里 `RT_USING_DEVICE` 开了，这组才进构建，**Kconfig 与 scons 的接合点就在 depend**。- `DefineGroup`：把"组名+源文件+依赖+头文件路径"注册给构建系统，scons 收齐所有 group 后编。

SConstruct 顶层做三件事：设工具链、读 rtconfig.h、递归遍历所有 SConscript 收 group、链接出 `.elf`、objcopy 出 `.bin/.hex`。关键一句：

```
# SConstruct（节选）
env = Environment(tools=['gccarm'], ARM_GCC_PATH=rtconfig.EXEC_PATH)
```

scons 与 CMake 的差别（回 B7 第六节）：CMake 是"生成器"（生成 Makefile/Ninja 再构建），scons 是"直接构建器"（Python 脚本直接驱动 gcc），少一层"生成"步骤，但依赖 Python 运行时。**SConscript 用 Python 比纯 DSL 灵活**：循环、条件、字符串拼接都是原生 Python，不用学专用语法。代价是构建速度依赖 scons 实现（`-j8` 并行比 Ninja 慢一档），但中小工程无感。

四、软件包生态：在线点单

`pkgs --update` 是 RT-Thread 生态的"外卖骑手"：menuconfig 里勾的软件包（cJSON/MQTT/AT/...）存在线包索引里，`pkgs --update` 按选择拉源码到 `packages/` 目录，下次 scons 就编进去。

```
pkgs --update          # 按 .config 拉选中包
pkgs --list            # 看本地已装包
pkgs --upgrade         # 升级有新版的包
```

版本选择在 menuconfig 里：每个包可选 latest/特定 tag/特定版本，生产固件建议钉版本（防上游更新引入不兼容）。离线场景：提前 `pkgs --update` 拉全，把 `packages/` 目录一起带，离线构建不依赖网络。

软件包生态是 RT-Thread 区别于 FreeRTOS 的最大护城河（回 R0）：FreeRTOS 只给内核，外设驱动/网络栈/文件系统全靠自己拼；RT-Thread 一个 menuconfig 就能拉 MQTT 客户端、cJSON、lwIP，"点外卖式做固件"由此得名。R7 移植完标准版，回来勾一个软件包跑通，就能体感这条护城河。

五、BSP 解剖：三区职责

一个标准版 BSP 目录长这样：

```
bsp/stm32f407-zeo/
├── board/            # 板级：时钟/堆/外设初始化、链接脚本
│   ├── board.c       # rt_hw_board_init 实现
│   ├── linker.ld     # FLASH/RAM 布局
│   └── CubeMX_Config/# 引脚配置（HAL 生成）
├── libraries/        # HAL 驱动层：STM32 HAL + rt-thread 设备驱动
│   ├── HAL_Drivers/  # rt_device 包装（uart/spi/i2c）
│   └── STM32F4xx_HAL/# ST 官方 HAL 库
├── applications/     # 应用：main.c、业务代码
├── Kconfig           # 本 BSP 的菜单项（板级选项）
├── SConscript        # 构建脚本
└── rtconfig.h        # 裁剪配置（生成物）
```

三区职责分明： - **board/**：板级知识（时钟树、堆边界、引脚、链接脚本），换板改这里。 - **libraries/**：HAL 驱动，把 STM32 HAL 包装成 `rt_device`（回 R4）。 - **applications/**：业务代码，与板级解耦，换板不丢。

骨架坑列表第四条"在 applications 里改驱动"的根因：板级驱动该去 board/ 与 libraries/，applications 只放应用，否则换 BSP 全丢。这是 RT-Thread 工程化的核心纪律：**板级与应用解耦**，靠设备框架（R4）做接合层。R7 标准版移植的"改板三步"全在 board/ 里改，applications/ 一行不动，这就是解耦的红利。

六、对比 IDF：Kconfig 同源、构建异构

把 RT-Thread 与 ESP-IDF 的配置/构建体系摆一起（回 B7 第六节）：

维度	ESP-IDF	RT-Thread
配置系统	Kconfig → sdkconfig → sdkconfig.h	Kconfig → .config → rtconfig.h（同源）
构建系统	CMake（生成 Ninja）	scons（Python 直接构建）
组件声明	idf_component_register（CMake DSL）	SConscript（Python 函数）
包管理	idf.py add-dependency	pkgs --update（在线包索引）
调度入口	idf.py build/flash	scons / scons --target=mdk5
工具环境	IDF 安装器（自带 Python/cmake/ninja）	Env（自带 Python/scons/gcc/menuconfig）

两者都把"组件化+配置化"叠在 B1 那四步之上。差别在构建脚本语言：CMake 是专用 DSL（跨平台好、但语法另学），scons 是 Python（灵活、但要装 Python）。**Kconfig 是共用层**——同一套语法，菜单项=宏的链路两边都成立。会一种就能读另一种，会两种就能看穿"配置与构建是正交两维"（B7 的结论）。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **menuconfig 点单，pkgs 取货，scons 下厨；Kconfig 管编什么，scons 管怎么编：点外卖式做固件。**

延伸：menuconfig 点单流水线动画见 R6 动画；ESP-IDF Kconfig 对照见 P3；构建系统在 B7 详解。

实物实验

- 勾一个软件包（如 cJSON）→ 应用里 `#include "cJSON.h"` 解析一段 JSON 串口打印：从零到用上社区库，十分钟。

常见坑

- **menuconfig 保存后忘 scons 重编**：宏已变固件未新，"勾了没用"的第一原因。
- **pkgs --update 网络失败**：代理/镜像问题，pkgs 支持镜像源配置，公司网先配好。
- **Kconfig 依赖断链**：选项灰着选不上=父依赖没开，按？看依赖表达式。
- **在 applications 里改驱动**：改板级驱动请去 board/ 与 libraries/，applications 只放应用，否则换 BSP 全丢。
- **rtconfig.h 手改后被覆盖**：rtconfig.h 是 menuconfig 生成物，手改的下次 menuconfig 全冲掉。要改默认值改 Kconfig 的 `default` 或直接编辑 `.config`，别直接改 rtconfig.h。

短自测

1. menuconfig/pkgs/scons 三板斧各管什么？为什么说"配置与构建是正交两维"？

▼ 看答案
menuconfig 管"编什么"（写 .config，决定哪些组件进、哪些宏开），pkgs 管"取什么"（按选择下载软件包源码），scons 管"怎么编"（按 SConscript 描述的依赖图驱动 gcc 出固件）。说它们正交是因为：同一份 Kconfig 配置可以喂给 scons 也可以喂给 CMake；同一套 scons 可以编不同 Kconfig 配置，两维各自独立演化，不互相绑死（这是 B7 的结论）。

2. Kconfig 的 `select` 与 `depends on` 各是什么？灰着选不上的选项什么原因？

▼ 看答案
`select` 是"选中我时自动勾选依赖项"，勾 I2C 自动开 DEVICE，省得忘开依赖。`depends on` 是"父依赖开了我才能选"，父依赖没开时子选项灰着选不上。灰选项的根因就是 `depends on` 表达式里的父依赖没满足，按问号键看依赖表达式，先开父依赖即可。

3. rtconfig.h 为什么不该手改？要改默认值怎么办？

▼ 看答案
rtconfig.h 是 menuconfig 按 .config 生成的产物，手改的下次 menuconfig 会按 .config 重新生成，全冲掉。要改默认值改 Kconfig 里的 `default`，或直接编辑 .config（下次 menuconfig 会读它）。rtconfig.h 该被 gitignore，该提交的是 Kconfig 与 .config（或 .config.defaults 种子文件）。

4. SConscript 里 `DefineGroup` 的 `depend` 参数起什么作用？

▼ 看答案
`depend` 指定这组源码进构建的前提：只有 rtconfig.h 里对应的宏开了，这组才编。比如 `depend=['RT_USING_DEVICE']` 表示设备框架开了才编这组驱动。这是 Kconfig 与 scons 的接合点：Kconfig 决定宏开不开，SConscript 的 depend 决定宏开了才进构建，两维在这里汇合。

5. BSP 三区（board/libraries/applications）各放什么？为什么改驱动不能去 applications？

▼ 看答案
board 放板级知识（时钟、堆、引脚、链接脚本），libraries 放 HAL 驱动（把 STM32 HAL 包装成 rt_device），applications 放业务代码。改驱动去 applications 会把板级耦合进应用，换 BSP 时 applications 全丢。正确做法是板级驱动放 board/libraries，靠设备框架（R4）做接合层，applications 只调 rt_device 接口，换板不动应用。

对照表：本章概念 → 仓库与上游落点

概念	落点
Env 三板斧（menuconfig/pkgs/scons）	env 控制台；本章第一节
Kconfig → .config → rtconfig.h 链路	BSP 目录的 Kconfig + rtconfig.h
SConscript 的 DefineGroup/depend	bsp/*/SConscript
软件包在线点单	pkgs --update；packages/ 目录
BSP 三区（board/libraries/applications）	bsp/stm32f407-*/ 目录结构
scons --target=mdk5/iar 一键导出	本章第一节命令
Kconfig 与 ESP-IDF 同源	B7 第五节；P3

你做到了

- RT-Thread 生态的"点单-取货-下厨"全流程跑通；
- 配置体系与构建体系的分工彻底清晰：B7 的理论在此落地。

落点在 rtconfig.h：菜单那一下勾选，最终变成这个生成文件里的一行宏，代码编译时读的也是它。手改它就像改收据，下次 menuconfig 一开，墨迹全无。

✅ 下一站：R7 移植到霸天虎：Nano 手动移植到标准版，两步走全记录。

通往单片机之路第 52 章

R7 移植 RT-Thread 到霸天虎：两步走

难度 ★★★☆☆ · 约 40 分钟

本章罗盘

问题

FreeRTOS 靠 SVC 起第一个任务，RT-Thread 为什么不用，Nano 移植到底要手写哪几件？

交付

一份 Nano 四件套清单（内核源码＋rtconfig.h＋board.c＋链接脚本）与 board.c 三件事实现（时钟/SysTick 1ms/rt_system_heap_init 堆边界），配五项验收记录：tick 在涨、双线程异频闪、finsh 应答、list_device 在册、malloc 不崩。

边界

本仓库配套工程（code/rtos/02-rt-thread-lab）只走 Nano 路线；标准版「就近抄 BSP 改三步（时钟/引脚/内存）」是方法论，无配套工程与实测背书；源码行为以 RT-Thread master 为准（未固定 tag、不钉行号），HardFault 取证细节回 S16。

依据锚点 `f8 · f1 · mcu-code · mcu-anim`

> 🎯 F8 我们手动接管了 FreeRTOS 的三个异常；这一章走两条路：先用 **Nano**（3KB 的迷你内核）手动移植感受"最小成本"，再用**标准版 BSP** 体验"设备框架+finsh"的完整生态——两条路都走通，RT-Thread 的版图在你心里就完整了。

本章精髓

1. Nano 手动移植四件套：内核源码（src/ + libcpu/arm/cortex-m4）+ `rtconfig.h`（裁剪配置）+ board.c（实现 `rt_hw_board_init`：时钟/SysTick/堆）+ 三异常接管（与 F8 同款：PendSV/SysTick/HardFault），内核无关板卡，板卡全在 board.c。
2. SysTick 的双重身份：`rt_hw_board_init` 里配好 1ms tick，ISR 里调 `rt_tick_increase()`，与 FreeRTOS 的 xPortSysTickHandler 异曲同工。
3. 标准版的价值在"带 BSP"：RT-Thread 仓库自带大量 STM32 BSP（同系 F407 BSP 可就近借鉴），复制最近似 BSP 改引脚/时钟/外设，比从零写快十倍；finsh 与设备框架随 `rtconfig.h` 开关即来。

- **能理解**：为什么 board.c 是 Nano 移植唯一的"板卡知识"集中地；为什么标准版抄 BSP 比从零写快十倍；为什么 RT-Thread 不像 FreeRTOS 用 SVC 启动第一个任务。
- **能用**：Nano 路线跑通双线程异频闪灯；标准版路线选最近似 BSP 改三步，出 `list_device` 看到 pin/uart1 在册。

先修

- F8 FreeRTOS 移植（对照锚点）、R0、R6 Env。

先跑起来（10 分钟 quick win）

```
# Nano 路线终点（配套工程：code/rtos/02-rt-thread-lab，与 01-freertos-lab 同构）
cd code/rtos/02-rt-thread-lab
make                    # 或 make DEMO_SCENE=6 / DEMO_SCENE=6 USE_MUTEX=1
# 烧录后串口应见：
#   scene=1 two threads blink (200ms vs 500ms)   # 双线程异频闪灯
#   [0] L take / [0] L got lock, cpu work 3000ms / ...   # scene 6 优先级反转日志
```

两线程异频闪 + 串口打印——最小 RT-Thread 诞生。Nano 版本与 finsh 版本二选一（Nano 不带 finsh）：Nano 验内核（双线程切换），标准版验生态（finsh+设备框架）。两条路都走通，RT-Thread 的版图在你心里就完整了。本仓库配套工程走 Nano 路线；标准版路线按下文 BSP 步骤自行搭建。

进工程之前先押个乡音：四件套里真正带着这块板子口音的文件有几个？押完数一数，答案只剩一个名字。

动画：Nano 移植四件套装配

内核源码、rtconfig.h、board.c、异常接管四件逐一套上：内核无关板卡，板卡知识全部收进 board.c，装配完成即得最小 RT-Thread。

R7 Nano 移植四件套装配

小节结构

小节	内容	四件套
Nano 四件套	源码/rtconfig.h/board.c/异常接管逐项	库解析
board.c 逐行	rt_hw_board_init：时钟/SysTick/堆边界符号（与链接脚本配合）	代码分析
三异常接管	与 F8 的对照表；HardFault 处理差异	配置
标准版移植	就近 BSP 选择→改板三步（时钟/引脚/内存）	配置
finsh 接入	串口设备注册给 finsh 的全过程（R4+R5 闭环）	代码分析
验收清单	tick/切换/finsh/list_device 逐项验证	库解析

一、Nano 四件套：源码/rtconfig.h/board.c/异常接管

RT-Thread Nano 是约 3KB 的迷你内核，只保留调度/IPC/内存，不带设备框架/finsh/软件包。手动移植到霸天虎 F407，四件套：

件	内容	来源
内核源码	src/（线程/IPC/内存/调度）+ libcpu/arm/cortex-m4（上下文切换/异常）	RT-Thread 仓库
rtconfig.h	裁剪配置（开哪些 IPC/调度选项、TICK_HZ、堆大小）	官方模板改
board.c	rt_hw_board_init（时钟/SysTick/堆）+ 三异常接管	自己写
链接脚本	FLASH/RAM 布局；堆边界符号（与 board.c 配合）	回 B3

Nano 的哲学：**内核无关板卡，板卡全在 board.c**。内核源码（src + libcpu）跨所有 Cortex-M4 通用，F407/F411/GD32F4 全用同一份；板卡的差异（时钟频率、HSE 晶振、引脚、堆位置）全部集中到 board.c 一个文件。这与 F8 的 FreeRTOS 移植同理，port 层通用，差异在 FreeRTOSConfig.h + board。

四件套到手后的工程结构：

```
rtthread-nano-port/
├── src/                    # 内核：thread.c/ipc.c/scheduler.c/...
├── libcpu/arm/cortex-m4/   # 上下文切换：cpuport.c/context_rvds.S
├── include/                # rtthread.h/rtdef.h/...
├── board/
│   ├── board.c             # ★ 板卡知识全在这
│   ├── board.h
│   └── linker.ld           # 链接脚本
├── rtconfig.h              # 裁剪配置
└── applications/main.c     # 双线程闪灯
```

`src/` 与 `libcpu/` 直接从仓库拷，不动一行；`rtconfig.h` 从官方模板改（关掉用不到的 IPC、定 `RT_TICK_PER_SECOND` 与堆大小）；board.c 是唯一的"原创"，这也是为什么说 Nano 移植"最小成本"：要写的代码集中在一个文件。

二、board.c 逐行：rt_hw_board_init

board.c 是 Nano 移植的"板卡知识全集"，核心函数 `rt_hw_board_init`：

```
// board.c（节选）
void rt_hw_board_init(void)
{
    /* 1) 硬件初始化：时钟/中断优先级分组 */
    HAL_Init();
    SystemClock_Config();              // 时钟树（回 S2），HSE 以板卡规格书为准

    /* 2) 系统滴答：SysTick 配 1ms tick */
    SysTick_Config(SystemCoreClock / RT_TICK_PER_SECOND);  // 通常 1000

    /* 3) 堆：把链接脚本划出的区间交给内核 */
    rt_system_heap_init((void *)HEAP_BEGIN, (void *)HEAP_END);

    /* 4) 组件初始化钩子（标准版走 rt_components_init，Nano 可留空） */
#ifdef RT_USING_COMPONENTS_INIT
    rt_components_board_init();
#endif
}
```

三件事的分工：- **时钟**：调 `SystemClock_Config`（S2 那套 PLL 公式），HSE 晶振频率以板卡规格书为准（霸天虎回 S2 核对 HSE）。- **SysTick**：配 1ms tick：`SystemCoreClock / RT_TICK_PER_SECOND`，`RT_TICK_PER_SECOND` 在 rtconfig.h（通常 1000）。- **堆**：把链接脚本里划出的堆区间交给内核：`rt_system_heap_init(HEAP_BEGIN, HEAP_END)`，`HEAP_BEGIN/END` 是链接脚本符号（回 B3）。

堆边界符号对不上是骨架坑列表第二名：链接脚本的 `end` / `_estack` 与 rt_hw 堆参数对不上，首次 malloc 即崩，这是 B3 链接脚本符号的复训点。内核堆的"池子"就是链接脚本里 `.bss` 之后到栈底之间那块，与 F8 的 `configTOTAL_HEAP_SIZE` 是两种风格：FreeRTOS 用宏定池子大小（一个定长数组），RT-Thread 用链接脚本符号定池子边界（用 leftover 空间）。

三、三异常接管：与 F8 同款

Nano 的内核上下文切换也挂在 Cortex-M 的三个异常上：PendSV/SysTick/HardFault（与 F8 同款，但 RT-Thread 不用 SVC 启动第一个任务，区别在后面）：

异常	RT-Thread 函数	干什么	F8 对照
PendSV（14）	rt_hw_pendsv（libcpu）	上下文切换：保存当前线程、恢复下一个	xPortPendSVHandler
SysTick（15）	rt_hw_systick（board.c）	tick 递增：调 rt_tick_increase()	xPortSysTickHandler
HardFault（3）	rt_hw_hard_fault（board.c）	故障取证：打印 PC/LR/栈帧	自写（S16）

SysTick 的双重身份：`rt_hw_board_init` 里配好 1ms tick，SysTick ISR 里调 `rt_tick_increase()`：递增全局 tick 计数、检查延时到期的线程、必要时挂起 PendSV 请求切换。这与 FreeRTOS 的 `xPortSysTickHandler` 异曲同工（回 F8 第二节）：同一颗 SysTick，同一个 1ms 节拍，不同内核的"报时员"。

SysTick ISR 的最小实现：

```
// board.c
void SysTick_Handler(void)
{
    rt_interrupt_enter();          // 进中断钩子（嵌套计数 +1）
    rt_tick_increase();            // ★ tick + 1，检查延时，可能挂 PendSV
    rt_interrupt_leave();          // 离中断钩子（计数归零可能触发切换）
}
```

`rt_interrupt_enter/leave` 是 RT-Thread 的中断嵌套计数：`leave` 时若计数归零且有挂起的 PendSV，就真正触发切换。这是 RT-Thread 与 FreeRTOS 在中断处理层的细微差别：FreeRTOS 的 `xPortSysTickHandler` 直接在 ISR 里 portYIELD；RT-Thread 用 enter/leave 钩子统一管理临界区与切换时机。

HardFault 处理差异：F8 那套是 FreeRTOS 自带（V11 默认关，靠 configASSERT）；RT-Thread 的 Nano 移植要自己写 `rt_hw_hard_fault`，打印出错时的 PC/LR/SP，回 S16 HardFault 取证 的栈帧挖 PC 方法。这是 Nano "不带 BSP" 的代价，标准版 BSP 已写好。

为什么 RT-Thread 不用 SVC：FreeRTOS 用 SVC 异常手工"启动第一个任务"（恢复它的现场，从此不返回）；RT-Thread 不用 SVC：`rt_thread_startup` 只把线程挂进就绪表，真正的第一次切换在 `rt_system_scheduler_start` 里：它挑出最高优先级线程，调 `rt_hw_context_switch_to`，后者把 `from` 记为 0、置位 PendSV，开中断后 PendSV 立刻进来恢复目标线程现场（`libcpu/arm/cortex-m4/context_gcc.S`）。这是两个内核启动模型的差别：一个靠异常跳进第一个任务，一个靠普通调度路径自然滑进去。

四、标准版移植：就近抄 BSP

标准版的价值在"带 BSP"：RT-Thread 仓库 `bsp/` 自带大量 STM32 BSP：stm32f407-atk-explorer、stm32f407-st-discovery、stm32f407-zeo... 同系 F407 BSP 可就近借鉴。流程：

第 1 步：选最近似 BSP，优先选同芯片同封装同外设的，差异最小。霸天虎 F407 找同系 F407 BSP 作母本（如 `stm32f407-zeo`）。

第 2 步：改板三步，母本 BSP 复制后改三处：

改什么	在哪改	参照
时钟源	board/board.c 的 SystemClock_Config + linker.ld 的 FLASH/RAM	S2 核对 HSE 晶振
引脚	board/CubeMX_Config 重新配 LED/USART 引脚	S3；霸天虎 LED=PF6/7/8 共阳（回 S3/S0）
内存	linker.ld 的 FLASH/RAM 起址与大小	芯片 datasheet

这就是骨架坑列表第四条"BSP 时钟源不改"的翻车现场：母本 BSP 用 25MHz 晶振，而霸天虎不同频。不改 HSE，PLL 算出来的 SYSCLK 全错；串口波特率依赖 APB 时钟，跟着全错，打印全是乱码。**改板第一步永远是核对 HSE**（回 S2）。

第 3 步：menuconfig 裁剪 + scons，`menuconfig` 关掉母本有但霸天虎没有的外设、开需要的（finsh/uart）；`scons -j8` 编；`scons --target=mdk5` 导出 IDE 工程。finsh 与设备框架随 rtconfig.h 开关即来，不需要手写一行 finsh 代码，勾 `RT_USING_FINSH` 自动进构建（回 R5）。

"比从零写快十倍"的原因：母本 BSP 的 board.c/libraries/Kconfig 已经按 F407 调好，时钟树、HAL 驱动包装、设备框架注册全是现成的，你只改引脚/晶振/内存这三个板卡差异点。从零写 board.c 要调时钟树（S2）、写 HAL 包装（R4）、注册设备，三天活；抄 BSP 改三步，半小时。

五、finsh 接入：R4 + R5 闭环

标准版移植的最后闭环是 finsh 接入，串口设备注册给 finsh 的全过程（回 R4 设备框架 + R5 finsh）：

```
USART1 硬件 ──HAL 驱动包装──> rt_serial_device ──注册──> 设备容器 "uart1"
                                                          ↓
                                                    finsh 组件
                                                          ↓
                                                    msh /> 提示符
```

三步： 1. **HAL 包装**：libraries/HAL_Drivers 把 STM32 HAL 的 USART 包装成 `rt_serial_device`（R4 的 rt_device 接口：open/read/write/control）。 2. **设备注册**：`rt_hw_serial_register` 把 uart1 注册进设备容器， `list_device` 能看到它。 3. **finsh 绑定**：rtconfig.h 开 `RT_USING_FINSH`，finsh 启动时查 `rt_console_set_device` 找注册的串口，绑定成控制台， `msh />` 提示符出来。

`menuconfig` 里勾 `RT_USING_FINSH` + `RT_USING_SERIAL`，scons 自动把 finsh 源码和 uart 驱动编进固件，配置驱动开发，不用手写一行 finsh 代码。这就是 R0 说的"组件层当水电"：finsh 是水电，menuconfig 是开关，R6 的"点单系统"在 R5 的 finsh 上落地。

六、验收清单：tick/切换/finsh/list_device

移植完按这个清单逐项验：

验收项	怎么验	通过判据
tick 走起来	finsh 敲 `list_thread`，看 tick 计数在涨	数字在动 = SysTick 接管成功
线程切换	起两个线程异频闪灯（PF6/7/8），看是否交替闪	两灯不同步 = 调度器在切换
finsh 响应	串口回车出 `msh />`，敲 `help`	命令清单出 = finsh+uart 闭环
设备注册	敲 `list_device`	看到 pin/uart1 在册 = 设备框架工作
堆可用	起线程 + malloc 一块	不崩 = 堆边界符号对

骨架坑列表的"首次 malloc 即崩"就卡在堆边界符号：`list_thread` 能跑但一 malloc 就 HardFault，根因是链接脚本的 HEAP_BEGIN/END 与 `rt_system_heap_init` 参数对不上（回 B3）。Nano 路线没有 finsh，验收用双线程异频闪灯 + 串口裸打印 tick 计数即可。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **Nano 四件手动装，board.c 是全部板卡知识；标准版就近抄 BSP，改钟改脚改内存，finsh 一响，移植功成。**

延伸：RTT Nano 移植四件套动画见 R7 动画；FreeRTOS 移植对照见 F8；双 OS 移植对比见 X1。

实物实验

- Nano 版双线程 + finsh 版本二选一跑通；
- 标准版 BSP 改造后 `list_device` 看到 pin/uart 设备注册在册，生态落地的实锤。

常见坑

- **rtconfig.h 与源码版本不配**：配置项名随版本漂移，从官方模板改，不抄旧文。
- **board.c 堆边界符号错**：链接脚本符号（如 `end` / `_estack`）与 rt_hw 堆参数对不上，首次 malloc 即崩（B3 复训）。
- **SysTick 冲突**：裸机延时与内核 tick 抢 SysTick，同 F8 教训，统一交给内核。
- **BSP 时钟源不改**：母本 BSP 用 25MHz 晶振而霸天虎不同频，串口波特率全错的经典翻车（回 S2 核对 HSE）。
- **libcpu 选错架构**：拿了 cortex-m3 的 libcpu 给 F407（cortex-m4），上下文切换没存浮点寄存器，浮点任务间互踩。判据是 `-mcpu=cortex-m4` 必须配 libcpu/arm/cortex-m4（同 F8 的 ARM_CM4F 教训）。

短自测

1. Nano 四件套是哪四件？为什么说"内核无关板卡，板卡全在 board.c"？

> **▼ 看答案**
> 四件套是内核源码（src/ + libcpu/arm/cortex-m4）、rtconfig.h（裁剪配置）、board.c（rt_hw_board_init + 三异常接管）、链接脚本。说"内核无关板卡"是因为 src/ 与 libcpu/ 跨所有 Cortex-M4 通用，F407/F411/GD32F4 全用同一份；板卡的差异（时钟频率、HSE 晶振、引脚、堆位置）全部集中到 board.c 一个文件，内核代码不动一行，只改 board.c 就能换板。

2. SysTick 在 RT-Thread 里的双重身份是什么？与 FreeRTOS 的 xPortSysTickHandler 如何对照？

> **▼ 看答案**
> 双重身份：`rt_hw_board_init` 把 SysTick 配成 1ms tick（硬件定时器）；SysTick ISR 里调 `rt_tick_increase`（内核报时员），递增全局 tick、检查延时到期线程、必要时挂起 PendSV 请求切换。与 FreeRTOS 的 xPortSysTickHandler 异曲同工：同一颗 SysTick、同一个 1ms 节拍、都靠 PendSV 切换。细微差别是 RT-Thread 用 rt_interrupt_enter/leave 钩子统一管理切换时机，FreeRTOS 在 ISR 里直接 portYIELD。

3. 为什么 RT-Thread 不像 FreeRTOS 那样用 SVC 启动第一个任务？

> **▼ 看答案**
> FreeRTOS 用 SVC 异常手工恢复第一个任务的现场，从此不返回，靠 SVC"起死回生"。RT-Thread 不用 SVC。`rt_thread_startup` 只把线程挂进就绪表；第一次切换发生在 `rt_system_scheduler_start` 里，由 `rt_hw_context_switch_to` 置位 PendSV，开中断后自然进 PendSV 恢复第一个线程。这是两个内核启动模型的差别：一个靠异常跳进第一个任务，一个靠普通调度路径自然滑进去。

4. 标准版移植"改板三步"是哪三步？为什么抄 BSP 比从零写快十倍？

▼ 看答案
改板三步：时钟源（board.c 的 SystemClock_Config + linker.ld）、引脚（CubeMX_Config 重配 LED/USART）、内存（linker.ld 的 FLASH/RAM 起址大小）。抄 BSP 快是因为母本 BSP 的 board.c/libraries/Kconfig 已按 F407 调好，时钟树、HAL 驱动包装、设备框架注册全现成，只改引脚/晶振/内存三个板卡差异点。从零写要调时钟树（S2）、写 HAL 包装（R4）、注册设备，三天活；抄 BSP 改三步，半小时。

5. list_device 看到 pin/uart1 在册，说明哪条链路全部打通了？

▼ 看答案
这说明从硬件到 finsh 的整条链路打通了：USART1 硬件经 HAL 驱动包装成 `rt_serial_device`，注册进设备容器（`list_device` 能看到 uart1），再由 finsh 组件绑定串口成控制台。这是 R4 设备框架 + R5 finsh + R6 menuconfig 的三闭环。勾 RT_USING_SERIAL 与 RT_USING_FINSH 后，scons 自动把驱动和 finsh 编进固件，不用手写一行接合代码。

对照表：本章概念 → 仓库与上游落点

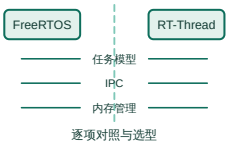
概念	仓库落点
Nano 内核源码 src/ + libcpu	rt-thread/src, libcpu/arm/cortex-m4
rtconfig.h 裁剪配置	工程根 rtconfig.h（官方模板改）
board.c 的 rt_hw_board_init	board/board.c
SysTick ISR 调 rt_tick_increase	board/board.c SysTick_Handler
三异常接管 PendSV/SysTick/HardFault	libcpu + board.c；对照 F8
标准版 BSP 母本	bsp/stm32f407-* 系列
改板三步（时钟/引脚/内存）	board.c + linker.ld + CubeMX_Config
finsh 接入闭环	R4 + R5
list_device 验收	finsh 命令

你做到了

- 两条移植路线都握在手里：要极简用 Nano，要生态上标准版；
- 双 OS 移植方法论成型：换任何 Cortex-M 芯片你都敢接。

乡音只在 board.c：内核不认板子，时钟、SysTick、堆全在这一个文件里落地。换任何 Cortex-M 芯片，内核一行不动，动的都是它。

✅ R 篇收官。下一站：对比 0 双 OS 对照：把两套内核摆上同一张解剖台。



PART PX

第六部分 双 OS 对照与选型

同一个概念在两套内核里各长什么样：逐项对照任务、调度、IPC 与内存管理，把选型从口味之争变成按约束推导。

> 把两套内核摆上同一张解剖台：二十个维度逐项对照，从优先级数值方向、选优算法，到堆管理与配置哲学。对照的目的不是评高下，而是练迁移能力：知道哪些参数要翻译、哪些习惯必须扔，最后落进一棵可执行的选型决策树。读完的标志：面对新项目，能用约束（预算、生态、认证）推导选型，而不是凭口味。

本部 2 章 · 约 50 分钟（0.8 小时）

53 . 对比 0 同概念双实现对照

54 . 对比 1 选型决策树

通往单片机之路第 53 章

对比 0 同概念双实现对照

难度 ★★☆☆☆ · 约 30 分钟

本章罗盘

问题

学完两套内核怎么才不串味，同一个概念两副答案，迁移时哪些参数要翻译、哪些习惯必须扔？

交付

一张 20 维度总览对照表（任务对象/优先级方向/选优算法/堆管理/驱动模型/配置哲学…）+ 迁移坑清单（优先级数值反向、pvPortMalloc 与 rt_malloc 不可混用、静态创建参数形态不同），再拿 Zephyr 按同维度建档一遍。

边界

对照停在机制与 API 形态层，无双 OS 同板性能实测；FreeRTOS 侧结论对 V11.1.0、RT-Thread 侧对 master（不钉行号），两列证据状态不对等；Zephyr 列是 30 分钟建档练习不是深评测；内存分歧的学术背景只在延伸阅读点名（D3/D4）。

依据锚点 `f1 · f8 · d3 · d4 · mcu-anim`

> 🎯 学完两套内核再看对方，处处是"熟悉的陌生人"：同一个问题，两副答案。这张大对照表把 F/R 两篇的全部机制压到一页：复习时按表索骥，迁移时按行翻译。

对表之前先押个左右舵：FreeRTOS 里数值越大优先级越高，这条习惯开进 RT-Thread，方向盘会不会整个换边？再押一个：任务通知到了对面，有现成替身，还是得拿信号量顶上？押完往下读，总览表与机制深对照里都有判词。

本章精髓

- **能记住**：两句话："FreeRTOS 把内核做到没有一两赘肉，RT-Thread 把生态做到开箱即用"；优先级方向反（FreeRTOS 数大=高，RT-Thread 数小=高）。
- **能理解**：为什么 FreeRTOS 的信号量基于队列实现而 RT-Thread 是独立 IPC 对象；为什么 FreeRTOS 有任务通知而 RT-Thread 没有，根子是思想差异：一个抠性能到极致，一个求模型统一。
- **能用**：拿到任何第三个 RTOS（Zephyr/ThreadX/µC/OS），能按这张对照表的维度快速建档，30 分钟内给出"与 F/R 的异同"初判。

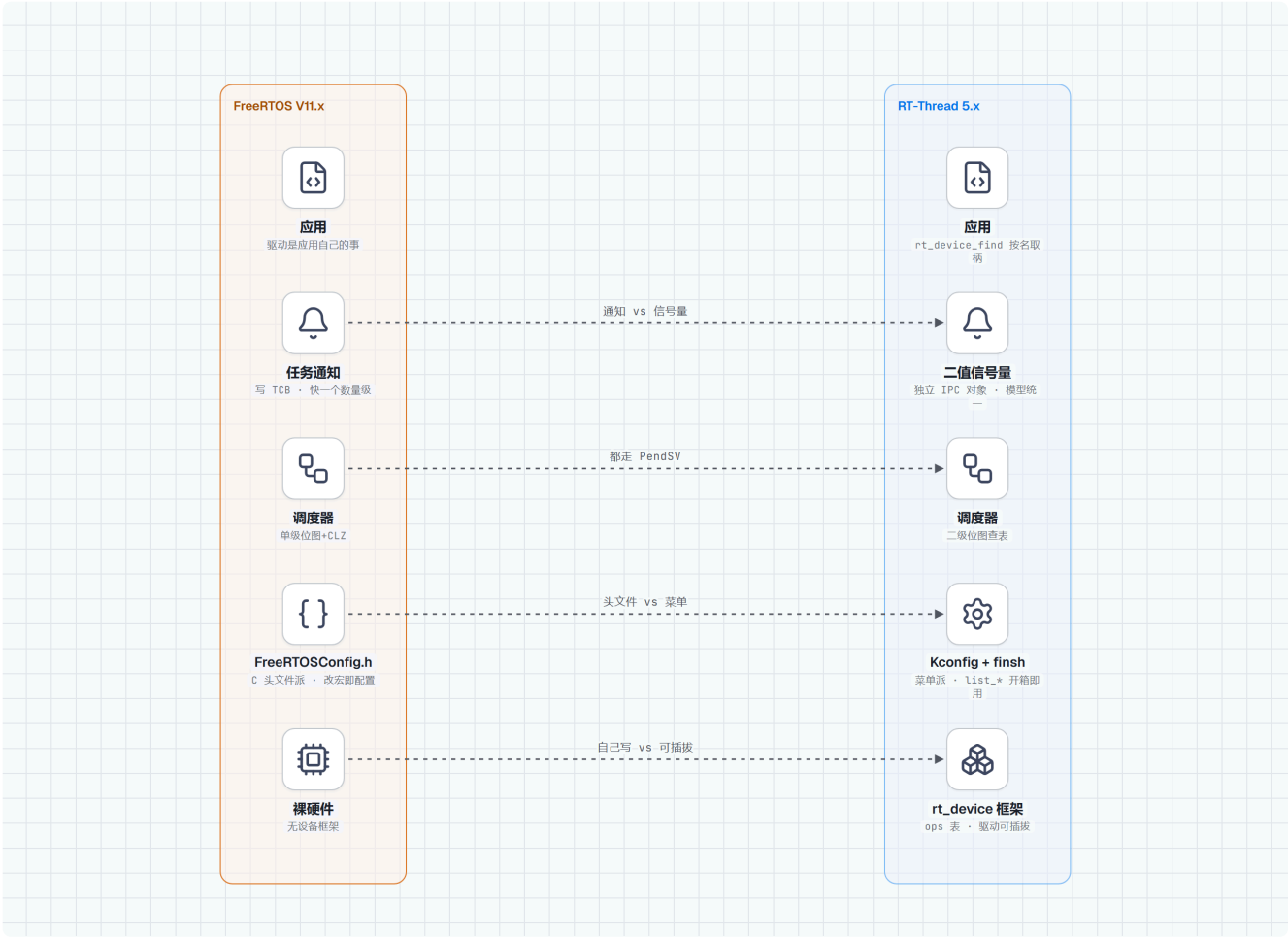
动画：双 OS 同概念对照

任务通知 vs 二值信号量、有无设备框架、FreeRTOSConfig.h vs Kconfig，三组同概念双实现并排演进，两个内核"抠性能"与"求统一"的思想差异一眼看穿。

双 OS 同概念对照：通知 vs 无 / 设备框架 / 配置哲学

总览对照

对照表太密？先看图：五层解剖台，左边 FreeRTOS 右边 RT-Thread，虚线是同一个问题的两种答案。



FreeRTOS × RT-Thread 同一张解剖台：五层对照

维度	FreeRTOS (V11.x)	RT-Thread (5.x)
定位	极致精简内核	内核+组件+生态
任务对象	tskTCB	rt_thread（继承 rt_object）
优先级方向	数值大=高	数值小=高（0 最高）
优先级数	configMAX_PRIORITIES（常 32）	32/256（二级位图）
选优算法	单级位图+CLZ	二级位图+查表
上下文切换	PendSV（xPortPendSVHandler）	PendSV（rt_hw_context_switch*）
tick	xTaskIncrementTick	rt_tick_increase
延时对象	延时列表	线程内嵌 rt_timer
队列	Queue（环形+双阻塞表）	消息队列（变长环形）
信号量/互斥	基于队列实现；互斥带继承	独立 IPC 对象；互斥带继承
轻量同步	任务通知（TCB 内 32 位）	无直接等价（用信号量/事件）
事件	事件组（24 位）	事件集（32 位）
软件定时器	守护任务执行	定时器线程（可系统/用户）
堆管理	heap_1~heap_5 选一	memheap/slab/TLSF+内存池
静态创建	xTaskCreateStatic 家族	rt_thread_init 家族
驱动模型	无（裸 API）	rt_device 设备框架（ops 表）
控制台	无内置	finsh/msh
构建/配置	FreeRTOSConfig.h 手改	Kconfig+menuconfig+scons
软件生态	靠移植/第三方	软件包中心（数百个）
可观测	RunTimeStats/Trace	list_* 命令开箱即用

机制深对照（三例）

1. **通知 vs 无**：FreeRTOS 任务通知把"一对一同步"做到极致（写 TCB）；RT-Thread 没有等价物，用二值信号量。思想差异：一个抠性能到极致，一个求模型统一。
2. **设备框架**：FreeRTOS 的世界里"驱动"是应用自己的事；RT-Thread 用 ops 表把驱动变成"可插拔商品"，这是软件包生态的地基。
3. **配置哲学**：FreeRTOSConfig.h 是"C 头文件派"（简单直接，无工具依赖）；Kconfig 是"菜单派"（依赖自动检查，生态可扩展）；小项目爱前者，大系统要后者。

通知 vs 无：极致性能 vs 模型统一

FreeRTOS 的任务通知（F6 任务通知/事件组/软件定时器）把"一对一同步"压到极致：不创建任何对象，直接写目标 TCB 里的 32 位通知值字段，一次 `xTaskNotifyGive` + `ulTaskNotifyTake` 就是完整的一对一同步，比信号量快一个数量级。代价是"只能一对一"（通知值存在 TCB 里，没有独立句柄，不能广播）。

RT-Thread 没有等价物：一对一同步用二值信号量（R2 IPC 全家桶）。信号量是独立 IPC 对象（继承 rt_ipc_object），有挂起列表、有对象容器登记、finsh 能 `list_sem` 看到它。代价是多一层对象开销，收益是模型统一，所有同步走同一副骨架，可观测性开箱即用。

选型含义：你从 FreeRTOS 迁移到 RT-Thread，把"任务通知"全替换成"二值信号量"：功能等价，性能略降，但代码更统一可观测。反过来从 RT-Thread 迁到 FreeRTOS，二值信号量可以原样用，想追求性能再改成任务通知。

设备框架：驱动是应用的事 vs 驱动是可插拔商品

FreeRTOS 的世界里没有"设备框架"，驱动是应用自己的事。UART 收发要自己写寄存器或找 HAL 库，换芯片要改应用层。这是"裸内核"哲学：内核只管调度与 IPC，外设归应用。

RT-Thread 的 `rt_device` 设备框架（R4 设备框架）把驱动变成"可插拔商品"：驱动作者实现 ops 表（init/open/read/write/control），应用作者 `rt_device_find("uart1")` 按名取柄调用，换板只换 ops 表，应用纹丝不动。这是软件包生态的地基：软件包（如 MQTT 客户端）依赖的是 `rt_device` 接口（网络 socket → 网卡设备），不是具体硬件，所以同一个软件包能在不同 BSP 上跑。

选型含义：功能纯粹的控制器（传感器采集+串口上报）不需要设备框架，FreeRTOS 的极简更香；要拼"联网+存储+UI+OTA"的产品，RT-Thread 的设备框架让软件包即插即用，省几个月。

配置哲学：C 头文件派 vs 菜单派

FreeRTOSConfig.h 是"C 头文件派"（B7 构建系统）：`#define configMAX_PRIORITIES 32` 直接改宏，简单直接，无工具依赖，小项目爱这种。代价是"依赖检查靠人"：你开了 `USE_MUTEXES` 但没开 `USE_RECURSIVE_MUTEXES`，编译能过但运行时行为不对，得自己核对。

RT-Thread 的 Kconfig 是"菜单派"（R6 Env 与 menuconfig）：每个选项声明依赖（`depends on RT_USING_MUTEX`），menuconfig 自动检查依赖、灰化不可选项，`.config` 生成 `rtconfig.h`（生态可扩展，软件包自带 Kconfig 自动进菜单），大系统要这种。代价是多一层工具（menuconfig/scons），学习曲线略陡。

两种哲学在 ESP-IDF 里合流：IDF 用 Kconfig（菜单派）管 FreeRTOS 的配置，`CONFIG_FREERTOS_USE_MUTEXES` 在 menuconfig 里勾，生成 `sdkconfig.h` 供 FreeRTOSConfig.h 引用。所以 ESP32 开发者拿到的是"菜单管 FreeRTOS"的混合体。

短自测

1. FreeRTOS 优先级 5 迁移到 RT-Thread 应该填什么数？方向反了会怎样？

▼ 看答案
FreeRTOS 数值大=高优先级，RT-Thread 数值小=高优先级（0 最高）。迁移要把"FreeRTOS 优先级 5"按你的优先级映射表转成 RT-Thread 的某个更小的数（如 RT-Thread 优先级 25，如果总 256 级且你想要"中等偏高"）。方向反了会让"本该高优先级的任务"变成最低优先级，被低优先级任务抢占，调度行为全反，现象是"高优先级任务不响应"。

2. FreeRTOS 的任务通知在 RT-Thread 里用什么替代？两者的性能与可观测性差异？

▼ 看答案
RT-Thread 用二值信号量替代任务通知。性能：任务通知直接写 TCB 字段，比信号量快一个数量级（无对象开销、无挂起列表操作）；可观测性：信号量是独立 IPC 对象，finsh 能 list_sem 看到，任务通知藏在 TCB 里、list_thread 看通知值但不直观。选型：RT-Thread 牺牲一点性能换模型统一与可观测，FreeRTOS 牺牲可观测换极致性能。

3. 为什么说 RT-Thread 的设备框架是"软件包生态的地基"？

▼ 看答案
软件包（如 MQTT 客户端）依赖的是 rt_device 接口（网络 socket → 网卡设备的 ops），不是具体硬件。设备框架保证"同一个软件包能在不同 BSP 上跑"（BSP 作者实现网卡设备的 ops 表，软件包作者只调 rt_device API，两者解耦）。没有设备框架，软件包要么绑定具体硬件（换板不能跑）、要么自己抽象硬件（每个软件包重复造轮子）。设备框架是"可插拔商品"的统一接口。

4. FreeRTOSConfig.h 与 Kconfig 两种配置哲学各适合什么场景？

▼ 看答案
FreeRTOSConfig.h（C 头文件派）：简单直接、无工具依赖、改宏即生效，适合小项目、功能纯粹、配置项少且固定的场景；代价是依赖检查靠人。Kconfig（菜单派）：菜单勾选、依赖自动检查、生态可扩展（软件包自带 Kconfig 自动进菜单），适合大系统、配置项多且有依赖关系、要拼软件包生态的场景；代价是多一层工具（menuconfig/scons）学习曲线。ESP-IDF 是混合体：Kconfig 管 FreeRTOS 的配置。

5. 给你一个 Zephyr RTOS，怎么按本章的对照表快速建档？

▼ 看答案
按总览对照表的维度逐行填 Zephyr 列：定位（Zephyr 是 Linux 基金会托管的工业级 RTOS）、任务对象（k_thread 结构）、优先级方向（Zephyr 数值小=高，与 RT-Thread 同向）、选优算法（k_thread 优先级数组+红黑树就绪队列）、上下文切换（PendSV/_zephyr_svc）、tick（z_clock_announce）、队列（k_fifo/k_lifo 基于环形）、信号量/互斥（k_sem/k_mutex）、堆管理（k_heap/k_mem_slab）、静态创建（K_THREAD_DEFINE 宏）、驱动模型（device tree + driver API，类似 RT-Thread）、控制台（shell 子系统）、构建配置（Kconfig+west/CMake）。30 分钟内能给出"与 F/R 的异同"初判。

对照表：本章概念 → 仓库与上游落点

概念	落点
总览对照表（20 维度）	本章总览对照节；F 篇 + R 篇全章
通知 vs 无	F6 任务通知 vs R2 信号量
设备框架对照	R4 设备框架；FreeRTOS 无对应
配置哲学对照	B7 构建系统 + R6 menuconfig + P3 IDF
优先级方向差异	F1 TCB vs R1 rt_thread
堆管理对照	F7 heap vs R3 memheap/slab/TLSF
上下文切换对照	F2 上下文切换 vs R7 移植三异常

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **FreeRTOS 把内核做到没有一两赘肉，RT-Thread 把生态做到开箱即用；前者读懂只要一周，后者用全只要一天。**

常见坑

- **迁移时数值方向忘换**：FreeRTOS 优先级 5 → RT-Thread 应是"某个更小的数"，方向反，调度行为全反。
- **带 FreeRTOS 习惯找通知**：RT-Thread 没有就用二值信号量替代，别硬造轮子。
- **带 RT-Thread 习惯找设备框架**：FreeRTOS 里 uart 收发要自己写/找驱动库（F4 队列范式就是你的框架）。
- **堆管理迁移忘换家族**：FreeRTOS 的 `pvPortMalloc` / `vPortFree` 与 RT-Thread 的 `rt_malloc` / `rt_free` 不是同一族，混用就是内存灾难，迁移时全局替换并核对 free 家族。
- **静态创建 API 形态差异**：FreeRTOS 的 `xTaskCreateStatic` 要传 StaticTask_t+栈数组两个缓冲；RT-Thread 的 `rt_thread_init` 要传 rt_thread 结构+栈数组+栈大小（参数个数与顺序不同），别按肌肉记忆填。

延伸阅读

两家在内存管理上分道扬镳的学术背景：

- **[D3]** Masmano et al. 2004（TLSF）· **[D4]** Wilson et al. 1995（分配器综述）— 先读综述再读 TLSF，分歧的来龙去脉就清楚了。

你做到了

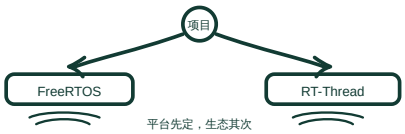
- 一页纸看穿两个 OS；
- 拿到任何第三个 RTOS（Zephyr/ThreadX）都能按这张表快速建档。

✅ 下一站：对比 1 选型决策树——你的下一个项目，该牵谁的手。

通往单片机之路第 54 章

对比 1 选型决策树

难度 ★☆☆☆☆ · 约 20 分钟



本章罗盘

问题

面对同一块板子，凭什么在 FreeRTOS 与 RT-Thread 之间做选择，哪些差异真实影响你的项目？

交付

一张按项目约束（ROM/RAM 预算、生态、中间件、认证）打分的决策表，并写明自己当前项目该选谁、为什么。

边界

结论只对中小型裸机迁移场景负责；安全认证、大型中间件生态的需求要另行调研当期版本。

依据锚点 `f1 · f8 · mcu-code`

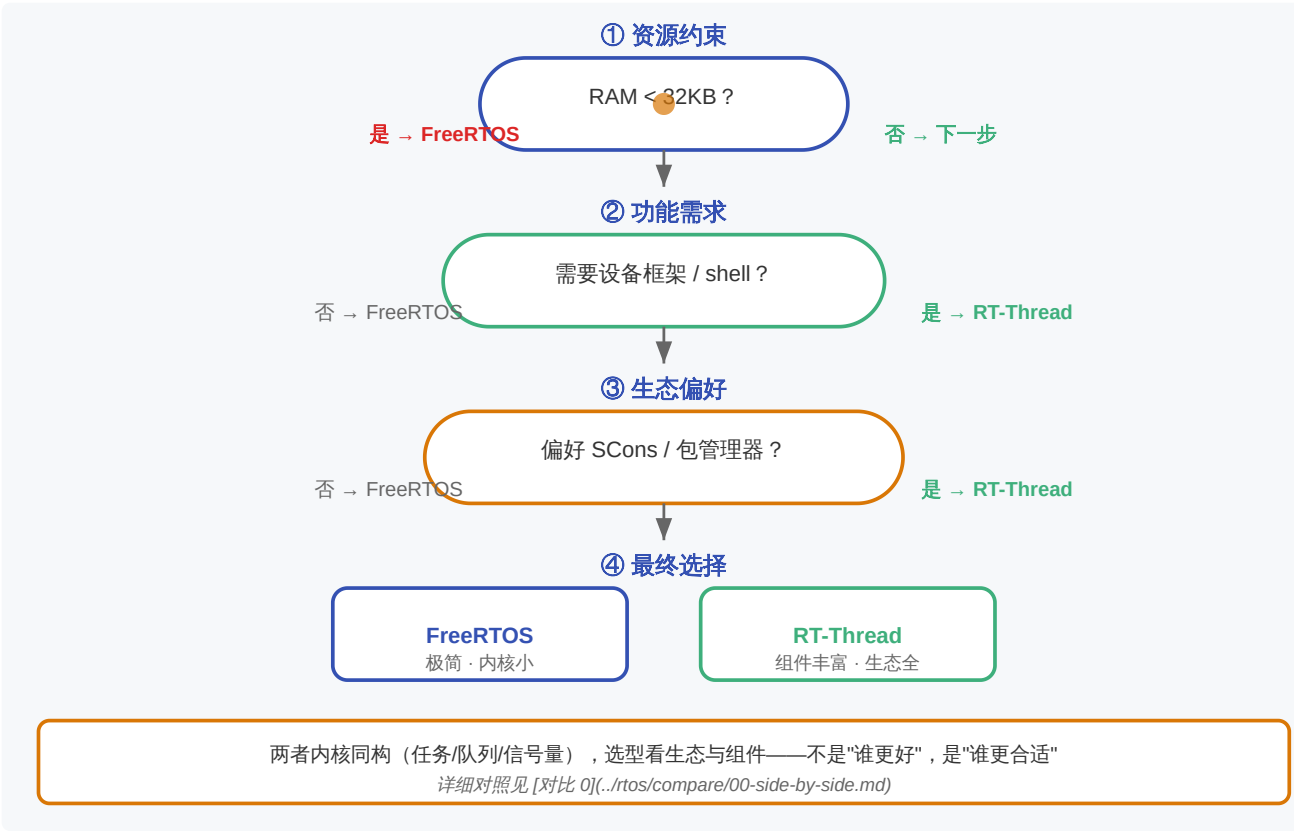
> 🎯 "FreeRTOS 还是 RT-Thread？"这个问题没有普世答案，只有"你的项目长什么样"。把项目特征当输入，顺着决策树走一遍，答案自然浮出来。

走树之前先押个站位：带屏、联网、文件系统、OTA 四样全占的项目，树会把你领到哪一家？再押一个：电池供电的极简节点，树上有没有唯一答案？押完顺着树往下走，场景练习就是记分牌。

本章精髓

- **能记住**：选型三定律——"平台先定，生态其次，团队兜底"。
- **能理解**：为什么 ESP32 就用 IDF 自带 FreeRTOS（平台绑定，协议栈深度集成 SMP/电源管理）；为什么功能纯粹的控制器选 FreeRTOS 而要拼产品的选 RT-Thread 标准版。
- **能用**：拿到一个新项目立项，30 分钟内按决策树走一遍给出有依据的选型结论，不靠拍脑袋。

决策树



X1 RTOS 选型决策树

项目立项

↓

芯片平台已定？

- ↳ **ESP32 系列**　ESP-IDF 自带 FreeRTOS，别折腾，直接用
- ↳ **STM32 / 国产 MCU**　继续 ↓

需要现成组件吗？（网络 / 文件系统 / UI / OTA）

- ↳ **需要且想省事**　RT-Thread 标准版，软件包点单
- ↳ **安全认证场景**　见下方认证分支
- ↳ **不需要或自研**　继续 ↓

团队熟悉谁？

- ↳ **FreeRTOS**　FreeRTOS，极简可控
- ↳ **RT-Thread**　RT-Thread 标准版

有认证包袱？（车规 / 医疗）

- ↳ **有**　考虑商用认证内核，或静态化 FreeRTOS
- ↳ **无**　回到「团队熟悉谁」

资源够吗？（标准版约 32K Flash 起步）

- ↳ **紧张**　RT-Thread Nano：3K 起步 + 手工扩展
- ↳ **宽裕**　RT-Thread 标准版

静态版由在线版 mermaid 决策树整理而来；逐节点依据见正文。

三句话版

1. **平台决定论**：ESP32 就用 IDF 自带 FreeRTOS（深度集成 SMP/电源管理），换 RTOS 是自找苦吃。
2. **生态决定论**：要快速拼出"联网+存储+UI+OTA"的产品，RT-Thread 的软件包中心能省几个月；反之功能纯粹的控制器，FreeRTOS 的极简更香。
3. **人决定论**：团队熟悉哪个用哪个。RTOS 的坑都长在"不熟悉"上，学习成本往往大于技术差异。

平台决定论：ESP32 为什么直接用 IDF 自带 FreeRTOS

ESP32 系列芯片的 Wi-Fi/BLE 协议栈、电源管理、双核 SMP 调度都深度集成在 ESP-IDF 里（P4 中断与双核），而 IDF 内置的就是 FreeRTOS（且是 SMP 适配版）。换 RTOS 意味着要把协议栈、电源管理、双核调度全部重新对接，工作量以人月计，且享受不到乐鑫的持续维护。所以 ESP32 的选型答案几乎是唯一的：**IDF 自带 FreeRTOS，别折腾**。

这背后的逻辑是"平台绑定"：芯片厂商把 RTOS 与 SDK 深度耦合，换 RTOS 的成本远大于换 RTOS 的收益。STM32/国产 MCU 没有这种绑定，FreeRTOS 和 RT-Thread 都能跑，选型才成为真问题。

生态决定论：什么时候 RT-Thread 软件包能省几个月

一个"带屏+联网+文件系统+OTA"的产品，从零搭起来需要：网络栈（LwIP/HTTP/MQTT）、文件系统（FatFS/LittleFS）、UI 框架（LVGL）、OTA 框架。这些在 RT-Thread 软件包中心（R6 Env 与 menuconfig）里都是 `menuconfig` 勾选 + `pkgs --update` 下载即用，加起来可能省几个月的开发时间。

反过来，功能纯粹的控制器（传感器采集+串口上报、电机驱动、简单状态机）不需要这些组件，FreeRTOS 的极简内核（几 KB Flash）更香，因为没有"杀鸡用牛刀"的生态开销。RT-Thread Nano（3KB 起步）也能做，但标准版的生态优势在这个场景用不上。

判断标准很朴素：**你的项目要碰"联网/存储/UI/OTA"里的几个？碰 3 个以上，RT-Thread 标准版的生态优势显著；碰 0-1 个，FreeRTOS 的极简更划算。**

人决定论：团队熟悉度往往大于技术差异

RTOS 的坑都长在"不熟悉"上：FreeRTOS 的优先级数值大=高、RT-Thread 数值小=高（对比 0），方向反了调度行为全反，这种坑只有"踩过一次"才记住。团队如果熟悉 FreeRTOS，切 RT-Thread 的学习成本（重新建心智模型 + 踩坑）往往大于两个 OS 之间的技术差异。

所以当平台和生态都没有压倒性倾向时（如 STM32 上做中等复杂度产品），**团队熟悉哪个用哪个**：这是最务实的选型。学习成本是真实的工程成本，不比技术差异小。

本项目场景练习

场景	建议	理由
霸天虎做传感器采集+串口上报	FreeRTOS	功能纯粹，内核即够
霸天虎做带屏+文件系统+网络的终端	RT-Thread 标准版	组件现成，生态提速
立创 S3 做小智语音终端	ESP-IDF（FreeRTOS）	平台绑定，协议栈深度集成
电池供电极简节点	RT-Thread Nano 或 FreeRTOS	两者皆可，看团队

场景练习的解读

四个场景覆盖了决策树的全部路径：

- **传感器采集+串口上报**：不碰联网/存储/UI/OTA（0 个组件），FreeRTOS 极简内核几 KB 就够，团队如果熟悉 FreeRTOS 直接选。RT-Thread Nano 也能做，但没有生态优势。
- **带屏+文件系统+网络终端**：碰 UI+存储+联网（3 个组件），RT-Thread 标准版的软件包能省几个月，生态优势压倒性。FreeRTOS 也能做但要自己拼 LwIP/FatFS/LVGL，工作量大。
- **小智语音终端**：平台绑定。S3 用 ESP-IDF，IDF 自带 FreeRTOS SMP 适配版，换 RTOS 是自找苦吃。协议栈+电源管理+双核调度都深度集成。
- **电池供电极简节点**：资源紧张（Flash < 32K）、功能纯粹，两者都能做。RT-Thread Nano 3K 起步、FreeRTOS 几 KB 起步，看团队熟悉度。这是"人决定论"的典型场景。

短自测

1. ESP32 项目为什么几乎一定用 IDF 自带 FreeRTOS？换 RTOS 的代价是什么？

> **▼ 看答案**
> ESP32 的 Wi-Fi/BLE 协议栈、电源管理、双核 SMP 调度都深度集成在 IDF 里，IDF 内置的就是 FreeRTOS SMP 适配版。换 RTOS 要把协议栈、电源管理、双核调度全部重新对接，工作量以人月计，且享受不到乐鑫的持续维护。换 RTOS 的成本远大于收益，所以"平台绑定"让选型答案几乎唯一。

2. 什么项目特征让你选 RT-Thread 标准版而不是 FreeRTOS？给出量化判断标准。

> **▼ 看答案**
> 项目要碰"联网/存储/UI/OTA"组件中的 3 个以上，RT-Thread 标准版的软件包中心（menuconfig 勾选+pkgs --update）能省几个月。量化标准：数你的项目要用的组件数，碰 3 个以上选 RT-Thread 标准版（生态优势压倒性），碰 0-1 个选 FreeRTOS（极简无生态开销），碰 2 个看团队熟悉度。

3. "团队熟悉 FreeRTOS，新项目用 RT-Thread 会更好"：这个决策什么时候成立、什么时候不成立？

▼ **看答案**
成立条件：RT-Thread 的生态优势（软件包省几个月）显著大于团队学习 RT-Thread 的成本（建心智模型+踩坑），即项目要碰 3+ 个组件且团队愿意承担学习成本。不成立条件：项目功能纯粹（0-1 个组件），RT-Thread 的生态优势用不上，团队切 RT-Thread 只有学习成本没有收益，这时"团队熟悉哪个用哪个"更务实。学习成本是真实工程成本，不比技术差异小。

4. 决策树里"资源紧张"分支为什么推荐 RT-Thread Nano 而非标准版？Nano 与标准版的资源差是多少？

▼ **看答案**
标准版要 ~32K Flash 起步（带 finsh+设备框架+组件），资源紧张（如 64K Flash 芯片）放不下或太挤。Nano 是 3K Flash 起步的裁剪版（只内核，无设备框架/finsh/组件），资源紧张时选 Nano 手工扩展需要的部分。Nano vs 标准版的资源差约 10 倍（3K vs 32K），差距来自组件层（设备框架/finsh/文件系统等）的有无。

5. 给"电池供电极简节点"场景，FreeRTOS 和 RT-Thread Nano 各有什么取舍点？

▼ **看答案**
两者都能做（几 KB Flash、μA 级睡眠时 RTOS 开销可忽略）。FreeRTOS：极简、社区大、资料多、Tickless Idle（低功耗 tick）成熟；RT-Thread Nano：3K 起步、rt_object 可观测性好（list_thread）、与 RT-Thread 标准版平滑升级（产品后期要加组件可上标准版）。取舍：看团队熟悉度 + 产品后期是否会加组件，会加选 Nano（平滑升级路径），不会加两者皆可。

对照表：本章概念 → 仓库与上游落点

概念	落点
决策树（平台/生态/人三定律）	本章决策树节 + 三句话版节
平台决定论（ESP-IDF 绑定）	P4 中断与双核 SMP 适配
生态决定论（软件包省时）	R6 Env 与 menuconfig 软件包中心
配置哲学对照	对比 0 机制深对照第三节
Nano vs 标准版资源差	R0 架构 Nano vs 标准版节
优先级方向迁移坑	对比 0 常见坑第一节
场景练习四例	本项目霸天虎/S3 场景；E01~E08

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **平台先定，生态其次，团队兜底；要拼产品选生态，要抠资源选极简。**

延伸：RTOS 选型决策树动画见 X1 动画；双 OS 同概念对照见 X0；FreeRTOS 入门见 F0，RT-Thread 入门见 R0。

常见坑

- **为"学习目的"在 product 项目里换 RTOS**：学习用开发板随便换，产品线换 OS=全量回归测试。
- **低估 SMP 差异**：双核 S3 上 FreeRTOS 的核间亲和/互斥实现与单核不同，换核不审阅，灵异 bug 遍地（P4 联动）。
- **把生态当免费午餐**：软件包质量参差，生产用前读 issue/看维护频率，R6 教的方法用起来。
- **只看 Flash 占位不看 RAM**：标准版 32K Flash 起步，但 finsh+设备框架+多任务栈的 RAM 开销也要算，64K SRAM 芯片可能 Flash 够 RAM 紧。
- **选型时忽略调试可观测性**：RT-Thread 的 finsh `list_*` 命令在调试期极有价值（省接调试器的时间），量产可裁剪；FreeRTOS 靠 Tracealyzer/手动插桩，调试效率也是选型因素。

你做到了

- RTOS 篇收官：机制、移植、对照、选型四关全过；
- 下次立项，30 分钟内能给出有依据的选型结论。

✅ RTOS 篇收官。下一站：P 篇 ESP32-S3：去双核+Wi-Fi 的世界里，看 FreeRTOS 的"出厂形态"。

PART PE

第七部分 ESP32-S3 + ESP-IDF

换一条技术路线：GPIO 矩阵、IDF 工程解剖、双核中断、驱动框架、Wi-Fi 与 ESP-NOW、分区表/NVS/OTA、低功耗、音频链路与看门狗健康监督。

> 换一条技术路线：GPIO 矩阵的引脚自由、双核与 IRAM 纪律、驱动框架的事件模型、Wi-Fi 状态机与退避重连、NVS/OTA 的掉电安全、音频链路的三道门。IDF 把 FreeRTOS 当出厂形态，P 篇也是 F 篇知识在真实产品里的落点。读完的标志：明白为什么 ISR 必须进 IRAM、拿到 IP 之后为什么还要自己写指数退避。

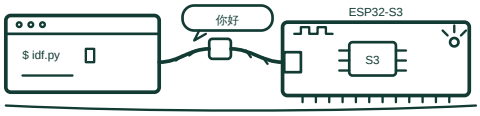
本部 14 章 · 约 525 分钟（8.8 小时）

55 . P0 环境搭建：ESP-IDF 与小智板的第一次对话
56 . P1 S3 架构与启动：从复位到 app_main
57 . P2 GPIO 与引脚矩阵：引脚自由换岗的秘密
58 . P3 IDF 工程解剖：组件化构建流水线
59 . P4 中断与双核：IRAM 纪律与核间分工
60 . P5 UART 驱动解析：成熟驱动长什么样
61 . P6 SPI/I2C 驱动框架：板载外设当教材
62 . P7 定时器与 LEDC：各管一段时间
63 . P8 Wi-Fi 精髓：事件循环与连接状态机
64 . P9 蓝牙与 ESP-NOW：协议栈的另一种打开方式
65 . P10 Flash、分区表、NVS 与 OTA
66 . P11 低功耗：睡眠矩阵与 ULP
67 . P12 音频链路：从 I2S 时序到"hello 语音
68 . P13 看门狗与健康监督：不要喂反了

通往单片机之路第 55 章

P0 环境搭建：ESP-IDF 与小智板的第一次对话

难度 ★☆☆☆☆ · 约 20 分钟



本章罗盘

问题

为什么装 ESP-IDF 要 1GB『全家桶』，idf.py build 和 flash 背后分别是谁在干活？

交付

一条可复跑的全链路记录：ESP-IDF CMD 里 idf.py --version 出 v5.5.x、设备管理器认出 CH340 COM 口、flash monitor 打印芯片信息并 10 秒周期重启，外加一张手默的最小工程结构图（顶层 CMakeLists/main 组件/sdkconfig/build）。

边界

本章只保证『跑得通』：idf.py 的调度细节 P3 才拆、三个 bin 的关系 P1 才讲；流程以 Windows 离线安装器 + 立创实战派 S3 为主线，macOS/Linux 与其他板卡不在射程。

依据锚点 a9 · mcu-code · mcu-lab · mcu-anim

> 🎯 给一块板子装"开发环境"，听起来像给手机装 App，其实更像办签证：编译器、烧录器、串口驱动、构建系统，四个章一个都不能少。这章一次盖齐，从此 `idf.py` 走天下。

① 安装
arm-none-eabi-gcc 15.2.1
+ OpenOCD
+ ST-Link 驱动

→

② 配置
Makefile 指定
`-mcpu=cortex-m4`
`-T xxx.ld`
链接脚本 + 启动文件

→

③ 编译
`make`
四步构建
E → S → c → T
✓ firmware.elf

→

④ 烧录
`make flash`
OpenOCD → SWD
→ Flash 0x08000000
✓ 校验通过

→

⑤ 验证
LED
闪烁

自查三命令

```
arm-none-eabi-gcc --version → 版本确认
arm-none-eabi-gcc -print-search-dirs → 搜索路径
where arm-none-eabi-gcc → PATH 优先级
```

STM32 (S0)
野火 F407 霸天虎
ST-Link V2 板载
`make MODE=0 flash`

ESP32 (P0)
ESP32-S3 小智板
USB-JTAG 内置
`idf.py flash monitor`

GD32 (G0)
复用 STM32 工具链
换 GD32 DFP 包
`make flash`（同 S0）

常见坑

1. PATH 混用多套工具链 → where gcc 查优先级
2. 目标芯片与工具链不匹配 → arm-none-eabi 编裸机，arm-linux 编 Linux

通用环境搭建五步闭环：安装→配置→编译→烧录→点灯

本章精髓

1. ESP-IDF 为什么是"全家桶"？因为编译器（xtensa-esp32s3）、构建系统（CMake+Ninja）、烧录调试（esptool/OpenOCD）、操作系统（FreeRTOS）、协议栈（Wi-Fi/BT）一站配齐，而且全部开源。
2. `idf.py` 一条命令背后发生了什么？它是 CMake 的"翻译官"：`idf.py build` ≈ cmake + ninja，`idf.py flash` ≈ esptool 走串口协议灌固件（P3 章拆给你看）。
3. 与 STM32 裸机的世界观差异：你的 `app_main` 一出生就跑在 FreeRTOS 任务里。RTOS 不是选修，是出厂配置。

先修

- 无。唯一要求：Windows 10/11 + 立创实战派 S3 板 + 一根**带数据能力**的 Type-C 线。

第一步：装 ESP-IDF（离线安装器，最稳）

装之前先押个出身：app_main 一睁眼，是裸机程序独占整机，还是生下来就跑在某个任务里？押完再往下装，装完翻本章精髓对账。

1. 打开乐鑫下载页 `dl.espressif.com/dl/esp-idf/`，选 **ESP-IDF v5.5.x Windows 离线安装器**（Offline Installer，约 1GB，自带 Python/Git/工具链，不挑网络）。
2. 安装时保持默认选项；目标芯片勾选 **ESP32-S3**（全选也无妨，多占点磁盘）。
3. 装完桌面出现两个快捷方式：**ESP-IDF 5.5 CMD** 和 **ESP-IDF 5.5 PowerShell**。这俩是注入了全部环境变量的"特权终端"，以后所有 `idf.py` 命令都在这里面敲。

验证（在 ESP-IDF CMD 里）：

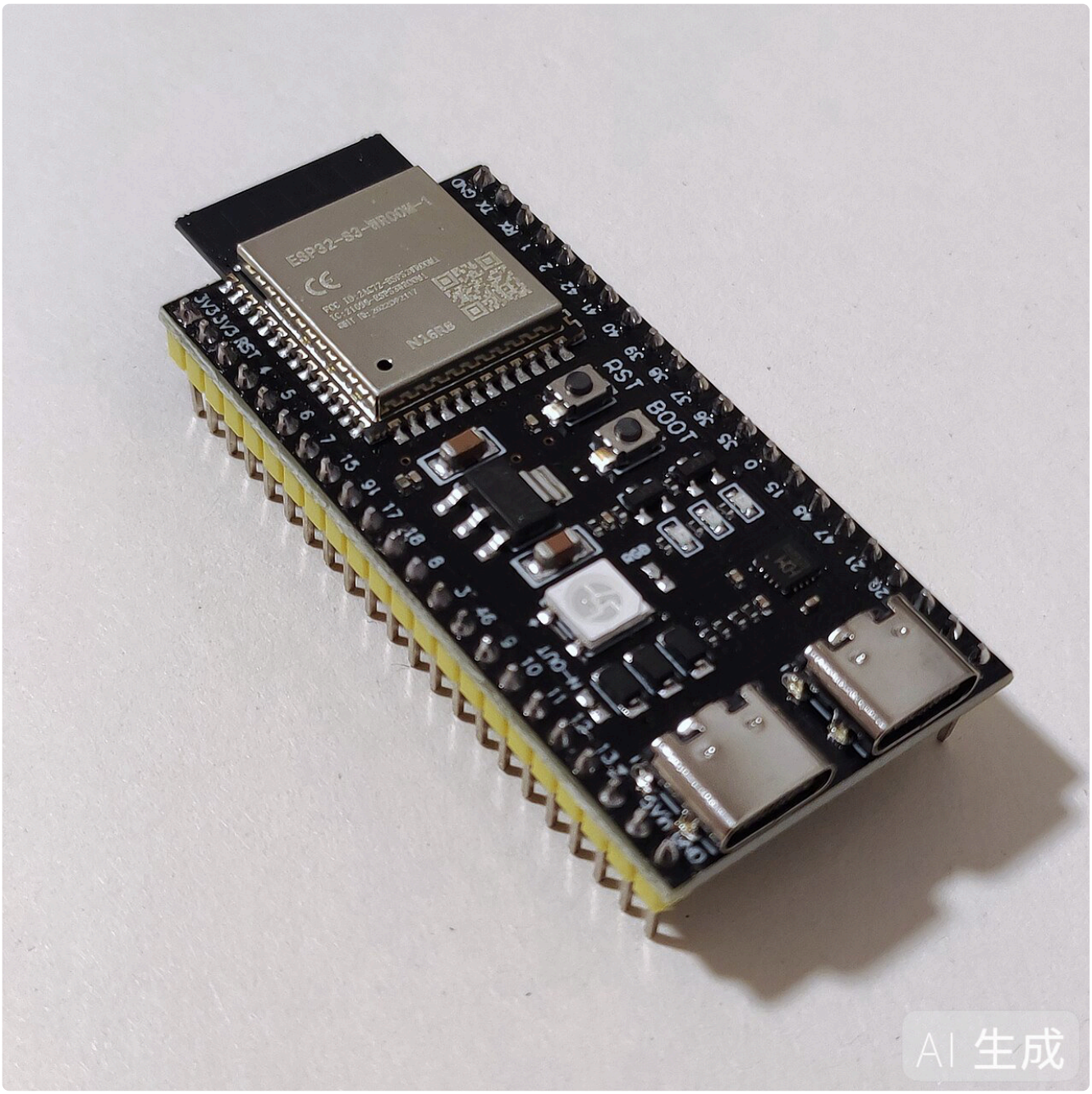
```
idf.py --version
```

看到 `ESP-IDF v5.5.x` 即成功。

> ⚠️ **注意·路径纪律**
>
> 安装路径别带中文/空格/特殊字符。默认 `C:\Espressif` 就很好。这条纪律和 STM32 侧完全一致：嵌入式工具链对路径的挑剔是祖传的。

第二步：认识你的板子

立创·实战派 ESP32-S3（小智板）核心配置：模组 **ESP32-S3-WROOM-1-N16R8**（16MB Flash + 8MB PSRAM，LX7 双核 240MHz），USB 转串口芯片 **CH340K**。板子 Type-C 口身兼三职：供电、烧录、串口监视，一根线搞定。板卡外观、原理图与引脚分配见 立创官方 wiki。



ESP32-S3-WROOM-1-N16R8 模组：屏蔽罩上印型号，N16R8 表示 16MB Flash + 8MB PSRAM

> 模组实物参考（一款搭载同款模组的通用开发板；来源：Wikimedia Commons，CC BY-SA 4.0，作者 VectorVoyager）。

插上电脑，打开设备管理器看"端口 (COM 和 LPT)"：

- 出现 `USB-SERIAL CH340 (COMx)` → 驱动 OK，记下 COM 号；
- 出现黄色感叹号或"未知设备" → 装 CH340 驱动（搜沁恒官网 CH341SER，CH340/CH340K 通用）。

第三步：拿到第一个工程

直接用仓库里配好的逐行注释版（和官方 hello_world 行为一致，注释更密）：

```
cd code/esp32/00-hello
```

工程最小结构，一眼看穿：

```
00-hello/
├── CMakeLists.txt          # 顶层：拉进 IDF 构建系统 + 起工程名
├── main/
│   ├── CMakeLists.txt      # 组件声明：我有哪些源文件
│   └── hello_main.c        # app_main 在这
└── sdkconfig               # build 后才生成：全部配置的"存档点"
```

第四步：指定目标芯片

```
idf.py set-target esp32s3
```

这步生成 `sdkconfig`（两千多行配置的默认值）并锁定工具链为 xtensa-esp32s3。**换芯片必须重新 set-target**，否则固件刷进去就是板砖候选人。

第五步：构建

```
idf.py build
```

首次构建 3–10 分钟（要编整个 IDF）。成功末尾：

```
Project build complete. To flash, run:
 idf.py -p PORT flash
```

产物在 `build/`：`hello-mcu.bin`（应用）、`bootloader.bin`（二级引导）、`partition-table.bin`（分区表）。三者什么关系，正是 P1 章的主题。

第六步：烧录 + 监视

把第二步记下的 COM 号代入（示例 COM5）：

```
idf.py -p COM5 flash monitor
```

一条命令干两件事：烧录（esptool 走串口灌三个 bin），然后自动打开串口监视。预期输出：

```
Hello! 这颗芯片是 ESP32-S3，2 核，16 MB 外挂 Flash
无线能力: Wi-Fi BLE
SRAM 剩余：3xxxxx 字节；PSRAM 剩余：83xxxxx 字节
10 秒后重启……
……
重启！
```

退出监视：`Ctrl + ]`。

恭喜，你已经和小智板完成了第一次对话。回头逐行对照 `hello_main.c`，每一行输出都能找到出处。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **IDF 四步曲：`set-target`（定芯片）→ `menuconfig`（改配置）→ `build`（编固件）→ `flash monitor`（烧+看）。** 首次可以不 menuconfig，但顺序不能乱。

延伸：环境搭建五步闭环动画见 P0 动画；IDF 与 STM32 工具链对照见 B0；menuconfig 与 RT-Thread 同源机制见 R6。

实物实验

本章即实验。观测点：

- 设备管理器出现 CH340 COM 口；
- `idf.py build` 末尾出现 "Project build complete"；
- monitor 打印芯片信息且 10 秒周期重启；
- 板载喇叭/屏幕本章不用，它们在 P12 音频链路 和 P6 驱动章 等你。

短自测

1. `idf.py build` 背后实际调用了哪两个工具？`idf.py flash` 又调用了谁？

> **▼ 参考答案**
> `idf.py build` 依次调用 **CMake**（生成构建文件）和 **Ninja**（执行并行编译）。`idf.py flash` 调用 **esptool.py**，它通过串口 bootloader 协议把固件写入 Flash。`idf.py` 本身只是 Python 封装：它解析子命令、拼参数、调外部工具，不直接编译也不直接烧录。

2. ESP-IDF 的 `app_main` 和标准 C 的 `main` 有什么本质区别？为什么说"FreeRTOS 不是选修"？

> **▼ 参考答案**
> 标准 C 的 `main` 是进程唯一的执行线程；ESP-IDF 的 `app_main` 一进入就已经跑在 FreeRTOS 的一个任务里（叫 `main task`，默认栈 8KB，优先级 1）。你可以从 `app_main` 里 `xTaskCreate` 创建更多任务，也可以 `return` 结束这个主任务，但其他任务继续运行。这意味着从第一行代码起，你就在多任务环境里：串口中断、Wi-Fi 事件、看门狗都在各自的任务或 ISR 里并行工作。

3. `idf.py set-target esp32` 误设成经典 ESP32（不是 S3），会怎样？怎么恢复？

> **▼ 参考答案**
> 经典 ESP32（Xtensa LX6 双核）与 ESP32-S3（Xtensa LX7 双核）架构不同，指令集和内存布局不兼容，刷错 target 的固件无法启动。恢复方法：`idf.py set-target esp32s3` 重新设定（会清空 `build/` 目录和 `sdkconfig`），然后重新 `idf.py build`。如果之前 `menuconfig` 改过配置，需要重设，因为 `sdkconfig` 已被覆盖。

常见坑

- **`idf.py` 不是内部或外部命令**：你在普通终端里。必须用"ESP-IDF 5.5 CMD/PowerShell"快捷方式，或先执行安装目录下的 `export.bat`。
- **烧录报 `Failed to connect to ESP32-S3`**：① COM 号选错；② 串口被别的程序占用（关掉串口助手/其它 monitor）；③ 线只有充电功能。极少数情况需手动进下载模式：按住 BOOT 键→点按 RESET→松开 BOOT。
- **monitor 里全是乱码**：波特率不对。IDF 默认 115200，`idf.py monitor` 会自动配对，用第三方串口工具要手动设。
- **set-target 成 esp32（不带 S3）**：classic ESP32 与 S3 架构不同，刷错 target 的固件起不来。`idf.py set-target esp32s3` 重来（会清 build 目录）。
- **公司网/代理导致在线安装器失败**：换离线安装器；或先装再配 `IDF_TOOLS_PATH` 镜像。

你做到了

- 盖齐四个"签证章"：编译器、构建、烧录、串口。
- 从源码到串口输出的全链路亲手跑通一遍。
- 建立了 ESP32 世界观：app_main 是任务，IDF 是全家桶。

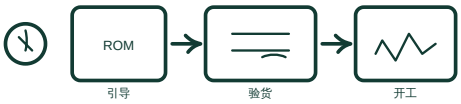
出身揭晓：app_main 生在 FreeRTOS 的任务里，不是裸机独占整机。这个答案会改写你读 E 篇每一章代码的姿势：主角不是从上到下跑一遍的 main，而是一个被调度的小卒。签证章盖齐之外，你还顺手领到了整台机器的运行规矩。

✅ 下一站：P1 S3 架构与启动全过程：bootloader、分区表、app 三段接力，讲清上电到你的 printf 之间发生了什么。

通往单片机之路第 56 章

P1 S3 架构与启动：从复位到 app_main

难度 ★★☆☆☆ · 约 35 分钟



本章罗盘

问题

从复位到 app_main 之间是谁在接力？串口只打到 rst:0x1 就没了该查哪一棒？

交付

一张『串口 banner 行 → 启动三棒』排查表和 app_main 身世证据链（esp_startup_start_app → xTaskCreatePinnedToCore(main_task) → app_main → vTaskDelete），配 idf.py partition-table 实表与 partitions_singleapp.csv 的对照记录。

边界

讲到『app_main 被谁调用』为止，main task 之后的调度与 SMP 细节归 RTOS 篇；IRAM 铁律只立不拆（P4 展开）；banner 对照与栈溢出实验标注『待上板回填』，现有证据是源码行号加本机 idf.py size/.map 构建产物，行号以 v5.5.2 为准。

依据锚点 `a7 · a9 · f2 · mcu-code · mcu-anim`

> 🎯 STM32 上电是"一条指令到 main"；ESP32-S3 上电是一场三棒接力：**ROM 引导 → 二级 bootloader → 你的 app**。多出来的两棒不是多余：它们换来了分区表、OTA、Flash 加密这些"出厂就是产品级"的能力。

本章精髓

1. 双核 LX7 + 内存分级：PRO_CPU/APP_CPU 两个核（`SOC_CPU_CORES_NUM = 2`），外部 Flash 经 **Cache 映射**执行，代码实际从"缓存窗口"跑，这决定了"ISR 要放 IRAM"等一系列铁律（P4 展开）。
2. 三棒启动：ROM（固化，查 strapping 脚决定启动模式）→ 二级 bootloader（初始化 Flash/Cache、读分区表、选 app 并校验）→ app（FreeRTOS 初始化 → 创建 main task → 调你的 `app_main`）。
3. 分区表是"地产证"：bootloader 不烧死 app 地址，而是读分区表（CSV 定义 `factory` / `ota` / `nvs` / `phy` 等）。OTA 双 app 轮换、参数存储全靠这张表。

- **能理解**：为什么你的 `app_main` **不是** `main()`：它跑在一个名叫 `main` 的 FreeRTOS 任务里（`xTaskCreatePinnedToCore(main_task, "main", ...)`）。
- **能用**：从复位到 `app_main` 的每一棒都能指到源码行号；出问题时会判断"卡在哪一棒"（ROM 打印？ bootloader 打印？ app 打印？）。

先修

- P0 环境与工具链：能跑 `idf.py build` 之后再来。
- S1 架构与内存：Flash 映射、向量表（STM32 侧的对照系）。
- B1 四步构建：预处理/编译/汇编/链接的视角，看 ESP 侧有何不同。

先跑起来（10 分钟 quick win）

```
cd code/esp32/00-hello
idf.py build          # 需要 IDF 环境（见 P0）
```

构建日志里找这几行（真实日志见 `.trellis/ref/idf-00-hello.log`）：

```
Project build complete. To flash, run:
 idf.py flash
```

monitor 起来之前先押一棒：banner 的第一行是谁打的？你写的代码，还是芯片里比你的代码老得多的那位？押完再看串口，第一行当场判分。

然后 `idf.py flash monitor`，串口会打出启动 banner。**banner 里的每一行都对应下面某一棒**，这是本章最实用的调试入口：

串口看到什么	说明卡在哪一棒
`rst:0x1 (POWERON)` + ROM 信息	ROM 引导（第一棒）——还没到 bootloader
`I (27) boot: ESP-IDF ...`	二级 bootloader（第二棒）正在跑
`I (xxx) app_start: Starting scheduler`	内核起来了（第三棒）
`I (xxx) app_start: Calling app_main()`	即将进你的代码

ROM 打印完就没了 / 反复重启 / 只有 `waiting for download`：三种现象分别指向三棒的不同故障，第 5 节逐条对照。

小节结构

小节	内容	四件套
双核与内存分级	PRO_CPU/APP_CPU；IRAM/DRAM；Cache 映射执行	配置
第一棒 ROM	固化引导、strapping 脚、下载模式	库解析
第二棒 bootloader	初始化 Flash/Cache、读分区表、校验并选 app	库解析
第三棒 app	`esp_startup_start_app` → main task → `app_main`	代码分析
分区表	CSV 四列含义、`factory` / `nvs` / `phy` 的真实条目	配置
和 STM32 对照	一条直线 vs 三棒接力；启动失败的排查入口	配置

一、双核与内存分级：先说清楚"跑在哪"

ESP32-S3 是**双核**：

```
#define SOC_CPU_CORES_NUM              2     // soc/esp32s3/include/soc/soc_caps.h:145
```

- **PRO_CPU**（CPU0）：协议核，负责 WiFi/蓝牙协议栈、启动流程。
- **APP_CPU**（CPU1）：应用核，跑你的任务。

这只是**默认分工**，不是硬约束：FreeRTOS 在 S3 上是 **SMP**（对称多处理）单实例，任务默认可以跑在任意核上，也可以用 `xTaskCreatePinnedToCore` 钉死到某个核（P4 展开）。

内存分级才是关键差异。S3 的地址空间里，代码可以待在两个地方：

位置	特点	谁在用
IRAM（内部 SRAM，可执行）	取指零等待，不受 Cache 影响	启动早期代码、被标记 `IRAM_ATTR` 的 ISR
Flash（外部，经 **Cache 映射**）	容量大、便宜；Cache 关闭时**不可取指**	绝大部分应用代码与 rodata

这就是那条铁律的来源：**普通代码放在 Flash 里，靠 Cache 映射窗口执行**。当系统要写 Flash（NVS 提交、OTA、分区操作）时会**暂时关闭 Cache**，此刻若发生中断，而 ISR 在 Flash 里，CPU 取指无门，直接崩。所以有 IRAM 纪律（P4 正面拆解）。

> STM32 侧没有这个概念：F407 的代码就在 Flash 上原地执行，不需要 Cache 窗口，也不存在"关 Cache 期间不能取指"。这是两平台**最本质的架构差异**。

二、第一棒 ROM：芯片里烧死的引导

先把三棒接力画出来：ROM 认路、bootloader 验货、app 开工，串口前缀就是探针。

ESP32-S3 上电三棒接力动画

ROM 引导是**固化在芯片里**的（不是你能改的代码），上电后第一段执行的就是它。它做三件事：

1. **读 strapping 脚**，决定启动模式。"按住 BOOT 键再复位"能进下载模式，就是因为这一步。
2. 做最早期的基础初始化（时钟、基础外设）。
3. 找到并跳转到**二级 bootloader**。

调试信号：串口第一行 `rst:0x1 (POWERON)` 之类就是 ROM 打印的。**如果连这行都没有**，说明问题比软件更早：供电、晶振、串口接线，或者芯片根本没在跑。

三、第二棒 bootloader：验货 + 选货

二级 bootloader 是一段**真实的、可升级的**代码（在 `components/bootloader/` 里）。它的主线在 `components/bootloader/subproject/main/bootloader_start.c`：

```
    bootloader_utility_load_boot_image(&bs, boot_index);   // bootloader_start.c:62
```

在这行之前，它要完成：

- 初始化 Flash 与 Cache 映射（否则后面的代码根本读不到）；
- **读分区表**，拿到各分区的地址与大小；
- 按顺序选择要启动的 app：先试 `factory`，再试 `ota_0` / `ota_1`，并**校验镜像**（校验失败就回退到另一个）。

调试信号：串口打 `I (xx) boot: ESP-IDF ...` 就是它。常见故障是**校验失败导致回退或重启**，此时日志里会有 `invalid header` / `image verification failed` 之类字样。

> **为什么要两棒而不是一棒**：ROM 是烧死的，不可升级。把"读分区表、校验、回退"这些需要**频繁演进**的逻辑放进可升级的二级 bootloader，芯片出厂后仍能修复启动逻辑、支持新分区方案。这是用两棒换来的灵活性。

四、第三棒 app：`app_main` 到底是谁调用的

这是本章最值得记住的一段源码。app 侧的启动主线在 `components/esp_system/startup.c`，末尾是：

```
    esp_startup_start_app();        // startup.c:222
    ESP_INFINITE_LOOP();
```

而 `esp_startup_start_app` 的实现**不在 `esp_system`，在 `freertos`**，见 `components/freertos/app_startup.c:64`：

```
void esp_startup_start_app(void)
{
    ...
    esp_crosscore_int_init();                   // 初始化核间中断（CPU0）

    BaseType_t res = xTaskCreatePinnedToCore(main_task, "main",
                                             ESP_TASK_MAIN_STACK, NULL,
                                             ESP_TASK_MAIN_PRIO, NULL, ESP_TASK_MAIN_CORE);
    assert(res == pdTRUE);

    ESP_EARLY_LOGD(APP_START_TAG, "Starting scheduler on CPU0");
    vTaskStartScheduler();                      // 从这里开始，调度器接管
}
```

关键认识：`app_main` 不是 `main()`。 它是一个叫 `main` 的 FreeRTOS 任务里的函数调用（`app_startup.c:158-212`）：

```
static void main_task(void* args)
{
    ESP_LOGI(MAIN_TAG, "Started on CPU%d", (int)xPortGetCoreID());
    ...
    ESP_LOGI(MAIN_TAG, "Calling app_main()");
    extern void app_main(void);
    app_main();                         // ← 你的代码在这里被调用
    ESP_LOGI(MAIN_TAG, "Returned from app_main()");
    vTaskDelete(NULL);                  // ← app_main 返回，任务自杀、栈被回收
}
```

这段话信息量很大：

- **`app_main` 返回不致命**，任务会自我删除（`vTaskDelete(NULL)`），栈内存被回收。而 STM32 上 `main()` 返回是未定义行为。
- **`app_main` 跑在哪个核**由 `ESP_TASK_MAIN_CORE` 决定，它来自 `CONFIG_ESP_MAIN_TASK_AFFINITY`（`components/esp_system/Kconfig:233` 附近，**默认 `ESP_MAIN_TASK_AFFINITY_CPU0`**）。
- **它的栈多大**由 `ESP_TASK_MAIN_STACK` 决定 = `CONFIG_ESP_MAIN_TASK_STACK_SIZE`（`components/esp_system/include/esp_task.h:57`）。这个 Kconfig 项的**默认值是 3584**（字节），定义在 `components/esp_system/Kconfig:227`：

```
    config ESP_MAIN_TASK_STACK_SIZE
        int "Main task stack size"
        default 3584
```

- **它的优先级**是 `ESP_TASK_MAIN_PRIO = ESP_TASK_PRIO_MIN + 1`（`esp_task.h:56`），即**倒数第二低**。你的 `app_main` 优先级很低，这解释了"为什么初始化任务常常一创建就抢走 CPU"。

> 顺带：`main` 任务的栈**不像别的任务那样从 heap 切**，它有独立的启动栈（`heap_caps_enable_nonos_stack_heaps()` 那句注释说的就是"启动栈现在可以还给 heap 了"）。

五、分区表：bootloader 的"地产证"

bootloader 怎么知道 app 在哪？**不硬编码地址，而是读分区表**。单 app 方案的默认表在 `components/partition_table/partitions_singleapp.csv`：

```
# Name,   Type, SubType, Offset,  Size, Flags
nvs,      data, nvs,     ,        0x6000,
phy_init, data, phy,     ,        0x1000,
factory,  app,  factory, ,        1M,
```

四列的含义：

列	含义	例子
`Name`	分区名，代码里用它找分区（`esp_partition_find`）	`nvs` / `factory`
`Type`	大类：`app` 还是 `data`	`app`
`SubType`	子类型：app 有 `factory` / `ota_0` …；data 有 `nvs` / `phy` / `fat` …	`factory`
`Size`	大小；`1M` = 1MB	`0x6000` = 24KB

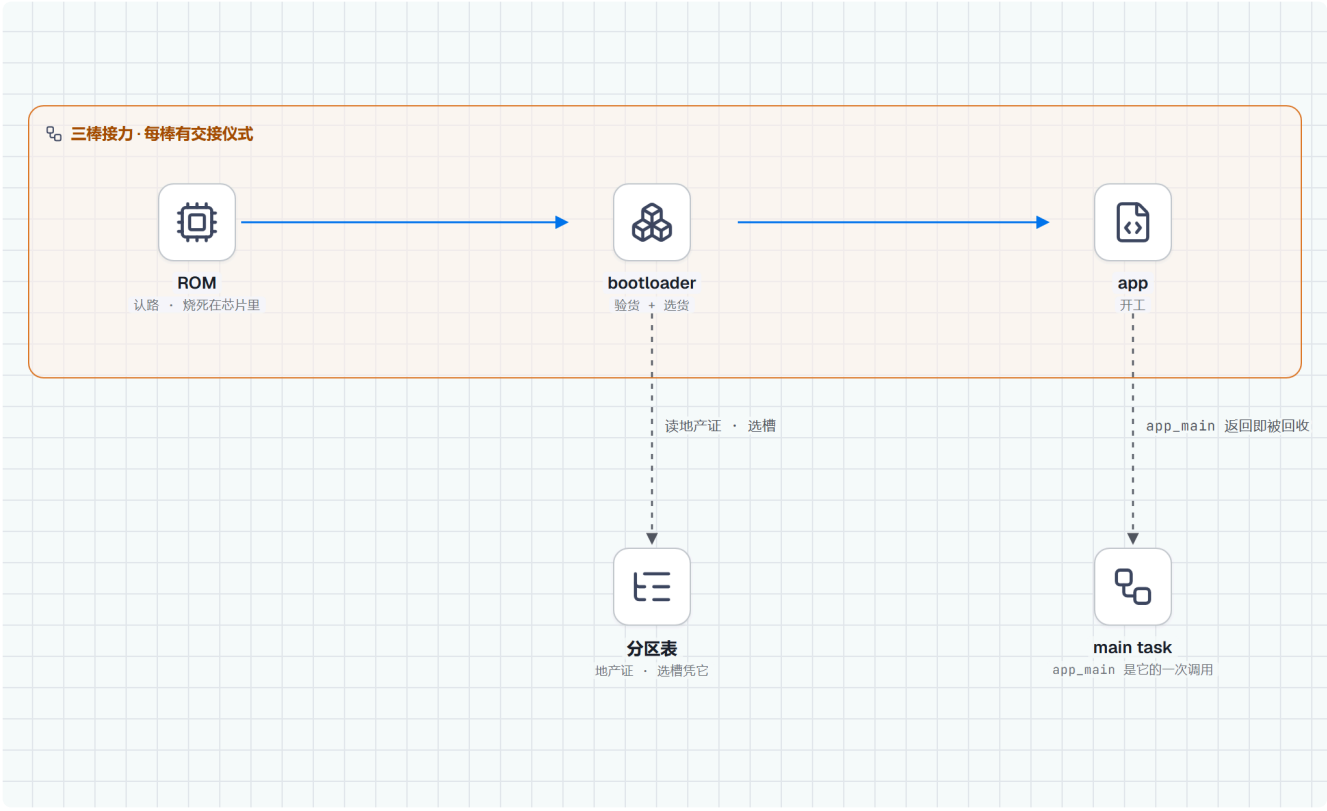
`Offset` 那一列留空是有意的：留空表示"由工具自动排布"（`gen_esp32part.py` 负责填），避免手写地址撞车。

调试信号：`idf.py partition-table` 能打印出**实际生成**的表（含工具填好的偏移）。**分区表是 OTA 的地基**，双 app 轮换就是往 `ota_0` / `ota_1` 两个分区里交替烧、再改引导指针。

> 对照 STM32：B3 链接脚本 里我们是**手写** `MEMORY` 段把 Flash 切成几个区域；ESP32 把这件事**数据化**成了一张可运行时查询的表。代价是必须多一棒 bootloader 来读它。

六、和 STM32 对照：一条直线 vs 三棒接力

对照之前先看静态版图：三棒的位置关系与两条依赖（bootloader 读地产证选槽、app_main 寄居 main task），动画演过程，这张图定结构。



S3 上电三棒静态结构：ROM→bootloader→app，依赖分区表与 main task

	STM32（F407）	ESP32-S3
上电第一步	复位向量 → `Reset_Handler`	ROM 引导（固化）
中间层	无	二级 bootloader（可升级）
到 main 的路径	启动文件汇编 → `main()`	bootloader → app → FreeRTOS → main task → `app_main()`
代码执行位置	Flash 原地执行	Flash 经 Cache 映射（关 Cache 时不可取指）
分区信息	链接脚本里写死	运行时读分区表
`main` 返回	未定义行为	任务自杀，栈回收

排查入口就是本章 quick win 的那张表：**看串口打印停在哪一棒**。这是 ESP32 调试的第一性方法，比 STM32 侧还重要，因为多出来的两棒都在你看到 `app_main` 之前。

附录：工程完整源码

00-hello 的工程描述（最小 IDF 工程的 `CMakeLists.txt`，P3 会逐行解剖它）：

<<< ../../code/esp32/00-hello/CMakeLists.txt

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **上电三棒：ROM 认路、bootloader 验货、app 开工；`app_main` 是 main task 里的一次调用，返回就被回收。**

延伸：启动接力动画见 P1 动画；STM32 启动流程对照见 B4；FreeRTOS 任务创建在 F1 展开。

实物实验

- `idf.py build` 后 `idf.py flash monitor`，把串口 banner 逐行对照第五节那张"卡在哪一棒"的表（**待上板回填**）。
- `idf.py partition-table` 打印实际分区表，与 `partitions_singleapp.csv` 对照，找出工具自动填上的 `Offset` 值。
- 故意把 `CONFIG_ESP_MAIN_TASK_STACK_SIZE` 改小（如 1024），在 `app_main` 里加一个大数组，观察栈溢出时的报错，这是 F7 那套 RAM 账在 ESP 侧的同一门课。
- 查 `idf.py size` 的输出，看你的代码有多少落在 IRAM、多少在 Flash。本机实测（`code/esp32/09-iram-discipline`，ESP-IDF v5.5.2，目标 esp32s3，`sh probe.sh` 第 7 段可复现）：IRAM `.text` 15356 字节 + `.vectors` 1028 字节（这一行 size 工具按 16384 字节计，显示 100%）；DIRAM 用 55463 / 341760 字节（16.23%）；Flash Code 78770 字节、Flash Data 41324 字节，整镜像 189693 字节。注意这张表的分组是 size 工具自己的口径，链接器真正用的区间要看 `.map` 里的 `Memory Configuration`（同一脚本第 3 段打印：`iram0_0_seg` 起 `0x40374000` 长 `0x57700`）。

常见坑

- **以为 `app_main` 就是 `main`**：它跑在一个叫 `main` 的 FreeRTOS 任务里（`app_startup.c:158`），优先级倒数第二低、栈 3584 字节起，这些都能配置，别当固定值。
- **在 `app_main` 返回后还想跑代码**：返回即任务自杀（`vTaskDelete(NULL)`）。要常驻就自己创建任务或写成死循环。
- **把 ISR 随手放 Flash**：Cache 关闭期间不可取指 → 崩。这是 IRAM 纪律的由来（P4 详解）。
- **手写分区偏移**：`Offset` 留空让工具排布；手写容易与 bootloader/NVS 区撞车。
- **忽略启动日志**：ESP32 的启动日志是分层的，`rst:` 来自 ROM、`boot:` 来自 bootloader、`app_start:` 来自 app，**读日志前缀就能定位故障在哪一棒**。

短自测

1. 为什么 ESP32 需要二级 bootloader，STM32 不需要？

▼ **看答案**
因为 ROM 里的引导代码是**固化不可升级**的。把"读分区表、校验镜像、OTA 回退"这些需要频繁演进的逻辑放进**可升级**的二级 bootloader，芯片出厂后仍能修复启动逻辑、支持新分区方案。STM32 没有分区表/OTA 这套运行时启动策略，链接脚本写死地址即可，所以启动文件一条直线到 `main()`。

2. `app_main()` 是被谁调用的？它返回后会怎样？

▼ **看答案**
被 `main_task()` 调用（`components/freertos/app_startup.c:158` 起，调用点在 `app_main();` 那行）。这个任务由 `esp_startup_start_app()` 用 `xTaskCreatePinnedToCore(main_task, "main", ...)` 创建，随后 `vTaskStartScheduler()` 启动调度器。`app_main` 返回后打印 "Returned from app_main()"，然后 `vTaskDelete(NULL)`，**任务自杀、栈被回收**（不像 STM32 的 `main` 返回是未定义行为）。

3. 串口只打出 `rst:0x1 (POWERON)` 就没了，问题在哪一棒？

> **▼ 看答案**
> **第一棒 ROM 之后就没接上**。`rst:` 是 ROM 打印的，紧接着应该出现 bootloader 的 `I (xx) boot:` 行。没有它说明 ROM 找不到/跳不进二级 bootloader：可能是分区表或 bootloader 镜像损坏、Flash 里还是空白、或芯片在下载模式等待。先确认 `idf.py flash` 真的写进去了。

4. `SOC_CPU_CORES_NUM` 是多少？它在哪个文件？

> **▼ 看答案**
> **2**。定义在 `components/soc/esp32s3/include/soc/soc_caps.h:145`。这决定了 FreeRTOS 在 S3 上是 SMP 配置（任务可跑任意核），也是 `xTaskCreatePinnedToCore` 有意义的前提。

5. 分区表 CSV 里 `Offset` 一列留空是什么意思？

> **▼ 看答案**
> 表示**由工具自动排布**（`gen_esp32part.py` 根据各分区大小顺序填充），避免手写地址互相撞车。真实表可以用 `idf.py partition-table` 打印出来看填充后的结果。默认单 app 表见 `components/partition_table/partitions_singleapp.csv`。

对照表：本章概念 → 仓库落点

概念	仓库落点
双核数量（`SOC_CPU_CORES_NUM = 2`）	`$IDF_PATH/components/soc/esp32s3/include/soc/soc_caps.h:145`
app 启动入口 `esp_startup_start_app()` 的调用	`$IDF_PATH/components/esp_system/startup.c:222`
创建 main task + 启动调度器	`$IDF_PATH/components/freertos/app_startup.c:64`（`xTaskCreatePinnedToCore` → `vTaskStartScheduler`）
`main_task` 本体与 `app_main()` 调用点	同上 `:158`（`Calling app_main()` 日志与 `app_main();` 那行）
main 任务优先级 / 栈 / 核	`$IDF_PATH/components/esp_system/include/esp_task.h:56-58`
主任务栈默认 3584 字节	`$IDF_PATH/components/esp_system/Kconfig:227-234`
二级 bootloader 选镜像	`$IDF_PATH/components/bootloader/subproject/main/bootloader_start.c:62`
默认分区表 CSV	`$IDF_PATH/components/partition_table/partitions_singleapp.csv`
分区表工具	`$IDF_PATH/components/partition_table/gen_esp32part.py`
最小 ESP32 工程	`code/esp32/00-hello/`（`CMakeLists.txt` + `main/`）
真实构建日志	`.trellis/ref/idf-00-hello.log`

> 表中 `$IDF_PATH` = `C:/Users/zhugu/.espressif/v5.5.2/esp-idf`（本机 ESP-IDF v5.5.2）。路径与行号以该版本为准，换版本需重核。

你做到了

- 上电三棒在你眼里不再是黑箱，每一棒都能指到源码行号；
- 知道 `app_main` 的真实身份，以及它的优先级/栈从哪来；
- 拿到一块出问题的板子，会先看串口日志前缀定位到"哪一棒"，而不是盲目重烧。

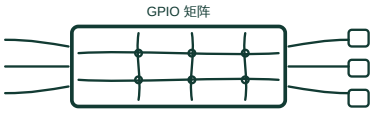
你押的那一棒，赢家是 ROM：出厂就焊死在芯片里，改不了删不掉，你的代码再大也轮不到它先开口。三段接力，第一棒不由你写，往后两棒才是你的地盘。

✅ 下一站：P3 IDF 工程解剖：把那几个 `CMakeLists.txt` 和 `sdkconfig` 拆开看，理解这条流水线怎么把上千个文件编成你刚烧进去的镜像。

通往单片机之路第 57 章

P2 GPIO 与引脚矩阵：引脚自由换岗的秘密

难度 ★★★☆☆ · 约 25 分钟



本章罗盘

问题

ESP32-S3 为什么任何外设都能接到几乎任何引脚，哪些引脚偏偏不行？

交付

一张 GPIO 矩阵路由图和 strapping/Flash/USB 引脚『禁区』清单，配一份可复跑的 io_mux 对照实验。

边界

矩阵解决数字信号路由；引脚的电气规格（驱动能力/耐压）仍要查数据手册，矩阵改变不了。

依据锚点 `a7 · a8 · a9 · f2 · mcu-code`

> 🎯 波形从旧引脚消失了，新引脚却一片安静。外设没停，信号也没丢，它可能只是被送到一个不存在、被占用或仍处于输入态的“工位”。ESP32-S3 的 GPIO Matrix 像总机，能让外设信号自由换岗；真正难的不是拨号，而是确认新工位**存在、空闲、接得通**。

本章精髓

1. **两条路进引脚**：**IO_MUX**（直连快车道：信号↔固定引脚，延迟低，少数高速信号必须走它）与 **GPIO Matrix**（交换矩阵：外设信号↔任意有效引脚，灵活但多一级延迟）。选型 = 速度 vs 布局自由。
2. **"任意引脚"的真实含义**：任意 **有效焊盘 + 该脚支持的输入/输出能力 + 模组没占用 + 板卡实际接出来** 的引脚。S3 芯片有 45 个物理 GPIO（0~21、26~48），但 N16R8 模组的 FLASH+PSRAM 占掉 12 个（八线 PSRAM 占 IO35/36/37 一带），板上真正引出来的更少，立创实战派的多功能扩展口只引出 **GPIO10、GPIO11**。
3. **焊盘属性是独立一维**：上下拉/驱动强度/开漏/毛刺过滤（S3 新增），与路由无关，`gpio_set_pull_mode` 家族管这片；路由对了但方向没配，信号照样出不来。

- **能理解**：为什么"任意映射"不等于"随便用"；为什么 PSRAM 占用的脚一碰就崩。
- **能用**：用 `uart_set_pin` / LEDC 的 `gpio_num` 把信号在两个扩展口脚之间换岗，并用仪器确认。

先修

- P1 架构；S3 STM32 GPIO（复用概念对照）。

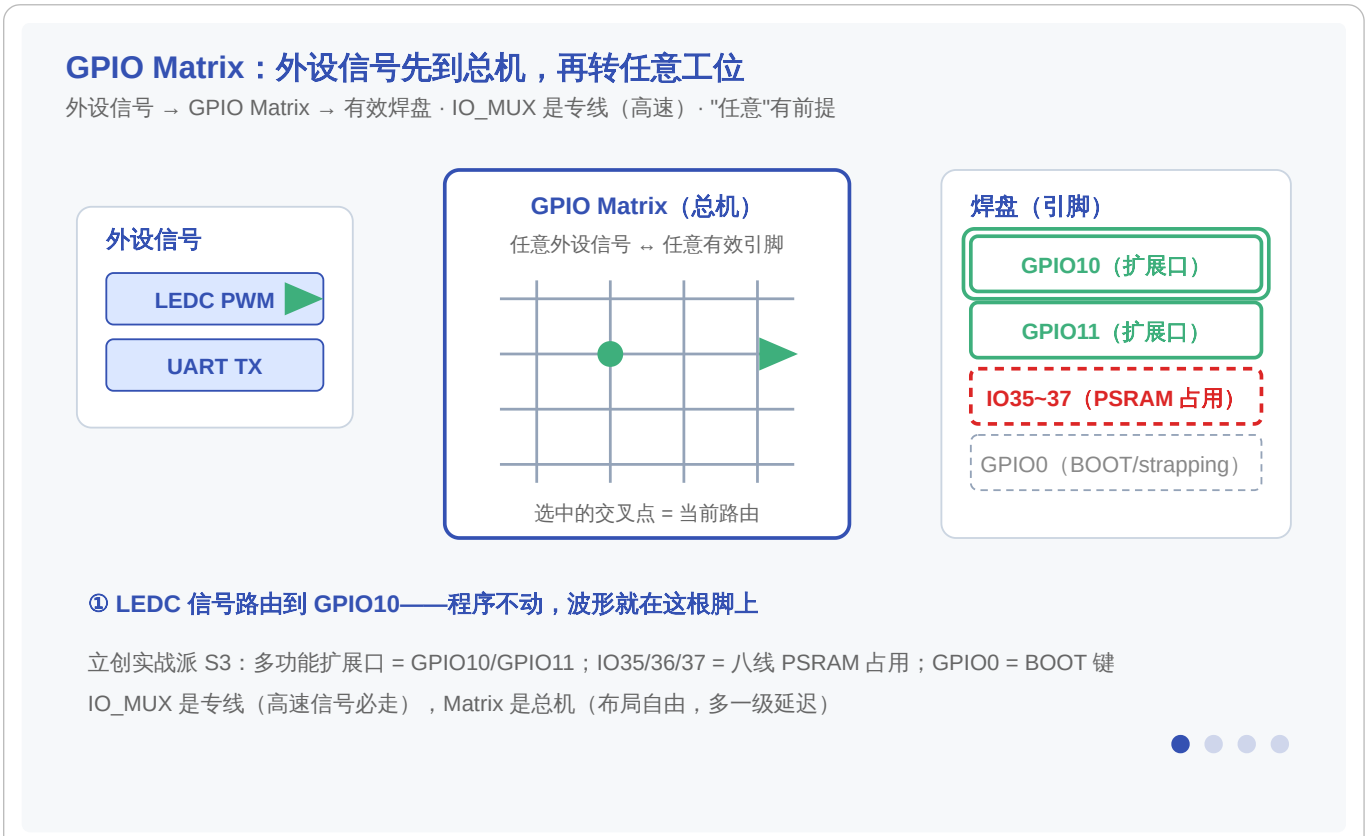
先跑起来（10 分钟 quick win）

`code/esp32/01-gpio-matrix`：同一个 LEDC PWM 信号，先在 GPIO10 上出波形，三秒后换到 GPIO11，程序不动、波形换脚。再用 `uart_set_pin` 把 UART1 换到扩展口两个脚（见 P5 的 `02-uart-events`）。

运行前先押结果：换岗后旧脚会立刻安静，还是两个脚一起输出？新脚没波形时，你会先改路由、改方向，还是换一个未占用焊盘？工程里的 `gpio_reset_pin(旧引脚)` 会给第一题答案；后面“路由线”和“焊盘线”分开检查，会给第二题一套不靠乱试的顺序。

动画：信号怎么走到焊盘

外设信号出发 → 到交换矩阵 → 选一个有效焊盘 → 焊盘属性（方向/上下拉）生效。盯住"被占用的脚"那一栏：PSRAM 占用的 IO35~37 直接打叉，驱动不拦你（它只查焊盘合法性），失败发生在物理层：波形出不来，还会干扰 PSRAM 总线；若固件使能了 octal PSRAM，可能当场跑飞。



GPIO 矩阵路由动画

板卡与版本前提

- 板卡：立创·实战派 ESP32-S3（模组 ESP32-S3-WROOM-1-N16R8，FLASH 16MB + PSRAM 8MB）。
- 板上用户可自由使用的引脚（立创 wiki 核实）：**BOOT 键 = GPIO0**（输入）、**多功能扩展口 = GPIO10、GPIO11**；I2C 扩展口与板载传感器共用（只能当 I2C 用）。
- **IO35/36/37 被八线 PSRAM 占用，不可用**；IO46 下载模式必须低电平（板上有下拉电阻）。
- 框架：ESP-IDF **v5.5.2**（本机实测版本，读 `esp_idf_version.h` 确认）。

小节结构

小节	内容	四件套
一、两条路由	IO_MUX vs Matrix 结构图与延迟差	配置
二、输出路由	信号源枚举 + 引脚选择	库解析
三、输入路由	反向路由与"一信号多听"	库解析
四、焊盘属性	上下拉/驱动/开漏/毛刺过滤	配置
五、strapping 避让	上电采样脚清单与设计禁忌	引脚
六、代码分析	`01-gpio-matrix/main.c` 的换岗全链路	代码分析

一、两条路由：专线 vs 总机

外设信号到引脚有两条路：

- **IO_MUX**：信号直接绑到固定引脚，中间不过交换矩阵，延迟低、时序干净。SPI 高速模式等场合必须走它，这也是"某些脚是某外设的'原生脚'"的来源。
- **GPIO Matrix**：信号先进交换矩阵，再路由到任意有效引脚：灵活，代价是多一级延迟。

STM32 的 AF 复用表相当于"只有专线、可选工位有限"；S3 的 Matrix 相当于"总机可转任意工位"。代价对照：**Matrix 多一级延迟，高速信号走 IO_MUX**。

二、输出路由：信号源选外设，引脚选目的地

输出的路由方向是"外设 → Matrix → 引脚"。IDF 里你通常不直接写 Matrix 寄存器，而是用外设驱动的引脚参数（比如 LEDC 的 `gpio_num`、`uart_set_pin` 的 TX/RX）。驱动内部帮你把信号路由过去。

要点：路由成功后**焊盘方向仍要单独配**（输出使能是独立一步），驱动一般会帮你做，但手写 Matrix 路由时这是最容易漏的一步。

三、输入路由：反向路由与"一信号多听"

输入方向相反：引脚信号 → Matrix → 外设输入。Matrix 的一个特性是**一个引脚信号可以同时喂给多个外设输入**：比如把一根线同时接进 UART RX 和 RMT 的输入做协议分析。这是 STM32 的 AF 复用做不到的。

四、焊盘属性：与路由无关的另一维

焊盘属性与路由是两条独立的线

上下拉、驱动强度、开漏、毛刺过滤（S3 新增）都由焊盘电路管，与信号路由完全独立。所以配置顺序是两条线并行：

- 路由线：`uart_set_pin` / `ledc_channel_config`（信号去哪）
- 焊盘线：`gpio_set_pull_mode` / `gpio_set_drive_capability`（脚的电气属性）

五、strapping 避让：上电那一拍的角色

GPIO0（BOOT）、GPIO45、GPIO46 等在上电/复位时被采样，决定启动模式。**上电瞬间这些脚不是普通 GPIO**，外接上拉/下拉错了，板子每次上电都进下载模式。本板的 IO46 已经板载下拉电阻（保证下载模式安全）；你要用的是 GPIO0（BOOT 键）时记住它默认上拉、按下为低。

六、代码分析：`01-gpio-matrix/main.c`

- `ledc_channel_config(.gpio_num = PAD_A)`：LEDC 通道绑到 GPIO10；
- `ledc_set_pin(PAD_B, ...)`：三秒后同一信号换到 GPIO11，**Matrix 重新路由，程序其余部分不动**；
- `gpio_reset_pin(旧引脚)`：先断开旧输出再路由新引脚，否则一个信号可同时留在两个输出脚；GPIO35/36/37 被模组占用，示例不向这些脚发送测试信号；
- `SOC_LEDC_CHANNEL_NUM` / `SOC_LEDC_TIMER_BIT_WIDTH` 打印：芯片能力以 `soc_caps.h` 为准，不是凭印象。

这段代码最值得故意遮住的一行是“断开旧输出”。先猜删掉它会怎样，再看 Matrix 的真实能力：一个信号可以继续留在旧脚，同时被送到新脚。换岗不是天然的“搬走”，而是重新接线；是否拆掉旧线，是你的责任。开场那条消失的波形，也因此有了两条完全不同的嫌疑路径：信号没接到新脚，或旧脚被主动复位而新脚的焊盘条件不成立。

附录：工程完整源码

<<< ../../code/esp32/01-gpio-matrix/main/main.c

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **IO_MUX 是专线，Matrix 是总机；高速走专线，布局走总机；"任意"= 有效焊盘 + 没被占用 + 板卡接出来；strapping 脚上电别惹。**

延伸：GPIO 矩阵路由动画见 P2 动画；STM32 GPIO 对照见 S3；strapping 引脚与启动模式在 P1 启动 有完整列表。

实物实验

- quick win 的 PWM 换岗：示波器/逻辑分析仪先夹 GPIO10，三秒后波形消失、出现在 GPIO11；
- 资料核对：在模组资料中找出 GPIO35/36/37 的 PSRAM 用途，不通过向占用脚输出信号验证；
- "一信号多听"（选做）：UART TX 同时路由给 UART RX 与另一外设输入，两路分析仪通道对照。

常见坑

- **strapping 脚当普通 IO**：GPIO0（BOOT）上拉电阻接错，板子每次上电进下载模式；
- **Flash/PSRAM 占用脚乱动**：八线 PSRAM 占用 IO35/36/37 一带，一碰就崩；
- **Matrix 路由后忘配焊盘方向**：信号路由对了，引脚还是输入态，输出使能是独立一步；
- **USB D+/D- 脚（GPIO19/20）当 GPIO**：用原生 USB 时这两脚有主，冲突外设先让位；
- **把"任意引脚"当"随便哪个脚"**：模组占用脚、strapping 脚、没引出的脚都不在内。

短自测

1. IO_MUX 和 GPIO Matrix 的区别是什么？什么信号必须走 IO_MUX？

> **▼ 参考答案**
> IO_MUX 是信号到固定引脚的直连（延迟低、时序干净）；GPIO Matrix 是交换矩阵（外设信号路由到任意有效引脚，多一级延迟）。高速信号（如部分 SPI 模式）必须走 IO_MUX。

2. S3 上"任意引脚"的真实前提是什么？

> **▼ 参考答案**
> 四个条件同时成立：引脚是有效焊盘（S3 有 45 个物理 GPIO）；该脚支持你要的方向/能力；模组没占用（FLASH/PSRAM 占 12 个，N16R8 的八线 PSRAM 占 IO35~37）；板卡把它实际引出来了。

3. 为什么 strapping 引脚不能随便接上拉/下拉？

> **▼ 参考答案**
> 上电/复位瞬间这些脚被采样以决定启动模式，不是普通 GPIO。上拉/下拉接错了，板子每次上电都进下载模式或起不来。

对照表：本章概念 → 仓库与上游落点

概念	落点
IO_MUX vs Matrix	ESP-IDF GPIO 文档（v5.5.2）+ TRM
芯片能力宏	`components/soc/esp32s3/include/soc/soc_caps.h`
板卡引脚分配	立创 wiki（GPIO0=BOOT、GPIO10/11=扩展口、IO35~37=PSRAM）
换岗实验	code/esp32/01-gpio-matrix
动画	gpio-matrix-routing.svg

你做到了

- "引脚自由"从宣传语变成可配可测的机制；
- S/P 两篇的 GPIO 观完成对照：固定 AF 表 vs 自由矩阵，各有设计哲学；
- "任意引脚"的四个前提，你以后不会再被宣传语带偏。

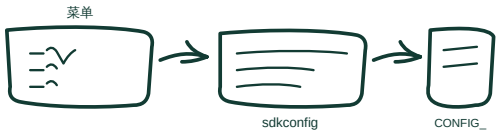
回到开场那次失败换岗：先问焊盘是否可用，再问信号是否路由，最后问方向与电气属性是否生效。把“总机接线”和“工位通电”分开查，GPIO Matrix 就从玄学宣传语变成一张可逐段验通的线路图。

✅ 下一站：P3 IDF 工程解剖：组件化 CMake 与 Kconfig 的流水线全拆。

通往单片机之路第 58 章

P3 IDF 工程解剖：组件化构建流水线

难度 ★★☆☆☆ · 约 35 分钟



本章罗盘

问题

三行 CMakeLists 凭什么编出上千个文件？menuconfig 里的选择怎么变成代码里的宏？

交付

一条 CONFIG 项的三段取证链：Kconfig 定义 → sdkconfig 固化值 → sdkconfig.h 宏各记一处（gpio-matrix 实测 890 项），加一份删掉 REQUIRES 后的构建报错样本和一个自己拆的最小组件骨架。

边界

拆的是 IDF 构建流水线的『形状』，CMake 语言与手写 Makefile 对照归 B7；idf.py --help 子命令对照标注『待上机回填』；890 项是 gpio-matrix 工程的实测口径，换工程换版本数字会变；源码行号以本机 v5.5.2 为准。

依据锚点 `a9 · f2 · mcu-code · mcu-anim`

> 🎯 STM32 侧我们手写 Makefile 管 4 个文件；IDF 管着上千个文件、上百个组件还很从容。秘密两条：**组件化**（每部分源码是一个自描述的 CMake 组件）与 **Kconfig**（用菜单决定编什么）。这一章把流水线拆开——这正是 ESP 侧的"库文件解析"主场。

本章精髓

1. 组件 = 自描述的源码包：`idf_component_register(SRCS ... INCLUDE_DIRS ... REQUIRES ...)` 声明源码/头文件/依赖，IDF 按依赖图拓扑排序编译，`main` 只是"一个叫 main 的特殊组件"。
2. Kconfig 三段论：组件自带的 `Kconfig` 文件描述选项 → `menuconfig` 写 `sdkconfig`（文本）→ 构建生成 `sdkconfig.h`（宏）。归拢成一句：**菜单 → 文本 → 宏**，代码里 `#if CONFIG_XXX` 应声生效。
3. `idf.py` 是司令不是士兵：`idf.py build` = cmake 配置 + 构建器干活；`flash` 要调 esptool；`menuconfig` 要调 kconfig 工具，它是调度层，不是编译器。

- **能理解**：你那个只有三行的顶层 `CMakeLists.txt` 为什么就够了：`include($ENV{IDF_PATH}/tools/cmake/project.cmake)` 一行把整套规则拉进了你的工程。
- **能用**：新建一个自己的组件（哪怕只是把 LED 驱动从 `main` 里拆出去），知道该写哪几行；出构建错误时知道去翻 `sdkconfig` 还是 `CMakeLists.txt`。

先修

- P0 环境与工具链：能跑 `idf.py build`。
- P1 S3 架构与启动：三棒启动，本章讲的是"第三棒的代码是怎么被编出来的"。
- B7 构建系统：Makefile/CMake 的基本概念，本章做对照。

先跑起来（10 分钟 quick win）

```
cd code/esp32/00-hello
idf.py build
```

构建完，去 `build/config/` 目录看三个**自动生成**的文件：

```
ls build/config/
# sdkconfig.h  sdkconfig.cmake  sdkconfig.json
```

`build/config/sdkconfig.h` 就是"菜单 → 文本 → 宏"的终点站。打开它搜一个你认识的配置项（比如 `CONFIG_IDF_TARGET`），再回到工程根目录的 `sdkconfig` 里搜同名项：**同一个选项，两种形态**。这一章就是讲这两个文件之间的那段流水线。

往下读之前先押个方向：sdkconfig 和 sdkconfig.h，谁是谁的译文、谁又是原文？方向押反了，后面整条流水线都会读倒。

动画：Kconfig 三阶管线

从 Kconfig 定义到 sdkconfig 文本再到 sdkconfig.h C 宏，配置沿管线单向流动。

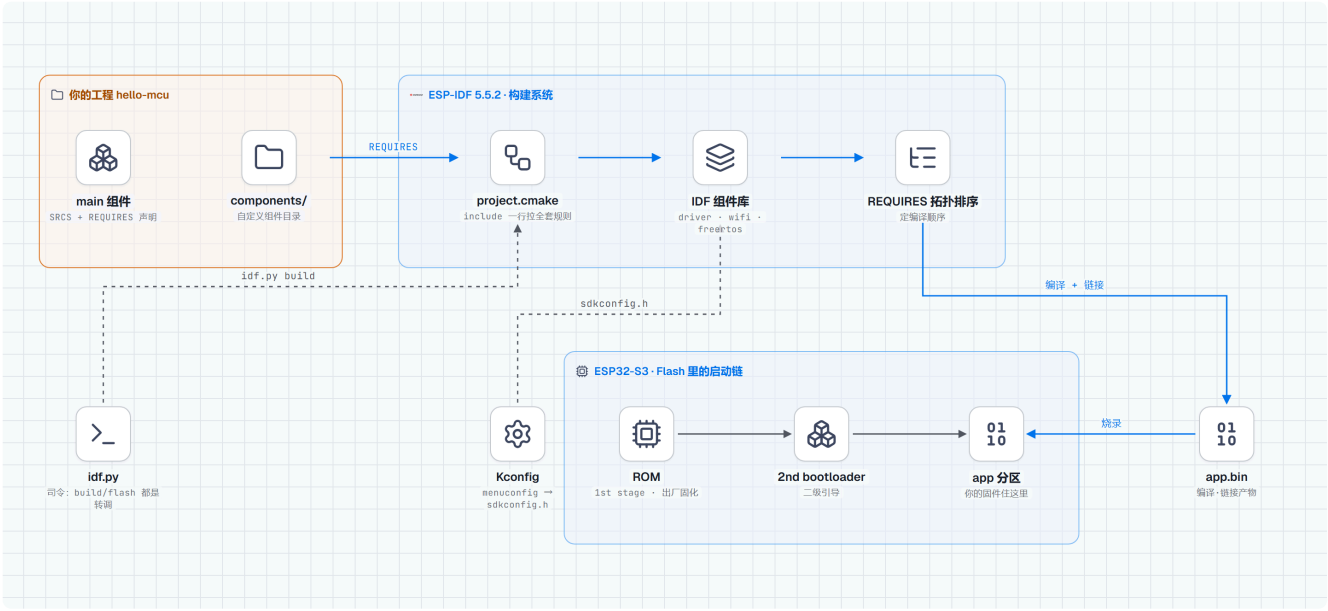
Kconfig 三阶管线动画

小节结构

小节	内容	四件套
三行顶层 CMake	`cmake_minimum_required` / `include` / `project` 各自干什么	配置
组件是什么	`idf_component_register` 逐参数；`REQUIRES` 构成依赖图	库解析
main 特殊在哪	为什么 `main` 不用 `REQUIRES` 也能用别的东西	库解析
Kconfig 三段论	Kconfig → sdkconfig → sdkconfig.h；生成的三个文件	配置
idf.py 的调度	各子命令调谁；和手写 Makefile 的分工对照	库解析
自己写组件	最小可用的组件骨架与常见错误	配置

一、三行顶层 CMake：你的工程为什么这么短

先看解剖图：你的工程、IDF 构建系统、芯片里的启动链，三个圈子各管一段。三行 CMake 就是第一个圈子递给第二个圈子的接力棒。



一个 ESP32 工程的解剖图：你的工程、IDF 构建系统、芯片启动链

看 `code/esp32/00-hello/CMakeLists.txt` 的正文（注释之外）：

```
cmake_minimum_required(VERSION 3.16)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(hello-mcu)
```

三行，一个可编译的 ESP32 工程。**第二行是全部魔法所在**：

```
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
```

这行把 IDF 的整套构建规则拉进你的工程。`project.cmake` 自己再往下拉别的规则（`tools/cmake/project.cmake:21,29`）：

```
include(${CMAKE_CURRENT_LIST_DIR}/targets.cmake)   # 21 行：芯片目标定义
include(${CMAKE_CURRENT_LIST_DIR}/idf.cmake)       # 29 行：IDF 本体规则
```

`project(hello-mcu)` 不是普通的 CMake 工程声明：`project.cmake` 里重定义了它（`tools/cmake/build.cmake:67-73` 附近解析 `project()` 的参数），所以这一行之后，构建系统额外获得了几件事：

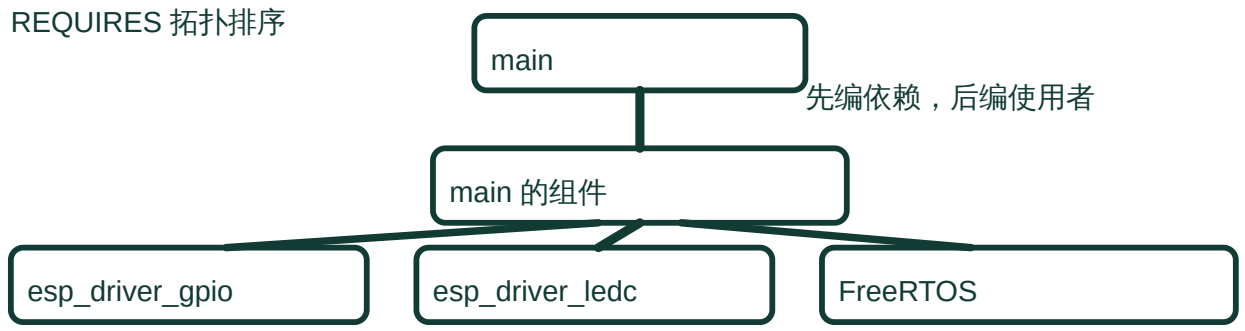
- **组件扫描**：从 `IDF_PATH/components/` 和你的 `components/` 目录收集所有组件；
- **依赖图排序**：按 `REQUIRES` 做拓扑排序，决定编译顺序与头文件可见性；
- **Kconfig 处理**：生成配置头文件（第三节）。

还有一行值得注意：本工程自己加的**目标守卫**：

```
if(NOT IDF_TARGET STREQUAL "esp32s3")
    message(FATAL_ERROR "This example requires ESP32-S3. Run: idf.py set-target esp32s3")
endif()
```

这是防止"配错芯片"的实用技巧：如果工程目录里残留了一个指向经典 ESP32 的 `sdkconfig`，构建会**静默**按错的目标编。加这三行守卫，错配直接失败。`code/esp32/*/CMakeLists.txt` 每个工程都有这段。

二、组件是什么：一行声明交代全部



组件依赖图：REQUIRES 决定拓扑编译顺序

一个组件 = 一个目录 + 一个 `CMakeLists.txt`。本仓库最小的例子，`code/esp32/00-hello/main/CMakeLists.txt` 正文只有一行：

```
idf_component_register(SRCS "hello_main.c"
                    INCLUDE_DIRS "")
```

复杂一点的，`code/esp32/01-gpio-matrix/main/CMakeLists.txt`：

```
idf_component_register(SRCS "main.c" INCLUDE_DIRS "." REQUIRES esp_driver_ledc esp_driver_gpio)
```

这行函数就是"自描述"的全部。它的完整参数表在 `tools/cmake/component.cmake:443-449`：

```
function(idf_component_register)
    set(options WHOLE_ARCHIVE)
    set(single_value KCONFIG KCONFIG_PROJBUILD)
    set(multi_value SRCS SRC_DIRS EXCLUDE_SRCS
                    INCLUDE_DIRS PRIV_INCLUDE_DIRS LDFRAGMENTS REQUIRES
                    PRIV_REQUIRES REQUIRED_IDF_TARGETS EMBED_FILES EMBED_TXTFILES)
```

常用参数的含义：

参数	作用	例子
`SRCS`	直接列出源文件	`SRCS "main.c"`
`SRC_DIRS`	列目录，自动收里面所有源文件	`SRC_DIRS "drivers"`
`INCLUDE_DIRS`	**公开**头文件目录（别的组件能 `#include`）	`INCLUDE_DIRS "."`
`PRIV_INCLUDE_DIRS`	私有头文件目录（只有本组件能用）	内部实现头
`REQUIRES`	**公开**依赖：我也会把这些组件的头文件暴露给依赖我的人	`REQUIRES esp_driver_ledc`
`PRIV_REQUIRES`	私有依赖：我用，但不传递	`PRIV_REQUIRES esp_timer`
`REQUIRED_IDF_TARGETS`	限定只能在这些芯片上编	`REQUIRED_IDF_TARGETS esp32s3`
`EMBED_FILES`	把文件当作二进制数据嵌进固件	网页、字体
`EMBED_TXTFILES`	同上，但附加 `\0` 结尾（当字符串用）	证书、HTML

`REQUIRES` 构成的是一张有向图。IDF 收齐所有组件的声明后做**拓扑排序**，得到编译顺序，所以你在 `main` 里 `#include "driver/ledc.h"` 能不能成功，取决于 `main` 有没有（直接或传递地）`REQUIRES esp_driver_ledc`。

> 对照 STM32 侧：我们手写 Makefile 时，头文件路径靠 `-I` 一个个列、链接靠 `.o` 一个个点。IDF 把这两件事合并成一句"我依赖谁"，剩下的交给构建系统。**代价是依赖声明写错时报错信息比较绕**：这是下面常见坑里的第一条。

三、`main` 特殊在哪：一个特别的组件

IDF 里 `main` 是一个**约定的特殊组件名**。它的特殊之处：

- **不需要被别的组件 `REQUIRES`** 就会自动链接进 app；
- 它可以（通常也应该）`REQUIRES` 别的组件；
- 你的 `app_main()` 就住在这里（P1 讲的启动第三棒会调用它）。

构建系统确实对 `main` 有专门处理，`tools/cmake/build.cmake:465` 附近可以看到 `main` 目录被当作**项目组件**（project components）收集的路径：

```
__project_component_dir("${CMAKE_CURRENT_LIST_DIR}/main" "project_components")
```

"项目组件"与"IDF 内置组件"（`IDF_PATH/components/`）是两个来源，都会被拉进同一个依赖图。

实践含义：`main/` 里塞得太多时，正确做法是把功能拆成 `components/你的组件名/`，然后在 `main/CMakeLists.txt` 里 `REQUIRES` 它。拆分后好处是：

- 编译粒度更细（改一处不用重编全部）；
- 依赖关系显式化（谁用谁写在纸上）；
- 组件可以被多个工程复用。

四、Kconfig 三段论：菜单 → 文本 → 宏

这是 IDF 配置系统的全部秘密，本仓库里就能看到每一段的实物。

第一段：菜单（Kconfig 文件）

每个组件可以放一个 `Kconfig`，用 Kconfig 语言描述选项（类型、默认值、依赖、提示文案）。IDF 本体的 `Kconfig` 到处都是，例如 `components/esp_system/Kconfig:227` 起：

```
    config ESP_MAIN_TASK_STACK_SIZE
        int "Main task stack size"
        default 3584
        help
            Configure the "main task" stack size. ...
```

这就是 P1 里那个"main 任务栈默认 3584 字节"的**原始出处**。

第二段：文本（sdkconfig）

`idf.py menuconfig` 打开菜单，你的选择被写进工程根目录的 `sdkconfig`，一个纯文本文件。本仓库 `code/esp32/01-gpio-matrix/sdkconfig` 的头部写着：

```
#
# Automatically generated file. DO NOT EDIT.
# Espressif IoT Development Framework (ESP-IDF) 5.5.2 Project Configuration
#
CONFIG_SOC_CAPS_ECO_VER_MAX=301
CONFIG_SOC_ADC_SUPPORTED=y
...
```

这个文件里有多少项？实测：

```
grep -cE "^CONFIG_" code/esp32/01-gpio-matrix/sdkconfig
# 890
```

890 个配置项，一个传感器示例工程的配置规模。

> `DO NOT EDIT` **是认真的**：这个文件由工具生成。要改配置就 `idf.py menuconfig`（或直接 `set-target` 等命令），手改会在下次 menuconfig 时被覆盖。想让某项配置进版本控制又能被追踪，IDF 提供了 `sdkconfig.defaults` 机制。

第三段：宏（build/config/sdkconfig.h）

构建时，`sdkconfig` 被翻译成 C 头文件。这一步在 `tools/cmake/kconfig.cmake:208-214`：

```
    set(sdkconfig_header ${config_dir}/sdkconfig.h)
    ...
    set(kconfgen_output_options
    --output header ${sdkconfig_header}
    --output cmake ${sdkconfig_cmake}
    --output json ${sdkconfig_json}
```

注意它一次生成**三个**产物：

产物	给谁用
`build/config/sdkconfig.h`	**C 代码**：`#include "sdkconfig.h"` 后用 `CONFIG_xxx` 宏
`build/config/sdkconfig.cmake`	**CMake**：构建脚本里判断配置
`build/config/sdkconfig.json`	**工具**：脚本/IDE 读取配置

于是代码里就能这么写，本仓库的真实例子，`code/esp32/00-hello/main/hello_main.c:25`：

```
            CONFIG_IDF_TARGET_ESP32S3 ? "S3" : "?",
```

`CONFIG_IDF_TARGET_ESP32S3` 这个宏从哪来？追回去就是：IDF 本体的 `Kconfig` 定义选项 → 你（或 `set-target`）写进 `sdkconfig` → 构建时 `kconfig.cmake` 生成 `sdkconfig.h` → 编译器看到 `-D` 或被包含的头文件。**三段闭环。**

> 对照 R6 RT-Thread 的 menuconfig：同一套 Kconfig 语言的另一套实现，学会了这套，RT-Thread 那边是横向迁移。

五、`idf.py` 是司令不是士兵

`idf.py` 本身**不编译任何东西**，它是个调度层（Python 脚本）。子命令各调各的：

子命令	实际调谁	说明
`idf.py build`	CMake（配置）+ 构建器（ninja 或 make）	生成/更新 `build/`，编出 `.elf/.bin`
`idf.py menuconfig`	kconfig 工具	改 `sdkconfig`
`idf.py flash`	esptool	把 bin 烧进 Flash
`idf.py monitor`	串口终端	看日志
`idf.py size`	`size` 类工具	看体积构成
`idf.py partition-table`	分区表工具	打印实际分区表

为什么这个分工重要：出问题时能定位到**哪一层**。

- 配置不对（缺宏、选错芯片）→ 查 `sdkconfig` / `menuconfig`；
- 构建规则不对（找不到头文件、缺依赖）→ 查 `CMakeLists.txt` 的 `SRCS` / `INCLUDE_DIRS` / `REQUIRES`；
- 编译错误 → 编译器（消息里有文件行号）；
- 烧录失败 → esptool（消息里有串口名/波特率）；
- 运行时崩 → P1 的启动三棒排查法。

> 与 B7 构建系统 的对照：STM32 侧我们手写 Makefile，编译/汇编/链接三件事都看得见（B1 四步构建）；IDF 把这三件事封进 `idf.py build`。 **代价是透明度，收益是规模**：上千个文件、上百个组件、多芯片目标，手写 Makefile 管不过来。这也是为什么"两边都学"比"只学一边"更能看清构建系统到底在做什么。

六、自己写一个组件：最小骨架

想把自己的驱动从 `main/` 里拆出来，目录结构是：

```
components/
└── my_led/
    ├── CMakeLists.txt
    ├── include/
    │   └── my_led.h        ← 对外接口（公开头文件）
    └── my_led.c
```

`components/my_led/CMakeLists.txt`：

```
idf_component_register(SRCS "my_led.c"
                       INCLUDE_DIRS "include"
                       REQUIRES esp_driver_gpio)
```

然后 `main/CMakeLists.txt` 加上依赖：

```
idf_component_register(SRCS "main.c"
                       INCLUDE_DIRS "."
                       REQUIRES my_led)
```

要点：`INCLUDE_DIRS` 写的是**目录**（不是文件），IDF 会把该目录加进 `-I`；头文件放在 `include/` 下是惯例（`my_led.h` 里其他组件写 `#include "my_led.h"` 而非 `#include "include/my_led.h"`）。

附录：工程完整源码

顶层 CMakeLists.txt（三行正文 + 目标守卫，00-hello 版）：

<<< ../../code/esp32/00-hello/CMakeLists.txt

main 组件的 CMakeLists.txt（最小可用形态）：

<<< ../../code/esp32/00-hello/main/CMakeLists.txt

带 REQUIRES 的真实例子（01-gpio-matrix）：

<<< ../../code/esp32/01-gpio-matrix/main/CMakeLists.txt

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **组件自报家门（SRCS/INCLUDE_DIRS/REQUIRES），Kconfig 三段变身（菜单→sdkconfig→sdkconfig.h），idf.py 只当司令不搬砖。**

延伸：Kconfig 三阶管线动画见 P3 动画；RT-Thread 同源 Kconfig 机制见 R6；CMake 构建系统在 B7 详解。

实物实验

- `idf.py build` 后翻 `build/config/`，打开 `sdkconfig.h` 找到 `CONFIG_IDF_TARGET_ESP32S3`，再回工程根 `sdkconfig` 找 `CONFIG_IDF_TARGET=`：**同一个选项的两种形态**，把两处都截图/记录。
- 数一下你的工程的配置规模：`grep -cE "^CONFIG_" sdkconfig`（本仓库 gpio-matrix 是 **890** 项）。
- 把 `code/esp32/01-gpio-matrix/main/CMakeLists.txt` 的 `REQUIRES` 删掉，重新 `idf.py build`，**记录报错信息**，这就是"依赖声明写错时长什么样"，以后再见到能一眼认出来。
- 查看 `idf.py --help` 的子命令列表，对照第五节那张表，确认每个命令各调谁（**待上机回填**）。

常见坑

- **手改 `sdkconfig`**：文件头写着 `DO NOT EDIT`，它由工具生成。下次 `menuconfig` 或 `set-target` 会覆盖你的手改。要持久化配置用 `sdkconfig.defaults`。
- **`INCLUDE_DIRS` 写成头文件路径**：它要的是**目录**。写成 `include/my_led.h` 会让 `-I` 指向文件，编译期找不到头文件。
- **忘了 `REQUIRES`**：代码里能用是因为**头文件恰好被别的组件的公开头传递带进来了**，这种"侥幸编译通过"很脆弱，换个 IDF 版本就可能断。显式声明依赖。
- **`main` 里越堆越多**：`main` 是特殊组件，但不是什么都能往里塞。功能成规模就拆 `components/`。
- **残留错目标的 `sdkconfig`**：从别的芯片工程拷来的 `sdkconfig` 会让构建静默按错目标编，本仓库每个工程都用 `if(NOT IDF_TARGET STREQUAL "esp32s3")` 守卫住了，你自己的工程也建议加。
- **顶层 `CMakeLists.txt` 里加了一堆逻辑**：它只需要三行 + 可选守卫。真正的构建逻辑属于组件。

短自测

1. 你那个只有三行的顶层 `CMakeLists.txt` 为什么够用？

▼ 看答案
因为第二行 `include($ENV{IDF_PATH}/tools/cmake/project.cmake)` 把 IDF 的整套构建规则拉了进来（它内部再 include `targets.cmake` 与 `idf.cmake`，见 `tools/cmake/project.cmake:21,29`）。之后 `project(名字)` 被 IDF 重定义，触发组件扫描、依赖图排序、Kconfig 处理。你的工程只需要声明"我是谁"。

2. `INCLUDE_DIRS` 和 `PRIV_INCLUDE_DIRS` 有什么区别？

▼ 看答案
`INCLUDE_DIRS` 是**公开**的，依赖你这个组件的其他组件也能包含这些头文件；`PRIV_INCLUDE_DIRS` 是**私有**的，只有本组件内部能用。这是"接口 vs 实现"在构建系统里的落地：把内部头文件放进私有目录，别人就编译不到你的实现细节。同理 `REQUIRES` / `PRIV_REQUIRES` 是依赖的公开与私有版本。

3. Kconfig 三段论是哪三段？各产出一个什么文件？

▼ 看答案
菜单（组件里的 `Kconfig` 文件，如 `components/esp_system/Kconfig:227`）→ **文本**（`menuconfig` 写出的 `sdkconfig`，纯文本，本仓库 gpio-matrix 有 890 项）→ **宏**（构建时生成的 `build/config/sdkconfig.h`，代码里用 `CONFIG_xxx`）。生成逻辑在 `tools/cmake/kconfig.cmake:208-214`，同时还会产出 `sdkconfig.cmake`（给 CMake）和 `sdkconfig.json`（给工具）。

4. `main` 组件特殊在哪？什么时候该把东西拆出去？

▼ 看答案
`main` 是约定名：不需要被别的组件 `REQUIRES` 就会自动链接进 app，`app_main()` 住在里面。构建系统对它专门处理（`tools/cmake/build.cmake:465` 把 `main` 目录当"项目组件"收集）。当 `main` 里的功能成规模、或你想复用/细化编译粒度时，就该拆成 `components/你的组件名/`，再在 `main` 里 `REQUIRES` 它。

5. 构建报"找不到头文件"，你应该先查哪几个地方？

▼ 看答案
按顺序：**① 你的 `CMakeLists.txt` 里 `INCLUDE_DIRS` 是否包含了该头文件所在的**目录**（不是文件）；**② 提供该头文件的组件是否在 `REQUIRES` 里（直接或传递）；**③ 该组件是否被构建系统收集到（目录位置对不对：`components/` 下还是 IDF 内置）；**④ 若报错来自配置分支，查 `sdkconfig` 里相关 `CONFIG_*` 是否如预期。这四条覆盖了绝大多数"找不到头文件"。

对照表：本章概念 → 仓库落点

概念	仓库落点
三行顶层 `CMakeLists.txt` + 目标守卫	`code/esp32/00-hello/CMakeLists.txt`
`project.cmake` 内部 include 链	`$IDF_PATH/tools/cmake/project.cmake:21,29`
`idf_component_register` 完整参数表	`$IDF_PATH/tools/cmake/component.cmake:443-449`
最小组件声明（一行）	`code/esp32/00-hello/main/CMakeLists.txt`
带 `REQUIRES` 的真实组件声明	`code/esp32/01-gpio-matrix/main/CMakeLists.txt`
`main` 作为"项目组件"被收集	`$IDF_PATH/tools/cmake/build.cmake:465`
Kconfig 选项的真实例子（main 任务栈）	`$IDF_PATH/components/esp_system/Kconfig:227-234`
`sdkconfig` 实物（890 项，`DO NOT EDIT`）	`code/esp32/01-gpio-matrix/sdkconfig`
生成 `sdkconfig.h` / `.cmake` / `.json` 三产物	`$IDF_PATH/tools/cmake/kconfig.cmake:208-214`
代码里用 `CONFIG_*` 的真实例子	`code/esp32/00-hello/main/hello_main.c:25`
构建产物落点	`code/esp32/00-hello/build/config/`（构建后生成）
与手写 Makefile 的对照	B7 构建系统、B1 四步构建
Kconfig 另一套实现（RT-Thread）	`docs/rtos/rtthread/06-env-menuconfig.md`

> 表中 `$IDF_PATH` = `C:/Users/zhugu/.espressif/v5.5.2/esp-idf`（本机 ESP-IDF v5.5.2）。行号以该版本为准，换版本需重核。

你做到了

- 那个"三行就够"的顶层 CMake 不再神秘，你知道它拉进了什么；
- 能给自己的驱动写一个合规的组件（`SRCS` / `INCLUDE_DIRS` / `REQUIRES` 三板斧）；
- 走通了 Kconfig 三段论，能追着一个 `CONFIG_xxx` 从菜单追到宏；
- 构建报错时会分层定位（配置 / 组件声明 / 编译 / 烧录），而不是瞎改。

方向押完翻底牌：原文其实是 Kconfig，sdkconfig 与 sdkconfig.h 都是它的译文，一份给人看，一份给编译器看。追着任何一个配置项从菜单走到宏，这条链你现在是顺着的。

✅ 下一站：P4 中断与双核：IRAM 纪律与核间分工，把"Cache 关了你的 ISR 还能不能跑"这件事彻底说透。

通往单片机之路第 59 章

P4 中断与双核：IRAM 纪律与核间分工

难度 ★★★☆☆ · 约 40 分钟

本章罗盘

问题

Cache 关闭窗口里中断来了，ISR 放在 Flash 里为什么会崩？标了 IRAM_ATTR 还不够吗？

交付

probe.sh 四条调用链的 nm 落址表：isr_leaky 本体在 0x4037xxxx 而 callx8 目标落在 0x42xxxxxx 的『断链』证据，加 99 中断源（0–98）× 7 级 × 2 核的 esp_intr_alloc 参数选型表。

边界

落址取证是链接期静态证据（本机实跑、无需板子）；『Cache 关闭期间真的崩/被推迟』、钉核落点、LEVEL3 对比实验均标注『待上机回填』，动态行为尚无板上记录；地址区间是 esp32s3 口径，初代 ESP32 的 0x400xxxxx 不能互抄；SMP 调度细节在 RTOS 篇。

依据锚点 `a7 · a9 · f1 · f2 · mcu-code · mcu-anim`

> 🎯 ESP32 最著名的事故现场：代码在 Flash 里，系统在写 Flash（NVS/OTA 常干）——Cache 一关，取指无门，中断来了直接崩。所以有了铁律：**声明 `ESP_INTR_FLAG_IRAM` 的中断必须保证可达代码/数据均在内部存储；普通非 IRAM-safe 中断在 Flash 操作期间会被推迟**。

本章精髓

1. **`IRAM_ATTR` 是生存问题，不是性能问题**。Cache 关闭的那段窗口里，只有 IRAM/ROM 里的代码能取指。所以 ISR 本体、它调用的函数、它用到的 rodata，三样都要排查。`esp_intr_alloc` 的 `ESP_INTR_FLAG_IRAM` 标志会强制这个约束。
2. **中断分配器是"总机"**。S3 有 **99 个中断源编号**（实测 `ETS_MAX_INTR_SOURCE = 99`，编号 0–98），经中断矩阵分配到两个核的 **7 级**中断。`esp_intr_alloc` 一次把外设源、优先级、所属核三样绑在一起。
3. 双核 SMP 的共享规矩：FreeRTOS 在 S3 上是 SMP 单实例（任务可跑任意核，也可 `xTaskCreatePinnedToCore` 钉核），核间共享数据用自旋锁/ `portMUX`，**中断与任务、核与核之间 `volatile` 完全不够**（回 C3 的边界清单）。

- **能理解**：`IRAM_ATTR` 只是往链接器段里一塞（`".iram1"`），真正保证"能在关 Cache 时跑"的是**整条调用链**都在 IRAM，包括它读的常量。
- **能用**：给外设挂中断时，能正确选 `ESP_INTR_FLAG_LEVELx` 与 `ESP_INTR_FLAG_IRAM`；并知道什么时候必须用 `portMUX`，而不是 `volatile`。

先修

- P1 S3 架构与启动：双核、IRAM/DRAM、Cache 映射执行，本章的铁律全是它的推论。
- C3 volatile：`volatile` 能做什么、不能做什么，本章直接引用其边界清单。
- S4 中断与 NVIC：优先级、向量表、ISR 礼仪，单核侧的对照系。

先跑起来（10 分钟 quick win）

挂中断的写法本身（`code/esp32/01-gpio-matrix`）：

```
/* 写法 A：普通 ISR —— 代码在 Flash 里 */
static void IRAM_ATTR my_isr(void *arg)   /* 先看 B，再把这个 IRAM_ATTR 删掉体会差别 */
{
    BaseType_t hp = pdFALSE;
    xQueueSendFromISR(evt_queue, &tick, &hp);
    portYIELD_FROM_ISR(hp);
}

/* 挂中断：指定优先级 + 是否 IRAM-safe */
esp_intr_alloc(ETS_GPIO_INTR_SOURCE, ESP_INTR_FLAG_LEVEL1 | ESP_INTR_FLAG_IRAM,
               my_isr, NULL, NULL);
```

probe 之前先认个内鬼：四条链里，哪条看着最安全、关 Cache 的窗口一开却第一个跪？先押一个再跑构建，nm 的归类表就是判决书。

落址不用背，本机链接完直接查。 `code/esp32/09-iram-discipline` 把实践清单的四条链一次摆全：不标的、只标本体的、整条链都标的、代码进了 IRAM 但常量还在 Flash 的。`sh probe.sh` 一条命令跑完构建与取证，最后那段断言逐条打勾、错一条直接非零退出才算数。下面是本机实测（ESP-IDF v5.5.2，目标 esp32s3）：

```
$ sh probe.sh                # 第 3 段：链接器真正用的区间，出自 .map 的 Memory Configuration
  iram0_0_seg 0x40374000 0x00057700    # 内部 SRAM，可执行，关 Cache 也取得到
  iram0_2_seg 0x42000020 0x007fffe0    # Flash 映射的取指窗口
  drom0_0_seg 0x3c000020 0x01ffffe0    # Flash 映射的常量窗口

$ # 第 4 段：nm 的地址 + 当场按上面三行归类
  0x42008f08  isr_plain    → iram0_2_seg     # 没标 IRAM_ATTR，留在 Flash
  0x42008f1c  helper_far   → iram0_2_seg     # 被 IRAM 里的 ISR 调用，却没标
  0x4037668c  isr_leaky    → iram0_0_seg     # 标了，本体确实在 IRAM
  0x40376698  helper_near  → iram0_0_seg
  0x403766ac  isr_tight    → iram0_0_seg     # 整条链都标
  0x40376668  isr_rodata   → iram0_0_seg
  0x3c028810  k_table      → drom0_0_seg     # 常量还在 Flash

$ # 第 6 段：断链的现场在调用指令的目标地址里，不在段名里
  4037668c <isr_leaky>:
    4037668f:  l32r  a8, 403746a0 <_iram_text_start+0x29c> (42008f1c <helper_far>)
    40376692:  callx8 a8
    40376695:  retw.n
  403766ac <isr_tight>:
    403766af:  call8 40376698 <helper_near>
    403766b2:  retw.n
```

判据：`isr_leaky` 的每一条指令都在 IRAM，可它 `callx8` 过去的那个地址落在 Flash 映射段。Cache 一关，`entry` 之后第一条取指就无门。`isr_tight` 的 `call8` 目标还在 IRAM 区间内，这才是真的 IRAM-safe。地址区间一律以 `.map` 里的 `Memory Configuration` 为准。esp32s3 的 IRAM 取指窗口起点是 `0x40374000`，Flash 映射是 `0x42xxxxxx`，常量映射是 `0x3Cxxxxxx`。初代 ESP32 则是 `0x400xxxxx` 一片——两代的数字不能互抄。

还有一个只有实跑才会撞见的坑。这个工程里没有真的外设去 `esp_intr_alloc` 这些 ISR，而 IDF 默认带 `-Wl,--gc-sections`，**没人引用的函数会被整段收掉**。第一版探针工程 `nm` 里只剩 `app_main` 和 `g_hits`，七个符号一个都不剩。现在 `main.c` 里那张 `g_isr_table` 就是为了钉住它们，顺便把这条记下来：你写在磁盘上的 ISR，链接器可能根本没让它进 ELF。

小节结构

小节	内容	四件套
Cache 与取指	Flash 映射执行；为什么关 Cache 会"代码消失"	配置
IRAM 纪律	`IRAM_ATTR` 的定义；整条调用链与 rodata 的连带责任	库解析
中断分配器	99 个源 × 7 级 × 2 核；`esp_intr_alloc` 五参数	库解析
双核分工	SMP vs 钉核；`xTaskCreatePinnedToCore` 的核参数	代码分析
共享数据规矩	`portMUX` 自旋锁、队列、通知；`volatile` 的边界	配置
与 NVIC 对照	单核向量表 vs 中断矩阵 + 分核	配置

一、Cache 与取指：为什么"代码会消失"

先看这张图：取指的两条路，和 Cache 关闭窗口里一个中断的两种命运。

IRAM 纪律与 Cache 关闭窗口动画

P1 讲过内存分级，这里把它推到结论。ESP32-S3 上你的应用代码**绝大多数放在外部 Flash 里**，通过 **Cache 映射窗口**执行。这意味着"执行一条 Flash 里的指令"实际上是：

```
CPU 取指 → Cache 命中？ → 命中：直接返回指令
                        → 未命中：去 Flash 读（慢，但能读）
```

关键在"Cache 关闭"这个状态。系统在做这些事时会**暂时禁用 Cache**：

- 写 Flash（NVS 提交、OTA、`esp_partition_write`）；
- 某些低层 Flash 操作与加密操作。

Cache 一关，映射窗口失效：此时 CPU 若要从 Flash 取指，**没有任何路径能拿到那条指令**。硬件层面的结果就是异常/崩溃。

所以问题不是"会不会遇到"，而是"你的哪个 ISR 会在那个窗口里被触发"。 中断是**异步**的：NVS 正在写 Flash，恰好你的 GPIO 中断来了。如果你在 `esp_intr_alloc` 时**没有**声明 IRAM-safe，系统会让这个中断**推迟**到 Cache 恢复；如果声明了却**没做到**整条链在 IRAM，就是直接崩。

二、IRAM 纪律：`IRAM_ATTR` 只是入场券

定义（`components/esp_common/include/esp_attr.h:23`）：

```
#define IRAM_ATTR _SECTION_ATTR_IMPL(".iram1", __COUNTER__)
```

就这么简单：`IRAM_ATTR` 干的事，是**把这段代码放进名为 `.iram1` 的链接段**，再由链接脚本安排到内部 SRAM（可执行区）。它不是"加速"，是"改变落址"。

但把它加在 ISR 函数上只是第一步。真正的约束是**整条执行链**都要可达：

要排查的东西	为什么
ISR 本体	显然——它自己要取指
ISR 调用的**所有**函数	调用即取指；内联的不算问题，没内联的就是问题
ISR 用到的 **rodata**（字符串常量、查表、`const` 数组）	数据默认在 Flash 映射区，关 Cache 时读不到

实践清单（写 IRAM-safe ISR 时逐条过）：

1. ISR 本体标 `IRAM_ATTR`；
2. 它调用的每个非内联函数也标 `IRAM_ATTR`（编译器若内联则免除，但**不要依赖内联**）；
3. 所有 `const` 查表/字符串也标 `IRAM_ATTR`（或放进 DRAM）；
4. **不要**在里面调 `printf`、`malloc`、任何可能碰 Flash 的 API；
5. 需要跨 ISR/任务传数据 → 用 `...FromISR` API（它们本身就是 IRAM-safe 的）。

`ESP_INTR_FLAG_IRAM` 是声明侧的开关（`components/esp_hw_support/include/esp_intr_alloc.h:42`）。

```
#define ESP_INTR_FLAG_IRAM          (1<<10) ///< ISR can be called if cache is disabled
```

它的语义是**承诺**：我保证这个 ISR 在 Cache 关闭时也能跑。系统的回应是：**那就不推迟它**，Cache 关着也照样调用。所以：

- **不标** `ESP_INTR_FLAG_IRAM`：系统会在 Flash 操作期间**推迟**该中断（安全，但有延迟）；
- **标了** `ESP_INTR_FLAG_IRAM`：系统不推迟（低延迟），但**你承诺的必须是真的**：链条上任何一环在 Flash 里，就是崩。

> 这是本章最值得记住的**权责对等**：低延迟不是免费的，你用自己的正确性给它做担保。

三、中断分配器："总机"上的 99 条线

中断分配器：99 条线分到 CPU0/CPU1

STM32 上中断和向量是**一一对应**的（S4：`EXTI0_IRQn = 6` → 固定向量槽 22），定向量槽）。ESP32-S3 是**中断矩阵**：外设的中断源可以**路由**到某个核的某个中断级。

规模（实测 `components/soc/esp32s3/include/soc/interrupts.h`，`sh probe.sh` 第 1 段让编译器把整张枚举打出来）：

- 枚举体在第 18–117 行，从 `ETS_WIFI_MAC_INTR_SOURCE = 0` 排到 `ETS_DMA_EXTMEM_REJECT_SOURCE = 98`；随后第 118 行是 `ETS_MAX_INTR_SOURCE`，编译器打印它的值是 **99**，注释写着 "number of interrupt sources"。
- **99 是编号槽位的数量（0–98），不是"有 99 个不同的名字"**。这张表里 98 个名字只覆盖 95 个不同编号：23、33、34、46 是保留位没有名字，57/58/59 各被两个名字占用（`ETS_SYSTIMER_TARGETn_EDGE_INTR_SOURCE = ETS_SYSTIMER_TARGETn_INTR_SOURCE`）。MER_TARGETn_INTR_SOURCE`，三条 EDGE 是同值别名）。
- 所以按源编号路由时，`ETS_MAX_INTR_SOURCE` 才是数组边界的正确写法（`esp_isr_names[ETS_MAX_INTR_SOURCE]` 就是这么声明的）。别拿"行数"当数量。

中断级（`components/esp_hw_support/include/esp_intr_alloc.h:33-43`）：

标志	值	含义
`ESP_INTR_FLAG_LEVEL1`	`1<<1`	最低优先级的中断向量
`ESP_INTR_FLAG_LEVEL2` … `LEVEL6`	`1<<2` … `1<<6`	逐级升高
`ESP_INTR_FLAG_NMI`	`1<<7`	最高优先级（不可屏蔽）
`ESP_INTR_FLAG_SHARED`	`1<<8`	允许 ISR 共享
`ESP_INTR_FLAG_EDGE`	`1<<9`	边沿触发
`ESP_INTR_FLAG_IRAM`	`1<<10`	关 Cache 时也能调用（第二节）
`ESP_INTR_FLAG_INTRDISABLED`	`1<<11`	分配后先禁用

源文件第 32 行还有一条**给实现者的提醒**：`Keep the LEVELx values as they are here; they match up with (1<<level)`，意思是**位号就是级别号**，这不是巧合，是接口约定（`probe.sh` 第 8 段逐条把 `LEVELn` 的位移量对回 n）。

分配函数（同文件 `:149`）：

```
esp_err_t esp_intr_alloc(int source, int flags, intr_handler_t handler, void *arg, intr_handle_t *ret_handle);
```

五个参数各自的角色：

参数	填什么	从哪来
`source`	中断源（如 `ETS_GPIO_INTR_SOURCE`）	`soc/esp32s3/include/soc/interrupts.h` 的枚举
`flags`	级别 + 特性（OR 起来）	上面那张表
`handler`	你的 ISR 函数	你写的
`arg`	传给 ISR 的参数	你定的（常用于传句柄）
`ret_handle`	返回句柄，用于后续 `esp_intr_free`	可以传 `NULL`

绑到哪个核由 `flags` 里的级别与源决定（见 `esp_intr_alloc` 文档头里对 level 的说明）；简单记法是：**级别越高越"紧急"，也越不能用普通 API**。

四、双核分工：SMP 与钉核

S3 是双核（`SOC_CPU_CORES_NUM = 2`，P1 已引）。FreeRTOS 在 S3 上跑 **SMP**：默认情况下任务可以调度到任意核。

需要控制核归属时，用钉核版 API（`components/freertos/FreeRTOS-Kernel/include/freertos/task.h:382-389`）。

```
extern BaseType_t xTaskCreatePinnedToCore( TaskFunction_t pxTaskCode,
                                           const char * const pcName,
                                           const configSTACK_DEPTH_TYPE usStackDepth,
                                           void * const pvParameters,
                                           UBaseType_t uxPriority,
                                           TaskHandle_t * const pvCreatedTask,
                                           const BaseType_t xCoreID );
```

比 `xTaskCreate` 多最后一个参数 `xCoreID`：

- `0` / `1`：钉到 PRO_CPU / APP_CPU；
- `tskNO_AFFINITY`：不钉核（调度器自由安排）。实测 `task.h:206` 把它定义为 `( BaseType_t ) 0x7FFFFFFF`，紧跟的第 207 行还有一条 `Todo: Update tskNO_AFFINITY value to -1` 的注释。所以它**不是 -1**，别按 -1 去猜值。

函数注释紧接着写明了这一点（`:391-392`）：**"Call the 'PinnedToCore' version with `tskNO_AFFINITY` to create an unpinned task."**。而且 `xTaskCreate` 的函数体（`:394` 起）只有一句 `return xTaskCreatePinnedToCore( pxTaskCode, ... )`。包装关系明明白白写在源码里，不是推测。

什么时候该钉核：

- 任务与**核绑定的外设**强相关（某些外设的中断固定在某核）；
- 需要**核间负载**明确分工（一核专管通信、一核专管控制）；
- 排查"任务跑到哪个核"引起的问题时先钉住复现。

反过来说，**能不钉就不钉**：钉核会降低调度器的灵活性，也让负载均衡失效。

五、共享数据：`volatile` 不够，要用自旋锁

这是 C3 volatile 边界清单 的直接延续。在双核 + 中断的 ESP32 上，`volatile` **保证不了**任何一件事。把需求拆开看：

需求	`volatile` 能做吗	正确做法
阻止编译器优化掉访问	✅ 能	`volatile` 本行
读-改-写不被打断	❌ 不能	`portMUX` 自旋锁 / 临界区
核与核之间的可见性	❌ 不能	自旋锁 / 原子操作 / 队列
ISR 与任务之间传数据	❌ 不能	`...FromISR` API（队列/通知）

`portMUX_TYPE` 的实现（`components/freertos/FreeRTOS-Kernel/portable/xtensa/include/freertos/portmacro.h:188`）：

```
typedef spinlock_t                          portMUX_TYPE;               /**< Spinlock type used by FreeRTOS critical sect
#define portMUX_INITIALIZER_UNLOCKED        SPINLOCK_INITIALIZER        /**< Spinlock initializer */
```

而 `spinlock_t` 本体定义在 `components/esp_hw_support/include/spinlock.h:45` 。

三种场景的正确选择：

```
/* 场景 1：任务里保护共享变量的读-改-写（可能被其它核或 ISR 打断） */
static portMUX_TYPE my_mux = portMUX_INITIALIZER_UNLOCKED;

portENTER_CRITICAL(&my_mux);
shared_counter++;
portEXIT_CRITICAL(&my_mux);

/* 场景 2：ISR → 任务 传数据（唯一推荐的做法） */
xQueueSendFromISR(q, &data, &hp);      /* ISR 里 */
xQueueReceive(q, &data, portMAX_DELAY); /* 任务里 */

/* 场景 3：任务 → 任务 或 核 → 核 传数据 */
xQueueSend(...) / xTaskNotifyGive(...)  /* 用内核对象，别自己造轮子 */
```

为什么临界区只包"读-改-写"而不是整个函数：`portENTER_CRITICAL` 在有自旋锁的平台上会**关中断**（或自旋），持锁时间越长，实时性越差。**临界区要尽可能短**：这是 F0 那个"任务越少越优雅"的同一条原则在锁上的体现。

六、与 NVIC 对照：单核与中断矩阵

	STM32 F407（S4）	**ESP32-S3**
中断控制	NVIC（每核一个，F407 单核）	**中断矩阵**（源 → 核 → 级）
中断源数	按外设固定（IRQn 枚举）	**99 个源编号**（实测 0–98），可路由
优先级	4 位，`IP 字节 = 逻辑优先级 << 4`	**7 级**（`ESP_INTR_FLAG_LEVEL1..6` + NMI）
向量表	固定地址数组，`IRQn + 16`	由分配器动态绑定
ISR 落址	Flash 原地执行，无所谓	**Flash 经 Cache 映射**——关 Cache 时不可取指 → IRAM 纪律
分配方式	`NVIC_EnableIRQ(IRQn)`	`esp_intr_alloc(source, flags, ...)`
核间	不存在	自旋锁 / `portMUX` / 队列

差异概括：STM32 的中断是**静态的**（谁挂哪个向量、什么优先级，编译期定死）；ESP32 的中断是**动态分配的**（运行时把源路由到核与级）。动态化换来灵活性，代价就是你多了一份责任：**分配时声明的一切，运行时要真做到**：IRAM 承诺如此，优先级选择亦然。

附录：工程完整源码

01-gpio-matrix 的 main（本章实验的起点：一个真实外设工程，`app_main` → 外设初始化；「先跑起来」的 ISR 就挂在它的 GPIO 上）：

<<< ../../code/esp32/01-gpio-matrix/main/main.c

它的组件声明（`REQUIRES esp_driver_ledc esp_driver_gpio`，P3 讲过的依赖声明；中断 API 就来自其中的驱动组件）：

<<< ../../code/esp32/01-gpio-matrix/main/CMakeLists.txt

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **Cache 一关 Flash 里没代码：ISR 要么进 IRAM（还要带上它调的函数和常量），要么老实被推迟。双核共享别信 volatile，上 portMUX。**

延伸：IRAM 纪律动画见 P4 动画；volatile 的局限性在 C3 详解；STM32 NVIC 对照见 S4。

实物实验

- ~~写一个带 `IRAM_ATTR` 的 ISR 和一个不带的，`idf.py build` 后用 `arm-none-eabi-nm` 对比符号地址~~。这条已经在「先跑起来」里用**本机实跑**填掉了。原写法有两处错：ESP-IDF 用的不是 `arm-none-eabi-*`，而是 `xtensa-esp-elf-nm`；判据地址也不是 `0x400xxxxx`（那是初代 ESP32）。上板只剩一件静态取证做不到的事：**真的把 Cache 关一次**，在 NVS 提交期间触发这个中断，看它崩不崩。
- 给一个 GPIO 中断同时挂 `ESP_INTR_FLAG_LEVEL1` 与 `ESP_INTR_FLAG_IRAM`，在 ISR 里**故意**调 `printf`，观察它在 Flash 操作期间是否崩溃（**这是反例实验，小心使用**；**待上机回填**）。
- 用 `xTaskCreatePinnedToCore(..., xCoreID)` 分别传 `0`、`1`、`tskNO_AFFINITY` 创建同一个任务，在任务里打印 `xPortGetCoreID()`，记录三种情况下任务实际落在哪个核（**待上机回填**）。
- 把 `ESP_INTR_FLAG_LEVEL1` 改成 `LEVEL3`，观察中断响应行为（**待上机回填**）。

常见坑

- **只给 ISR 本体标 `IRAM_ATTR`**：它调用的函数、它读的字符串还在 Flash 里，关 Cache 时照样崩。**整条链**都要排查。
- **声明了 `ESP_INTR_FLAG_IRAM` 却没做到**：这比不声明更糟，不声明只是被推迟（安全），声明了却是**直接崩**。
- **ISR 里调 `printf` / `malloc`**：既可能碰 Flash，又可能违反 ISR 礼仪（S4 说过"中断里只做快递分拣"）。用 `...FromISR` API 或只置标志。
- **用 `volatile` 代替锁**：在双核 + 中断面前，`volatile` 既保证不了原子性也保证不了可见性。核间共享必须上 `portMUX` 或内核对象。
- **无脑钉核**：`xTaskCreatePinnedToCore` 用得越多，调度器越没得选，负载均衡失效。只在**有理由**时钉。
- **中断级越高越好**：级别高意味着它可能**抢占内核自己的临界区**，而高优先级 ISR **不能**调用大部分内核 API。默认从 `LEVEL1` 起步，确有需要再升。

短自测

1. 为什么"代码在 Flash 里"这条在 STM32 上无所谓，在 ESP32 上是生死问题？

> **▼ 看答案**
> STM32 的 Flash 是**原地执行**的（地址空间直接映射，CPU 直接从 Flash 取指，没有中间层）。ESP32-S3 的代码在外部 Flash，通过 **Cache 映射窗口**执行：系统写 Flash 时会**暂时禁用 Cache**，此时映射窗口失效，CPU 从 Flash **取不到指令**。所以 ISR 若在 Flash 里，关 Cache 期间被触发就是崩溃（除非没声明 IRAM-safe，被推迟）。

2. `IRAM_ATTR` 到底做了什么？加在 ISR 上够吗？

> **▼ 看答案**
> 它把代码放进 `.iram1` 段（`components/esp_common/include/esp_attr.h:23`：`#define IRAM_ATTR _SECTION_ATTR_IMPL(".iram1", __COUNTER__)`），_)`），由链接脚本安排到内部 SRAM。**加在 ISR 本体上不够**：ISR 调用的所有非内联函数、以及它读的所有 rodata（字符串、`const` 查表）也必须在 IRAM，否则整条链上有一环在 Flash 就崩。

3. 不声明 `ESP_INTR_FLAG_IRAM` 会怎样？

> **▼ 看答案**
> 系统会在 Flash 操作（Cache 禁用）期间**推迟**这个中断，ISR 不会被调用，等到 Cache 恢复再执行。这是**安全但可能有延迟**的行为。相反，声明了 `ESP_INTR_FLAG_IRAM` 就是承诺"我能在关 Cache 时跑"，系统不再推迟，代价是你必须真的保证整条链可达。

4. S3 有多少个中断源？几个中断级？

> **▼ 看答案**
> **99 个中断源编号（0–98）**，实测 `components/soc/esp32s3/include/soc/interrupts.h`：枚举体第 18–117 行，最后一项 `ETS_DMA_EXTMEM_REJECT_SOURCE` 编译器打印出来是 98，紧跟的第 118 行 `ETS_MAX_INTR_SOURCE` = **99**。注意 99 说的是编号槽位：那 98 个名字只覆盖 95 个不同编号（23/33/34/46 是保留位，三条 `..._EDGE_INTR_SOURCE` 与各自 TARGET 同值）。 **7 个级别**：`ESP_INTR_FLAG_LEVEL1..LEVEL6`（`1<<1`..`1<<6`）加 `ESP_INTR_FLAG_NMI`（`1<<7`，源码注释直说 "Level 7 interrupt vector"），见 `components/esp_hw_support/include/esp_intr_alloc.h:33-39`。位号就是级别号，`probe.sh` 第 8 段逐条对过账。

5. 双核之间共享一个计数器，为什么 `volatile int n; n++;` 是错的？

> **▼ 看答案**
> 两个原因：**① 非原子**，`n++` 是"读-改-写"三步，另一个核或 ISR 可能插在中间，导致丢失更新；**② 无可见性/顺序保证**，`volatile` 只阻止编译器优化掉这次访问，不提供跨核的同步语义。正确做法是用 `portENTER_CRITICAL(&mux)` 包住读-改-写（`portMUX_TYPE` 是自旋锁，`portmacro.h:188`）；或者干脆用内核对象（队列、通知）代替裸变量。

对照表：本章概念 → 仓库落点

概念	仓库落点
`IRAM_ATTR` 定义（放进 `.iram1` 段）	`$IDF_PATH/components/esp_common/include/esp_attr.h:23`
`ESP_INTR_FLAG_IRAM`（`1<<10`，关 Cache 可调用）	`$IDF_PATH/components/esp_hw_support/include/esp_intr_alloc.h:42`
中断级别标志 `LEVEL1..6` / `NMI`	同上 `:33-39`
其余特性标志（SHARED/EDGE/INTRDISABLED）	同上 `:40-43`
中断源数量（99）与枚举	`$IDF_PATH/components/soc/esp32s3/include/soc/interrupts.h:18-118`
`esp_intr_alloc` 五参数签名	`$IDF_PATH/components/esp_hw_support/include/esp_intr_alloc.h:149`
`xTaskCreatePinnedToCore` 七个参数	`$IDF_PATH/components/freertos/FreeRTOS-Kernel/include/freertos/task.h:382-389`
不钉核 = 传 `tskNO_AFFINITY`	同上 `:394` 附近注释
`portMUX_TYPE` 是自旋锁	`$IDF_PATH/components/freertos/FreeRTOS-Kernel/portable/xtensa/include/freertos/portmacro.h:188`
`spinlock_t` 定义	`$IDF_PATH/components/esp_hw_support/include/spinlock.h:45`
双核数量（`SOC_CPU_CORES_NUM = 2`）	`$IDF_PATH/components/soc/esp32s3/include/soc/soc_caps.h:145`
ISR 写法参考（真工程）	`code/esp32/01-gpio-matrix/main/`
四条链的落址取证探针（本机实跑，不需要板）	`code/esp32/09-iram-discipline/probe.sh`
`volatile` 的边界清单	C3 volatile
单核 NVIC 的对照系	S4 中断与 NVIC

> 表中 `$IDF_PATH` = `C:/Users/zhugu/.espressif/v5.5.2/esp-idf`（本机 ESP-IDF v5.5.2）。行号以该版本为准，换版本需重核。

你做到了

- 知道了"IRAM 纪律"不是玄学：它是 Cache 映射执行的必然推论；
- 能正确给外设挂中断：选对源、级别、IRAM 标志，并知道每个选择的责任；
- 双核共享数据时不再指望 `volatile`，会用 `portMUX` 与 `--FromISR` API；
- 能对着 STM32 的 NVIC 说清"静态向量 vs 动态中断矩阵"的差异。

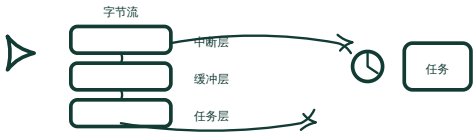
内鬼就是那条半截链：本体搬进了 IRAM，帮手还留在 Flash 里。关 Cache 的窗口一开，取指断在半路。整条链同进同出才是 IRAM 纪律的全文，也是你往后审每一份中断代码的尺子。

✅ 下一站：RTOS 篇把双核 SMP 的调度细节补齐，或回 P0 复查环境配置。

通往单片机之路第 60 章

P5 UART 驱动解析：成熟驱动长什么样

难度 ★★★☆☆ · 约 25 分钟



本章罗盘

问题

IDF 的 UART 驱动比手写中断+环形缓冲多买了什么？事件和字节各走哪条路？

交付

一张三层缓冲数据路径图（硬件 FIFO 128B → 驱动 ring buffer → 用户 buf，标出 FIFO_OVF/BUFFER_FULL 两个溢出点）和一段用 timeout_flag 判帧尾的不定长接收代码（02-uart-events 可烧可跑）。

边界

只解剖 UART 驱动的事件模型与用法，不逐行读 uart.c 两千行实现；RS485/IRDA 等特殊模式与波特率误差计算不展开（回 S7）；115200 连发 10KB、调小 ring buffer 复现溢出属实验清单项，本章未附实测记录。

依据锚点 `a7 · a9 · f2 · mcu-code · mcu-anim`

> 🎯 S7 我们手写了一个串口驱动：中断+环形缓冲，能用但单薄。IDF 的 UART 驱动是"产品级"的答案：中断 → **事件队列** → 用户任务、硬件 FIFO + 环形缓冲协作、超时与帧间隔处理——把"成熟驱动"四个字拆开给你看。

本章精髓

1. **事件驱动的完整闭环**：ISR 只做搬运与发事件（`UART_DATA` / `UART_FIFO_OVF` / `UART_BUFFER_FULL` …），`uart_event_t` 经队列交给用户任务处理，是**中断礼仪的教科书示范**（回 C5 记账不办事）。
2. **三层缓冲各司其职**：硬件 FIFO（S3 上是 128 字节，`SOC_UART_FIFO_LEN`）→ 驱动环形缓冲（rx_ring_buf）→ 用户 buf，溢出三处各自的告警与处置，数据丢失的排查路径就藏在这三层。
3. **事件 ≠ 数据**：`UART_DATA` 事件的 `size` / `timeout_flag` 只是**元数据**（"现在有多少字节可读、是不是超时收尾"），真正的字节要你自己 `uart_read_bytes` 取走。别把事件结构体当串口字节负载。

- **能理解**：为什么事件队列满了会丢数据；为什么 `uart_read_bytes` 等的是 ring buffer 不是 FIFO。
- **能用**：用事件队列写"不定长帧接收"，并与 S7 手写版对照。

先修

- S7 USART（硬件层对照）、F4 队列、P4 中断。

先跑起来（10 分钟 quick win）

`code/esp32/02-uart-events`：UART1 经 GPIO Matrix 路由到 GPIO10(TX)/GPIO11(RX)（多功能扩展口，立创 wiki 核实）。扩展口发任意数据，串口监视打印事件类型与长度，并原样回显，第一次看到"驱动在替你记事件账"。

发数据之前先押一层：事件报"有数据到"的那一刻，你发的字节正睡在哪一层？押完再发，答案在事件模型那一节。

动画：字节与事件走两条路

字节路径：FIFO → ring buffer → 用户 buf；事件路径：ISR → queue → 你的任务。两条路并行，事件只负责"通知你发生了什么"，字节在另一条路上等你取。

IDF UART 事件动画

板卡与版本前提

- 板卡：立创·实战派 ESP32-S3；UART1 走 GPIO10/11（多功能扩展口）。
- 框架：ESP-IDF **v5.5.2**。UART 驱动源码在 `components/esp_driver_uart/src/uart.c`（5.5.x 已拆到独立组件；4.x 时代的 `driver/uart.c` 路径不再适用）。
- 芯片能力：`SOC_UART_FIFO_LEN = 128`（S3 硬件 FIFO 128 字节，soc_caps.h）。

小节结构

小节	内容	四件套
一、数据路径	三层缓冲全图与溢出点	库解析
二、事件模型	事件枚举全表；ISR→队列→任务的闭环	库解析
三、uart_config_t	逐字段落位（波特率/帧格式/流控/时钟源）	配置
四、不定长接收	DATA 事件 + 超时标志判帧尾	代码分析
五、发送路径	uart_write_bytes 的等待逻辑	代码分析
六、与手写版对照	S7 手写版 / IDF 版 / RT-Thread 设备版三方对照	库解析

一、数据路径：三层缓冲与三个溢出点

```
RX 线 → 硬件 FIFO(128B) → 驱动 ring buffer(rx_buf_size) → 你 uart_read_bytes 的用户 buf
              ↑                    ↑                          ↑
         FIFO_OVF 事件       BUFFER_FULL 事件            你自己负责别开太小
```

每一层都有自己的溢出告警：

- **FIFO_OVF**：ISR 没及时把 FIFO 搬进 ring buffer，通常是你把中断优先级配太低，或 ISR 被长阻塞堵住；
- **BUFFER_FULL**：ring buffer 满了你还没读，任务优先级不够，或队列消费太慢；
- 第三层：你的 `uart_read_bytes` 给的 buf 太小，截断是你自己的。

排查路径固定：先开日志看哪种 OVF/FULL 事件来了，就锁定是哪一层的问题。

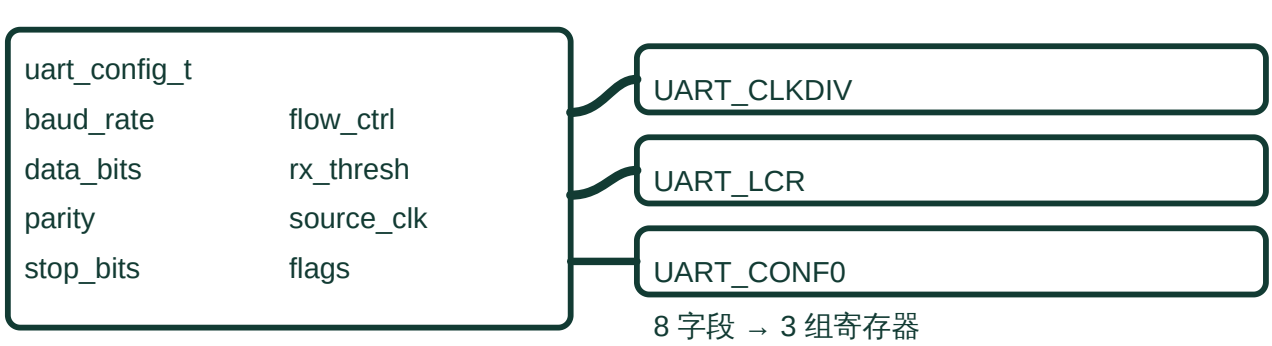
二、事件模型：ISR 记账，任务办事

`uart_event_t`（`esp_driver_uart/include/driver/uart.h`）的事件枚举：

事件	含义
`UART_DATA`	收到数据（`size` = 现在可读字节数，`timeout_flag` = 是否因超时收尾）
`UART_BREAK`	检测到 break（持续低电平）
`UART_BUFFER_FULL`	ring buffer 满
`UART_FIFO_OVF`	硬件 FIFO 溢出
`UART_FRAME_ERR`	帧错误（没等到停止位）
`UART_PARITY_ERR`	校验错

ISR 只负责把字节从 FIFO 搬进 ring buffer、把事件丢进队列。**所有"处理"都在任务上下文**，这就是"记账不办事"的中断礼仪。

三、uart_config_t：八字段落位



uart_config_t 八字段落位到 CLKDIV/LCR/CONF0

```
uart_config_t cfg = {
    .baud_rate  = 115200,          // 波特率（驱动内部算分频）
    .data_bits  = UART_DATA_8_BITS,
    .parity     = UART_PARITY_DISABLE,
    .stop_bits  = UART_STOP_BITS_1,
    .flow_ctrl  = UART_HW_FLOWCTRL_DISABLE,
    .source_clk = UART_SCLK_DEFAULT,   // 时钟源，决定波特率精度上限
};
```

与 S7 手写版对照：你不再需要自己算 BRR，驱动替你算，但**时钟源的精度上限仍然存在**（source_clk 选 APB 还是晶振，影响高波特率下的误差）。机制没变，只是封装了一层。

四、不定长接收：DATA + 超时标志

不定长帧的接收套路（与 S7 的 IDLE 判帧是同一招）：

```
case UART_DATA: {
    int len = uart_read_bytes(UART_PORT, buf, evt.size, 0);
    // evt.timeout_flag==1 说明这次 DATA 是"超时收尾"——帧尾到了
    break;
}
```

`timeout_flag` 就是 IDF 版的"IDLE 标志"：RX 线空闲超过阈值，驱动认为这一帧结束了。**字节在 ring buffer 里，帧边界在事件的 timeout_flag 里**：两条路合起来就是完整答案。

五、发送路径：uart_write_bytes 的等待逻辑

`uart_write_bytes` 把字节拷进 TX FIFO（无 TX ring 时）或 TX ring buffer，然后等"发完"。注意它与 S7 的 TXE/TC 是同一件事：**写进去 ≠ 发完**，`uart_wait_tx_done` 才是等待"停止位跑完"的那一个（回 S7 的 TXE/TC）。

六、与手写版对照：同一个问题，三种答案

维度	S7 手写版	IDF 版	RT-Thread 设备版
帧边界	IDLE 标志自判	timeout_flag 事件	设备框架回调
缓冲	自己维护环形数组	驱动 ring buffer	框架缓冲区
中断纪律	自己写 ISR	ISR 只发事件	框架封装
适用	学机制	做产品	跨平台统一接口

机制是一样的：FIFO + 环形 + 事件/中断 + 判帧边界。学透一个，另外两个只是换皮。

附录：工程完整源码

<<< ../../code/esp32/02-uart-events/main/main.c

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **ISR 记账发事件，三层缓冲防溢出；事件只带元数据，字节要自己读；判帧靠超时标志，发送完要等 done。**

延伸：IDF UART 驱动事件动画见 P5 动画；STM32 USART 对照见 S7；环形缓冲在 C8 有完整实现。

实物实验

- quick win + 压力测试：115200 连发 10KB，记录有无 OVF/FULL 事件；把 ring buffer 调小一半复现溢出，"溢出可观测"就是专业驱动的样子；
- 对照实验：同一串数据，S7 手写版（STM32）与 IDF 版（S3）的日志并排看：机制同构，眼见为实。

常见坑

- `uart_read_bytes` **阻塞时机误解**：它等的是 ring buffer，不是硬件 FIFO，超时参数按层理解；
- **事件队列没消费导致丢数据**：队列满了新事件被丢（有日志），事件任务优先级要给够；
- **流控脚没接 RTS/CTS 却开流控**：发送卡死查无原因，先核对 flow_ctrl 配置与接线；
- **多任务并发读写同一 UART**：驱动有锁但帧级互斥语义自行保证，协议帧级互斥用互斥量（F5 复训）；
- **把事件结构体当字节负载**：`UART_DATA` 事件的 `size` 只是"现在可读多少"的元数据，字节要 `uart_read_bytes` 读。

短自测

1. IDF UART 的三层缓冲各是什么？溢出分别报什么事件？

> **▼ 参考答案**
> 硬件 FIFO（128B，溢出报 FIFO_OVF）→ 驱动 ring buffer（满了报 BUFFER_FULL）→ 用户 buf（开太小是你自己的截断）。排查路径：看哪种事件来了，就锁定哪一层。

2. `UART_DATA` 事件的 `size` 是数据本身吗？字节在哪？

> **▼ 参考答案**
> 不是。size 是元数据（"现在有多少字节可读"），字节在 ring buffer 里，要 uart_read_bytes 取走。事件结构体不是串口字节负载。

3. 不定长帧的帧尾怎么判？与 STM32 的 IDLE 是什么关系？

> **▼ 参考答案**
> 看 UART_DATA 事件的 timeout_flag：RX 空闲超过阈值就置位，表示帧尾。它与 STM32 的 IDLE 标志是同一个思路：硬件管搬运，帧边界由"空闲/超时"事件告诉你。

对照表：本章概念 → 仓库与上游落点

概念	落点
事件枚举与 uart_event_t	ESP-IDF v5.5.2 `components/esp_driver_uart/include/driver/uart.h`
驱动实现	同上 `components/esp_driver_uart/src/uart.c`（5.5.x 新路径）
FIFO 长度	`soc/soc_caps.h` SOC_UART_FIFO_LEN=128
实验	code/esp32/02-uart-events
动画	idf-uart-events.svg
STM32 手写版对照	S7 USART

你做到了

- "成熟驱动"的评判标准具体化：事件账、缓冲层、配置对象、并发纪律；
- 读懂 2 千行驱动的勇气和方法，P6 的 SPI/I2C 直接复用；
- 三条路线在同一个"串口接收"问题上完成了第一次三方对照。

字节睡在中间那层：FIFO 只管收，事件只管喊，真正等你来取的是 ring buffer。下次丢数据，从三层各自的家门口查起，别一上来就怪驱动。

✅ 下一站：P6 SPI/I2C 驱动框架：拿板载 ST7789 与 QMI8658 当活教材。

通往单片机之路第 61 章

P6 SPI/I2C 驱动框架：板载外设当教材

难度 ★★★☆☆ · 约 50 分钟

本章罗盘

问题

为什么驱动要先配总线再挂设备？一句 transmit_receive 把 I2C 的时序礼仪代劳了什么？

交付

一份 QMI8658 链路自检记录（7 位地址 0x6a、WHO_AM_I 回 0x05、SDA=GPIO1/SCL=GPIO2、10Hz 六轴打印）加 ST7789 点亮三步流程（初始化序列 → CASET/RASET 开窗 → RAMWR 刷 RGB565，DC 电平分令数）。

边界

QMI8658 链路是 E07 工程实测，但 ST7789 的 GPIO 编号与初始化序列以立创 wiki/屏手册为准、成稿待核对，刷屏帧率（40M/80M 对比）与板上现象标注『待实测』；I2C/SPI 电气细节与时序手法回 S11/S12，本章不重复。

依据锚点 `a9 · f2 · mcu-code · mcu-lab · mcu-anim`

> 🎯 立创实战派 S3 板载一块 ST7789 彩屏（SPI）和一颗 QMI8658 姿态传感器（I2C）。学驱动框架最好的方式就是：**让屏幕亮起来，让姿态数据流出来**。本章用 IDF 的新版主机驱动框架（spi_master / i2c_master）打通这两件事。

本章精髓

1. 总线与设备两级模型：`spi_bus_config_t` 先配"哪条线"（MOSI/MISO/SCLK），`spi_device_interface_config_t` 再配"跟谁说话"（CS/速率/模式/队列深度）：总线可复用，设备可挂多个（与 R4 的总线模型异曲同工）。
2. 事务（transaction）是 SPI 的灵魂：`spi_transaction_t` 描述一次"命令+地址+数据"的完整交互，队列化异步执行，屏刷带宽就靠"队列里永远有事务在飞"。
3. 新 I2C 驱动的对象化：`i2c_new_master_bus` 出总线句柄，`i2c_master_bus_add_device` 出设备句柄，`i2c_master_transmit_receive` 一把梭"写寄存器地址+读数据"。对比 S11 的逐拍寄存器级，框架把"时序礼仪"全部代劳。

- **能理解**：为什么 SPI 屏要用 DC 引脚区分命令/数据；为什么异步事务的 buffer 生命周期必须覆盖传输期。
- **能用**：读通 E07 工程的 I2C 链路（QMI8658 WHO_AM_I + 六轴），把 ST7789 点亮流程画出来。

先修

- S11 I2C、S12 SPI（硬件时序）、P2 引脚矩阵。

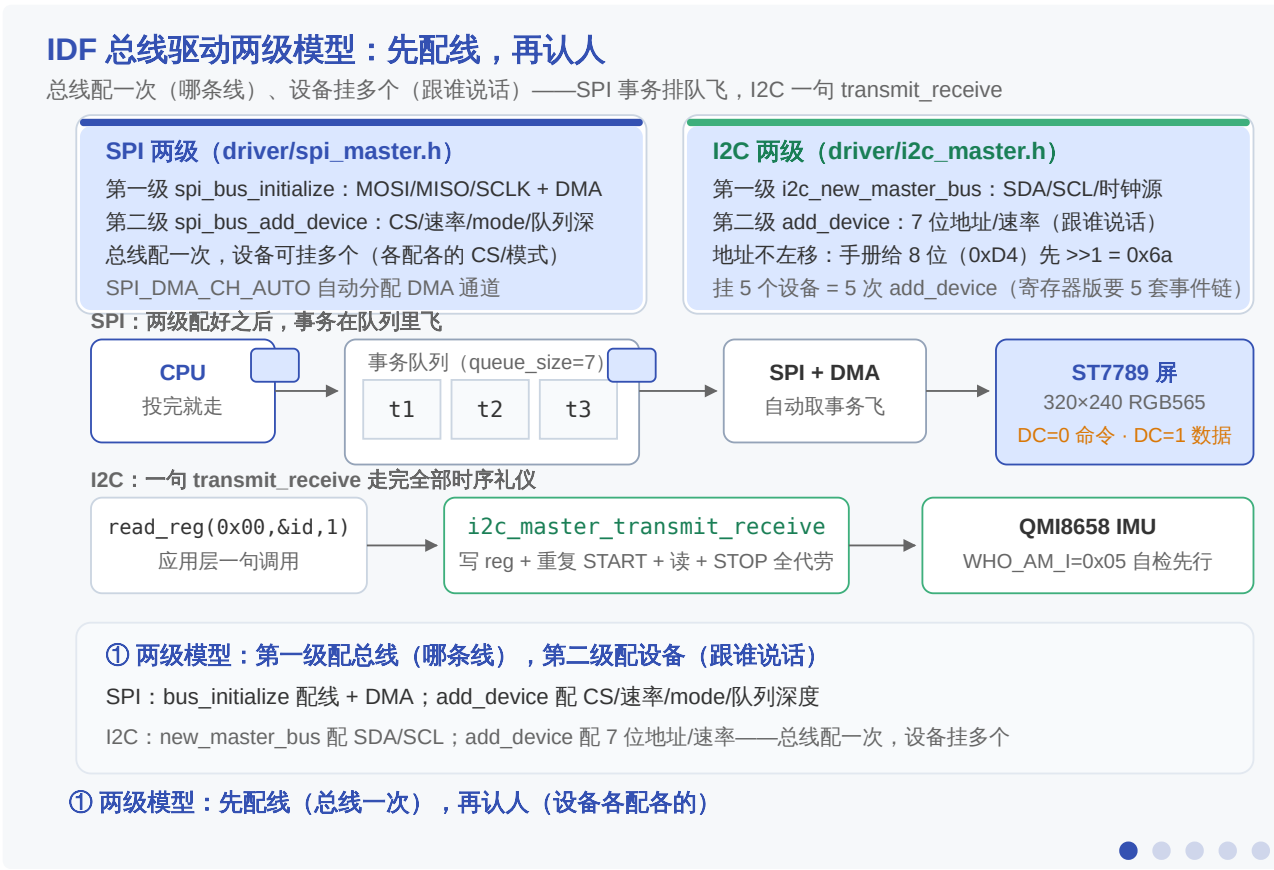
先跑起来（10 分钟 quick win）

I2C 先通：读 QMI8658 WHO_AM_I（寄存器 0x00），串口打印出期望的 ID 值 0x05，一次事务就能证明整条链路。E07 工程已提供完整源码，`idf.py flash monitor` 即可。

flash 之前先押个分家人：MOSI 上命令和像素排同一条队，谁在路口把两队分开？押完翻 SPI 事务那一节，点亮屏幕时对一眼那根线。

动画：两级模型与事务队列

先配线再认人的两级配置、SPI 事务在队列里连续飞向屏幕、I2C 一句 transmit_receive 走完全部时序礼仪：事务方块从 CPU 飞到 ST7789 的过程，就是屏刷带宽的来源。



IDF 总线驱动两级模型：先配线，再认人

板卡事实

- 立创实战派 S3（N16R8 模组）板载：ST7789 彩屏（SPI，320×240）、触摸 FT6336G（I2C，INT=IO17）、姿态 QMI8658（I2C）、音频 ES8311/ES7210（I2C+I2S）。**全部 GPIO 分配与 I2C 地址以立创 wiki 原理图页为准**（wiki.lckfb.com/zh-hans/szpi-esp32s3/）。
- QMI8658 事实（E07 工程已核实，`code/esp32/04-qmi8658/main/main.c`）：SDA=GPIO1、SCL=GPIO2、7 位地址 **0x6a**、WHO_AM_I 寄存器 0x00 回 **0x05**、六轴数据从 `AX_L(0x35)` 起连读 12 字节。
- ST7789 事实（以立创 wiki 原理图为准）：SPI 接口、支持 mode 0/3、DC 引脚区分命令/数据、CS 引脚选片，具体 GPIO 编号查板卡 wiki 原理图页，成稿时按你手上板子核对标注。

小节结构

小节	内容	四件套
两级模型	总线配置 vs 设备配置逐字段	配置
SPI 事务	transaction 结构与队列化；DC 线的 GPIO 角色	库解析
ST7789 点亮	初始化序列逐条；开窗/写像素 DMA 路径	代码分析
I2C 新驱动	master_bus/device 句柄模型；transmit_receive 组合事务	库解析
QMI8658 数据	WHO_AM_I→量程配置→六轴读取→换算	代码分析
与 S 篇对照	手写时序 vs 框架事务：各自的适用场景	库解析

一、两级模型：先配线，再认人

IDF 的新版 SPI/I2C 驱动都把"总线"和"设备"分成两层配置：这与 S11/S12 手写寄存器时"外设实例+引脚+模式"一把配的思路不同，但更贴合"一条总线挂多个设备"的现实。

SPI 两级（`driver/spi_master.h`）：

```
/* 第一级：总线—配"哪条线" */
spi_bus_config_t bus_cfg = {
    .mosi_io_num = MOSI_PIN, .miso_io_num = MISO_PIN, .sclk_io_num = SCLK_PIN,
    .max_transfer_sz = 4096, .flags = SPICOMMON_BUSFLAG_MASTER,
};
ESP_ERROR_CHECK(spi_bus_initialize(SPI2_HOST, &bus_cfg, SPI_DMA_CH_AUTO));

/* 第二级：设备—配"跟谁说话" */
spi_device_interface_config_t dev_cfg = {
    .clock_speed_hz = 40 * 1000 * 1000,  /* 40MHz */
    .mode = 0,                           /* CPOL=0/CPHA=0（ST7789 支持 0/3） */
    .spics_io_num = CS_PIN,              /* 片选引脚 */
    .queue_size = 7,                     /* 事务队列深度：屏刷带宽靠它 */
    .flags = SPI_DEVICE_HALFDUPLEX,
};
spi_device_handle_t spi;
ESP_ERROR_CHECK(spi_bus_add_device(SPI2_HOST, &dev_cfg, &spi));
```

总线配一次（MOSI/MISO/SCLK 固定），设备可挂多个（每个设备自己的 CS/速率/模式）。共享 SPI 总线挂屏+挂 Flash 就是这么来的。`SPI_DMA_CH_AUTO` 让 IDF 自动分配 DMA 通道（S8 DMA 的"外设侧请求"由框架代劳）。

I2C 两级（`driver/i2c_master.h`，E07 工程实测版 `code/esp32/04-qmi8658/main/main.c`）：

```
/* 第一级：总线 */
i2c_master_bus_config_t bus_cfg = {
    .i2c_port = I2C_NUM_0, .sda_io_num = 1, .scl_io_num = 2,
    .clk_source = I2C_CLK_SRC_DEFAULT, .glitch_ignore_cnt = 7,
    .flags.enable_internal_pullup = true,
};
ESP_ERROR_CHECK(i2c_new_master_bus(&bus_cfg, &bus));

/* 第二级：设备 */
i2c_device_config_t dev_cfg = {
    .dev_addr_length = I2C_ADDR_BIT_LEN_7, .device_address = 0x6a, .scl_speed_hz = 100000,
};
ESP_ERROR_CHECK(i2c_master_bus_add_device(bus, &dev_cfg, &imu));
```

对比 S11 的寄存器级：那里你手写 CR2.FREQ/CCR/TRISE、自己拆 START/ADDR/数据/STOP 的事件链；这里 IDF 把整套时序包成 `i2c_new_master_bus` + `i2c_master_bus_add_device`，你只管"总线+设备"两个结构体。**框架的价值在规模**：挂 5 个 I2C 设备，寄存器版要管 5 套地址+5 套事件链，框架版只是 5 次 `add_device`。

二、SPI 事务：队列化异步执行 + DC 引脚

SPI 屏刷的核心数据结构是 `spi_transaction_t`，它描述一次完整交互：

```
spi_transaction_t t = {
    .length = 8 * n,            /* 位数 = 字节数 × 8 */
    .tx_buffer = buf,           /* 发送缓冲（NULL = 只收） */
    .rx_buffer = NULL,          /* 接收缓冲（NULL = 只发） */
    .flags = 0,                  /* 可配 SPI_TRANS_USE_RXDATA 等 */
};
ESP_ERROR_CHECK(spi_device_polling_transmit(spi, &t));  /* 同步等完成 */
/* 或异步：spi_device_queue_trans + spi_device_get_trans_result */
```

队列化的精髓（`queue_size = 7`）：你可以连续投 7 个事务进队列，CPU 不必等每个完成，SPI 外设+DMA 自己从队列里取事务飞，CPU 投完就去准备下一批。屏刷 60fps 的带宽就靠"队列里永远有事务在飞"，CPU 不会卡在 `while(TXE)` 上（S12 的寄存器版要自己等每个标志）。

DC 引脚：ST7789 这类屏的 SPI 帧不只是数据，还有命令（开窗、设显示模式等）。DC 引脚电平区分当前帧是命令（DC=0）还是数据（DC=1）。这就是 S12 提到的"SPI 没有地址阶段，靠应用层约定"的屏版实现。`spi_device_interface_config_t` 里 `.spics_io_num` 是 CS，DC 一般另配一个 GPIO，每次发事务前按命令/数据拉低/拉高。

骨架坑列表的"CS 与 DC 搞反"：DC 是数据/命令选择（每帧切），CS 是片选（整段事务期间低）；接错或配置错位，症状是"全屏噪声"（命令被当数据、数据被当命令，初始化序列全乱）。

三、ST7789 点亮：初始化序列→开窗→刷像素

点亮流程三步（伪代码，完整初始化序列以屏手册 + 立创 wiki 为准）：

```
/* ① 初始化序列：一串命令+参数，每条走一次 SPI 事务（DC=0 命令、DC=1 参数） */
static const uint8_t init_cmds[] = { /* SLPOUT=0x11, DISPON=0x29, ... 以屏手册为准 */ };
for (each cmd in init_cmds) {
    gpio_set_level(DC_PIN, 0);  spi_send(cmd);
    gpio_set_level(DC_PIN, 1);  spi_send(params);
}

/* ② 开窗：CASET=0x2A（列地址）+ RASET=0x2B（行地址）设定刷写区域 */
gpio_set_level(DC_PIN, 0); spi_send(0x2A);           /* 命令 */
gpio_set_level(DC_PIN, 1); spi_send(col_start, col_end);  /* 数据：4 字节 */

/* ③ 写像素：RAMWR=0x2C 后连续发 RGB565 数据，DC=1 */
gpio_set_level(DC_PIN, 0); spi_send(0x2C);
gpio_set_level(DC_PIN, 1); spi_send(pixel_buf, n * 2);  /* 每像素 2 字节 RGB565 */
```

刷一帧 320×240×2 = 153600 字节。40MHz SPI 理论带宽 5MB/s，一帧约 30ms。60fps（16.7ms/帧）吃紧，所以屏刷常用 DMA + 队列化事务（第二节）让 CPU 不卡在传输上。

四、I2C 新驱动：transmit_receive 一把梭

E07 工程读 QMI8658 寄存器的完整代码（`code/esp32/04-qmi8658/main/main.c` 第 19–27 行）：

```
static esp_err_t read_reg(uint8_t reg, uint8_t *bytes, size_t count)
{
    return i2c_master_transmit_receive(imu, &reg, 1, bytes, count, 100);
}
static void write_reg(uint8_t reg, uint8_t value)
{
    const uint8_t bytes[] = {reg, value};
    ESP_ERROR_CHECK(i2c_master_transmit(imu, bytes, sizeof(bytes), 100));
}
```

对照 S11 的寄存器级写 EEPROM：那里你手写 `CR1 |= START` →等 SB→ `DR = 地址` →等 ADDR→清 ADDR（先 SR1 后 SR2）→ `DR = 数据` →等 TXE/BTF→ `CR1 |= STOP`，六步事件链一步不能错；这里 `i2c_master_transmit_receive(imu, &reg, 1, bytes, count, 100)` 一句把"写寄存器地址 + 重复 START + 读 N 字节 + STOP"全套礼仪代劳。100 是超时 ms。

地址处理的坑（骨架坑列表）：IDF 用 7 位地址（`device_address = 0x6a`，不左移）；手册给 8 位地址（如 0xD4）要先 `>>1`（S11 的"0x50 上线变 0xA0"在 IDF 这边自动处理）。QMI8658 手册标 7 位地址 0x6a，直接填；若某器件手册给 8 位 0xD4，填 `(0xD4 >> 1) = 0x6a`。

五、QMI8658 数据：WHO_AM_I → 量程 → 六轴读取

E07 工程的完整初始化与读取流程（`code/esp32/04-qmi8658/main/main.c`）：

```
/* ① 链路自检：读 WHO_AM_I */
uint8_t id;
ESP_ERROR_CHECK(read_reg(0x00, &id, 1));
if (id != 0x05) { ESP_LOGE(TAG, "WHO_AM_I=0x%02x (expected 0x05)", id); return; }

/* ② 量程配置（寄存器地址以 QMI8658 手册为准） */
write_reg(0x60, 0xb0);              /* RESET */
vTaskDelay(pdMS_TO_TICKS(20));
write_reg(0x08, 0x00);              /* CTRL7：先关闭采样 */
write_reg(0x02, 0x40);              /* CTRL1：地址递增 */
write_reg(0x03, 0x15);              /* CTRL2：±4g、250Hz */
write_reg(0x04, 0x55);              /* CTRL3：±512dps、250Hz */
write_reg(0x08, 0x03);              /* CTRL7：加速度+角速度使能 */

/* ③ 周期读取：状态位 + 连读 12 字节 */
for (;;) {
    vTaskDelay(pdMS_TO_TICKS(100));   /* 10Hz */
    uint8_t status, bytes[12];
    if (read_reg(0x2e, &status, 1) == ESP_OK && (status & 0x03) == 0x03) {
        read_reg(0x35, bytes, sizeof(bytes));   /* AX_L 起 12 字节 */
        /* 换算：32768/4=8192 LSB/g；32768/512=64 LSB/(deg/s) */
    }
}
```

三个要点对应 S11 的纪律：

1. **WHO_AM_I 先行**：和 S12 读 W25Q 的 JEDEC ID 同理，链路自检的第一步，ID 错就停（E07 的 `return` 不无限等待）。
2. **量程与灵敏度换算**：±4g → 8192 LSB/g（32768/4），±512dps → 64 LSB/(deg/s)（32768/512）。这是 S9 ADC "分辨率/量程"在 IMU 上的同构。
3. **状态位等数据就绪**：`status & 0x03 == 0x03` 表示加速度+角速度都完成了一次采样，再连读 12 字节，避免读到半新半旧的样本。E07 的教学口径是"每 100ms 取最新样本，不声称无丢样采集"（main.c 第 5 行注释），诚实标注。

字节组装（main.c 第 29–33 行）显式小端 + 符号扩展，不依赖宿主字节序和指针别名。这是 C7 严格别名纪律的实战：用 `p[0] | (p[1] << 8)` 而非 `*(int16_t*)p`，避免别名违规。

六、与 S 篇对照：手写时序 vs 框架事务

维度	S 篇手写寄存器	P 篇 IDF 框架
配置粒度	每个寄存器位手填	结构体字段 → 框架翻译
时序礼仪	逐拍等事件（SB/ADDR/TXE/BTF）	`transmit_receive` 一句代劳
多设备	每设备一套事件链	`add_device` 复用总线
DMA	手配流/通道/请求映射	`SPI_DMA_CH_AUTO` 自动分配
调试	看寄存器位即知状态	看 ESP_LOG + 协议分析仪
适用	学习时序/极致性能	快速接入器件/多设备

选型很朴素：**学时序手写，做产品用框架**。S 篇让你懂"框架在替你做什么"，P 篇让你"用框架快速接入器件"。两层能力都备，才是嵌入式工程师的完整武器库。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **先配线再认人（总线/设备两级），SPI 靠事务排队飞，I2C 一句 transmit_receive 走完礼仪；屏要 DC 分令数，传感器先问 WHO_AM_I。**

延伸：IDF 总线驱动两级模型动画见 P6 动画；STM32 SPI/I2C 对照见 S11 和 S12；GD32 外设编号差异见 G4。

实物实验

- E07 读 QMI8658：摇板子看三轴数据跳变；
- ST7789 刷屏：纯色→色带→帧率粗测（改 SPI 时钟 40M/80M 对比）。

常见坑

- **SPI 模式配错**：ST7789 用 mode0/3（以屏手册为准），模式错白屏花屏。
- **CS 与 DC 搞反**：DC 是数据/命令选择不是片选，接线/配置错位症状为"全屏噪声"。
- **I2C 地址左移忘处理**：IDF 用 7 位地址（不左移），手册给 8 位地址要先 >>1（S11 复训）。
- **事务缓冲用局部变量做异步传输**：函数返回后缓冲失效：异步事务的 buffer 生命周期必须覆盖传输期（C1 复训）。
- **I2C 总线未上拉**：`enable_internal_pullup = true` 适合原型；高速/多设备时外接 4.7k 上拉更稳（S11 上拉节）。

短自测

1. SPI/I2C 的"两级模型"分别配什么？为什么总线只配一次、设备可挂多个？

> **▼ 参考答案**
> 第一级配总线（SPI: MOSI/MISO/SCLK；I2C: SDA/SCL+时钟源），第二级配设备（SPI: CS/速率/模式/队列深度；I2C: 地址/速率）。总线是物理线，一条线只有一个引脚分配，配一次就够；设备是挂在总线上的器件，每个有自己的 CS（SPI）或地址（I2C）+速率+模式，所以可挂多个、各配各的。共享总线是"线复用"的实现。

2. ST7789 的 DC 引脚干什么？为什么 CS 和 DC 搞反会"全屏噪声"？

> **▼ 参考答案**
> DC 是数据/命令选择引脚：DC=0 当前 SPI 帧是命令（如开窗命令 0x2A），DC=1 是数据（如像素 RGB565）。CS 是片选，整段事务期间低。搞反后：命令被当数据写入像素 RAM、数据被当命令送进命令解释器，初始化序列全乱，屏不知道显示什么，全屏随机噪声。

3. E07 读 QMI8658 为什么先读 WHO_AM_I？ID 错为什么直接 return 而不是重试？

▼ **参考答案**
WHO_AM_I 是链路自检：读对说明 I2C 总线+设备地址+寄存器地址全对，链路可用；读错说明接线/地址/上拉有问题，继续写配置无意义（写进错误的器件）。return 而非重试是教学工程的诚实口径：ID 错先看硬件，不在软件层无限等待掩盖问题。产品代码可加重试+日志，但自检失败仍要停。

4. IDF 的 7 位地址 `0x6a`，手册给 8 位地址 `0xD4`，怎么填？

▼ **参考答案**
IDF 用 7 位地址（device_address 字段不左移），手册给 8 位地址要先 >>1：0xD4 >> 1 = 0x6a。QMI8658 手册直接标 7 位 0x6a 就直接填。判断方法：8 位地址最低位通常是 0（写）或 1（读），7 位地址最低位是 0；或者看手册"7-bit address"字段直接拿。

5. 异步 SPI 事务为什么不能用局部变量做 tx_buffer？

▼ **参考答案**
异步事务投进队列后，SPI 外设+DMA 会在"未来的某个时刻"才读 buffer 内容。如果 buffer 是局部变量，函数返回后栈帧释放，那块内存被复用，DMA 读到的是垃圾。异步事务的 buffer 生命周期必须覆盖整个传输期：用全局变量、堆分配（直到 get_trans_result 后释放）、或静态缓冲池。同步事务（polling_transmit）等完成才返回，局部变量安全。

对照表：本章概念 → 仓库与上游落点

概念	落点
I2C 两级模型 + transmit_receive	`code/esp32/04-qmi8658/main/main.c` 36–47、19–27 行（E07 工程实测）
QMI8658 WHO_AM_I + 量程 + 六轴	同上 48–80 行；寄存器地址以 QMI8658 手册为准
SPI 两级模型 + 事务队列化	IDF `driver/spi_master.h`；ST7789 初始化以屏手册+立创 wiki 为准
DC 引脚命令/数据区分	本节第二、三节；S12 SPI 无地址阶段
7 位地址 vs 8 位地址	S11 I2C 地址节 + 本节第四节
字节组装避免别名违规	E07 main.c 29–33 行；C7 严格别名

你做到了

- 两块板载外设听你调遣：屏会亮，姿态会报数；
- 总线/设备/事务的框架语言成型，以后接任何 SPI/I2C 器件都是同一套流程。

分家人是 DC：一根线的高低电平，替屏幕省掉"这笔是命令还是像素"的猜谜。这也是它和 CS 一样必须进设备配置的原因。总线只管送，听懂不听懂，是设备端自己的事。

✅ 下一站：P7 定时器与 LEDC：GPTimer 和 LEDC 的分工，呼吸灯与背光调光。

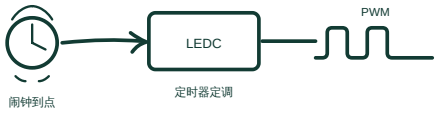
可独立运行的最小实验

QMI8658 六轴读取 已提供独立工程、配置说明和完整源码。板上现象待实测。

通往单片机之路第 62 章

P7 定时器与 LEDC：各管一段时间

难度 ★★☆☆☆ · 约 35 分钟



本章罗盘

问题

S3 为什么把定时器和 PWM 拆成 GPTimer/LEDC 两个外设？分辨率和频率为什么不能同时拉满？

交付

一段可跑的两级配置 + 硬件 fade 呼吸灯代码（GPIO10、5kHz/10 位，03-ledc-fade），配 f= 时钟/2^位数的频率-位数取舍表（舵机 50Hz/14 位 ↔ 音频 20kHz/≤10 位）和 S3 能力清单（8 通道/14 位/仅低速，soc_caps.h 实读）。

边界

S3 只有 8 通道、14 位、低速模式，经典 ESP32 的 16 通道/高速模式结论不能外推；GPTimer 抖动方差与 fade 线性度要逻辑分析仪实测（E03）才有数字，章内只给量级判断；MCPWM 与输入捕获不涉及（回 S6）。

依据锚点 `a7` · `a9` · `f2` · `mcu-code` · `mcu-lab` · `mcu-anim`

> 🎯 STM32 里 TIM 一肩挑（时基+PWM）；ESP32-S3 把两件事分给了两个外设：**GPTimer** 管"什么时候叫我"（高精度报警），**LEDC** 管"以什么节奏眨眼"（PWM 调光调速）。分工即选型。

本章精髓

1. **GPTimer 是"闹钟工厂"**：计数器+报警值，到点回调；回调跑在 ISR 上下文（IRAM 纪律回 P4），微秒级分辨率，适合测距、超时、采样节拍。
2. **LEDC 是"8 通道 PWM 乐团"（S3 上只有低速模式）**：定时器（分辨率+频率）与通道（绑定 GPIO+占空比）两级配置。占空比渐变用硬件 `ledc_fade`（无需 CPU 逐点写）。**别套经典 ESP32 的 16 通道/高速模式**：S3 是 8 通道（`SOC_LEDC_CHANNEL_NUM=8`）、定时器位宽 14 位（`SOC_LEDC_TIMER_BIT_WIDTH=14`）、仅低速模式（soc_caps.h 核实）。
3. **分辨率×频率是此消彼长**：`f = 时钟 / 2^位数`：位数越高，可达频率越低。舵机 50Hz 可上 14 位，音频级应用就得降位数。

- **能理解**：为什么改 duty 必须 update（双缓冲）；为什么 fade 与手动 set 不能混用；为什么 S3 不能套经典 ESP32 的结论。
- **能用**：写出 LEDC 两级配置链；用硬件 fade 做呼吸灯；说出分辨率与频率的取舍。

先修

- S6 TIM（PWM 本质）、P2 引脚、P4 中断。

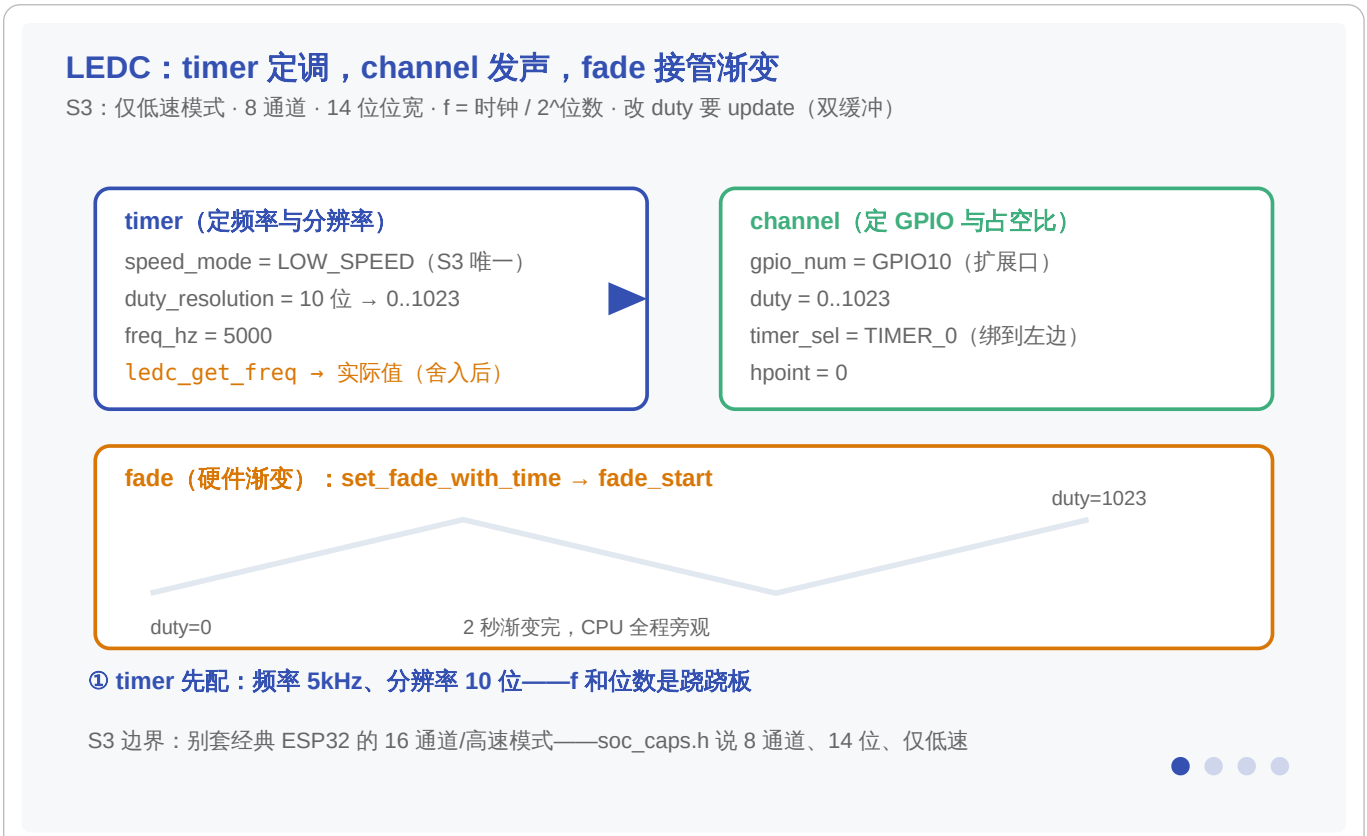
先跑起来（10 分钟 quick win）

`code/esp32/03-ledc-fade`：LEDC 通道绑到 **GPIO10**（多功能扩展口，立创 wiki 核实），5kHz/10 位起步，硬件 fade 做呼吸，占空比的物理意义第一次"看得见"。

呼吸灯亮起来之前先押个闲差：占空比从零爬到满的那几秒，CPU 在忙循环、在被打断、还是根本不用管？押完再 flash，答案会让你重新认识"定时"两个外设的分工。

动画：timer 定调，channel 发声

LEDC 的两级分工一目了然：timer 定频率与分辨率（"多快、多细"），channel 定 GPIO 与占空比（"从哪出、多高"），fade 引擎按步进改比较值。



LEDC timer 与 channel 动画

板卡与版本前提

- 板卡：立创·实战派 ESP32-S3；PWM 从 **GPIO10**（多功能扩展口）输出。
- 框架：ESP-IDF **v5.5.2**；LEDC 驱动在 `components/esp_driver_ledc/`。
- 芯片能力（soc_caps.h）：S3 是 8 通道、14 位位宽、仅低速模式，与经典 ESP32 的 16 通道/高速模式不是一回事。

小节结构

小节	内容	四件套
一、GPTimer 报警	gptimer_config/alarm 回调；ISR 上下文纪律	配置
二、采样节拍	1kHz 定时 vs vTaskDelay 抖动实测	代码分析
三、LEDC 两级	timer/channel 配置逐字段；S3 低速模式的边界	配置
四、硬件渐变	ledc_fade 系列；阻塞/非阻塞两模式	代码分析
五、分辨率取舍	频率-位数换算表；舵机/背光案例	配置
六、对照 TIM	STM32 TIM 一肩挑 vs S3 分工设计的哲学	库解析

一、GPTimer 报警：闹钟工厂

GPTimer 的配置对象：`gptimer_config_t`（时钟源、分辨率、计数方向）→ `gptimer_alarm_config_t`（报警值、自动重载）→ 回调注册。回调跑在 **ISR 上下文**（IRAM 纪律回 P4）：回调里只办事不等待，不能用会阻塞的 API。

二、采样节拍：1kHz 定时 vs vTaskDelay

`vTaskDelay(1)` 的实际周期是 tick 粒度（本工程 1ms），抖动就是 ±1 tick；GPTimer 报警回调是硬件定时，抖动在微秒级。做采样节拍、测距超时这类"时间要准"的事，用 GPTimer 不是用 vTaskDelay。

三、LEDC 两级：timer 定调，channel 发声

两级配置链：

```
/* timer：多快、多细 */
ledc_timer_config_t timer = {
    .speed_mode      = LEDC_LOW_SPEED_MODE,   /* S3 只有低速模式 */
    .duty_resolution = LEDC_TIMER_10_BIT,     /* 占空比 0..1023 */
    .timer_num       = LEDC_TIMER_0,
    .freq_hz         = 5000,
    .clk_cfg         = LEDC_AUTO_CLK,
};

/* channel：从哪出、占多少 */
ledc_channel_config_t ch = {
    .gpio_num   = GPIO_NUM_10,
    .speed_mode = LEDC_LOW_SPEED_MODE,
    .channel    = LEDC_CHANNEL_0,
    .timer_sel  = LEDC_TIMER_0,
    .duty       = 0,
    .hpoint     = 0,
};
```

`ledc_get_freq` 回读实际频率：请求值与实际值可能因分频舍入不同（回 S6 的"+1"教训）。

四、硬件渐变：ledc_fade 系列

`ledc_fade_func_install()` 装渐变引擎 → `ledc_set_fade_with_time()` 设目标占空比与用时 → `ledc_fade_start()` 启动。**fade 期间 CPU 不逐点干预**：这是它比"定时器模拟 PWM 渐变"高级的地方。

两模式：`LEDC_FADE_NO_WAIT`（启动就返回，fade 在后台跑）vs `LEDC_FADE_WAIT_DONE`（等渐变完）。选哪个取决于你要不要在渐变途中干别的。

注意纪律：fade 与手动 `ledc_set_duty` 不能混用，渐变途中手动 set 会被 fade 引擎覆盖，先停 fade 再接管。

五、分辨率取舍：频率-位数跷跷板

需求	频率	位数	取舍
舵机	50Hz	14 位	低频高分辨率，随便选
LED 调光	1~5kHz	10~13 位	常见组合
音频级	>20kHz	≤10 位	高频就得降位数

本质：`f = 时钟 / 2^位数`，时钟有限，位数和频率只能二选一。`ledc_get_freq` 回读实际值，别假设请求值一定生效。

六、对照 TIM：一肩挑 vs 分工

STM32 的 TIM 一个外设包干时基+PWM+捕获；S3 拆成 GPTimer（计时）+ LEDC（PWM）。设计哲学的对照：**复用度高但配置纠缠** vs **职责单一但各司其职**。学到的东西是相通的：比较值、预装载、双缓冲、fade/渐变，只是名字换了。

附录：工程完整源码

<<< ../../code/esp32/03-ledc-fade/main/main.c

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **GPTimer 定闹钟（ISR 里快办事），LEDC 开乐团（timer 定调 channel 发声）；渐变交给硬件 fade，位数频率跷跷板；S3 只有低速 8 通道，别套经典 ESP32。**

延伸：LEDC 定时器与通道动画见 P7 动画；STM32 TIM PWM 对照见 S6；定时器输入捕获在 S6 TIM 展开。

实物实验

- 呼吸灯 + E03 分析仪看占空比 的 S3 版：fade 曲线抓取，验证硬件渐变的线性；
- 抖动对比：同一 1kHz 节拍，GPTimer 回调版 vs vTaskDelay 版，逻辑分析仪看周期方差；
- 分辨率实测：5kHz/10 位改成 5kHz/14 位，看 `ledc_get_freq` 回读是否还成立。

常见坑

- **改 duty 忘 update**：LEDC 双缓冲，`ledc_update_duty` 才装载（与 TIM 直接写 CCR 不同）；
- **fade 与手动 set 混用**：渐变途中手动 set 被 fade 引擎覆盖，停 fade 再接管；
- **套经典 ESP32 结论**：S3 是 8 通道、14 位、仅低速模式，没有高速模式可选；
- **GPTimer 回调里做长活**：回调=ISR，阻塞一毫，系统抖一分（P4 复训）；
- **请求频率当实际频率**：分频舍入让两者不等，用 `ledc_get_freq` 回读。

短自测

1. S3 的 LEDC 与经典 ESP32 的 LEDC 差在哪？

> **▼ 参考答案**
> S3 是 8 通道、定时器位宽 14 位、只有低速模式（soc_caps.h: SOC_LEDC_CHANNEL_NUM=8、SOC_LEDC_TIMER_BIT_WIDTH=14）。经典 ESP32 的 16 通道/高速模式结论不能套。

2. 为什么改完 `ledc_set_duty` 还要 `ledc_update_duty`？

> **▼ 参考答案**
> LEDC 是双缓冲：set 只写影子寄存器，update 才装载到硬件生效。漏了 update，波形不变。

3. 分辨率位数和频率为什么不能同时拉满？

> **▼ 参考答案**
> f = 时钟 / 2^位数。时钟有限，位数每加 1，同档频率减半。舵机 50Hz 可以上 14 位，音频级 20kHz+ 就得降位数。

对照表：本章概念 → 仓库与上游落点

概念	落点
S3 通道数/位宽/低速模式	`soc/soc_caps.h` SOC_LEDC_CHANNEL_NUM=8、SOC_LEDC_TIMER_BIT_WIDTH=14
fade API 与线程安全	ESP-IDF v5.5.2 `components/esp_driver_ledc/include/driver/ledc.h`
实验	code/esp32/03-ledc-fade
动画	ledc-timer-channel.svg
STM32 对照	S6 TIM

你做到了

- S3 的"时间双雄"各就各位；
- PWM 从"能用"到"用好"：fade、分辨率、速度组边界全在掌握；
- 与 STM32 TIM 的对照让你看清"同一个问题、两种设计"。

闲差押中了：那几秒 CPU 是真闲，fade 由硬件自己爬坡，一步都不用你盯。把"什么时候叫"和"以什么节奏眨"拆给两个外设，拆的就是这个意思。

✅ 下一站：P8 Wi-Fi 精髓：esp_event 事件循环与连接状态机，联网的第一性原理。

通往单片机之路第 63 章

P8 Wi-Fi 精髓：事件循环与连接状态机

难度 ★★★☆☆ · 约 50 分钟

本章罗盘

问题

esp_wifi_connect 之后该怎么等？路由器一重启，设备为什么就永远离线了？

交付

教科书骨架六步初始化代码（NVS→netif→event loop→注册回调→init→start）+ 带 1s→60s 指数退避、esp_timer 外置的重连 handler，加一张 WIFI_EVENT/IP_EVENT 与 reason code 速查分诊表。

边界

只走 STA 联网主线，AP/SoftAP、多接口并存与 ESP-NOW 共存不展开（P9）；socket/LwIP 应用编程不在范围，GOT_IP 只是网络可用的起点；退避上限 60s 是产品权衡值不是规范；重启路由器量化断连/重连耗时属实物实验项。

依据锚点 a9 · f2 · mcu-anim

> 🎯 新手写 Wi-Fi：`esp_wifi_connect()` 之后 `while(1);` 干等，连上了靠运气，断线了干瞪眼。高手只信一件事：**Wi-Fi 是个状态机，事件循环是它的语音播报**。听懂播报（WIFI_EVENT/IP_EVENT），就掌握了联网的第一性原理。

本章精髓

1. esp_event 是 IDF 的"神经系统"：事件循环任务（默认 loop）+ 事件基（WIFI_EVENT/IP_EVENT）+ 注册回调：驱动/协议栈出事件，你的代码只响应，不轮询（这就是"事件驱动架构"，与 P5 的驱动事件同思想）。
2. 连接状态机就六个状态：init→config→start→connect→(SCAN_DONE/AUTH/DISASSOC 中间态)→GOT_IP。`WIFI_EVENT_STA_DISCONNECTED` 必须处理**重连策略**，否则路由器一重启设备就永远离线。
3. LwIP 在幕后：拿到 IP 后才有 socket：`IP_EVENT_STA_GOT_IP` 是"网络可用"的唯一合法起点，之前调 connect() 都是空气。

- **能理解**：为什么 handler 运行在事件任务上下文、不能阻塞；为什么断线重连要用任务/定时器外置而不是在 handler 里 sleep。
- **能用**：写出"教科书骨架"六步初始化 + 带指数退避的重连，看懂 `idf.py flash monitor` 日志里的状态迁移。

先修

- F6 事件思想、P3 Kconfig（Wi-Fi 配置项）。

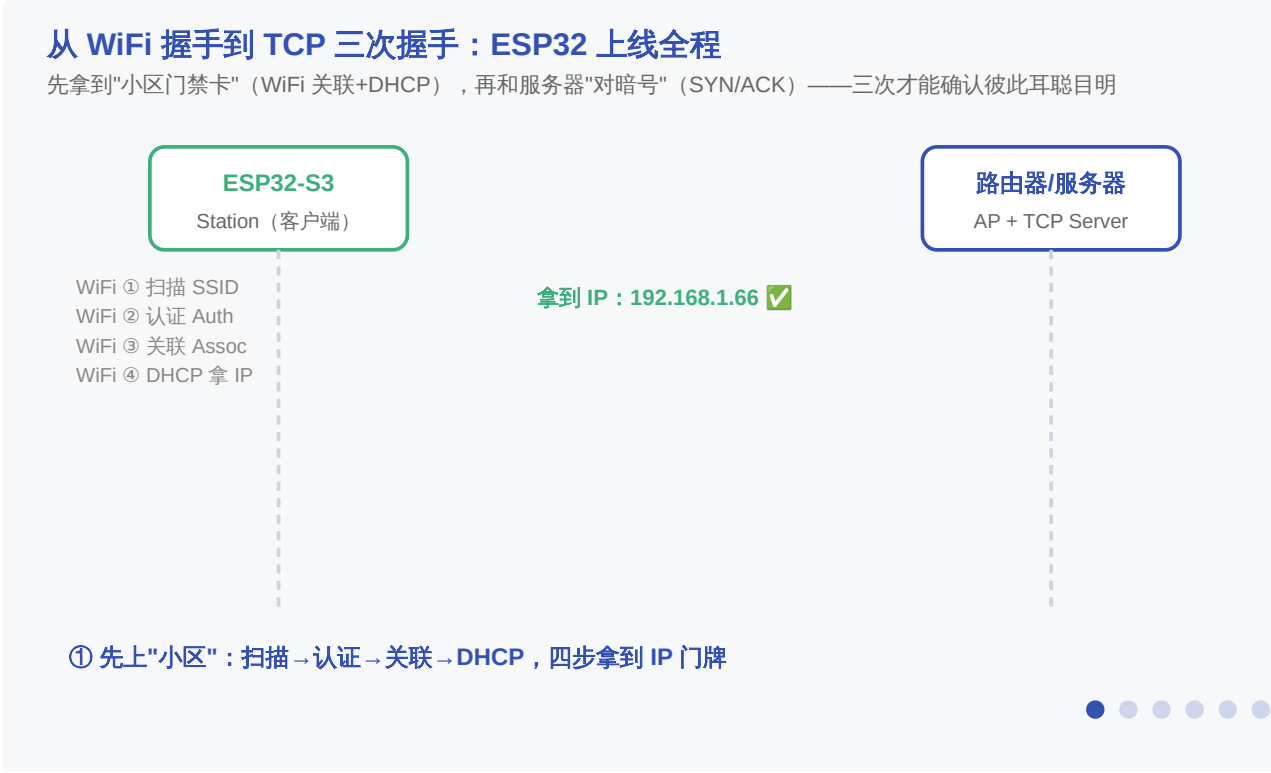
先跑起来（10 分钟 quick win）

menuconfig 填上 Wi-Fi 账号密码（示例工程惯例），`idf.py flash monitor`：观察日志从 `wifi:state: init` 到 `got ip` 的完整旅程，把日志与状态机逐条对号入座。

monitor 之前先押趟车：日志从 init 一路走到拿到 IP，这条旅程要停几个大站？押个整数，跑完拿事件全表对号。

动画：从 WiFi 握手到 TCP 三次握手

先拿"小区门禁卡"（关联+DHCP 得 IP），再和服务器对暗号：SYN(seq=100) → SYN+ACK(seq=300,ack=101) → ACK(ack=301)。**三次才能证明双向耳聪目明**：序号是防迟到旧报文骗开门的。



TCP 握手动画

配套视频

> 🎬 **一条视频讲清楚 TCP 与 UDP：三次握手与四次挥手（65 万播放，和上图逐帧对照）** @ 哔哩哔哩
> 离线书不内嵌在线视频；去 哔哩哔哩 观看 ↗

小节结构

小节	内容	四件套
事件循环	esp_event 架构；默认 loop vs 自建 loop	库解析
骨架六步	教科书代码逐行；每步的错误码含义	代码分析
事件全表	WIFI_EVENT_*IP_EVENT*_ 速查与触发时机	配置
重连策略	DISCONNECTED 的原因码；退避算法实现	代码分析
NVS 与校准	Wi-Fi 为什么需要 NVS（射频校准数据）	配置
状态机对照	日志级别调高看协议栈内部状态迁移	库解析

一、事件循环：IDF 的"神经系统"

esp_event 是 IDF 所有"异步事务"的统一通道：Wi-Fi、IP、BLE、UART 驱动等都在这里发事件，你的代码注册回调接收：**只响应，不轮询**。这是事件驱动架构（EDA）在 IDF 的落地，与 P5 UART 驱动 的"驱动事件→任务处理"同思想，与 F6 任务通知/事件组 同源。

三件套：

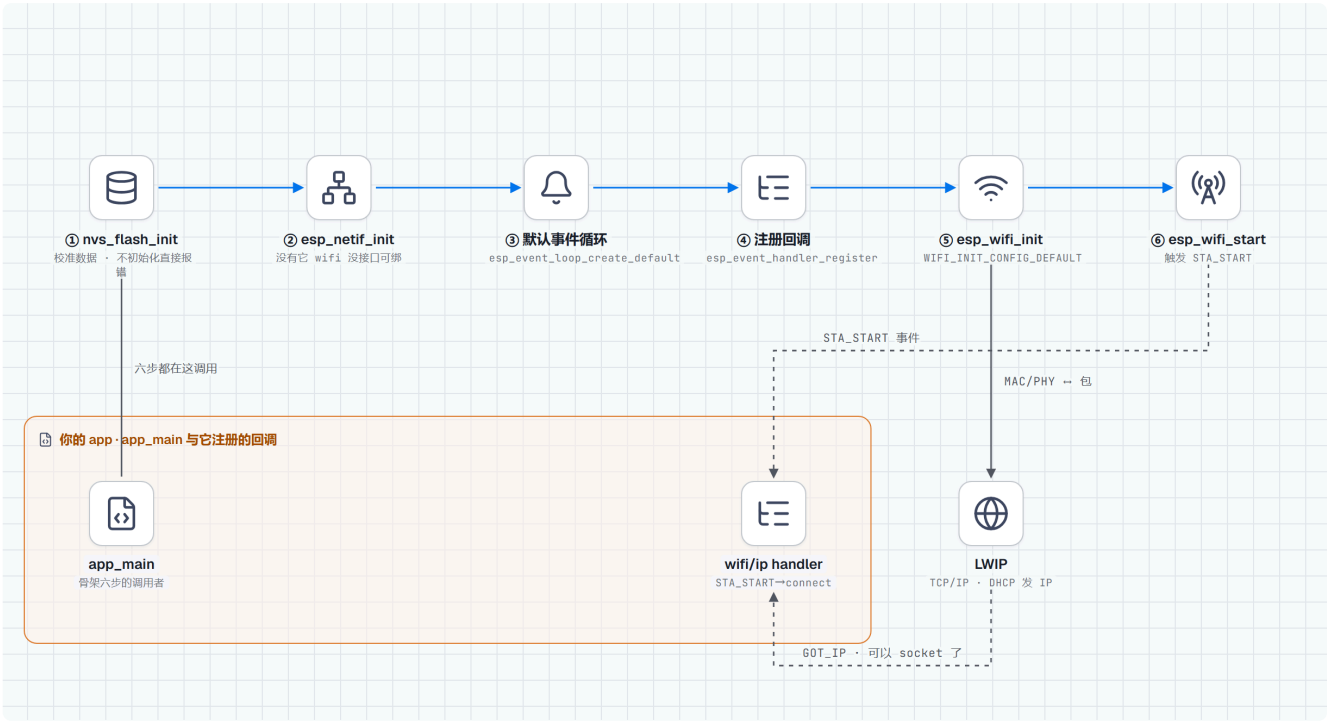
- **事件循环任务**（event loop task）：默认 loop 由 `esp_event_loop_create_default()` 创建，跑在一个 FreeRTOS 任务里，所有 handler 在这个任务的上下文执行。
- **事件基**（event base）：`WIFI_EVENT`、`IP_EVENT`、`BLUETOOTH_EVENT` 等，是一组相关事件的"姓氏"。
- **注册回调**：`esp_event_handler_register(WIFI_EVENT, WIFI_EVENT_STA_DISCONNECTED, handler, arg)`，指定"什么基的什么事件"由谁处理。

handler 签名：`void handler(void *arg, esp_event_base_t base, int32_t id, void *data)`，`data` 携带事件特有数据（如 `wifi_event_sta_disconnected_t` 含 reason code）。

关键纪律：handler 跑在事件任务上下文，**不能阻塞**。事件任务是单线程的，一个 handler 阻塞（`vTaskDelay(1000)`、等信号量），后续所有事件（别的 handler）都堵在队列里。Wi-Fi 断线了你的重连 handler 睡 1 秒，这 1 秒里 IP 事件、BLE 事件全丢。骨架坑列表第一名的"handler 里阻塞重连"就是这么翻车的。重连等长操作要外置到独立任务或定时器（第四节）。

二、骨架六步：教科书初始化

六步长什么样、事件怎么回流，先看图再读代码；顺序铁律就画在实线上。



Wi-Fi 骨架六步与两条事件回流

STA 模式连路由器的标准初始化（六步，每步失败后果列在注释里）：

```
/* ① NVS：Wi-Fi 校准数据存储（不初始化，esp_wifi_init 直接报错） */
esp_err_t ret = nvs_flash_init();
if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
    ESP_ERROR_CHECK(nvs_flash_erase());   /* NAND 布局变了，擦了重来 */
    ret = nvs_flash_init();
}
ESP_ERROR_CHECK(ret);

/* ② netif：创建默认网络接口（STA 模式） */
ESP_ERROR_CHECK(esp_netif_init());              /* 没有 it，esp_wifi 没有接口可绑 */

/* ③ 事件循环：默认 loop（handler 跑在这） */
ESP_ERROR_CHECK(esp_event_loop_create_default());
esp_netif_create_default_wifi_sta();            /* STA 接口实例 */
wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();

/* ④ 注册回调：在 wifi_init 之前注册，防止漏掉 init/start 事件 */
ESP_ERROR_CHECK(esp_event_handler_register(WIFI_EVENT, ESP_EVENT_ANY_ID, &wifi_handler, NULL));
ESP_ERROR_CHECK(esp_event_handler_register(IP_EVENT, IP_EVENT_STA_GOT_IP, &ip_handler, NULL));

/* ⑤ Wi-Fi 初始化 + 配置 STA */
ESP_ERROR_CHECK(esp_wifi_init(&cfg));
wifi_config_t wifi_cfg = { .sta = { .ssid = CONFIG_WIFI_SSID, .password = CONFIG_WIFI_PASSWORD } };
ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &wifi_cfg));

/* ⑥ 启动 + 连接 */
ESP_ERROR_CHECK(esp_wifi_start());   /* 触发 WIFI_EVENT_STA_START，handler 里再 connect */
/* connect 不在这调——等 STA_START 事件再调，防止"start 还没好就 connect" */
```

六步的顺序铁律：NVS → netif → event loop → 注册回调 → wifi_init → start。调换任一步，前面那步的依赖没就位：比如 event loop 没建就注册回调，注册直接报错。

SSID/password 用 `CONFIG_WIFI_SSID`（Kconfig 宏）而非硬编码。骨架坑列表的"凭据泄漏事故"靠 menuconfig 注入（P3 的 Kconfig 自定义项）。

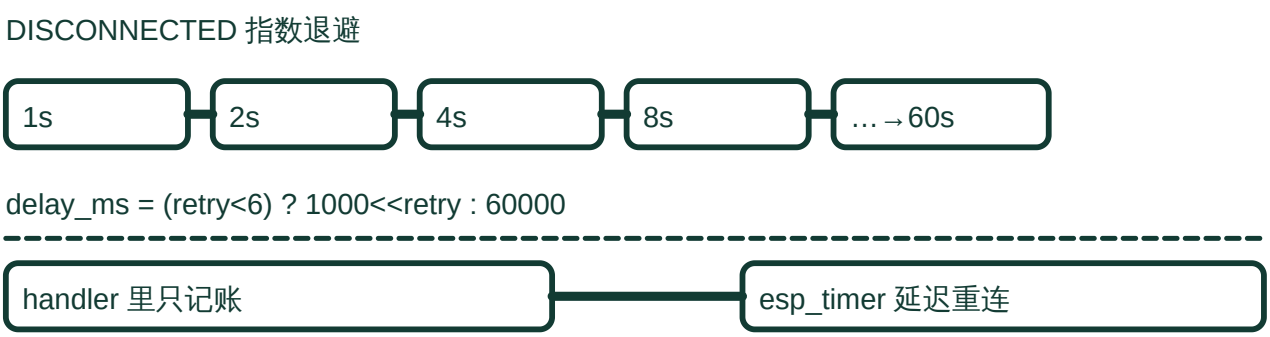
三、事件全表：WIFI_EVENT / IP_EVENT 速查

连接链路上的关键事件（按时间顺序）：

事件	触发时机	典型处理
`WIFI_EVENT_STA_START`	esp_wifi_start 完成	调 esp_wifi_connect（启动连接）
`WIFI_EVENT_STA_CONNECTED`	关联路由器成功	等 DHCP（不直接调 socket）
`WIFI_EVENT_STA_DISCONNECTED`	关联失败/断线	reason code 判断 + 重连策略
`IP_EVENT_STA_GOT_IP`	DHCP 拿到 IP	网络可用，可以 socket
`IP_EVENT_STA_LOST_IP`	IP 丢失	停止网络业务

最容易漏的：`WIFI_EVENT_STA_DISCONNECTED` 在**关联失败、认证失败、路由器重启、信号弱**时都会触发：reason code 区分原因（如 201=未找到 AP、15=认证超时、4=关联被拒）。新手只处理 GOT_IP 不处理 DISCONNECTED，路由器一重启设备就永远离线，因为 `esp_wifi_connect` 失败后不会自动重试。

四、重连策略：DISCONNECTED 的退避算法



Wi-Fi 重连退避：1s→2s→…→60s 封顶

handler 里的重连必须**外置**到定时器或任务，不能 `vTaskDelay` 在 handler 里等（第一节纪律）。指数退避标准写法：

```
static int retry_count = 0;
static esp_timer_handle_t reconnect_timer;

static void reconnect_now(void *arg) {
    esp_wifi_connect();   /* 定时器回调里调，不在事件 handler 里 */
}

static void wifi_handler(void *arg, esp_event_base_t base, int32_t id, void *data) {
    if (id == WIFI_EVENT_STA_START) {
        esp_wifi_connect();   /* 首次连接 */
    } else if (id == WIFI_EVENT_STA_DISCONNECTED) {
        wifi_event_sta_disconnected_t *e = (wifi_event_sta_disconnected_t *)data;
        ESP_LOGW(TAG, "disconnected, reason=%d", e->reason);
        /* 指数退避：1s/2s/4s/.../60s 上限 */
        int delay_ms = retry_count < 6 ? (1000 << retry_count) : 60000;
        retry_count++;
        esp_timer_start_once(reconnect_timer, delay_ms * 1000);  /* 外置定时器 */
    }
}
```

退避算法：1s→2s→4s→8s→16s→32s→60s（上限），路由器重启期间不会狂重连把 AP 打爆。`reconnect_timer` 用 `esp_timer`（IDF 软件定时器），回调在 esp_timer 任务上下文执行，不堵事件任务。

五、NVS 与校准：Wi-Fi 为什么需要 NVS

`nvs_flash_init` 是骨架第一步，不只是存 Wi-Fi 凭据：**射频校准数据**也存 NVS。每颗 ESP32 的射频特性略有差异，出厂时校准数据写进 NVS，Wi-Fi 协议栈启动时读取校准数据补偿射频；没 NVS，Wi-Fi 初始化直接报错（新手第一条报错信息）。

NVS 满了或版本不匹配的处理（骨架第 ① 步）：`ESP_ERR_NVS_NO_FREE_PAGES`（页满了）/ `ESP_ERR_NVS_NEW_VERSION_FOUND`（NAND 布局变了，可能是别的固件写的）→ 擦了重来 `nvs_flash_erase()`。擦 NVS 会丢校准数据和凭据，但校准数据会重新生成（首次连时重新校准）。这是 NVS 的"自愈"机制，对照 S13 手写 Flash 的"半擦"防御，NVS 把这套防御做进了库。

六、状态机对照：日志级别调高看协议栈内部

`idf.py menuconfig` → Component config → Wi-Fi → Log verbosity 调到 Debug/Verbose，连一次路由器看日志：

```
wifi:state: init                                  ← ⑥ start 之前
wifi:state: sta_start                             ← STA_START 事件
wifi:scan:start                                   ← 关联前扫描
wifi:state: associate                             ← AUTH 阶段
wifi:state: associated                            ← STA_CONNECTED
wifi:state: got ip:192.168.x.x                    ← GOT_IP（网络可用）
```

每行对应状态机一个状态。把日志与第三节的"事件全表"逐行对号入座，Wi-Fi 从"玄学调用"变成"可推理的状态机"。断线时日志会打印 `reason=` 加 reason code，与第四节退避算法的 `e->reason` 对上，这就是分诊的入口。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **Wi-Fi 是状态机，事件循环是播报员；GOT_IP 才算联网，DISCONNECTED 必须安排后路。**

延伸：TCP 握手动画见 P8 动画；Wi-Fi 与低功耗配合见 P11 低功耗；ESP-NOW 无网直连见 P9。

实物实验

- 重连实测：连上后重启路由器，日志记录断连检测耗时与重连成功耗时，"可用性"第一次被量化。

常见坑

- **handler 里阻塞重连**：事件任务被堵，后续事件全丢，重连用任务/定时器外置。
- **忘 nvs_flash_init**：Wi-Fi 校准数据无处安放，初始化直接报错（新手第一条报错信息）。
- **ssid/password 硬编码进仓库**：凭据泄漏事故，menuconfig/构建注入（P3 的 Kconfig 自定义）。
- **多网卡混淆**：STA+AP 双模时 netif 选错，esp_netif 句柄按模式分清。
- **connect 在 start 后立即调**：start 还没完成，connect 失败，等 STA_START 事件再调。

短自测

1. esp_event 的"三件套"是什么？handler 为什么不能阻塞？

> **▼ 参考答案**
> 三件套：事件循环任务（默认 loop，跑在独立 FreeRTOS 任务）、事件基（WIFI_EVENT/IP_EVENT 等一组相关事件）、注册回调（指定基+事件+handler）。handler 跑在事件任务上下文，这个任务是单线程的，一个 handler 阻塞（vTaskDelay/等信号量），后续所有事件（别的 handler）全堵在队列里：Wi-Fi 断线的重连 handler 睡 1 秒，这 1 秒里 IP/BLE 事件全丢。长操作必须外置到独立任务或定时器。

2. 骨架六步的顺序为什么是 NVS→netif→event loop→注册回调→wifi_init→start？

> **▼ 参考答案**
> 每步依赖前面那步：netif 要 event loop（绑接口需要 loop）；event loop 要在注册回调之前建（否则注册报错）；注册回调要在 wifi_init 之前（防止漏掉 init/start 事件）；wifi_init 要 NVS（校准数据）；start 要 wifi_init 完成。顺序错了，前面那步的依赖没就位，直接报错或漏事件。

3. `WIFI_EVENT_STA_DISCONNECTED` 不处理会怎样？reason code 干什么用？

> **▼ 参考答案**
> 不处理则路由器重启/信号弱/关联失败后设备永远离线，esp_wifi_connect 失败后不自动重试。reason code 区分断线原因（201=未找到 AP、15=认证超时、4=关联被拒等），分诊用：找不到 AP 可能是 SSID 错或 AP 没开，认证超时可能是密码错或信号弱，关联被拒可能是 AP 黑名单。

4. 指数退避重连为什么上限设 60s？为什么不直接 1s 固定重连？

▼ 参考答案
1s 固定重连在 AP 长时间宕机时会狂打 AP（路由器重启可能要 30s+），且耗电（射频反复唤醒）。指数退避 1→2→4→8→16→32→60 让重连间隔越来越长，AP 恢复后第一次成功重连就复位 retry_count 回到 1s。上限 60s 是"用户可接受的检测延迟"与"省电省 AP"的平衡：产品可调，但别无限退（永远不重连）也别太短（狂打 AP）。

5. 为什么 `esp_wifi_connect` 要在 `WIFI_EVENT_STA_START` 的 handler 里调，而不是 start 之后立即调？

▼ 参考答案
esp_wifi_start 是异步的：它启动 Wi-Fi 协议栈后立即返回，但协议栈真正就绪要等 STA_START 事件。start 后立即 connect，协议栈还没就绪，connect 失败或静默无效。在 STA_START handler 里调，保证协议栈就绪后再发起连接，这是事件驱动架构的纪律：每步等事件确认而非等固定延时。

对照表：本章概念 → 仓库与上游落点

概念	落点
esp_event 三件套	IDF `esp_event.h`；与 P5 UART 驱动 事件模型同思想
骨架六步初始化	IDF Wi-Fi example `wifi/getting_started/station`；本节第二节
事件全表 + reason code	IDF `esp_wifi_types.h` WIFI_EVENT_*；ESP-IF WiFi reason codes
指数退避重连	本节第四节；esp_timer 外置
NVS 射频校准	P10 NVS 机制；nvs_flash_init 第五节
TCP 三次握手	动画 tcp-handshake.svg + 配套视频

你做到了

- 联网从"玄学调用"变成"状态机+事件"的可推理系统；
- 断线重连这个 99% 教程跳过的生产问题，你有了解法。

大站点完名：从 init 到拿到 IP 一共六站，日志里一站都不缺席。断线那一站最考验人：默认没人替你重连，路由器一重启，设备就安静掉线。

✅ 下一站：P9 蓝牙与 ESP-NOW：无路由直连的另一种可能。

通往单片机之路第 64 章

P9 蓝牙与 ESP-NOW：协议栈的另一种打开方式

难度 ★★☆☆☆ · 约 40 分钟

本章罗盘

问题

两块板不接路由器怎么隔空通信？BLE 的 GATT 和 ESP-NOW 各适合什么场景？

交付

一套双板『按键→灯』ESP-NOW 互传工程（init→add_peer→send 三步、send_cb 回执失败计数当链路质量计），加 NimBLE/Bluedroid 取舍表和 GATT 服务-特征-CCC 订阅模型图。

边界

S3 是单模 BLE、无经典蓝牙，Bluedroid 的功能全优势在这用不上；ESP-NOW 定位小数据快通道，单包 250 字节（v1.0）之外的分片重组、大文件传输要应用层自己造；双板互传与边走边测丢包率需两块板，属实物实验。

依据锚点 a9 · f2 · mcu-lab · mcu-anim

> 🎯 Wi-Fi 要路由器，蓝牙要配对——而 ESP-NOW 说：两块板子，加电就通。乐鑫自研的这个"轻量直连协议"是遥控器、传感器组网的秘密武器；蓝牙（BLE）则是手机互联的正道。本章把两条路都铺出来。

本章精髓

1. 协议栈是"预编译巨人"：Wi-Fi/BT 协议栈以库形式链接，经 **hci/phy** 层与射频硬件对话：你调的是 API，跑的是几 MB 的固件级状态机（这是 ESP 与纯 MCU 的本质差异）。
2. BLE 的主角是 GATT：服务（Service）→ 特征（Characteristic，读/写/通知三种属性）：手机"订阅通知"= 设备主动推送的通道；NimBLE vs Bluedroid 两套主机栈的取舍（体积 vs 功能全）。
3. ESP-NOW 极简模型：无连接、配对靠 MAC、单包 250 字节、动作帧（action frame）传输：**先 esp_now_init 再 add_peer 最后 send**，回调收发送回执；与 Wi-Fi 可共存（同信道协商）。

- **能理解**：为什么 S3 上 NimBLE 比 Bluedroid 更省内存；为什么 ESP-NOW 必须先 add_peer 再 send；为什么连了路由器的板子跑 ESP-NOW 会卡在信道不一致。
- **能用**：两块板完成"按键→灯"隔空互传，看懂 send_cb 的回执计数，能用错误码分诊链路问题。

先修

- P8 Wi-Fi（事件模型复用）、F4 队列（接收回调→任务）。

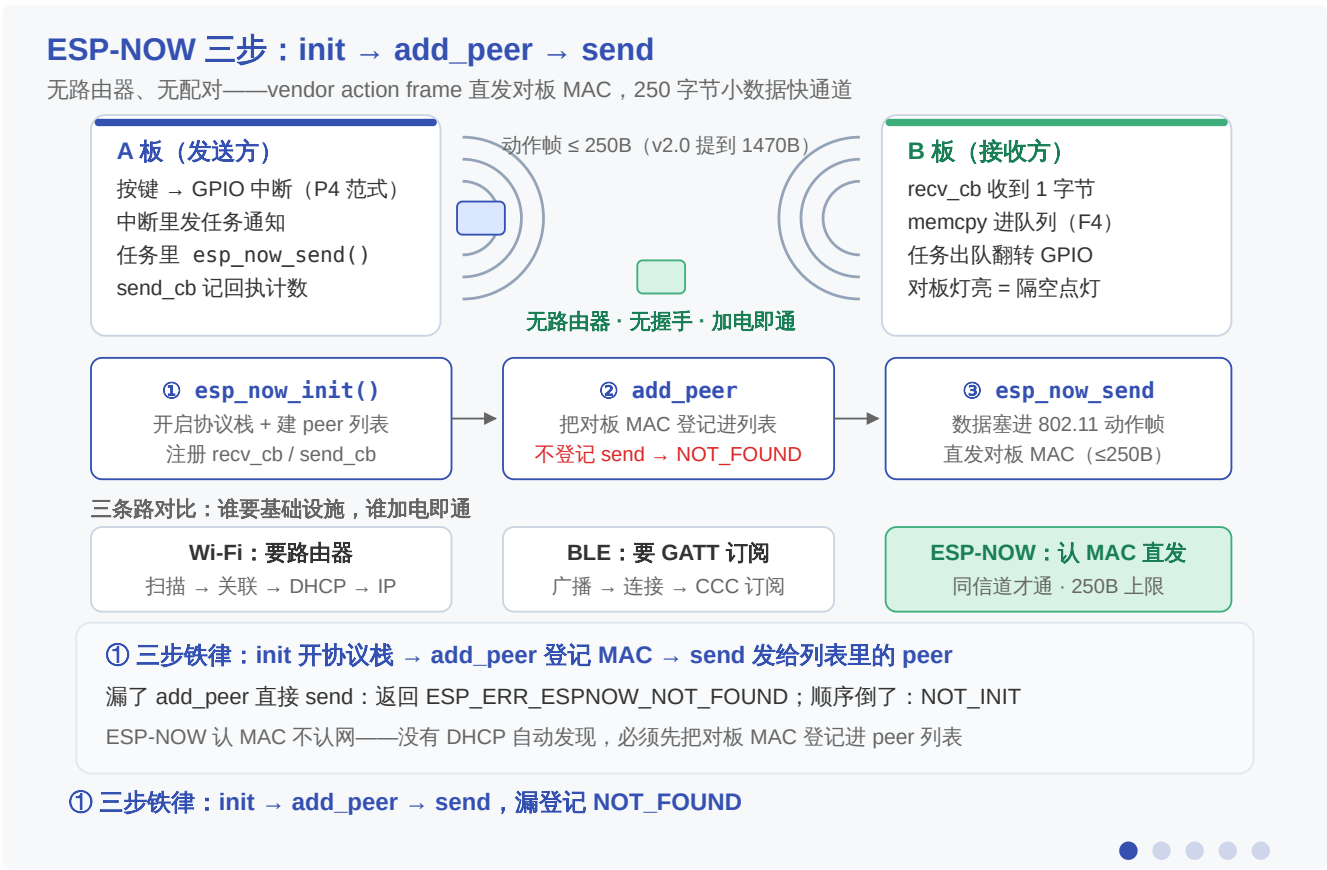
先跑起来（10 分钟 quick win）

两块板各烧本章示例：A 板按用户键，B 板灯翻转，100 行代码内完成"隔空点灯"，成就感拉满。

加电之前先押个没省掉的：免路由直连省掉了路由器和配对，没省掉的是哪一步？押一个再往下读，三步直发里一步都躲不开它。

动画：ESP-NOW 三步直发

init → add_peer → send 的顺序铁律、动作帧在两板之间直发、回执包反向飞回，无路由器无握手的"加电即通"，回执失败计数就是链路质量的第一手感。



ESP-NOW 三步：init → add_peer → send

小节结构

小节	内容	四件套
协议栈架构	控制器/主机/hci 分层；NimBLE vs Bluedroid	库解析
GAP 与广播	广播包结构；手机发现设备的全过程	配置
GATT 模型	服务/特征/描述符；notify 订阅机制	库解析
ESP-NOW 三步	init/peer/send 骨架；发送回调回执	代码分析
双板互传	按键→灯完整链路（E07 姊妹实验）	代码分析
共存与信道	ESP-NOW 与 Wi-Fi 同频共存的规则	配置

一、协议栈架构：控制器、主机与 hci 分层

ESP32-S3 的蓝牙是**单模 BLE**（不支持经典蓝牙 Classic BT），符合 BLE 5.0 规范。和 Wi-Fi 一样，协议栈是个"预编译巨人"，分三层：

- **Controller（控制器）**：跑在射频硬件 + ROM/IRAM 固件里，管链路层（打广播、跳频、加密连接），你看不到源码。
- **Host（主机）**：以库形式链接进你的固件，管上层协议（GAP/GATT/L2CAP），**这一层你有两套选**：
- **NimBLE**（Apache Mynewt 开源）：BLE 专用、轻量，运行时 RAM 占用小，S3 上的首选；
- **Bluedroid**：历史栈，功能全（在支持 Classic BT 的芯片上能跑经典蓝牙），体积大、API 偏底层。 二者在 menuconfig 里互斥：`Component config → Bluetooth → Host → NimBLE` 或 `Bluedroid`（Kconfig 符号 `CONFIG_BT_NIMBLE` vs `CONFIG_BT_BLUEDROID`，同启会报错）。S3 既然没有 Classic BT，选 NimBLE 即可——除非你要复用老 Bluedroid 例程。
- **HCI**：SoC 内部 Controller 与 Host 都在同一颗芯片，没有物理 UART HCI 线，走的是 **VHCI**（virtual HCI，软件回调对接）。所以代码里看不到 `/dev/hci0`，调 `esp_bt_controller_init / enable` 就把控制器拉起来。

板卡与版本前提（本章事实基准）：立创·实战派 ESP32-S3、ESP-IDF **v5.5.2**；ESP-NOW 头在 `components/esp_wifi/include/esp_now.h`，BT 协议栈在 `components/bt/`。和 P8 Wi-Fi 一样，蓝牙初始化也要 `nvs_flash_init`：校准数据与 MAC 都存 NVS（同 P8 第一坑）。

二、GAP 与广播：手机怎么发现你的板子

GAP（Generic Access Profile）管"发现与连接"，是 BLE 的门面。四个角色：broadcaster（只广播）、observer（只扫描）、peripheral（被连接，板子常当这个）、central（发起连接，手机常当这个）。你的板子要被手机看到，就是当 peripheral 广播。

广播包（advertising packet）是设备在三个广播信道上周期性发的"小名片"，legacy 广播包有效负载 31 字节，另可附 31 字节的 scan response（扫描请求回来再补一份）。BLE 5.0 的扩展广播可以更大，但手机端兼容性以 legacy 为准。名片里塞什么：

字段	内容
Flags	LE general discoverable 等模式标志
Complete/Incomplete Service UUID	暴露的服务（让 nRF Connect 这类 app 识别）
Local Name	设备名（如 "My-Bulb"）
Manufacturer Data	厂商自定义（iBeacon 的 UUID 就塞这里）

广播间隔是个 tradeoff：间隔短→被很快发现但费电；间隔长→省电但手机要扫更久才看到。电池设备常 1s 级，插电设备 100ms 级。手机扫到广播→发连接请求→进入 GATT 阶段（下一节）。这一步和 P8 的"AP 扫描→关联"是对偶：Wi-Fi 找路由器，BLE 找手机。

三、GATT 模型：服务、特征与订阅通知

GATT（Generic Attribute Profile）管"连上之后的数据"，规定了一个**层级容器**：

```
Profile
  └─ Service（UUID 标识）
       └─ Characteristic（UUID + 值 + 属性）
            └─ Descriptor（如 CCC，订阅开关）
```

- **Service**：一组相关特征的集合，用 UUID（16 位如 `0x180F` 电池服务，或 128 位自定义）标识。
- **Characteristic**：数据单元，有 value + properties（读 `READ`、写 `WRITE` / `WRITE_NR`、通知 `NOTIFY`、指示 `INDICATE`）。
- **Descriptor**：特征的元数据，最关键的是 **CCC**（Client Characteristic Configuration，UUID `0x2902`）：手机往这个描述符写 `0x0001` 就开了 notify，写 `0x0000` 关掉。

notify 是 BLE 的"主动推送"：设备侧值变了，直接把数据包推给已订阅的手机，手机不用轮询读。这就是本章精髓说的"手机订阅通知=设备主动推送通道"。心率传感器每秒推一次心跳、键盘按下即推键值，都走 notify。

NimBLE 和 Bluedroid 的 GATT API 表面不同（NimBLE 用 `ble_svc_*` 系列声明属性表，Bluedroid 用 `esp_ble_gatts_*` 运行时注册），但**模型完全一样**：服务/特征/描述符三级 + 属性位 + CCC 订阅。学透模型，换栈只是换 API 名。GATT 回调（连接事件、读写请求）也跑在蓝牙任务上下文，**不能阻塞**，长操作入队回任务处理（F4 队列 范式，复用 P5/P8 纪律）。

四、ESP-NOW 三步：init / add_peer / send

ESP-NOW 是乐鑫自研的无连接 Wi-Fi 协议，把应用数据塞进 802.11 的 vendor-specific action frame 直发：**不要路由器、不要握手、加电即通**。最小骨架三步（顺序铁律）：

```
#include "esp_now.h"
#include "esp_wifi.h"

/* ① init：开启 ESP-NOW（依赖 esp_wifi 已 init/start） */
ESP_ERROR_CHECK(esp_now_init());
ESP_ERROR_CHECK(esp_now_register_recv_cb(on_recv));   /* 收 */
ESP_ERROR_CHECK(esp_now_register_send_cb(on_send));   /* 发送回执 */

/* ② add_peer：把对板 MAC 登记进 peer 列表（不登记，send 直接 NOT_FOUND） */
esp_now_peer_info_t peer = {0};
peer.channel = 0;                       /* 0 = 跟随当前 Wi-Fi 信道 */
peer.ifidx   = WIFI_IF_STA;             /* 走 STA 接口 */
peer.encrypt = false;                   /* 不加密；true 时填 lmk[16] */
memcpy(peer.peer_addr, peer_mac, 6);    /* 对板 MAC */
ESP_ERROR_CHECK(esp_now_add_peer(&peer));

/* ③ send：发到指定 peer；peer_addr=NULL 广播给所有 peer */
uint8_t payload = 0x01;
esp_now_send(peer_mac, &payload, sizeof(payload));
```

`esp_now_peer_info_t` 字段（v5.5.2 `esp_now.h` 实测）：`peer_addr[6]`、`lmk[16]`（加密密钥）、`channel`、`ifidx`、`encrypt`、`priv`。两个回调签名也是头里写死的：

```
typedef void (*esp_now_recv_cb_t)(const esp_now_recv_info_t *info,
                                  const uint8_t *data, int data_len);
typedef void (*esp_now_send_cb_t)(const esp_now_send_info_t *tx_info,
                                  esp_now_send_status_t status);  /* SUCCESS / FAIL */
```

`send_cb` 给你**发送回执**，这正是 ESP-NOW 比"发了就忘"的 UDP 强的地方：回执失败计数 = 链路质量的第一手感（实物实验里"边走边看丢包率"就靠它）。`recv_cb` 里的 `esp_now_recv_info_t` 是栈上临时变量（头文件原话："it can only be used in the callback"），回调返回即失效，要存数据就 memcpy 走，别存指针。

单包上限： `ESP_NOW_MAX_DATA_LEN = 250`（v1.0，即动作帧 vendor IE 上限）；v2.0 协议 `ESP_NOW_MAX_DATA_LEN_V2 = 1470`。混网时 v1.0 收 v2.0 长包会截断或丢弃，大文件请自己在应用层分片重组，ESP-NOW 定位是"小数据快通道"不是流。加密 peer 上限 6 个（`ESP_NOW_MAX_ENCRYPT_PEER_NUM`），总 peer 上限 20（`ESP_NOW_MAX_TOTAL_PEER_NUM`）。

五、双板互传：按键→灯的完整链路

两块 S3（或一块 S3 + 一块 ESP32）互传，完整链路是 ESP-NOW 的"hello world"，也是姊妹实验 E07 的内容：

1. 两板各自 `esp_wifi_init` + `esp_now_init`，先 `esp_wifi_get_mac(WIFI_IF_STA, mac)` 打印自身 MAC，**人工把对板 MAC 填进 add_peer**（ESP-NOW 认 MAC 不认网，没有 DHCP 自动发现）。
2. A 板：GPIO 按键中断（P4 范式）→ 中断里发任务通知 → 任务里 `esp_now_send(B_mac, &toggle, 1)`。
3. B 板：`recv_cb` 收到 1 字节 → memcpy 进队列（F4 队列，回调不办事）→ 任务出队翻转 GPIO 灯。
4. A 板 `send_cb` 记 `ESP_NOW_SEND_FAIL` 次数，这就是你"隔空点灯"失败几次的客观度量。

为什么 recv_cb 要入队、不在回调里直接点灯？和 P5 UART 的中断礼仪同构：回调跑在 esp_now 任务上下文（系统任务），直接 `vTaskDelay` 或长操作会堵住后续收包。回调只 memcpy + 入队，处理交给自己的任务，这是全篇反复出现的"记账不办事"纪律。

整套链路 100 行内能写完，成就感拉满，但更重要的是它把"物联网的网"字第一次具象化：你写的不是 `LED=!LED`，而是 `LED_on_board_B = !LED_on_board_B`，跨空间、无线、可观测回执。

六、共存与信道：和 Wi-Fi 同频不打架

ESP-NOW 不另起射频，它复用 Wi-Fi 的 PHY/MAC，所以**和 Wi-Fi 同信道**是它和路由器共存的前提，也是最常见的失败根因。规则：

- **未连 Wi-Fi 时**：ESP-NOW 仍能跑（无连接模式），用默认信道；两块都没连路由器、信道一致即可互通。
- **连了路由器时**：板子的 Wi-Fi 信道被路由器锁死（比如路由器在 6 信道）。这时 `peer.channel = 0` 让对板"跟随当前 STA 信道"，但**对板也得在同一信道**，否则 `esp_now_send` 返回 `ESP_ERR_ESPNOW_CHAN`（头里明确列了这个错误码）。常见坑第二坑就是这个：A 板连了路由器在信道 6，B 板没连、停在信道 1，互相发不出去。
- **STA + ESP-NOW 同启**：照 P8 骨架六步初始化 Wi-Fi，再 `esp_now_init`，两者共用底层。初始化顺序是 wifi_init/start 在前、esp_now_init 在后（ESP-NOW 依赖 Wi-Fi 已 start）。

排查信道不一致： `esp_wifi_get_channel(&primary, &second)` 打印两板当前信道对比；不一致就调整让对板跟随，或干脆两板都不连路由器走默认信道。这套分诊思路和 P8 的 reason code 分诊同构：错误码是入口，不是终点。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **BLE 靠 GATT 立服务，订阅通知主动推；ESP-NOW 认 MAC 不认网，加电即通 250 字节。**

延伸：ESP-NOW 三步动画见 P9 动画；Wi-Fi 联网在 P8 展开；BLE 与 Wi-Fi 功耗对比见 P11 低功耗。

实物实验

- quick win 双板互传 + 测距粗实验：边走边看丢包率（回执失败计数），无线链路质量的第一手感。

常见坑

- **NVS 没初始化**：BT 校准/MAC 存储也要 NVS，同 P8 的第一坑。
- **ESP-NOW 与 Wi-Fi 信道不一致**：连了路由器的板子信道被路由器锁死：peer 板要跟随同信道。
- **GATT 回调里做长操作**：蓝牙任务被堵断连：事件入队回任务处理（P5 范式复用）。
- **250 字节上限忘分包**：大图片/文件传输要自分片重组，ESP-NOW 定位是"小数据快通道"。
- **recv_cb 里存 esp_now_recv_info 指针**：头文件原话该结构是回调内局部变量，返回即失效，要存数据 memcpy 走，别存 `info` 指针跨调用用。

短自测

1. S3 上蓝牙协议栈为什么要选 NimBLE 而不是 Bluedroid？两套栈的 Kconfig 怎么配？

▼ 参考答案
S3 是单模 BLE、没有 Classic BT，Bluedroid"同时支持经典蓝牙"的优势用不上，反而体积大、RAM 占用高。NimBLE 专为 BLE 设计、轻量，是 S3 的首选。menuconfig 走 Component config → Bluetooth → Host，选 NimBLE 或 Bluedroid，对应 Kconfig 符号 CONFIG_BT_NIMBLE / CONFIG_BT_BLUEDROID，二者互斥，同启会报错。要复用老 Bluedroid 例程才切 Bluedroid。

2. BLE 的 GAP 和 GATT 各管什么？手机"订阅通知"具体是往哪个描述符写了什么？

▼ 参考答案
GAP 管"发现与连接"：广播、扫描、建立连接、角色（peripheral/central 等）。GATT 管"连上之后的数据"：Service → Characteristic → Descriptor 三级容器。订阅通知是手机往 Characteristic 的 CCC 描述符（UUID 0x2902）写 0x0001 开 notify（0x0000 关），之后设备值变化就主动推给手机，不用轮询。

3. ESP-NOW 的三步顺序为什么必须是 init→add_peer→send？把 add_peer 漏了会怎样？

▼ 参考答案
esp_now_init 开启协议栈并建好 peer 列表；add_peer 把对板 MAC 登记进列表（带信道/接口/加密参数）；send 只能给列表里的 peer 发。漏了 add_peer 直接 send，返回 ESP_ERR_ESPNOW_NOT_FOUND（peer 不存在）；顺序倒了（send 在 init 前）返回 ESP_ERR_ESPNOW_NOT_INIT。ESP-NOW 认 MAC 不认网，没有 DHCP 自动发现，必须先登记。

4. ESP-NOW 单包上限是多少？v2.0 有什么变化？要传一张 2KB 的图片怎么办？

▼ 参考答案
v1.0 上限 ESP_NOW_MAX_DATA_LEN = 250 字节（vendor action frame IE 上限）；v2.0 提升到 1470 字节。混网时 v1.0 设备收 v2.0 长包会截断或丢弃，所以跨版本别假设对端能收长包。2KB 图片要在应用层自分片（每片不超过 250 字节）+ 序号 + 重组，并处理丢片重传：ESP-NOW 定位是"小数据快通道"，不是文件传输。

5. （开放）A 板连了路由器在信道 6，B 板没连路由器，两板用 ESP-NOW 互发数据失败，从信道角度怎么排查？

▼ 参考答案
根因大概率是信道不一致：A 板被路由器锁在信道 6，B 板没连 Wi-Fi 停在默认信道（常为 1），两边不在同一信道，esp_now_send 返回 ESP_ERR_ESPNOW_CHAN。排查：用 esp_wifi_get_channel 打印两板当前 primary 信道对比。解法三选一：① 把 peer.channel 设为对板实际信道；② 两板都不连路由器，走同一默认信道；③ 让 B 板也连同一路由器，自然同信道。和 P8 的 reason code 分诊同构：错误码是入口不是终点。

对照表：本章概念 → 仓库与上游落点

概念	落点
协议栈三层 + NimBLE/Bluedroid	ESP-IDF v5.5.2 `components/bt/`；Kconfig `CONFIG_BT_NIMBLE` / `CONFIG_BT_BLUEDROID`
GAP 广播 / GATT 服务模型	Bluetooth Core Spec 5.0；IDF `bt_host` 层（NimBLE `ble_svc_*` / Bluedroid `esp_ble_gatts_*`）
ESP-NOW 全套 API + 错误码	`components/esp_wifi/include/esp_now.h`（v5.5.2 实测）
单包上限 250 / 1470	同上 `ESP_NOW_MAX_DATA_LEN` / `ESP_NOW_MAX_DATA_LEN_V2`
peer 结构与加密	`esp_now_peer_info_t`（lmk[16]/channel/ifidx/encrypt）；加密 peer 上限 6、总 peer 上限 20
双板互传姊妹实验	E07（本章实物实验）；IDF `examples/wifi/esp_now/`
共存信道与 ESP_ERR_ESPNOW_CHAN	`esp_now.h` 错误码；与 P8 信道锁定对偶
回调入队范式	F4 队列、P5 UART 中断礼仪

你做到了

- 无线世界的两条主干道都走过：正规军（Wi-Fi/BLE）与轻骑兵（ESP-NOW）；
- 双板实验成就感达成，物联网的"网"字第一次在你手里具象化。

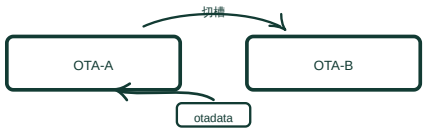
没省掉的是 add_peer：直发不等于盲发，先把对端的 MAC 登记在册，再开口说话。send 之后也别急着走，回执是链路质量的第一手感，丢包多少它都记着。

✅ 下一站：P10 Flash/分区/NVS/OTA，16MB 的版图管理与永不翻车的升级。

通往单片机之路第 65 章

P10 Flash、分区表、NVS 与 OTA

难度 ★★★☆☆ · 约 45 分钟



本章罗盘

问题

16MB Flash 谁来分地？NVS 写一半断电还作数吗？OTA 为什么必须两个等大的 app 槽？

交付

一张 16MB 双 ota 槽分区表 CSV（app offset 0x10000 对齐）、一个断电五次计数 1→5 的 boot_count 持久化最小例、一次完整本地 OTA 演练日志（写另一槽→切槽→自检转正→故意不转正看 PENDING_VERIFY 自动回滚）。

边界

回滚状态机只在 CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE 开启后存在（默认未启用），不开时坏固件照样坐庄、不会自动回滚，不能把默认行为套进回滚流程；立创板默认表多为 factory+大存储、无 ota 槽，玩 OTA 先换表；Flash 加密/安全启动只画概念地图不实操（eFuse 不可逆）。

依据锚点 `a9 · f2 · d9 · mcu-anim`

> 🎯 STM32 的 Flash 布局是你链接脚本里画的；ESP32 的 16MB Flash 归一张 **CSV 分区表** 管：哪块放 bootloader、哪块放 app、哪块存参数。而 OTA 的全部秘密，就是"两个 app 分区轮流坐庄 + 一个 otadata 记票"。

本章精髓

1. 分区表 CSV 逐字段 name/type/subtype/offset/size/flags：`factory`（出厂 app）、`ota_0/ota_1`（升级候选）、`nvs`（键值存储）、`phy_init`（射频校准）、`spiffs/fatfs`（文件系统）。16MB 的每一 KB 都有户口。
2. NVS 是"磨损均衡的键值库"：nvs_set/get 背后是按页管理+CRC 校验+掉电安全的迷你文件系统。Wi-Fi 凭据、设备配置、计数器的标准住所（别再自己写 Flash 扇区了，对照 S13 的手工方案）。
3. OTA 双槽机制：新固件写进另一个 ota 分区→校验→otadata 指向新槽→重启。**新固件自检通过后 esp_ota_mark_app_valid_cancel_rollback**，这一步只在启用 CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE 时需要；否则下次启动才会回滚旧槽。"防变砖"不是口号，是这套状态机。

回滚选项默认未启用；先配置 bootloader，再验证 NEW → PENDING_VERIFY → VALID/ABORTED 状态。不能将不开回滚时的行为套入此流程。

- **能理解**：为什么 app 分区 offset 要 0x10000 对齐而 size 只需 4KB 对齐；为什么"追加写"一个动作同时买到磨损均衡、掉电安全、页满整理；回滚选项为什么默认不开：不开时 NEW/PENDING_VERIFY 状态根本不会被使用，不能把默认行为套进回滚流程。
- **能用**：改 partitions.csv 并用 `idf.py partition-table` 验证；写出 boot_count 掉电持久化最小例；走通"写入另一槽 → set_boot_partition → 重启 → 自检 → mark valid"全流程，并亲手演示一次 PENDING_VERIFY 自动回滚。

先修

- P1 启动（分区表登场）、S13 Flash（手工方案对照）。

先跑起来（10 分钟 quick win）

hello 工程加 NVS：每次启动 `boot_count++` 存回，断电重启后打印：你的第一笔"持久化资产"。

跑之前先押一笔：拔电再上电，boot_count 是从零重数，还记得上次的数？押完就拔，这是全书少有的敢拿断电当判分的实验。

动画：OTA 双槽轮流坐庄

ota_0/ota_1 两个槽轮流上岗、otadata 记票；升级期间旧固件一直在岗，新固件先过"试用期"，自检通过转正，失败自动回滚。双槽的保险机制一图演完。

OTA 双槽轮流坐庄：下载 → 切槽 → 自检 → 回滚

板卡事实

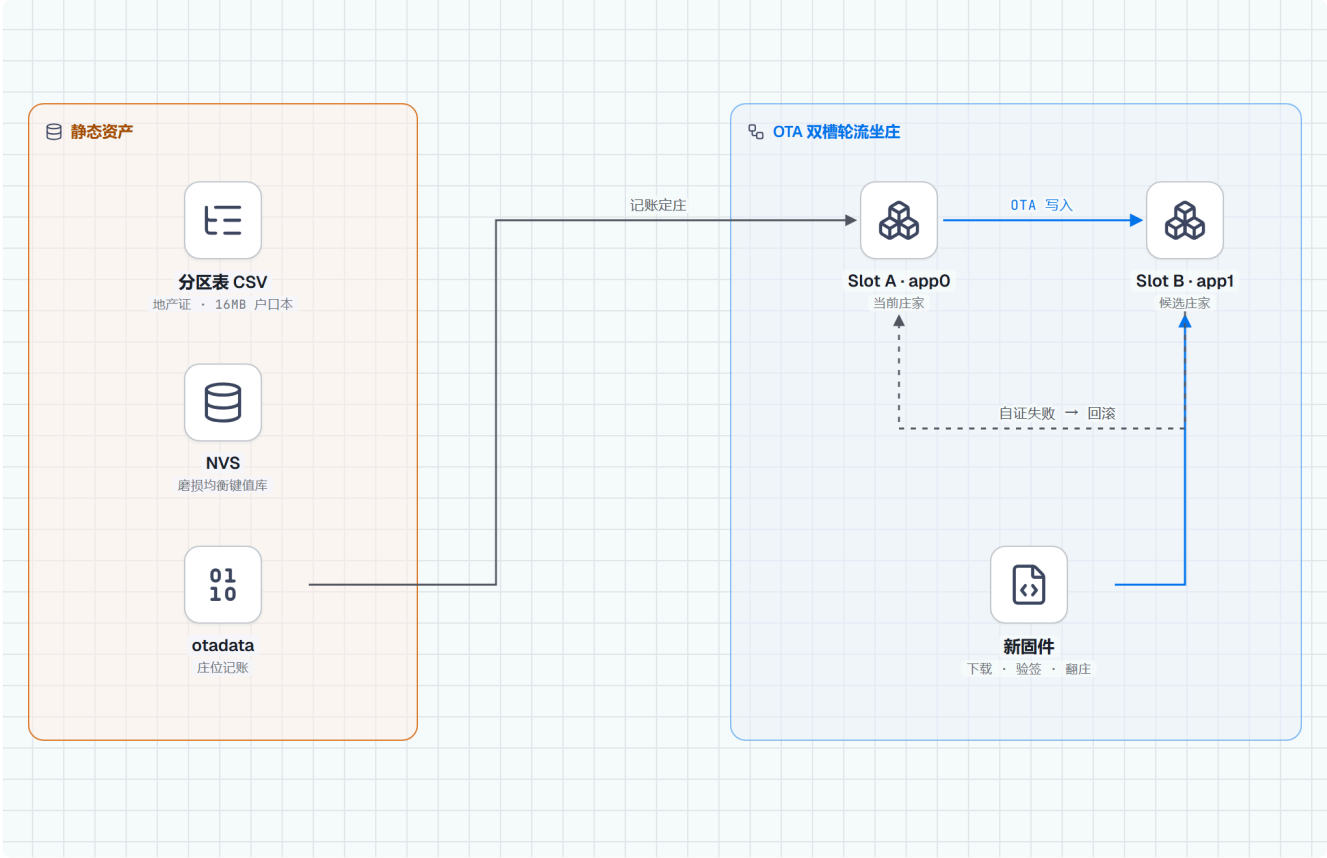
- 立创实战派 S3：16MB Flash（N16R8 模组），默认分区表多为 factory + 大容量存储区布局，以 `idf.py partition-table` 实际读出为准。

小节结构

小节	内容	四件套
CSV 逐字段	默认表逐行；offset 对齐与 size 约束	配置
NVS 机制	页/条目/CRC；命名空间与类型 API	库解析
boot_count 实战	掉电持久化最小例	代码分析
OTA 状态机	双槽+otadata+valid/pending 状态全图	库解析
本地 OTA 演练	版本号/切槽/回滚三步实操	代码分析
安全选项	Flash 加密与安全启动的概念地图（生产话题）	配置

一、CSV 逐字段：16MB 的户口本

版图先看图：左边静态资产，右边双槽轮流坐庄。



分区表、NVS 与 OTA 双槽：记账定庄、下载写入、自证失败回滚

对照 S13：STM32 的 Flash 布局画在链接脚本里，是"编译期契约"；ESP32 反过来：布局写在一张 CSV 里，bootloader 每次启动读它（P1），换布局不动链接脚本，换张表就行。

16MB 的头部有三块"公摊面积"，不归 CSV 管：

- 0x0 起：第二级 bootloader 的地皮（S3 的 bootloader 基址就是 0x0）；
- 0x8000：分区表本身（默认 CONFIG_PARTITION_TABLE_OFFSET）。表长 0xC00、最多 95 条，结尾跟一个 MD5 完整性校验，正好占满一个 0x1000 扇区；
- 所以 CSV 里第一个分区必须从 0x9000 或更靠后起步。

一张适配 16MB 的教学示例表（数值按项目需要调整；立创实战派 S3 的实际默认表以 `idf.py partition-table` 读出为准）：

```
# Name,   Type, SubType,  Offset,  Size,     Flags
nvs,      data, nvs,      0x9000,  0x5000,
otadata,  data, ota,      0xE000,  0x2000,
phy_init, data, phy,      0x10000, 0x1000,
ota_0,    app,  ota_0,    0x20000, 0x300000,
ota_1,    app,  ota_1,    0x320000,0x300000,
spiffs,   data, spiffs,   0x620000,0x9E0000,
```

16MB = 0x1000000：两个 3MB 的 ota 槽 + 约 9.8MB 文件系统，首尾相接正好铺满。立创实战派的默认布局多为 factory + 大存储区、往往没有 ota 槽，要玩 OTA，第一件事就是换一张这样的双槽表。

逐字段（对齐约束是工具链硬性检查，写错构建期直接报错）：

字段	作用	约束/事实
name	分区名，esp_partition_find_first 按名查	最长 16 字节（含结尾空字符，即 15 字符）
type	app / data 两大类（0x00/0x01，0x40 起留自定义）	bootloader 只认 app 和 data
subtype	app：factory/ota_0~ota_15/test；data：ota/nvs/phy/spiffs/fat/nvs_keys/coredump	决定库把这块地当什么用
offset	起始地址	可留空自动分配；所有分区至少 0x1000（一个扇区）对齐；app 分区必须 0x10000 对齐
size	大小	支持 K/M 后缀；app 分区大小须 4KB 对齐
flags	encrypted / readonly	app、bootloader、分区表、otadata、nvs_keys 五类无条件加密，与 flags 无关

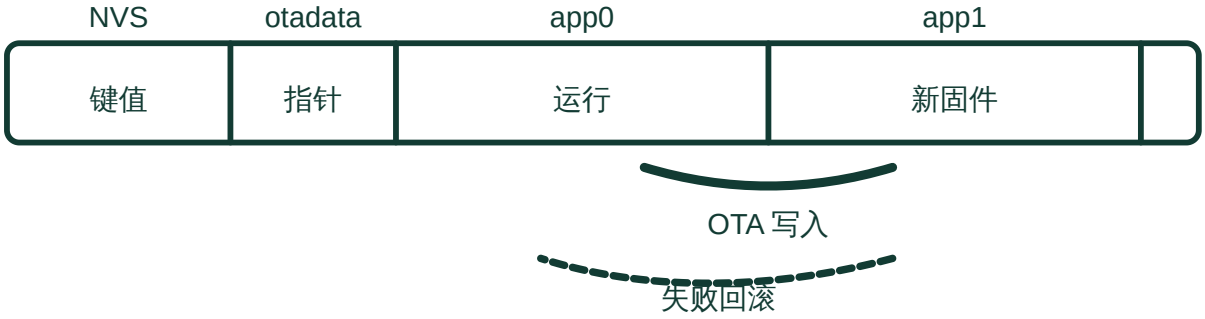
subtype 户口本（本章主角全在这）：

- **factory**：出厂 app 槽，otadata 无效时 bootloader 的兜底（第四节）；不参与 anti-rollback（第六节）。
- **ota_0 / ota_1**：升级双槽，轮流坐庄，OTA 的前提配置。
- **ota（data 子类型，名字常叫 otadata）**：记票板，存"下次启动谁坐庄"。
- **nvs**：键值库（第二节）。官方建议至少 0x3000，至少三页（含一个空页）才能边写边整理；0x1000/0x2000 的小分区会被库按只读对待。
- **phy**：射频初始化数据（名字常写 phy_init）；按设备校准的射频数据另住 NVS（P8 第五节）。
- **spiffs / fat**：文件系统分区，网页、语音、配置文件的家（大文件别塞 NVS，塞这）。

三条工具链事实：

1. 启用自定义表：menuconfig → Partition Table 选 "Custom partition table CSV"（CONFIG_PARTITION_TABLE_CUSTOM_FILENAME，默认文件名 partitions.csv）。
2. 构建时 gen_esp32part.py 把 CSV 编译成二进制烧进 0x8000；`idf.py partition-table` 打印解析结果，改完表先跑它，对齐、越界当场报错，不用等烧录翻车。
3. 内置的 "Factory app, two OTA definitions" 是 4MB 视角的参考布局（factory/ota_0/ota_1 各 1M），16MB 板照着扩容即可。

实践铁律：**两个 ota 槽等大**。轮流坐庄的两个庄家地位必须对称。不等大不会立刻报错，但构建系统会在"固件只装得进部分 app 槽"时警告（Partition Size Checks），OTA 轮转的隐患从第一天就埋下。



ESP32 Flash 分区与 OTA：otadata 指向当前 app 槽，新固件写入另一槽；失败可回滚

二、NVS 机制：磨损均衡的键值库

在 STM32 上手写 Flash 存储（S13），三件事全得自己扛：擦除粒度、掉电半写、寿命摊薄。ESP-IDF 把它们打包成了 NVS（Non-Volatile Storage），**带磨损均衡的键值库**，Wi-Fi 凭据、设备配置、计数器的标准住所。官方定位说得很直白：为"大量小值"设计，别拿它塞大 blob（大文件走文件系统分区）。

三层结构，官方 Internals 章节给了精确解剖图：

- 分区（subtype nvs）按 4096 字节一页切，一页正好一个物理扇区；
- 页 = 页头 32 字节（状态 4 + 序号 4 + 版本 1 + 保留 19 + CRC32 4）+ 条目状态位图 32 字节 + 126 个条目；
- 条目 32 字节：命名空间索引 + 类型 + 跨度 + CRC32 + key（最长 15 字符）+ 数据（整型 8 字节；字符串/blob 可跨多个条目）。

核心机制是**追加写（append-only）**。官方 Internals 原话：键值对顺序存放、新对加在日志尾部；**更新一个 key 不是改旧条目，而是尾部追加新条目、把旧条目标记为 erased**。一个设计买到三样东西：

1. **磨损均衡**：每次写都落在不同位置，擦写压力摊到整个分区（官方量化说法：页+条目组织让擦除频率相对写入频率降低 126 倍）。对照 S13 的单扇区方案：同一个地址反复擦写，寿命按那个扇区单点烧。
2. **掉电安全**：条目和页头都带 CRC32。写一半断电，半条目校验不过，被当作不存在，读到的是上一个完整条目。官方承诺：任意时刻断电，除"正在写的那个键值对"外不丢数据；Flash 里是随机数据也能正常初始化。
3. **页满整理**：页状态机 Active（唯一活动页，只往这写）→ Full（写满，只能标记删除）→ Erasing（整理中：把页里仍然有效的最新条目搬去别的页，再整页擦除回收）。整理全自动，断电打断也会在上电后接着完成。

页状态机还有 Corrupted（页头 CRC 不过、整页弃用），这是库的"自愈"边界：坏一页不至于起不来。

命名空间与类型 API：

```
nvs_handle_t h;
nvs_open("counter", NVS_READWRITE, &h);  /* 命名空间=姓、key=名；同名 key 在不同命名空间互不打架（上限 254 个） */
nvs_set_i32(h, "boot_count", 42);        /* i8/i16/i32/i64、u8/u16/u32/u64、str、blob 全套 */
char buf[32]; size_t len = sizeof(buf);
nvs_get_str(h, "ssid", buf, &len);       /* str/blob：带缓冲直接读；不知道长度先传 NULL 问大小再二次取值 */
nvs_erase_key(h, "ssid");
nvs_commit(h);                           /* 官方文档原话：nvs_commit 之前不真正落盘 */
nvs_close(h);
```

三条纪律：

- **commit 是句号**：`nvs_set_*` 的 API 文档明说"实际存储直到 nvs_commit 才更新"，每批写入后必调；断电前没 commit 的写全部不算数。
- **读不存在的 key 不是错误**：返回 ESP_ERR_NVS_NOT_FOUND，调用方据此区分"第一次"（第三节正这么用）。类型对不上才是错误（存的是 str、按 i32 读，直接报错）。
- **初始化要接住错误码**：`ESP_ERR_NVS_NO_FREE_PAGES`（没有空页，常见于分区被改小）或 `ESP_ERR_NVS_NEW_VERSION_FOUND`（NVS 格式版本变了，多半是别的固件写的）→ `nvs_flash_erase()` 擦了重新 init。会丢凭据和校准数据，但校准数据下次校准会重新生成（P8 第五节）。

（进阶储备：NVS 在 RAM 里维护查找缓存，开销随分区和键数增长：官方口径约 1MB 分区 22KB RAM、每 1000 个键再加 5.5KB；带 PSRAM 的项目可开 CONFIG_NVS_ALLOCATE_CACHE_IN_SPIRAM 把缓存挪出内部 RAM。）

三、boot_count 实战：第一笔持久化资产

IDF 官方 examples/storage/nvs_rw_value 就是一个"重启计数器"（键名 restart_cnt）。我们把它还原成最小形态：

```
#include <stdio.h>
#include "nvs.h"
#include "nvs_flash.h"

void app_main(void)
{
    /* ① 初始化：接住两个错误码（第二节纪律 3） */
    esp_err_t err = nvs_flash_init();
    if (err == ESP_ERR_NVS_NO_FREE_PAGES || err == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        ESP_ERROR_CHECK(nvs_flash_erase());
        err = nvs_flash_init();
    }
    ESP_ERROR_CHECK(err);

    /* ② 打开命名空间 */
    nvs_handle_t h;
    ESP_ERROR_CHECK(nvs_open("counter", NVS_READWRITE, &h));

    /* ③ 读旧值：首次没有这个 key，返回 NOT_FOUND，count 保持 0 */
    int32_t count = 0;
    (void)nvs_get_i32(h, "boot_count", &count);

    /* ④ 加一写回 + commit 收尾 */
    count++;
    ESP_ERROR_CHECK(nvs_set_i32(h, "boot_count", count));
    ESP_ERROR_CHECK(nvs_commit(h));
    printf("boot count: %d\n", (int)count);

    nvs_close(h);
}
```

烧录后连续断电重启五次，monitor 里看到 1→2→3→4→5。三个关键点值得逐行咂摸：

- **首次启动**：第 ③ 步 NOT_FOUND，count 保持初值 0，打印 1："读不到旧值"被翻译成"第一次"，不需要额外的标志位。
- **重启不等于丢数据**：断电、按 EN 复位、monitor 里 Ctrl-T Ctrl-R 热复位，三种方式对 NVS 毫无区别——重启后读到的都是上次 commit 落盘的那个值。掉电安全不靠运气，靠的是第二节"半条目 CRC 不过被当不存在"的机制。
- **第 ④ 步在 Flash 里发生了什么**：set 没有碰旧条目，而是在活动页追加了一个新条目（同 key 以最新为准）；重启几十次后活动页写满，库自动做 Erasing 整理（页满后的自动整理行为），你一行代码都不用写。

举一反三：把"每次启动 +1"换成"上次连的 Wi-Fi""累计运行秒数""报警次数"：所有"断电要记住的事"都是这一个模式的变体。

泼一瓢冷水（呼应常见坑 2）：boot_count 的写入频率是"每次启动一次"，NVS 毫无压力；把同样的代码搬进主循环每秒 +1，Flash 寿命就开始燃烧：高频数据先进 RAM，定期落盘。

四、OTA 状态机：双槽轮流坐庄

为什么必须两个 app 槽？单 app 分区没法自我升级，总不能把正在运行的那块地当场铲掉。双槽的本质：**运行的槽和接收新固件的槽永远不是同一个**。

- **ota_0 / ota_1**：两个等大的 app 分区，一个坐庄（运行中），一个备选（收新固件）。
- **factory**（可选）：出厂槽。
- **otadata**（data/ota，0x2000）：记票板。注意它的掉电安全设计：官方文档明说这分区是"两个扇区"，各写一份同样的票、带计数器轮流写，断电后两份不一致就按计数器裁决谁更新。给记票板自己做双备份，比给固件买保险还讲究。

启动决策（bootloader 每次上电重演一遍）：

1. 读 otadata：票指向 ota_X 就启动 ota_X；
2. 记票板全空（全 0xFF，从未 OTA 过）→ 启动 factory；表里没有 factory → 启动第一个 ota 槽（通常 ota_0）。

写入流程的 API 五步（数据来源随意：HTTP 下载、预烧分区、SD 卡，都汇入同一条管道）：

```
const esp_partition_t *next = esp_ota_get_next_update_partition(NULL); /* 自动轮转挑"另一个槽" */
esp_ota_handle_t ota;
esp_ota_begin(next, OTA_SIZE_UNKNOWN, &ota);  /* 大小未知则先整槽擦除 */
esp_ota_write(ota, chunk, len);               /* 循环分块写 */
esp_ota_end(ota);                              /* 校验闸：镜像不合法在此拦下 */
esp_ota_set_boot_partition(next);              /* 改票：下次启动坐庄的是新槽 */
esp_restart();
```

两处设计值得品味：

- `esp_ota_get_next_update_partition()` 从当前运行槽出发轮转找下一个，你不用算"现在 0 号所以写 1 号"。
- `esp_ota_end()` 是校验闸门：刚写完的镜像不是合法 app（或开了安全启动后签名不过），直接返回 ESP_ERR_OTA_VALIDATE_FAILED。坏 bin 在这被拦下，而不是变砖后才发现。

然后是本章最重要的一段事实：上面五步只是"换庄"。防变砖的"自证清白"状态机，**只在 CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE 打开后才存在（默认未启用）**。官方文档的 App OTA State 全图：

状态	bootloader 会选它吗	谁设置
ESP_OTA_IMG_VALID	正常选	esp_ota_mark_app_valid_cancel_rollback()：自检通过、转正
ESP_OTA_IMG_UNDEFINED	正常选	esp_ota_set_boot_partition()，**未开回滚时**设的就是它
ESP_OTA_IMG_INVALID	永不选	esp_ota_mark_app_invalid_rollback_and_reboot()：主动判死
ESP_OTA_IMG_ABORTED	永不选	bootloader：发现上次停在 PENDING_VERIFY，改写为 ABORTED
ESP_OTA_IMG_NEW	只给一次启动机会，且立即被改写为 PENDING_VERIFY	esp_ota_set_boot_partition()，**开了回滚时**设它
ESP_OTA_IMG_PENDING_VERIFY	不选，改写为 ABORTED 后弃用	bootloader 首次启动 NEW 镜像时设置

开了回滚之后，一次 OTA 的完整生命周期：

1. `esp_ota_set_boot_partition()` 把新镜像标 **NEW**；
2. 重启，bootloader 看到 NEW → 改写为 **PENDING_VERIFY** → 启动新固件（"临时工上岗"）；
3. 新固件自检（点灯、联网、关键外设应答，任选）：- 通过 → `esp_ota_mark_app_valid_cancel_rollback()` → **VALID**，转正；- 失败 → `esp_ota_mark_app_invalid_rollback_and_reboot()` → **INVALID**，立即重启回旧槽；
4. 若 PENDING_VERIFY 没确认就断电/崩溃/看门狗复位，**下次启动** bootloader 发现它还停在 PENDING_VERIFY，改写为 **ABORTED**、弃用，自动回滚旧槽。只要不重启，PENDING_VERIFY 可以一直挂着；回滚永远发生在"下次启动"，不在运行中。

而**默认配置（未开回滚）时，这套状态机根本不启用**。官方文档原话：此时两个 mark 函数"是可选的"，`ESP_OTA_IMG_NEW` 和 `ESP_OTA_IMG_PENDING_VERIFY` 两个状态"不会被使用"；set_boot_partition 设的是 UNDEFINED，bootloader 直接启动新槽、立即视为有效。新固件是坏的也照样坐庄，不会自动回滚，只能人工重烧。所以精髓第 3 条反复划的边界："自检通过后 mark valid"是**开回滚后的强制环节**，不是 OTA 的默认流程；不开回滚时，你在固件里调不调 mark valid，启动决策一个字都不会变。

三个附带事实（第五节的演练里都会碰到）：

- 回滚状态记在 otadata 分区里，不在固件镜像里：镜像不变，变的只是"票"。
- 只有 ota 槽会回滚，factory 槽不参与回滚（官方明确）。
- 开了回滚、当前固件还停在 PENDING_VERIFY 时就发起下一次 OTA，`esp_ota_begin()` 直接报 ESP_ERR_OTA_ROLLBACK_INVALID_STATE：先把上一版转正，再升级下一版。

最后一条工具链事实：CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE 编译在 **bootloader** 里。menuconfig 改完必须 `idf.py flash` 连 bootloader 一起重烧，只烧 app 不生效，与常见坑 4"分区表与 boot 不同步"是同一类失配。

五、本地 OTA 演练：切庄、转正与回滚三连

"本地 OTA"= 新 bin 不出局域网：起一个局域网 HTTP(S) 服务放 bin，或干脆把新 bin 预烧进数据分区，设备自己搬进 ota 槽。官方 examples/system/ota/native_ota_example 演示的就是 app_update 原生 API 全流程；网络侧想省事用 esp_https_ota 组件（内部包着同一套 esp_ota_* 管道）。骨架说的"版本号/切槽/回滚"三件事，这里分五步走：

第 0 步：开回滚 + 换双槽表。 menuconfig：Bootloader config 打开 CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE；Partition Table 选 Custom、指向第一节的示例 CSV（两个 3MB ota 槽）。`idf.py flash` 全量烧录（bootloader 一起），`idf.py partition-table` 确认 ota_0/ota_1/otadata 三块地皮都挂牌。**没这步就没有回滚可演示**：默认配置下第 3 步什么都不会发生。

第 1 步：版本号，这出戏的角色名牌。 CMakeLists.txt 里 `set(PROJECT_VER "1.0.0")`（不设则由 git 推导，再不行默认 1），运行时打印版本和当前槽位，分清谁在坐庄：

```
#include "esp_app_desc.h"

const esp_partition_t *running = esp_ota_get_running_partition();
printf("ver=%s, running=%s\n",
       esp_app_get_description()->version, running->label);
/* 首次烧录：ver=1.0.0, running=ota_0 */
```

第 2 步：把 1.1.0 写进另一个槽并切庄。 改 PROJECT_VER 为 1.1.0 构建新 bin，喂给设备（本地分区或局域网服务器），核心管道就一段：

```
#include "esp_ota_ops.h"

const esp_partition_t *next = esp_ota_get_next_update_partition(NULL); /* ota_1 */
esp_ota_handle_t ota;
ESP_ERROR_CHECK(esp_ota_begin(next, OTA_SIZE_UNKNOWN, &ota));
char buf[512];
int n;
while ((n = src_read(buf, sizeof(buf))) > 0) {  /* src：本地分区/HTTP，随你 */
    ESP_ERROR_CHECK(esp_ota_write(ota, buf, n)); /* 分块搬进 ota_1 */
}
ESP_ERROR_CHECK(esp_ota_end(ota));               /* 校验闸门 */
ESP_ERROR_CHECK(esp_ota_set_boot_partition(next)); /* 改票 */
esp_restart();
```

重启后日志：`ver=1.1.0, running=ota_1`，庄家换了，但此刻它只是 PENDING_VERIFY 的"临时工"。

第 3 步：自检转正。 app_main 尽早的位置跑一段自检（官方推荐的骨架）：

```
const esp_partition_t *running = esp_ota_get_running_partition();
esp_ota_img_states_t st;
if (esp_ota_get_state_partition(running, &st) == ESP_OK
        && st == ESP_OTA_IMG_PENDING_VERIFY) {
    if (self_test_ok()) {   /* 点灯/联网/外设应答，任选一招 */
        ESP_ERROR_CHECK(esp_ota_mark_app_valid_cancel_rollback());
        /* PENDING_VERIFY → VALID，正式坐庄 */
    } else {
        ESP_ERROR_CHECK(esp_ota_mark_app_invalid_rollback_and_reboot());
        /* INVALID，立刻回旧槽 */
    }
}
```

第 4 步：故意不转正，看自动回滚。 再 OTA 一版 2.0.0，但这版**故意删掉第 3 步的自检代码**。2.0.0 跑起来很欢（版本号都打印出来了），此时手动 `esp_restart()`，下次启动 bootloader 发现 ota_1 还停在 PENDING_VERIFY：改写为 ABORTED、弃用、回滚，日志回到 `ver=1.1.0, running=ota_0`，**防变砖机制亲手验证完毕**（实物实验留档）。想演"主动回滚"，把 self_test_ok 换成永远返回假，走 INVALID 那条立即回滚的路。

对照组（事实纪律）：如果第 0 步没开回滚选项，同样的"故意不转正"什么也不会发生：重启后照样是 2.0.0 坐庄，bootloader 根本不查这套状态。回滚不是玄学魔法，是 bootloader 多做的一次查票；不开回滚，就没有这次查票。

六、安全选项：从"明文裸奔"到量产的概念地图

到这一节为止，你的 Flash 是明文的：拆下 Flash 芯片用编程器能读走固件，改个 bin 就能刷进去。量产前要补的板子，这里只建概念地图不实操（细节以 IDF Security 文档为准；eFuse 烧错没有后悔药，先拿测试板练手）：

- **Flash 加密**：Flash 落盘密文、运行时由硬件透明加解密，密钥固化在 eFuse（一次性烧写），防"拆片读固件"。menuconfig 分 Development/Release 两档：前者可反复试错，后者一次性烧死。第一节 flags 提过：app、bootloader、分区表、otadata、nvs_keys 这五类**无条件加密**，其余分区要落密文得标 encrypted。
- **安全启动（Secure Boot）**：信任链 ROM→bootloader→app：bootloader 用 eFuse 里的公钥摘要验 app 签名。效果：**没签名的 bin bootloader 直接拒收**。配合 OTA：esp_ota_end 的校验闸从"拦坏镜像"升级到"拦未签名镜像"（签名验证失败同样返回 ESP_ERR_OTA_VALIDATE_FAILED）。
- **anti-rollback**：第四节的状态机防"新固件挂了"（可用性），防不了"故意刷旧版"（安全性：旧版的漏洞还没修）。CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK（与回滚选项配套使用）在镜像描述符 esp_app_desc_t 里引入 secure_version。bootloader 只选"版本号不低于芯片内已固化值"的镜像，只许升不许降。两个细节：secure_version 只有 16 位（升级次数有限）；factory/test 槽不参与这套机制。

- **NVS 加密**：前门上了锁（固件加密），后门还敞着（Wi-Fi 密码明文躺在 NVS）：nvs_keys 分区存密钥，把参数也密文化。官方提醒：NVS 加密防篡改防读取，但防不了"整分区擦除"这种暴力。
- **不开安全启动也要签 OTA**：CONFIG_SECURE_SIGNED_APPS_NO_SECURE_BOOT，不上安全启动硬件链，但 OTA 镜像仍要求签名验证，性价比很高的中间档。

这张地图的作用：让你在"量产"这个词出现时知道去哪查。学习期到第五节为止，明文 + 回滚已够用；真要发产品，Flash 加密、安全启动、anti-rollback 的开启顺序和 Release 模式的不可逆性，IDF Security 章节有保姆级流程：**先在测试板上练**。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **分区表是地产证，NVS 是带掉电保护的抽屉，OTA 是双槽轮流坐庄——新固件不自证清白，旧槽随时复位。**

实物实验

- 本地 OTA 全流程 + 故意"不标 valid"看自动回滚，亲手验证防变砖机制（日志存档）。

常见坑

- **app 超分区大小**：build 过但烧录/启动失败，改 CSV 给 app 扩容或开 -Os 瘦身（B5 方法）。
- **nvs 当高频日志存储**：Flash 寿命按擦写次数计，高频数据先入 RAM 定期落盘。
- **OTA 后忘调 mark_valid**：新固件跑挺欢，重启回旧版：自检+标记是 OTA 的最后一公里。
- **分区表与 boot 不同步**：改了 CSV 只烧 app，`idf.py flash` 全套重烧解决。

短自测

1. 分区表 CSV 的六个字段是什么？app 分区与 data 分区在 offset/size 对齐上各有什么硬约束？写错了什么时候会暴露？

▼ 看答案
六字段：name/type/subtype/offset/size/flags。约束：所有分区 offset 至少 0x1000（一个 4KB 擦除扇区）对齐；app 分区 offset 必须对齐 0x10000，size 只需 4KB 对齐；offset 可留空由 gen_esp32part 自动分配对齐。写错不用等烧录翻车：构建期、`idf.py partition-table` 解析时就直接报错。

2. NVS 更新同一个 key 时，Flash 里到底发生了什么？为什么说追加写"一个设计买到三样东西"？

▼ 看答案
更新 = 在日志尾部追加一个新条目，旧条目标记为 erased，读取以最新为准。三样：磨损均衡（写压力摊到整个分区，官方量化：擦除频率相对写入频率降低 126 倍）；掉电安全（半写条目 CRC32 不过被当不存在，任意时刻断电只丢"正在写的那个键值对"）；页满整理（页状态 Active→Full→Erasing：有效条目搬走、整页擦除回收，全自动且断电可续）。

3. OTA 写入的 API 五步顺序是什么？esp_ota_end 和 esp_ota_set_boot_partition 各把哪道闸？

▼ 看答案
esp_ota_get_next_update_partition（自动轮转挑另一个 ota 槽）→ esp_ota_begin（OTA_SIZE_UNKNOWN 时先整槽擦除）→ esp_ota_write 循环分块写 → esp_ota_end → esp_ota_set_boot_partition → esp_restart。esp_ota_end 是校验闸：验证刚写完的镜像是不是合法 app（开安全启动还要验签），坏 bin 返回 ESP_ERR_OTA_VALIDATE_FAILED 当场拦下；set_boot_partition 是改票闸：更新 otadata，让下次启动换庄。

4. 开了回滚后，新镜像的完整状态生命周期怎么走？关键事实题：默认不开回滚时，"故意不 mark valid"会发生什么？

▼ 看答案
开回滚：set_boot_partition 标 NEW → 重启后 bootloader 把 NEW 改写为 PENDING_VERIFY 并启动 → 自检通过调 esp_ota_mark_app_valid_cancel_rollback 转 VALID（转正）；自检失败调 esp_ota_mark_app_invalid_rollback_and_reboot 标 INVALID 立即回滚；没确认就重启（断电/崩溃/看门狗），下次启动 bootloader 把 PENDING_VERIFY 改写为 ABORTED、弃用，自动回滚旧槽。默认不开回滚：NEW 和 PENDING_VERIFY 两个状态根本不被使用，set_boot_partition 设的是 UNDEFINED，重启直接启动新固件、立即视为有效，坏固件不会自动回滚，只能人工重烧；mark valid 调不调都不影响启动决策。

5. otadata 为什么定 0x2000、内部怎么防掉电？记票板全空时 bootloader 启动谁？回滚选项为什么必须重烧 bootloader？

▼ 看答案
otadata 是两个扇区（0x2000）：写票时两份轮流写、各带计数器，断电后两份不一致按计数器裁决谁更新，记票板自己的掉电安全。全空（全 0xFF，从未 OTA）→ 启动 factory 槽；没有 factory 则启动第一个 ota 槽（通常 ota_0）。回滚选项 CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE 编译在 bootloader 二进制里，menuconfig 改完必须 idf.py flash 连 bootloader 一起重烧，只烧 app 不生效，与"改了分区表只烧 app"是同一类失配。

对照表：本章概念 → 仓库与上游落点

概念	落点
分区表 CSV 六字段与对齐规则	IDF components/partition_table（gen_esp32part.py）；实际读数以 `idf.py partition-table` 为准
0x8000 分区表（0xC00 条目区 + MD5，占满 0x1000 扇区）	IDF《Partition Tables》；P1 启动
NVS 页/条目/CRC 内部结构（页头 32B、条目 32B、126 条目/页）	IDF《NVS Internals》；`nvs_flash.h` / `nvs.h`
boot_count 掉电持久化	IDF examples/storage/nvs/nvs_rw_value（官方重启计数示例）；本节第三节
OTA 五步 API 与双槽机制	IDF `esp_ota_ops.h`（app_update 组件）；examples/system/ota/native_ota_example
回滚状态机 + otadata 双扇区计数器	IDF《OTA》App Rollback 一节；CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE（默认未启用）
anti-rollback（secure_version 单调门槛）	CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK；esp_app_desc_t.secure_version
Flash 加密 / 安全启动 / NVS 加密	IDF Security 章节（Flash Encryption、Secure Boot、NVS Encryption）；nvs_keys 分区
NVS 与 Wi-Fi 校准数据的渊源	P8 Wi-Fi 第五节
手工 Flash 方案对照（擦除粒度/掉电/寿命）	S13 Flash

延伸阅读

NVS 页头与条目的 CRC32 不是装饰：

- **[D9]** Koopman & Chakravarty 2004 — 多项式与报文长度决定检错能力；这篇解释了 32 位 CRC 对几十字节记录"够用"的边界在哪。

你做到了

- 16MB 的每一寸都归你规划；
- 持久化与远程升级两大产品能力原理到手，小智板离"产品"只差你的应用。

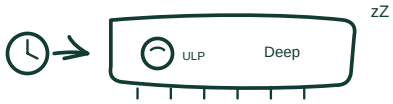
断电判分的结果：数没丢。CPU、内存、寄存器全体失忆，只有 Flash 里那页条目还记得你数到几。公摊面积之外，这是你在这张版图上划出的头一块自留地。

✅ 下一站：P11 低功耗：睡眠矩阵、ULP 与电流实测。

通往单片机之路第 66 章

P11 低功耗：睡眠矩阵与 ULP

难度 ★★☆☆☆ · 约 35 分钟



本章罗盘

问题

Deep sleep 一觉醒来为什么像冷启动从 app_main 重跑？变量怎么活过断电？

交付

Light/Deep 四列对照表（RAM/外设/唤醒方式/唤醒后状态）+ 周期上报器六步骨架（RTC_DATA_ATTR 跨睡计数、esp_sleep_get_wakeup_cause 分诊），附 CR2032≈90 天、18650≈150 天两笔续航手算。

边界

电流数值以 datasheet §Power consumption 与 E06 实测为准（章内明确『不编造』），续航是理论上限、实测要打 LDO 漏电折扣；ULP 只给两代对比与开发流程地图、不教编程；UART 深睡唤醒能力依芯片而异、见 TRM，本章不展开。

依据锚点 `a7` · `a8` · `a9` · `mcu-lab` · `mcu-anim`

> 🎯 Wi-Fi 芯片谈低功耗，像让短跑运动员学睡觉。但只要回答三个问题，**睡多深、谁叫醒、醒来还剩什么**，电池供电的 ESP32 产品就成立。Deep Sleep 下电流做到 μA 级，ULP 协处理器替你站岗。

本章精髓

1. **两档睡眠**。Light sleep 只是 CPU 暂停，RAM 保持、外设可唤醒，毫秒级回神。Deep sleep 把数字域整个断电，只留 RTC 域，电流降到 μA 级。 代价是：**Deep sleep 醒来 = 复位重启**，程序从 `app_main` 重新跑。对照 S14 的 Sleep/Stop/Standby 三档，ESP 少一档、但把 RTC 域这场戏演得更重。
2. **唤醒源有五路**：定时器 `esp_sleep_enable_timer_wakeup`、GPIO（EXT0/EXT1）、触摸、UART、ULP。 选哪一路，决定了产品怎么设计：定时上报就选定时器，按键唤醒就选 EXT。
3. **ULP 是守夜人**。主核睡死时，这颗超低功耗小核还在跑。它能读 ADC/I2C、能数脉冲，只有达到阈值才把主核叫醒。 这是电池产品的终极省电手段（本章简介 FSM 与 RISC-V 两代 ULP）。

- **能理解**：为什么 Deep sleep 唤醒≈复位重启（数字域断电，CPU/RAM 全没）；为什么 EXT0/EXT1 只能用 RTC GPIO（普通 GPIO 在数字域，断电就废）；为什么 Wi-Fi 没关就睡电流下不来（射频/协议栈还在跑）。
- **能用**：写出周期上报器的六步骨架（醒来→采→连→发→关 Wi-Fi→睡）。用 `RTC_DATA_ATTR` 跨复位保存计数，用 `esp_sleep_get_wakeup_cause` 判断这次为什么醒来。

先修

- S14 低功耗（对照）、P10 NVS（持久化分工）。

先跑起来（10 分钟 quick win）

`esp_sleep_enable_timer_wakeup(5s)` + `esp_deep_sleep_start()`：5 秒后自动复位，串口打印 RTC 内存里的计数，睡-醒循环第一次跑通。

睡前先押份遗产：Deep Sleep 五秒醒来，普通全局变量还认得你吗？押认得或失忆，再看串口。这一觉的账单，比你想的重。

动画：睡眠矩阵与唤醒路径

Light 与 Deep 的四列差异、RTC 内存的“遗言”、ULP 守夜人：Deep 阶段数字域 CPU/SRAM 变灰断电，boot_count 跨复位累加，唤醒箭头周期射入。

ESP32 睡眠矩阵：Light 眯一会 · Deep 睡死 · ULP 守夜

小节结构

小节	内容	四件套
睡眠矩阵	Light/Deep 四列对照；与 STM32 三档对表	配置
RTC 内存与慢速内存	RTC_DATA_ATTR/RTC_SLOW 的存放规则	代码分析
唤醒源配置	定时器/EXT0/EXT1/触摸/UART 逐个	配置
周期上报器	醒来→采数据→发 Wi-Fi→再睡的完整骨架	代码分析
ULP 概念	两代 ULP 能力；适用场景与开发流程地图	库解析
电流实测	三档电流记录与续航估算（CR2032/18650 两例）	代码分析

一、睡眠矩阵：Light vs Deep 四列对照

ESP32 的睡眠分两档：Light sleep 与 Deep sleep。它们大致对应 S14 STM32 的 Stop 与 Standby。先看四列对照：

维度	Light sleep	Deep sleep
CPU	暂停（时钟门控）	断电
RAM	保持	断电（数字域 RAM 全失）
外设	大多保持，可配置	数字域外设断电，仅 RTC 域活
唤醒方式	GPIO/定时器/UART 等任意外设	仅 RTC 域唤醒源（Timer/EXT0/EXT1/Touch/UART/ULP）
唤醒后	从暂停点继续执行	= 复位重启（setup() 从头）
电流	几 mA 量级	μA 级
回神时间	ms 级	ms 级（外加复位重启代价）

哲学与 S14 一致：睡得越深，省得越多，但醒来代价越大。STM32 三档里，ESP 的 Light ≈ Stop（RAM 保持、外设部分活），ESP 的 Deep ≈ Standby（域断电、靠 RTC 留遗言）。差别在于 ESP 没有对应 STM32 Sleep 的那一档。STM32 的 Sleep 只停 CPU、外设全活；ESP 的"轻睡"默认连射频一起关（Modem sleep）。两者省电根本不是一个量级。

关键认知：**Deep sleep 唤醒 = 复位重启**。`setup()` 从头跑一遍，全局变量全丢（除非放 RTC 内存）。这是新手最大的认知颠覆："我刚才不是还在跑吗？怎么又 setup 了？"因为数字域 RAM 失电了，CPU 从复位向量重新启动。Light sleep 不一样：从 `esp_light_sleep_start()` 下一行继续跑，像 `vTaskDelay` 一样。

二、RTC 内存与慢速内存：跨 Deep sleep 的"遗言"

Deep sleep 会断掉整个数字域 RAM，但 RTC 域还活着。RTC 域里有一块 RTC slow memory，专门用来存"跨睡眠还想过夜"的状态。IDF 用属性宏把变量放进这块内存。只要给变量加个宏，它就住在 RTC 域里：

```
RTC_DATA_ATTR int boot_count = 0;   /* 跨 Deep sleep 保持；仅上电冷启动才清零 */

void app_main(void) {
    boot_count++;
    printf("第 %d 次从 Deep sleep 醒来\n", boot_count);
    esp_sleep_enable_timer_wakeup(5 * 1000000ULL);  /* 5s */
    esp_deep_sleep_start();
    /* 不会到这里——Deep sleep 后复位 */
}
```

`RTC_DATA_ATTR`：变量放进 RTC slow memory。Deep sleep 期间保持；**只有上电（冷启动）才清零**，看门狗复位同样保持。所以 `boot_count` 每次 Deep sleep 醒来 +1，能跨复位累计。

存放规则速查：

属性	位置	跨 Deep sleep	跨冷启动
`RTC_DATA_ATTR`	RTC slow memory	保持	复位为 0
`RTC_NOINIT_ATTR`	RTC slow memory	保持	**不初始化**（脏值）
`RTC_SLOW_ATTR`	RTC slow memory（同 RTC_DATA）	保持	复位为 0
`RTC_FAST_ATTR`	RTC fast memory	保持	复位为 0
普通全局变量	数字域 SRAM	**失电**	复位为 0

`RTC_NOINIT_ATTR` 用途：调试时想看上一次 crash 时的变量值（不被复位清零），但脏值风险大：除非你确定要这个语义，否则用 `RTC_DATA_ATTR`。

容量注意：RTC slow memory 8 KB 量级（具体看 TRM），ULP 程序也住这里：ULP 用得多留给主核的就少，规划时要权衡。

三、唤醒源配置：五种主流唤醒

Deep sleep 只能被 RTC 域唤醒源叫醒（普通 GPIO 在数字域，断电就废了）。主流唤醒源有五种，逐个看：

1. 定时器唤醒（最常用，周期上报）：

```
esp_sleep_enable_timer_wakeup(5 * 1000000ULL);  /* 5s = 5,000,000 μs */
esp_deep_sleep_start();
```

参数是微秒（`uint64_t`），常量要带 `ULL` 后缀防 32 位溢出（5s 在 32 位内没事，但 1 小时 = 3.6e9 μs 就溢出了）。

2. EXT0 唤醒（单 RTC GPIO，电平触发）：

```
esp_sleep_enable_ext0_wakeup(GPIO_NUM_0, 0);  /* GPIO0 拉低唤醒（BOOT 键） */
```

一个 RTC GPIO，指定电平（0 或 1），到了就醒。适合"按键唤醒"单脚场景。

3. EXT1 唤醒（多 RTC GPIO，组合触发）：

```
/* 位掩码：GPIO 0 | GPIO 2，任一高电平唤醒 */
esp_sleep_enable_ext1_wakeup(BIT64(GPIO_NUM_0) | BIT64(GPIO_NUM_2), ESP_EXT1_WAKEUP_ANY_HIGH);
```

位掩码用 `BIT64(n)`（GPIO 编号可能 ≥ 32，所以用 64 位掩码）。模式两种：`ESP_EXT1_WAKEUP_ANY_HIGH`（任一高）、`ESP_EXT1_WAKEUP_ALL_LOW`（全部低）。醒来后用 `esp_sleep_get_ext1_wakeup_status()` 查是哪只脚触发的。

4. 触摸唤醒（触摸传感器在 RTC 域）：

```
touch_pad_init();
/* 配置触摸阈值... */
esp_sleep_enable_touchpad_wakeup();
```

触摸传感器是 RTC 域外设，Deep sleep 也能跑。适合"触摸按键唤醒"产品。

5. UART 唤醒（特定模式）。

```
esp_sleep_enable_uart_wakeup(0);  /* UART0 收到一定数量字节唤醒 */
```

UART 唤醒主要用在 Light sleep。Deep sleep 下的 UART 唤醒能力有限，且因芯片而异（详见 TRM）。本章不展开，配置见 E06。

唤醒后分诊原因（任何低功耗产品必做）：

```
esp_sleep_wakeup_cause_t cause = esp_sleep_get_wakeup_cause();
switch (cause) {
    case ESP_SLEEP_WAKEUP_TIMER:    printf("定时器叫醒\n"); break;
    case ESP_SLEEP_WAKEUP_EXT0:     printf("EXT0 GPIO 叫醒\n"); break;
    case ESP_SLEEP_WAKEUP_EXT1:     printf("EXT1 GPIO 叫醒，pin=0x%llx\n", esp_sleep_get_ext1_wakeup_status()); break;
    case ESP_SLEEP_WAKEUP_TOUCHPAD: printf("触摸叫醒\n"); break;
    case ESP_SLEEP_WAKEUP_UART:     printf("UART 叫醒\n"); break;
    case ESP_SLEEP_WAKEUP_ULP:      printf("ULP 叫醒\n"); break;
    case ESP_SLEEP_WAKEUP_UNDEFINED: printf("冷启动（首次上电）\n"); break;
}
```

`ESP_SLEEP_WAKEUP_UNDEFINED` 是冷启动（首次上电或外部复位），区别于"Deep sleep 醒来"，新设备首次跑要识别这个状态走不同分支。

四、周期上报器：醒来→采数据→发 Wi-Fi→再睡

电池 IoT 产品的经典骨架：每 N 分钟醒一次，采传感器、连 Wi-Fi、上报、断 Wi-Fi、再睡。完整代码骨架：

```
RTC_DATA_ATTR int cycle = 0;   /* 跨 Deep sleep 累计周期数 */

void app_main(void) {
    /* ① 分诊：是不是 Deep sleep 醒来 */
    if (esp_sleep_get_wakeup_cause() == ESP_SLEEP_WAKEUP_TIMER) {
        printf("第 %d 次周期醒来\n", ++cycle);
    } else {
        printf("冷启动，cycle 复位\n");
        cycle = 0;
    }

    /* ② 采数据（这里读 ADC/传感器） */
    int sensor_val = read_sensor();

    /* ③ 连 Wi-Fi（用 P8 骨架六步，这里简化） */
    wifi_connect_blocking();   /* 阻塞等到 GOT_IP */

    /* ④ 上报（HTTP/MQTT/ESP-NOW 任选） */
    report_to_server(sensor_val);

    /* ⑤ 关 Wi-Fi（射频必须关，否则电流下不来） */
    esp_wifi_disconnect();
    esp_wifi_stop();           /* 协议栈停 */
    /* 可选：esp_wifi_deinit() 释放更多资源 */

    /* ⑥ 配置下次唤醒 + 睡 */
    esp_sleep_enable_timer_wakeup(10 * 60 * 1000000ULL);  /* 10 分钟 */
    esp_deep_sleep_start();    /* 不会返回 */
}
```

六步铁律：分诊→采→连→发→**关 Wi-Fi**→睡。第 ⑤ 步是新手最容易漏的：Wi-Fi 协议栈还在跑就 Deep sleep，射频/协议栈状态机乱了，电流也下不来（见常见坑）。`esp_wifi_stop()` 之后再睡，醒来重新 `esp_wifi_start()`（在 wifi_connect_blocking 里）。这是低功耗与 P8 联网骨架 的拼装点。

`wifi_connect_blocking()` 的实现：在 P8 事件骨架基础上用信号量等 `IP_EVENT_STA_GOT_IP`，主任务等信号量，事件 handler 给信号量。这是"事件驱动 + 同步等待"的标准拼装。

续航估算（只算量级）。假设活跃 200 mA 持续 2 秒（连接+发送），Deep sleep 10 μA 持续 600 秒，平均电流 ≈ 200mA·2s/602s + 10μA·600s/602s ≈ 0.68 mA。CR2032（约 220 mAh）续航 ≈ 220/0.68 ≈ 323 小时 ≈ 13 天。如果唤醒间隔拉到 1 小时，平均电流降到约 0.11 mA，续航约 90 天。**唤醒间隔越长，Deep sleep 占比越高，续航越长**：这是电池 IoT 产品的核心设计杠杆。具体数值以 datasheet §Power consumption 与 E06 实测 为准。

五、ULP 协处理器：守夜人

主核进入 Deep sleep 后，ULP 协处理器仍在 RTC 域里运行。这颗超低功耗小核就是电池产品的"守夜人"：周期性醒来读 ADC/I2C、数脉冲，达到阈值才唤醒主核。主核能睡得更久（不用频繁醒来轮询），ULP 的 μA 级功耗远低于主核频繁唤醒的 mA 级。

两代 ULP：

维度	ULP（FSM，原始）	ULP-RISC-V（S2/S3）
架构	有限状态机（汇编宏）	RISC-V 核心
编程	`ulp_process_macros`（宏汇编）	C/RISC-V 工具链
算力	极弱（FSM 级指令集）	弱但有 C 编译器
能访问	RTC GPIO、ADC、I2C（专用指令）	RTC GPIO、ADC、I2C
唤醒主核	SW 中断	SW 中断

ULP（FSM）的"程序"是一组宏汇编指令，编译后放进 RTC slow memory，主核睡后 ULP 周期性执行。开发流程地图（FSM 版）：

1. 用 `ulp_process_macros` 写宏汇编（读 ADC、比较、条件唤醒）
2. 程序加载到 RTC 内存（链接脚本与 `ulp_load_binary`）
3. 配置 ULP 唤醒周期（`ulp_set_wakeup_period`）
4. 主核 `esp_deep_sleep_start()`，ULP 自动接管

ULP-RISC-V（S2/S3）更现代：直接用 C 写，有 GCC 工具链，算力稍强，但还是"哨兵"定位，别拿它跑业务。

适用场景地图：

- **用 ULP**：主核要长时间睡、但有"阈值触发"需求（ADC 超阈值、按键长按、计数脉冲到 N），ULP 替你站岗，达到条件才叫醒主核。

- **不用 ULP**：周期性定时上报（用 Timer 唤醒即可）；唤醒条件简单（按键电平变化用 EXT0），ULP 开发复杂度不值得。
- **别让 ULP 干业务**：ULP 算力/内存极小（RTC slow memory 几 KB），它只适合"读-比-唤醒"哨兵循环，跑业务逻辑会爆。

六、电流实测：三档记录与续航估算

P8 骨架 拿到逻辑通了，但低功耗产品要拿电流表说话。E06 实验 用万用表 μA 档（或 Power Profiler）测三档电流：

档位	测点	量级	备注
活跃（连 Wi-Fi TX）	工作电流峰值	数百 mA 峰值	射频突发，看示波器波形
活跃（待机，不连）	平均电流	数十 mA	主核在跑、射频待机
Light sleep	平均电流	几 mA	Modem sleep 关射频
Deep sleep	平均电流	μA 级	仅 RTC 域
Deep sleep + ULP	平均电流	几十 μA 起	ULP 周期活动

具体数值以 datasheet §Power consumption 与 E06 实测为准（不编造）。实测要点：

1. **量程切换**：活跃用 mA 档、Deep 用 μA 档，手动切换：直接用 μA 档测活跃会被瞬间电流冲爆（保险丝烧）。
2. **测点选电源脚**：测 3V3 进线电流，别测 USB（USB 走了板载 LDO/CP2102，漏电大，不准）。
3. **断掉 USB 调试**：USB CDC 跑着电流下不来：测 Deep sleep 要拔 USB，用电池/外部 3V3 供电。

续航估算两例（量级）：

- **CR2032 纽扣电池**（约 220 mAh）：唤醒间隔 1 小时、活跃 2s@200mA、Deep 10μA → 平均约 0.11 mA → 续航约 90 天。适合"低频上报"产品（环境监测、门磁）。
- **18650 锂电**（约 2500 mAh）：唤醒间隔 10 分钟、活跃 2s@200mA、Deep 10μA → 平均约 0.68 mA → 续航约 150 天。适合"中频上报"加较大体积产品。

注意：实测续航要打"折扣"：板载 LDO 漏电、传感器静态电流、外设未关都吃电。理论续航是上限，实际产品要逐项掐电（每颗芯片的使能脚、上拉电阻值、外设电源开关）。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **Light 眯一会（记忆在），Deep 睡死（靠 RTC 留遗言），ULP 守夜看门——唤醒即复位，状态藏 RTC。**

延伸：ESP32 睡眠矩阵动画见 P11 动画；STM32 低功耗对照见 S14；RTC 与 NVS 数据保持见 P10。

实物实验

- 周期上报器 + 电流三档实测；进阶：GPIO 唤醒（用户键）与定时唤醒并存，分辨唤醒原因（esp_sleep_get_wakeup_cause）。

常见坑

- **Deep sleep 后找变量**：普通 RAM 已失电，跨 sleep 状态必须 RTC_DATA_ATTR 或 NVS。
- **Wi-Fi 没关就睡**：射频/协议栈在跑，电流下不来：睡前 esp_wifi_stop/disconnect。
- **GPIO 唤醒脚选错**：Deep sleep 只有 RTC GPIO 能唤醒，查 TRM 的 RTC 引脚清单。
- **ULP 当主核用**：ULP 算力/内存极小，它只适合做"阈值哨兵"，别让它干业务。

短自测

1. Light sleep 和 Deep sleep 在"RAM/外设/唤醒方式/唤醒后状态"四列的差异是什么？

▼ 看答案
Light sleep：CPU 暂停（时钟门控）、RAM 保持、外设大多保持可配置、唤醒后从暂停点继续执行（像 vTaskDelay 醒来），电流几 mA。Deep sleep：CPU 断电、RAM 断电（数字域全失）、外设仅 RTC 域活、唤醒=复位重启（setup() 从头跑），电流 μA 级。本质区别是"RAM 保持还是断电"，决定了醒来是"继续"还是"重来"。

2. 为什么 Deep sleep 唤醒=复位重启？普通全局变量为什么会丢？

▼ 看答案
Deep sleep 把数字域（CPU、SRAM、大多数外设）电源全断了，只留 RTC 域。CPU 没时钟没电源，重新上电时从复位向量启动，等于一次冷复位，setup()/app_main() 从头跑。普通全局变量存在数字域 SRAM，断电即失电，醒来后 RAM 是"全新"的，变量全复位为 0。要跨 Deep sleep 保持状态，必须用 RTC_DATA_ATTR 把变量放进 RTC slow memory（RTC 域不断电），或写 NVS（Flash，永久）。

3. EXT0/EXT1 唤醒为什么只能用 RTC GPIO？普通 GPIO 不行吗？

▼ 看答案
Deep sleep 期间数字域断电，普通 GPIO 的输入检测电路也在数字域，断电就检测不了电平变化。只有 RTC GPIO（GPIO 多路复用器接到 RTC 域的引脚）在 Deep sleep 时还有电，能检测电平变化触发唤醒。所以选唤醒脚必须查 TRM 的 RTC GPIO 清单，不是任意 GPIO 都能 EXT0/EXT1 唤醒。

4. 周期上报器的六步铁律里，"关 Wi-Fi"（esp_wifi_stop）这一步为什么不能省？

▼ 看答案
Wi-Fi 协议栈和射频在 Deep sleep 前没停：状态机还在跑，射频可能还在 Beacon 唤醒周期。直接睡下去会有两个后果。其一，协议栈状态会乱，醒来 `start` 时报错或重连失败。其二，射频和协议栈的功耗会把 Deep sleep 的 μA 级拉高到 mA 级，省电效果全部作废。esp_wifi_disconnect 加 esp_wifi_stop 把协议栈停干净，醒来重新 start，干净利落。

5. ULP 适合做什么、不适合做什么？为什么主核睡死时 ULP 还能跑？

▼ 看答案
ULP 适合"阈值哨兵"，周期性读 ADC/I2C、数脉冲、比较阈值，达到条件才唤醒主核。主核能睡更久不用频繁醒来轮询，ULP 的 μA 级远低于主核频繁唤醒的 mA 级。不适合跑业务逻辑：ULP 算力/内存极小（RTC slow memory 几 KB），FSM 版只有有限指令集，RISC-V 版虽有 C 编译器但仍弱。主核睡着时 ULP 还能跑，原因在供电分区：ULP 在 RTC 域，数字域断电不影响它。ULP 周期性执行 RTC slow memory 里的程序，满足条件就通过 SW 中断唤醒主核（主核走复位重启）。

对照表：本章概念 → 仓库与上游落点

概念	落点
Light/Deep 睡眠矩阵	IDF `esp_sleep.h`；对照 S14 STM32 低功耗 三档
RTC 内存属性	`RTC_DATA_ATTR` / `RTC_NOINIT_ATTR` / `RTC_SLOW_ATTR` / `RTC_FAST_ATTR`（IDF `esp_attr.h`）
五种唤醒源	`esp_sleep_enable_*_wakeup` 系列；唤醒原因 `esp_sleep_get_wakeup_cause`
周期上报器骨架	本节第四节；与 P8 Wi-Fi 骨架 拼装
ULP 协处理器	IDF `ulp/` 目录；FSM（`ulp_process_macros`）+ RISC-V（S2/S3）
电流实测	E06 实验；datasheet §Power consumption

你做到了

- 电池供电 ESP32 产品的全套低功耗工具到手；
- S/P 两篇低功耗观完成对照："睡觉"这门学问你现在是双学位。

遗产清点：醒来即复位，普通变量全体失忆，活下来的只有 RTC 域里那份记账。真做周期上报的产品，睁眼第一件事是问"昨晚谁叫的我"，把唤醒原因分诊清楚，再决定这一轮干什么。

✅ 下一站：P12 音频链路：ES8311+ES7210，小智板的看家本领，从 I2S 时序到"hello 语音"。

通往单片机之路第 67 章

P12 音频链路：从 I2S 时序到"hello 语音

难度 ★★★☆☆ · 约 45 分钟

本章罗盘

问题

音频芯片为什么 I2C 和 I2S 两套线并用？MCLK 没给或给错为什么会无声？

交付

一张 麦→ES7210→I2S RX→S3→I2S TX→ES8311→NS4150B→喇叭 全链路图（带宽账：16kHz×16bit→512kHz BCK、MCLK=256×fs=4.096MHz），加一段 E08 实测能响的『滴』（stereo 双槽位填同一份样本、PA_EN 走 PCA9557 bit1 而非 GPIO）。

边界

E08 只实测打通播放（ES8311 DAC）路径；ES7210 采集与耳返回环给的是骨架，往返约 20ms+ 的延迟账是估算、待上板实测；AEC/VAD/Opus/唤醒词只给位置地图（『另一个故事』）；I2C 地址与 I2S 引脚以立创 wiki 原理图为准、成稿待逐项核对。

依据锚点 `b3 · a9 · mcu-code · mcu-lab · mcu-anim`

> 🎯 立创实战派 S3 不是普通开发板——它天生是"会说话的终端"：双麦克风 + 1W 喇叭 + ES8311 解码 + ES7210 采集。打通这条音频链路，你就站在了"小智 AI 语音助手"的家门口。

本章精髓

1. I2S 是音频的"传送带协议"：三根线（BCK 位时钟/WS 声道选择/DATA 数据）按帧传输：**采样率×位宽×声道数=带宽**（16kHz×16bit×1=256kbps 有效 PCM 数据；两个 16bit 槽位的 I2S 帧时钟为 512kHz），MCLK 供给 codec 的 PLL（与 S12 SPI 对照：都是同步移位，I2S 多了"帧=左右声道"的语义）。
2. codec 是"模拟↔数字翻译官"：ES8311（DAC：数字→喇叭）与 ES7210（ADC：麦克风→数字）都挂 I2C 配置、走 I2S 传数据：**控制面（I2C）与数据面（I2S）分离**是音频芯片的通用范式（这就是四件套的完整用武之地）。
3. 功放与麦是"最后一厘米"：NS4150B D 类功放放大功率推喇叭；双模拟麦经 ES7210 四通道（用其三）采集：回声消除/波束成形的算法空间就藏在这"双麦"里。

- **能理解**：为什么 codec 要分控制面和数据面；为什么 stereo 双槽位要填同一份单声道样本；为什么 PA_EN 不在 ESP32 GPIO 上而在 I2C 扩展器。
- **能用**：照 E08 配出 16kHz/16bit 的 I2S + ES8311，让喇叭"滴"一声；看懂 main.c 里初始化链路的每一步在干什么。

先修

- P6 I2C/SPI 驱动（控制面）、S12 SPI（同步移位）、F4 队列（音频流缓冲）。

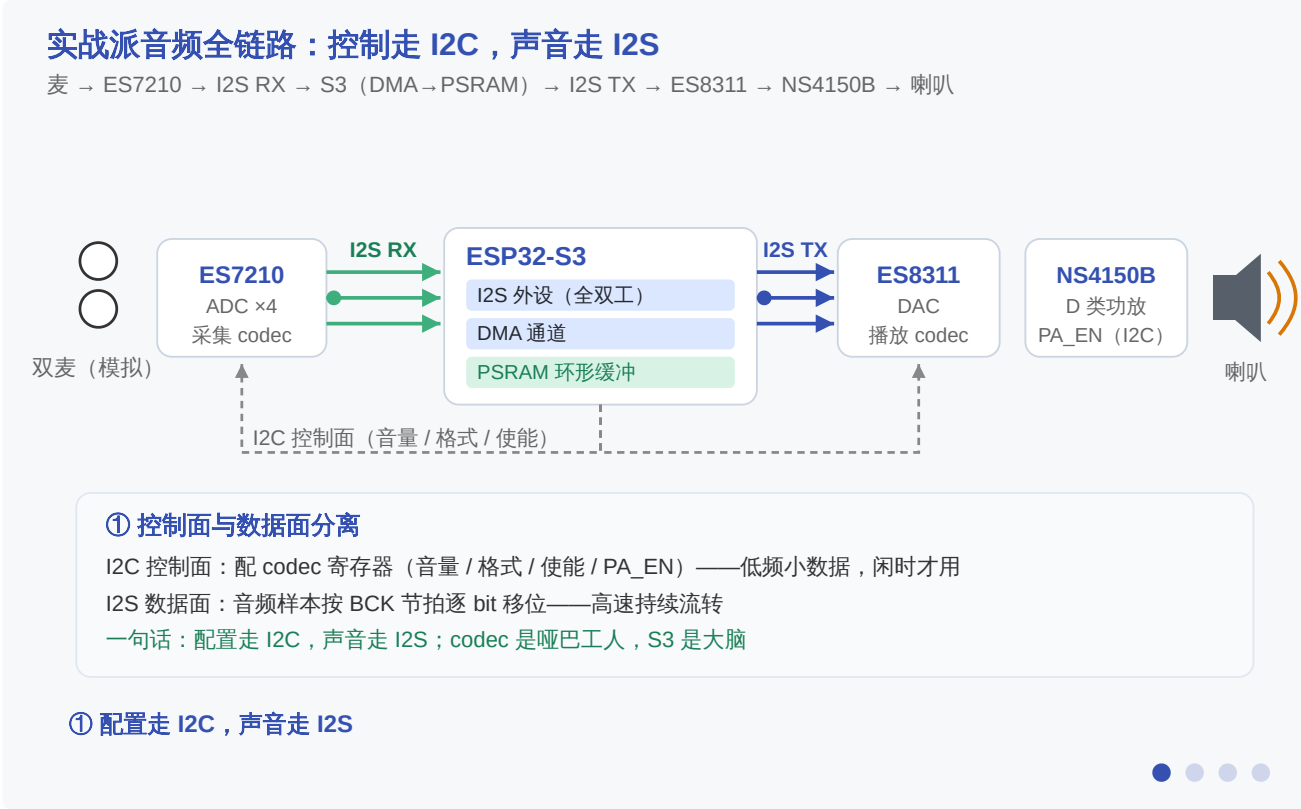
先跑起来（10 分钟 quick win）

播"滴"：内置一段 1kHz 正弦 PCM 数组（const，回 C1），I2S 写出：喇叭第一次在你代码控制下发声。

让喇叭滴之前先押个数量级：采样率、位宽、声道数相乘，这条传送带每秒要走掉多少 bit？心里先按下一个数量级，再拿精髓第一行的算式对账。差一位，后面 I2S 的时钟账就全对不上。

动画：实战派音频全链路

采集链与播放链一条传送带画完：麦 → ES7210 → I2S RX → S3（DMA→PSRAM）→ I2S TX → ES8311 → 功放 → 喇叭；I2C 控制面与 I2S 数据面分离，样本令牌在链路上持续流动。



实战派音频全链路：控制走 I2C，声音走 I2S

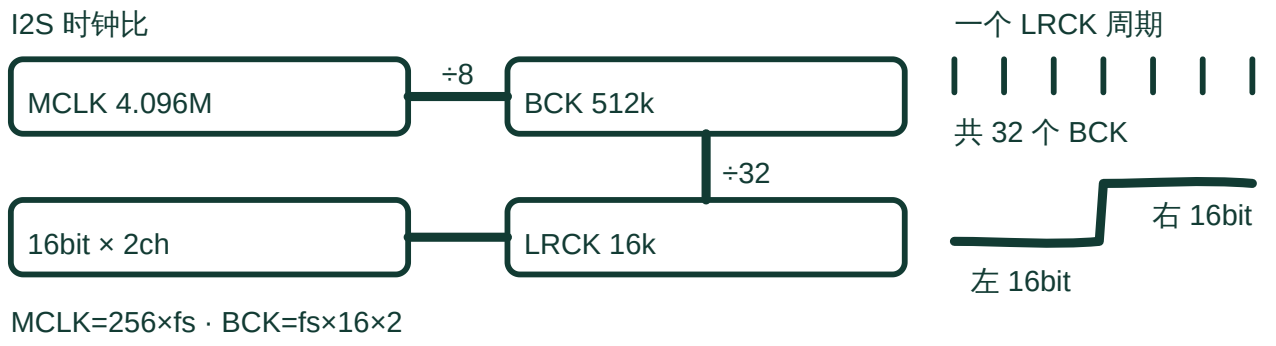
板卡事实

- 音频 codec：ES8311（DAC，I2C 配置+I2S 数据）；ES7210（四通道 ADC，I2C+I2S）；功放 NS4150B（D 类，单声道）；双麦 ZTS6216（模拟）。
- **全部 I2C 地址、I2S 引脚分配、功放使能脚以立创 wiki 原理图页为准**，成稿时逐项核对标注。

小节结构

小节	内容	四件套
I2S 协议	BCK/WS/DATA/MCLK 时序；帧与声道语义	配置
控制面 vs 数据面	I2C 配 codec（音量/使能/格式）+ I2S 跑数据	配置
ES8311 播放	初始化序列→I2S TX→正弦发声	代码分析
ES7210 采集	I2S RX 读 PCM；RMS 能量计算	代码分析
全双工回环	采集→直通播放（耳返实验）；缓冲与延迟账	代码分析
通往小智	语音链路之后：Opus 编码/AEC/唤醒词的位置地图	库解析

一、I2S 协议：音频的"传送带"



I2S 时钟账：MCLK/BCK/LRCK 的比例

I2S（Inter-IC Sound）是飞利浦定下的芯片间音频串行协议，本质是"按帧移位"，和 S12 SPI 同源（都是同步移位），但多了一层"帧=左右声道"的语义。

四根线：

- **BCK（Bit Clock）位时钟**：每一个 bit 一拍，移位基准。
- **WS（Word Select）声道选择**：又叫 LRCK。WS=0 传左声道，WS=1 传右声道。WS 频率 = 采样率 fs。
- **DATA（Serial Data）数据线**：按 BCK 节拍移位，MSB first。
- **MCLK（Master Clock）主时钟**：给 codec 内部 PLL 用，通常是 fs 的整数倍（256×fs 最常见）。

带宽公式：

```
BCK 频率 = 采样率 × 位宽 × 声道数
MCLK 频率 = 采样率 × mclk_multiple（常用 256/384/512）
```

举例（E08 实测，main.c:18 `SAMPLE_RATE=16000`）：

- 16kHz × 16bit × 2ch(stereo) = 512kHz 的 BCK
- MCLK = 256 × 16000 = 4.096MHz（main.c:95 `I2S_MCLK_MULTIPLE_256`，main.c:126 日志 `MCLK=4096000`）

为什么 MCLK 是 256×fs：codec 内部有个 PLL，要从 MCLK 分频出 fs 的所有内部时钟（DAC 转换、滤波器等）。MCLK 必须是 fs 的高倍整数，PLL 才能稳锁。256 是历史惯用值（也兼容 384/512）。MCLK 没给或给错频率，codec PLL 锁不住，这是常见坑第一名。

帧与声道语义：一个 WS 周期是一"帧"，包含左槽位 + 右槽位。Philips 标准里 WS 在 BCK 下降沿提前一个 BCK 切换（给 codec 时间对齐）。stereo 模式两槽位都传；mono 模式只传一个，但 I2S 硬件通常仍按 stereo 跑，软件把同一份样本填两槽位（ES8311 就是这么干的，见第三节）。

二、控制面 vs 数据面：I2C 配、I2S 跑

音频 codec 不是"配一次就忘"的简单外设，它有两个截然不同的接口：

接口	走什么	干什么	频率
控制面（I2C）	寄存器读写	配音量/使能/格式/PLL/mute	偶发（初始化+调音量）
数据面（I2S）	PCM 样本流	实时音频数据	持续（按 fs 流）

为什么分离：I2C 慢（100kHz/400kHz），适合"偶尔改个寄存器"；I2S 快（MHz 级 BCK），适合"连续流数据"。如果控制也走 I2S，每次改音量都要打断数据流，不现实。所以业内通用范式就是"控制面 I2C + 数据面 I2S"，P6 I2C/SPI 驱动 的 I2C 在这里是 codec 的"遥控器"。

E08 实测（main.c:71-80）：

- I2C 总线：SDA=1, SCL=2（main.c:72），与 P6 同一套 I2C master 驱动
- ES8311 挂这条 I2C，地址 `ES8311_CODEC_DEFAULT_ADDR`（main.c:99）
- PCA9557 扩展器也挂这条 I2C，地址 0x19（main.c:78）：PA_EN 不在 ESP32 GPIO 上，而在扩展器的 bit1（main.c:81 注释明说"PA_EN 是 PCA9557 bit1，不是 ESP32 GPIO1"）

esp_codec_dev 把这套"控制面+数据面"封装成统一接口：`audio_codec_new_i2c_ctrl`（控制面，main.c:101）+ `audio_codec_new_i2s_data`（数据面，main.c:103）+ `es8311_codec_new`（芯片抽象，main.c:112）：你只跟 `esp_codec_dev_open/write` 打交道，I2C/I2S 的细节藏在下面。

三、ES8311 播放：从初始化到"滴"一声

ES8311 是单声道音频 codec（DAC+ADC，但板子上只用 DAC 路径推喇叭）。E08 的初始化链路（main.c:84-122）：

1. **建 I2S TX 通道**（main.c:84-87）：`I2S_NUM_0`、master 角色、`auto_clear=true`（写不动时自动输出静音，避免残留噪声）。

2. **配 I2S std 模式**（main.c:88-96）：- `I2S_STD_CLK_DEFAULT_CONFIG(16000)`：fs=16kHz - `I2S_STD_PHILIPS_SLOT_DEFAULT_CONFIG(16BIT, STEREO)`：Philips 标准、16bit、stereo - 引脚：mclk=38/bclk=14/ws=13/dout=45（main.c:92），din 未用（只播不采）- `mclk_multiple = 256`（main.c:95）
3. **建 codec 控制面+数据面+GPIO 抽象**（main.c:98-105）：I2C 控制面接 ES8311，I2S 数据面接 TX 通道。
4. **建 ES8311 实例**（main.c:106-112）：`codec_mode = DAC`、`master_mode = false`（ESP32 当 I2S master，codec 当 slave）、`use_mclk = true`、`mclk_div = 256`。
5. **包成 esp_codec_dev**（main.c:114-122）：`ESP_CODEC_DEV_TYPE_OUT`、采样信息 bits=16/channel=2/channel_mask=3/fs=16000。
6. **开 codec + 设音量**（main.c:122-123）：`esp_codec_dev_open` + `set_out_vol(40)`。

发声核心是 `tone()` 函数（main.c:48-67）：

```
int16_t pcm[BLOCK_FRAMES * 2];   /* 双槽位：左右各一份 */
for (unsigned i = 0; i < count; ++i) {
    int16_t value = (int16_t)(3000.0f * envelope *
        sinf(6.28318530718f * frequency * n / SAMPLE_RATE));
    pcm[2*i] = value;      /* 左槽位 */
    pcm[2*i+1] = value;    /* 右槽位：同一份单声道样本 */
}
esp_codec_dev_write(codec, pcm, count * 2U * sizeof(int16_t));
```

关键细节：

- **stereo 双槽位放同一份单声道样本**（main.c:50 注释、main.c:62-63）：ES8311 是单声道 DAC，但 I2S 配的是 stereo，所以左右槽位填同一个值，否则只有一边出声。
- **首尾 10ms 淡入淡出**（main.c:59）：`envelope = fminf(n/160, (frames-1-n)/160)`，160 个样本 = 10ms（16000/1000×10）。避免方波边沿的"啪"声。
- **先静音 100ms 让 DMA 稳定**（main.c:124 `tone(0, 100)`）：DMA 启动有填充延迟，直接发声会丢头。
- **PA_EN 静音-发声-静音三段**（main.c:81/125/137）：上电先静音防爆音，发声前才 `amplifier(true)`，结束先静音再关功放。

四、ES7210 采集：双麦 PCM 与 RMS 能量

ES7210 是四通道音频 ADC，板子上用其中两个通道接双麦（见板卡事实）。E08 工程只演示了播放（ES8311 DAC 路径），采集路径需要照同一套 esp_codec_dev 范式扩展：**引脚分配以立创 wiki 原理图与器件手册为准**，下面给骨架。

采集链路与播放镜像对称：

1. **建 I2S RX 通道**：`i2s_new_channel(&channel_cfg, NULL, &rx)`（第二参数 TX 给 NULL，第三参数 RX）。
2. **配 std 模式**：和 TX 同样 fs=16000/16bit，但 `din` 接 ES7210 的数据脚，`dout` 未用（引脚以立创 wiki 为准）。
3. **建 ES7210 codec 实例**：`es7210_codec_new`，`codec_mode = ESP_CODEC_DEV_WORK_MODE_ADC`，控制面挂同一 I2C 总线（地址以器件手册为准）。
4. **包成 esp_codec_dev**：`ESP_CODEC_DEV_TYPE_IN`。
5. **open + read** 循环读 PCM。

RMS 能量计算（判断"有没有人说话"的最简算法）：

```
int16_t pcm[BLOCK_FRAMES * 2];   /* 双麦：左右槽位是两路麦 */
esp_codec_dev_read(codec, pcm, count * 2 * sizeof(int16_t));
float sum_sq = 0;
for (unsigned i = 0; i < count; ++i) {
    int16_t mic0 = pcm[2*i];     /* 麦 0 */
    int16_t mic1 = pcm[2*i+1];   /* 麦 1 */
    sum_sq += (float)mic0 * mic0;
}
float rms = sqrtf(sum_sq / count);
ESP_LOGI(TAG, "mic RMS=%.1f", rms);
```

RMS 越大说明能量越高：对着板子说话，RMS 跳动；安静时 RMS 接近本底噪声。这是"语音活动检测"（VAD）的最朴素形态，也是唤醒词的前置门槛。

双麦的意义：两个麦同一时刻采到不同相位的声波，波束成形（beamforming）靠相位差定向声源，回声消除（AEC）靠双通道互相关分离回声。这块板子天生为"小智类语音终端"准备，双麦不是冗余，是算法空间。

五、全双工回环：耳返实验与延迟账

"耳返"=采集到的声音实时从喇叭放出来，像歌手戴耳机听自己。这是音频链路最苛刻的测试：延迟超过 150ms 人耳就能察觉"回声感"，超过 300ms 就没法说话了。

回环结构（E08 扩展，待上板实测）：

```
while (1) {
    esp_codec_dev_read(rx_codec, pcm, BLOCK_FRAMES * 2 * sizeof(int16_t));
    esp_codec_dev_write(tx_codec, pcm, BLOCK_FRAMES * 2 * sizeof(int16_t));
}
```

延迟账（每一段都是延迟来源）：

延迟来源	估算	说明
采集 DMA 缓冲	BLOCK_FRAMES/fs	一块填满才能读出
处理（直通≈0）	≈0	不做 AEC/编码
播放 DMA 缓冲	BLOCK_FRAMES/fs	一块填满才能开始放
codec 内部延迟	~几 ms	ES8311/ES7210 数字滤波器
总往返	~2×BLOCK_FRAMES/fs + 几 ms	

E08 的 `BLOCK_FRAMES=160`（main.c:19），160/16000=10ms，往返约 20ms+：耳返能听见自己但不至于回声感。如果 BLOCK 拉到 512（32ms），往返 64ms+，开始难受。

缓冲与丢样本：采集端缓冲太小（F4 队列 账复用）会溢出丢样本，播放端没数据时出"嗤啦"杂音。容量按"采样率×容忍延迟"算：16kHz×50ms=800 样本，留 2 倍余量 = 1600 样本队列。

六、通往小智：Opus/AEC/唤醒词的位置地图

这条音频链路打通后，"小智 AI 语音助手"的剩下拼图在哪里：

模块	在链路哪一段	干什么
AEC（回声消除）	采集后、编码前	用播放参考信号减掉喇叭回采，避免"自己说的被听进去"
VAD（语音活动检测）	采集后	RMS/神经网络判断"有人说话吗"，没人就不上传省带宽
唤醒词（KWS）	采集后	"小智小智"本地关键词检测，触发上传
Opus 编码	VAD/KWS 后	PCM→Opus 压缩，16kHz×16bit 单声道 → ~10kbps，省上传带宽
网络上传	Opus 后	走 P8 Wi-Fi 的 socket，发到云端 ASR/TTS
Opus 解码	网络下行后	Opus→PCM，喂给 ES8311 播放
TTS（文本转语音）	云端	云端把回答文本转 PCM/Opus

整条链路：麦→ES7210→I2S RX→AEC→VAD/KWS→Opus 编码→Wi-Fi 上传→云端 ASR→LLM→TTS→Opus 下行→Wi-Fi 下载→Opus 解码→I2S TX→ES8311→NS4150B→喇叭。本章打通的是首尾两端（采集+播放），中间的 AEC/VAD/Opus 是"语音终端"的软件层：那是另一个故事，本章只给你"音频管道"的物理基础。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **控制走 I2C，数据走 I2S，codec 是翻译官，功放推喇叭；先滴一声，再听见自己——全双工一回环，小智就在眼前。**

延伸：音频全链路动画见 P12 动画；I2S 与 SPI/I2C 的总线对比见 P6；DMA 搬运在 STM32 的对照见 S8。

实物实验

- E08 音频放音：正弦"滴"→PCM 小音乐片段；
- 耳返实验：采集直通播放，拍手听延迟，缓冲深度与延迟的取舍一手掌握。

常见坑

- **MCLK 没给/给错频率**：codec PLL 锁不住，无声或变调：256×fs 的经典关系先核对。
- **I2S 位宽/声道与 codec 配置不一致**：左右串、杂音、速度错：两边参数对齐表逐项打勾。
- **功放使能脚忘拉高**：I2S 数据在跑，喇叭无声：原理图上 PA_EN 脚是新手盲区。
- **采集缓冲太小溢出**：I2S RX 环形缓冲深度不足丢样本：按"采样率×容忍延迟"算容量（F4 队列账复用）。

短自测

1. I2S 的四根线各是什么？为什么 MCLK 通常是 256×fs？

▼ 看答案
四根线：BCK（位时钟，每 bit 一拍）、WS（声道选择，又叫 LRCK，频率=fs）、DATA（串行数据，MSB first）、MCLK（主时钟，给 codec PLL）。MCLK 是 256×fs 因为 codec 内部 PLL 要从 MCLK 分频出 fs 的所有内部时钟（DAC 转换、滤波器），MCLK 必须是 fs 的高倍整数才能稳锁。256 是历史惯用值（兼容 384/512）。MCLK 没给或给错，codec PLL 锁不住，无声或变调。

2. 为什么音频 codec 要分"控制面（I2C）+ 数据面（I2S）"？能不能只用 I2S？

▼ 看答案
I2C 慢（100k/400k），适合偶发改寄存器（音量/使能/格式）；I2S 快（MHz 级 BCK），适合连续流 PCM。如果控制也走 I2S，每次改音量都要打断数据流，不现实。两套接口各司其职：I2C 当"遥控器"，I2S 当"传送带"。这是音频芯片的通用范式，不是 ES8311 独有。

3. E08 里 stereo 双槽位为什么填同一份单声道样本？不填会怎样？

▼ 看答案
ES8311 是单声道 DAC，但 I2S 配的是 stereo 模式（I2S_SLOT_MODE_STEREO），硬件按左右槽位取数。如果只填左槽位、右槽位留 0，codec 只在一半帧拿到数据，音量减半或失真；如果左右填不同值，codec 取哪个取决于内部路由。填同一份样本保证两槽位等价，codec 任取都对：这是"单声道内容走 stereo 通道"的标准做法。

4. PA_EN 不在 ESP32 的 GPIO 上，而在哪？为什么这样设计？

▼ 看答案
PA_EN 在 PCA9557 I2C 扩展器的 bit1（I2C 地址 0x19），不是 ESP32 GPIO。E08 main.c 第 81 行注释明说"PA_EN 是 PCA9557 bit1，不是 ESP32 GPIO1"。这样设计是因为 ESP32 的 GPIO 资源紧张（音频链路已占了 mclk/bclk/ws/dout 4 个），用 I2C 扩展器省 GPIO：代价是功放使能要走 I2C 总线，比直接 GPIO 慢，但功放开关不频繁，可接受。

5. 耳返实验的延迟从哪里来？BLOCK_FRAMES=160 时往返延迟约多少？

▼ 看答案
延迟来源：采集 DMA 缓冲（一块填满才能读出，BLOCK_FRAMES/fs）+ 播放 DMA 缓冲（一块填满才能放，BLOCK_FRAMES/fs）+ codec 内部数字滤波器延迟（几 ms）。直通处理约等于 0。BLOCK_FRAMES=160、fs=16000 时，单段 10ms，往返约 20ms 加几 ms codec 延迟，约 25ms：人耳能听见自己但不至于回声感（大于 150ms 才明显）。BLOCK 拉到 512（32ms）往返 64ms+，开始难受。

对照表：本章概念 → 仓库与上游落点

概念	落点
I2S 四线协议	飞利浦 I2S 标准；与 S12 SPI 同源（同步移位）
控制面 vs 数据面	P6 I2C/SPI 驱动 I2C 当遥控器；esp_codec_dev 封装
ES8311 播放	E08 工程 main.c；esp_codec_dev 接口
ES7210 采集	立创 wiki 原理图；esp_codec_dev ADC 范式
全双工回环	F4 队列 缓冲账；耳返延迟实测
通往小智	Opus/AEC/KWS 软件层；P8 Wi-Fi 上传
MCLK=256×fs	E08 main.c:95 `I2S_MCLK_MULTIPLE_256`、main.c:126 日志
PA_EN=PCA9557 bit1	E08 main.c:81 注释；I2C 地址 0x19

你做到了

- 板子在你代码下"能听会说"；
- P 篇收官：从 GPIO 到 Wi-Fi 到音频，立创 S3 的每一寸都被你点亮：综合项目（小智类终端）的门槛已经踏平。

带宽对完账：控制面 I2C 只在开工前把 codec 的家什摆好，声音本身一刻不停走 I2S。两条面各司其职，这声"滴"才算落了地；采集那一侧的麦克风，走的也是同一套分工。

✅ P 篇收官。下一站：实验中心 把散落各章的实验串成项目；或回 RTOS 篇 深化系统内功。

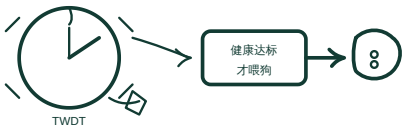
可独立运行的最小实验

ES8311 提示音与旋律 已提供独立工程、配置说明和完整源码。本章仍在建设中，最小实验不依赖本章其余内容；板上现象待实测。

通往单片机之路第 68 章

P13 看门狗与健康监督：不要喂反了

难度 ★★★☆☆ · 约 45 分钟



本章罗盘

问题

任务都在按时让出 CPU 却一直不出活，默认 TWDT 为什么看不见？喂狗条件怎么写才算数？

交付

一个可跑的 health user 监督工程（07-watchdog-health）：锁内读时间/锁外 reset_user 的 supervise 循环、fault 1/2 两条时间线预算（约 t0+2.6~2.8s 复位、监督停喂记 mask 0x80）和复位后 esp_reset_reason + RTC_NOINIT_ATTR 记录回放。

边界

时间线数字是按任务周期推的模型，实际复位时刻与 panic 后的 reset reason 枚举值均『待上板记录』（章末实验记录表当前为空）；只动 TWDT，IWDT 保持默认、RTC_WDT/XTWDT 不涉及；连过 OpenOCD 的板子 IWDT/TWDT 已被关且不再打开，测复位必须重新上电。

依据锚点 `a9` · `f1` · `f2` · `mcu-code` · `mcu-lab` · `mcu-anim`

> 🎯 FreeRTOS 的任务都还在被调度，但采集任务已经半分钟没交过一份结果。TWDT 默认只盯空闲任务，它看得出来吗？

本章精髓

1. **调度正常不等于工作完成**：TWDT 默认只订阅每个核的空闲任务，只能发现"有任务霸占 CPU 不让出"。任务按时让出 CPU、却一直没完成工作，它发现不了。本例额外注册一个名为 health 的 user，只有 S17 的那份 health 模块判定全部按期，监督任务才替它 reset。
2. **条件写反就是没装狗**：`health_supervise` 返回 true 才 `reset_user`；返回 false 表示"该停喂了"。停止监督也不能 `delete_user`，删掉被监视的对象等于取消了超时来源。
3. **跨核共享要真正的锁**：worker、通信、监督三个任务可能在两个核上跑。C 的 volatile 不提供互斥，也不保证一致的快照。`portMUX` 自旋锁保护 health 状态，而且要**拿到锁之后再读时间**。

- **能理解**：为什么注册一个 user 而不是把每个任务都 `esp_task_wdt_add`；为什么 `init` 返回 `ESP_ERR_INVALID_STATE` 时要改用 `reconfigure`；为什么用 OpenOCD 调过的板子要重新上电才能测看门狗。
- **能用**：分别构建普通固件和故障固件；用 `fault` 命令停掉 worker 或监督，按时间预算预测复位时刻，用默认控制台的复位日志核对。

先修

- 必需：P4 中断与双核、F8 FreeRTOS F407 移植、C8 串口协议。
- 建议先读 S17 看门狗：health 模块的期限、宽限期和锁存在那里讲过，本章直接复用。

先跑起来（10 分钟 quick win）

在 IDF 5.5.2 环境里：

```
cd code/esp32/07-watchdog-health
idf.py set-target esp32s3
idf.py build flash monitor -p COMx      # 端口必须显式指定
```

flash 之前先押个盲区：worker 任务回回都乖乖让出 CPU，调度器无可挑剔，可它半分钟没交过一份数据，默认的 TWDT 会叫吗？押会或不会，第一屏日志见分晓。

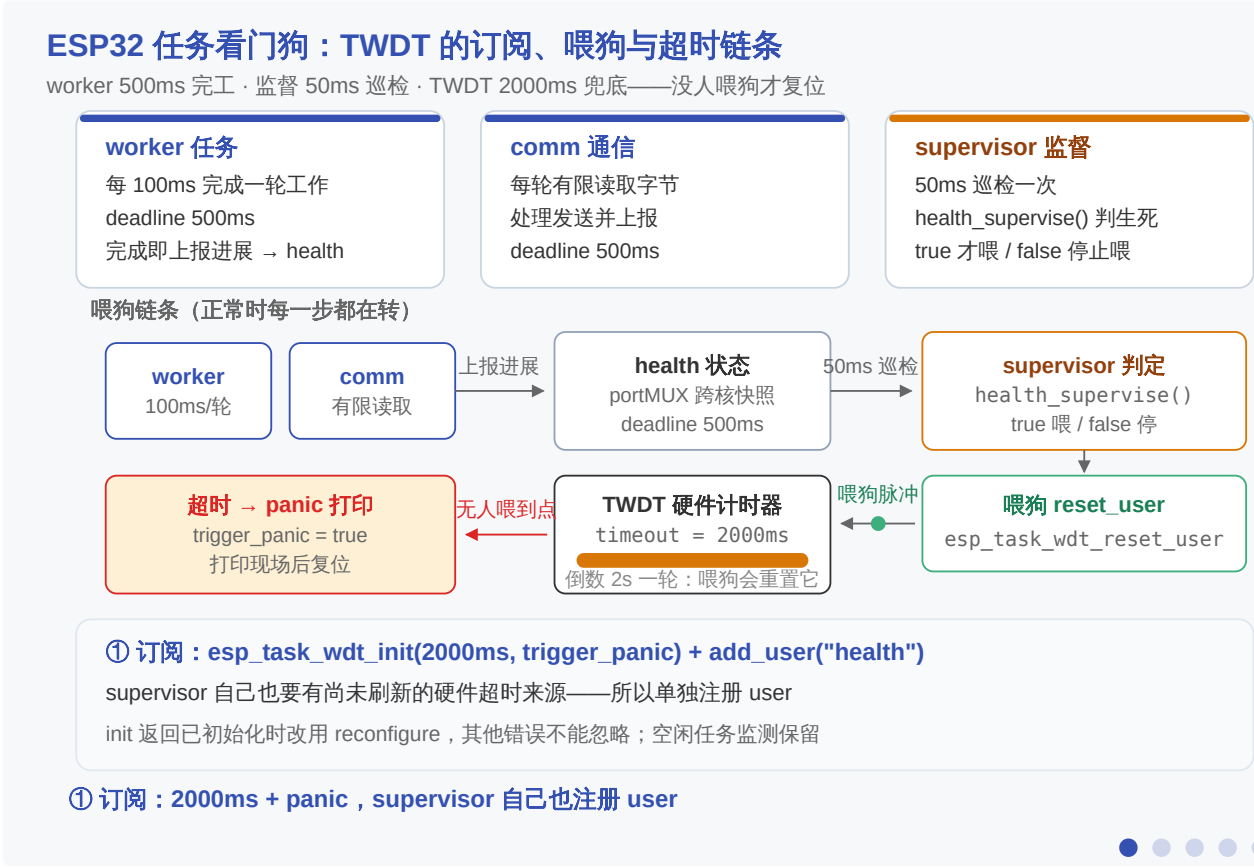
默认控制台第一屏会打出一行 `reset=… retained=0 overdue=0`：普通上电时没有有效的历史记录。接着在仓库根用数据口（GPIO10/11）查状态：

```
python3 scripts/device-console.py status --port COMy
```

隔几秒再查一次，运行毫秒在涨、过期 mask 为 0。主机一个字节都不发，系统也应保持健康。

动画：TWDT 订阅、喂狗与超时链条

worker 上报进展、supervisor 巡检确认、reset_user 喂狗、TWDT 两秒倒数——喂狗脉冲还在闪就没有超时；fault 注入后链条断在哪一环，一图看完。



ESP32 任务看门狗：TWDT 的订阅、喂狗与超时链条

板卡事实

- 立创实战派 S3 的 GPIO2 是板载 I2C 的 SCL。本例不驱动 GPIO2 做"心跳灯"，免得和板上 I2C 器件打架。
- 数据口 UART1：GPIO10 TX、GPIO11 RX，和 C8 相同。文本日志走默认控制台，不混入数据口。
- TWDT 基于定时器组 0 的 MWDT，IWDT 基于定时器组 1 的 MWDT（IDF 5.5.2 Watchdogs 文档）。

小节结构

小节	内容	四件套
一、S3 的四种看门狗	RTC_WDT / IWDT / TWDT / XTWDT 各管什么	配置
二、订阅模型	订阅任务 vs 注册 user；本例为什么只要一个 user	库解析
三、跨核共享	portMUX、锁内读时间、锁外 reset	代码分析
四、初始化	init / reconfigure、三个配置字段、panic 行为	配置
五、故障注入与时间预算	两种故障、两条时间线	代码分析
六、复位原因与 RTC 记录	`esp_reset_reason` + `RTC_NOINIT_ATTR`	代码分析
七、OpenOCD 与看门狗	断点会关掉看门狗，而且不再打开	引脚

一、S3 的四种看门狗

S3 四种看门狗：RTC/IWDT/TWDT/XTWDT 各管一段

看门狗	监督什么	谁喂	本例
RTC_WDT	上电到用户 `app_main` 之前的启动过程	引导代码；默认在进入用户主函数前关闭	不动
IWDT（中断看门狗）	中断是不是被长时间屏蔽、ISR 是不是跑太久	每个核的 FreeRTOS tick 中断	保持默认开启
TWDT（任务看门狗）	订阅的任务/用户是否在期限内报到	被订阅者自己调 `reset` / `reset_user`	**本章主角**
XTWDT	外部 32kHz 晶振是否停振	硬件	未启用

（依据：IDF 5.5.2 *Watchdogs* 文档。）

IWDT 和 TWDT 的分工要分清。IWDT 管的是"中断还能不能及时跑"：临界区太长、ISR 里死等，都会让 tick 中断进不来，触发 IWDT。TWDT 管的是"任务有没有按时完成"。本例故意让故障任务**继续调用 `vTaskDelay` 让出 CPU**，这样触发的一定是 TWDT 的 health user，而不是空闲任务饿死或 IWDT。

二、订阅模型：一个 user，而不是三个任务

TWDT 有两种订阅方式：

- `esp_task_wdt_add(task)`：订阅一个任务，这个任务自己调 `esp_task_wdt_reset()`；
- `esp_task_wdt_add_user("health", &handle)`：注册一个用户，拿到句柄，由谁调 `esp_task_wdt_reset_user(handle)` 都行。

最省事的写法是把 worker 和通信任务直接 `add`，各自 reset。本例没这么做，原因有三个：

1. **每个任务一个期限。** TWDT 只有一个全局超时（这里是 2000ms）。health 模块给每个任务单独设期限（这里都是 500ms，可以各不相同），还有上电宽限期和锁存。
2. **能说出是谁超时。** STATUS 的过期 mask 和 RTC 记录都能告诉主机"是 worker 还是通信"。只靠 TWDT，复位前打出的 backtrace 里才有这个信息。
3. **和 F407 共用一份代码。** `health.c` 在 S17 里驱动 IWDG/WWDG，在这里驱动 TWDT，判定逻辑完全相同，主机上测一次就够了。

那监督任务自己卡死了怎么办？它不再调 `reset_user`，health user 的超时照样到期。所以必须**单独注册一个由监督任务负责的 user**：主管自己出问题时，也要有一个没人刷新的超时来源。

停止监督的故障（`fault 2`）也不能写成 `esp_task_wdt_delete_user`。删掉 user，TWDT 就不再等它，超时来源没了，系统反而不会复位。正确的表现是：user 保持注册，但没人再 reset 它。

三、跨核共享：锁内读时间，锁外喂狗

三个任务都用 `xTaskCreate` 创建，没有绑核，调度器可能把它们放到两个核上同时跑。共享的东西有 `health`、`supervisor`、`service` 和几个故障标志，全部用一个 `portMUX_TYPE` 保护：

```
static portMUX_TYPE mux=portMUX_INITIALIZER_UNLOCKED;
static void progress(uint8_t id){
    portENTER_CRITICAL(&mux);
    health_progress(&health,id,now_ms());   /* 先拿锁，再读时间 */
    portEXIT_CRITICAL(&mux);
}
```

S3 上的 `portENTER_CRITICAL` 做两件事：关掉**当前核**的中断，然后自旋等另一个核释放锁。所以有两条规矩。

锁内要短。 临界区关中断，正好是 IWDT 要抓的东西（"Critical Sections (also disables interrupts)"）。本例最长的一段是通信任务：一轮最多喂 128 字节给接收状态机，再做一次超时检查。这是有界的；不能在锁里调 `uart_read_bytes`、`vTaskDelay` 或任何会阻塞的函数。

时间要在锁里读。 假设 supervisor 写成先读时间、再拿锁：`uint32_t now=now_ms(); portENTER_CRITICAL(&mux); health_evaluate(&health,now);`。读时间和拿锁之间，另一个核可能插进来：

1. supervisor 在核 1 读到 now=1000，还没拿到锁；
2. worker 在核 0 拿到锁，读到 1001，写入 `last[0]=1001`，释放锁；
3. supervisor 拿到锁，算 `now-last[0]` = 1000−1001，无符号减法回绕成 4294967295，大于 500，判成过期；
4. 锁存，停止喂狗，2 秒后一次莫名其妙的复位。

规则是：**所有一方都在拿到锁之后才读时间**。这样锁内的先后顺序就是时间戳的先后顺序：评估时看到的每个 `last` 都不晚于它自己的 `now`，减法不会倒过来。上报进展的一方同样遵守这条，免得被同核更高优先级的任务抢占之后，才把一个早就过时的时间戳写进去。

supervise 循环把"判定"和"喂狗"拆在锁的两边：

```
portENTER_CRITICAL(&mux);
/* 到点的故障生效；判定；更新 STATUS 字段；算出要记录的 mask */
health_result_t r=health_evaluate(&health,now_ms());
bool allowed=!stop_supervisor&&health_supervise(&supervisor,r);
portEXIT_CRITICAL(&mux);
if(allowed)ESP_ERROR_CHECK(esp_task_wdt_reset_user(health_user));
else if(!recorded){fault_record_save(&retained,mask);recorded=true;}
```

`esp_task_wdt_reset_user` 内部有自己的锁，放在我们的临界区里会嵌套自旋，还会拉长关中断的时间，所以放到锁外。锁内算好的 `allowed` 是一个局部快照，出锁后再用它不会有竞争。`ESP_ERROR_CHECK` 包住它：句柄无效之类的错误不能静默忽略。

至于 volatile：它只保证编译器每次都真的去内存读写，不提供互斥，也不保证几个字段一起读出来是同一时刻的值。本例的共享状态都在锁内访问，不需要 volatile；只有 RTC 记录用 volatile，因为它要在复位前后"真的写进 RTC 内存"。

四、初始化：init 失败不一定是错

```
esp_task_wdt_config_t wdt={
    .timeout_ms=2000,
    .idle_core_mask=(1U<<portNUM_PROCESSORS)-1U,   /* S3 双核：0b11 */
    .trigger_panic=true,
};
esp_err_t err=esp_task_wdt_init(&wdt);
if(err==ESP_ERR_INVALID_STATE)err=esp_task_wdt_reconfigure(&wdt);
ESP_ERROR_CHECK(err);
ESP_ERROR_CHECK(esp_task_wdt_add_user("health",&health_user));
```

字段	本例	含义
`timeout_ms`	2000	所有订阅者共用的超时
`idle_core_mask`	0b11	两个核的空闲任务都订阅，保留"有人霸占 CPU"的检测
`trigger_panic`	true	超时进入 panic；默认 false 时只打印警告和回溯，程序**继续运行**

`CONFIG_ESP_TASK_WDT_INIT` 默认开启，IDF 启动时已经初始化过 TWDT。这时再调 `esp_task_wdt_init` 会返回 `ESP_ERR_INVALID_STATE`，意思是"已经初始化了"，不是硬件坏了。正确的处理是改用 `esp_task_wdt_reconfigure` 换成我们的配置。其他错误码一律交给 `ESP_ERROR_CHECK`。

`trigger_panic=true` 之后的动作由 panic 配置决定。本例 `sdkconfig.defaults` 设了 `CONFIG_ESP_SYSTEM_PANIC_PRINT_REBOOT=y`：打印寄存器和回溯，然后重启。生产环境一般就是这个选项；调试时可以改成停住等 GDB。

初始化顺序也有讲究：先 `health_init`（宽限期 1000ms 从这一刻算），再配 TWDT、注册 user，最后才创建三个任务。user 注册到监督任务第一次 reset 之间有一小段空档，2000ms 的超时足够覆盖。

五、故障注入与时间预算

故障命令只在测试固件里存在。故障固件用独立的构建目录和 sdkconfig，合并 `sdkconfig.defaults.faults`（打开 `CONFIG_RELIABILITY_TEST_FAULTS`）：

```
idf.py -B build-faults -D SDKCONFIG=build-faults/sdkconfig \
  -D "SDKCONFIG_DEFAULTS=sdkconfig.defaults;sdkconfig.defaults.faults" build
idf.py -B build-faults -p COMx flash monitor
```

普通固件收到 TEST_FAULT 会回"不支持"，不会因为一个误发的命令重启。

`fault --fault 1 --task 0`：worker 停止上报进展。 设主机发出命令的时刻为 t0，各任务周期：worker 100ms、supervisor 50ms、TWDT 2000ms。

时刻	事件
t0	服务层先把应答排队，再记下 `fault_at=now+200`
t0+200～250ms	supervisor 的下一轮发现到点，置 `pause_mask` bit0
t0+100～250ms	worker 最后一次成功上报（取决于它的 100ms 周期落在哪）
t0+600～800ms	`now-last[0] >= 500`，判定过期，锁存，停止 `reset_user`，写 RTC 记录
约 t0+2.6～2.8s	距最后一次 `reset_user` 满 2000ms，TWDT 超时 → panic → 重启

这里的数字是按代码周期推的模型，实际值待上板记录。两个时间尺度要叠加着看：500ms 期限决定"多快判定"，2000ms TWDT 超时决定"判定后多久复位"。想让复位更快，该缩短的是 TWDT 超时，而不是任务期限。

`fault --fault 2`：监督任务自己停止喂狗。 生效后 `allowed` 恒为 false，记录的 mask 是 0x80（不是任何任务位，表示"监督被停"）。最后一次 reset 发生在生效前的那一轮，复位大约在 t0+2.15～2.25s。这个故障模拟的是"主管自己出问题"：没有任何任务过期，TWDT 照样复位。

STATUS 里 `required` 恒为 0x03，`overdue` 单独一个字节。主机看到 `overdue=0x01` 就知道是 worker；复位后 `retained=1 overdue=0x1` 是同一个结论在复位后的回放。

最后分清两类错误：串口上收到坏帧、CRC 不对、帧超时，是协议层的事，由 C8 的重同步和计数器处理，不该让系统重启。看门狗只处理"失去进展"。

六、复位原因与 RTC 记录

`app_main` 一开头读 `esp_reset_reason()`，然后决定要不要信 RTC 里的记录：

```
uint32_t reset=(uint32_t)esp_reset_reason(),previous=0;
bool valid=fault_record_take(&retained,
    reset==ESP_RST_TASK_WDT||reset==ESP_RST_INT_WDT||reset==ESP_RST_PANIC,&previous);
```

`RTC_NOINIT_ATTR` 把 `retained` 放进 RTC 内存里不被启动代码清零的区域。软件复位和看门狗复位后内容还在；**冷上电后内容是随机的**。所以记录的有效性要过三关：

1. **复位原因对得上**：只有 TWDT、IWDT、panic 这三类暖复位才去读，冷上电直接不信；
2. **格式和校验对得上**：magic `0x57444731`、版本 1、CRC-16 全部正确（和 S17 是同一份 `record.c`）；
3. **读完就作废**：`fault_record_take` 读完立刻把 magic 第一个字节清零，下一次无关的暖复位不会把旧证据再报一遍。

写入顺序也是为复位准备的：先清 magic，再写内容和 CRC，最后写 magic。写到一半复位，读出来的 magic 不对，记录就当不存在。

TWDT 触发 panic 后，`esp_reset_reason()` 在这块板上到底报 `ESP_RST_TASK_WDT` 还是 `ESP_RST_PANIC`，日志里的原始数值待上板记录；代码对两种都接受。日志打印的是原始枚举值，便于对照 `esp_system.h`。

七、OpenOCD 与看门狗

IDF 文档写得很明确：用 OpenOCD 调试时，每次停在断点，OpenOCD 都会**关掉 IWDT 和 TWDT 的硬件定时器**，离开断点后**也不会再打开**。只要通过 JTAG 连过 OpenOCD，这两个看门狗就等于没装，不会有任何警告或 panic。

和 F407 的冻结位是一个道理：用调试器连过的板子上测"会不会复位"，"不复位"说明不了任何问题。固定流程是：烧录 → 关掉 OpenOCD → 板子重新上电 → 再发故障命令。

附录：工程完整源码

<<< ../../code/esp32/07-watchdog-health/main/main.c

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **TWDT 默认只看空闲任务；注册一个 health user，判定通过才 reset，停喂不删 user；锁里读时间、锁外喂狗；init 报已初始化就 reconfigure；连过 OpenOCD 必须重新上电。**

实物实验

- **装备**：立创实战派 S3、USB 线（默认控制台）、3.3V USB-TTL 接 GPIO10/11。
- **实验 1（worker 过期）**：故障固件，重新上电后执行 `fault --fault 1 --task 0 --port COMy`，在默认控制台记录 panic 打印的时刻和复位后的 `reset=… retained=1 overdue=1`，和第五节的 t0+2.6～2.8s 对照。
- **实验 2（监督停喂）**：`fault --fault 2`，复位后 overdue 应为 0x80，复位时刻应比实验 1 早约 0.5s。
- **实验 3（冷上电）**：拔掉 USB 再插上，日志应为 `retained=0`，证明冷上电不会误用 RTC 里的旧内容。

实验记录

模型/编译验证不等于上板实测。当前待上板；请记录日期、板版本、固件版本、接线、输入、原始输出与结论。

日期	板卡/版本	故障输入	原始结果	结论
待实测				

常见坑

- **只靠默认的空闲任务订阅**：任务按时 `vTaskDelay`、却一直没完成工作，TWDT 永远不叫（第二节）。
- **`trigger_panic` 没开**：TWDT 超时只打印一行警告，程序接着跑，以为看门狗复位过了（第四节）。
- **把 `ESP_ERR_INVALID_STATE` 当致命错误**：IDF 已经自动初始化过 TWDT，应该 reconfigure（第四节）。
- **停止监督时 `delete_user`**：超时来源没了，系统永远不复位（第二节）。
- **锁外读时间**：跨核插队后无符号减法回绕，误判过期（第三节）。
- **临界区里阻塞**：关中断太久，触发的是 IWDT 而不是你想测的 TWDT（第三节）。
- **连着 OpenOCD 测复位**：看门狗已被关掉（第七节）。

短自测

1. 把 worker 和通信任务直接 `esp_task_wdt_add`、各自 `esp_task_wdt_reset()`，能不能替代本例的设计？丢了什么？

> **▼ 参考答案**
> 能发现"任务卡死"，但丢了三样：每个任务各自的期限（只剩 2000ms 一个全局超时）、上电宽限期和锁存、以及复位后"是哪个任务"的记录（STATUS 的过期 mask 和 RTC 记录）。也没法和 F407 共用 health.c。

2. 监督任务里把 `if(allowed)` 写成 `if(!allowed)`，会发生什么？

> **▼ 参考答案**
> 健康时不喂，系统上电约 2 秒就复位，反复重启；真出故障时反而一直在喂，永远不复位。条件写反等于把看门狗反着装。

3. 为什么 fault 2 记录的 mask 是 0x80，而不是 0？

> **▼ 参考答案**
> 0 会和"没有任何任务过期"混在一起。0x80 不对应任何任务位，专门表示"监督自己被停"，主机复位后看到它就知道不是 worker 或通信任务的问题。

4. 冷上电后 RTC 里的随机内容恰好 magic 和 CRC 都对，会被误报吗？

> **▼ 参考答案**
> 不会。第一关是复位原因：冷上电的原因不是 TWDT/IWDT/panic，`fault_record_take` 直接返回 false，根本不看内容。CRC 是第二道防线，防的是暖复位时写到一半的记录。

对照表：本章概念 → 仓库与文档落点

概念	落点
进展、期限、宽限期、锁存	`code/common/reliability/health.c`
三个任务、portMUX、supervise 循环	`code/esp32/07-watchdog-health/main/main.c`
TWDT 配置与 user 接口	IDF 5.5.2 `esp_task_wdt.h`：`esp_task_wdt_init/reconfigure/add_user/reset_user`
四种看门狗、OpenOCD 行为	IDF 5.5.2 *Watchdogs* 文档（API Reference → System）
暖复位记录	`code/common/reliability/record.c`；`RTC_NOINIT_ATTR`（`esp_attr.h`）
panic 后重启	`sdkconfig.defaults`：`CONFIG_ESP_SYSTEM_PANIC_PRINT_REBOOT=y`
故障固件	`sdkconfig.defaults.faults`：`CONFIG_RELIABILITY_TEST_FAULTS=y`

延伸阅读

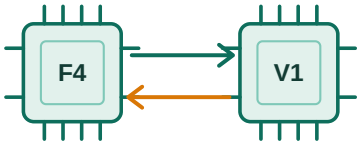
- **[D12]** Ganssle《Great Watchdog Timers for Embedded Systems》：任务看门狗该由谁喂、喂狗条件怎么写，才能真的反映"系统健康"。

你做到了

- 能说清 TWDT 默认看什么、看不到什么，以及为什么要加一个 health user；
- 跨核共享状态会用 portMUX，知道时间戳为什么要在锁里读；
- 能按 500ms + 2000ms 两段预算预测复位时刻，并用 RTC 记录在复位后回放原因。

不叫的原因：TWDT 默认看的是出勤，不是产出。任务按时让位它就满意，活干没干完它管不着；所以这章给 health 单写了一条新规矩：按期交结果，才有人替它 reset。狗要盯的是产出，这把尺子比默认配置严得多。

✅ 下一站：J2 S3 数据记录器——把协议、健康监督和看门狗装进一个完整的采集项目。



PART PG

第八部分 GD32 双系对照

G 篇把 GD32F4xx 与 STM32F407 逐字段对照，V 篇用 GD32VF103 丈量 RISC-V 内核、CLIC 中断与 MTIME。迁移能力就是在对照里练出来的。

> 国产化迁移不是「换头文件」：G 篇把 GD32F4xx 与 STM32F407 逐字段对账：同名不同义的位、多出来的串口、200MHz 的分频账；V 篇用 GD32VF103 丈量 RISC-V：工具链、启动文件、CLIC 中断、MTIME 时基整套换血。对照是练迁移能力最狠的方式：差异表就是迁移清单。读完的标志：拿到任何「类 STM32」芯片，知道先对哪几笔账。

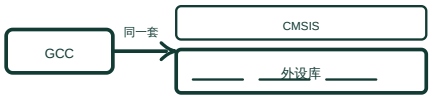
本部 10 章 · 约 385 分钟（6.4 小时）

69 . G0 环境与工具链：复用一套 GCC，解剖官方库 V3.3.3
70 . G1 RCU 时钟树：200MHz 是怎么算出来的
71 . G2 GPIO 与 AF 复用对照：七大寄存器同名不同姓
72 . G3 USART 增强点：从命名到数量的差异清单
73 . G4 GD32 外设差异：有没有与一不一样
74 . V0 RISC-V 工具链与启动文件
75 . V1 Bumblebee 内核与 CLIC 中断
76 . V2 RCU 与 108MHz
77 . V3 MTIME 与裸机延时
78 . V4 GPIO 最小系统

通往单片机之路第 69 章

G0 环境与工具链：复用一套 GCC，解剖官方库 V3.3.3

难度 ★★☆☆☆ · 约 25 分钟



本章罗盘

问题

GD32 与 STM32 同内核，工具链真能一字不改复用吗？官方库 V3.3.3 那堆文件各归哪层？

交付

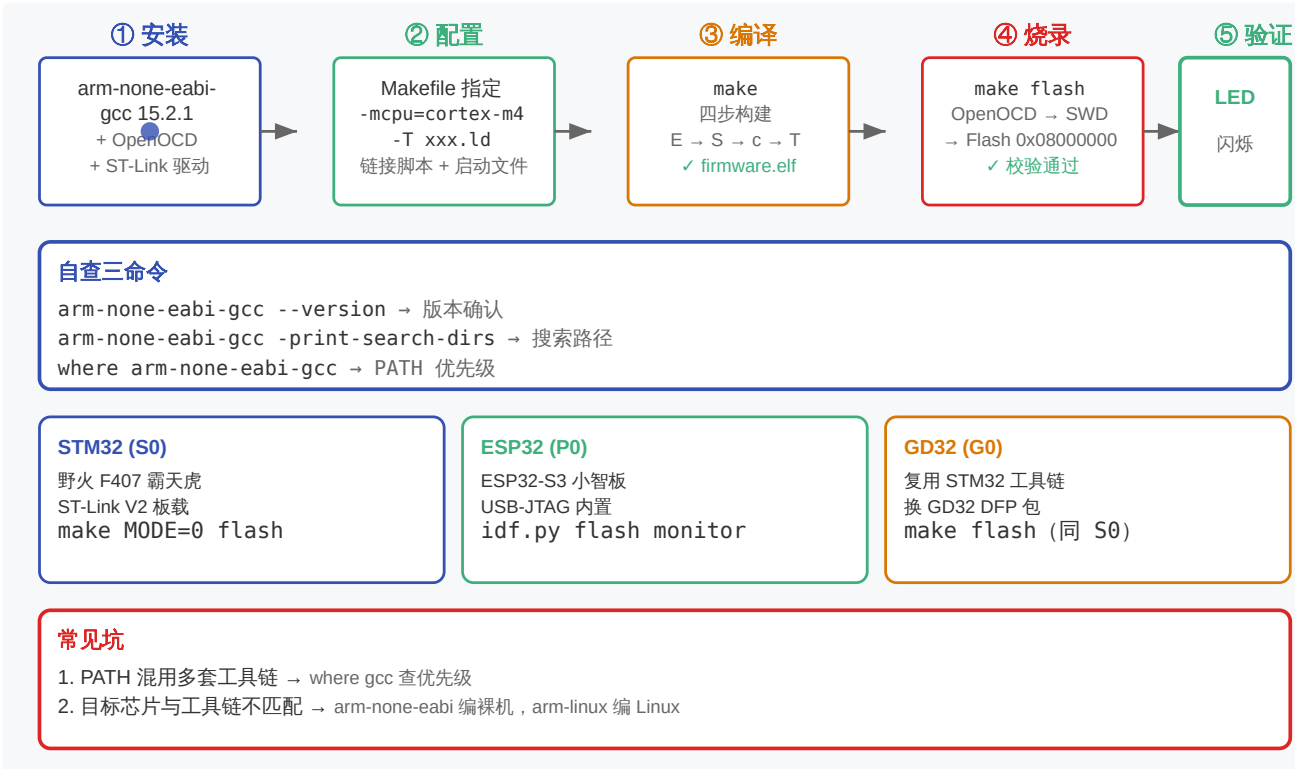
一次 make 编过 G1 工程的记录（证明 xPack 工具链零增量），加一张官方库两层结构的文件归类表。

边界

全部结论停在宿主机「编过」这一层，未上板；固件包顶层目录名待官方包核对，F470 容量事实出自 Product Brief 而非数据手册；链接官方库 .c 的库函数级路线不在本章范围。

依据锚点 `f6 · a12 · mcu-code · mcu-anim`

🎯 STM32F407 你装过 xPack arm-none-eabi-gcc——好消息：GD32F4xx 也是 Cortex-M4F，同一套交叉编译器一个字都不用改就能编。要换的只有"事实字典"：把 ST 的 CMSIS 头文件换成兆易创新的官方固件库 V3.3.3。本章就把这本字典的目录拆给你看。



通用环境搭建五步闭环：安装→配置→编译→烧录→点灯

本章精髓

1. **工具链零增量**：GD32F4xx 是 ARM Cortex-M4F，交叉编译旗 `-mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard` 与 STM32F407 **逐字相同**（B0 已装好的 xPack arm-none-eabi-gcc 15.2.1 直接复用）；只需多一个 `-DGD32F450` 选芯片型号。

2. **官方库 V3.3.3 是两层结构**：上层是 **CMSIS 设备层**（`gd32f4xx.h` + `system_gd32f4xx.c` + 启动模板），下层是 **标准外设库层**（`gd32f4xx_rcu.h/.c`、`gd32f4xx_gpio.h/.c` ……每个外设一对）。所有头文件统一标 `version 2026-02-05, V3.3.3`。
3. **本站 GD32 工程是"寄存器级"，不 `-I` 官方库**：官方 `.h` 当**事实字典**读（查偏移、查位名），main.c 里手写 `*(volatile uint32_t*)` 访问，和 S 篇同款姿势。官方库源码缓存于 `.trellis/ref/gd32/`，不随仓库分发。

- **能理解**：为什么 GD32 不需要装新编译器（同为 Cortex-M4F，ABI 一致）；官方库 `.h` 与 `.c` 各自的角色。
- **能用**：拿到一份 GD32F4xx 工程，能指出哪个文件来自 CMSIS 设备层、哪个来自标准外设库层，并自己加一个外设的头文件进来。

先修

- 无强制先修；建议先读 B0 工具链全景 把 xPack arm-none-eabi-gcc 装好，再看 S0 环境 的安装步骤对照。

先跑起来（10 分钟 quick win）

确认你电脑上那套 xPack 工具链能编 GD32。在仓库根目录跑：

```
arm-none-eabi-gcc --version    # 应显示 15.2.1（xPack，与 CI 同源）
cd code/gd32/01-rcu-clock && make    # G1 工程，编过就说明工具链对 GD32 也通
```

先押个开箱单：行李是零，三件套还是三张老面孔。谁管向量表、谁把时钟树拉到 200MHz、谁划 Flash 和 SRAM 的地盘？押完再敲 make。

`make` 走完应该看到 `arm-none-eabi-size` 打印 text/data/bss 三列体积，和 S 篇工程长得一模一样。这一条证明：**编 STM32 和编 GD32，是同一套 gcc、同一组编译旗，只是源码和链接脚本换了**。

小节结构

小节	内容
一、工具链复用	同一颗 Cortex-M4F，编译旗逐字相同；`-DGD32F450` 选型号
二、官方库两层结构	CMSIS 设备层 + 标准外设库层；目录解剖
三、三件套：启动 / system / 链接脚本	startup_gd32f4xx.s、system_gd32f4xx.c、gd32f4xx.ld
四、寄存器级工程怎么"用"官方库	.h 当字典读，main.c 手写访问

一、工具链复用：同一套 GCC 编两颗芯

GD32F4xx（F450/F470）的内核是 **ARM Cortex-M4F**，和 STM32F407 同一个内核、同一套 FPU（FPv4-SP-D16）、同一套 AAPCS 硬浮点 ABI。所以 B0 装好的 xPack arm-none-eabi-gcc 15.2.1，编 GD32 不用动一个字节。

实证看 G1 工程的 Makefile（`code/gd32/01-rcu-clock/Makefile:8-11`）：

```
CC      := arm-none-eabi-gcc
CFLAGS  := -mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard \
           -g3 -O1 -Wall -Wextra -std=c11 -ffreestanding -fno-builtin \
           -DGD32F450 \
           -Wa,--noexecstack
```

把这套旗和 S 篇工程并排看：`-mcpu/-mthumb/-mfpu/-mfloat-abi` 四件**完全相同**（这四件描述的是"ARM Cortex-M4F 这个内核"，与芯片厂无关）。唯一的 GD32 专属是 `-DGD32F450`：它是一个**型号选择宏**，官方 `gd32f4xx.h` 顶部根据它挑出 Flash/RAM 容量、中断向量条数等型号相关参数。

> **【注】为什么 GD32 不需要"GD32 专用编译器"？** 交叉编译器的 target 是 **ARM 体系**（`arm-none-eabi-` 的 `arm`），不是某家芯片厂。Cortex-M4F 是 ARM 设计的内核，STM32F407 和 GD32F450 都买这同一个内核，所以同一套 gcc 都能编。芯片厂的差异在外设（RCU/GPIO/USART 寄存器布局），那部分由 **CMSIS 设备头文件** 描述，不归编译器管。这是"学透一颗再举一反三"能成立的底层原因。

工具链自查纪律照搬 B0：`arm-none-eabi-gcc --version` 看版本、`-print-search-dirs` 看搜索路径、`where arm-none-eabi-gcc`（Windows）看 PATH 优先级。本站 CI 的 arm-host job 与本地同源 xPack 15.2.1，本地能编过 CI 就能编过。这条对 STM32 和 GD32 一视同仁。

二、官方库 V3.3.3 两层结构：目录解剖

兆易创新放出的 **GD32F4xx Firmware Library V3.3.3** 是一个标准固件包，分两层。本站参考缓存 `.trellis/ref/gd32/` 收录的是这两层的关键文件子集（按版本核对，`@10d02f4`）：

```
GD32F4xx_Firmware_Library_V3.3.3/
├── Firmware/
│   ├── CMSIS/
│   │   └── GD/GD32F4xx/
│   │       ├── Include/                  ── 设备头：gd32f4xx.h、system_gd32f4xx.h
│   │       └── Source/
│   │           ├── system_gd32f4xx.c     ── 时钟树/系统初始化（G1 全章解剖）
│   │           └── Templates/            ── 启动文件：gcc/startup_gd32f4xx.s 等
│   └── GD32F4xx_standard_peripheral/
│       ├── Include/                      ── gd32f4xx_rcu.h、gd32f4xx_gpio.h、gd32f4xx_usart.h …
│       └── Source/                       ── gd32f4xx_rcu.c、gd32f4xx_gpio.c …
└── Examples/                             ── 外设示例（G1 提到全量约 3295 个文件）
```

两层分工：

层	代表文件	角色	本站用法
CMSIS 设备层	`gd32f4xx.h` 、`system_gd32f4xx.c`	描述"这颗芯片"：基址、中断号、寄存器总线挂载；`system_*.c` 是上电后第一段 C 代码（时钟树初始化）	`gd32f4xx.h` 是**事实总字典**（G1 的 RCU 基址、G2 的 GPIO 基址全从这里查）
标准外设库层	`gd32f4xx_rcu.h/.c`、`gd32f4xx_gpio.h/.c`	每个外设一对：`.h` 给寄存器位定义 + 库函数原型，`.c` 给库函数实现	`.h` 当位域字典读；`.c` 看官方"标准做法"对照（G1 第七节就是逐行对照 `system_gd32f4xx.c`）

版本取证：每个 `.h` 顶部都印一行 `version 2026-02-05, V3.3.3, firmware for GD32F4xx`（见 `gd32f4xx_gpio.h:5`），这是本站"按版本核对"的锚点。

> **【注】目录路径以官方包 README 为准。** 上面这棵树是 GD32 固件库 V3.3.3 的**标准包布局**：文件夹名（`Firmware/CMSIS/GD/GD32F4xx/...`）遵循兆易创新一贯的打包约定。本站仓库不随包分发，只在 `.trellis/ref/gd32/` 缓存了关键 `.h/.c` 用于事实核对；要完整跑官方 Examples，需自行从兆易创新官网下载 V3.3.3 包。具体顶层文件夹名如有更新，以官方包 README 为准（**待官方包核对**）。

三、三件套：启动文件、system 文件、链接脚本

把一颗 GD32F4xx 从复位跑到 `main()`，最少要三个文件，本站 G1 工程三个都齐了：

文件	来源	干什么
`startup_gd32f4xx.s`	官方 `Templates/` 的 GCC 版（本站按教学精简）	向量表（16 内核槽 + 82 外设槽）+ Reset_Handler：开 FPU → 拷 `.data` → 清 `.bss` → `main`
`system_gd32f4xx.c`	官方 CMSIS 设备层	`SystemInit()`：上电后第一段 C，配时钟树到 200MHz（G1 全章解剖）
`gd32f4xx.ld`	自写（参考官方 GCC 链接脚本）	内存映射：Flash 起始 0x08000000、SRAM 起始 0x20000000、堆栈位置

- **启动文件**：官方 `Templates/` 同时给 **ARM（Keil）/IAR/GCC** 三套启动文件，因为这三家工具链的汇编语法、段名约定不同。本站用 GCC 版（`.s` 用 GNU 汇编语法，`.syntax unified`、`.thumb_func`）。G1 的 `startup_gd32f4xx.s` 在官方基础上**精简为弱符号 + Default_IRQHandler**（向量槽统一填弱符号，应用可同名覆盖，见 `code/gd32/01-rcu-clock/startup_gd32f4xx.s:30-33`）。
- **system 文件**：这是 GD32 与 STM32 的一个关键差异点：`system_gd32f4xx.c` 里 `SystemInit()` 把 PLL 直接拉到 **200MHz**（F450 档），而 ST 的 `system_stm32f4xx.c` 默认留在 168MHz。G1 整章就是解剖这个文件，这里只点名它的存在。
- **链接脚本**：`gd32f4xx.ld` 描述 GD32F4xx 的 Flash/RAM 地址范围。Flash 基址 `0x08000000`、SRAM 基址 `0x20000000`，这俩是 ARM Cortex-M 的约定，和 STM32F4 相同；具体容量（F450/F470 不同）由 `-DGD32F450` 选定的型号决定。链接脚本的写法和 B3 链接脚本 同款，只是地址表换成了 GD32 的。

四、寄存器级工程怎么"用"官方库

本站 GD32 工程和 S 篇一样是**寄存器级**：不调官方库函数、不 `-I` 官方库头文件，main.c 自己手写 `*(volatile uint32_t*)` 访问寄存器。那官方库拿来干嘛？**当事实字典读**。

以 G1 工程为例（`code/gd32/01-rcu-clock/main.c:23-29`）：

```
/* ================= 基地址（gd32f4xx.h @10d02f4） ================= */
#define AHB1_BUS_BASE   0x40020000U
#define RCU_BASE        (AHB1_BUS_BASE + 0x3800U)   /*!< RCU（gd32f4xx.h:343） */
#define FMC_BASE        (AHB1_BUS_BASE + 0x3C00U)   /*!< FMC（gd32f4xx.h:344） */
#define PMU_BASE        (APB1_BUS_BASE + 0x7000U)   /*!< PMU（gd32f4xx.h:330） */
```

每个 `#define` 后面都挂了**凭证行号**：`gd32f4xx.h:343` 是 RCU 基址的出处。写代码的人打开 `.trellis/ref/gd32/gd32f4xx.h`，跳到第 343 行，抄下 `RCU_BASE = AHB1_BUS_BASE + 0x3800`，再回到 main.c 写宏。**官方 `.h` 是字典，main.c 是作文**——字典不进作文，但作文每个字都得查过字典。

这套"读字典、手写访问"的姿势带来三个好处：

1. **事实可核验**：每个魔法数字都挂了头文件行号，读者一条条能对得上（G1 的"事实来源"表就是这么来的）。
2. **不绑死官方库版本**：V3.3.3 升到 V3.4，库函数签名可能变，但寄存器偏移不会变，寄存器级代码跨版本稳。
3. **对照 STM32 一目了然**：手写 `RCU_AHB1EN` 和手写 `RCC_AHB1ENR` 并排看，"同名不同姓"立刻显形（这正是 G1/G2 的写法）。

> 【注】什么时候才真的链接官方库 `.c`？当你想要库函数的便利（`gpio_af_set()` 一行配好 AF）、又不在乎版本绑定时，把 `gd32f4xx_gpio.c` 加进 `SRCS`、把 `Include/` 加进 `-I` 即可。本站教学工程为了"寄存器级透明"选择不链接，但 G2 会把官方 `gpio_mode_set` / `gpio_af_set` 的实现拆给你看，证明它就是我们手写那几行的 for 循环封装。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **工具链零增量（同 Cortex-M4F）、官方库两层（CMSIS 设备层 + 标准外设库层）、三件套（启动/system/链接脚本）；本站工程读 `.h` 当字典、手写访问，魔法数字挂行号。**

延伸：环境搭建五步闭环动画见 G0 动画；STM32 工具链对照见 B0；SPL 库结构解剖见 S15。

实物实验

- **quick win 复现**：`cd code/gd32/01-rcu-clock && make`，看到 `size` 打印三列体积，证明 xPack 工具链对 GD32 通。
- **目录自检**：打开 `.trellis/ref/gd32/`，数一数 CMSIS 设备层文件（`gd32f4xx.h`、`system_gd32f4xx.c`）与标准外设库文件（`gd32f4xx_rcu.h`、`gd32f4xx_gpio.h`、`gd32f4xx_fmc.h` ……）各几个，印证两层结构。
- **型号对照（选做）**：把 Makefile 的 `-DGD32F450` 暂改成 `-DGD32F470`，看 `gd32f4xx.h` 顶部 `#ifdef` 分支怎么挑容量。型号选择宏已实证：`gd32f4xx.h:46` 列出 6 种型号（F450/F405/F407/F470/F425/F427），F450 与 F470 共享容量分支（:171-213）；F470 系列容量已按官方 Product Brief 实证：200MHz、Flash 512~3072KB、SRAM 256~768KB（最高配 3MB Flash + 768KB SRAM）。

常见坑

- **以为编 GD32 要装"GD32 专用工具链"**：不需要。GD32F4xx 是 Cortex-M4F，与 STM32F407 同内核同 ABI，xPack arm-none-eabi-gcc 原样复用，只多一个 `-DGD32F450`。
- **拿 STM32 的 CMSIS 头文件去编 GD32**：基址凑巧相同（GPIOF 都是 0x40021400），但 RCU 叫 RCU、寄存器名也不同，用错头文件会编过但跑飞。GD32 工程必须用 `gd32f4xx.h`。
- **把官方库 `.c` 全链接进来却说"寄存器级"**：本站教学工程不 `-I`、不链接官方库 `.c`，main.c 手写访问。链接官方库是另一条路线（库函数级），别混着说。
- **漏 `-DGD32F450`**：不定义型号宏，`gd32f4xx.h` 顶部的容量/中断分支拿不到默认值，编译可能报错或编出错的向量表。
- **照抄官方 `system_gd32f4xx.c` 的 `while(1)` 死等**：晶振不来就死机，G1 已点名这是官方 demo 的坏习惯，教学工程要改成超时回退。

短自测

1. 编 GD32F450 需要换一套新的交叉编译器吗？为什么？
2. 官方固件库 V3.3.3 的两层结构各叫什么？各举一个代表文件。
3. G1 工程的 Makefile 里，哪一组编译旗和 STM32 工程逐字相同？哪一个是 GD32 专属？
4. 本站 GD32 工程是"寄存器级"，那官方库 `.h` 文件在这个工程里扮演什么角色？
5. `system_gd32f4xx.c` 和 ST 的 `system_stm32f4xx.c` 在"上电默认频率"上有什么差别？

▼ **参考答案（先自己想完再展开）**

1. 不需要。GD32F4xx 是 ARM Cortex-M4F，与 STM32F407 同内核、同 FPU（FPv4-SP-D16）、同硬浮点 ABI。xPack arm-none-eabi-gcc 的 target 是 ARM 体系（`arm-none-eabi-` 的 `arm`），与芯片厂无关。芯片厂差异在外设寄存器，由 CMSIS 设备头文件描述，不归编译器管。 2. 两层：**CMSIS 设备层**（代表 `gd32f4xx.h`、`system_gd32f4xx.c`，描述"这颗芯片"：基址、中断号、上电时钟初始化）与**标准外设库层**（代表 `gd32f4xx_rcu.h/.c`、`gd32f4xx_gpio.h/.c`，每个外设一对，给寄存器位定义 + 库函数）。 3. `-mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard` 与 STM32 工程逐字相同（这四件描述 Cortex-M4F 内核，与芯片厂无关）。GD32 专属的是 `-DGD32F450`：型号选择宏，挑 Flash/RAM 容量与中断向量条数。 4. 当**事实字典**读：写代码时打开 `.trellis/ref/gd32/gd32f4xx.h`，查寄存器偏移与位名，抄进 main.c 的 `#define`，并在注释里挂头文件行号凭证。`.h` 不进编译（不 `-I`），main.c 手写 `*(volatile uint32_t*)` 访问。 5. GD32 的 `system_gd32f4xx.c` 里 `SystemInit()` 把 PLL 直接拉到 **200MHz**（F450 官方默认档，PSC25/N400/P2/Q9）；ST 的 `system_stm32f4xx.c` 默认留在 **168MHz**。另外 GD32 官方文件**不设 Flash 等待周期**（FMC_WS 全文检索零命中），这是 G1 第四节点名的"官方留白"。

对照表：本章概念 → 仓库落点

本章说的	仓库里哪一行
工具链零增量、`-DGD32F450`	code/gd32/01-rcu-clock/Makefile:5-11
启动文件（GCC 版向量表 + Reset_Handler）	code/gd32/01-rcu-clock/startup_gd32f4xx.s:15-40
链接脚本（Flash 0x08000000 / SRAM 0x20000000）	code/gd32/01-rcu-clock/gd32f4xx.ld
寄存器级手写访问 + 行号凭证	code/gd32/01-rcu-clock/main.c:23-35
官方库 V3.3.3 版本锚	.trellis/ref/gd32/gd32f4xx_gpio.h:5（`V3.3.3`）
CMSIS 设备头（基址总线挂载）	.trellis/ref/gd32/gd32f4xx.h:305-357（@10d02f4）
system 文件（200MHz 时钟树初始化）	.trellis/ref/gd32/system_gd32f4xx.c:936-1003（G1 全章解剖）

你做到了

- 工具链从"还要再装一套吗"变成"零增量复用"：GD32F4xx 和 STM32F407 是同一个 Cortex-M4F 内核；
- 官方固件库 V3.3.3 的两层结构（CMSIS 设备层 + 标准外设库层）不再是一坨乱文件，每个 `.h` 都能归类；
- 知道本站 GD32 工程是"读 `.h` 当字典、手写访问"，下一章 G1 的每个魔法数字你都能在 `.trellis/ref/gd32/` 里查到出处。

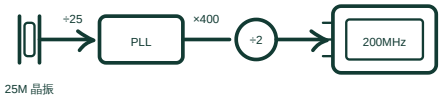
开箱对上：startup_gd32f4xx.s 管向量表，Reset_Handler 里开 FPU、拷 .data、清 .bss 再进 main；system_gd32f4xx.c 把时钟树拉到 200MHz；gd32f4xx.ld 划地盘，Flash 起始 0x08000000、SRAM 起始 0x20000000。行李是零：同为 Cortex-M4F，B0 装好的那套 xPack arm-none-eabi-gcc 直接复用，旗逐字相同，只添 -DGD32F450 一个宏。往后每一章查偏移、查位名，花的都是这份零行李红利。

✅ 下一站：G1 RCU 时钟树：200MHz 是怎么算出来的，与 STM32F4 的 RCC 逐字段对照。

通往单片机之路第 70 章

G1 RCU 时钟树：200MHz 是怎么算出来的

难度 ★★★☆☆ · 约 45 分钟



本章罗盘

问题

GD32F450 默认 200MHz 的 PLL 账怎么算？比 STM32 多出的电压握手与 Flash 等待留白怎么处理？

交付

一笔手算的 25÷25×400÷2=200MHz 分频账、本工程与官方 system 文件的逐条对照表，和 CK_OUT0 量频对账方案。

边界

Flash 零等待只实证到 F407 数据手册的 168MHz 档，F450/F470 的 200MHz 档明确不做外推；两份手册均无「频率↔等待」对照表，保守超配值与 PA8≈50MHz 量频都待上板实测。

依据锚点 `a1` · `a10` · `a11` · `f6` · `mcu-code` · `mcu-anim`

> 🎯 最危险的移植往往不是完全陌生，而是“看起来一模一样”：PLL 位域熟悉、时钟公式熟悉、输出脚也熟悉，于是你顺手照搬 STM32 的经验。GD32 偏偏在电压回执、Flash 等待与外设时钟源上换了合同。今天的任务不是背另一棵时钟树，而是抓出哪些相似可以复用，哪些相似专门会骗熟手。

本章精髓

1. **PLL 寄存器和 STM32 同布局**（PSC/N/P/Q 同位域），但官方参数更激进：VCO 拉到 **400MHz**（STM32 的 N=336），25M 晶振 ÷25 ×400 ÷2 = **200MHz**。
2. **电压档是"三件套"且有回执**：LDOVS → HDEN → HDS，后两者各有就绪标志位 **HDRF/HDSRF**，官方代码真的轮询等待。这和 F405/407 的"写入即生效、无位可等"正好相反。
3. **官方 system 文件不设 Flash 等待周期，而且确实不用设**：UM §2.4.1 实证 WSCNT 是 4 位域 **0~15 档**（比 STM32 的 0~7 宽一倍），且必须先置 FMC_WSEN 才生效；而 GD32F407xx datasheet 开篇明写 168MHz 下 **Flash 零等待**："官方留白"不是偷懒，是这条取指路径本就不需要。两份手册都**不给"频率↔等待"对照表**（全文检索实证），与 ST 在 RM0090 直接给整张表的做法正好相反。

先修

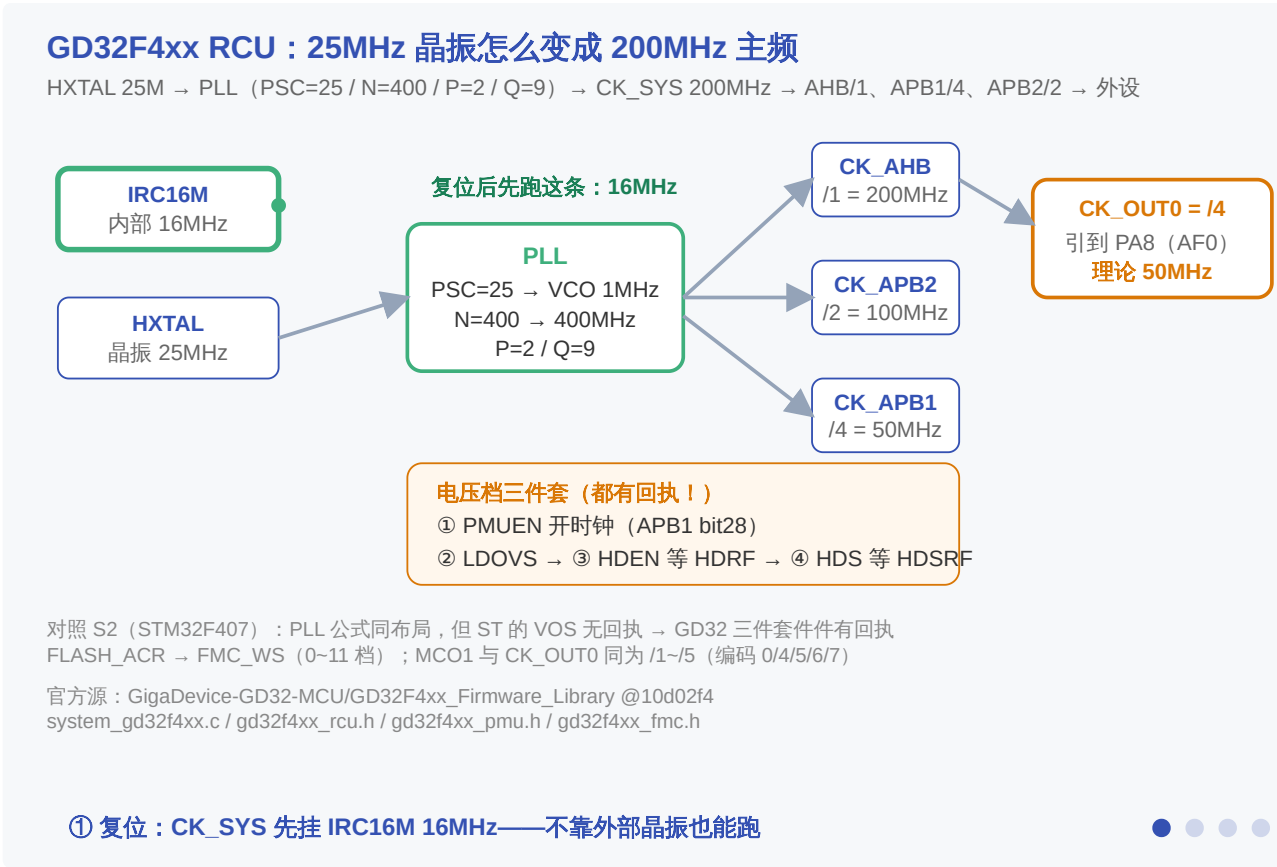
- S2 的"六步提频"与"MCO1 对账"思想（方法论完全复用）。
- GD32 命名：`CK_SYS` / `CK_AHB` / `CK_APB1` / `CK_APB2` 对应 STM32 的 SYSCLK/HCLK/PCLK1/PCLK2。

先跑起来（10 分钟 quick win）

先遮住下文结论，只看官方 PLL 参数，自己算一遍输入、VCO 与系统时钟；再回答两个迁移判断：STM32 的电压档“写完就走”能否照搬？MCO 的分频编码能否按“分频比减一”填写？算完再往下读。前一道题考公式，后两道题专门抓“太熟所以没查手册”的惯性。

即时判分标准很简单：频率公式可以复用，电压握手不能省，输出分频编码必须查枚举。相似的位域帮你快速定位，不同的状态位与编码才决定代码能不能安全运行。

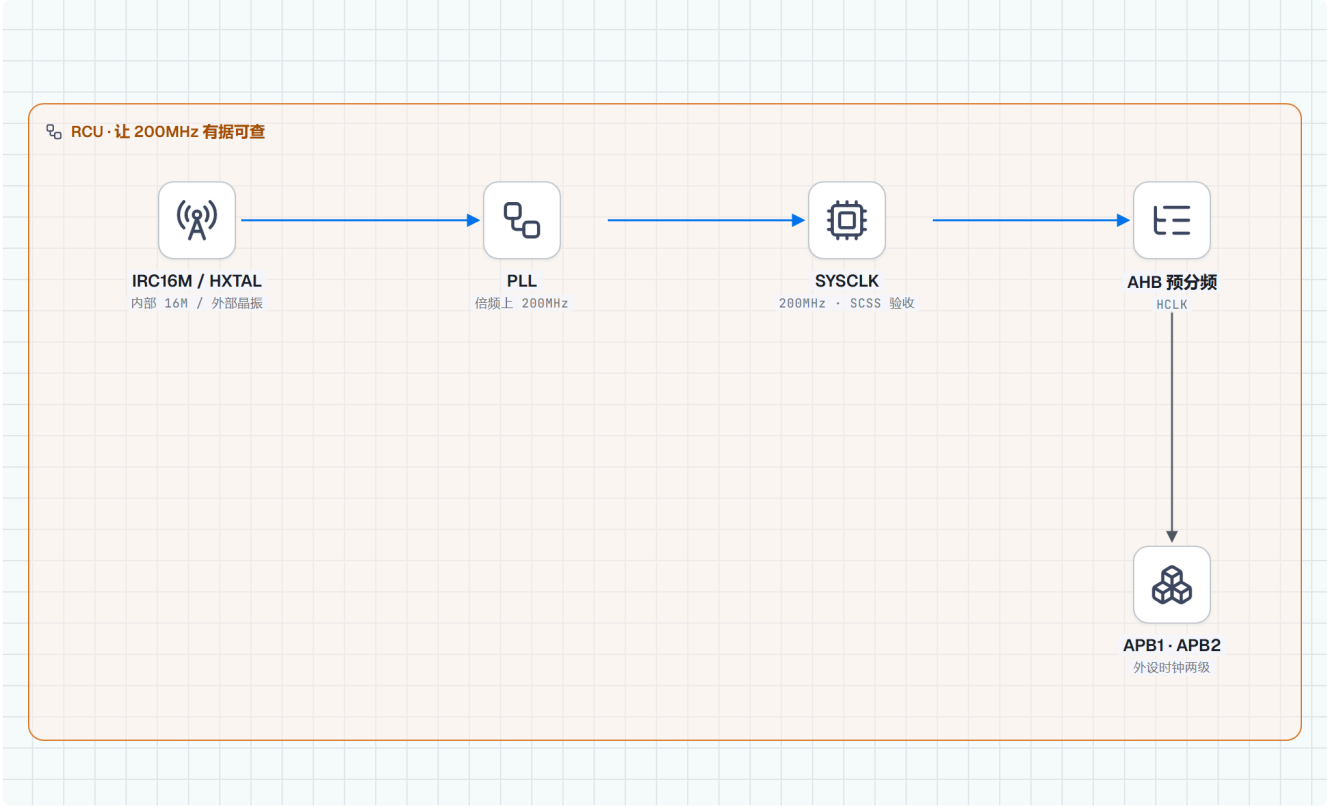
动画：GD32 的时钟树长什么样



GD32 RCU 时钟树动画

一、命名对照：同名不同姓

时钟树先看图：源、PLL、SYSCLK、AHB/APB 逐级分发；切换顺序铁律在图底。



GD32 RCU 时钟树：IRC16M/HXTAL→PLL→SYSCLK→AHB→APB 两级

STM32F407	GD32F4xx	凭证
RCC	**RCU**（基址 0x40023800，AHB1）	gd32f4xx.h:343
SYSCLK / HCLK / PCLK1/2	**CK_SYS / CK_AHB / CK_APB1/2**	gd32f4xx_rcu.h 注释
HSI 16M / HSE	**IRC16M / HXTAL**	gd32f4xx_rcu.h:812-814
FLASH_ACR.LATENCY	**FMC_WS.WSCNT**（bits[3:0]，0~15 档）	gd32f4xx_fmc.h:150-165
PWR + VOS（无回执）	**PMU + LDOVS/HDEN/HDS**（有 HDRF/HDSRF 回执）	gd32f4xx_pmu.h:59-73
MCO1（PA8，/1~/5，编码 0/4/5/6/7）	**CK_OUT0**（PA8，/1~/5，编码同款）	gd32f4xx_rcu.h:886-901；HAL stm32f4xx_hal_rcc.h:314-318
PLLRDY / SWS	**PLLSTB / SCSS**	gd32f4xx_rcu.h:88,102-103

二、PLL：同布局，更激进

`RCU_PLL` 偏移 0x04、`RCU_CFG0` 偏移 0x08（gd32f4xx_rcu.h:44-46），与 STM32 的 PLLCFGR/CFGR 排布一致，位域也一样：

```
RCU_PLL = PSC[5:0] | N[14:6] | P[17:16]=(P/2)-1 | SRC[22] | Q[27:24]
```

官方 200M 档写入的参数（system_gd32f4xx.c:946-948）：

```
RCU_PLL = (25U | (400U << 6U) | (((2U >> 1U) - 1U) << 16U) |
           (RCU_PLLSRC_HXTAL) | (9U << 24U));
```

即 **PSC=25，N=400，P=2，SRC=HXTAL，Q=9**：25MHz ÷ 25 = 1MHz 进 VCO，× 400 = **400MHz**，÷ P2 = **200MHz**。对比 STM32F407 的 8÷8×336÷2=168：公式一模一样，N 的上限更敢用（400 > 336）。

Q=9 给出 400/9 ≈ **44.4MHz**。这里有个值得停一下的细节：**USB 要的是 48MHz，44.4 不是 48**。UM §4.2.2 已核验：USBFS / USBHS / TRNG / SDIO 四个外设统一吃一路叫 **CK48M** 的时钟，而 CK48M 由 **PLLQ / PLLSAIP / IRC48M 三选一**（`RCU_ADDCTL` 的 `PLL48MSEL` + `CK48MSEL` 两位）。所以官方 200MHz 档下 PLLQ 根本凑不出 48MHz，USB 必须改走另外两源。GD32 多给的那颗 **IRC48M（片内 48MHz RC）** 正是为这种场合准备的。

对照着看 ST 的选择就更有意思：STM32F407 **没有** 48MHz 片内 RC，CK48M 只能由 PLLQ 出，所以 ST 把 N 定在 336，336/7 刚好 48.000MHz，提频公式是被 USB 反向约束过的。GD32 敢把 N 推到 400（VCO 400MHz 整），代价是 PLLQ 凑不出整 48，补偿是多给一颗 IRC48M。**同一条公式，两种取舍。**

> 🧮 这里在在线版是一个可交互组件（ClockCalc，改参数即时出数）；离线书请按正文给定的参数手算，或到在线版该页把玩。

三、电压档三件套：这次真的"等就绪"

GD32 把"能不能上 200MHz"拆成三次握手，每一步都有硬件回执（system_gd32f4xx.c:960-984）：

步	写什么	等什么	回执位
1	`PMU_CTL.LDOVS`（bits[14:15]）	无需等待	—
2	`PMU_CTL.HDEN`（bit16）高驱动使能	`PMU_CS.HDRF`（bit16）	官方代码真轮询
3	`PMU_CTL.HDS`（bit17）切入高驱动	`PMU_CS.HDSRF`（bit17）	官方代码真轮询

前提：`RCU_APB1EN.PMUEN`（bit28）先给 PMU 时钟，GD32 的 PMU 挂 **APB1**（gd32f4xx_rcu.h:261）。

和 S2 的结论对照着记：**F405/407 的 VOS 没有就绪位，写完就当生效；GD32F4xx 有 HDRF/HDSRF，官方流程必须等**。另外官方文件在提频前还塞了一个 `_soft_delay_()` 软延时，注释明说"防止 Vcore 波动影响，强烈建议保留"（system_gd32f4xx.c:226-231）。电压档切换不是免费的。

四、Flash 等待：官方不管，你来管

`system_gd32f4xx.c` 全文件**没有任何 FMC 等待周期设置**；对官方库**全量 Examples（3295 个文件，浅克隆逐目录检索）**搜 `fmc_wscnt_set` / `FMC_WS`，**零命中**。官方示例从不设 Flash 等待。而 FMC 的等待档位实际有 **0~15 共 16 档**，WSCNT 是 4 位域（`FMC_WC_WSCNT = BITS(0,3)`，gd32f4xx_fmc.h:65），头文件给出 WS_WSCNT_0 ~ WS_WSCNT_15 全部宏（:149-165），比 STM32F407 的 0~7（3 位）宽一倍。另外 GD32 还有一枚 STM32 没有的 **FMC_WSEN@0xFC** 等待使能寄存器（`FMC_WSEN_WSEN` bit0，:117-118）。两件套都要自己管。

> ✅ **这个坑已经填了**：官方示例不设等待却跑高频，答案在 datasheet 第一页：GD32F407xx 的 Cortex-M4 "operating at 168 MHz frequency with **Flash accesses zero wait states**"（GD32F407xx Datasheet Rev2.7 §1）。GD32 的 Flash 取指路径按零等待设计，所以官方 system 文件不写 FMC_WS 也能跑满 168MHz。那 WSCNT 存在的意义是什么？UM §2.4.1 的一句话给了线索（"The WSCNT valid when **WSEN** bit in FMC_WSEN is set"）：**等待周期默认整条路关着**，要用得先开 FMC_WSEN@0xFC，这是给特殊电压/温度条件或更高主频档留的余量，不是常规必配项。
>
> ⚠️ 一个诚实的边界：我们手上的 datasheet 是 **F407xx（168MHz 档）**。官方 system 文件默认的 **200MHz 档属于 F450/F470**，那两颗的零等待上限要各自 datasheet 确认，本站**不做外推**。
>
> ⚠️ 本工程策略：`FMC_WS_VALUE` 宏默认取**保守超配值**（多配等待周期只会稍慢，欠配会取指跑飞）。位域与使能链已全部核验（WSCNT 4 位 0~15 + WSEN 使能，UM §2.4.1 / §2.4.10）；但手册**确实不提供**"频率↔等待"对照表，所以"这颗板子这个频率该配几拍"没有纸面答案，只能上板实测。在没有对照表的情况下，保守超配是唯一安全的默认值。

五、CK_OUT0 对账：MCO1 的同岗同事

- 引脚 **PA8，AF0**（官方 CKOUT 示例 gpio_af_set(GPIOA, GPIO_AF_0, GPIO_PIN_8)，example_ckout_main.c:135），和 STM32 MCO1 同脚同 AF。
- 源可选 IRC16M/LXTAL/HXTAL/**PLLP**（gd32f4xx_rcu.h:886-889），分频 **/1 /2 /3 /4 /5**（:897-901），**与 STM32 MCO1 完全同款**：两边都不是 2 的幂序列，编码也一样（0xx=不分频、100=/2、101=/3、110=/4、111=/5；HAL `RCC_MCODIV_1..5`=0/4/5/6/7，stm32f4xx_hal_rcc.h:314-318）。"看名字以为是 STM32 的坑、其实两边一样"本身就是对照阅读的价值。
- 编码雷区：/4 的编码是 **6**，不是 3：写成 3 会掉进"不分频"区。（S2 工程曾真踩此坑，已修。）
- 选 PLLP + /4 → **理论 50MHz** 上 PA8，示波器一量就知时钟树对不对（**待上板实测**）。

六、代码分析：`code/gd32/01-rcu-clock/main.c`

工程与 S2 的 `01-rcc-clock` 同构，差异点全部点名：

- `hxtal_enable()`：HXTALEN + 轮询 HXTALSTB，**超时回退 IRC16M**（官方 demo 是 `while(1)` 死等，system_gd32f4xx.c:942-945；教学工程不照抄这个坏习惯）。
- `pmu_highdrive_enter()`：PMUEN → LDOVS → HDEN 等 HDRF → HDS 等 HDSRF，带超时。
- `fmc_ws_set()`：写 FMC_WS.WSCNT。
- `pll_start()`：按官方参数写 RCU_PLL，PLLSTB 带超时。
- `clock_init()`：总线分频 AHB/1、APB2/2、APB1/4（与官方 200M 档一致），SCS=PLLP 后等 SCSS=PLLP。
- `clock_tree_readback()`：从寄存器反推真实时钟树，**源按 SCSS 如实解码**、PLL 频率按 RCU_PLL 参数重算，存进 `g_clock_tree[]`；故障回退路径也报真账（不传"调用方以为的值"）。
- `ckout0_init()`：PA8 配 AF0，CK_OUT0 = PLLP/4。

七、上游对照：官方 `system_gd32f4xx.c` 逐条对照（@10d02f4）

官方（行号）	本工程	结论
HXTALSTB 超时后 `while(1)`（:942-945）	超时回退 IRC16M + `g_clock_status=1`	工程更稳；官方 demo 简化了错误路径
PMUEN→LDOVS→HDEN/HDS 等回执（:960-984）	同序，加超时	一致
不设 FMC_WS（全文检索为空）	显式设置 + 保守超配	工程补上了官方留白
AHB/1、APB2/2、APB1/4（:962-966）	同	一致
PLL 参数 PSC25/N400/P2/Q9（:946-948）	同	一致
SCS=PLLP 后等 SCSS（:993-999）	同	一致
`while(0 != (RCU_CFG0 & RCU_SCSS_IRC16M))`（:208-210）	——	**官方小瑕疵**：`RCU_SCSS_IRC16M = CFG0_SCSS(0) = 0`（gd32f4xx_rcu.h:817），这个"等回执"恒假、一次都不等。对照 S2 里 HAL 切钟后老老实实轮询 SWS（hal_rcc.c:681）——**官方库也会有空转，读源码时要带脑子**。本工程的回读改为按 SCSS 如实解码，故障路径也不报假账。

附录：工程完整源码

<<< ../../code/gd32/01-rcu-clock/main.c

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **先电压三件套（LDOVS→HDEN→HDS 带回执）、再 FMC 等待、后切钟；写完 SCS 看 SCSS；CK_OUT0 一量，自欺现形。**

延伸：GD32 RCU 时钟树动画见 G1 动画；STM32 RCC 对照见 S2；Flash 等待周期在 B5 有体积影响。

实物实验

- CK_OUT0 量频：构建烧录后量 PA8，应为 ≈50MHz（200/4）；切 IRC16M 档应为 4MHz。
- HSE 对账：量 HXTAL 实际频率（晶振标称 25M 常有 ±20ppm 偏差），回填到 `HXTAL_VALUE_HZ`。
- 翻车实验（选做）：把 FMC_WS 改小 2 档跑 200MHz，观察取指异常，再用调试器读 FMC_WS 复位值。

常见坑

1. **PMU 时钟没开就写 PMU_CTL**：`RCU_APB1EN.PMUEN`（bit28）是第一步，否则写进去的影子都没有。
2. **照抄官方 demo 的 `while(1)`**：晶振不来就死机；教学工程的超时回退才配叫"代码"。
3. **以为官方帮你设了 Flash 等待**：`system_gd32f4xx.c` 不设 FMC_WS，欠配等待周期上 200M 必跑飞。

短自测

1. GD32F450 官方默认档从 25MHz HXTAL 算到 200MHz，写全 PLL 公式。和 STM32F407 的 N=336 有什么差别？

> ▼ **参考答案**
> 公式：25MHz ÷ 25（PSC=24，分频系数=PSC+1=25）× 400（N=399，倍频系数=N+1=400）÷ 2（P=1，分频系数=P+1=2）= **200MHz**。VCO 频率 = 25 ÷ 25 × 400 = 400MHz。STM32F407 的 VCO 封顶 336MHz（N=335），SYSCLK = VCO ÷ 2 = 168MHz。GD32 把 VCO 拉到 400MHz，同样 ÷2 就是 200MHz。PLL 寄存器布局完全相同，只是参数更激进。

2. HDEN 和 HDS 各控制什么？为什么两个都要等标志位？

> ▼ **参考答案**
> **HDEN**（High Density Mode Enable）使能高密度模式，拉高 LDO 输出，让芯片能在 200MHz 下稳定运行。**HDS**（High Density Step）进一步微调 LDO 档位到最高性能档。两者各有就绪标志：**HDRF**（HD Enable Ready Flag）和 **HDSRF**（HD Step Ready Flag）。必须轮询等待是因为 LDO 电压爬升需要时间（微秒

级），如果不等就绪就提频，内核可能因供电不足跑飞。官方 `system_gd32f4xx.c` 里两步都有 `while(!HDRF)` / `while(!HDSRF)` 的等待循环。

3. CK_OUT0 输出在哪个引脚？/4 分频的编码值是多少？误写成"分频比 − 1 = 3"会落入哪个编码区？

▼ 参考答案
CK_OUT0 输出在 **PA8**（AF0 复用功能）。/4 分频的编码值是 **6**：官方宏依次使用 0、4、5、6、7 表示 /1 到 /5。误写 3 会落入 `0xx` 的不分频区，输出源时钟而不是 /4。这里不能套“分频系数减一”，必须按官方枚举写值；本章代码使用的 `RCU_CKOUT0_DIV4` 也正是 `6UL << 24`。

事实来源

结论	出处
RCU 基址/外设库命名	gd32f4xx.h:343（@10d02f4）
CTL/PLL/CFG0 偏移 0x00/0x04/0x08，AHB1EN 0x30，APB1EN 0x40	gd32f4xx_rcu.h:44-56
HXTALEN/HXTALSTB bit16/17，PLLEN/PLLSTB bit24/25	gd32f4xx_rcu.h:83-88
SCS/SCSS/AHBPSC/APB1PSC/APB2PSC/CKOUT0 位域	gd32f4xx_rcu.h:102-112
官方 200M 档 PLL 参数与提频序列	system_gd32f4xx.c:936-1003
LDOVS/HDEN/HDS 与 HDRF/HDSRF	gd32f4xx_pmu.h:59-73
PMUEN bit28	gd32f4xx_rcu.h:261
FMC_WS.WSCNT 0~15 档 + FMC_WSEN 使能	gd32f4xx_fmc.h:65,117-118,149-165（V3.3.3 已核验）
WSCNT 必须 WSEN 置位才生效	**UM Rev3.0 §2.4.1 / §2.4.10 已核验**（p69 / p76）
168MHz 下 Flash 零等待	**GD32F407xx Datasheet Rev2.7 §1 已核验**（F450/F470 的 200/240MHz 档未外推）
UM 与 datasheet 均无"频率↔等待"对照表	两份手册全文检索零命中（已核验）
CK48M 三源 PLLQ/PLLSAIP/IRC48M，由 RCU_ADDCTL 的 PLL48MSEL+CK48MSEL 选	**UM Rev3.0 §4.2.2 已核验**（p94）
PA8 AF0 配 CK_OUT0	example_ckout_main.c:135（@10d02f4）+ **datasheet §2.6.6 Table 2-9 AF0 列已核验**

你做到了

- 你算出了 GD32 的 200MHz，也掌握了“同名不同姓”的对照读法。拿到任何一颗 Cortex-M4，都能顺着时钟树把它提上去。
- 你知道电压档要等回执、Flash 等待要自己管：这两条是官方文档不会替你记住的坑。

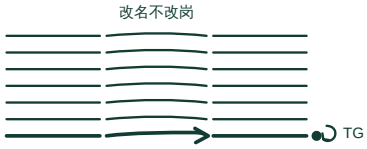
回到开场那次“看起来一样”的移植：你现在会先复用结构，再逐项核对合同。公式相同就大胆算，状态位不同就老实等，编码表长得眼熟也要重新查。真正可迁移的不是寄存器写法，而是这套对照取证的方法。

✅ G1 收官。下一站：G2 GPIO 与 AF 复用对照：七大寄存器“同名不同姓”，逐个点名。

通往单片机之路第 71 章

G2 GPIO 与 AF 复用对照：七大寄存器同名不同姓

难度 ★★☆☆☆ · 约 60 分钟



本章罗盘

问题
GD32F4 的 GPIO 与 STM32F4 逐字段对照，哪些位同名同义、哪些同名不同义、哪些干脆换了位置？

交付
一张 GPIO 寄存器 F4↔GD32F4 逐位对照表，标出必须改代码的差异位和可以直接照抄的部分。

边界
对照只覆盖本书实验用到的模式；未列出的封装/引脚要以 GD32 数据手册对应料号重新核对。

依据锚点 `a1 · a10 · a11 · f6 · mcu-anim`

> 🎯 S3 你把 STM32 的 MODER/OTYPER/OSPEEDR/PUPDR 七张表拆到逐位——到了 GD32，这七张表一张都没多、一张都没少，位置也一模一样，只是**每个都换了名字**：CTL、OMODE、OSPD、PUD……本章把这些"同名不同姓"逐个点名，再揪出 GD32 比 STM32 多出来的两件秘密兵器（BC 与 TG）。

本章精髓

1. **七大寄存器全部改名、不改岗**：CTL↔MODER、OMODE↔OTYPER、OSPD↔OSPEEDR、PUD↔PUPDR、ISTAT↔IDR、OCTL↔ODR、BOP↔BSRR。偏移 0x00~0x1C 逐字相同，位宽与编码也相同，**名字是唯一差异**（gd32f4xx_gpio.h:52-63）。
2. **GD32 多两件兵器**：`BC`（0x28，独立清零）和 `TG`（0x2C，**硬件翻转**），STM32F4 压根没有这两个寄存器。点灯的"读-改-写翻转"在 GD32 是一条 `str`。
3. **基址与使能位与 STM32F4 完全相同**：GPIOA=0x40020000、GPIOF=0x40021400，时钟使能 `RCU_AHB1EN` bit0~bit8（GPIOA~GPIOI）；连"忘开时钟"这个新手第一大坑，两边都长一个样。
4. **编码一字不差**：模式 0/1/2/3=输入/输出/复用/模拟，上下拉 0/1/2=浮空/上/下，推挽开漏 0/1，S3 背过的"先清后置"肌肉记忆**原样可用**。

- **能理解**：为什么"基址相同 + 偏移相同 + 编码相同"意味着 S3 的知识几乎全额平移；BC/TG 各自解决什么痛点。
- **能用**：拿 S3 点灯的 PF6 流程，把每个寄存器名换成 GD32 版写出来；查 `gd32f4xx_gpio.h` 确认每一项。

先修

- G1 RCU 时钟树：GD32 的 RCU/CK_* 命名体系与"对照读法"；
- S3 GPIO：七个寄存器位级图解：MODER/OTYPER/OSPEEDR/PUPDR/IDR/ODR/BSRR 的位级理解，本章只讲"哪里不一样"。

先跑起来（10 分钟 quick win）

打开 `.trellis/ref/gd32/gd32f4xx_gpio.h` 第 52~63 行（寄存器偏移定义），对照 S3 板卡事实里的 STM32F4 偏移表（RM0090 §8.4：MODER 0x00 / OTYPER 0x04 / OSPEEDR 0x08 / PUPDR 0x0C / IDR 0x10 / ODR 0x14 / BSRR 0x18 / LCKR 0x1C / AFRL 0x20 / AFRH 0x24）：**前 10 个偏移逐个对得上，只有名字换了**。5 分钟点名完，你已经会 GD32 GPIO 的 80%。

先押个平移：S3 背过的"先清后置"肌肉记忆，到 GD32 还能原样用吗？翻转点灯这件小事，GD32 会不会专门备一件兵器，让"读-改-写"缩成一条指令？

动画：AF 复用的 16 选 1 选择器

一根引脚一根物理线，AF0~AF15 排队过门；每脚 4 位的 AFSEL 字段决定谁过门。PA8 写 AF0、CK_OUT0 上路这个实例，动画演一遍，"换号 = 换接线"一眼看懂。

GD32 AF 复用：每脚 4 位的选择器，16 选 1 过门

小节结构

小节	内容
一、改名点名	七大寄存器 + LOCK/AFSEL 的 STM32↔GD32 对照表
二、基址与使能	GPIO 基址、RCU_AHB1EN 使能位——与 STM32F4 完全同款
三、多出来的两件兵器	BC 独立清零、TG 硬件翻转；STM32 的曲线实现对比
四、编码对照	模式/上下拉/速度/输出类型四位配置的逐位核对
五、AF 复用	AFSEL0/1 与 datasheet 复用表查法
六、库函数对照	gpio_mode_set + gpio_output_options_set vs GPIO_Init

一、改名点名：七大寄存器"同名不同姓"

GD32F4xx 每个 GPIO 端口有 **12 个寄存器**（gd32f4xx_gpio.h:52-63），STM32F4 是 10 个。先看点名表：

偏移	STM32F407（RM0090 §8.4）	GD32F4xx（gd32f4xx_gpio.h）	岗位
0x00	MODER	**CTL**（port control）	模式：输入/输出/复用/模拟，每脚 2 位
0x04	OTYPER	**OMODE**（output mode）	推挽/开漏，每脚 1 位
0x08	OSPEEDR	**OSPD**（output speed）	输出速度，每脚 2 位
0x0C	PUPDR	**PUD**（pull-up/pull-down）	上拉/下拉/浮空，每脚 2 位
0x10	IDR	**ISTAT**（input status）	读引脚真实电平，只读
0x14	ODR	**OCTL**（output control）	输出数据寄存器
0x18	BSRR	**BOP**（bit operate）	低 16 位置位 + 高 16 位复位，原子写
0x1C	LCKR	**LOCK**（configuration lock）	引脚配置锁定
0x20	AFRL	**AFSEL0**（AF selected 0）	引脚 0~7 的复用号，每脚 4 位
0x24	AFRH	**AFSEL1**（AF selected 1）	引脚 8~15 的复用号，每脚 4 位
0x28	—（无）	**BC**（bit clear）	GD32 独有：独立清零寄存器
0x2C	—（无）	**TG**（bit toggle）	GD32 独有：硬件翻转寄存器

前 10 行的规律一眼看穿：**偏移、位宽、编码全部相同，只有名字换了**。

GD32 的命名习惯是"把类型塞进缩写"：CTL 是 control，OMODE 是 output mode，OSPD 是 output speed，PUD 是 pull-up/down，ISTAT 是 input status，OCTL 是 output control。名字更长，但和 STM32 一样能按"每脚占多少位"直接推。

改名记忆钩子：

- **模式叫 CTL 不叫 MODER**，最易踩的一个：搜文档搜 "MODER" 永远搜不到；
- **输出两兄弟都带 O**：OMODE 管形态（推挽/开漏）、OSPD 管快慢、OCTL 管电平，三 O 打头；
- **输入叫 ISTAT**：input status，比 STM32 的 IDR 直白；
- **BOP 是 BSRR 的 GD32 名**：bit operate，布局同款（低 16 置位、高 16 复位，gd32f4xx_gpio.h:174-206）；
- **复用叫 AFSEL0/AFSEL1**：比 AFRL/AFRH 更好认：数字 0 管 pin0~7、数字 1 管 pin8~15，见名知义。

> 【注】**LOCK 寄存器两边都有，坑位相同**。STM32 的 LCKR 有著名的 LCKK 写序列坑；GD32 的 LOCK 键序列已按用户手册实证（F4xx UM p185）：**Write 1 → Write 0 → Write 1 → Read 0 → Read 1**。
>
> 细节也和 STM32 同款：LKK 在 bit16，序列期间 LK[15:0] 必须保持不变，锁定后直到复位才解锁。本站工程暂不用锁定功能。

二、基址与使能：连地址都没换

更狠的事实：**GPIO 基址和时钟使能位与 STM32F4 完全相同**。

事实	GD32F4xx	凭证	STM32F407
GPIO 总基址	GPIO_BASE = AHB1_BUS_BASE + 0x00 = **0x40020000**	gd32f4xx.h:313,341	0x40020000
GPIOA	0x40020000（+0x00）	gd32f4xx_gpio.h:41	0x40020000
GPIOF	0x40021400（+0x1400，步进 0x400）	gd32f4xx_gpio.h:46	0x40021400
端口数	A~I 共 9 个（F407 常用 A~I）	gd32f4xx_gpio.h:40-49	A~I
时钟使能寄存器	RCU_AHB1EN（RCU + 0x30）	gd32f4xx_rcu.h:53	RCC_AHB1ENR（+0x30）
GPIOA 时钟位	RCU_AHB1EN_PAEN = **bit0**	gd32f4xx_rcu.h:209	bit0
GPIOF 时钟位	RCU_AHB1EN_PFEN = **bit5**	gd32f4xx_rcu.h:214	bit5
GPIOI 时钟位	RCU_AHB1EN_PIEN = **bit8**	gd32f4xx_rcu.h:217	bit8

GPIOF 开时钟一行，两边只差寄存器名：

```
/* STM32F407（S3 工程）                          GD32F4xx（同名不同姓） */
RCC_AHB1ENR |= (1UL << 5);          RCU_AHB1EN |= (1UL << 5);   /* 都是 bit5 */
```

为什么能像到这个份上？F4 系的 AHB1 外设布局早就是 ARM 生态的"行业事实标准"。GD32F4xx 对 STM32F4 做的正是**高兼容设计**：外设基址、总线挂载都刻意对齐，迁移成本压到"改宏名"这一级。这正是 G 篇"对照学习"能省力七成的底牌。

但"兼容"≠"相同"：G1 已经演示过 RCU 内部位域有差异（HDRF/HDSRF 回执、SCSS 解码），本章第四节会看到速度档标签的小差异。**基址相同给你迁移便利，位域还得逐个核**，这是 G 篇的总纪律。

> **【注】"忘开时钟"坑两边一个指纹**：S3 说"不开 GPIOF 时钟，寄存器读回全是 0"，GD32 一模一样，连症状带解法（先写 RCU_AHB1EN）都原样复用。新手第一大坑，在 GD32 上还是第一大坑。

三、多出来的两件兵器：BC 与 TG

STM32F4 的 GPIO 到 0x24（AFRH）就结束了；GD32F4xx 在 0x28 和 0x2C 又加了两个寄存器，这是**这颗芯片对 GPIO 的真增强**：

BC（0x28）：独立清零寄存器

低 16 位每脚一位，写 1 清零对应引脚（gd32f4xx_gpio.h:247-263）。

STM32 里要清一个引脚，两条路：BSRR 高 16 位（`GPIOx_BSRR = 1UL << (pin+16)`）或 ODR 读-改-写。GD32 把"清零"单独发一个寄存器：**低 16 位直接写脚号位**，不用 +16 挪半档：

```
RCC_...                                  /* STM32：BSRR 高 16 位清零     */
GPIOF_BSRR = (1UL << (6 + 16));          GPIOF_BC = (1UL << 6);      /* GD32：BC 直接写 */
```

顺手一提：BOP（0x18）的高 16 位同样能清零（CR0~CR15，gd32f4xx_gpio.h:191-206），所以 GD32 是**清零有两条路**（BOP 高 16 位、BC 低 16 位），置位只有一条（BOP 低 16 位）。官方库 `gpio_bit_reset()` 走的是 BC（查 gd32f4xx_gpio.c 可核，**本页以 .h 位定义为准**）。

TG（0x2C）：硬件翻转寄存器

低 16 位每脚一位，**写 1 让引脚电平翻转**（gd32f4xx_gpio.h:265-281）。

这一下就动了 STM32 的奶酪。S3 讲过：STM32 翻转一个引脚要么 ODR 读-改-写（`GPIOF_ODR ^= 1UL<<6`，三条指令、可被中断插队）、要么 BSRR 配合读 ISTAT/ODR 判断当前态。GD32 的答案是**硬件翻转一条写指令**：

```
GPIOF_TG = (1UL << 6);    /* 一条 str：PF6 电平翻转，原子、无读操作 */
```

点灯场景直接受益：blink 不再需要"记住当前状态"，往 TG 写脚号位就行；官方库也给了对应封装 `gpio_bit_toggle()`（gd32f4xx_gpio.h:402）和整端口翻转 `gpio_port_toggle()`（:404）。

> **【注】TG 与 BOP/BC 一样是"写 1 生效、写 0 无影响"**，对 TG 用读-改-写毫无意义（它不需要读）。但注意 TG 只翻转**输出路径的电平**，输入侧要看 ISTAT；高频翻转时 OSPD 速度档不够会看到边沿变缓，速度档配置见第四节。

三剑客变四剑客

S3 的"数据三剑客"（IDR/ODR/BSRR）在 GD32 变成**四剑客**：ISTAT（看）、OCTL（写）、BOP（原子置位/复位）、加上 BC/TG 双兵器。对照记忆：

动作	STM32F407	GD32F4xx
读电平	IDR	ISTAT
写电平	ODR（读-改-写有隐患）	OCTL（同隐患）
原子置位/复位	BSRR 低/高 16 位	BOP 低/高 16 位（同款）
独立清零	—（只有 BSRR 高 16 位）	**BC** 低 16 位
翻转	ODR `^=`（三条指令）	**TG** 低 16 位（一条写）

四、编码对照：四位配置一字不差

S3 背过的"配置四维"编码，在 GD32 **逐位相同**（全部出自 gd32f4xx_gpio.h 的常量定义）：

维度	取值编码	GD32 常量（行号）	STM32 对应
模式（CTL）	00 输入 / 01 输出 / 10 复用 / 11 模拟	GPIO_MODE_INPUT/OUTPUT/AF/ANALOG = 0/1/2/3（:288-291）	MODER 同款 00/01/10/11
上下拉（PUD）	00 浮空 / 01 上拉 / 10 下拉	GPIO_PUPD_NONE/PULLUP/PULLDOWN = 0/1/2（:295-297）	PUPDR 同款
输出类型（OMODE）	0 推挽 / 1 开漏	GPIO_OTYPE_PP/OD = 0/1（:331-332）	OTYPER 同款
速度（OSPD）	00 / 01 / 10 / 11 四档	GPIO_OSPEED_2MHZ/25MHZ/50MHZ/MAX = 0/1/2/3（:336-345）	OSPEEDR 同款四档

唯一值得点名的是**速度第四档的标签**：STM32F4 的 11 档标 **100MHz**（rm0090），GD32 的 LEVEL3 标 **GPIO_OSPEED_MAX**，注释明说 "max speed more than 50MHz"（gd32f4xx_gpio.h:345）：**库头文件不承诺具体数字**。

但 datasheet 承诺了，而且比 100MHz 更激进。GD32F407xx Datasheet Rev2.7 **Table 4-28 I/O port AC characteristics**（p101）把四档全标了名、还按负载电容给了实测上限：

OSPD[1:0]	datasheet 档位名	CL=10pF	CL=30pF	CL=50pF
00	IO_Speed = 2 MHz	30 MHz	25 MHz	15 MHz
01	IO_Speed = 25 MHz	95 MHz	80 MHz	50 MHz
10	IO_Speed = 50 MHz	160 MHz	125 MHz	90 MHz
11	**IO_Speed = 200 MHz**	200 MHz	170 MHz	130 MHz

三件事带走：

1. **第四档的真名是 200MHz**，不是库注释里含糊的"more than 50MHz"，也不是 STM32 的 100MHz。对照 STM32F407 datasheet 的同款表（2/25/50/100MHz 四档），GD32 把第四档的标称值翻了一倍。
2. **档位名是标称值，不是能跑到的频率**。`IO_Speed = 2 MHz` 这档在 10pF 轻载下实测能到 30MHz；反过来 200MHz 档挂 50pF 只剩 130MHz。真正决定边沿的是**档位 + 负载电容**两个变量，表里每档三列就是这个意思。
3. datasheet 注 (4) 还压了一道天花板：**最高频率不得超过 168 MHz**（F407 的核心上限）。也就是说，200MHz 这个标称值在 F407 上根本吃不满——它是给 F450/F470 的高主频档留的。

> ⚠️ **四个引脚是例外**。datasheet 注 (3) 明说 **PC13 / PC14 / PC15 / PI8 走 Power Switch 供电**，只能取到很小的电流。它们做输出时**速度不得超过 2 MHz**（最大负载 30pF）。这四个脚在 STM32F407 上同样是"备份域弱驱动"脚（PC13-PC15 接 LXTAL/侵入检测），两边都别拿来驱动高速信号或直推 LED。

所以 S3 的 PF6 点灯流程翻译成 GD32，一个字都不用改逻辑，只换寄存器名：

```
/* GD32F4xx：PF6 推挽输出点灯（对照 S3 的 00-blink，仅换名） */
RCU_AHB1EN  |= (1UL << 5);                          /* GPIOF 时钟：bit5，同 STM32  */
GPIOF_CTL   &= ~(3UL << (6 * 2));                   /* 先清：CTL[13:12]            */
GPIOF_CTL   |=  (1UL << (6 * 2));                   /* 后置：01 = 输出             */
GPIOF_OMODE &= ~(1UL << 6);                          /* 0 = 推挽                    */
GPIOF_OSPD  &= ~(3UL << (6 * 2));                    /* 00 = 低速                   */
GPIOF_PUD   &= ~(3UL << (6 * 2));                    /* 00 = 浮空                   */
GPIOF_BC    =  (1UL << 6);                           /* 清零 PF6 → 拉低（共阳则亮） */
```

"先清零再置位"的纪律（S3 的【注】）原样生效：CTL 是两位格子，不清就置可能踩出意外组合；复位后 PA13/PA14 等调试脚默认复用模式的提醒也照搬。

五、AF 复用：AFSEL0/1 与复用表查法

复用机制的框架与 S3 第四节完全一致，改名与分界如下：

项	STM32F407	GD32F4xx	凭证
复用模式位	MODER[1:0] = 10	**CTL[1:0] = 10**（GPIO_MODE_AF=2）	gd32f4xx_gpio.h:290
AF 号寄存器	AFRL（pin0~7）/ AFRH（pin8~15）	**AFSEL0 / AFSEL1**，每脚 4 位	gd32f4xx_gpio.h:227-245
AF 号范围	AF0~AF15	**AF0~AF15**（GPIO_AF_0..GPIO_AF_15）	gd32f4xx_gpio.h:353-368
nibble 定位	AFR[pin >> 3]	同款算法（AFSEL1 管 pin8~15）	—
哪脚能当什么	datasheet 复用功能表（Table 9，一张表管全部端口）	**datasheet §2.6.6，Table 2-9 ~ 2-17 共九张表**（Port A~I 一端口一张）	GD32F407xx Datasheet Rev2.7 p60-68（已核验）

G1 已核过一个实例：**PA8 输出 CK_OUT0 用 AF0**（`gpio_af_set(GPIOA, GPIO_AF_0, GPIO_PIN_8)`，官方 example_ckout_main.c:135）；datasheet Table 2-9 的 AF0 列也正是 `CK_OUT0`，与 STM32 MCO1 的 PA8/AF0 同脚同号。

本章把那条"逐脚核对"的作业也做了：**USART0_TX = AF7**，候选脚 **PA9 / PB6 / PA15**（datasheet Table 2-9 / 2-10 的 AF7 列，p60-63）。前两个与 STM32F407 的 USART1_TX（PA9/PB6，AF7）**同号同脚**：GD32 的 USART0 就是 STM32 USART1 的岗位（G3 会讲这套错位编号），连 AF 号都没动。

但第三个脚是 GD32 加的。STM32F407 的 PA15 只有 JTDI / TIM2_CH1 / SPI1_NSS / SPI3_NSS，**没有 USART1_TX**（STM32F407 datasheet Table 9，p66）。同一张表看两遍，"大量重合"和"全部重合"的差别就在这种脚上。

> **所以纪律不能省**：拿 STM32 的 AF 表当 GD32 用，PA9/PB6 这种脚会碰巧对，PA15 这种脚会让你以为"GD32 没有"；反过来把 GD32 工程搬回 STM32，PA15 当 TX 用会**静默失效**（AF7 在 STM32 PA15 上是空位）。GD32 多出来的那几个串口（USART5~7，G3 讲）在 STM32F407 上更没有对应岗位，AF 号无从对照。迁移时的正确姿势永远是**按脚查表**，不是按记忆填号。

配置姿势照搬 S3"复用要成对"：CTL 写 10（复用模式）+ AFSEL 选号，缺一不可；只写 AFSEL 不改 CTL，等于章盖了门没开。

```
/* GD32F4xx：PA8 复用 AF0（CK_OUT0），对照 S3 的 USART1_TX 配置样式 */
GPIOA_CTL    &= ~(3UL << (8 * 2));
GPIOA_CTL    |=  (2UL << (8 * 2));                  /* CTL[17:16] = 10，复用模式    */
GPIOA_AFSEL1 &= ~(0xFUL << 0);                       /* PA8 是 AFSEL1 的第 0 个 nibble */
GPIOA_AFSEL1 |=  (0x0UL << 0);                       /* AF0 = CK_OUT0                */
```

注意 PA8 属于 pin8~15，落在 **AFSEL1**（对应 STM32 的 AFRH），nibble 索引是 `pin & 7`，不是 pin 本身。S3 踩过的"PA9 是 AFRH 第 1 个 nibble"坑，在 GD32 原样存在，只是寄存器名换成 AFSEL1。

六、库函数对照：GD32 拆两步，STM32 一把抓

官方库的 GPIO 配置 API 两边思路不同：**STM32 SPL 一个结构体一把抓，GD32 拆成两个语义函数**：

语义	STM32 SPL	GD32 V3.3.3（gd32f4xx_gpio.h:372-404）
配模式 + 上下拉	GPIO_Init（连同输出参数一把抓）	`gpio_mode_set(gpiox, mode, pupd, pin)` → 写 **CTL + PUD**
配输出形态 + 速度	同上（同一个结构体）	`gpio_output_options_set(gpiox, otype, speed, pin)` → 写 **OMODE + OSPD**
配复用号	GPIO_PinAFConfig（单独调）	`gpio_af_set(gpiox, af, pin)` → 写 **AFSEL0/1**
置位/复位/写	GPIO_SetBits / ResetBits / WriteBit	gpio_bit_set / bit_reset / bit_write
读输入/输出	GPIO_ReadInputDataBit / ReadOutputDataBit	gpio_input_bit_get / gpio_output_bit_get
翻转	—（手写 ODR 翻转）	**gpio_bit_toggle / gpio_port_toggle**（走 TG）
锁定	GPIO_LockPin	gpio_pin_lock

GD32 的拆分有它的道理：**纯输入引脚根本不需要输出形态和速度**，`gpio_mode_set` 一行就配完（CTL+PUD），不碰 OMODE/OSPD；要输出了再调 `gpio_output_options_set`。STM32 的 GPIO_Init 结构体里 `GPIO_Speed/GPIO_OType` 对输入脚是"填了也白填"。对照记忆：**SPL 一把抓，GD32 按需分两次**，像点菜，套餐改单点。

剥开 `gpio_mode_set` 的实现（gd32f4xx_gpio.c，@10d02f4），骨架就是我们第四节的"先清后置"：

```
/* 伪代码骨架（原文件逻辑意译） */
GPIO_CTL(gpiox) &= ~GPIO_MODE_MASK(pin);            /* 清 CTL 的两位格子   */
GPIO_CTL(gpiox) |=  GPIO_MODE_SET(pin, mode);       /* 写模式              */
GPIO_PUD(gpiox) &= ~GPIO_PUPD_MASK(pin);            /* 清 PUD              */
GPIO_PUD(gpiox) |=  GPIO_PUPD_SET(pin, pupd);       /* 写上下拉            */
```

这里一个字都没超出本章第四节：**库函数就是手写寄存器的 for 循环封装**。S3 第五节对 `GPIO_Init` 说过的话，对 GD32 原样成立。掩码/置位宏 `GPIO_MODE_SET(n, mode)`、`GPIO_MODE_MASK(n)` 就定义在 gd32f4xx_gpio.h:319-324。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **七大寄存器只换名：CTL/OMODE/OSPD/PUD/ISTAT/OCTL/BOP；基址使能全同款；GD32 多 BC 清零、TG 翻转两件兵器；编码一字不差，先清后置照旧；复用找 AFSEL0/1，nibble 算法同款。**

延伸：GD32 AF 复用动画见 G2 动画；STM32 GPIO 对照见 S3；指针操作寄存器在 C2 详解。

实物实验

- **桌面点名（5 分钟）**：合上本文，对照 `.trellis/ref/gd32/gd32f4xx_gpio.h:52-63` 把 12 个寄存器的偏移从 0x00 默写到 0x2C，标出哪两个是 STM32 没有的。
- **翻译练习（10 分钟）**：打开 S3 的 00-blink main.c，把每行 STM32 寄存器操作翻译成 GD32 版（PF6→PF6，MODER→CTL……），再翻回本文核对。
- **上板（待基准板）**：拿到 GD32F450 板后，用 TG 寄存器写最小 blink（`GPIOF_TG = 1UL<<6` 循环+延时），与 OCTL 读-改-写版对比，示波器上看边沿应完全一致；量 PF6 翻转频率还能顺手验证 G1 的 200MHz 时钟树（**待上板实测**）。

常见坑

1. **拿着 STM32 名字搜 GD32 文档**：搜 "MODER""OTYPER" 永远搜不到：GD32 叫 CTL/OMODE。反向同理：看 GD32 例程里的 OCTL 别当成 STM32 的输出时钟配置（那是 RCC 的活）。
2. **以为 BOP 高 16 位不能用、非写 BC 不可**：两者都能清零（BOP 高 16 位、BC 低 16 位），官方库走 BC。但**置位只有 BOP 低 16 位**一条路，不存在"置位版 BC"。
3. **用 OCTL 读-改-写翻转**：GD32 有现成 TG 一条写搞定，还不用读：OCTL `^=` 的中断插队坑（S3 第二节）在 GD32 完全可以绕开。
4. **AF 写了 AFSEL 忘改 CTL**：CTL[1:0] 还是 00/01（输入/输出），外设接不进引脚：复用要成对（CTL=10 + AFSEL 选号），S3 的坑原样平移。
5. **速度档照抄 STM32 的"100MHz"**。GD32 第四档库里叫 `GPIO_OSPEED_MAX`，注释写 "more than 50MHz"；datasheet 的真名是 **IO_Speed = 200 MHz**（Table 4-28）。两边都不是 100MHz。照 STM32 的 100MHz 设计时序余量，在 GD32 上要么保守浪费、要么按错档算边沿。
6. **拿 PC13~PC15 / PI8 当普通高速脚**。这四个脚走 Power Switch 供电，输出模式**上限 2MHz**（datasheet 注 (3)）。配 200MHz 档也不会变快，只会让人以为"配了就有"。

短自测

1. 不看表默写：GD32 的七个核心 GPIO 寄存器（0x00~0x18）分别叫什么？对应 STM32 哪七个名字？
2. BC 和 BOP 高 16 位都能清零，那置位有几条路？走哪个寄存器？
3. STM32 里翻转一个引脚要几条指令？GD32 用哪个寄存器几条？各有什么隐患/优势？
4. PA11 要配复用功能，写 AFSEL0 还是 AFSEL1？nibble 索引怎么算？
5. GD32 的 GPIO_OSPEED_MAX（11）和 STM32F4 OSPEEDR 的 11 档（100MHz）有什么微妙差别？

▼ 参考答案（先自己想完再展开）

1. 0x00 CTL（↔MODER）、0x04 OMODE（↔OTYPER）、0x08 OSPD（↔OSPEEDR）、0x0C PUD（↔PUPDR）、0x10 ISTAT（↔IDR）、0x14 OCTL（↔ODR）、0x18 BOP（↔BSRR）。偏移、位宽、编码全同，只有名字换了（gd32f4xx_gpio.h:52-63）。 2. 置位只有**一条路**：BOP 低 16 位（BOP0~BOP15）。清零有两条路：BOP 高 16 位（CR0~CR15）或 BC 低 16 位，官方库 gpio_bit_reset 走 BC。不存在"独立置位寄存器"。 3. STM32 翻转要 ODR 读-改-写三条指令（ldr/eor/str），中间可被中断插队、丢状态；GD32 写 TG 对应脚位一条 str 原子完成，还不需要读，点灯循环连状态变量都省了。 4. PA11 属于 pin8~15，写 **AFSEL1**（对应 STM32 的 AFRH）；nibble 索引按 pin 对 8 取余（11 对 8 = 3），即 AFSEL1 的 bits[15:12]（第 3 个 nibble）。写成 bits[43:40] 或 pin 本身当索引是典型错。 5. STM32F4 的 11 档明确标 100MHz。GD32 的 LEVEL3 库里只叫 `GPIO_OSPEED_MAX`，注释写 "max speed more than 50MHz"，**库头文件不承诺数字**；但 datasheet Table 4-28 承诺了：**IO_Speed = 200 MHz**，并给出 200/170/130MHz（CL=10/30/50pF）三档上限。 所以差别有两层：**库含糊 vs datasheet 明确**，以及**标称 200MHz vs STM32 的 100MHz**。迁移时别按 100MHz 设计时序余量，也别忘了 datasheet 注 (4) 的 168MHz 天花板。

对照表：本章概念 → 仓库落点

本章说的	仓库里哪一行
12 个寄存器偏移点名（CTL 0x00 ~ TG 0x2C）	.trellis/ref/gd32/gd32f4xx_gpio.h:52-63（@10d02f4）
模式/上下拉编码 0/1/2/3 与 0/1/2	gd32f4xx_gpio.h:288-297
速度四档（第四档 MAX 标签）	gd32f4xx_gpio.h:336-345
速度四档的 datasheet 真名与按负载上限（2/25/50/**200** MHz；200/170/130MHz@10/30/50pF；168MHz 天花板）	GD32F407xx Datasheet Rev2.7 **Table 4-28**（p101，已核验）
PC13~PC15 / PI8 输出不得超 2MHz（Power Switch 供电）	同上 datasheet 注 (3)（已核验）
USART0_TX = AF7，候选脚 PA9/PB6（同 STM32）+ PA15（GD32 独有，STM32F407 PA15 无 USART1_TX）	GD32 datasheet **§2.6.6 Table 2-9 / 2-10**（p60-63）↔ STM32F407 datasheet Table 9（p66-67），双边已核验
九张端口 AF 表（Port A~I，Table 2-9 ~ 2-17）	datasheet §2.6.6（p60-68，已核验）
推挽/开漏编码	gd32f4xx_gpio.h:331-332
BOP 低 16 置位 + 高 16 复位布局	gd32f4xx_gpio.h:174-206
BC 独立清零 / TG 硬件翻转位定义	gd32f4xx_gpio.h:247-263 / :265-281
AFSEL0/1 每脚 4 位 + GPIO_AF_0~15	gd32f4xx_gpio.h:227-245 / :353-368
GPIO 基址 0x40020000、GPIOF +0x1400	gd32f4xx.h:313,341；gd32f4xx_gpio.h:41-49
RCU_AHB1EN bit0~8（GPIOA~I）	gd32f4xx_rcu.h:53,209-217
库函数 API 清单（mode_set/output_options_set/af_set/toggle）	gd32f4xx_gpio.h:372-404
PA8 AF0 配 CK_OUT0 实例	example_ckout_main.c:135（G1 第五节引用）
STM32 侧对照锚（MODER~AFRH 偏移与 PF6 流程）	code/stm32/00-blink/main.c；RM0090 §8.4

你做到了

- GD32F4xx 的 GPIO 从"又要学一套"变成"改七个宏名"，S3 的位级理解全额平移；
- 知道 GD32 的真增强在哪（BC/TG），点灯翻转从此一条指令；
- 会查 `.trellis/ref/gd32/gd32f4xx_gpio.h` 核对任何寄存器事实，为 G3 的 USART 对照备好方法。

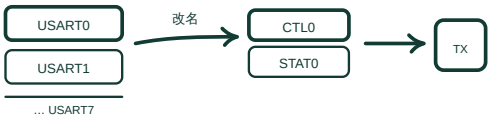
平移是全额：编码一字不差，上下拉、推挽开漏全照搬；那件兵器叫 TG，硬件翻转，一条 str 搞定，旁边还站着管独立清零的 BC，STM32F4 压根没这待遇。

✅ 下一站：G3 USART 增强点：gd32f4xx_usart.h 里比出的差异清单，接着把串口打印跑通。

通往单片机之路第 72 章

G3 USART 增强点：从命名到数量的差异清单

难度 ★★☆☆☆ · 约 30 分钟



本章罗盘

问题

STM32 串口代码搬到 GD32，编号、寄存器名、中断号各错位多少？按什么纪律翻译才不出错？

交付

一张 8 串口对照表（编号/总线/中断号/STM32 对应）加 CR→CTL、BRR→BAUD 的寄存器映射表与增强点位号清单。

边界

位级事实只核到 V3.3.3 头文件，CLEN/SCEN 等增强的时序语义待上板实测；USART5/UART6/7 为 GD32 独有，引脚分配只能查 GD32 数据手册，STM32 引脚表无从对照。

依据锚点 `a1 · f6 · mcu-code`

> 🎯 STM32F407 有 6 个串口（USART1/2/3/6 + UART4/5），GD32F4xx 有 **8 个**（USART0/1/2/5 + UART3/4/6/7）——多出来的不只是数量：寄存器命名也从 CR1/CR2/CR3 换成了 CTL0/CTL1/CTL2。本章把差异逐项点名，迁移时按表翻译。

本章精髓

1. 数量差异：GD32F4xx 有 **8 个串口**（USART0/1/2/5 + UART3/4/6/7），比 STM32F407 的 6 个多两个；命名从"USART1~6 + UART4/5"变成"USART0~5 + UART3/4/6/7"，**编号从 0 起不是从 1 起**。
2. 寄存器命名大换血：STM32 的 CR1/CR2/CR3 → GD32 的 **CTL0/CTL1/CTL2**；BRR → **BAUD**；GTPR → **GP**；SR/DR → **STAT/DATA**：功能等价，名字全变。
3. GD32 USART 有"不连续时钟"（CTL1.CLEN）与"智能卡"（CTL2.SCEN/NKEN）模式的增强（位级事实已按 `gd32f4xx_usart.h` V3.3.3 核验，见第四节）。

- **能理解**：为什么 GD32 从 0 起编号（与中断号对齐，USART0_IRQn=37）；为什么命名从 CR 改 CTL（GD32 库的统一风格，所有外设控制寄存器都叫 CTL）。
- **能用**：拿到 GD32 USART 工程，把 STM32 肌肉记忆的 CR1/CR2/CR3 翻译成 CTL0/1/2，把 BRR 翻译成 BAUD。

先修

- G1 RCU 时钟树、S7 USART（对照锚点）。

先跑起来（10 分钟 quick win）

用 `gd32f4xx.h` 查 USART0 基址（APB1 + 0x4400），对照 S7 的 USART1 基址（APB2 + 0x1000）：**总线不同、基址不同、但偏移布局神似**。

先押排门牌：串口数量 8 对 6，GD32 独有的是哪三个名字？这八路各自挂哪条总线，有没有跟大部队分开走的？押完看中断号地图。

小节结构

小节	内容	四件套
数量与编号	8 个串口 vs 6 个，0 起编号	配置
寄存器命名	CR1/2/3→CTL0/1/2，BRR→BAUD 全表	库解析
中断号地图	8 个 USART_IRQn 与总线归属	配置
增强点	GD32 独有功能（已核验）	库解析
库函数对照	gd32 USART 配置 API 与 SPL 对照	库解析

一、数量与编号：8 个串口，0 起编号

先讲清楚"为什么从 0 起"，后面那张表就不用背了。

GD32F4 是 ARM Cortex-M4F 内核（和 STM32F4 同门），中断控制器仍是 NVIC。它没有另起一套向量表：**中断号布局沿用 STM32F4**，改的只是外设命名——第一个串口从 USART1 叫成了 USART0，中断号 37 原地不动。

打个比方：小区的门牌号没换（37 号还是 37 号），只是楼名从"1 号楼"改叫"0 号楼"。

迁移时的实用推论只有一条：**别按编号找对应**。看到 GD32 的 USART0，先看它在哪条总线（APB2）、中断号多少（37），再回 STM32 表里找"同总线 + 同中断号"的那个，答案自然是 USART1。

GD32F4xx 串口全家福（中断号已在 `gd32f4xx.h` 核对）：

串口	类型	总线	中断号	STM32 对应
USART0	同步/异步	APB2	37	USART1（APB2）
USART1	同步/异步	APB1	38	USART2（APB1）
USART2	同步/异步	APB1	39	USART3（APB1）
USART5	同步/异步	APB2	71	**STM32F407 无**
UART3	异步	APB1	52	UART4（APB1）
UART4	异步	APB1	53	UART5（APB1）
UART6	异步	APB1	82	**STM32F407 无**
UART7	异步	APB1	83	**STM32F407 无**

三条差异一眼看出：

1. **编号从 0 起**：GD32 的 USART0 对应 STM32 的 USART1，迁移时别按编号找对应。
2. **多 3 个串口**：USART5（APB2）、UART6/7（APB1）是 GD32 独有，STM32F407 没有。
3. **USART5 挂 APB2**：与其他 USART 的 APB1 归属不同，时钟使能位在 RCU_APB2EN，别找错。

> 【注】 GD32 的 USART0/1/2/5 是"同步/异步"（带 SCK 引脚，可做 SPI 主从）；UART3/4/6/7 是"纯异步"（无 SCK），对应 STM32 的 USART vs UART 区分。

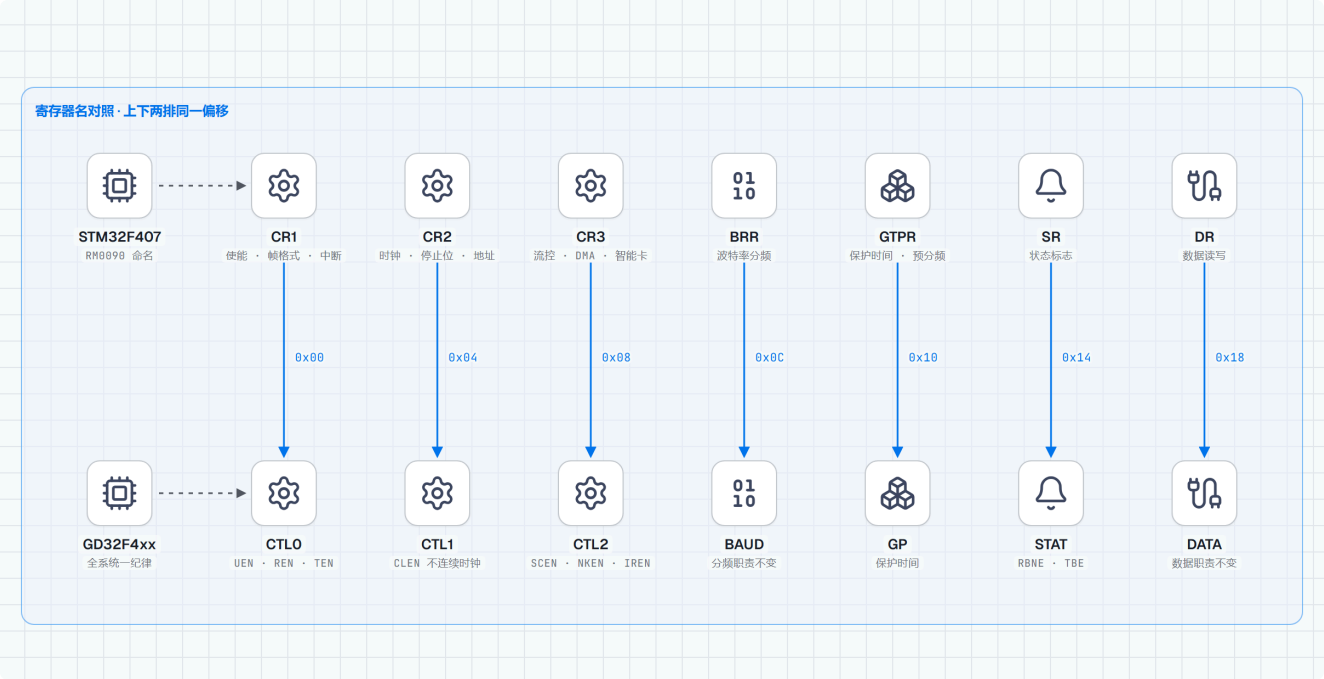
二、寄存器命名：CR1/2/3 → CTL0/1/2

改名的动机不是"换个花样"，而是 GD32 全系统一的命名纪律：**每一个外设的控制寄存器都叫 CTL，状态寄存器都叫 STAT，数据寄存器都叫 DATA**。

把它想成物业公司换门禁面板：以前每栋楼一个牌子（CR1/CR2/CR3、SR、DR），电工上门要背三套；现在全小区统一面板（CTL0/CTL1/CTL2、STAT、DATA），新来的电工看一次就懂。

代价是"肌肉记忆失效"：你写惯的 `USART_CR1` 在 GD32 里编译不过，得换成 `USART_CTL0`。收益是"一个规律管八个串口"：`USART_CTL0`、`UART_CTL0` 写法一致，换外设不用重新记名字。

改名纪律落到 USART 是七条偏移轨：上排是 STM32 的名字，下排是 GD32 的名字，同一列就是同一个偏移。



USART 寄存器改名对照：CR1/CR2/CR3/BRR/GTPR/SR/DR 与 CTL0/CTL1/CTL2/BAUD/GP/STAT/DATA 逐偏移对齐

GD32 库的统一命名风格：所有外设的控制寄存器都叫 **CTL**（Control），STM32 叫 **CR**（Control Register）。USART 的三组控制寄存器命名映射：

STM32	GD32	偏移	功能
CR1	**CTL0**	0x00	使能/帧格式/中断使能
CR2	**CTL1**	0x04	时钟/停止位/地址
CR3	**CTL2**	0x08	流控/DMA/智能卡
BRR	**BAUD**	0x0C	波特率分频
GTPR	**GP**	0x10	保护时间/预分频
SR	**STAT**	0x14	状态标志
DR	**DATA**	0x18	数据读写

偏移布局与 STM32 **完全一致**（0x00~0x18 逐行对齐）：功能等价，只是名字换了。迁移纪律：**按偏移不按名字**，0x00 永远是主控制寄存器，0x0C 永远是波特率。

位级命名差异（已知方向，具体位号以 `gd32f4xx_usart.h` 为准）：

STM32 CR1 位	GD32 CTL0 位（方向）	说明
UE（bit13）	**UEN**（使能）	名字加 EN 后缀
RE/TE	**REN/TEN**	同上加 EN
RXNE/TXE	**RBNE/TBE**	接收缓冲/发送缓冲，命名更直白
TC	**TC**（不变）	传输完成
IDLE	**IDLE**（不变）	空闲

GD32 的位命名风格有两条规律。**动作位加 EN 后缀**（UEN/REN/TEN）；**状态位用 B（Buffer）替代 X**（RBNE vs RXNE = Receive Buffer Not Empty vs Receive Not Empty）。

三、中断号地图：8 个 IRQn

GD32F4xx 的 USART 中断号已在 `gd32f4xx.h` 的 IRQn_Type 枚举核对（非连续编号）：

```
USART0_IRQn = 37,    // APB2，对应 STM32 USART1
USART1_IRQn = 38,    // APB1，对应 STM32 USART2
USART2_IRQn = 39,    // APB1，对应 STM32 USART3
UART3_IRQn  = 52,    // APB1，对应 STM32 UART4
UART4_IRQn  = 53,    // APB1，对应 STM32 UART5
USART5_IRQn = 71,    // APB2，GD32 独有
UART6_IRQn  = 82,    // APB1，GD32 独有
UART7_IRQn  = 83,    // APB1，GD32 独有
```

中断号不连续：39→52 跳了 13，53→71 跳了 18。GD32 的中断向量表布局与 STM32 不同。迁移时**不要按 STM32 的中断号找 GD32 的**，按上表对照。

四、增强点：GD32 独有功能（已按 V3.3.3 头文件核验）

GD32F4xx 的 USART 相比 STM32F407 的增强点，位级事实已对照 `gd32f4xx_usart.h`（V3.3.3，本地 .trellis/ref/gd32/fw）核验：

1. **USART5 的存在**：多一个同步/异步串口（APB2），STM32F407 没有对应（gd32f4xx_usart.h:47），可用于额外的高速同步通信。
2. **UART6/7 的存在**：多两个异步串口（APB1，gd32f4xx_usart.h:45-46），多串口产品（如 8 通道数据采集）的直接收益。
3. **不连续时钟（CLEN）**：`USART_CTL1_CLEN` BIT(8)（gd32f4xx_usart.h:104）：CTL1 的 CK 长度位，控制 8 位帧出 7/8 个 CK 脉冲（`USART_CLEN_NONE`）还是 8/9 个（`USART_CLEN_EN`，:290-292）。STM32F407 无此位。
4. **智能卡增强（SCEN/NKEN）**：`USART_CTL2_SCEN` BIT(5) 开智能卡模式（:117）、`USART_CTL2_NKEN` BIT(4) 选 NACK 使能（:116），都在 CTL2，对应 STM32 CR3 的位置但字段不同。
5. **IrDA 模式（IREN）**：`USART_CTL2_IREN` BIT(1)（:113），与 STM32 的 CR3.IREN 同语义。

> 【注】 以上位级事实已对照 `gd32f4xx_usart.h`（V3.3.3）逐一核验并标注行号；与用户手册的语义一致性（如时序细节）仍以上板实测为准。

五、库函数对照：GD32 与 SPL

GD32 标准外设库的 USART API 与 STM32 SPL 的对照（函数名风格不同但参数对应）：

SPL（STM32）	GD32 库	说明
`USART_Init(USART1, &cfg)`	`usart_init(USART0, baud, word_len, stop, parity, mode)`	GD32 拆成参数不传结构体
`USART_Cmd(USART1, ENABLE)`	`usart_enable(USART0)`	名字加 enable/disable
`USART_SendData(USART1, b)`	`usart_data_transmit(USART0, b)`	动词更明确
`USART_ReceiveData(USART1)`	`usart_data_receive(USART0)`	同上
`USART_GetFlagStatus(USART1, TXE)`	`usart_flag_get(USART0, USART_FLAG_TBE)`	TBE 替代 TXE
`USART_ITConfig(USART1, RXNE, ENABLE)`	`usart_interrupt_enable(USART0, USART_INT_RBNE)`	RBNE 替代 RXNE

GD32 库的风格：**函数名小写下划线**（`usart_init` vs `USART_Init`）；**参数不打包结构体**（直接传 baud/word_len 等）；**宏名加 FLAG/INT 前缀**。读库姿势照旧（S15 SPL 解剖）：拿到 `usart_init` 先问"它动了 CTL0/1/2 的哪几位"，按偏移对答案。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **8 个串口 0 起编号，CTL0/1/2 替代 CR1/2/3；BAUD 替代 BRR，STAT/DATA 替代 SR/DR：偏移不变名字变，迁移按偏移不按名。**

延伸：STM32 USART 对照见 S7；USART 帧波形动画见 S7 动画；GD32 外设编号差异总览见 G4。

实物实验

- 在 `code/gd32/01-rcu-clock` 工程上加 USART0 初始化（APB2 时钟使能），串口打印"GD32 USART0"，验证基址与命名翻译。

常见坑

- **按 STM32 编号找 GD32 串口**：USART1 在 GD32 是 USART0，编号差 1，找错基址全乱。
- **USART5 时钟在 APB2**：其他 USART 在 APB1，USART5 单独在 APB2，时钟使能位找错不工作。
- **中断号不连续**：39→52 跳 13，53→71 跳 18，按 STM32 中断号填 GD32 向量表会越界。
- **位名肌肉记忆**：RXNE→RBNE、TXE→TBE：写 `USART_FLAG_RXNE` 在 GD32 编不过，要用 `USART_FLAG_RBNE`。
- **多出来的串口未查手册**：USART5/UART6/7 是 GD32 独有，引脚分配以 GD32 datasheet 为准，别套 STM32 引脚表。

短自测

1. GD32F4xx 有几个串口？STM32F407 有几个？多出来的是哪几个？

▼ 看答案
GD32F4xx 有 8 个（USART0/1/2/5 + UART3/4/6/7），STM32F407 有 6 个（USART1/2/3/6 + UART4/5）。GD32 多 3 个：USART5（APB2 同步/异步）、UART6（APB1 异步）、UART7（APB1 异步）。

2. STM32 的 CR1/CR2/CR3/BRR 在 GD32 里叫什么？偏移布局是否一致？

▼ 看答案
GD32 叫 CTL0/CTL1/CTL2/BAUD。偏移布局完全一致：0x00 主控制、0x04 时钟/停止位、0x08 流控/DMA、0x0C 波特率。迁移纪律：按偏移不按名字，0x00 永远是主控制寄存器。

3. GD32 USART0 的中断号是多少？对应 STM32 的哪个串口？为什么总线不同？

▼ 看答案
GD32 USART0_IRQn=37，对应 STM32 的 USART1（STM32 USART1 也在 APB2）。两者都挂 APB2（高速总线），因为 USART0/1 在 GD32 是同步/异步串口需要较高时钟，APB2 比 APB1 快。

4. RXNE 和 TXE 在 GD32 里叫什么？这种命名变化的规律是什么？

▼ 看答案
GD32 叫 RBNE（Receive Buffer Not Empty）和 TBE（Transmit Buffer Empty）。规律：状态位用 B（Buffer）替代 X（STM32 的 RXNE = Receive Not Empty，GD32 加了 Buffer 更直白）；动作位加 EN 后缀（UEN/REN/TEN）。迁移时把 STM32 的 RXNE/TXE 宏名换成 RBNE/TBE。

5. GD32 库的 `usart_init` 与 STM32 SPL 的 `USART_Init` 在 API 风格上有什么差异？

▼ 看答案
STM32 SPL 用结构体打包（USART_InitTypeDef 含 baud/word_len/stop/parity/mode 传给 USART_Init）；GD32 库直接传参数（usart_init(USART0, baud, word_len, stop, parity, mode) 不打包）。命名风格：STM32 驼峰大写（USART_Init），GD32 小写下划线（usart_init）。功能等价，代码风格不同。

对照表：本章概念 → 仓库与上游落点

概念	落点
8 个串口中断号	`gd32f4xx.h` IRQn_Type 枚举（USART0_IRQn=37 等）
寄存器命名映射	`gd32f4xx_usart.h`（V3.3.3，本地已缓存核验）；偏移对照 STM32 RM0090
USART0 基址	`gd32f4xx.h` USART_BASE = APB1 + 0x4400
库函数对照	GD32 V3.3.3 库 `usart_init` / `usart_enable` vs STM32 SPL `USART_Init` / `USART_Cmd`
STM32 对照锚点	S7 USART 全章
增强点（CLEN/SCEN/NKEN/IREN）	`gd32f4xx_usart.h`（V3.3.3 已核验：CLEN=CTL1:8、SCEN=CTL2:5、NKEN=CTL2:4、IREN=CTL2:1）

你做到了

- GD32 USART 从"换个名字的 STM32"变成"有明确差异清单的独立芯片"；
- 迁移时按偏移不按名字的纪律到手，这套方法在 G4 外设差异上直接复用。

门牌对上：GD32 独有 USART5、UART6、UART7；USART0 与 USART5 挂 APB2，其余六路 APB1，找对应按总线加中断号，别按编号，起跑线挪到 0 为的也是跟中断号对齐。
CR1/CR2/CR3 改姓 CTL0/CTL1/CTL2，BRR 换 BAUD、SR/DR 换 STAT/DATA，偏移布局神似，迁移按偏移不按名字。

✅ 下一站：G4 GD32 外设差异：USBHS/EXMC/CAN 逐项过堂，看"有没有、一不一样"。

通往单片机之路第 73 章

G4 GD32 外设差异：有没有与一不一样

难度 ★★☆☆☆ · 约 30 分钟

本章罗盘

问题

把 STM32F407 工程移植到 GD32F4xx，哪些外设根本没有、哪些看着同名位宽却藏坑？

交付

一张外设「有没有」对照表加移植三步清单，附位级实证：CAN_BT 复位值 0x01230000 全等、SDIO DIV 第 9 位藏在 bit31。

边界

USBHS 与 OTG_HS 的寄存器位级未逐位断言，以 drv_usb_regs.h 与 UM §29 为准；ULPI 个别 D 线候选脚在 datasheet 宽表抽取中有歧义，未逐一断言；结论限定 F4xx↔F407，子型号以各自 datasheet 为准。

依据锚点 `a1` · `a10` · `a11` · `b6` · `f6` · `mcu-anim`

> 🎯 G1~G3 逐个外设比了"同名不同姓"；这一章把 GD32F4xx 与 STM32F407 的外设全家拉到一张表上——**有的 GD32 多了、有的 STM32 没有、有的都有但内部有差异**。迁移前先查这张表，别等编译/运行时才发现"这个外设我没有"。

本章精髓

1. 有没有：GD32F4xx 比 STM32F407 多 USBHS（高速 USB）、TRNG（真随机数）、更多串口（G3）；STM32F407 有而 GD32 可能没有或不同的：OTG_FS/OTG_HS（GD32 用 USBHS 替代）。
2. 一不一样：都有 CAN/SDIO/EXMC(FSMC)/ENET，但寄存器命名与位布局有差异：GD32 统一用 CTL 命名风格（G3 已见）。位级差异已按官方库头文件（V3.3.3）+ **用户手册 Rev3.0** 双层核验。CAN 位时序**逐位相同到连复位值 0x0123 0000 都一样**；ENET 描述符同为 4 字/8 字双模；**SDIO 分频器 GD32 比 STM32 宽一位**（DIV[8:0] vs CLKDIV[7:0]，第 9 位藏在 bit31）。
3. 引脚可以不动：USBHS 的 **ULPI 12 根信号与 STM32F407 OTG_HS_ULPI 同脚同 AF（AF10）**。跑 ULPI PHY 的板子换芯片不用改布线，换的是固件库。
4. 迁移纪律：先查"有没有"（外设存在性），再查"一不一样"（寄存器映射**与位宽**），最后查"引脚分配"（datasheet）。三步缺一步就翻车。

- **能理解**：为什么"有没有"比"一不一样"更优先：外设不存在，寄存器映射再对也白搭。
- **能用**：拿到一个 STM32 工程想移植到 GD32，先用这张表扫一遍外设存在性，再逐个查寄存器。

先修

- G1 RCU 时钟树、G2 GPIO、G3 USART。

先跑起来（10 分钟 quick win）

打开 `gd32f4xx.h`，搜索 `USBHS` / `EXMC` / `CAN` / `ENET` / `TRNG` 的基址定义：存在即有这个外设，搜不到即没有（或叫别的名字）。这是"有没有"的最快自查法。

先押个家底：GD32F4xx 比 STM32F407 多出哪些硬货？两边都有的 SDIO，分频器的宽窄又差在哪一位？

小节结构

小节	内容	四件套
有没有对照表	GD32 vs STM32 外设存在性全表	配置
USBHS 替代 OTG	高速 USB 的方案差异	库解析
EXMC 替代 FSMC	外部存储器控制器的命名与功能差异	库解析
CAN/ENET/SDIO	两者都有但有差异的外设	库解析
TRNG 等独有	GD32 独有外设简介	配置
移植三步清单	存在性→寄存器→引脚	代码分析

一、有没有对照表：GD32F4xx vs STM32F407

GD32 vs STM32 外设对照

外设	STM32F407	GD32F407	差异
SPI	3 个（SPI1/2/3）	3 个（SPI0/1/2）	编号差 1
I2C	3 个（I2C1/2/3）	3 个（I2C0/1/2）	编号差 1
USART	6 个（USART1-3, UART4-5）	6 个（USART0-2, UART3-4）	编号差 1
TIM	14 个（TIM1-14）	14 个（TIM0-13）	编号差 1

命名规律

STM32 从 1 开始编号
GD32 从 0 开始编号
数量相同，编号差 1

寄存器差异

大部分寄存器同名同义
少数位域命名不同
详见芯片参考手册

时序差异（关键）

SPI/I2C 时序参数略有不同（建立/保持时间、时钟极性）—— **代码移植时必须核对时序**
详见 [S11 I2C 时序](../stm32/11-i2c.md) 与 [S12 SPI 时序](../stm32/12-spi.md)

移植口诀：**编号减一，时序核对，寄存器查表**——三步完成 STM32 → GD32 迁移

GD32 vs STM32 外设差异对照

外设	STM32F407	GD32F4xx	差异类型
GPIO	A~I（9 组）	A~I（9 组）	命名不同（G2），数量同
USART/UART	6 个（USART1/2/3/6 + UART4/5）	**8 个**（USART0/1/2/5 + UART3/4/6/7）	GD32 多 3 个（G3）
SPI	3 个（SPI1/2/3）	3 个（SPI0/1/2）	编号差 1，数量同（时序差异见 S12）
I2C	3 个（I2C1/2/3）	3 个（I2C0/1/2）	编号差 1，数量同（时序差异见 S11）
TIM	14 个（TIM1~14）	14 个（TIMER0~13）	命名 TIM→TIMER，数量同
ADC	3 个（ADC1/2/3）	3 个（ADC0/1/2）	编号差 1，数量同
DAC	2 通道（DAC1/2）	2 通道（DAC0/1）	编号差 1，数量同
DMA	2 个（DMA1/2）	2 个（DMA0/1）	编号差 1，数量同
USB	OTG_FS + OTG_HS	**USBHS**（替代 OTG_HS）	方案不同
EXMC	FSMC（外部存储器）	**EXMC**（改名）	命名不同，功能类似
CAN	2 个（CAN1/2）	2 个（CAN0/1）	编号差 1，可能有位差异
ENET	有（以太网 MAC）	有（ENET）	两者都有，命名已核验（见第四节）
SDIO	有	有（SDIO）	两者都有，命名已核验（见第四节）
TRNG	**无**	**有**（真随机数发生器）	GD32 独有
RTC	有	有	命名同；寄存器布局与 STM32 同构（TIME/DATE/CTL/STAT/PSC/WUT/ALRM0TD，gd32f4xx_rtc.h:45-55），细节已按头文件核验
WDT/WWDT	有	有（FWDGT/WWDGT）	命名改 WDT→WDGT

三条结论：

1. **GD32 编号从 0 起**：几乎所有外设编号比 STM32 小 1（SPI1→SPI0、TIM1→TIMER0）。G3 已见的 USART 模式是全系列通用规律。
2. **GD32 多 USBHS + TRNG**：STM32F407 的 OTG_HS 在 GD32 换成 USBHS（方案不同），TRNG 是 GD32 独有。
3. **命名风格统一**：GD32 所有控制寄存器叫 CTL（G3 已见），看门狗叫 WDGT（Watchdog Timer 加 T），FSMC 叫 EXMC，统一加长命名。

二、USBHS 替代 OTG：高速 USB 的方案差异

STM32F407 用 **OTG_FS + OTG_HS**（On-The-Go 全速+高速 USB）；GD32F4xx 用 **USBHS**（USB High Speed）。两者都支持高速 USB（480Mbps），但实现方案与寄存器布局不同：

维度	STM32F407 OTG_HS	GD32F4xx USBHS
命名	OTG_HS（OTG 兼容）	USBHS（专用高速）
寄存器前缀	OTG_HS_	USBHS_
OTG 模式	支持（主机/设备双模）	有（V3.3.3 库含 device 与 host 双栈，`drv_usb_dev.h` / `drv_usb_host.h`）
ULPI 接口	有（外接 ULPI PHY 跑高速）	有，**引脚已核验**：12 根信号全在 AF10 组，与 STM32 一一对脚（见下表）
ULPI 时钟源	—	外部 ULPI PHY 时钟 **或** 片内 CK48M，由 `USBHS_GUSBCS.EMBPHY` 选（UM §4.2.2）

迁移含义：STM32 的 USB 代码（OTG_HS 驱动）不能直接套 GD32，寄存器名与方案都不同。GD32 有自己的 USBHS 库（V3.3.3 为 `GD32F4xx_usb_library`，寄存器宏在 `driver/Include/drv_usb_regs.h`，不再叫 `gd32f4xx_usbhs.h`），移植要换库。

但引脚可以不动。 datasheet §2.6.5/§2.6.6 逐脚核对的结果是：GD32F407 的 ULPI 12 根信号，与 STM32F407 OTG_HS_ULPI **同脚同 AF 号（AF10）**——两颗芯片共用同一套 ULPI 引脚排布：

ULPI 信号	GD32F407 引脚	对 STM32F407
ULPI_CK	PA5	同脚（OTG_HS_ULPI_CK）
ULPI_D0 ~ D7	PA3 / PB0 / PB1 / PB10 / PB11 / PB12 / PB13 / PB5	八根全同脚
ULPI_DIR	PC2（另有候选 **PI11**）	同脚
ULPI_STP	PC0	同脚
ULPI_NXT	PC3（另有候选 **PH4**，与 I2C1_SCL 共脚）	同脚

硬件层面这意味着：**跑 ULPI PHY 的 STM32F407 板子，PCB 可以原样换上 GD32F407**，换的是固件库，不是布线。

> 【注】 本表只核到"哪根信号在哪个脚、哪个 AF 号"这一层。USBHS 与 OTG_HS 的**寄存器位级**差异仍以 `GD32F4xx_usb_library/driver/drv_usb_regs.h` 与 UM §29 为准（**OTG 双模已由官方库 host+device 双栈证据背书**，V3.3.3）。几个 D 线还有额外候选脚（文字版 datasheet 的宽表抽取有歧义，未逐一断言）。真要排 PCB，请回查 Table 2-9 ~ 2-17 的 AF10 列。

三、EXMC 替代 FSMC：外部存储器控制器的改名

STM32F407 叫 **FSMC**（Flexible Static Memory Controller），GD32F4xx 叫 **EXMC**（External Memory Controller）。两者功能类似，都用来驱动 SRAM/NOR/PSRAM/NAND/SDRAM，但命名与寄存器布局有差异：

维度	STM32F407 FSMC	GD32F4xx EXMC
命名	FSMC	EXMC
寄存器前缀	FSMC_	EXMC_
SDRAM 支持	有（Bank 5/6）	有（2 个 SDRAM 设备：SDCTL0/1，gd32f4xx_exmc.h:264）
NOR/PSRAM	有（Bank 1 四区）	有（EXMC_NOR_PSRAM 映射区）
NAND	有（Bank 2/3）	有（NAND/PC card 3 组：NPCTL1/2/3，gd32f4xx_exmc.h:203）

迁移含义：FSMC 的寄存器名（FSMC_BCR/FSMC_BTR 等）在 GD32 换成 EXMC 前缀，功能映射类似但位级布局可能不同。移植时按偏移对照，别按名字。

> 【注】 EXMC 的 bank 分区与时序寄存器位级以 `gd32f4xx_exmc.h`（V3.3.3，本地 .trellis/ref/gd32/fw）为准：NOR/PSRAM、NAND/PC card、SDRAM 三段映射清晰，位级对照已核验；SDRAM 为 2 设备（SDCTL0/1），NAND/PC 为 3 组寄存器。

四、CAN/ENET/SDIO：两者都有但有差异

三个外设 STM32F407 与 GD32F4xx 都有，寄存器命名已按 V3.3.3 头文件核验（CTL 风格统一）。这一节把 UM 也翻了，**位级结论比"名字不同"有意思得多**：

外设	STM32F407	GD32F4xx	已核验的差异
CAN	bxCAN（CAN1/2）	CAN（CAN0/1）	编号差 1；寄存器名 CR→CTL（CAN_CTL@0x00，gd32f4xx_can.h:46）。**位时序寄存器逐位相同**：CAN_BT@0x1C 复位值 **0x0123 0000**，SCMOD b31 / LCMOD b30 / SJW[1:0] b25:24 / BS2[2:0] b22:20 / BS1[3:0] b19:16 / BAUDPSC[9:0] b9:0——与 STM32 CAN_BTR 的 SILM/LBKM/SJW/TS2/TS1/BRP **位位置与复位值全等**（UM §26.4.8 p804 ↔ RM0090 §32.9 p1104）
ENET	以太网 MAC	ENET	命名同方向；MAC/DMA 寄存器同构（gd32f4xx_enet.h:180-188,431）。**描述符格式已核验**：DFM=0 → 4 字描述符（Descriptor0-3）、DFM=1 → 8 字增强描述符（Descriptor0-7），Tx/Rx 两侧都是这套双模，每个描述符最多指两个 buffer——与 STM32 ETH 的 normal/enhanced 双模同款（UM §27 p833-854）
SDIO	SDIO	SDIO	命名同；寄存器名改 CTL 风格（PWRCTL/CLKCTL/CMDCTL，gd32f4xx_sdio.h:44-47）。**分频位宽真有差异**：见下文

CAN 的"复位值指纹"

最能说明两家关系的不是名字，是**复位值**。CAN_BT 和 CAN_BTR 都在偏移 0x1C、复位值都是 `0x0123 0000`，这个数字拆开看是 SJW=0、BS2=1、BS1=2、BAUDPSC=0 的一组默认位时序。**同一个偏移、同一组位域、同一个非零复位值**，巧合不到这个程度：位时序这块是逐位复刻的。好消息是迁移时 CAN 波特率计算公式可以**原样搬过来**：`tq=(1+BAUDPSC)×tPCLK1`、`tBS1=(1+BS1)×tq`、`tBS2=(1+BS2)×tq`（UM §26.3.7）。连算出来的寄存器值都不用改。

SDIO 的分频器比 STM32 宽一位

这是本节唯一一处**真的不一样**，而且藏得很隐蔽：

项	STM32F407 SDIO_CLKCR	GD32F4xx SDIO_CLKCTL
分频位宽	CLKDIV[7:0]，**8 位**（0~255）	DIV[**8**:0]，**9 位**（0~511）
第 9 位在哪	—	**DIV[8] 在 bit31**，正好落在 ST 的保留位区
分频公式	SDIO_CK = SDIOCLK / (CLKDIV + 2)	SDIO_CLK = SDIOCLK / (**DIV[8:0]** + 2)——公式同款
其余位	HWFC_EN b14 / NEGEDGE b13 / WIDBUS b12:11 / BYPASS b10 / PWRSAV b9 / CLKEN b8	HWCLKEN b14 / CLKEDGE b13 / BUSMODE b12:11 / CLKBYP b10 / CLKPWRSAV b9 / CLKEN b8——**位位置全同，只换名**

凭证：UM §24.8.2（p700-701）↔ RM0090 §31.9.2。

迁移含义，两条。其一，从 GD32 搬到 STM32 时，若原代码用了 DIV > 255，**bit31 会被当保留位丢掉**，SDIO_CK 悄悄变快（分频变小）。卡识别阶段要求 < 400kHz，这正是最容易踩的地方。其二，反方向从 STM32 搬到 GD32 完全安全，高位本来就是 0。

这也是"**按偏移对照还不够，还要按位宽对照**"的典型案例：寄存器名、偏移、公式三样全同，偏偏位宽多一位。

这三个外设的移植策略：**功能存在性确认后，逐个寄存器按偏移 + 按位宽双重对照**。CAN 的 CR1→CTL0 命名迁移同 G3 USART 的规律（GD32 统一 CTL 命名）。

五、TRNG 等独有：GD32 独有外设

GD32F4xx 有而 STM32F407 没有的外设：

- **TRNG**（True Random Number Generator）：真随机数发生器，用噪声源生成真随机数。STM32F407 没有独立 TRNG（只有用 ADC 噪声的软件方案）。适合加密密钥生成。
- **USART5/UART6/7**：多 3 个串口（G3 已详述）。
- **可能的额外外设**：GD32F4xx 的某些子型号可能有 STM32 没有的外设，以具体型号 datasheet 为准。

TRNG 的存在让 GD32 在安全/加密场景比 STM32F407 多一个硬件基础：不用靠 ADC 噪声凑随机数，直接读 TRNG 寄存器出真随机。

六、移植三步清单：存在性 → 寄存器 → 引脚

把一个 STM32F407 工程移植到 GD32F4xx，三步检查：

1. **存在性检查**：扫工程用到的外设（GPIO/USART/SPI/USB/...），逐个查第一节"有没有"表：GD32 没有的要换方案（如 OTG_HS→USBHS），GD32 多的可利用（如 TRNG）。
2. **寄存器映射检查**：对存在的外设，逐个查寄存器命名（CR→CTL、BRR→BAUD）与偏移，按偏移不按名字（G3 纪律）。位级差异以 `gd32f4xx_*.h` 头文件为准。
3. **引脚分配检查**：查 GD32F4xx datasheet 的引脚复用表：GD32 的 AF 编号可能与 STM32 不同（G2 已见），同一功能可能在不同引脚。

三步走完，移植的"大坑"基本扫清。剩下的细节（时钟树、电压档、Flash 等待）在 G1 已述。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **GD32 编号从 0 起，USBHS 替代 OTG，EXMC 替代 FSMC，TRNG 是独有；移植三步：有没有→一不一样→引脚在哪。**

实物实验

- 在 `code/gd32/01-rcu-clock` 工程基础上，尝试加一个 USART0 初始化（G3 的命名翻译）+ 一个 CAN0 初始化（CR→CTL0 迁移），验证"按偏移不按名字"的移植纪律。

常见坑

- **按 STM32 外设名找 GD32**：FSMC 在 GD32 叫 EXMC，OTG_HS 叫 USBHS：名字不同，搜不到以为"没有"。

- **编号差 1 忘换**：STM32 SPI1 → GD32 SPI0、TIM1 → TIMER0，编号不换基址全错。
- **USB 方案直接套代码**：STM32 的 OTG_HS 驱动不能直接套 GD32 USBHS。方案不同，要换库。
- **TRNG 当 ADC 噪声用**：GD32 有真 TRNG，别再用 ADC 噪声凑随机数：硬件 TRNG 更安全更快。
- **引脚复用号不查 GD32 表**：GD32 的 AF 编号可能与 STM32 不同（G2 已见），同一功能可能在不同引脚。查 GD32 datasheet，别套 STM32 引脚表。
- **只对偏移不对位宽**：SDIO 分频就是反例：寄存器名、偏移、`/(DIV+2)` 公式三样全同，偏偏 GD32 的 DIV 多一位且第 9 位藏在 bit31。从 GD32 往 STM32 搬时这一位会被当保留位丢掉，SDIO_CK 悄悄变快。卡识别阶段要求 < 400kHz，正是最容易炸的地方。

短自测

1. GD32F4xx 有而 STM32F407 没有的外设有哪几个？

▼ 看答案
TRNG（真随机数发生器）+ USART5/UART6/7（多 3 个串口，G3 已详述）。TRNG 让 GD32 在加密场景多一个硬件基础，不用靠 ADC 噪声凑随机数。

2. STM32 的 FSMC 在 GD32 里叫什么？OTG_HS 呢？

▼ 看答案
FSMC → EXMC（External Memory Controller）；OTG_HS → USBHS（USB High Speed）。两者功能类似（驱动外部存储器/高速 USB），但命名与寄存器布局不同，移植要换库、按偏移对照。不过 **ULPI 的 12 根引脚两边同脚同 AF（AF10）**：改固件不改布线。

3. GD32 的外设编号与 STM32 有什么规律性差异？举三例。

▼ 看答案
GD32 几乎所有外设编号都从 0 起，比 STM32 小 1：SPI1→SPI0、TIM1→TIMER0、USART1→USART0、ADC1→ADC0、CAN1→CAN0、DMA1→DMA0。迁移时别按 STM32 的编号去找 GD32 基址。

4. 把一个 STM32F407 工程移植到 GD32F4xx，三步检查是什么？

▼ 看答案
①存在性检查：扫工程用到的外设，查"有没有"表：GD32 没有的换方案（OTG→USBHS），GD32 多的可利用（TRNG）。②寄存器映射检查：对存在的外设查命名（CR→CTL、BRR→BAUD）与偏移，按偏移不按名字，位级以头文件为准，**还要对位宽**（SDIO 的 DIV 多一位）。③引脚分配检查：查 GD32 datasheet 的引脚复用表，AF 编号可能不同，同一功能可能在不同引脚。

5. GD32 的 TRNG 相比 STM32F407 的随机数方案有什么优势？

▼ 看答案
STM32F407 没有独立 TRNG，要用 ADC 噪声做软件随机数方案（质量差、有偏、慢）。GD32 有硬件 TRNG（真随机数发生器），用物理噪声源生成真随机数，质量高、无偏、快，适合加密密钥生成。移植时把 STM32 的 ADC 噪声随机数方案换成 GD32 TRNG 直接读。

6. CAN_BT 和 CAN_BTR 哪些地方相同到"不可能是巧合"？这对迁移意味着什么？

▼ 看答案
偏移同为 **0x1C**，复位值同为 **0x0123 0000**，六个字段的位位置逐位相同（b31 静默模式 / b30 回环 / b25:24 SJW / b22:20 BS2↔TS2 / b19:16 BS1↔TS1 / b9:0 BAUDPSC↔BRP）。连位时序计算公式都一样。 这意味着 **CAN 波特率算出来的寄存器值可以原样搬**，只需改寄存器名和外设编号（CAN1→CAN0）。

7. SDIO 的分频器两边差在哪？哪个迁移方向有风险？

▼ 看答案
STM32 是 CLKDIV[7:0]（8 位，0~255），GD32 是 DIV[8:0]（9 位，0~511），第 9 位 **DIV[8] 单独放在 bit31**，正好落在 ST 的保留位区。公式两边同为 `SDIOCLK/(DIV+2)`。风险方向是 **GD32 → STM32**：原来用了 DIV > 255 的配置，bit31 被当保留位丢掉后分频变小、SDIO_CK 变快，卡识别阶段（要求 < 400kHz）会直接失败。反方向（STM32 → GD32）安全，高位本来是 0。

对照表：本章概念 → 仓库与上游落点

概念	落点
外设存在性对照表	本章第一节；`gd32f4xx.h` 基址定义
USBHS 替代 OTG_HS	`GD32F4xx_usb_library/driver/Include/drv_usb_regs.h`（V3.3.3，本地已缓存并核验：device+host 双栈）
EXMC 替代 FSMC	`gd32f4xx_exmc.h`（V3.3.3，本地已缓存并核验：NOR/PSRAM、NAND/PC card、SDRAM 三段映射）
编号差 1 规律	G3 USART USART0 vs USART1；G2 GPIO
移植三步清单	本章第六节；G1 RCU 时钟树差异
STM32 对照锚点	S3 GPIO、S7 USART
ULPI 12 根信号同脚同 AF10	GD32F407xx Datasheet Rev2.7 §2.6.5/§2.6.6（已核验：PA3/PA5/PB0/PB1/PB5/PB10~PB13/PC0/PC2/PC3，DIR 另有 PI11、NXT 另有 PH4）
CAN_BT 与 CAN_BTR 位级全等（含复位值 0x0123 0000）	**UM Rev3.0 §26.4.8 p804 ↔ RM0090 §32.9 p1104**（双边已核验）
CAN 位时序公式 tq/tBS1/tBS2	UM Rev3.0 §26.3.7 p791（已核验）
ENET 描述符 4 字（DFM=0）/ 8 字增强（DFM=1），Tx/Rx 双侧	UM Rev3.0 §27 p833-854（已核验）
SDIO DIV[8:0] 九位、DIV[8] 在 bit31、`SDIOCLK/(DIV+2)`	**UM Rev3.0 §24.8.2 p700-701 ↔ RM0090 §31.9.2 p1061**（双边已核验）
ULPI 时钟源由 USBHS_GUSBCS.EMBPHY 选（外部 PHY 时钟 / CK48M）	UM Rev3.0 §4.2.2 p94（已核验）
命名风格 CTL 统一	G3 USART CR→CTL；S15 SPL 解剖 读库姿势

延伸阅读

两家"同脚同 AF、位时序连复位值都一样"不是巧合，是都按同一份规范来：

- **[B5]** ULPI 1.1 — USBHS 那 12 根信号的接口规范。
- **[B6]** Bosch CAN 2.0 / ISO 11898-1 — 位时序（SJW/BS1/BS2）的定义，规范把参数定死了，寄存器自然长一样。

你做到了

- GD32F4xx 与 STM32F407 的外设差异在你眼里是一张可查的表；
- 移植三步清单到手，换任何国产 MCU（如 HK32/CH32/AT32）都能按这套方法建档。

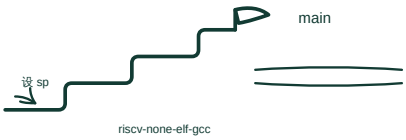
家底两边各有：GD32F4xx 多出 USBHS 与 TRNG，STM32F407 的 OTG_FS/OTG_HS 到这边由一件 USBHS 替代；SDIO 的差藏在第 9 位，DIV[8:0] 比 CLKDIV[7:0] 宽一位，那一位还躲在 bit31。跑 ULPI 的板子最省心，12 根信号同脚同 AF，换芯片不用改布线，换的是固件库。

✅ G 篇收官。下一站：R 篇 RT-Thread：国产内核配国产操作系统；或直接进 V 篇 RISC-V，去看 GD32VF103 的 Bumblebee 内核，那个没有 NVIC 的世界。

通往单片机之路第 74 章

V0 RISC-V 工具链与启动文件

难度 ★★☆☆☆ · 约 30 分钟



本章罗盘

问题

从 ARM 搬到 RISC-V，编译旗和启动文件到底哪几处变了？为什么没有 FPU 要开、栈要自己设？

交付

一张 -march/-mabi 对 ARM 四旗的编译旗对照表，加 start_gd32vf103.S 与 ARM 启动文件的逐段工序对照（点名删掉的「开 FPU」一节）。

边界

结论停在宿主机编译与反汇编层，未上板；复位向量装入是 Bumblebee 的实现定义行为，不能外推到其他 RISC-V 核；108M 档 Flash 等待拍数 UM 无映射表，待实测。

依据锚点 `c8` · `a13` · `f7` · `mcu-anim`

> 🎯 `arm-none-eabi-gcc` 换成 `riscv-none-elf-gcc`、`startup_stm32f407xx.s` 换成 `start_gd32vf103.S`——名字变了，"上电到 main"的工序却几乎一模一样。但有三处根本不同：没有 FPU 要开、没有"硬件两读"栈顶、复位靠的是 `reset_handler` 而不是向量表第 1 项。学会这三处差异，B4 的知识就能整个平移到 RISC-V。

本章精髓

1. **工具链命名遵循 RISC-V 规范**：`riscv-none-elf-gcc`（曾用 `riscv-none-embed-gcc` / `riscv64-unknown-elf-gcc`），编译标志从 `-mcpu/-mthumb/-mfloat-abi/-mfpu` 换成 `-march=rv32imac -mabi=ilp32`："目标三元组 + march/mabi"是 RISC-V 工具链的身份证。
2. **启动文件 `start_gd32vf103.S` 与 B4 的 `startup_stm32f407xx.s` 同构**：设 sp → 搬 .data → 清 .bss → 调 SystemInit → `call main`。**删掉的**是"开 CPACR"那一节：RV32IMAC 没有 FPU，硬浮点编译选项根本不存在。
3. **RISC-V 复位不做"硬件两读"**：PC 由复位向量直接装入，sp 必须由启动代码显式 `la sp, __stack_top` 设好。向量表那套机制在 V1 才登场（ECLIC 的向量直跳），复位这一刻它不参与。

先修

无（建议对照 B0 工具链全景、B4 启动过程 作锚点）。

先跑起来（10 分钟 quick win）

```
riscv-none-elf-gcc --version
# 输出里找 "riscv-none-elf-gcc" —— 工具链到位的标志
# 若 "command not found"，先装 xPack 的 riscv-none-elf-gcc 发行包

riscv-none-elf-gcc -march=rv32imac -mabi=ilp32 -print-libgcc-file-name
# 能打印出 libgcc 路径，说明 rv32imac/ilp32 这个 multilib 组合在工具链里有效
```

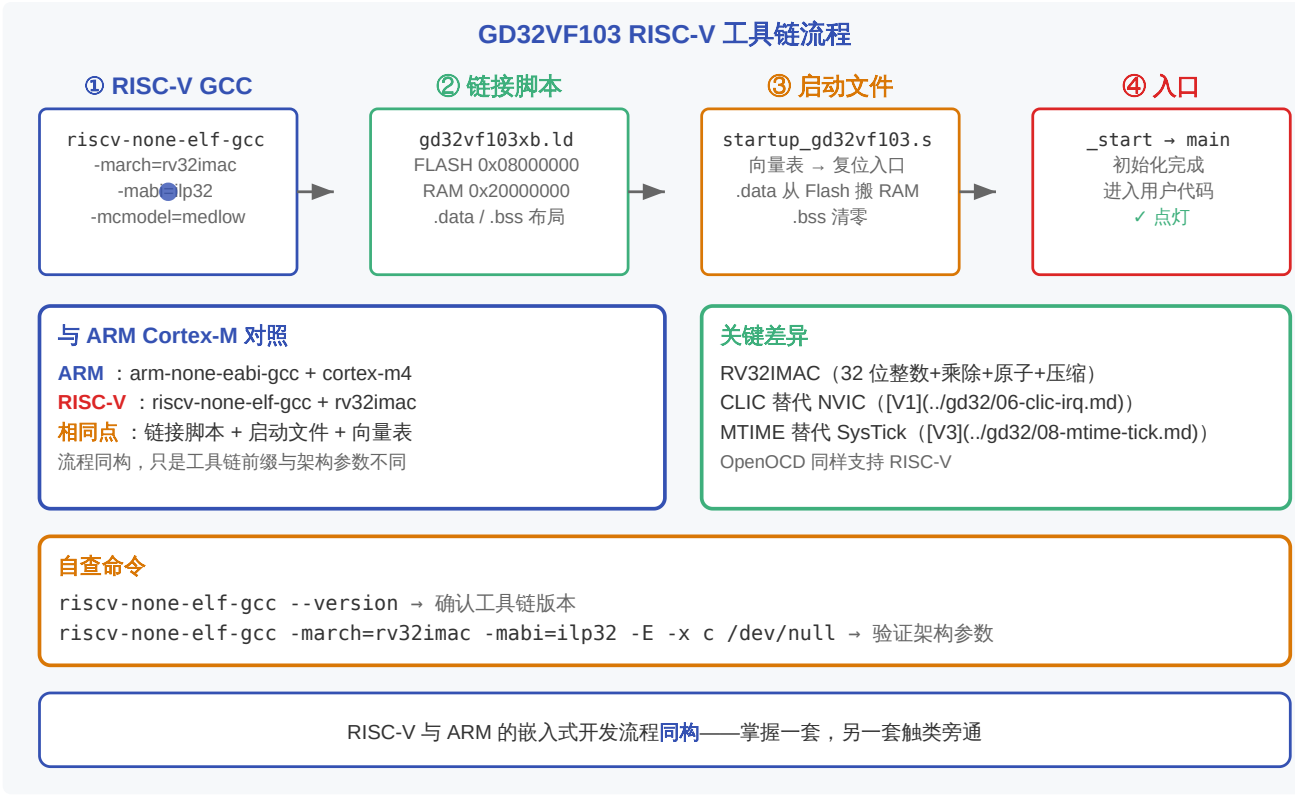
读两行输出：版本号、libgcc 路径里藏的 `rv32imac/ilp32`，这就是 RISC-V 工具链的身份证。本章每个标志位都能对回这两行。

先押张工序单：身份证正面是工具链，背面印的是启动工序。搬 .data、清 .bss、调 SystemInit、进 main 这几道，RISC-V 那边还用同一套指令拼写吗？押完对第二节的对照表。

小节结构

小节	内容	四件套
一、工具链换岗	riscv-none-elf-gcc 命名、-march/-mabi 标志、multilib	配置
二、启动文件骨架	start_gd32vf103.S 与 B4 启动文件的工序逐段对照	代码分析
三、没有 CPACR	RV32IMAC 无 FPU，"开 FPU"那一节为何整体删除	配置
四、没有硬件两读	RISC-V 复位如何装 PC、sp 谁来设	代码分析
五、链接脚本	gcc_gd32vf103.ld 的 MEMORY/ENTRY/段与 B3 五符号	配置
六、SystemInit	system_gd32vf103.c 复位 RCU、main 之前拉到 108M	代码分析

一、工具链换岗：arm-none-eabi → riscv-none-elf



RISC-V 工具链流程

B0 讲过"四个软件一台戏"：编译器、汇编器、链接器、调试器。换到 RISC-V，这四件套全有，只是前缀变了：

ARM 裸机	RISC-V 裸机	备注
`arm-none-eabi-gcc`	`riscv-none-elf-gcc`	曾用 `riscv-none-embed-gcc`（xPack 早期）、`riscv64-unknown-elf-gcc`（更早）
`arm-none-eabi-objdump`	`riscv-none-elf-objdump`	同
`arm-none-eabi-gdb`	`riscv-none-elf-gdb`	同
`arm-none-eabi-objcopy`	`riscv-none-elf-objcopy`	同

目标三元组的读法：`riscv`（厂商）+ `none`（无操作系统/裸机）+ `elf`（目标格式）。ARM 那边写 `none-eabi` 是因为 ARM 用 EABI 规范；RISC-V 这边直接 `elf`，ABI 由 `-mabi` 单独指定。

编译标志对照，这是最关键的一行差异：

项	ARM (STM32F407)	RISC-V (GD32VF103)
架构标志	`-mcpu=cortex-m4 -mthumb`	`-march=rv32imac`
浮点标志	`-mfloat-abi=hard -mfpu=fpv4-sp-d16`	**无**（IMAC 无 F/D 扩展）
ABI 标志	（隐含在 -mfloat-abi）	`-mabi=ilp32`

把 `-march=rv32imac` 拆开念：`rv32` = 32 位 RISC-V；`I` = 基础整数指令；`M` = 乘除法；`A` = 原子指令；`C` = 压缩指令（16 位编码，省 Flash）。`-mabi=ilp32` = ILP32 数据模型（int/long/指针都是 32 位），不带 `f` / `d` 后缀，因为没有浮点硬件，浮点运算走 libgcc 软件模拟。

> **【注】别把 `riscv64-unknown-elf-gcc` 当 64 位用**。它支持 rv32 multilib，但默认目标是 rv64。不显式带 `-march=rv32imac -mabi=ilp32`，链接脚本里 `ORIGIN(RAM)=0x20000000` 这种 32 位地址会和 64 位默认重定位对不上，报一堆 `relocation truncated to fit`。

二、启动文件骨架：与 B4 逐段对照

`start_gd32vf103.S`（注意大写 `.S` = 要过预处理器，所以能用 `#include` 和宏）与 B4 的 `startup_stm32f407xx.s` 工序几乎一一对应：

B4 工序	B4 实现（ARM 汇编）	V0 实现（RISC-V 汇编）	差异
设栈	硬件两读自动装 MSP	`la sp, __stack_top`	**软件显式**（见第四节）
开 FPU	`ldr r0,=0xE000ED88; orr #(0xF<<20)`	**删除**	架构级删除（见第三节）
搬 .data	`ldr r0,=_sdata...ldr r4,[r2,r3]; str`	`la t0,_sidata; la t1,_sdata; lw/sw/addi`	同义换指令
清 .bss	`ldr r2,=_sbss; str r3`	`la t0,_sbss; sw zero`	同义换指令
调 SystemInit	`bl SystemInit`	`call SystemInit`	同
进 main	`bl main; b .`	`call main; j .`（或 `wfi` 循环）	同

搬运 .data 的循环骨架（RISC-V 版，与 B4 第三节对照读）：

```
    la      t0, _sidata      /* Flash 源起点（链接脚本给的） */
    la      t1, _sdata       /* RAM 目标起点 */
    la      t2, _edata       /* RAM 目标终点 */
    mv      t3, zero         /* 偏移 = 0 */
.L_copy_loop:
    bgeu    t1, t2, .L_copy_done    /* 目标到终点了？先判后干 */
    lw      t4, 0(t0)        /* 从 Flash 读一个字 */
    sw      t4, 0(t1)        /* 写到 RAM */
    addi    t0, t0, 4
    addi    t1, t1, 4
    j       .L_copy_loop
.L_copy_done:
```

`_sidata = LOADADDR(.data)`、`_sdata` / `_edata` 是 B3 链接脚本讲的五个符号里的三个，这套约定跨架构通用，RISC-V 链接脚本照样发这些"地址条"。`bgeu t1, t2`（无符号大于等于则跳）对应 B4 的 `cmp r4, r1; bcc`，同样是"先判后干"，长度为零的 .data 也安全。

清 .bss 更瘦：源操作数都不需要，一路 `sw zero` 扫过去，与 B4 第四节同思路，只是指令换成了 RISC-V 的 `sw`。

三、没有 CPACR：RV32IMAC 无 FPU 这件事

B4 的"第 0 步"是开 `SCB->CPACR` 的 CP10/CP11，否则第一条 FPU 指令触发 `UsageFault(NOCP)`，程序静默卡死。这一步在 V0 **整体删除**——不是省略，是架构上不存在：

- **指令集没有浮点扩展**：`-march=rv32imac` 里的 `IMAC` 不含 `F`（单精度）也不含 `D`（双精度），编译器根本不会生成 FPU 指令。
- **没有"协处理器使能"机制**：ARM 用 CPACR 管协处理器权限是因为它有 CP0~CP15 的协处理器编号空间；RISC-V 的自定义扩展走 CSR 编码空间，浮点若有则由 `mstatus.FS` 字段管，但 IMAC 没有浮点，`mstatus.FS` 都不用碰。
- **`-mfloat-abi=hard` 是非法组合**：在 `rv32imac` 上加 `-mfloat-abi=hard`，工具链要么报错，要么默默走软浮点（libgcc 软件模拟）。这与 B4 里"硬浮点工程忘开 CPACR"的坑是镜像关系：ARM 是"有 FPU 但没开门"，RISC-V 是"门都没有"。

推论：GD32VF103 上做浮点运算（`float a = 1.5f * 2.0f;`）走的是 libgcc 的软浮点函数调用，慢且占 Flash，但能跑，不会 fault。要不要用浮点、用多少，是个工程权衡，不是"开了就崩"的开关。

四、没有硬件两读：RISC-V 复位与 Cortex-M 的根本差异

B4 第一节讲过：Cortex-M 复位时**硅片硬逻辑**从 0 号地址读 4 字节装 MSP、从 4 号地址读 4 字节装 PC，然后开始取指。RISC-V 没有这套机制：

	Cortex-M (B4)	**RISC-V (V0)**
初始 sp	硬件从 0x0 读 MSP	**sp = 0**，启动代码 `la sp, __stack_top` 设
初始 PC	硬件从 0x4 读 Reset_Handler	复位向量地址装入（实现定义）
向量表位置	Flash 起始，第 0 项是栈顶	无"向量表"概念，复位不靠它
谁先干活	硬件两读，再跳 Reset_Handler	直接跳 `reset_handler`，sp 自己设

所以 `start_gd32vf103.S` 的 `reset_handler` 第一件事不是搬数据，是**设栈**：

```
reset_handler:
    la      sp, __stack_top    /* 没人替你设 sp，自己来 */
    /* ... 之后才是搬 .data、清 .bss、call SystemInit、call main ... */
```

`__stack_top` 由链接脚本给（见第五节），值 = `ORIGIN(RAM) + LENGTH(RAM)`，栈从 RAM 顶向下长，与 B4 的 `_estack` 同思路，只是 B4 那是"硬件读进去"，这里是"软件装进去"。

> **【坑】** 别以为 RISC-V 也有"向量表第 0 项是栈顶值"的约定，把 `__stack_top` 写进链接脚本当向量：RISC-V 复位不读它，sp 还是 0，第一条用到栈的指令就跑飞。向量机制在 V1 讲（ECLIC 向量直跳），复位这一刻它完全不参与。

五、链接脚本：gcc_gd32vf103.ld 与 B3 五符号

`gcc_gd32vf103.ld`（官方固件库 `Firmware/GD32VF103/GCC/LinkerScripts/` 下）与 `stm32f407xx.ld` 同构：

```
MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 256K    /* gd32vf103.h:193 */
    RAM  (rwx) : ORIGIN = 0x20000000, LENGTH = 32K     /* gd32vf103.h:194 */
}
ENTRY(reset_handler)

SECTIONS
{
    .text : { *(.text*) } > FLASH
    .data : { *(.data*) } > RAM AT> FLASH              /* 运行在 RAM，初值存 Flash */
    .bss  : { *(.bss*)  } > RAM
    __stack_top = ORIGIN(RAM) + LENGTH(RAM);
}
```

关键事实核验（`gd32vf103.h:193-194`）：

```
#define FLASH_BASE  ((uint32_t)0x08000000U)   /* main FLASH base */
#define SRAM_BASE   ((uint32_t)0x20000000U)   /* SRAM base       */
```

与 STM32F407 完全相同的地址（见 S1 架构总览）。这是 GD32 刻意保持的移植兼容点：把 STM32 固件搬到 GD32VF103，Flash 和 RAM 起始地址不用改。

B3 讲的五个符号在这里同名同义：`_sidata = LOADADDR(.data)`（初值仓库在 Flash 的位置）、`_sdata` / `_edata`（.data 在 RAM 的头尾）、`_sbss` / `_ebss`（.bss 在 RAM 的头尾）、`__stack_top`（栈顶）。这套符号是链接脚本约定，跨架构通用。

唯一差异：RISC-V 链接脚本没有 `.isr_vector` 段（ARM 把向量表放 Flash 起始，对应 `g_pfnVectors`）。RISC-V 的"向量"由 ECLIC 管理（V1 详述），不需要在 Flash 起始摆一张大表。

六、SystemInit：main 之前把时钟拉到 108M

`system_gd32vf103.c:150` 的 `SystemInit()` 干两件事：复位 RCU 配置位 → 调 `system_clock_config()`。后者默认走 `system_clock_108m_hxtal()`（`system_gd32vf103.c:62` 定义 `__SYSTEM_CLOCK_108M_PLL_HXTAL`）。

108M 流程（`system_gd32vf103.c:751-830`）：

1. 使能 HXTAL：`RCU_CTL |= HXTALEN`（bit16），轮询 `HXTALSTB`（bit17）
2. 总线分频：AHB = SYSCLK/1、APB2 = AHB/1、APB1 = AHB/2
3. PLL 配置：`RCU_PLL_MUL27`（× 27）+ PREDV0 分频 - 8M 晶振（V_EVAL/C_START/T_START）：PREDV0_DIV2 → 4MHz × 27 = **108MHz** - 25M 晶振（R_START）：经 PREDV1_DIV5 × PLL1_MUL8 → 40MHz，再 PREDV0_DIV10 → 4MHz × 27 = 108MHz
4. 使能 PLL：`RCU_CTL |= PLLEN`（bit24），轮询 `PLLSTB`（bit25）
5. 选 PLL 为系统时钟：`RCU_CFG0 |= RCU_CKSYSSRC_PLL`，轮询 `SCSS = PLL`

这与 STM32 RCC 提频六步同构，只是寄存器名 `RCC→RCU`、位名 `PLLRDY→PLLSTB`、`SWS→SCSS`（G1 已在 GD32F4 上逐字段对照过）。**同样"官方留白"**：`system_gd32vf103.c` 全文不设 Flash 等待周期，但 VF103 的 FMC_WS 寄存器确实存在（@0x4002 2000 + 0x00，gd32vf103_fmc.h:45,64），而且 UM 已核验出一个 G1 没有的收窄：**VF103 的 WSCNT[2:0] 只有 000/001/010 三档有效，011~111 全部保留**（UM §2.4.1），同样要先置 FMC_WSEN 才生效。108MHz 档实际该配几拍，UM 不给映射表（与 G1 在 GD32F4 的发现一致），**待上板实测**。

启动序列全景：

```
reset_handler
  → la sp, __stack_top          /* 设栈 */
  → 搬 .data (Flash → RAM)
  → 清 .bss
  → call SystemInit             /* 把 RCU 配成 PLL×27 = 108M */
  → call main                   /* main 一进来就在 108MHz 跑 */
```

`SystemInit` 在 `main` 之前把时钟拉到 108M，所以你的 `main` 第一行就在 108MHz 跑。跳过它就只在 8M IRC8M 跑，串口波特率全错。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **工具链换 riscv-none-elf，march rv32imac 配 ilp32；启动删 FPU、栈自己设；FLASH 0x0800 / RAM 0x2000 与 STM32 同址，main 进来就在 108M。**

延伸：RISC-V 工具链流程动画见 V0 动画；ARM 工具链对照见 B0；链接脚本在 B3 详解。

实物实验

- `riscv-none-elf-objdump -d your.elf | findstr reset_handler`：看 `reset_handler` 第一条是 `la sp`（设栈），与 Cortex-M reset 第一条"读向量表"对比。
- GDB `monitor reset halt` 后 `info registers sp pc`：sp = 0、pc = `reset_handler` 地址，亲眼看到"sp 没人设"。
- 进阶：在 `reset_handler` 的 `la sp, __stack_top` 行下断，单步过后 sp 变成 RAM 顶值（0x20000000 + RAM 长度）；再单步过 `call SystemInit`，`RCU_CFG0` 的 SCSS 字段应变成 PLL。

常见坑

1. **用了 `riscv64-unknown-elf-gcc` 旧名又不带 `-march/-mabi`**：默认 rv64，链接脚本里 32 位地址对不上，报 `relocation truncated to fit` 一堆。要么换 `riscv-none-elf-gcc`，要么每次显式带 `-march=rv32imac -mabi=ilp32`。
2. **`-march=rv32imac` 误写成 `rv32imacF`**：加了 F 但芯片没 FPU，工具链报 `unsupported ISA extension`。GD32VF103 就是 IMAC，别画蛇添足。
3. **以为 RISC-V 也有"向量表第 0 项是栈顶"**：把 `__stack_top` 当向量写，复位还是不读它，sp 留 0，第一条 push 就跑飞。RISC-V 复位只跳 `reset_handler`，栈是软件设。
4. **忘了 `call SystemInit`**：`main` 在 8MHz IRC8M 跑，串口波特率、延时全错，现象是"程序能跑但时序诡异"，难查。
5. **照搬 STM32 的 `-mfloat-abi=hard`**：RISC-V 上无 F 扩展，编译报错或默默走软浮点；想要硬件浮点得换带 F/D 的 RISC-V 芯片，GD32VF103 不行。

短自测

1. `riscv-none-elf-gcc` 替代 `arm-none-eabi-gcc` 后，编译标志哪两套变了？

> ▼ **看答案**
> **架构标志**与 **ABI/浮点标志**两套。ARM 写 `-mcpu=cortex-m4 -mthumb -mfloat-abi=hard -mfpu=fpv4-sp-d16`；RISC-V 写 `-march=rv32imac -mabi=ilp32`。RISC-V 没有 `-mcpu`（用 march 指令集代替），也没有 `-mfloat-abi/-mfpu`（IMAC 无 FPU，浮点走软模拟）。

2. `start_gd32vf103.S` 相比 `startup_stm32f407xx.s` 删除了哪一节？为什么？

▼ 看答案
开 CPACR / 开 FPU 那一节（B4 的"第 0 步"）。RV32IMAC 指令集不含 F/D 浮点扩展，编译器不生成 FPU 指令，也没有"协处理器使能"机制可开。这是架构级删除，不是省略。

3. RISC-V 复位后 sp 是多少？谁来设？与 Cortex-M 有何不同？

▼ 看答案
sp = 0。由启动代码 `la sp, __stack_top` 显式设，`__stack_top` 由链接脚本给（`ORIGIN(RAM) + LENGTH(RAM)`）。Cortex-M 复位时硬件从 0 号地址读 4 字节自动装 MSP，软件不用管；RISC-V 没有"硬件两读"机制，sp 必须软件设。

4. GD32VF103 的 FLASH 与 RAM 起始地址是多少？和 STM32F407 比如何？

▼ 看答案
FLASH = 0x08000000、RAM = 0x20000000（`gd32vf103.h:193-194`）。与 STM32F407 **完全相同**，这是 GD32 刻意保持的移植兼容点，把 STM32 固件搬到 GD32VF103，Flash 和 RAM 起始地址不用改。

5. `main` 一进来就在 108MHz 跑，是谁干的？跳过会怎样？

▼ 看答案
`reset_handler` 在 `call main` 之前先 `call SystemInit`（`system_gd32vf103.c:150`），后者调 `system_clock_108m_hxtal()` 把 RCU 配成 PLL × 27 = 108M。跳过它，`main` 只在 8M IRC8M 跑，程序能跑但串口波特率、延时时序全错，难查。

对照表：本章概念 → 仓库落点

概念	仓库/文件落点
`riscv-none-elf-gcc` 命名、替代 `arm-none-eabi-gcc`	工具链发行说明；Makefile
`-march=rv32imac -mabi=ilp32`	启动工程 Makefile / 启动文件头注释
`start_gd32vf103.S` 工序对照	官方固件库 `Firmware/GD32VF103/Source/` 启动文件
搬 .data / 清 .bss 的 RISC-V 实现	`start_gd32vf103.S` `reset_handler` 段
RV32IMAC 无 FPU、删 CPACR 节	架构事实（IMAC 无 F/D 扩展）
复位无硬件两读、sp 软件设	`start_gd32vf103.S` 首条 `la sp, __stack_top`
`__stack_top = ORIGIN(RAM)+LENGTH(RAM)`	`gcc_gd32vf103.ld`
`_sidata/_sdata/_edata/_sbss/_ebss` 五符号	`gcc_gd32vf103.ld`（与 B3 同名同义）
FLASH/SRAM 基址 0x08000000 / 0x20000000	`gd32vf103.h:193-194`
`SystemInit` 复位 RCU	`system_gd32vf103.c:150`
108M 流程 HXTAL→MUL27→PREDV0	`system_gd32vf103.c:751-830`
HXTALEN/STB bit16/17、PLLEN/STB bit24/25	`gd32vf103_rcu.h:67-68 / :71-72`
默认 `__SYSTEM_CLOCK_108M_PLL_HXTAL`	`system_gd32vf103.c:62`
VF103 的 WSCNT 只有 0/1/2 三档有效（011~111 保留），须 WSEN 使能	**GD32VF103 UM EN V1.0 §2.4.1 已核验**（p40）；FMC 基址 0x4002 2000
108M 档实际等待拍数	UM 不给"频率↔等待"映射表（已检索）——**待上板实测**

你做到了

- 工具链从 ARM 换到 RISC-V，编译标志两套差异、multilib 选择都讲得出来；
- 启动文件逐段对照 B4，知道哪节删了（FPU）、哪节换了写法（搬数据指令）、哪节完全照搬（清 bss、调 main）；
- RISC-V 复位"sp 自己设、PC 由 reset 装入"的机制入肌肉记忆，不再误以为有"硬件两读"。

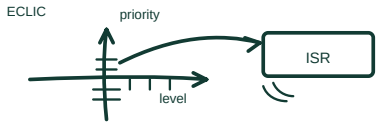
工序单对上：搬 .data、清 .bss 同义换指令，ldr/str 那套换成 la 与 lw/sw；调 SystemInit、进 main 工序照旧。背面另两行空缺：一是"开 CPACR"那一节整个消失，RV32IMAC 没有 FPU，硬浮点编译选项根本不存在；二是复位不做"硬件两读"，PC 由复位向量直接装入，sp 由启动代码显式设好。这就是架构差异落在启动文件上的分量。

✅ 下一站：V1 Bumblebee 内核与 CLIC 中断：去看没有 NVIC 的世界里，向量怎么"直跳"。

通往单片机之路第 75 章

V1 Bumblebee 内核与 CLIC 中断

难度 ★★★☆☆ · 约 35 分钟



本章罗盘

问题

RISC-V 的 CLIC 与 Cortex-M 的 NVIC 在中断入口、优先级和压栈上有哪些本质不同？

交付

一张 NVIC↔ECLIC 机制对照表和一份 CLIC 中断入口汇编的逐行注解。

边界

Bumblebee 的 CLIC 是规范草案的厂商增强实现；结论不能直接平移到其他 RISC-V 内核。

依据锚点 `a13 · a14 · c9 · f7`

> 🎯 同样是"按键打断 CPU"，Cortex-M4 走 NVIC → ISER → IPR → 向量槽 22；GD32VF103 走 ECLIC → clicintie → level+priority → 向量直跳。没有 NVIC、没有 SysTick、没有 `SCB->VTOR`、没有 PendSV——但中断照样嵌套、照样抢占。这一章用 S4 的六步清单当尺子，量出 RISC-V 的中断世界差在哪、又奇巧地神似在哪。

本章精髓

1. **中断控制器换名换架构**：**ECLIC**（Enhanced Core-Level Interrupt Controller，Bumblebee 内核私有，Nuclei 与 Andes 联合开发）取代 ARM 的 NVIC。RISC-V 标准的 software / timer / external 三类中断它都管，外部中断源数交给芯片定（GD32VF103 在 `gd32vf103.h` 里用 `CLIC_INT_*` 与 `*_IRQn` 编号）。
2. **优先级变成两维**：**level（抢占级，最多 16 级，决定能否打断别人）+ priority（同级排序，最多 16 级）**。不是 ARM 那套"4 位单一优先级 + PRIGROUP 切分组 + 0x5FA 钥匙"。
3. **"向量"换语义**：ARM 是"向量表第 N 项放 ISR 地址，硬件读表跳转"；ECLIC 是"每个中断有 shv 位，置 1 即**向量直跳**到该中断专属入口，置 0 则进公共入口再软件分发"，省了一张大表，但每个要向量的中断都得显式配入口。

先修

- V0 RISC-V 工具链与启动（知道复位序列与启动文件）；
- S4 NVIC 与 EXTI（六步清单与三层链路，本章的对照锚点）。

先跑起来（10 分钟 quick win）

在 GD32VF103 官方固件库的 `gd32vf103.h` 里翻中断号（`findstr /n "CLIC_INT _IRQn" gd32vf103.h`），本章骨架就在这五行：

```
101:     CLIC_INT_SFT               = 3,       /*!< Software interrupt */
102:     CLIC_INT_TMR               = 7,       /*!< CPU Timer interrupt */
104:     CLIC_INT_PMOVI             = 18,      /*!< Performance Monitor */
113:     EXTI0_IRQn                 = 25,      /*!< EXTI line 0 interrupts */
144:     USART0_IRQn                = 56,      /*!< USART0 interrupt */
171:     ECLIC_NUM_INTERRUPTS                  /* 中断总数哨兵 */
```

读完本章再回看这五行，每个编号你都能讲出它与 ARM `IRQn` 的区别。

先押两扇门：硬件读表跳转的老剧情，这边还演不演？每个中断名下那位 shv，置 1 与置 0 各进哪扇门、谁在门口分发？ARM 那把 0x5FA 钥匙，这边还认不认？

动画：ECLIC 两维优先级

level 决定能否打断别人（抢占），priority 决定同 level 内谁先响应（排序）。两维配合完成中断嵌套与同级排序，无需 ARM 的 PRIGROUP 分组钥匙。

ECLIC 两维优先级：level 定抢占，priority 定序

小节结构

小节	内容	四件套
一、内核换魂	Cortex-M4 → Bumblebee RV32IMAC，少了什么	配置
二、ECLIC 取代 NVIC	增强版 CLIC 的职责、16 level + 16 priority、向量/非向量、尾链、NMI	配置
三、中断号地图	CLIC_INT_* / *_IRQn 与 ARM IRQn+16 的差异	配置
四、两维优先级	level 抢占 + priority 同级 vs ARM 单一优先级 + PRIGROUP	配置
五、向量直跳	shv 模式与 ARM 向量表的机制差异	代码分析
六、MTIME 不是 SysTick	内存映射 mtime/mtimecmp、mtime_toggle_a 频率	配置
七、对照落地	配通 EXTI0 的步骤与 S4 六步逐条对比	配置

一、内核换魂：Cortex-M4 → Bumblebee RV32IMAC

GD32VF103 把 ARM Cortex-M4 换成了 Nuclei/Andes 联合定制的 **Bumblebee** 内核。先把"少了什么"一次列清（凭证来自 Bumblebee 处理器内核简明手册 §1.1 / §2.4 / §2.6 / §2.13）：

维度	Cortex-M4 (STM32F407)	Bumblebee (GD32VF103)	凭证
ISA	ARMv7E-M (Thumb-2)	RV32IMAC	手册 §1.1
流水线	3 级	2 级可变 + 简单分支预测 + 预取 2 条	手册 §1.1
特权模式	Handler/Thread + 特权/非特权	Machine / User	手册 §2.4
中断控制器	NVIC	**ECLIC**	手册 §1.1 / §2.6
内核定时器	SysTick（24 位递减）	**MTIME**（64 位递增，内存映射）	手册 §2.13
向量表重定位	`SCB->VTOR`	无 SCB、无 VTOR；ECLIC 自管向量	—
PendSV	有（RTOS 上下文切换用）	无	—
FPU	有（CP10/CP11）	无（IMAC 无 F/D）	V0 第三节
复位两读	硬件读 0x0 / 0x4	无	V0 第四节
PMP	有（可选）	无	手册 §2.7

少了 NVIC、SysTick、`SCB->VTOR`、PendSV：这四样是 Cortex-M 内核的"标配"，RISC-V 一个都没有。 但中断该有的能力（嵌套、抢占、向量、NMI）ECLIC 全给，只是实现的姿势不同。本章就是把这"不同"逐项讲清。

二、ECLIC 取代 NVIC：增强版 CLIC 的职责

ECLIC 是 Bumblebee 核的私有外设（与 DEBUG、TIMER 并列在 Core 层级下），通过内存地址访问（手册 §2.6）。它的职责（手册 §1.1 ECLIC 条目）：

- 管 RISC-V 标准三类中断：**software / timer / external**；
- 外部中断源数交给芯片定（GD32VF103 见 `gd32vf103.h` 的 `IRQn_Type`）；
- **最多 16 级 level + 16 级 priority**，软件可动态改；
- 基于 level 的抢占；
- 快速向量中断处理（fast vectored interrupt processing）；
- 快速中断尾链（tail-chaining）；
- 支持 NMI。

与 NVIC 的结构对照（ECLIC 寄存器精确偏移已由官方固件驱动实证，见 n200_eclic.h：基址 0xD2000000、cliccfg@0x0、clicintip@0x1000、clicintie@0x1001、clicintattr@0x1002、clicintctl@0x1003，每中断间隔 4 字节；这里讲结构与维度）：

	ARM NVIC (S4)	ECLIC (V1)
使能	ISER/ICER 位图	clicintie[i] 每中断一位
挂起	ISPR/ICPR 位图	clicintip[i] 每中断一位
优先级	IPR 字节（高 4 位有效）	clicintctl[i]（level + priority 两维）
触发沿	放在 EXTI 寄存器	clicintattr[i]（属性寄存器收进来管）
向量模式	向量表读表跳转	clicintattr[i].shv（向量/非向量）
全局配置	SCB->AIRCR（PRIGROUP）	cliccfg（全局配置，如 nlbits）

注意一个收敛点：ARM 把"触发沿"放在外设（EXTI RTSR/FTSR）、把"优先级"放在内核（NVIC IPR），两处分；ECLIC 把这些都收进来统一管：每个中断一组完整的属性（使能、挂起、触发沿、level/priority、向量模式）。

三、中断号地图：CLIC_INT_* 与 ARM IRQn+16 的差异

S4 第四节讲过：ARM 的"槽号 = IRQn + 16"（前 16 项是内核异常），外设中断从槽 16 开始排。RISC-V ECLIC **没有这个 +16 偏移**：中断号就是 ECLIC 的索引。

`gd32vf103.h:97-172` 的 `IRQn_Type` 给了全部编号（节选）：

```
CLIC_INT_RESERVED = 0,       /* RISC-V reserved */
CLIC_INT_SFT      = 3,       /* Software interrupt       */
CLIC_INT_TMR      = 7,       /* CPU Timer interrupt      */
CLIC_INT_BWEI     = 17,      /* Bus Error interrupt      */
CLIC_INT_PMOVI    = 18,      /* Performance Monitor      */
WWDGT_IRQn   = 19,
EXTI0_IRQn   = 25,           /* EXTI line 0              */
EXTI5_9_IRQn = 42,           /* EXTI[9:5]                */
USART0_IRQn  = 56,
EXTI10_15_IRQn = 59,         /* EXTI[15:10]              */
USBFS_IRQn   = 86,
ECLIC_NUM_INTERRUPTS         /* 中断总数哨兵（gd32vf103.h:171） */
```

关键差异：EXTI0 在 STM32F407 是 `IRQn = 6`（槽 22，`stm32f407xx.h:83`），在 GD32VF103 是 `IRQn = 25`（ECLIC 索引 25，无 +16）。**同一条 EXTI0 线，两颗芯片的中断号差很远**，移植时不能照搬 S4 那张映射表。

> 【混血特征】 GD32VF103 的外设脱胎自 GD32F1（与 STM32F1 同源）：EXTI 在 `APB2 + 0x400 = 0x40010400`（`gd32vf103.h:220`），AFIO 在 `APB2 + 0x000 = 0x40010000`（`gd32vf103.h:219`），这俩地址与 STM32F1 完全一致。所以 GD32VF103 是"**外设像 F1、内核是 RISC-V**"的混血：第七节配 EXTI 时，外设层那几步几乎照搬 F1，只有控制器层（第 6 步）换成 ECLIC。

四、两维优先级：level + priority vs ARM 单一优先级

S4 第三节讲过 ARM 的优先级：4 位单一字段，PRIGROUP 把它切成"抢占 + 子"两份，写 AIRCR 必须带 0x5FA 钥匙。ECLIC **天生两维**，不用切分组：

	ARM NVIC (S4)	ECLIC (V1)
优先级位宽	4 位（高 4 位有效）	level 4 位 + priority 4 位（手册"up to 16"）
抢占/子 切分	PRIGROUP 切 4 位	天生两维，不用切
全局配置	AIRCR + 0x5FA 钥匙	cliccfg（如 nlbits 设 level 位数）
调优先级 API	`NVIC_SetPriority(n, x)`	写 clicintctl[n] 的 level + priority
分组钥匙	`0x5FA`	无钥匙机制

- **level**（抢占级，最多 16 级）：决定"能不能打断别人"。高 level 抢断低 level，类似 ARM 的抢占优先级。
- **priority**（同级排序，最多 16 级）：决定"同 level 同时挂起时谁先上"，类似 ARM 的子优先级，但**没有打断能力**。

推论一：移植 RTOS 时"优先级数值没变、行为全变"的灵异事件（S4 3.2 讲的 PRIGROUP 尺子换了）在 RISC-V 上不会发生：没有"分组尺子"可换。但 RTOS 仍要求"全部 level 给抢占"这类约定，对应到 cliccfg 的 nlbits 设置（官方驱动已给掩码与移位：`ECLIC_CFG_NLBITS_MASK=0x1E`、`NLBITS_LSB=1`，n200_eclic.h:54-55；**精确语义**仍以 ISA 手册为准）。

推论二：ARM 那套 `NVIC_SetPriorityGrouping()` / `0x5FA` 钥匙在 RISC-V 上完全不存在。看到 GD32VF103 例程里"设优先级分组"的写法，那是在配 cliccfg，不是 ARM 的 AIRCR。

五、向量直跳：shv 模式与 ARM 向量表的机制差异

S4 第四节讲过 ARM 向量表：Flash 起始一张大表，槽 N 放 ISR 函数指针，硬件进异常时读槽跳转；函数名拼错 = weak 默认顶替 = 死循环。

ECLIC 有两种向量模式（手册 §1.1 "fast vectored interrupt processing"）：

- **向量模式（shv = 1）**：每个中断有自己的入口地址，硬件**直接跳到该入口**，叫"向量直跳"。不需要"读一张大表"，每个中断的向量独立配置在 ECLIC 内部。
- **非向量模式（shv = 0）**：所有中断进公共入口（`mtvec` 指向），由软件读 `mcause` / clicintip 分发，省向量存储，但多一次软件分派。

	ARM NVIC (S4)	ECLIC (V1)
向量存储	Flash 起始一张大表（98 项 × 4 字节）	每中断入口地址存 ECLIC 内部（clicintattr.shv + 向量寄存器）
跳转方式	硬件读槽 → 跳	shv=1 硬件直跳专属入口；shv=0 进公共入口软件分
函数名约定	必须与向量表 weak 同名	向量模式下入口地址直接配进 ECLIC，名字不一定要对表
函数名拼错	weak 符号顶替 → 死循环	取决于配置；公共入口模式下"没注册"进默认处理
异常返回	`EXC_RETURN` 魔法值（0xFFFFFFFD）+ 硬件弹栈 8 字	`mret` + 从 mepc 恢复（软件在 prologue 压栈）

ECLIC 的"向量直跳"省了 ARM 那张大表，但代价是每个要用向量的中断都得显式配入口，灵活但要写更多配置代码。

> **【注】RISC-V 异常返回没有"魔法值"**。ARM 进 ISR 时 LR 被硬件换成 `EXC_RETURN`（0xFFFFFFFD），出 ISR 一条 `bx lr` 触发硬件弹栈。RISC-V 进 ISR 时硬件把返回地址存进 `mepc`、把中断使能存进 `mstatus.MPIE`，ISR 末尾一条 `mret` 指令恢复：没有"LR 换魔法值"这一步，压栈也是软件在 prologue 干（不像 ARM 硬件自动压 8 字）。
>
> 精确的 shv 位位置、向量寄存器布局、clicintattr 字段：官方固件驱动已实证偏移（clicintattr@+0x1002、SHV 位 0x01、TRIG_LEVEL/EDGE=0x00/0x02、TRIG_POS/NEG=0x00/0x04，n200_eclic.h:40-46），语义细节仍以 ISA 手册为准。

六、MTIME 不是 SysTick：内存映射定时器

S5 讲过 SysTick：24 位递减计数器、CLKSOURCE 选 HCLK/8 或 HCLK、IRQn = -1（内核异常）、寄存器在 SCS 区 `0xE000E010`。Bumblebee 的 MTIME 完全不同（手册 §2.13）：

- **RISC-V 标准 64 位递增定时器**，CLK 由系统低速 RTC 频率驱动；
- **mtime**：64 位当前计数值；
- **mtimecmp**：64 位比较值，**mtime 不小于 mtimecmp 时触发定时器中断**；
- **mtime/mtimecmp 不是 CSR**：手册明说："not CSR registers, but Memory Address Mapped system registers"（内存映射系统寄存器）；
- 具体映射地址 RISC-V 架构不定义，由实现定；Bumblebee 把它们做在 TIMER 单元里。官方固件驱动实证：`TIMER_CTRL_ADDR = 0xD1000000`，mtime @ +0x0、mtimecmp @ +0x8（n200_timer.h:24-30）。

	SysTick (Cortex-M, S5)	MTIME (Bumblebee)
位数	24 位递减	64 位递增
比较方式	倒数到 0 触发	mtime 不小于 mtimecmp 触发
寄存器位置	SCS 内存映射 0xE000E010	TIMER 单元内存映射（官方驱动实证 0xD1000000）
是否 CSR	否	**否**（常被误以为 CSR）
IRQn	-1（内核异常）	`CLIC_INT_TMR = 7`（ECLIC，gd32vf103.h:102）
重装	写 LOAD 自动重装	**软件写 mtimecmp = mtime + 周期**
RTOS tick	直接用	直接用（V3 详述）

驱动频率（手册 §3.1 + GD32VF103 注记）：mtime 由 SoC 的 `mtime_toggle_a` 脉冲驱动，每检测到一个边沿 mtime 加 1。GD32VF103 上 `rtc_clk = core_clk_aon / 4`，所以 **mtime 自增频率 = core_clk_aon / 4**。若核心跑 108MHz，mtime 约 27MHz 自增（`core_clk_aon` 与核心时钟同源同频的关系已由 V3 收案：手册 §2.1）。

推论：MTIME 是 64 位递增 + 软件比较，意味着"周期定时"要软件每次中断后把 mtimecmp 加一个周期（不像 SysTick 硬件自动重装）。这是 RTOS port 到 RISC-V 必须处理的差异（V3 详述）。SysTick 的"写一次 LOAD 一劳永逸"在 MTIME 上不成立。

七、对照落地：配通 EXTI0，与 S4 六步逐条对比

以 PA0 → EXTI0 为例，与 S4 第六节的六步逐条对比：

#	S4 (STM32F407)	V1 (GD32VF103)	差异
1	开 GPIOF 时钟 `RCC_AHB1ENR`	开 GPIOA 时钟 `RCU_APB2EN.PAEN`（bit2）	RCU 命名，APB2 上（`gd32vf103_rcu.h:165`）
2	PF6 配输入	PA0 配输入	同
3	开 SYSCFG 时钟 `RCC_APB2ENR` bit14	开 AFIO 时钟 `RCU_APB2EN.AFEN`（bit0）	F1 风格 AFIO（`gd32vf103_rcu.h:164`）
4	SYSCFG EXTICR 选门	AFIO_EXTISS 选门	F1 风格选门（AFIO@0x40010000）
5	EXTI IMR/FTSR 设沿放行	EXTI IMR/FTSR 设沿放行	**外设同源（F1），几乎照搬**
6	`NVIC_SetPriority` + `EnableIRQ`（ISER）	ECLIC clicintie[25]=1 + clicintctl[25] 设 level/priority + clicintattr[25] 配 shv	**控制器不同，三动作**
ISR	`EXTI0_IRQHandler` 清 PR	同名 ISR 清 PR	ISR 写法照搬（EXTI 外设同 F1）

关键差异在第 6 步：ARM 是"使能 + 设优先级"两动作；ECLIC 是"使能 + 设 level/priority + 配向量模式"三动作。多出来的"配向量模式"决定该中断是直跳专属入口（shv=1）还是进公共入口再分发（shv=0）。

ISR 写法照搬（因为 EXTI 外设与 F1 同源，挂起位还是 `EXTI_PD` 写 1 清除）：

```
void EXTI0_IRQHandler(void)          /* GD32VF103 上 EXTI0 的 ECLIC 中断号 = 25 */
{
    EXTI_PD = (1UL << 0);            /* 第一件事：清挂起（与 S4 5.3 同语义） */
    /* ... 记账式干活，快进快出 ... */
}
```

GD32VF103 的"混血"在这里体现得最清楚：外设层（EXTI/AFIO/GPIO）是 ARM F1 的亲戚，所以第 1~5 步、ISR 清挂起都几乎照搬；只有控制器层（第 6 步）换成 ECLIC 的三动作。**会用 STM32F1 的人，学完 ECLIC 第 6 步就能在 GD32VF103 上配通任意外部中断。**

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **控制器 NVIC→ECLIC，优先级单维→level+priority 两维，向量大表→每中断 shv 直跳；SysTick→MTIME 内存映射、64 位递增软件比较；EXTI 外设照搬 F1，第 6 步换 ECLIC 三动作。**

实物实验

- 用 GDB 读 `mtvec`（`info registers mtvec`）与 cliccfg，看 ECLIC 全局配置（寄存器偏移已实证，语义细节见 ISA 手册）。
- 配 EXTI0 为 shv=1 向量模式，按键触发，在 ISR 入口设断点，应直接断在专属入口，不停在公共入口。
- 配同 EXTI0 为 shv=0 非向量模式，再触发，应先停在公共入口，单步后分发到 ISR。两种模式一对比，"向量直跳"就直观了。
- MTIME 实验（V3 详述）：读 mtime 两次间隔，反推自增频率，验证 `core_clk_aon / 4`。

常见坑

1. **用 STM32 的"槽号 = IRQn + 16"算 GD32VF103 中断号**：EXTI0 应是 25 不是 22（S4 表里 EXTI0 槽 22 是 STM32F407 的），算错就配错 ECLIC 寄存器。
2. **以为 ECLIC 也有 AIRCR / 0x5FA 钥匙、PRIGROUP 分组**：RISC-V 没这套，level + priority 本来就两维，硬写 AIRCR 无效。看到"设优先级分组"的代码，那是配 cliccfg，不是 ARM 的 AIRCR。
3. **把 mtime/mtimecmp 当 CSR 用 `csrr` / `csrw` 读写**：它们是内存映射，要用 `lw` / `sw` 普通访存指令，`csrr` 会触发非法指令异常。
4. **SysTick 周期定时照搬到 MTIME**：以为硬件自动重装 → MTIME 是软件写 mtimecmp 加周期，不写就只响一次。RTOS tick 必须每次中断后更新 mtimecmp。
5. **EXTI 外设照搬 F1，但忘了 GD32VF103 时钟使能位编码不同**：`RCU_APB2EN.PAEN` 是 bit2（`gd32vf103_rcu.h:165`），不是 F1 的 bit2（PAEN 恰好同位），但 AFEN 是 bit0、USART0EN 是 bit14：照搬 F1 位号前要核对 `gd32vf103_rcu.h`，别想当然。

短自测

1. GD32VF103 上 EXTI0 的中断号是多少？和 STM32F407 的 EXTI0 比，编号差多少？

> ▼ **看答案**
> `EXTI0_IRQn = 25`（`gd32vf103.h:113`）。STM32F407 的 `EXTI0_IRQn = 6`（槽 22，`stm32f407xx.h:83`）。差很多：RISC-V ECLIC 没有"IRQn + 16"偏移，ECLIC 索引就是中断号本身。S4 那张"槽号 = IRQn + 16"的映射表在 GD32VF103 上完全不适用，移植不能照搬。

2. ECLIC 的优先级有哪两个维度？各自管什么？ARM 用一个什么机制实现类似效果？

> ▼ **看答案**
> **level**（抢占级，最多 16 级，决定能否打断别人）+ **priority**（同级排序，最多 16 级，决定同 level 同时挂起时谁先上，无打断能力）。ARM 用单一 4 位优先级 + PRIGROUP 切分组（写 AIRCR 带 0x5FA 钥匙）实现"抢占 + 子"两份。ECLIC 天生两维，不用切分组，也没有 0x5FA 钥匙。

3. mtime 与 mtimecmp 是 CSR 吗？放在哪？怎么读写？

> ▼ **看答案**
> **不是 CSR**。Bumblebee 手册 §2.13 明说它们是 "Memory Address Mapped system registers"，做在 TIMER 单元里，精确地址由实现定（官方固件驱动实证：0xD1000000，n200_timer.h:29）。要用 `lw`/`sw` 普通访存指令读写，不能用 `csrr`/`csrw`，后者会触发非法指令异常。

4. ECLIC 的 shv=1 与 shv=0 各是什么模式？与 ARM 向量表机制比有何不同？

> ▼ **看答案**
> shv=1 是**向量模式**：硬件直跳到该中断专属入口（向量直跳）；shv=0 是**非向量模式**：进公共入口（`mtvec` 指向）再软件读 `mcause`/clicintip 分发。ARM 是"Flash 起始一张大表，硬件读槽跳"，所有中断都走读表；ECLIC 每中断独立配，可直跳（shv=1）也可分发（shv=0），省了一张大表但每个要向量的中断都得显式配入

口。

5. GD32VF103 上 mtime 的自增频率约为什么？依据是什么？

▼ 看答案
约为 `core_clk_aon / 4`。依据：mtime 由 SoC 的 `mtime_toggle_a` 脉冲驱动（每检测到边沿 mtime 加 1），Bumblebee 手册 §3.1 注记 GD32VF103 上 `rtc_clk = core_clk_aon / 4`，而 mtime_toggle_a 推荐 由 rtc_clk 驱动。若核心跑 108MHz，mtime 约 27MHz 自增（`core_clk_aon` 与核心时钟同源同频已由 [V3](08-mtime-delay.md) 收案：手册 §2.1）。

对照表：本章概念 → 仓库/手册落点

概念	仓库/手册落点
Bumblebee RV32IMAC、2 级流水线、Machine/User 模式	Bumblebee 手册 §1.1 / §2.4
无 NVIC / SysTick / SCB-VTOR / PendSV	Bumblebee 手册 §1.1 / §2.6 / §2.13
无 PMP	Bumblebee 手册 §2.7
ECLIC 16 level + 16 priority、向量、尾链、NMI	Bumblebee 手册 §1.1 ECLIC 条目 / §2.6
`CLIC_INT_SFT=3` / `CLIC_INT_TMR=7` / `BWEI=17` / `PMOVI=18`	`gd32vf103.h:101-104`
`EXTI0_IRQn=25` / `USART0_IRQn=56` / `USBFS_IRQn=86`	`gd32vf103.h:113` / `:144` / `:169`
`ECLIC_NUM_INTERRUPTS` 哨兵	`gd32vf103.h:171`
EXTI_BASE=0x40010400、AFIO_BASE=0x40010000（F1 风格）	`gd32vf103.h:220` / `:219`
`RCU_APB2EN.PAEN=bit2` / `AFEN=bit0`	`gd32vf103_rcu.h:165` / `:164`
MTIME 64 位、mtime/mtimecmp 内存映射、非 CSR	Bumblebee 手册 §2.13
mtime_toggle_a、rtc_clk = core_clk_aon / 4	Bumblebee 手册 §3.1 + GD32VF103 注记
ARM 对照：IRQn+16、4 位优先级、PRIGROUP、0x5FA、向量表	S4 第二/三/四节
ECLIC 寄存器精确偏移、shv 位、clicintattr/clicintctl 字段	官方固件驱动实证（n200_eclic.h:28-49）；语义细节以 ISA 手册为准
mtime/mtimecmp 精确映射基地址	官方固件驱动实证：0xD1000000（n200_timer.h:24-30）；nlbits 语义待 ISA 手册
`core_clk_aon` 与核心时钟的精确关系	已收案：同源同频（V3，手册 §2.1）

延伸阅读

ECLIC 的标准原型与权威寄存器表：

- **[C9]** RISC-V CLIC 规范草案 — "级别 + 优先级"双字段、向量直跳模式的标准原型，ECLIC 是它的增强实现。
- **[A14]** Bumblebee Core Datasheet — ECLIC 寄存器与 CSR 的权威表，GD32VF103 的中断源编号以它为准。
- **[C8]** RISC-V Privileged 20211203 — mstatus/mtvec/mie 的架构定义。

你做到了

- 中断控制器从 NVIC 换到 ECLIC，三处架构差异（控制器、两维优先级、向量直跳）入肌肉记忆；
- 中断号不再"IRQn + 16"，能从 `gd32vf103.h` 直接读 ECLIC 索引，知道同一条 EXTI0 线在两颗芯片上编号差很远；
- MTIME 不是 SysTick：64 位递增、内存映射、软件比较、`mtime_toggle_a` 驱动，RTOS tick 的地基（V3 接着盖）；
- 手握"EXTI 外设照搬 F1、第 6 步换 ECLIC 三动作"的清单，能在 GD32VF103 上配通任意外部中断。

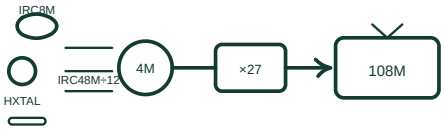
两扇门对答案：shv 置 1，硬件直跳该中断的专属入口；置 0，进公共入口由软件读 mcause 分发，ARM 那张向量大表没了。优先级另拆成两张：level 管抢占、priority 管同级排序。0x5FA 那把钥匙，这个世界不认。

✅ 下一站：V2 RCU 与 108MHz：G1 的 RCU 知识平移到 RISC-V，预设档 48/72/108M 一次算清。

通往单片机之路第 76 章

V2 RCU 与 108MHz

难度 ★★☆☆☆ · 约 30 分钟



本章罗盘

问题

同样叫 RCU，VF103 的 0x04 为什么不是 PLL？108MHz 分频链怎么推、G1 的哪些步骤消失？

交付

每种时钟源一条手推分频链（8M÷2×27、25M 绕 PLL1 两圈、IRC8M÷2），加 SystemCoreClockUpdate 回读公式的手算对账。

边界

CKOUT0 引脚 UM 全文不点名，PA8 只是 F1 血统约定、待上板扫脚确认；108M 档 Flash 等待拍数待实测；档位函数按 HXTAL_VALUE 板宏分叉，结论绑定 8M/25M 两类板。

依据锚点 `f6` · `a13` · `f7` · `mcu-anim`

> 🎯 G1 花一整章把 GD32F450 拉到 200MHz：大 VCO、电压三件套、FMC 留白。到了 VF103，时钟这摊事反而变简单——没有大 VCO、没有升压仪式，就一句"4MHz × 27"。但同名 RCU 里藏着陷阱：F4xx 的 RCU+0x04 是 PLL 寄存器，VF103 的 RCU+0x04 是 CFG0：照搬 G1 的地址，第一笔就写错地方。

本章精髓

1. **VF103 的 PLL 没有"大 VCO"**：F1 血统的两步走：CK_PLL = 输入 ÷ PREDV0（1~16）× PLLMF（×2~×32，还有 ×6.5 彩蛋）。G1 的 PSC/N/P/Q 四位域在这颗芯上不存在；108M 就是 **4MHz × 27**。
2. **预设档是"编译期开关"，且全家都是 4MHz 家族**：`system_gd32vf103.c` 顶部一排注释掉的 `__SYSTEM_CLOCK_*` 宏，取消注释即换档（默认 108M_HXTAL）。七个 HXTAL 档的乘数 ×6~×27 全部作用在 **4MHz** 的 PLL 输入上。三条来路殊途同归：8M 晶振 ÷2 得 4M；IRC8M ÷2 也得 4M；25M 晶振绕 PLL1 两圈还是 4M。
3. **同名不同图，"官方留白"却原样复现**：VF103 的 RCU 基址 0x40021000（跟 STM32F103 的 RCC 同址，不跟 F4xx 的 RCU 走）；0x04 偏移是 CFG0 不是 PLL。G1 的电压三件套在 VF103 上彻底不存在（只有管深睡电压的 DSV），而 `system_gd32vf103.c` 同样**全文不设 Flash 等待周期**。UM 实证 WSCNT 仅 0/1/2 三档有效，108M 档实际拍数待上板实测。

先修

- V0 RISC-V 工具链与启动文件（SystemInit 在 main 之前的角色）；
- V1 Bumblebee 内核与 CLIC 中断（知道 Bumblebee 是颗什么核、外设是 F1 血统）；
- G1 RCU 时钟树（RCU/CK_SYS 命名体系与"回读对账"纪律，本章的对照锚点）。

先跑起来（10 分钟 quick win）

打开官方固件库的 `system_gd32vf103.c`（本仓库核对副本在 `.trellis/ref/gd32/`，不随仓库分发），检索预设档：

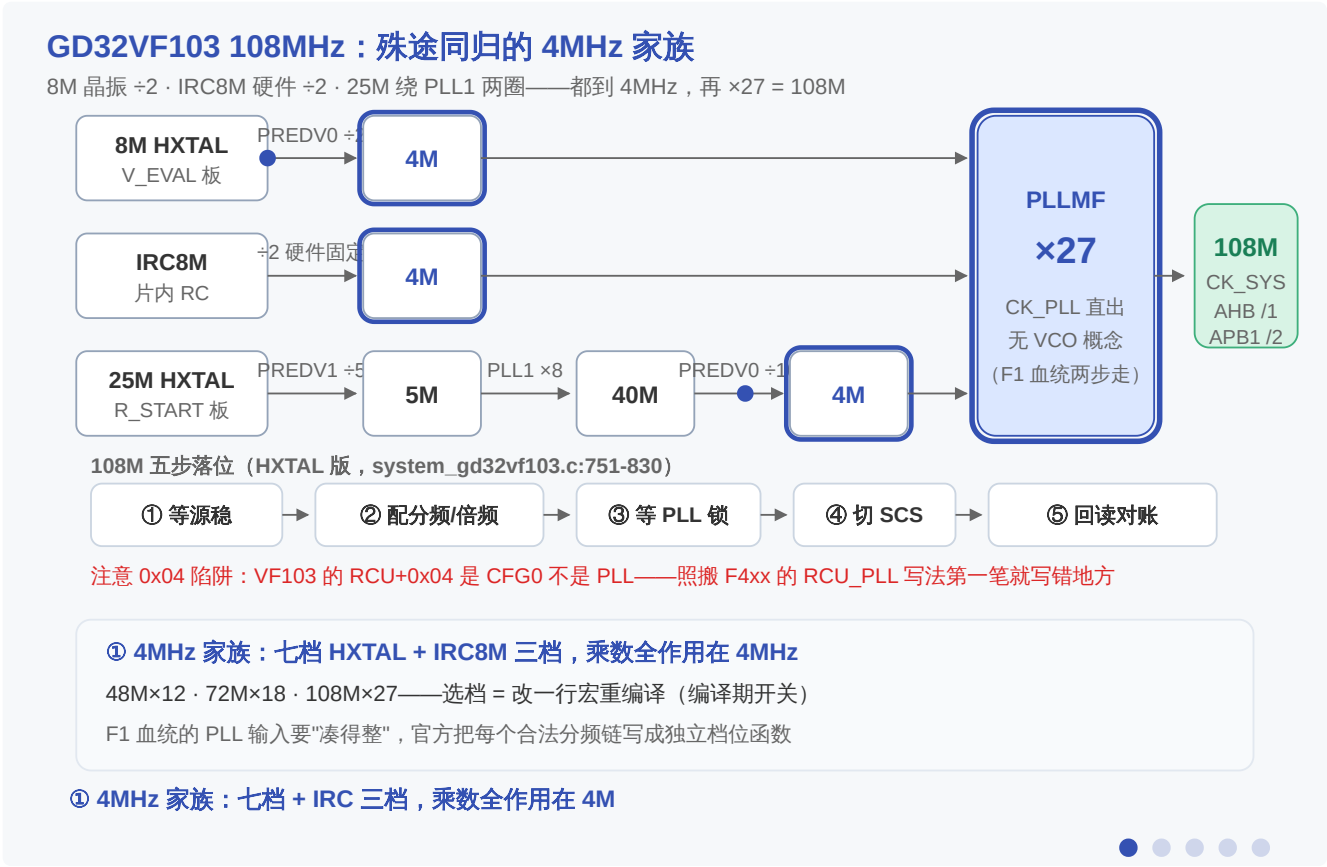
```
findstr /n "SYSTEM_CLOCK" system_gd32vf103.c
```

数一数注释掉的档位：IRC8M 系 3 个（48/72/108）、HXTAL 系 7 个、外加晶振直跑 1 个。再找唯一没注释的那行：`:62 __SYSTEM_CLOCK_108M_PLL_HXTAL`，官方默认档。最后心算验一遍：8MHz ÷ 2 × 27 = **108MHz**。本章每一节都是把这一行账拆开看。

先押把算盘：25M 那块板直分凑不出 4，卡在 25 的什么性质上？官方绕 PLL1 那一圈，每一步各凑哪个数？押完翻第三节对账。

动画：殊途同归的 4MHz 家族

8M 直路、IRC8M 硬件分频、25M 绕 PLL1 两圈，三条链殊途同归到 4MHz 再 ×27 出 108M；频率珠沿链流动，五步落位与回读对账一图看完。



GD32VF103 108MHz：殊途同归的 4MHz 家族

小节结构

小节	内容
一、两张 RCU 地图	基址、偏移逐项对照；0x04 陷阱与消失的 PLL 寄存器
二、预设档阶梯	编译期选档机制；4MHz 家族的完整乘数表
三、25M 的绕圈路	PLL1 中转链的推导：直分为什么凑不出 4MHz
四、逐位看 108M 五步	CTL/CFG0/CFG1 的位级落点；官方裸奔的"等就绪"
五、回读对账	SystemCoreClockUpdate 的解码公式与两个彩蛋
六、G1 清单对账	电压三件套、FMC 留白、CKOUT0 在 VF103 上的状态

一、两张 RCU 地图：同名 RCU，隔代布局

都叫 RCU，但 F4xx 与 VF103 是隔了两代的时钟控制器。先把地图摆开：

项	GD32F4xx（G1）	GD32VF103（本章）	凭证
RCU 基址	0x40023800（AHB1+0x3800）	**0x40021000**（AHB1+0x9000）	gd32f4xx.h:343；gd32vf103.h:202,226
0x00	RCU_CTL	RCU_CTL（同）	gd32f4xx_rcu.h:44；gd32vf103_rcu.h:46
0x04	**RCU_PLL**（PSC/N/P/Q）	**RCU_CFG0**	gd32f4xx_rcu.h:45；gd32vf103_rcu.h:47
0x08	RCU_CFG0	RCU_INT	gd32f4xx_rcu.h:46；gd32vf103_rcu.h:48
外设使能	AHB1EN 0x30 / APB1EN 0x40	**AHBEN 0x14 / APB2EN 0x18 / APB1EN 0x1C**	gd32f4xx_rcu.h:53,56；gd32vf103_rcu.h:51-53
PLL 参数放哪	独立 RCU_PLL 一个寄存器	拆进 CFG0（PLLMF 等）+ CFG1（PREDV0 等）	gd32vf103_rcu.h:79-91,220-227

两个值得停一拍的观察：

- **0x40021000 恰是 STM32F103 的 RCC 老地址**。VF103 的外设跟 F1 同源（V1 第三节已用 EXTI/AFIO 基址验过血统），连时钟控制器的门牌号都跟 F1 走。名字跟 F4xx 叫 RCU，地址跟 F1 住，这就是"混血"。
- **0x04 偏移陷阱**：G1 写 PLL 参数是 `RCU_PLL = ...`；VF103 头文件里根本没有 `RCU_PLL` 这个宏（寄存器清单见 gd32vf103_rcu.h:46-58，共 13 个、偏移 0x00~0x34），照搬"0x04 是 PLL"的直觉，写进去的是 **CFG0**：SCS/AHBPSC/PLLMF 全被砸中，切钟当场乱套。跨代芯片对照，**流程可以复用，地址必须重查**。

CTL 是两代最像的寄存器，公共位一模一样：HXTALEN bit16 / HXTALSTB bit17 / PLLEN bit24 / PLLSTB bit25（gd32vf103_rcu.h:67-72）；VF103 在此之上多了 PLL1EN/PLL1STB bit26/27、PLL2EN/PLL2STB bit28/29（:73-76），F1 时代为 USB/I2S 准备的两颗辅助 PLL。F4xx 用单 PLL 的 PSC/N/P/Q 替掉了这套，VF103 原样保留。

二、预设档阶梯：官方替你把账配好

G1 的 `system_gd32f4xx.c` 只给一套 200M 默认参数；VF103 的 `system_gd32vf103.c` 给了一整排"官方背书的合法组合"（:48-62）。选档 = 改一行宏 + 重编译：`SystemCoreClock` 初值取该宏（:69-105 的条件编译链），`system_clock_config()` 在编译期绑定对应函数（:116-142），这是编译期开关，不是运行时 API。

为什么 VF103 需要预设档而 F4xx 不怎么需要？F1 血统的 PLL 输入要"凑得整"：CK_PLL = PLL 输入 × 整数（2~32），还要同时满足"PREDV0 整数分频"与"CK_PLL 不超上限"，手配容易配出非法组合。官方干脆把每个合法档的分频链写成独立函数。已核对的乘数全家福（PLL 输入一律 4MHz）：

档位	乘数	凭证（system_gd32vf103.c）	档位	乘数	凭证
24M	×6	:342	72M	×18	:630
36M	×9	:413	96M	×24	:705
48M	×12	:484	108M	×27	:779
56M	×14	:558	IRC8M 48/72/108	×12/×18/×27	:868/:924/:980

主推三档 48/72/108 的乘数 12/18/27 作用在同一个 4MHz 上。三种时钟源怎么各自走到 4MHz，看下一节。

> 🧮 这里在在线版是一个可交互组件（`ClockCalc`，改参数即时出数）；离线书请按正文给定的参数手算，或到在线版该页把玩。

三、25M 的绕圈路：殊途同归的 4MHz

8M 晶振板（V_EVAL/C_START/T_START，gd32vf103.h:63-65）与 IRC8M 走的是直路：

```
8M HXTAL：8MHz ÷ PREDV0_DIV2 = 4MHz ── ×27 ──▶ 108MHz   (system_gd32vf103.c:799-801)
IRC8M   ：8MHz ÷ 2（PLLSEL 选 IRC8M_DIV2，硬件固定÷2）= 4MHz ── ×27 ──▶ 108MHz   (:980-982)
```

25M 晶振板（R_START，gd32vf103.h:60-61）凑不出 4M，只能绕。PREDV0 是整数分频（1~16），25 的任何整数分频（25/1、25/2、…、25/16）都除不出 4MHz：25 是 5 的倍数，4 不是 5 的倍数，整除链天然断掉。官方的解法是借 PLL1 当"频率整形器"（:783-786）：

```
25MHz ÷ PREDV1_DIV5 = 5MHz ─×8（PLL1）─▶ 40MHz ÷ PREDV0_DIV10 = 4MHz ─×27─▶ 108MHz
```

PREDV0SEL（CFG1 bit16）选 PREDV0 的进门（gd32vf103_rcu.h:225,605-606）：0 = HXTAL 直连（8M 板），1 = CK_PLL1 中转（25M 板）。`SystemCoreClockUpdate` 的回读路径里有同款分叉（:220-233）：25M 板要先算完 PLL1 链再算 PREDV0。

一个官方代码的小迷惑点：108M 的 8M 分支顺手配了 `PLL1_MUL20 | PLL2_MUL20`（:800-801）却没使能它们：PREDV0SEL=HXTAL 时 PLL1 根本不在路上；25M 分支则把 PLL2 也使能了（:794-798），而 PLL2 伺候的是 I2S，不参与系统时钟。**"配而不用"不等于"有用"**：读官方代码时拿回读公式当裁判。

还有一处档位差异要记：IRC8M 的启动超时是 0x0500，HXTAL 是 0xFFFF（gd32vf103.h:73,83）：片内 RC 起得快、晶振起得慢，官方连"等就绪的耐心值"都替你分好了档。

四、逐位看 108M 五步：HXTAL 版（:751-830）

步	写什么	等什么	官方实现
1	RCU_CTL.HXTALEN（bit16）	HXTALSTB（bit17）	do-while 计数超时 0xFFFF，但失败后 `while(1)` 死等（:756-769）
2	CFG0.AHBPSC/APB2PSC/APB1PSC	无需等	AHB=/1、APB2=/1、APB1=/2 → 108M/108M/54M（:772-777）
3	CFG0.PLLSEL + PLLMF=27；CFG1.PREDV0	无需等	8M：PREDV0_DIV2；25M：÷5×8÷10（:780-815）
4	RCU_CTL.PLLEN（bit24）	PLLSTB（bit25）	while 裸轮询，**无超时**（:817-821）
5	CFG0.SCS=2（CKSYSSRC_PLL）	SCSS=2（SCSS_PLL）	while 裸轮询，无超时（:824-829）

宏名各就各位：`RCU_PLL_MUL27 = PLLMF_4 | CFG0_PLLMF(10)`（gd32vf103_rcu.h:506）：bit29（乘数第 5 位）+ bits[18:21]=10；`RCU_PREDV0_DIV2`（:542）写 CFG1 bits[0:3]=1；`RCU_CKSYSSRC_PLL`（:429）写 SCS bits[0:1]=2。

与 G1 同款的批评：**官方的"等就绪"五处有两处裸奔**：PLLSTB 与 SCSS 的等待没有任何超时，晶振不起、PLL 不锁，程序就永远停在那行 while 上。G1 在 F4xx 上批过 `while(1)` 死等（G1 §六），VF103 这边一模一样，教学工程照旧要加超时回退（回 IRC8M 档）。另外 SystemInit 在进 main 前还干了一遍"大扫除"：清 SCS/分频/PLLMF、CFG1 归零、RCU_INT 写 0x00FF0000 清全部中断标志（:153-175）。你的提频代码永远从一张干净的桌面开始。

五、回读对账：SystemCoreClockUpdate 的解码公式（:187-262）

G1 的纪律原样适用：**变量 `SystemCoreClock` 是账本，SCSS/CFG0/CFG1 才是真账**。官方给了完整的回读函数，PLL 分支的解码链：

1. SCSS bits[2:3]：0=IRC8M、1=HXTAL、2=PLL（:193,198-208）；
2. PLLSEL：选 IRC8M_DIV2 → 输入 = IRC8M/2；选 HXTAL → 看 PREDV0SEL，若为 CKPLL1 先算 PLL1 链（PREDV1+1、PLL1MF+2，**17→20 特例**，:224-228），再 ÷ PREDV0（+1，:231-232）；
3. PLLMF：raw = bits[18:21]，bit29 补成第 5 位；**raw 小于 15 时 +2、不小于 15 时 +1**（:242-246）；
4. 结果恰为 15 → 不是 ×15，是 **×6.5**，用 `ck*6 + ck/2` 整数式实现（:250-253）。

两个彩蛋值得停一拍：

- **×6.5 不是小数乘法**：`ck * 6U + ck / 2U`：整型世界没有 0.5，只有"×6 再补一半"。
- **raw=15 是表上的黑洞**：头文件的 MUL 宏从 raw14（×16）直接跳到 raw16（×17，gd32vf103_rcu.h:495-496），raw15 无定义；但回读公式会把 raw15 也算成 ×16。用不到的编码写进了逻辑，眼下不出 bug，却提醒你：**官方库也会有随手写的分支，读源码要带脑子**（G1 在 F4xx 上抓过同款"恒假的等待"）。

对账动作：改档重编译后，GDB 断在 main，把 CFG0/CFG1 的值代进上面公式手算一遍，再与 `SystemCoreClock` 变量比对，两边相等，档位才算真落下。

六、G1 清单对账：三件套的 VF103 状态

G1 事项	GD32F4xx	GD32VF103	凭证
电压档三件套	LDOVS→HDEN→HDS，带 HDRF/HDSRF 回执	**不存在**。RCU 里只有 DSV（深睡电压 1.2/1.1/1.0/0.9V 档）	gd32vf103_rcu.h:617-622
Flash 等待	FMC_WS 0~15 档（WSCNT 4 位域）+ FMC_WSEN 使能，官方留白	**同样留白**：system_gd32vf103.c 全文零 FMC 写入（检索实证）；VF103 的 FMC_WS 就在 @0x00，WSCNT[2:0] 仅 0/1/2 三档有效（011~111 保留，UM §2.4.1），同样要 WSEN 使能，108M 档拍数待实测	两库头文件 + 全文检索 + UM
CKOUT0 对账	PA8，/1~/5 分频	源 9 选（NONE/CKSYS/IRC8M/HXTAL/CKPLL 除 2/CKPLL1/CKPLL2 除2/EXT1/CKPLL2），**无分频器字段**（F1 风格源直出）；UM 实证输出能力 **0.09375 ~ 108 MHz**，但**全文不点引脚名**，只要求"把对应 GPIO 配成 AFIO 模式"	gd32vf103_rcu.h:521-530 + UM §5.2 p67
PLL 结构	单 PLL：PSC/N/P/Q，VCO 400M 再 ÷2	无 VCO 概念：PREDV0 分频 + PLLMF 倍频，CK_PLL 直出；另有 PLL1/PLL2 伺候 USB/I2S	gd32vf103_rcu.h:474-511

"108M 为什么不用升压"：VF103 是 3.3V 固定供电的 F1 外设工艺家族，主频上限就是数据手册标的 108MHz，没有 F4xx 那套"调电压换频率"的档位经济学。**G1 的提频六步到 VF103 收敛成四步：等源稳 → 配分频/倍频 → 等 PLL 锁 → 切 SCS 等 SCSS**：少掉的"电压"与"Flash 等待"两步，前者不存在，后者官方留白给你。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **0x04 是 CFG0 不是 PLL；108M = 4M×27，48/72/108 全是 4MHz 家族；25M 绕 PLL1 两圈才到 4M；三件套没有、FMC 照样留白；账本不如 SCSS，回读公式手算一遍。**

延伸：GD32VF103 108MHz 动画见 V2 动画；STM32 RCC 对照见 S2；GD32F4xx RCU 对照见 G1。

实物实验

- **SCSS 对账**：GDB 断在 main，读 RCU_CFG0 的 SCSS（bits[2:3] 应为 2=PLL）与 PLLMF 字段、RCU_CFG1 的 PREDV0。代回第五节公式应得 108000000，与 SystemCoreClock 变量比对。
- **换档实操**：注释 108M、放行 72M（:62 附近换宏），重编译烧录，再读 SCSS/PLLMF，两分钟一次"编译期选档"的肌肉记忆；注意 72M 档 APB1 变 36M，外设波特率要跟着重配。
- **CKOUT0 量频**：CKOUT0SEL 选 IRC8M 或 HXTAL（8MHz 好量），示波器对账时钟源真伪。UM 给的输出能力是 **0.09375 ~ 108 MHz**（§5.2 p67），够量全档；但 UM 自己**不点引脚名**（全文 8 处 CK_OUT0 命中无一处给引脚），只说"把对应 GPIO 配成 AFIO 模式"。F1 传统在 PA8，**上板用示波器扫一遍 PA8 就能确认**（这本身就是一个好实验：手册留白处，仪器说话）。
- **mtime 频率联动**：108M/72M 两档分别读 mtime 差值反推自增频率（27MHz/18MHz），给 V3 的延时标定铺路。

常见坑

1. **照搬 G1 的 RCU_PLL**：VF103 头文件里没有这个宏，0x04 是 CFG0，PLL 参数写进时钟配置寄存器，切钟当场乱。跨代对照，流程可复用、地址要重查。
2. **25M 板抄 8M 板的 PREDV0_DIV2**：25/2×27 = 337.5MHz 超频一倍多，PLLSTB 永远等不到。档位函数内部按 `HXTAL_VALUE` 分叉（gd32vf103.h:60-65 的板宏），改板不改宏必炸。
3. **带着 F4 肌肉记忆找电压三件套**：VF103 没有 LDOVS/HDEN/HDS，唯一的"电压"寄存器 DSV 管深睡模式，108M 档无电压仪式，白找。
4. **等就绪不加超时**：官方 PLLSTB/SCSS 两处裸 while，晶振不起就死机。教学工程要超时回退 IRC8M（G1 同款批评）。
5. **只信 SystemCoreClock 变量**：改了档忘了让变量跟上（或反之），延时/波特率全错。判据永远是 SCSS+CFG0+CFG1 的回读公式。

短自测

1. VF103 的 RCU 基址是多少？它跟 GD32F4xx 的 RCU、STM32F103 的 RCC 各是什么关系？

> ▼ **看答案**
> 0x40021000（gd32vf103.h:202,226：AHB1+0x9000）。它不跟 F4xx 的 RCU（0x40023800）同址，反而与 STM32F103 的 RCC 老地址 0x40021000 一致：VF103 外设是 F1 血统（V1 用 EXTI/AFIO 基址验过），门牌号也跟 F1 走。"RCU"这个名字来自 GD32 家族统一命名。

2. 48/72/108 三档的共同点是什么？写出 8M 晶振下 108M 档的完整分频链。

> ▼ **看答案**
> 三档的 PLL 输入都是 4MHz，乘数分别是 ×12/×18/×27（system_gd32vf103.c:484/630/779），总线分频都是 AHB/1、APB2/1、APB1/2。8M 晶振的 108M 链：8MHz ÷ PREDV0_DIV2 = 4MHz，× PLLMF(27) = 108MHz，最后 SCS 切 PLL、等 SCSS 回读确认。

3. 25M 晶振配 108M 为什么要绕 PLL1？直接 PREDV0 分频为什么凑不出来？

> ▼ **看答案**
> PREDV0 只能整数分频 1~16，25 的任意整数分频都得不到 4MHz：25 是 5 的倍数而 4 不是，整除关系天然不存在。官方借 PLL1 整形：25M ÷ PREDV1_DIV5 = 5M，× PLL1_MUL8 = 40M，÷ PREDV0_DIV10 = 4M，再 ×27 = 108M（system_gd32vf103.c:783-786）。PREDV0SEL=CKPLL1 时 PLL1 才进链路；8M 板选 HXTAL 直连，PLL1 配了也不在道上。

4. SystemCoreClockUpdate 里 raw=13 的 PLLMF 解出什么乘数？怎么实现的？

> ▼ **看答案**
> raw13 小于 15，先 +2 得 15；15 命中特例：不是 ×15 而是 ×6.5，用整数式 ck×6 加 ck/2 实现（system_gd32vf103.c:242-253）。头文件侧 RCU_PLL_MUL6_5 = CFG0_PLLMF(13)（gd32vf103_rcu.h:494）。整型世界没有小数乘法，只有"乘整数再补一半"。

5. G1 的"电压三件套"在 VF103 上对应什么？为什么 108M 档不用配它？

▼ **看答案**
什么都不对应：不存在。F4xx 的 LDOVS/HDEN/HDS 是"调电压换频率"的机制，由 PMU 承载（G1 第三节）；VF103 是 3.3V 固定供电的 F1 外设家族，RCU 里唯一的电压字段 DSV 只管深睡模式电压（gd32vf103_rcu.h:617-622）。提频步骤因此比 G1 少两步：没有电压档可切，Flash 等待官方留白。

对照表：本章概念 → 仓库落点

概念	落点
RCU 基址 0x40021000 = AHB1+0x9000	gd32vf103.h:202,226
0x04 是 CFG0 不是 PLL	gd32vf103_rcu.h:47；gd32f4xx_rcu.h:45
RCU 13 个寄存器、偏移 0x00~0x34	gd32vf103_rcu.h:46-58
HXTALEN/STB bit16/17、PLLEN/STB bit24/25、PLL1/PLL2 四位	gd32vf103_rcu.h:67-76
CFG0/CFG1 位域（SCS/SCSS/PLLMF/PREDV0/CKOUT0SEL...）	gd32vf103_rcu.h:79-91,220-227
预设档清单与默认 108M_HXTAL	system_gd32vf103.c:48-62
全家乘数 ×6/×9/×12/×14/×18/×24/×27	system_gd32vf103.c:342/413/484/558/630/705/779
IRC8M 档 ×12/×18/×27	system_gd32vf103.c:868/924/980
108M 五步流程与裸奔等待	system_gd32vf103.c:751-830
回读公式（SCSS/PLL1 链/PLLMF 修正/×6.5）	system_gd32vf103.c:187-262
SystemInit 大扫除（清位/CFG1 归零/INT 写 0x00FF0000）	system_gd32vf103.c:150-179
板级 HXTAL 值分叉（R_START 25M、其余 8M）	gd32vf103.h:60-65
启动超时 HXTAL 0xFFFF / IRC8M 0x0500	gd32vf103.h:73,83
无电压三件套、DSV 深睡电压档	gd32vf103_rcu.h:617-622
CKOUT0 九源无分频	gd32vf103_rcu.h:521-530
CKOUT0 输出能力 0.09375 ~ 108 MHz	**GD32VF103 UM EN V1.0 §5.2 已核验**（p67）
官方不设 Flash 等待；108M 档等待拍数	**UM 实证**：FMC_WS 的 WSCNT[2:0] 仅 000/001/010 三档有效（011~111 全部保留，UM §2.4.1 p40）；UM 未给"频率↔拍数"映射，实际拍数待上板或厂方推荐
CKOUT0 引脚与 AF	UM §5 只列 CKOUT0SEL 九源选择位与"配成 AFIO 模式"要求、**全文未给引脚名**（8 处命中逐一检索确认）——F1 血统约定 PA8，待硬件实测确认

你做到了

- 两个 RCU 的"同名不同图"记进肌肉：0x04 陷阱、地址跟 F1 走、三件套消失、FMC 留白复现；
- 108M 不再是魔法数：4MHz 家族的分频链你能对每种晶振独立推算，回读公式能手算对账；
- 预设档机制上手：换档是改宏重编译，不是运行时调用。

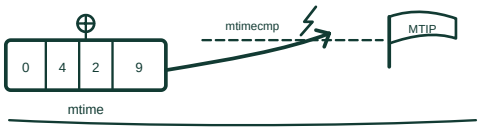
算盘对上：25 是 5 的倍数、4 不是，整数分频的链天然断掉，只能借 PLL1 做频率整形，÷ PREDV1_DIV5、×8、再 ÷ PREDV0_DIV10，才落回 4；8M 晶振 ÷2、IRC8M ÷2 是直路，全家压到 4MHz 再 ×27 出 108M。G1 那张 PSC/N/P/Q 四位域的图在这颗芯上不存在，0x04 偏移住的也不是 PLL，是 CFG0。

✅ 下一站：V3 MTIME 与裸机延时：时钟拉到 108M 之后，看 RISC-V 拿哪块"表"接班 SysTick。

通往单片机之路第 77 章

V3 MTIME 与裸机延时

难度 ★★☆☆☆ · 约 45 分钟



本章罗盘

问题

没有 SysTick 的 RISC-V 拿什么计时？为什么 mtime 按主频四分之一走、64 位读写还有纪律？

交付

一段宿主可跑的 tick 预演（108M 档 1ms=27000），加 delay_ms 与周期 tick ISR 代码：高-低-高读、先高后低写、cmp 加周期续约。

边界

mtime 地址以本芯片官方库实证（0xD1000000）为准，不是 QEMU 惯例布局，换核必重查；调试器指定执行期间 mtime 停走，标定必须脱机做；mcountinhibit 位定义待 ISA 手册核验。

依据锚点 `a14 · c8 · f7 · mcu-anim`

> 🎯 S5 给了 Cortex-M 一颗 24 位"倒数秒表"：写 LOAD、看 COUNTFLAG、硬件自动重装。RISC-V 换成一块 64 位"顺数电表"：mtime 从不回绕、没有归零标志，想定时就自己拿 mtimecmp 比个数，比较到了，MTIP 中断走 V1 的 ECLIC 上门。还有笔反直觉的账：这块表按主频的四分之一走，108MHz 的芯，1ms 只有 27000 格。

本章精髓

1. **mtime/mtimecmp 是内存映射，不是 CSR**（Bumblebee 手册 §2.13 原文明说）：64 位顺数，mtime 不小于 mtimecmp 即产生定时器中断。ECLIC 编号 `CLIC_INT_TMR = 7`（gd32vf103.h:102）。SysTick 的四件套（CTRL/LOAD/VAL/CALIB）在这换成"两张内存表 + 一个比较"。
2. **驱动链三步走，主频除以四**：mtime_toggle_a 每个边沿加 1（手册 §3.1）。GD32VF103 的 rtc_clk = core_clk_aon/4（§3.1 注记）。core_clk_aon 与 core_clk 同源同频（§2.1）。三步连起来：**mtime 自增频率 = SystemCoreClock/4**。108M 档 27MHz、1ms = 27000 tick；V1 遗留的"待核验"在此收案。
3. **64 位是福气也是机关**：回绕要 2.2 万年，"防回绕"的担心可以退休。但 RV32 读它必须"高-低-高"循环防撕裂、写 mtimecmp 必须"先高后低"防假中断。周期 tick 没有"自动重装"：ISR 里 `mtimecmp += 周期`，忘了就只响一次。

先修

- V0 RISC-V 工具链与启动文件（SystemInit 与 108M 档）；
- S5 SysTick：内核的心跳（周期 = LOAD+1、COUNTFLAG 读清、回绕安全写法，本章的对照锚点）。

先跑起来（10 分钟 quick win）

延时公式不必上板才信，宿主 gcc 五分钟先算一遍：

```
#include <stdint.h>
#include <stdio.h>

int main(void)
{
    uint64_t ticks_per_ms = 108000000ULL / 4U / 1000U;  /* mtime = 主频/4 */
    uint64_t t0 = 0x00000000FFFFFFFFULL;                /* 造一个"低位即将进位"的时刻 */
    uint64_t t1 = t0 + 3 * ticks_per_ms;                /* 3ms 后 */
    printf("1ms = %llu ticks, 3ms = %llu ticks\n",
           (unsigned long long)ticks_per_ms,
           (unsigned long long)(t1 - t0));
    return 0;
}
```

`gcc demo.c -o demo && ./demo` 输出 `1ms = 27000 ticks, 3ms = 81000 ticks`。先减再比的无符号差值跨过 32 位边界也不出错。本章全部延时数学都能这样在宿主预演。

先押个仪表：SysTick 的 CTRL/LOAD/VAL 四件套，到 mtime 这边换成什么阵型？S5 背熟的减一、读清两个魔咒，有几个跟过河？

动画：MTIME 64 位读写纪律

RV32 读 64 位 mtime 走"高-低-高"循环防撕裂，写 mtimecmp 必须"先高后低"防假中断。两条读写纪律保住跨 32 位边界的时间账。

MTIME 64 位读写纪律：hi-lo-hi 防撕裂

小节结构

小节	内容
一、从秒表到电表	SysTick 四件套 vs mtime/mtimecmp 两张表；地址机关
二、主频除四	mtime_toggle_a/rtc_clk/core_clk_aon 三步链条与 27MHz 账
三、64 位读写纪律	高-低-高防撕裂、先高后低防假中断、非 CSR
四、查询式延时	delay_ms 的完整实现与分辨率账
五、周期 tick	MTIP 走 ECLIC 编号 7；ISR 里 mtimecmp 加周期续约

一、从秒表到电表：SysTick 的四件套换成两张表

先把 S5 的旧账和新表摆在一起：

	SysTick（S5，Cortex-M）	MTIME（本章，Bumblebee）
方向/位数	24 位倒数	**64 位顺数**
寄存器	CTRL/LOAD/VAL/CALIB 四个	**mtime、mtimecmp 两张内存表**
位置	SCS 区 0xE000E010（CMSIS core_cm4.h）	内核私有 TIMER 单元，内存映射——gd32vf103.h 里**没有**它的宏
周期	LOAD+1，写一次自动重装	**无重装**：mtime 不小于 mtimecmp 即中断，之后软件续约
标志	COUNTFLAG（读过自清）	无标志，比大小
中断号	IRQn=-1（内核异常）	ECLIC `CLIC_INT_TMR = 7`（gd32vf103.h:102）
使能	CTRL.ENABLE	没有 CTRL——想停表只能动 CSR mcountinhibit

最后一行值得停一拍：**mtime 连"开关"都没有**。上电就顺数，能停它的只有 CSR `mcountinhibit`（手册 §2.13.2，位定义待 ISA 手册核验）；但"不产生中断"你说了算：mtimecmp 挂得足够高，MTIP 永远安静。

地址机关：RISC-V 架构不定义 mtime/mtimecmp 的地址（手册 §2.13 明说"由实现定义"）。在 GD32VF103 上它们落在内核私有 **TIMER 单元**，官方固件库驱动 `n200_timer.h` 给出实证：`TIMER_CTRL_ADDR = 0xD1000000`，mtime 偏移 +0x0、mtimecmp 偏移 +0x8（64 位各占 8 字节，n200_timer.h:24-30）。`n200_func.c:61-69` 的 `mtime_lo()/mtime_hi()` 直接按此地址读写。注意这不是 RISC-V 规范里 QEMU 惯例的 0x0200_0000 布局，**以本芯片官方库为准**。工程里自己写宏：

```
#define CLINT_BASE      0xD1000000UL            /* GD32VF103 TIMER 域（官方库 n200_timer.h 实证） */
#define MTIMECMP_LO     (*(volatile uint32_t *)(CLINT_BASE + 0x8U))
#define MTIMECMP_HI     (*(volatile uint32_t *)(CLINT_BASE + 0xCU))
#define MTIME_LO        (*(volatile uint32_t *)(CLINT_BASE + 0x0U))
#define MTIME_HI        (*(volatile uint32_t *)(CLINT_BASE + 0x4U))
```

拆成 32 位两半不是多此一举：RV32 的访存指令一次只有 32 位，第三节会讲拆法里的讲究。

二、主频除四：mtime 的驱动链

mtime 不是拿内核时钟直接数的。手册给了一条三步链条，每步都有原文背书：

1. **mtime_toggle_a 双边沿触发**：TIMER 单元由 SoC 送来的脉冲信号驱动，"上升沿和下降沿都采样，检测到任何边沿就触发 mtime 加 1"（手册 §3.1 表 3-1）；
2. **GD32VF103 的 rtc_clk**：手册 §3.1 注记写明，rtc_clk 的频率是 core_clk_aon 的四分之一，因此 mtime 自增频率 = core_clk_aon/4；
3. **core_clk_aon 就是内核时钟**：手册 §2.1 说，"core_clk 与 core_clk_aon 频率相同、相位相同、来自同一时钟源"，后者只是"常开"分支（睡着了表也走）。

三步连乘：**mtime 自增频率 = core_clk/4 = SystemCoreClock/4**。108M 档 = 27MHz，一个 tick 约 37ns；72M 档 = 18MHz。V1 第六节留的"core_clk_aon 与核心时钟关系待 UM 核验"到此收案，答案在手册 §2.1 的第一句。

两笔衍生账：

- **分辨率账**：1ms = 27000 tick；延时的粒度上限是 1 tick（约 37ns），比 CPU 周期（9.3ns）粗四倍。mtime 管 ms 级"宏观时间"，单周期级标定请用 mcycle/minstret（手册 §2.12）或外设定时器。
- **回绕账**：64 位表有 2 的 64 次方格，按 27MHz 数完一圈要约 2.2 万年。S5 里 uint32_t"49 天必爆"的担心，在 mtime 上以"两万年都不爆"的形态退休。

调试课代表还要知道一条：**调试器指定程序执行期间 mtime 自动停走**（手册 §2.13.1）。本意是不让调试程序污染计时，副作用是断点、单步期间的时间读数不可信，延时标定要脱机跑。

三、64 位读写纪律：撕不烂、打不响

读：高-低-高循环。mtime_lo 从 0xFFFFFFFF 进到 0x00000000 的那一拍，mtime_hi 同时加 1。若先读高位再读低位，低位可能已经是"新一轮"的，旧高配新低，读数差出一个 2 的 32 次方。纪律是读两次高位当证人：

```
uint64_t mtime_read(void)
{
    uint32_t hi, lo;
    do {
        hi = MTIME_HI;
        lo = MTIME_LO;
    } while (hi != MTIME_HI);   /* 高位没变 = 低位没跨过 32 位边界 */
    return ((uint64_t)hi << 32) | lo;
}
```

为什么这样就安全：高位不变恰说明这期间没有发生 2 的 32 次方进位，低位的读取落在同一段"高位纪元"里，拼起来必是真值。

写：先高后低。把 deadline 往后推（新比较值比当前大）时，必须先写 MTIMECMP_HI 再写 MTIMECMP_LO。反过来先写低位：新低配旧高，若旧高小于 mtime 当前高位，拼出的"半新半旧"值可能瞬间满足"mtime 不小于 mtimecmp"，假 MTIP 当场到账。先写高位把比较值推上高空，中途怎么比都撞不响，最后写低位让它落位。

非 CSR 再钉一遍：mtime/mtimecmp 用 `lw/sw` 普通访存读写，`csrr/csrw` 会触发非法指令异常。手册 §2.13 原文："not CSR registers, but Memory Address Mapped system registers"（V1 坑 3 的实操版）。

四、查询式延时：S5 的骨架换 64 位心脏

```
extern uint32_t SystemCoreClock;   /* system_gd32vf103.c 的账本（[V2](07-rcu-108m.md)） */

void delay_ms(uint32_t ms)
{
    uint64_t start = mtime_read();
    uint64_t ticks = (uint64_t)ms * (SystemCoreClock / 4U / 1000U);
    while ((mtime_read() - start) < ticks) {
        /* 先减再比：64 位无符号差值，跨"低位边界"也不出错 */
    }
}
```

三个要点，条条对着 S5 的旧账：

1. **先减再比**。S5 第五节的回绕安全写法在 64 位上更是天经地义；宿主 quick win 里 0xFFFFFFFF 起点那笔账就是证明。
2. **ticks 现算不写死**。SystemCoreClock/4 跟着档位走：108M 档 27000，换 72M 档自动变 18000。写死 27000 的延时，换个档全慢 1.5 倍。
3. **S5 的两个魔咒在这消失**。没有"减一"（比较语义没有 off-by-one，27000 就是 27000）；没有"读清"（查询的是计数本身，不是读过就没的标志）。COUNTFLAG 那个"调试器偷看一眼就死等"的坑，结构性不存在。

分辨率提示：函数调用加两次 mtime_read 本身就有百余 ns 级开销，delay_us 若要精到个位数微秒，请换外设定时器（后续实验篇）；mtime 的主场是 ms 级以上的宏观时间。

五、周期 tick：没有自动重装，软件续约

想要 1ms 周期心跳（RTOS tick、毫秒时基），先把比较值挂上：

```
#define TICK_MS  1U

static uint64_t mtimecmp_read(void)   /* 与 mtime_read 同款纪律 */
{
    uint32_t hi, lo;
    do { hi = MTIMECMP_HI; lo = MTIMECMP_LO; } while (hi != MTIMECMP_HI);
    return ((uint64_t)hi << 32) | lo;
}

void tick_init(void)
{
    uint64_t cmp = mtime_read() + TICK_MS * (SystemCoreClock / 4U / 1000U);
    MTIMECMP_HI = (uint32_t)(cmp >> 32);   /* 先高后低（第三节纪律） */
    MTIMECMP_LO = (uint32_t)cmp;
    /* ECLIC 侧三动作：clicintie[7] 使能、clicintctl[7] 设 level/priority、
       clicintattr[7] 配向量模式——见 [V1 第七节](06-clic-irq.md)，偏移已实证（n200_eclic.h） */
}

void mtime_isr(void)                   /* MTIP → ECLIC 编号 7 分发 */
{
    uint64_t cmp = mtimecmp_read();
    cmp += TICK_MS * (SystemCoreClock / 4U / 1000U);   /* 续约：加周期，不是 now 加周期 */
    MTIMECMP_HI = (uint32_t)(cmp >> 32);
    MTIMECMP_LO = (uint32_t)cmp;
    g_tick++;                          /* 全站毫秒时基；volatile 见 C3 */
}
```

续约那一行是 RISC-V tick 的灵魂，两个对比记牢：

- **对比 SysTick**：S5 写一次 LOAD 硬件自动重装；mtimecmp 到点只响不重置，**不写 `cmp += 周期` 就只响一次**。这是 RISC-V RTOS port 必须处理的第一差异（F8 篇的伏笔）。
- **对比 now 加周期**：`cmp = mtime_read() + 周期` 看着等价，实则每次把"中断延迟"都吃进下一拍，长期走时越走越慢。`cmp += 周期` 把节拍锚在绝对时间轴上。S5 第三节"忘减一相位漂移"的教训在这换了马甲：**写 LOAD 忘减一，tick 慢几 ppm；续约用 now 加周期，tick 慢的是每次中断延迟之和**。RTOS 心跳，认准加法续约。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **电表不是秒表：64 位顺数、比大小发中断、非 CSR；主频除四走（108M 档 1ms=27000）；读高-低-高、写先高后低；周期全靠软件续约：cmp 加周期，别用 now 加周期。**

实物实验

- **频率标定**：裸机循环里连续 mtime_read() 取差值，反推自增频率。108M 档应约 27MHz、72M 档约 18MHz；与 V2 的换档实验联动做。
- **delay 对账**：delay_ms(500) 翻转 LED（V4 把这个闭环拼完整），逻辑分析仪量周期，误差应在整 tick 范围内。
- **调试器盲区**：断点下读两次 mtime 差值，对比脱机跑的同款数据，手册 §2.13.1 的"调试停走"亲手复现，从此不在调试会话里标定延时。
- **撕裂复现（选做）**：把 mtime_read 换成"只读低位"跑长延时，观察进位时刻的读数跳变。再换回高-低-高循环，症状消失。

常见坑

1. **csrr 读 mtime**：不是 CSR（手册 §2.13 原文），非法指令异常。用 lw/sw 走普通访存。
2. **一条 `volatile uint64_t` 直读**：RV32 编译器拆成两次 32 位读，跨越低位进位时高低位撕裂，读到"过去"或"未来"；高-低-高循环是唯一安全读法。
3. **mtimecmp 先写低位**：半新半旧的比较值可能被 mtime 追平，假 MTIP 立刻上门；推 deadline 永远先写高 32 位。
4. **SysTick 肌肉记忆找"归零标志"、等"自动重装"**：mtime 两样都没有。tick ISR 忘写 `cmp += 周期` 就只响一次，RTOS 起不来。
5. **延时常数写死 27000**：换 72M 档后 mtime 是 18MHz，全慢 1.5 倍；用 SystemCoreClock/4 现算，让档位变化自动跟随。

短自测

1. 108M 与 72M 档下，1ms 各对应多少个 mtime tick？写出推导。

> **▼ 看答案**
> mtime 自增频率 = SystemCoreClock/4。驱动链三步：mtime_toggle_a 双边沿触发；GD32VF103 的 rtc_clk = core_clk_aon/4；core_clk_aon 与 core_clk 同源同频（见手册 §3.1 与 §2.1）。108M 档：108000000/4/1000 = 27000；72M 档：18000。没有"减一"，比较语义没有 off-by-one，算出多少就是多少。

2. 为什么 mtime 必须按"高-低-高"循环读？直接声明成 64 位变量一次读会怎样？

> **▼ 看答案**
> mtime_lo 进位回 0 的那一拍，mtime_hi 加 1。RV32 一次访存只有 32 位，64 位读取必然拆两次：若先读高后读低，低位可能已是"新一轮"，旧高配新低，值差出一个 2 的 32 次方量级。高-低-高循环用"第二次读高位未变"证明这期间没发生进位，低位的读取属于同一"高位纪元"，拼出来必为真值。

3. 把 deadline 从 t 推到 t+3ms，mtimecmp 先写哪半字？反着写有什么后果？

> **▼ 看答案**
> 先写高 32 位再写低 32 位。反着写：新低位配旧高位，若旧高位小于 mtime 当前高位，拼出的比较值可能瞬间被 mtime 追平，假 MTIP 立刻触发。先写高位把比较值推上高空，中途任何比较都撞不响，写完低位才落位到目标。

4. mtime 的周期中断重装谁负责？与 S5 写一次 LOAD 有什么本质不同？RTOS tick 的 ISR 必须写哪条语句？

> **▼ 看答案**
> 软件负责：mtime 达到 mtimecmp 只触发一次 MTIP，硬件不重置比较值。SysTick 是"写 LOAD 一劳永逸"（硬件自动重装），mtime 是"每次 ISR 续约"。必须写 `mtimecmp += 周期`，不是 `mtime_read() + 周期`（那会把每次中断延迟吃进下一拍，长期走时变慢；加法续约把节拍锚在绝对时间轴上）。

5. mtime 自增频率的三步手册链是什么？哪一步把 V1 遗留的"待核验"收了案？

> **▼ 看答案**
> 三步：① mtime_toggle_a 上升、下降沿都触发 mtime 加 1（手册 §3.1 表 3-1）。② GD32VF103 的 rtc_clk = core_clk_aon/4，故自增频率 = core_clk_aon/4（§3.1 注记）。③ core_clk_aon 与 core_clk 同源同频（§2.1）。第 ③ 步收了 V1 的案：V1 曾标注"core_clk_aon 与核心时钟的精确关系待 UM 核验"，答案就在手册 §2.1："same frequency and phases... same clock source"。

对照表：本章概念 → 仓库/手册落点

概念	落点
mtime/mtimecmp 非 CSR、64 位、比较触发	Bumblebee 手册 §2.13
mtime_toggle_a 双边沿触发	Bumblebee 手册 §3.1 表 3-1
GD32VF103 rtc_clk = core_clk_aon/4	Bumblebee 手册 §3.1 注记
core_clk_aon 与 core_clk 同源同频	Bumblebee 手册 §2.1
调试器指定程序执行期间 mtime 停走	Bumblebee 手册 §2.13.1
mcountinhibit 可停表	Bumblebee 手册 §2.13.2（位定义待 ISA 手册核验）
mcycle/minstret 做周期级标定	Bumblebee 手册 §2.12
CLIC_INT_TMR = 7	gd32vf103.h:102
mtime/mtimecmp 精确地址（TIMER 域布局）	官方库实证：0xD1000000（mtime@+0x0、mtimecmp@+0x8，n200_timer.h:24-30）；UM 复核后更新
delay 的 ticks 公式与 27000	本章第四节；V2 SystemCoreClock
ECLIC 三动作（使能/level/priority/向量）	V1 第七节
回绕安全超时写法（先减再比）	S5 第五节

延伸阅读

64 位电表的架构定义：

- **[C8]** RISC-V Privileged 20211203 "Machine Timer Registers"一节。要点：mtime/mtimecmp 64 位、从不回绕、"比较到了就置 MTIP"。时钟源不归 ISA 管，所以 1/4 主频那笔账要看 **[A14]** Bumblebee Core Datasheet。

你做到了

- 时基换了心脏也会开方：mtime 的驱动链、27MHz 账、读写纪律、续约语句一气呵成；
- S5 的两个魔咒（减一、读清）在 RISC-V 上的"结构性消失"讲得清楚，64 位的两条新纪律（撕裂、假中断）入了肌肉；
- RTOS 篇的 tick 地基打好：mtimecmp 续约那行代码，就是 F8 移植时的第一颗螺丝。

仪表是电表：mtime 与 mtimecmp 是两张内存表，64 位顺数，mtime 不小于 mtimecmp 即产生定时器中断；减一、读清两个魔咒结构性消失，换来的是读要"高-低-高"循环防撕裂、写 mtimecmp 要"先高后低"防假中断。1ms 还是 27000 tick，只是驱动链多一道主频除以四。

✅ 下一站：V4 GPIO 最小系统：时钟、时基都齐了，配上 GPIO 点亮第一盏 RISC-V 的灯。

通往单片机之路第 78 章

V4 GPIO 最小系统

难度 ★★☆☆☆ · 约 55 分钟

本章罗盘

问题

G2 的「七表全同」为什么到 VF103 失效？怎么判出 F1 血统并拼出第一个点灯闭环？

交付

一个可烧录的五步点灯工程（启动+链接+SystemInit+CTL 配置+mtime 延时）和三条上板验收记录：1 秒闪、SCSS=2、mtime≈27MHz。

边界

G2 的 F4xx 结论对本章不适用，掩码/移位/总线全要重查；LED 极性与引脚待仓库基准板指定后回填；速度档电气语义上板前按 UM 复核，VF103 无 TG 仅为头文件实证。

依据锚点 `a1` · `a13` · `f7` · `mcu-anim`

> 🎯 G2 带来过好消息：GD32F4xx 的 GPIO 与 STM32F4 七张表逐字相同，只换名字。到了 VF103，好消息一条不成立——它的 GPIO 是 F1 血统：两张配置表每脚 4 位、复用走 AFIO 重映射、时钟挂在 APB2 不在 AHB1。判据不用猜，就在 `gd32vf103.h` 的三行宏里。本章用 V0 的启动、V2 的 108M、V3 的 mtime 延时，拼出第一个完整的 RISC-V 点灯闭环，V 篇到此收官。

本章精髓

1. **血统鉴定看头文件**：`gd32vf103.h` 给了 `AFIO_BASE = APB2+0x0000`（:219）：AFIO（复用重映射机关）是 F1 时代的活化石，F4xx 早已用每脚 4 位的 AFSEL0/1 替掉它（G2 第五节）。**有 AFIO 就是 F1 家**：VF103 的 GPIO 因此是 CTL0/CTL1 两张配置表、每脚 4 位（MD+CTL 合体），不是 G2 那套七寄存器家族。
2. **数据四剑客换编制不换岗**：ISTAT/OCTL/BOP/BC 对应 S3 的 IDR/ODR/BSRR：BOP 低 16 位置位、高 16 位复位，写 0 无影响；BC 是独立清零寄存器（STM32F407 没有，得用 BSRR 高 16 位绕）；"读-改-写暗伤"与"原子写"的教训（S3 第二节）原样平移。
3. **五步闭环即收官**：开时钟（RCU_APB2EN.PAEN = bit2，注意在 APB2）→ CTL0 四位配置 → OCTL 设初值 → BOP/BC 原子翻转 → mtime delay_ms（V3）：V0~V3 的产出在此合体烧录，点灯即毕业。

先修

- V0 RISC-V 工具链与启动文件（启动文件、链接脚本、Makefile 标志）；
- V1 Bumblebee 内核与 CLIC 中断（AFIO/EXTI 的 F1 血统与 V 篇全局观）；
- V2 RCU 与 108MHz（SystemInit 默认拉到 108M 档）；
- V3 MTIME 与裸机延时（delay_ms 的 mtime 实现）。

先跑起来（10 分钟 quick win）

拿你板子的原理图（GD32VF103 板各家 LED 引脚不同，Sipeed、GigaDevice、自制板各归各；仓库基准板指定后本节接线回填），查出 LED 的端口、脚号与极性（共阳低亮/共阴高亮）。然后做两笔账：

```
① 核基址：GPIO_BASE = APB2 + 0x0800 = 0x40010000 + 0x0800 = 0x40010800（gd32vf103.h:201,221）
② 算配置：推挽输出 2MHz → MD=10（2MHz 档）、CTL=00（推挽）→ 四位合体 0b1000
   落位：CTL0 的 [pin*4+3 : pin*4] = 0b1000；掩码 0xF、移位 pin*4（S3 是 0x3、pin*2——别混）
```

十分钟后你写的每行代码都有账可查：本章 main.c 就是这两笔账的展开。

先押个家谱：VF103 的 GPIO，是 G2 那套七寄存器家族原班人马，还是另认祖宗换了编制？判据就藏在头文件的一个符号里，押完翻 gd32vf103.h 找它。

动画：RISC-V 点灯闭环

开时钟 → CTL0 四位配置 → OCTL 设初值 → BOP/BC 原子翻转 → mtime 延时，五步装配，V0~V4 的产出在此合体成第一个完整的 RISC-V 点灯闭环。

RISC-V 点灯闭环：五步装配

小节结构

小节	内容
一、血统鉴定	AFIO 判据；S3/G2/V4 三代 GPIO 对照表
二、四位一坑	CTL0/CTL1 的 MD+CTL 编码与 pin×4 落位
三、数据四剑客	ISTAT/OCTL/BOP/BC 对照 S3；BC 的直路价值
四、点灯五步	完整 main.c：时钟→CTL→OCTL→BOP/BC→mtime 延时
五、工程拼图	启动+链接+Makefile+main 组成可烧工程；验收三件事
六、收官对账	V0~V4 迁移清单总表：ARM 世界的五件替身

一、血统鉴定：有 AFIO 就是 F1 家

先看判据，再谈结论。`gd32vf103.h` 的外设地图（:201-227）里躺着三行：

```
#define AFIO_BASE   (APB2_BUS_BASE + 0x00000000U)   /* :219 复用重映射机关 */
#define EXTI_BASE   (APB2_BUS_BASE + 0x00000400U)   /* :220 */
#define GPIO_BASE   (APB2_BUS_BASE + 0x00000800U)   /* :221 → 0x40010800 */
```

AFIO 是"复用重映射"的 F1 制机关：F1 家族的复用功能只有固定几个引脚候选，靠 AFIO 的选择寄存器"选门"；F4xx 起改为每脚 4 位的 AFSEL 编号自由 mux（G2 第五节）。**AFIO_BASE 的存在本身就是血统证明**：F4xx 头文件里没有这个外设。V1 第三节用 EXTI/AFIO 基址验过"外设脱胎自 GD32F1"，GPIO 是同一血统的下一块拼图。

三代对照一次摆开：

维度	STM32F407（S3）	GD32F4xx（G2）	GD32VF103（本章）
配置寄存器	MODER/OTYPER/OSPEEDR/PUPDR 四张	同四张改名 CTL/OMODE/OSPD/PUD	**CTL0/CTL1 两张，每脚 4 位**
每脚位宽	2+1+2+2 分居四表	同左	**4 位一张表统管（MD+CTL）**
复用	AFRL/AFRH 每脚 4 位编号	AFSEL0/1 同款	**AFIO 选门 + CTL 选复用形态**
时钟总线	RCC_AHB1ENR bit0~8	RCU_AHB1EN bit0~8	**RCU_APB2EN bit2~6（PA~PE）**
数据寄存器	IDR/ODR/BSRR	ISTAT/OCTL/BOP/BC/TG	ISTAT/OCTL/BOP/BC
GPIO 基址	0x40020000（GPIOA）	0x40020000	**0x40010800——与 STM32F103 同址**

最后一行再补一记铁证：0x40010800 恰是 STM32F103 GPIOA 的老地址，端口步进 0x400（A~E 依次 +0x400，与 F103 完全同址；逐口基址已核验：GPIOA=GPIO_BASE+0，B~E 各 +0x400，gd32vf103_gpio.h:41-45）。V2 的 RCU 跟 F1 同门牌、本章 GPIO 也跟 F1 同门牌，"外设层像 F1、内核是 RISC-V"这个混血判断，处处有地址背书。

二、四位一坑：CTL0/CTL1 的 MD+CTL 编码

F1 血统把 S3 的四张表压成一张：每脚 4 位，低 2 位是 **MD**（速度/方向），高 2 位是 **CTL**（输入/输出形态）。CTL0（偏移 0x00）管脚 0~7，CTL1（偏移 0x04）管脚 8~15。四位语义已核验（gd32vf103_gpio.h:53-104：每脚 MD 占低 2 位、CTL 占高 2 位，逐位编码与 F1 惯例一致）：

MD	含义	CTL	输入时	输出/复用时
00	输入模式	00/01/10	模拟/浮空/上拉下拉	——
01	输出 10MHz	00/01/10/11	——	推挽/开漏/复用推挽/复用开漏
10	输出 2MHz	同上	——	同上
11	输出 50MHz	同上	——	同上

点灯用"推挽输出 2MHz"：MD=10、CTL=00 → 四位 `1000`。落位两步走，S3 的"先清后置"纪律原样有效，但**掩码和移位都换挡**：

```
/* S3（F407，每脚 2 位）                    本章（VF103，每脚 4 位） */
GPIOx_MODER &= ~(3UL << (pin * 2U));      GPIO_CTL0(x) &= ~(0xFUL << (pin * 4U));  /* 掩 0xF 不是 0x3 */
GPIOx_MODER |=  (1UL << (pin * 2U));      GPIO_CTL0(x) |=  (0x8UL << (pin * 4U));  /* 四位合体 0b1000 */
```

照搬 S3 的 0x3 掩码会留下两个旧位，是"模式改不干净"的暗病。配复用脚时尤其致命：残留位能把推挽变开漏，信号半死不活。

要上拉/下拉输入：MD=00、CTL=10，再写 OCTL 对应位。F1 血统的"输出锁存器"兼任上拉/下拉选择（写 1 上拉、写 0 下拉），这是与 S3 PUPDR 独立寄存器的又一个结构差异（已按用户手册实证：UM §7.5 配置表 Input pull-up=CTL10+OCTL1、pull-down=CTL10+OCTL0）。复用输出（如 USART_TX）：MD 选速度、CTL 选"复用推挽"，重映射候选脚多时还要动 AFIO：三件套见 V1 第七节 第 3、4 步。

三、数据四剑客：ISTAT/OCTL/BOP/BC

岗位	STM32F407（S3）	VF103（偏移已核验，gd32vf103_gpio.h:55-58）	备注
看电平	IDR（只读）	**ISTAT**（+0x08）	输入的耳朵
写电平	ODR（读写）	**OCTL**（+0x0C）	读-改-写暗伤同 S3 第二节
原子置位/复位	BSRR（低置高复）	**BOP**（+0x10）	低 16 置位、高 16 复位，写 0 无影响
独立清零	——（用 BSRR 高 16 位绕）	**BC**（+0x14）	F407 要 +16 挪半档，这里直路

四条纪律全部从 S3 平移，只有第四条是"减负"：

- **OCTL 的暗伤**：`OCTL 或上一个值` 是"读-改-写"三条指令，中断插进读与写之间就冲掉别人的改动，S3 第二节的翻车现场在 RISC-V 上一字不差。
- **BOP 只能纯赋值**：读回恒 0、写 0 无影响：对它做读-改-写等于把自己的位全清空（S3 坑 5 的 F1 版）。
- **BC 是 GD32 家的直路**：F407 想清一个脚得往 BSRR 高 16 位写（pin+16 半档）；VF103 直接把 1 左移 pin 位写 BC，不用挪半档（G2 第三节 在 F4xx 上已点名这件兵器）。
- **没有 TG**：G2 介绍的硬件翻转寄存器在 VF103 上没有（头文件实证：寄存器列表只有 CTL0/1、ISTAT、OCTL、BOP、BC、LOCK 六枚，无 TG，gd32vf103_gpio.h:53-59），翻转老老实实 BOP/BC 各写一次。

四、点灯五步：完整 main.c

五步闭环，每一行都标了出处（LED 接线以你板子的原理图为准；仓库基准板指定后本节回填）：

```
#include <stdint.h>

/* ---------- 板级接线：查原理图回填（基准板指定后更新） ---------- */
#define LED_PORT_BASE   0x40010800U     /* 示例 GPIOA；B~E 依次 +0x400 */
#define LED_PIN         1U              /* 示例 PA1 */

/* ---------- RCU（gd32vf103.h:226；gd32vf103_rcu.h:52,165） ---------- */
#define RCU_BASE        0x40021000U
#define RCU_APB2EN      (*(volatile uint32_t *)(RCU_BASE + 0x18U))
#define RCU_APB2EN_PAEN (1UL << 2)      /* PA 时钟位 bit2；PB~PE 是 bit3~6 */

/* ---------- GPIO（偏移已核验，gd32vf103_gpio.h:53-58） ---------- */
#define GPIO_CTL0(x)    (*(volatile uint32_t *)((x) + 0x00U))
#define GPIO_OCTL(x)    (*(volatile uint32_t *)((x) + 0x0CU))
#define GPIO_BOP(x)     (*(volatile uint32_t *)((x) + 0x10U))
#define GPIO_BC(x)      (*(volatile uint32_t *)((x) + 0x14U))

/* ---------- mtime 延时（V3 第四节原样搬运；基址按官方库实证更新） ---------- */
#define CLINT_BASE      0xD1000000U      /* GD32VF103 TIMER 域（n200_timer.h:29） */
#define MTIME_LO        (*(volatile uint32_t *)(CLINT_BASE + 0x0U))
#define MTIME_HI        (*(volatile uint32_t *)(CLINT_BASE + 0x4U))
extern uint32_t SystemCoreClock;        /* V2：SystemInit 后为 108000000 */

static uint64_t mtime_read(void)
{
    uint32_t hi, lo;
    do { hi = MTIME_HI; lo = MTIME_LO; } while (hi != MTIME_HI);
    return ((uint64_t)hi << 32) | lo;
}

static void delay_ms(uint32_t ms)
{
    uint64_t start = mtime_read();
    uint64_t ticks = (uint64_t)ms * (SystemCoreClock / 4U / 1000U);
    while ((mtime_read() - start) < ticks) { }
}

#define CTL_OUT_PP_2M   0x8UL           /* MD=10（2MHz）+ CTL=00（推挽） */

int main(void)
{
    /* 启动文件已 call SystemInit：时钟在 108M 档（V0 第六节） */

    RCU_APB2EN |= RCU_APB2EN_PAEN;                                  /* ① 时钟先行 */
    GPIO_CTL0(LED_PORT_BASE) &= ~(0xFUL << (LED_PIN * 4U));         /* ② 先清四位 */
    GPIO_CTL0(LED_PORT_BASE) |=  (CTL_OUT_PP_2M << (LED_PIN * 4U));
    GPIO_OCTL(LED_PORT_BASE) |=  (1UL << LED_PIN);                  /* ③ 初值：先置高（按极性定亮灭） */

    while (1) {
        GPIO_BOP(LED_PORT_BASE) = (1UL << LED_PIN);                 /* ④ 原子置位 */
        delay_ms(500);
        GPIO_BC(LED_PORT_BASE)  = (1UL << LED_PIN);                 /*    原子清零；⑤ mtime 延时 */
        delay_ms(500);
    }
}
```

五步对号：**① 开时钟 → ② CTL0 四位配置 → ③ OCTL 初值 → ④ BOP/BC 原子翻转 → ⑤ mtime delay**。三点说明：

- **时钟位换总线了**：S3/G2 的 GPIO 时钟在 AHB1ENR/RCU_AHB1EN；VF103 在 **RCU_APB2EN** bit2~6（PA~PE 逐一对应，gd32vf103_rcu.h:165-169），"外设像 F1"的又一处落地（F1 的 GPIO 时钟就在 APB2）。照搬 F407 的 AHB1 肌肉记忆，连寄存器都摸错。
- **④ 的极性由硬件定**：共阳灯低电平亮：初值给高（灭），BOP 置高是"灭"、BC 清零才是"亮"；共阴反过来。不看原理图抄极性，程序对灯不亮。
- **⑤ 的时间观来自 V3**：mtime 27MHz、1ms=27000 tick，delay 精度受整 tick 限制，500ms 误差在微秒级，肉眼与逻辑分析仪都无感。

五、工程拼图：四份产出合成一个能烧的工程

组成	出处	作用
`start_gd32vf103.S`	V0 第二/四节	设栈、搬 .data、清 .bss、call SystemInit、call main
`gcc_gd32vf103.ld`	V0 第五节	FLASH 0x08000000 / RAM 0x20000000、`__stack_top`
`system_gd32vf103.c`	官方库（V2）	SystemInit 把 RCU 拉到 108M 档
`main.c`	本章第四节	点灯五步 + V3 延时
Makefile 标志	V0 第一节	`-march=rv32imac -mabi=ilp32`（无 FPU 标志——RV32IMAC 没有 F）

烧录与调试沿用 V0 实验的 GDB 流程（调试后端按你的板卡定）。上电验收三件事：

1. **灯闪**：1 秒周期（500ms 亮 500ms 灭），闭环 OK；
2. **GDB 读 SCSS**：RCU_CFG0 bits[2:3] 应为 2（PLL，V2 实物实验），时钟真在 108M；
3. **mtime 差值**：连续读差反推约 27MHz（V3 实物实验），时基真的在跑。

三条全中，说明 V0~V4 没有一环是"抄来的"：每章的账都在你的板上对上了。

六、收官对账：ARM 世界的五件替身

V 篇五章，替你把 Cortex-M 的"标配"换成了 RISC-V 的对应物：

ARM 世界的件	RISC-V 替身	章
arm-none-eabi 工具链 + FPU 四标志	riscv-none-elf + march/mabi 两标志	V0
复位硬件两读、向量表第 0 项栈顶	reset_handler 显式设栈，无向量表	V0
NVIC：IRQn+16 槽号、PRIGROUP 分组、大向量表	ECLIC：索引直用、level+priority 两维、向量直跳	V1
RCC：PLL 大 VCO、电压三件套、提频六步	RCU：PREDV0×PLLMF 两步走、4MHz 家族、提频四步	V2
SysTick：24 位倒数、LOAD 自动重装、COUNTFLAG	mtime：64 位顺数、软件续约、比大小（主频/4）	V3
GPIO：MODER 七表、AHB1 时钟、AFSEL 编号	GPIO：CTL0/CTL1 四位表、APB2 时钟、AFIO 选门	V4（本章）

外设层与 F1 同源的"混血"红利贯穿全程：EXTI 配置、ISR 清挂起、GPIO 编码，凡是 F1 有的几乎照搬；要重学的只有内核三件：启动方式、中断控制器、时基。**这就是"学透一颗再举一反三"的实战版：先 STM32、再 GD32F4xx、最后 RISC-V，每一步都只学"差的那一点"。**

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **有 AFIO 就是 F1 家：两张表每脚四位，掩 0xF 移 pin×4；时钟在 APB2 bit2~6 不在 AHB1；数据四剑客 ISTAT/OCTL/BOP/BC，BC 清脚不用 +16；五步闭环：时钟、CTL、OCTL、BOP/BC、mtime delay。**

延伸：RISC-V 点灯闭环动画见 V4 动画；STM32 GPIO 对照见 S3；MTIME 延时在 V3 展开。

实物实验

- **基础**：跑通五步闭环，LED 1 秒周期翻转；逻辑分析仪量高电平段，应为 500ms 整（整 tick 误差内）。
- **换档联动**：V2 切 72M 档重编译，delay 用 SystemCoreClock 现算，灯仍应 500ms 翻转；若变慢，查 V3 坑 5（常数写死）。
- **输入侧复刻**：LED 脚改输入浮空/上拉（CTL0 改 MD=00、CTL=01/10），GDB 读 ISTAT，杜邦线拉地看 1 变 0，S3 第六节的实验在 RISC-V 上再走一遍。
- **七色扩展**：板上有 RGB 三灯的，三脚各配一段 CTL0/CTL1，主循环按位图轮流点亮，顺手练"多位配置不串位"。

常见坑

1. **忘开 RCU_APB2EN.PAEN**：症状与 S3 一模一样（写入无效、读回全 0），但解法换门牌：bit2~6 对应 PA~PE（gd32vf103_rcu.h:165-169），且寄存器是 **APB2EN** 不是 AHB1EN。第一大坑跨架构永恒，坑的坐标每颗芯都要重查。
2. **照搬 S3 的 0x3 掩码 / pin×2 移位**：VF103 每脚 4 位：掩 0xF、移 pin×4。掩浅了留旧位，配出的"半新半旧"模式是间歇性怪病的温床。

3. **OCTL 读-改-写翻转 + 对 BOP/BC 也读-改-写**：前者是 S3 第二节的暗伤（中断插队丢状态），后者更隐蔽：BOP 读回恒 0，读-改-写等于清空自己。原子寄存器只配纯赋值。
4. **极性与基址拍脑袋**：共阳/共阴不查原理图就抄亮灭逻辑；端口基址步进记错（B 口是 +0x400，不是 +0x100）。
5. **复用三件套缺一件**：F1 血统的复用 = CTL 选"复用推挽/开漏" + AFIO_EXTISSx 选门 + RCU_APB2EN.AFEN（bit0）开 AFIO 时钟，缺一件外设就哑（V1 第七节）。

短自测

1. 给出 VF103 GPIO 是 F1 血统的头文件级判据，并说出它带来的两处结构差异。

▼ **看答案**
`gd32vf103.h` 给了 AFIO_BASE = APB2+0x0000（:219）：AFIO 是 F1 制的复用重映射机关，F4xx 没有（F4xx 用每脚 4 位的 AFSEL0/1 替掉，见 G2 第五节）。有 AFIO 就是 F1 家。两处结构差异：① 配置从四张表（MODER 家族）变两张表 CTL0/CTL1、每脚 4 位（MD+CTL）；② 复用从"AFSEL 编号自由 mux"变"AFIO 选门 + CTL 选复用形态"。附带证据：GPIO 基址 0x40010800 与 STM32F103 的 GPIOA 同址。

2. PA1 配"推挽输出 2MHz"：CTL0 的目标 32 位值是多少？写出掩码与移位。

▼ **看答案**
MD=10（2MHz 输出）、CTL=00（推挽）→ 四位合体 0b1000 = 0x8。PA1 在 CTL0 占 bits[7:4]：先把该四段清零（掩码 0xF 左移 4 位后取反相与），再或上 0x8 左移 4 位，目标段值 0x00000080。注意每脚 4 位：掩 0xF、移 pin×4（S3 是 0x3、pin×2）。

3. "清零一个引脚"在 STM32F407 与 VF103 上各怎么写？BC 的价值是什么？

▼ **看答案**
F407 没有独立清零寄存器，用 BSRR 高 16 位复位区：把 1 左移 (pin+16) 位后写入。VF103 有 BC（GD32 家兵器，F4xx 同款见 G2 第三节）：把 1 左移 pin 位直接写 GPIO_BC，不用 +16 挪半档。两者都是纯赋值原子写、写 0 无影响。

4. 以 PA 口为例：点灯五步的第一步写哪个寄存器的哪一位？依据在哪？

▼ **看答案**
写 RCU_APB2EN（RCU+0x18）的 bit2，PAEN。依据 gd32vf103_rcu.h:165（RCU_APB2EN_PAEN = BIT(2)），bit3~6 依次对应 PB~PE。注意 GPIO 时钟在 APB2 不在 AHB1（F1 血统），别用 F407 的 AHB1ENR bit5 肌肉记忆。

5. 为什么说 V4 是"闭环"和"收官"？V0~V3 各贡献了哪个件？

▼ **看答案**
点灯需要"起得来、有时钟、有时间观、能动引脚"四件，正好对应 V0~V3 的产出：V0 给启动文件与链接脚本（程序起得来，SystemInit 被调起）；V2/SystemInit 给 108MHz 档（时钟）；V3 给 mtime 延时（时间观）；V4 本章给 GPIO 五步（动引脚）。合起来是第一个能从上电跑到现象的完整 RISC-V 工程，所以是闭环；V 篇"从工具链到点灯"的迁移故事在此讲完，所以是收官。

对照表：本章概念 → 仓库落点

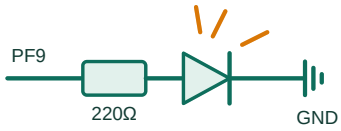
概念	落点
AFIO/EXTI/GPIO 基址（F1 血统判据）	gd32vf103.h:219-221
APB2_BUS_BASE = 0x40010000	gd32vf103.h:201
GPIO 时钟使能 PA~PE = APB2EN bit2~6	gd32vf103_rcu.h:165-169
AFEN = APB2EN bit0	gd32vf103_rcu.h:164
RCU 基址 0x40021000、APB2EN 偏移 0x18	gd32vf103.h:226；gd32vf103_rcu.h:52
CTL0/CTL1/ISTAT/OCTL/BOP/BC 偏移与四位编码	gd32vf103_gpio.h:53-104（已核验）
端口基址步进 0x400、逐口地址	gd32vf103_gpio.h:41-45（已核验，与 F103 同址）
读-改-写暗伤与 BSRR 纪律	S3 第二节
BOP/BC/TG 与 F4xx 对照	G2 第一/三节；VF103 无 TG 已实证（寄存器列表无 TG 项）
mtime 延时实现	V3 第四节
启动文件/链接脚本/Makefile 标志	V0
SystemInit 默认 108M 档	V2 第二节；system_gd32vf103.c:62

你做到了

- 第一个完整的 RISC-V 工程从你手里跑起来：启动、时钟、时基、GPIO 四环都验过账；
- "外设像 F1、内核是 RISC-V"的混血地图刻进脑子，以后看任何 F1 兼容芯，知道哪部分照搬、哪部分重学；
- V 篇毕业：从 arm-none-eabi 到 riscv-none-elf，你完成了一次架构迁移的全流程演练。

家谱是 F1：AFIO_BASE 还钉在 APB2 头上，有 AFIO 就是 F1 家；于是 CTL0/CTL1 两张配置表每脚 4 位，掩码 0xF、移位 pin*4*，*别拿 S3 的 0x3*、*pin*2 去套；数据四剑客 ISTAT/OCTL/BOP/BC 换编制不换岗，BC 独立清零是 STM32F407 没有的待遇。

✅ **V 篇收官**。下一站：实验中心把闭环上板实测；或直奔 RTOS 篇：看 SysTick 的 tick 到了 RISC-V 怎么落进 mtimecmp。



PART PL

第九部分 实物实验中心

逻辑分析仪、示波器、电流表下场：UART 抓帧、PWM 实测、优先级反转复现、I2C 解码、功耗账本，每个理论结论都在真实波形与读数上钉一次。

> 眼见为实需要仪器下场：逻辑分析仪抓一个 'A' 字节的低位先行、示波器对账 PSC/ARR/CCR 与每个沿、双 OS 复现优先级反转里被吃掉的 3500ms、I2C 抓包逐字节标注 ACK、电流表算出电池能活几天。每个实验都写明仪器、接线与预期现象，未回填的实测老老实实标「待上板实测」。读完的标志：看到「不工作」，你有一套可观察、可二分的排查动作。

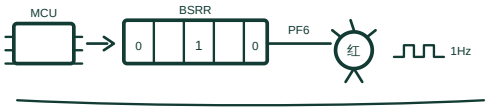
本部 8 章 · 约 445 分钟（7.4 小时）

79 . 实验 E01 点亮霸天虎的 RGB 红灯
80 . 实验 E02 逻辑分析仪抓 UART 帧
81 . 实验 E03 示波器看 PWM
82 . 实验 E04 优先级反转复现
83 . 实验 E05 I2C 抓包读 EEPROM
84 . 实验 E06 低功耗电流实测
85 . 实验 E07 S3 板载姿态传感器
86 . 实验 E08 S3 音频链路放音

通往单片机之路第 79 章

实验 E01 点亮霸天虎的 RGB 红灯

难度 ★☆☆☆☆ · 约 30 分钟



本章罗盘

问题

为什么拉低 PF6 红灯才亮，而不开 GPIOF 时钟连写 BSRR 都无效？

交付

一段能跑的寄存器版 blink 固件（构建→烧录→灯闪全链路走通）加一张秒表实测表：默认与改短延时各粗测 10 个周期，并能指认 RCC/MODER/BSRR 各自动了哪一位。

边界

红灯闪烁现象待上板实测回填，约 1Hz 只是软件延时推算（±30% 属正常）；计时精度不在承诺内，精确时序留给 SysTick 章；只走板载灯的寄存器直控路线，外接 LED 电路与库函数开发不涉及。

依据锚点 `a1 · mcu-code · mcu-lab`

> 🎯 嵌入式世界的仪式感：第一盏灯必须自己亲手点，而且要知道它为什么亮：不是"调了个库"，是"往 0x40021418 写了一个数"。

实验信息卡

时长	约 30 分钟
难度	★☆☆☆☆
配套代码	工程已提供 · PF6 红灯工程，见下方构建步骤。
上板验证	待上板实测；下文为预期现象，空白记录不表示验证通过

项	内容
编号	E01
对应章节	S0 环境搭建、S3 GPIO
目标板	野火 F407 霸天虎

实验目标

- 现象：霸天虎板载 RGB 灯的**红色**以约 1Hz 闪烁。
- 能力：说清"开时钟→配模式→控数据"三步各自动了哪个寄存器的哪一位。

装备

装备	数量	备注
霸天虎开发板	1	RGB 红灯=PF6，共阳低电平点亮
ST-Link（或 J-Link/DAP）	1	霸天虎不板载仿真器
Micro-USB 线	2	板子供电 + 仿真器
电脑（装好工具链）	1	见 S0

> 板卡外观与原理图见 野火霸天虎官方资料页。

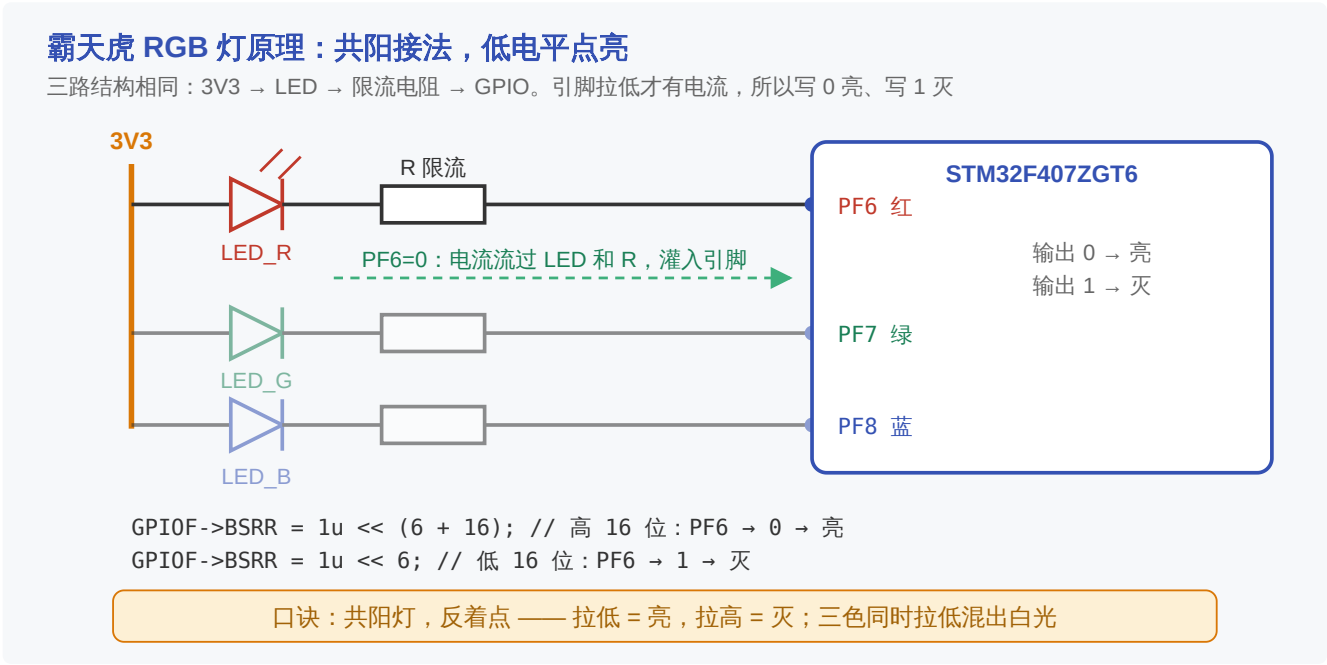
原理一句话

LED 阳极共接 3.3V，阴极经限流电阻到 PF6：**引脚拉低才有电流**，所以 `BSRR` 高 16 位写 1（拉低）= 亮，低 16 位写 1（拉高）= 灭。

改参数前先猜：延时砍到五分之一，灯会快几倍？先押一个数，答案在秒表上。

接线

本实验用板载灯，无需外接元件。原理图如下（绿 PF7、蓝 PF8 结构相同，供扩展）：



霸天虎 RGB 灯原理

SWD 接线与供电纪律见 S0 第五步。

步骤

1. **构建**：`cd code/stm32/00-blink && make` → 检查 `build/blink.elf` 生成。
2. **烧录**：`make flash` → 日志出现 `Verified OK`。
3. **观测**：复位后看红灯，用手机秒表粗测 10 次亮灭周期（约 10 秒）。
4. **改参数**：把 `main.c` 里两个 `delay(2000000)` 改成 `delay(400000)`，重新 `make flash`，观察闪烁明显变快。
5. **换颜色（选做）**：把 `LED_R_PIN` 从 `6` 改成 `7`，看绿灯闪；改成 `8` 看蓝灯。三个都点亮会混出什么颜色？

预期现象

- 默认固件：红灯约 1Hz 闪烁（软件延时，±30% 属正常，S5 SysTick 再追求精准）。
- 改短延时后：明显急促。
- 换引脚后：对应颜色闪烁。

实测记录

日期	板子	观测手段	结果	备注
	霸天虎	肉眼+秒表		
	霸天虎	肉眼（改延时）		

故障排查

症状	最可能原因	处置
`Verified OK` 但灯不亮	引脚号错（教程串板）	确认代码是 PF6 不是 PF9/PA8
灯常亮不闪	没进 while / 延时被优化掉	确认 `-O0` ；`delay` 循环体有 `__asm__ volatile ("nop")` 兜底——少这一句才真会被删，见 C3 六变体实测
OpenOCD 连接失败	驱动/接线/供电	回 S0 常见坑
`make` 报 missing separator	Makefile 缩进是空格	换成 Tab
改完代码现象不变	只 `make` 没 `make flash`	重新烧录

思考题

1. 为什么 `BSRR` 写高 16 位是"拉低"？设计这个寄存器的人为什么不让 ODR 直接原子操作？（提示：读-改-写在中断面前会发生什么）
2. 不开 `RCC_AHB1ENR` 的 GPIOF 时钟，直接写 MODER/BSRR，会发生什么？为什么？（答案在 S3）
3. `delay` 的参数为什么要声明 `volatile` ？去掉会怎样？（答案在 C3）

你做到了

- 亲手完成"源码→固件→板子→灯闪"全链路。
- 第一次用寄存器地址跟芯片对话成功。
- 拿到了后续所有实验的"通行证"（工具链 + 烧录流程）。

✅ 下一站：实验 E02 逻辑分析仪抓 UART 帧：把串口波形抓到屏幕上，一位一位对照手册。

通往单片机之路第 80 章

实验 E02 逻辑分析仪抓 UART 帧

难度 ★★☆☆☆ · 约 40 分钟

本章罗盘

问题

没有时钟线的 UART，分析仪凭什么把一串电平还原成字节？'A' 的 0x41 为何要倒着数？

交付

一屏 'A' 帧逐位标注的抓包记录（起始位→低位先行的 1000 0010→停止位），附采样率、触发与解码器参数，外加一笔用最窄脉宽反推波特率的手算。

边界

抓包与解码现象待上板实测回填；本章只教读波形与反推波特率，USART 外设配置细节在 S7；24MHz 分析仪的能力上限（更高波特率、边沿质量、电平标准）不可外推，16 倍过采样机理只作思考题。

依据锚点 `a1 · mcu-code · mcu-lab · mcu-anim`

> 🎯 教科书上的"起始位+8 数据位+停止位"听过一百遍，不如亲手把字母 'A' 的波形抓到屏幕上数一遍：0x41 = 0100 0001，LSB 先行，看一次记一辈子。（想要方格旗似的对称波形就发 0x55，两者共用同一副骨架，只是数据不同。）

实验信息卡

时长	约 40 分钟
难度	★★☆☆☆
配套代码	工程已提供 · UART/DMA 工程；抓包需另备逻辑分析仪。
上板验证	待上板实测；下文为预期现象，空白记录不表示验证通过

项	内容
编号	E02
对应章节	S7 USART
目标板	霸天虎（TTL 串口经 USB 转 TTL 或直接测 PA9）

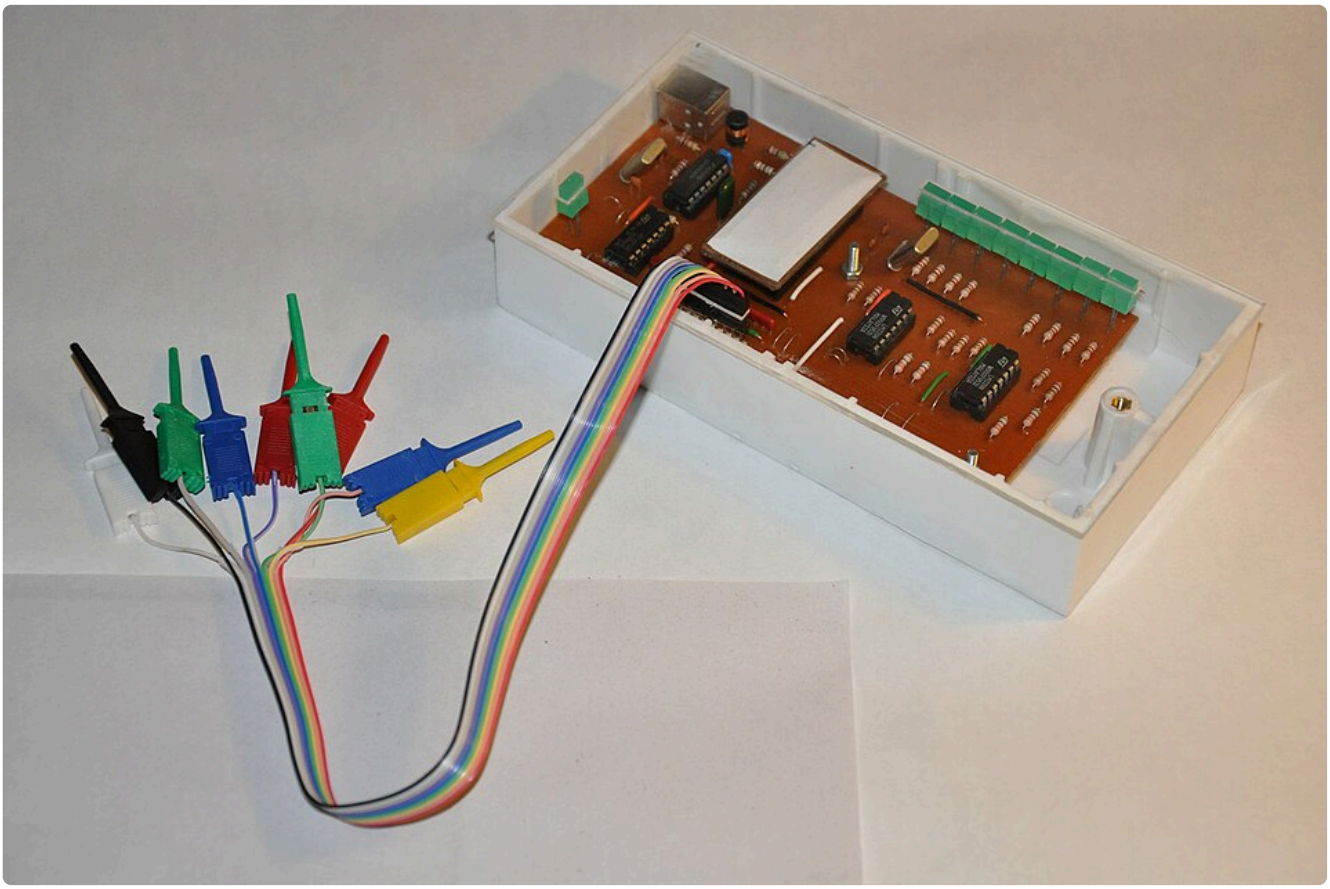
实验目标

- 现象：PulseView 解码出固件打印的字符串，逐位展开 'A'/'5' 的波形。
- 能力：会接线、会采样、会解码，再用波形反推波特率，这是时序分析的第一战。

装备

装备	数量	备注
24MHz 逻辑分析仪	1	saleae 兼容即可
霸天虎（烧好 S7 串口固件）	1	115200-8-N-1
杜邦线	3	CH0 接 TX、GND 共地

> 板卡外观与原理图见 野火霸天虎官方资料页。



自制 8 通道 USB 逻辑分析仪：开盖外壳、彩色排线与测试钩

逻辑分析仪形态参考（资料参考图，不是上板验证证据）：图为自制 8 通道款，¥30 的 24MHz saleae 兼容款是火柴盒大小的铝壳，接法相同。来源：Wikimedia Commons（CC BY-SA 2.0，作者 Dilshan Jayakody）。

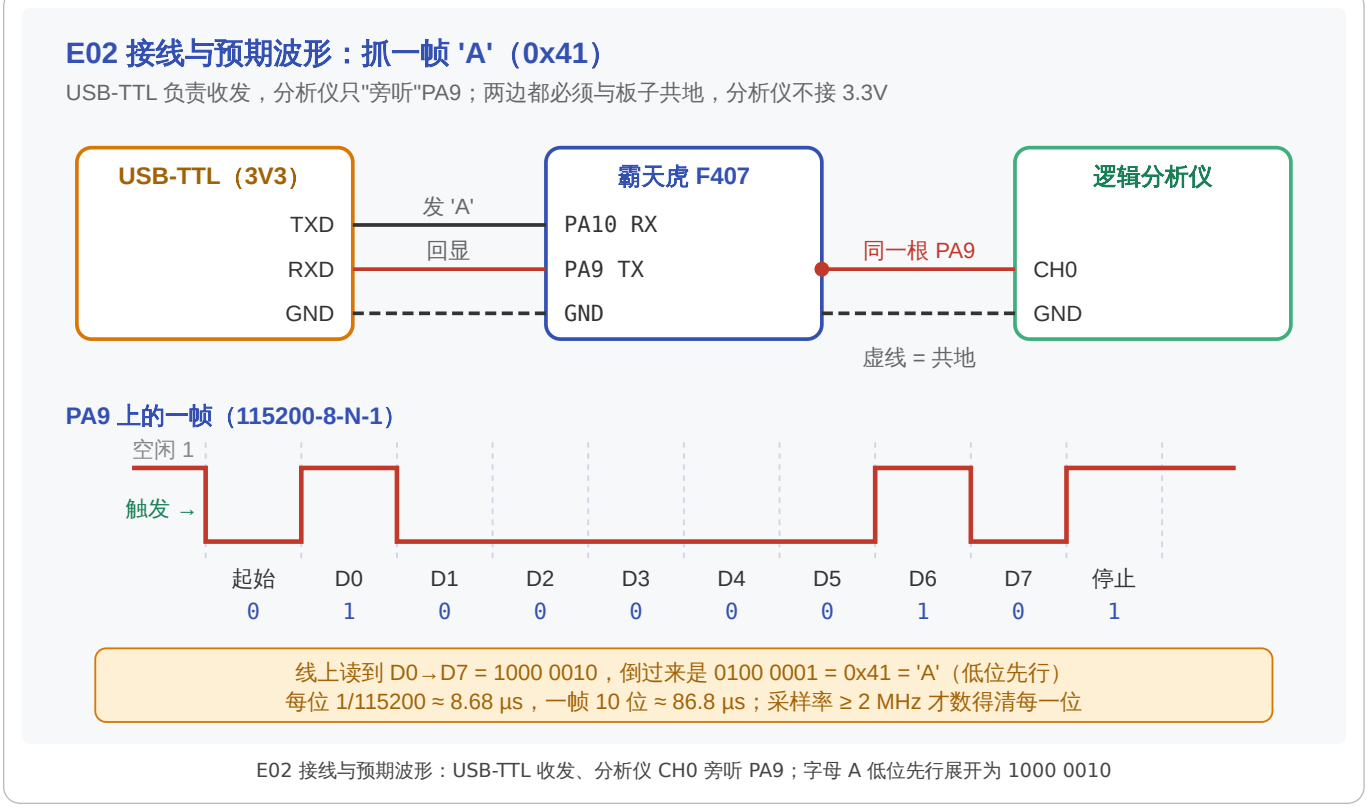
原理一句话

UART 是异步协议：没有时钟线，收发双方靠约定的波特率切时间片。分析仪按同样的规则切片，就能把波形还原成字节。

抓包前先估：'A' 这一帧铺在时间轴上有多宽？押一个微秒级的数，再用光标量。量过一次，波特率对你就是长度，不只是数字。

接线

- 分析仪 CH0 → 霸天虎 PA9（USART1_TX），GND ↔ GND；
- USB-TTL TX → PA10、RX → PA9、GND 共地，用于向固件发送待回显的字符；
- 纪律：共地必须有，只接两根线即可，别接 3.3V。



E02 接线与预期波形：USB-TTL 收发、分析仪 CH0 旁听 PA9；字母 A 低位先行展开为 1000 0010

一帧怎么逐位移出去，见 S7 USART 的帧动画。

步骤

1. 在仓库根执行 `cd code/stm32/03-uart-dma`、`make`、`make flash`（ST-Link）；安装 PulseView，选 24MHz 采样率，按 ≥16 倍波特率留余量时 115200 至少选 2MHz；
2. 触发方式设 CH0 下降沿（起始位就是下降沿）；
3. 开始采集；USB-TTL 串口助手设为 115200 8N1，发送单个 `A`，固件在 PA9 回显。复位时也会输出 `uart-dma ready`，但它不含大写 `A`；
4. 添加 UART 解码器：波特率 115200、8 数据位、无校验、1 停止位；
5. 找到字母 'A' 的帧，放大逐位数电平，对照 ASCII 0x41 = 0100 0001（注意线上是**低位先行**）。

预期现象

- 解码行显示与串口助手完全一致的字符串；
- 'A' 帧可辨：起始位低 → 1,0,0,0,0,0,1,0（LSB 先行的 0x41）→ 停止位高。

实测记录

日期	板子	观测手段	结果	备注
	霸天虎	PulseView		

故障排查

症状	最可能原因	处置
满屏噪声	没共地	补 GND
解码乱码	波特率设错	用"最窄脉宽"反推：1/脉宽≈波特率
抓不到数据	通道接错/触发错	确认接的是 TX 不是 RX

思考题

1. 为什么 UART 需要"过采样"（16 倍）？中点判决抗的是什么？
2. 把波特率改到 921600 再抓：对分析仪采样率的新要求是什么？

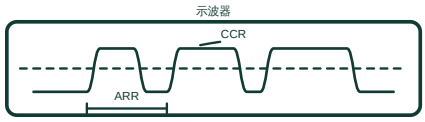
你做到了

- 时序分析技能开张；
- 从"背帧格式"到"读得懂波形"；I2C/SPI 抓包（E05）可以直接复用这套方法。

通往单片机之路第 81 章

实验 E03 示波器看 PWM

难度 ★★☆☆☆ · 约 45 分钟



本章罗盘

问题

示波器上那段方波，怎样从 PSC/ARR/CCR 的配置反推出每个沿的来历？

交付

一张定时器参数→理论波形→实测波形的对照记录（改 CCR 的占空比、改 ARR 的频率、fade 渐变各一组），附探头档位设置。

边界

板上波形以实测回填为准，文中预期值与误差范围未上板前只是推算；高频效应（振铃、串扰）超出本章仪器能力；RC 低通只做『穷人版 DAC』的定性验证，不进入滤波器设计。

依据锚点 `a1` · `mcu-code` · `mcu-lab` · `mcu-anim`

> 🎯 `CCR=500, ARR=999` 只是两个数字；示波器上那条"一半时间高电平"的方波，才是 PWM 的物理真身。数字与波形对上的那一刻，定时器就再也不是黑盒。

实验信息卡

时长	约 45 分钟
难度	★★☆☆☆
配套代码	工程已提供 · F407 PWM 与 S3 LEDC 两个工程均有独立说明。
上板验证	待上板实测；下文为预期现象，空白记录不表示验证通过

项	内容
编号	E03
对应章节	S6 TIM、P7 LEDC
目标板	霸天虎 或 立创 S3（任选一，双做更佳）

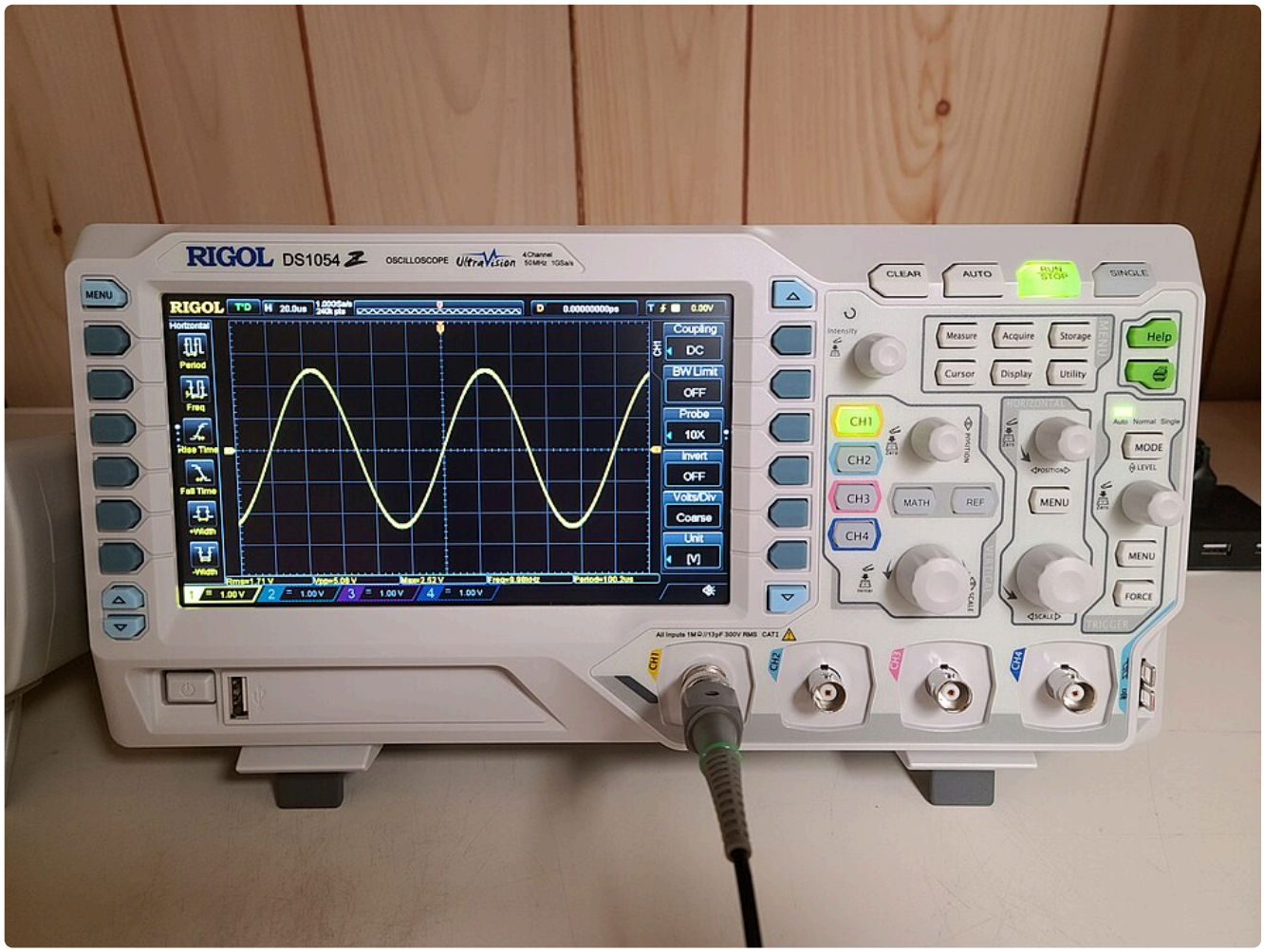
实验目标

- 现象：波形占空比随手算值变化；改 ARR 波形频率变化；呼吸灯的渐变曲线被记录。
- 能力：PWM 三参数（频率/占空比/分辨率）的测算对照；仪器读数与寄存器值互查。

装备

装备	数量	备注
示波器（或逻辑分析仪）	1	示波器可看模拟特性（RC 滤波）
霸天虎（S6 PWM 固件）或 S3（P7 LEDC 固件）	1	F407 1kHz 呼吸；S3 5kHz fade
杜邦线	2	通道+地

> 板卡外观与引脚分配见 野火霸天虎资料页 / 立创 S3 wiki。



RIGOL DS1054Z 四通道数字示波器，屏幕显示一条正弦波

示波器形态参考（资料参考图，不是上板验证证据）：RIGOL DS1054Z。仪器的 Probe 档位要与探头上的 ×1/×10 开关一致，否则幅值差 10 倍。来源：Wikimedia Commons（CC BY-SA 4.0，作者 Draconichiaro）。

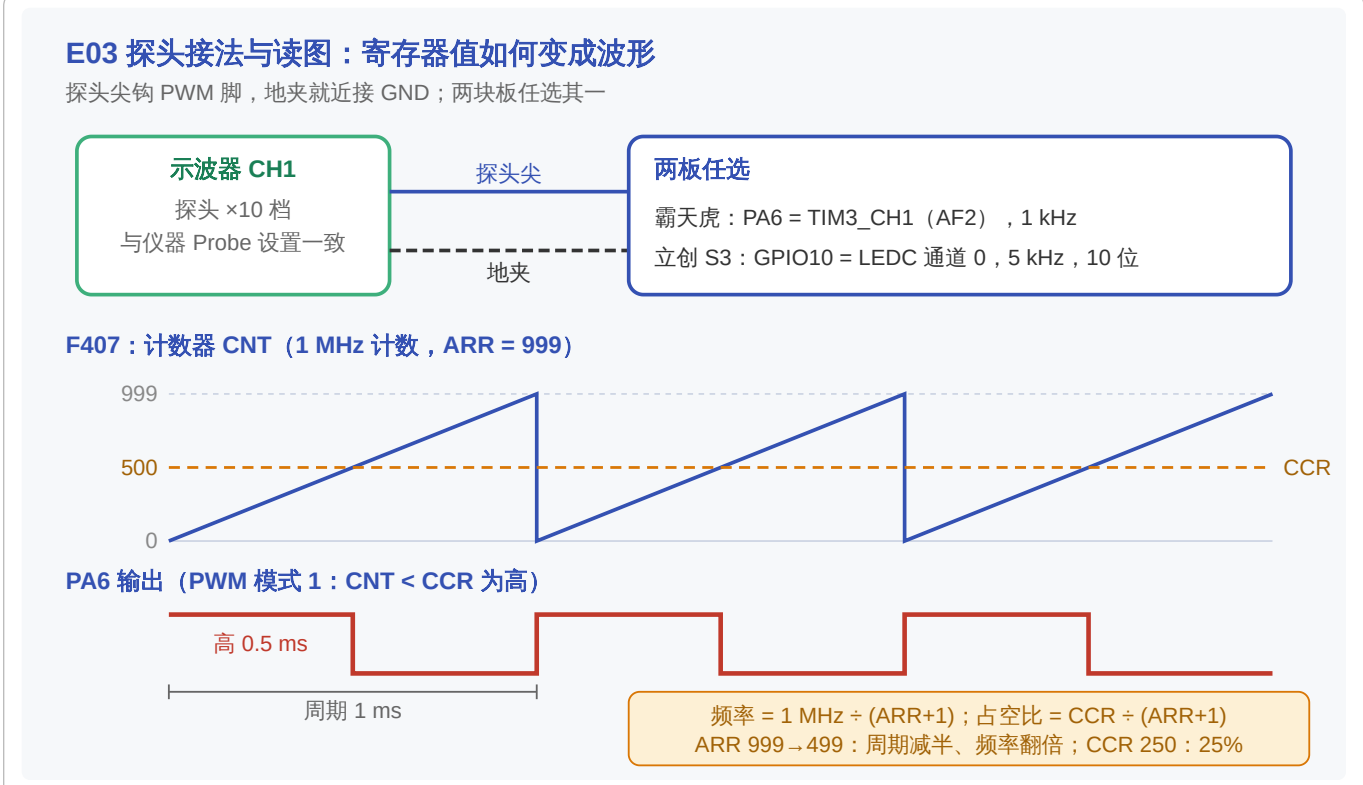
原理一句话

PWM=周期内高电平时间占比；占空比=CCR/(ARR+1)（STM32）或 duty/2^bits（LEDC）：波形的每个参数都能从寄存器值算出来。

上探头前先猜：手算的 25%，示波器会读出多少？误差是小数点后的事，还是会被仪器坑出几个百分点？押完再上探头。

接线

- 探头接 PWM 输出脚（霸天虎本批固件固定 **PA6**，TIM3_CH1 AF2；S3 本批固件固定 **GPIO10**，LEDC 低速通道 0，多功能扩展口），地夹共地。



E03 探头接法与读图：CNT 锯齿与 CCR 比较得到 PA6 方波，周期 1ms、高电平 0.5ms

计数器与比较值的动态过程见 S6 TIM；LEDC 的定时器/通道关系见 P7。

步骤

1. F407：进入 `code/stm32/02-tim-pwm`，`make` 后 `make flash`；默认 PA6 输出 1kHz，占空比随主循环变化。S3：进入 `code/esp32/03-ledc-fade`，执行 `idf.py set-target esp32s3`、`idf.py build`、`idf.py -p COMx flash monitor`；默认 GPIO10 输出 5kHz fade。先记录各自频率及占空比变化；
2. （F407 固定值练习）将主循环内 `ccr` 的计算式改成 `(PWM_PERIOD_TICKS + 1U) / 4U`，仍调用 `tim3_set_duty(ccr)`，重新构建烧录，对比 25% 占空比；
3. （F407）将 `PWM_PERIOD_TICKS` 从 999 改成 499，保持计数时钟不变，重新构建后频率应翻倍；
4. （S3）默认已启用硬件 fade，记录 0→1023/1024 占空比的渐变时间（目标约 2 秒），最大值不是精确 100%；
5. （进阶）输出经 RC 低通（10kΩ+100nF），示波器看"模拟电压≈VCC×占空比"——DAC 的穷人版现身。

预期现象

- 比较测量值与手算结果，记录时钟源与仪器误差；不预填未测得的误差范围；
- fade 曲线呈阶梯式渐变（硬件渐变的最小步进可辨）。

实测记录

日期	板子	观测手段	结果	备注

故障排查

症状	最可能原因	处置
无波形	CC 通道没使能/引脚复用错	回 S6/P7 配置清单
频率翻倍或减半	APB 倍频规则（S6）或速度组时钟（P7）	重算时钟链
占空比反了	输出极性位	检查 OC 极性配置

思考题

1. 为什么 LED 调光用 1kHz 看不出闪烁，而 50Hz 能？
2. RC 滤波后的纹波与 PWM 频率什么关系？怎么选 RC？

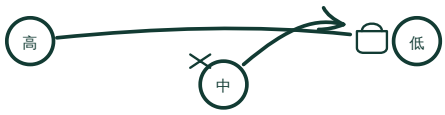
你做到了

- 定时器参数与物理波形建立换算直觉；
- "数字配置→模拟现象"的验证方法论再下一城。

通往单片机之路第 82 章

实验 E04 优先级反转复现

难度 ★★★☆☆ · 约 90 分钟



本章罗盘

问题

同一剧本下 H 的等待，信号量版比互斥量版多出的 3500ms 究竟被谁吃掉了？

交付

一张双 OS×两版本的四组对照表（FreeRTOS/RT-Thread × 信号量/互斥量），记录 H 等待时长并逐项与理论时间线对表。

边界

理论时间线是按 1ms 节拍剧本推算的非实测数据，结论以上板日志为准（节拍抖动会带来几 ms 偏差）；继承只缓解不根治，天花板协议与死锁仅思考题点到；临界区误写成 vTaskDelay 实验即失效，本页只警示不展开。

依据锚点 `d2` · `d11` · `f1` · `mcu-code` · `mcu-anim` · `mcu-lab`

> 🎯 这是本站最有"戏剧张力"的实验：三个任务演一出戏，高优先级被"饿死"，凶手却不是任何一个任务，而是**机制本身**。火星探路者就为它重启过。你将在自己的板子上导演这出戏，再亲手改结局。

实验信息卡

时长	约 90 分钟
难度	★★★☆☆
配套代码	工程已提供 · FreeRTOS scene 6（信号量版+互斥量版）已进 rtos-01-freertos-lab；RT-Thread 版已进 rtos-02-rt-thread-lab（R7 移植工程）。
上板验证	待上板实测；下文为预期现象，空白记录不表示验证通过

项	内容
编号	E04
对应章节	F5、R2
目标板	霸天虎（FreeRTOS 版 + RT-Thread 版各一遍）

实验目标

- 现象：二值信号量版里，高任务迟迟得不到执行；互斥量版里，高任务及时完成。两组日志对比。
- 能力：复现反转、解释继承、说出"反转/死锁/优先级天花板"的层级关系。

装备

装备	数量	备注
霸天虎（F8/R7 移植工程）	1	串口日志观测
逻辑分析仪（可选）	1	GPIO 打点看执行序更直观

> 板卡外观与原理图见 野火霸天虎官方资料页。

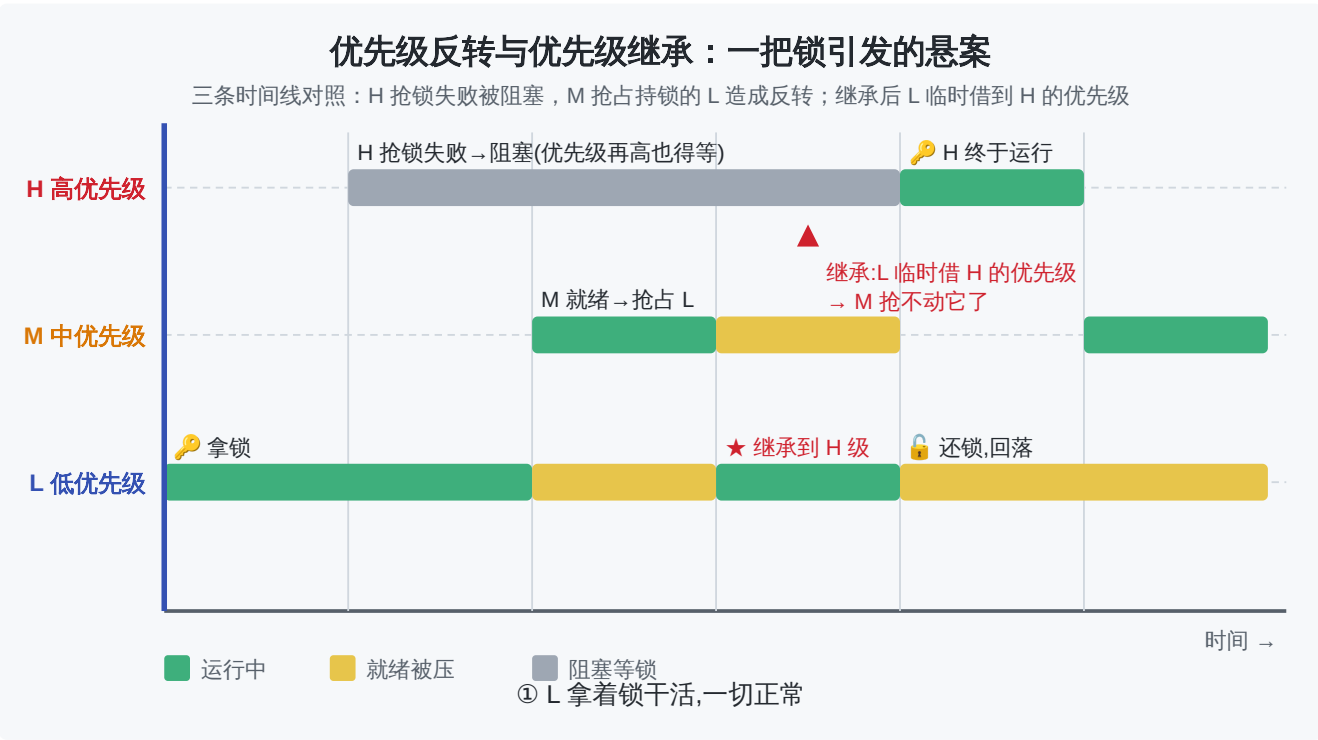
原理一句话

高任务等锁 → 锁在低任务手里 → 中任务抢占低任务 → 低任务永远还不了锁 → 高任务"被中任务间接阻塞"；互斥量的继承把低任务临时抬到高优先级，中任务插不进来。

烧录前先押一注：6000 与 2500 是理论账，你的板子会各量出多少？两个工程同剧本，谁离理论更近？串口日志判卷。

动画：优先级反转与继承

三任务的时间轴：L 拿锁进临界区→H 等锁→M 抢占 L（信号量版 H 被饿死）vs L 被继承抬级（互斥量版 M 插不进、H 及时完成）。两种结局并排对照，继承机制的"抬级"在画面上直观可见。



优先级反转与继承动画

剧本（三任务）

任务	优先级	行为
L（低）	1	拿锁→干 3 秒活（临界区）→还锁
M（中）	2	死循环干活（不碰锁），延时错峰启动
H（高）	3	延时 1 秒后尝试拿锁→打印"拿到锁"

步骤

1. **FreeRTOS·信号量版**： `make DEMO_SCENE=6` 烧录，串口看 H 的"拿到锁"比预期晚多久；
2. **FreeRTOS·互斥量版**： `make DEMO_SCENE=6 USE_MUTEX=1` 重跑，对比 H 的等待时间；
3. **RT-Thread 版**：同剧本在 RT-Thread 工程上复现（rt_sem vs rt_mutex），日志对比；
4. 打点版（可选）：三任务各翻一个 GPIO，分析仪上看执行序列，这就是反转的"心电图"。

配套工程

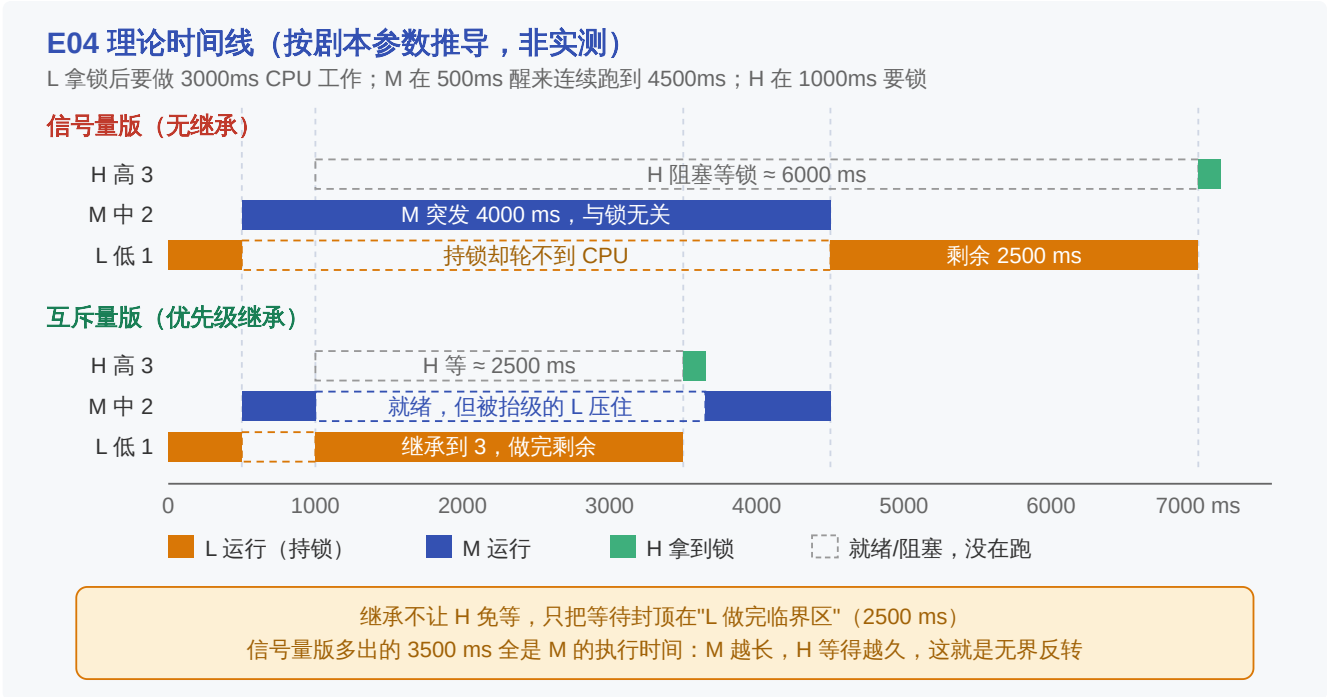
FreeRTOS scene 6 已进 `code/rtos/01-freertos-lab`（`make DEMO_SCENE=6` 信号量版 / `make DEMO_SCENE=6 USE_MUTEX=1` 互斥量版）。三任务优先级 1/2/3，时间戳日志直接打印 H 的等待时长。剧本是**单次**的：三个任务演完一轮就睡，复位重演，日志不会被下一轮搅乱。

RT-Thread 版已进 `code/rtos/02-rt-thread-lab`（R7 移植工程，同一命令矩阵）：`make DEMO_SCENE=6` 信号量版 / `make DEMO_SCENE=6 USE_MUTEX=1` 互斥量版，剧本与 FreeRTOS 版同构（L=22 低 / M=21 / H=20，RT-Thread 数字越小优先级越高）。

> ⚠️ **注意 · 反转只在"真占 CPU"时出现**
>
> 最容易写错的地方：把 L 的临界区写成 `vTaskDelay(3000)`。这时 L 是**睡着**的，M 抢不抢它，L 都在 3 秒后由定时器唤醒，两版 H 的等待一样长，实验什么也证明不了。所以工程里 L 用 `cpu_work_ms(3000)` 消耗"自己真正跑到的"3000ms（被抢占的时间不算），M 用 `busy_until(4500)` 不阻塞地连续占满 CPU。RT-Thread 版另有一个同类陷阱：`rt_thread_create` 之后不调 `rt_thread_startup`，线程停在 INIT 态永远不跑，编译链接却全过。

理论时间线

按剧本参数（1ms 节拍）推出的理论值，用来和串口日志对表；**不是实测数据**。



E04 剧本理论时间线：信号量版 H 等约 6000ms，互斥量版约 2500ms

时刻（ms）	信号量版	互斥量版
0	L 拿锁，开始 3000ms CPU 工作	同左
500	M 醒来抢占 L，L 已做 500ms	同左
1000	H 要锁被挡；L 仍是优先级 1，M 继续跑	H 要锁被挡；**L 继承到优先级 3**，抢回 CPU
3500	M 还在跑，L 一动不动	L 做完剩余 2500ms 并还锁，H 拿到锁（等 ≈2500）
4500	M 跑完，L 才继续剩余 2500ms	M 跑完
7000	L 还锁，H 拿到锁（等 ≈6000）	—

互斥量版的 2500ms 正是 L 临界区剩下的工作量：继承不能让 H 免等，只把等待**封顶**在"低任务把临界区做完"；信号量版多出来的 3500ms 全部是 M 的执行时间，与锁毫无关系，这就是"无界反转"。

预期现象

- 信号量版：日志中 `H got lock, waited ms` 理论 ≈6000，且排在 `M burst end` 之后；
- 互斥量版：理论 ≈2500，排在 `M burst end` 之前（M 被继承抬级的 L 压住，H 还锁后 M 才接着跑完）；
- 两 OS 行为同构（继承机制一致），日志并排即证据；实测值以上板记录为准，节拍抖动会带来几 ms 偏差。

实测记录

日期	版本	H 等待时间	M 是否插足	备注
	FreeRTOS 信号量			
	FreeRTOS 互斥量			
	RT-Thread 信号量			
	RT-Thread 互斥量			

故障排查

症状	最可能原因	处置
现象不明显	错峰延时没调好	M 要在 H 等锁时已在跑
日志乱序	printf 竞争	打印加互斥或缩短
直接死锁	L 忘还锁/递归 take	检查 give 配对

思考题

1. 优先级继承为什么只是"缓解"不是"根治"？（提示：L 的临界区时长仍是 H 的下限）
2. 优先级天花板协议（priority ceiling）与继承的差异是什么？查资料补一行。

延伸阅读

你在板上复现的，是 1997 年火星上发生过的事：

- **[D11]** Reeves 1997 — JPL 工程师的一手复盘；读完再看实验现象，会更有画面。
- **[D2]** Sha, Rajkumar, Lehoczky 1990 — 修复方案（优先级继承）的论文。

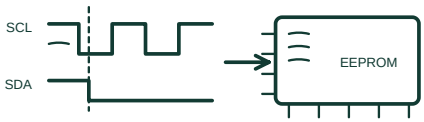
你做到了

- 亲手复现航天史上著名的 bug 并修复；
- 双 OS 同剧本对照，你的实验证据比教科书插图更有说服力。

通往单片机之路第 83 章

实验 E05 I2C 抓包读 EEPROM

难度 ★★★☆☆ · 约 60 分钟



本章罗盘

问题

一次『写后读』要在总线上发出哪些字节？重复 START 与末字节 NACK 为什么不能省？

交付

两屏解码记录（写事务与随机读事务各一，逐字节标注 START/ACK/重复 START/NACK/STOP）加上拉电阻对比，及串口 read=0x5A PASS 证据。

边界

抓包与读回现象待上板实测回填；只覆盖单主 100kHz 场景，时钟拉伸、多主仲裁与快速模式不展开，重复 START 的原子性动机只给提示；EEPROM 写入中途断电的损坏行为未验证，仅用 ACK polling 规避写周期。

依据锚点 `b1` · `a1` · `mcu-code` · `mcu-lab` · `mcu-anim`

> 🎯 I2C 学得扎不扎实，一块 AT24C EEPROM + 一台逻辑分析仪就能验出来：把"写 0x5A 到地址 0x10 再读回"的完整事务抓下来，每一拍对应手册的哪个状态、驱动代码的哪一行。全对上了，I2C 才算毕业。

实验信息卡

时长	约 60 分钟
难度	★★★☆☆
配套代码	工程已提供 · 完整 I2C EEPROM 工程；外接 AT24C02，非假定板载接线。
上板验证	待上板实测；下文为预期现象，空白记录不表示验证通过

项	内容
编号	E05
对应章节	S11 I2C
目标板	霸天虎

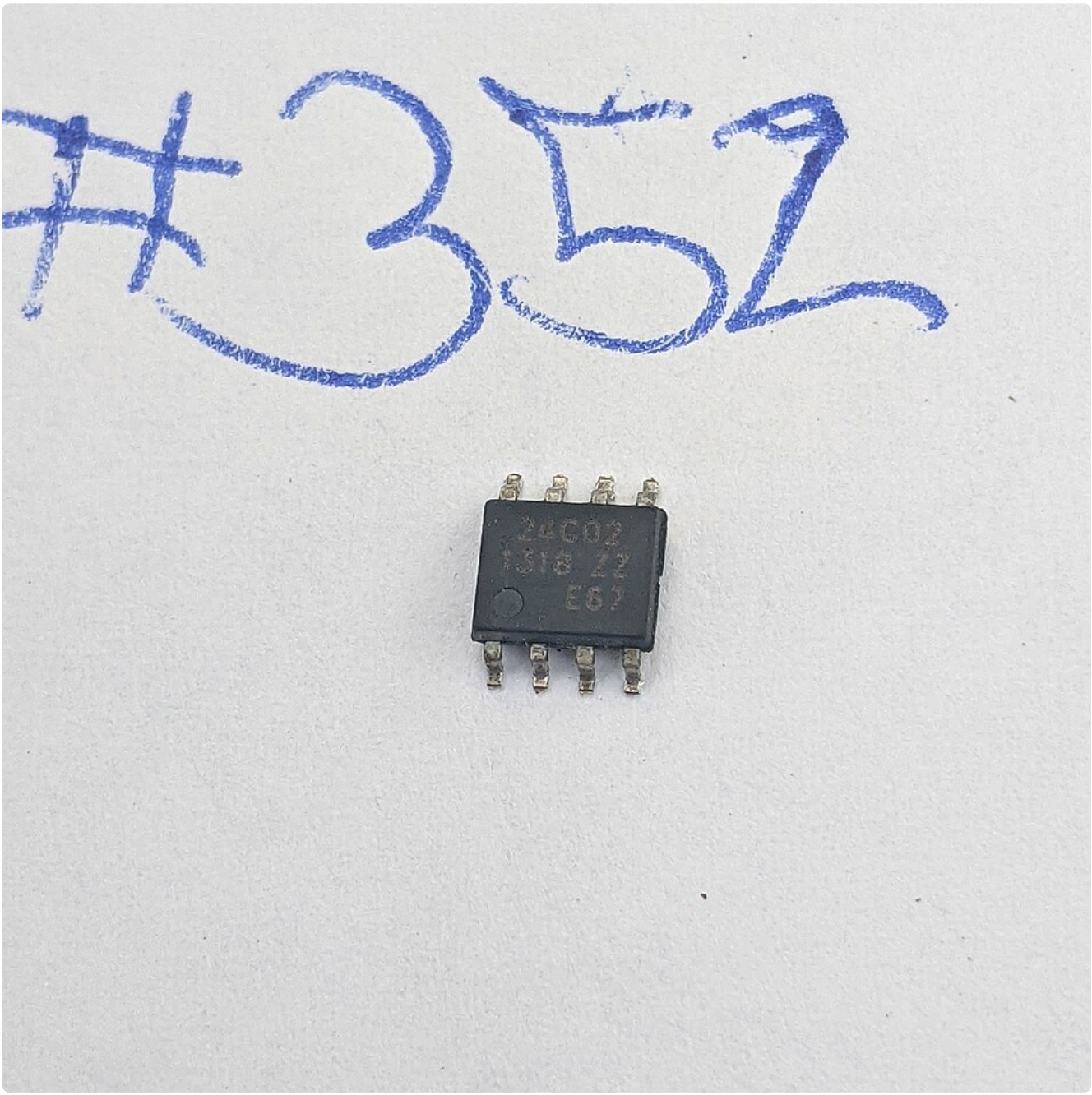
实验目标

- 现象：EEPROM 写入并读回 0x5A；分析仪解码出完整事务（START/地址/ACK/数据/重复 START/NACK/STOP）。
- 能力：I2C 波形解码、事件位与波形互查、上拉电阻对波形的影响评估。

装备

装备	数量	备注
AT24C02 模块	1	地址 0x50（A0-A2 接地）
逻辑分析仪	1	两通道：SDA/SCL
上拉电阻 4.7kΩ ×2	2	模块自带则免
霸天虎（本页 EEPROM 工程）	1	I2C1 PB6/PB7，外接模块按下文接线

> 板卡外观与原理图见 野火霸天虎官方资料页。



24C02 EEPROM 芯片，SOIC-8 贴片封装，表面丝印 24C02

被测芯片形态参考（资料参考图，不是上板验证证据）：SOIC-8 封装，圆点为 1 脚（A0），逆时针编号，8 脚 VCC、4 脚 GND。来源：Wikimedia Commons（CC BY-SA 2.0，作者 cole8888）。

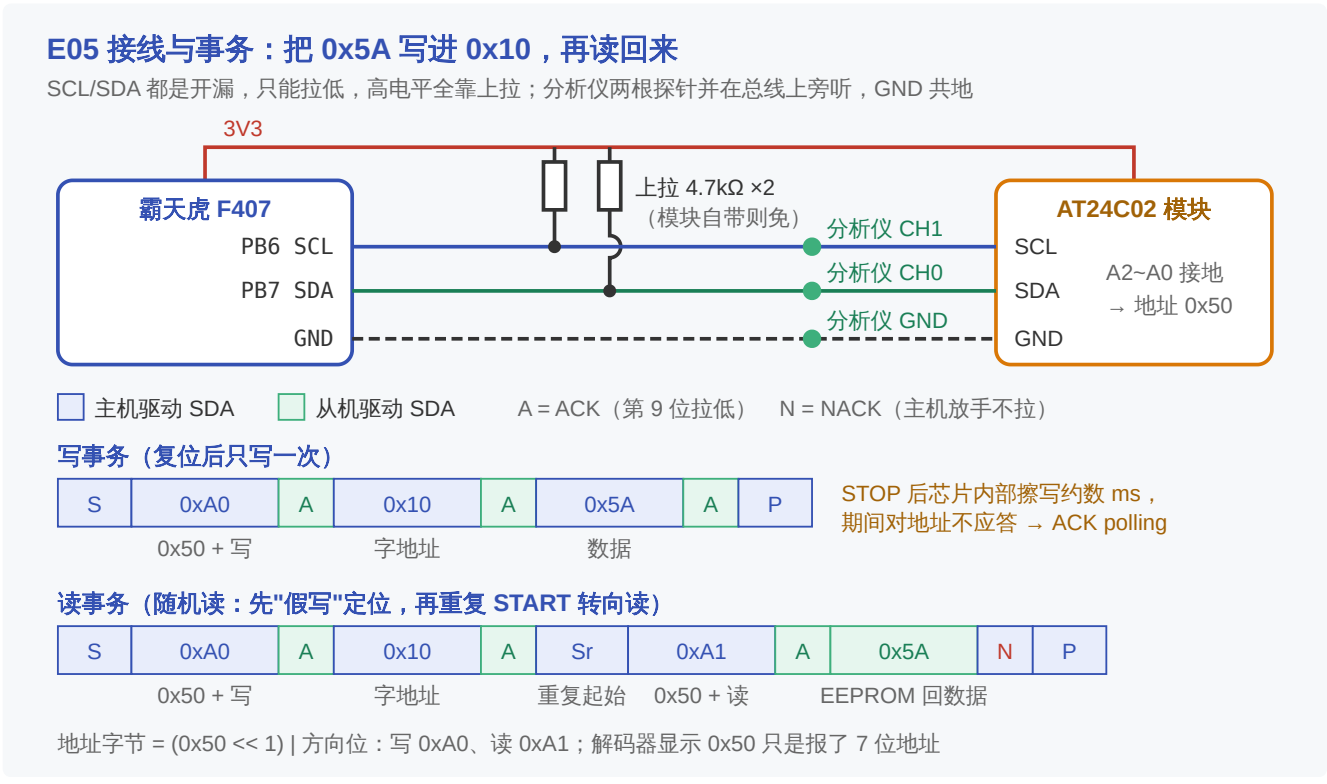
原理一句话

I2C 的每次交互都是"START + 地址帧 + ACK + 数据帧(+ACK) + STOP"的积木组合，寄存器级驱动就是按事件位依次搭建这些积木。

换挡前先猜：上拉从 10kΩ 换到 1kΩ，上升沿会陡一半，还是陡好几倍？押一个倍数，波形自己公布。

接线

- PB6→SCL、PB7→SDA、GND 共地、3.3V 供电；
- 分析仪 CH0→SDA、CH1→SCL；
- 若无上拉：SCL/SDA 各经 4.7kΩ 上拉到 3.3V。



E05 接线与事务：PB6/PB7 接 AT24C02，SCL/SDA 上拉，分析仪 CH0 接 SDA、CH1 接 SCL；下方为写事务与随机读事务的字节条

START/STOP/ACK 的逐拍电平变化见 S11 I2C 的时序动画。

步骤

1. 在仓库根运行 `cd code/stm32/04-i2c-eeprom`，`make` 后 `make flash`（ST-Link）；USB-TTL RX 接 PA9、GND 共地，以 115200 8N1 观察 `read=0x5A PASS`。程序每次复位仅写一次 0x10，并用 ACK polling 等写周期完成。
2. 分析仪 1MHz 采样抓取完整事务；
3. 添加 I2C 解码器，核对：START→0xA0（写地址+W）→ACK→0x10（字地址）→ACK→0x5A→ACK→STOP；
4. 再核对读事务：START→0xA0→ACK→0x10→ACK→**重复 START**→0xA1（读地址+R）→ACK→0x5A→**NACK**→STOP；
5. 换上拉电阻 10kΩ vs 1kΩ 各抓一屏：对比上升沿斜率。

预期现象

- 读回值 0x5A；解码与手册时序逐拍吻合；
- 1kΩ 上拉上升沿明显更陡（功耗换速度）。

实测记录

日期	板子	观测手段	结果	备注

故障排查

症状	最可能原因	处置
全 NACK	地址错/没上拉	0x50 左移=0xA0；查上拉
波形全是低	总线被拉死	查 SDA/SCL 是否短路、器件损坏
读到 0xFF	写周期没等够	EEPROM 写周期 5ms，用 ACK polling
解码错位	采样率不足	≥10 倍于 SCL（100k→1MHz）

思考题

1. 为什么读事务要"重复 START"而不是 STOP 后再 START？（提示：多主竞争与原子性）
2. NACK 在"主机读最后一字节"时的作用是什么？

你做到了

- I2C 从波形到代码全链路打通；
- 抓包对表法毕业；以后遇到任何新协议（1-Wire/自定义），都用这把刀。

附录：工程完整源码

构建、接线与排查见工程 README。

<<< ../../code/stm32/04-i2c-eeprom/main.c

通往单片机之路第 84 章

实验 E06 低功耗电流实测

难度 ★★★☆☆ · 约 60 分钟

本章罗盘

问题

手册里的 μA 数字怎么换算成电池能活几天？芯片睡眠电流为什么不等于整板电流？

交付

一张双板各四档的电流实测表（F407 全速/Sleep/Stop/Standby，S3 含 Wi-Fi 各档），加一笔 CR2032 220mAh 续航手算（按占空比分解出平均电流）与一个『偷电贼』拔除前后对比。

边界

本实验无独立配套工程（固件为 planned），需把 S14/P11 内联代码搬进自己工程；文中续航算例数值是演示假设，各档电流不预填 μA 结论；芯片手册电流≠整板输入电流，LDO 与板载负载须另计，深睡档结论受 μA 档仪器分辨率制约。

依据锚点 `a2 · a8 · mcu-lab · mcu-anim`

> 🎯 "低功耗"三个字在手册里是 μA 数字，在你表里是电池能活几天。本实验把两种板子的各档睡眠电流全部实测一遍，算出真实续航，从此"省电"是算出来的，不是吹出来的。

实验信息卡

时长	约 60 分钟
难度	★★★☆☆
配套代码	工程建设中 · 双板分档低功耗固件尚未提供独立工程； S14/P11 章节内联代码可直接进项目主组件。
上板验证	待上板实测；下文为预期现象，空白记录不表示验证通过

项	内容
编号	E06
对应章节	S14、P11
目标板	霸天虎 + 立创 S3（双板对比）

实验目标

- 现象：记录双板"全速/睡眠/深睡"各档电流；算出 CR2032 供电的理论续航。
- 能力：电流测量接线、断电影响因素排查、续航估算模型。

装备

装备	数量	备注
万用表（μA 档）或功耗仪	1	μA 分辨率必需
双板各一	2	烧好低功耗固件
电池盒/稳压电源	1	旁路板载 LDO 更佳

DT830D 数字万用表：旋钮档位含直流电压、直流电流 200µ/2m/20m/200m/10A、电阻

万用表形态参考（资料参考图，不是上板验证证据）：DT830 系列最低电流档 200µA，测深睡眠电流分辨率往往不够，要换带 µA 档的表或功耗仪。来源：Wikimedia Commons（CC0，作者 Ranjithkumar Murugesan）。

> 板卡外观与引脚分配见 野火霸天虎资料页 / 立创 S3 wiki。

原理一句话

续航(小时) ≈ 电池容量(mAh) ÷ 平均电流(mA)：平均电流由"工作时长占比×工作电流 + 睡眠占比×睡眠电流"决定；睡眠电流每降一个数量级，续航涨一个数量级。

上电前先押一记：拔掉仿真器，睡眠电流掉一成、一半，还是整整一个数量级？万用表说了算。

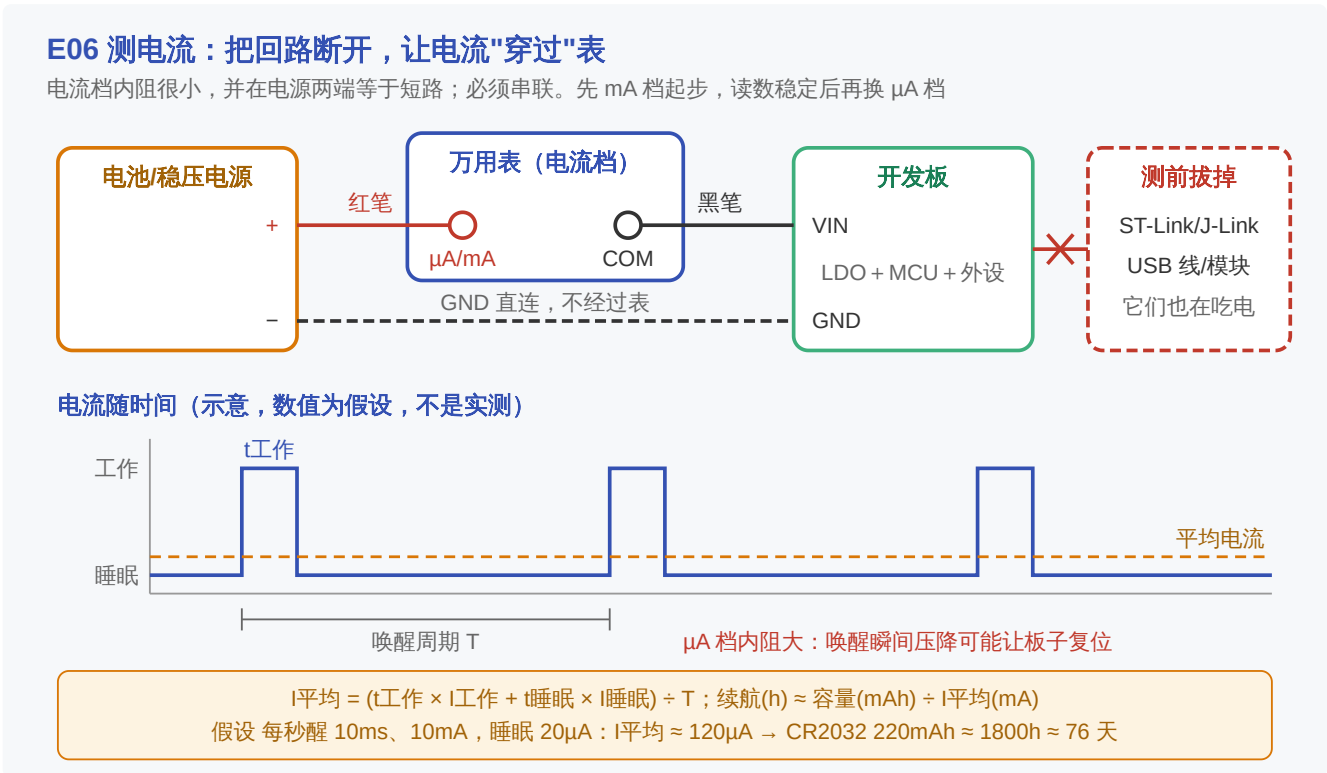
两板低功耗事实速查

维度	霸天虎（STM32F407）	立创 S3（ESP32-S3）
睡眠档位	Sleep/Stop/Standby（S14）	Light/Deep sleep（P11）
最深档电流	Standby：nA~μA 级	Deep sleep：μA 级
唤醒方式	EXTI/RTC 闹钟/WKUP 引脚	定时器/GPIO EXT0/EXT1/触摸/UART/ULP
唤醒后状态	Stop=继续；Standby=复位	Deep=复位重启
跨睡眠记忆	备份寄存器/备份 SRAM（4KB）	RTC 内存（RTC_DATA_ATTR）
代码出处	S14 内联代码（enter_stop/enter_standby）	P11 内联代码（esp_sleep_enable_timer_wakeup）

两板的低功耗代码已在 S14 与 P11 以内联代码呈现，可直接进项目主组件。本实验不另建独立工程，把对应章节的代码段搬进你的工程即可。

接线

- 万用表串在电源正极与板子供电输入之间（电流档，先 mA 档起步防过流，稳定后换 μA 档）；
- **拔掉仿真器与一切外接模块**（它们都在吃电）。



E06 测电流接法：万用表电流档串在电源正极与板子 VIN 之间，测前拔掉调试器与外接模块；下方为平均电流示意与续航算例

图中算例数值是假设，用来演示公式，不是实测。各睡眠档关掉了什么，见 S14 与 P11 的动画。

步骤

1. 霸天虎：全速运行→Sleep→Stop→Standby 四档各记录一次；
2. S3：Wi-Fi 活跃→Modem sleep→Light sleep→Deep sleep 四档各记录一次；
3. 记录每档的"唤醒周期"与"工作时长"，算平均电流；
4. 代入 CR2032（约 220mAh）估算两板的理论续航；
5. 进阶：找出并消除一个"偷电贼"（悬空 IO/调试接口/板载 LED）。

预期现象

- 比较各档电流；芯片手册的休眠电流不等于整块开发板输入电流，LDO、屏幕、codec 等负载必须计入；
- 实际电流和数量级差异待测量，不预填 μA 结论；
- 拔掉仿真器后睡眠电流明显下降（亲眼抓到一个偷电贼）。

实测记录

日期	板子	档位	电流	备注

故障排查

症状	最可能原因	处置
睡眠电流虚高	仿真器/外设未拔	全拔再测
μA 档无读数	表笔还在电压孔	换电流孔+串联
板子睡死唤不醒	唤醒源没配	回 S14/P11 检查唤醒配置

思考题

1. 为什么"工作时长占比"比"工作电流"更主导续航？（duty cycle 数学）
2. 如果只能用一节 CR2032 活一年，你的设计预算（平均电流上限）是多少 μA？

你做到了

- 低功耗从参数表变成实测数据；
- 续航估算模型在手；电池产品的第一张"预算表"，你会算了。

通往单片机之路第 85 章

实验 E07 S3 板载姿态传感器

难度 ★★☆☆☆ · 约 60 分钟

本章罗盘

问题

从 WHO_AM_I 应答 0x05 到串口打出 g/dps，地址、量程、字节序哪一环错了会读到 0xFF？

交付

一段能跑的 IDF 读数工程（WHO_AM_I 自检→配置→10Hz 打印），加三组动作实测记录：平放 Z≈1g、竖立重力换轴、摇晃合值偏离 1g。

边界

板上读数待实测回填；10Hz 串口打印不是无丢样采集（芯片内部 250Hz），采样率结论不可外推；陀螺漂移、融合与校准明确留作后话，本章止于原始量换算；换算链按 ±4g/±512dps 固定量程，改量程须重算 LSB。

依据锚点 `a7 · a9 · f2 · mcu-code`

> 🎯 立创 S3 板子里藏着一颗六轴姿态传感器（三轴加速度+三轴陀螺仪）——不用接一根线，I2C 一读，摇一摇板子，数据就在串口里跳舞。这是"传感器应用"最丝滑的第一次。

实验信息卡

时长	约 60 分钟
难度	★★☆☆☆
配套代码	工程已提供 · 独立 IDF I2C master 工程，直接读取六轴，不依赖 P6 成稿。
上板验证	待上板实测；下文为预期现象，空白记录不表示验证通过

项	内容
编号	E07
对应章节	P6 SPI/I2C 驱动
目标板	立创实战派 S3

实验目标

- 现象：串口以 10Hz 输出六轴数据；静止时 Z 轴≈1g，翻转板子各轴变化符合重力方向。
- 能力：I2C 器件读 WHO_AM_I→配置量程→连续读取→物理量换算的完整套路。

装备

装备	数量	备注
立创实战派 S3	1	QMI8658 板载
Type-C 数据线	1	烧录+监视

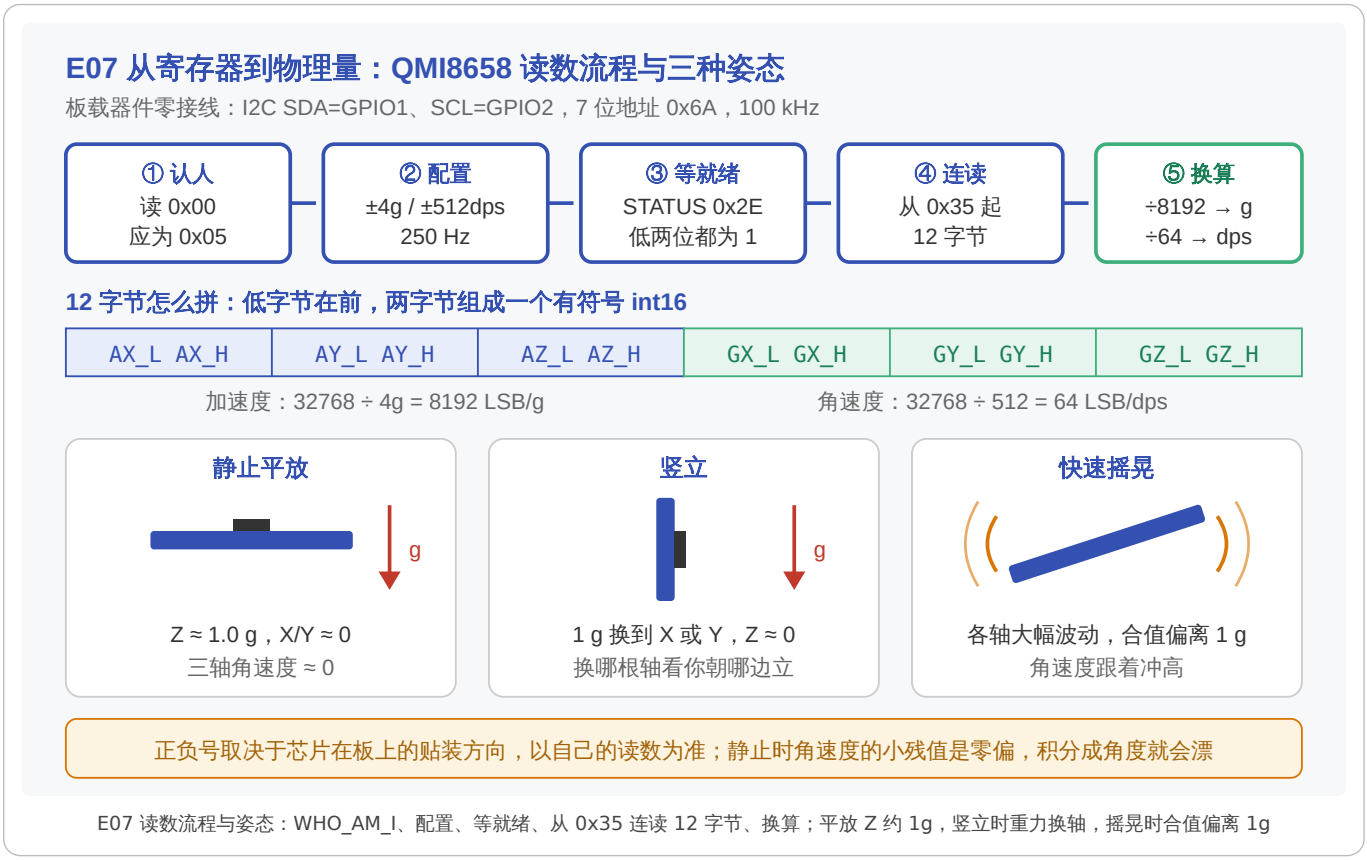
原理一句话

加速度计测的是"比力"：静止时重力加速度沿各轴的分量直接暴露板子的姿态；陀螺仪测角速度，积分得角度（漂移是它的天性，融合算法是后话）。

动手前先猜：加速度计静止读 1g 人人会背；陀螺仪呢，板子纹丝不动时三轴读到几？恰好全 0，还是带一点偏置？串口见分晓。

接线

板载器件，零接线；SDA=GPIO1、SCL=GPIO2、7 位地址=0x6A，见立创官方教程，工程顶部集中定义。



E07 读数流程与姿态：WHO_AM_I、配置、等就绪、从 0x35 连读 12 字节、换算；平放 Z 约 1g，竖立时重力换轴，摇晃时合值偏离 1g

步骤

1. 进入 `code/esp32/04-qmi8658`，在 IDF 5.5.2 终端执行 `idf.py set-target esp32s3`、`idf.py build`、`idf.py -p COMx flash monitor`；将 COMx 换为本机串口（Linux 如 /dev/ttyACM0）。
2. 读 WHO_AM_I（0x00）确认应答（链路自检第一步）；
3. 写配置：加速度 ±4g、陀螺仪 ±512dps、输出速率 250Hz，关闭自测试；每 100ms 读取最新数据，串口打印 10Hz，不是无丢样采集；
4. 循环读 12 字节原始数据（6 轴×int16），换算成 g 与 dps；
5. 10Hz 串口打印，做三组动作：静止平放→竖立→快速摇晃，观察数据特征。

预期现象

- WHO_AM_I 返回 0x05；
- 平放时 Z≈1.0g、X/Y≈0；竖立时重力"换轴"；摇晃时各轴大幅波动。

实测记录

日期	板子	观测手段	结果	备注

故障排查

症状	最可能原因	处置
WHO_AM_I 读 0xFF/0x00	I2C 地址错/引脚错	核对 wiki 原理图；地址是否左移
数据恒定不变	器件未使能（待机态）	查 CTRL 寄存器使能位
换算差 256 倍	原始数据字节序/量程搞错	按 datasheet 重算 LSB 权重

思考题

1. 为什么静止时加速度计"测到 1g"？它与自由落体时的读数矛盾吗？（等效原理）
2. 仅靠加速度计能否区分"匀速运动"和"静止"？这对航姿解算意味着什么？

你做到了

- 传感器应用全流程一次走完（ID→配置→读数→换算）；
- 立创 S3 的第一颗板载外设被你点亮，屏和音频（P6/P12）在排队。

附录：工程完整源码

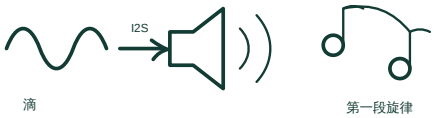
构建、接线与排查见工程 README。

<<< ../../code/esp32/04-qmi8658/main/main.c

通往单片机之路第 86 章

实验 E08 S3 音频链路放音

难度 ★★★☆☆ · 约 60 分钟



本章罗盘

问题

一声『滴』要过三道门：I2C 配 codec、I2S 送样本、扩展器开功放，哑火时先查哪道？

交付

一段能跑的放音工程（1kHz 滴声 1 秒 + 8 音符旋律，串口显示进度），加一次调小 DMA 缓冲引发断音再恢复的实测记录，MCLK/BCLK 可由 16kHz/16bit 复算。

边界

放音现象待实测，DMA 缓冲断音临界值未测出、本章不宣称下限；采样率音质取舍与全双工耳返只出现在思考题；功放控制路径（PCA9557 bit1）以立创 wiki 为据核对，尚非板上验证证据。

依据锚点 `a7 · a9 · f2 · mcu-code · mcu-anim`

🎯 这是立创 S3 的"加冕实验"：codec 配置、I2S 时序、DMA 缓冲，全部章节知识汇聚成一声清脆的"滴"——然后是第一段在你代码控制下播放的旋律。小智，从这一声开始。

实验信息卡

时长	约 60 分钟
难度	★★★☆☆
配套代码	工程已提供 · 独立 IDF 音频工程，含 codec 与扩展器功放控制。
上板验证	待上板实测；下文为预期现象，空白记录不表示验证通过

项	内容
编号	E08
对应章节	P12 音频链路
目标板	立创实战派 S3

实验目标

- 现象：喇叭播放 1kHz 正弦"滴"声（1 秒），随后播放一段 8 音符旋律；串口打印播放进度。
- 能力：codec 初始化、I2S TX 配置、PCM 数据组织、缓冲深度与断音的关系。

装备

装备	数量	备注
立创实战派 S3	1	喇叭板载（1W）
Type-C 数据线	1	
耳机/示波器（可选）	1	观察输出波形

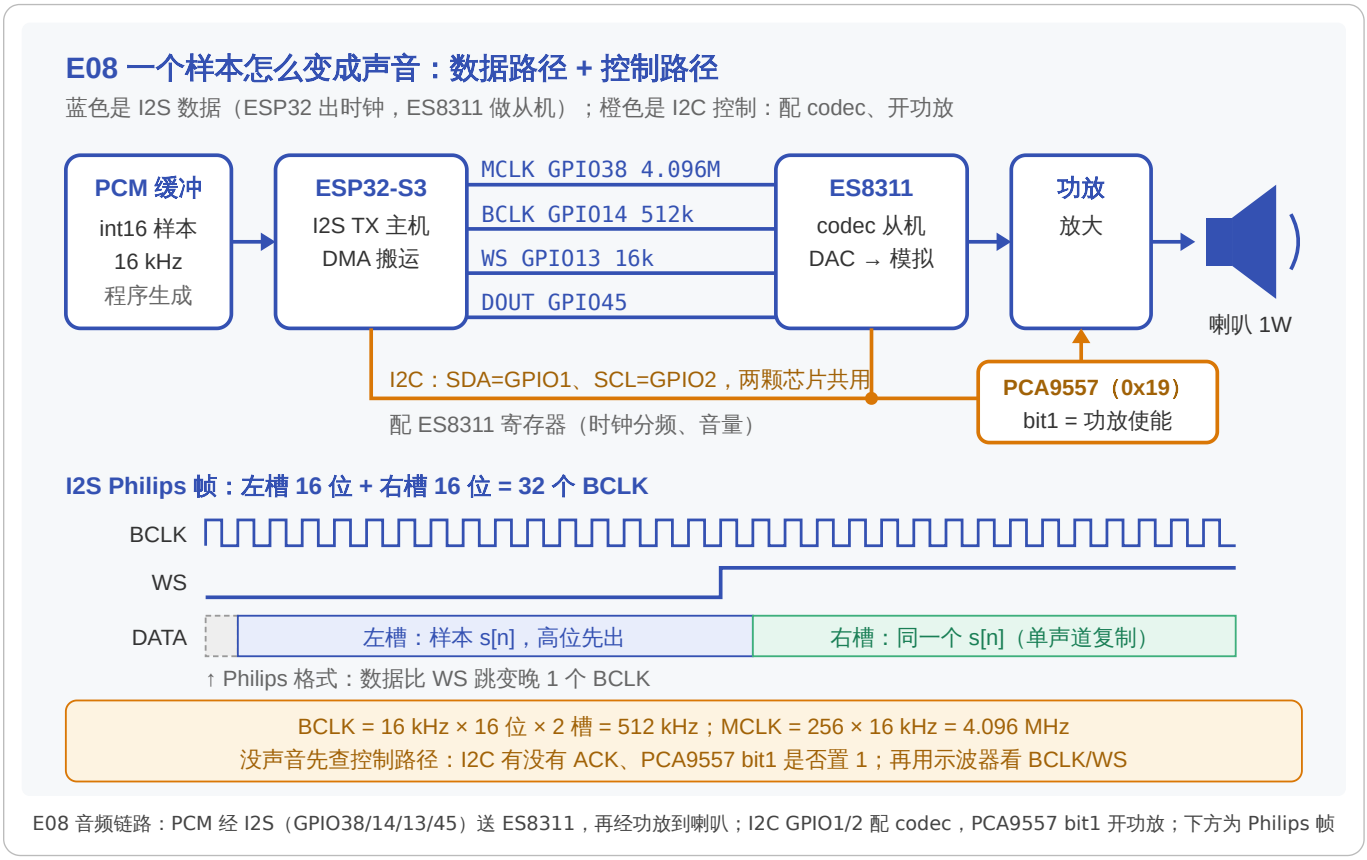
原理一句话

I2S 把内存里的 PCM 样本按"位时钟×声道帧"的节奏推给 codec，codec 转成模拟电压，功放放大后喇叭发声：**你写什么波形，它唱什么歌**。

开播前先押耳朵的判断：DMA 缓冲一次次减半，第几次开始断音？断音是周期性的还是随机的？答案在喇叭里。

接线

板载全集成，零接线；引脚与 PCA9557 功放控制见下面的已核对接线说明。



E08 音频链路：PCM 经 I2S（GPIO38/14/13/45）送 ES8311，再经功放到喇叭；I2C GPIO1/2 配 codec，PCA9557 bit1 开功放；下方为 Philips 帧

控制走 I2C、声音走 I2S 的动态过程见 P12 音频链路。

步骤

1. 进入 `code/esp32/05-audio-play`，在 IDF 5.5.2 终端运行 `idf.py set-target esp32s3`、`idf.py build`、`idf.py -p COMx flash monitor`，COMx 换为实际串口。首次构建自动下载固定版本 esp_codec_dev 1.3.4。
2. 工程用 GPIO1/2 的 I2C 配置 ES8311，PCA9557（0x19）的 bit1 控制功放；不是直接拉高 ESP32 的 GPIO1。
3. I2S 使用 Philips 格式：16kHz、16bit、两个槽位各放同一份单声道样本；MCLK=4.096MHz，BCLK=512kHz。MCLK/BCLK/WS/DOUT 分别为 GPIO38/14/13/45。
4. 程序生成 1kHz 正弦播放 1 秒，再播放 8 个音符；串口显示音符进度，结束自动关闭功放。首尾有淡入淡出，无需下载音乐文件。
5. 完成基本验证后再研究 DMA 缓冲与断音；本示例不宣称已测出缓冲临界值。

板卡控制路径依据：立创官方音频教程。

预期现象

- "滴"声清晰无杂音；旋律可辨；
- 缓冲过小时出现断音，恢复后正常（缓冲/延迟关系亲测）。

实测记录

日期	板子	观测手段	结果	备注

故障排查

症状	最可能原因	处置
完全无声	PA_EN 没拉/codec 初始化失败	查使能脚与 I2C 应答
声音变调	MCLK 与采样率不配	核对 256×fs
杂音爆裂	缓冲下溢/位宽错配	加大缓冲；核对 16bit 对齐
声音极小	codec 音量寄存器默认值	调音量寄存器

思考题

1. 16kHz 采样率对语音够用吗？电话音质 vs CD 音质的采样率差多少、为什么？
2. 如果要边采边放（耳返），单向缓冲与全双工缓冲的设计差在哪？

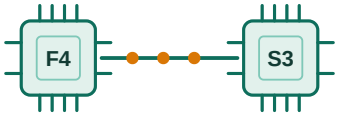
你做到了

- 音频链路全通，立创 S3 的"灵魂功能"归你指挥；
- 集 P 篇大成的一战：你离自己的语音终端只差应用层创意。

附录：工程完整源码

构建、接线与排查见工程 README。

<<< ../../code/esp32/05-audio-play/main/main.c



双板项目

PART PJ

第十部分 双板综合项目

把全部知识组装成两台可交付的传感器记录器：采样、滤波、队列、持久化、故障注入与恢复路径，全程可观察。

> 把全部知识组装成两台可交付的记录器：电位器/六轴 → 滤波 → 16 槽队列 → 主机 CSV，外加配置双槽轮转、看门狗健康监督与四种故障注入。J1/J2 共用公共核心层，port 层只随硬件变，这就是跨平台复用的实战答案。读完的标志：面对「做个小设备」，你先列故障清单，再写第一行代码。

本部 2 章 · 约 120 分钟（2.0 小时）

87 . J1 STM32F407 传感器记录器

88 . J2 ESP32-S3 传感器记录器

通往单片机之路第 87 章

J1 STM32F407 传感器记录器

难度 ★★★☆☆ · 约 60 分钟

本章罗盘

问题

传感器失联、队列积压、配置写一半断电，记录器如何把每种故障变成可观察的明确状态？

交付

一套跑通『电位器→滤波→16 槽队列→主机 CSV』全链路的 F407 工程与 console 脚本操作记录，附 fault 1–4 注入日志对照与一次 configure→save→重启加载新 generation 的验证。

边界

全页板上结果待实测：故障注入表标注的是预期逻辑结果，EEPROM 写入中断电与看门狗真实复位路径未上板验证；ADC 原码未测 VDDA 不得当校准电压；无流控 UART 不能确认对端真实接收；S3 平台差异与 NVS 结论归 J2。

依据锚点 `a1 · f1 · mcu-code · mcu-lab`

> 🎯 数据能打印一次不难；让它在传感器失联、队列积压和配置写入中断后仍有明确状态，才是一套完整记录器。

本章精髓

采集、滤波、队列、通信、配置保存与监督各有一个职责。两平台共享协议和纯 C 核心，驱动差异留在 port 适配层。健康判据是完成有界工作，传感器错误不自动等于任务卡死。

先修

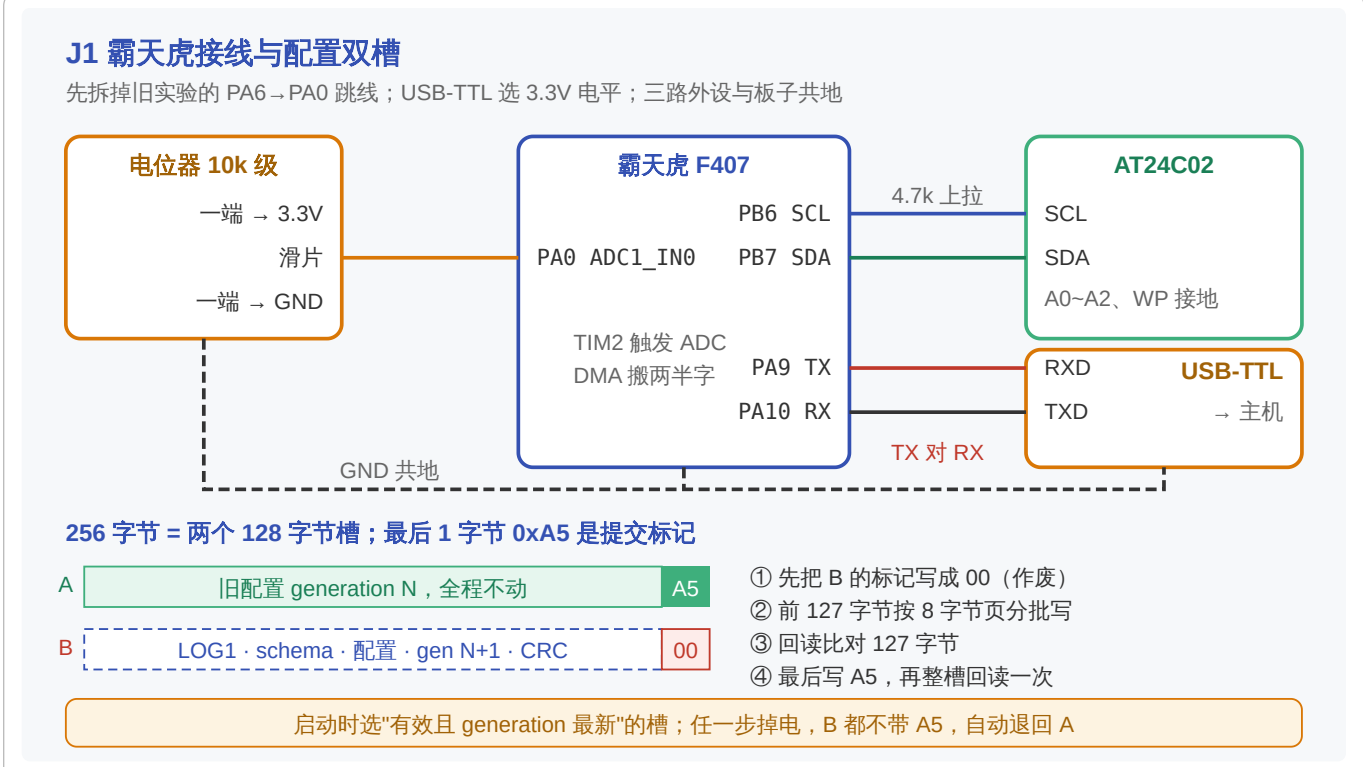
必需：C8 协议、S17 看门狗、P13 健康监督；板卡路线补 S9 ADC 或 P6 I2C。另一平台作为并行对照。

工程信息

时长	约 60 分钟
难度	★★★☆☆
配套代码	工程已提供 · 真实外设驱动与完整运行路径已提供；物理板卡结果待测。
上板验证	待上板实测；下文为预期现象，空白记录不表示验证通过

配置与接线

PA0 为 ADC1_IN0：外接电位器中间脚，另外两端接 3.3V 和 GND；先移除旧 PA6→PA0 测频跳线。PB6/PB7 接 AT24C02 SCL/SDA、各 4.7k 上拉至 3.3V，A0–A2 和 WP 接地。模块 256 字节全部作为配置双槽使用，会覆盖旧 EEPROM 实验数据。USART1 PA9 TX/PA10 RX 接 3.3V USB-TTL。



J1 接线与配置双槽：电位器滑片接 PA0，PB6/PB7 接 AT24C02 并上拉，PA9/PA10 接 USB-TTL；EEPROM 两个 128 字节槽的四步保存顺序

F407 基准为复位 HSI16+FreeRTOS V11.1.0；S3 为 IDF5.5.2。共地、TX/RX 交叉，COMx 换实际端口，绝不把 5V 信号直接接入 3.3V GPIO。

先跑起来

在已激活工具链环境执行：

```
cd code/stm32/07-sensor-logger
make MODE=0 FAULTS=0
make MODE=0 FAULTS=0 flash
```

随后回仓库根，在主机执行：

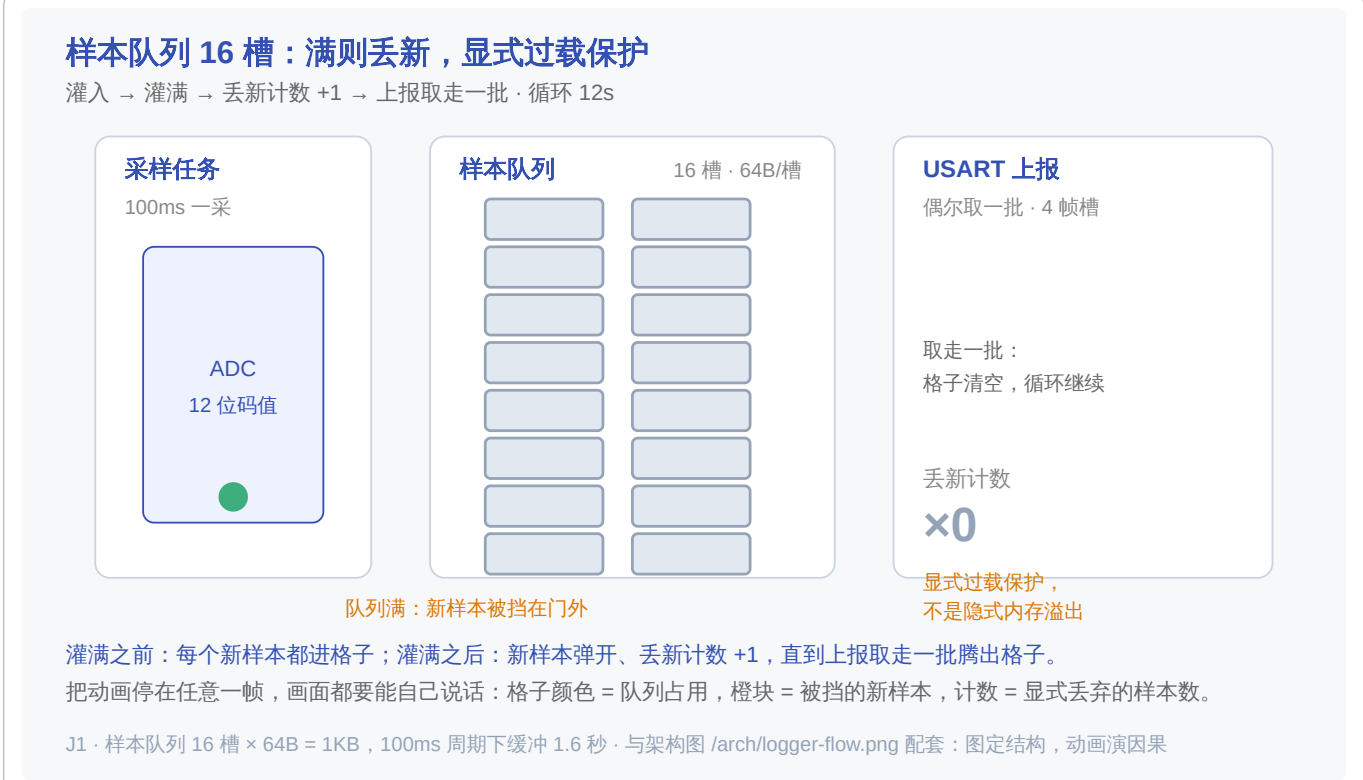
```
python3 -m pip install -r scripts/device-console-requirements.txt
python3 scripts/device-console.py info --port COMx
python3 scripts/device-console.py status --port COMx
python3 scripts/device-console.py configure --period 200 --filter-shift 2 --port COMx
python3 scripts/device-console.py save --port COMx
python3 scripts/device-console.py record --seconds 10 --output samples.csv --port COMx
python3 scripts/device-console.py stop --port COMx
python3 scripts/device-console.py start --port COMx
```

断电之前先押个记性：如果刚才只跑到 configure、还没 save 就断了电，重启之后记录器会用哪个周期干活？新改的、原来的，还是配置干脆丢掉？押完再往下读；不服，就真拔一次电，答案在重启后的 status 里。

record 写入新的 CSV 文件；请使用新的输出路径，避免覆盖之前的数据。默认周期 100ms；period 允许 100–1000ms 且为 10ms 倍数，filter-shift 允许 0–6，0 为直通。configure 只改变运行配置，save 成功才持久化；旧 generation 的在途采样会丢弃计数，防止误标新配置。

动画：样本队列的过载保护

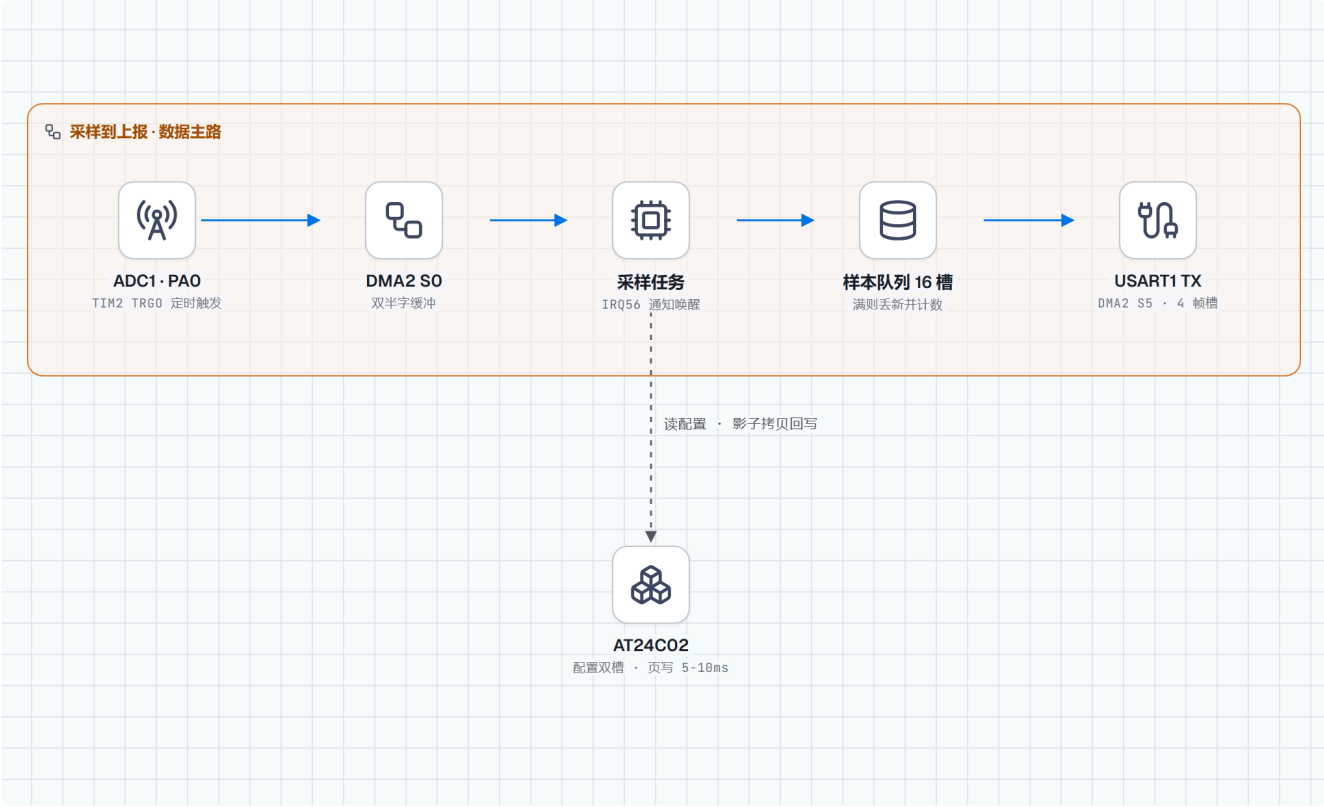
队列灌满之后发生什么，是本章最重要的一个设计决策：新样本被挡在门外、丢新计数 +1——显式过载保护，不是隐式内存溢出。



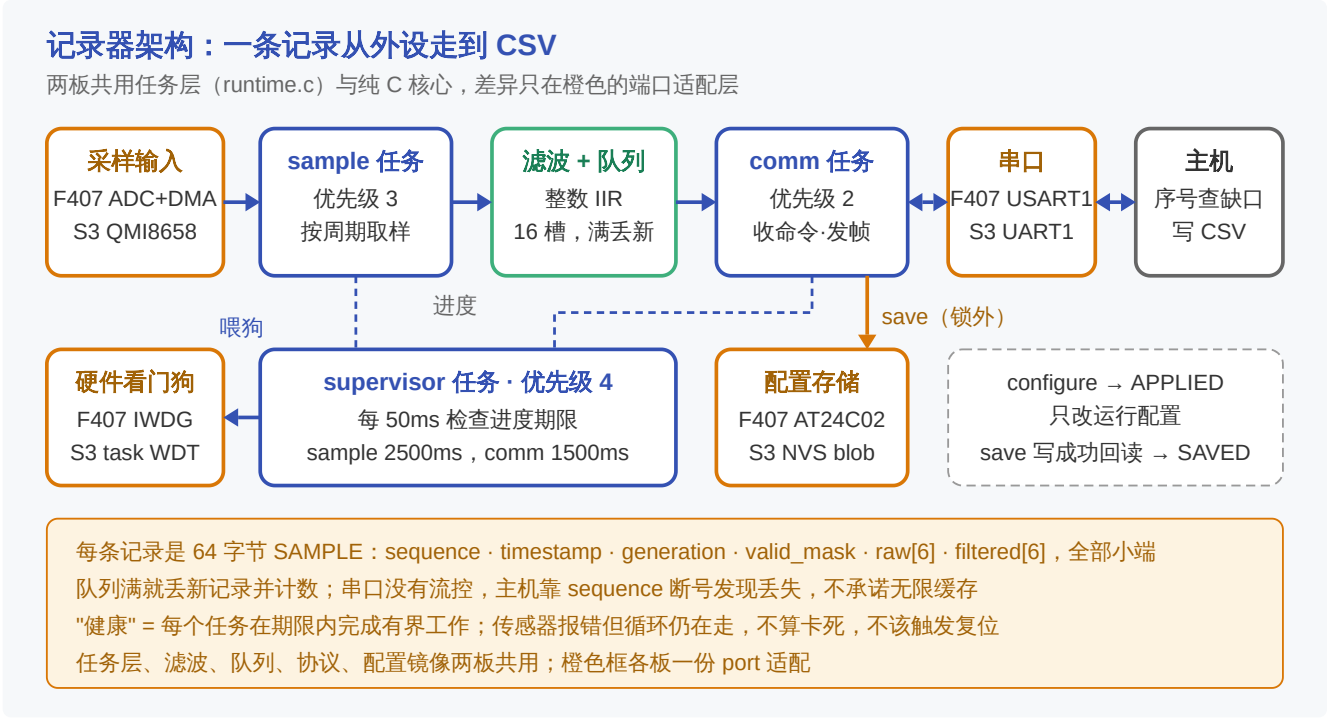
样本队列过载保护动画：灌入、灌满、丢新计数、取走一批

四层实现

五站数据流先收进一张图：采样主路之外，还有一条影子拷贝的配置通路。



数据记录器五站接力：ADC→DMA→采样任务→样本队列→USART，配置走 EEPROM 影子拷贝



记录器架构：采样输入经 sample 任务、滤波与 16 槽队列、comm 任务和串口到主机；supervisor 按期限检查进度并喂硬件看门狗；save 在锁外写配置存储

层	实际职责	验证
外设适配	ADC或I2C、串口、存储、硬件watchdog	平台编译与待上板日志
公共核心	滤波、配置镜像、16槽样本队列	生产C函数的宿主断言
RTOS运行层	采样/通信/监督任务，短时状态互斥	故障注入和期限预算
主机	独立编解码、序号匹配、CSV	C/Python交叉向量

系统设计决策

为什么选四层架构而非单体？ 嵌入式项目常见诱惑是"反正就这几个功能，全塞 main.c"。但当我们需要在 F407 和 S3 两个平台复用核心逻辑时，单体结构会让代码分叉难以维护。四层分离让公共核心（滤波、协议、队列）在两个平台共享同一份 C 代码，平台适配层只处理寄存器差异。代价是多了一层间接调用，但换来的是：①主机断言可以在 PC 上验证核心逻辑；②新增平台（如 GD32）只需写适配层；③代码审查时能清晰区分"平台 bug"和"逻辑 bug"。

为什么配置保存不持有采样锁？ EEPROM 页写需要 5-10ms，如果在此期间持有采样状态锁，采样任务会被阻塞至少一个周期（100ms 默认周期下就是 10% 的数据丢失）。设计原则是"保存用影子拷贝"：先把当前配置 memcpy 到临时缓冲区（<1μs），释放锁，再慢慢写 EEPROM。采样任务继续用原始配置运行，不受影响。锁的粒度应该覆盖"读取-修改"的原子性，而不覆盖"写入慢速外设"的等待。这个决策的代价是需要额外的 128 字节栈空间存放影子拷贝，但换来的是采样连续性。

为什么样本队列只有 16 槽？ 理论上队列越大越不容易丢数据，但每个槽是 64 字节，16 槽就是 1KB RAM。F407 有 192KB RAM 看似充裕，但 FreeRTOS 任务栈、DMA 缓冲、协议栈都在抢这块空间。16 槽在 100ms 采样周期下能缓冲 1.6 秒数据，足够覆盖通信任务短暂阻塞（如 USB 转串口芯片的流控）。如果通信持续失败，队列满后丢新样本并计数，这是显式的"过载保护"，而非隐式的内存溢出。调大队列前应先用 STATUS 命令查高水位，确认是否真的需要。

为什么健康监督用期限而非心跳？ 心跳模式（任务定期报告"我还活着"）的问题是：任务卡死在某个阻塞调用上时，仍可能在超时前最后一次报告心跳。期限模式要求任务在固定窗口内完成固定工作（采样任务 2500ms 内至少完成 20 次采样），未完成即视为卡死。这能检测到"任务还活着但进度停滞"的隐性故障。代价是需要为每个任务定义"合理工作量"，但这迫使开发者思考任务的时间约束，而非假设"只要不崩就行"。

定时采样与滤波

TIM2 TRGO 以毫秒分频定时触发 ADC1，DMA2 Stream0 Channel0 使用两个半字缓冲，IRQ56 发通知唤醒采样任务。IRQ 优先级 6，满足本项目 MAX_SYSCALL=5 的限制；USART1 使用 DMA2 Stream5，不共用流。原始数据是 12 位 ADC 码值，不能未测 VDDA 就声称是校准电压。

默认整数 IIR 每次走向新值的 1/4，使用 64 位中间差值避免溢出；整数舍入会产生小死区，滤波延迟不是零。有效位清除的通道不更新滤波，恢复后第一样本重新初始化。STOP 停止采样服务，但通信和监督仍执行，不应该触发看门狗。

协议与缓冲

SAMPLE 固定 64 字节：sequence/u32、timestamp_ms/u32、generation/u32、sensor_type/u8、valid_mask/u8、status/u16、raw[6]/i32、filtered[6]/i32，全部 LE。ADC 仅 bit0 有效；IMU 最多六位有效。INFO 给原码比例，STATUS 的 TLV 给运行配置、保存代次、丢弃/传感器/存储错误和队列高水位。

样本队列 16 槽，满则丢新记录并计数。TX 另有 4 帧槽和 100ms 发送期限，部分帧失败后先发零同步。无硬件流控的 UART 不能确认对端真正接收，也不能可靠检测接收线断开；主机用序号发现缺口，不承诺无限离线缓存。

配置保存与恢复

AT24C02 两个 128 字节槽。写目标槽前使提交标记无效；按 8 字节页边界写前 127 字节，回读校验后最后写提交标记。旧槽在整个过程中保留，启动挑选最新有效 generation。每个写步骤的中断都用生产存储逻辑测试；硬件断电/时序仍待实测。

配置代次用有界模序号比较；不同 schema 不能直接按新结构解释。保存发生在状态锁之外，采样优先级高于通信，避免慢存储长期阻塞采样。重复保存相同代次不额外写入。

健康与故障实验

采样服务期限 2500ms、通信 1500ms，监督每 50ms 评估；硬件 watchdog 名义 2s。无流量、已 STOP、传感器返回错误但工作循环仍前进，都不属于卡死。首次超时锁存停喂狗，故障记录只辅助暖复位诊断，不能代替真实上板观测。

测试构建才开放以下显式命令，普通固件返回 UNSUPPORTED：

输入	预期逻辑结果（非实测）
fault 1 / task 0或1	指定任务停止上报，随后watchdog复位
fault 2	监督停止喂狗，注册的watchdog仍保留
fault 3	1秒模拟传感器失联，有效位清零，系统保持健康
fault 4	TX暂停2秒，队列积压/丢弃可观察，随后同步恢复

命令示例：`python3 scripts/device-console.py fault --fault 3 --port COMx`。应答入队后 200ms 执行，不自动重试故障。F407 以 FAULTS=1 构建；S3 用独立 sdkconfig 合并 sdkconfig.defaults.faults。断电配置试验先使用专用存储，不对有重要数据的模块做实验。

短自测

1. CRC 校验正确的报文，为什么仍不能信任"无效位"为 0 的通道数据？

> **▼ 参考答案**
> CRC 只保证报文字节在传输中没有被检测出损坏，它不验证传感器是否真正完成了采样。如果传感器失联或处于错误状态，驱动可能返回全零或上一次缓存值，CRC 仍然正确（因为 CRC 算的是报文本身，不是传感器状态）。有效位（valid bit）是驱动层对传感器健康状态的判断，独立于 CRC。只看 CRC 不看有效位，等于把"信封没破"当成"信的内容正确"。

2. 配置保存到 EEPROM 时，为什么不持有采样状态锁？

> **▼ 参考答案**
> EEPROM 的页写入需要 5-10ms 等待（AT24C02 的页写周期），如果在此期间持有采样锁，采样任务会被阻塞至少一个周期。设计原则是"保存用影子拷贝"：先把当前配置 memcpy 到临时缓冲区，释放锁，再慢慢写 EEPROM。采样任务继续用原始配置运行，不受影响。锁的粒度应该覆盖"读取-修改"的原子性，而不覆盖"写入慢速外设"的等待。

3. 什么情况下重启后仍加载旧参数？

> **▼ 参考答案**
> 两种情况：①只调了 `configure` 修改运行时参数但没调 `save`：新参数只活在 RAM 里，断电即失；②调了 `save` 但 EEPROM 写入失败（总线忙、页写未完成就断电、WP 引脚意外拉高）。健壮的做法是 save 后立即 readback 校验，不匹配则重试或报警。旧有效配置应保留在 EEPROM 的另一槽（双槽轮转），直到新配置校验通过才切换。

无数据先查 START 状态和端口/交叉接线，再查 STATUS 有效性与错误计数；无响应查看 RX 错误和 TX 丢弃；意外复位检查 reset_reason、过期 mask、调试器冻结设置。不要先加长看门狗掩盖卡死。

附录：完整运行路径

公共任务层：

<<< ../../code/common/logger/runtime.c

平台适配：

<<< ../../code/stm32/platform/logger_port.c

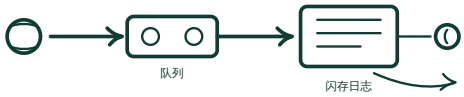
你做到了

可追踪一条记录从外设到 CSV，能分清配置应用、保存和恢复，也能把故障从静默错误变成明确状态。上板记录请注明板版本、固件提交、接线、输入和原始日志，本页当前仍为待实测。

通往单片机之路第 88 章

J2 ESP32-S3 传感器记录器

难度 ★★★☆☆ · 约 60 分钟



本章罗盘

问题

同一份记录器核心跨到 S3，配置存储和看门狗为什么不能照搬 F407 方案，port 层要换什么？

交付

S3 版记录器跑通『六轴→滤波→队列→CSV』同一条链路，附 NVS set_blob→commit→回读比对验证、fault 1–4 日志，及 F407/S3 决策差异（NVS 单 blob、TWDT）对照笔记。

边界

全页待实测：NVS 掉电恢复明确要求本板实验、不能从 F407 双槽模型直接推导；故障表为预期逻辑结果非实测；QMI8658 内部 250Hz 不等于应用侧 10Hz 无丢样，采集率结论不可互换；OTA 回滚只在自测题出现，不构成本章结论。

依据锚点 `a7 · a9 · f2 · mcu-code · mcu-lab`

> 🎯 数据能打印一次不难；让它在传感器失联、队列积压和配置写入中断后仍有明确状态，才是一套完整记录器。

本章精髓

采集、滤波、队列、通信、配置保存与监督各有一个职责。两平台共享协议和纯 C 核心，驱动差异留在 port 适配层。健康判据是完成有界工作，传感器错误不自动等于任务卡死。

先修

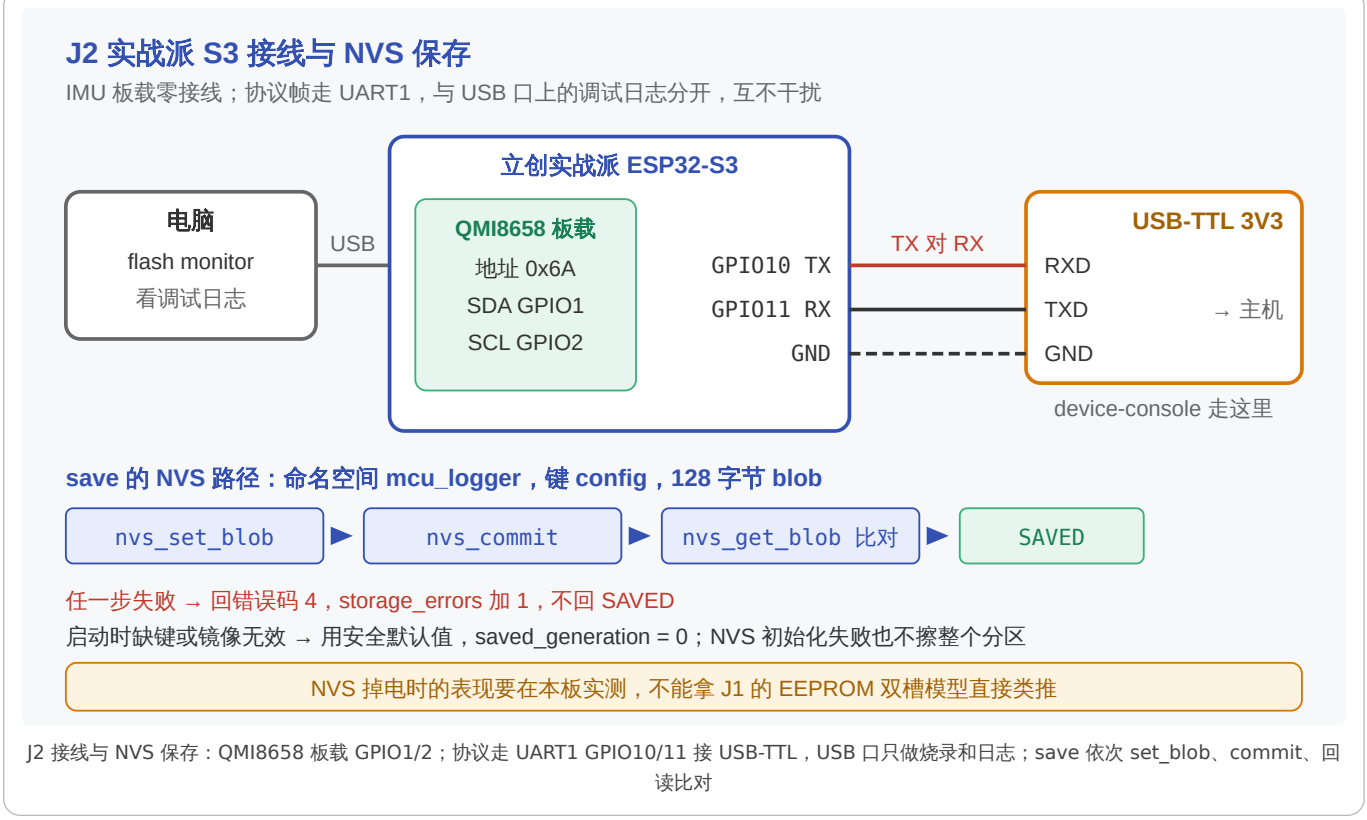
必需：C8 协议、S17 看门狗、P13 健康监督；板卡路线补 S9 ADC 或 P6 I2C。另一平台作为并行对照。

工程信息

时长	约 60 分钟
难度	★★★☆☆
配套代码	工程已提供 · 真实外设驱动与完整运行路径已提供；物理板卡结果待测。
上板验证	待上板实测；下文为预期现象，空白记录不表示验证通过

配置与接线

板载 QMI8658：GPIO1 SDA、GPIO2 SCL，7 位地址 0x6A。GPIO10 UART1 TX、GPIO11 RX 接 3.3V USB-TTL；默认控制台单独输出调试日志。NVS 只访问 mcu_logger 命名空间的 config 键，不自动擦整个分区。



J2 接线与 NVS 保存：QMI8658 板载 GPIO1/2；协议走 UART1 GPIO10/11 接 USB-TTL，USB 口只做烧录和日志；save 依次 set_blob、commit、回读比对

F407 基准为复位 HSI16+FreeRTOS V11.1.0；S3 为 IDF5.5.2。共地、TX/RX 交叉，COMx 换实际端口，绝不把 5V 信号直接接入 3.3V GPIO。

先跑起来

在已激活工具链环境执行：

```
cd code/esp32/08-sensor-logger
idf.py set-target esp32s3
idf.py build
idf.py -p COMx flash monitor
```

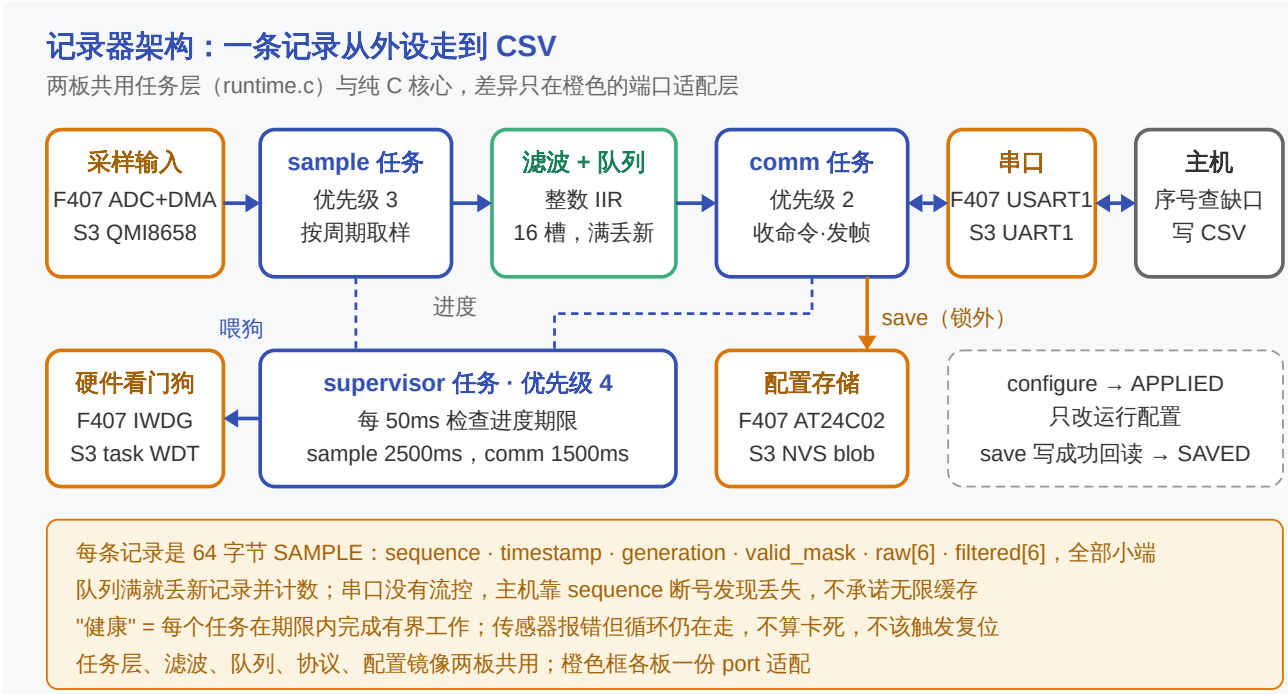
随后回仓库根，在主机执行：

```
python3 -m pip install -r scripts/device-console-requirements.txt
python3 scripts/device-console.py info --port COMx
python3 scripts/device-console.py status --port COMx
python3 scripts/device-console.py configure --period 200 --filter-shift 2 --port COMx
python3 scripts/device-console.py save --port COMx
python3 scripts/device-console.py record --seconds 10 --output samples.csv --port COMx
python3 scripts/device-console.py stop --port COMx
python3 scripts/device-console.py start --port COMx
```

拔电之前先押个证词：如果刚才只跑到 configure、还没 save 就断了电，重启之后 NVS 里存的是新周期还是旧周期？押完再往下读；不服，就真拔一次电，让重启后的 status 作证。

record 写入新的 CSV 文件；请使用新的输出路径，避免覆盖之前的数据。默认周期 100ms；period 允许 100–1000ms 且为 10ms 倍数，filter-shift 允许 0–6，0 为直通。configure 只改变运行配置，save 成功才持久化；旧 generation 的在途采样会丢弃计数，防止误标新配置。

四层实现



记录器架构：采样输入经 sample 任务、滤波与 16 槽队列、comm 任务和串口到主机；supervisor 按期限检查进度并喂硬件看门狗；save 在锁外写配置存储

层	实际职责	验证
外设适配	ADC或I2C、串口、存储、硬件watchdog	平台编译与待上板日志
公共核心	滤波、配置镜像、16槽样本队列	生产C函数的宿主断言
RTOS运行层	采样/通信/监督任务，短时状态互斥	故障注入和期限预算
主机	独立编解码、序号匹配、CSV	C/Python交叉向量

系统设计决策

为什么选四层架构而非单体？ 嵌入式项目常见诱惑是"反正就这几个功能，全塞 main.c"。但当我们需要在 F407 和 S3 两个平台复用核心逻辑时，单体结构会让代码分叉难以维护。四层分离让公共核心（滤波、协议、队列）在两个平台共享同一份 C 代码，平台适配层只处理驱动差异。代价是多了一层间接调用，但换来的是：①主机断言可以在 PC 上验证核心逻辑；②新增平台（如 ESP32-C3）只需写适配层；③代码审查时能清晰区分"平台 bug"和"逻辑 bug"。

为什么用 NVS 单 blob 而非 EEPROM 双槽？ F407 用 AT24C02 双槽轮转保证断电安全：写新槽前旧槽完整，写失败可回退。ESP32 的 NVS 是 Flash 分区，底层已有磨损均衡和掉电保护机制，再实现双槽是重复造轮子。但 NVS 的代价是：①单次 blob 最大 1984 字节（我们的配置 128 字节远未到限）；②nvs_commit 是原子操作但耗时不可预测（Flash 擦写），所以仍在锁外执行；③NVS 损坏时需要整分区擦除，不如 EEPROM 双槽可以"回退到旧槽"。选择 NVS 是因为 IDF 生态支持，但必须接受"掉电恢复需本板实测，不能从 F407 双槽模型直接推导"。

为什么样本队列只有 16 槽？ 理论上队列越大越不容易丢数据，但每个槽是 64 字节，16 槽就是 1KB RAM。ESP32-S3 有 512KB SRAM 看似充裕，但 Wi-Fi/BLE 协议栈、FreeRTOS 任务栈、PSRAM 缓存都在抢这块空间。16 槽在 100ms 采样周期下能缓冲 1.6 秒数据，足够覆盖通信任务短暂阻塞（如 USB CDC 流控）。如果通信持续失败，队列满后丢新样本并计数，这是显式的"过载保护"，而非隐式的内存溢出。调大队列前应先用 STATUS 命令查高水位，确认是否真的需要。

为什么健康监督用 TWDT 而非软件看门狗？ ESP-IDF 的 Task Watchdog Timer (TWDT) 是内核级机制，每个 CPU 核心独立监控。相比软件看门狗（一个任务定期喂另一个任务），TWDT 的优势是：①即使主任务卡死在临界区，TWDT 仍能触发；②可以区分"Pro 核卡死"和"App 核卡死"；③复位原因寄存器保留故障现场。代价是 TWDT 超时后直接触发系统复位，不如软件看门狗可以"先记录故障再复位"。本项目选择 TWDT 是因为 S3 没有外部看门狗芯片，且 TWDT 的 2 秒超时足够覆盖最坏情况（采样 2.5 秒 + 通信 1.5 秒）。

定时采样与滤波

周期任务用 xTaskDelayUntil 定时读取 QMI8658 最新六轴样本；内部 250Hz 采样不等于应用 10Hz 无丢样采集。加速度原码除 8192 为 g、角速度原码除 64 为 dps，量程固定 ±4g/±512dps。I2C 失败时标记无效并每秒重新尝试初始化，通信和监督继续。

默认整数 IIR 每次走向新值的 1/4，使用 64 位中间差值避免溢出；整数舍入会产生小死区，滤波延迟不是零。有效位清除的通道不更新滤波，恢复后第一样本重新初始化。STOP 停止采样服务，但通信和监督仍执行，不应该触发看门狗。

协议与缓冲

SAMPLE 固定 64 字节：sequence/u32、timestamp_ms/u32、generation/u32、sensor_type/u8、valid_mask/u8、status/u16、raw[6]/i32、filtered[6]/i32，全部 LE。ADC 仅 bit0 有效；IMU 最多六位有效。INFO 给原码比例，STATUS 的 TLV 给运行配置、保存代次、丢弃/传感器/存储错误和队列高水位。

样本队列 16 槽，满则丢新记录并计数。TX 另有 4 帧槽和 100ms 发送期限，部分帧失败后先发零同步。无硬件流控的 UART 不能确认对端真正接收，也不能可靠检测接收线断开；主机用序号发现缺口，不承诺无限离线缓存。

配置保存与恢复

配置作为单个 128 字节 NVS blob 保存，包含 magic、schema、generation 和 CRC。只有 nvs_set_blob、nvs_commit、回读都成功才回 SAVED。初始化失败不会全盘擦除；缺键或无效镜像使用安全默认值，saved_generation=0。NVS 实际掉电恢复需本板实验，不能从宿主双槽模型直接推导。

配置代次用有界模序号比较；不同 schema 不能直接按新结构解释。保存发生在状态锁之外，采样优先级高于通信，避免慢存储长期阻塞采样。重复保存相同代次不额外写入。

健康与故障实验

采样服务期限 2500ms、通信 1500ms，监督每 50ms 评估；硬件 watchdog 名义 2s。无流量、已 STOP、传感器返回错误但工作循环仍前进，都不属于卡死。首次超时锁存停喂狗，故障记录只辅助暖复位诊断，不能代替真实上板观测。

测试构建才开放以下显式命令，普通固件返回 UNSUPPORTED：

输入	预期逻辑结果（非实测）
fault 1 / task 0或1	指定任务停止上报，随后watchdog复位
fault 2	监督停止喂狗，注册的watchdog仍保留
fault 3	1秒模拟传感器失联，有效位清零，系统保持健康
fault 4	TX暂停2秒，队列积压/丢弃可观察，随后同步恢复

命令示例：`python3 scripts/device-console.py fault --fault 3 --port COMx`。应答入队后 200ms 执行，不自动重试故障。F407 以 FAULTS=1 构建；S3 用独立 sdkconfig 合并 sdkconfig.defaults.faults。断电配置试验先使用专用存储，不对有重要数据的模块做实验。

短自测

1. CRC 校验正确的报文，为什么仍不能信任"无效位"为 0 的通道数据？

▼ 参考答案
CRC 只保证报文字节在传输中没有被检测出损坏，它不验证传感器是否真正完成了采样。如果传感器失联或处于错误状态，驱动可能返回全零或上一次缓存值，CRC 仍然正确（因为 CRC 算的是报文本身，不是传感器状态）。有效位（valid bit）是驱动层对传感器健康状态的判断，独立于 CRC。只看 CRC 不看有效位，等于把"信封没破"当成"信的内容正确"。

2. 配置保存到 NVS 时，为什么不持有采样状态锁？

▼ 参考答案
NVS（Non-Volatile Storage）写入需要擦除 Flash 页，耗时可达数十毫秒。如果在此期间持有采样锁，采样任务会被阻塞至少一个周期。设计原则是"保存用影子拷贝"：先把当前配置 memcpy 到临时缓冲区，释放锁，再写 NVS。采样任务继续用原始配置运行，不受影响。锁的粒度应该覆盖"读取-修改"的原子性，而不覆盖"写入慢速外设"的等待。

3. 什么情况下 OTA 回滚后仍运行旧固件？

▼ 参考答案
两种情况：①新固件启动后没有及时调用 `esp_ota_mark_app_valid_cancel_rollback()` 标记自身有效：看门狗会在超时后触发回滚到旧分区，但如果新固件在标记之前就崩溃重启，回滚机制已经生效，下一次启动仍是旧固件；②NVS 中的 `otaseq` 分区序号在写入过程中断电，导致分区表指向不一致。健壮的做法是新固件入口第一件事就标记有效，并在 NVS 写入完成后做 readback 校验。

无数据先查 START 状态和端口/交叉接线，再查 STATUS 有效性与错误计数；无响应查看 RX 错误和 TX 丢弃；意外复位检查 reset_reason、过期 mask、调试器冻结设置。不要先加长看门狗掩盖卡死。

附录：完整运行路径

公共任务层：

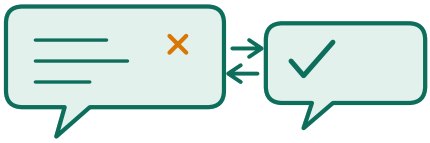
<<< ../../code/common/logger/runtime.c

平台适配：

<<< ../../code/esp32/08-sensor-logger/main/port.c

你做到了

可追踪一条记录从外设到 CSV，能分清配置应用、保存和恢复，也能把故障从静默错误变成明确状态。上板记录请注明板版本、固件提交、接线、输入和原始日志，本页当前仍为待实测。



PART PA

第十一部分 AI 辅助开发

把大模型变成你的结对工程师：提示词给约束、输出对手册、工具化工作流、幻觉有防御，最后用这本书自身的生产线自证。

> 模型写代码很快，错得也快：总线归属张冠李戴、位语义张冠李戴、你的板子接线它永远看不见。这一部不教你用哪个 AI，教你怎么当 AI 产出的审计员：三段式提示词先把需求钉死，RM0090 逐位对账把假设打回证据，MCP 工具流让模型自己读手册跑构建，最后一章解剖这本书自己的 AI 生产线。读完的标志：模型给你的每一段寄存器代码，你先看它替你做主了哪些决定。

本部 6 章 · 约 250 分钟（4.2 小时）

89 . A0 为什么把大模型当结对工程师
90 . A1 提示词模式：三段式，把决定权从模型手里拿回来
91 . A2 手册对账：模型写的寄存器代码，逐位验证给它看
92 . A3 工具流：让模型读手册、跑构建、看串口
93 . A4 幻觉防御：八类幻觉的类型学与验证决策树
94 . A5 这本书是怎么和 AI 一起写成的：生产线的自证

通往单片机之路第 89 章

A0 为什么把大模型当结对工程师

难度 ★☆☆☆☆ · 约 35 分钟

本章罗盘

问题

AI 都能写出能跑的代码了，我为什么还要学寄存器和 RTOS？

交付

一张「模型能替你做与不能替你做」清单 + 2026 工具地图（Copilot/Claude Code/Cursor/MCP 各一条官方可引定位）+ 学习阶段与工作阶段两用法边界。

边界

只立工作观；对账在 A2、提示词在 A1，不展开任何模型原理；不评价任何具体产品优劣，只引官方定位（访问日期 2026-10-27，界面会变、方法论不变）。

依据锚点 `ai-copilot-vscode-overview` · `ai-claude-code-overview` · `ai-cursor-run-modes` · `ai-mcp-spec` · `mcu-code`

> 🎯 AI 都能写出能跑的代码了，我为什么还要学寄存器和 RTOS？这个问题值得当面问出来，也值得一个有证据的回答：模型交出的是假设，你手里的手册、原理图与示波器交出的才是判据；它出草稿，你出终审，这是结对工程师与代笔的全部区别。本章立这个工作观，发你一张「能替你做与不能替你做」的清单，画一张只引官方定位的 2026 工具地图，再划清学习阶段与工作阶段两种用法的边界。界面会变，方法论不变。

本章精髓

1. **模型给假设，人给判据，终审权在人**。工具官方文档把分工写得比任何警告都直白：你描述结果、评审改动、验证产物；红线句更直白：输出可能不正确甚至含安全漏洞，评审与测试是你的活。结对工程师的意思是它当快手草稿员，你拿手册与原理图当终审法官，审不过的稿子不许进代码库。
2. **能与不能的分界线是你的板子**。写样板代码、解释报错、写测试、起草寄存器代码，它能做，因为这些事的正确性定义在公共知识里；知道你的原理图、替你上板、为你的事故负责、替你做工程取舍，它永远不能，因为这些事的正确性定义在你的硬件与你的责任里。这张清单背下来，比记任何快捷键都值钱。
3. **学习阶段要它解释，工作阶段它出稿你审计**。学习阶段让它代写，等于给「CubeMX 也能点灯，但你学会了什么」这道老答辩题换一个更贵的替身：灯亮了，你没学会。工作阶段反过来用，它量产草稿，你按对账三问终审；审计动作在 A2，工单写法在 A1。

- **能理解**：为什么「AI 会写代码」不构成「你可以不学寄存器」的理由；「模型输出是分布不是事实」的经验含义；四种工具官方定位的分工差异。
- **能用**：背出能/不能清单并在下次提问前自查；按学习、工作两阶段切换用法；各用一条官方定位向同伴说清 Copilot、Claude Code、Cursor、MCP 分别是什么。

先修

- S2 总线与时钟：AHB1/APB1 总线归属（先跑起来实验的标准答案在这里）；
- S3 GPIO：点灯全流程与霸道者 F407 板卡事实（「你的原理图」这条不能项从那里来）；
- C0 为什么嵌入式 C 是另一种 C：没有弹窗只有事故的世界观（终审权为什么在人的硬件版理由）；
- F0 为什么需要 RTOS：中断上下文的纪律（第五节的拦截案例从这里来）。

先跑起来（一致性实验，10 分钟）

打开你手边任意一个代码模型，网页对话框就够用。开三个互不可见的会话（隐身窗口、退出重登、换一台设备都行），把同一句原样各问一遍：

发问之前先押个一致性：同一个问题问三个互不可见的会话，三份答案会全对、全错，还是有对有错？光看语气，你分得出哪份是幻觉吗？押完再动手，这一把量的是你手边这个模型的分布。

> STM32F407 的 GPIOF 挂在哪条总线上？它的时钟使能位在哪个寄存器的哪一位？

标准答案你在 S2 已经学过：GPIOF 挂 AHB1，使能位是 `RCC_AHB1ENR` 的 bit5（GPIOFEN）。判据不用背，查就行：RM0090 §2.3 表 1 的存储器映射写着 GPIOF 基地址 0x40021400 落在 AHB1 外设区段，§7.3 的 RCC 寄存器定义写着 AHB1ENR 的 bit5 是 GPIOFEN。

把三份回答并排，填这张表：

会话	总线归属	寄存器名	位号	语气
第 1 次				
第 2 次				
第 3 次				

两个实验纪律先说清。其一，三个会话必须互不可见：在同一个会话里追问三次，模型会参考自己上一轮的答案，你测到的就不是分布，而是它维护一致性的能力。其二，选题有讲究：好的实验题要满足手册里有唯一正确答案（否则你无法判对错）、语料密度适中（太热的题测不出分布，太冷的题超出能力没有教学价值）。GPIOF 恰好卡在中间：它在全网的点灯教程里出现频率高，但「挂哪条总线」这个问题，错法有足够的想象空间。

实验报告按这个格式存档（进版本控制或实验笔记，下次直接调用）：

字段	内容
日期与模型	哪天问的、用的哪个模型与版本
问题原文	原样记录，一个字不改
三份答案	总线归属、寄存器名、位号，逐条摘录
一致性判定	全对 / 全错 / 有对有错
手册判据	RM0090 §2.3 表 1 + §7.3，标准答案的出处
结论	这次观察到的分布形态

三种结果，三种解读：

- **三次全对**：别急着宣布模型可靠。GPIOF 是全书反复使用的例子，语料密度高；换一个你不熟的型号再问三次，比如 F103 的 GPIOA，或者 ESP32-S3 的某个信号该走 GPIO 矩阵还是 IO_MUX 直通（背景在 P2），再看它的一致性。
- **三次全错**：恭喜，你用手里的一道题抓到了幻觉现场。先别纠正它，把三份答案存档，去 A2 看怎么把这种现场变成对账练习。
- **有对有错**：这是信息量最大的结果。同一个模型、同一个问题、同样自信的语气，答案却不同——你观察到的这个东西叫分布。

为了让你认得出长相，错的那份答案通常是这样的（编造示例，语气还原）：

> GPIOF 挂在 APB1 总线上，时钟使能位是 RCC_APB1ENR 的第 5 位。

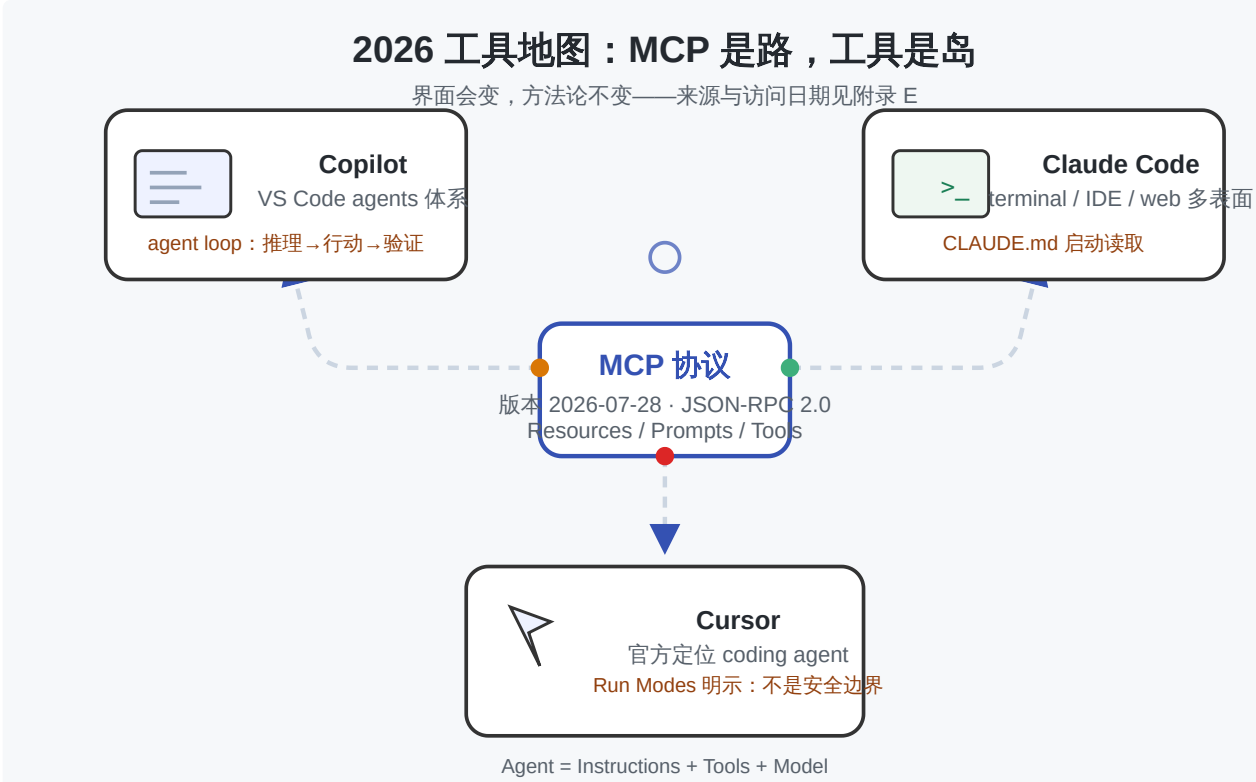
工整、笃定、带寄存器名带位号，与标准答案的差别只有 A 和 H 一个字母。这不是它不用心，是它真的分不清：语气从来不由正确性供电，这个结论在 A4 还会再收一次利息。

这个实验的产出不是「模型不行」四个字，是本章工作观的经验底座：**模型输出是分布，不是事实**。分布收敛成事实的动作叫对账，全流程在 A2；本章只负责让你在接受这个工作观之前，先亲手摸一次分布。

为什么把实验放在全章最前面？因为工作观靠说服立不住，靠观察立得住。读十篇「AI 会幻觉」的文章，不如亲手抓到一次「同问不同答」。你填的那张表，就是你与结对工程师共事的第一份档案。

动画：2026 工具地图

三种工位加一条总线：Copilot 住在 VS Code 的 agents 体系里，核心机制是「推理 → 行动 → 验证」的 agent loop；Claude Code 从终端出发，同一个引擎覆盖 terminal、IDE 扩展、desktop、web 四种表面；Cursor 把整款产品定位成 coding agent，Agent 由 Instructions、Tools、Model 三构件组成；MCP 不是第四个工具，是前三者接外部数据源的开放协议，Hosts/Clients/Servers 三方、Resources/Prompts/Tools 三类原语。



2026 工具地图：Copilot / Claude Code / Cursor 三种工位，MCP 一条总线

读这张图只记一层关系：三个工位是产品，随版本换代；一条总线是协议，随治理中立而沉淀。你的终审工位不在图上，因为图里没有一个工具能替你坐那个位置。

小节结构

一、结对工程师不是代笔：官方分工句与红线句怎么读；二、能替你做与不能替你做：四加四清单与分界线；三、学习阶段与工作阶段：同一工具两种相反用法；四、2026 工具地图：Copilot、Claude Code、Cursor、MCP 各一条官方定位；五、回到开头的问题：为什么还要学寄存器和 RTOS。

一、结对工程师，不是代笔

这一章排在整部最前，是因为工作观决定后面所有章节的姿势：提示词怎么写（A1）、对账怎么做（A2）、工具怎么接（A3）、幻觉怎么防（A4），全是同一个工作观在不同工序上的投影。观不正，后面每章的技巧都会用歪成代笔的辅助工具。

先把两个比喻坐实，因为它们决定你接下来用 AI 的全部姿势。

代笔的关系是：你出题，它交稿，你照抄。这个关系里没有验证工位，稿子的正确性由交稿者的语气担保。嵌入式世界里语气一文不值：编译器只查语法，不认识你的板子；烧录器只管搬运，不认识你的原理图；能让灯亮的只有「位写对了地方」这一件事——而这件事只有手册、原理图与上板现象说了算。

结对工程师的关系是：它出草稿，你出判据；它给假设，你给验证。这个分工不是本书的发明，是工具官方文档自己写的。VS Code 的 agents 文档给了一句可以贴在工位上的分工句：

> For a task that spans files and tools, an agent can find relevant code, make changes, and run checks without you directing each search, edit, and test run. You describe the outcome, review the changes, and verify the result.

前半句是它的活：跨文件找代码、做修改、跑检查，不用你逐步指挥。后半句三个动词，describe、review、verify，主语全是你。

把这三个动词逐个翻译成嵌入式动作。**describe 是下工单**：芯片型号、手册章节号、板卡事实、约束条件写清楚，工单质量决定草稿质量，三段式写法在 A1 展开。**review 是对账**：寄存器名与总线归属对不对、每个位域的值对不对、初始化顺序对不对，三问清单全流程在 A2。**verify 是上板**：烧录、按复位、看灯、量波形，工具链与调试器用法在 B6。

三个动词有一个共同点：它们都要求你手里先有判据。describe 要求你知道自己要什么；review 要求你能查出它错在哪；verify 要求你认得物理世界的证据。没有判据的人用 agent，三个动词全部退化成同一个动作：点头。

同一份文档还给这个工作机制起了名字：

> An agent uses a language model to reason about your request and tools to act on your development environment...This repeated reasoning, action, and validation is the **agent loop**.

推理、行动、验证，循环往复。注意 loop 里的「验证」是机器侧的：跑测试、看编译输出、读 lint 报错。机器侧验证只能证明代码在它触达的环境里表现如何，证明不了位定义与你的板子一致。GPIOF 挂 AHB1 还是 APB1，编译器不报警，测试框架也不知道，这道终审只能在你手里。

把机器侧验证的边界再划细一点。编译器检查的是语法与类型；测试检查的是你写下的断言；lint 检查的是风格与常见坏味道。三样都不认识 RM0090，不认识你的原理图，不认识共阳与共阴的区别。嵌入式代码的错误有一大半活在这三样的盲区里：总线归属错了，编译通过；上下拉配反了，编译通过；把 BSRR 的高 16 位当置位用，编译还是通过。机器侧验证是 agent loop 的内环，你的手册与示波器是外环，外环缺席的内环转得再快也只是在原地空转。

官方对这道终审的表述，一条比一条硬。GitHub 的 Copilot 文档写得一字未曲：

> Both chat and agentic experiences on GitHub.com can produce incorrect or suboptimal code, including code that contains security vulnerabilities. Review and test the output before using it in production.
>
> Logs do not replace your own review and testing.

VS Code 的 agents 文档把责任主体点得更明：

> You remain responsible for deciding which changes reach your codebase.

三句连起来读：输出可能有错，包括安全漏洞；日志代替不了你的评审与测试；哪些改动进你的代码库，由你负责。这是厂商白纸黑字的文档口径，不是本书的危言。把这些表态当免责套话翻过去，是 AI 时代嵌入式开发者最常踩的认知坑：厂商比你更清楚模型的边界在哪，并且已经告诉你了。

为什么厂商要写这种「自拆台」的红线？因为责任划分必须清晰：工具提供生成能力，你提供正确性担保。这跟示波器厂商在手册里写「测量结果的正确解读由使用者负责」是同一种条款。区别在于，示波器的波形不会自己辩解，模型的输出会：它用流畅的注释与自信的口吻包装每一个假设，包括对的部分与错的部分。语气越自信，红线越重要。

所以「结对工程师」四个字要拆开读：「工程师」描述它的产出能力（读代码、改文件、跑命令），「结对」描述责任形态（终审与责任都在你）。两种关系的对照：

维度	代笔	结对工程师
初稿	模型写	模型写
正确性判据	模型语气	手册 + 原理图 + 上板现象
验证环节	缺失	机器侧 agent loop + 人的终审
出错时排查起点	从头怀疑人生	对账三问（A2）
事故责任	你（盲签）	你（审过，留了证据）

代笔关系的结局只有三种，按发生频率排：

1. **运气好**：代码恰好对，灯亮了。你什么都没学到，还强化了「照抄就行」的肌肉记忆，下次赌得更大。
2. **坏得明显**：灯不亮。你没有判据，只能从头怀疑人生：板子坏了？线松了？型号错了？排查没有起点。
3. **坏得隐蔽**：灯亮了，时序错了；功能对了，边界炸了。这种雷不在实验室爆，在凌晨的生产现场爆，排查成本是实验室里的十倍。

结对关系不改变模型出错的概率，改变的是错误的落点：雷在对账台上爆，不在现场爆。

盯住对照表最后一行：无论哪一列，责任都是你。这就是把终审工位建起来的理由：责任逃不掉，能选的只是盲签还是有证据地签。两种签法的差异不在出不出事，在出事之后：盲签的人从零开始怀疑人生，有证据地签的人从对账记录往回倒一层就找到断点。

最后把「结对」的信任结构说准：结对不是不信任模型，是把信任放对地方。信任它的速度、语料广度与不知疲倦；不信任它对任何具体事实的断言，凡是具体事实都回到手册与原理图。也信任官方红线：厂商说输出可能有错，这句话本身是可用的元信息。信任放对地方之后，它犯的每个错都只是流程里的一次普通迭代，不是事故。

还要给比喻划个边界：结对工程师不是人。它不会愧疚，不记得昨天的对话（跨会话记忆要靠机制与文件，A3），更不会为你的 deadline 焦虑。所以与它协作靠流程纪律，不靠人际关系技巧：工单写清楚、判据摆上桌、证据留档案。把对人的期待投射到它身上（「它应该会改」「它应该知道」），是代笔心态的另一种形态。

二、能替你做，与不能替你做

这张清单不是凭感觉拍的，它的每一项都来自两类出处：官方文档写明的能力与边界（附录 E 来源卡），以及前面章节里被反复验证的嵌入式事实（总线归属、共阳接法、中断纪律）。清单会随工具进化微调，分界线的逻辑不变。

能做的四件事，按可靠度从高到低排：

1. **写样板代码**。工程骨架、Makefile、CMakeLists、CI 配置、.gitignore。这类内容的正确性定义在公共知识里，全网语料密度最高，幻觉率最低。它几十秒交出的骨架，你手敲要半小时，这是结对关系里最划算的一笔交易。
2. **解释报错**。编译错误、链接脚本报错、HardFault 的寄存器现场，它读得比你快，给出的排查方向常常靠谱。两个纪律：报错要贴原文，贴「跑不起来」它只能陪你猜；它的解释仍是假设，成立与否靠你复现。
3. **写测试**。测试用例、边界值枚举、参数化测试骨架，它量产极快且不知疲倦。测试代码本身也有幻觉（编出来的库函数名能以假乱真，类型学在 A4），但测试跑不过这件事会自己报警，自带纠错回路。
4. **起草寄存器代码**。注意「起草」二字。给它芯片型号、手册章节号、板卡事实、明确约束，它能交出结构像样的初始化框架；然后每一个位都要过对账（A2），因为总线归属、位语义、共阳共阴这些「你这块板子」的事实，它一概不知。

能侧四件的共同特征：正确性都可以在「你的板子」之外定义清楚。Makefile 对不对，构建系统说了算；报错解释对不对，复现说了算；测试对不对，跑一遍说了算；寄存器草稿对不对，RM0090 说了算。判据全部外在于你本人的记忆与偏好，所以可以放心让它起草，再由判据收口。

不能做的四件事，按危险度从高到低排：

1. **为你的事故负责**。烧错的板子、炸掉的驱动管、误期的交付，责任主体只有你。第一节三句官方红线已经把这件事写得很白，不再重复。
2. **知道你的原理图**。霸道者 F407 的 RGB 灯 PF6/7/8 共阳接 3.3V、低电平点亮，这个事实只活在你的原理图和 S3 的板卡事实表里。模型没见过你的板子，它有的是全网最常见接法的统计印象，而最常见接法未必是你的接法。
3. **替你上板**。烧录、按复位、看灯、量波形、摸芯片温度，物理世界的终审工具在你手里：示波器、逻辑分析仪、万用表。它能读你贴回来的波形描述与日志（怎么让它读得顺，在 A3），但探头只能你去夹。
4. **替你做工程取舍**。功耗与响应速度的权衡、引脚分配与 PCB 布局的妥协、交期与严谨度的平衡。这些决策的输入是你的产品约束与现场条件，它不知道你的约束，给出的「最佳实践」只是统计意义上的常见答案。

不能侧四件的共同特征：正确性定义在物理世界与法律关系里，不在软件能力里。原理图是物理事实，上板是物理动作，责任是法律关系，取舍是你的约束函数。这四样都不随模型升级而改变：下一版模型代码写得更好，它仍然没见过你的板子、夹不了探头、签不了字。不能清单是这张表里最稳定的部分。

四加四的分界线只有一条：**这件事的正确性定义在公共知识里，还是在你的板子与你的责任里**。前者放心让它起草，后者它连入场券都没有。

拿一张真实工单演示拆分。需求：「给霸道者 F407 加一个 UART 命令行，能命令点亮、熄灭 PF6 红灯」。拆给模型的部分：命令行解析的骨架代码、环形缓冲区的测试用例、链接报错的解释。必须自己扛的部分：PF6 共阳低电平点亮这个写进提示词的板卡事实；UART 波特率分频值的复算（手册公式加计算器，模型算术错不罕见）；中断回调里只设标志不办事这条纪律的执行（中断里为什么不调阻塞函数，F0 有完整推导）；最后烧上板看现象。一张工单拆完，模型拿到的是速度与广度，你留下的是判据与责任，这就是分工。

有一个容易误判的中间地带要说透：RM0090 是公共的，谁都能读；但「你的板子怎么用这些位」是你的，全网语料里没有你的原理图。所以寄存器代码落在能侧，却永远带对账条件。RTOS 同理：队列与信号量的语义有官方文档定义，属公共知识；但你的任务优先级怎么排、栈开多大、看门狗超时怎么定，输入是你的产品，它只能给起点，不能给答案。

速查表贴在工位上：

任务	能不能甩给它	附带条件
工程骨架、构建脚本	能	跑一遍构建再信
报错解释	能	贴原文；解释当假设，靠复现确认
测试用例	能	测试代码也要审，编的库函数名在 A4 有案底
寄存器初始化草稿	能	每一位过对账三问（A2）
原理图相关决策	不能	它没见过你的板子
上板烧录与量测	不能	探头在你手里
事故责任	不能	官方红线三句，第一节
工程取舍	不能	约束输入在你这里

用法很简单：提问之前先看这张表。落在「能」侧的任务，放心甩，甩完按附带条件验收；落在「不能」侧的任务，问它之前先问自己：我是不是在把本该自己扛的判据外包出去？外包判据的瞬间，你就从审计员降级成了盖章员。

自查再简化成三问，提问前过一遍：这件事的正确性谁定义？定义物（手册、原理图、测试、规范）在我手里吗？验收动作我做得起吗？三问都答得上来，甩给它；任何一问卡壳，先补判据再提问。这张三问卡与 A2 的对账三问是同一套方法论的两个侧面：一个管提问前，一个管收货后。

三、学习阶段与工作阶段：两种用法

同一个工具，两个阶段，用法相反。混用是新人最容易犯的错：学习阶段用了工作阶段的代写法，工作阶段又用学习阶段的逐字手敲硬扛交付压力。

学习阶段：要它解释，不要它代写。

全书 FAQ 里有一条针对 CubeMX 的答辩口径：工作中能用图形化配置工具，学习阶段先手写一遍；理由是工具把引脚复用、时钟分频这些「会出错的决策」藏起来了，你只有先自己错过一次，之后才看得懂它替你做了什么选择。

AI 代写把这个坑放大了两级。CubeMX 生成的代码至少是确定性的：同样的勾选永远生成同样的代码，你可以对着生成结果反推配置逻辑。AI 代写连确定性都没有：同一句提示词三次给三个版本，每个版本配着一本正经的注释。用它代写作业，「你学会了什么」这道答辩题照样答不出，还多一层自欺：我好像看懂了。看懂别人的代码与写出自己的代码之间，隔着的就是亲手错那一次。

学习阶段的正确用法有四种，每一种的产出物都留在你脑子里：

角色	你怎么用	示例提示词
讲解员	让它解释概念，再追要不同角度的例子	「解释 BSRR 高 16 位为什么是复位，给三个不同角度的例子」
出题官	让它出题考你，你答它批改	「出十道 GPIO 位定义填空题，我答完你逐题批改」
答辩对手	让它扮演面试官追问你	「追问我：中断里为什么不能调阻塞函数，直到我答不上来」
错题分析员	把你写错的代码给它，只许指疑点不许给答案	「指出这段初始化里可疑的三处，不要给正确写法」

注意第四种：不给答案是关键。它给了答案，你的排查肌肉就萎缩一次。检验标准朴素：合上聊天窗口，你能把刚学的东西重写一遍、再讲给别人一遍。做不到，说明刚才是它学了一遍，不是你。

「讲解员」用法的一次完整示范，照这个节奏来：

> 你：解释 BSRR 高 16 位为什么是复位，给三个不同角度的例子。
>
> 它：（给出解释与三个例子。）
>
> 你：我用自己的话复述一遍：高 16 位写 1 把对应引脚拉低，因为共阳 LED 低电平点亮，所以「写高 16 位」等于点灯。我复述得对吗？
>
> 它：（确认或纠正你的复述。）
>
> 你：出一道变式题考我：如果 LED 改成共阴接法，BSRR 该怎么写？

同一节奏搬到 RTOS 一样成立：

> 你：解释中断里为什么不能调带超时的队列接收，从调度器的视角讲。
>
> 它：（给出解释。）
>
> 你：我复述：中断上下文没有任务控制块，调度器不知道中断阻塞了该切给谁。对吗？
>
> 你：出题考我：中断里要通知任务处理数据，正确做法有哪些，各适合什么场景？

注意这个节奏里代码一行都没让它写，但每一步都在用你的话重建概念。它输出的是解释，你输出的是理解；理解到位了，代码你自己写得出来，写的过程再卡壳，回 S3 查。

学习阶段还有三种自欺要提防，全是「它学了不是你学了」的变种：

1. **看懂了等于会了**：读它的解释频频点头，合上书一个寄存器名都写不出。检验：能不能默写。
2. **改对了等于懂了**：它指出错误你照改，灯亮了。你学会的是「听它的」，不是「为什么」。检验：换个引脚、换个外设，能不能独立再来一遍。
3. **问过了等于学过**：聊天记录越来越长，脑子里的越来越少。检验：这道题的判据，你能不能讲给没用过 AI 的同伴听。

为什么不能跳级？因为嵌入式知识是层层咬合的：总线归属咬合存储器映射，位语义咬合数据手册，RTOS 纪律咬合中断模型。代写跳过的每一层，都会在后一层的排查现场连本带利讨回来：灯不亮时你连「先查时钟还是先查模式」的顺序都没有，AI 给的五个猜测你只能挨个试。学习阶段的手写不是为了怀旧，是为了让排查时有顺序。

工作阶段：审计员模式。

交付压力下分工反过来：它当快手草稿员，你当审计员。草稿员的价值在速度，审计员的价值在判据。你的审计工位上摆三件工具：手册在位（RM0090，位定义可查）、原理图在位（板卡事实可查）、上板现象在位（终审判决可查）。审计动作就是对账三问：名对不对、位对不对、序对不对，全流程在 A2。

审计员模式有两条配套纪律。其一，工单质量决定草稿质量：上下文（芯片型号、手册章节、板卡事实）、约束（资源、时序、风格）、验收（可编译、上板现象、对账动作）三段式写法在 A1 展开；这里只立原则，不给上下文的工单等于让它猜你的板子。其二，审计员必须懂行：不懂总线归属的审计员，看不出 `RCC->APB1ENR |= (1<<5)` 错在哪；不懂 RTOS 的审计员，看不出中断服务函数里那句阻塞调用是死锁配方。这条纪律直接把本章开头那个问题的答案顶了出来，第五节收拢。

一张最小工单长这样（三段式的骨架，完整写法在 A1）：

> 上下文：STM32F407 霸道者板，PF6 红灯共阳接 3.3V，低电平点亮；手册 RM0090 §7.3、§8.4。
>
> 约束：只用裸寄存器，不用 HAL；时钟使能必须第一行；给出每一行的位定义出处。
>
> 验收：我会逐位对账并上板，灯亮且引脚读回低电平才算过。

对照这张工单，回看 A2 里那段自信满满的点灯代码，你会发现雷全是工单没写清的地方爆的：没给板卡事实，它就按最常见接法猜；没要求出处，它的注释就敢写得像查过手册。工单是草稿质量的上限。

审计员模式的标准动作序列，五步成环：

1. **下工单**：三段式提示词交底（A1），上下文给足，约束写死，验收标准列清；
2. **收草稿**：拿到输出先别读注释，注释是它的语气，不是它的证据；
3. **对账**：名、位、序三问逐位过（A2），手册与原理图是唯二判据；
4. **上板**：烧录取证，现象与寄存器读回值留档；
5. **回喂**：把证据句写回对话（实测 + 手册出处 + 板卡事实），让草稿基于证据修正，迭代到闭环。

五步里模型参与第一步与第五步的接收端，中间三步全是你的主场。这个环转熟了，单次迭代从小时级压到分钟级，这就是结对工程师的产出形态：它不替你会，它让你的会转得更快。

两阶段之间有个过渡带，判定标准也给你：当你对某个外设能做到「不看生成结果，自己手写也能点亮」，这个外设就从学习阶段毕业，进入工作阶段的代写区。毕业一门甩一门，永远只在已毕业的领域用审计员模式，在未毕业的领域保持手写生涩但扎实的节奏。全领域的毕业进度，就是你的能力地图。

同一个人的两个下午，对照一下两阶段的差别。学习阶段的下午：目标是搞懂 DMA 循环模式，你让它讲原理、出变式题、批改你的默写，代码你亲手写，写错它只指疑点。工作阶段的下午：目标是交付一个已毕业外设的驱动，你下三段式工单，它出草稿，你按三问对账、上板取证、证据回喂，五步环转到绿。两个下午都用 AI，产出物完全不同：一个产出你脑子里的判据，一个产出仓库里审过的代码。混淆两个下午的用法，是最常见也最隐蔽的浪费。

工作阶段也有三样不甩：架构决策（模块边界与数据流，错一层代价放大十倍）、安全相关代码（官方红线明写输出可能含安全漏洞，这类代码的每一行都要能追溯到你的评审）、量产固件的最终签发（签字的是你）。审计员模式管的是效率，这三样管的是命脉。

四、2026 工具地图：三种工位，一条总线

口径先行：本节每条只引官方文档与官方规范的能力描述、规范定义（来源卡与访问日期 2026-10-27 见附录 E），营销话术不引，产品优劣不评。四家文档都在高频改版：VS Code 文档 2026-10-21 刚把 chat 与 agent mode 重组为 Agents 章节，GitHub 的旧 coding-agent 文档页已 301 跳转到新地址。任何写到按钮位置的教程都在倒计时过期，所以这一节只写分工与边界：界面会变，方法论不变。

Copilot：编辑器里的 agents 体系。 分工句在第一节引过了：你描述结果、评审改动、验证产物；agent 在 VS Code 内跨文件找代码、做修改、跑检查，工作机制叫 agent loop，推理、行动、验证循环往复。这个机制的雏形早在 2025-02-24 的 agent mode 预览公告里就写明了：它响应编译与 lint 错误、监视终端与测试输出、自动修正并循环直到任务完成。嵌入式可用性有两条官方事实打底：C/C++ 是 VS Code 的一等支持语言，Microsoft 官方 C/C++ 扩展提供 IntelliSense 与调试，Copilot 在 C/C++ 工作流内提供编码建议；Copilot code review 自 2025-04-23 起支持 C/C++，官方 changelog 称覆盖 PR 中逾 90% 的常见文件类型。另有云端形态：Copilot cloud agent 在临时云开发环境里改代码、跑测试与 linter，官方硬限制是单仓库、单分支、每任务一个 PR、每次会话上限 59 分钟。云端跑得了测试跑不了你的板子，59 分钟的会话上限也装不下一次完整的上板排查，这两条限制本身就是「不能替你上板」的官方注脚。

Claude Code：终端里的 agentic coding 工具。 官方仓库 README 的定位句（截取能力部分）："an agentic coding tool that lives in your terminal, understands your codebase, edits files, runs commands"。一个引擎四种表面：terminal、IDE 扩展（VS Code、JetBrains）、desktop、web，官方口径是你的 CLAUDE.md、设置与 MCP servers 在四种表面间通用。CLAUDE.md 是放在项目根的 markdown 文件，每个会话启动时读取；官方同时明示它把这类文件当上下文而非强制配置，想硬拦某个动作要用 hooks 这类机制。软约束不是硬门禁，这条分界线 A3、A4 会反复用到。另有一个对嵌入式极顺手的性质：遵循 Unix 哲学、可管道化，`tail -200 app.log | claude -p "分析这段日志"` 是官方文档给出的合法用法；把 app.log 换成串口抓回来的日志，就是 A3 工具流的入口。

Cursor：整款产品定位成 coding agent。 官方文档的能力分句：理解你的代码库、规划并构建特性、修 bug、评审改动。Agent 由三构件组成：Instructions（系统提示与规则）、Tools（编辑文件、跑终端命令、浏览器等）、Model；官方明示一个任务内工具调用次数无上限，运行中可以反问澄清、可以排队追加指令。产物管理有一条官方口径值得照抄：Checkpoints 是会话期间的本地代码快照，只用于撤销 Agent 的改动，官方明说永久版本控制请用 Git。agent 产物必须进你自己的版本控制，这条纪律与工具无关。安全口径必须引原文：运行模式分 Auto-review、Allowlist、Run Everything 三档，Auto-review 下白名单内的命令直跑、其余尽量进沙箱、剩下的交后端分类器判断，而 "Auto-review is not a security boundary"，官方明说分类器会犯错。权限闸门不能外包给一个会犯错的分类器，A3 的权限边界一节由此展开。

MCP：不是工具，是协议。 最短定义来自 Claude Code 官方文档："The Model Context Protocol (MCP) is an open standard for connecting AI tools to external data sources." 规范侧的事实清单（协议版本 2026-07-28）：消息格式 JSON-RPC 2.0；架构分三方，Hosts 是发起连接的 LLM 应用，Clients 是 host 内的连接器，Servers 是提供上下文与能力的服务；server 向 client 暴露三类原语：Resources（数据）、Prompts（提示模板）、Tools（可执行能力）。安全原则写在规范里：调用任何工具前 host 须获得用户明示同意；工具代表任意代码执行，须谨慎对待。生态口径：官方在册 reference servers 仅 7 个（Everything、Fetch、Filesystem、Git、Memory、Sequential Thinking、Time），官方明示其定位是教学示例而非生产方案，更多服务器走 MCP Registry。治理口径：2025-12，MCP 由 Anthropic 捐赠给 Linux Foundation 下设的 Agentic AI Foundation（2025-12-09 成立），与 Block 的 goose、OpenAI 的 AGENTS.md 同为创始项目。一个协议有了中立治理与官方注册中心，它就更像 USB 口而不是私有排线，这是它进地图的理由。

三类原语用嵌入式的手翻译一遍：Resources 是数据，你的手册 PDF、原理图导出文本、仓库文件都可以作为 Resource 进模型的上下文；Prompts 是提示模板，A1 的三段式工单可以固化成一个 Prompt 原语全组复用；Tools 是可执行能力，跑构建、读串口日志、查寄存器头文件都能包成 Tool。连接拓扑也写在官方架构页里：host 为每个 server 建一个 client，本地 STDIO server 通常只服务一个 client，远程 Streamable HTTP server 通常服务多个。这套拓扑决定了工具权限的颗粒度：每个 server 的能力清单独立授权，A3 讲权限边界时会用到。

四格速览：

工具	官方定位（一条）	工位形态	核心边界
Copilot	VS Code agents 体系，agent loop = 推理 → 行动 → 验证	编辑器内 + 云端 agent	输出可能有错，评审与测试是你的活
Claude Code	住在终端的 agentic coding 工具，一个引擎四种表面	终端为主，IDE/desktop/web 同引擎	CLAUDE.md 是软约束，不是硬门禁
Cursor	整款产品即 coding agent，Agent = Instructions + Tools + Model	编辑器即 agent	Auto-review is not a security boundary
MCP	连接 AI 工具与外部数据源的开放协议	协议，不是产品	7 个 reference servers 是教学示例；工具调用须用户明示同意

工位选择按工作形态对号：

你的主战场	先看哪格
日常在 VS Code 里写代码，偶尔甩后台任务	Copilot
日常在终端，日志与脚本要能管道化	Claude Code
想让编辑器本身就是 agent	Cursor
要让模型读到手册、仓库、串口日志	MCP（搭配任一工位）

连起来读：前三格是结对工程师的三种工位，选哪个看你的编辑器与终端习惯；MCP 是它们接外部世界的共用插座标准，手册 PDF、串口日志、构建系统都能经这只插座进模型的手。消费侧怎么用这只插座，在 A3。

地图的用法两条。其一，选型按工作形态不按热度：热度是别人项目的答案，工作形态才是你的问题，每天坐在哪就先看哪格，工位换了再换格，地图跟人走。其二，查证按来源卡不按记忆：本节每个断言都对应附录 E 里的一张来源卡（官方文档 URL + 访问日期 2026-10-27），界面与条款改版时以官方当前文档为准。这一节的时效口径与全书一致：界面会变，方法论不变。

三份互不隶属的官方文档（VS Code agents 文档、GitHub Copilot 文档、Claude Code 仓库 README）措辞指向同一个分工：模型出方案，工具作用于环境，人负责验收。三个来源互相印证，这本身就是本章工作观的外部证据：结对而非代笔，不是本书的口味，是工具提供者自己写下的边界。

这一节刻意不教安装与配置，原因有二。其一，四家界面都在高频改版，安装步骤的保质期按月计，官方文档永远比任何二手教程新。其二，装哪个工具不是本章的决策，怎么用才是：工位可以换，终审工位不能撤。装好之后的第一件事也不是跑起来，是把这个工具的红线条款找出来读一遍，每家的文档里都写着，位置不同，意思相同。

云端与本地也值得分一眼。Copilot cloud agent 这类形态把「改代码、跑测试」搬进了临时云环境，适合边界清晰、验收可自动化的任务；嵌入了上板环节的活留在本地，因为终审工具（烧录器、示波器）接在你的机器上。云的干云的，桌上的干桌上的，分工界限与「能/不能清单」同一条线。

五、为什么还要学寄存器和 RTOS

回到本章开头的问题：AI 都能写出能跑的代码了，我为什么还要学寄存器和 RTOS？

现在可以给出完整答案了。**终审需要判据，判据就是寄存器与 RTOS 本身。** 模型交出 `RCC->APB1ENR |= (1<<5)`，编译过、注释工整、语气自信。学过 S2 的人一眼看到 GPIOF 在 AHB1 不在 APB1；学过 S3 的人知道共阳 LED 要低电平点亮，BSRR 该写高 16 位；学过 F0 与调度器原理的人知道中断上下文没有任务控制块，调阻塞函数是死锁配方。这些「一眼」不是天赋，是前面章节一篇一篇攒下的判据。没有判据的审计员，审什么都不成立。

AI 把「写」的成本打到接近零，反而把「审」的价值抬到最高。 它生成越快，单位时间堆到你终审工位上的代码越多。软件世界的终审工具是测试与类型系统，嵌入式还要再加三样：手册、原理图、示波器。判据的厚度，就是你在 AI 时代的不可替代性。

看个具体场景。模型给你写了一段 UART 中断服务函数，里面调了一个带超时等待的队列接收，注释写着「等待数据到达后处理」。代码能编译，逻辑读起来通顺，注释甚至很贴心。学过 F0 的人看到这里警报已经响了：中断上下文没有任务控制块，阻塞调用会让调度器不知道该切给谁，结果是死锁或内存损坏。没学过的人看到的是一个贴心的助手帮你处理了等待逻辑。同一段代码，两种读者，两种结局：一个在对账时拦下，一个在凌晨的生产现场追查偶发死机。终审权不是抽象的权力，是这种具体场景里的拦截能力。

判据的库存长什么样，给你一张审计员检查单。寄存器侧：总线归属（§2.3 表 1）、使能位（§7.3）、位定义与偏移（§8.4）、时钟先行。RTOS 侧：中断上下文没有 TCB，不能调阻塞函数；队列传数据副本、信号量传事件通知，选型不同结局不同；任务栈要留水位余量；看门狗超时按最坏执行时间定，不按愿望定。这些条目没有一条是 AI 时代的新知识，全部是前面章节的老功夫。新的是使用场景：它们从「你写代码时的规范」变成了「你审代码时的拦截网」。

还有个更实际的角度：工具换代淘汰的是打字员，留下的是懂行的人。会查手册、会对账、会上板取证的人，模型越强他的产出越大，因为草稿便宜了，终审的吞吐量成了瓶颈。这一部剩下的章节，就是把这套工作观拆成可操作的动作：A1 提示词模式，教你怎么把芯片型号、手册章节、板卡事实写进工单；A2 手册对账，给你终审的三问清单；A3 工具流，教模型自己读手册、跑构建、看串口；A4 幻觉防御，给你幻觉的类型学与决策树；A5 用这本书自己的生产线自证，这套方法是产线不是口号。

判据还有复利。每学透一个外设，你对同类外设的审计速度翻几倍；每排过一次现场，你的提示词里就多一条写给模型的约束。模型每几个月换一版，你的判据只增不减。寄存器与 RTOS 不是 AI 时代的过期知识，是你与模型分工里不可替代的那一半。

记忆锚点

全章压缩成一条可携带的判据：

> 💡 **提示 · 一句话记住**
>
> **模型出草稿，你出手册；它给假设，你给判据。学习阶段要它解释，工作阶段它出稿你审计：终审权永远在人。**

延伸：工单怎么写（三段式提示词）在 A1；终审怎么做（对账三问）在 A2；让模型自己动手的工具流在 A3；幻觉的类型学在 A4；这套工作观的产线证据在 A5。

常见坑

- **把能编译当能烧**：编译器只查语法，不认识你的板子、总线与共阳接法。AI 代码过编译只是拿到对账的入场券，终审在手册与上板，动作见 A2。
- **学习阶段让模型代写**：作业交了，脑子空转，还多一层「我好像会了」的幻觉。FAQ 里 CubeMX 的答辩口径对 AI 同样成立：工作可用，学习先手写。
- **把官方定位句当能力担保**：定位是定位，担保是担保。GitHub 红线写着输出可能有错、含安全漏洞；Cursor 写着 Auto-review is not a security boundary。把定位读成承诺，是新型迷信。
- **不给上下文就信输出**：你的原理图、共阳接法、时序预算，模型一概不知。不给上下文，等于让它猜你的板子，猜错的代价你付。上下文三段式怎么写，在 A1。
- **追着界面记按钮位置**：四家官方文档都在高频改版，2026-10 的界面截图到明年就是考古资料。记分工与边界不过期：界面会变，方法论不变。
- **让模型自己验证自己**：agent loop 里的机器侧验证是内环，只能证明代码在它触达的环境里表现如何。内环全绿不等于外环过关，外环是手册、原理图与上板。
- **实验与对账不留档**：三次一致性实验的答案、对账发现的问题、上板的现象，都是可复用的证据。进版本控制或实验笔记，下次同型号直接调用；不存档等于每次都从零开始认识同一个模型。

短自测

五题全答得出，本章才算闭环；答不出的回到对应节复读。

1. 「模型给假设，人给判据」。嵌入式开发里的「判据」具体是哪三样东西？终审权在人的官方出处是哪几句？

> ▼ **看答案**
> 判据三样：手册（RM0090 的位定义，如 §7.3 的 RCC 使能位、§8.4 的 GPIO 位定义）、板卡原理图（共阳共阴、接线）、上板现象（灯亮不亮、波形对不对）。官方出处三句：VS Code agents 文档「You describe the outcome, review the changes, and verify the result.」；GitHub 文档「Review and test the output before using it in production.」与「You remain responsible for deciding which changes reach your codebase」。分工的落点一致：评审与验证是人的活。

2. 能/不能清单各四条是什么？它们的共同分界线是哪一条？

▼ 看答案
能：写样板代码、解释报错、写测试、起草寄存器代码（草稿必须过对账）。不能：为你的事故负责、知道你的原理图、替你上板、替你做工程取舍。分界线：这件事的正确性定义在公共知识里，还是在你的板子与你的责任里。中间地带只有一类：手册是公共的，但你的板子怎么用这些位是你的，所以寄存器代码永远落在能侧、永远带对账条件。

3. 学习阶段为什么「要它解释，不要它代写」？用 FAQ 里 CubeMX 的问答说明。

▼ 看答案
FAQ 口径：工作中能用 CubeMX 这类图形化工具，学习阶段先手写一遍；工具把会出错的决策藏起来，你先自己错过一次，之后才看得懂它替你做了什么选择。AI 代写把这个坑放大两级：生成结果没有确定性，还配一本正经的注释，让你误以为看懂了。学习阶段四种正确用法：讲解员、出题官、答辩对手、错题分析员，共同点是把产出物留在你的脑子里，检验标准是合上聊天窗能重写、能讲给别人。

4. 各用一条官方定位说清 Copilot、Claude Code、Cursor、MCP 的区别。

▼ 看答案
Copilot：VS Code agents 体系内的结对者，机制是 agent loop（推理 → 行动 → 验证循环）。Claude Code：官方 README 定位「住在终端的 agentic coding 工具，理解代码库、编辑文件、运行命令」，一个引擎覆盖 terminal / IDE / desktop / web 四种表面，CLAUDE.md 启动读取。Cursor：整款产品定位 coding agent，Agent = Instructions + Tools + Model，官方明示 Auto-review is not a security boundary。MCP：不是工具是协议，连接 AI 工具与外部数据源的开放标准，JSON-RPC 2.0，Hosts/Clients/Servers 三方，Resources/Prompts/Tools 三类原语，协议版本 2026-07-28。

5. 「GPIOF 挂哪条总线」三次实验的结论是什么？为什么说结论不是「模型不行」？

▼ 看答案
结论：模型输出是分布，不是事实。同一模型、同一问题、不同会话，答案可能不一致，而每份答案的语气一样自信。这不是「模型不行」：它对最常见写法的拟合近乎完美，但 GPIOF 这类事实恰好落在幻觉高发区，全网示例里 APB 总线出现频率高，总线归属张冠李戴的概率不低（A2 第一节有实锤案例）。分布是这类工具的性质而非缺陷；把分布收敛成事实的动作叫对账，在 A2。

对照表：本章概念 → 仓库与上游落点

概念	落点
模型给假设、人给判据	VS Code agents 文档分工句「You describe the outcome, review the changes, and verify the result.」（附录 E 来源卡 ai-copilot-vscode-overview，访问 2026-10-27）
终审权在人	GitHub 文档红线「Review and test the output before using it in production.」「Logs do not replace your own review and testing.」（ai-copilot-github-cloud）
agent loop 定义	VS Code agents 文档：推理 → 行动 → 验证的循环（ai-copilot-vscode-overview）
嵌入式 C 可用性	VS Code C/C++ 语言页（ai-copilot-vscode-cpp）；Copilot code review 2025-04-23 changelog（ai-copilot-ccr-cpp）
Claude Code 定位与四表面	官方 README（ai-claude-code-repo）+ overview 文档（ai-claude-code-overview）
CLAUDE.md 启动读取、软约束口径	Claude Code memory 文档（ai-claude-code-memory）
Cursor Agent 三构件	Cursor Agent 文档（ai-cursor-agent）；定位句（ai-cursor-docs）
Auto-review 非安全边界	Cursor Run Modes 文档（ai-cursor-run-modes）
MCP 规范事实	MCP spec 2026-07-28（ai-mcp-spec）+ 架构页（ai-mcp-architecture）+ servers README 7 个 reference（ai-mcp-servers）
MCP 治理事实	Anthropic 2025-12 捐赠公告，Linux Foundation AAIF（ai-mcp-anthropic-donation）
实验靶子 GPIOF/AHB1	S3 板卡事实（GPIOF = 0x40021400、AHB1ENR bit5）；RM0090 §2.3 表 1、§7.3
学习阶段用法哲学	全书 FAQ「能用 CubeMX 吗」答辩口径：工作可用，学习先手写
提问前自查三问	本章第二节；收货后对账三问在 A2
审计员五步环	本章第三节（下工单、收草稿、对账、上板、回喂）；回喂格式细节在 A2 证据三元组
中断拦截案例	F0：中断上下文没有 TCB，阻塞调用即死锁配方
终审动作	A2 手册对账：名/位/序三问清单

你做到了

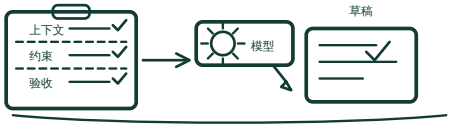
- 能当面回答「AI 都会写代码了为什么还要学寄存器」：终审需要判据，判据就是寄存器与 RTOS 本身；
- 手里有一张能/不能清单，下次提问前知道哪类活该甩给它、哪类决定必须自己拍板；
- 能各用一条官方定位向同伴说清 Copilot、Claude Code、Cursor、MCP，并且知道去哪里查当前版本（附录 E 来源卡，访问日期 2026-10-27）；
- 分得清两个阶段的用法：学习阶段它当讲解员，工作阶段它当草稿员，你始终坐在终审工位；
- 做完了三次一致性实验，亲眼见过「分布不是事实」，对账的手已经开始痒。

✅ 工作观立住了：它出假设，你出判据。下一站：**A1 提示词模式**，学怎么下一份它没法自由发挥的工单；或直接进 **A2 手册对账**，拿 RM0090 给它上一课，这门课它迟早要补。

通往单片机之路第 90 章

A1 提示词模式：三段式，把决定权从模型手里拿回来

难度 ★★☆☆☆ · 约 40 分钟



本章罗盘

问题

同样一个外设需求，为什么有人一句话搞定、有人贴五十行还被模型带偏？

交付

三段式提示词模板（上下文/约束/验收）+ 同一霸道者点灯任务的弱强提示对照实录 + 模型「替你做主」决定清单。

边界

工具深度配置在 A3，对账验收细则只引 A2 不展开；提示词中文书写、术语保留英文原词；案例全部基于霸道者点灯任务，不评价具体产品优劣。

依据锚点 `a1` · `ai-cursor-agent` · `ai-claude-code-overview` · `mcu-code`

> 🎯 同样一个点灯需求，有人发给模型三行字，拿回能用的代码；有人贴出五十行描述，照样被带进沟里。差距不在字数，也不在模型，在提示词里有多少决定是你亲手做的。模型讨厌留白：你没说的，它替你补；它替你补的每一个决定，都是一颗可能的雷。这一章给你三段式——上下文、约束、验收，把签字的笔一段一段拿回来。

本章精髓

1. **弱提示的本质是让渡决定权**。你没指定的总线、极性、顺序、参数，模型不是不知道，是替你做主：它选全网最常见的写法，不选你这块板子的正确写法。被带偏不是模型坏，是该你签的字让它代签了。
2. **三段式各堵一个缺口**。上下文堵事实缺口：给它查得到的锚点（芯片型号、手册章节号、板卡接线）；约束堵边界缺口：封死自由裁量（资源上限、时序纪律、禁用项）；验收堵终裁缺口：定义做完长什么样（可编译、上板现象、对账动作）。
3. **提示词是迭代出来的，不是憋出来的**。每一轮把模型踩的雷翻译成证据三元组（实测、手册出处、板卡事实）回喂，再固化成提示词里的新条款。提示词跟代码一样进版本库，越改越短，越短越准。

- **能理解**：为什么同一个需求，有人三行提示词搞定、有人贴五十行照样被带偏；模型替你做主的决定落在哪几个类别；三段式各自封死哪类风险。
- **能用**：把任意外设需求写成上下文、约束、验收三段式；拿到模型输出先数它替你签了哪些字，再决定信几分；每一轮用证据三元组迭代提示词本身，而不是只改它的回答。

先修

- S3 GPIO：点灯全流程与板卡事实，本章提示词的靶子与素材库；
- S2 RCC 时钟树：AHB1 与 APB1 的总线归属，上下文段的常客；
- A2 手册对账：四类幻觉实锤与证据三元组，本章只引用结论，对账细则以它为准。

先跑起来（A/B 对照实验，10 分钟）

手边有任意一个代码模型就能做。两组对话，必须开两个独立会话：在同一个会话里连着问，它记得你上一轮说过的话，对照就脏了。

发之前先押个雷数：同一句提示，一份原样发，一份补上那行板卡事实，两边的雷会差多少？差不多、少几颗，还是一颗不剩？押完再发，两份输出摆在一起对答案。

实验 A：原样发送这句弱提示。

> 用寄存器方式初始化 STM32F407 的 GPIOF 引脚 6，推挽输出，点亮 LED。

实验 B：新会话里发同一句提示词，只在末尾补一行板卡事实。

> 用寄存器方式初始化 STM32F407 的 GPIOF 引脚 6，推挽输出，点亮 LED。板子是霸道者 F407，PF6 接红灯，共阳接 3.3V，低电平点亮；GPIOF 基地址 0x40021400，时钟使能位是 RCC_AHB1ENR bit5。

两份回答都先别烧。对照 A2 的四类雷表，各打一遍勾：总线选错没有？电平反了没有？顺序乱没乱？然后填这张便签：

对照项	实验 A	实验 B
中的雷（颗）		
它替你做的决定（个）		
它向你提问（次）		

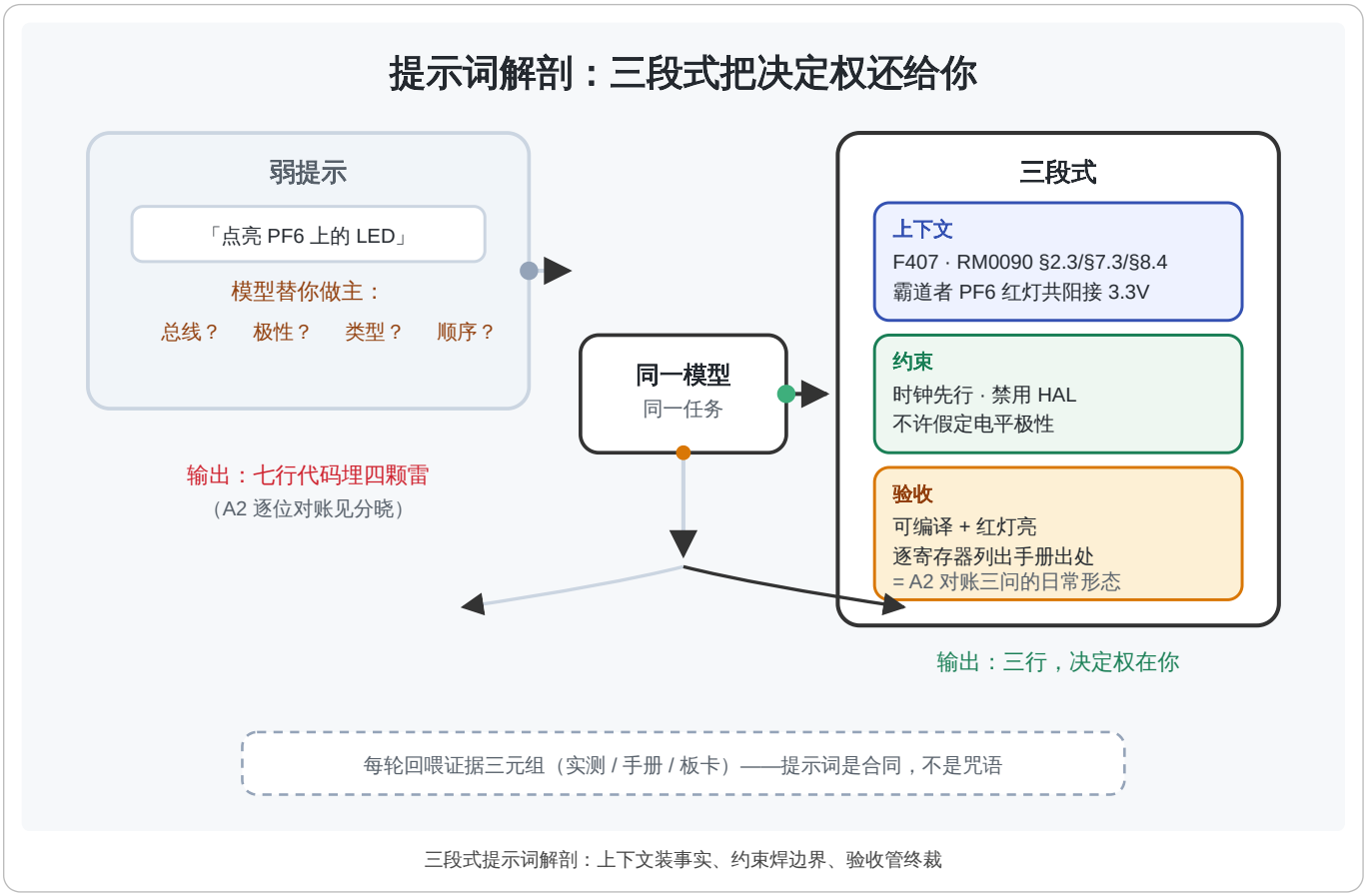
判读时多看一眼实验 B 的提问数：它有没有反问你没给的事实？有，说明「不确定就提问」这类条款值得常驻你的提示词；没有，说明你不能指望它自觉，第三节细说。

多数情况下，实验 B 会少掉总线与极性两类雷；如果你的结果不是这样，带着两份输出读第一节，对得上号。而如果实验 B 还在总线上栽，说明那一行事实里缺了总线锚点，这正是三段式要系统化的东西。一行板卡事实能换掉几颗雷，你自己数。

这张便签留着，读完第二节再回来对照一次：三段式就是把实验 B 的那「一行」，扩写成三段。

动画：三段式解剖

一条外设需求从左边进来，剖面分三层：上下文层装事实（芯片、手册章节号、板卡接线），约束层焊边界（资源、时序、禁用项），验收层站终审（编译、上板、对账）。弱提示三层全空，模型在每层替你签字；三段填满，签字笔回到你手里。



三段式提示词解剖：上下文装事实、约束焊边界、验收管终裁

小节结构

一、弱提示实录：一条回答里，模型替你签了几个字；二、三段式解剖：上下文、约束、验收各管什么；三、强提示实录：同一任务，输出变了什么；四、替你做主决定清单：通用审计入口；五、迭代纪律：证据回喂，提示词进版本库。

一、弱提示实录：模型替你签了多少字

先承认，这句提示词不算烂：

> 用寄存器方式初始化 STM32F407 的 GPIOF 引脚 6，推挽输出，点亮 LED。

有芯片型号，有引脚号，有输出类型，比「帮我点个灯」强得多。但它把至少四个决定留白给了模型。

选点灯当靶子不是偶然：它是你在这本书里跑通的第一个外设，每一个位都已经被你自己的手对过账，模型说错什么你手里都有判决书。拿熟透的东西试提示词，才分得清是它的错还是你的不懂。模型的典型回答长这样（注释是它自己加的，雷也是它自己埋的；逐雷拆解在 A2 第一节）：

```
/* 点亮 PF6 上的 LED */
void led_init(void)
{
    /* 1. 使能 GPIOF 时钟（GPIOF 挂在 APB1 总线上） */
    RCC->APB1ENR |= (1UL << 5);          /* 雷 1：总线归属 */

    /* 2. PF6 推挽输出 */
    GPIOF->MODER   |= (1UL << 12);       /* 模式：输出 */
    GPIOF->OTYPER  |= (1UL << 6);        /* 雷 2：开漏 */
    GPIOF->OSPEEDR |= (3UL << 12);       /* 100MHz */

    /* 3. 拉高 PF6 点亮 LED */
    GPIOF->BSRR = (1UL << 6);            /* 雷 3：位语义 */
}
```

别急着挑雷，这一节先数它替你签的字。读下面这张表时，盯住中间那列：每个决定本来都该是你的。

#	模型替你做的决定	它的典型选择	对应的雷（A2 分类）
1	GPIOF 挂哪条总线	APB1，教程里的高频写法	总线归属幻觉：实为 AHB1，RCC_AHB1ENR bit5
2	什么电平算点亮	置位输出高，写 BSRR 低 16 位	位语义幻觉：共阳要拉低，该写高 16 位 bit22
3	推挽还是开漏	OTYPER 写 1，开漏	硬件语境幻觉：你明说了推挽，它照写开漏
4	先开时钟还是先配寄存器	看运气，这次恰好没炸	顺序幻觉：时钟先行是铁律
5	输出速度档	OSPEEDR 拉满 100MHz	不炸，但点灯用不上：决定权被顺手拿走

注意第五行跟前四行不一样：速度档这个决定没炸。决定与雷是两回事，替你做主本身就是问题，哪怕这次结果碰巧能用。因为它下次还会替你做主，而你不一定每次都这么好运。

它的选择逻辑也值得看清：APB1 是教程高频总线，置位点亮是教程高频写法，开漏是 I2C 场景的高频配置。每一个决定都在拟合「最常见」，因为你的提示词里没有放任何能抵消分布的筹码。上下文段的全部意义，就是往天平上放筹码。

三个观察。

观察一：**连明约束都会被吞**。提示词里白纸黑字写了「推挽」，模型照写开漏。弱提示里，你的要求只是它训练分布里的一个弱信号，高频写法一冲就没了。所以光「提要求」不够，要求得变成可检查的条款，这是约束段与验收段存在的理由。

观察二：**它从不为留白提问**。共阳还是共阴、挂哪条总线，这些信息它确实没有，但它选择猜，不选择问。猜的判据是「全网最常见」，不是你的板子。你不给事实，它就还你一个最像答案的答案。

观察三：**留白面积与含雷量几乎成正比**。A2 的四类雷换一个说法，就是四个「你没签、它代签」的决定。提示词每空一处，模型就多一次替你签字的机会。

这个比例关系就是本章的工作假设：想降含雷量，不必换模型，先把留白面积砍下来。三段式，就是砍留白的三刀。

那贴五十行长提示为什么也救不了你？因为它经常是这种（浓缩示意）：

> 我有一块霸道者 F407 开发板，去年买的，上面有个 RGB 灯。我以前学过 51，现在想学寄存器编程，听说要点灯得先配 MODER 寄存器，也有人说直接用库函数方便。我想用寄存器的方式，把 GPIOF 第 6 个脚上的红灯点亮，请给我完整代码，注释写详细点，谢谢。

把这段话按三段式过一遍筛子：

三段式要的东西	这段五十行里有没有
芯片型号	有：STM32F407
手册章节号	无，它只能凭训练记忆编
板卡事实（共阳、低电平点亮）	无，「RGB 灯」三个字撑不起极性
资源上限	无，它顺手多配几个引脚都不犯规
时序纪律	无，时钟先行全凭运气
禁用项	无，「想用寄存器」是愿望不是禁令
可观察验收	无，「点亮」没接任何终裁

七项里缺五项。字数花在叙事上，约束一项没落地：没有章节号，它编；没有共阳事实，它猜；没有禁用项，它顺手就上库函数；没有验收，它把过编译当终点。「请」「谢谢」「写详细点」占用的是你的输入预算，不是它的约束力；模型不需要客气，需要条款。

弱不是短，是缺结构。三段式治的不是「提示词太短」，是「三段全缺」。反过来，三行字的提示词只要三段齐全，也比五十行叙事强。

把这句弱提示的缺口摆开，正好三个：事实缺位（板卡接线与手册锚点一个没给）、边界缺位（没纪律没禁用项，它想用什么招用什么招）、终裁缺位（没说怎么算对）。三个缺口，正好引出三段式。

这三个缺口还是诊断工具：拿到别人写好的提示词，先看三段齐不齐，比逐字挑毛病快得多。

二、三段式解剖：上下文、约束、验收

三段式不是格式癖，一段堵一个缺口：上下文堵事实，约束堵边界，验收堵终裁。

顺序也有讲究：事实在前，锚点先立住；约束居中，在事实圈定的范围里焊边界；验收殿后，终裁标准定义这趟活的终点。

上下文：给它查得到的锚点。上下文段只装三样东西。

一是芯片型号：STM32F407，要写到后缀字母（比如 STM32F407ZGT6）：容量与封装由后缀决定，写全了，资源类回答才不会按错档次。光有型号还不够，同型号不同板子的接线天差地别，所以还要后两样。

二是手册章节号：RM0090 §2.3 表 1 管存储器映射（哪个外设挂哪条总线），§7.3 管 RCC 使能位，§8.4 管 GPIO 寄存器位定义。给章节号等于给书签：模型对手册内容有训练记忆，索引却不可靠，你给了锚点它才有地方站稳；你不给，它就编一个自信满满的页码回赠你。

引章节号，不引页码：页码随手册版本与 PDF 排版漂移，章节号稳定可查。模型编页码是最容易穿帮也最常见的一种幻觉（完整类型学在 A4 幻觉防御），这里只留一条：章节号是成本最低的防幻觉件。

三是板卡事实：霸道者 F407，PF6 接红灯，共阳接 3.3V，低电平点亮；GPIOF = 0x40021400，RCC = 0x40023800，AHB1ENR 偏移 0x30、bit5（事实出处 S3）。这类事实最特殊：手册里没有，全网也没有，只活在你的原理图里。它是模型唯一不可能自己补上的上下文，所以必须你亲手给。

板卡事实从哪来？三个来源：原理图（共阳接 3.3V 这种接线事实的唯一权威）、板卡手册与丝印（引脚编号、跳线默认位置）、你自己上板测过的读回值（S3 里读 IDR 的进阶实验就是动手取证）。写进上下文之前，每条事实先问一句：它有出处吗？没出处的事实，跟模型的幻觉是同一物种。

给事实时顺手给推导链，效果比裸事实好：「共阳接 3.3V，所以引脚低电平点亮」，比只说「低电平点亮」多给模型一步推理支点，它自己就能推出该写 BSRR 高 16 位，而不是死记一个结论。给结论也给理由，是上下文段的进阶写法。

约束：负面清单优先。约束段封死模型的自由裁量，三类条款。

资源上限：只用哪个引脚、哪个定时器、哪段内存，其余不许动。「只配置 PF6，不动 GPIOF 其他引脚」，少了这句，它可能顺手帮你把 PF7、PF8 也配了。

资源上限还有第二种形态：性能与功耗预算。主频跑多少、电流花多少，点灯任务用不上，换成电机控制或电池供电设备就是硬条款。约束段的原则是：你在乎什么，就封什么；没封的，模型按最省事的来。

时序纪律：时钟先行，初始化顺序显式写死。这条是 S3 的铁律（「时钟永远先于一切」，新手第一大坑），也是顺序幻觉的高发区。写进约束，等于把纪律从「你应该记得」变成「你被要求」。

禁用项：不许用 HAL 与标准库，不许假设电平极性，不许省略对账表。禁用项的逻辑很直接：模型的默认路径是训练分布里的高频路径，嵌入式 C 语料的高频路径是库函数。正面描述（「用寄存器方式」）是给它指路，禁用项（「禁用 HAL」）是把老路堵死；指路会走丢，堵路没得走。第一节被吞的那条明约束，吞的正是指路型条款。

禁用项还能封写法风格：不许用宏魔法藏起寄存器地址，必须用 CMSIS 原名，方便你对账时一眼对上手册。风格条款看着软，审计时最省时间。

约束不是要把模型捆死。三段式封的是「不许碰的」，剩下的空间全是它的：变量怎么命名、注释怎么措辞、先写哪个辅助宏，随它发挥。边界内给自由，比处处微管理好管，也比处处留白安全。

验收：定义做完长什么样。验收段回答「这活怎么算干完」，三层缺一不可。

可编译：过 arm-none-eabi-gcc 交叉编译（工具链见 B0）。注意是交叉工具链，不是宿主机 gcc：语法哪里都一样，地址映射只有目标平台认。这是最弱的一层，只证明语法合法，但它是门槛，先写上。

上板现象：红灯亮。终裁在硬件，不在编译器，也不在模型的语气。现象必须可观察：「红灯亮」可以，「工作正常」不行。

对账动作：随代码附一张表，逐寄存器列出写入值与依据的手册章节号。这一条把 A2 的对账前置了一半：模型自己先把账对了，你从全手工对账降级成抽查。

对账动作只要求它交表，不等于相信它自查正确；表的意义是让你的抽查有抓手。抽哪几行、按什么风险分级，A2 第五节有现成答案。

三层验收对应三种风险：编译层挡语法错，现象层挡硬件语境错，对账层挡手册幻觉，缺哪层，哪层的风险就裸奔。对账动作写进验收还有个隐藏收益：模型为了交出这张表，写代码时就得先想依据，等于把审计成本前移到了生成时刻。

三条验收的共性只有两个字：可查。编译有退出码，灯有人眼看，表有章节号可翻。验收段的写作练习只有一道题：这条验收，明天交给同事，他能不能不看你的脸色，独立判出通过不通过？

反面教材各来一个，对照着看：

段落	写了等于没写	能用的写法
上下文	「参考手册」	「位定义查 RM0090 §8.4，使能位查 §7.3」
约束	「代码写规范点」	「禁用 HAL；只配置 PF6；时钟使能第一句」
验收	「代码要正确」	「过 -Wall 编译；红灯亮；附逐寄存器章节号表」

「代码要正确」这类写法对模型不是判据，是愿望：它没法拿「正确」自查，你也没法拿它终审。VS Code 官方文档给 agent 的分工定义就一句：You describe the outcome, review the changes, and verify the result（你描述结果、评审改动、验证结果；能力描述，访问日期 2026-10-27）。验收段，就是「描述结果」的落地处。

模板：直接抄。三段拼起来，点灯任务的完整强提示长这样：

> 【上下文】芯片 STM32F407（霸道者开发板），手册 RM0090。目标：GPIOF 引脚 6 接板载红灯，LED 共阳、阳极接 3.3V，引脚输出低电平点亮。已知：GPIOF 基地址 0x40021400，时钟使能位 RCC_AHB1ENR bit5；总线归属查 §2.3 表 1，使能位查 §7.3，位定义查 §8.4。
>
> 【约束】只用 CMSIS 寄存器直写，禁用 HAL 与标准库；只配置 PF6，不动其他引脚；时钟使能必须是第一句（时钟先行）；电平极性按共阳事实推导，不许假设；每行寄存器写操作，注释该行依据的手册章节号。
>
> 【验收】过 arm-none-eabi-gcc -Wall 编译无警告；上板红灯亮；随代码附对账表，逐寄存器列写入值与 RM0090 章节号；你不确定的事实不要猜，列成问题问我。

最后那句是整个模板的保险丝，第三节专门说它。

点灯模板换个外设照样套。通用骨架长这样，占位符换成你任务的：

> 【上下文】芯片与板卡：*。手册与章节号：*。目标外设与引脚：*。板卡事实（接线、极性、默认电平）：*。
>
> 【约束】写法边界（禁用哪些库）：*。资源白名单（只用哪些引脚、定时器、通道）：*。时序纪律（谁先谁后）：*。不许假设的事：*。
>
> 【验收】编译判据：*。上板现象（可观察、可复现）：*。对账动作（列什么表、挂什么出处）：___。不确定就提问。

填空的过程本身就是一次设计评审：哪一格填不出来，哪格就是你还没想清楚的地方。提示词写不下去，往往不是表达问题，是设计没想完。

把这张骨架存进项目仓库（比如 prompt-template.md），新需求先复制再填空。模板的存在感越低越好：最好变成你开新任务时的肌肉记忆。

一次性对话与项目级上下文。三段式贴在消息里，管一次对话；同一个项目天天用，就把它固化到工具的项目级机制里。Claude Code 的官方文档写明：CLAUDE.md 是放在项目根的 markdown 文件，每次会话开始时被读取。Cursor 的官方文档把 Agent 拆成 Instructions、Tools、Model 三构件，其中的 Instructions 层（system prompt 与 rules）就是提示词工程的落点（均为官方文档能力描述，访问日期 2026-10-27）。界面与文件名会变，方法论不变：把板卡事实与禁用项放在模型每次都能看见的地方。这些机制怎么配、权限边界在哪，归 A3 工具流，本章只立提示词本身。

判定该贴消息还是该固化，标准也简单：这条上下文，下个任务还用得上吗？用得上就进项目级文件，用不上就留在消息里。项目级文件只装跨任务稳定的东西：板卡事实、编码纪律、验收习惯。

三、强提示实录：输出变了什么

把上一节的模板原样发给同一个模型，回答的典型变化有三处。具体措辞每个模型、每天都可能不同，那是天气；你要看的是地形。

变化一：决定权回到你手里。弱提示里它替你签的五个字，总线、极性、类型、顺序、速度，这次全在提示词里签好了，它只剩执行。代码从「七行注释埋四颗雷」收敛成三行干净货：

```
RCC->AHB1ENR   |= (1UL << 5);    /* GPIOFEN：RM0090 §7.3，AHB1ENR bit5 */
GPIOF->MODER   |= (1UL << 12);   /* MODER6=01 通用输出：§8.4 */
GPIOF->BSRR     = (1UL << 22);   /* 高 16 位写 1 拉低，共阳点亮：§8.4 */
```

这三行，每一行都能在你的提示词里找到出处：bit5 来自你给的已知事实，01 来自你指的 §8.4，bit22 来自共阳事实的推导。它没有创造任何决定，这正是你要的状态：模型从决策者降回执笔人。

变化二：不确定性以提问返还，不再是硬猜。模板最后那句保险丝起了作用：这轮如果故意把板卡事实从上下文里删掉，它的典型反应是先反问，而不是替你选一个。返还的提问长这样：

> 1. LED 是共阳还是共阴？
> 2. 引脚上有没有限流电阻，阻值多少？
> 3. 初始化需要返回错误码吗，还是上电即配死？

每个问题都对应决定清单里的一类签字。提问的质量也是提示词的镜子：它问得越贴硬件（共阳、限流、返回值），说明上下文给的支点越对；它还在问「要点什么颜色的灯」，说明你的上下文跑偏了。反问不是玄学，是工具链里真实存在的一环：Cursor 官方文档给 Agent 列的工具清单里就有 ask questions，任务进行中向你反问澄清（能力描述，访问日期 2026-10-27）。

但别把反问当默认行为：它不问的时候居多，所以「不确定就提问」要写进提示词，不是写在心里。

如果保险丝失灵了呢？它没反问，直接开猜，输出一份看起来齐活的代码。处理办法有两步：先把「上下文缺事实时必须提问，不许假设」从尾句升级成验收段的明条款；再养成一个阅读习惯，拿到回答先找它提没提问，再读代码。提问数为零而上下文明明有缺口，这本身就是危险信号，跟编译零警告的新代码一样，值得多看一眼。

变化三：输出自带对账表。验收段要求了「逐寄存器列章节号」，它就得交：

寄存器写操作	写入值	依据
RCC->AHB1ENR bit5 置位	GPIOFEN = 1	RM0090 §7.3，RCC 寄存器定义
GPIOF->MODER bits 13:12	01，通用输出	RM0090 §8.4，MODER 位定义
GPIOF->BSRR bit22 写 1	PF6 输出低	RM0090 §8.4，BSRR 高 16 位复位；共阳事实来自上下文段

你在 A2 练的手工对账，从逐位重查变成抽查：它给的表里抽一行，对得上 §8.4，可信度上一档；对不上，整份退回。注意边界：表是它给的，账还是你审；表一样可能是编的，好在每行都指向手册的具体位置，这正是它可审的原因：查不着的出处，跟没有出处是同义词。对账细则不在这里展开，以 A2 为准。

两份输出摆在一起：

对照项	弱提示的输出	强提示的输出
替你做的决定	至少 4 个：总线、极性、顺序、速度	0 个，决定全在提示词里
已知雷数	4 颗（A2 第一节实锤）	0 颗（抽查口径）
向你提问	0 次，有缺口就猜	漏事实时反问
手册章节号	0 处，注释全是自然语言	逐行引用
你的活	全手工对账，加口头 patch	抽查，加迭代提示词

最后一行才是重点：强提示省下的不只是模型的错，更是你的审计成本。

还有一个隐形变化：代码变短了。弱提示下它要替你猜，猜就得铺陈，铺陈就出注释比代码长的「教科书」；上下文给足了，训练分布的引力被锚点抵消，它只写该写的三行。输出的啰嗦程度，反过来也是提示词质量的一枚仪表。

你复现实验时，拿到的具体句子多半跟本节不同：同一模型，不同日期的输出会变。盯结构三指标：它替你做了几个决定、问了几个问题、引了几处章节号。措辞天天变，这三个数是提示词质量的稳定读数。

四、替你做主决定清单：通用审计入口

点灯案例里的五个决定不是 GPIO 特产。拿到任何一段模型输出，先别问「对不对」，先问「它替我签了哪些字」。签字高发区一共六类：

类别	模型常替你签的字	收回到哪一段
硬件事实	总线归属、电平极性、上下拉、接法	上下文：原理图事实加手册章节号
资源占用	用哪个定时器、通道、DMA、引脚	约束：白名单，「只用……」
顺序依赖	初始化先后、时钟先行、解锁序列	约束：显式时序纪律
参数取值	速度档、波特率、分频系数、超时	上下文给数值，或约束给推导口径
写法风格	HAL 还是寄存器、阻塞还是中断	约束：禁用项负面清单
范围边界	顺手重构、顺手加功能、顺手改风格	约束：「只改这个函数」

拿第一节那段点灯代码过一遍这张表：硬件事实类，总线归属与电平极性两个字被代签；写法风格类，开漏顶掉你明写的推挽；顺序依赖类，这次恰好没炸；参数取值类，OSPEEDR 100MHz 顺手拉满；范围边界类，这次没越界，换个复杂需求就难说。一段七行的代码，六个类别里中了四类。

这就是审计入口的用法：先归类，再对照右列找到该收紧的段落，把决定写回提示词。

六类里有两类要特别警惕。参数取值类经常「不炸但错」：OSPEEDR 拉满点得亮灯，可你没授权它做这个选择；换到功耗敏感的场景，这就是事故。范围边界类最伤版本库：你让它改一个驱动，它顺手重排了三个文件的缩进，diff 评审成本翻倍，还可能改掉别处的行为。这两类不触发任何编译错误，恰恰只能靠提示词约束提前封死。

反过来，硬件事实与顺序依赖这两类最「响」：错了灯就不亮、板就没反应，反而好抓。评审的力气分配跟响度成反比：越安静的代签，越值得盯。

把这张表用进日常评审，流程只有三步：

- 读输出之前，先默写提示词里你签了哪些字；
- 读输出时对照六类清单，找它代签的部分，逐条归类；
- 把每一类代签翻译成一行提示词条款，写回对应段落。

三步走完，你得到的不只是这份输出的可信度评估，还有提示词的下一个版本。

再换一个任务试这张表：让它初始化串口。它替你签的字立刻变成另一组：波特率取多少、帧格式怎么定、挂哪条总线、阻塞收发还是中断收发、要不要顺带把 printf 重定向也写了。前四项分属参数取值、硬件事实、写法风格，最后一项是范围边界的经典越界。任务换了，六类不变，这就是清单能当通用入口的原因。

这张表本身也是活的：每做完一个项目，把新遇到的代签类别补进去。你的清单比任何通用模板都懂你的项目。

每次评审模型输出，都是在给提示词做增量维护，而不是在给模型挑错。这个心态换过来，评审就从苦役变成了资产积累。

五、迭代纪律：提示词进版本库

强提示不是一次憋出来的，是迭代出来的。纪律三条。

迭代单位是提示词，不是回答。模型写错总线，你回一句「不对，GPIOF 在 AHB1」，它这轮改对了；然后呢？下个任务、下个会话，它接着替你选 APB1。口头 patch 修好的是这一轮的回答，提示词里的雷一颗没排。

正确的修法是把雷翻译成条款。用点灯任务走一遍完整迭代，看条款是怎么长出来的：

轮次	这轮炸的雷	翻译成的条款	落到哪段
1	代码里混入 HAL 库调用	「禁用 HAL 与标准库」	约束
2	总线写成 APB1	「已知：RCC_AHB1ENR bit5；归属查 §2.3 表 1」	上下文
3	按共阴写法点亮，逻辑全反	「共阳接 3.3V，低电平点亮」「不许假设极性」	上下文加约束
4	收敛：输出自带对账表，抽查通过	无新增，验收段原本就有	验收

每颗雷对应一行条款，提示词就是这样长出来的。注意第 4 轮：没有新条款可长，就是收敛信号。

条款本身也有写法要求：必须可检查。「时钟使能放第一句」查一眼就知，「代码写得健壮点」查无可查。写不出检查办法的条款，跟写不出判据的验收是同一种病，删掉重写。

回喂用证据三元组。吐槽对模型是零信息，证据才有校正力：实测现象（灯不亮、引脚读回是高电平）、手册出处（RM0090 §7.3，AHB1ENR bit5 才是 GPIOFEN；APB1ENR bit5 是 TIM7EN，A2 已实锤）、板卡事实（PF6 共阳，低电平点亮）。三元组的写法与案例细则在 A2 第四节，这里只立纪律：每轮迭代必须带证据，不带证据的轮次算浪费。

提示词跟代码同仓、同评审、同演进。三段式模板存进项目仓库，或者固化进上节说的项目级上下文文件，跟驱动代码一起进版本控制。三个月后的你、后来的同事，看到的不是「当时好像这么调的」，是 commit 历史里每一行条款的来由。

条款也会过期：换芯片、换板子、换库政策，旧条款就得拿出来重审。从 F407 换到别的板子，霸道者那组事实整段作废，上下文不是一次投资。收敛的标志是提示词越改越短：每收回一个决定，模型要猜的就少一个；最终形态是三张清单，事实清单、负面清单、验收清单，一句寒暄都不剩。

提示词自己也能被评审。不管评审别人的还是三个月前自己的，问四个问题：三段齐不齐？每条验收是不是可观察？禁用项有没有落在明处？章节号随手抽一个能不能查到？四个问题有一个答不上，这条提示词就还没到入库标准。

条款有血统：从别家项目抄来的提示词，堵的是别家的雷。借鉴可以，逐条问一句「这条堵的是我的哪颗雷」，答不上的别入库。

最后划一条边界：提示词再强，压缩的也只是模型的发挥空间，替代不了对账。强提示把雷从四颗压到零颗，是这一章的全部承诺；确认它真是零颗，是 A2 的活。「提示词写好了所以对账免了」这条推论，从来不存在。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **弱提示让模型替你签字，三段式把签字笔拿回来：上下文装事实（型号、章节号、板卡接线），约束焊边界（资源、时序、禁用项），验收管终裁（编译、上板、对账）；每颗炸过的雷翻译成一行条款，提示词进版本库。**

延伸：对账三问与证据三元组的完整细则在 A2 手册对账；三段式的项目级固化（CLAUDE.md、rules、MCP 与权限边界）在 A3 工具流；模型编章节号、编位定义的幻觉类型学在 A4 幻觉防御。

常见坑

- **验收标准写「正确」二字等于没写**：「代码要正确」对模型不是判据，是愿望。验收必须可观察：编译器过不过、灯亮不亮、对账表里有没有章节号。写不出可观察判据的验收段不如删掉，至少不制造虚假的安全感。
- **约束漏禁用项，模型默认走 HAL**：嵌入式语料的高频路径是库函数，正面描述是指路，禁用项是堵路；指路会走丢，堵路没得走。少了「禁用 HAL」，交上来的「寄存器代码」里很可能藏着库调用。
- **上下文不给手册章节号，模型开始编页码**：「参考手册」四个字给不了锚点，它回赠你自信满满的虚构引用。给 §2.3 表 1、§7.3、§8.4 这种能查证的锚点，成本一行字，省的是整轮对账。
- **口头 patch 两轮就当提示词合格**：「不对，重来」「把 BSRR 改一下」，修的是这一轮的回答，提示词里的留白原样保留，换个任务接着炸。每轮的修法只有一种：把雷翻译成条款写回提示词。
- **板卡事实觉得显而易见就不写**：共阳接法在你脑子里、在原理图上，唯独不在模型的训练数据里。你越觉得「这还用说」的事实，越是必须写进上下文的事实。
- **把三段式当咒语背，不理解各管什么**：照着模板填得满满当当，上下文里抄进半本手册，约束里写满不相干的禁令。三段式不是仪式，是分工：每段只堵自己的缺口，抄来的条款不堵你的雷。
- **把强提示当终点，不烧就合并**：三段式把雷压到零，是概率上的零；烧上板之前，「零颗」只是期望值。验收段里「红灯亮」三个字不是修辞，是终审，终审只能发生在硬件上（A2 闭环的最后一环）。

短自测

1. 三段式是哪三段？各自堵住弱提示的哪个缺口？

> **▼ 看答案**
> 上下文、约束、验收。上下文堵事实缺口：芯片型号、手册章节号（RM0090 §2.3 表 1、§7.3、§8.4）、板卡事实（霸道者 PF6 共阳、低电平点亮）；约束堵边界缺口：资源上限（只配置 PF6）、时序纪律（时钟先行）、禁用项（禁用 HAL、不许假设极性）；验收堵终裁缺口：可编译、上板现象（红灯亮）、对账动作（逐寄存器列章节号）。三个缺口来自弱提示实录的观察：事实缺位、边界缺位、终裁缺位。

2. 弱提示「用寄存器方式初始化 STM32F407 的 GPIOF 引脚 6，推挽输出，点亮 LED」里，模型替你做了哪四个以上的决定？各对应 A2 的哪类幻觉？

> **▼ 看答案**
> 至少四个。总线归属（它选 APB1，实为 AHB1）：总线归属幻觉；点亮电平（它写 BSRR 低 16 位置位输出高，共阳其实要拉低）：位语义幻觉；输出类型（你明说推挽，它照写开漏）：硬件语境幻觉，还说明弱提示连明约束都护不住；初始化顺序（先配 MODER 还是先开时钟全凭运气）：顺序幻觉。外加一个不炸但越权的：OSPEEDR 速度档拉满 100MHz，决定权被顺手拿走。

3. 为什么「验收：代码正确」等于没写？合格的验收三件套是什么？

> **▼ 看答案**
> 「正确」不是可执行、可观察的判据：模型拿它无法自查，你拿它无法终审，写与不写对输出质量没有影响。合格三件套：可编译（过 arm-none-eabi-gcc 交叉编译，最弱但是门槛）、上板现象（红灯亮，终裁在硬件）、对账动作（随代码附逐寄存器的 RM0090 章节号引用表，把你的审计从全手工降级成抽查）。三层从弱到强，缺一不可。

4. 为什么约束段里禁用项比正面描述管用？举嵌入式场景最典型的一条。

> **▼ 看答案**
> 模型的默认路径是训练分布里的高频路径，嵌入式 C 语料的高频路径是库函数。正面描述「用寄存器方式」只是指路，高频写法一冲就丢（弱提示里明写的「推挽」照样被吞）；禁用项「禁用 HAL 与标准库」是把老路直接堵死，它没得选只能走新路。最典型的一条就是「禁用 HAL」：少了它，交上来的代码里很可能藏着 GPIO_InitTypeDef 与库函数调用。

5. 每轮迭代的回喂为什么必须是证据三元组？「口头 patch 回答」和「迭代提示词」差在哪？

> **▼ 看答案**
> 证据三元组是实测现象、手册出处、板卡事实（细则在 A2 第四节）。「又错了」只给模型一个布尔值，它下轮只能换个猜法；三元组给的是可验证的更正链，它本轮就能收敛。口头 patch 修好的是这一轮的回答，提示词里的留白原样保留，下个任务同样的雷接着炸；迭代提示词是把雷翻译成条款（总线错了就加「归属查 §2.3 表 1」），雷被排掉而不是被绕开。判断标准看版本库：提示词有没有跟代码一起演进。

对照表：本章概念 → 仓库与上游落点

概念	落点
弱提示四类雷	A2 第一节实锤；RM0090 §2.3 表 1、§7.3、§8.4
板卡事实	S3：霸道者 PF6/7/8 共阳，GPIOF = 0x40021400，RCC = 0x40023800，AHB1ENR 偏移 0x30、bit5
时钟先行铁律	S3「时钟永远先于一切」；S2 总线归属
可编译验收	B0：arm-none-eabi-gcc 交叉工具链
证据三元组回喂	A2 第四节，本章只立纪律不展开
明约束被吞现象	本章第一节观察一；对策在第二节禁用项
长提示缺结构案例	本章第一节五十行筛子表
三段式通用骨架	本章第二节填空模板
验收分工口径	VS Code 官方文档 agents 总览：You describe the outcome, review the changes, and verify the result（访问日期 2026-10-27）
反问澄清能力	Cursor 官方文档 Agent 页，工具清单 ask questions（访问日期 2026-10-27）
项目级上下文机制	Claude Code 官方文档 memory 页（CLAUDE.md）、Cursor 官方文档（Instructions、Tools、Model 三构件）（访问日期 2026-10-27）；配置细节在 A3

你做到了

- 一句外设需求能拆成上下文、约束、验收三段，决定权在你手里，不在模型的训练分布里；
- 拿到模型输出先数它替你签了哪些字，再按六类清单逐类收回；
- 每轮踩的雷会翻译成条款，提示词进版本库，越改越短，越短越准；
- 别人的提示词拿来先过三段筛子，再决定抄不抄。

✅ 三段式在手，签字笔回到你手里。下一站：A2 手册对账，拿换来的输出逐位验证给它看；之后 A3 工具流，让模型自己读手册、跑构建，你从管道升回监工。

通往单片机之路第 91 章

A2 手册对账：模型写的寄存器代码，逐位验证给它看

难度 ★★☆☆☆ · 约 40 分钟

本章罗盘

问题

模型给的 RCC/GPIO 初始化能过编译，烧进板子灯不亮，怎么知道它错在哪一位？

交付

对账三问清单（名/位/序）+ 以霸道者 PF6 共阳红灯为靶子的逐位对账全表 + 证据三元组回喂句范例，含总线归属、硬件语境、位语义、顺序四类幻觉的实锤改法。

边界

对账流程以 RM0090 §2.3/§7.3/§8.4 与板卡原理图为唯一判据；点灯例子的上板现象属预期逻辑结果，读者须在自己的板子上复现；弱强提示对照与工具流分别在 A1/A3，本章不展开；不评价任何具体 AI 产品。

依据锚点 `a1 · mcu-code`

> 🎯 模型三十秒生成的 GPIO 初始化，过编译、长得像教科书、注释写得比你工整——烧进板子，灯不亮。它错在哪一位？你手里有一样它永远没有的东西：RM0090。这一章教你把「AI 生成的代码」从结论降级为假设，再用手册逐位对账，最后上板终审。对账不是不信任模型，是给模型装上游标卡尺。

本章精髓

1. **模型输出是假设，不是事实**。它对「最常见写法」的拟合近乎完美，对「你这块板子的具体接法」一无所知：总线归属、上下拉语境、共阳共阴，全是它可以一本正经编错的地方。验证的对象不是模型，是手册的位定义表。
2. **对账三问：名对不对、位对不对、序对不对**。寄存器名与总线归属查 §7.3 时钟树与存储器映射；位域值查 §8.4 寄存器位定义表；初始化顺序查「时钟先行」这条铁律。三问全过，代码才配烧上板。
3. **闭环的最后一环是回喂**。上板现象（灯亮不亮、示波器有没有波形）要回到提示词里，让模型基于证据修正，而不是基于你的吐槽修正。「实测：BSRR 高 16 位是复位」比「你写错了」对模型的校正效率高一个数量级。

- **能理解**：为什么过编译是最弱的一种「正确」；模型幻觉在寄存器层面的四种常见形态；对账的依据永远是手册章节号而不是模型自信程度。
- **能用**：拿到任意一段 AI 生成的寄存器初始化代码，按三问清单逐位对账，把发现的问题改写成可回喂的证据句。

先修

- S3 GPIO：MODER/OTYPER/BSRR 的位定义与点灯全流程（本章的对账靶子）；
- S2 总线与时钟：AHB1/APB1 总线归属与使能逻辑（幻觉高发区）；
- A1 提示词模式：三段式提示词（本章的对账素材从那里来）。

先跑起来（对账预演，5 分钟）

把下面这段提示词原样扔给你手边的任何一个代码模型：

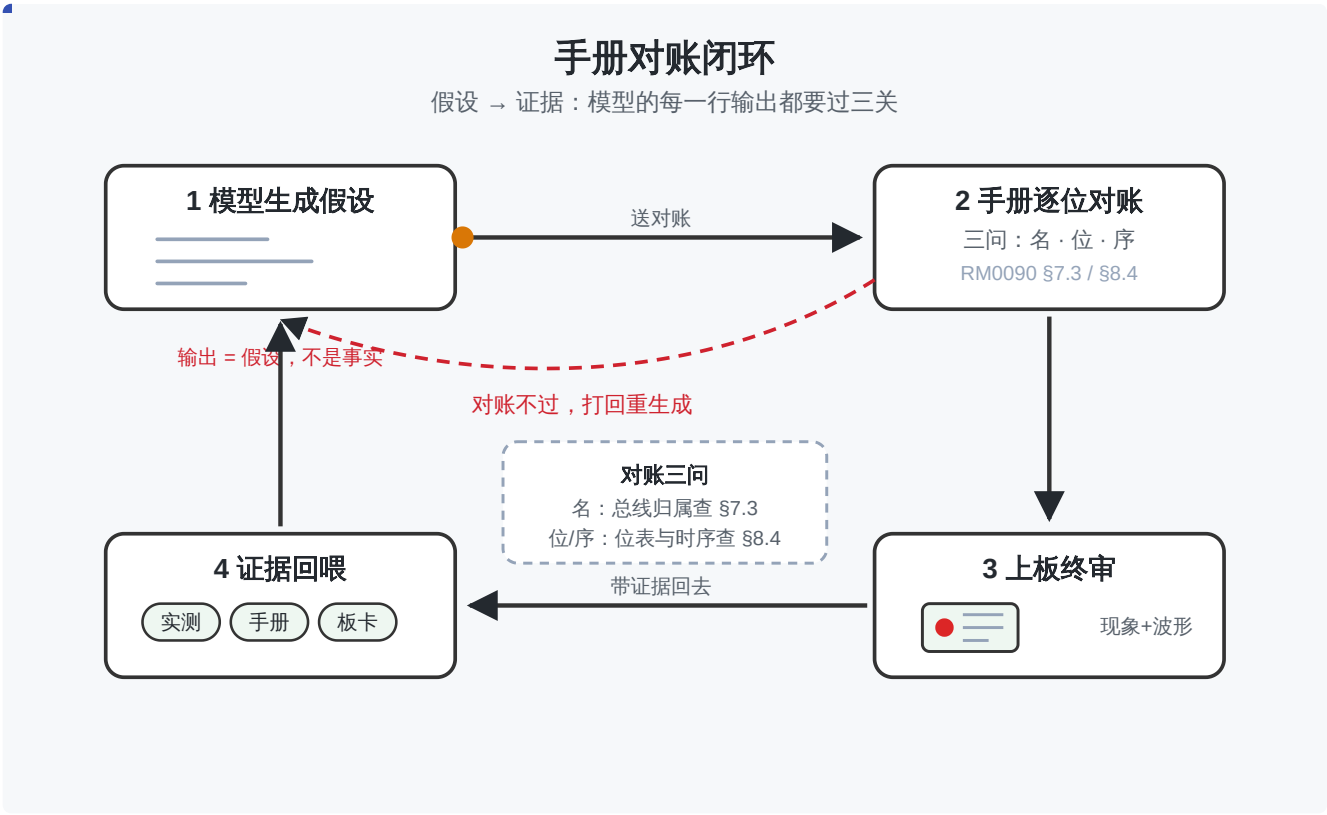
扔出去之前先押个签字数：这句话扔给模型，它会替你签几个字？一个、三个，还是全都签了？押个数再扔，拿到代码当场数。

> 用寄存器方式初始化 STM32F407 的 GPIOF 引脚 6，推挽输出，点亮接在这个引脚上的 LED，给出完整的初始化代码。

拿到代码别烧。数一下它替你做了几个你没有要求的决定：选哪个寄存器、挂哪条总线、什么顺序。这些没问你的决定，就是对账的入口。

动画：对账闭环

模型生成假设 → 手册逐位对账（RM0090 §7.3/§8.4）→ 上板终审 → 证据回喂 → 模型修正假设，循环收敛。



手册对账闭环：生成、对账、上板、回喂

对账素材

- 靶子板：霸道者 F407（GPIOF PF6 红灯，**共阳**接 3.3V，低电平点亮）；GPIOF = 0x40021400，RCC = 0x40023800。
- 手册：RM0090 §7.3（RCC，复位和时钟控制）与 §8.4（GPIO 寄存器位定义），地址与偏移以 ST CMSIS 头文件逐项核对过（见 S3 板卡事实）。
- 被对账对象：一段典型的「模型自信满满」初始化代码（下一节全文给出）。

小节结构

一、一个真实的幻觉：四颗埋在点灯代码里的雷；二、对账工具箱：手册怎么看、表怎么填；三、逐位对账实操：三问清单全表打勾；四、上板终审与证据回喂；五、对账的边界：哪些逐位、哪些抽样。

一、一个真实的幻觉

模型对这道点灯题的回答，十次里有八次长这样（注释是它自己加的，雷也是它自己埋的）：

```
/* 点亮 PF6 上的 LED */
void led_init(void)
{
    /* 1. 使能 GPIOF 时钟（GPIOF 挂在 APB1 总线上） */
    RCC->APB1ENR |= (1UL << 5);          /* 雷 1：总线归属 */

    /* 2. PF6 推挽输出 */
    GPIOF->MODER   |= (1UL << 12);       /* 模式：输出 */
    GPIOF->OTYPER  |= (1UL << 6);        /* 雷 2：开漏 */
    GPIOF->OSPEEDR |= (3UL << 12);       /* 100MHz */

    /* 3. 拉高 PF6 点亮 LED */
    GPIOF->BSRR = (1UL << 6);            /* 雷 3：位语义 */
}
```

编译一把过。逐行读，每一行都像教科书。问题在哪？四个，一个比一个隐蔽：

雷	模型写的内容	实际正确口径	幻觉类别
1	`RCC->APB1ENR |= (1<<5)`，注释「GPIOF 挂在 APB1」	GPIOF 挂在 **AHB1**，`RCC->AHB1ENR` bit5（GPIOFEN）	**总线归属幻觉**：编造一条似是而非的总线分配
2	`OTYPER |= (1<<6)`，输出类型开漏	共阳 LED 需要低电平拉电流点亮，必须是**推挽输出**（OTYPER=0）	**硬件语境幻觉**：无视「共阳接 3.3V」这一接线事实
3	`BSRR = (1<<6)` 置位	共阳 LED **低电平点亮**：要拉低该置 BSRR 高 16 位的复位位（`(1UL << (6+16))`），或者写 ODR；置位反而熄灭	**位语义幻觉**：BSRR 高 16 位复位 / 低 16 位置位的规则张冠李戴
4	（前面三颗之外）三行寄存器操作的隐含顺序	模型这段恰好顺序没错，但它有相当概率把「配 MODER」写在「开时钟」前面	**顺序幻觉**：违反「时钟永远先于一切」铁律（S3 原话）

注意一个共同点：这四颗雷**没有一颗能被编译器抓到**。编译器不认识你的板子，不认识共阳共阴，不认识 AHB1 和 APB1 哪条总线上挂着 GPIOF。过编译只证明语法合法，不证明任何一个位写对了地方。这就是为什么对账的对象是手册，不是编译器，也不是模型的语气。

那模型为什么错得这么「像」？因为它学的是全网的代码与文本，而全网关于点灯的示例里，APB1 出现得极其频繁（USART/SPI/I2C 都在 APB 上），OTYPER=1 的开漏写法在 I2C 场景里天天见，BSRR 低 16 位置位是 90% 教程的写法。模型给你的是「最常见写法」，不是「你的板子的正确写法」。这个区别，就是对章存在的全部理由。

二、对账工具箱

对账只需要三样东西：手册、板卡事实、一张表。

手册怎么查。RM0090 的 §2.3 管「哪个外设挂哪条总线」，§7.3 管「时钟使能位是哪个」，§8.4 管「每个寄存器每一位是什么」。查总线归属看 §2.3 的存储器映射（表 1）；查使能位看 §7.3 的 RCC 寄存器定义；查位定义看 §8.4 的寄存器位表：MODER 偏移 0x00、OTYPER 0x04、OSPEEDR 0x08、PUPDR 0x0C、IDR 0x10、ODR 0x14、BSRR 0x18、LCKR 0x1C、AFRL 0x20、AFRH 0x24。这张偏移表在 S3 已经抄过一遍，对账时直接搬。

板卡事实从哪来。原理图。霸道者的 PF6 红灯共阳接 3.3V。这个事实模型不知道，手册也不知道，它只活在你的原理图和 S3 的板卡事实表里。对账永远发生在「手册」与「板卡事实」的交叉点上：手册说 BSRR 高 16 位写 1 会把引脚拉低，板卡事实说拉低才亮，两个一拼，模型那句 `BSRR = (1<<6)` 的死刑判决书就写好了。

表怎么填。三问清单，一行一行来，不许跳：

#	对账项	查哪里	判据
1	寄存器名与总线归属	RM0090 §2.3 + §7.3	基地址落在哪条总线（§2.3 表 1），使能位就在哪条总线的 ENR（§7.3）
2	位域值	RM0090 §8.4 位定义表	每个写进去的数值能对回某几位的定义，无一位悬空
3	初始化顺序	手册时序图 + 「时钟先行」铁律	先使能时钟，再碰外设寄存器
4	硬件语境	板卡原理图	电平极性、上下拉、开漏推挽，与接线一致

三、逐位对账实操

拿第一节那段代码，把表填完：

对账项 1：总线归属。查 §2.3 存储器映射：GPIOF 基地址 0x40021400，落在 AHB1 区段（AHB1 外设从 0x40020000 起）。结论：使能位在 `RCC->AHB1ENR`，§7.3 的 RCC 寄存器定义里 bit5 是 GPIOFEN。模型写的 `APB1ENR |= (1<<5)` 双重错误：总线错了，而且 APB1ENR bit5 是 TIM7EN，等于顺手打开了定时器 7 的时钟：一颗雷炸出两处伤。改法：`RCC->AHB1ENR |= (1UL << 5);`。

对账项 2：位域值。逐寄存器过：

寄存器	模型写的	§8.4 定义	判定
MODER	`(1<<12)`	MODER6 = bits 13:12，01 = 通用输出	✓ 值对（输出模式）
OTYPER	`(1<<6)`	bit6=1 开漏；共阳 LED 需推挽（=0）	✗ 雷 2，删掉这行
OSPEEDR	`(3<<12)`	OSPEEDR6 = bits 13:12，11 = 100MHz	✓ 值合法（点灯其实 2MHz 足够，但不算错）
BSRR	`(1<<6)`	低 16 位置位输出高；高 16 位（bit22）复位输出低	✗ 雷 3，共阳要低电平，改 `(1UL << 22)`

对账项 3：顺序。这段代码恰好先开时钟再配 MODER，顺序这颗雷没爆。但对账不能赌运气。把顺序显式写进你的检查习惯：任何外设初始化，第一行使能时钟，这是 S3 说的「新手第一大坑，没有之一」，模型踩中的概率不低。

对账项 4：硬件语境。原理图：PF6 红灯共阳。推导链：共阳 → LED 另一端接 3.3V → 引脚输出低电平才有电流 → 推挽 + 拉低。模型不知道这条链，你必须替它知道。

四问全过，修正版才三行：

```
RCC->AHB1ENR   |= (1UL << 5);    /* GPIOFEN：AHB1，bit5 */
GPIOF->MODER   |= (1UL << 12);   /* PF6 通用输出 */
GPIOF->BSRR     = (1UL << 22);   /* 高 16 位：PF6 输出低，共阳 LED 点亮 */
```

从模型自信满满的七行注释，到对账后的三行。中间消失的部分，就是幻觉的体积。

四、上板终审与证据回喂

对账全过，才配烧上板。烧。灯亮，闭环完成一半；灯不亮，回到第三节，必有一问你填得太快。

终审的意义不只是「亮没亮」。拿示波器或者最简单的串口打印，确认你看到的亮是推挽拉低的亮（引脚电平 0V 附近），而不是某个悬空引脚被恰好接通的假象。一次上板，三个产出：现象（亮/不亮）、证据（波形/寄存器读回值）、修正后的代码。

然后是最值钱的一步：**回喂**。把证据句写进下一轮提示词，不是写情绪句：

回喂句	模型能学到什么
「你写错了，灯不亮」	几乎什么都学不到，只会道歉
「实测：GPIOF 挂在 AHB1 不是 APB1，RM0090 §7.3 存储器映射 0x40021400 属 AHB1 区段；APB1ENR bit5 是 TIM7EN」	总线归属这条，本轮记住
「实测：BSRR 高 16 位写 1 复位引脚输出低，低 16 位写 1 置位输出高；本板 LED 共阳需低电平」	位语义 + 硬件语境，本轮记住

补一个现场细节：喂进证据句之后，它的下一轮多半以「您说得对」开头。这四个字出现得再诚恳，也担保不了下一轮的正确率：道歉是它的文体，不是它的修正动作。账对没对，仍然只认 §7.3 与 §8.4。

回喂的格式就是证据三元组：**实测 + 手册出处 + 板卡事实**。这跟你在 S3 学寄存器时的取证习惯是同一套——对账不是 AI 时代的新发明，是这本书从头到尾的方法论，只是现在多了一个会犯错的结对对象。

五、对账的边界：哪些逐位、哪些抽样

逐位对账很费人。全部代码逐位对，你很快就弃坑。按风险分级：

代码层	对账强度	理由
时钟使能（RCC 各 ENR）	**永远逐位**	总线归属是模型幻觉第一名，错了编译不报错、上板没反应，排查成本最高
模式/类型/上下拉寄存器（MODER/OTYPER/PUPDR）	**永远逐位**	与板卡接线强相关，模型永远不知道你的原理图
数据路径寄存器（ODR/BSRR/DR）	逐位 + 上板现象	位语义幻觉高发（高 16 位/低 16 位、MSB/LSB）
时序参数（波特率分频、定时器 PSC/ARR）	逐位 + 计算复算	模型算术错不罕见，拿计算器重算一遍
应用层逻辑（状态机、协议解析）	抽样 + 单测	幻觉密度低，测试比手册快
纯软件算法（滤波、CRC）	单测对拍	有现成数学定义，不需要手册

分界线只有一条：**这段代码的「正确性」定义在手册里，就逐位；定义在数学或测试里，就用计算和单测**。前者是嵌入式独有的对账，后者是普通软件工程的老本行。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **模型输出是假设：名对没对查 §7.3，位对没对查 §8.4，序对没对时钟先行；板卡接线是你的领地；上板拿证据，回喂写三元组。**

延伸：弱提示与强提示的对照实录在 A1 提示词模式；让模型自己读手册跑构建的工具流在 A3 工具流；幻觉的完整类型学与防御决策树在 A4 幻觉防御。

常见坑

- **拿编译通过当对账通过**：编译器不认识你的板子，也不认识 AHB1 和 APB1 的区别。过编译是最弱的一种正确。
- **对账对到一半信了你自己的记忆**：使能位是 bit5 还是 bit6，OTYPER 偏移 0x04 还是 0x0C？手边没手册就别对账，凭记忆的对账只是两个人一起幻觉。
- **回喂只喂情绪不喂证据**：「不对」「错了」「再试试」对模型是零信息。证据三元组：实测现象、手册章节号、板卡接线。
- **对账粒度一把梭**：应用层代码逐位对到地老天荒。按第五节的分级表分配力气，RCC 和模式寄存器才是逐位区。
- **忘了顺序也是审计项**：模型把 MODER 写在使能时钟前面，对账清单如果只查「位」不查「序」，这颗雷就漏了。

短自测

1. 模型的点灯代码 `RCC->APB1ENR |= (1<<5)` 错在哪两处？查 RM0090 §2.3 与 §7.3 说明你的判据。

▼ 看答案
两处错。其一，GPIOF 挂在 AHB1 总线，使能位在 RCC->AHB1ENR 的 bit5（GPIOFEN），不在 APB1ENR；其二，APB1ENR 的 bit5 是 TIM7EN，这句代码顺手打开了定时器 7 的时钟。判据：§2.3 存储器映射（表 1），GPIOF 基地址 0x40021400 落在 AHB1 外设区段（0x40020000 起）；§7.3 RCC 寄存器定义，AHB1ENR bit5 标注 GPIOFEN、APB1ENR bit5 标注 TIM7EN。总线归属错了编译器不报错，这是必须逐位对账的原因。

2. 共阳 LED 接 PF6，为什么 `GPIOF->BSRR = (1<<6)` 点不亮？正确的写法是什么？

▼ 看答案
BSRR 低 16 位写 1 把对应引脚置为输出高，高 16 位写 1 置为输出低（BSRR 偏移 0x18，§8.4）。共阳 LED 的阳极接 3.3V，引脚输出低电平才有电流流过，输出高反而两端等电位熄灭。所以正确写法是写高 16 位：`GPIOF->BSRR = (1UL << (6 + 16))`，即 bit22；或者写 ODR 对应位为 0。模型 90% 的教程学来的是「置位点亮」，那对共阴或低电平有效的电路成立，对你的板子不成立：这就是硬件语境幻觉。

3. 对账三问是哪三问？各自查手册的哪个位置？

▼ 看答案
名对不对：寄存器名与总线归属，查 §2.3 的存储器映射（基地址落哪条总线）与 §7.3 的 RCC 寄存器定义（使能位在哪条 ENR）。位对不对：每个写进代码的数值对回位定义表，查 §8.4（以及各外设章节的寄存器说明），不许有一位悬空。序对不对：初始化顺序，查手册时序要求加「时钟永远先于一切」铁律：先使能时钟，再配置外设寄存器。

4. 为什么「你写错了」对模型的校正价值几乎为零？合格回喂句的三元组是什么？

▼ 看答案
模型从「你写错了」里只能得到一个布尔值：错。它不知道是总线错、位错还是顺序错，更不知道正确口径，下一轮只能换一个猜法。合格回喂句的三元组：实测现象（灯不亮/引脚读回高/波形无翻转）、手册出处（RM0090 §2.3 存储器映射：0x40021400 属 AHB1 区段）、板卡接线（PF6 红灯共阳）。三元组给了模型一条可验证的更正链，本轮对话内它就能收敛。

5. 按第五节分级表，HAL 库的 GPIO 时钟使能函数 `__HAL_RCC_GPIOF_CLK_ENABLE()` 内部那一行，要不要逐位对账？

▼ 看答案
要。凡是你代码路径上经过的时钟使能，无论裸寄存器还是库函数，都属「永远逐位」级：总线归属错了，库函数也救不了你（虽然 HAL 的总线归属是对的，但读一遍它的实现，就是对 RCC->AHB1ENR bit5 的封装，把「名对不对」这问闭环，成本一行）。分级表按代码层不按写法层：库函数只是封装，不降低时钟使能的风险等级。

对照表：本章概念 → 仓库与上游落点

概念	落点
对账三问	RM0090 §7.3（名/总线）、§8.4（位）、S3 时钟先行（序）
GPIOF/AHB1 事实	S3 板卡事实：0x40021400 / 0x40023800 / AHB1ENR 偏移 0x30 / bit5
BSRR 高 16 位语义	RM0090 §8.4，BSRR 偏移 0x18；S3 位操作
总线归属幻觉案例	本章第一节雷 1；TIM7EN = APB1ENR bit5（§7.3 RCC 寄存器定义）
证据回喂三元组	A1 提示词模式 验收标准段；A3 工具流 日志回喂
时钟先行铁律	S3：「新手第一大坑，没有之一」

你做到了

- 拿到任何一段 AI 生成的寄存器代码，三问清单在心里自动弹出来；
- 能把「灯不亮」翻译成一条模型学得会的证据句；

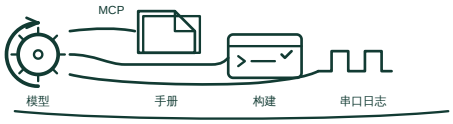
- 对账的力气会分级了：RCC 与模式寄存器逐位，应用层交给测试。

✅ 对账闭环在手，它的输出从此都要过你的三问。下一站：**A1 提示词模式**，回到上游，让幻觉从源头少起来；或 **A3 工具流**，让它自己翻手册，你先把搬运工的马甲脱了。

通往单片机之路第 92 章

A3 工具流：让模型读手册、跑构建、看串口

难度 ★★★☆☆ · 约 45 分钟



本章罗盘

问题

让模型自己看手册、自己跑构建，和把错误粘给它，差在哪？

交付

MCP 三角色架构解说（Host/Client/Server、三类原语）+ 三个嵌入式场景实录（读手册快照/跑交叉构建/串口日志回喂证据三元组）+ agent 权限与安全边界清单。

边界

消费侧为主，不教自建 MCP server；对账与验收细则只引 A2 不展开；场景演示全用仓库既有工程，不引入新工程；引用官方口径带访问日期，界面会变、方法论不变。

依据锚点 `ai-mcp-spec` · `ai-mcp-architecture` · `ai-mcp-servers` · `ai-cursor-run-modes` · `ai-claude-code-memory`

> 🎯 前两章的模型是「手无寸铁」的：你粘手册它背手册，你粘错误它猜错误，你粘日志它读切片，每个来回你都是它与终端之间的人肉管道。这一章给它装上三件工具：读文件的手册、跑命令的终端、看日志的眼睛；协议底座是 MCP，权限闸门握在你手里。装上工具的模型还是不是假设机？是。变的不是可信等级，是闭环的位置：机器侧的内环它自己转，你守住手册与上板的外环。

本章精髓

1. **工具流省的不是复制粘贴，是信息的完整与新鲜**。你粘的是切片、是上一刻、是孤本；它自己取的是全量、是当下、自带行号与工程语境。同一条编译错误，切片喂法它陪你对着症状猜，全量喂法它自己顺着第一条错误挖到根因。差的不是步骤数，是闭环的位置，以及你在闭环里的角色。
2. **MCP 是插座标准，不是又一个工具**。开放协议（版本 2026-07-28，消息格式 JSON-RPC 2.0），Hosts/Clients/Servers 三方，host 为每个 server 建一个 client；server 向 client 暴露 Resources/Prompts/Tools 三类原语：手册文本是 Resource，三段式工单模板是 Prompt，跑构建是 Tool。协议 2025-12 起归 Linux Foundation 下设中立机构治理，学它不押注任何一家厂商。
3. **接入即授权，闸门要建在硬处**。MCP 规范把「工具代表任意代码执行」写进安全原则；Cursor 官方明说 Auto-review is not a security boundary；CLAUDE.md 被官方定性为 context, not enforced configuration。软约束管态度，硬门禁管放行：允许列表、hooks、构建门、对账三问，一层都别省。

- **能理解**：为什么「它自己跑构建」比「你把错误粘给它」信息量高一个量级；Host/Client/Server 三角色各坐在哪；三类原语与嵌入式素材的对应关系；三档运行模式各自把风险放在哪。
- **能用**：给手边的 agent 工位接上读文件与跑命令两类能力，跑通读手册、跑构建、看串口三个场景；给每个接入的工具过一张四问权限清单；把超长日志切成模型吃得下的证据片。

先修

- A0 为什么把大模型当结对工程师：能/不能清单与工具地图，本章所有「不许」从那里来；
- A1 提示词模式：三段式工单，本章三个场景的提示词都按它写；
- A2 手册对账：对账三问与证据三元组，工具流的终审动作，本章只引不展开；
- B0 交叉工具链：arm-none-eabi-gcc 的安装与验证，场景二的构建靶子；
- S3 GPIO：霸道者 F407 板卡事实，场景一的对账靶子。

先跑起来（同一条错误，两种喂法，15 分钟）

准备工作两件：B0 的工具链装好，本书前面任选一个能构建通过的 STM32 裸寄存器工程（S3 的点灯工程即可）；再有一个能跑命令的 agent 工位（A0 工具地图三格任选一格）。

开跑之前先押个轮数：同一处构建错误，把 5 行粘给它，和让它自己捞全量输出，各自几轮修到绿？一边押一个数，两张表填完对答案。

实验 A（搬运工模式）：把工程里改错一行，比如 `RCC->AHB1ENR` 误写成 `RCC->APB1ENR`（A2 的总线归属雷）。自己跑一遍构建，把终端输出的最后 5 行复制下来，粘进一个不能跑命令的普通聊天窗，问：怎么修？

实验 B（工具流模式）：把同一处错误留着，换到 agent 工位，只发一句：

> 构建这个工程，把编译修到通过。只许动 main.c，改完自己重新构建验证后再交。

两轮都结束，填这张对照表：

对照项	实验 A	实验 B
它问你要信息的次数		
修到编译通过用的轮数		
它看到的错误输出（切片/全量）		
有没有顺手改别的东西		

判读两条。其一，看第一轮判断差多远：搬运工模式它只能凭你粘的 5 行猜，而构建错误的根因通常在输出最前面的第一条，后面几十行全是连锁反应；工具流模式它自己捞全量输出，第一刀就砍在根因上。其二，数实验 B 里它跑了几次构建：这个「跑、读、改、再跑」的循环就是官方说的 auto-correct，你不用当管道，它自己转到绿。

如果实验 B 里它转了五轮还没绿、还开始乱改，别急着救场，先记下现场：这正是第六节权限闸门的预习材料，「只许动 main.c」这条约束有没有被遵守，是你评审的第一项。乱改的典型症状是开始动与错误无关的文件：那不是它在努力，是它在表演努力，评审时按越界改动记账。

实验做完把错误改回去，这个工程后面还要用。

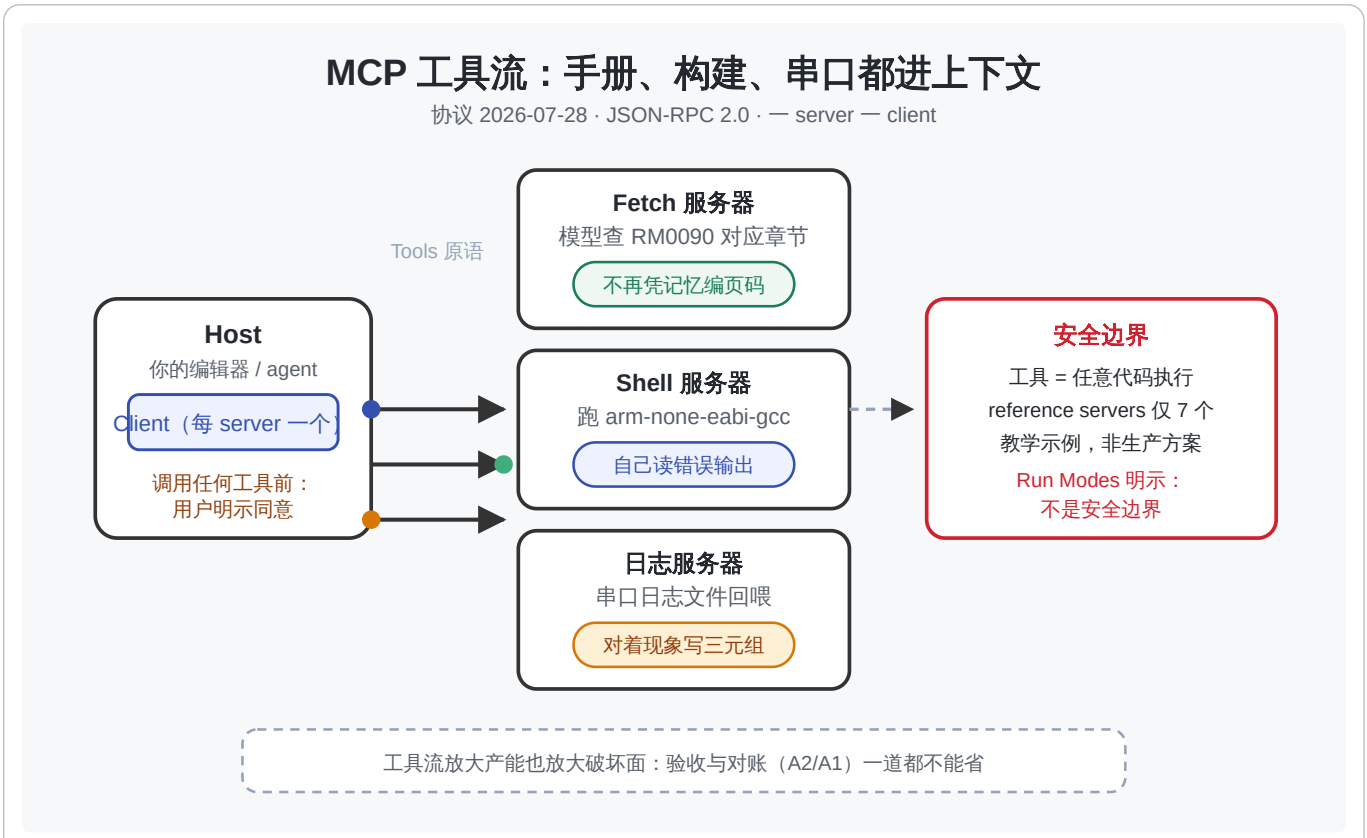
实验报告按这个格式存档（进版本控制或实验笔记）：

字段	内容
日期与工位	哪天做的、用的哪个 agent 工具与版本
错误原文	改坏了哪一行，构建输出的第一条错误
实验 A 表现	问了你几次、第几轮方向才对
实验 B 表现	自己跑了几轮构建、有没有越界改动
结论	这次观察到的差距形态

这份报告与 A0 的一致性实验档案是同一类资产：下次评估新工具、新模型时翻出来对照，你的判断就有了自己的数据底座，不用信任何评测文章。

动画：工具流全景

左边是你的工位（Host）：VS Code、Cursor、Claude Code 任一；Host 内部给每个外部能力配一个连接器（Client），一对一接到服务（Server）上：Filesystem server 管读手册文本与仓库文件，Git server 管版本史，Fetch server 管抓官方文档页。Server 向 Client 暴露三类原语：Resources（数据）、Prompts（模板）、Tools（可执行能力），消息格式 JSON-RPC 2.0。图的右侧是你永远不下班的终审工位：手册、原理图、上板。



MCP 工具流：Host/Client/Server 三角色与三类原语

读这张图只记一层关系：Client 是 Host 内部的配角，一对一服务一个 Server；你打交道的是 Host，谈判能力清单的对象是 Server。

图分三层读。上层是产品层：工位随版本换代，今天的按钮位置明年就是考古资料。中层是协议层：JSON-RPC 2.0 消息在三角色间流动，Resources/Prompts/Tools 三类原语是全部词汇表。下层是终审层：手册、原理图、上板，没有一个图标是机器能替你坐的。三层里只有中层值得背，上层查文档，下层长在你手里。

小节结构

一、两种喂法差在哪：切片、时效、语境三重损耗；二、MCP 解剖：三角色、三原语、一条拓扑；三、场景实录一读手册；四、场景实录二跑构建；五、场景实录三看串口；六、权限与安全边界：软约束与硬门禁。

一、把错误粘给它，与它自己跑，差在哪

先给两种协作方式起名字。**搬运工模式**：模型不动手，你当它与终端之间的管道，报错靠你复制，文件靠你粘贴，日志靠你截图转文字。**工具流模式**：模型自己有手，读文件、跑命令、看输出，你只下工单与终审。

A0 的能/不能清单没有变：它依然不知道你的原理图，依然不能替你上板。变的是信息到达它手里的路径。搬运工模式里，这条路有三重损耗，每一重你都在日常里见过：

损耗	现场	后果
切片损耗	构建吐了 80 行错误，你复制了最后 10 行	第一条错误才是根因，后面全是连锁；它陪你对着症状开药
时效损耗	你粘的是五分钟前的输出，中间自己又改了两行	它基于过期现场推理，修一个可能已经不存在的问题
语境损耗	错误文本到了，出错的文件、行号、编译命令、工程结构没到	它只能猜环境，猜错方向你还得再粘一轮

工具流模式把三重损耗一次消掉：它自己跑构建，拿到全量输出，第一条错误在最前面；它自己读文件，行号与上下文自带；它改完自己再跑一遍验证，不用等你当下一次管道。

把切片损耗演一遍给你看。一次真实的构建，输出 80 行：第 3 行是 `main.c:42: 'RCC_AHB1ENR' undeclared`，后面 77 行全是这个未声明宏引发的连锁报错。你复制最后 10 行粘过去，模型看到的是一堆莫名其妙的类型错误，于是开始陪你猜：是不是头文件顺序错了，是不是编译选项少了。它猜得有理有据，全是症状的关联推理。而直接把全量输出给它（或者让它自己跑），它第一眼看到第 3 行：宏名打错了，补个头文件或改对名字，一刀收工。同一条错误，两种喂法，一个是会诊，一个是误诊。你当管道的每一分钟，都在为切片损耗付利息。

这不是本书的推演，是工具官方写好的工作机制。VS Code 的 agents 文档给这个机制起了名字（规范定义，访问日期 2026-10-27）：

> An agent uses a language model to reason about your request and tools to act on your development environment...This repeated reasoning, action, and validation is the **agent loop**.

推理、行动、验证，循环往复。2025-02-24 的 agent mode 预览公告把这个循环描述得更具体（能力描述）："It responds to compile and lint errors, monitors terminal and test output, and auto-corrects in a loop until the task is completed." 响应编译与 lint 错误、监视终端与测试输出、自动修正直到任务完成。Claude Code 官方仓库 README 的定位句同构（能力描述）："edits files, runs commands"。三份互不隶属的来源说的是同一件事：机器侧的内环，机器自己转。

回忆 A0 的两环结构：内环是机器侧验证（编译、测试、lint），外环是你的终审（手册、原理图、上板）。搬运工模式里，内环的传动轴是你，每一圈都要亲手搬一次；工具流模式把内环整个还给机器，你的人肉管道退役，回到审计员的位置。这就是「差在哪」的完整答案：差的不是谁打字快，是闭环的位置，以及你在闭环里的角色。

两种模式摆一张总表：

维度	搬运工模式	工具流模式
信息到达	你复制的切片，上一刻的孤本	它自己取的全量，当下的现场
内环传动	人肉管道，一圈一搬	机器自闭环，auto-correct 转到绿
你的角色	管道工兼审计员	纯审计员
外环终审	手册、原理图、上板	手册、原理图、上板（不变）
新增风险	无新能力，无新风险	接入即授权，闸门要建（第六节）

盯住最后一行：工具流不是免费的午餐，它用「授权风险」换「信息完整」。这笔交易值不值，取决于第六节的三层防线你建没建。

顺带说清本章的排位逻辑：工具流排在 A1 与 A2 之后，是因为工单与对账是它的前置技能。没有三段式，你不知道给它下什么工单；没有对账三问，它自己跑出来的绿你会当成终审。先会验收，再发工具，顺序不能反。

不变的那一半也要看见。agent loop 里的「验证」仍然是机器侧验证：编译绿、测试绿、lint 绿，三个绿都不认识 RM0090，不认识共阳接法。它把 `RCC->AHB1ENR` 误写成 `RCC->APB1ENR`，编译照样全绿，灯照样不亮。内环越顺，这个盲区越隐蔽：人很容易把「它自己跑通了」误读成「它对了」。终审不撤，对账三问照跑（A2），工具流时代反而更要盯紧这一条。

二、MCP 解剖：三个角色、三类原语、一条拓扑

要让模型读手册、跑构建，得先有一条把外部能力接进模型的标准插座。这条插座就是 MCP。最短定义用 Claude Code 官方文档的说法（能力描述，访问日期 2026-10-27）：

> The Model Context Protocol (MCP) is an open standard for connecting AI tools to external data sources.

规范侧的硬事实（MCP 官方规范，协议版本 2026-07-28，规范用语遵循 BCP 14 / RFC 2119）：消息格式 JSON-RPC 2.0，方法与参数都是结构化 JSON，抓包能逐字读；架构分三方；server 暴露三类原语。逐层拆。

消费侧接触 MCP 的入口通常是一份配置文件：在工位的设置里声明要接哪些 server、各自用什么方式启动（本地命令还是远程地址）。界面与文件名随工具与版本漂移，各家写法不同，查各自官方文档的当前版本；不变的是三件要决定的事：接哪个 server、给它什么权限、它出不出网。这份文件读到底是一张授权表：每添一行，就多一个能摸到你文件的角色；填表的手续在第六节，四问清单就是从这三件事长出来的。

三个角色。Hosts 是发起连接的 LLM 应用：你的 VS Code、Cursor、Claude Code 工位都是 host。Servers 是提供上下文与能力的服务：读文件、查 Git、抓网页各是一个 server。Clients 是 host 内部的连接器，host 为每个 server 建一个 client。盯住「一 server 一 client」这条拓扑（官方架构页）：接三个 server，host 里就是三条各自独立的排线，一条连手册柜、一条连版本史、一条连外网文档。排线的好处是各拔各的：哪条出事拔哪条，其余照常干活。这就是授权的颗粒度：能力按 server 分批接入、分批授权，出问题时按 server 定位与摘除。

拓扑还有一层工程含义。本地 STDIO server 通常只服务一个 client，跑在你自己机器上，权限跟着你的用户走；远程 Streamable HTTP server 通常服务多个 client，你的代码与日志要出机器、进别人的服务。消费侧的默认姿势是本地优先，第六节展开。规范把协议切成两层：data layer（消息与能力协商）与 transport layer（STDIO 与 Streamable HTTP 两种搬运方式）。消费侧只需抓住一条：同一套原语，本地管道与远程 HTTP 都能跑。

三类原语。server 向 client 暴露 Resources、Prompts、Tools 三样东西，用嵌入式的手各翻译一遍：

原语	规范里的角色	嵌入式翻译
Resources	数据，server 提供给模型的上下文	RM0090 章节文本、原理图导出、仓库文件、串口日志
Prompts	提示模板，可复用的提示词	A1 的三段式工单固化成模板，全组共用
Tools	可执行能力，模型可以调用的动作	跑一次 arm-none-eabi-gcc 构建、读一段串口日志

client 侧还有一个 Elicitation 原语：server 可以主动向用户反问补齐信息，与 A1 里「不确定就提问」那条保险丝同一个意图。一个时效提醒：旧教程常提的 Sampling 与 Logging 两个 client 原语，自协议版本 2026-07-28 起已被官方标记为 deprecated（架构页明示），学新不学旧。

三类原语与后面三节场景实录是一一对应的：场景一「读手册」消费的是 Resources，把手册文本当数据喂进上下文；场景二「跑构建」消费的是 Tools，把构建命令包成模型可调用的动作；场景三「看串口」两者都吃，日志文件是 Resource，管道直喂是 host 自带的壳能力。至于 Prompts，你的三段式工单模板固化进去，全组下工单就走同一张骨架。原语只有三类，组合是无限的：消费侧的功夫全在「拿什么素材配什么原语」这一件事上。

生态与治理。官方在册 reference servers 仅 7 个：Everything、Fetch、Filesystem、Git、Memory、Sequential Thinking、Time（官方 servers 仓库 README，访问日期 2026-10-27）。官方明示其定位是教学示例，"not as production-ready solutions"；早期的 GitHub、GitLab、Google Drive、Postgres、SQLite、Slack 等参考实现已归档进 servers-archived；更多服务器走官方 MCP Registry。治理侧：2025-12，Anthropic 把 MCP 捐赠给 Linux Foundation 下设的 Agentic AI Foundation（2025-12-09 成立），与 Block 的 goose、OpenAI 的 AGENTS.md 同为创始项目。一个协议有了中立治理与官方注册中心，就从某家的私有排线变成行业的标准插座，这是它值得进你工具箱的理由。

本章立场是消费侧：怎么选 server、怎么授权、怎么用在嵌入式流程里；自建 server 是另一门课，本书不展开。时效口径与全书一致：本节每个事实都对着附录 E 的一张来源卡（访问日期 2026-10-27），网上旧文引的旧版协议号（如 2025-06-18）已过时，以 2026-07-28 为准。界面与清单会变，「三角色、三原语、一 server 一 client」的方法论不变。

为什么押协议而不是押某个工具？官方架构页自己给了类比：LSP（语言服务器协议）当年把「编辑器接语言智能」这件事标准化，换来的是任何编辑器都能接任何语言。MCP 想做的是同一件事在 AI 工具侧的翻版：工具与数据源写一遍 server，任何 host 都能用。协议层沉淀、产品层换代，你的学习投资应该放在沉淀的那层。规范本身也在快速演进：2025-11-25 的规范版本引入了异步操作、无状态化、server 身份与官方 extensions 等机制，到 2026-07-28 版本又有调整；这恰好是「学协议骨架、不背界面细节」这条口径的官方证明。

三、场景实录一：读手册，别靠背

A1 里你学过给手册章节号当锚点：§2.3 表 1 查总线归属，§7.3 查使能位，§8.4 查位定义。那是「靠锚点」级：模型对 RM0090 的训练记忆相当广，锚点能帮它站稳，但记忆终究是记忆，它能把 MODER 的偏移说对，也可能在下一轮一本正经地编一个不存在的位名。工具流把这个场景再升一级：不靠记忆，靠文件。

三级对照：

级别	做法	幻觉风险
L0 靠记忆	什么都不给，直接问	编页码、编位名，语气与答对时一样自信（A1 实锤）
L1 给锚点	提示词里写手册章节号	站稳大半，内容仍靠它的记忆转述
L2 给文件	手册文本进仓库，Filesystem server 授权只读	它引的每行都有仓库原文可对

实录一遍 L2 的姿势。把 RM0090 相关章节的文本导出（比如 GPIO 寄存器章、RCC 章），放进工程仓库的 datasheets/ 目录；这份文本你自己核对过，从此是「手册的仓库快照」，与代码同仓演进。给工位配 Filesystem server，授权范围只读这个目录（授权怎么审，第六节）。工单从 A1 的「位定义查 §8.4」升级成：

> 【上下文】……位定义读 datasheets/ 下 RM0090 的 GPIO 章文本，MODER/OTYPER/OSPEEDR/BSRR 各寄存器的偏移与位域以该文件为准，引用时给出原文行。

输出的变化肉眼可见：它给的每个位定义后面跟着仓库文件里的原文行。你的对账动作不变，还是三问（A2），但抽查成本骤降：它写「MODER 偏移 0x00」，你打开 datasheets/ 搜一下就知道它读没读对，不用翻整本 PDF。

两条边界先说死。其一，它读到的原文经它转述仍可能走样：读串行、截断、把相邻寄存器的位域张冠李戴，这类错工具流不免疫；所以抽查不撤，只是从「怀疑它的记忆」降级为「核对它的引用」。其二，手册快照是你的资产：官网改版、PDF 换版，你的 datasheets/ 目录不动，全组对的是同一份文本。网页版手册与厂商文档页也可以经 Fetch server 抓，同一口径：抓来的内容进证据链时带来源与日期，与 A0 的来源卡纪律一致。

快照从哪来、放哪去，定三条规矩就够。来源：手册原文只能从厂商官方渠道拿（ST 官网的 RM0090 PDF），文本导出后你自己核对一遍关键表再入库，导出工具的错字与模型的幻觉是同一物种，都不能免检。范围：datasheets/ 只放团队核实过的章节，按芯片与外设分文件，别整本糊成一个巨型文本，检索精度与上下文预算都吃不消。演进：手册换版时更新快照并在 commit 里写明版本差异，模型引用的「原文行」必须能对回你手里的那份，而不是对回官网的最新版。

为什么是文本快照而不是直接喂 PDF？三个现实原因。其一，PDF 的位定义表排版复杂，两栏、跨页、合并单元格，解析成文本后错行错列是常态，你核对过的纯文本比现场解析的 PDF 可信。其二，上下文预算：参考手册体量庞大，整本进上下文既不经济也不必要，你关心的一直是当前外设那几章。其三，可评审：文本进了版本库，diff 看得见，PDF 换版你只能靠记忆发现。哪天工具的 PDF 解析可靠到你能抽查通过，再换不迟：方法不变，载体可换。

L0 与 L2 的回答摆在一起对照，差别一眼可见：

对照项	L0 靠记忆	L2 给文件
位定义的出处	「根据 RM0090……」（记忆转述）	指着仓库里的原文行
你的对账动作	翻整本 PDF 逐位重查	打开 datasheets/ 抽查引用行
幻觉被抓的时机	上板之后（最坏的时机）	对账台上（最好的时机）
团队复用	每人各信各的记忆	全组对同一份快照

读手册这个场景是三个场景里收益最稳的一个：零写入、零执行，风险最低，而幻觉高发区恰好就在手册事实上。工具流的第一课从这里开始：先把「它的记忆」换成「你的快照」，再谈跑命令这种带电操作。

四、场景实录二：跑构建，内环还给机器

B0 教你装 arm-none-eabi-gcc 时，构建是你亲手跑的：敲命令、读红字、改、再敲。这个循环正是 agent loop 的天然靶子，2025-02-24 的官方公告把这套动作写进了产品定义（第一节引过原文）。实录工单长这样：

> 【上下文】工程：S3 点灯工程，霸道者 F407，板卡事实照旧：PF6 红灯共阳接 3.3V，低电平点亮；GPIOF 基地址 0x40021400，时钟使能位 RCC_AHB1ENR bit5。
>
> 【约束】只许动 main.c；禁用 HAL 与标准库；时钟使能第一句；不许用压警告的方式消警告。
>
> 【验收】arm-none-eabi-gcc 交叉编译通过、零警告；随代码附对账表；你自己重新构建验证后再交。

发出去，观察，不要插手。它跑构建、读错误、改代码、再跑构建，转到零警告交卷。这个过程你只盯三件事，盯的时候手里拿一张观察记录表：

轮次	它跑了什么命令	它改了什么	结果（错误数变化）
1			
2			
3			

这张表就是 agent 会话的对账单。它交卷时你不只看最终的绿，还看这张表：每一步是不是都在收敛，有没有一轮改完错误数反而变多（那是在拆东墙），有没有同一个文件被来回改（那是在绕圈）。

它修的是因还是症状。报 undefined reference，修因是补上定义或改对名字，修症状是把调用删了；报警告，修因是改代码，修症状是加编译选项把警告压掉。「不许用压警告的方式消警告」写进约束段，封的就是后一条路。判断因与症状，用的还是你在前面章节攒下的判据：工具流不生产判据，只消耗判据。

修症状有三个肉眼可辨的信号，评审 diff 时对照着看：其一，删代码多于加代码，功能随着错误一起消失了；其二，改动出现在构建配置而不是源码里，警告被 flag 压掉、检查被选项绕过；其三，它解释不清为什么这样修，注释从「因为手册说」退化成「这样可以过」。三个信号中任何一个出现，这一轮就不能算收敛，退回让它修因，修不了就换你上。

它跑了几次构建。一次过，说明任务太简单或运气好；三到五次收敛是常态；十次以上还在转，多半在绕圈，叫停，看它卡在哪个错误上：多半是缺上下文，按 A1 的迭代纪律补锚点，比让它硬转便宜。Cursor 官方文档明示一个任务内工具调用次数无上限（能力描述，访问日期 2026-10-27），所以「让它自己转到天荒地老」不是策略，是账单的代名词。

它有没有顺手越界。约束写了只许动 main.c，它有没有动 Makefile、有没有顺手「优化」别的文件。Cursor 官方文档给过一个配套机制：Checkpoints 是会话期间的本地代码快照，只用于撤销 agent 的改动，永久版本控制请用 Git（能力描述，访问日期 2026-10-27）。agent 的产物必须进你自己的版本控制、逐 diff 评审，这条纪律与工具无关。

验收的换算关系随之更新。A1 的三层验收是可编译、上板现象、对账动作；工具流把「可编译」这一层整个包给机器，你的终审从三层收敛到两层：对账与上板。省力是真，陷阱也是真：内环越自动化，「编译绿」的迷惑性越强。它自己跑通的代码，位照样可能错：AHB1 写成 APB1，编译器不报警，它也不报警，因为它和那台编译器一样，不认识你的板子。绿灯只放行到对账台，不放行到烧录器（A2）。

它交卷之后，你的动作序列固定四步：

1. **看 diff 不看总结**：它的「修好了」是语气，git diff 是证据；逐 hunk 过，顺手改动当场揪出；
2. **看对账表抽查**：它附的逐寄存器章节号表，抽一行对回手册，对不上整份退回；
3. **自己复跑一遍构建**：它的「零警告」在你的机器上复现才算数，环境差异是常客；
4. **上板取证**：烧录、按复位、看灯，终审动作一个不少。

四步与 A0 的审计员五步环同构，只是把「下工单」到「收草稿」之间的活整个外包给了机器。外包的是手，不是眼睛。

五、场景实录三：看串口，日志变证据

嵌入式与纯软件的工具流，差在最后这一公里：你的「测试输出」在串口上。本书 E08 放音工程（P12 音频链路）是个现成例子：ESP32-S3 经 I2S 推 ES8311 让喇叭「滴」一声，16kHz/16bit，MCLK = 256 × fs = 4.096MHz，固件启动时串口打出一行 `MCLK=4096000`（main.c:126）。喇叭没响的时候，这行日志在不在、值对不对，是排查的第一站。

日志进模型有两种合规喂法。其一，串口输出抓到文件（uart.log），Filesystem server 授权只读，模型自己读。其二，管道直喂：Claude Code 官方文档明示它遵循 Unix 哲学、可管道化，原话例子就是 `tail -200 app.log | claude -p "…"`（能力描述，访问日期 2026-10-27）。把 app.log 换成你的串口日志，这条路就是为嵌入式准备的。

抓日志本身也有姿势，按排查阶段选：常抓求全（调试期全程落盘，出事后慢慢切），触发抓求准（只在故障复现时开抓，日志短而准），双份抓求对照（正常跑一份、故障跑一份，喂给它做 diff，它找差异比你肉眼快一个量级）。三种姿势的产出都是文件，都走同一个切片纪律。

让它读日志，下工单时把证据三元组写进验收（细则在 A2）：

> 读 uart.log，回答：喇叭为什么没响？回答格式：实测现象（日志里有什么、缺什么）+ 出处（固件哪一行该打这行日志）+ 假设（最可能的原因，按可能性排序）。不许改代码，只出分析报告。

一个合格输出长这样：日志里 I2S 初始化行在、`MCLK=4096000` 在，但功放使能行缺失；查 main.c 第 81 行注释，PA_EN 走 PCA9557 的 bit1（I2C 地址 0x19），不是 ESP32 的 GPIO；假设：功放没被使能，喇叭物理上没通电。注意这条结论链的每一环都有日志行或代码行作锚：实测、出处、板卡事实，证据三元组齐活。

把三元组在本节的落点摆成一张表，回 A2 第四节对照：

三元组	A2 点灯案例	本节 E08 案例
实测现象	灯不亮、引脚读回高电平	日志缺功放使能行、喇叭无声
手册出处	RM0090 §7.3、§8.4	main.c:81 注释、I2C 地址 0x19
板卡事实	PF6 共阳接 3.3V	PA_EN 走 PCA9557 bit1 而非 GPIO

同一个格式，换一个外设照样成立：这就是把回喂纪律写成验收条款的意义，格式对了，它的分析报告才有可审性。

纪律两条，都是 A0 不能清单的延伸。其一，它读的是日志，不是现象本身：日志是固件自报的二手现象，固件说「I2S 初始化完成」不等于示波器上真有 BCK 波形；日志与示波器冲突时，信示波器。其二，它能读你抓回来的日志，不能替你夹探头：串口线你插，日志你抓，终审的物理动作一个没少。工具流改变的是「读」的分工，不改变「测」的分工。

切片怎么下刀，拿 E08 的 uart.log 演一遍。喇叭没响，日志三千行，三刀切出证据片：

1. **尾部窗口**：`tail -200 uart.log`，最后 200 行通常覆盖出事前后的现场；
2. **锚点检索**：`grep -n "MCLK\|I2S\|PA_EN\|ERR" uart.log`，把初始化链路与报错行连同行号一起捞出来；
3. **时间窗**：知道故障在第 40 秒前后发生，就按日志时间戳切那一小段，前后各留 20 行余量。

切完的三片拼起来喂给它，附一句「这是三段切片，分别来自日志尾部、锚点检索与故障时间窗」。它拿到的信息密度比三千行全量高得多，你省下的上下文预算，留给它的分析报告用。切片的原则只有一条：有名有姓。每一片从哪来、为什么切它，写在切片开头；无名切片与断章取义是同义词。

六、权限与安全边界：软约束不是硬门禁

工具接得越多，闸门越要建在硬处。这一节摆三份官方口径，它们说的是同一件事：别把安全押在模型的自觉上。

口径一：MCP 规范的安全原则。协议版本 2026-07-28 的规范原文三句（规范定义）：

> Users must explicitly consent to and understand all data access and operations.
>
> Hosts must obtain explicit user consent before invoking any tool.
>
> Tools represent arbitrary code execution and must be treated with appropriate caution.

翻译过来：数据访问与工具调用，用户必须明示同意且知情；工具等于任意代码执行，按这个危险等级对待。「接入即授权」不是修辞：你每接一个 server，都是一份实打实的能力让渡。

「明示同意」在嵌入式工位上要翻译成一份逐次点头的清单：读文件可以批量放行，写文件要看路径，跑构建命令可以入白名单，推仓库与发网络请求必须逐次确认。凡是让比特离开这台机器的动作（推送、上传、发请求），都不配进「不再询问」的名单；这条线画在哪，取决于这次泄露的后果你赔不赔得起。

口径二：Cursor Run Modes 三档。官方文档（访问日期 2026-10-27；版本线索取自其页内 Changelog：v3.6 于 2026-05-29 把 Auto-review 定为推荐默认，v3.23 于 2026-10-01 新增 Read access Workspace 模式）：

模式	行为
Auto-review	allowlist 内的命令直跑；其余尽量进沙箱；再剩下的交后端 LLM 分类器判读
Allowlist	只跑白名单内的命令，其余逐条问你
Run Everything	全部直跑，不问

沙箱是操作系统级的实现：macOS Seatbelt、Linux Landlock+seccomp。Auto-review 的分类器能借助 ReadFile/Grep/Glob/ListDir 这类只读工具判读命令内容。Cloud Agents 不走 Run Modes（官方明示）。最关键的一句官方原话，值得贴在工位上：

> The classifier can make mistakes…Auto-review is not a security boundary.

分类器会犯错，Auto-review 不是安全边界。厂商自己都不敢让模型给模型当门卫，你更不该。

口径三：CLAUDE.md 是软约束。Claude Code 官方 memory 文档写得明白：CLAUDE.md 每次会话启动时被读取，但（能力描述，访问日期 2026-10-27）：

> Claude treats them as context, not enforced configuration. To block an action…use a PreToolUse hook.

上下文，不是强制配置。你在 CLAUDE.md 里写「禁用 HAL」，它大概率遵守，也可能在某一轮忘了；想硬拦一个动作，官方指的出路是 hooks 这类机制。这与本书的立场同构：三段式提示词是软约束，管态度与质量；构建门、对账三问、diff 评审是硬门禁，管放行。A1 教你写好软约束，A2 给你硬门禁的对账动作，两者缺一不可，且不能互相替代。

三份口径摆在一起，交集清晰得近乎苛刻：规范说工具调用前须用户明示同意，Cursor 说自己的自动分类器不是安全边界，Anthropic 说自家的记忆文件不是强制配置。三家厂商，三份各自独立的文档，同一句潜台词：自动化的每一层都会错，最后一道闸门必须是不依赖模型自觉的东西。这不是本书的偏执，是供给侧的自白。

三层防线摆一张表：

层	例子	性质	失手后果
软约束	三段式、CLAUDE.md、rules	模型每轮读到，靠它遵守	质量漂移，含雷量回升
工具闸门	Run Modes、allowlist、沙箱	拦截执行，但分类器会错	危险命令可能漏放
硬门禁	hooks、构建门、对账三问、diff 评审	确定性检查，不依赖模型自觉	不绿不放行，雷留在对账台上

落到操作：每个接入的工具先过四问清单，答完再授权。把这一步当成入职面试：四道题就是背景审查，授权就是发工牌；答不上题的候选人，演示再精彩也不发。

1. **读什么**：路径范围收到最小，datasheets/ 只读，别把整块硬盘放出去；
2. **写什么**：默认零写入，构建类能力只给构建目录；
3. **执行什么**：命令白名单到具体命令（make、arm-none-eabi-gcc），不给任意 shell；
4. **出不出网**：本地 STDIO 优先；远程 server 意味着你的代码与日志进别人的服务，涉敏工程直接否。

四问之外加一条凭据纪律：生产环境的凭据（OTA 服务器、量产烧录、云控制台）永不进 agent 的运行环境。它能跑构建，不等于它该握着产线的钥匙。常见坑第一条就是这条的事故形态。

把四问清单用在两个典型 server 上，答案长这样：

四问	**Filesystem server（读手册）**	**构建能力（跑命令）**
读什么	datasheets/ 与 docs/，其余不可见	工程目录只读，构建脚本内容可审计
写什么	零写入	只写构建输出目录（build/）
执行什么	不执行任何命令	白名单：make 与 arm-none-eabi-gcc 系列
出不出网	不出	不出；拉依赖走你自己的包管理，不走它

两张答案卡填完，授权范围就定死了；填不出来的格子，等于候选人交了张空白简历，不录。

授权的顺序也有讲究，按风险从低到高爬坡：先本地后远程（STDIO 不出机器，HTTP 出机器），先只读后写入（读手册、读日志先于改文件、跑命令），先单 server 后多 server（一条连接跑顺了再接下一条，出问题好定位）。每爬一级，把这一级的行为观察一个任务周期再放行下一级。权限一旦放出去就很难完整收回，爬坡容易下坡难。

最后把第六节收拢成一句立场：工具流把模型的手接进了你的工程，手的每一根手指都要经过你的闸门。软约束让它知道规矩，工具闸门让它难越界，硬门禁让越界变不成事故。三层都在，工具流才是生产力；只剩软约束，工具流就是把风险自动化。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **它自己取，比你粘的完整且新鲜：手册给文件，构建给它跑，日志切片喂；MCP 三角色、一 server 一 client、三原语；工具即任意代码执行，软约束管态度，硬门禁管放行。**

延伸：幻觉的类型学与防御决策树在 A4；这套工具流服务本书自身生产线的实录在 A5；证据三元组与对账三问的细则回 A2；三段式工单的写法回 A1；能/不能清单与两环结构的源头回 A0。

常见坑

- **把 Auto-review 当安全边界**：官方原话 "Auto-review is not a security boundary"，分类器会犯错。它的定位是省你的确认点击，不是保你的安全。真正管放行的只有硬门禁：allowlist、沙箱、hooks、你的 diff 评审。
- **给模型生产环境凭据**：OTA 升级服务器 token、量产烧录座、云控制台账号，进了 agent 环境就等于进了它的行动半径。一句「帮我部署一下」配一把真钥匙，事故报告都不知道从哪写起。开发与生产的凭据物理隔离，agent 只碰开发侧。
- **整段日志塞进上下文**：串口跑了半小时，几千行日志一把糊给它，关键行被淹没，它开始一本正经地编「日志里有」的行。切片三招：tail 窗口取最后 200 行；按时间戳或锚点行（如 `MCLK=`）筛出关键段；分段喂，每段带一句「这是第几段、共几段」。给的是切片，但要给得有名有姓。
- **拿 7 个 reference servers 上生产**：官方 README 明示其定位是教学示例，"not as production-ready solutions"。学拓扑、跑实验尽管用；进产线、进团队流程之前，按第六节四问清单重新审一遍，或换生产级实现。
- **工具接上了，对账撤了**：内环全绿（编译过、它自己验证过）是最容易放松终审的时刻。它不认识你的板子这件事，与它能跑多少工具无关。绿灯只放行到对账台：三问照跑，上板照烧（A2）。
- **抄别人的 MCP 配置不审能力清单**：网上抄一段配置，server 一连，工具一多，看着挺热闹。接入即授权：每个 server 先问它暴露了哪些 Tools、读到什么路径、出不出网，四问过不完不授权。别人的配置堵的是别人的风险。
- **把 agent 的总结当交付**：它说「修好了，全部测试通过」，diff 里却混着三处顺手改动、一处警告被 flag 压掉。交付物以 diff、构建门与对账表为准，不以它的总结为准；总结是语气，diff 是证据。

短自测

五题全答得出，本章才算闭环；答不出的回到对应节复读。

1. MCP 的 Hosts/Clients/Servers 各自是什么？「一 server 一 client」的拓扑决定了什么？

▼ 看答案
Hosts 是发起连接的 LLM 应用（你的 VS Code、Cursor、Claude Code 工位）；Servers 是提供上下文与能力的服务（读文件、查 Git、抓网页各一个）；Clients 是 host 内部的连接器，host 为每个 server 建一个 client。这条拓扑决定授权的颗粒度：能力按 server 分批接入与授权，出问题按 server 定位与摘除；本地 STDIO server 通常只服务一个 client、权限随你的用户，远程 Streamable HTTP server 通常服务多个 client、你的数据要出机器。

2. Resources/Prompts/Tools 三类原语，各自对应嵌入式工具流里的什么素材？

▼ 看答案
Resources 是数据：RM0090 章节文本、原理图导出、仓库文件、串口日志；Prompts 是提示模板：A1 的三段式工单固化后全组复用；Tools 是可执行能力：跑一次 arm-none-eabi-gcc 构建、读一段串口日志。client 侧的 Elicitation 对应「不确定就提问」那条保险丝。另记一条时效：Sampling 与 Logging 两个 client 原语自协议版本 2026-07-28 起已 deprecated，旧教程的说法不要学。

3. 「你粘错误」与「它自己跑构建」差在哪三重损耗？为什么内环还给机器后，你反而更不能下班？

▼ 看答案
切片损耗（你粘最后 10 行，根因常在第一条）、时效损耗（你粘的是上一刻的现场）、语境损耗（文件、行号、编译命令、工程结构没跟过去）。内环还给机器后你从管道回到审计员：内环绿只证明语法与环境层面的正确，总线归属、共阳极性这类「你的板子」的事实，编译器与 agent 都不认识；对账与上板两个终审动作反而成了你的主战场。

4. Cursor 官方为什么说 "Auto-review is not a security boundary"？三层防线各管什么？

▼ 看答案
Auto-review 把白名单外的命令交后端 LLM 分类器判读，官方明示分类器会犯错，所以它不是安全边界，定位是省确认点击。三层防线：软约束（三段式、CLAUDE.md，管态度与质量；官方定性 context, not enforced configuration）、工具闸门（Run Modes、allowlist、沙箱，拦截执行但会漏）、硬门禁（hooks、构建门、对账三问、diff 评审，确定性检查，不绿不放行）。

5. 串口日志回喂的切片三招是什么？为什么说模型读日志不改变「测」的分工？

▼ 看答案
切片三招：tail 窗口取尾部（官方例 tail -200）、按锚点行或时间戳筛关键段、分段喂并标注段号。日志是固件自报的二手现象：固件说 I2S 初始化完成，不等于示波器上有 BCK 波形，日志与示波器冲突时信示波器。插串口线、抓日志、夹探头这些物理动作仍是你的活，工具流只改变「读」的分工，不改变「测」的分工。

对照表：本章概念 → 仓库与上游落点

概念	落点
agent loop（推理 → 行动 → 验证）	VS Code agents 文档（附录 E 来源卡 ai-copilot-vscode-overview，访问 2026-10-27）
auto-correct 循环	VS Code 官方博客 2025-02-24 agent mode 公告（ai-copilot-vscode-agentmode-blog）
edits files, runs commands	Claude Code 官方仓库 README（ai-claude-code-repo）
管道喂日志 `tail -200`	Claude Code overview 文档（ai-claude-code-overview）
MCP 定义、三角色、三原语、安全原则三句	MCP 官方规范，协议版本 2026-07-28（ai-mcp-spec）
一 server 一 client、STDIO/HTTP、两层划分、Sampling/Logging deprecated	MCP 官方架构页（ai-mcp-architecture）
reference servers 仅 7 个、教学示例口径、Registry	modelcontextprotocol/servers 仓库 README（ai-mcp-servers）
治理：2025-12 捐赠与 AAIF	Anthropic 官方公告（ai-mcp-anthropic-donation）
Run Modes 三档与「非安全边界」原话	Cursor Run Modes 文档（ai-cursor-run-modes）
工具调用次数无上限、Checkpoints 归 Git	Cursor Agent 文档（ai-cursor-agent）
CLAUDE.md 软约束与 PreToolUse hook	Claude Code memory 文档（ai-claude-code-memory）
板卡事实与手册章节号	S3；RM0090 §2.3 表 1、§7.3、§8.4
交叉工具链与构建命令	B0
E08 日志锚点 `MCLK=4096000`、PA_EN=PCA9557 bit1	P12，main.c:126 与 main.c:81
证据三元组与对账三问	A2
三段式工单与「不确定就提问」	A1
能/不能清单与两环结构	A0

你做到了

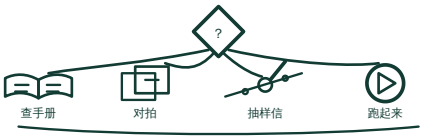
- 分得清搬运工模式与工具流模式：差的不是步骤，是闭环位置与你的角色；
- 能用三角色、三原语、一条拓扑向同伴讲清 MCP，知道它归谁治理、哪些实现只是教学示例；
- 三个场景各跑通一次：手册进 datasheets/ 只读授权、构建它自己转到绿、串口日志切片回喂出证据三元组；
- 手里有四问权限清单与三层防线：软约束管态度，硬门禁管放行，生产凭据永不进 agent 环境；
- 知道每条工具事实的出处与访问日期，界面改版时回官方文档查证，不靠记忆背书；
- 授权按风险爬坡：先本地后远程，先只读后写入，先单 server 后多 server。

✅ 工具流在手，你从人肉管道升进了终审席。下一站：**A4 幻觉防御**，工具越多，幻觉的伪装越讲究，去认一认那八张面孔；或回 A2 手册对账，把终审动作再练一遍。

通往单片机之路第 93 章

A4 幻觉防御：八类幻觉的类型学与验证决策树

难度 ★★★☆☆ · 约 45 分钟



本章罗盘

问题

模型编的寄存器名能有多像真的？

交付

八类幻觉类型学（A2 四类手册层 + 假库函数/假引证/假勘误/假芯片特性四类话语层）+ 「正确性定义在哪」验证决策树 + 每类最低成本验证动作清单。

边界

案例限于 S 篇/P 篇已学外设（GPIO/USART/LEDC/Matrix），不引新外设知识；A2 的对账流程不重复，本章只讲「错的样子」与分流入口；不评价具体产品。

依据锚点 `a1` · `ai-claude-code-memory` · `ai-copilot-github-cloud` · `mcu-code`

> 🎯 模型编的寄存器名能有多像真的？像到命名合乎规范、参数顺序自洽、注释比你工整，末尾还附一行手册章节号。A2 给了你验证的流程，本章给你另一半：「错的样子」的完整图鉴。八类幻觉、一条按「正确性定义在哪」分流的决策树、每类幻觉三十秒到十分钟的最低成本验证动作。读完这一章，模型的自信语气在你眼里会从证据降级为反指标。

本章精髓

1. **幻觉不是随机噪声，是平行宇宙里自洽的事实**。它拟合训练分布里最高频的形态（A0 的分布口径），并保持命名与格式的局部一致性：假函数名符合 HAL 命名规范，假引证符合手册格式，假勘误符合 errata 文体。读一遍抓不出来，只能拿判据对出来。
2. **先分流，再验证，按最低成本动作出手**。起点只有一问：这段说法的正确性定义在哪？定义在手册就逐位对账，定义在数学就复算对拍，定义在头文件就 grep，定义在经验就抽样信，哪都不在就跑起来看现象。存在性检查三十秒，逐位对账分钟级，上板终审十分钟级，永远从便宜的开始。
3. **软约束降概率，硬门禁挡漏网**。CLAUDE.md 与 AGENTS.md 的官方口径是「被当作上下文，不是强制配置」：写进去的纪律是往分布里加筹码，不构成拦截。能挡住幻觉的只有不经过模型意志的东西：手册、头文件、构建门、对账流程、上板现象。

- **能理解**：八类幻觉各自「像真的」的机制；为什么语料越稀、型号越冷，幻觉越敢凭空构造；为什么精确的引证（页码、条目号）反而最值得怀疑。
- **能用**：拿到任何一段模型输出，先按决策树分流到最低成本验证动作；把「它说得好像有道理」翻译成一条可检索、可对拍、可烧板的问题；说清软约束与硬门禁的分工。

先修

- A2 手册对账：对账三问与四类手册层幻觉（本章类型学以它为底，验证流程细则以它为准）；
- A0 结对工程师：「模型输出是分布，不是事实」的经验底座（第二节的机制从那里来）；
- A1 提示词模式：三段式与「明约束会被吞」（第五节软约束的上游）；
- S2 总线与时钟：AHB1 与 APB1 的归属判据（总线幻觉的标准答案）；
- S3 GPIO：板卡事实与 BSRR 位语义（案例靶子）；
- P2 GPIO 矩阵：IO_MUX 与 Matrix 的分工、毛刺过滤为 S3 新增（假芯片特性案例的判据）。

先跑起来（三道陷阱题，10 分钟）

开三个互不可见的会话（纪律同 A0：同一会话里追问，它会参考自己上一轮的答案，测到的就不是分布），各发一道，原样照抄。选题逻辑与 A0 一致：判据唯一可查（头文件、目录、位表各管一道），语料密度适中；三题都不需要烧板，成本压在十分钟内，产出是你对当前所用模型的第一份幻觉档案。

发题之前先押个命中率：三道题发出去，它会对几道？如果错，错的会是哪道？押完再发，记住你下这注时有多自信，回头拿存档对答案，看这份自信值几个钱。

> 题 1：STM32F4 的 HAL 库里，点亮 GPIOF 引脚 6 用 `HAL_GPIO_SetPin` 还是 `HAL_GPIO_WritePin`？给出函数原型与它所在的头文件名。
>
> 题 2：RM0090 里 GPIO 寄存器 BSRR 的位定义在哪一页哪一节？给出页码与章节号。
>
> 题 3：STM32F407 的 GPIO 输入毛刺滤波器怎么开？给出寄存器名与位号。

拿到回答先别信，当场用本章的最低成本动作验证：

题	它交出什么	你的动作	判据
1	一个选择加原型	`grep -n "HAL_GPIO_SetPin" stm32f4xx_hal_gpio.h`	搜不到即假：真身是 `HAL_GPIO_WritePin`
2	页码加章节号	打开 RM0090 目录检索	只信章节号：GPIO 位定义在 §8.4；页码随版本漂移
3	寄存器加位号加注释	§8.4 位定义表检索	位表里没有的东西不存在：毛刺过滤是 ESP32-S3 焊盘的特性（P2）

三种结果都正常。它全答对：这三道题在语料里出现频率不低，换更冷门的型号再问一轮；它答错：恭喜抓到现场，把输出存档，读完第一节给它归档。无论对错，注意它的语气：错的那些回答，语气与对的那些一样自信。语气不是证据，这是本章全部工作的前提。

把三道题的结果当场存档，这是你的幻觉案例库的第一页（四栏格式在第四节展开）：

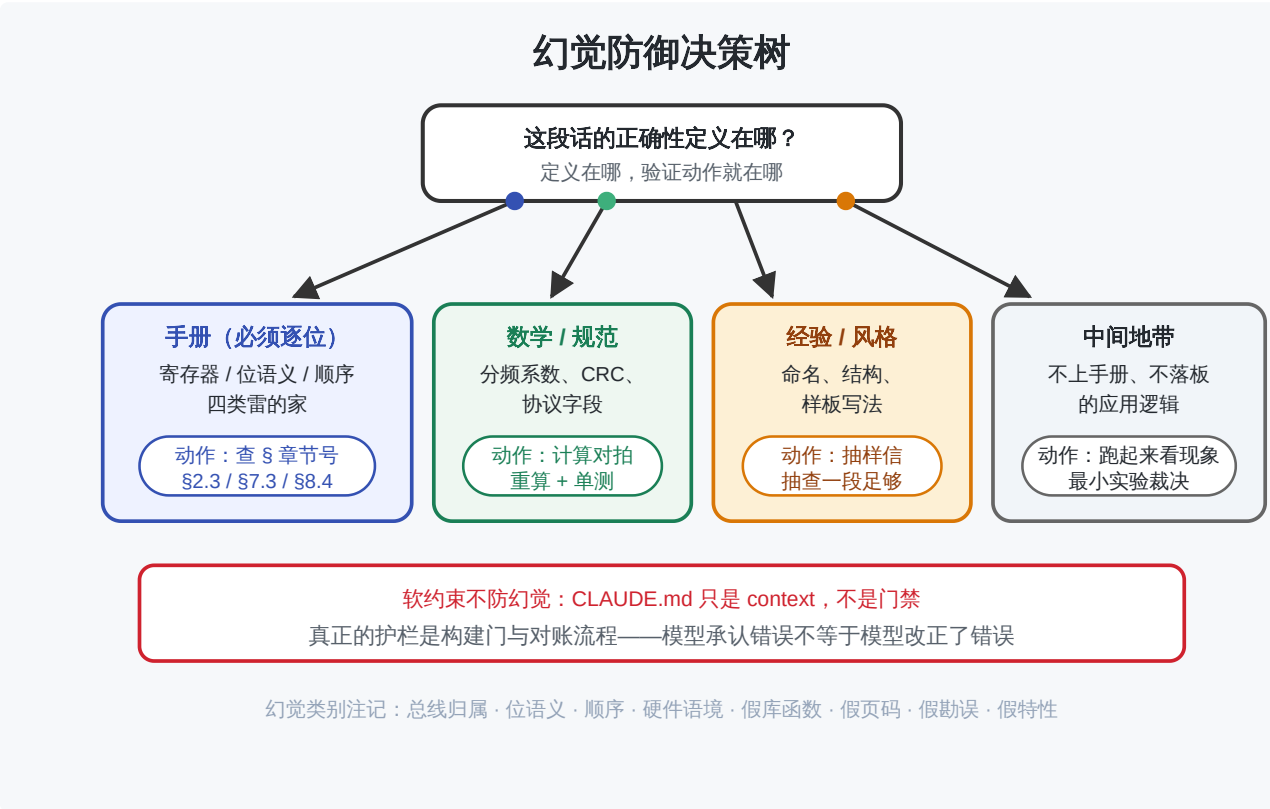
题	它的回答要点	验证结果	幻觉类型
1			
2			
3			

三道题各对应一类话语层幻觉：假库函数、假引证、假芯片特性。它们比 A2 的四类更隐蔽，因为它们错的不在代码，在证据。

判读时多记一笔信号来源。题 1 它主动告诉你真身是 `HAL_GPIO_WritePin`，与你自己 grep 出真身，是两种不同的可信度：前者说明这道题语料密，后者才是你的判据在工作。题 2 它给页码时有没有主动声明手册版本，也值得记下：声明了，说明它知道页码会漂；没声明还言之凿凿，你对它的引证类输出就该整体降一档信任。

动画：幻觉类型学与验证决策树

模型输出从顶端进入，先过第一层分流：错在代码与位的四类（手册层），错在证据与出处的四类（话语层）。每类幻觉挂一个最低成本验证动作：§号检索、头文件 grep、勘误表检索、特性表检索、上板看现象。树梢是模型的自信语气，树根是你手里的判据。



幻觉类型学与验证决策树：八类幻觉各挂一个最低成本验证动作

读这张图只记一条主线：每类幻觉都有一条判据通路，没有一类靠「再读一遍」兜底。

小节结构

一、八类幻觉全景：手册层四类加话语层四类；二、像真的机制：分布拟合与局部一致性；三、验证决策树：按正确性定义分流；四、最低成本验证动作清单；五、软约束与硬门禁：CLAUDE.md 不是护栏；六、一页速查：能贴在工位边的全章压缩。

一、八类幻觉全景：从写错代码到编出证据

A2 的第一节你在一段点灯代码里拆过四颗雷：总线归属、位语义、顺序、硬件语境。这四类有一个共同的落点：错在代码与位，判据在手册与原理图。本书管它们叫**手册层幻觉**。

本章补全另一半：**话语层幻觉**。模型不只写错代码，还会编出支持错误代码的证据：不存在的库函数、不存在的章节号、不存在的勘误条目、不存在的芯片特性。这四类的落点不是代码，是你的判断：它们伪装成判据本身。

为什么要一张类型学？因为逐个排雷是打地鼠，认识「错的样子」是看图识雷。同一类幻觉共享同一个判据与同一个最低成本动作：认出类型，动作自动跟上，不用再现想怎么查。这张表值得你打印出来贴在屏幕边上。

全景一张表：

#	类型	层	经典长相	判据在哪	最低成本动作
1	总线归属	手册层	GPIOF 挂 APB1（实为 AHB1，`RCC_AHB1ENR` bit5）	RM0090 §2.3 表 1	基地址检索
2	位语义	手册层	BSRR 高 16 位当置位（实为复位）	RM0090 §8.4	位表逐位对
3	顺序	手册层	先配 MODER 后开时钟	时钟先行铁律	时序审查
4	硬件语境	手册层	共阳当共阴，置位点亮	你的原理图	对照板卡事实
5	假库函数	话语层	`HAL_GPIO_SetPin(GPIOF, 6)`	头文件	grep
6	假引证	话语层	「RM0090 第 217 页，见 §7.4」	手册目录	§号检索
7	假勘误	话语层	编号、触发条件、规避方案齐全的 errata 条目	ST 勘误表	条目检索
8	假芯片特性	话语层	F407 的 GPIO 有输入毛刺滤波	数据手册特性表	特性表与位表检索

前四类的实锤与对账细则在 A2，不重复展开。本节把话语层四类逐个拆开：每一类给长相、给机制、给判据。

读这张表看三列就够：「层」决定从哪里下手，手册层从代码下手，话语层从出处下手；「判据在哪」决定动作；「最低成本动作」决定顺序，永远先做最便宜的。以后遇到八类之外的新长相，问自己同一问：它的判据在哪？答得出来就能塞进这张表，答不出来就走中间地带分支（第三节）。

假库函数：命名规范的外推。模型的函数名是按命名约定生成的，不是按头文件生成的。HAL 的命名是三段式（库前缀、外设、动作），于是 `HAL_GPIO_SetPin` 这种从未存在的名字被造出来时，形态上与真身 `HAL_GPIO_WritePin` 一致得无懈可击。三行对照，只有一行能活过头文件：

```
HAL_GPIO_SetPin(GPIOF, 6);                          /* 编造：HAL 没有这个函数 */
HAL_GPIO_WritePin(GPIOF, 6, GPIO_PIN_SET);          /* 名字对了，签名错了：掩码当引脚号 */
HAL_GPIO_WritePin(GPIOF, GPIO_PIN_6, GPIO_PIN_SET);  /* 真身加真签名 */
```

同一族的常见变体：`HAL_RCC_GPIOF_CLK_ENABLE()`，真身是双下划线开头的宏 `__HAL_RCC_GPIOF_CLK_ENABLE`，差两条下划线；`uart_send_bytes()`，ESP-IDF 的真身是 `uart_write_bytes`。假函数分两档：纯编造的，编译器多半能抓（隐式声明告警，前提是你没把它关掉）；名字真实存在但签名错位的，编译全过：第二行的 `6` 顺顺当当编过，可第二参数是引脚掩码，`GPIO_PIN_6` 是 0x0040，传 6 等于同时操作引脚 1 与引脚 2。后一档只能 grep 头文件对原型，编译器救不了你：C 语言里 `6` 与 `GPIO_PIN_6` 都是合法的 `uint16_t`，类型系统无话可说，语义错误活在类型之下。这也是为什么「它编过了」在库函数这层同样是最弱证据：编过只淘汰纯编造档，签名错位档全员放行。

抓到假函数后的回喂句式照 A2 证据三元组：「实测：stm32f4xx_hal_gpio.h 中不存在 HAL_GPIO_SetPin，写函数真身是 HAL_GPIO_WritePin，原型三参数为端口、引脚掩码、电平；请按真实原型重写并注明出处头文件。」出处具体到文件，它下一轮才有地方站稳；只说「函数不对」，它只会换一个猜法。

ESP-IDF 侧同理：下划线小写加动词后缀的命名约定一样可被外推，「理应存在」的路由类函数名最容易被造出来。判据同样是头文件而不是直觉，IDF 还多一层陷阱：信号路由往往没有公开 API，正确入口是外设驱动的引脚参数（`uart_set_pin`、`gpio_num`，P2 口径），它若给你一个直接操作 Matrix 的函数，先 grep 再谈。

假引证：精确格式免费赠送。模型对手册的内容有训练记忆，对手册的结构只有模糊印象，输出时却套上精确的格式：「RM0090 第 217 页」「见 §7.4」。页码随手册版本与 PDF 排版漂移，本来就不能当判据（A1 口径：引章节号，不引页码）；章节号更微妙，§7.4 这种编号看着离 §8.4 不远，恰好是「相邻宇宙的章节号」：格式全对，落点全错。判据只有一个：打开手册目录，检索章节号，对不上即假。引证的存在性，永远先于引证的内容被验证。

手册版本漂移值得单独说透。RM0090 有多个修订版，同一内容的页码随修订移动；模型的训练语料里混着各个版本的引用，它报出的页码可能真在某个旧版里存在过。这解释了假引证最迷惑人的一点：偶尔能对上。对上是巧合，对不上是常态，判据因此只能是你手里当前版本的目录，不是它的记忆。章节号稳定得多，这是全书坚持「引章节号不引页码」的原因（A1 上下文段口径）。

回喂句式：「实测：RM0090 当前版本目录中 GPIO 寄存器位定义在 §8.4，§7.4 查无此内容；请按当前版本章节号重新引用，页码不必给。」它若换一个新章节号补回来，一样先检索再信：引证类输出在你这里的默认状态是「未验证」，检索通过才改状态。

假勘误：最权威的幻觉文体。勘误条目的文体极有辨识度：编号、触发条件、影响、规避方案，四件套齐全。模型对这种文体的拟合近乎完美，于是它能交出这样的东西（以下为编造示例，请勿当真）：

> 「Errata 2.1.3：当 BSRR 的高 16 位与低 16 位在同一语句写入时，复位操作可能被忽略。规避方法：先读 ODR，再分两次写入。」

编号像、条件像、规避方案像，语气比手册还像手册。这类幻觉的杀伤方式很独特：它不往你代码里埋雷，它把你本来正确的代码「纠正」成错的，或者让你为一个不存在的芯片 bug 加上一段多余的规避逻辑。排查时你连怀疑对象都找不到，因为它披着判据的外衣。

为什么它最像真的？勘误文体在语料里高度模板化：编号、条件、影响、规避，四件套的格式几十年没变。模板化越强的文本，模型拟合越完美；而「某条目是否存在于某型号某版本」是稀疏事实，拟合最弱。格式最强项叠上事实最弱项，假勘误因此是八类里像真度最高的一类。判据：ST 官网该型号的 errata sheet，检索编号与关键词；检索不到，即编造；检索得到，再核适用条件与批次。

回喂句式：「实测：该型号现行 errata sheet 检索不到此条目编号，关键词 BSRR、ODR 均无匹配；请撤回该条目或给出可检索出处。」注意措辞是「撤回或给出处」，不是「你再查查」：前者把举证责任交回给断言方，后者只是邀请它再编一条。

假芯片特性：跨芯片移植。训练语料里，各家芯片的教程混在「MCU 的 GPIO」这一个大类下，特性在分布里互相串味。两个典型方向。其一，把别家的特性安进来：「F407 的 GPIO 输入可以开毛刺滤波」，而毛刺过滤是 ESP32-S3 焊盘的新增能力（P2），RM0090 §8.4 的 GPIO 寄存器表里根本没有这项配置。其二，把自家的限制说没了：「ESP32 的 UART 只能走固定引脚」，实际上 IO_MUX 有原生脚、Matrix 可把信号换岗到任意有效引脚，P2 的换岗实验用 `uart_set_pin` 与 LEDC 的 `gpio_num` 在 GPIO10 与 GPIO11 之间搬过信号。STM32 侧是固定 AF 复用表，ESP32-S3 侧是 IO_MUX 加交换矩阵，两边的真相都在手册里，只是不在同一本。

还有第三个方向：同一家族不同型号之间互串。STM32 家族型号众多，教程常不写型号后缀，分布里「STM32 的 GPIO」被压成一个大类，F1 时代的写法会被搬进 F4 的工程，别的 F4 型号的特性会被安到 407 头上。型号后缀写得越全，这一类越少（A1 的上下文段纪律在此兑现）；它照样编得出来。三个方向的判据同一套：数据手册特性表与对应外设章节的寄存器位表；ESP32 侧还有一条更硬的，芯片能力宏以 `soc_caps.h` 为准（P2 的口径），grep 一下比问它快。

回喂句式：「实测：RM0090 §8.4 GPIO 寄存器表无滤波配置位，该特性不存在于 F407；请按手册重写配置步骤。」特性类回喂多带一句「正确归属」效果更好：「该特性是 ESP32-S3 的焊盘能力」。点破它串了哪两颗芯片，比只说「没有」更能压住同类再发。

两类幻觉的防御思路相同：判据在代码之外，就在手册、头文件、勘误表、特性表里。区别只在话语层多一步：先验证「证据存在」，再验证「证据说的对」。跳过后一步会信错，跳过前一步会被骗。

两层不是非此即彼，有的雷两层都沾：`HAL_GPIO_SetPin(GPIOF, GPIO_PIN_6)` 这个调用，函数是假的（话语层）；就算函数存在，「置位点亮」对共阳板也是反的（手册层）。分层处理的意思是各查各的：先 grep 存在性，再对账极性，两道门都过才算数。

这八类不是天花板，是基线。工具换代会带来新长相，判据结构不变：凡断言，先问正确性定义在哪。遇到第九类的那天，按同一问归档进你的案例库。

二、像真的机制：分布拟合与局部一致性

为什么这八类都读不出来？两个机制，都来自你在 A0 亲手摸过的那个东西：分布。

机制一：分布拟合，高频形态是默认答案。模型对每个问题的回答，都是对训练分布里最高频形态的采样。APB1 在点灯教程里出现频率高，于是 GPIOF 被挂上 APB1；置位点亮是多数教程的写法，于是共阳板收到一句 `BSRR = (1<<6)`。由此得到幻觉密度的经验曲线：

语料密度	幻觉形态	例子
高（点灯、USART 回显）	选错常见写法	GPIOF 挂 APB1、BSRR 当置位
中（型号变体之间）	张冠李戴	把别家的毛刺过滤安到 F407
低（冷门外设、新版手册）	凭空构造	假勘误、假章节号、假函数

同一块板子的同一个问题，三次回答三个样，你在 A0 的先跑起来里见过。把那条实验结论平移到本章：分布给不出事实，只能给出候选；语料越稀，候选越野。

「同问不同答」与「幻觉」其实是同一枚硬币的两面：分布意味着每次采样都有方差，方差落在事实边界内叫「措辞不同」，落在边界外就叫幻觉。所以「它上次答对了」不构成这次的对：每次回答都是一次独立采样，判据必须每次都到场。这条推论直接决定你的工作习惯：对账不是新代码的入职体检，是每一次交付的例行检查。

机制二：局部一致性，谎言内部是自洽的。幻觉不是随手乱写。GPIOF 若真挂在 APB1，`APB1ENR` bit5 这个位置就「合理」：这个位的真实身份是 TIM7EN（RM0090 §7.3），同样言之凿凿。假函数名符合命名规范，假章节号符合手册编号习惯，假勘误符合 errata 文体，假特性符合「MCU 的 GPIO 都该有」的直觉。模型拟合最强的是格式与命名，最弱的是事实与现实的对应关系。

拿 A2 的 BSRR 雷走一遍局部一致性。「BSRR 低 16 位写 1 置位」这句本身是真的（§8.4 位定义如此），模型错在把它接到「共阳板点亮」这个它不知道的语境上。幻觉常常不是整句都假，是每一半都真、接起来才假：真的位定义，加真的置位动作，加一条只有你知道的板卡事实，等于一颗雷。这就是为什么逐句读查不出问题：每句都对，错在句子与你现实的接缝处。验证因此必须把语句接到你的现实上才算完，手册与原理图就是接缝处的两个锚点。

第一节那条编造的勘误示例，是局部一致性的极致形态，值得逐件拆：

- 编号「2.1.3」：符合 errata 编号习惯，格式真；
- 条件「高低 16 位同一句写入」：与 BSRR 的双半字结构自洽，像真；
- 影响「复位操作可能被忽略」：符合勘误的症状措辞，像真；
- 规避「先读 ODR 再分两次写入」：符合 workaround 文体，像真。

四件全像，整体是假：检索不到即不存在。每一层都经得起读、只有出处经不起查，这就是话语层幻觉的标准画像。

做个小实验你就服了：把一段含雷的模型输出拿给没学过对账的同事读，他能读出「通顺、专业、像教科书」，读不出任何错。这不怪他：通顺与专业是文本属性，对错是文本与现实的对应关系，后者只有判据能查，眼睛查不了。

两个机制合起来，推出本章最反直觉的一条：**语气与精确度是反指标**。读起来通顺、命名规范、注释自信、引证精确的输出，只说明它高分通过了「像」这一关；而「像」恰恰是拟合强度最高、成本最低的部分。页码、条目号、位号这些精确细节，在分布里是格式，不是事实。审计顺序要倒过来：越具体的引证越先查，越自信的断言越先验。

顺带划清嵌入式与纯软件的风险差。纯软件的假 API，编译与链接多半会炸；嵌入式的假位定义编译全过（编译器不认识你的板子，A2 口径），假引证与假勘误更彻底，它们不进代码，进你的脑子。手册层幻觉的受害者是固件，话语层幻觉的受害者是你的判断，后者的排查成本高一个数量级。

有人会问：模型升级能不能直接消灭这些？升级让拟合更好，错法会更隐蔽、格式会更工整，但「你的板子」永远不在它的分布里，事实对应这个弱项不靠升级补齐。所以类型学会随版本变老，具体的假函数名会换，判据不会：手册、头文件、勘误表、特性表，这四样在哪个版本都是终审。

三、验证决策树：这段说法的正确性定义在哪

A2 给了你一条验证流水线（对账三问加证据回喂），本章解决流水线的入口问题：不是每段说法都值得逐位对账，也不是每段说法都能抽样信。分流的判据只有一问：**这段代码或说法的正确性，定义在哪里？**

正确性定义在	典型说法	验证动作	成本量级
手册位定义	寄存器、总线、位语义、初始化顺序	逐位对账，三问全过（A2）	分钟级每寄存器
头文件与源码	库函数存在性、原型、宏定义	grep 头文件，对签名与注释	30 秒
手册引证体系	章节号、页码	目录检索章节号，页码当噪声	1 分钟
勘误表	芯片 bug、触发条件、规避方案	errata sheet 检索编号	分钟级
数据手册特性表	「这颗芯片有没有这项特性」	特性表与位表检索；ESP32 侧 grep `soc_caps.h`	分钟级
数学与规范	CRC、滤波、分频计算、协议状态机	复算加对拍：标准向量、单测	分钟级
经验与风格	命名、目录布局、注释口味	抽样信	秒级
哪都不在	工具链行为、运行时表现	跑起来看现象	十分钟级

逐行说透三个容易站错队的：

数学与规范分支。模型写的 CRC16、波特率分频、定时器 PSC 与 ARR，正确性有数学定义，不需要手册全程陪跑。A2 第五节的分级表已经把这条钉死：时序参数拿计算器重算，纯软件算法拿单测对拍。标准测试向量（比如 CRC 的已知输入输出对）比逐行读实现快，也比它可靠。

经验与风格分支。命名习惯、文件布局、注释风格，这类输出的正确性定义在团队约定里，不在手册里。抽样看一眼，不顺眼就改，别为它启动对账。对账的力气是稀缺资源，花在定义明确的地方。

中间地带分支。有些说法的正确性既不在手册里，也不是数学：「这个编译警告可以忽略」「这个函数在新版 IDF 里已弃用」「这样配时序裕量够」。判据只剩行为本身，动作也只剩一个：跑起来看现象。编译一遍看警告还在不在，烧上板看灯与波形，串口把寄存器读回值打出来（S3 读 IDR 的取证实验就是这条路）。中间地带的纪律：现象是唯一判据，模型对现象的解释仍是假设。

完整走一遍。模型说「这个警告可以忽略」。分流：正确性不在手册、不在数学，落中间地带。最小实验：保留警告编译一次，看清它指哪一行；那行若是它顺手加的调试残留，删掉；若是有意保留的暂存变量，问它为什么需要，答不出所以然也删。现象（警告指向的行）是判据，「可以忽略」只是假设。中间地带最容易被「它的语气很肯定」糊弄过去，纪律只有一条：先跑，再谈。

一段输出横跨多行是常态。模型一次交付初始化代码、一个 CRC 函数、一句「据 RM0090 第 217 页」、一组命名建议，拆开各走各的门：

断言	分支	动作
`RCC->AHB1ENR |= (1UL << 5)`	手册位定义	§7.3 对使能位
CRC16 查表实现	数学与规范	标准向量对拍
「RM0090 第 217 页」	手册引证体系	目录检索章节号
变量命名风格	经验与风格	抽样信

分流是逐断言的，不是逐文件的。分流错了代价是双向的：手册层的说法抽样信，会漏雷；风格层的说法逐位对，会累死。

两条入口纪律。其一，养成 30 秒分诊习惯：拿到输出先不读实现，只数断言：它声称了几个事实、引了几处出处、用了几个 API？每个断言当场贴一个分支标签，贴不出的默认升级到更严的分支（拿不准就当手册层处理）。分流错了可以纠正，漏分流不会自己报警。其二，决策树不替代对账三问，它是三问的分拣台：分到「手册位定义」的断言，走 A2 全流程，一步不减；分到别处的，按上表动作执行，执行完同样按证据三元组归档。

这张表还有一个隐藏读法：从上到下，判据的外置程度递减，你允许的信任可以递增。手册与头文件是硬判据，条条过；数学与规范次之，对拍过就算；经验与风格最软，抽样就够。信任的方向与判据的硬度同向，与模型的自信程度无关：它在风格题上唯唯诺诺、在手问题上斩钉截铁，也丝毫不改变两边的验证强度。

两个分流事故各看一眼。把手册层当风格信：「名字挺规范，应该没问题」，GPIOF 挂上 APB1，编译过、灯不亮，排查从零开始。把风格当手册对：给变量命名逐位对账，一下午对完零产出，真正的雷在别处置闲。分流的意义不是完美分类，是让每一类付出配得上它的风险。

最后说清本章与 A2 的边界：A2 讲流程，怎么验证；本章讲库，错的样子，加入口，先分流。两章共用同一套判据：手册与板卡事实。判据没有增加，增加的是识别速度与出手的便宜程度。

四、最低成本验证动作清单

四类话语层幻觉加中间地带，各配一个十分钟以内的动作。前四类各有一个窗口要跑：头文件、手册目录、勘误表、特性表，问法相同——先问在不在，再问对不对。总原则：**先存在性，后语义；先检索，后对位；先跑通，后深究**。按成本升序排列：

动作 1：头文件 grep（库函数类，30 秒）。

```
grep -n "HAL_GPIO_SetPin" stm32f4xx_hal_gpio.h    # 无输出：函数不存在
grep -n "HAL_GPIO_WritePin" stm32f4xx_hal_gpio.h  # 有输出：存在，继续对原型
grep -rn "uart_send_bytes" components/driver/     # ESP-IDF 侧同理
```

三层判据按序过：名字在不在（存在性）、原型对不对（签名：参数类型与顺序）、注释说什么（语义：它到底干什么）。grep 到名字只算过了第一层，`GPIO_PIN_6` 与 `6` 的差别活在第二层。搜不到的处理：把「头文件里没有这个函数，请改用真实存在的 API 或给出出处」回喂给模型，它多半会换真身；换完仍要 grep 第二遍，新名字一样要过这三层。

第二层「对原型」实操长这样：grep 命中 `uart_write_bytes` 后顺手把原型读全，参数是串口号、缓冲指针、长度、超时四个；模型给的调用若只传了三个实参，签名这层就没过。原型在头文件里只有一行，读它的成本以秒计，省掉它的代价可能是一次莫名其妙的队列阻塞或 HardFault。

动作 2：§章节号检索（寄存器与引证类，1 分钟）。打开手册目录，检索章节号。RM0090 的口径全书统一：§2.3 表 1 管存储器映射，§7.3 管 RCC 使能位，§8.4 管 GPIO 位定义。对不上即假引证；对得上，再看那一节的内容是否支持它的断言：章节号存在而内容不支持，是更精细的一种幻觉，一样按假处理。页码一律当噪声：版本一换，页码就漂。检索不到的处理：要求模型给出可检索的出处（文档名加章节号），给不出即弃用该断言；它道歉后补来的新引证，重新检索，流程不变，这道门对道歉免疫。要双保险就再翻一次 §2.3 表 1 的存储器映射：基地址落点与总线归属互为印证，A2 的总线幻觉就是这么定案的。

动作 3：勘误表检索（勘误类，分钟级）。ST 官网器件页面有该型号的 errata sheet（器件勘误文档），全文检索它给的条目编号与关键词。检索不到：编造，归档进你的幻觉案例库。检索得到：别急着照做，先核适用条件（芯片批次、封装、外设配置），勘误的每一条都有边界，拿着半截条件套自己的板子，等于替模型完成幻觉。检索到但条件不适用你的批次，归档为「真实但无关」：别为别人的批次加规避代码，多写的每一行规避逻辑都是未来的维护债。

动作 4：特性表检索（特性类，分钟级）。芯片有没有某项特性，权威出处是数据手册的特性清单与外设章节，不是教程，更不是模型。STM32 侧的终裁：对应外设的寄存器位定义表，位表里没有这项配置，特性就不存在（F407 的 GPIO 毛刺滤波即死于此条）。通过判据两条：特性清单列了名，或寄存器位表有对应位；两条都不满足，特性即不存在，无论它说得多像真的。ESP32 侧的终裁更直接：`soc_caps.h` 里的能力宏（P2 的口径：芯片能力以它为准，不凭印象），例如：

```
grep -n "SOC_LEDC" components/soc/esp32s3/include/soc/soc_caps.h
```

动作 5：跑起来看现象（中间地带，十分钟级）。最小实验三形态：编译（警告即证据）、烧板（灯与波形即证据）、读回（寄存器读回值即证据）。三形态按成本升序用：先编译，再读回，最后烧板；能在前一形态解决的，不推到后一形态。产出按 A2 的证据三元组归档：实测现象、手册出处、板卡事实。现象与它的解释冲突时，信现象：解释只是假设，现象是物理世界的判决书。

用钱的逻辑用这张清单：先花 30 秒 grep，通过再决定要不要逐位对账；先查存在性，通过再决定要不要信内容。倒过来做（拿到输出就逐位对），你会在假函数上浪费一次完整的对账：函数本身就不存在，位对得再细也是给幽灵验尸。

五个动作还有一个共同点：都不需要模型配合。它不在场，这些动作一样成立；它的道歉、解释、再保证，对这些动作的结果零影响。这是它们与「问它对不对」的本质区别。

五个动作汇总一表，越往下越贵：

动作	目标类	成本	通过判据
头文件 grep	假库函数	30 秒	名字在、原型对、注释合
§号检索	假引证、寄存器类	1 分钟	章节号存在且内容支持断言
勘误表检索	假勘误	分钟级	编号可检索且条件适用你的批次
特性表检索	假芯片特性	分钟级	特性清单或位表有据，ESP32 侧 `soc_caps.h` 有宏
跑起来看现象	中间地带	十分钟级	现象可复现，与断言一致

最后一件长期资产：个人幻觉案例库。每抓到一条，按四栏归档进你的笔记：类型（八类之一）、长相（它怎么说的）、判据（你拿什么戳穿的）、出处（手册章节号或头文件路径）。这个库是你的私人分布：它告诉你你常用的模型在你的领域爱在哪爆，比任何通用清单都准。攒到二十条，你对「错的样子」的直觉就建立了，分诊那 30 秒会越来越快。

五、软约束与硬门禁：CLAUDE.md 不是护栏

读到这里一个自然的想法：把「不许编造寄存器名」「引证必须给章节号」写进 CLAUDE.md，让模型自我约束。该写，但要先认清它是什么。官方文档的原话（能力描述，访问日期 2026-10-27）：

> Claude treats them as context, not enforced configuration. To block an action…use a PreToolUse hook.

它把 CLAUDE.md 这类文件当上下文，不当强制配置；想真正拦截一个动作，得用 hooks 这类机制（怎么配在 A3 工具流展开）。上下文的作用机制你在 A1 已经见过：往分布里加筹码，能显著降幻觉率，但明约束会被吞（A1 观察一：提示词里明写「推挽」，它照写开漏）。筹码不是闸门。

同口径的还有两句，一并摆上桌面。Cursor 运行模式文档："Auto-review is not a security boundary"（自动审阅不是安全边界，官方明说分类器会犯错；能力描述，访问日期 2026-10-27）。GitHub 的 Copilot 文档："Logs do not replace your own review and testing"（日志代替不了你的评审与测试；官方纪律口径，访问日期 2026-10-27）。三句连起来读：提示词文件不是门禁，自动分类不是门禁，运行日志不是门禁。

那什么是门禁？定义只有一条：**不经过模型意志、失败会自己报警的机制**。

三句官方口径放在一起，读出的不是厂商的谦逊，是责任结构：工具提供生成与上下文机制，你提供判据与终审。这与 A0 第一节的红线句是同一条款的两侧：那边说「输出可能有错，评审与测试是你的活」，这边说「约束文件不是强制配置」，合起来是一句完整的话：没有任何一层软件替你签字。按这个定义清点你的资产：

层	成员	作用
软约束层	CLAUDE.md、AGENTS.md、三段式提示词	降幻觉概率，往分布里加筹码
检索门禁	手册目录、头文件 grep、勘误表、特性表	话语层幻觉的死刑执行地
构建门禁	交叉编译 -Wall、lint、测试断言、构建门	语法与类型层面的自动拦截
对账门禁	三问清单、证据三元组（A2）	手册层幻觉的逐位审判
终审门禁	上板现象、示波器、读回值	物理世界的最终判决

门禁定义里「自己报警」四个字值得单独说。依赖你记得去查的清单不是门禁，是习惯；门禁的判据外在于模型意志，且失败状态主动可见：编译退出码非零、测试红、构建门拒、grep 无输出。对账表由人执行，为什么算门禁？因为判据不在模型手里：名对不对查 §7.3，位对不对查 §8.4，执行者是你，判决书是手册。人是执行者没关系，判据外置才算数。

给你一个当场分拣任何防御件的判据：模型完全不配合时，它还拦得住吗？拦得住是门禁：编译器不认识模型的道歉，grep 不接受模型的辩解，手册目录不理会模型的语气。拦不住是筹码：提示词里的禁令、对话里的约定、它自己的承诺，全都指望它的配合。AGENTS.md 与各类约定文件同理，一律归入软约束层；文件名与工具会换，层级不变。

两层思维收尾：软约束层负责让幻觉少发生，四层门禁负责让漏网的幻觉死在你手里。省掉软约束，门禁会被幻觉流量淹没；拿软约束当门禁，被吞的那条约束就是事故的入口。界面会变，方法论不变：文件名叫 CLAUDE.md 还是 AGENTS.md 会随工具换代，「上下文不是配置、门禁不经意志」这条不会。

这套分层不是纸上谈兵。本书自己的生产线就跑着同一结构：文稿层有写作约定与提示词（软约束），流水线层有构建门、链接检查、口癖计数（硬门禁），任何一章无论谁执笔，都过同一套不经意志的闸门。约束归约束，门禁归门禁，这条结构你读完 A5 会看到它如何被当成 AI 协作的案例本身。

六、把决策树揣进口袋：一页速查

全章压缩成一张能贴在工位边的清单，使用顺序就是阅读顺序。

第一步，分诊（30 秒）：

- 数断言：它声称了几个事实、引了几处出处、用了几个 API；
- 贴标签：每个断言归一条分支（手册、头文件、引证、勘误、特性、数学、风格、中间地带）；
- 贴不出的，默认升级到更严的分支，拿不准就当手册层。

第二步，按类出手（永远从便宜的开始）：

- 库函数：grep 头文件，存在、签名、语义三层全过才算；
- 引证：手册目录检索章节号，页码一律当噪声；
- 勘误：errata sheet 检索编号，再核适用条件与批次；
- 特性：数据手册特性表加位定义表，ESP32 侧 grep `soc_caps.h`；
- 寄存器：A2 对账三问，名、位、序全过；
- 数学：复算加对拍，标准测试向量优先；
- 风格：抽样信，不花对账成本；
- 中间地带：跑起来看现象，现象是唯一判据。

第三步，归档与回喂（1 分钟）：

- 证据三元组：实测现象、手册出处、板卡事实；
- 进案例库：类型、长相、判据、出处，四栏齐全；
- 回喂模型用证据句，不用情绪句；道歉后的新输出，过同一道门。

红线自查（每次交付前过一遍）：

- 我信了哪条只凭语气的断言？
- 哪条引证我没检索就放了？
- 哪个函数名我没 grep 就用了？
- 哪条特性我凭「应该有」就信了？
- 它道歉后给的新代码，过门了吗？

这张速查是压缩版，不是替代版：分诊卡壳回第三节，动作细节回第四节，软硬之辨回第五节。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **八类幻觉分两册：手册层错在位，话语层错在证据；分流只问正确性定义在哪，验证从最便宜的动作出手；CLAUDE.md 是筹码不是闸门，门禁必须不经过模型意志。**

延伸：分布口径与同问不同答实验在 A0；对账三问与证据回喂全流程在 A2；提示词怎么往分布里加筹码在 A1；hooks 与权限边界这类真拦截机制，A3 工具流展开。

常见坑

七条坑按悔意浓度排序，条条有案底：

- **把道歉当修复**：模型承认「你说得对」之后交出的新代码，仍是同一分布的又一次抽样，必须过同一道门。承认错误是文本行为，不是代码行为；它的歉意不计入任何判据。新输出到手，grep 照跑、章节号照查、对账照做。
- **用模型验证模型**：把 A 模型的说法贴给 B 模型问对不对。B 说「对」只是分布的又一个样本，两个分布互证不构成证据；判据只能是手册、头文件、勘误表与上板现象。
- **引证越细越信**：「§8.4 第 217 页」比「参考手册里写着」更危险。精确是格式，格式是拟合强度最高、成本最低的部分；引证的存在性永远先于内容被验证。
- **grep 到名字就收工**：存在性只是第一层。`HAL_GPIO_WritePin` 真实存在，可第二参数是引脚掩码不是引脚号，传 `6` 与传 `GPIO_PIN_6`（0x0040）的差别是操作引脚 1、2 还是引脚 6。签名与语义再对一层。
- **拿软约束当硬门禁**：在 CLAUDE.md 写明「禁止编造引证」就放心下班。官方口径它是 context；真拦截靠门禁：检索、构建、对账、上板，一层都不能省。
- **对账过一次就终身免疫**：同一块板卡换个外设、同一外设换个型号，分布与手册都要重新对。对账记录是你的资产，但只对当次断言有效；每次回答都是独立采样，「上次它对了」不构成这次的对。
- **替它补全出处**：看到「§8.4」三个字就先入为主，自己翻开手册替它对上了号。这是引证幻觉最舒服的落点：你的脑补替它完成了验证。纪律反过来：它引的，它负责可被检索；你只负责检索，不负责找补。

短自测

1. 模型说「STM32F407 的 GPIO 输入毛刺滤波器在 PUPDR 之后配置」，属于哪类幻觉？最低成本验证动作是什么？

▼ 看答案
假芯片特性：把 ESP32-S3 焊盘的能力安到了 F407 头上（毛刺过滤是 S3 新增，见 P2 GPIO 矩阵）。验证走特性类动作：RM0090 §8.4 的 GPIO 寄存器位定义表检索，位表里没有这项配置，特性即不存在；要双保险就再翻数据手册特性清单。判据的要点：芯片有没有某项特性由手册定义，不由「MCU 的 GPIO 都该有」的直觉定义。

2. 模型给出 `HAL_GPIO_SetPin(GPIOF, 6)` 并附注释「标准 HAL 用法」。这段代码里有几处错？各用什么动作验证？

▼ 看答案
两处。其一，函数不存在：HAL GPIO 的写函数真身是 HAL_GPIO_WritePin，grep stm32f4xx_hal_gpio.h 三十秒定案（存在性）。其二，就算换成真身，第二参数是引脚掩码：GPIO_PIN_6 是 0x0040，传 6 等于同时操作引脚 1 与引脚 2（签名与语义层）。教训：grep 到名字只过第一层，原型与注释才是后两层。那句「标准 HAL 用法」的注释不是证据，是语气。

3. 三样东西各走决策树的哪条分支：(a) 模型写的 CRC16 查表实现；(b) 模型说「RM0090 第 217 页写明 BSRR 高 16 位是复位」；(c) 模型给变量起的命名风格？

▼ 看答案
(a) 数学与规范分支：CRC 的正确性有数学定义，拿标准测试向量对拍加单测，不必翻手册。(b) 手册引证分支：页码当噪声（随版本漂移），检索章节号，对到 §8.4 再看内容是否支持断言；引证存在性先于内容。(c) 经验与风格分支：正确性定义在团队约定，抽样看一眼即可，不消耗对账成本。分流错了代价是双向的：手册层的说法抽样信会漏雷，风格层的说法逐位对会累死。

4. 假勘误为什么比假位定义更危险？拿到一条模型给的勘误条目，第一步动作是什么？

▼ 看答案
假位定义错在代码里，对账三问能抓到；假勘误伪装成判据本身，会把你本来正确的代码「纠正」成错的，或让你为不存在的芯片 bug 加多余的规避逻辑，排查时你连怀疑对象都找不到。文体上它最权威：编号、触发条件、影响、规避方案四件套齐全。第一步动作：ST 官网该型号 errata sheet 检索条目编号与关键词，检索不到即编造；检索得到，先核适用条件（批次、封装、配置），再决定动不动代码。

5. 在 CLAUDE.md 里写明「所有手册引证必须给章节号，禁止编造」之后，为什么不能省掉检索这道动作？用官方口径回答。

▼ 看答案
官方文档明示：Claude 把 CLAUDE.md 这类文件当作 context，而非 enforced configuration（能力描述，访问日期 2026-10-27）。写进去的纪律是往分布里加筹码：能降概率，不构成拦截，A1 的「明约束会被吞」是同一现象的提示词版。拦截幻觉的是不经过模型意志的硬门禁：§号检索、头文件 grep、构建门、对账与上板。软约束与硬门禁各司其职：前者让幻觉少发生，后者让漏网的死在你手里。

对照表：本章概念 → 仓库与上游落点

概念	落点
手册层四类幻觉	A2 第一节实锤（总线/位语义/顺序/语境）
分布口径	A0 先跑起来：同问不同答的一致性实验
明约束会被吞	A1 观察一；软约束机制的提示词版
假芯片特性判据	P2：毛刺过滤为 S3 新增；AF 固定表 vs Matrix；能力宏以 `soc_caps.h` 为准
板卡事实	S3：PF6/7/8 共阳、GPIOF = 0x40021400、`AHB1ENR` bit5
手册引证口径	RM0090 §2.3 表 1 / §7.3 / §8.4：引章节号，不引页码
软约束官方口径	Claude Code memory 文档「context, not enforced configuration」（能力描述，访问日期 2026-10-27）
硬门禁体系	检索/构建/对账/终审四层；对账流程见 A2，hooks 与权限边界见 A3 工具流
中间地带判据	上板现象与读回值；S3 读 IDR 取证实验
个人幻觉案例库	本章第四节四栏格式；A0 实验报告存档口径
一页速查	本章第六节：分诊、出手、归档、红线四步，可打印贴工位

你做到了

合上这一章，自检五条：

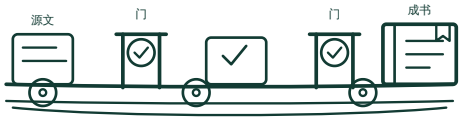
- 拿到任何一段模型输出，分流问题先于阅读：这段说法的正确性定义在哪？
- 八类幻觉的「像」骗不过你了：通顺、规范、自信、精确，这四样只是拟合强度的读数；
- 验证动作有了成本序：30 秒 grep 存在性，1 分钟检索章节号，分钟级对特性表与勘误表，逐位对账留给真正需要它的寄存器；
- 软约束降概率、硬门禁挡漏网，你不再把筹码当闸门；
- 手里多了一页能贴工位的速查：分诊、出手、归档、红线自查，四步成环。

✅ 八张面孔认齐，自信的语气从此在你眼里只是反指标。下一站：**A2 手册对账**，把三问清单练成肌肉记忆；或 **A3 工具流**，看 hooks 与权限边界这类真拦截怎么配。

通往单片机之路第 94 章

A5 这本书是怎么和 AI 一起写成的：生产线的自证

难度 ★★☆☆☆ · 约 45 分钟



本章罗盘

问题

「把大模型当结对工程师」到底灵不灵？这本书本身就是实验。

交付

本书 AI 生产线解剖：八道工序与十几道门、三个真实事故与长出的门（滤镜光栅化/幽灵连字符/数字漂移）、AI 的四个角色（采集/起草/评审/对账）、每条论断配复现命令的度量表、四条 AI 不许碰的边界。

边界

讲工作流证据不讲工具安装；行号引证以成稿当期仓库为准；「待上板实测」纪律不因自动化而豁免；门的阈值只许变严不许变松。

依据锚点 `mcu-code` · `ai-copilot-vscode-overview`

> 🎯 前四章教你把大模型当结对工程师。这一章把被告叫上证人席：你手里这本书，从起草到印前，就是按那套方法论由 AI 和人结对生产的——而且它给自己装了十几道机器门。自证不是炫技，是把「模型给假设、人拿终审权」落成可复现的命令。你在这一章读到的每个论断，都能在仓库里跑一条命令复现。

本章精髓

1. **软约束挡不住流水线事故，硬门禁才挡得住**。AGENTS.md 里的写作约定是 context，真正能拦住缺陷的是门：印前 13 门、契约测试、口癖门、题图守卫、数字声明对账：每一道门背后都有一次真实事故，本章逐个解剖。
2. **AI 写的文字要过口癖门，AI 参与的构建要过证据门**。口癖有预算（破折号散文面 ≤2/章），事实有指纹（事实 token 多重集）：`—` 超了是风格问题，「27000 个 tick」被润色成「2.7 万个」是事实事故，两种问题各有一道机器门。
3. **自证的意义是让方法论变成公共财产**。A0 讲定位、A1 讲提示词、A2 讲对账、A3 讲工具流、A4 讲幻觉防御，这些不是道理，是这本书生产线上的工作记录：每一章的写作过程本身就用了那一章的方法。

- **能理解**：为什么「相信 AI」的反面不是「不信 AI」而是「给 AI 的输出装门禁」；一道机器门如何从一次事故里长出来；为什么软约束（风格指南、记忆文件）必须有硬门兜底。
- **能用**：给自己的项目搭一条最小生产线：一个可复现的构建 + 一条数字声明对账 + 一个事实指纹脚本。

先修

- A0 结对工程师：模型给假设、人拿终审权的定位；
- A2 手册对账：对账三问与证据三元组（本章的「门」就是对账三问工业化后的样子）；
- A3 工具流：MCP 与工具链（本章的评审通道与采集通道）。

先跑起来（5 分钟）

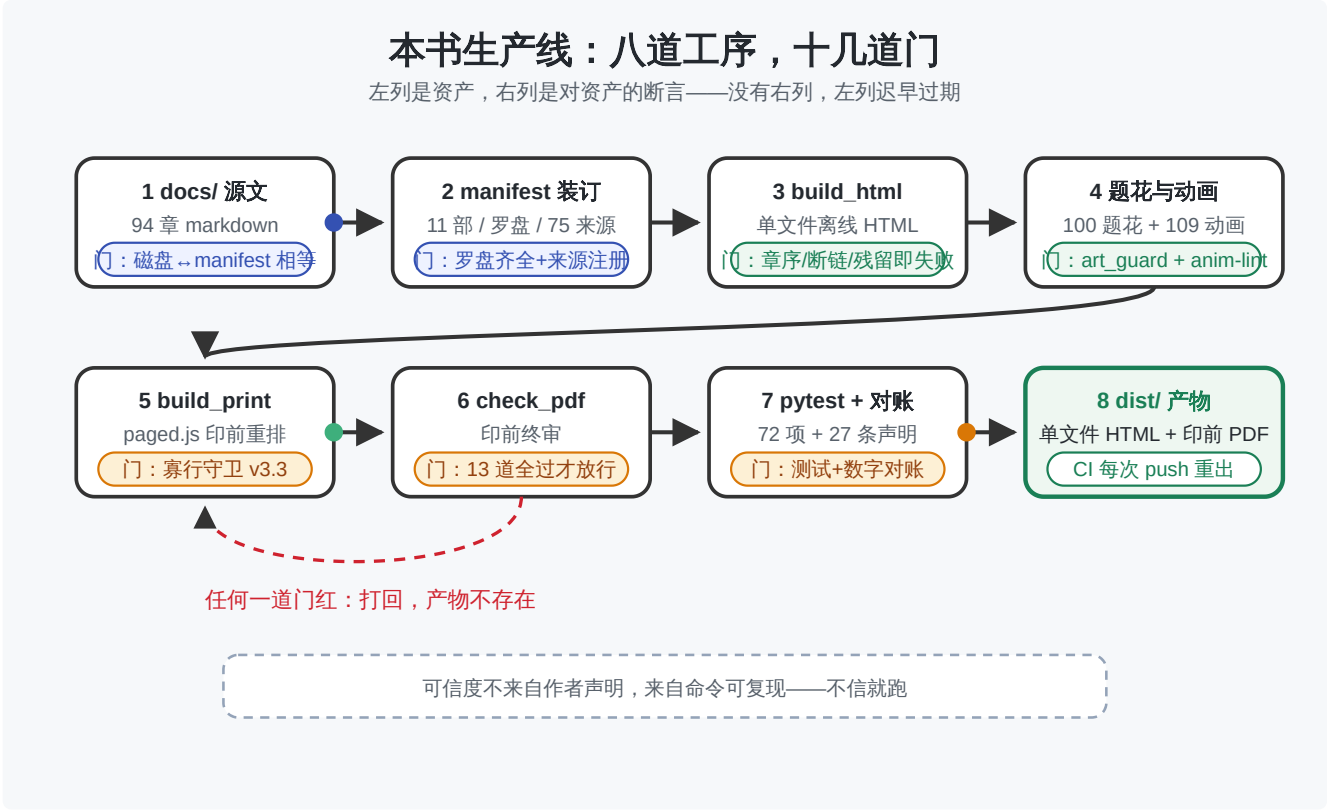
打开终端跑三条命令，看这本书的生产线现场：

```
python mcu-book/check_pdf.py mcu-book/dist/mcu-deep-book-print.pdf   # 印前 13 门
python mcu-book/tools/prose_guard.py --single docs/ai/02-manual-reconciliation.md   # 口癖门
npm run docs:check   # 数字声明对账（27 条手写数字 vs manifest 真相）
```

三条命令各有退出码。这三条就是「这本书可信」的全部理由：不是因为它是谁写的，而是因为每条论断都有门守着。

动画：生产线全景

正文源文 → manifest 装订 → 契约对账 → 印前重排 → 13 门终审 → 产物；每道工序一台门，令牌沿着流水线走，任何一道门红，产物就不存在。



本书生产线：源文、构建、门禁与产物

小节结构

一、生产线全景：八道工序与它们的门；二、三个真实事故解剖（门是怎么长出来的）；三、AI 在生产线上的四个角色（起草/研究/评审/对账，附本任务运行实录）；四、度量：一张每条都能复现的指标表；五、边界：哪些环节 AI 不许碰。

一、生产线全景：八道工序，十几道门

这本书的生产线不是一条「写完就发」的管道，是八道工序、每道工序至少一台机器门：

工序	工具	守它的门
正文源文	`docs/` 91 章 markdown（成稿时 94 章）	磁盘↔manifest 集合相等（`test_build.py`）
装订清单	`manifest.json`（11 部 / 罗盘 / 75 项来源）	罗盘四字段非空、来源 ID 已注册
离线 HTML	`mcu-book/build_html.py`	构建期断言：章序/罗盘数/附录集/零断链/组件残留即失败
题花与插图	100 枚题花 + 109 张机制动画	`art_guard`（纯矢量、≤6000B、笔压分级）+ `anim-lint`（色板映射、连续动画）
印前重排	`mcu-book/build_print.py`（paged.js + Chromium）	寡行守卫 v3.3（断点前移 + 边界抬升）
终审	`mcu-book/check_pdf.py`	**13 道门**：字数、字体、页眉页码、目录页码、U+2011、灰点矢量扫描……
回归测试	`mcu-book/tests/`	pytest 72 项（契约、守卫、门禁自身）
站点一致性	`scripts/docs-check.mjs`	27 条手写数字声明 ↔ manifest 真相逐条对账

注意这张表的结构：**左列是资产，右列是对资产的断言**。没有右列，左列就是一堆迟早过期的文件。这本书能宣称「与在线站点同源」，靠的不是承诺，是 `docs-check.mjs` 每跑一次就把 README、入口页、维护说明里的手写数字抓出来跟 manifest 对一遍账。

二、三个真实事故：门是怎么长出来的

门不是设计出来的，是事故撞出来的。三个例子，全部有仓库现场可查。

事故一：滤镜题图被印前光栅化

章首题花的手绘感最初靠 SVG 滤镜（feTurbulence 抖动）表达。直接内联进 HTML 没问题，但走印前路径时，Chromium 把带滤镜的题图**整块光栅化**，全书平白多出 94 张位图，印刷分辨率与文字可搜索性净损。更糟的是另一条流水线会话把滤镜版题图批量写回过目录，对抗发生。

长出的门：题花一律纯矢量（零 filter/id/url/href），`mcu-book/tools/art_guard.py` 全目录把守（viewBox 320×88、≤6000B、笔压至少两档、纯 LF），`build_html.py` 装配期再加一道硬门：带 feTurbulence 的题图直接构建失败。回归测试 `tests/test_print_vectors.py` 钉死「印前产物不得出现题图位图」。

教训（A4 决策树的叶子一）：「图画得对不对」的正确性定义在印前产物里，不在屏幕预览里，所以验证动作是矢量扫描产物，不是看图。

事故二：断点处的幽灵连字符 U+2011

印前重排用 paged.js 的分页引擎。寡行守卫要把「页首只剩一行」的段落断点前移一行，实现上要在溢出 Range 上设边界。有一类边界落在文本节点偏移 0，paged.js 的 hyphenateAtBreak 取 `prevLetter` 得到 `undefined`，`String(undefined)` 是字符串 `"undefined"`，首字母 `u` 恰好命中单词字符正则，于是页尾被悄悄追加一个连字符 U+2011。系统字体回退渲染它，字体门当场红。

更深的事故还在后面：修掉连字符后，元素形式的边界 `(code, 0)` 会在上一行末尾留下一个**空 `<code>` 元素**：inline-code 的背景色块凭空出现在行尾，文字检索完全看不见（空元素没有文本）。三处现场（p492/p577/p716）用像素级采样才抓到：行尾约 2.5×11pt 的灰色小片。

长出的门：寡行守卫 v3.3 把边界「抬升」过内联元素（`build_print.py:306` 的 walk-up），外加全书矢量扫描：`get_drawings()` 按填充色 `#f2f2f2`、宽 `<6pt`、高 `>7pt` 全书搜空内联灰点。这道扫描现在是印前电池的常驻成员。

教训：幽灵缺陷的共同形态是「肉眼看不见、文本检索看不见」。对这类缺陷，验证动作必须落在渲染产物或矢量结构上，而不是源文。

事故三：手写数字与真相漂移

新增第十一部时，构建器里钉着一个常量 `EXPECTED_CHAPTERS = 88`。接线的第一天它就在五处断言里把「加一章」变成「改五个数」。同一天还发现：README、入口页、维护说明里的手写数字（章数/部数/附录数/来源数/动画数）全部滞后于真相，`docs-check` 一口气报了 24 处不符。

长出的门与修法：常量删除，改为 `chapter_count(manifest)` 从清单推导（`build_html.py:49`）；测试断言同步改为推导式；数字声明交给 `docs-check.mjs:340` 的 27 条 CLAIMS 对账：连「历史上 v1.0.0 是 88 章」这样的句子都要改写成不命中正则的措辞，因为门的语义是「任何命中的数字都必须等于当前真相」。

教训：手写数字是债。能推导的推导，不能推导的（比如历史版本号）就改写成「当时的」口径，并用门的正则形状约束自己。

一个附赠事故：对账章引错手册

A2 是全书的「引证纪律」章，初稿里写着「查 §7.3 存储器映射」。评审通道抓出来：存储器映射是 RM0090 §2.3 表 1，§7.3 是 RCC 寄存器定义。错处当场分家：基地址区段引 §2.3，使能位引 §7.3，正文、对账表、自测答案、动画注释、manifest 罗盘边界八处同步改正。

这条值得单列，因为它就是 A2 自己的论点：模型（这次是给书起草章节的模型）给的引证是假设，判据在手册。对账章都没豁免，你的代码当然也不豁免。

三、AI 在生产线上的四个角色

这本书的第十一部是这样生产出来的（全部过程在仓库 `.trellis/tasks/10-27-ai-dev-deepen/` 里留痕）：

角色一：研究采集。研究代理只读官方一手文档（厂商文档/规范仓库/官方博客），产出 15 条带来源、快照与访问日期的引用条目，营销话术逐条隔离。对应你在 A0 学到的「来源卡纪律」：采集时就把「可引」和「好听」分开。

角色二：批次起草。章节由起草代理按罗盘（问题/交付/边界）+ 试点章样板成批撰写，每章落盘即过 `prose_guard --single` 口癖门与链接白名单。运行实录里最值钱的一条纪律是 **file-first**：子代理 15 分钟运行时上限，两个代理都曾因调研耗尽上限而零产出；恢复指令只有一句：先落完整骨架，再打磨。这条纪律你在任何长任务里都该用。

角色三：评审通道。独立的评审代理对每一轮改动给出意见，本轮一共采纳 7 条（引证分离、无效 SVG 动画属性、旧计数漂移、站点门禁、隔页覆盖测试等）、驳回 3 条过时或误报。评审意见不是命令，是要逐条验真的输入；其中「试点章引错手册」那一条，直接改写了 A2 的八处引证。

角色四：对账与终审。所有数字/版本/行号引用在成稿时逐条核过当期仓库；每条论断给复现命令；终审权在人，这是 VS Code agents 文档的红线句在本书生产线上的落实。

四个角色合起来，就是 A0 那张「能/不能清单」的工业化形态：能起草、能采集、能评审、能对账；不能替你做终审、不能替你上板、不能替你为错误负责。

四、度量：每条论断配一条命令

论断	复现命令
印前 13 门全过	`python mcu-book/check_pdf.py mcu-book/dist/mcu-deep-book-print.pdf`
契约测试 72 项	`python -m pytest mcu-book/tests/`
题花 100 枚 0 违规	`python mcu-book/tools/art_guard.py --dir mcu-book/book/assets/chapter-art`
动画 109 张全过 lint	`node scripts/anim-lint.mjs`
数字声明 27 条对账	`npm run docs:check`
新章口癖预算内	`python mcu-book/tools/prose_guard.py --single <章文件>`
站点 127+ 页全可达	`npm run nav:check`
寡行/孤儿/幽灵字符全 0	印前电池（寡行检测 + 孤儿标题直扫 + U+2011 全文检索 + 灰点矢量扫描）

这张表的正确读法：不是「这本书有 13 道门所以可信」，而是「不信的时候，你自己跑」。可信度不来自作者声明，来自命令可复现，这正是全书从 C 篇第一章就在用的方法论，现在它回过头来担保这本书自己。

五、边界：哪些环节 AI 不许碰

生产线再自动化，有几道环节 AI 永远不许单独碰：

1. **手册事实的终审**。模型可以读手册、抄位表，但「这个位是什么」的最终判定必须落在手册原文与你的复核上。A2 的对账三问没有代理。
2. **上板实测**。凡标「待上板实测」的论断，只有仪器说了才算数。模型可以帮你写测试代码，不能替你看见示波器。
3. **出版裁决**。门禁全绿是出版的必要条件，不是充分条件。发不发布，人说了算。
4. **门的阈值放宽**。阈值只能变严不能变松，这是防「为了过门而松门」的元规则。本轮所有阈值的变动历史都在仓库里，每一条都是收紧。

这四条合起来，就是 A0 里那句红线的完整版：模型对产物负责的部分是「生成与建议」，人对产物负责的部分是「判定与发布」。

记忆锚点

> 💡 **提示 · 一句话记住**
>
> **这本书就是证据：软约束防不住事故，硬门禁才防得住；每道门背后都有事故现场；不信就跑命令——可信度不来自作者声明，来自可复现。**

延伸：方法论的前四章是 A0 定位、A1 提示词、A2 对账、A3 工具流、A4 防御；门的完整清单与阈值史见仓库 `mcu-book/README.md`；任务过程档案在 `.trellis/tasks/10-27-ai-dev-deepen/`。

常见坑

- **把软约束当门禁用**：CLAUDE.md/AGENTS.md 是 context，不是门禁：写得再严，模型也只是「通常照做」。防线要落在可执行检查上（脚本 + 退出码）。
- **门禁只建不审**：门会漂。数字声明手写一次就过时一次，要么推导（chapter_count），要么对账（docs-check），不许靠记忆维护。
- **评审意见照单全收**：评审代理也会过时（它的意见基于更早的仓库状态）。每条意见先验真再采纳，本轮 10 条里 3 条被驳回。
- **子代理长任务不设 file-first**：调研到运行时上限、零落盘是真实发生过的失败形态。骨架先行，加深靠追加。
- **给 AI 的职责越界**：让模型「顺便决定发不发版」是边界事故。终审与发布永远在人。

短自测

1. 这本书「可信」的理由是什么？为什么说这个理由与作者无关？

> ▼ **看答案**
> 理由是每条论断都有一道可复现的机器门守着：印前 13 门、契约测试、口癖门、题图守卫、数字声明对账，全部给出命令与退出码。与作者无关的含义：可信度不来自「这是谁写的/谁审核的」这种声明，而来自「不信你可以自己跑」的可复现性。作者声明可以被编造，退出码不能。

2. 事故二里幽灵连字符 U+2011 与灰色小片为什么文本检索抓不到？各自的正确探测手段是什么？

> ▼ **看答案**
> U+2011 是渲染产物里的字符：它由分页引擎在断点处临时追加，源文里根本不存在，搜源文永远搜不到；探测手段是检索印前产物（PDF 文本层）或字体门的回退报告。灰色小片是空 code 元素（inline-code）的背景色块：空元素没有文本，文本层同样看不见；探测手段是渲染产物的矢量扫描（get_drawings 按填充色与尺寸搜）。共同教训：幽

灵缺陷的探测必须落在产物上，不是源文上。

3. 手写数字为什么是债？本书怎么处置它？

▼ 看答案
手写数字与真相之间没有任何强制同步机制，真相变一次它就过时一次，这就是债。处置分两类：能推导的改推导：章数从 manifest 求和（chapter_count），进度块由 docs:gen 扫描生成；不能推导的（历史版本的真实数字）改写成「当时的」口径，让数字声明门的正则不再命中它。门的语义是任何命中的数字必须等于当前真相。

4. 第十一部生产过程中的 file-first 纪律是什么？它治什么病？

▼ 看答案
file-first：子代理先落完整骨架（全部必备结构段落齐、正文可到七成），再迭代打磨，落盘优先于完美。它治的病是「调研耗尽运行时上限、零文件落盘」：本轮两个起草代理都曾因此撞 15 分钟上限被取消，恢复指令就是这句纪律。推广到任何长任务：先有一个可构建的整体，再加深局部。

5. AI 在本书生产线上有哪四个角色？哪一个环节它永远不许单独碰？

▼ 看答案
四角色：研究采集（官方文档带来源卡）、批次起草（罗盘+口癖门+链接白名单）、评审通道（意见逐条验真后采纳）、对账终审辅助（数字/行号核当期仓库）。永远不许单独碰的环节有四个：手册事实终审、上板实测、出版裁决、门的阈值放宽：前三个是终审权问题，第四个是防「为过门而松门」的元规则。

对照表：本章概念 → 仓库与上游落点

概念	落点
印前 13 门	`mcu-book/check_pdf.py`（13 个 check 函数，main 在 :619）
寡行守卫 v3.3	`mcu-book/build_print.py:80` WIDOW_GUARD_JS；:306 边界抬升 walk-up；:531 注入
口癖门（单文件）	`mcu-book/tools/prose_guard.py:229` `--single`（——散文 ≤2/章）
题花守卫	`mcu-book/tools/art_guard.py`（320×88/≤6000B/纯矢量/笔压两档）+ `build_html.py` 装配硬门
数字声明对账	`scripts/docs-check.mjs:340` CLAIMS 27 条
章数推导	`mcu-book/build_html.py:49` `chapter_count(manifest)`（替代 EXPECTED_CHAPTERS 常量）
隔页题图覆盖门	`mcu-book/tests/test_part_art.py`（manifest×part_art 非空、零滤镜）
任务过程档案	`.trellis/tasks/10-27-ai-dev-deepen/`（prd/design/implement/研究/评审记录）
§2.3/§7.3 引证分离	`docs/ai/02-manual-reconciliation.md` 工具箱节；评审记录见任务档案

你做到了

- 能给自己的项目搭最小生产线：一个可复现构建 + 一条数字对账 + 一个事实指纹脚本；
- 会区分软约束与硬门禁，并知道每道门背后应该有一次事故记录；
- 读完了这本书的「出生证明」：它的每一页，都是它教你的方法论跑出来的。

✅ 第十一部收官，全书收官。下一站：**附录区**：附录 A 工程深读看帧协议的五个契约（生产线门的最完整范例）；附录 E 看 75 项冻结来源的注册表，本章每条论断的出处都在那里等你复查。

通往单片机之路APPENDIX

附录 A 工程深读：从 IDLE 中断到完整帧协议的五个契约

本书 S7 USART、S8 DMA、C5 函数指针、状态机与环形缓冲、C8 串口协议 与 J1 记录器 各讲了这条链路的一段。这一附录把它们接成**一条完整的字节旅程**：从线路上的一个电平跳变，到业务层手里的一帧数据，中间每一段都签着一份"合同"。违反任何一条，链路都会以一种典型症状死给你看。而症状往往出现在离违约现场很远的地方。

三份配套工程是本附录的证据源：`code/stm32/03-uart-dma`（IDLE+DMA 环形接收的寄存器级最小实现）、`code/stm32/05-framed-protocol`（帧协议落地）与 `code/common/reliability`（双板共用的 codec 与传输层，测试在 `tests.c`）。

0. 全景：一个字节的三段旅程

```
线路 ──USART──▶ DMA 环形缓冲 ──ISR──▶ 字节队列 ──解析器──▶ 帧 ──▶ 业务
       [硬件]       [S8]          [C5]            [C8]        [J1]
```

段	所有权人	这一段的"唯一真相"	违约的典型症状
线路 → DMA 缓冲	DMA 控制器	`NDTR`（剩余计数）	数据撕裂、整段丢失
DMA 缓冲 → 字节队列	UART/DMA 中断服务	环形读写指针	重复搬运或漏段
字节队列 → 帧 → 业务	解析器状态机	`0x00` 分隔符 + CRC	两段残帧拼成一段"合法"帧

发送方向是同一张表的镜像（业务 → TX 队列 → USART），契约相同，只是"消费者"换成了线路。下面逐条展开五个契约。

契约一：DMA 只认地址，不认语义

DMA 是一条"给地址就搬"的传送带。它不知道什么是"一帧"，也不知道你的缓冲区有几个逻辑区段。它只知道三件事：从哪搬（`PAR`）、搬到哪（`M0AR`）、还剩多少（`NDTR`）。所以第一份合同签给配置顺序与读数口径：

1. **改配置必须先停流。** `CHSEL` / `DIR` / `CIRC` 这些位只有在 `EN=0` 时才允许改（这是 RM0090 的硬性规定，不是风格建议）。03-uart-dma 的初始化顺序就是"关流 → 配 PAR/M0AR/NDTR → 配控制位 → 最后置 EN"。
2. **写位置是从 NDTR 减出来的。** 环形模式下，DMA 写到了哪里：

```
c uint32_t pos = (RX_BUF_LEN - DMA_S5_NDTR) % RX_BUF_LEN;
```

`NDTR` 是**剩余量**不是已写量；跑满一圈回装原值，所以要取模。读它的时机也有讲究：先读 NDTR、再读 HISR；如果采样间隙恰好跨了一次回绕，事后读到的标志位能提示你"刚才那一下越界了"。 3. **清标志写 HIFCR，不碰 HISR。** Stream5 的标志在高位寄存器（HISR/`HIFCR`），且 HISR 只读，往只读寄存器写 1 什么也清不掉。这是新手"标志永远清不掉、中断一进就是风暴"的第一根因。 4. **"DMA 完成"不等于"CPU 看见"。** 传送完成中断只证明 DMA 把请求做完了；CPU 要读到最新数据，还隔着缓冲区所有权这一层（契约三）。在带 D-Cache 的芯片上（如 H7、S3 的可用 DMA 内存区域限制）这一条还要再拆成"可达性"与"一致性"两个独立问题。

违约方式	症状	在哪一章展开
EN=1 时改 CHSEL	通道根本没换，数据进了别的流	S8
忘掉 `% RX_BUF_LEN`	一圈后 pos 越界，读走别人的内存	S8
往 HISR 写 1 清标志	标志永不清，IRQ 风暴	S4 NVIC

契约二：IDLE 是"该搬积压了"，不是帧边界

IDLE 标志的含义只有一个：**线路静默了一个帧时间**。它不携带任何"这是一帧"的承诺。03-uart-dma 在源码注释里把这条写死在第一屏：

> IDLE 只提示总线空闲；应用协议必须自行定义帧边界。

所以 `g_frames` 这个观测量的注释也诚实地写着：它是 **IDLE 通知次数**，不是协议帧数。到了 05-framed-protocol，IDLE 的角色进一步降级为"该把积压搬走一趟了"的搬运通知；帧边界完全交给 COBS + `0x00`（契约四）。

错误标志的清除同样有口径：`ORE/FE/NE` 必须按"读 SR 后读 DR"的顺序才会应答清除；IDLE 的清除方式与之相同，但它**不是**协议边界，混在一起处理就会把"线路静默"误当成"收完一帧"。

事件	硬件语义	应用该做什么	错误用法
IDLE	一个帧时间内无新字节	把 DMA 缓冲里的新增区间搬进字节队列	当作帧边界直接切帧
ORE	收到字节时 RX 还没被读走	记数、读 DR 清除、考虑加大搬运频率	静默丢弃不计数
FE/NE	帧错误/噪声	记数；连续出现时先查波特率与时钟源	只清不查根因

处理 ORE 的位置值得多说一句：它意味着"数据在 USART 的单字节缓冲里挤丢了"。这时候**计数比恢复更重要**，因为丢的 byte 已经找不回来，能留下的是"链路在什么速率下开始丢"的证据。

契约三：环形缓冲的所有权与有序性

ISR 把新增字节从 DMA 缓冲搬进字节队列，这一段是 C5 的主场。免锁的前提是**所有权分离**，四条不变量：

1. 写指针只有生产者（ISR）能改；读指针只有消费者（主循环/任务）能改。
2. 双方各自改**自己那一个**指针，且对它的写对另一方可见即可，单生产者单消费者场景下不需要临界区（[D5] 的结论）。
3. 判空判满必须先声明容量口径：留一格的 A 派（物理 N 格、可用 N−1）或计数 B 派（可用 N，多一个计数变量）。两派混用是"时好时坏"类 bug 的经典来源。
4. `atomic_signal_fence` 只挡**编译器**重排；它不是内存屏障，不解决多核可见性（那是 DMB/DSB 与 Cache 一致性的领地，见 P4）。

一旦有第二个生产者（比如两个中断往同一个队列塞数据），免锁立即作废：所有权分离被破坏，必须回到关中断或 RTOS 队列（F4）。

```
/* 队列满时禁止静默覆盖：宁可丢、必须数。 */
if (q->count == Q_SLOTS) { q->dropped = sat_add(q->dropped, 1); return; }
```

上面这行来自 `code/common/logger/logger.c`。注意连**计数本身都用饱和加法**（`sat_add`）：溢出场景下计数器自己先溢出归零，是最讽刺的翻车方式。

契约四：帧协议要能从任意字节流再同步

IDLE 不可靠、字节可能丢、线路上可能从帧中间开始听。所以协议层的合同是：**无论从哪个字节开始听，都能在一个帧时间内恢复同步**。`code/common/reliability` 的线上格式：

```
wire = COBS(raw) + 0x00                       # 最长 74 字节
raw  = version:u8=1 | type:u8 | sequence:u16LE | length:u16LE
       | payload[0..64] | crc:u16LE
CRC-16/CCITT-FALSE（poly 0x1021，init 0xffff，不反射）
检查串 "123456789" → 0x29B1
```

三个设计决定各自对应一条纪律：

- **`0x00` 只当帧尾**：COBS 编码保证 raw 里不会出现 `0x00`，于是分隔符可以无歧义；丢失或损坏一段后，**下一个 `0x00` 就是重新同步点**，不用等"恰好对齐"。这是选 COBS 而不是转义或长度前缀的核心理由（[D13]）。
- **CRC-16/CCITT-FALSE 而不是随手挑的多项式**：嵌入式网络环境下这个多项式的检错表现有据可查（[D9]）；`0x29B1` 检查串让你能独立验证自己的 CRC 实现；书里 C8 手算过一笔 `seq=7 INFO` 请求从 raw 到 wire 的全过程。
- **sequence 回显配对**：响应必须回显请求的 seq。串线、重连、两台主机抢一个设备时，这是防止"旧响应被当成新请求的答案"的最后一道闸。

解析器只有三个状态：`EMPTY → COLLECT → DISCARD`。DMA 错误（TEIF）、UART 错误（ORE/FE/NE）、字节队列溢出，任何一样都把状态机推入 DISCARD：丢掉不确定的残段，在下一个 `0x00` 重新开始。**"可疑就整段丢弃"比"试着抢救"永远便宜。**

契约五：慢消费者必须可见地失败

链路的每一层都会遇到"下游来不及"。合同只有一条：**禁止静默覆盖，失败必须变成可读的计数**。传输层（`transport.c`）的实现是一份范本：

层	满时的行为	可观察量
TX 响应队列（4 帧）	新响应直接丢弃，**不覆盖正在发送的帧**	`tx_drop` 计数
TX 发送截止期	每帧 100ms 发不出去算失败，补发同步分隔符	失败计数
RX 字节队列（512 B）	溢出即失步：推解析器进 DISCARD	溢出计数
记录器槽位（J1 的 16 槽）	丢新样本，保住未取走的历史	`dropped`（饱和加法）
配置代次（J1 双槽）	代次失配的旧读数不进记录	`dropped`（按代次分账）

"丢新的还是丢旧的"是每个系统的显式决定：TX 队列选**保旧弃新**（正在发送的帧不能撕），记录器选**保历史弃新样**（故障时刻的数据最值钱）。没有对错，但**必须写下来**。这正是 S17 与 P13 健康监督的底座：监督的不是"任务活着"，而是"每一层的失败计数没有增长"。

六、跨到 ESP32-S3：同一 codec，换一副骨架

这条链路不是 F407 专属。`code/esp32/06-framed-protocol` 与 F407 版**共用同一个 `code/common/reliability` codec**（源文件级复用，不是复制粘贴），只换了搬运骨架：

环节	F407 寄存器版	ESP-IDF 版
收字节	IDLE 中断 + DMA 环形，NDTR 轮询位置	驱动事件回调 + ring buffer（P5）
帧边界	COBS + `0x00`（同一份 `protocol.c`）	同左
溢出语义	计数 + DISCARD（同一份 `transport.c`）	同左
交叉验证	`codec-tool.c` encode/decode ↔ Python CLI	同一份 CLI 对两块板

这正是 J1/J2 双板记录器"port 层"的含义：**协议与健康逻辑是公共资产，随硬件变的只有搬运层**。换芯片时重写的代码量被压缩到"怎么把字节搬进来/发出去"这一件事上。

七、最小验证矩阵

整链路的验收不是"串口助手里看起来收到了"，而是每一条输入都能用位置、计数与字节比较解释：

输入	预期	证据
1、127、128 字节后停顿	字节逐一相同，无越界	接收长度、逐字节比较
恰好半缓冲后停顿	不重复、不提前重启	NDTR 轨迹与标志序列
超过一圈的连续流	跨尾/头连续	`old_pos → pos` 轨迹
业务暂停直到队列满	溢出计数增长，系统不死锁	`dropped` 与恢复后的帧序号
人为制造 ORE / 线路噪声	错误可见、DISCARD 后可再同步	错误计数、下一个合法帧的 CRC
seq 乱序 / 旧响应	拒绝配对	seq 回显核对

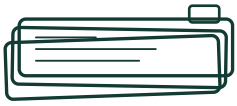
`code/common/reliability/tests.c` 把这些场景做成了可跑的 C 断言（全切分点、时间回绕、背压、饱和计数、守护式模糊），`sh probe.sh` 一条命令全绿。上板之前，协议层的正确性已经在宿主机上被钉死过一遍。

八、一页总图

五个契约背下来，你在任何一颗芯片上搭串口接收链路，要做的都只剩同一件事：**给每一段找对所有的主人，把失败变成数字。**

通往单片机之路APPENDIX

附录 B 配套工程索引（47 个可编译验证的工程）



47 个可编译工程

全书正文里每一处"配套工程"都指向 `code/` 下的一个独立目录。本附录是全量索引：47 个工程与章节一一对应，入口、类型与验证方式由 `code/projects.json` 登记，CI 按同一清单逐个构建验证（ARM 工程用 arm-none-eabi-gcc，ESP-IDF 工程用 `espressif/idf:v5.5.2` 容器、target `esp32s3`，宿主工程用本机 gcc）。构建器在出书时会核对本表与 `projects.json` 一致：漏登一个工程，书就构建不出来。

类型说明

类型	含义	怎么跑
`host-probe`	宿主机取证脚本，不需要开发板	进入目录 `sh probe.sh`
`arm-make`	F407/GD32 裸机 Makefile 工程	`make`（有板时 `make flash`）
`esp-idf`	ESP-IDF 工程（esp32s3）	`idf.py build` / `idf.py flash monitor`

C 篇与构建篇（宿主机取证）

工程	类型	入口	对应章节	内容
c-01-memory-model	host-probe	probe.sh	C1 内存模型	段归属实测：nm 地址/介质对账、const 经济学
c-02-pointer	host-probe	probe.sh	C2 指针	读写六函数宽度对照汇编、const 三组合
c-03-volatile	host-make	Makefile	C3 volatile	-O0/-O2 反汇编对照（volatile 有无各一版）
c-04-struct-abi	host-probe	probe.sh	C4 结构体与 ABI	offsetof/_Static_assert 偏移验证、位域禁区
c-05-func-pointer	host-probe	probe.sh	C5 函数指针	状态机+环形缓冲 560 断言、免锁口径
c-06-abi-stack	host-probe	probe.sh	C6 调用约定与栈帧	AAPCS 传参/栈帧实测、宿主 vs M4 帧账
c-07-ub-misra	host-probe	probe.sh	C7 UB 与 MISRA	优化等级翻脸实录、三道防线编译旗
toolchain-01-four-steps	host-probe	probe.sh	B1 四步构建	手动四步的全套中间产物
toolchain-02-elf	host-probe	probe.sh	B2 ELF 解剖	五刀解剖：段表/程序头/nm 对账
toolchain-03-linker	host-probe	probe.sh	B3 链接脚本	删 KEEP/AT> 的六刀取证

STM32F407 寄存器主线

工程	类型	入口	对应章节	内容
stm32-00-blink	arm-make	Makefile	S0 环境 · S3 · E01	第一个寄存器版点灯
stm32-01-arch	host-probe	probe.sh	S1 架构	位带别名断言 + -O2 反汇编互证
stm32-01-arch-board	arm-make	Makefile	S1 架构	位带的板上验证工程
stm32-01-rcc-clock	arm-make	Makefile	S2 时钟树	168MHz 提频六步
stm32-02-tim-pwm	arm-make	Makefile	S6 定时器 · E03	PWM 呼吸灯 + 输入捕获
stm32-03-uart-dma	arm-make	Makefile	S7 USART · S8 DMA · 附录 A	IDLE+DMA 环形接收寄存器版
stm32-04-nvic-exti	host-probe	probe.sh	S4 NVIC	IRQn→ISER/IP 换算 52 断言
stm32-04-nvic-exti-board	arm-make	Makefile	S4 NVIC	按键外部中断上板工程
stm32-05-systick	host-probe	probe.sh	S5 SysTick	LOAD 手算/回绕安全 42 断言
stm32-05-systick-board	arm-make	Makefile	S5 SysTick	精确延时上板工程
stm32-09-adc	host-probe	probe.sh	S9 ADC	12 轮 SAR 试探表全算
stm32-14-pwr	host-probe	probe.sh	S14 低功耗	三档睡眠换算与 CMSIS 对账
stm32-16-debug-hardfault	host-probe	probe.sh	S16 HardFault	CFSR/BFAR 解读与取证演练
stm32-05-framed-protocol	arm-make	Makefile	C8 帧协议 · 附录 A	COBS+CRC 帧协议寄存器版
stm32-06-watchdog-health	arm-make	Makefile	S17 看门狗	IWDG+WWDG+健康监督（8 变体矩阵）
stm32-07-sensor-logger	arm-make	Makefile	J1 记录器	F407 传感器记录器整机（故障注入）
stm32-04-i2c-eeprom	arm-make	Makefile	S11 I2C · E05	EEPROM 读写 + 总线恢复

RTOS 双精讲

工程	类型	入口	对应章节	内容
rtos-00-why-rtos	host-probe	probe.sh	F0 为什么 RTOS	裸机可行性账与超级循环翻车实录
rtos-01-freertos-lab	arm-make	Makefile	F1–F8 FreeRTOS	FreeRTOS 全场景实验（6 场景）
rtos-02-rt-thread-lab	arm-make	Makefile	R0–R7 RT-Thread	RT-Thread Nano 移植与验收
rtos-03-scheduler	host-probe	probe.sh	F3 调度器	双路径选优 O(1) 对照实测
rtos-05-sem-mutex	host-probe	probe.sh	F5 信号量 · E04	优先级反转剧本双版本复现
rtos-06-timers	host-probe	probe.sh	F6 软件定时器	守护任务 RAM 账与五幕推演

ESP32-S3 + ESP-IDF

工程	类型	入口	对应章节	内容
esp32-00-hello	esp-idf	CMakeLists.txt	P0 环境	第一个 IDF 工程与 monitor
esp32-01-gpio-matrix	esp-idf	CMakeLists.txt	P2 GPIO 矩阵	io_mux 对照实验
esp32-02-uart-events	esp-idf	CMakeLists.txt	P5 UART 驱动	事件驱动不定长接收
esp32-03-ledc-fade	esp-idf	CMakeLists.txt	P7 定时器与 LEDC	硬件 fade 呼吸灯
esp32-04-qmi8658	esp-idf	CMakeLists.txt	P6 驱动框架 · E07	六轴传感器读数
esp32-05-audio-play	esp-idf	CMakeLists.txt	P12 音频 · E08	I2S 放音"滴"与旋律
esp32-06-framed-protocol	esp-idf	CMakeLists.txt	C8 帧协议 · 附录 A	与 F407 同 codec 的 S3 版
esp32-07-watchdog-health	esp-idf	CMakeLists.txt	P13 看门狗	TWDT 健康监督（故障注入）
esp32-08-sensor-logger	esp-idf	CMakeLists.txt	J2 记录器	S3 版记录器整机（故障注入）
esp32-09-iram-discipline	esp-idf	CMakeLists.txt	P4 中断与双核	IRAM/Flash 落址取证

GD32 双系

工程	类型	入口	对应章节	内容
gd32-01-rcu-clock	arm-make	Makefile	G1 RCU 时钟树	200MHz 分频账与官方库对照

公共核心层（双板共用，宿主可测）

工程	类型	入口	对应章节	内容
common-behavior	host-probe	probe.sh	J1 · J2	行为层（滤波/状态）断言
common-reliability	host-probe	probe.sh	C8 · 附录 A	帧 codec+健康监测全断言
common-logger	host-probe	probe.sh	J1 · J2	记录槽位/代次/饱和计数

证据状态

工程分两档证据：**宿主验证**（host-probe 与公共层：断言全绿、CI 必跑）与**目标板验证**（arm-make/esp-idf：CI 验证"能干净构建"，板上现象按各章「待上板实测」口径另行回填（见 实验中心 的实测记录制度））。哪个工程处于哪一档，以各工程 README 的清单为准，本表不代填。

通往单片机之路**APPENDIX**

附录 C 章末自测参考答案

13 个章节带「🎯 自测」（正文里以蓝色题目框出现，共 60+ 题）。离线书里题目留在原章、答案集中在这里：**先选完再翻**，每题给答案字母、正确项原文与一段"为什么"，错在哪、对的凭什么是哪一章的哪张表。

第 04 章 · C3 volatile 自测

1. CMSIS 的 USART_TypeDef 寄存器字段本身带 volatile。轮询 `while (!(USART1->SR & RXNE))` 还需要再套 volatile 吗？

答案 B不需要——字段自带 volatile，这个读已经是真访存；再套只会多出不必要的访存

volatile 是"传染"的：外设结构体字段自带 __IO 限定，`USART1->SR` 每次都会真读硬件。SPL 的 I2C_GetFlagStatus 里那两个 __IO 局部变量就是"可加而不必加"的活样本。

2. 软件延时循环 `for (i=0;i<N;i++);` 在 -O2 下被优化成一条 ret，为什么合法？

答案 C没有副作用的空循环，标准允许编译器假定其终止（C11 起不终止按未定义处理）；非 volatile 的 i 又可被缓存折叠——两重权利叠加

as-if 规则（N1570 §5.1.2.3）：不可观察行为都可以删。i 非 volatile 则"循环内没人能改它"，比较结果可缓存；空循环本体按 C11 视为可假定终止。delay 想活着，计数变量必须 volatile。

3. 主循环 `flag ^= 1;`、中断里 `flag = 0;`，flag 已声明 volatile。这样安全吗？

答案 B不安全——volatile 只保证每步真访存，不保证读-改-写三步之间不被抢占

主循环读旧值后被中断抢占：中断写 0，主循环回来把基于旧值算出的结果写回，中断的更新丢了。对策：临界区包住读-改-写、改成"只置位/只清位"协议，或用外设侧原子寄存器（如 BSRR）。

4. `volatile uint32_t buf[256]` 当 DMA 缓冲区，代价和边界是什么？

答案 B每次普通访问都被强制真实访存，memcpy 无法块搬运优化；且 F407（无数据 cache）基本够用，ESP32-S3（有 cache）还必须做 cache 清理/无效化

volatile 管"别优化掉访存"，管不了 cache 一致性：Cortex-M4 没有数据 cache，volatile+屏障基本够；ESP32-S3 有数据 cache，DMA 与 CPU 共享的缓冲还要 cache clean/invalidate——光 volatile 不够。

5. `asm volatile("nop")` 里的 volatile 和变量上的 volatile 是一回事吗？

答案 B不是一回事：这是 inline asm 的属性，阻止的是汇编语句被删除/移动/合并，与内存可见性无关

两条独立防线：变量 volatile 管访存真实性；asm volatile 管这条汇编指令的存在与位置。实验对照过——只去掉 asm 参数里的 volatile（B 行），灯照闪；把 nop 也去掉（C 行），延时才消失：两道防线各管各的，谁也替代不了谁。

第 20 章 · S2 时钟树自测

1. 为什么切 168MHz 之前要先配 Flash 等待周期和电压档？

答案 B不配够等待周期会取指超时跑飞，VOS='0' 时 HCLK 上限仅 144MHz

Flash 取指在 3.3V 下 150~168MHz 要 5 个等待周期，不配够就取指超时、随机跑飞；电压档 VOS='0' 时 HCLK 上限是 144MHz，不配电压档连 168MHz 的资格都没有。顺序上必须先慢下来（电压档+等待周期）再提速。

2. HSE=8MHz 时，哪组 M/N/P/Q 的 VCO 输入更干净（抖动更低）？

答案 BM=4, N=168, P=2, Q=7（VCO 输入 2MHz）

RM0090 推荐 VCO 输入 2MHz 以限制 PLL 抖动。M=4/N=168 给出 VCO 输入 2MHz，比 M=8/N=336 的 1MHz 更干净。两组 SYSCLK 和 USB 结果相同（168MHz / 48MHz），但第二组参考输入更纯净。

3. HCLK=168MHz、APB1 分频 /4 时，TIM2 的时钟是多少？

答案 B84MHz

PCLK1 = 168/4 = 42MHz；APB 分频 ≠ 1 时定时器时钟 ×2 → TIM2 时钟 = 84MHz。只记 42MHz 会把所有定时器算慢一半——这是 STM32 时钟树最经典的隐藏规则。

4. 写完 RCC_CFGR.SW 后立刻读 SWS，可能看到什么？

答案 B可能是旧值——硬件切换需要时间

可能看到旧值——硬件切换需要时间。正确做法是轮询 SWS 直到它等于你写的 SW，并加超时；读不到回执就当切换失败处理。

5. 宏写 HSE_VALUE_HZ=25000000、板上实际焊 8MHz。probe.sh 第 2 步实测这一串的表现是什么？

答案 B一声不吭：pll_params_from_hse() 返回 0、回读算出 168000000，引脚上实际 53760000 Hz

实测写入 PLLCFGR 的是 M=25 N=336 → 0x07405419，pll_params_from_hse() 返回值 0（它只校验宏本身：25/25=1MHz 落在 1~2MHz 就放行），clock_tree_readback() 也算出 168000000；引脚上实际 53760000 Hz、USB 那一路 15360000 Hz，差 114240000 Hz。回读只证明「你写的位和你的宏自洽」，不证明晶振为真——要让程序开口，得自己加一步 MCO1 实测对账。

6. HPRE 是 4 位字段、PPRE 只有 3 位。照 HPRE 给 PPRE 开一张 16 格的分频表，症状是什么？

答案 C/2 /4 /8 /16 四档全部落在第一格，APB 分频被静默读成 /1，屏幕上出现 336MHz

3 位索引最大只到 7，表的后 8 格永远读不到，/2 /4 /8 /16 全部落在第一格 → APB 分频被静默读成 /1。症状不是崩溃而是离谱：APB1 被算成 168MHz，再套上「分频≠1 定时器 ×2」，屏幕上就出现 336MHz 这种这颗芯片根本给不出的频率。表长要按字段位宽开，{1,1,1,1,2,4,8,16} 八格才对（probe.sh 第 4b 节实测）。

第 21 章 · S3 GPIO 自测

1. 没开 GPIOF 时钟就往 MODER 写配置，随后读回 MODER，会看到什么？

答案 B全是 0——外设没通电，写入无效、读回也无效

时钟永远先于一切：RCC_AHB1ENR 不合闸，GPIOF 的寄存器就是没通电的橱窗——写进去不响，读回全是 0。这是新手第一大坑的指纹。

2. PF6 在 GPIOF_MODER 里占哪两位？要配成通用输出该写什么？

答案 Cbits[13:12]，写 01

每脚 2 位，PF6 占 bits[6×2+1 : 6×2] = [13:12]；MODER 取值 00 输入 / 01 通用输出 / 10 复用 / 11 模拟，输出写 01。正文的 PF6 实拍演算就是先清 0x3000 再置 0x1000。

3. 为什么改一个引脚电平要用 BSRR 而不是 ODR |= ？

答案 CODR |= 是读-改-写三条指令，中断插在中间会把别人的改动冲掉；BSRR 一条 str 原子完成且写 0 无影响

ODR |= 编译出 ldr/orr/str 三条指令，① 和 ③ 之间插进一个也改 ODR 的中断，返回后旧账整本写回、中断的改动被当场冲掉——症状是偶尔丢一次电平。BSRR 没有读、没有改，中断无从插足。

4. I2C 的 SDA 为什么必须开漏 + 上拉？

答案 B多设备共享一根线且谁都有权拉低；推挽会一推一拉对轰短路，开漏"只会拉、想高放手"实现线与

SDA 上挂主机和好几个从机，发 0、应答、clock stretching 都要能拉低。若用推挽，一个脚推高一个脚拉低就是两个输出级直通短路烧脚；开漏 + 上拉的"线与"制度让多设备共享一根线而不斗殴。

5. PA9 用作 USART1_TX，除了查 datasheet 复用表得知 AF7，寄存器上还要写什么？

答案 AMODER[19:18] = 10（复用），AFRH[7:4] = 0x7

复用要成对：MODER=10 把数据线从 ODR 切到外设，AFR 选号决定接哪个外设。PA9 在 AFRH（管 pin8~15）的第 1 个 nibble 即 [7:4] 写 0x7。只写 AF 不改 MODER，等于章盖了门没开。

第 22 章 · S4 中断自测

1. USART1_IRQn = 37，它的使能位在 NVIC 的哪里？

答案 BISER[1] 的 bit 5（地址 0xE000E104）

位图是数组：下标 = 37 / 32 = 1，位号 = 37 % 32 = 5，所以使能位在 ISER[1] bit5，地址 0xE000E100 + 4 = 0xE000E104。IP 字节是另一回事：0xE000E400 + 37 = 0xE000E425。probe 的七中断映射表两列全中。

2. NVIC_SetPriority(EXTI0_IRQn, 5) 实际写进 IP 字节的值是多少？

答案 **B**0x50

F407 的 __NVIC_PRIO_BITS = 4，IP 字节只有高 4 位有效，逻辑优先级要左移 4 位：5 << 4 = 0x50。低 4 位永远是 0，读回来也是 0——probe 编码表实测 5 → 0x50。

3. 要把优先级分组切成"2 位抢占 + 2 位子"（PRIGROUP=5），往 SCB->AIRCR 写的完整 32 位值是多少？

答案 **C**0x05FA0500

AIRCR 带锁：高 16 位必须写 VECTKEY = 0x5FA，否则整个写被硬件无视；组值放 PRIGROUP 位 [10:8]，5 << 8 = 0x500。合起来 0x05FA0500——probe 分组表第三列直接可查。

4. EXTI10 要接 PF10，SYSCFG 的哪几位写什么值？

答案 **B**EXTICR3[11:8] = 0x500

公式：寄存器 = 线号/4（10/4=2 → 第 3 个，即 EXTICR3，因为 EXTICR1 管 0~3），字段起始位 = (10%4)×4 = 8 → [11:8]；F 口编码 0x5，写入值 0x5 << 8 = 0x500。别忘了先开 SYSCFG 时钟（RCC_APB2ENR bit14）。

5. "把 HardFault 的优先级调低一点，方便调试嵌套"——这句话错在哪？

答案 **B**HardFault 的优先级硬件固定、根本不可编程：IRQn 枚举里 -13 被故意跳过，SHP 里也没有它的字节

stm32f407xx.h 的 IRQn_Type 里 NonMaskableInt=-14 之后直接跳到 MemoryManagement=-12——HardFault_IRQn(-13) 被故意省略。Reset/NMI/HardFault 优先级硬件固定（-3/-2/-1），能设的只有 -12~-1（走 SHP）和 IRQn≥0（走 IPR）。

6. `NVIC_SetPriority(EXTI0_IRQn, 16)` 写下去，IP 字节里实际存的是什么？

答案 **B**0x00 —— 变成最高优先级 0（在本字节里被截断）

官方头文件那行原话是 `NVIC->IP[IRQn] = (uint8_t)((priority << (8U - __NVIC_PRIO_BITS)) & 0xFFUL)`（core_cm4.h:1818）——按了下标取的就是**这一个字节**，越界既不换行也不报错，只在本字节里截断。probe 实测：16 → 0x00（读回来当作优先级 0，最高）、17 → 0x10（当作 1）、20 → 0x40（当作 4）、255 → 0xF0（当作 15）。这四条都在 52 条断言里。

第 24 章 · S6 定时器自测

1. 为什么改 CCR 不会改 PWM 的频率？

答案 **B**频率由 PSC 和 ARR 决定，CCR 只决定周期内哪一拍翻转电平

频率由 PSC 和 ARR 决定（计数周期）；CCR 只决定'在一个周期里的哪一拍翻转电平'。改 CCR 只是移动开关点，不改周期的总长度。

2. TIM3 挂在 APB1 上，APB1 分频 /4，PCLK1=42MHz，TIM3 的时钟是多少？

答案 **B**84MHz

84MHz。规则：APB 分频 ≠ 1 时定时器时钟 = PCLK × 2。只记 42MHz 会把所有定时器算慢一半。

3. 用 PSC=83、ARR=999 配 1kHz，为什么写成 PSC=84、ARR=1000 是错的？

答案 **B**硬件数的是 0 到 PSC 和 0 到 ARR，84 对应分频 85、1000 对应周期 1001，结果 ≈987Hz

硬件数的是'0 到 PSC'和'0 到 ARR'：84 对应分频 85，1000 对应周期 1001。更新频率 = 84MHz/85/1001 ≈ 987Hz，差一个 '%' 级。

4. 输入捕获里'相邻两个上升沿的 CCR 差值'在 CNT 回绕时怎么算才对？

答案 **B**先确认计数模数，用无符号差值自动处理回绕

先确认计数模数等于 2^32、间隔小于整圈且未丢边沿，本例 TIM2 才可用 uint32_t 模差；ARR=999 的 TIM3 不能套用。无符号整数减法天然处理回绕（如 0x0002 - 0xFFFE = 4）。

第 25 章 · S7 串口自测

1. 0x41 在串口线上的位序是什么样的？为什么是这个顺序？

答案 **B**空闲高 → 起始位低 → 1,0,0,0,0,0,1,0（LSB 先发）→ 停止位高，UART 协议规定低位先发

LSB 先发是 UART 的协议规定，双方约定俗成；逻辑分析仪按这个顺序解码才对（对照 0x55 与 0x41：同一波特率下波形完全不同，帧结构一模一样）。

2. PCLK2=16MHz，要配 115200 波特率，BRR 应取多少？

答案 **B**139

BRR = PCLK/波特率 = 16000000/115200 ≈ 138.9 → 取 139。实际波特率 ≈ 115108，误差 -0.08%，完全可用。BRR 是除法结果不是波特率本身。

3. RS-485 半双工发完要切方向，应该等哪个标志？

答案 **C**TC=1 才切——停止位跑完、移位寄存器也空了

TXE 只表示 DR 空了可塞下一个字节，移位寄存器还在一位一位往外发；TC 才是'真的发完'的唯一证据。等 TXE 就切会把最后一位拦腰砍断，对方帧校验直接挂。

4. IDLE 标志怎么清除？

答案 **B**读 SR 之后再读 DR

硬件规定清除序列是'读 SR 再读 DR'：光写 SR 没用，不把 DR 里的字节取走，IDLE 就一直挂着，中断会无限进——少了这一对读就是中断风暴。

5. 为什么不能拿 1MHz 时钟的总线波特率发生器配 115200？

答案 **B**分频后误差约 -3.549%，单项就吃掉 8N1 帧总预算的六成

波特率跟着 PCLK 走：低频时钟下 BRR 的取整误差被放大。1MHz 配 115200，BRR = round(1e6/115200) = 9，实际 111111.1，误差 -3.549%（probe 实测）——8N1 一帧的总容差只有 5.882%，这一项就占去六成。'BRR 写死'也是同一类坑——时钟树一改串口就乱码。

第 26 章 · S8 DMA 自测

1. USART1 的 RX 数据来了半天不来，配置看起来全对——最先该查什么？

答案 **B**DMA 控制器选错：USART1 在 APB2，请求归 DMA2，不在 DMA1

F4 的 DMA 分工按 APB 总线划：APB1 外设→DMA1，APB2 外设→DMA2。USART1 在 APB2 上，所以它的请求在 DMA2（Stream5/Stream2 的 Channel4）——直觉上'数字小的外设配数字小的 DMA'正是常见选错点。

2. 环形模式下 NDTR 数到 0 会发生什么？

答案 **C**NDTR 自动回装成初始值，从头继续写——保证'不停'，不保证'不丢'

环形模式保证传输永不停：NDTR 硬件自动重装。但如果 CPU 还没来得及读走旧数据，新数据就会把它盖上——处理期限是硬约束，'不丢'要靠半满/全满中断的水位管理。

3. 清 DMA 中断标志应该写哪个寄存器？

答案 **B**HIFCR 或 LIFCR 对应位写 1（按流所在半区选）

HISR/LISR 是只读状态寄存器，往它写 1 什么都不会清，中断会无限进。清标志必须写 HIFCR（流 4~7）或 LIFCR（流 0~3），对应位写 1 清除——这条已经在坑里验证过。

4. 在 EN=1（传输进行中）时修改 CR 的 CHSEL/CIRC 位，结果是什么？

答案 **C**改不进去——这些位只有 EN=0 时可写，必须先关流并等 EN 落 0

硬约束：CHSEL/CIRC/MINC 等流属性只在 EN=0 时可改。手写代码要 CR=0 后轮询等 EN 落 0；HAL_DMA_Init() 同样先禁用再轮询，只是多了 5ms 超时兜底不死锁（hal_dma.c L208-227）。

5. 把 DMA 接收缓冲区放到 CCM（核心耦合内存）里会发生什么？

答案 **C**DMA 写不进 CCM，数据静悄悄丢失

F407 的 CCM（0x10000000）DMA 访问不到（RM0090 总线矩阵），配置都对但数据就是不来——不报错、不触发异常，静默丢数据，最难查的一类坑。缓冲区要放在主 SRAM。

第 27 章 · S9 ADC 自测

1. 12 位 SAR ADC 为什么只需要 12 次比较？

答案 **B**每轮比较把候选区间对半砍：4096 → 2048 → … → 1，log2(4096) = 12，比较次数恒等于位数

probe 打印的 12 行试探表里，每轮 trial = 保留位 | 当前试探位，候选每轮对半砍——位数就是"猜数字"的提问次数：12 问定 4096 档。

2. VREF=3.30 V、Vin=2.00 V，理想 12 位 ADC 锁定的码值是多少？

答案 A2482（0x9B2）

LSB = 3.30/4096 = 805.7 μV；逐次逼近按"≤ 就保留"向下取整，floor(2.00/LSB) = 2482 = 0x9B2，回算 1.9997 V，量化误差 0.42 LSB——落在 [0,1) LSB 区间。probe 实测输出即此。

3. 100 kΩ 源阻抗、f_ADC=21 MHz，采样时间图省事一律配 3 拍，后果是什么？

答案 B残差约 2886 LSB 的系统性偏低——保持电容没充满就转，读数基本是垃圾

τ = (100k+2k)·4pF = 408 ns，沉到半 LSB 要 t > 9.01·τ ≈ 77.2 个周期，应选 84 拍档。只给 3 拍是"没充满的照片"——这不是噪声，是系统性偏低，12 轮比较再准也逼近的是虚焦值。

4. 三通道规则扫描 + DMA 采集，SQR1 的 L 字段和 DMA 的 NDTR 各写什么？

答案 BL = 2（通道数−1），NDTR 与通道数对齐（3 或 3 的整数倍）

序列总长 L 在 SQR1 bit23:20，宏 ADC_SQR1(3) = (3-1)<<20——L 是通道数减一；通道号按名次填 SQR3 前三个 5 位字段。NDTR 必须与通道数对齐，否则数据错位——S8 的 NDTR 语义在这里直接用。

5. ADC 输入引脚为什么必须配成模拟模式（MODER=11）而不是输入浮空？

答案 C模拟模式把施密特触发器和上下拉全部断开，引脚直通 ADC——漏电流最小、采样最真；配成数字输入读数是玄学

数字输入路径上有施密特触发器和可选上下拉，它们都会干扰模拟电压；MODER=11 把它们全部断开让引脚直通 ADC。忘配模拟模式是本章常见坑第一位——"读数是玄学"。

第 29 章 · S11 I2C 自测

1. 为什么 SDA 的数据跳变必须落在 SCL 低电平区？

答案 BSCL 高电平期间的 SDA 跳变被定义为控制信号（START/STOP），数据跳变会被误读

SCL 高电平期间的 SDA 跳变被定义为控制信号：下落=START、上升=STOP。数据若在高区变，会被全场误读为'开始/结束'，帧当场报废。所以协议规定数据只在 SCL 低区换脸，高区只负责被采样。

2. 清 ADDR 标志的法定动作是什么？

答案 B先读 SR1，再读 SR2

先读 SR1，再读 SR2。这是硬件约定的清除序列；不读或读反，ADDR 一直挂着，后续事件判断全卡死——I2C 寄存器版最经典的死循环现场。

3. TXE 和 BTF 有何区别？为什么最后一字节必须等 BTF？

答案 BTXE=数据寄存器空；BTF=移位寄存器也空。最后一字节只看 TXE 就 STOP 会截断正在移出的字节

TXE=数据寄存器 DR 空（可以写下一字节）；BTF=移位寄存器也空了（当前字节完整上了总线）。最后一字节只看 TXE 就 STOP，会把正在移出的字节拦腰砍断，从机收到半帧回 NACK。

4. 向 AT24C02 从地址 0 连写 20 字节会发生什么？

答案 B页大小 8 字节，超出部分页内回绕覆盖页首数据

页大小 8 字节，超出部分页内回绕覆盖页首附近数据；且 STOP 后有约 5ms 擦写期。正确做法：按 8 字节分页写，每页之间用 ACK polling 等擦写完成。

第 30 章 · S12 SPI 自测

1. 读 W25Q 的 JEDEC ID 时，为什么每收一个字节都要先往 DR 写 0xFF？

答案 B移位环两侧每拍同步交换——不写 DR，主机移位寄存器就没有"料"推过去顶出从机的数据；发哑数据就是花钱雇时钟

SPI 没有"只收"：片选拉低成环后，时钟每跳一下两边各换一位。想读就得"假写"——用主机的移位输出把从机字节一拍一拍顶回 MISO，8 拍之后 DR 里躺着要的字节。发即是收，收即是发。

2. 循环里只写 DR 从不读回，攒到第二个字节交换完成时会发生什么？

答案 BSR 的 OVR（bit6）置位，后续接收悄悄断线——这是"只写不读"设计错误的必然结局

每写一次 DR，硬件就同步收进一个字节；上一个没读走、新的又收满，OVR 置位。清法是读 SR 再读 DR，但根治是像 spi_xfer 那样收发成对——第 ④ 步"丢弃返回值也要读走"就是这份合同的另一半。

3. 逻辑分析仪上怎么一眼区分模式 0 和模式 3？

答案 A看 CS 拉低后、第一个数据变化前 SCK 停在哪：低=模式 0，高=模式 3

CPOL 定空闲电平（0=低、1=高），CPHA 定第几沿采样。模式 0/3 都是"一沿变、一沿采"的互补节奏，数据在稳定期被采样，所以 W25Q 这类器件两档常可互换——但主从必须同款，错配不报错、数据整体错位一位。

4. 单主机、片选走普通 GPIO 的工程，为什么还常配 SSM=1 + SSI=1？

答案 BNSS 信号内部强制为高、外部引脚彻底脱钩——即使 NSS 脚被干扰拉低也不会触发 MODF，SPE 不会被偷偷清掉

SSM=1 时 NSS 由 SSI 位内部提供，SSI=1 即"一直没人选我"；这是"软件片选 + 硬件免疫"的组合拳。反面教材：主模式忘配 SSOE 而 NSS 又被外部拉低 → MODF（SR bit5），SPE 被清、MSTR 降级——一个静默的"我咋不发了"。

5. 给 SPI1 配长帧 DMA 传输，流该选在哪一侧？

答案 BDMA2——F407 的 SPI1 请求在 DMA2，且收发两路要成对配置，只发不配收流 OVR 照样等你

SPI1 挂 APB2，它的 DMA 请求不在 DMA1（S8 章内表既有结论）——配错流连"不工作"都算幸运。DMA 版同样遵守"发即是收"：TXDMAEN/RXDMAEN 成对开，两路流成对配。

第 32 章 · S14 低功耗自测

1. Stop 与 Standby 在寄存器层的差别，说到底动了哪两位？

答案 BSCB->SCR.SLEEPDEEP 与 PWR_CR.PDDS

本机 probe.sh 实跑三档定档：Sleep → CR=0x4200 / SCR=0x10（CR 一个位没动）；Stop → CR=0x4201 / SCR=0x14；Standby → CR=0x4202 / SCR=0x14。三档的差别只有 SLEEPDEEP(bit2) 与 PDDS(bit1)：SLEEPDEEP=0 是 Sleep；SLEEPDEEP=1 且 PDDS=0 是 Stop；SLEEPDEEP=1 且 PDDS=1 是 Standby。LPDS/FPDS/VOS 只是 Stop 下的省电选项，不决定档位。

2. Stop 醒来把 SysTick 恢复成 168MHz 的装载值，但 USART 的 BRR 留着没重算（PCLK2 此刻还是 8MHz）。实际打出去的波特率是多少？

答案 C10973 baud，差 5%，对端收到的是帧错

usart_baud_from_brr(729, 8MHz) 实测 10973 baud。BRR 是分频数不是波特率本身：729 在 84MHz 下是 115226.0（+0.023%，能通），时钟掉到 8MHz 后同一个数变成 10973。8MHz 下要 115200 得重算成 BRR=69。凡是『数 = 时钟 / 目标』的寄存器（SysTick LOAD、BRR、TIM ARR/PSC）醒来都得重算。

3. BKPSRAM 写入前只开了 DBP、没开 PWR_CSR.BRE。probe.sh 实测 bkp_store() 的返回值和数组内容是？

答案 B返回 −2，数组仍是 0——看着写成功，其实是空气

实测：DBP=0 → −1；有 DBP 无 BRE → −2 且假寄存器数组全 0；超容量 → −3；三关全过 → 0。DBP（PWR_CR bit8）是写保护开关，BRE（PWR_CSR bit9）是备份调压器的供电开关，两个寄存器两个职责。缺供电时写总线不会报错，只会静默丢掉——比崩溃更难查。

4. 想让 RTC 闹钟（EXTI 线 17）唤醒 Stop，只把 PWR_CSR.EWUP 置 1 够不够？

答案 C不够，EWUP 只管 PA0/EXTI 线 0；线 17 得写 EXTI_IMR

探针实测：只配 EWUP 再查 17 号线 → −1，照样睡死；EXTI_IMR 写 1<<17 落进去的是 0x00020000，配好之后 17 号才返回 0。EWUP 是 PA0（EXTI 线 0）专用的唤醒使能，不在 EXTI_IMR 里；RTC 闹钟是线 17、RTC 唤醒是线 18，都走 EXTI_IMR。RTCEN 管的是 RTC 上不上电，不是唤醒路由。

5. IWDG 的 PR 有 8 个编码值，RLR=4095、LSI=32kHz 时最长超时是多少？按『每个码再翻一倍』推到 /512 会算成多少？

答案 A32760 ms；想当然会算成 65520 ms——高估整整两倍

八个码对应 /4 /8 /16 /32 /64 /128 /256 /256 —— PR=6 与 PR=7 同为 /256，最后一个码不新增分频。最长超时 256×4095×1000/32000 = 32760 ms。按 /512 想算出 65520 ms，正好高估两倍：你以为狗还能撑一分多钟，它半分钟就咬。LSI ±5% 时下界是 31122 ms，喂狗周期要按最坏的算。

6. 反汇编 enter_standby()，wfi 之后还有 7 条指令 / 18 个字节。这说明了什么？

答案 C这段代码永远不执行：Standby 的醒法是复位，18 字节静静躺在 Flash 里

同一次实跑里 Sleep 的 wfi 之后是 1 条 / 2 字节（直接返回），Stop 是 6 条 / 14 字节（会执行，所以重配时钟写在这里才有地方放），Standby 是 7 条 / 18 字节且永不执行——Standby 关掉调压器，醒来走的是上电复位序列。这也是为什么状态必须睡前写进备份域：函数里 wfi 之后那半截连它准备读复位标志的 ldr 都不会跑。

第 40 章 · F4 队列自测

1. 为什么说"信号量就是队列"？源码依据是什么？

答案 B QueueDefinition 用 union 复用存储：uxItemSize=0 时发送只维护计数、不 memcpy——信号量=无数据的队列

同一个 QueueDefinition：uxItemSize=0 时 prvCopyDataToQueue 不做数据拷贝只记计数；互斥锁再挂上持有者指针和优先级继承逻辑。xSemaphoreGive 实际展开成 xQueueGenericSend。

2. 队列已满，xQueueGenericSend 与 xQueueOverwrite 的行为差异是？

答案 B普通发送阻塞在 xTasksWaitingToSend；Overwrite 直接覆盖队首旧值、永远成功

普通发送"一个都不能丢"（命令/数据流），满了就挂起等空位；Overwrite 面向"只要最新值"（心跳/状态广播），要求队列长度 1，旧值被直接替换、发送永不失败。

3. 中断里调队列 API 为什么必须用 FromISR 版本？

答案 C中断上下文没有"任务"可挂起、不能中途切换上下文；是否需要让出用 pxHigherPriorityTaskWoken 带回，在中断末尾统一结算

阻塞/切换都必须发生在"任务"上下文。FromISR 把叫醒谁、要不要 YIELD 作为输出参数带出，配合 portYIELD_FROM_ISR 在中断出口统一结算——让出时机合法且只切一次。

4. 发送方拷贝完数据后可能当场 YIELD，判断依据是什么？

答案 B被叫醒的等待者优先级高于当前任务——xTaskRemoveFromEventList 返回 pdTRUE

若 xTasksWaitingToReceive 非空就叫醒等待者；等待名单按优先级排序，被叫醒者优先级更高时返回 pdTRUE，queueYIELD_IF_USING_PREEMPTION 当场让出——数据一到，最高优先级的消费者立刻接手。

5. prvCopyDataToQueue 写数据时队列缓冲区已到尾端，怎么处理？

答案 C分两段拷贝：先写到物理尾端，再从头写剩余字节（环形回绕）

队列存储是固定环形缓冲：跨越尾端的数据按 pcWriteTo 分两段 memcpy 回绕写入——这就是"环形存储+两张名单"里环形二字的来历，FreeRTOS-Kernel V11.1.0 queue.c 的 prvCopyDataToQueue。

第 43 章 · F7 内存管理自测

1. heap_4 释放一块内存时，什么情况下会合并？

答案 B只有当被释放的块与空闲链表中的块地址相邻时才合并

只有当被释放的块与空闲链表中的块**地址相邻**时才合并（前邻接尾或后邻接头）。不相邻就留洞——这就是碎片的物理来源。落点：heap_4.c 的 prvInsertBlockIntoFreeList。

2. '总空闲还有 4KB'为什么不能保证能分出 2KB？

答案 B空闲可能碎成多块不连续的小块，分配器要的是最大连续块

空闲可能碎成多块不连续的小块。分配器要的是**最大连续块**，不是总空闲。场景 5 的第 3、4 步就是证据。

3. `xPortGetMinimumEverFreeHeapSize` 在哪些 heap 版本上有？

答案 C只有 heap_4 和 heap_5

只有 heap_4 和 heap_5 有（V11.1.0 逐文件核对）。它回答'历史最低水位是多少'——峰值压力有多大，不是碎片率。

4. 动态创建的任务，它的栈存在哪里？

答案 B由内核 pvPortMalloc 从 heap 池子里分

动态创建（xTaskCreate）的栈由内核 pvPortMalloc 从 heap_4 的池子里分；静态创建（xTaskCreateStatic）的栈由调用方提供的静态数组给。MSP 主栈（中断用）在链接脚本里，与堆无关。

通往单片机之路**APPENDIX**

附录 D 术语速查：一句话讲透一个名词

看不懂的章节，八成不是难，是有一个名词从第一页就没被说清。这里把全书会遇到的名词按板块摊开：每个一句话 + 一句"最容易误会成什么"，然后告诉你去哪一章深读。

怎么用这一附录

- 读正文卡住 → 先在这里找名词，看一句话版本，再顺"深读"列跳回正文；
- 一句话版本刻意**不用**另一个生词解释它；如果还是看不懂，说明你缺的是前置章，按"深读"列往回退一级；
- 标 🔧 的词条与具体芯片强相关，本书只保证在 STM32F407 霸天虎 / ESP32-S3 实战派 这两块板上的说法成立。

语言与构建

术语	一句话	别误会成	深读
MMIO（内存映射外设）	外设寄存器被编进地址空间，读写一个地址就是读写硬件	不是"内存"，地址后面没有存储单元，是外设引脚/状态机	C1 内存模型
`volatile`	告诉编译器：这个对象每次使用都必须真实访存，不许缓存、不许删、不许无中生有	不保证原子性、不保证顺序，不是锁	C3 volatile
位域 / 掩码	一个 32 位寄存器里塞好几个字段，用移位+按位与各自取用	字段位置各芯片不同，别拿别家手册的偏移往这里套	C4 结构体与 ABI
ABI / 调用约定	函数怎么传参、谁清栈、返回时哪些寄存器必须原样还回来的"合同"	不是某个库的 API，是编译器与汇编都要共同遵守的接口规则	C6 调用约定与栈帧
栈帧	一次函数调用在栈上划出的私有区域：局部变量、保存的寄存器、返回地址	栈溢出不是"变量太多"这么简单，中断嵌套也会吃栈	C6 调用约定与栈帧
UB（未定义行为）	标准不管的行为，编译器有权假设它不发生——于是优化出你看不懂的代码	不是"结果随机"，是"任何结果都可能，包括看起来对的结果"	C7 UB 与 MISRA-C
预处理 / 编译 / 汇编 / 链接	四步构建：展开替换 → C 转汇编 → 汇编成目标文件 → 多目标文件按地址拼成可执行镜像	"编译报错"和"链接报错"是两类问题，后者多是符号/地址/段	B1 四步构建
ELF / HEX / BIN	ELF 带符号和段信息给调试器；BIN 是纯二进制按地址烧；HEX 是带地址文本封装	烧录用 BIN/HEX，调试用 ELF，别拿 BIN 去看函数名	B2 ELF 解剖
链接脚本	规定各段放到哪块地址、栈从哪里开始的地图	段属性字母有限集，抄来的老写法可能在新 binutils 上报错	B3 链接脚本
启动文件	从复位向量到 `main` 之间那段汇编/C：填向量表、配栈、搬 .data、清 .bss	不是"厂家加的胶水"，它决定你的全局变量凭什么有初值	B4 启动过程
map 文件	链接器输出的地址账本：每个符号落在哪个地址、哪个段占多大	体积优化第一步是读懂它，不是猜	B5 map 与体积

STM32 / Cortex-M

术语	一句话	别误会成	深读
GPIO	通用输入输出：一个引脚可被软件配成输入读电平、或输出驱动高低	"通用"不等于"随意"——复用功能、推挽/开漏、上下拉、速度都还要另外配	S3 GPIO
时钟树 🔧	晶振/内部 RC 经分频、倍频后分配到内核与各总线外设的那张"供水管网"	频率不是越高越好，Flash 等待周期与 APB 上限会先卡住你	S2 RCC 时钟树
HSE / HSI / PLL	外部晶振 / 内部 RC 振荡器 / 锁相环倍频输出	PLL 要轮询就绪位再切换时钟源，不是一写就生效	S2 RCC 时钟树
AHB / APB	高速总线与外设总线两级分频，外设挂在 APB1/APB2 上	挂在 APB 上的外设时钟 ≠ 系统时钟，计算分频时要顺着往下算	S2 RCC 时钟树
AF（复用功能）🔧	引脚的第二人生：把引脚的数据源从 ODR 换成某个外设	引脚和外设的对应关系以 datasheet 复用表为唯一真值	S3 GPIO
MODER / OTYPER / OSPEEDR / PUPDR	配置一个引脚的四个维度：模式 / 推挽或开漏 / 边沿速度 / 上下拉	只配 MODER 就以为配完了，电气特性还差三项	S3 GPIO
BSRR vs ODR	改一位不扰邻居用 BSRR（原子置/复位）；ODR 是读-改-写，中断插进来会翻车	ODR 能用不等于该用	S3 GPIO
中断向量表	每个中断号对应一个函数地址的表，硬件按编号取地址跳转	表里的顺序是硬件定的，改名不改顺序	S4 NVIC 与 EXTI
NVIC	管中断的开关、优先级与挂起状态的控制器	优先级分组规则不同，"抢占 0 位"和"子优先级 16 级"是两个世界	S4 NVIC 与 EXTI
EXTI	把外部引脚电平变化变成中断请求的机制	触发沿要配、线要清，漏一步就是"进一次中断再也出不来"	S4 NVIC 与 EXTI
SysTick	内核自带的倒计时器，常用作系统心跳	它是"内核的"，不属于某个外设总线	S5 SysTick
DMA	不占 CPU 的外设到内存搬运工	配置错方向/宽度时搬的字节数会骗过肉眼，要上仪器看	S8 DMA
循环模式 / 双缓冲	一次配置反复搬、或两块缓冲交替供数	环形缓冲区读写指针不写对，数据会撕裂	S8 DMA
波特率	每秒符号数，收端要靠过采样找回每个位的中心	时钟源算错，"9600"就不是你以为的 9600	S7 USART
波特率误差预算	总容差 = 0.5 ÷ (帧长 − 1.5)，量化误差与两端时钟容差共同瓜分这张饼	只看"实际波特率差 0.3%"就放心——帧尾采样点可能已经跨界	S7 USART
HardFault	出大错时硬件统一跳过去的异常出口	它不是 bug 本身，是 bug 的墓碑——要靠压栈现场反查元凶	S16 HardFault 与排错
IAP	应用自己在运行中写内部 Flash，常用于升级	写 Flash 期间取指会卡，要么同扇区不执行，要么搬进 RAM	S13 内部 Flash 与 IAP
Flash 存储器	掉电不丢的程序与数据存储器，按扇区擦除、按页/字写入	写之前必须先擦，且擦写次数有限；写 Flash 期间取指会卡住	S13 内部 Flash 与 IAP
RCC	复位与时钟控制：选时钟源、配 PLL、分频到各总线并逐个使能外设时钟	外设时钟默认是关的，不使能就读写寄存器会得到 0 或直接 HardFault	S2 RCC 时钟树
USART / UART	串行收发器：UART 指异步那一档，USART 还能对外出时钟	"串口"是口语，硬件上它是个 USART 外设；TTL 电平不等于 RS232	S7 USART
SPI	四线同步串行总线（SCK/MOSI/MISO/CS），靠 CPOL/CPHA 两个位配对模式	模式配错照样能通信，但数据整个偏一位；片选要软件自己管	S12 SPI
I2C	两线（SDA/SCL）多主多从总线，靠地址寻址 + 开漏线与仲裁	上拉电阻不能省；时钟延展和重复起始是抓包时最容易懵的两件事	S11 I2C

术语	一句话	别误会成	深读
ADC	把模拟电压量化成数字量的转换器，有规则组与注入组两套通道	采样时间不够读数会偏低；参考电压不稳，位数再多也没意义	S9 ADC
DAC	把数字量还原成模拟电压的输出通道	输出带载能力很弱，要推负载得外加运放	S10 DAC
TIM / PWM	定时器做计数、输入捕获、输出比较；PWM 是比较输出的一种用法	周期与占空比是两个不同寄存器（ARR / CCR），改一个不等于改另一个	S6 TIM 定时器
RTC	掉电后靠备用电源继续走的日历时钟	它挂在独立的低速时钟域上，读写有同步要求，不能当普通寄存器用	S14 电源与功耗
FPU	硬件浮点单元，让 float 运算由指令直接完成而非软件模拟	编译器不开浮点选项，FPU 就白装了；异常时浮点上下文要额外压栈	S16 HardFault 与排错
CMSIS	ARM 定的内核与外设软件接口标准，让代码与工具链、厂商解耦	它不是库，是一层头文件与命名约定	S1 架构概览
SPL	ST 标准外设库：把寄存器操作封装成结构体 + 函数	本书主线是寄存器版，SPL 只作对照阅读，别两套混着用	S15 SPL 解剖

RTOS

术语	一句话	别误会成	深读
任务	一段独立执行流 + 它专属的栈和控制块	不是"函数"，函数没有自己的栈和调度身份	F1 任务与 TCB
TCB	任务户口本：栈顶指针、优先级、状态链表项、任务名	栈顶指针必须是结构体第一个成员，这是移植层与内核的约定	F1 任务与 TCB
上下文切换	把 CPU 寄存器现状存进旧任务栈、从新任务栈恢复	切换有真实开销，"任务多一个无所谓"是错觉	F2 上下文切换
抢占式调度	更高优先级任务一就绪，当前任务立刻让位	不是时间片轮转；时间片只是同级之间的事	F3 调度器
就绪 / 阻塞 / 挂起	等 CPU / 等事件 / 不参与调度	阻塞态不耗 CPU，但占栈和链表，任务开太多先耗尽 RAM	F3 调度器
队列	带拷贝语义的线程安全收发管道	队列传的是副本，塞指针进去要自己保证生命周期	F4 队列
二值信号量 / 计数信号量	一个令牌 / 一串令牌的计数通知	信号量不继承优先级	F5 信号量与互斥量
互斥量	带所有权与优先级继承的锁	"谁加锁谁解锁"，用信号量当锁就会丢继承	F5 信号量与互斥量
优先级反转	中优先级把持有锁的低优先级饿死，高优先级跟着一起等	光靠"提高优先级"治不了，要靠继承或天花板	实验 E04 优先级反转复现
任务通知	内核把 TCB 本身当事件载体，免建对象、比队列快	只能一对一，广播场景还得事件组	F6 通知/事件/软件定时器
软件定时器	由调度器/定时任务服务线程统一回调的延时动作	回调跑在定时器任务的上下文里，别在里面阻塞或长耗时	F6 通知/事件/软件定时器
临界区	短暂关中断/挂起调度保护的代码段	临界区一长，实时性立刻崩，还会漏中断	F7 内存管理
heap_4	可合并空闲块的动态分配实现	能合并仍会产生碎片，长跑项目要看最坏值不只看总量	F7 内存管理
设备框架	把外设抽象成统一接口，应用只认句柄与操作集	抽象不等于慢，抽象掉的是重复代码	R4 设备框架
finsh / MSH	串口控制台里直接敲命令调函数	它要占 Flash 参与链接，命令表得让对象真正被编进来	R5 finsh 控制台
FreeRTOS	本书精讲的开源实时内核：任务、队列、信号量/互斥量、通知、软件定时器	它是内核不是操作系统——没有文件系统、没有网络栈，那些是组件	F0 为什么需要 RTOS
RT-Thread	国产开源 RTOS，除内核外自带设备框架、finsh 控制台与组件生态	"内核 + 组件"两层要分开看，别把一个组件的行为算到内核头上	R0 架构与设计
tick（系统节拍）	内核的心跳：每到一个 tick 检查延时到期、同级轮转与超时	tick 频率越高越灵敏但开销越大；vTaskDelay 是以 tick 为单位睡	F3 调度器
信号量	用于同步的令牌（二值）或资源计数（计数），不携带数据	信号量没有所有权与优先级继承，拿它当锁护数据等于裸奔	F5 信号量与互斥量

ESP32-S3 / ESP-IDF

术语	一句话	别误会成	深读
IDF	Espressif 官方开发框架：FreeRTOS + 协议栈 + 驱动全家桶	不是"只能 ESP32 用"，它是框架名，芯片是芯片	P0 环境搭建
menuconfig	IDF 的图形化配置工具，生成 sdkconfig 头文件	不是"改完就生效"，要重新 build 才编进固件	P3 IDF 解剖
GPIO 矩阵 🔧	把任意 GPIO 路由到任意外设的信号交换矩阵	不是"引脚随便选"，高速信号仍走 IO_MUX 专线；strapping/Flash/USB 引脚是禁区	P2 GPIO 与引脚矩阵
strapping 引脚	上电瞬间被采样以决定启动模式的引脚	把它们当普通输出口用，可能让芯片进不了下载模式	P2 GPIO 与引脚矩阵
IRAM	指令 RAM：Cache 关闭时仍能执行的代码所在	不是"所有代码都能放"，ISR 必须进 IRAM	P4 中断与双核
中断与双核	中断可绑到指定核，跨核要软中断	"关中断"只关当前核，另一核照跑	P4 中断与双核
LEDC	LED Controller：硬件 PWM 发生器，支持渐变；也是"产生 PWM 的专用外设"	通道数、位宽、高/低速模式各型号不同，S3 是 8 通道/14 位/仅低速	P7 定时器与 LEDC
I2S	Inter-IC Sound：串行音频总线	不是"只能传声音"，高速同步数据也能用	P12 音频链路
分区表	把 Flash 划成 app / nvs / otadata 等各用途区间	改分区会动到 OTA 双槽逻辑，改前先备份当前映射	P10 Flash/分区/OTA
NVS	Non-Volatile Storage：Flash 分区里的键值存储，配磨损均衡	不是文件系统，也不是"随便写 Flash"；存大块数据要用别的方案	P10 Flash/分区/OTA
OTA	把新固件写进另一个 app 槽，校验通过再切换启动；断电安全靠回滚/标记	不是"写完再跳"；默认配置不开回滚时坏固件照样坐庄	P10 Flash/分区/OTA
ESP-NOW	不需配网的设备间点对点无线帧	默认不加密，要保密得自己配密钥或在上层加一层	P9 蓝牙与 ESP-NOW
深度睡眠	关掉大部分逻辑与时钟，靠唤醒源重启	唤醒后 RAM 内容未必还在，"睡前的变量"不能假设自动复原	P11 低功耗
TWDT	Task Watchdog Timer：内核级任务看门狗	不是"软件喂狗"，是硬件监控每个 CPU 核心	P13 看门狗与健康监督
ULP	Ultra Low Power：独立低功耗协处理器	不是"主核休眠就行"，ULP 能在主核休眠时跑简单任务	P11 低功耗

GD32（与 STM32 对照）

术语	一句话	别误会成	深读
RCU	Reset and Clock Unit：GD32 的时钟控制单元，岗位等于 STM32 的 RCC	不是"换个名字而已"，寄存器位域和切换流程有差异，逐字段对账才算读完	G1 RCU 时钟树
CK_SYS / CK_AHB / CK_APBx	从源时钟逐级分频下来的实际频率账	"主频 200MHz"不等于每个外设都吃到 200MHz	G1 RCU 时钟树
IRC16M	GD32 的内部 16MHz RC 振荡器，对应 STM32 的 HSI 岗位	精度与温漂不如晶振，做时间基准要心里有数	G1 RCU 时钟树
RISC-V	开源指令集架构；V 篇的 GD32VF103 用它而非 ARM Cortex-M	指令集不同意味着工具链、启动与中断控制器全都要换一套	V1 Bumblebee 内核与 CLIC
Bumblebee 内核	芯来科技为 GD32VF103 实现的 RISC-V 内核	它不是标准 RISC-V 的全部，CLIC 等部分是厂商扩展；也不是开源核，是商业核但文档公开	V1 Bumblebee 内核与 CLIC
CLIC / ECLIC	Enhanced Core-Local Interrupt Controller：Bumblebee 的中断控制器	不是"RISC-V 版 NVIC"，入口、优先级模型、压栈都和 NVIC 不同，别照搬 NVIC 代码	V1 Bumblebee 内核与 CLIC
MTIME / MTIMECMP	RISC-V 标准的 64 位机器定时器与比较寄存器，裸机延时的时间基准	它不属于某个外设总线，是内核级时间源；读写要分高低 32 位防撕裂	V3 MTIME 与裸机延时

项目与系统

术语	一句话	别误会成	深读
四层架构	外设适配 / 公共核心 / RTOS 运行层 / 主机的分层设计	不是"层越多越好"，是为跨平台复用和可测试性	J1 F407 记录器
影子拷贝	保存配置时先 memcpy 到临时缓冲区再释放锁写慢速外设	不是"多此一举"，是避免慢存储阻塞实时采样	J1 F407 记录器
期限监控	要求任务在固定窗口内完成固定工作，否则视为卡死	不是"心跳包"，能检测到"活着但停滞"的隐性故障	J1 F407 记录器
配置代次	用有界模序号区分新旧配置，防止回退到旧版本	不是"时间戳"，模序号能处理回绕	J1 F407 记录器
双槽轮转	EEPROM 配置保存用两个槽交替写入，保证断电安全	不是"备份"，是原子切换机制	J1 F407 记录器

实物与仪器

术语	一句话	别误会成	深读
逻辑分析仪	只看高低电平，按通道抓时间序列，擅长解协议	它不看电压值，模拟量分不清 1.8V 还是 3.3V	实验 E02 抓 UART 帧
示波器	看真实电压波形、边沿与幅值	探头衰减比设错，读数会整倍偏掉	实验 E03 示波器看 PWM
共阳 / 共阴 LED	决定点亮是"输出低"还是"输出高"	接反了灯不亮不代表 GPIO 坏	实验 E01 点亮 RGB 红灯

> 词条不够用？本书的原则是每个名词第一次出现都配"类比 + 精确定义"双轨。缺哪个词，去仓库开 Issue 点名。

通往单片机之路**APPENDIX**

附录 E 来源、版本与许可

可溯源

本书的每一句"手册说""标准规定""论文证明过"都必须能翻到原文。这一附录交代三件事：证据纪律、正文引用标签与来源注册表的对应关系、以及许可与再分发边界。下方注册表由构建器从 `mcu-book/manifest.json` 自动渲染，与本附录文字同源。

证据纪律

1. **版本号是结论的一部分。** 寄存器位看手册修订版（Rev），行号看代码 tag/commit，API 看文档版本。三样对不上，先怀疑口径，再怀疑教程，最后才怀疑芯片。
2. **代码基准冻结点：** FreeRTOS-Kernel **V11.1.0**（ARM_CM4F 端口）、ESP-IDF **v5.5.2**、CMSIS device_f4 **v2.6.11**、SPL **V1.8.0**、GD32 固件库 V3.3.3 / @7ab0521。RT-Thread 未固定 tag（master 稀疏检出），这是本站已知的可复现性缺口：R 篇行号可能随上游漂移，已在注册表 `f8` 如实标注。
3. **"待上板实测"制度：** 凡是没有接仪器量过的现象，正文与实验页一律标注「待上板实测」，空白记录表不表示验证通过。本书不把这些数字当已验证数据引用。
4. **社区内容只作线索：** 会改变产品行为的结论必须回到目标版本的一手资料（手册/规范/冻结源码）与实测。

正文标签对照

章节正文沿用站点的参考文献标签体系（`[D2]` 这类写法），与本附录注册表 ID 的对应关系是**字母区相同、编号相同**：正文 `[A1]` = 注册表 `a1`（RM0090），`[D2]` = `d2`（优先级继承论文），`[F1]` = `f1`（FreeRTOS V11.1.0），以此类推；三个小写 `mcu-*` 条目是本仓库自有的工程/实验/动画资产。每个章节开头的「本章罗盘 · 依据锚点」列出的就是该章结论所依赖的注册表条目。

许可与再分发边界

- **书稿正文、版式与机制动画**：Apache-2.0（与仓库 LICENSE 一致），可自由转载与二次创作，需保留署名与许可声明。
- **被逐行引用的上游代码**（FreeRTOS、ESP-IDF、CMSIS、HAL、SPL、GD32 固件库、RT-Thread）：本书仅链接、引用行号并独立表述，不随书再分发其源码；各归其上游许可。
- **外部手册、规范、论文与书籍**：只作链接与事实引用（文档号/DOI/ISBN 与版本），权利归原权利人；注册表每张卡片都标了许可边界。
- **商标**：STM32 是 ST 的商标，ESP32 是 Espressif 的商标，GD32 是兆易创新的商标，FreeRTOS/RT-Thread 归各自权利人；本书作为独立教学材料使用这些名称仅出于指称目的。

来源注册表

以下卡片由 `manifest.json` 的 `sources` 渲染。快照与访问日期是本书构建时点（2026-10-23）的冻结口径。

`a1` RM0090 — STM32F405/415/407/417 Reference Manual

权威层级	ST 官方参考手册
冻结点	Rev 22，2026-05；使用时核对 ST 文档中心当前修订版
访问日期	2026-10-23
许可边界	ST 文档条款（本书仅链接并引用文档号与页码）
适用范围	S 篇全部寄存器口径：时钟树、GPIO、NVIC/EXTI、TIM、USART、DMA、ADC/DAC、I2C/SPI、Flash/IAP、PWR、看门狗。

打开来源 ↗

`a2` DS8626 — STM32F405xx/407xx Datasheet

权威层级	ST 官方数据手册
冻结点	Rev 12，2026-03
访问日期	2026-10-23
许可边界	ST 文档条款（仅链接与引用）
适用范围	引脚复用表、电气参数、ADC 采样时间、功耗典型值。

打开来源 ↗

a3 PM0214 — STM32 Cortex-M4 Programming Manual

权威层级 ST 官方编程手册
冻结点 未核版本；使用时核对当前修订版
访问日期 2026-10-23
许可边界 ST 文档条款（仅链接与引用）
适用范围 NVIC/SCB、异常模型、SysTick、故障寄存器与内核行为。

打开来源 ↗

a4 CMSIS_5 core_cm4.h（内核外设定义）

权威层级 Arm 官方头文件
冻结点 5.9.0，sha256 f5b63d52…（CI 逐字节校验）
访问日期 2026-10-23
许可边界 Apache-2.0（上游仓库条款）
适用范围 NVIC/SysTick/SCB 寄存器定义与内核内联函数口径。

打开来源 ↗

a5 Cortex-M4 Technical Reference Manual（DDI 0439）

权威层级 Arm 官方技术参考
冻结点 未核版本
访问日期 2026-10-23
许可边界 Arm 文档条款（仅链接与引用）
适用范围 内核框图、指令时序、SysTick 与调试组件。

打开来源 ↗

a6 ARMv7-M Architecture Reference Manual（DDI 0403E.e）

权威层级 Arm 官方架构手册
冻结点 E.e
访问日期 2026-10-23
许可边界 Arm 文档条款（仅链接与引用）
适用范围 异常入栈帧、EXC_RETURN、BASEPRI、懒堆叠的权威定义。

打开来源 ↗

a7 ESP32-S3 Technical Reference Manual

权威层级 Espressif 官方技术参考手册
冻结点 未核版本；使用时核对当前修订版
访问日期 2026-10-23
许可边界 Espressif 文档条款（仅链接与引用）
适用范围 P 篇 GPIO 矩阵、EXTI/双核、LEDC、I2S、RTC 与看门狗寄存器。

打开来源 ↗

a8 ESP32-S3 Series Datasheet

权威层级 Espressif 官方数据手册
冻结点 未核版本
访问日期 2026-10-23
许可边界 Espressif 文档条款（仅链接与引用）
适用范围 引脚、电气、模组 N16R8 的 PSRAM 占用（IO35/36/37）。

打开来源 ↗

a9 ESP-IDF Programming Guide v5.5.2

权威层级 Espressif 官方编程指南
冻结点 v5.5.2（与代码基准同版；URL 必须带 v5.5）
访问日期 2026-10-23
许可边界 Espressif 文档条款（仅链接与引用）
适用范围 P 篇全部 API：驱动框架、Wi-Fi、NVS/OTA、功耗管理、TWDT。

打开来源 ↗

a10 GD32F4xx User Manual

权威层级 兆易创新官方用户手册
冻结点 Rev 3.0 , 2024-01
访问日期 2026-10-23
许可边界 厂商文档条款（仅链接与引用）
适用范围 G 篇 RCU/GPIO/USART 等外设与 STM32F4 的逐字段对照基准。

打开来源 ↗

a11 GD32F407xx Datasheet

权威层级 兆易创新官方数据手册
冻结点 Rev 2.7
访问日期 2026-10-23
许可边界 厂商文档条款（仅链接与引用）
适用范围 G 篇引脚与 AF 复用表。

打开来源 ↗

a12 GD32F470 Product Brief

权威层级 兆易创新官方简介
冻结点 当前版
访问日期 2026-10-23
许可边界 厂商文档条款（仅链接与引用）
适用范围 G0 选型对比（F407 vs F470）。

打开来源 ↗

a13 GD32VF103 User Manual

权威层级 兆易创新官方用户手册
冻结点 Rev 1.0，2019-06
访问日期 2026-10-23
许可边界 厂商文档条款（仅链接与引用）
适用范围 V 篇外设（RCU/GPIO/USART）。

打开来源 ↗

a14 Bumblebee Core Datasheet

权威层级 芯来科技内核文档
冻结点 Rev 1.0，2019-08-28
访问日期 2026-10-23
许可边界 厂商文档条款（仅链接与引用）
适用范围 V 篇 ECLIC、MTIME/MTIMECMP 与 1/4 主频时间源。

打开来源 ↗

b1 UM10204 — I²C-bus specification and user manual

权威层级 NXP 官方规范
冻结点 Rev. 7，2021-10
访问日期 2026-10-23
许可边界 NXP 文档条款（仅链接与引用）
适用范围 S11/E05 起始/停止/ACK 时序、重复起始与总线恢复依据。

打开来源 ↗

b2 Motorola SPI Block Guide（S12SPIV3）

权威层级 Motorola/Freescale 工程文档（事实标准）
冻结点 V03.06
访问日期 2026-10-23
许可边界 文档条款（仅链接与引用）
适用范围 S12 CPOL/CPHA 四模式的出处。

打开来源 ↗

b3 I²S bus specification

权威层级 Philips 规范
冻结点 1986-02，1996-06-05 修订
访问日期 2026-10-23
许可边界 文档条款（仅链接与引用）
适用范围 P12/E08 左右声道与 WS 时序。

打开来源 ↗

b4 Universal Serial Bus Specification Revision 2.0

权威层级 USB-IF 规范
冻结点 Revision 2.0，2000-04-27
访问日期 2026-10-23
许可边界 USB-IF 文档条款（仅链接与引用）
适用范围 G4 USBHS 背景。

打开来源 ↗

b5 UTMI+ Low Pin Interface (ULPI) Specification

权威层级 行业标准规范
冻结点 Revision 1.1，2004-10-20
访问日期 2026-10-23
许可边界 规范条款（仅链接与引用）
适用范围 G4：GD32F4 与 F407 的 ULPI 12 根信号同脚同 AF。

打开来源 ↗

b6 CAN Specification · ISO 11898-1:2015

权威层级 Robert Bosch GmbH / ISO 标准
冻结点 Version 2.0，1991 · ISO 11898-1:2015
访问日期 2026-10-23
许可边界 标准条款（仅链接与引用）
适用范围 G4 bxCAN 位时序逐位对照。

打开来源 ↗

c1 ISO/IEC 9899:2011（C11）；免费草案 WG14 N1570

权威层级 ISO C 标准及工作组草案
冻结点 N1570，2011-04-12
访问日期 2026-10-23
许可边界 标准草案公开分发（引用条款号）
适用范围 C2/C3/C7：对象模型、volatile 语义、附录 J.2 未定义行为清单。

打开来源 ↗

c2 MISRA C:2012（含 Amendment）· MISRA C:2023

权威层级 MISRA 编码指南
冻结点 2013 · 2023
访问日期 2026-10-23
许可边界 MISRA 条款（仅引用规则编号）
适用范围 C7 规则取舍与「嵌入式十条」。

打开来源 ↗

c3 Executable and Linking Format (ELF) Specification

权威层级 TIS 行业规范
冻结点 Version 1.2，1995-05
访问日期 2026-10-23
许可边界 规范公开分发（仅链接与引用）
适用范围 B2/C1 文件视图与段视图两副目录。

打开来源 ↗

c4 Procedure Call Standard for the Arm Architecture（AAPCS32，IHI 0042）

权威层级 Arm 官方 ABI 标准
冻结点 abi-aa 仓库当前版
访问日期 2026-10-23
许可边界 Arm 文档条款（仅链接与引用）
适用范围 C4/C6 对齐与填充、r0–r3 传参、偶奇寄存器对。

打开来源 ↗

c5 AN298 — Cortex-M4(F) Lazy Stacking and Context Switching

权威层级 Arm 官方应用笔记
冻结点 DAI0298A，2012-03
访问日期 2026-10-23
许可边界 Arm 文档条款（仅链接与引用）
适用范围 F2 为什么大多数中断不付 FPU 帧的钱。

打开来源 ↗

c6 Cortex-M4 Devices Generic User Guide（DUI 0553）

权威层级 Arm 官方用户指南
冻结点 未核版本
访问日期 2026-10-23
许可边界 Arm 文档条款（仅链接与引用）
适用范围 S 篇 CMSIS 内核函数（__WFI、NVIC_*）口径。

打开来源 ↗

c7 GNU ld 手册（binutils）

权威层级 GNU 工具链官方手册
冻结点 当前版；区域属性只认 r/w/x/a/i/l
访问日期 2026-10-23
许可边界 GNU Free Documentation License
适用范围 B3 MEMORY/SECTIONS 语法；B5 map 三板斧。

打开来源 ↗

c8 RISC-V ISA：Vol. I Unprivileged · Vol. II Privileged

权威层级 RISC-V International 官方规范
冻结点 20191213 · 20211203
访问日期 2026-10-23
许可边界 规范公开分发（仅链接与引用）
适用范围 V0/V3 指令集、mtime/mtimecmp 语义、CSR 编号。

打开来源 ↗

c9 Core-Local Interrupt Controller (CLIC) 规范草案

权威层级 RISC-V 工作组草案
冻结点 草案仓库当前版
访问日期 2026-10-23
许可边界 仓库许可（仅链接与引用）
适用范围 V1：Bumblebee 的 ECLIC 是它的增强实现。

打开来源 ↗

d1 Liu & Layland. Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment. J. ACM 20(1): 46–61, 1973

权威层级 同行评审论文
冻结点 DOI 10.1145/321738.321743
访问日期 2026-10-23
许可边界 版权方保留权利；本书仅链接与引用结论
适用范围 F3 固定优先级抢占式调度与 69.3% 利用率上界。

打开来源 ↗

d2 Sha, Rajkumar & Lehoczky. Priority Inheritance Protocols. IEEE Trans. Computers 39(9): 1175–1185, 1990

权威层级 同行评审论文
冻结点 DOI 10.1109/12.57058
访问日期 2026-10-23
许可边界 版权方保留权利；仅链接与引用
适用范围 F5/R2 优先级继承、E04 复现依据。

打开来源 ↗

d3 Masmano et al. TLSF: a New Dynamic Memory Allocator for Real-Time Systems. ECRTS 2004, pp. 79–88

权威层级 同行评审论文
冻结点 DOI 10.1109/EMRTS.2004.1311009
访问日期 2026-10-23
许可边界 版权方保留权利；仅链接与引用
适用范围 R3 两级隔离适配 O(1) 分配的原始出处。

打开来源 ↗

d4 Wilson et al. Dynamic Storage Allocation: A Survey and Critical Review. IWMM 1995, LNCS 986, pp. 1–116

权威层级 同行评审综述
冻结点 DOI 10.1007/3-540-60368-9_19
访问日期 2026-10-23
许可边界 版权方保留权利；仅链接与引用
适用范围 F7/R3 碎片、首次适配/最佳适配/隔离适配的代价。

打开来源 ↗

d5 Lamport. Proving the Correctness of Multiprocess Programs. IEEE TSE SE-3(2): 125–143, 1977

权威层级 同行评审论文
冻结点 DOI 10.1109/TSE.1977.229904
访问日期 2026-10-23
许可边界 版权方保留权利；仅链接与引用
适用范围 C5 单生产者单消费者环形缓冲不加锁为什么是对的。

打开来源 ↗

d6 McCreary & Gray. All-MOS Charge Redistribution ADC Techniques — Part I. IEEE JSSC 10(6): 371–379, 1975

权威层级 同行评审论文
冻结点 DOI 10.1109/JSSC.1975.1050629
访问日期 2026-10-23
许可边界 版权方保留权利；仅链接与引用
适用范围 S9 片上 SAR ADC 电容阵列电荷再分配。

打开来源 ↗

d7 Eide & Regehr. Volatiles Are Miscompiled, and What to Do About It. EMSOFT 2008, pp. 255–264

权威层级 同行评审论文
冻结点 DOI 10.1145/1450058.1450093
访问日期 2026-10-23
许可边界 版权方保留权利；仅链接与引用
适用范围 C3 十三款编译器 volatile 错编实测与访问函数对策。

打开来源 ↗

d8 Wang et al. Towards Optimization-Safe Systems: Analyzing the Impact of Undefined Behavior. SOSP 2013, pp. 260–275

权威层级 同行评审论文
冻结点 DOI 10.1145/2517349.2522728
访问日期 2026-10-23
许可边界 版权方保留权利；仅链接与引用
适用范围 C7 优化器利用 UB 删掉安全检查的真实案例集。

打开来源 ↗

d9 Koopman & Chakravarty. CRC Polynomial Selection for Embedded Networks. DSN 2004, pp. 145–154

权威层级 同行评审论文
冻结点 DOI 10.1109/DSN.2004.1311885
访问日期 2026-10-23
许可边界 版权方保留权利；仅链接与引用
适用范围 C8 为什么选 CRC-16/CCITT；P10 NVS 的 CRC32。

打开来源 ↗

d10 Dijkstra. Solution of a Problem in Concurrent Programming Control. Commun. ACM 8(9): 569, 1965

权威层级 同行评审论文
冻结点 DOI 10.1145/365559.365617
访问日期 2026-10-23
许可边界 版权方保留权利；仅链接与引用
适用范围 F5/C5 互斥这件事的起点。

打开来源 ↗

d11 Reeves. What Really Happened on Mars? RISKS Digest 19.49, 1997-12

权威层级 JPL 工程师一手复盘（工程随笔）
冻结点 RISKS Digest 19.49
访问日期 2026-10-23
许可边界 转载条款（仅链接与引用）
适用范围 F5/E04 火星探路者优先级反转事故。

打开来源 ↗

d12 Ganssle. Great Watchdog Timers for Embedded Systems

权威层级 资深工程师工程随笔（非规范）
冻结点 Wayback Machine 存档（2024-12-14）
访问日期 2026-11-05
许可边界 作者站点条款（仅链接与引用）
适用范围 S17/P13 窗口看门狗为什么比定时喂狗可靠。

打开来源 ↗

d13 Cheshire & Baker. Consistent Overhead Byte Stuffing. IEEE/ACM Trans. Networking 7(2): 159–172, 1999

权威层级 同行评审论文
冻结点 DOI 10.1109/90.769765
访问日期 2026-10-23
许可边界 版权方保留权利；仅链接与引用
适用范围 C8 帧协议选 COBS 而不是转义或长度前缀的理由。

打开来源 ↗

e1 Joseph Yiu. The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors, 3rd ed. Newnes, 2013

权威层级　教科书（ISBN 978-0-12-408082-9）
冻结点　3rd ed., 2013
访问日期　2026-10-23
许可边界　版权书（仅引用观点与章节）
适用范围　S1/S4/S16、F2：异常模型、双栈指针、懒堆叠。

打开来源 ↗

e2 John R. Levine. Linkers and Loaders. Morgan Kaufmann, 2000

权威层级　教科书（ISBN 1-55860-496-0）
冻结点　2000 版；作者站点有章节草稿
访问日期　2026-10-23
许可边界　版权书（仅引用）
适用范围　B2–B4 符号决议与重定位。

打开来源 ↗

e3 俞甲子、石凡、潘爱民.《程序员的自我修养——链接、装载与库》. 电子工业出版社, 2009

权威层级　中文教科书（ISBN 978-7-121-08511-9）
冻结点　2009 版
访问日期　2026-10-23
许可边界　版权书（仅引用）
适用范围　B 篇最佳中文伴侣书。

打开来源 ↗

e4 Richard Barry. Mastering the FreeRTOS Real Time Kernel, v1.1.0

权威层级　FreeRTOS 官方教程书
冻结点　v1.1.0
访问日期　2026-10-23
许可边界　上游许可（仅链接与引用）
适用范围　F 篇 API 口径（本站讲内核源码，它讲用法）。

打开来源 ↗

e5 Knuth. The Art of Computer Programming, Vol. 1, 3rd ed., §2.5

权威层级　教科书（ISBN 978-0-201-89683-1）
冻结点　3rd ed., 1997
访问日期　2026-10-23
许可边界　版权书（仅引用）
适用范围　F7 边界标记法——heap_4 合并空闲块的祖师爷。

打开来源 ↗

e6 Kernighan & Ritchie. The C Programming Language, 2nd ed. Prentice Hall, 1988

权威层级 教科书（ISBN 0-13-110362-8）
冻结点 2nd ed., 1988
访问日期 2026-10-23
许可边界 版权书（仅引用）
适用范围 C 篇默认先修。

打开来源 ↗

e7 刘火良、杨森.《RT-Thread 内核实现与应用开发实战指南——基于 STM32》. 机械工业出版社, 2019

权威层级 中文教科书（ISBN 978-7-111-62677-4）
冻结点 2019 版
访问日期 2026-10-23
许可边界 版权书（仅引用）
适用范围 R 篇伴侣书（从零手写内核的路线）。

打开来源 ↗

f1 FreeRTOS/FreeRTOS-Kernel

权威层级 FreeRTOS 官方内核源码
冻结点 V11.1.0，portable/GCC/ARM_CM4F；ESP-IDF 内置 V10.5.1 SMP 改版不混读
访问日期 2026-10-23
许可边界 MIT（上游仓库）
适用范围 F 篇全部行号与结构体依据。

打开来源 ↗

f2 espressif/esp-idf

权威层级 Espressif 官方框架源码
冻结点 v5.5.2；CI 用 espressif/idf:v5.5.2 容器构建
访问日期 2026-10-23
许可边界 Apache-2.0（上游仓库）
适用范围 P 篇驱动框架与 soc_caps.h 结论。

打开来源 ↗

f3 STMicroelectronics/cmsis_device_f4（stm32f407xx.h）

权威层级 ST 官方 CMSIS 器件头文件
冻结点 v2.6.11，sha256 bd21220e…（CI 逐字节校验）
访问日期 2026-10-23
许可边界 Apache-2.0（上游仓库）
适用范围 寄存器基址与位域第二来源（GPIOF=0x40021400 等）。

打开来源 ↗

f4 STMicroelectronics/stm32f4xx_hal_driver

权威层级 ST 官方 HAL 源码
冻结点 @ 1f6451c
访问日期 2026-10-23
许可边界 BSD-3-Clause（上游仓库）
适用范围 S2/S6/S7/S8 的库源码上游对照。

打开来源 ↗

f5 STM32F4 StdPeriph Library（SPL）

权威层级 ST 官方标准外设库
冻结点 V1.8.0（官方未随 GitHub 分发，取自板卡资料盘）
访问日期 2026-10-23
许可边界 ST 固件条款（仅链接与讲解 API）
适用范围 S3/S11/S12/S15 的库封装对照。

打开来源 ↗

f6 GigaDevice GD32F4xx Firmware Library

权威层级 兆易创新官方固件库
冻结点 V3.3.3 @ 10d02f4
访问日期 2026-10-23
许可边界 上游许可（仅链接与引用）
适用范围 G 篇 RCU/FMC 逐字段对照。

打开来源 ↗

f7 GigaDevice GD32VF103 Firmware Library

权威层级 兆易创新官方固件库
冻结点 @ 7ab0521
访问日期 2026-10-23
许可边界 上游许可（仅链接与引用）
适用范围 V 篇固件库与启动文件。

打开来源 ↗

f8 RT-Thread/rt-thread

权威层级 RT-Thread 官方内核源码
冻结点 未固定 tag：CI 稀疏检出 master（src、include、libcpu/arm/cortex-m4）——已知可复现性缺口
访问日期 2026-10-23
许可边界 Apache-2.0（上游仓库）
适用范围 R 篇行号依据；行号可能随上游漂移。

打开来源 ↗

`mcu-code` 本仓库配套工程（code/，47 项）

权威层级 项目原始资料
冻结点 本书构建时的 code/ 与 code/projects.json（CI 按清单矩阵逐个验证）
访问日期 2026-10-23
许可边界 Apache-2.0
适用范围 章节配套的寄存器版/RTOS/IDF 示例工程：入口、构建方式与验证状态。

打开来源 ↗

`mcu-lab` 实物实验中心与装备清单

权威层级 项目原始资料
冻结点 本书构建时的 docs/guide/hardware.md 与 docs/lab/
访问日期 2026-10-23
许可边界 Apache-2.0
适用范围 基准板卡（霸天虎/实战派 S3）、仪器与接线实测口径。

打开来源 ↗

`mcu-anim` 机制动画库（docs/public/anim/，103 张 SMIL SVG）

权威层级 项目原始资料
冻结点 本书构建时的动画目录；版式与色板规范见 .trellis/spec/docs-site/animation.md
访问日期 2026-10-23
许可边界 Apache-2.0
适用范围 正文机制动画的图形化讲解；HTML 版随书内嵌播放。

打开来源 ↗

`ai-copilot-vscode-overview` Build with AI in VS Code（agents 总览；原 Copilot agent mode 文档现汇入此体系）

权威层级 Microsoft — Visual Studio Code 官方文档
冻结点 页面标注 10/21/2026；docs 已将 chat/agent mode 重排为 Agents 章节，旧 /docs/copilot/chat/chat-agent-mode 已跳转
访问日期 2026-10-27
许可边界 厂商官方文档条款（仅链接与引用断言）
适用范围 A0 工具地图（VS Code 内 agent 定位与 harness 概念）；A3（agent loop 官方定义、permissions/approvals 权限机制、MCP servers 接入）；A4（『AI can produce incorrect code…You remain responsible』）。

打开来源 ↗

`ai-copilot-vscode-agentmode-blog` Introducing GitHub Copilot agent mode (preview) — VS Code 官方博客

权威层级 Microsoft — Visual Studio Code 官方博客
冻结点 2025-02-24 发布；agent mode 首次以 preview 亮相的公告文
访问日期 2026-10-27
许可边界 厂商官方文档条款（仅链接与引用断言）
适用范围 A3 工具流：agent 模式『读错误输出—自动修正—循环直至任务完成』工作机制的一手时间锚点与机制描述。

打开来源 ↗

`ai-copilot-vscode-cpp` C/C++ for Visual Studio Code（C/C++ 语言支持页）

权威层级 Microsoft — Visual Studio Code 官方文档

冻结点 页面标注 10/21/2026

访问日期 2026-10-27

许可边界 厂商官方文档条款（仅链接与引用断言）

适用范围 A0 嵌入式 C 工作流可用性：Microsoft C/C++ 扩展（ms-vscode.cpptools）提供 IntelliSense/调试，同页确认 GitHub Copilot 在 C/C++ 工作流内提供编码建议——即嵌入式 C 开发在 VS Code 内可直接使用 Copilot 全家桶。

打开来源 ↗

`ai-copilot-github-cloud` GitHub Copilot on GitHub.com（含 Copilot cloud agent；旧 about-coding-agent 页 301 至此）

权威层级 GitHub 官方文档

冻结点 页面无显式日期（docs.github.com 持续更新），2026-10-27 抓取；文内已含 cloud agent / code review / automations 新口径

访问日期 2026-10-27

许可边界 厂商官方文档条款（仅链接与引用断言）

适用范围 A0 云端 agent 定位；A3（cloud agent 在 ephemeral 云开发环境编辑文件、跑测试与 linter）；A4（官方明示 59 分钟会话上限、单分支单 PR 限制，以及『Logs do not replace your own review and testing』『can produce incorrect or suboptimal code, including…security vulnerabilities』）。

打开来源 ↗

`ai-copilot-ccr-cpp` GitHub Copilot code review now supports C, C++, Kotlin, and Swift — GitHub Changelog

权威层级 GitHub 官方 changelog

冻结点 2025-04-23 发布

访问日期 2026-10-27

许可边界 厂商官方文档条款（仅链接与引用断言）

适用范围 A0/A4 嵌入式 C 可用性：Copilot code review 自 2025-04 起支持 C/C++（覆盖 PR 中逾 90% 文件类型）；配合 Using GitHub Copilot code review 文档（docs.github.com）可见评审入口覆盖 VS Code / Visual Studio / JetBrains（含 CLion）/ Xcode / gh CLI。

打开来源 ↗

`ai-claude-code-overview` Claude Code Docs — Overview

权威层级 Anthropic 官方文档（code.claude.com/docs）

冻结点 页面无显式日期，2026-10-27 抓取；内容已含 Desktop/Web 表面、Agent SDK、scheduled tasks 等 2026 年新口径

访问日期 2026-10-27

许可边界 厂商官方文档条款（仅链接与引用断言）

适用范围 A0 工具地图（Claude Code 定义与四种表面：terminal/IDE extension/desktop/web）；A1（CLAUDE.md 官方说法）；A3（MCP 官方说法、hooks、skills、sub-agents、CLI 可管道化 `claude -p`）。

打开来源 ↗

`ai-claude-code-memory` How Claude remembers your project（memory 机制页）

权威层级 Anthropic 官方文档

冻结点 页面无显式日期；正文引用 CLI v2.1.211–v2.1.285 行为差异，判定为 2026 年下半年现行版本

访问日期 2026-10-27

许可边界 厂商官方文档条款（仅链接与引用断言）

适用范围 A1/A3 记忆机制底稿：CLAUDE.md（user/project/local/managed 四作用域、目录层级加载、@import）与 auto memory（~/.claude/projects/<project>/memory/、MEMORY.md 索引首 200 行或 25KB 载入）；AGENTS.md 兼容；『Claude treats them as context, not enforced configuration』——A4 幻觉/纪律防御的制度化对照。

打开来源 ↗

`ai-claude-code-repo` anthropics/claude-code — 官方仓库 README

权威层级 Anthropic 官方 GitHub 仓库

冻结点 README 现行版，2026-10-27 抓取

访问日期 2026-10-27

许可边界 厂商官方文档条款（仅链接与引用断言）

适用范围 A0 agentic coding 一手定义：『an agentic coding tool that lives in your terminal, understands your codebase, edits files, runs commands』——比 overview 页营销语更贴『结对工程师』口径。

打开来源 ↗

`ai-cursor-docs` Cursor Documentation 首页（产品定位）

权威层级 Anysphere — Cursor 官方文档

冻结点 页面无显式日期；Models 表已列 Claude Opus 5.5 / GPT-5.6 / Gemini 3.8 等 2026 下半年模型，判定为现行版

访问日期 2026-10-27

许可边界 厂商官方文档条款（仅链接与引用断言）

适用范围 A0 工具地图：『Cursor is a coding agent for building ambitious software』——官方把整款产品定位为 coding agent 而非『带 AI 的编辑器』（定位语，部分属营销口径，引用时取能力句不取形容词）。

打开来源 ↗

`ai-cursor-agent` Cursor Docs — Agent（Agent 机制总览）

权威层级 Anysphere — Cursor 官方文档

冻结点 页面无显式日期，2026-10-27 抓取；含 /goal、side chats 等 2026 新特性

访问日期 2026-10-27

许可边界 厂商官方文档条款（仅链接与引用断言）

适用范围 A3 工具流：Agent 三构件（Instructions/Tools/Model）；工具清单（编辑文件、跑终端命令、浏览器、图片生成、提问澄清）；『There is no limit on the number of tool calls』；Checkpoints 快照回滚；运行中 steering/排队消息——与 VS Code/Claude Code 的工具流图互为对照。

打开来源 ↗

`ai-cursor-run-modes` Cursor Docs — Run Modes（Auto-review / Allowlist / Run Everything 与沙箱）

权威层级 Anysphere — Cursor 官方文档

冻结点 页内 Changelog 表：v3.6（2026-05-29）Auto-review 成推荐默认、v3.23（2026-10-01）新增 Read access Workspace 模式——日期线索以此表为准

访问日期 2026-10-27

许可边界 厂商官方文档条款（仅链接与引用断言）

适用范围 A3 权限与安全边界主来源：三档运行模式、后端 LLM 分类器（可读 ReadFile/Grep/Glob/ListDir）、『Auto-review is not a security boundary』、macOS Seatbelt / Linux Landlock+seccomp 沙箱、Cloud Agents 不走 Run Modes。

打开来源 ↗

`ai-mcp-spec` Model Context Protocol Specification（latest，协议版本 2026-07-28）

权威层级 Model Context Protocol 官方规范（现隶属 Linux Foundation AAIF）

冻结点 协议版本 2026-07-28（latest 别名解析至此）；规范用语遵循 BCP 14 / RFC 2119

访问日期 2026-10-27

许可边界 厂商官方文档条款（仅链接与引用断言）

适用范围 A3 MCP 是什么的一手规范定义：开放协议、JSON-RPC 2.0、Hosts/Clients/Servers 三方、Server 三大特性 Resources/Prompts/Tools、安全原则（用户明示同意、工具调用前须授权）——ai-mcp-toolflow.svg 的规范层底稿。

打开来源 ↗

`ai-mcp-architecture` MCP — Architecture overview

权威层级 Model Context Protocol 官方文档

冻结点 对应协议版本 2026-07-28，2026-10-27 抓取

访问日期 2026-10-27

许可边界 厂商官方文档条款（仅链接与引用断言）

适用范围 A3 架构图底稿：host 为每个 server 建一个 client；本地 STDIO server 通常服务单 client、远程 Streamable HTTP 服务多 client；两层划分（data layer / transport layer）；三大 primitives 定义与 */list、tools/call 方法；声明 LSP 式生态标准化愿景。

打开来源 ↗

`ai-mcp-servers` modelcontextprotocol/servers — 官方参考实现仓库 README

权威层级 MCP steering group 官方仓库

冻结点 README 现行版（仓库 91.6k stars，2026-10-27 抓取）；reference servers 仅 7 个，更多服务器走 MCP Registry

访问日期 2026-10-27

许可边界 厂商官方文档条款（仅链接与引用断言）

适用范围 A3『官方 reference servers 有哪些类别』的直接答案：在册 reference 为 Everything/Fetch/Filesystem/Git/Memory/Sequential Thinking/Time；GitHub、GitLab、Google Drive、Postgres、SQLite、Slack 等已归档至 servers-archived；官方明示其为教学示例『not as production-ready solutions』。

打开来源 ↗

`ai-mcp-anthropic-donation` Donating MCP to the Agentic AI Foundation — Anthropic 官方公告

权威层级	Anthropic 官方新闻
冻结点	2025-12 公告（Linux Foundation 2025-12-09 新闻稿口径一致）
访问日期	2026-10-27
许可边界	厂商官方文档条款（仅链接与引用断言）
适用范围	A0/A3 治理事实：MCP 由 Anthropic 捐赠给 Linux Foundation 下设 Agentic AI Foundation（与 Block goose、OpenAI AGENTS.md 同为创始项目；Google/Microsoft/AWS/Cloudflare/Bloomberg 支持）；官方 Registry 上线；2025-11-25 spec 引入异步操作、无状态化、server 身份与官方 extensions。

打开来源 ↗

通往单片机之路**APPENDIX**

附录 F 常见问题 FAQ

这些问题在评论区、私信里被问了几十遍，答案都一样。集中写在这里，省得每个人重新走一遍弯路。全篇按"起步 / 学习路线 / 内容与协作 / 技术细节"四区组织，技术细节区的每条都标了深读章节。

起步

我该买哪块板子？

只有一块预算，**野火 STM32F407 霸天虎**：S 篇与 RTOS 移植全部按它写，资料最多、引脚全引出、调试接口标准。做语音/AIoT 才选 **立创·实战派 ESP32-S3**。两块都想买的话，先 STM32 后 ESP32：裸机地基打在前者更牢。清单与别花冤枉钱的配件见在线版装备清单。

没有数电/模电基础，能学吗？

能，但别硬撑。你需要的是"看得懂原理图、知道上下拉和上电顺序"这一档，不是会设计滤波器。缺的这部分在本书靠两处补：手册地图 教你查资料，实验中心 让你在板子上看到现象。

SPL、HAL、寄存器，到底学哪个？

顺序是**寄存器 → 库**。先在寄存器层把一个外设的"配置寄存器 + 数据寄存器 + 状态寄存器"三件套走通，再回头看库把它封成了什么，本书每章的"库源码解析"就是这个对照。反过来先学 HAL，你会卡在"CubeMX 生成的代码为什么这样写"，而那是没有答案的问题。详见 S15 SPL 库解剖。

能用 CubeMX / 图形化配置工具吗？

工作中能用，学习阶段先手写一遍。理由很实际：工具把引脚复用、时钟分频这些"会出错的决策"藏起来了，你只有先自己错过一次，之后才看得懂它替你做了什么选择。

Windows 够用吗？

够。本书全部工程在 Windows 上构建验证（工具链 + VSCode/命令行）。ESP-IDF 走官方安装器，注意路径不要带中文与空格。步骤在 S0 环境搭建 与 P0 环境搭建。

没有示波器/逻辑分析仪能做实验吗？

E01/E02/E05 用串口打印或 LED 就能验；E03/E06/E07/E08 需要仪器（示波器、毫安级电流测量）。不必一次买齐：先做不需要仪器的实验，等需要看波形时再补。

板子到手第一件事做什么？

点灯，而且要**知道自己在往哪个地址写什么**。（实验 E01）

学习路线

学到什么程度算入门？

三条检验：① 换一个没玩过的芯片，只看手册能在两小时内把 USART 收发跑起来；② 程序跑飞时，你有排查顺序而不只是重烧一遍；③ 能向别人讲清"中断里为什么不能调用阻塞函数"。这三条分别对应 S 篇、B4/S16、F 篇。

FreeRTOS 和 RT-Thread 先学哪个？

先 FreeRTOS：内核小、能一行行读完，读完你就懂"RTOS 到底做了什么"。之后 RT-Thread 的重点变成设备框架与组件生态，那是另一套问题。对照结论在双实现对照与选型决策树。

为什么 C 篇要放在最前面？

因为指针、`volatile`、栈帧这三样不补上，后面每一章的"库源码解析"都会变成玄学。见 C0 为什么嵌入式 C 是另一种 C。

章节显示"建设中"，还值得读吗？

骨架页的**结构、精髓、先修关系、常见坑**已经写完，可以当提纲用；正文、动画与工程还没补齐。在线首页地图实心=成稿、虚线=建设中，数字由脚本全站统计，不会骗你。（本书收录章节均为成稿。）

内容与协作

数字和事实怎么保证不出错？

寄存器地址、引脚号必须有出处，写不出就标"以某手册某节为准"，禁止编数值；核对过的数值要有第二来源（CMSIS 头文件/官方手册）。这条纪律写在贡献指南，未回填的坑也都在页面上明说。

发现错误怎么办？

开 Issue 或直接提 PR。勘误是本项目的正式贡献类型，不是打扰。

可以转载/商用吗？

内容是 Apache-2.0，引用的上游源码遵守各自许可证且已注明出处；实物照片与图表请勿单独摘走。

为什么这个项目参考"通往AGI之路"？

学它的**开源、体系化、面向初学者**，但不学它把深度让渡给广度：这里每一个知识点都要追到寄存器、追到源码、追到示波器上的波形。

技术细节

为什么寄存器地址都是 0x4002xxxx 这种？

ARM Cortex-M 把外设编进内存地址空间（MMIO），0x4002xxxx 是 AHB/APB 总线外设的区域。读一个地址就是读一个寄存器，不是访问内存。详见 C1 内存模型。

volatile 到底保什么？

只保"每次读写都真实访存"，不保原子性、不保顺序。多个 volatile 变量之间的操作顺序，编译器仍可能重排。需要顺序用 `__DSB()` / `__DMB()`，需要原子用中断保护或 RTOS 互斥量。详见 C3 volatile。

为什么中断里不能调用阻塞函数？

阻塞函数（如 vTaskDelay、xQueueReceive 带超时）会让出 CPU 等事件。中断上下文没有任务控制块（TCB），调度器不知道"中断阻塞了该切到谁"，结果要么是死锁、要么是内存损坏。中断里只做"记账"（设标志、发通知），办事交给任务。详见 F0 为什么需要 RTOS。

FreeRTOS 的队列和信号量有什么区别？

队列传数据副本（你塞进去 4 字节，接收端拿到的是拷贝），信号量传事件通知（二值信号量=1 个令牌，计数信号量=N 个令牌）。队列适合"生产者-消费者"数据流，信号量适合"事件发生"同步。详见 F4 队列 和 F5 信号量与互斥量。

为什么看门狗不能无限加长？

看门狗超时时间应该覆盖"最坏情况下的任务执行时间"，而不是"我希望它多久复位一次"。无限加长等于放弃了故障检测能力。正确做法是分析任务期限预算，让看门狗超时略大于最坏执行时间。详见 S17 看门狗 和 P13 健康监督。

I2C 上拉电阻怎么选？

标准模式（100kHz）通常 4.7kΩ，快速模式（400kHz）用 2.2kΩ。电阻太小功耗大、电平低；太大上升沿慢、通信失败。具体值要看总线电容和器件手册。详见 S11 I2C。

SPI 模式 0/1/2/3 怎么记？

CPOL=0 表示空闲时 SCK 低电平，CPOL=1 表示空闲时高电平；CPHA=0 表示第一边沿采样，CPHA=1 表示第二边沿采样。模式 0（CPOL=0, CPHA=0）最常用，但传感器可能用模式 3。**以传感器手册为准，不要假设**。详见 S12 SPI。

为什么 DMA 配置错了"看起来正常"？

DMA 搬的是字节数，不是"语义"。如果你配错了方向（外设到内存 vs 内存到外设）或数据宽度（8 位 vs 16 位），DMA 仍会默默搬完，只是搬错了内容。肉眼看不出区别，必须用示波器或逻辑分析仪验证。详见 S8 DMA。

RTOS 任务栈多大合适？

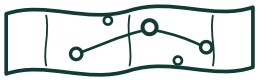
太小会溢出（HardFault 或数据损坏），太大会浪费 RAM。经验值：简单任务 256 字节，带 printf 的任务 512 字节，复杂任务 1024 字节。用 FreeRTOS 的 `uxTaskGetStackHighWaterMark()` 查剩余量，留 20% 余量。详见 F1 任务与 TCB。

为什么 ESP32 的 ISR 要放 IRAM？

ESP32 的代码默认从 Flash 执行，靠 Cache 加速。但某些操作（如 Flash 写入）会关闭 Cache，此时如果 ISR 在 Flash 里就会取指失败。把 ISR 放进 IRAM（指令 RAM）保证 Cache 关闭时仍能执行。详见 P4 中断与双核。

通往单片机之路APPENDIX

附录 G 开源项目与社区地图



30 个仓库 · 6 个知识点

本书的每条结论都要求"能指到文件行"。开源仓库是最便宜的一手证据来源：不用许可、不用保密协议，`git clone` 之后就能看见厂商和社区各自怎么实现同一件事。本附录给三样东西——一张按篇章组织的仓库地图、六个只有读源码才拿得到的知识点、以及社区提问的最小纪律。

星标数与活跃度取 2026-10-26 的 GitHub API 返回值；许可取自各仓库根目录的 `LICENSE`；知识点的文件路径按当日默认分支。**星标多是"人多"，不是"该读"**——该不该读只看一条：仓库里有没有你要找的那个 `.h` 或那个函数。

仓库地图

篇	仓库	星标	许可	先读什么	对
C	ARM-software/CMSIS_6	483	Apache-2.0（见仓库 `LICENSE`）	`CMSIS/Core/Include/core_cm4.h` 里内联函数与屏障宏	C3 vo S4
C	STMicroelectronics/cmsis-device-f4	161	Apache-2.0（Cube 许可表逐组件标注）	`Include/stm32f407xx.h`：寄存器结构体与位域定义	S3 S1
构建	platformio/platformio-core	9564	见仓库 `LICENSE`	`platformio/builder/`：谁决定 `-mcpu` 与链接脚本	B7 统
构建	riscv-collab/riscv-gnu-toolchain	4651	GPL 家族（见仓库 `LICENSE`）	`Makefile.in` 的 `--with-arch` / `--with-abi` 缺省值	V0 V
STM32	STMicroelectronics/STM32CubeF4	1361	HAL/BSP/Projects = BSD-3-Clause；CMSIS/mbedTLS = Apache-2.0；USB/STemWin/TouchGFX = SLA0044	`Drivers/STM32F4xx_HAL_Driver/Src/` 里任一外设的 `Init`	S1 解
STM32	libopencm3/libopencm3	3671	见仓库 `COPYING`	`include/libopencm3/stm32/f4/memorymap.h`：把手册翻成宏	S3
STM32	stm32duino/Arduino_Core_STM32	3375	见仓库 `LICENSE`	`system/Drivers/` 与 `cores/arduino/` 的边界	C4 与
RTOS	RT-Thread/rt-thread	12309	Apache-2.0	`src/object.c` 对象容器、`src/components.c` 初始化档位	R1 调
RTOS	FreeRTOS/FreeRTOS-Kernel	4609	MIT（官方许可页）	`tasks.c` 就绪表与切换、`portable/GCC/ARM_CM4F/`	F3
RTOS	zephyrproject-rtos/zephyr	16906	Apache-2.0（`LICENSE` 正文）	`include/zephyr/device.h`：设备树的运行期投影	F0 需 RT
ESP32	espressif/esp-idf	19347	Apache-2.0（`esp_err.h` 的 SPDX 头可证）	`components/esp_common/include/esp_err.h`	P4 双 P1 与
ESP32	espressif/arduino-esp32	17629	见仓库 `LICENSE`	`cores/esp32/esp32-hal-*.c`：Arduino API 怎么落到 IDF	P5 驱
GD32	CommunityGD32Cores/gd32-pio-spl-package	10	见仓库 `LICENSE`	GD32 SPL 的重打包规则	G0
调试	openocd-org/openocd	2365	GPL-2.0（见 `COPYING`）	`src/flash/nor/stm32f2x.c`：Flash 算法怎么写	B6 与
调试	pyocd/pyOCD	1473	Apache-2.0（`LICENSE` 正文）	`pyocd/core/` 的 target 抽象：调试协议怎么变成 API	B6 与
调试	blackmagic-debug/blackmagic	3719	见仓库 `COPYING`	**已归档**（`archived: true`），仅作阅读	S1 Ha
运行时	micropython/micropython	22154	见仓库 `LICENSE`	`ports/*/mpconfigport.h`：移植层要暴露什么	C6 定
运行时	rust-embedded/cortex-m	1010	MIT / Apache-2.0 双许可	`src/peripheral/` 与 `asm.rs`：裸机抽象的另一条路	C1 型
控制器	grbl/grbl	6309	见仓库 `COPYING`	`grbl/stepper.c`：中断里只做最少事	S6

篇	仓库	星标	许可	先读什么	对
RTOS	apache/nuttx	4087	Apache-2.0（`LICENSE` 正文）	`sched/` 任务模型、`drivers/` 分层注册	F3 对
RTOS	RIOT-OS/RIOT	5827	**LGPL-2.1**（`LICENSE` 正文）	`core/sched.c` 极简调度、`sys/` 抽象层	C6 定
存储	littlefs-project/littlefs	6980	BSD-3-Clause（`LICENSE.md`）	`lfs.c` 元数据对与 CTZ 跳表、`lfs_alloc` 磨损均衡	S1 Fla · P NV
存储	pellepl/spiffs	1632	MIT（`LICENSE`）	`spiffs_nucleus.c` GC 触发与页状态机	S1 种
网络	lwip-tcpip/lwip（GitHub 镜像，上游在 savannah）	1765	BSD-3-Clause（`COPYING`，SICS 版）	`src/core/pbuf.c` 零拷贝链、`src/core/tcp_in.c` 状态机	P8
网络	apache/mynewt-nimble	895	Apache-2.0（`LICENSE` 正文）	`nimble/host/src/ble_gap.c` 连接状态机	P9
网络	hathach/tinyusb	7238	MIT（`LICENSE`）	`src/device/usbd.c` 端点与描述符装配、`src/class/cdc/`	G4 异
图形	lvgl/lvgl	25010	MIT（`LICENCE.txt`）	`src/core/lv_obj.c` 对象树、`src/draw/sw/` 脏矩形	P1 数 制
测试	ThrowTheSwitch/Unity	5432	MIT（`LICENSE.txt`）	`src/unity.c` 断言宏与失败信息	C5 书 pr 的 言 照 未 它）
测试	ThrowTheSwitch/CMock	846	MIT（`LICENSE.txt`）	`src/cmock.c` mock 生成与调用序断言	C8
SDK	raspberrypi/pico-sdk	5004	BSD-3-Clause（`LICENSE.TXT`）	`src/rp2_common/`：寄存器→高层 API 的分层	B0 库

六个知识点：只有读源码才拿得到

读一个陌生仓库，先问三个问题：**谁在什么时候调用谁**、**数据在内存里长什么样**、**我猜错时会看到什么现象**。下面六条都是这三问的答案，每条都给出机制、踩坑现场和一行可验证的办法。

1. RT-Thread 的初始化是"按书脊编号上架"，不是"按你搬来的顺序"

原理：链接器是图书管理员，它只认书脊上的编号做排序，不认你把书搬进馆的先后。

机制：`INIT_EXPORT(fn, "level")` 把函数指针塞进一个以字符串命名的链接段（`.rti_fn.<level>`）。链接时同名前缀的段被拼成一段连续数组，**按段名做字典序排序**；启动时 `rt_components_init()` 从段首走到段尾，挨个调用。于是 `"0" < "0.end" < "0.post" < "1" < "1.end" …`：`src/components.c` 里 `INIT_EXPORT(rti_start, "0")`、`INIT_EXPORT(rti_board_start, "0.end")` 就是这条传送带的两个端点。

踩坑现场：把 `INIT_BOARD_EXPORT / INIT_PREV_EXPORT / INIT_DEVICE_EXPORT / …` 当成"六道闸门、编号 1～6"，于是断定"设备一定在板级之后初始化"。实际上档位是一串可插值的字符串。`bsp/tm4c129x/applications/board.c` 的注释写得很白：为了让 `rt_hw_cpu_init` 跑在所有 `INIT_BOARD_EXPORT` 之前，作者用的是 `INIT_EXPORT(rt_hw_cpu_init, "0.post")`。不改宏，只改字符串，就插了队。

自己验：`nm 固件 | grep rti_fn` 看段名排序；或在两个相邻档位之间插一句打印，看它落在谁前面。这一条也解释了 R1 里"设备注册早于调度器启动"的真正原因：不是规定，是排序。

2. `esp_err_t` 就是 `int`，错误码是"身份证号"，前几位是省份

原理：错误码不是枚举，是一套分段的编号规则，看段号就知道该找哪个部门。

机制：`components/esp_common/include/esp_err.h` 第一句是 `typedef int esp_err_t;`，返回值就是普通整数。编号分两截：低段 `0x101` ~ `0x10D` 是通用错误（`ESP_ERR_INVALID_ARG 0x102`、`ESP_ERR_TIMEOUT 0x107`、`ESP_ERR_INVALID_CRC 0x109`），负数只有 `ESP_FAIL -1`。高段按模块划分：`ESP_ERR_WIFI_BASE 0x3000`、

`MESH 0x4000`、`FLASH 0x6000`、`HW_CRYPTO 0xc000`、`MEMPROT 0xd000`。

踩坑现场：串口打印 `0x6001`，你去查"哪个参数传错了"，查了半天没有。0x6000 段属于 Flash 层：它在说"擦除/写入没成"，可能压根没走到参数校验。

自己验：排障时同时打印 `esp_err_to_name(err)` 和十进制值；拿到 `0x` 开头的高位，先按段号定位模块，再进那个模块找。

3. `ESP_ERROR_CHECK` 在发布构建里是拆掉的报警器

原理：烟雾报警器只在装修时通电，正式营业时断电，标语还挂在墙上。

机制：同一个宏有三个分支。正常构建：失败就走 `_esp_error_check_failed()`，打印文件、行号、表达式并停机。`CONFIG_COMPILER_OPTIMIZATION_ASSERTIONS_SILENT`：只 `abort()`，不打印。`#ifdef NDEBUG`：只有一句 `(void) sizeof(err_rc_);`，**表达式照求值一次（副作用保留），结果直接丢掉，不判错**。

为什么这么设计：发布固件要省 flash、省串口日志；断言被当作开发期工具。这是有意为之，不是 bug。

踩坑现场：单元测试全绿，现场却"明明返回了错误还继续往下跑"，故障点漂到几百行之外。你以为是驱动吞了错误，其实是这个宏根本没检查。

自己验：`gcc -E` 展开看一眼，或确认构建有没有定义 `NDEBUG`。要在发布版留下兜底，就自己写 `if (err != ESP_OK) { … }`。

4. 错误码的字符串表是"链接期钉上布告栏"，不是运行期查字典

原理：`esp_err_to_name()` 不是翻字典，是去看墙上那张布告，而布告在你的代码编译之前就钉好了。

机制：组件在自己 `CMakeLists.txt` 里调 `idf_define_esp_err_codes()` 登记错误码，构建系统从头文件里抽取编号与名字，生成一张放进 `.esp_err_msg_tbl` 链接段的表；运行期函数在这张表里查。

踩坑现场：自己加了个 `MY_ERR_BASE + 3`，调用 `esp_err_to_name()` 得到 `"UNKNOWN ERROR"`，于是怀疑函数有 bug。其实是你只写了 `#define`，没在组件里登记：**便签没交给管理员，墙上当然没有**。

自己验：新增错误码后跑一次 `esp_err_to_name()`；返回 UNKNOWN 就回去补登记，而不是查函数。

5. 厂商 SDK 的许可是"配料表"，不是一锅汤

原理：整包 SDK 不是一锅炖好的汤，是一张配料表，每样配料有自己的保质条款。

机制：`STM32CubeF4/LICENSE.md` 本身就是一张表：STM32F4 HAL、全部 BSP、FatFS、LwIP、Utilities、basic Examples = BSD-3-Clause；CMSIS、CMSIS Device、mbedTLS = Apache-2.0；STM32_USB_Device_Library、USB_Host、STemWin、TouchGFX、STM32_Audio = SLA0044（ST 专有条款）；LibJPEG = Independent JPEG Group License。

踩坑现场："Cube 是 BSD 许可"这句话只对 HAL 与 BSP 成立。产品里链接了 USB 库或 TouchGFX，义务就换了一套；按 BSD 去理解 SLA0044，会在合规审查时返工。

自己验：选中间件之前先翻这张表，把要用的每个目录对上它那一行。

6. GitHub 上的"归档"与"只读镜像"是两码事

原理：一个是"门牌还挂着，屋里没人"，一个是"这屋放的是影印件，原件在别处"。

机制：`openocd-org/openocd` 的描述写着 "Official OpenOCD Read-Only Mirror (no pull requests)"：仓库本身活着，但 GitHub 这侧不收 PR，补丁要走上游。`ARM-software/CMSIS_5` 与 `blackmagic-debug/blackmagic` 的 `archived` 字段为 `true`：只读、不再受理变更；CMSIS 的后续在 CMSIS_6，Black Magic 的后续在别的分支/仓库。

踩坑现场：在镜像上开 PR，等一个月没人回；在归档仓库等新版本，等到的只有"已停止维护"。

自己验：看描述里有没有 `mirror` / `no pull requests`，看页头有没有归档横条。两个信号，三秒判断。

许可边界：能不能链进你的固件

仓库地图里那列"许可"不是装饰，它决定你能不能把代码拿进产品：

许可族	能静态链进闭源固件吗	义务	地图里的例子
MIT / BSD-3-Clause	可以	保留版权与许可声明	FreeRTOS-Kernel、littlefs、lwIP、LVGL、TinyUSB、Unity、CMock、pico-sdk
Apache-2.0	可以	同上 + 保留 `NOTICE`、显著标注改动	Zephyr、RT-Thread、NuttX、ESP-IDF、pyOCD
LGPL-2.1	**静态链接需谨慎**	允许动态链接/独立进程；静态链入闭源固件通常要提供目标文件或源码	RIOT
GPL 家族（GPL-2.0/3.0）	**不可以**	工具链/调试器类随工具分发；不要并进固件	OpenOCD、grbl、riscv-gnu-toolchain
厂商 SLA / 多组件	逐组件看	STM32CubeF4 是"每目录一许可"，USB/STemWin/TouchGFX 走 SLA0044	STM32CubeF4

两条纪律：**下单看 `LICENSE` 原文，不看 README 的徽章**；**多组件仓库（Cube、IDF）按目录逐个核**。 同一仓库里 BSD-3、Apache-2.0 与 SLA 可以并存。本书的 47 个工程全部自写，不复制上述项目的代码。

社区：提问之前先做五件事

1. **读 issue 模板**。模板问什么，就是维护者最缺什么；跳过模板的 issue 通常第一个被关。
2. **把复现压到最小**。能贴 20 行就别贴 200 行；能删掉的初始化全部删掉，直到现象消失的前一步。
3. **版本三件套**：芯片型号（含后缀）、SDK/内核版本（commit 或 tag）、工具链版本。缺一个，回答只能靠猜。
4. **给证据不给形容词**。"不工作"没有信息量；串口日志、`nm` / `objdump` 片段、示波器截图、`idf.py monitor` 的寄存器转储有。
5. **先搜再开**。重复 issue 是最快被忽略的方式；搜的时候用报错原文，不要用你理解的转述。

复刻清单：六个能一周啃完的小工程

仓库	规模锚点	读哪一处	练到的手艺
grbl/grbl（GPL-3.0）	`stepper.c` 一个文件	中断服务里只做"发一个脉冲 + 记账"	实时路径的最小化；GPL 系：**读可以，抄进闭源固件不行**
libopencm3/libopencm3	`stm32/f4/memorymap.h` 与同目录外设头	把参考手册的一节翻成 `#define`	寄存器文档化
RT-Thread/rt-thread	`src/object.c`	`rt_object_init()` 与类型强转	用 C 做继承
FreeRTOS/FreeRTOS-Kernel	`tasks.c` + `portable/GCC/ARM_CM4F/`	`vTaskSwitchContext()` 前后	就绪表与上下文切换
pyocd/pyOCD	`pyocd/core/`	target 抽象层的接口设计	把协议写成 API
micropython/micropython	`ports/*/mpconfigport.h`	一个最小 port 暴露了哪些宏	移植层的边界

证据口径

- 星标数、`archived` 字段、仓库描述：GitHub API，2026-10-26；表中 30 个仓库同日逐一复核。
- 许可：仓库根 `LICENSE` / `COPYING` / `LICENSE.md` 原文；STM32CubeF4 按其 `LICENSE.md` 的逐组件表引用。
- 代码路径与函数：当日默认分支的 GitHub 代码检索命中；版本漂移后路径可能变化，读之前先 `git log -1` 确认分支。
- 本附录不引用任何未在上述来源中出现的事实；仓库若改名或归档，以仓库当前状态为准。

勘误与贡献

发现错误？到仓库提 Issue：https://github.com/zhuguang-ZFG/STM32/issues。请带上章节号（如 S7）、原文一句、以及你认为正确的说法与出处——「证据驱动」对勘误同样适用。

想参与？校对、翻译、新章节与实验复现都欢迎；入口见仓库 `CONTRIBUTING.md`，排版与书稿工艺的讨论见 `docs/reference/book.md`。

贡献者：zhuguang-ZFG 与贡献者；完整名单见 GitHub Contributors。

全书规模：94 章正文（11 部），约 63 小时通读，186 张机制动画与插图，47 个配套工程（附录 B），65 道自测题（附录 C），75 个冻结来源（附录 E）。

本书由 `mcu-book/build_html.py` 从 88 章源文档构建；来源、冻结点与许可见附录 E。